Electronics and Computing

Book 4 — The Electronics & Computing Core

Electronic devices, analog and digital circuits, computer architecture and digital hardware, programming and scientific computing, embedded and real-time systems, power electronics, CMOS and VLSI, and hardware–software co-design.

The Studio of Sadiq · Golam Kuadir Khan Prince

Contents

Part I — Electronic Devices

Chapter 1 — Electronic Device Fundamentals

Chapter 2 — Semiconductor Junction Devices

Chapter 3 — Bipolar Junction Transistors

Chapter 4 — Field-Effect Transistors

Chapter 5 — Advanced Semiconductor Devices

Part II — Analog Electronics

Chapter 6 — Diode Circuits

Chapter 7 — Transistor Amplifiers

Chapter 8 — Operational Amplifiers

Chapter 9 — Feedback and Oscillators

Chapter 10 — Analog Integrated Circuits

Part III — Digital Electronics

Chapter 11 — Digital Number Systems

Chapter 12 — Boolean Algebra and Logic Gates

Chapter 13 — Combinational Digital Systems

Chapter 14 — Sequential Digital Systems

Chapter 15 — Digital System Design

Part IV — Computer Architecture and Digital Hardware

Chapter 16 — Computer Architecture

Chapter 17 — Microprocessors and Microcontrollers

Chapter 18 — Memory and Programmable Logic

Chapter 19 — Hardware Description Languages

Part V — Programming and Computational Systems

Chapter 20 — Programming Fundamentals

Chapter 21 — C/C++ for Engineering

Chapter 22 — Python for Engineering

Chapter 23 — Algorithms and Data Structures

Chapter 24 — Scientific Computing

Part VI — Embedded and Real-Time Systems

Chapter 25 — Embedded-System Architecture

Chapter 26 — Embedded Interfaces

Chapter 27 — Real-Time Computing

Chapter 28 — Embedded System Design

Part VII — Power Electronics

Chapter 29 — Power-Electronic Fundamentals

Chapter 30 — DC–DC Converters

Chapter 31 — AC–DC and DC–AC Conversion

Chapter 32 — Motor Drives and Power Conversion

Part VIII — Integrated Circuits and VLSI

Chapter 33 — CMOS Fundamentals

Chapter 34 — VLSI Design

Chapter 35 — Hardware–Software Co-Design

Part I — Electronic Devices

Every circuit, processor and converter in this book is built from a handful of semiconductor devices, and every abstraction it relies on (the amplifier, the logic gate, the switch, the instruction) holds only while those devices stay inside their limits. Part I turns device physics into the working knowledge needed to use real parts: how a nonlinear device is modelled at the level of accuracy a design requires, how it is biased and linearised, how its parameters move with temperature and from part to part, and what its datasheet guarantees. It moves from ideas common to all devices, through the junction and the two transistor families, to the power, wide-bandgap and optoelectronic devices that carry the same principles to kilovolts, gigahertz and light.

By the end of this part you should understand

- how a device is described by its voltage–current characteristics, and how to choose among ideal, piecewise-linear, exponential and compact models for a given purpose;
- why operating regions exist, how to bias a device at a stable operating point, and when it may be replaced by a linear small-signal model;
- how temperature, part-to-part spread, breakdown, safe operating area, parasitics and ageing limit every model, and how datasheet ratings and characteristics express those limits;
- how real p–n and Schottky junctions depart from the ideal diode law, and how rectifier, Zener, Schottky, photo-, light-emitting, solar, varactor and protection diodes are chosen and sized;
- how the bipolar transistor works in its four regions and three configurations, how it is biased against β and temperature spread, and what limits its gain, switching speed and bandwidth;
- how JFETs and MOSFETs are modelled, from the square law to short-channel and power devices;
- how thyristors, IGBTs, wide-bandgap and optoelectronic devices extend these ideas, and how devices are selected against their ratings.

Chapters

Chapter	Central question	What it delivers
Chapter 1 — Electronic Device Fundamentals	How is a nonlinear device modelled, biased and linearised, and which limits end the validity of every model?	Characteristics, load lines and Newton solution, the model hierarchy, operating regions, bias sensitivity, small-signal linearisation and its distortion limit, temperature coefficients and self-heating, datasheet structure and derating, breakdown, SOA, noise, spread and reliability
Chapter 2 — Semiconductor Junction Devices	How does a real junction depart from the ideal diode law, and how is it specialised for different tasks?	Depletion-region numbers, breakdown and stored charge, the real diode characteristic and reverse recovery, rectifier, Zener and Schottky diodes, photodiodes, LEDs and solar cells at device level, varactors, and TVS, ESD, flyback and reverse-polarity protection
Chapter 3 — Bipolar Junction Transistors	How does the BJT amplify and switch, and how is a circuit made independent of its poorly controlled current gain?	Structure and Ebers–Moll model, regions and saturation voltage, the CB, CE and CC configurations, stable bias design, load lines, the hybrid-π model, intrinsic gain and degeneration, switching times and storage, f_T and the Miller effect
Chapter 4 — Field-Effect Transistors	How does a gate voltage control a channel, and how do FET models, bias and switching differ from the BJT's?	JFET operation, MOSFET structure, enhancement and depletion modes, threshold voltage, regions and short-channel effects, biasing, small-signal models and amplification, switching, and power-MOSFET fundamentals
Chapter 5 — Advanced Semiconductor Devices	Which devices handle kilovolts, kiloamperes, light and microwave frequencies, and how are devices selected against their ratings?	Thyristors, SCRs, TRIACs and DIACs, IGBTs, SiC and GaN power devices, optoelectronic and compound-semiconductor devices, and device selection and ratings

How this part connects to the rest of the book

Part I builds on three sibling books and feeds every later part of this one. Band theory, carrier statistics, doping, drift–diffusion and the one-section sketches of the diode, BJT and MOSFET come from *Engineering Physics: Top Down* (Ch. 6, Ch. 14 and Ch. 18); semiconductor material properties, families and optoelectronic materials from *Materials Science and Engineering* (Ch. 32–35); and circuit laws, dependent sources, two-ports and frequency response from *Circuits, Signals and Communications* (Part I).

Within this book, the devices of Part I become the analog circuits of Part II: diode circuits (Chapter 6), transistor and operational amplifiers (Chapters 7–8), feedback and oscillators (Chapter 9) and analog integrated circuits (Chapter 10). The switching behaviour of transistors becomes the logic gates of Part III and, at the transistor level, the CMOS circuits and VLSI design of Part VIII (Chapters 33–34). The power and wide-bandgap devices of Chapters 4 and 5, with the thermal and safe-operating-area limits of Chapter 1, are the switches of the converters in Part VII (Chapters 29–32), whose system-level use belongs to *Electrical Energy and Machines*.

Chapter 1 — Electronic Device Fundamentals

Part I · Electronic Devices

A battery-powered soil-moisture logger is redesigned to run from a single 1.2 V nickel-metal-hydride cell instead of a 3.3 V regulator. The designer keeps the old status-LED and diode-OR circuits and recalculates the currents with the rule everyone learns first: "a silicon diode drops 0.7 V". On the bench every current is 30 to 50 % higher than calculated, a small-signal detector that worked at 3.3 V now distorts, and in a cold-chamber test at −40 °C the diode-OR stops sharing current between the cell and the backup supply. Nothing is faulty. The 0.7 V rule is a *model*, chosen once for one range of current and temperature, and the redesign moved the circuit outside that range. Every electronic device used in this book, from a signal diode to a 1200 V SiC MOSFET, is a nonlinear element whose behaviour can be summarised by simpler models only inside stated limits of voltage, current, frequency and temperature. This chapter is about those models and those limits: how a device is described by its voltage–current characteristic, how a circuit containing it is solved, how it is biased at an operating point and linearised about that point, how its parameters move with temperature, and how to read the datasheet that defines what can be relied on.

> **How is a nonlinear device described, and how do engineers choose a model simple enough to design with but accurate enough to trust?**
> **What is an operating point, why must it be set and stabilised, and when may the device be treated as linear around it? Which parameters on a datasheet are guaranteed, which drift with temperature, and which practical limits end the validity of every model?**

Concept	Engineering destination
Voltage–current characteristics	Reading datasheet curves; identifying sources, sinks and power flow
Nonlinear devices	Rectifiers, mixers, log amplifiers, switches; numerical circuit solution
Device models	Hand analysis, SPICE compact models, worst-case design
Operating regions	Choosing the right model; checking a design's assumptions
Biasing	Amplifier operating points, LED currents, reference circuits
Small-signal concepts	Gain, impedance and distortion of amplifiers and sensor front ends
Temperature effects	Diode thermometers, bandgap references, thermal runaway, derating
Device parameters	Datasheet interpretation, part selection, guaranteed versus typical design
Practical limitations	Breakdown, safe operating area, parasitics, noise, reliability

Roadmap. The characteristic as the device's description (1.1) → what nonlinearity does to circuit solution (1.2) → the hierarchy of models (1.3) and the regions in which each holds (1.4) → fixing an operating point (1.5) and linearising about it (1.6) → how temperature moves everything (1.7) → what a datasheet promises (1.8) → the limits that end every model (1.9).

1.1 Voltage–current characteristics

Where *Engineering Physics* leaves off

Engineering Physics carries the semiconductor from bands and the Fermi level (Ch. 6 §6.5) through carrier concentrations, doping and the mass-action law $np = n_i^2$ (§6.6), to drift–diffusion transport and the ideal p–n junction law $I = I_0(e^{eV/k_BT} - 1)$ (Ch. 14 §14.5), and in Ch. 18 §18.3 it sketches how the diode, the BJT and the MOSFET become

circuit models with an ideality factor, a small-signal resistance $r_d = \eta V_T / I_D$ and a transconductance. The material side (dopant ionisation, mobility, lifetime and their temperature dependence) is in *Materials Science and Engineering*, Ch. 32. This book takes those results as given. It starts where the physics stops: with the terminal behaviour an engineer measures, the models a designer chooses among, the regions where each model holds, and the ratings, spreads and failure limits that a real part carries. Where a device is pushed beyond the validity of the physics sketch (high injection, breakdown, self-heating, high frequency), we say so.

The characteristic as the description of a two-terminal device

For a circuit designer a two-terminal device *is* its static voltage–current characteristic $i = f(v)$, measured with the passive sign convention (current entering the terminal marked +), plus whatever stores charge or energy (capacitance, inductance, thermal mass) when signals change quickly. Figure 1.1 shows five characteristics side by side. A resistor is a straight line through the origin. An ideal voltage source is a vertical line, an ideal current source a horizontal one. A diode is a curve that is nearly flat for negative voltage and rises exponentially for positive voltage. A Zener diode adds a second steep branch at a negative breakdown voltage.

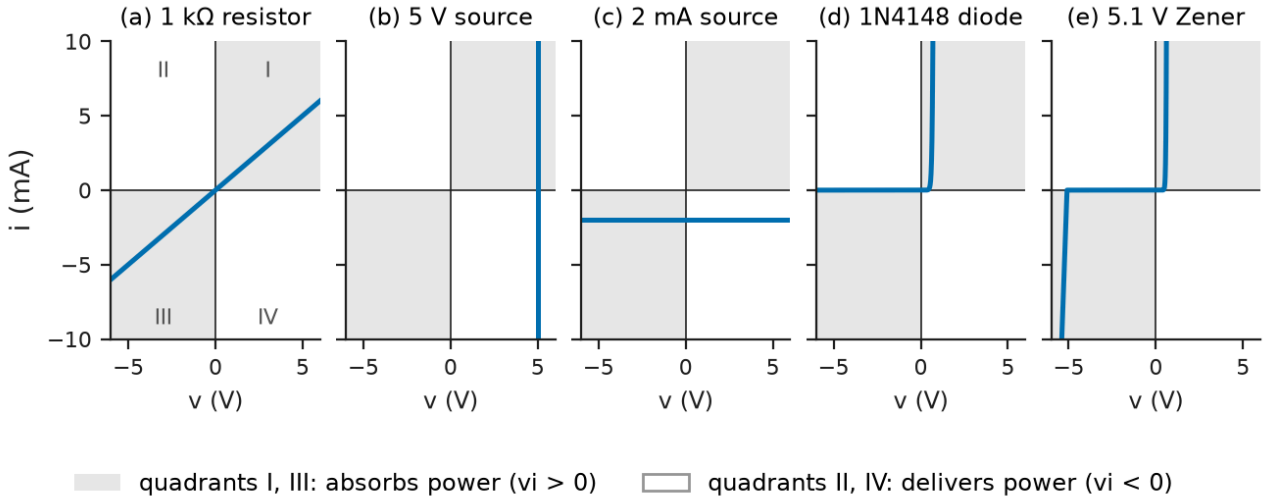


Figure 1.1 — Static voltage–current characteristics of five two-terminal elements, with the power-absorbing and power-delivering quadrants shaded.

The quadrant in which the operating point lies tells whether the element absorbs or delivers power. With the passive sign convention (*Circuits, Signals and Communications*, Ch. 1), $p = vi > 0$ means absorption: a resistor, a forward-biased diode and a reverse-biased Zener in breakdown live in quadrants I and III. A battery being discharged, an illuminated solar cell and a photodiode in photovoltaic mode deliver power and operate in quadrant IV.

Static and dynamic resistance

Two different "resistances" can be read off any characteristic at an operating point (V_Q, I_Q):

- the **static (chordal) resistance** $R = V_Q / I_Q$, the slope of the line from the origin to the point;
- the **dynamic (incremental, differential) resistance** $r = dv/di$ evaluated at the point, the inverse slope of the tangent.

For a resistor they are equal. For a diode carrying 1 mA they differ by an order of magnitude: a 1N4148 at 1 mA has $V_Q \approx 0.58$ V, so $R \approx 580\ \Omega$, while $r_d \approx 45\ \Omega$ (Section 1.6). The static value sets DC power ($P = V_Q I_Q$); the dynamic value sets what happens to small signals, ripple and noise. Confusing them is the most common source of factor-of-ten errors in first designs.

Three- and four-terminal devices: families of curves

A transistor has a characteristic for each pair of terminals, and one terminal voltage or current acts as a parameter. The usual presentation is a family of **output characteristics** (I_C against V_{CE} with I_B or V_{BE} as parameter for a BJT; I_D against V_{DS} with V_{GS} as parameter for a MOSFET) together with a **transfer characteristic** (I_C against V_{BE}, I_D against V_{GS}). The output family shows the regions of Section 1.4 directly: a steep region near the origin where the device behaves as a low resistance, a flat region where it behaves as a controlled current source, and a breakdown region at high voltage. Chapters 3 and 4 read these families in detail. Table 1.1 classifies the characteristics met in this book by their dominant mathematical form.

Table 1.1 — Dominant forms of device characteristics (typical, small-signal silicon devices near 25 °C).

Device and region	Dominant law	Typical scale parameter
p–n diode, forward	exponential, $I \propto e^{V/nV_T}$	60–120 mV per decade of current
Schottky diode, forward	exponential (thermionic)	60–70 mV per decade
BJT, forward active	exponential in V_{BE}, nearly linear in I_B	60 mV per decade; β = 100–400
MOSFET, saturation (long channel)	square law in $V_{GS} - V_{th}$	$k'W/L$ = 0.1–100 mA/V²
MOSFET, subthreshold	exponential, $I \propto e^{V_{GS}/nV_T}$	70–100 mV per decade
MOSFET, triode (small V_{DS})	linear resistor, $R_{DS(on)}$	1 mΩ (power) to 10 Ω (small signal)
Zener/avalanche breakdown	near-vertical, small slope	dynamic resistance 1–100 Ω
Thermistor (NTC)	exponential in $1/T$ (via self-heating)	B = 3000–4500 K

1.2 Nonlinear devices

What linearity buys and what nonlinearity costs

A network of resistors and linear sources obeys superposition, has a unique solution that a single matrix inversion produces, and can be replaced at any port by a Thévenin equivalent (*Circuits, Signals and Communications*, Ch. 3–4). Insert one diode and all three properties are lost. The response to two inputs is no longer the sum of the responses; a circuit with positive feedback may have several solutions; and a Thévenin equivalent exists only for the linear part *seen by* the device, not through it.

Nonlinearity is not a defect to be minimised: it is the reason electronics works. Rectification (Chapter 6), mixing and frequency conversion, logarithmic compression, the regenerative switching of a logic gate (Chapter 33) and the on/off action of every power switch (Chapter 29) all exist because a characteristic is curved. The engineering task is to exploit the curvature where it is wanted and to suppress it where a linear response is required, usually by biasing (Section 1.5) and linearisation (Section 1.6), or by negative feedback (Chapter 9).

The load-line method

The simplest nonlinear circuit is a device in series with a linear network. Replace the linear part by its Thévenin equivalent (V_{Th}, R_{Th}). KVL gives the **load line**

$$i = \frac{V_{Th} - v}{R_{Th}}$$

a straight line in the (v, i) plane with intercepts V_{Th} on the voltage axis and V_{Th}/R_{Th} on the current axis. The device must also lie on its own characteristic, so the operating point is the intersection (Figure 1.2). The construction separates what

the device does from what the circuit does, and it is still how engineers think about bias points, the swing of an amplifier (Chapter 3, Section 3.7) and the transitions of a switch.

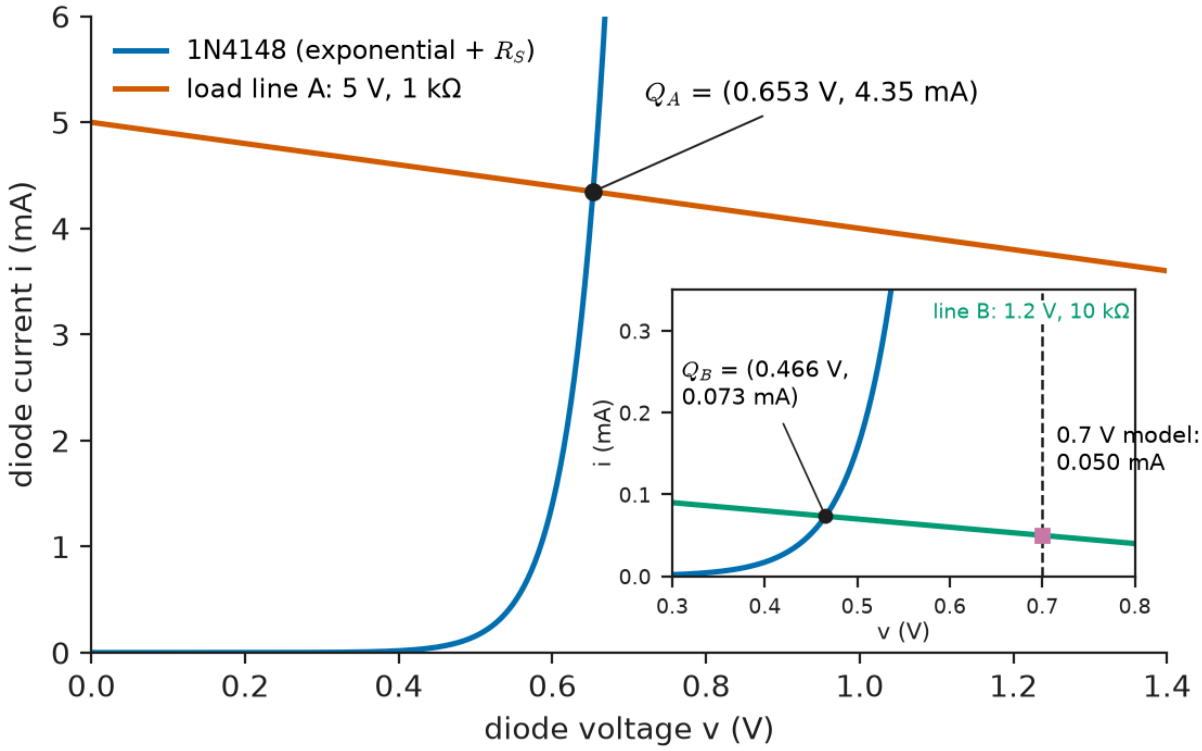


Figure 1.2 — Load-line solution of a diode in series with a resistor, for two supply and resistor combinations.

Numerical solution

Graphical intersection is not a design method for circuits with many devices. Circuit simulators solve the nonlinear nodal equations by **Newton–Raphson iteration**: guess the node voltages, replace each device by its tangent (a conductance in parallel with a current source, the "companion model"), solve the resulting linear network, and repeat. The method converges quadratically near the solution, but the exponential characteristic can throw a guess far beyond the physical range, so simulators limit the voltage step across each junction. The mathematics of Newton's method and its convergence is in *The Mathematics Universe*, Ch. 22 §22.4; the listing below applies it to the diode circuit with the published SPICE parameters of a 1N4148 ($I_S = 2.52$ nA, $n = 1.752$, $R_S = 0.568\ \Omega$).

```
import math

def diode_bias(Vs, R, Is=2.52e-9, n=1.752, Rs=0.568, T=300.0):
    """Solve Vs = V + (R + Rs) * Is*(exp(V/(n*VT)) - 1) for V by damped Newton."""
    VT = 1.380649e-23 * T / 1.602176634e-19
    V = 0.5                                  # starting guess
    for k in range(50):
        e = math.exp(V / (n * VT))
        f = Vs - V - (R + Rs) * Is * (e - 1.0)
        df = -1.0 - (R + Rs) * Is * e / (n * VT)
        step = max(-0.1, min(0.1, -f / df))   # limit the step (SPICE does the same)
        V += step
        if abs(step) < 1e-12:
            break
    I = Is * (math.exp(V / (n * VT)) - 1.0)
    return V + I * Rs, I, k + 1              # terminal voltage, current, iterations

for Vs, R in [(5.0, 1e3), (3.3, 10e3), (1.2, 10e3)]:
    V, I, k = diode_bias(Vs, R)
    print(f"Vs={Vs:4.1f} V R={R/1e3:4.0f}k: V_D={V:.4f} V  I_D={I*1e3:.4f} mA  "
          f"(0.7 V model: {(Vs-0.7)/R*1e3:.4f} mA, {k} iterations)")
```

The variable `V` is the voltage across the ideal junction; the series resistance is added back to give the terminal voltage. The step limit of 0.1 V is the same device that SPICE calls junction voltage limiting. Running it gives:

```
 Vs= 5.0 V R=   1k: V_D=0.6529 V  I_D=4.3471 mA  (0.7 V model: 4.3000 mA, 8 iterations)
Vs= 3.3 V R=  10k: V_D=0.5260 V  I_D=0.2774 mA  (0.7 V model: 0.2600 mA, 6 iterations)
Vs= 1.2 V R=  10k: V_D=0.4656 V  I_D=0.0734 mA  (0.7 V model: 0.0500 mA, 6 iterations)
```

Worked example — the logger's diode at 5 V and at 1.2 V. With a 5 V supply and 1 kΩ the exponential model gives $V_D = 0.653$ V and $I_D = 4.347$ mA. The 0.7 V rule gives $(5 - 0.7)/1000 = 4.300$ mA, an error of 1.1 %: the resistor drops 4.35 V, so a 47 mV error in the diode voltage hardly matters. With 1.2 V and 10 kΩ the diode sits at 0.466 V and 73.4 µA, while the rule predicts $(1.2 - 0.7)/10\,000 = 50.0$ µA, an error of −32 %. The diode voltage is now a large fraction of the supply, and its 234 mV departure from the assumed 0.7 V (the diode carries only 73 µA, two decades below the current at which 0.7 V is right) feeds straight into the current. This is the logger's first symptom. The general lesson: a device model is accurate in *relative* terms only when the device voltage is a small part of the total, or when the model's parameters were fitted at the actual operating current.

> **Status of the principle: Load line and Newton solution of a nonlinear circuit**
> *Derived*. The load line is KVL applied to a Thévenin equivalent, and Newton iteration solves the resulting equations exactly to within a tolerance; the accuracy of the answer is limited only by the accuracy of the device model, not by the method.

Multiple solutions and negative resistance

When a characteristic has a region of negative slope (the tunnel diode, the snapback of an ESD clamp, a thyristor's switching region, Chapter 5) a load line can cross it three times. The middle intersection is unstable: any small disturbance grows, and the circuit jumps to one of the outer points. The same mechanism, created by positive feedback rather than device physics, gives the two stable states of a latch (Chapter 14) and the hysteresis of a Schmitt trigger (Chapter 8). With a device of negative incremental resistance $-r$ in series with R_{Th}, the rule depends on the shape of the characteristic. For a voltage-controlled (N-shaped) characteristic such as the tunnel diode's, a single stable intersection requires $R_{Th} < r$, so such devices are biased from a stiff, low-resistance source. For a current-controlled (S-shaped)

characteristic such as a thyristor's or a gas discharge's, stability requires $R_{Th} > r$, which is why such devices always need a series ballast resistor.

1.3 Device models

A hierarchy, not a single truth

Every device is described by a ladder of models of increasing accuracy and cost. Choosing the rung is an engineering decision: the model must capture the effect the design depends on, and nothing more. Figure 1.3 compares four diode models, and Table 1.2 lists where each is used.

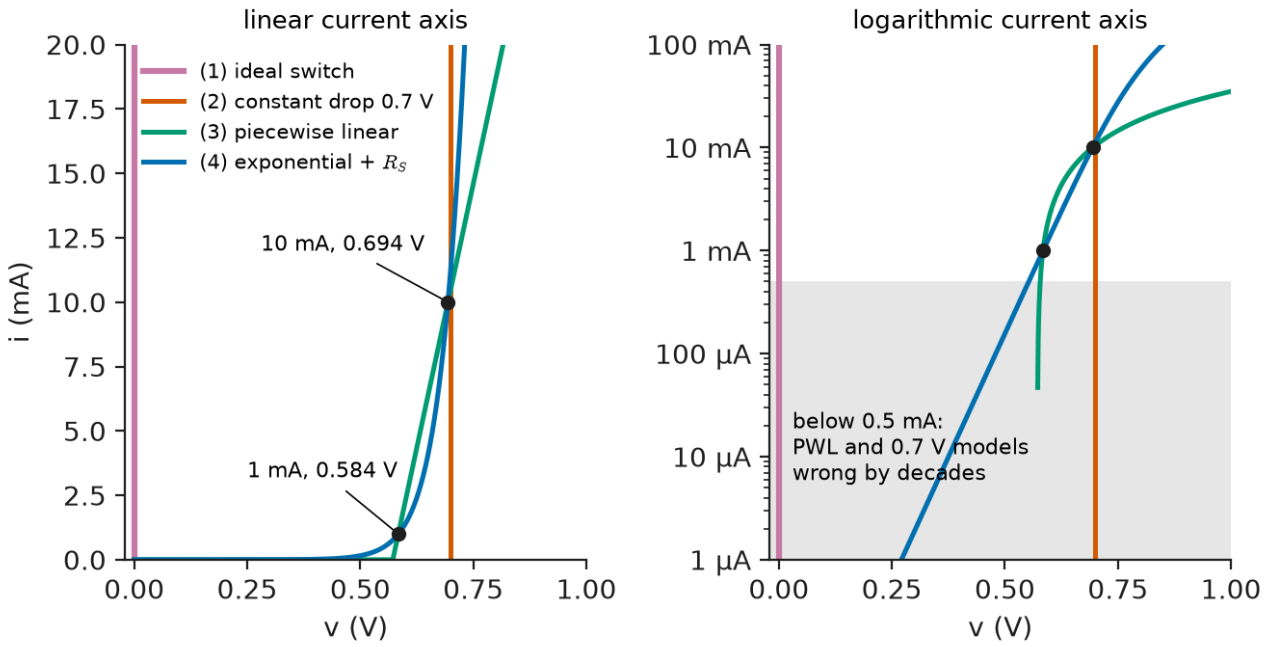


Figure 1.3 — Four diode models compared with the exponential characteristic of a 1N4148.

Table 1.2 — The diode model hierarchy.

Model	Equation	Where it is adequate	Where it fails
Ideal switch	$v = 0$ if $i > 0$; $i = 0$ if $v < 0$	Rectifiers and switches whose voltages are $\gg 1$ V; logic of steering circuits	Low supply voltages; precision; power loss
Constant drop	$v = V_{D0}$ (0.6–0.7 V Si, 0.2–0.4 V Schottky) when on	Bias estimates when $V_D \ll$ other voltages	Low supplies; currents far from the fitting point; temperature
Piecewise linear	$v = V_{D0} + r_D i$ when on	Conduction loss in power diodes (V_{F0}, r_T on datasheets)	Low currents; temperature unless refitted
Exponential with n, R_S	$i = I_S(e^{(v-iR_S)/nV_T} - 1)$	Bias, logarithmic circuits, small-signal design, temperature	Breakdown, high injection, reverse leakage (generation), fast transients
Compact model (SPICE)	Exponential + $C_j(v)$, transit time, breakdown, temperature laws	Transient and AC simulation, worst-case and Monte Carlo analysis	Only as good as its fitted parameters and their range

The piecewise-linear model in the third row is fitted by drawing a straight line through two points of the exponential curve. Through the 1N4148's points at 1 mA (0.584 V) and 10 mA (0.694 V), it has $r_D = (0.694 - 0.584)/9\ \text{mA} = 12.2\ \Omega$ and intercept $V_{D0} = 0.572$ V. It predicts 4.37 mA in the 5 V/1 kΩ circuit (0.6 % high) but 62.7 µA in the 1.2 V/10 kΩ circuit (15 % low), because 73 µA lies outside the range over which it was fitted.

> **Status of the principle: Constant-drop and piecewise-linear diode models**
> *Approximate*. They replace an exponential by a straight-line fit valid over roughly one decade of current around the fitting point, at the fitting temperature; outside that window the error grows by about $nV_T \ln 10 \approx 100$ mV per decade.

Large-signal, small-signal and switching models

Models are also classified by the signals they serve:

- **Large-signal (static) models** relate total DC voltages and currents and are used to find the operating point: the exponential diode law, Ebers–Moll for the BJT (Chapter 3), the square law and its short-channel corrections for the MOSFET (Chapter 4).
- **Small-signal models** are linear circuits of resistances, transconductances and capacitances, valid for small excursions about the operating point (Section 1.6): r_d for the diode, hybrid-π for the BJT and MOSFET. Their parameter values depend on the operating point found with the large-signal model.
- **Switching models** describe a device that moves between two extreme regions: an on-resistance or saturation voltage, an off-state leakage, and the charges or capacitances that set the transition times. They are used for logic (Chapter 33) and power converters (Chapter 29).
- **Thermal models** convert dissipated power into junction temperature through a thermal-resistance and capacitance network (Section 1.7).

Compact models in simulators

A SPICE diode model card for the 1N4148 reads, in part, `IS=2.52n N=1.752 RS=0.568 CJO=4p M=0.4 VJ=0.5 TT=20n BV=100 IBV=100u`. Each parameter has a physical origin in *Engineering Physics* (saturation current and ideality from §14.5.4 and §18.3.1, junction capacitance from §18.3.1, transit time from minority-carrier storage), but in a model card they are fitted numbers, extracted from measurements on sample parts. Two consequences follow. First, a model card represents a *typical* part at nominal temperature; it says nothing about the spread guaranteed by the datasheet unless the manufacturer supplies corner models. Second, the fitted values are trustworthy only within the measured range; a model extracted between 1 µA and 100 mA predicts nothing reliable about leakage at 150 °C or the voltage during a 10 A surge. MOSFET compact models (BSIM, PSP, EKV) carry hundreds of parameters (Chapters 4 and 33).

1.4 Operating regions

A region is the domain of a simpler model

The characteristic of a real device is a single smooth curve. Engineers divide it into **operating regions** because inside each region a much simpler model holds. A forward-biased diode is an exponential, a reverse-biased diode a small constant leakage plus a capacitance, a diode in breakdown a voltage source with a small series resistance. The boundaries are not sharp: they are chosen where one term of the full model becomes negligible compared with another, and the choice depends on the accuracy required.

Table 1.3 — Operating regions of the main devices, with the simplified model used in each.

Device	Region	Condition (silicon, typical)	Simplified model
Diode	Forward	$V_D > 0.3$ V and $I_D \gg I_S$	$I = I_S e^{V/nV_T}$, or constant drop
Diode	Reverse	$-V_{BR} < V_D < -3V_T$	$I \approx -I_R$ (nA–μA), $C_j(V)$
Diode	Breakdown	$V_D < -V_{BR}$	$V \approx -V_{BR} - r_z I$
BJT	Forward active	B–E forward, B–C reverse	$I_C = \beta I_B$, $I_C = I_S e^{V_{BE}/V_T}$
BJT	Saturation	Both junctions forward	$V_{CE} = V_{CE(sat)} \approx 0.05$–$0.3$ V
BJT	Cutoff	Both junctions reverse	All currents ≈ leakage
MOSFET	Cutoff/subthreshold	$V_{GS} < V_{th}$	Exponential leakage, ≈ 0
MOSFET	Triode (linear)	$V_{GS} > V_{th}$, $V_{DS} < V_{GS} - V_{th}$	Voltage-controlled resistor
MOSFET	Saturation	$V_{GS} > V_{th}$, $V_{DS} > V_{GS} - V_{th}$	Square law, current source

Device names for regions are inconsistent across the field, and one example causes real errors. The *saturation* region of a MOSFET is where its drain current saturates and it acts as a current source, the analogue of the BJT's *forward-active* region. The *saturation* region of a BJT is where the transistor is fully on and acts as a closed switch, the analogue of the MOSFET's *triode* region. An engineer moving between the two families must translate.

The assume–solve–check procedure

Because the region decides the model, every hand analysis of a circuit with devices follows the same loop:

1. **Assume** a region for each device (e.g. the BJT is forward-active, the diode is on).
2. **Solve** the circuit with the corresponding simple models.
3. **Check** that the solution satisfies the region's conditions (is $V_{CE} > V_{CE(sat)}$? is $I_D > 0$?).
4. If a check fails, change the assumption and repeat.

Simulators use a single all-region model instead, but the loop remains the engineer's main tool for catching nonsense from a simulator.

> **Status of the principle: Region-by-region device modelling**
> *Approximate.* Each region's model is an asymptote of the full characteristic, valid where the neglected terms are small; results near a region boundary (a diode at a few V_T of forward bias, a MOSFET near V_{th}, a BJT at the edge of saturation) carry the largest error and must be checked with the full model.

Regions and abstraction

The rest of this book can be read as a list of abstractions built on one region each. Linear amplifiers (Chapters 7–10) keep the transistor in its current-source region. Logic gates (Chapters 12 and 33) drive transistors hard between cutoff and the low-resistance region and treat the transit as a delay. Power converters (Chapters 29–31) do the same with large currents and voltages, and count the energy lost in the transit. Voltage references (Chapter 10) use the forward diode's predictable temperature behaviour or the breakdown region of a Zener. An abstraction fails when a device leaves its region, which is why the regions' boundaries reappear as specifications: an amplifier's output swing, a gate's noise margin, a converter's maximum duty cycle.

1.5 Biasing

The operating point

Biasing fixes the DC voltages and currents of a device, its **quiescent** or **operating point** (Q-point), so that the device sits in the intended region with the intended small-signal parameters. A diode used as a small-signal resistor needs a fixed DC current, because r_d is set by it. An LED needs a fixed current, because its light output is proportional to current while its forward voltage varies from part to part. A transistor amplifier needs its Q-point placed so that the largest expected signal does not drive it out of the forward-active region (Section 3.7). In each case the bias network defines the Q-point, and the quality of a bias design is measured by how little the Q-point moves when things it does not control change.

Voltage bias versus current bias

The exponential characteristic makes the choice between imposing a voltage and imposing a current decisive. Imposing a voltage on a forward junction is a poor way to set its current: a change of $nV_T \ln 10$, about 60–120 mV, changes the current tenfold, and the voltage needed for a given current falls by about 2 mV/K (Section 1.7). Imposing a current is robust: the junction voltage adjusts itself logarithmically and the current stays where it was set. The rule "bias junctions with currents, not voltages" runs through every bias circuit in this book, from the series resistor of an LED (Chapter 2) to the emitter resistor of a BJT stage (Chapter 3) and the current mirrors of integrated circuits (Chapter 10).

Worked example — biasing an indicator LED for part-to-part spread. A green InGaN LED with a datasheet forward voltage of 3.0 V typical (2.7 V minimum, 3.4 V maximum) at 5 mA is driven from a 3.3 V rail. With a series resistor sized for the typical part, $R = (3.3 - 3.0)/5\ \text{mA} = 60\ \Omega$, the current would range from $(3.3 - 2.7)/60 = 10.0$ mA to $(3.3 - 3.4)/60 < 0$, i.e. from double brightness to dark. The resistor drop of 0.3 V is too small compared with the 0.7 V spread of V_F. Driving from the 5 V rail with $R = (5.0 - 3.0)/5\ \text{mA} = 400\ \Omega$ (390 Ω standard) gives currents between $(5 - 3.4)/390 = 4.1$ mA and $(5 - 2.7)/390 = 5.9$ mA, a ±18 % spread that the eye barely notices. A true current source (a constant-current LED driver, Chapter 10) removes the dependence entirely. The design principle is quantitative: the fractional current error equals $\Delta V_F / V_R$, the device-voltage spread divided by the voltage across the current-setting element.

Sensitivity and stability of the operating point

For a Q-point quantity X that depends on a parameter p, the normalised **sensitivity**

$$S_p^X = \frac{p}{X}\,\frac{\partial X}{\partial p}$$

is the percentage change in X per percent change in p. A good bias design makes S small for the parameters that vary most: the BJT's current gain β (a factor of three between minimum and maximum on most datasheets), its V_{BE} (−2 mV/K), the MOSFET threshold V_{th} (±0.5 V or more between parts) and the supply voltage. In the LED example, the sensitivity of current to V_F is $S = -V_F/V_R$: −10 with a 3.3 V supply and −1.5 with a 5 V supply. Section 3.6 applies the same measure to BJT bias networks, where negative feedback through an emitter resistor reduces the sensitivity to β by a factor of ten or more.

> **Status of the principle: Bias sensitivity**
> *Derived.* The sensitivity is the first-order Taylor coefficient of the Q-point with respect to a parameter; it predicts the effect of small parameter changes accurately and of large ones (a factor-of-three spread in β) only approximately, so worst-case corners should be recomputed with the full equations.

1.6 Small-signal concepts

Linearisation about the operating point

Once a device is biased at (V_Q, I_Q), a small change v in voltage produces a change i in current. Expanding the characteristic $I = f(V)$ in a Taylor series about the Q-point,

$$i = f'(V_Q)\,v + \tfrac{1}{2}f''(V_Q)\,v^2 + \tfrac{1}{6}f'''(V_Q)\,v^3 + \dots$$

Keeping only the first term gives a linear relation $i = g\,v$ with the **small-signal conductance** $g = f'(V_Q) = 1/r$. The device is replaced, for signals only, by a resistor (or a transconductance, for a three-terminal device) whose value depends on the bias. The DC and signal problems then separate: the bias is found with the large-signal model, the signal response with a linear network in which DC sources are set to zero (DC voltage sources become shorts and DC current sources open circuits) and capacitors that are large at signal frequencies become shorts. All the linear-network methods of Book 3 then apply to the signal circuit. Book 3 introduces exactly this linearisation as the origin of dependent sources (*Circuits, Signals and Communications*, Ch. 1).

For the exponential diode, $I = I_S e^{V/nV_T}$ gives

$$r_d = \frac{nV_T}{I_Q}$$

which is the result stated in *Engineering Physics* Ch. 18 §18.3.1. It is remarkable for not depending on I_S, the parameter that varies most between parts and with temperature. A 1N4148 ($n = 1.752$) at 1 mA and 300 K has $r_d = 1.752 \times 25.85\ \text{mV}/1\ \text{mA} = 45.3\ \Omega$; an ideal junction ($n = 1$) would have 25.9 Ω. The same structure gives the BJT transconductance $g_m = I_C/V_T$, which depends only on bias current, and the MOSFET $g_m = 2I_D/(V_{GS} - V_{th})$ (Chapters 3 and 4).

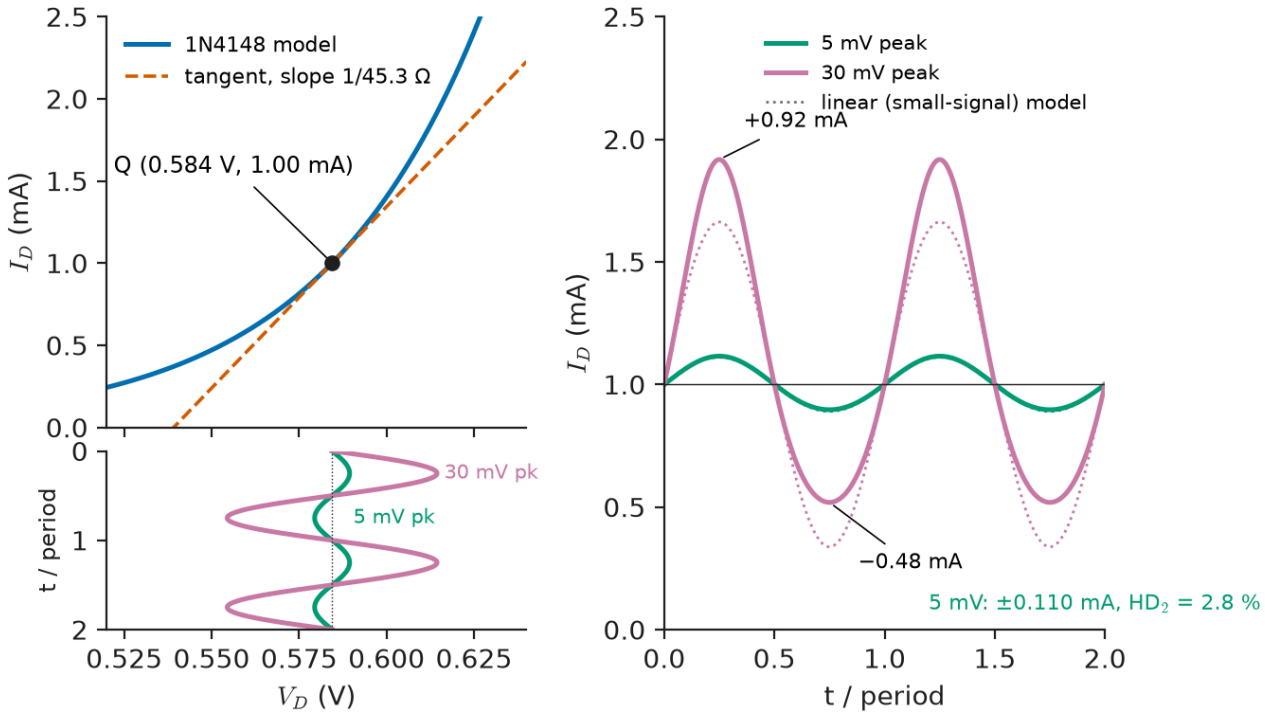


Figure 1.4 — Linearisation of the diode characteristic about a 1 mA operating point, and the distortion of the current when the signal is not small.

How small is small?

The linear model is useful only if the neglected terms are small. For the exponential with a sinusoidal signal $v = V_p \cos \omega t$, write $x = V_p / n V_T$. The current is $I_Q e^{x \cos \omega t}$, whose harmonic amplitudes are modified Bessel functions; for small x the ratio of the second harmonic to the fundamental is

$$\mathrm{HD}_2 \approx \frac{x}{4} = \frac{V_p}{4 n V_T}$$

so the linear model holds to 1 % second-harmonic distortion only for $V_p < 0.04\, n V_T$, about 1.8 mV for the 1N4148 and 1.0 mV for an ideal junction or the BJT's base–emitter junction. Above roughly $n V_T$ the waveform is visibly distorted (Figure 1.4). Square-law devices are more forgiving: the MOSFET in saturation has $\mathrm{HD}_2 = V_p / [4(V_{GS} - V_{th})]$, so a 200 mV overdrive tolerates a signal some eight times larger than an exponential junction. This is one of the main trade-offs between bipolar and MOS amplifiers, and Chapter 7 returns to it.

Worked example — a diode as a small-signal attenuator. A 1N4148 biased at 1 mA is used as the shunt arm of a signal attenuator fed from a 1 kΩ source, a classic gain-control element in old AGC circuits. The small-signal attenuation is $r_d / (R + r_d) = 45.3 / 1045.3 = 0.0433$, i.e. −27.3 dB. Doubling the bias to 2 mA halves r_d to 22.6 Ω and gives −33.1 dB: the attenuation is set electronically by the bias current. For a 5 mV peak signal at the diode, $x = 5 / 45.29 = 0.110$, and the second harmonic is $x/4 = 2.8$ % of the fundamental (the exact Bessel-function ratio $I_2(x)/I_1(x)$ also gives 2.76 %). If the signal across the diode is to stay below 1 % distortion, it must be below 1.8 mV peak, which caps the useful input at about $1.8\ \mathrm{mV} / 0.0433 = 42$ mV peak at the source. These are the numbers that made diode attenuators acceptable only for low-level signals, and that motivated the multiplier and variable-gain amplifier circuits of Chapter 10.

> **Status of the principle: Small-signal linearisation**
> *Derived.* The small-signal model is the first-order Taylor expansion of the large-signal characteristic; it is *approximate* in use, valid when the signal is small compared with the characteristic voltage scale ($n V_T$ for junctions, $V_{GS} - V_{th}$ for MOSFETs) and at frequencies where the bias-point capacitances are included.

Small-signal capacitances

At higher frequency the linearisation must include the incremental capacitances dQ/dV of the device's stored charges: the **depletion (junction) capacitance** $C_j(V)$ stated in *Engineering Physics* Ch. 18 §18.3.1, and the **diffusion capacitance** $C_d = \tau_T I_Q / n V_T$ of the minority carriers stored in a forward-biased junction with transit time τ_T. For the 1N4148 ($\tau_T = 20$ ns in its model) at 1 mA, $C_d = 20\ \mathrm{ns} / 45.3\ \Omega = 0.44$ nF, a hundred times its 4 pF zero-bias junction capacitance. Diffusion capacitance is why a forward-biased junction is slow to turn off (Chapter 2, Section 2.2) and why the BJT's C_π grows with collector current (Chapter 3, Section 3.11).

1.7 Temperature effects

Why everything moves

Device characteristics depend on temperature through the thermal voltage $V_T = k_B T / q$ (25.85 mV at 300 K), through the intrinsic concentration $n_i^2 \propto T^3 e^{-E_g / k_B T}$ that sets saturation and leakage currents (*Engineering Physics*, Ch. 6 §6.6.1), through mobility, which falls roughly as $T^{-2.4}$ for electrons in lightly doped silicon, and through the breakdown mechanisms, whose temperature coefficients differ in sign (*Materials Science and Engineering*, Ch. 32 §32.9). The engineering task is to know the size and sign of each effect at the device terminals.

The forward junction voltage: −2 mV/K

The saturation current of a p–n junction is the steepest temperature function in electronics. Compact models write it as

$$I_S(T) = I_S(T_0)\left(\frac{T}{T_0}\right)^{X_{TI}/n} \exp\left[\frac{E_g}{nV_T}\left(\frac{T}{T_0}-1\right)\right]$$

with $X_{TI} \approx 3$ and $E_g \approx 1.11$ eV for silicon. For the 1N4148 near room temperature this is a rise of 9.2 % per kelvin: I_S doubles every 7.9 K and grows by a factor of 85 between 25 °C and 85 °C. At constant current the forward voltage must therefore fall. Differentiating $V_D = nV_T \ln(I/I_S)$ at constant I gives

$$\frac{dV_D}{dT} = \frac{V_D - E_g/q - X_{TI}V_T}{T}$$

which, for $V_D = 0.6$ V and $T = 300$ K, is $(0.6 - 1.11 - 0.078)/300 = -1.96$ mV/K. The familiar "−2 mV/K" is therefore not an empirical rule but a derived result, and it is less negative at higher currents (larger V_D). It is the basis of diode and transistor thermometers in every CPU and power module, of the PTAT and CTAT voltages combined in bandgap references (Chapter 10), and of the main bias-stability problem of BJT amplifiers (Chapter 3).

> **Status of the principle: Temperature coefficient of the forward junction voltage**
> *Derived*. It follows from the Shockley law with the Boltzmann temperature dependence of n_i^2; the value, about −1.5 to −2.5 mV/K for silicon junctions, holds in the diffusion-dominated range of current and is modified at high current by the positive temperature coefficient of the series resistance.

Worked example — a 1N4148 from −40 °C to 125 °C. At a constant 1 mA, the model with $X_{TI} = 3$ gives $V_D =$ 0.711 V at −40 °C, 0.581 V at 25 °C, 0.457 V at 85 °C and 0.374 V at 125 °C. Between 25 °C and 85 °C the average slope is −2.06 mV/K. Over the automotive range (−40 °C to 125 °C, AEC-Q101) the forward voltage moves by 337 mV, more than half its room-temperature value. Return to the logger of the chapter opening: at −40 °C, a diode-OR made of two diodes from different batches, each forced to the same voltage by the common output, shares current according to I_S, which may differ by a factor of two between parts; and the battery-side diode now needs 0.71 V where 0.58 V was budgeted, leaving a 1.2 V cell too little headroom to supply the load. A Schottky diode or an "ideal diode" MOSFET controller (Chapter 2) would have avoided both problems.

Reverse leakage

Reverse current in silicon has two parts: the diffusion current I_S, which follows n_i^2, and the generation current from the depletion region, which follows n_i and dominates at room temperature in silicon. The practical rule is that silicon junction leakage roughly **doubles every 10 K**, and Schottky leakage (Chapter 2) every 6–8 K. A 1N4148 with 25 nA at 25 °C and 20 V therefore leaks of the order of 1.6 µA at 85 °C and 25 µA at 125 °C, enough to discharge a sample-and-hold capacitor or offset a high-impedance sensor input.

Self-heating and junction temperature

A device dissipating power P heats its junction above the ambient temperature T_A by

$$T_J = T_A + P\,R_{\theta JA}$$

where $R_{\theta JA}$ is the junction-to-ambient thermal resistance, the series sum of junction-to-case, case-to-heatsink and heatsink-to-ambient resistances. The thermal-resistance network, its electrical analogy and the transient thermal RC are derived in *Engineering Physics*, Ch. 14 §14.7; here they are design tools. Datasheets give $R_{\theta JC}$ for the package and $R_{\theta JA}$ for a specified board (JEDEC JESD51 test boards), and the device's maximum junction temperature $T_{J(max)}$, typically 150 °C for silicon signal and power devices, 175 °C for automotive-grade MOSFETs and up to 200 °C for some SiC devices.

Worked example — junction temperature of a rectifier on a small pad. A 1 A rectifier diode in an SMA package carries 1.0 A DC in a 24 V industrial input, with $V_F \approx 0.95$ V at the operating temperature. Its datasheet gives $R_{\theta JA} = 75$ K/W on a minimum-pad FR-4 board. The dissipation is 0.95 W, the temperature rise $0.95 \times 75 = 71$ K, and in a cabinet at 40 °C the junction reaches 111 °C. That is inside the 150 °C limit with a 39 K margin, but every extra 10 K of cabinet temperature or a smaller copper pad eats a quarter of it.

Thermal runaway and electrothermal feedback

Self-heating closes a feedback loop: dissipation raises temperature, and temperature changes the characteristic. If a temperature rise *increases* dissipation, the loop has positive feedback, and if the loop gain exceeds one the temperature runs away. A BJT biased at constant V_{BE} is the classic case: I_C rises about 7 % per kelvin at fixed V_{BE}, so the collector power rises with temperature. Parallel-connected bipolar devices and diodes suffer the same instability, with the hottest device taking more current, so they need emitter or series ballast resistors. MOSFETs at high gate drive are the opposite case: their on-resistance rises with temperature (mobility falls), a hot device takes less current, and paralleled MOSFETs share current naturally, a property of the material (*Materials Science and Engineering* Ch. 32 §32.9) that power electronics relies on (Chapter 29). The stability condition can be stated in one line: the loop is stable if $R_{\theta JA}\,\partial P/\partial T_J < 1$.

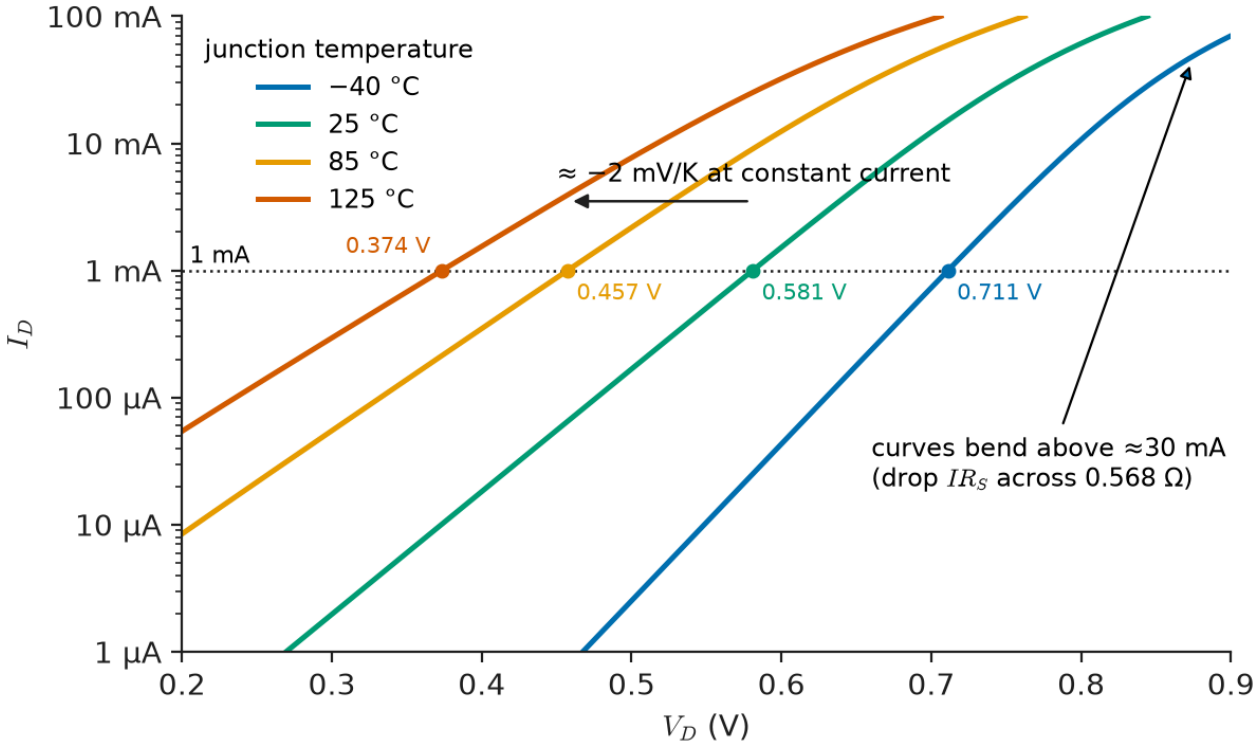


Figure 1.5 — Forward characteristic of a silicon signal diode at four junction temperatures.

1.8 Device parameters

How a datasheet is organised

A datasheet is a contract between the manufacturer and the designer, and its structure tells which numbers can be designed on. Most follow the layout of the IEC 60747 series and the JEDEC registration format:

1. **Absolute maximum ratings.** Limits that must never be exceeded, even momentarily, under any condition: maximum reverse or collector–emitter voltage, continuous and peak current, power dissipation at a stated case or ambient temperature, junction and storage temperature, ESD withstand. They are not operating conditions. A device operated at its absolute maximum has no margin and an undefined lifetime.
2. **Recommended operating conditions** (for ICs and some discrete devices): the range over which the electrical characteristics apply.
3. **Electrical characteristics.** Parameters with minimum, typical and maximum values, each at stated test conditions (current, voltage, temperature, pulse width). Minimum and maximum values are guaranteed, by test or by design and characterisation. Typical values are the mean of a sample and are not guaranteed.

4. **Typical characteristic curves.** Graphs of parameters against current, voltage and temperature for a typical part. They show trends and are essential for understanding a device, but they carry no guarantee.
5. **Thermal characteristics and package data**, including $R_{\theta JC}$, $R_{\theta JA}$ on a specified board, and derating curves.

The single most important habit in reading a datasheet is to design on guaranteed limits and to use typical curves only to understand behaviour. A bias circuit that works only with a β of 200 when the datasheet guarantees a minimum of 100 is not a design.

Table 1.4 — Key parameters of four common small-signal devices (datasheet values; typical where marked).

Parameter	1N4148 (signal diode)	1N4007 (rectifier)	2N3904 (NPN BJT)	2N7002 (N-MOSFET)
Maximum blocking voltage	$V_{RRM} = 100$ V	$V_{RRM} = 1000$ V	$V_{CEO} = 40$ V	$V_{DS} = 60$ V
Maximum continuous current	200 mA	1 A average	$I_C = 200$ mA	115–300 mA (by vendor)
Forward/on voltage	$V_F \le 1.0$ V at 10 mA	$V_F \le 1.1$ V at 1 A	$V_{CE(sat)} \le 0.2$ V at 10 mA/1 mA	$R_{DS(on)} \approx 1$–$2\ \Omega$ typ at 10 V
Leakage	$I_R \le 25$ nA at 20 V	$I_R \le 5$ µA at 1000 V	$I_{CEX} \le 50$ nA	$I_{DSS} \le 1$ µA
Gain or threshold	—	—	h_{FE} = 100–300 at 10 mA	$V_{GS(th)}$ = 1.0–2.5 V
Speed	$t_{rr} \le 4$ ns	$t_{rr} \approx 2$ µs (typ, unspecified)	$f_T \ge 300$ MHz	$t_{on}, t_{off} \approx 20$ ns
Capacitance	$C_j \le 4$ pF at 0 V	≈ 15 pF at 4 V	$C_{obo} \le 4$ pF	$C_{iss} \approx 20$–50 pF
Power, package	500 mW, DO-35	— , DO-41	625 mW, TO-92	200–350 mW, SOT-23

Two features of this table recur throughout the book. Gains and thresholds have wide guaranteed ranges (a factor of three for h_{FE}, 1.5 V for $V_{GS(th)}$), and good circuits are designed so that the result does not depend on them. And the same part number from different manufacturers can differ in ratings (the 2N7002's current rating ranges from 115 mA to 300 mA by vendor), so a bill of materials must name the actual source or a qualified set of sources.

Test conditions and derating

Every guaranteed parameter carries test conditions, and a value is meaningless without them. A MOSFET's $R_{DS(on)}$ at $V_{GS} = 10$ V and 25 °C may be half its value at $V_{GS} = 4.5$ V and 125 °C. A diode's V_F is often measured with a 300 µs pulse to keep the junction at 25 °C, so it is lower than the voltage the same diode shows when it has warmed up under continuous current. Power ratings are given at $T_C = 25$ °C for power devices, a condition that requires an infinite heatsink, or at $T_A = 25$ °C on a specified board for small packages. The **derating curve** reduces the allowed dissipation linearly to zero at $T_{J(max)}$:

$$P_{max}(T_A) = \frac{T_{J(max)} - T_A}{R_{\theta JA}}$$

For the 2N3904, 625 mW at 25 °C with $T_{J(max)} = 150$ °C implies $R_{\theta JA} = 200$ K/W and a derating of 5.0 mW/K; at 85 °C only 325 mW are allowed (Figure 1.6). Professional practice adds **design derating**: voltages and junction temperatures are commonly kept to 70–80 % of their ratings in industrial equipment, and space standards (ECSS-Q-ST-30-11) are stricter still.

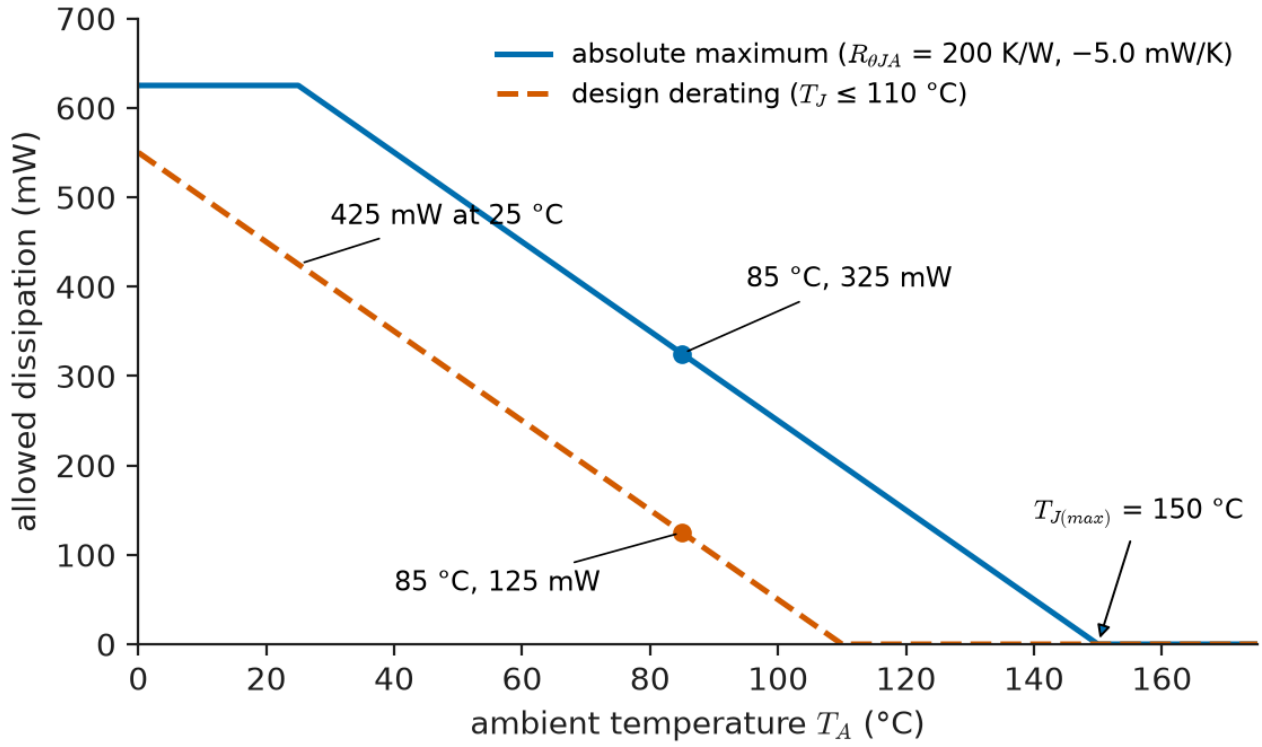


Figure 1.6 — Power derating of a small-signal transistor in a TO-92 package, with a design-derating envelope.

> **Status of the principle: Absolute maximum ratings versus characteristics**
> *Empirical.* Ratings and guaranteed limits come from the manufacturer's characterisation and production testing of a population of parts, often with statistical guard bands; they are valid only at the stated test conditions and say nothing about behaviour outside them.

1.9 Practical limitations

Every model in this chapter fails somewhere. The failures are not arbitrary: each is a piece of physics the model leaves out, or a change of regime. The list below is the checklist an experienced designer runs through for every device in a circuit.

Voltage: breakdown

Every junction breaks down at a high enough reverse voltage, by avalanche multiplication above about 6 V in silicon and by band-to-band tunnelling (the Zener effect) below about 5 V (*Materials Science and Engineering*, Ch. 32 §32.10). Breakdown is not destructive in itself, and Zener and TVS diodes are designed to operate in it (Chapter 2), but the dissipation $V_{BR}I$ is large and concentrated. Gate oxides break down irreversibly: the 20 V V_{GS} limit of a typical power MOSFET is set by a gate oxide 50–100 nm thick, and the 1–4 V limits of logic transistors by oxides a few nanometres thick. Voltage ratings must survive transients (inductive kick, ESD, line surges), which is why protection devices exist (Chapter 2, Section 2.10).

Current and power: the safe operating area

Current is limited by bond wires and metallisation (fusing, electromigration under JEDEC JEP122 lifetimes) and, more usually, by heat. For transistors the combined limits are drawn as a **safe operating area** (SOA) in the I–V plane: a current limit, a power-dissipation hyperbola $IV = P_{max}$, a voltage limit, and for bipolar transistors a **second-breakdown** line where current crowds into hot spots and the device fails below its average power rating. Short pulses

are allowed larger areas because the die's thermal capacitance absorbs the energy (the transient thermal impedance $Z_{\theta JC}(t)$). Power-device SOA and its use in switching design belong to Chapters 5 and 29.

Frequency: capacitance and transit time

Static models ignore stored charge. Junction and diffusion capacitances (Section 1.6) form RC time constants with the circuit, and the transit time of carriers across a base or channel sets the fastest response the device can give, summarised in the transition frequency f_T (Chapter 3, Section 3.11). Package parasitics add series inductance, about 1 nH per millimetre of lead and 0.5–5 nH for surface-mount packages, which matters for fast edges: a 10 A/ns current slope in a power stage through 5 nH induces 50 V. When the device or its interconnections approach a fraction of a wavelength, lumped models fail altogether (*Engineering Physics*, Ch. 18 §18.9; transmission lines in *Circuits, Signals and Communications*, Ch. 14).

Noise

Every device adds random fluctuations to the signals it carries. The thermal noise of resistances ($\sqrt{4k_BTR} = 4.07\ \text{nV}/\sqrt{\text{Hz}}$ for 1 kΩ at 300 K), the shot noise of currents crossing a junction ($\sqrt{2qI} = 17.9\ \text{pA}/\sqrt{\text{Hz}}$ at 1 mA) and flicker ($1/f$) noise are defined in *Circuits, Signals and Communications*, Ch. 29 §29.3. What this book adds is their device origin and scale: shot noise follows the bias current, so low-noise amplifiers choose the bias that balances current and voltage noise against source impedance; flicker noise is far larger in MOSFETs than in BJTs, with corner frequencies from a few hertz in precision bipolar op-amps to hundreds of kilohertz in minimum-size CMOS (Chapters 8 and 10).

Spread, tolerance and worst-case design

No two devices are identical. Gains, thresholds and saturation currents vary by factors of two or three between parts and between lots; matching between adjacent transistors on one die is far better (0.1–1 % in I_S, a few millivolts in V_{BE} or V_{th}), which is why integrated circuits rely on ratios rather than absolute values (Chapter 10). A design must work at every combination of tolerances it can meet. **Worst-case analysis** sets each parameter to the limit that pushes the output in the same direction; it is safe but can be very pessimistic when many parameters vary. **Root-sum-square** and **Monte Carlo** analyses treat parameters as independent random variables and predict a distribution and a yield. Safety-critical, aerospace and medical designs usually require the worst-case method; consumer products often accept a statistical yield of a few sigma.

Reliability and ageing

Devices wear out. Electromigration in metal, hot-carrier injection and bias-temperature instability in MOSFETs, time-dependent dielectric breakdown of oxides, and solder and bond-wire fatigue under thermal cycling all depend on temperature. Their rates are usually modelled with an Arrhenius acceleration factor

$$AF = \exp\left[\frac{E_a}{k_B}\left(\frac{1}{T_{use}} - \frac{1}{T_{stress}}\right)\right]$$

with activation energies E_a of 0.3–1.0 eV depending on mechanism. With $E_a = 0.7$ eV, a part operated at a junction temperature of 85 °C ages about 9.8 times faster at 125 °C and about 8 times slower at 55 °C. This is the quantitative reason for the design derating of Section 1.8 and for the 1000-hour high-temperature operating-life tests of JEDEC JESD22-A108 and AEC-Q101.

> **Status of the principle: Arrhenius acceleration of failure mechanisms**
> *Empirical.* The exponential temperature dependence is motivated by thermally activated processes, but activation energies are fitted per mechanism and per technology; extrapolating far outside the stress range, or to mechanisms driven by cycling rather than steady temperature, is unreliable.

ESD and handling

A person walking on carpet can charge to 10–15 kV. The IEC 61000-4-2 test models a discharge from a human finger through 330 Ω and 150 pF, with a peak current of 30 A within 1 ns at an 8 kV contact discharge. Component-level models (human-body model, 1.5 kΩ and 100 pF, ANSI/ESDA/JEDEC JS-001; charged-device model, JS-002) set the handling ratings of 0.5–8 kV printed on datasheets. MOSFET gates and the inputs of CMOS ICs are the most vulnerable; on-chip clamps protect them during handling, and external TVS arrays protect connectors in the field (Chapter 2).

What remains open

Open question	Where it is answered
How does a real p–n junction depart from the ideal law, and how are diodes specialised for rectification, regulation, light, varactor tuning and protection?	Chapter 2
How is the BJT modelled from Ebers–Moll to hybrid-π, biased against β and temperature spread, and switched?	Chapter 3
How do threshold voltage, short-channel effects and the power-MOSFET structure change the square-law model?	Chapter 4
What do safe operating areas, thyristor latching and wide-bandgap devices add to the limits of this chapter?	Chapter 5
How are biased devices assembled into amplifiers, and how does negative feedback reduce sensitivity to device spread and nonlinearity?	Chapter 7, Chapter 9
How do matching and temperature behaviour become current mirrors and bandgap references?	Chapter 10
How are switching losses, junction temperature and protection handled in power converters?	Chapter 29
What are noise processes and noise figure as signal-level quantities?	*Circuits, Signals and Communications*, Ch. 29
What physics sets I_S, n_i, mobility and lifetime in the first place?	*Engineering Physics*, Ch. 6 and 14; *Materials Science and Engineering*, Ch. 32

Next: Chapter 2 applies these ideas to the p–n junction and its specialised descendants, the rectifier, Zener, Schottky, photodiode, LED, solar cell, varactor and protection diodes.

Chapter 2 — Semiconductor Junction Devices

Part I · Electronic Devices

A 24 V industrial sensor module passes every bench test and then fails in the field at a rate of a few per thousand, always on lines fed by long cables through the plant. The post-mortem finds three different junction devices at fault. A reverse-polarity diode in the supply input has run hot enough in a 60 °C cabinet to discolour the board. The transient-voltage suppressor on the same line is rated for the right working voltage but clamps too high during the surges of nearby contactors, so the regulator behind it breaks down. And an infrared photodiode in the optical position sensor, run with no reverse bias to "save a component", is too slow for the new, faster conveyor. Each failure comes from a property the ideal diode law does not contain: series resistance and heating, avalanche breakdown and its dynamic resistance, voltage-dependent junction capacitance. This chapter takes the junction from that law to the specialised devices built on it and the datasheet parameters by which each is chosen.

> **How does a real p–n junction depart from the ideal Shockley law in forward bias, reverse bias, breakdown and switching?**
> **How are junctions specialised to rectify, regulate, switch fast, detect and emit light, convert sunlight, tune circuits and absorb transients, and what limits each? Which datasheet parameters decide the choice of each device, and how are its losses and temperatures estimated?**

Concept	Engineering destination
PN-junction operation	Depletion width, field and capacitance; breakdown voltage; stored charge
Diode characteristics	Ideality, series resistance, leakage, reverse recovery in every diode circuit
Rectifier diodes	Mains rectifiers, freewheel and boost diodes, surge and recovery ratings
Zener diodes	Shunt references, clamps, gate protection, level shifting
Schottky diodes	Low-voltage rectification, OR-ing, SiC boost diodes in PFC and EV chargers
Photodiodes	Optical receivers, lidar, pulse oximeters, encoders, light sensors
LEDs	Indicators, lighting, backlights, optocouplers and IR links
Solar cells	PV modules, energy harvesting for IoT nodes, spacecraft power
Varactor diodes	VCOs, PLL synthesisers, tuners, RF phase shifters
Protection diodes	ESD and surge protection (IEC 61000-4-2/-5, ISO 7637-2), flyback clamps, reverse-polarity protection

Roadmap. The junction's electrostatics and switching (2.1) → the real diode characteristic (2.2) → specialised by voltage, current and speed: rectifier (2.3), Zener (2.4) and Schottky (2.5) diodes → junctions and light: photodiode (2.6), LED (2.7), solar cell (2.8) → the junction as a capacitor (2.9) → the junction as a protector (2.10).

2.1 PN-junction operation

Where *Engineering Physics* leaves off

Engineering Physics derives the ideal diode law $I = I_0(e^{eV/k_BT} - 1)$ from minority-carrier diffusion, with I_0 set by diffusion lengths and the minority densities $n_{p0} = n_i^2/N_A$ and $p_{n0} = n_i^2/N_D$ (Ch. 14 §14.5.4); it adds the ideality factor, the small-signal resistance and the junction capacitance $C_j = C_{j0}(1 - V_D/\phi_B)^{-m}$ in Ch. 18 §18.3.1, and gives the solar cell's ideal I_{SC} and V_{OC} in Ch. 14 §14.5.5, thermionic emission over a metal–semiconductor barrier in §14.6, and light

absorption and emission in Ch. 12. Recombination, lifetime and impact ionisation as material properties are in *Materials Science and Engineering*, Ch. 32 §32.7–32.10, and the optoelectronic materials in Ch. 35. Chapter 1 supplied the modelling vocabulary. This chapter adds what lies beyond the ideal law: depletion-region numbers, the real characteristic, breakdown, stored charge, and the specialised junction devices with their ratings.

Depletion-region numbers

In the depletion approximation, an abrupt junction with doping N_A on the p side and N_D on the n side has a built-in potential and, for the common one-sided case $N_A \gg N_D$, a depletion width, peak field and capacitance per unit area

$$V_{bi} = V_T \ln\frac{N_A N_D}{n_i^2}, \qquad W = \sqrt{\frac{2\varepsilon_s (V_{bi} + V_R)}{q N_D}}, \qquad \mathcal{E}_{max} = \frac{q N_D W}{\varepsilon_s}, \qquad \frac{C_j}{A} = \frac{\varepsilon_s}{W}$$

The depletion region spreads almost entirely into the lightly doped side, which therefore decides breakdown voltage and capacitance, while the heavily doped side decides injection.

Worked example — a p⁺n junction at 0, 5 and 10 V. Take $N_A = 10^{19}$ cm^{-3}, $N_D = 10^{16}$ cm^{-3}, silicon at 300 K ($n_i = 1.0 \times 10^{10}$ cm^{-3}, $\varepsilon_s = 11.7\varepsilon_0$, $V_T = 25.85$ mV). Then $V_{bi} = 0.02585 \ln(10^{35}/10^{20}) = 0.893$ V. At zero bias $W = 0.340$ µm, $\mathcal{E}_{max} = 52.6$ kV/cm and $C_j/A = 30.5$ nF/cm², i.e. 30.5 pF for a 0.1 mm² die. At 5 V reverse: $W = 0.873$ µm, $\mathcal{E}_{max} = 135$ kV/cm, 11.9 pF. At 10 V: $W = 1.19$ µm, $\mathcal{E}_{max} = 184$ kV/cm, 8.7 pF. The capacitance follows $(V_{bi} + V_R)^{-1/2}$, the abrupt-junction case $m = 0.5$ of the physics book's formula. Breakdown occurs when the peak field reaches the critical field, about 0.40 MV/cm at this doping (*Materials Science and Engineering* Ch. 32 §32.10), which gives $V_{BR} \approx \varepsilon_s \mathcal{E}_c^2/(2qN_D) = 52$ V. Repeating the estimate at 10^{15} and 10^{17} cm^{-3} gives about 290 V and 9 V: every decade less doping buys roughly a factor of 5.6 in blocking voltage, at the price of a wider, more resistive drift region, the trade-off that SiC, with ten times the critical field, relaxes (Chapter 5).

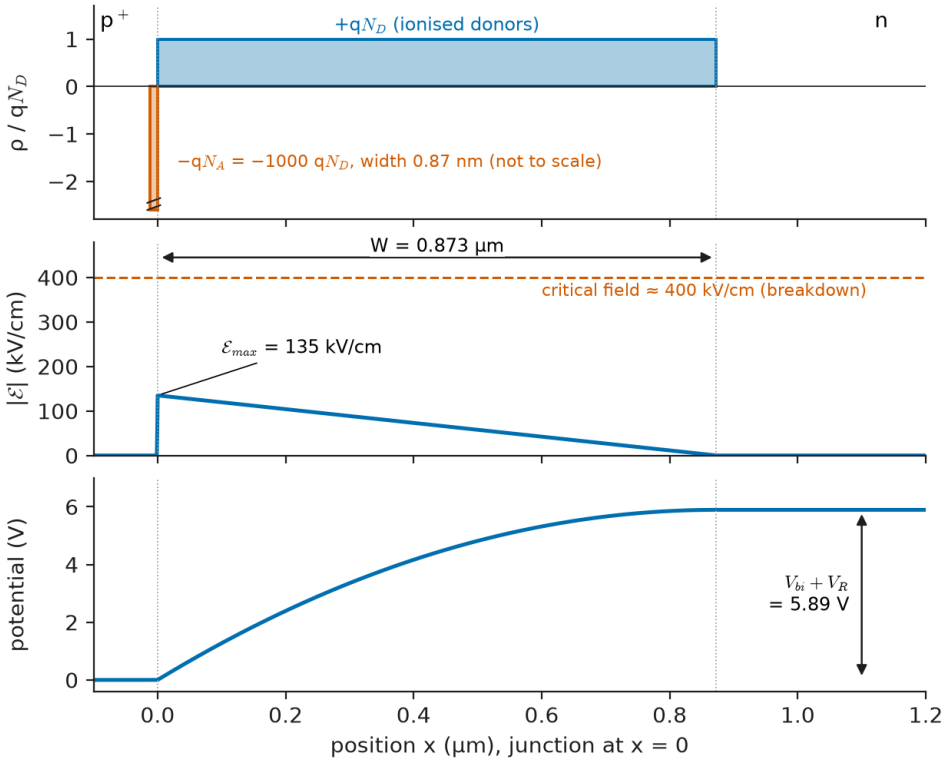


Figure 2.1 — Charge density, electric field and potential across an abrupt p+n silicon junction at 5 V reverse bias.

Breakdown: avalanche and Zener

Two mechanisms end the blocking region. In **avalanche breakdown** carriers accelerated by the depletion field gain enough energy to create electron–hole pairs by impact ionisation, and the multiplication runs away; it dominates in

silicon junctions breaking down above about 6 V, and its voltage *rises* with temperature, about +0.06 to +0.1 %/K, because phonon scattering steals energy from the carriers. In **Zener (tunnelling) breakdown**, in heavily doped junctions with depletion widths under about 10 nm, electrons tunnel directly from the valence band on the p side to the conduction band on the n side; it dominates below about 5 V, and its voltage *falls* with temperature because the band gap shrinks. Between about 5 and 6 V both contribute and the temperature coefficients cancel (Section 2.4). Both mechanisms are reversible; the device fails by heating, not by breakdown itself.

Stored charge and switching

A forward-biased junction holds a charge of injected minority carriers, $Q_s = \tau_T I_F$, where the transit time τ_T is close to the minority-carrier lifetime for a long-base diode and much shorter for a thin, narrow-base one. Before the junction can block reverse voltage this charge must be removed, by reverse current and by recombination. That is the origin of **reverse recovery** (Section 2.2), of the turn-off delay of saturated BJTs (Chapter 3), and of the switching losses of every bipolar power device. Lifetime is therefore a design variable: power-diode makers shorten it deliberately with gold, platinum or electron irradiation, accepting higher leakage and forward voltage to switch faster (*Materials Science and Engineering* Ch. 32 §32.8).

> **Status of the principle: Depletion approximation**
> *Approximate.* It assumes the depletion region is free of mobile carriers and has abrupt edges; it predicts width, field and capacitance to within a few per cent when $V_{bi} + V_R \gg V_T$, and fails in forward bias near and beyond V_{bi}, where injected carriers fill the depletion region.

2.2 Diode characteristics

The four regimes of forward current

Plotted on a logarithmic current axis, a real silicon diode's forward characteristic is not one straight line but four segments (Figure 2.2):

1. **Recombination current** (low current, typically below 1–10 μA in a signal diode): carriers recombine inside the depletion region through mid-gap traps, giving $I \propto e^{V/2V_T}$, a slope of 120 mV per decade.
2. **Diffusion current** (the ideal law): $I \propto e^{V/V_T}$, 60 mV per decade at 300 K. Over this range the device follows the physics book's equation.
3. **High injection**: when the injected minority density approaches the doping of the lightly doped side, the slope returns towards 120 mV per decade. The physics book's low-injection assumption is violated here.
4. **Ohmic region**: at the highest currents the drop across the bulk, contacts and leads, IR_S, dominates, and the curve bends towards a straight line on a linear plot.

A single exponential with an average ideality factor n between 1 and 2 and a series resistance R_S, the model of Chapter 1, fits about three decades of current around the range it was extracted from. The 1N4148's fitted $n = 1.752$ is such an average. Precision circuits (log amplifiers, temperature sensors, bandgap references) therefore use transistor base–emitter junctions, whose collector current keeps n within 1.00–1.01 over six or more decades because recombination current flows in the base (Chapter 3).

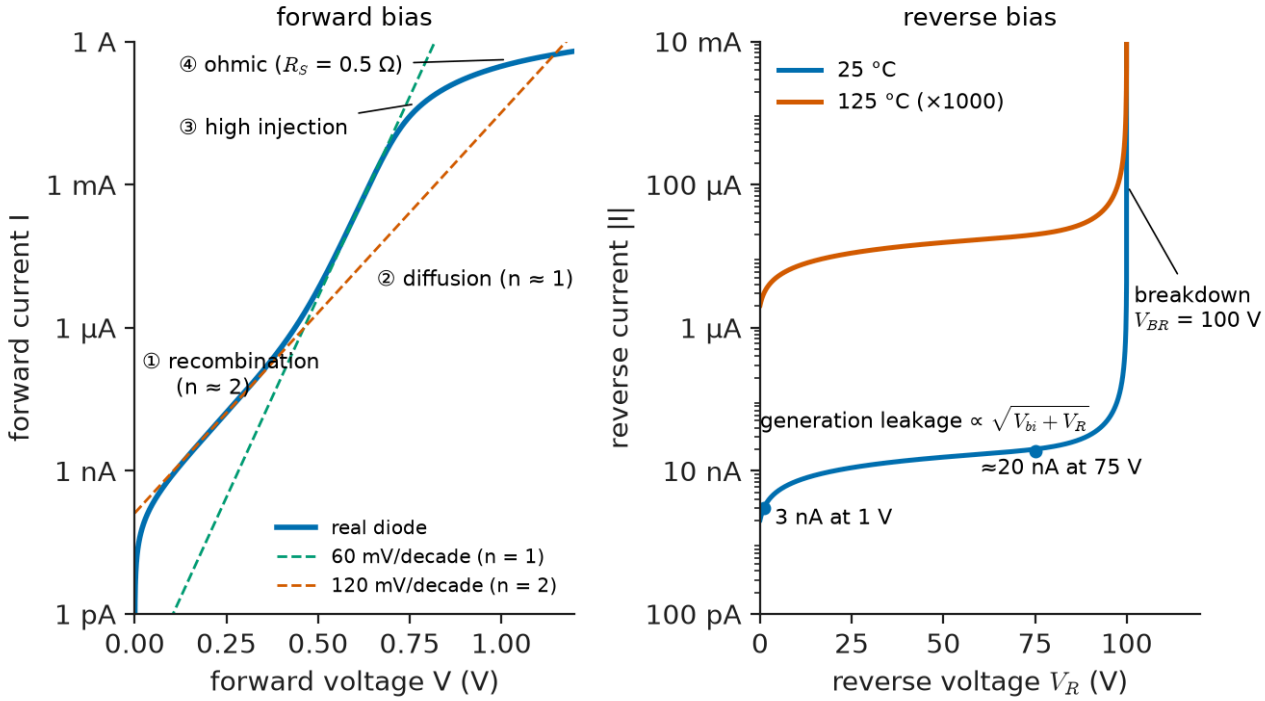


Figure 2.2 — Complete characteristic of a real silicon diode on a logarithmic current scale, showing the four forward regimes and the reverse leakage and breakdown.

Reverse current

In reverse bias the ideal law predicts a constant $-I_0$, picoamperes or less for a small silicon diode. Measured leakage is several orders of magnitude larger and grows with voltage, because it is dominated by **generation current** in the depletion region (proportional to $n_i W$, and so to roughly $\sqrt{V_{bi}+V_R}$) and by **surface leakage** at the edges of the die. It doubles roughly every 10 K in silicon (Chapter 1, Section 1.7). The hot value on the datasheet is the one that matters for sample-and-hold circuits, high-impedance inputs and battery standby current.

Capacitances

The **junction capacitance** C_j (Section 2.1) dominates in reverse bias: a few picofarads for signal diodes, tens to hundreds for rectifiers, and the working element of varactors (Section 2.9). The **diffusion capacitance** $C_d = \tau_T I_F / n V_T$ dominates in forward bias; it is the small-signal face of the stored charge, and it makes forward-biased PIN diodes useful as current-controlled RF resistors in attenuators and transmit–receive switches.

Reverse recovery

A conducting diode driven into reverse does not stop conducting at once. Figure 2.3 shows a diode commutated at a fixed di/dt (set by a switch and the commutation inductance). The current falls through zero and continues negative, supported by the stored charge, until the junction edge is depleted at the peak reverse-recovery current I_{RRM}; it then decays to the leakage value. The total time is the reverse-recovery time $t_{rr} = t_a + t_b$, the integral of the reverse current the **recovered charge** $Q_{rr} \approx \tfrac{1}{2} I_{RRM} t_{rr}$, and the ratio $S = t_b/t_a$ the **softness**. A snappy diode ($S \ll 1$) cuts off its reverse current abruptly, and $L\,di/dt$ in the circuit inductance then produces voltage overshoot and ringing, a common source of EMI and of overvoltage failure in converters.

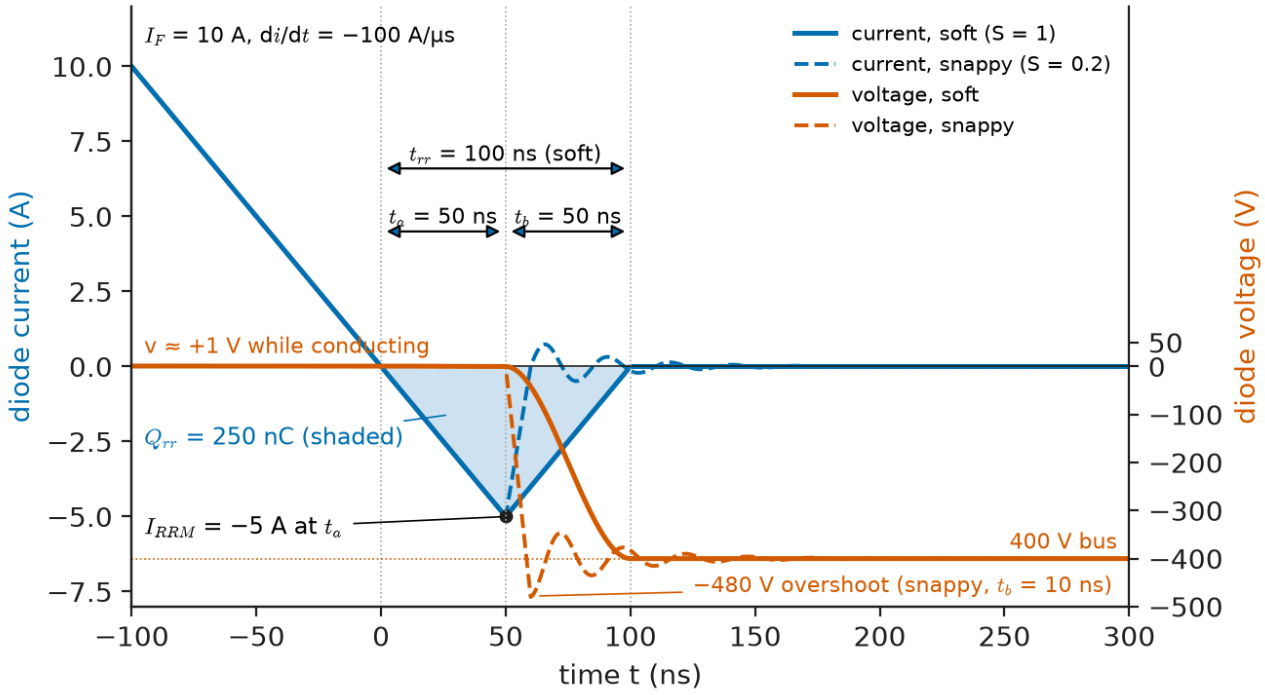


Figure 2.3 — Reverse-recovery current and voltage of a fast power diode commutated at a fixed rate, comparing soft and snappy recovery.

The loss caused by recovery appears mostly in the *opposite* switch, which must carry the load current plus the reverse current while it still supports most of the supply voltage. A rough upper estimate of the extra loss is $P_{rr} \approx Q_{rr} V_R f_{sw}$: a fast diode with $Q_{rr} = 50$ nC in a 400 V converter at 100 kHz costs about 2 W. This is why silicon PN diodes are replaced by SiC Schottky diodes, which have no stored charge, in high-voltage power-factor correctors and EV on-board chargers (Section 2.5 and Chapter 29).

> **Status of the principle: Recovered charge equals stored charge**
> *Approximate.* Q_{rr} is at most the stored charge $\tau_T I_F$ and is reduced by recombination during the fall of the current, so it depends on di/dt, forward current and temperature; datasheet values hold only at the stated test conditions, and Q_{rr} typically doubles between 25 °C and 125 °C.

2.3 Rectifier diodes

What a rectifier is specified for

A rectifier diode is a p–i–n structure optimised for current and blocking voltage, not for speed: a heavily doped p⁺ anode, a lightly doped n⁻ drift region whose thickness and doping set the voltage rating (Section 2.1), and an n⁺ cathode. In conduction the drift region is flooded with stored carriers (conductivity modulation), which keeps the forward voltage near 1 V even through tens of micrometres of nearly intrinsic silicon, but makes the diode slow to recover. The datasheet parameters are:

- V_{RRM}, the repetitive peak reverse voltage, and V_{RSM}, the non-repetitive surge value;
- $I_{F(AV)}$, the average forward current at a stated lead or case temperature, usually for half-sine conduction;
- I_{FSM}, the non-repetitive surge current, typically a single 8.3 ms or 10 ms half-sine (one half cycle of 60 or 50 Hz), and the fusing integral I^2t, which sets protection against capacitor-charging inrush;
- V_F at rated current, and for power devices the threshold–slope model $V_F = V_{F0} + r_T I_F$;
- I_R at V_{RRM}, at 25 °C and at high temperature;
- t_{rr} (or Q_{rr}) at stated I_F, di/dt and V_R.

Table 2.1 — Representative rectifier and fast diodes (datasheet maxima unless marked typical).

Part (class)	V_{RRM}	$I_{F(AV)}$	I_{FSM}	V_F at rated I_F	t_{rr}
1N4007 (standard recovery)	1000 V	1 A	30 A	1.1 V	≈ 2 µs (typ)
1N5408 (standard recovery)	1000 V	3 A	200 A	1.2 V	≈ 2 µs (typ)
UF4007 (ultrafast)	1000 V	1 A	30 A	1.7 V	75 ns
MUR460 (ultrafast)	600 V	4 A	110 A	1.28 V	75 ns
650 V, 10 A SiC Schottky	650 V	10 A	70–90 A	1.5 V (typ, 25 °C)	none (capacitive charge ≈ 25 nC)

Standard-recovery diodes belong in 50/60 Hz rectifiers, where their slow, gentle recovery is harmless and generates little conducted noise. At 20 kHz and above, ultrafast silicon or SiC devices are required. The general trade-off is visible in the table: faster recovery costs forward voltage, because the lifetime-killing that speeds recovery also reduces conductivity modulation.

Conduction loss and junction temperature

With the threshold–slope model, the average conduction loss of a diode carrying a current of average I_{av} and RMS value I_{rms} is

$$P_F = V_{F0}\, I_{av} + r_T\, I_{rms}^2$$

because V_{F0} multiplies the instantaneous current and r_T its square. The RMS value matters because rectifier currents are peaky: a diode feeding a reservoir capacitor conducts in short pulses whose RMS value is two to three times the average (Chapter 6 computes them).

Worked example — a 3 A rectifier in a capacitor-input supply. Each diode of a bridge in a 60 W, 24 V linear supply carries $I_{av} = 1.5$ A with $I_{rms} = 3.2$ A (pulsed conduction). For a 1N5408 class device, fit $V_{F0} = 0.75$ V and $r_T = 0.07\ \Omega$ to the typical curve (0.96 V at 3 A, inside the 1.2 V maximum). The loss is $0.75 \times 1.5 + 0.07 \times 3.2^2 = 1.13 + 0.72 = 1.84$ W per diode. With leads soldered to a board at 9.5 mm length, the datasheet gives $R_{\theta JA} \approx 20$ K/W, so in a 50 °C enclosure $T_J = 50 + 1.84 \times 20 = 87$ °C, comfortably below the 150–175 °C limit. The $r_T I_{rms}^2$ term is 39 % of the loss; using the average current in it instead of the RMS value would have underestimated the total by 30 %.

2.4 Zener diodes

Breakdown as a feature

A Zener diode is a junction doped so that it breaks down at a controlled voltage V_Z, from about 2.4 V to 200 V, and built to dissipate the power of operating in breakdown. Despite the name, only devices below about 5 V break down by the Zener (tunnelling) mechanism; above about 6 V they are avalanche diodes (Section 2.1). In breakdown the device is modelled as a voltage source V_Z in series with a **dynamic (Zener) impedance** $r_z = dV/dI$, specified at a test current I_{ZT}. Below a knee current I_{ZK}, the characteristic rounds off and r_z rises by an order of magnitude, especially for low-voltage parts, where the tunnelling knee is soft.

Table 2.2 — Zener diode behaviour versus voltage (typical small-signal 300–500 mW parts).

V_Z	Mechanism	r_z at 5 mA	Temperature coefficient	Comment
2.4–3.9 V	Zener (tunnelling)	70–100 Ω	about −2 mV/K	Soft knee; poor reference
4.7–5.6 V	Mixed	10–60 Ω	near zero (±1 mV/K)	Best temperature stability of plain Zeners
6.2–6.8 V	Avalanche	5–15 Ω	+2 to +3 mV/K	Lowest r_z; with a forward diode in series, a compensated reference
10 V	Avalanche	15–20 Ω	about +7 mV/K (+0.07 %/K)	
15–24 V	Avalanche	30–70 Ω	+12 to +20 mV/K	Clamps, gate protection

A 6.2 V avalanche diode in series with a forward-biased junction (whose −2 mV/K cancels the Zener's +2 mV/K) gives the temperature-compensated references of the 1N821–1N829 family, with coefficients down to 5 ppm/K. Buried Zeners in precision ICs reach a few ppm/K; bandgap references (Chapter 10) have replaced Zeners for most low-voltage references, but Zeners remain the simplest clamps.

The shunt regulator as a device application

The classic use of a Zener is a shunt clamp fed through a resistor from an unregulated supply, a direct exercise in the device's ratings (regulators proper are in Chapter 10).

Worked example — a 5.1 V Zener shunt from a 12 V ±10 % rail. The load draws 0 to 10 mA, and the Zener (BZX84C5V1 class, 300 mW in SOT-23, $r_z \approx 30\ \Omega$ typical at 5 mA) needs at least about 2 mA to stay beyond its knee. The worst case for starving the Zener is minimum input and maximum load: $R \le (10.8 - 5.1)/(10 + 2)\ \text{mA} = 475\ \Omega$; choose 470 Ω. The worst case for the Zener's dissipation is maximum input and no load: $I_Z = (13.2 - 5.1)/470 = 17.2$ mA and $P_Z = 5.1 \times 17.2 = 88$ mW, well inside 300 mW; the resistor dissipates $8.1^2/470 = 0.14$ W, so a 0.25 W part is the minimum. Regulation follows from the small-signal model: a 2.4 V change of input moves the output by $2.4 \times 30/(470 + 30) = 144$ mV (line regulation 6 %), and a 10 mA load step moves it by $10\ \text{mA} \times (30 \parallel 470) = 0.28$ V.

> **Status of the principle: Zener model, $V = V_Z + r_z I$**
> *Approximate.* A straight-line fit to the breakdown characteristic around I_{ZT}; it fails below the knee current, where r_z rises sharply, and at high dissipation, where self-heating shifts V_Z by the temperature coefficient times $P_Z R_{\theta JA}$.

2.5 Schottky diodes

The metal–semiconductor junction

A Schottky diode replaces the p⁺ region with a metal (or a metal silicide) on lightly doped n-type silicon. The rectifying barrier of height ϕ_B (0.5–0.9 eV for practical metals on n-Si) passes current by thermionic emission of majority carriers over the barrier, the Richardson–Dushman physics of *Engineering Physics* Ch. 14 §14.6, so the saturation current is

$$I_S = A A^* T^2 e^{-\phi_B / V_T}$$

with $A^* \approx 110$–$120\ \text{A cm}^{-2}\text{K}^{-2}$ for n-type silicon. Two engineering consequences follow.

- **Lower forward voltage.** For a 1 mm² die, $\phi_B = 0.60$ eV gives $I_S = 9.0$ μA at 300 K, against the order of 10^{-12} A for a silicon PN junction of similar area. At 1 A the ideal Schottky drop is $V_T \ln(1/I_S) = 0.30$ V, versus 0.71 V for the PN junction: a saving of about 0.4 V at every current. Lowering the barrier to 0.55 eV saves another 50 mV.

- **No stored minority charge.** Current is carried by majority carriers, so there is no reverse recovery: only the junction capacitance must be charged. Schottky diodes switch in nanoseconds, limited by C_j.

The cost is leakage. The same small barrier that lowers V_F raises the reverse saturation current exponentially, and image-force barrier lowering under reverse field makes leakage grow with voltage. Schottky leakage is milliamperes in a 3 A power part at 100 °C and doubles every 6–8 K. Each 0.1 eV of barrier trades about 100 mV of forward drop against a factor of 50 in leakage. Silicon Schottky diodes are also limited to about 200 V, because the thick drift region needed for higher voltage would add series resistance without the conductivity modulation that keeps PN rectifiers' forward drops low.

Table 2.3 — Representative Schottky diodes (datasheet values).

Part (class)	V_{RRM}	$I_{F(AV)}$	V_F	I_R at V_{RRM}
BAT54 (small signal)	30 V	200 mA	0.32 V at 1 mA; 0.40 V at 10 mA (max)	2 µA at 25 V
1N5819	40 V	1 A	0.60 V at 1 A (max)	1 mA (25 °C), 10 mA (100 °C)
SS34 (SMC)	40 V	3 A	0.50 V at 3 A (max)	0.5 mA (25 °C), 20 mA (100 °C)
650 V SiC Schottky (TO-220)	650 V	10 A	1.5 V at 10 A (typ, 25 °C)	1–50 µA

Worked example — Schottky versus fast PN diode at 1.75 A average. A freewheeling diode in a 3 A, 5 V point-of-load supply carries 1.75 A average. An SS34-class Schottky at its operating temperature drops about 0.45 V, losing $0.45 \times 1.75 = 0.79$ W; a fast-recovery silicon PN diode at about 0.85 V would lose 1.49 W plus its recovery loss. The Schottky saves about 0.7 W, roughly 5 % of a 15 W output. Its price is leakage: at a junction temperature of 100 °C and 12 V reverse, a few milliamperes flow for the 42 % of the time the diode blocks, some 20–40 mW, negligible here. In a sealed automotive enclosure where the junction runs 15 K hotter, leakage grows a further factor of four, and since leakage heats the junction, a poorly cooled Schottky can run away thermally in reverse, a failure PN rectifiers do not show. Low-voltage converters avoid both losses with a synchronous MOSFET (Chapter 30).

SiC Schottky diodes

In silicon carbide, with ten times silicon's critical field, a Schottky drift region for 650 or 1200 V is thin enough to have low resistance (Chapter 5). SiC Schottky diodes have only a small, temperature-independent capacitive charge and a positive temperature coefficient of V_F at rated current, so they parallel well; they are standard as boost diodes in server and telecom power-factor correctors and EV on-board chargers, removing the $Q_{rr} V_R f_{sw}$ loss of Section 2.2.

> **Status of the principle: Thermionic-emission Schottky law**
> *Approximate.* The ideal law with n close to 1.02–1.1 describes Schottky diodes over several decades of forward current; it fails at reverse bias, where barrier lowering and edge leakage make the current voltage-dependent, and at high current, where series resistance dominates.

2.6 Photodiodes

The junction as a light detector

A photon of energy $h\nu > E_g$ absorbed in or near the depletion region creates an electron–hole pair; the junction field separates it and a **photocurrent** I_{ph} flows in the reverse direction. The physics of absorption, cut-off wavelength and detector materials is in *Engineering Physics* Ch. 12 and *Materials Science and Engineering* Ch. 35 §35.5. At the terminals the photodiode is an ordinary diode with a current source in parallel,

$$I = I_S\left(e^{V/nV_T} - 1\right) - I_{ph}, \qquad I_{ph} = \mathcal{R}\,P_{opt}, \qquad \mathcal{R} = \frac{\eta q \lambda}{hc} = \eta\,\frac{\lambda\,[\text{nm}]}{1239.8}\ \text{A/W}$$

where $\mathcal{R}$ is the **responsivity** and η the external quantum efficiency (electrons collected per incident photon). A perfect detector at 850 nm would deliver 0.686 A/W; a good silicon PIN photodiode reaches 0.55–0.6 A/W there ($\eta \approx 80$–90 %).

The same equation describes three uses of one junction (Figure 2.4). In the **photoconductive mode** the diode is reverse biased (third quadrant), the current is $-I_{ph}$ almost independent of voltage, and the response is linear in light over eight to ten decades. In the **photovoltaic mode** the diode drives a high-impedance load or an open circuit, delivering power (fourth quadrant) and a voltage $V_{oc} = nV_T \ln(1 + I_{ph}/I_S)$ that is logarithmic in light; at zero voltage (into a transimpedance amplifier) the current is linear and dark current vanishes, the preferred mode for precision low-light work. The solar cell (Section 2.8) is the photovoltaic mode at large area; in the first quadrant a direct-gap junction emits light (Section 2.7).

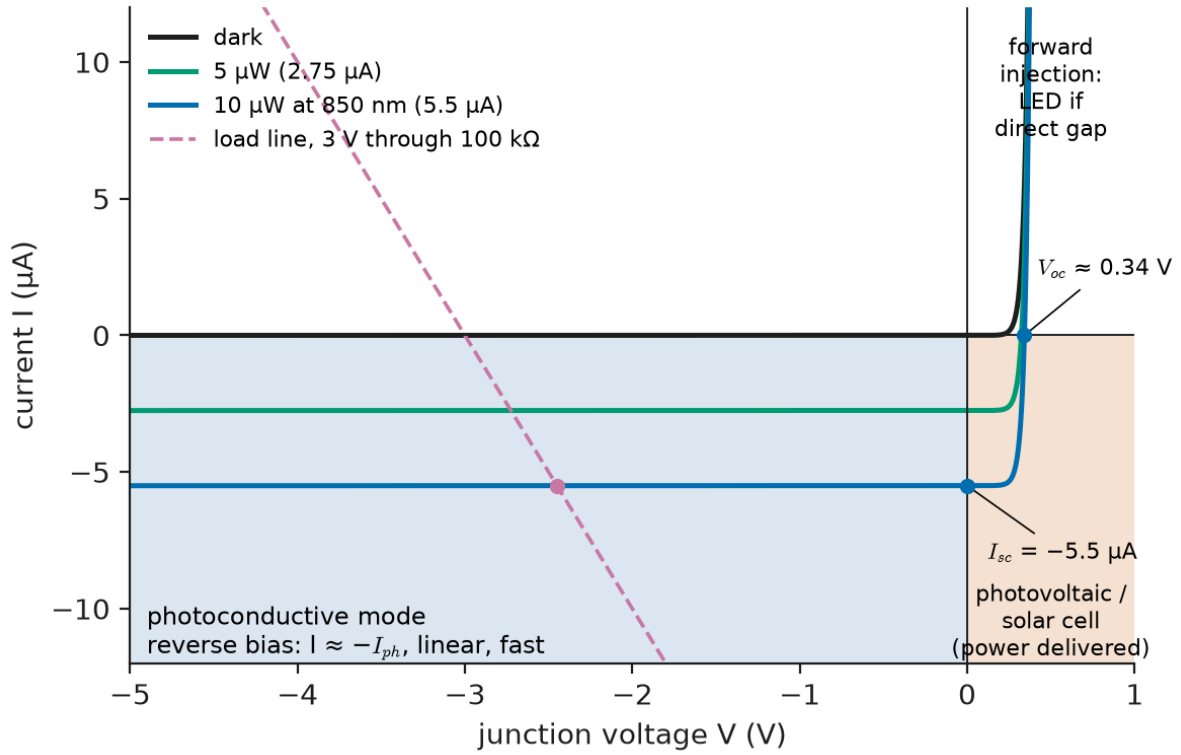


Figure 2.4 — One junction, three uses: the current–voltage characteristic of a p–n junction in the dark and under two illumination levels, with the photoconductive, photovoltaic and light-emitting operating regions.

PN, PIN and avalanche photodiodes

Speed and efficiency both want the light absorbed inside the depletion region, where carriers are collected by drift rather than by slow diffusion. A **PIN** photodiode inserts a nearly intrinsic layer, tens of micrometres thick in silicon, that is fully depleted at a few volts. Its bandwidth is set by the smaller of the RC limit $1/(2\pi R_L C_j)$ and the transit-time limit, roughly $0.45\, v_{sat}/W$: for a 50 µm drift region and $v_{sat} \approx 10^7$ cm/s, about 0.9 GHz. An **avalanche photodiode** (APD) adds a high-field multiplication region and provides internal gain M of 10–20 in InGaAs and 50–200 in silicon, at bias voltages of 30–60 V (InGaAs) and 100–400 V (Si), with an excess-noise penalty and a gain that falls with temperature (the avalanche coefficients of Section 2.1), so APD bias is temperature-compensated. APDs serve lidar, rangefinders and long-reach fibre receivers; single-photon avalanche diodes (SPADs), biased beyond breakdown, count photons in time-of-flight sensors.

Table 2.4 — Representative photodiodes (typical values).

Type (example class)	Spectral range	$\mathcal{R}$ at peak	Dark current	Capacitance	Typical use
Si PIN, 7.5 mm² (BPW34 class)	400–1100 nm	0.6 A/W at 900 nm	2 nA at 10 V	70 pF at 0 V, 25 pF at 3 V	Light sensors, IR remote, pulse oximetry
Si PIN, 1 mm², fast	320–1100 nm	0.5 A/W at 800 nm	0.1 nA at 10 V	3 pF at 10 V	Encoders, optical links to 100 MHz
InGaAs PIN, 75 µm	900–1700 nm	0.95 A/W at 1550 nm	0.5 nA	0.5 pF	Telecom receivers to 10 Gb/s
Si APD, 0.5 mm	400–1100 nm	50 A/W at $M = 100$	1 nA (unity gain)	1–2 pF	Lidar, rangefinders
InGaAs APD	900–1700 nm	9 A/W at $M = 10$	10 nA at $M = 10$	0.3 pF	Long-reach fibre, OTDR

Worked example — reverse bias and speed in the conveyor sensor. The position sensor of the chapter opening uses a BPW34-class photodiode receiving 10 µW of 850 nm light, with responsivity 0.55 A/W ($\eta = 0.55 \times 1239.8/850 = 0.80$), so $I_{ph} = 5.5$ µA. Read across a 1 kΩ load resistor it gives 5.5 mV, with an RC bandwidth of $1/(2\pi \cdot 1\ \text{k}\Omega \cdot 70\ \text{pF}) = 2.3$ MHz at zero bias. A 3 V reverse bias reduces C_j to 25 pF and raises the bandwidth to 6.4 MHz, what the faster conveyor needed. Noise sets the other limit: the photocurrent's shot noise, $\sqrt{2qI_{ph}} = 1.33\ \text{pA}/\sqrt{\text{Hz}}$, is below the resistor's thermal noise current, $\sqrt{4k_BT/R} = 4.07\ \text{pA}/\sqrt{\text{Hz}}$, so the resistor limits the signal-to-noise ratio; a transimpedance amplifier (Chapter 8) breaks this trade-off. In the dark, the 2 nA leakage at 10 V contributes a shot noise of 25 fA/√Hz, which corresponds to a **noise-equivalent power** of $25\ \text{fA}/\sqrt{\text{Hz}}/0.55\ \text{A/W} = 46\ \text{fW}/\sqrt{\text{Hz}}$, the ultimate sensitivity of the diode itself.

> **Status of the principle: Photocurrent proportional to optical power**
> *Fundamental.* Each absorbed photon creates at most one electron–hole pair (outside avalanche gain), so $I_{ph} = \eta q P_{opt}/h\nu$ holds exactly; the linearity of real photodiodes fails only at high photocurrent, when series and load resistance forward-bias the junction, and at low bias, when space charge slows collection.

2.7 LEDs

Forward voltage, photon energy and efficiency

A light-emitting diode is a forward-biased junction in a direct-gap semiconductor, where injected electrons and holes recombine radiatively and emit photons of energy close to E_g (*Engineering Physics*, Ch. 6 §6.6.3; emitter materials and alloys in *Materials Science and Engineering* Ch. 35 §35.4). Modern LEDs use heterostructures with quantum wells to confine carriers, so their electrical behaviour is that of a diode whose forward voltage sits slightly above the **photon voltage** $h\nu/q = 1239.8/\lambda[\text{nm}]$: 2.76 V at 450 nm, 1.98 V at 625 nm, 1.46 V at 850 nm. Series resistance and heterobarriers add a few hundred millivolts at rated current.

Three efficiencies describe an LED: the **internal quantum efficiency** (IQE, the fraction of injected carriers that recombine radiatively), the **external quantum efficiency** (EQE = IQE × light-extraction efficiency, photons out per electron in) and the **wall-plug efficiency** WPE $= P_{opt}/(I_F V_F)$. Blue InGaN LEDs reach EQE above 70 % at low current density, and white power LEDs (a blue die with a yellow phosphor) reach WPE of 35–50 % and luminous efficacies of 150–200 lm/W in production. All of them lose efficiency at high current density (**efficiency droop**, mainly from Auger recombination) and at high temperature. Every watt that is not light is heat in a tiny junction, so thermal design governs brightness, colour and life.

Table 2.5 — Representative LED characteristics (typical values at 25 °C).

Colour (material)	Peak wavelength	V_F at rated current	Light-output temperature coefficient	Use
Infrared (GaAs)	940 nm	1.2–1.4 V at 50 mA	−0.3 to −0.5 %/K	Remote controls, optocouplers
Infrared (AlGaAs)	850 nm	1.4–1.7 V at 100 mA	−0.3 to −0.5 %/K	Proximity sensing, IR illumination, IrDA
Red (AlInGaP)	625 nm	2.0–2.4 V at 20 mA	−0.5 to −1 %/K	Indicators, signals, automotive tail lights
Green (InGaN)	525 nm	2.9–3.4 V at 20 mA	−0.2 to −0.3 %/K	Indicators, displays, traffic lights
Blue (InGaN)	450 nm	2.8–3.2 V at 20 mA	−0.1 to −0.2 %/K	Displays, phosphor pump
White (InGaN + phosphor)	broadband	2.8–3.2 V at 350 mA	−0.1 to −0.3 %/K	Lighting, backlights, headlamps
UV-C (AlGaN)	275 nm	5–7 V at 100 mA	−0.3 to −0.5 %/K	Disinfection (EQE only a few %)

Driving an LED

An LED must be driven with a controlled current, for the reasons of Chapter 1 Section 1.5: light output follows current, while V_F varies by ±10 % between parts and falls by 2 to 4 mV/K as the junction heats. A series resistor from a supply well above V_F is adequate for indicators; lighting and backlights use constant-current drivers (linear or switching, Chapters 10 and 30). Brightness is controlled by **PWM dimming** of the rated current at hundreds of hertz to tens of kilohertz, which keeps colour constant (IEEE 1789 recommends flicker limits), or by **analog dimming**, which shifts the colour of phosphor-converted LEDs slightly. Parallel LEDs without ballast resistors share current badly (the hottest has the lowest V_F, the runaway of Chapter 1 Section 1.7), so LEDs are connected in series strings. They tolerate only about 5 V of reverse voltage, and InGaN dies are ESD-sensitive, so LED modules include protection diodes (Section 2.10).

Worked example — thermal design of a 1 W white LED. A white power LED runs at 350 mA with $V_F = 3.0$ V, so the electrical input is 1.05 W. With a wall-plug efficiency of 40 % it emits 0.42 W of light and dissipates 0.63 W as heat. The package has a junction-to-solder-point thermal resistance of 8 K/W, and the metal-core board and heatsink add 25 K/W to an ambient of 45 °C inside a luminaire. The junction runs at $45 + 0.63 \times (8 + 25) = 65.8$ °C. Light output falls by roughly 0.2 %/K for InGaN, so this LED gives about 8 % less light than at 25 °C, and its forward voltage is about 0.1 V lower. Lumen-maintenance life (L70, the hours until output falls to 70 %, extrapolated from LM-80 data by the TM-21 method) roughly halves for every 10–20 K of junction temperature, so a heatsink 10 K/W worse (+6.3 K) would cost 20–35 % of rated life while barely changing initial brightness.

> **Status of the principle: LED forward voltage near the photon voltage**
> *Approximate.* Energy conservation requires qV_F to exceed roughly $h\nu$ for efficient emission at useful current; actual V_F exceeds $h\nu/q$ by heterobarrier and resistive drops (0.1–0.5 V), and in deep-UV AlGaN devices by several volts, so the photon voltage is a lower bound, not a prediction.

2.8 Solar cells

From the ideal cell to the single-diode model

Engineering Physics Ch. 14 §14.5.5 gives the ideal solar cell, a photocurrent in parallel with a Shockley diode, its short-circuit current and the logarithmic open-circuit voltage $V_{OC} \approx V_T \ln(I_{SC}/I_0)$, and the Shockley–Queisser efficiency limit

of about 33 % for a single junction. A real cell needs three more elements, giving the **single-diode model** used in every PV datasheet fit and system simulator:

$$I = I_{ph} - I_0 \left[\exp\!\left(\frac{V + I R_s}{n V_T} \right) - 1 \right] - \frac{V + I R_s}{R_{sh}}$$

with an ideality factor n (1.0–1.5), a **series resistance** R_s (fingers, busbars, contacts, emitter sheet resistance) and a **shunt resistance** R_{sh} (edge leakage, crystal defects). The equation is implicit in I, so it is solved numerically. The figure of merit for how "square" the characteristic is, the **fill factor** $FF = P_{max}/(V_{OC} I_{SC})$, is the parameter most sensitive to R_s and R_{sh}.

Ratings are given at **standard test conditions** (STC: 1000 W/m², AM1.5G spectrum per IEC 60904-3, cell temperature 25 °C). The listing below fits a single-diode model to a modern 156.75 mm (M2, 244.3 cm²) monocrystalline PERC cell and explores series resistance, shunt resistance and temperature.

```
 import numpy as np
from scipy.optimize import brentq, minimize_scalar

K_EV = 8.617333e-5                      # Boltzmann constant in eV/K

def cell_current(V, T=298.15, Iph=9.80, I0=1.0e-10, n=1.05, Rs=0.003, Rsh=20.0):
    """Single-diode model, I0 scaled from its 25 C value with T^3*exp(-Eg/kT)."""
    VT = K_EV * T
    I0T = I0 * (T / 298.15)**(3 / n) * np.exp(1.12 / (n * K_EV) * (1 / 298.15 - 1 / T))
    IphT = Iph * (1 + 0.0005 * (T - 298.15))          # Isc tempco +0.05 %/K
    f = lambda I: IphT - I0T * np.expm1((V + I * Rs) / (n * VT)) - (V + I * Rs) / Rsh - I
    return brentq(f, -10 * Iph, 2 * Iph)               # implicit in I: solve numerically

def summary(**p):
    Isc = cell_current(0.0, **p)
    Voc = brentq(lambda V: cell_current(V, **p), 0.0, 0.8)
    res = minimize_scalar(lambda V: -V * cell_current(V, **p), bounds=(0, Voc), method="bounded")
    Vm = res.x; Im = cell_current(Vm, **p)
    return Isc, Voc, Vm, Im, Vm * Im, Vm * Im / (Isc * Voc)

for label, p in [("STC 25 C", {}), ("Rs = 10 mOhm", {"Rs": 0.010}),
                 ("Rsh = 2 Ohm", {"Rsh": 2.0}), ("cell at 60 C", {"T": 333.15})]:
    Isc, Voc, Vm, Im, Pm, FF = summary(**p)
    print(f"{label:12s} Isc={Isc:5.2f} A  Voc={Voc:.3f} V  Vmp={Vm:.3f} V  Imp={Im:4.2f} A  "
          f"Pmax={Pm:4.2f} W  FF={FF:.3f}  eta={Pm / (0.02443 * 1000) * 100:4.1f} %")
```

`cell_current` solves the implicit model with a bracketing root finder, scaling I_0 with temperature as in Chapter 1 Section 1.7; `summary` finds I_{SC}, V_{OC} and the maximum-power point:

```
 STC 25 C     Isc= 9.80 A  Voc=0.683 V  Vmp=0.572 V  Imp=9.31 A  Pmax=5.33 W  FF=0.797  eta=21.8 %
Rs = 10 mOhm Isc= 9.80 A  Voc=0.683 V  Vmp=0.515 V  Imp=9.19 A  Pmax=4.73 W  FF=0.707  eta=19.4 %
Rsh = 2 Ohm  Isc= 9.79 A  Voc=0.682 V  Vmp=0.572 V  Imp=9.06 A  Pmax=5.18 W  FF=0.777  eta=21.2 %
cell at 60 C Isc= 9.97 A  Voc=0.622 V  Vmp=0.509 V  Imp=9.36 A  Pmax=4.76 W  FF=0.768  eta=19.5 %
```

Worked example — what the numbers say about a cell. At STC the model cell delivers 5.33 W at 0.572 V and 9.31 A, 21.8 % efficient with a fill factor of 0.797. Raising R_s from 3 to 10 mΩ, the effect of a cracked finger or a poor solder joint to the ribbon, costs 11 % of the power and drops the fill factor to 0.707 without changing V_{OC} or I_{SC}: series resistance hides from the two numbers most often measured. A shunt of 2 Ω costs 2.8 % at full sun, and much more at low irradiance, where the shunt current is a larger fraction of the photocurrent. Heating the cell to 60 °C, a normal operating temperature on a sunny roof, lowers V_{OC} by 61 mV (−1.74 mV/K, or −0.26 %/K) and the power by 10.7 % (−0.31 %/K), while I_{SC} rises slightly. Datasheets of crystalline-silicon modules give power coefficients of −0.29 to −0.40

%/K; module temperatures are characterised by the nominal module operating temperature (NMOT, IEC 61215), typically 41–45 °C at 800 W/m² and 20 °C ambient.

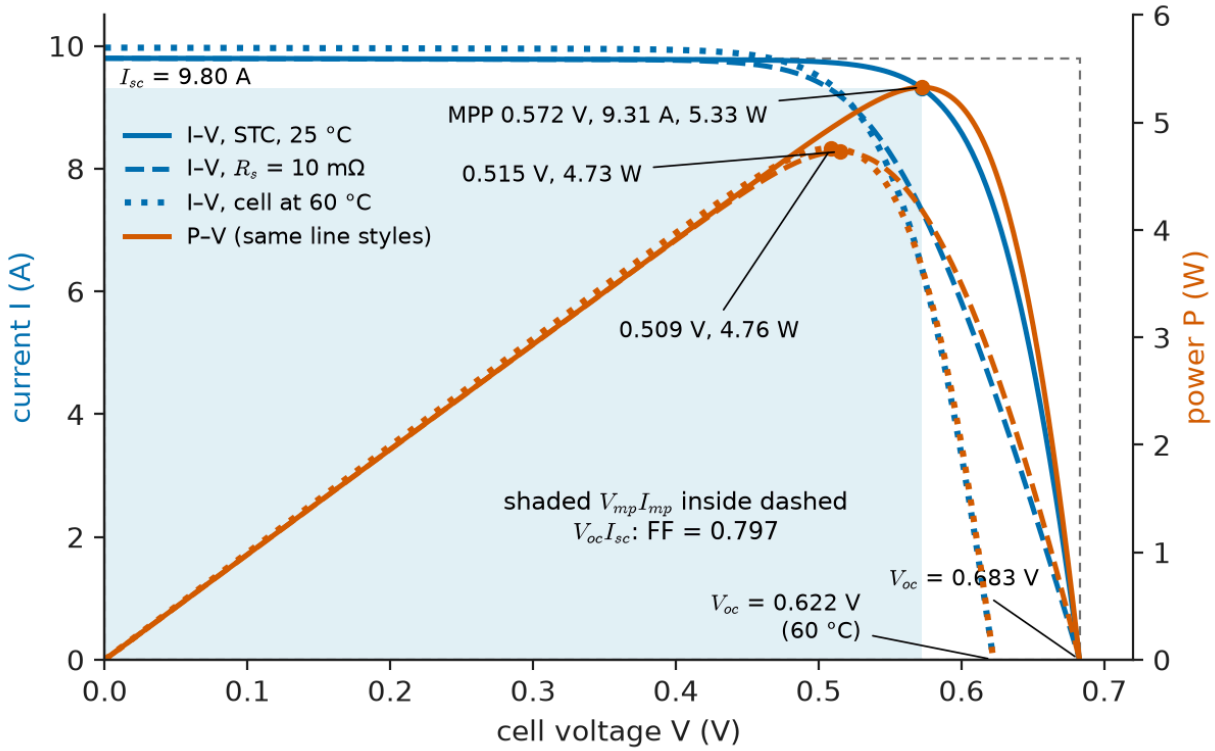


Figure 2.5 — Current–voltage and power–voltage curves of a single-diode model of a 244 cm2 PERC silicon cell, showing the effect of series resistance and temperature.

Cells in series, mismatch and bypass diodes

A 60-cell module connects its cells in series to reach about 34 V at the maximum-power point, so all cells carry the same current. A shaded cell is forced into reverse bias by the string current and dissipates the power of the others: a **hot spot** that can exceed 100 °C and crack glass or burn the backsheet. Modules therefore carry **bypass diodes**, usually three Schottky diodes in the junction box each across 20 cells. The shaded cell's reverse voltage is then limited to the other 19 cells' voltages plus one diode drop, about 11–12 V, and the module loses a third of its voltage rather than its whole output. The bypass diodes carry 10–14 A for hours at 70–85 °C, hence Schottky or MOSFET-based devices. Partial shading also gives the power–voltage curve several local maxima, which complicates maximum-power-point tracking (Chapter 32). The plant-level design of PV systems, inverters and their grid connection belongs to *Electrical Energy and Machines*, Ch. 15.

> **Status of the principle: Single-diode solar-cell model**
> *Approximate.* A lumped equivalent circuit fitted to measured I–V curves; it reproduces commercial silicon cells to within about 1 % near the maximum-power point, but distributed series resistance, a second recombination diode, reverse-bias breakdown of shaded cells and spectral effects require extended models.

2.9 Varactor diodes

The junction as a voltage-controlled capacitor

A reverse-biased junction is a capacitor whose value depends on the voltage, $C_j = C_{j0}(1 + V_R/\phi)^{-m}$ in the reverse-bias form of the physics book's law. A **varactor** (varicap) is a diode optimised for that capacitance: a controlled doping

profile to set the exponent m, a low series resistance R_s for high quality factor $Q = 1/(\omega R_s C_j)$, and low leakage. An abrupt junction has $m = 0.5$ (Section 2.1); **hyperabrupt** junctions, with doping that falls away from the junction, have m from 0.75 to about 2 over part of their range, and give a larger capacitance ratio for a given voltage swing, the key specification for wide tuning from a low supply. Representative devices have capacitance ratios of 2–15 over 1–25 V, with C from below 1 pF (microwave) to several hundred picofarads (AM broadcast tuners) and Q of 50–500 at their rated frequency.

Varactors tune the LC tanks of voltage-controlled oscillators in PLL frequency synthesisers (Chapter 9), in TV and radio tuners and in RF phase shifters and tunable filters. Because the capacitance depends on the *total* junction voltage, a large RF swing modulates its own capacitance, distorting the waveform and adding phase noise; tanks therefore use two varactors back to back, so that the modulation of one cancels the other's.

Worked example — a varactor-tuned FM-band oscillator. An oscillator tank with $L = 100$ nH must tune 88–108 MHz. The required total tank capacitance is $C = 1/(4\pi^2 f^2 L)$: 32.7 pF at 88 MHz and 21.7 pF at 108 MHz. With 5 pF of fixed parasitic and transistor capacitance, the varactor must swing from 27.7 pF to 16.7 pF, a ratio of 1.66. An abrupt varactor ($m = 0.5$, $\phi = 0.75$ V) starting at 1 V reverse bias needs $(V_R + 0.75)/(1 + 0.75) = 1.66^2$, so the top of the band is at $V_R = 4.06$ V, within a 5 V supply, and the varactor's zero-bias capacitance must be $C_{j0} = 27.7\sqrt{1.75/0.75} = 42.3$ pF. The average tuning sensitivity is $(108 - 88)/(4.06 - 1) = 6.5$ MHz/V, but it is not constant: 10.7 MHz/V at the bottom of the band and 4.3 MHz/V at the top, a variation of loop gain that the PLL designer must accommodate. With $R_s = 0.5\ \Omega$, the varactor's Q at 88 MHz is $1/(2\pi \cdot 88\ \text{MHz} \cdot 0.5\ \Omega \cdot 27.7\ \text{pF}) = 131$, adequate for a broadcast VCO.

2.10 Protection diodes

The threats

Electronic equipment must survive transients far larger than its working voltages. The main ones, each defined by a standard test waveform, are:

- **electrostatic discharge** (IEC 61000-4-2): up to 8 kV contact and 15 kV air discharge, a 30 A peak (at 8 kV contact) with a 0.7–1 ns rise and about 60 ns duration;
- **surges** (IEC 61000-4-5): 1.2/50 µs voltage and 8/20 µs current waveforms of 0.5–4 kV from lightning and power-system switching;
- **automotive transients** (ISO 7637-2, ISO 16750-2): inductive switching, load dump and reverse battery;
- **inductive kickback** from relays, solenoids and windings switched off by a transistor.

The diode answers are the **transient-voltage suppressor** (TVS, a large-area avalanche diode), **steering-diode arrays**, **flyback diodes** and **reverse-polarity** diodes or MOSFETs.

TVS diodes and their parameters

A TVS is specified by four voltages and a pulse rating:

- the **reverse stand-off (working) voltage** V_{RWM}, the largest voltage at which it conducts only leakage (choose it above the highest normal line voltage);
- the **breakdown voltage** V_{BR} at a test current of 1 mA, about 10–20 % above V_{RWM};
- the **clamping voltage** V_C at the peak pulse current I_{PP}, the voltage the protected circuit actually sees;
- the **peak pulse power** P_{PP}, usually for a 10/1000 µs waveform (400 W for SMA, 600 W for SMB, 1500 W for SMC packages), with a derating curve versus pulse width: shorter pulses are allowed more power because less heat reaches the die's surroundings.

In breakdown, a TVS behaves like a Zener: $V \approx V_{BR} + r_d I$, with dynamic resistance $r_d = (V_C - V_{BR})/I_{PP}$ of a fraction of an ohm for power types. Because V_C is typically 1.4–1.6 times V_{RWM}, the protected circuit must withstand that much

more than its working voltage; the opening example's regulator was rated below the TVS's clamping voltage. Bidirectional TVS diodes protect AC or bipolar lines.

Worked example — clamping an ISO 7637-2 pulse on a 12 V automotive input. Test pulse 2a (interruption of current in a series inductance) reaches $U_s = +112$ V at the highest severity, from a 2 Ω source, with a 50 µs duration. An SMBJ24A-class TVS has $V_{RWM} = 24$ V (above a 24 V jump-start), $V_{BR} \approx 28$ V typical and $V_C = 38.9$ V at 15.4 A, so $r_d \approx (38.9 - 28.0)/15.4 = 0.71$ Ω. The clamped current is $I = (112 - 28)/(2 + 0.71) = 31$ A and the clamping voltage $V_C = 28 + 0.71 \times 31 = 50$ V, a peak power of 1.55 kW. The SMB package's 600 W rating is for 10/1000 µs; its derating curve allows roughly 2.5–3 kW for a 50 µs pulse, so the device survives, but every component behind it, the regulator input above all, must be rated for 50 V, not 24 V. If the downstream parts are rated only 40 V, a larger SMC device (lower r_d) or a series resistance or inductance ahead of the TVS is needed.

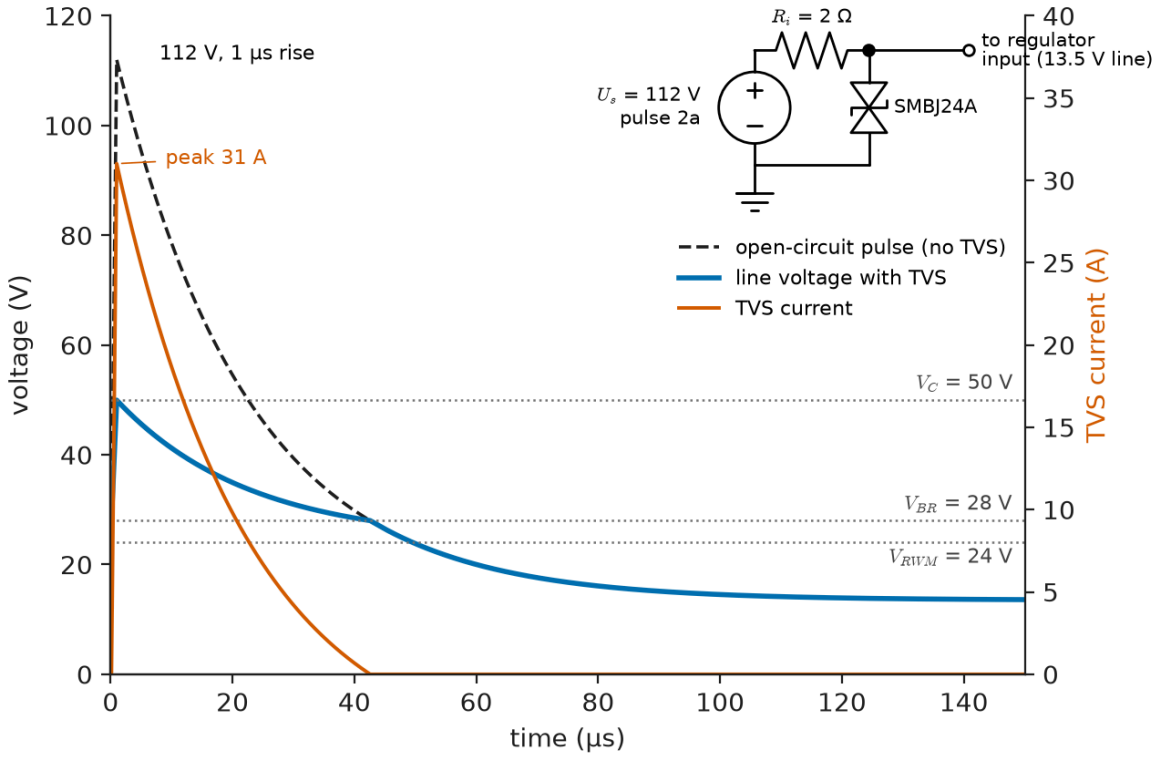


Figure 2.6 — Clamping of an automotive ISO 7637-2 pulse 2a by a 24 V TVS diode.

Data lines and ESD arrays

ESD arrays for USB, HDMI, Ethernet and CAN combine low-capacitance steering diodes, which route the discharge to the supply rail or ground, with one larger TVS from rail to ground. Line capacitance must stay below about 1 pF for USB 2.0 high speed (480 Mb/s) and 0.3–0.5 pF for USB 3.x and HDMI 2.x. The array must sit at the connector, with the shortest possible path to ground, because the 30 A, sub-nanosecond ESD edge drops about 30 V across each nanohenry of trace inductance ($L\,di/dt$ with $di/dt \approx 30$ A/ns).

Flyback diodes on inductive loads

When a transistor switches off a relay coil, solenoid or small motor, the inductor maintains its current, and with no path the collector voltage rises until something breaks down. A diode across the coil (cathode to the supply) gives the current a path, clamping the transistor voltage to the supply plus one diode drop. The penalty is slow release: the current then decays with the coil's own time constant. For a 12 V relay with a 400 Ω, 0.5 H coil (30 mA), the decay through a plain diode takes about $\tau \ln(1 + RI/V_D) = 1.25$ ms $\times \ln(1 + 12/0.8) = 3.5$ ms, which delays contact opening and increases arcing. Adding an 18 V Zener in series with the diode lets the collector rise to $12 + 18.8 = 30.8$ V, within a 40 V

transistor's rating, and shortens the decay to $1.25\ \text{ms} \times \ln(1 + 12/18.8) = 0.62\ \text{ms}$. Section 3.10 uses this in a transistor relay driver.

Reverse-polarity protection

A series diode in the supply input blocks reverse polarity but costs its forward drop continuously: 0.7–1 V and the corresponding loss, as in the overheating diode of the opening example (1 A at 0.95 V is nearly 1 W). A Schottky halves the loss. A P-channel MOSFET (or an N-channel MOSFET with an "ideal diode" controller), oriented so that its body diode conducts in the correct polarity and its gate is enhanced only by correct polarity, reduces the drop to $IR_{DS(on)}$, a few millivolts at 1 A; it is the standard automotive and industrial solution.

> **Status of the principle: TVS clamping model, $V_C = V_{BR} + r_d I$**
> *Approximate.* A linear fit to the avalanche characteristic at the datasheet test current; the clamping voltage also rises with junction heating during the pulse (the positive temperature coefficient of avalanche breakdown, about +0.1 %/K) and with the inductance of the connection, so measured clamping on a board is usually higher than the datasheet value.

What remains open

Open question	Where it is answered
How do two junctions sharing a thin base make a transistor, and how is it biased, modelled and switched?	Chapter 3
How do field-effect devices, body diodes and power MOSFETs compare with junction devices as switches and synchronous rectifiers?	Chapter 4
How do thyristors, IGBTs, SiC and GaN devices, laser diodes, optocouplers and other optoelectronic devices extend the junction?	Chapter 5
How are rectifiers, filters, clippers, clampers and multipliers designed around these diodes?	Chapter 6
How are photodiode transimpedance amplifiers designed for bandwidth and noise?	Chapter 8
How do varactor-tuned oscillators and bandgap references work as circuits?	Chapter 9, Chapter 10
How are diode reverse recovery and SiC Schottky diodes handled in converter loss budgets?	Chapter 29, Chapter 30
How is maximum-power-point tracking done, and how are PV plants designed and connected to the grid?	Chapter 32; *Electrical Energy and Machines*, Ch. 15
Which materials set LED colour, photodetector cut-off and solar absorber efficiency?	*Materials Science and Engineering*, Ch. 35

Next: Chapter 3 places two junctions back to back across a thin base and obtains the bipolar junction transistor, the first device in this book that amplifies.

Chapter 3 — Bipolar Junction Transistors

Part I · Electronic Devices

An industrial controller board uses a single 2N3904 per channel to drive a 12 V relay from a 3.3 V microcontroller pin, and a second 2N3904 as a small preamplifier for a 4–20 mA loop monitor. Production starts with transistors from one supplier and every board passes. Six months later a second-source lot arrives. The relay drivers still work, but they release the relays half a microsecond later and a few of them fail when an engineer removes the flyback diode "to save cost". The preamplifiers are worse: their output sits a volt away from where it used to, and on hot days some of them clip. The new transistors are within their datasheet in every respect; their current gain is simply near the top of the guaranteed 100–300 range, where the old ones were near the bottom. A circuit whose behaviour depends on β is a circuit that will change with every lot and every degree. The bipolar junction transistor is an excellent amplifier and switch exactly because its collector current is an exponential function of one voltage and nearly independent of another, but its current gain, its base–emitter voltage and its stored charge are among the least controlled numbers in electronics. This chapter shows how the BJT works as a circuit element, how its configurations differ, how it is biased so that the circuit does not depend on β or temperature, how it is modelled for small signals, and how it behaves as a switch and at high frequency.

> **How do the BJT's structure and Ebers–Moll model explain its operating regions, current gain and saturation voltage?**
> **How are the common-emitter, common-base and common-collector configurations different, and how is a transistor biased so that its operating point survives β spread and temperature? What limits a BJT's gain, switching speed and frequency response, and how are those limits calculated from datasheet parameters?**

Concept	Engineering destination
BJT structure	Reading datasheets; choosing small-signal, power, Darlington and SiGe devices
Operating regions	Amplifier versus switch design; checking hand analyses
Common-base configuration	Cascodes, RF and current-buffer stages, transimpedance inputs
Common-emitter configuration	Voltage-gain stages in discrete and integrated amplifiers
Common-collector configuration	Buffers, output stages, regulator pass elements
DC biasing	Operating points that survive β spread, temperature and supply variation
Load lines	Signal swing, clipping limits, choice of quiescent current
Small-signal models	Gain, impedance and frequency-response calculation; SPICE interpretation
Amplification	Intrinsic gain, degeneration, linearity limits
Switching behavior	Relay, LED and solenoid drivers; level shifters; storage-time limits
Frequency response	f_T, Miller effect, bandwidth of transistor stages

Roadmap. Structure and the Ebers–Moll model (3.1) → the four regions they imply (3.2) → the three configurations seen at device level (3.3–3.5) → setting the operating point (3.6) and seeing the signal swing on load lines (3.7) → the hybrid-π model (3.8) and what it says about gain (3.9) → the transistor as a switch (3.10) → charge, capacitance and f_T (3.11).

3.1 BJT structure

Where *Engineering Physics* leaves off

Engineering Physics Ch. 18 §18.3.2 introduces the BJT as two back-to-back junctions, states the active-region law $I_C = \beta I_B = I_S e^{V_{BE}/V_T}$, names the Ebers–Moll model and lists the hybrid-π parameters $g_m = I_C/V_T$, $r_\pi = \beta/g_m$ and $r_o = V_A/I_C$, with a common-emitter gain of $-g_m R_C$; the underlying minority-carrier diffusion and diffusion length are in Ch. 14 §14.5.3. Chapter 1 of this book provided the modelling vocabulary (regions, biasing, linearisation, temperature) and Chapter 2 the junction itself (depletion, breakdown, stored charge). This chapter builds the working engineering model: what sets β in a real structure, the Ebers–Moll equations used to find saturation voltages, the bias networks that make circuits independent of β, the complete small-signal model with its capacitances, and the switching and frequency limits that the one-section sketch leaves out. It also goes beyond the physics sketch's validity at high current (where β falls) and at high voltage (breakdown).

The real structure

A modern small-signal NPN transistor is a vertical, planar device (Figure 3.1). A heavily doped n⁺ emitter (10^{19}–10^{20} cm^{-3}) is diffused or implanted into a thin p-type base (10^{17}–10^{18} cm^{-3}, 0.1–1 µm wide), which sits in a lightly doped n-type collector epitaxial layer (10^{15}–10^{16} cm^{-3}) on an n⁺ substrate that carries the collector current to the back contact. The structure is deliberately asymmetric. The emitter is far more heavily doped than the base, so that the base–emitter junction injects mostly electrons into the base rather than holes into the emitter. The base is thinner than the electrons' diffusion length, so that almost all injected electrons reach the collector junction before recombining. The collector is lightly doped, so that the collector–base junction can block a large voltage, with its depletion region spreading mostly into the collector (Chapter 2, Section 2.1).

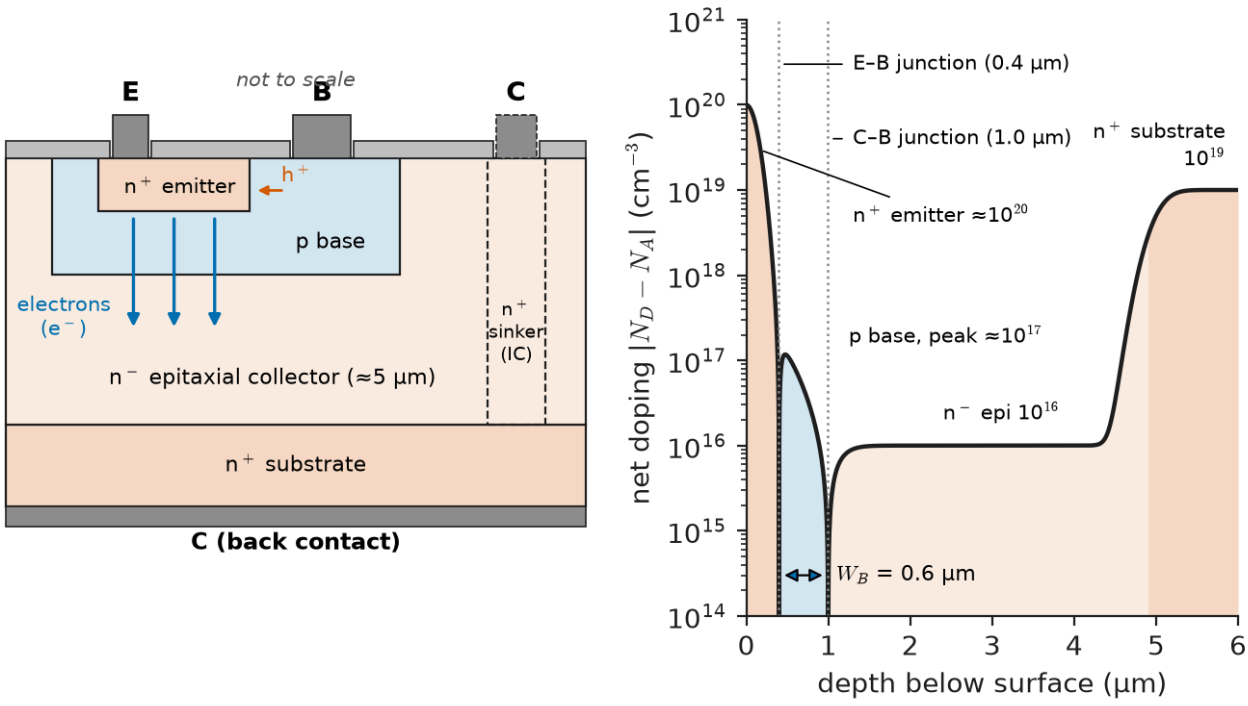


Figure 3.1 — Cross-section and doping profile of a vertical planar NPN transistor.

Three numbers describe how well the structure works. The **emitter injection efficiency** γ is the fraction of emitter current carried by electrons injected into the base; it approaches 1 when the emitter's Gummel number (doping times width) greatly exceeds the base's. The **base transport factor** α_T is the fraction of injected electrons that cross the base without recombining, approximately $1 - W_B^2/2L_n^2$ for base width W_B and diffusion length L_n. Their product is the common-base current gain $\alpha_F = I_C/I_E$, and the common-emitter current gain is

$$\beta_F = \frac{\alpha_F}{1-\alpha_F}$$

so that $\alpha_F = 0.995$ gives $\beta_F = 199$ and $\alpha_F = 0.998$ gives $\beta_F = 499$. A small change in the fraction of carriers lost, which depends on base doping and width, lifetime and surface condition, changes β by a large factor. That is why β varies so much between devices and lots: it is the reciprocal of a small, poorly controlled number.

> **Status of the principle:** $\beta_F = \alpha_F/(1-\alpha_F)$
> *Derived.* It follows from KCL, $I_E = I_C + I_B$, and the definitions of the two gains; it is exact at a given operating point, and the sensitivity it reveals ($d\beta/\beta = (1+\beta)\,d\alpha/\alpha$) explains the wide spread of β.

The Ebers–Moll model

The Ebers–Moll model treats the transistor as two coupled diodes and is valid in all four regions. In its transport form, with V_{BE} and V_{BC} as the junction voltages,

$$I_C = I_S\left(e^{V_{BE}/V_T} - e^{V_{BC}/V_T}\right) - \frac{I_S}{\beta_R}\left(e^{V_{BC}/V_T} - 1\right), \qquad I_B = \frac{I_S}{\beta_F}\left(e^{V_{BE}/V_T} - 1\right) + \frac{I_S}{\beta_R}\left(e^{V_{BC}/V_T} - 1\right)$$

The first term of I_C is the transport current that crosses the base, driven by the difference of the two junctions' exponentials; the other terms are the base currents of the two junctions, scaled by the forward and reverse current gains β_F and β_R. Because the structure is asymmetric, β_R is small, typically 0.1–5. With V_{BC} strongly negative the equations reduce to the physics book's active-region law $I_C = I_S e^{V_{BE}/V_T}$, $I_C = \beta_F I_B$. The SPICE Gummel–Poon model adds to Ebers–Moll the effects that matter in practice: the Early effect, the fall of β at low current (base–emitter recombination current, Chapter 2 Section 2.2) and at high current (high injection), and series resistances and capacitances. For the 2N3904 the commonly used model has $I_S = 6.73$ fA, $\beta_F = 416$, $\beta_R = 0.74$, $V_A = 74$ V, $\tau_F = 301$ ps.

> **Status of the principle: Ebers–Moll model**
> *Approximate.* It assumes low-level injection, one-dimensional current flow, constant base width and negligible series resistance; it predicts the regions and saturation voltages of a BJT well at moderate currents and must be extended (Gummel–Poon, VBIC, HICUM) for the Early effect, high injection, β roll-off and high-frequency behaviour.

Device families

Table 3.1 — Representative bipolar transistors (datasheet values; typical where marked).

Device (class)	V_{CEO}	$I_{C(max)}$	h_{FE} (at I_C)	f_T	P_D	Typical use
2N3904 (NPN, TO-92)	40 V	200 mA	100–300 (10 mA)	≥ 300 MHz	625 mW	General purpose
BC547B (NPN)	45 V	100 mA	200–450 (2 mA)	300 MHz (typ)	500 mW	Audio, low noise
2N2222A (NPN)	40 V	600 mA	100–300 (150 mA)	≥ 300 MHz	625 mW (TO-92)	Switching
BC807 (PNP, SOT-23)	45 V	500 mA	100–600 (100 mA)	100 MHz (typ)	310 mW	High-side switches
TIP31C (NPN, TO-220)	100 V	3 A	≥ 25 (1 A)	3 MHz	40 W (T_C = 25 °C)	Medium power
2SC5200 class (NPN, TO-264)	230 V	15 A	55–160 (1 A)	30 MHz	150 W	Audio output stages
TIP120 (Darlington)	60 V	5 A	≥ 1000 (3 A)	—	65 W	Solenoid, motor drivers
SiGe HBT (RF)	4 V	50 mA	150–300	40–50 GHz	0.2 W	LNAs, GNSS, Wi-Fi front ends

PNP transistors are the complement, with all voltages and currents reversed; for equal size they have lower f_T and somewhat lower β because holes are less mobile than electrons. A **Darlington** pair multiplies current gains (≥ 1000) at the cost of a saturation voltage of 1–2 V (one V_{BE} plus one $V_{CE(sat)}$) and slow turn-off. **Heterojunction bipolar transistors** (HBTs), with a SiGe base or in InP and GaAs, use a band-gap difference to block reverse injection, so the base can be heavily doped and very thin: SiGe BiCMOS processes reach f_T of 300 GHz and f_{max} of 500 GHz, the basis of automotive 77 GHz radar and optical-link transceivers, and GaAs HBTs dominate mobile-phone power amplifiers. In power electronics the bipolar transistor has been displaced by the MOSFET and IGBT (Chapters 4 and 5), whose gates need no steady drive current.

3.2 Operating regions

Four combinations of two junctions

Each junction may be forward or reverse biased, giving four regions (Table 3.2). For an NPN in a circuit, the region is decided by V_{BE} and V_{BC}, or equivalently V_{BE} and $V_{CE} = V_{BE} - V_{BC}$.

Table 3.2 — BJT operating regions (NPN, silicon, typical values).

Region	B–E junction	B–C junction	Behaviour	Use
Forward active	Forward (V_{BE} ≈ 0.6–0.75 V)	Reverse ($V_{CE} \gtrsim$ 0.3 V)	$I_C = \beta_F I_B$; current source controlled by V_{BE}	Amplifiers, current sources
Saturation	Forward	Forward	$V_{CE} = V_{CE(sat)}$ ≈ 0.05–0.3 V; $I_C < \beta_F I_B$	Closed switch
Cutoff	Reverse or < 0.4 V	Reverse	All currents ≈ leakage (nA)	Open switch
Reverse active	Reverse	Forward	β_R ≈ 0.1–5	Rare: some analogue switches, TTL inputs

The output characteristics of Figure 3.2 display the regions directly. In the forward-active region the curves are nearly horizontal, I_C set by I_B (or V_{BE}) and almost independent of V_{CE}. Their small upward slope is the **Early effect**:

increasing V_{CB} widens the collector depletion region, narrows the effective base and raises I_C. Extrapolated backwards, the curves meet near $V_{CE} = -V_A$, the **Early voltage**, 50–150 V for small-signal devices, and the model becomes

$$I_C = I_S e^{V_{BE}/V_T}\left(1 + \frac{V_{CE}}{V_A}\right)$$

Near the origin the curves collapse together into the saturation region; at high V_{CE} they turn upward into breakdown.

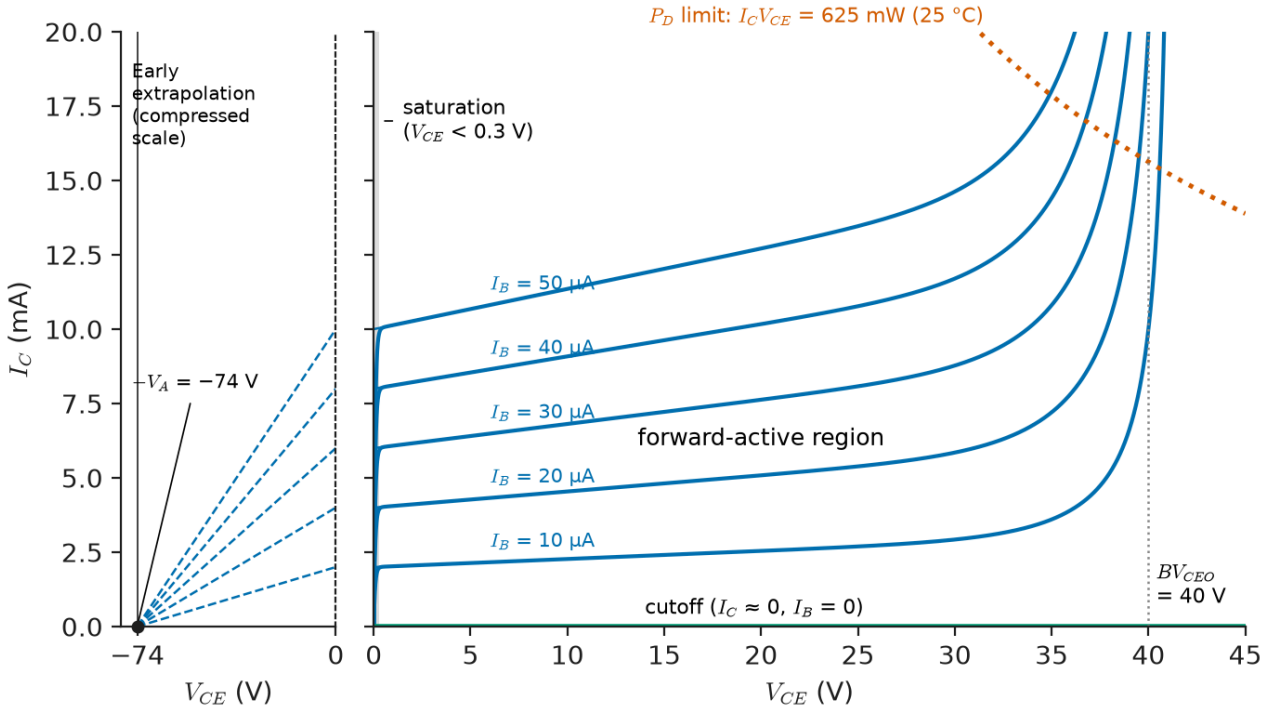


Figure 3.2 — Output characteristics of a small-signal NPN transistor with operating regions and the Early-voltage construction.

Saturation voltage from Ebers–Moll

A transistor used as a switch is driven with more base current than needed, so that $I_C < \beta_F I_B$. The ratio $\beta_{forced} = I_C/I_B$ is the **forced beta**. Solving the Ebers–Moll equations for V_{CE} with both junctions forward biased gives

$$V_{CE(sat)} = V_T \ln \frac{1/\alpha_R + \beta_{forced}/\beta_R}{1 - \beta_{forced}/\beta_F}$$

with $\alpha_R = \beta_R/(1+\beta_R)$. For the 2N3904 model ($\beta_F = 416$, $\beta_R = 0.74$) at a forced beta of 10, the intrinsic saturation voltage is $V_T \ln(16.3) = 72$ mV; at forced betas of 20 and 50 it rises to 89 and 113 mV. As β_{forced} approaches β_F the denominator vanishes and the transistor leaves saturation. Measured values are higher because of the collector series resistance: the datasheet guarantees $V_{CE(sat)} \le 0.2$ V at 10 mA with 1 mA of base current and ≤ 0.3 V at 50 mA with 5 mA. The rule of thumb $\beta_{forced} \approx 10$ comes from this curve: it gives a low, flat $V_{CE(sat)}$ with margin against the lowest β_F of a production lot at the lowest temperature.

> **Status of the principle: Saturation voltage from Ebers–Moll**
> *Derived.* It follows algebraically from the Ebers–Moll equations with both junctions forward biased; in practice it gives the intrinsic part only, and the collector and emitter series resistances add $I_C(R_C + R_E)$, which dominates in power transistors.

Breakdown

The collector–base junction breaks down by avalanche at BV_{CBO} (emitter open). With the base open, the breakdown voltage BV_{CEO} is lower, because holes generated by impact ionisation in the collector junction flow into the base, act as

base current and are amplified by β, so runaway needs an avalanche multiplication factor M exceeding 1 by only about $1/\beta$ ($M - 1 \approx 1/\beta$). For the 2N3904, $V_{CBO} = 60$ V but $V_{CEO} = 40$ V. A low base–emitter resistance (the BV_{CER} rating) or a reverse-biased base (BV_{CEX}) restores most of the difference, which is one reason switching transistors should never be left with a floating base. The base–emitter junction itself breaks down at only 5–7 V reverse (V_{EBO}), and repeated reverse breakdown degrades β permanently; inputs that can swing negative need a clamp diode.

3.3 Common-base configuration

Three ways to connect three terminals

A transistor used as an amplifier has one terminal common to input and output. The three choices give three configurations with very different impedances and gains. This chapter characterises each at device level, with the small-signal parameters of Section 3.8; Chapter 7 designs complete single- and multistage amplifiers around them. Throughout, the numbers refer to a transistor at $I_C = 1$ mA with $\beta = 200$, $V_A = 74$ V, so $g_m = 38.7$ mS, $r_\pi = 5.17$ kΩ and $r_o = 74$ kΩ, with a 4.7 kΩ collector load and no external load.

Common base

In the **common-base** (CB) configuration the signal enters at the emitter and leaves at the collector, and the base is held at signal ground. The input sees the emitter's small-signal resistance

$$r_e = \frac{\alpha}{g_m} \approx \frac{V_T}{I_E}$$

25.7 Ω at 1 mA, because a small change in V_{BE} produces a large change in emitter current. The current gain is α, just below 1: the stage passes current from emitter to collector almost unchanged, but from a low impedance to a high one. The voltage gain is $+g_m R_C = +182$ ignoring r_o, the same magnitude as the common-emitter stage but non-inverting. The output resistance looking into the collector is very high, of the order of βr_o when the emitter is driven from a high impedance, because the base is fixed and there is no internal feedback through the base.

Three properties make the CB stage valuable despite its low input impedance. It is a **current buffer**, taking a signal current from a low-impedance node and delivering it into a high-impedance load; this is the transimpedance input of photodiode receivers and the core of the **cascode**, where a CB transistor on top of a CE transistor shields it from the output voltage (Chapter 7, Chapter 10). The collector–base capacitance is not connected between input and output, so there is no Miller multiplication (Section 3.11) and the CB stage has the widest bandwidth of the three; it is a classic RF and video amplifier stage. And its low input impedance matches low-impedance sources such as 50 Ω transmission lines.

3.4 Common-emitter configuration

The voltage-gain stage

In the **common-emitter** (CE) configuration the signal drives the base, the output is taken from the collector and the emitter is at signal ground (directly or through a bypass capacitor). It is the only configuration with both voltage and current gain substantially greater than one, and therefore the highest power gain. Its small-signal properties are:

- input resistance $r_\pi = \beta / g_m = 5.17$ kΩ (plus the base spreading resistance r_x, 10–200 Ω);
- voltage gain $A_v = -g_m (R_C \parallel r_o) = -0.0387 \times 4.42\ \text{k}\Omega = -171$;
- current gain β;
- output resistance $R_C \parallel r_o = 4.42$ kΩ.

The gain $-g_m R_C = -I_C R_C / V_T$ is the ratio of the DC voltage across the collector load to the thermal voltage, a result *Engineering Physics* §18.3.2 states and that is worth remembering as a design rule: a CE stage with 4.7 V across its

collector resistor has a gain near -180 regardless of the transistor type or the current, as long as $r_o \gg R_C$.

Emitter degeneration

An unbypassed emitter resistor R_E (emitter degeneration) introduces local negative feedback (Chapter 9). The gain becomes

$$A_v = -\frac{g_m R_C}{1 + g_m R_E} \approx -\frac{R_C}{R_E} \quad (g_m R_E \gg 1)$$

and the input resistance rises to $r_\pi + (\beta + 1)R_E$. Gain is traded for three benefits: the gain is set by a resistor ratio instead of the transistor's g_m, so it no longer depends on I_C, temperature or the device; the input resistance is raised; and the linear input range grows by the factor $1 + g_m R_E$, because the base–emitter junction sees only a fraction of the input signal (Section 3.9). The same resistor, bypassed for signals but present at DC, is the main element of stable biasing (Section 3.6).

3.5 Common-collector configuration

The emitter follower

In the **common-collector** (CC) configuration, the **emitter follower**, the signal drives the base and the output is taken from the emitter, with the collector at signal ground (the supply). The emitter follows the base at one V_{BE} below it. With an emitter resistance R_E (here 4.7 kΩ):

- voltage gain $A_v = \dfrac{g_m R_E}{1 + g_m R_E} = 0.9945$, just below unity;
- input resistance $r_\pi + (\beta + 1)R_E = 950$ kΩ, the load multiplied by $\beta + 1$;
- output resistance $\dfrac{1}{g_m} + \dfrac{R_s}{\beta + 1} = 25.9 + 5.0 = 30.8\ \Omega$ for a 1 kΩ source, the source resistance divided by $\beta + 1$.

The emitter follower is an **impedance transformer**: it presents a high impedance to its source and a low impedance to its load, with no voltage gain but a current gain of $\beta + 1$. It buffers high-impedance nodes before low-impedance loads, forms the output stages of amplifiers and op-amps (Chapter 7, Chapter 10), and is the pass element of linear regulators. Its two practical hazards are worth knowing at device level. The output can sink current only through R_E (or a current source), so a capacitive load driven by a fast negative-going signal can make the transistor cut off. And an emitter follower driving a capacitive load from an inductive source can present a negative real input impedance at high frequency and oscillate, a classic problem cured by a small series base resistor of tens to hundreds of ohms.

Table 3.3 — The three configurations at I_C = 1 mA, β = 200, R_C or R_E = 4.7 kΩ (device-level small-signal values).

Property	Common emitter	Common base	Common collector
Input terminal / output terminal	Base / collector	Emitter / collector	Base / emitter
Input resistance	r_π = 5.17 kΩ	r_e = 25.7 Ω	$r_\pi + (\beta+1)R_E$ = 950 kΩ
Output resistance	$R_C \parallel r_o$ = 4.42 kΩ	≈ R_C = 4.7 kΩ	$1/g_m + R_s/(\beta+1)$ = 31 Ω (R_s = 1 kΩ)
Voltage gain	$-g_m(R_C \parallel r_o)$ = −171	$+g_m R_C$ ≈ +180	0.9945
Current gain	β = 200	α = 0.995	$\beta + 1$ = 201
Phase	Inverting	Non-inverting	Non-inverting
Main use	Voltage gain	Current buffer, cascode, RF	Buffer, output stage

> **Status of the principle: Configuration gain and impedance formulas**
> *Approximate*. They come from the hybrid-π model with r_x and r_o neglected where stated; they hold at low and mid frequencies and for small signals, and fail when source or load impedances become comparable with the neglected elements or when the capacitances of Section 3.11 matter.

3.6 DC biasing

What must stay constant

Biasing (Chapter 1, Section 1.5) sets the quiescent collector current I_{CQ} and voltage V_{CEQ}. A BJT bias network must hold them against three variations: β **spread**, a factor of three in a production lot, plus about +0.5 %/K with temperature; V_{BE} **drift**, about −2 mV/K, which at fixed V_{BE} would raise I_C by about 7 %/K (Chapter 1, Section 1.7); and **leakage** I_{CBO}, which doubles every 10 K but is negligible in modern silicon below about 100 °C.

Fixed bias: a cautionary circuit

The simplest network feeds the base from the supply through a large resistor R_B, fixing $I_B = (V_{CC} - V_{BE})/R_B$, so that $I_C = \beta I_B$. The collector current is directly proportional to β: the sensitivity $S_\beta^{I_C} = 1$. With $R_B = 2.2$ MΩ from 12 V, I_C is 0.52 mA for $\beta = 100$ and 1.55 mA for $\beta = 300$, a factor of three, and with a 4.7 kΩ collector resistor the quiescent output voltage moves by almost 5 V. This is the preamplifier of the chapter opening.

Voltage-divider bias with emitter degeneration

The standard discrete network (Figure 3.3) sets the base voltage with a divider R_1, R_2 and puts a resistor R_E in the emitter, bypassed for signals if gain is needed. Replacing the divider by its Thévenin equivalent $V_{Th} = V_{CC}R_2/(R_1+R_2)$, $R_{Th} = R_1 \parallel R_2$, KVL around the base–emitter loop gives

$$I_C = \frac{\beta\,(V_{Th} - V_{BE})}{R_{Th} + (\beta+1)R_E} \approx \frac{V_{Th} - V_{BE}}{R_E} \quad \text{when } (\beta+1)R_E \gg R_{Th}$$

The emitter resistor turns the base voltage into an emitter current, and the current no longer depends on β. The stage is a negative-feedback loop: if I_C rises, V_E rises, V_{BE} falls and I_C is pulled back. Two design rules follow from the equation. To reduce the β dependence, make $(\beta_{min}+1)R_E$ at least ten times R_{Th}. To reduce the V_{BE} dependence, make $V_E = I_E R_E$ large compared with the expected change in V_{BE}: a 0.12 V shift (60 K at −2 mV/K) changes I_C by $0.12/V_E$, i.e. by 10 % if $V_E = 1.2$ V. A common starting point is $V_E \approx V_{CC}/10$ and $V_{CE} \approx V_{CC}/2$.

> **Status of the principle: Emitter-degenerated bias**
> *Derived*. The bias equation follows from KVL and the active-region model; its insensitivity to β is a consequence of local negative feedback, holding to the extent that $(\beta+1)R_E \gg R_{Th}$ and $V_E \gg \Delta V_{BE}$.

Worked example — a bias network that survives a second-source lot. Design for $V_{CC} = 12$ V and $I_C \approx 1$ mA with a 2N3904 (β = 100–300). Choose $R_E = 1.2$ kΩ ($V_E \approx 1.2$ V) and $R_C = 4.7$ kΩ. The base must sit at about $1.2 + 0.65 = 1.85$ V; with the divider current about ten times the largest base current (10 µA at $\beta = 100$), take $R_2 = 15$ kΩ and $R_1 = 82$ kΩ, so $V_{Th} = 1.856$ V and $R_{Th} = 12.7$ kΩ. The condition $(\beta_{min}+1)R_E = 121\ \text{k}\Omega \gg 12.7\ \text{k}\Omega$ is met by a factor of 9.5. The listing below evaluates both this network and the fixed-bias circuit over the β range and at 25 °C and 85 °C, with V_{BE} falling 2 mV/K and β rising 0.5 %/K.

```
 VCC, RC, RE = 12.0, 4.7e3, 1.2e3              # supply and resistors of the CE stage
R1, R2, RB = 82e3, 15e3, 2.2e6                 # divider bias, and fixed-bias resistor

def divider_bias(beta, vbe):
    vth, rth = VCC * R2 / (R1 + R2), R1 * R2 / (R1 + R2)
    ic = beta * (vth - vbe) / (rth + (beta + 1) * RE)
    return ic, VCC - ic * RC - ic * (beta + 1) / beta * RE

def fixed_bias(beta, vbe):
    ic = beta * (VCC - vbe) / RB
    return ic, VCC - ic * RC                   # no emitter resistor

print("   T    beta | divider: IC (mA)  VCE (V) | fixed: IC (mA)  VCE (V)")
for t in (25, 85):
    vbe = 0.65 - 0.002 * (t - 25)              # -2 mV/K
    for beta in (100, 200, 300):
        b = beta * (1 + 0.005 * (t - 25))      # beta rises about 0.5 %/K
        icd, vd = divider_bias(b, vbe)
        icf, vf = fixed_bias(b, vbe)
        print(f"{t:4d} C {b:6.0f} |        {icd*1e3:6.3f}   {vd:6.2f}   |"
              f"       {icf*1e3:6.3f}   {vf:6.2f}")
```

```
   T    beta | divider: IC (mA)  VCE (V) | fixed: IC (mA)  VCE (V)
  25 C    100 |         0.901     6.68   |        0.516     9.58
  25 C    200 |         0.950     6.39   |        1.032     7.15
  25 C    300 |         0.967     6.29   |        1.548     4.73
  85 C    130 |         1.014     6.01   |        0.678     8.81
  85 C    260 |         1.058     5.75   |        1.356     5.63
  85 C    390 |         1.073     5.67   |        2.033     2.44
```

Over the full lot and a 60 K temperature range, the divider-biased stage keeps I_C within 0.90–1.07 mA and V_{CE} within 5.7–6.7 V. The fixed-bias stage spans 0.52–2.03 mA, a factor of four, and at the hot, high-β corner its V_{CE} has collapsed to 2.4 V, leaving little room for signal: this is the preamplifier that clipped on hot days. Integrated circuits go further and bias transistors with current mirrors whose collector current is copied from a reference (Chapter 10).

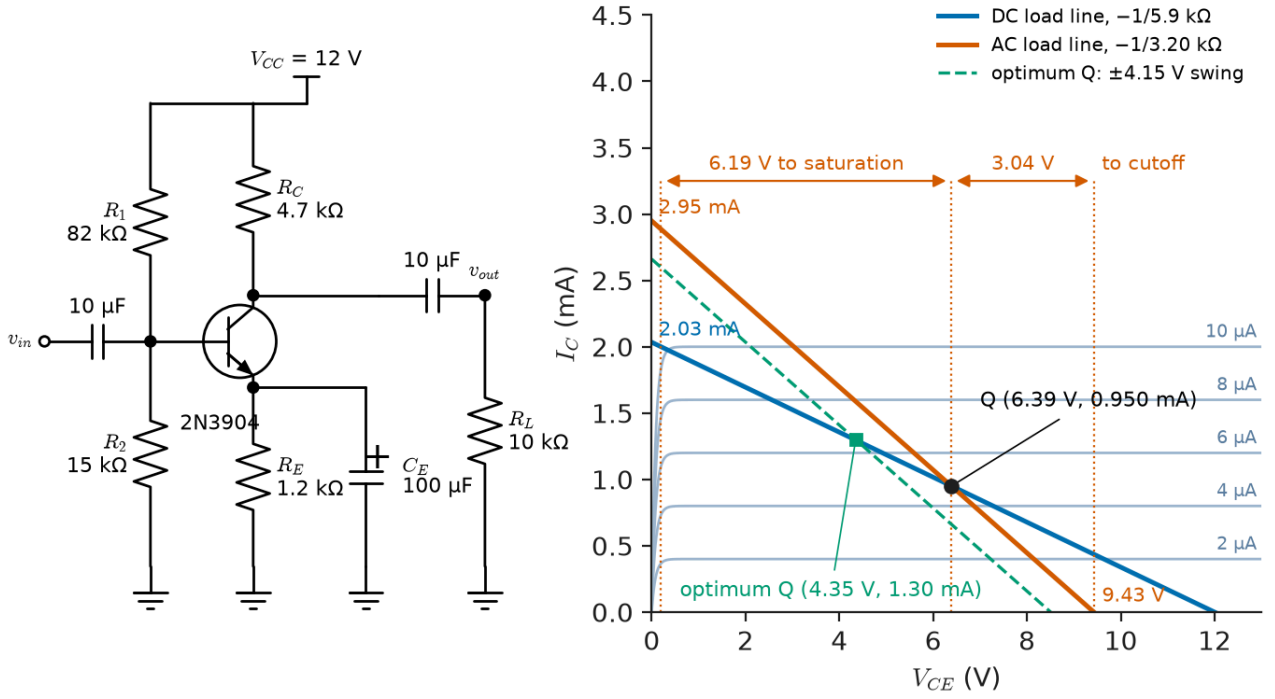


Figure 3.3 — Voltage-divider-biased common-emitter stage with its DC and AC load lines on the output characteristics.

3.7 Load lines

DC and AC load lines

The load-line construction of Chapter 1, Section 1.2 applies to the transistor's output. At DC, with the emitter resistor and collector resistor in series with V_{CE}, KVL gives the **DC load line**

$$V_{CE} = V_{CC} - I_C\,(R_C + R_E)$$

which runs from $(V_{CC}, 0)$ to $(0, V_{CC}/(R_C + R_E))$; the Q-point is its intersection with the output characteristic for the bias current. For signals the coupling and bypass capacitors are shorts, so the collector sees $R_{ac} = R_C \parallel R_L$ and the emitter is grounded. Signal excursions therefore follow the **AC load line**, through the Q-point with slope $-1/R_{ac}$, steeper than the DC line. The transistor can swing along the AC line until it reaches cutoff ($I_C = 0$), a change of $I_{CQ}R_{ac}$ in V_{CE}, or saturation, a change of $V_{CEQ} - V_{CE(sat)}$. The smaller of the two sets the largest undistorted output amplitude.

Worked example — swing of the biased stage. For the stage of Section 3.6 with $R_L = 10$ kΩ, the DC load line runs from 12 V to $12/5.9$ kΩ $= 2.03$ mA, and the Q-point is (6.39 V, 0.950 mA). The AC load resistance is $4.7 \parallel 10 = 3.20$ kΩ. Toward cutoff the collector can rise by 0.950 mA $\times$ 3.20 kΩ $= 3.04$ V; toward saturation it can fall by $6.39 - 0.2 = 6.19$ V. The output therefore clips first at the positive peak, at 3.04 V amplitude. For maximum symmetrical swing the Q-point should satisfy $I_{CQ}R_{ac} = V_{CEQ} - V_{CE(sat)}$ together with the DC load line, which gives $I_{CQ} = (V_{CC} - V_{CE(sat)})/(R_{ac} + R_C + R_E) = 11.8/9.10$ kΩ $= 1.30$ mA, $V_{CEQ} = 4.35$ V and a symmetrical swing of ±4.15 V (Figure 3.3). Note what this swing demands at the input: with R_E bypassed the gain is $g_m R_{ac} = 36.7$ mS $\times$ 3.20 kΩ $= 118$, so a 3 V output needs 26 mV at the base, about one V_T, where the exponential distorts heavily (Section 3.9). The load line tells how far the output can swing; linearity at that swing requires degeneration or feedback.

> **Status of the principle: Load-line analysis**
> *Derived.* It is KVL in the output loop plotted against the device characteristic; the AC line assumes the coupling and bypass capacitors are short circuits at the signal frequency, so it holds in the mid-band only.

3.8 Small-signal models

The hybrid-π model

Linearising the Ebers–Moll (or Gummel–Poon) model about the Q-point (Chapter 1, Section 1.6) gives the **hybrid-π model** of Figure 3.4, the standard small-signal BJT model for hand analysis and the form a simulator reports in its operating-point listing. Its elements and their values at $I_C = 1$ mA for a 2N3904-class transistor are listed in Table 3.4.

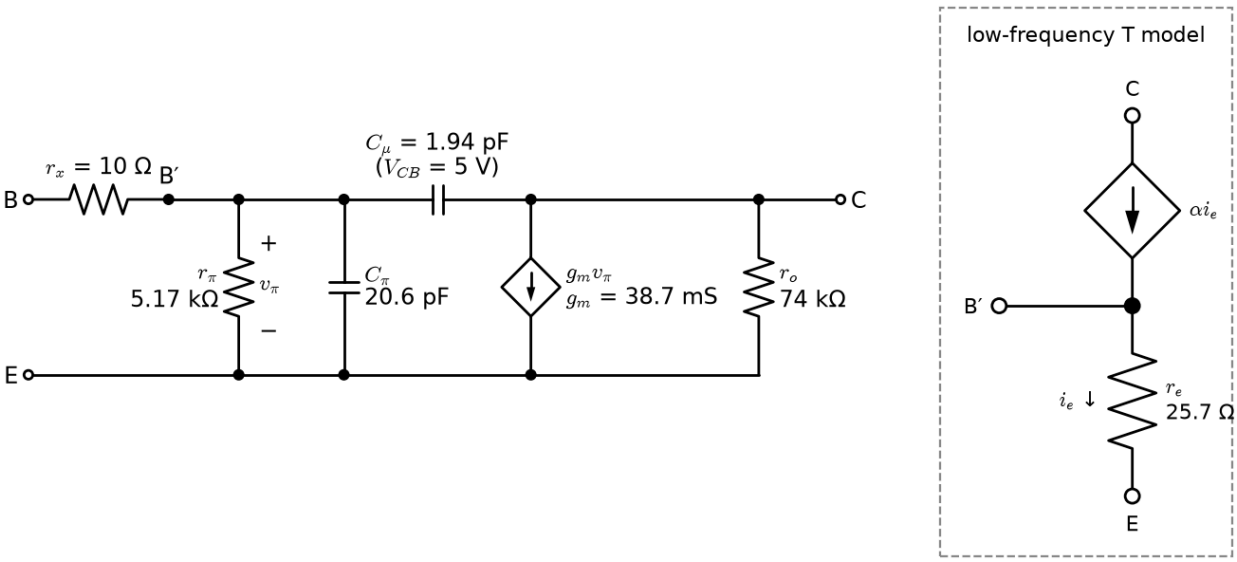


Figure 3.4 — Complete hybrid-pi small-signal model of a bipolar transistor with element values at IC = 1 mA.

Table 3.4 — Hybrid-π elements and representative values (2N3904 class, I_C = 1 mA, V_{CE} = 5 V, 300 K).

Element	Expression	Value at 1 mA	Scaling with I_C
Transconductance g_m	I_C/V_T	38.7 mS	$\propto I_C$
Input resistance r_π	β/g_m	5.17 kΩ (β = 200)	$\propto 1/I_C$
Output resistance r_o	V_A/I_C	74 kΩ	$\propto 1/I_C$
Base spreading resistance r_x	from layout	10–200 Ω	≈ constant
Base–emitter capacitance C_π	$g_m\tau_F + C_{je}$	20.6 pF	rises with I_C
Base–collector capacitance C_μ	junction capacitance at V_{CB}	1.9 pF at 5 V	falls with V_{CB}
Intrinsic gain $g_m r_o$	V_A/V_T	2860 (69 dB)	independent of I_C

The model has three remarkable properties. The transconductance depends only on bias current and temperature, not on the device: every silicon BJT at 1 mA has $g_m = 38.7$ mS at 300 K, whatever its size or type. The input resistance r_π inherits all of β's spread, which is why circuit gains that depend on r_π are as poorly defined as β itself. And the product $g_m r_o = V_A/V_T$, the **intrinsic gain**, is the largest voltage gain a single transistor can provide with an ideal current-source load, a few thousand for a BJT, compared with tens to a few hundred for short-channel MOSFETs (Chapter 4).

The **T-model** is an equivalent rearrangement with the emitter resistance $r_e = \alpha/g_m$ in the emitter lead and a current source αi_e from collector to base node; it gives the same answers and is more convenient for common-base and degenerated stages. The **h-parameter** description used on older datasheets maps onto hybrid-π as $h_{ie} = r_x + r_\pi$, $h_{fe} = \beta$, $h_{oe} = 1/r_o$ and $h_{re} \approx r_\pi/r_\mu$ (very small); two-port theory and the h-parameter conversions are developed in *Circuits, Signals and Communications*, Ch. 12.

> **Status of the principle: Hybrid-π small-signal model**
> *Derived*. It is the first-order linearisation of the large-signal model about the operating point; it is *approximate* in use, accurate for signals small compared with V_T at the base–emitter junction and up to frequencies of about $f_T/3$, beyond which distributed effects and excess phase require more elaborate models.

3.9 Amplification

Where gain comes from

A BJT amplifies because a small voltage at its input controls a current through a large resistance at its output: power from the supply is modulated by a signal that supplies very little. In small-signal terms the voltage gain of any stage built on one transistor is the product of the transconductance and the total resistance at the collector, $A_v = -g_m R_{out,total}$. Three limits follow.

1. **Gain is bounded by the intrinsic gain.** With an ideal current-source load the collector resistance is r_o and $|A_v| = g_m r_o = V_A/V_T$, about 2860 for the 2N3904. With a resistor load, $|A_v| = I_C R_C/V_T$ is limited by the supply: to get a gain of 1000 a resistor would need 26 V across it. This is why integrated amplifiers use active (current-source) loads (Chapter 10).
2. **Gain is set by bias.** Since $g_m \propto I_C$, any change in bias current changes gain proportionally. Stable gain requires stable bias, or degeneration that removes g_m from the gain expression.
3. **Gain is limited by linearity.** The collector current is an exponential function of v_{be}, so the input must stay well below V_T for linear operation.

Linearity and degeneration

The second-harmonic distortion of an exponential driven by a sinusoid of amplitude V_p is $\mathrm{HD}_2 \approx V_p/4V_T$ (Chapter 1, Section 1.6), 1 % at about 1 mV for an undegenerated CE stage. With emitter degeneration only the fraction $1/(1 + g_m R_E)$ of the input appears across the junction, and the distortion falls by the same factor (to first order; the feedback also reduces the third harmonic). A stage with $I_C = 1$ mA and $R_E = 470\ \Omega$ has $g_m R_E = 18.2$; its gain falls from -182 to $-R_C/(R_E + r_e) = -4700/496 = -9.5$, but its input can be 19 times larger for the same distortion. This trade of gain for linearity, input range and predictability is the elementary form of negative feedback, which Chapter 9 generalises.

Worked example — gain of the biased stage with partial bypass. In the stage of Section 3.6 ($I_C = 0.950$ mA, $g_m = 36.7$ mS, $R_C \parallel R_L = 3.20$ kΩ), the 1.2 kΩ emitter resistor is split into 100 Ω unbypassed and 1.1 kΩ bypassed. The mid-band gain is $-g_m R_{ac}/(1 + g_m R_{E1}) = -117.5/(1 + 3.67) = -25.1$, equal to the ratio form $-3200/(100 + 27.2) = -25.1$. The input resistance at the base rises from $r_\pi = 5.44$ kΩ to $r_\pi + (\beta + 1) \times 100 = 25.5$ kΩ at $\beta = 200$, and the divider's 12.7 kΩ in parallel gives 8.5 kΩ. For a 3 V output the base now needs 119 mV, of which only $119/4.67 = 25$ mV appears across the junction; the second harmonic is the open-junction value $25.5/(4 \times 25.85) = 25$ % divided by the loop factor 4.67, about 5 %, and a designer who needs less adds overall feedback (Chapter 9) or lowers the swing. The gain now changes by only about 0.2 % per 1 % change in I_C, instead of 1 % per 1 %.

3.10 Switching behavior

The transistor as a switch

As a switch a BJT alternates between cutoff (off, $I_C \approx 0$, the full supply across V_{CE}) and saturation (on, $V_{CE} = V_{CE(sat)}$). In both states its dissipation is small: leakage times voltage when off, $V_{CE(sat)} I_C$ when on. The design rules are to supply enough base current to saturate the transistor at the lowest β and the largest load current (forced beta 10–20, Section

3.2), to hold the base firmly off when the driver is off, and to protect the collector from inductive voltage spikes (Chapter 2, Section 2.10).

Stored charge and switching times

The transitions are not instantaneous. Datasheets specify four intervals (Figure 3.5), measured with a stated collector current and base drive:

- **delay time** t_d: charging the junction capacitances until the transistor starts conducting;
- **rise time** t_r: building up the base charge $\tau_F I_C$ needed to carry the collector current;
- **storage time** t_s: removing the *excess* base charge stored in saturation, during which the collector current does not change at all;
- **fall time** t_f: removing the active-region charge as the current falls.

For the 2N3904 the guaranteed maxima are $t_d = 35$ ns, $t_r = 35$ ns, $t_s = 200$ ns and $t_f = 50$ ns at $I_C = 10$ mA with $I_{B1} = I_{B2} = 1$ mA. Storage time dominates. In saturation both junctions are forward biased and the base and collector regions hold excess charge proportional to the overdrive $I_{B1} - I_C/\beta_F$. The charge-control model gives

$$t_s = \tau_s \ln \frac{I_{B1} + I_{B2}}{I_C/\beta_F + I_{B2}}$$

where I_{B1} is the forward base drive, I_{B2} the reverse current pulled out of the base at turn-off, and τ_s the storage time constant. Hard overdrive (large I_{B1}) speeds turn-on but lengthens storage; a strong reverse base current shortens it. Practical remedies are a **speed-up capacitor** across the base resistor, which supplies a pulse of charge at both edges, a **Baker clamp** (a Schottky diode from base to collector, as in Schottky TTL) that prevents deep saturation, and a low-impedance turn-off path.

> **Status of the principle: Charge-control model of switching**
> *Approximate.* It lumps the stored minority charge into one or two time constants and assumes constant β_F and drive currents; it predicts trends and order-of-magnitude storage times well, but quantitative values must be taken from datasheet test conditions or simulation.

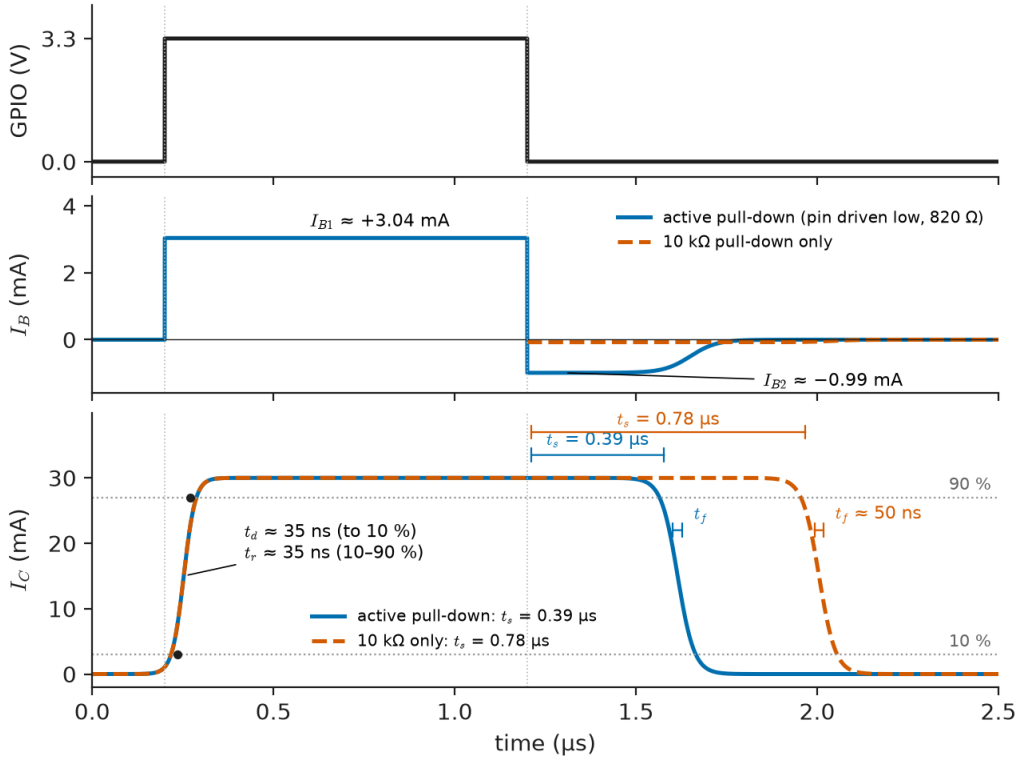


Figure 3.5 — Switching waveforms of a saturated NPN relay driver, showing delay, rise, storage and fall times.

Worked example — a relay driver from a 3.3 V microcontroller pin. A 12 V relay with a 400 Ω coil (30 mA) is switched by a 2N3904 from a 3.3 V GPIO pin. At 30 mA the minimum guaranteed h_{FE} is between 60 (at 50 mA) and 100 (at 10 mA), so a forced beta of 10 gives a generous margin: $I_B = 3$ mA. With $V_{BE} \approx 0.75$ V in saturation, $R_B = (3.3 - 0.75)/3\ \text{mA} = 850\ \Omega$; choose 820 Ω, which passes 3.11 mA; the 10 kΩ base–emitter resistor described next takes 75 µA of it, so the base drive is $I_{B1} \approx 3.04$ mA, within a typical MCU pin's 4–8 mA rating. The collector saturates at about 0.1–0.2 V and dissipates under 6 mW. A 10 kΩ resistor from base to emitter holds the transistor off while the pin is a floating input during reset. The coil needs a flyback diode (Chapter 2, Section 2.10); without it the collector would exceed the 40 V V_{CEO} at every turn-off, which is the failure the opening example's cost-cutting produced.

Storage time decides how soon the relay begins to release. Back-calculating from the datasheet ($t_s = 200$ ns at 10 mA, 1 mA, with $\beta_F \approx 150$) gives $\tau_s \approx 320$ ns. If the pin is switched to a high-impedance input, only the 10 kΩ pull-down removes base charge, $I_{B2} = 0.75\ \text{V}/10\ \text{k}\Omega = 75\ \mu\text{A}$, and $t_s = 320\ \text{ns} \times \ln[(3.04 + 0.075)/(0.2 + 0.075)] = 0.78\ \mu\text{s}$. Driving the pin actively low pulls about $0.75/820 = 0.91$ mA out of the base through the 820 Ω, plus the 75 µA in the pull-down, so $I_{B2} \approx 0.99$ mA and $t_s = 320\ \text{ns} \times \ln(4.03/1.19) = 0.39\ \mu\text{s}$. A higher-$\beta$ lot stores more excess charge for the same drive, which is the half-microsecond change seen after the second-source lot. For a relay, whose contacts take milliseconds, none of this matters; for a 100 kHz PWM drive, a storage time of 0.8 µs would be 8 % of the period, and a MOSFET (Chapter 4) or a Baker-clamped transistor would be chosen.

3.11 Frequency response

Charge, capacitance and the transition frequency

The hybrid-π capacitances of Section 3.8 make the transistor's gain fall at high frequency. C_π has two parts: the base–emitter junction capacitance C_{je} and the diffusion capacitance $g_m \tau_F$ of the minority charge in transit across the base, where the **forward transit time** τ_F is about $W_B^2/2D_n$ plus collector transit time, a few hundred picoseconds for a 2N3904 and under a picosecond in SiGe HBTs. C_μ is the collector–base junction capacitance.

With the collector shorted to AC ground, the current gain $h_{fe}(j\omega) = i_c/i_b$ falls from β at low frequency with a pole at $f_\beta = 1/[2\pi r_\pi (C_\pi + C_\mu)]$, and crosses unity at the **transition frequency**

$$f_T = \frac{g_m}{2\pi(C_\pi + C_\mu)} = \beta f_\beta$$

Inverting this, $1/(2\pi f_T) = \tau_F + (C_{je} + C_\mu)V_T/I_C$: at low current the junction capacitances dominate and f_T rises in proportion to I_C; at moderate current f_T approaches the transit-time limit $1/(2\pi\tau_F)$; and at high current it falls again as high injection and the base push-out (Kirk) effect widen the effective base. Figure 3.6 shows the curve. For the 2N3904 model at $V_{CE} = 5$ V, f_T is about 51 MHz at 0.1 mA, 273 MHz at 1 mA and 480 MHz at 10 mA; the datasheet guarantees at least 300 MHz at 10 mA and 20 V. The usable bandwidth of real stages is far below f_T, because of the Miller effect.

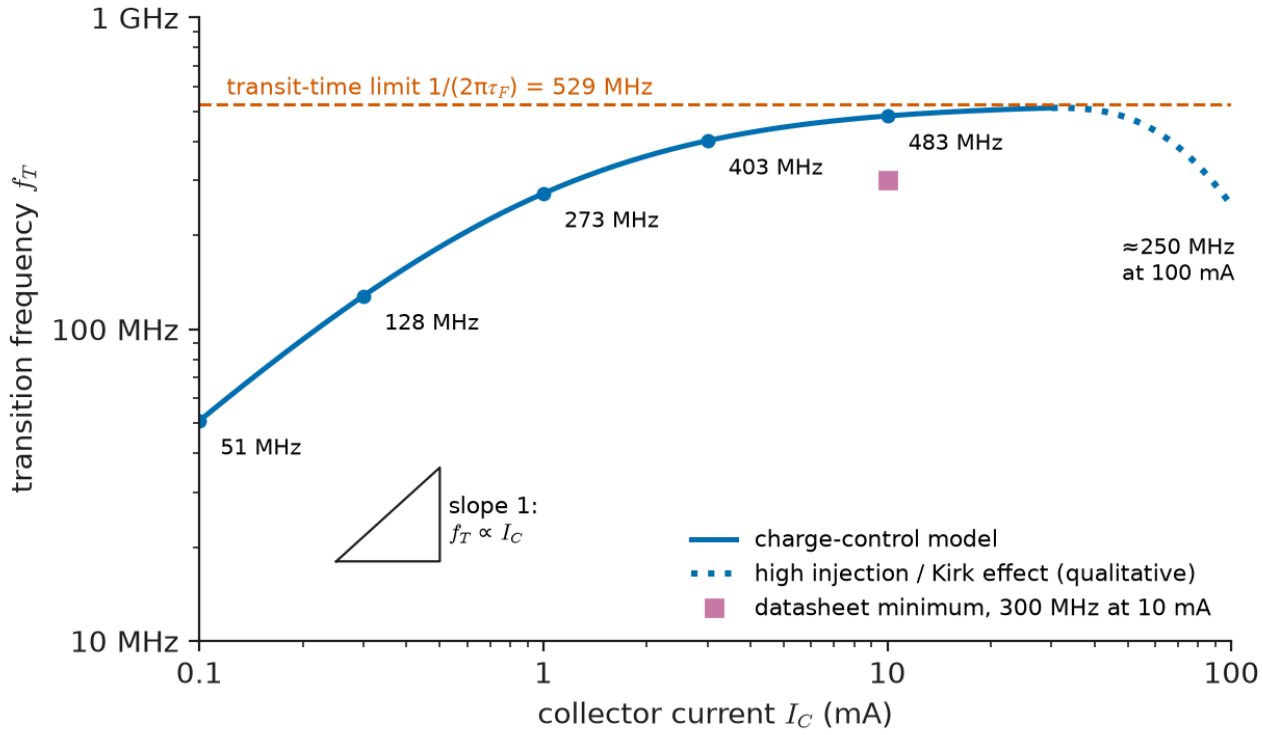


Figure 3.6 — Transition frequency of a small-signal NPN transistor versus collector current.

The Miller effect

In a common-emitter stage, C_μ connects the input (base) to the output (collector), which swings $-A_v$ times as much as the base. The current through C_μ is therefore $(1 + |A_v|)$ times larger than if its far end were grounded, and seen from the base it acts as a capacitance

$$C_M = C_\mu\,(1 + g_m R'_L)$$

where R'_L is the total collector load. This **Miller capacitance** usually dominates the input capacitance and sets the stage's upper cut-off frequency. The Miller approximation, its exactness conditions and the open-circuit time-constant method belong to amplifier analysis in Chapter 7; the frequency-response tools themselves (poles, Bode plots) are in *Circuits, Signals and Communications*, Ch. 13. At device level the lesson is that the transistor's C_μ, not its f_T, usually limits the bandwidth of a voltage-gain stage, and that configurations that keep C_μ away from the gain (common base, cascode, emitter follower) are the cure.

Worked example — bandwidth of a CE stage at 1 mA. A 2N3904 at $I_C = 1$ mA ($g_m = 38.7$ mS, $r_\pi = 5.17$ kΩ, $C_\pi = 20.6$ pF, $C_\mu = 1.94$ pF at $V_{CB} = 5$ V, $r_x = 10$ Ω) drives a total collector load of 2.5 kΩ from a 1 kΩ source. The mid-band gain from base to collector is $-g_m R'_L = -96.7$. The Miller capacitance is $1.94 \times 97.7 = 190$ pF, so the input capacitance is $20.6 + 190 = 210$ pF, ten times C_π. The resistance seen by it is $(1\text{ k}\Omega + 10\ \Omega) \parallel 5.17\text{ k}\Omega = 845\ \Omega$, and the input pole is at $1/(2\pi \times 845 \times 210\text{ pF}) = 0.90$ MHz. The transistor's f_T is 273 MHz at this current and f_β is 1.36 MHz,

yet the stage's bandwidth is under 1 MHz, and 90 % of the limiting capacitance is C_μ multiplied by the gain. The output node, with C_μ seen from the collector, has its pole at about 32 MHz and does not matter here. A cascode would hold the first transistor's collector at nearly constant voltage, reduce its Miller multiplication to about 2 and move the input pole to about 8 MHz (Chapter 7).

> **Status of the principle: Transition frequency** $f_T = g_m/2\pi(C_\pi + C_\mu)$
> *Approximate*. It follows from the hybrid-π model with the collector at AC ground and neglects r_x, excess phase and the feed-forward current through C_μ; it describes the device's intrinsic speed accurately to within 10–20 % below about $f_T/2$, and the maximum oscillation frequency $f_{max} = \sqrt{f_T/(8\pi r_x C_\mu)}$ is the better figure of merit for power gain at RF.

Speed versus current and size

The f_T curve gives the BJT designer a quantitative trade-off. Running a transistor at a current where f_T is near its peak maximises speed but costs power; running it at low current saves power but makes the junction capacitances dominate. Physically smaller devices have lower capacitances and reach a given f_T at lower current, which is why RF transistors are small and why the high-current fall-off sets a maximum current per unit emitter area. The same balance between current, capacitance and speed reappears in the MOSFET as the g_m/I_D design method (Chapter 4) and in CMOS logic as the power–delay product (Chapter 33).

What remains open

Open question	Where it is answered
How does a field-effect transistor compare with the BJT in gain, linearity, input current, switching and power capability?	Chapter 4
How do thyristors, IGBTs and wide-bandgap devices extend bipolar action to kilovolts and kiloamperes?	Chapter 5
How are complete CE, CB and CC stages, multistage amplifiers, the Miller approximation and amplifier distortion analysed and designed?	Chapter 7
How does negative feedback make gain, impedance and linearity independent of β?	Chapter 9
How are differential pairs, current mirrors, active loads and bandgap references built from matched BJTs?	Chapter 10
How do bipolar saturated switching and its storage time compare with MOSFET gate driving in power stages?	Chapter 29
How are two-port parameters and frequency responses manipulated in general?	*Circuits, Signals and Communications*, Ch. 12–13
What sets minority-carrier diffusion length and lifetime in the base?	*Engineering Physics*, Ch. 14 §14.5; *Materials Science and Engineering*, Ch. 32

Next: Chapter 4 turns to the field-effect transistor, in which a gate voltage controls a channel through an insulator instead of a junction, and compares its models, biasing and switching with those of the BJT.

Chapter 4 — Field-Effect Transistors

Part I · Electronic Devices

A firmware engineer adds a 12 V, 5 A solenoid valve to an irrigation controller and switches it with an IRF540N, a 100 V power MOSFET with a datasheet on-resistance of 44 mΩ, driven straight from a 3.3 V microcontroller pin. On the bench the valve clicks. In the field, on a warm afternoon, the MOSFET reaches 150 °C and the board browns. Nothing was out of specification on the front page of the datasheet: the 44 mΩ is guaranteed at a gate-source voltage of 10 V, and the threshold voltage of that part lies anywhere between 2 and 4 V. At 3.3 V of gate drive, a part near the top of its threshold spread is below threshold and barely conducts at all; one near 3 V conducts with a channel resistance of ohms rather than milliohms, and the 5 A load turns it into a heater. This chapter is about reading those lines: how a field-effect transistor's channel is formed and controlled, which model describes it in which region, how its threshold moves with process, body bias and temperature, how it is biased so that a circuit survives that spread, and how it behaves as an amplifier, as a logic switch and as a power switch.

> **How does a gate voltage control a channel, and which model of the drain current holds where? How do threshold spread, body effect and temperature decide whether a biasing scheme or a switch design survives production? What separates a power MOSFET from a signal MOSFET, and which datasheet limits govern it?**

Concept	Engineering destination
JFET operation	Low-noise preamplifiers for piezo, microphone and photodiode sensors; current-regulator diodes; analog switches
MOSFET structure	Reading symbols and datasheets; body diodes; oxide limits on gate voltage
Enhancement and depletion modes	Normally-off logic and power switches; normally-on start-up circuits and current sources
Threshold voltage	Logic-level versus standard gate drive; body effect in stacked and source-follower stages; leakage at temperature
Operating regions	Choosing square-law, short-channel or subthreshold models; switch versus amplifier operation
DC biasing	Discrete amplifier stages that survive a 2:1 threshold spread
Small-signal models	Gain, impedance and bandwidth estimates; g_m/I_D design
MOSFET amplification	Common-source, follower and cascode stages (developed in Ch. 7 and Ch. 10)
MOSFET switching	Load switches, low-side drivers, gate-charge timing from an MCU pin
Power MOSFET fundamentals	Motor drives, DC–DC converters, battery disconnects, hot-swap controllers

Roadmap. The junction FET as the simplest field-effect device (4.1) → the MOSFET's structure (4.2), its two modes (4.3) and its threshold (4.4) → the operating regions and the models that describe them (4.5) → biasing a discrete stage against spread (4.6) → the small-signal model (4.7) and what it gives an amplifier (4.8) → the MOSFET as a switch (4.9) → the vertical power MOSFET and its ratings (4.10).

4.1 JFET operation

Where *Engineering Physics* leaves off

Engineering Physics sketches the MOSFET in a single section (Ch. 18 §18.3.3): the inversion channel formed when $V_{GS} > V_{th}$, the triode and saturation square laws with a $(1 + \lambda V_{DS})$ correction, the transconductance $g_m = 2I_D/(V_{GS} - V_{th})$, and a remark that short-channel effects and gate tunnelling break the simple model. The doping and carrier-density results that set the threshold are in Ch. 6 §6.6, and the thermal-resistance network that turns dissipated power into junction temperature is in Ch. 14 §14.7. This chapter takes those results as given and does not re-derive them. It adds what the sketch leaves out: the junction FET, depletion-mode devices, the threshold's dependence on body bias and temperature, subthreshold conduction, the short-channel and velocity-saturated models that real devices follow, biasing against production spread, capacitances and gate charge, and the vertical power MOSFET with its datasheet ratings, safe operating area and electrothermal behaviour. Where it goes beyond the validity of the sketch (high fields, short channels, self-heating, avalanche), it says so.

Structure and pinch-off

A junction field-effect transistor (JFET) is a bar of doped semiconductor, the **channel**, with ohmic contacts at each end, the **source** and **drain**, and a reverse-biased PN junction along its side, the **gate** (Figure 4.1). In an n-channel JFET the channel is n-type and the gate p+. Making the gate more negative widens the depletion region that the junction pushes into the channel, narrowing the conducting cross-section. At the **pinch-off voltage** V_P (datasheets write $V_{GS(off)}$), negative for an n-channel device, the depletion regions meet and the channel closes. The gate junction is always kept reverse-biased (or at most forward-biased by less than about 0.5 V), so the gate draws only junction leakage: picoamperes in a small-signal JFET at 25 °C, doubling roughly every 10 °C.

With V_{DS} applied, the channel near the drain is more reverse-biased than near the source and pinches first. Beyond $V_{DS} = V_{GS} - V_P$ the drain current stops rising and the JFET is in **saturation** (the "pinch-off region" of older texts). There the drain current follows, to a good approximation, the Shockley square law

$$I_D = I_{DSS}\left(1 - \frac{V_{GS}}{V_P}\right)^2, \qquad V_P \le V_{GS} \le 0$$

where I_{DSS} is the saturated drain current at $V_{GS} = 0$. The JFET is therefore a **normally-on** (depletion-mode) device: it conducts its largest current with the gate tied to the source, and the gate can only turn it down.

> **Status of the principle: JFET square law**
> *Approximate.* It follows from the gradual-channel approximation for an abrupt junction and uniform channel doping; real channels are graded, so the exponent fitted to measured transfer curves is 1.8–2.2, and the law does not cover the subthreshold tail near V_P or the triode region at low V_{DS}.

Differentiating gives the transconductance

$$g_m = \frac{2I_{DSS}}{|V_P|}\sqrt{\frac{I_D}{I_{DSS}}} = g_{m0}\sqrt{\frac{I_D}{I_{DSS}}}$$

with $g_{m0} = 2I_{DSS}/|V_P|$ the value at $V_{GS} = 0$.

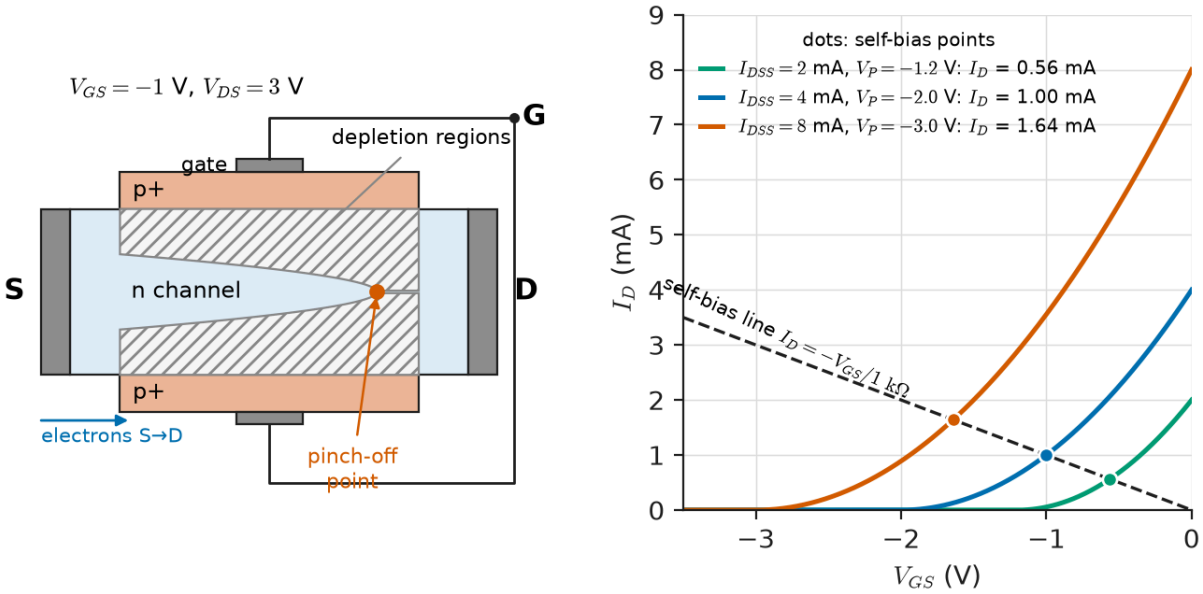


Figure 4.1 — N-channel JFET cross-section with depletion regions, and its transfer characteristic.

Why a JFET is still used

The JFET has no gate oxide and no surface channel. Its carriers flow in the bulk, away from the interface traps that make a MOSFET noisy at low frequency, and its gate leakage is a junction current rather than a tunnelling current. Three applications survive on these properties:

- **Low-noise, high-impedance front ends.** A piezoelectric accelerometer, an electret or condenser microphone, a pH electrode or a photodiode in photoconductive mode presents a source impedance of megohms to gigohms. A bipolar input would load it with base current and add shot noise. The JFET's picoampere gate current and voltage noise of about 1 nV/√Hz (LSK170/2SK170 class, at $I_D \approx 2$ mA) make it the input device of discrete preamplifiers and of JFET-input op-amps (TL072, OPA1641).
- **Current-regulator diodes.** A JFET with gate tied to source, or with a small source resistor, is a two-terminal constant-current source (the J500 and 1N5283 families, 0.24–4.7 mA), used to bias LEDs, references and sensor bridges.
- **Analog switches and voltage-controlled resistors.** In the triode region a JFET is a resistor controlled by V_{GS}, used for automatic gain control and in sample-and-hold switches.

Table 4.1 — Representative small-signal FETs (datasheet ranges at 25 °C).

Part (class)	Type	I_{DSS} or I_D rating	V_P or $V_{GS(th)}$	Other key data
J201	n-JFET	0.2–1.0 mA	−0.3 to −1.5 V	$I_{GSS} \le 100$ pA
2N5457	n-JFET	1–5 mA	−0.5 to −6 V	$g_{fs} = 1$–5 mS
LSK170 (grade B)	n-JFET	6–12 mA	−0.2 to −2 V	$e_n \approx 0.9$ nV/√Hz at 1 kHz
LND150	depletion n-MOSFET	1–3 mA	−1 to −3 V	500 V rating
2N7000	enhancement n-MOSFET	200 mA	0.8–3.0 V	$R_{DS(on)} \le 5\ \Omega$ at 10 V
BSS138	enhancement n-MOSFET	220 mA	0.8–1.5 V	logic level, level shifters

The spreads in the table are real: pinch-off depends on channel doping and thickness, which vary across a wafer, so manufacturers sort JFETs into I_{DSS} grades rather than control them tightly. A circuit that depends on the nominal I_{DSS} or V_P will not survive a production run.

Worked example — Self-biasing a JFET preamplifier against spread. A charge-amplifier front end for a piezoelectric vibration sensor uses an n-JFET with nominal $I_{DSS} = 4$ mA and $V_P = -2.0$ V, to be biased at $I_D = 1$ mA by a source resistor R_S with the gate returned to ground through 100 MΩ.

At 1 mA the required gate-source voltage is $V_{GS} = V_P\left(1 - \sqrt{I_D/I_{DSS}}\right) = -2.0(1 - 0.5) = -1.0$ V, so $R_S = 1.0\ \text{V}/1\ \text{mA} = 1\ \text{k}\Omega$. The transconductance there is $g_m = (2 \times 4/2.0)\sqrt{0.25} = 2$ mS.

Now suppose production parts span $(I_{DSS}, V_P) = (2\ \text{mA}, -1.2\ \text{V})$ to $(8\ \text{mA}, -3.0\ \text{V})$. With $V_{GS} = -I_D R_S$, the bias point solves $I_D = I_{DSS}(1 + I_D R_S/V_P)^2$, a quadratic in I_D. Taking the root with $V_{GS} > V_P$:

- low corner: $I_D = 0.56$ mA, $V_{GS} = -0.56$ V;
- nominal: $I_D = 1.00$ mA, $V_{GS} = -1.00$ V;
- high corner: $I_D = 1.64$ mA, $V_{GS} = -1.64$ V.

A 4:1 spread in I_{DSS} becomes a 2.9:1 spread in bias current. Fixed bias at $V_{GS} = -1.0$ V would have given 0.056 mA at the low corner and 3.56 mA at the high one, a 64:1 spread. The source resistor is negative feedback: more current raises $|V_{GS}|$, which reduces the current. Figure 4.1 shows the three intersections. A larger R_S returned to a negative supply tightens the spread further, at the cost of headroom.

4.2 MOSFET structure

The four terminals

A planar n-channel MOSFET (Figure 4.2) is built in a p-type **body** (or well). Two heavily doped n+ regions form the **source** and **drain**; between them, the **gate**, a conductor (heavily doped polysilicon in older processes, a metal on a high-κ dielectric since the 45 nm node), sits on a thin gate insulator over the channel region. The MOSFET is therefore a four-terminal device: gate (G), drain (D), source (S) and body (B). The gate is insulated: its DC current is tunnelling leakage, negligible in discrete devices and in processes with oxides thicker than about 3 nm, but amperes per square centimetre in the thinnest oxides, which is why high-κ dielectrics were introduced.

The gate insulator sets the gate capacitance per unit area,

$$C_{ox} = \frac{\varepsilon_{ox}}{t_{ox}}$$

For SiO_2 ($\varepsilon_{ox} = 3.9\varepsilon_0$) at $t_{ox} = 2$ nm, $C_{ox} = 17$ fF/μm²; at 10 nm, 3.5 fF/μm²; in a 60 V discrete power MOSFET with a 50 nm oxide, 0.69 fF/μm². The same thickness sets the gate's voltage limit: SiO_2 breaks down near 10 MV/cm, so a 2 nm oxide is limited to about 1 V in long-term use, and discrete power MOSFETs with 50–100 nm oxides are rated at $V_{GS(max)} = \pm 20$ V. Exceeding $V_{GS(max)}$ even briefly, for example by ringing on a long gate trace, punctures the oxide; it is the commonest way to destroy a MOSFET during bring-up.

In an integrated circuit the source and body are separate terminals, the body of every NMOS device being the common p-substrate tied to ground, and the body of PMOS devices an n-well tied to V_{DD}. In a discrete MOSFET the body is shorted to the source inside the package. That short turns the body–drain PN junction into an **intrinsic body diode** from source to drain (anode at source in an n-channel device), drawn in every power-MOSFET symbol. The body diode conducts whenever the drain is pulled below the source: an inductive load turning off, a synchronous rectifier, a reverse-connected battery. It is a real PN diode with forward drop, stored charge and reverse recovery, which matters in Section 4.10 and in Chapter 29.

Symmetry and the PMOS device

A planar MOSFET is symmetric: for an n-channel device the source is whichever n+ terminal is at the lower potential, which matters in analog switches where current flows both ways. The p-channel MOSFET is the complementary device. Hole mobility in silicon inversion layers is 2.5–3 times lower than electron mobility, so a PMOS device needs about 2.5

times the width for the same current, and discrete p-channel power MOSFETs are used only where their simple high-side gate drive outweighs their higher $R_{DS(on)}$: reverse-polarity protection, load switches, small motor bridges.

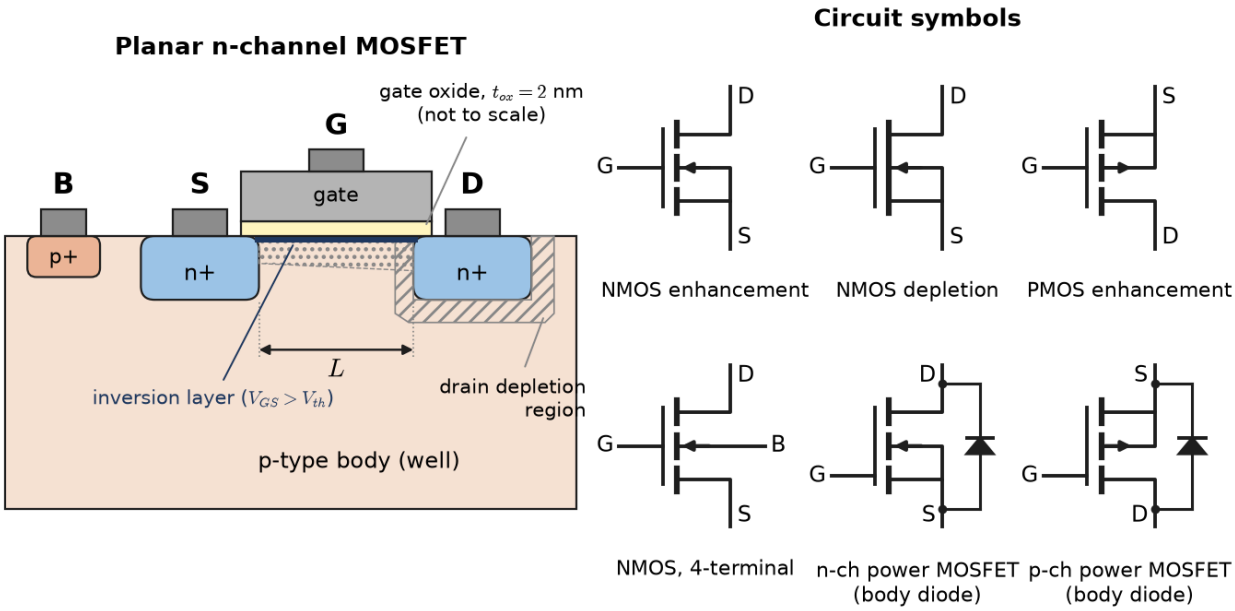


Figure 4.2 — Planar n-channel MOSFET cross-section and the common circuit symbols for enhancement, depletion and discrete power devices.

4.3 Enhancement and depletion modes

An **enhancement-mode** MOSFET has no channel at $V_{GS} = 0$; a channel must be induced by a gate voltage beyond the threshold, positive for NMOS, negative for PMOS. It is **normally off**, and almost every MOSFET in logic and power is of this kind, because a switch that is off when its driver is unpowered fails safe.

A **depletion-mode** MOSFET has a channel at $V_{GS} = 0$, built in by an implant of donors (for n-channel) under the gate. Its threshold is negative, typically −1 to −3 V for an n-channel device, and it is **normally on**: like the JFET, the gate must be driven negative to turn it off. Unlike the JFET, its gate is insulated, so it can also be driven positive to enhance the channel beyond I_{DSS}.

Depletion-mode MOSFETs are niche devices with distinctive jobs:

- **Start-up circuits for offline power supplies.** A 500–1000 V depletion MOSFET (LND150, BSS126) charges the controller's supply capacitor from the rectified mains at power-on and is turned off once the auxiliary winding takes over, saving the watts a bleeder resistor would waste.
- **Constant-current sources and inrush limiters**, using a source resistor exactly as in the JFET self-bias of Section 4.1.
- **Normally-on GaN HEMTs.** The two-dimensional electron gas of an AlGaN/GaN heterostructure exists without gate bias, so the natural GaN power transistor is a depletion-mode device; making it normally off for safety is one of the central device problems of wide-bandgap power electronics (Chapter 5).

Table 4.2 — The four MOSFET types.

Type	Channel at $V_{GS}=0$	Threshold	To turn on	To turn off	Typical use
n-channel enhancement	none	$V_{th}>0$ (0.3–4 V)	$V_{GS}>V_{th}$	$V_{GS}<V_{th}$	logic, low-side and bridge switches
p-channel enhancement	none	$V_{th}<0$ (−0.3 to −4 V)	$V_{GS}<V_{th}$	$V_{GS}>V_{th}$	CMOS pull-up, high-side load switch
n-channel depletion	present	$V_{th}<0$ (−1 to −3 V)	$V_{GS}=0$ suffices	$V_{GS}<V_{th}$	start-up, current sources
p-channel depletion	present	$V_{th}>0$	$V_{GS}=0$ suffices	$V_{GS}>V_{th}$	rare

4.4 Threshold voltage

What sets it

The threshold voltage is the gate-source voltage at which the surface under the gate inverts strongly enough to form a conducting channel. For an NMOS device it is the sum of three terms: the flat-band voltage V_{FB} (the work-function difference between gate and body minus the effect of fixed oxide charge), the band bending $2\phi_F$ needed to invert the surface (where ϕ_F is the body's Fermi potential, set by its doping through the carrier-density results of *Engineering Physics* Ch. 6 §6.6), and the voltage across the oxide needed to hold the depletion charge:

$$V_{th} = V_{FB} + 2\phi_F + \frac{\sqrt{2q\varepsilon_{Si}N_A\,(2\phi_F + V_{SB})}}{C_{ox}}$$

The process engineer adjusts V_{th} with a shallow channel implant and, in modern processes, with the gate metal's work function. The circuit engineer cares about three consequences: it depends on the source-body voltage, it depends on temperature, and it spreads.

Body effect

When the source is above the body ($V_{SB}>0$), the depletion charge grows and the threshold rises:

$$V_{th} = V_{th0} + \gamma\left(\sqrt{2\phi_F + V_{SB}} - \sqrt{2\phi_F}\right)$$

where V_{th0} is the threshold at $V_{SB}=0$ and $\gamma = \sqrt{2q\varepsilon_{Si}N_A}/C_{ox}$ is the **body-effect coefficient**, typically 0.2–0.6 $\mathrm{V}^{1/2}$ in CMOS processes.

> **Status of the principle: Body-effect threshold shift**
> *Approximate.* It follows from charge balance under the depletion approximation with uniform body doping; retrograde wells and halo implants in short-channel processes make the measured shift smaller and only approximately square-root, so foundry models (BSIM) fit it with extra parameters.

Worked example — Threshold of a stacked transistor. In a CMOS NAND gate, or in the upper device of a cascode, an NMOS transistor's source sits above ground. Take $V_{th0}=0.50$ V, $\gamma=0.4\ \mathrm{V}^{1/2}$, $2\phi_F=0.70$ V and a source 1.5 V above the grounded body:

$$V_{th} = 0.50 + 0.4\left(\sqrt{2.2}-\sqrt{0.7}\right) = 0.50 + 0.4(1.483-0.837) = 0.759\ \mathrm{V}$$

The threshold has risen by 0.26 V, more than half its nominal value. In a 1.8 V circuit this cuts the stacked device's overdrive, and with it its current, substantially. It is one reason that NAND gates with many inputs are slow, and why

source followers in ICs have gains below one even with ideal current-source loads (Section 4.7).

Temperature and subthreshold leakage

Through ϕ_F and the intrinsic carrier density, the threshold falls by about 1–3 mV/K in CMOS logic devices and 3–7 mV/K in power MOSFETs. Meanwhile, mobility falls roughly as $T^{-1.5}$. At high gate overdrive the mobility term wins and drain current *falls* with temperature; at low overdrive the threshold term wins and current *rises*. The two cancel at the **temperature-compensation point**, a gate voltage marked on power-MOSFET transfer curves (typically 4–6 V for a standard-gate part). Below it, a MOSFET in linear operation has positive thermal feedback, the root of the Spirito instability discussed in Section 4.10.

Below threshold the channel does not switch off abruptly. Drain current falls exponentially with V_{GS}, carried by diffusion over the source barrier, exactly as in a BJT:

$$I_D \approx I_{th}\exp\!\left(\frac{V_{GS}-V_{th}}{n V_T}\right), \qquad S = n V_T \ln 10$$

where $V_T = kT/q$, I_{th} is the current defined as "threshold" (datasheets use $I_D = 250\ \mu$A for power parts, foundries a few hundred nanoamperes per µm of width), $n = 1.2$–1.6 is the subthreshold slope factor, and S is the **subthreshold swing**, the gate voltage per decade of current: 60 mV/decade is the limit at 300 K for $n = 1$.

> **Status of the principle: Subthreshold exponential**
> *Approximate.* It follows from Boltzmann statistics of carriers crossing the source barrier, valid in weak inversion with V_{DS} above a few V_T; drain-induced barrier lowering in short channels makes I_{th} depend on V_{DS}, and at very low currents junction and gate leakage set a floor.

Worked example — Leakage of a low-threshold device at 100 °C. A low-V_{th} logic transistor has $V_{th} = 0.35$ V and $I_D = 1\ \mu$A at threshold at 25 °C, with $n = 1.4$. Its swing is $S = 1.4 \times 25.85\ \text{mV} \times 2.303 = 83.3$ mV/decade, so with the gate at 0 V its off-current is $1\ \mu\text{A} \times 10^{-0.35/0.0833} = 63$ pA.

At 100 °C the swing scales with absolute temperature to $83.3 \times 373/298 = 104.3$ mV/decade, and with $dV_{th}/dT = -1$ mV/K the threshold falls to 0.275 V. The off-current becomes $1\ \mu\text{A} \times 10^{-0.275/0.1043} = 2.3$ nA, 37 times larger. A processor with 10^9 such devices therefore leaks tens of times more at its hot corner than on a cool bench, which is why static power is specified at maximum junction temperature (Ch. 33) and why a phone's standby current rises in a hot car.

Threshold on a datasheet

Datasheets give $V_{GS(th)}$ as a minimum and maximum at a stated current, usually 250 µA or 1 mA with $V_{DS} = V_{GS}$. It is the voltage at which the device *starts* to conduct, not one at which it is usefully on. The useful number is the gate voltage at which $R_{DS(on)}$ is guaranteed: 10 V for standard-gate parts, 4.5 V (sometimes also 2.5 V or 1.8 V) for logic-level parts. The opening failure is the result of confusing the two: an IRF540N with $V_{GS(th)} = 2$–4 V is guaranteed only at 10 V, whereas an IRLZ44N, with $V_{GS(th)} = 1$–2 V, carries guaranteed $R_{DS(on)}$ figures of 22, 25 and 35 mΩ at 10, 5 and 4 V. Neither is specified at 3.3 V; a 3.3 V drive needs a part with a 2.5 V rating, or a gate driver.

4.5 Operating regions

The long-channel model and its regions

The square-law model of the inversion channel is summarised in *Engineering Physics* Ch. 18 §18.3.3; with $k'_n = \mu_n C_{ox}$ and overdrive $V_{ov} = V_{GS} - V_{th}$, it gives $I_D = k'_n (W/L)\left[V_{ov}V_{DS} - V_{DS}^2/2\right]$ in triode and $I_D = \tfrac{1}{2}k'_n(W/L)V_{ov}^2(1+\lambda V_{DS})$ in saturation. Table 4.3 collects the regions an engineer actually uses, including the ones the sketch does not cover.

Table 4.3 — MOSFET operating regions (n-channel; reverse all signs for p-channel).

Region	Condition	Drain current	Used for
Cut-off / subthreshold	$V_{GS} < V_{th}$	exponential in V_{GS}, slope S	off state, leakage; ultra-low-power analog (nA–µA)
Triode (ohmic, linear)	$V_{ov} > 0$, $V_{DS} < V_{ov}$	$\approx V_{DS}/R_{DS(on)}$ for small V_{DS}	switches, analog switches, voltage-controlled resistors
Saturation (active)	$V_{ov} > 0$, $V_{DS} \geq V_{ov}$	$\approx \frac{1}{2}k_n'(W/L)V_{ov}^2(1+\lambda V_{DS})$	amplifiers, current sources, linear regulators
Velocity saturation	short L, high V_{DS}	$\approx WC_{ox}v_{sat}(V_{ov} - V_{DS,sat})$, nearly linear in V_{ov}	digital CMOS below about 250 nm
Third quadrant	$V_{DS} < 0$	channel conducts in reverse; body diode beyond about −0.6 V	synchronous rectification
Breakdown / avalanche	$V_{DS} > V_{(BR)DSS}$	limited only externally	inductive clamping (rated in E_{AS})

Two things in the table deserve comment. The "linear" region of a MOSFET is the triode region, where it behaves as a resistor, whereas the "linear" region of a BJT is its active region, where it amplifies; the vocabulary is a perennial trap. And for small V_{DS} the triode law reduces to a resistance

$$R_{DS(on)} \approx \frac{1}{k_n'(W/L)\,(V_{GS} - V_{th})}$$

which falls as the gate overdrive rises. Doubling the overdrive halves the channel resistance; at an overdrive near zero the resistance goes to infinity, which is the opening failure in one formula.

Where the square law fails

The square law assumes a constant mobility and a channel long enough that carriers never approach their saturation velocity. Three effects break it in modern devices.

Mobility degradation. The vertical field from the gate pushes carriers against the oxide interface, where surface-roughness scattering lowers their mobility. The effective mobility falls as $\mu_{eff} = \mu_0/(1+\theta V_{ov})$, with $\theta \approx 0.1$–$1\ \mathrm{V}^{-1}$, so drain current rises more slowly than V_{ov}^2 at high gate drive.

Velocity saturation. In silicon the electron drift velocity saturates near $v_{sat} \approx 10^7$ cm/s at fields above about 30 kV/cm. A 1 V drain-source voltage across a 100 nm channel is 100 kV/cm, so carriers in every short-channel device travel at or near v_{sat}. The saturation current then becomes

$$I_{D,sat} \approx WC_{ox}v_{sat}\,(V_{ov} - V_{DS,sat})$$

nearly linear in the overdrive, and it no longer improves as $1/L$. The empirical **alpha-power law**, $I_{D,sat} \propto V_{ov}^{\alpha}$ with α between 1.2 and 1.5 in current processes, is the compact form used in delay estimates.

Short-channel electrostatics. When the drain's depletion region reaches toward the source, the drain helps the gate lower the source barrier: **drain-induced barrier lowering** (DIBL), of order 50–100 mV per volt of V_{DS} in a planar device. The threshold then falls with V_{DS} and with shorter L (V_{th} roll-off), and output resistance falls. FinFET and gate-all-around structures exist to restore gate control.

> **Status of the principle: Square-law MOSFET**
> *Approximate.* It assumes constant mobility, a gradual channel and no velocity saturation; it is a good model for channels longer than about 1 µm at modest overdrive (discrete small-signal parts and analog devices drawn long on purpose), and it overestimates current and transconductance in short-channel devices by factors of 1.5–3 at high overdrive.

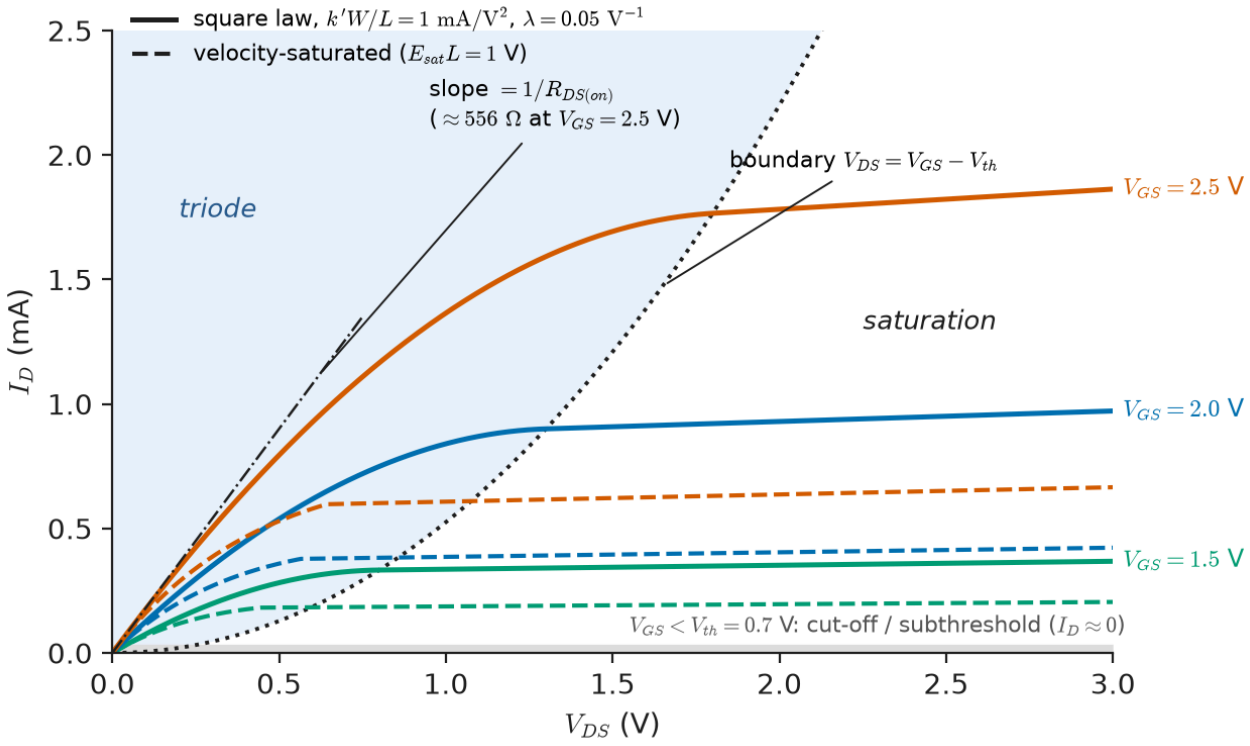


Figure 4.3 — Output characteristics of an n-channel MOSFET with operating regions marked, comparing long-channel square-law and velocity-saturated behaviour.

Figure 4.3 compares the two families of curves. Use the square law for insight on long-channel and discrete devices, the g_m/I_D method of Section 4.7 for short-channel analog design, and the foundry's compact model (BSIM-CMG, PSP) for anything that must be right to better than about 30 %.

4.6 DC biasing

The design problem

A MOSFET amplifier stage must sit in saturation, at a drain current chosen for gain, bandwidth, noise and power, with enough V_{DS} headroom for the signal swing. The obstacle is the threshold spread. A discrete 2N7000 has $V_{GS(th)}$ between 0.8 and 3.0 V; a production power MOSFET may span 2 V. Integrated circuits avoid the problem by matching (two transistors side by side share their threshold within millivolts) and bias with current mirrors (Ch. 10). Discrete stages must use feedback.

Fixed gate bias fails

Suppose a 2N7000-class part, modelled by the square law with $\frac{1}{2}k_n'W/L = K = 0.1$ A/V^2, is biased by a divider at $V_{GS} = 2.5$ V. At the low threshold, 0.8 V, it would conduct $0.1 \times 1.7^2 = 289$ mA, beyond its 200 mA rating; at the typical 2.1 V, 16 mA; at the high threshold, 3.0 V, nothing. A fixed gate bias is a design for one part, not for a production run.

Source degeneration

A resistor R_S in the source puts the bias current in series with the gate-source loop. With the gate held at V_G by a divider,

$$V_G = V_{GS} + I_D R_S = V_{th} + V_{ov} + I_D R_S$$

If $I_D R_S$ is large compared with the spread of $V_{th} + V_{ov}$, then $I_D \approx (V_G - V_{th})/R_S$, and a threshold change ΔV_{th} moves the current by only about $\Delta V_{th}/R_S$. The design rule is to make the voltage across R_S several times the expected

threshold spread, within the supply headroom. The other schemes are variations on the same feedback idea: **drain-to-gate feedback** (a resistor from drain to gate, which puts the device at $V_{GS} = V_{DS}$ and so guarantees saturation), **self-bias** for JFETs and depletion devices (Section 4.1), and **current-source bias**, where a mirror or a second transistor sets I_D directly.

> **Status of the principle: Bias stabilisation by source degeneration**
> *Derived.* It is series negative feedback around the gate-source junction: the sensitivity of I_D to V_{th} is $-1/(R_S + 1/g_m)$, reduced compared with fixed bias by the loop gain $g_m R_S$; it holds wherever the device stays in saturation.

Worked example — A degenerated 2N7000 stage. Use the same part from a 12 V supply with a divider setting $V_G = 6$ V, $R_S = 1$ kΩ and $R_D = 1$ kΩ. The bias point satisfies $V_G - V_{th} = V_{ov} + K R_S V_{ov}^2$, a quadratic in V_{ov}:

$V_{GS(th)}$	V_{ov}	I_D	V_{GS}	$V_{DS} = 12 - I_D(R_S + R_D)$
0.8 V (min)	0.223 V	4.98 mA	1.02 V	2.05 V
2.1 V (typ)	0.193 V	3.71 mA	2.29 V	4.59 V
3.0 V (max)	0.168 V	2.83 mA	3.17 V	6.34 V

Over the full datasheet threshold range the current spans 1.76:1 instead of 0-to-289 mA, and at every corner V_{DS} exceeds V_{ov} by more than 1.8 V, so the device stays in saturation. Notice that the overdrive is only about 0.2 V: with this large K the device needs very little overdrive for milliamperes, so almost all of $V_G - V_{th}$ appears across R_S, which is exactly what makes the bias robust. The price is signal gain, recovered by bypassing R_S with a capacitor at signal frequencies (Ch. 7). Figure 4.4 shows the construction graphically.

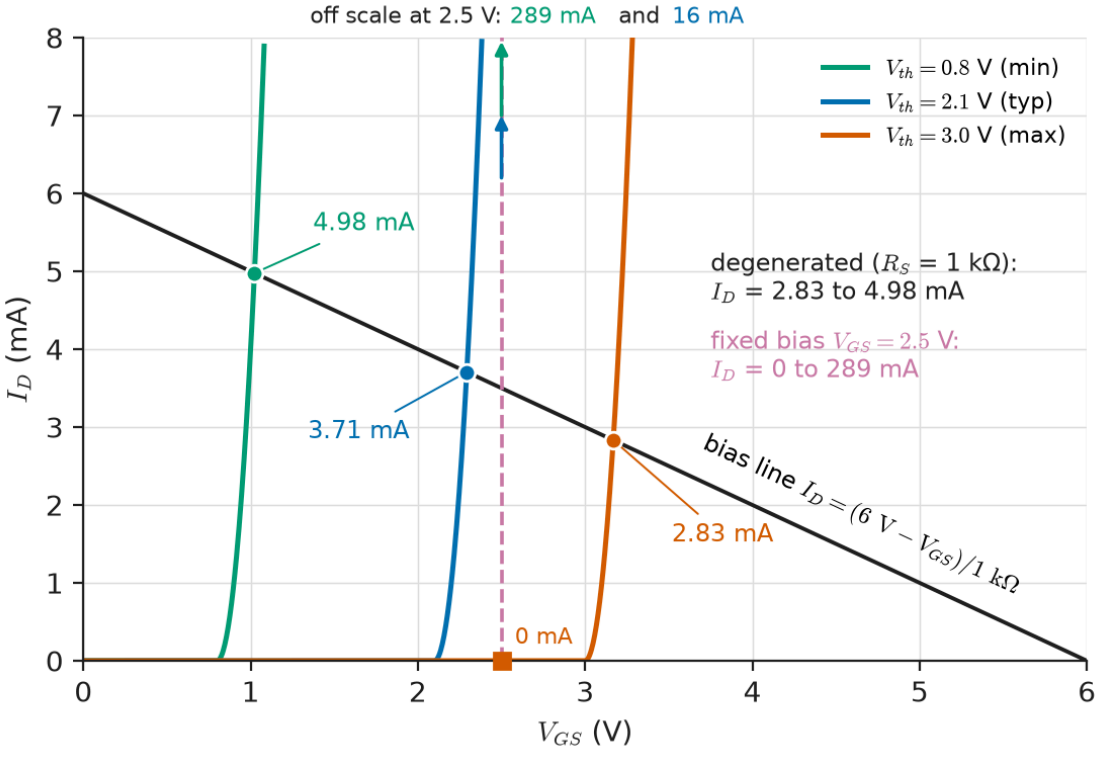


Figure 4.4 — Bias point of a source-degenerated MOSFET stage for minimum, typical and maximum threshold voltage, compared with fixed gate bias.

4.7 Small-signal models

The hybrid-π model of the MOSFET

Around a bias point in saturation, the MOSFET is linearised exactly as the diode and BJT are (Chapter 1 and Chapter 3). The low-frequency model has a voltage-controlled current source $g_m v_{gs}$, an output resistance r_o, and, when the source is not at the body potential, a second source $g_{mb} v_{bs}$ for the body effect. The gate draws no current at low frequency: there is no r_π.

The transconductance in saturation, from the square law, has three equivalent forms,

$$g_m = k'_n \frac{W}{L} V_{ov} = \sqrt{2 k'_n \frac{W}{L} I_D} = \frac{2 I_D}{V_{ov}}$$

and the others follow:

$$r_o = \frac{1}{\lambda I_D} = \frac{V_A}{I_D}, \qquad g_{mb} = \chi g_m, \quad \chi = \frac{\gamma}{2\sqrt{2\phi_F + V_{SB}}} \approx 0.1\text{–}0.3$$

with $V_A = 1/\lambda$ the Early voltage, proportional to channel length (roughly 5–50 V per µm of L in current processes).

> **Status of the principle: MOSFET hybrid-π model**
> *Derived.* It is the first-order Taylor expansion of the large-signal drain-current law about the bias point, valid for signal amplitudes small compared with the overdrive (the square law makes the second harmonic $v_{gs}/(4V_{ov})$ of the fundamental, Section 4.8); the numerical values of g_m inherit the accuracy of whichever large-signal model was differentiated.

Capacitances and f_T

At high frequency the model adds the gate-source capacitance C_{gs} (about $\frac{2}{3} W L C_{ox}$ in saturation plus overlap), the gate-drain capacitance C_{gd} (overlap only in saturation, but multiplied by the stage gain through the Miller effect), and the drain-body and source-body junction capacitances C_{db}, C_{sb}. The **transition frequency**, at which the short-circuit current gain falls to one,

$$f_T = \frac{g_m}{2\pi\,(C_{gs} + C_{gd})} \approx \frac{3 \mu_n V_{ov}}{4\pi L^2} \ \text{(long channel)}, \qquad f_T \approx \frac{v_{sat}}{2\pi L} \ \text{(velocity-saturated)}$$

summarises the device's speed. It rises as $1/L^2$ for long channels but only as $1/L$ once velocity saturation sets in: about 300 GHz for a 22 nm FinFET, 0.1–few GHz for discrete small-signal and RF MOSFETs, and tens of megahertz for a power MOSFET whose input capacitance is nanofarads.

MOSFET versus BJT, and the g_m/I_D view

At the same bias current, a BJT has more transconductance: $g_m = I_C/V_T = 38.7$ mS at 1 mA, against $g_m = 2I_D/V_{ov} = 10$ mS for a MOSFET at $V_{ov} = 0.2$ V. The ratio g_m/I_D, the transconductance efficiency, is $1/V_T \approx 38.7$ V^{-1} for a BJT at any current, but for a MOSFET it depends on the inversion level: about $2/V_{ov}$ (5–15 V^{-1}) in strong inversion, approaching $1/(nV_T)$ (25–30 V^{-1}) in weak inversion, where the MOSFET behaves like a BJT with a poorer ideality. Modern analog design picks g_m/I_D first, trading efficiency (weak inversion) against speed (strong inversion, high f_T), and reads device widths from simulated lookup tables.

Table 4.4 — Representative small-signal parameters at $I_D = 1$ mA.

Device	g_m	r_o	$g_m r_o$	f_T
BJT, 2N3904 class ($V_A \approx 100$ V)	38.7 mS	100 kΩ	≈ 3900	300 MHz
n-JFET, 2N5457 class	2–4 mS	50–200 kΩ	100–800	50–100 MHz
NMOS, 180 nm, $L = 0.18$ µm, $V_{ov} = 0.2$ V	10 mS	5–10 kΩ	50–100	20–40 GHz
NMOS, 180 nm, $L = 1$ µm, $V_{ov} = 0.2$ V	10 mS	50 kΩ	≈ 500	≈ 1.5 GHz
Discrete 2N7000 (near weak inversion)	15–25 mS	50–200 kΩ	> 1000	≈ 100 MHz

4.8 MOSFET amplification

The transistor-amplifier configurations, with their gain, impedance and frequency-response analysis, belong to Chapter 7, and the differential pairs, mirrors and active loads of integrated amplifiers to Chapter 10. This section records only what the device itself contributes.

The intrinsic gain

A common-source stage with drain load R_D has small-signal voltage gain

$$A_v = -g_m \left(R_D \parallel r_o\right)$$

Its upper bound, reached with an ideal current-source load, is the **intrinsic gain** $A_i = g_m r_o = 2V_A/V_{ov}$. It is a property of the device and bias alone, and it is low in short-channel MOSFETs: 10–30 for a minimum-length device in a 28 nm process, against thousands for a BJT. It explains the cascode (which multiplies output resistance by roughly another $g_m r_o$), non-minimum channel lengths in analog blocks, and digital calibration in fine processes.

Worked example — Gain of a common-source stage. An NMOS device biased at $I_D = 1$ mA with $V_{ov} = 0.2$ V and $\lambda = 0.02$ V^{-1} has $g_m = 2(1\text{ mA})/0.2\text{ V} = 10$ mS and $r_o = 1/(0.02 \times 1\text{ mA}) = 50$ kΩ, so its intrinsic gain is 500. With $R_D = 5$ kΩ the gain is $-10\text{ mS} \times (5 \parallel 50)\text{ k}\Omega = -10 \times 4.545 = -45.5$; loaded by a 10 kΩ input of the next stage, $R_D \parallel r_o \parallel R_L = 3.125$ kΩ and $A_v = -31.3$. A BJT at the same current would give $-38.7\text{ mS} \times 3.125\text{ k}\Omega = -121$ in the same circuit. The MOSFET's compensation is its infinite low-frequency input resistance, which lets it be driven by high-impedance sources and AC-coupled with small capacitors.

Followers, common gate and linearity

The source follower's gain, $g_m R_S/(1 + (g_m + g_{mb})R_S)$, stays below one even for $R_S \to \infty$ because of the body effect, and its output resistance is about $1/g_m$, 100 Ω at 10 mS. The square law also makes distortion predictable: a common-source stage driven by $v_{gs} = V\cos\omega t$ produces a second harmonic of relative amplitude $V/(4V_{ov})$, so keeping second-harmonic distortion below 1 % requires $V < 0.04 V_{ov}$ in an open-loop stage, or feedback (Ch. 7 and Ch. 9).

4.9 MOSFET switching

The MOSFET as a switch

As a switch the MOSFET operates in two states: cut-off, with leakage I_{DSS} (nanoamperes to microamperes, note the clash with the JFET's I_{DSS}), and deep triode, as a resistance $R_{DS(on)}$. It needs no steady gate current, which makes it the universal switch of logic (Ch. 33), load switching and power conversion. In the on state it is bidirectional: current can flow from source to drain through the channel, with a drop $I_D R_{DS(on)}$ smaller than the body-diode drop, which is the basis of synchronous rectification.

Three switch arrangements cover most designs:

- **Low-side switch.** An n-channel MOSFET between load and ground, gate driven relative to ground. The simplest and cheapest; used for solenoids, relays, LEDs and motors from a microcontroller.
- **High-side switch.** The switch is in the supply line of a grounded load. A p-channel MOSFET is driven by pulling its gate below the supply; an n-channel device needs a gate above the supply, from a charge pump or bootstrap (Ch. 29). Automotive loads are high-side switched, so that a wiring short to chassis does not energise them, by smart switches (BTS, VNQ families) with built-in current limit and diagnostics.
- **Load switch with reverse blocking.** Two MOSFETs source-to-source, so that their body diodes oppose, block in both directions: battery selection, USB power-path switching, analog multiplexers.

With an inductive load (solenoid, relay coil, motor) the switch must not be the only path for the load current at turn-off: the inductance drives the drain voltage up until something conducts. A freewheeling diode across the load (Chapter 2), a TVS or Zener clamp from drain to gate or drain to source, or the MOSFET's own avalanche capability (Section 4.10) must absorb the stored energy $\frac{1}{2}LI^2$.

Gate charge and switching time

The MOSFET's gate is a nonlinear capacitor. Rather than give C_{gs} and C_{gd}, which vary strongly with voltage, power-MOSFET datasheets give the **gate-charge curve** (Figure 4.5), measured by driving the gate with a constant current while the device switches a clamped inductive load. It has three segments:

1. **Charging** C_{gs} up to the threshold and then to the **Miller plateau** voltage $V_{plateau}$, at which the device carries the full load current; the charge used is Q_{gs}.
2. **The Miller plateau.** The drain voltage falls from the supply to nearly zero; the gate voltage stays flat at $V_{plateau}$ while the driver supplies the charge Q_{gd} to discharge the gate-drain capacitance. This is when the device dissipates most of its switching energy.
3. **Overdrive** from $V_{plateau}$ to the final drive voltage, which completes the total gate charge Q_g and brings $R_{DS(on)}$ down to its rated value.

The plateau voltage is the gate voltage needed for the load current: $V_{plateau} \approx V_{th} + \sqrt{I_D/K}$. The drain-voltage transition time is then set by the gate current during the plateau:

$$t_v \approx \frac{Q_{gd}}{I_G}, \qquad I_G = \frac{V_{drive} - V_{plateau}}{R_{G,total}}$$

> **Status of the principle: Gate-charge switching estimate**
> *Approximate.* It treats the driver as a voltage source behind a resistance and the gate-drain charge as delivered at constant gate voltage; it ignores source inductance (which subtracts $L_S\,di/dt$ from the effective drive) and the voltage dependence of C_{gd} within the plateau, and is good to about a factor of 1.5 for estimating edge times.

Worked example — Driving a logic-level MOSFET from a 3.3 V MCU pin. A logic-level 30 V MOSFET with $Q_{gs} = 1.5$ nC, $Q_{gd} = 2.0$ nC and $V_{plateau} = 2.2$ V at the 5 A load current is driven from a 3.3 V GPIO pin with an output resistance of about 50 Ω through a 22 Ω series gate resistor. During the plateau the gate current is $I_G = (3.3 - 2.2)/72\ \Omega = 15.3$ mA, and the drain-voltage edge takes $t_v = 2.0\,\text{nC}/15.3\,\text{mA} = 131$ ns. That is fine for a solenoid switched a few times per second and far too slow for a 200 kHz converter.

Two warnings follow. First, the pin's 1.1 V of overdrive beyond the plateau is all there is: if a hot part's plateau rises toward 3 V (higher current, higher-threshold sample), the gate current collapses and the device lingers in the plateau. Second, the opening's IRF540N, with a threshold up to 4 V, would have its plateau *above* 3.3 V at 5 A: the drain voltage would never fall fully, and the device would sit in its saturation region dissipating watts. The fix is a part whose $R_{DS(on)}$ is guaranteed at 2.5 V, or a gate driver (an MCP1416-class driver delivers 1.5 A peak and 12 V of drive); the driver design, the loss calculation and the switching-frequency trade-off belong to Chapter 29.

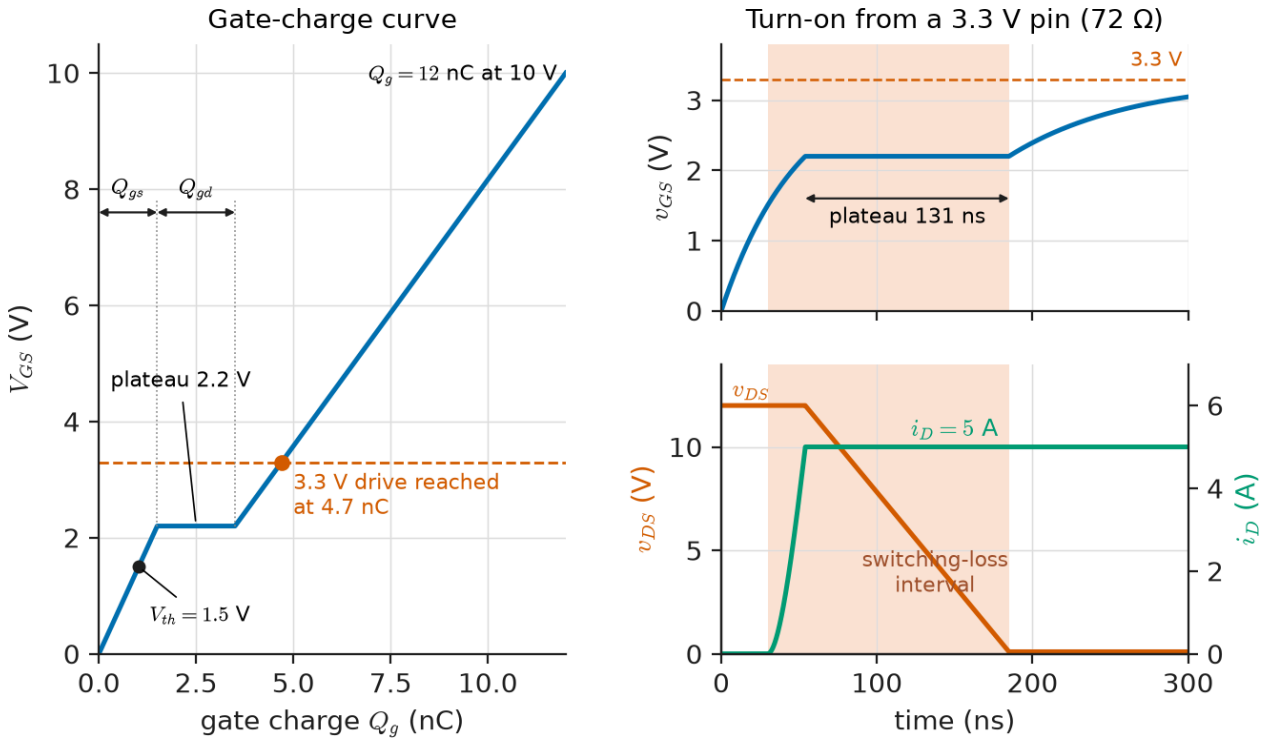


Figure 4.5 — Gate-charge curve of a power MOSFET and the corresponding turn-on waveforms in a clamped inductive circuit.

4.10 Power MOSFET fundamentals

Vertical structures

A planar MOSFET carries current along the surface, and its blocking voltage needs a long drift region in the same plane, wasting area. Power MOSFETs are **vertical**: source and gate on the top surface, drain on the back of the die, current flowing down through a lightly doped n– **drift region** that supports the blocking voltage (Figure 4.6). Thousands to millions of cells are paralleled on one die. Three generations dominate:

- **Planar DMOS (VDMOS)**: the channel is formed laterally at the surface in a double-diffused p-body, then current turns down into the drift region. Robust, used at 200–1000 V where the drift region dominates anyway.
- **Trench MOSFET**: the gate is buried in etched trenches and the channel runs vertically along the trench wall, eliminating the JFET region between cells and packing far more channel per area. Dominant below 250 V: a 40 V trench MOSFET in a 5 × 6 mm package reaches 1–2 mΩ.
- **Superjunction (SJ) MOSFET**: alternating n and p pillars in the drift region compensate each other's charge, so the drift region can be doped an order of magnitude more heavily for the same blocking voltage. Dominant at 500–900 V in silicon (CoolMOS, MDmesh, SuperFET families), with specific on-resistances near 1 Ω·mm² at 600 V.

On-resistance and the silicon limit

$R_{DS(on)}$ is a series sum: source and package resistance, channel, the JFET neck between cells (in planar devices), drift region and substrate. At low voltage the channel and package dominate; at high voltage the drift region dominates. For a conventional (non-superjunction) unipolar drift region the specific on-resistance rises steeply with blocking voltage: as V_B^2 for a constant critical field, as derived with the material figure of merit in *Materials Science and Engineering*, Ch. 33 §33.10, and closer to $V_B^{2.5}$ in silicon once the critical field's fall at the light doping of high-voltage drift regions is included. The consequence for selection is direct: a 600 V planar MOSFET has 100–300 times the specific on-resistance of a 60 V

device. Superjunctions break the one-dimensional limit in silicon; wide-bandgap materials lower it by orders of magnitude (Chapter 5).

Table 4.5 — Representative silicon power MOSFETs (typical datasheet values).

Class	$V_{(BR)DSS}$	$R_{DS(on)}$ at 10 V, 25 °C	Q_g	$R_{\theta JC}$	Typical use
30 V trench, logic level, 3 × 3 mm	30 V	3–6 mΩ	10–20 nC	2–4 K/W	point-of-load buck, laptop power
40 V trench, 5 × 6 mm	40 V	1–2 mΩ	50–100 nC	0.5–1 K/W	12 V motor drives, battery switches
100 V trench, TO-220 / D²PAK	100 V	3–8 mΩ	70–150 nC	0.4–0.8 K/W	48 V telecom and server, e-bike controllers
IRF540N (planar, legacy)	100 V	44 mΩ max	71 nC max	1.15 K/W	general purpose
650 V superjunction, TO-247	650 V	40–190 mΩ	30–110 nC	0.3–1 K/W	PFC, LLC converters, server PSUs

Temperature coefficient and paralleling

The on-resistance of a silicon MOSFET rises with temperature, because mobility falls: typically by a factor of 1.5–1.8 from 25 °C to 150 °C at rated gate drive, read from the normalised $R_{DS(on)}$ curve on the datasheet. The positive coefficient makes paralleled MOSFETs share current in the fully-on state: the hotter device's resistance rises and pushes current to its cooler neighbours. It also closes the electrothermal loop of Section 1.7: dissipation raises T_j, which raises $R_{DS(on)}$, which raises dissipation. With $R_{DS(on)} = R_{25}\,[1 + \alpha(T_j - 25)]$ and junction-to-ambient thermal resistance $R_{\theta JA}$ (the series network of *Engineering Physics* Ch. 14 §14.7), the steady state is

$$T_j = \frac{T_a + I^2 R_{25}(1 - 25\alpha) R_{\theta JA}}{1 - G}, \qquad G = I^2 R_{25}\,\alpha\, R_{\theta JA}$$

The electrothermal loop gain G must be less than one for an equilibrium to exist; as $G \to 1$ the temperature diverges, which is thermal runaway.

> **Status of the principle: Electrothermal equilibrium of a conducting MOSFET**
> *Derived.* It combines the linear temperature model of $R_{DS(on)}$ with a single thermal resistance; it is accurate for steady DC conduction, while pulsed loads need the transient thermal impedance $Z_{\theta JC}(t)$ from the datasheet, and the linear coefficient α is a fit valid between about 25 °C and 175 °C.

Worked example — A 48 V battery disconnect switch. An 80 V trench MOSFET in a battery-management unit for a 48 V e-bike pack (54.6 V at full charge) has $R_{25} = 4.0$ mΩ, rising 1.7 times by 150 °C ($\alpha = 0.7/125 = 0.0056$ K^{-1}). Mounted on a copper pour, it sees $R_{\theta JA} = 3.3$ K/W in a 60 °C enclosure. Listing 4.1 solves the equilibrium for several currents.

```
 # Junction temperature of a conducting power MOSFET whose R_DS(on) rises with T_j
R25, alpha = 4.0e-3, 0.0056   # ohm at 25 C; per kelvin (1.7x at 150 C)
Rth_ja, Ta = 3.3, 60.0        # K/W junction to ambient; ambient in C
Tj_max = 175.0

def operating_point(I):
    G = I**2 * R25 * alpha * Rth_ja          # electrothermal loop gain
    if G >= 1.0:
        return G, float("inf"), float("inf")  # no equilibrium: runaway
    Tj = (Ta + I**2 * R25 * (1 - 25*alpha) * Rth_ja) / (1 - G)
    P = (Tj - Ta) / Rth_ja
    return G, P, Tj

for I in (20, 30, 40, 50, 60, 70):
    G, P, Tj = operating_point(I)
    flag = "OK" if Tj <= Tj_max else "exceeds Tj,max"
    print(f"I = {I:2d} A   G = {G:4.2f}   P = {P:5.2f} W   Tj = {Tj:6.1f} C  {flag}")
print(f"runaway current (G = 1): {(1/(R25*alpha*Rth_ja))**0.5:.0f} A")
```

Its output:

```
 I = 20 A   G = 0.03   P =  1.97 W   Tj =   66.5 C  OK
I = 30 A   G = 0.07   P =  4.61 W   Tj =   75.2 C  OK
I = 40 A   G = 0.12   P =  8.68 W   Tj =   88.6 C  OK
I = 50 A   G = 0.18   P = 14.67 W   Tj =  108.4 C  OK
I = 60 A   G = 0.27   P = 23.47 W   Tj =  137.4 C  OK
I = 70 A   G = 0.36   P = 36.75 W   Tj =  181.3 C  exceeds Tj,max
runaway current (G = 1): 116 A
```

At the pack's 30 A continuous rating the junction runs at 75 °C with 4.6 W, a comfortable margin. Without the temperature coefficient, the 30 A estimate would have been $30^2 \times 4\ \text{m}\Omega = 3.6$ W: 22 % low. The 175 °C limit is reached at about 69 A, so the useful continuous rating is set by $T_{j,max}$, well before the 116 A at which the loop gain reaches one; note how the dissipation grows faster than I^2 as the current approaches that limit. For headroom, a designer would parallel two devices, which share current thanks to the positive coefficient.

Ratings, safe operating area and avalanche

The front page of a power-MOSFET datasheet is a list of absolute maximum ratings, defined in IEC 60747-8 and IEC 60134 as limits never to be exceeded under any condition:

- V_{DS} (equal to the guaranteed $V_{(BR)DSS}$ at 25 °C, rising about 0.1 %/K) and V_{GS} (typically ±20 V).
- I_D continuous, usually quoted at a case temperature of 25 °C and computed from $R_{\theta JC}$, a number no real heatsink achieves; the value at $T_C = 100$ °C is more honest, and the package lead rating (often 75–120 A for a TO-220) can be lower still.
- I_{DM} pulsed, and P_D at $T_C = 25$ °C.
- T_j from −55 °C to 150 °C or 175 °C.
- E_{AS}, the single-pulse **avalanche energy**: the energy the device can absorb in breakdown when an unclamped inductive load turns off (UIS). A device rated $E_{AS} = 200$ mJ can absorb the energy of 20 A in a 1 mH inductance once, starting from 25 °C; repetitive avalanche is rated separately, if at all.

The **safe operating area** (SOA) chart bounds I_D versus V_{DS} on log axes for single pulses of 10 µs to DC at $T_C = 25$ °C: a horizontal line at I_{DM}, a sloped line where $R_{DS(on)}$ limits current, constant-power lines set by the transient thermal impedance, and a vertical line at $V_{DS(max)}$. Modern trench MOSFETs add a steeper boundary at long pulse widths, the **Spirito** or linear-mode instability: in linear operation (hot-swap inrush limiting, eFuses, electronic loads) the device sits below its temperature-compensation point, so a hot spot draws more current and gets hotter. Hot-swap controllers in

servers and telecom backplanes are therefore paired with MOSFETs whose SOA is guaranteed for linear mode, not with the lowest-$R_{DS(on)}$ part.

The body diode carries its own ratings: forward voltage (0.7–1.2 V), continuous and pulsed current, and the reverse-recovery charge Q_{rr} and time t_{rr}. In superjunction MOSFETs Q_{rr} is large (several microcoulombs), which rules out using their body diodes in hard-switched bridges; this, and the remedies (fast-recovery body-diode variants, SiC MOSFETs, GaN HEMTs), are taken up in Chapter 5 for the devices and in Chapter 29 for the circuits.

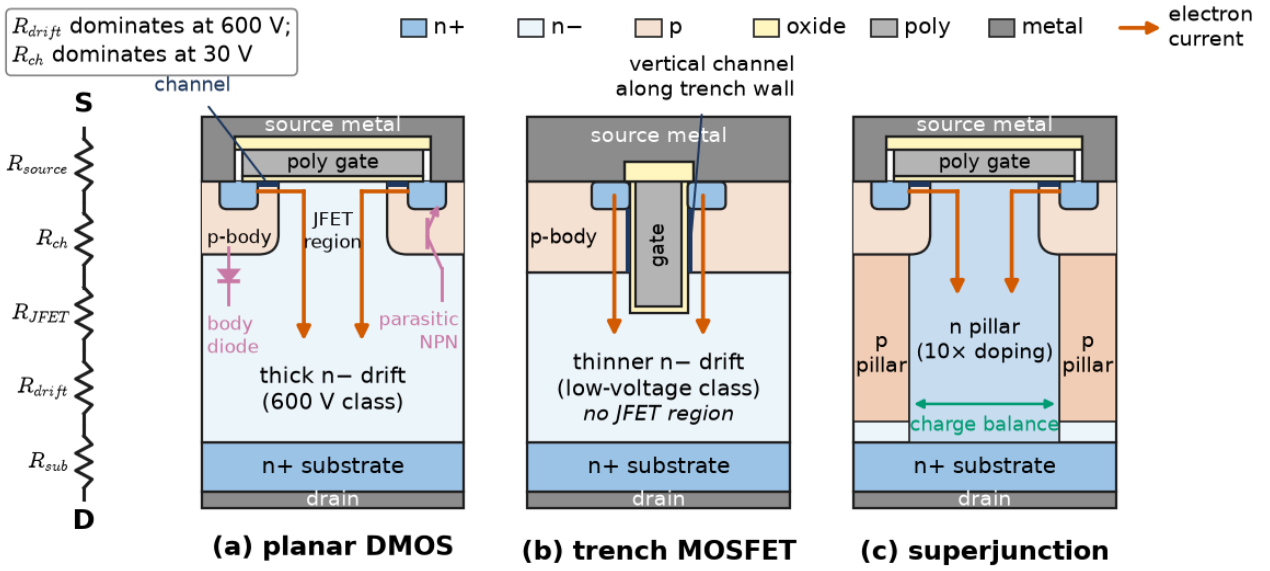


Figure 4.6 — Cross-sections of planar DMOS, trench and superjunction power MOSFET cells, with the components of on-resistance.

The parasitic NPN transistor formed by source, body and drift region (Figure 4.6) is held off by the source metal shorting body to source; under extreme dv/dt or avalanche current the lateral body resistance can let it turn on, and the device fails in second breakdown. Ruggedness ratings (E_{AS}, body-diode dv/dt) measure how well it is suppressed; Chapter 5 meets the same parasite as the latch-up mechanism of the IGBT.

What remains open

Open question	Where it is answered
How are thyristors, IGBTs, SiC MOSFETs and GaN HEMTs built, rated and chosen?	Chapter 5
How are common-source, follower and multistage FET amplifiers designed and analysed in frequency?	Chapter 7
How do current mirrors, differential pairs and cascodes exploit matched MOSFETs in an IC?	Chapter 10
How large are switching and conduction losses, and how is a power MOSFET's gate driven and protected?	Chapter 29
How do MOSFETs become CMOS gates, with delay, noise margins and dynamic power?	Chapter 33
What material properties set the unipolar limit and make SiC and GaN better?	*Materials Science and Engineering*, Ch. 33
How is heat removed from a junction through a package and heatsink?	*Engineering Physics*, Ch. 14 §14.7; Chapter 29 for converter thermal design

Next: Chapter 5 moves beyond the three classical transistors to the four-layer thyristor family, the IGBT and the wide-bandgap, optoelectronic and compound-semiconductor devices, and to the ratings by which any of them is selected.

Chapter 5 — Advanced Semiconductor Devices

Part I · Electronic Devices

An HVAC manufacturer replaces the relay that starts a 600 W single-phase fan motor with a triac, to remove contact wear and switching noise. The prototypes pass every test. In production, about one unit in fifty keeps the motor running after the controller removes the gate drive: the triac does not turn off. Nothing is broken. The motor draws 6 A at a power factor of 0.6, so when its current passes through zero the mains voltage is already at 260 V, and that voltage reappears across the triac within microseconds. The charge still stored in the triac from the half-cycle just ended is enough to turn it back on. The cure is a different class of triac, rated for a higher commutating current slope, or a snubber, and it comes from reading a line of the datasheet that relay designers never needed. The same factory's drives are moving from silicon IGBTs to SiC MOSFETs, and its control boards rely on optocouplers whose gain fades over a decade. Each of these devices extends the diode, BJT and MOSFET of Chapters 2–4 by adding a layer, a material or a photon, and each has limits of its own. This chapter covers how they work, how they are rated, and how one is chosen.

> **How do four-layer devices latch, and what makes them turn off or fail to?**
> **What does an IGBT, a SiC MOSFET or a GaN HEMT offer that a silicon MOSFET does not, and at what price? How are optoelectronic and compound-semiconductor devices specified, and how is any power device selected against its ratings?**

Concept	Engineering destination
Thyristors	Phase-controlled rectifiers, HVDC valves, crowbars, soft starters
SCRs	Battery chargers, controlled rectifiers, motor soft starters, overvoltage crowbars
TRIACs	AC switches for heaters, fans, pumps and lamps in appliances and HVAC
DIACs	Trigger devices for dimmers and phase controllers
IGBTs	Industrial drives, traction, induction cooking, welding, wind and solar inverters
Wide-bandgap power devices	EV inverters and chargers, data-centre and telecom power supplies, compact adapters
Optoelectronic devices	Galvanic isolation, gate-drive isolation, LiDAR, optical links, imaging
Compound-semiconductor devices	RF power amplifiers, low-noise receivers, radar, satellite and 5G front ends
Device selection and ratings	Choosing voltage class, current rating and technology; derating for life and cosmic rays

Roadmap. The four-layer structure and latching (5.1) → its three members: the SCR (5.2), the bidirectional TRIAC (5.3) and its trigger, the DIAC (5.4) → the MOS-gated bipolar IGBT (5.5) → silicon carbide and gallium nitride power devices (5.6) → devices that convert between photons and electrons beyond Chapter 2's LEDs and photodiodes (5.7) → III–V transistors for high frequency (5.8) → ratings and selection across all of them (5.9).

5.1 Thyristors

Where *Engineering Physics* leaves off

Engineering Physics carries the solid state as far as the PN junction, the BJT and the MOSFET in one section (Ch. 18 §18.3) and tabulates the band gaps of Si, GaAs, InP, SiC and GaN with their direct or indirect character (Ch. 6 §6.6.3); semiconductor lasers are sketched in Ch. 12 §12.8.3, and junction temperature from a thermal-resistance network in Ch. 14 §14.7. The material properties that make silicon carbide and gallium nitride attractive (critical field, saturation velocity, thermal conductivity, the polarization-induced two-dimensional electron gas and the Baliga figure of merit) are

developed in *Materials Science and Engineering*, Ch. 33 §33.4–33.5 and §33.10. None of these sources treats four-layer devices, MOS-gated bipolar devices or the ratings of real power, optoelectronic and RF parts. This chapter starts there. It uses the BJT of Chapter 3 to explain latching, the MOSFET of Chapter 4 to explain the IGBT, and the materials results above, without re-deriving them, to explain what wide-bandgap devices do in a circuit.

The four-layer structure

A thyristor is a PNPN stack with three junctions, J1 (anode P to N1), J2 (N1 to P2) and J3 (P2 to cathode N), and a gate contact on the P2 layer (Figure 5.1). With the anode positive, J1 and J3 are forward-biased and J2 blocks: the device is in **forward blocking**. With the anode negative, J1 and J3 block (**reverse blocking**). The lightly doped N1 base, tens to hundreds of micrometres thick, supports the voltage in both directions, which is why a thyristor, unlike a MOSFET, can block thousands of volts in either polarity.

The key to its behaviour is the **two-transistor model**: the stack is a PNP transistor (P–N1–P2) and an NPN transistor (N1–P2–N) sharing two layers, each transistor's collector driving the other's base. If α_1 and α_2 are their common-base current gains and I_{CO} the leakage of J2, the anode current is

$$I_A = \frac{\alpha_2 I_G + I_{CO}}{1 - (\alpha_1 + \alpha_2)}$$

> **Status of the principle: Thyristor latching condition**
> *Derived.* It follows from current continuity in the two-transistor model with each transistor described by its common-base gain; it predicts latching when $\alpha_1 + \alpha_2 \to 1$ and is qualitative rather than quantitative because both gains vary strongly with current, temperature and the lateral spread of conduction across the die.

At low current both gains are small (recombination dominates at low injection), the denominator is close to one and the device blocks. As the current rises, the gains rise; when $\alpha_1 + \alpha_2$ reaches one, the loop is regenerative and the current is limited only by the external circuit. Both transistors saturate, J2 becomes forward-biased, and the whole stack behaves like a single PIN diode with a forward drop of 1–2 V at rated current. The device has **latched**: removing the gate current changes nothing.

Anything that raises the current through J2 can trigger the loop: gate current (the intended method); forward voltage reaching the **breakover voltage** V_{BO}, where J2 avalanches; a fast-rising anode voltage, whose displacement current $C_{J2}\,dv/dt$ acts as gate current, so the datasheet's critical dv/dt (50–1000 V/µs) must exceed what the circuit applies; temperature, which raises leakage and gains together; and light, in the light-triggered thyristors of HVDC valves. Only the first is wanted; the circuit must keep the others away.

Holding, latching and turn-off

A thyristor turns off only when its anode current falls below the **holding current** I_H (milliamperes to a few hundred milliamperes) long enough for the stored charge in the bases to recombine. Right after triggering, the current must exceed the somewhat larger **latching current** I_L before the gate pulse ends, otherwise the device drops out again; a gate pulse that is too short into an inductive load, whose current rises slowly, is the classic cause of erratic firing. After the current reaches zero, the device needs the **circuit-commutated turn-off time** t_q (10–30 µs for fast inverter-grade thyristors, 100–500 µs for phase-control types) under reverse bias before forward voltage can be reapplied without retriggering.

Latching gives the thyristor the lowest forward drop per unit area of any high-voltage switch, no sustained gate drive and an enormous surge rating; it also means the gate cannot turn it off. The family (Table 5.1) answers that defect and the need for bidirectional conduction.

Table 5.1 — The thyristor family (representative ratings).

Device	Conducts	Turned off by	Typical ratings	Main uses
SCR (phase control)	one way	current zero (line commutation)	0.4–8.5 kV, 1 A–6 kA	controlled rectifiers, soft starters, HVDC
TRIAC	both ways	current zero	400–1000 V, 0.8–40 A RMS	appliance and HVAC AC switching, dimmers
DIAC	both ways (two-terminal)	current below holding	V_{BO} 28–45 V	trigger for TRIACs
GTO thyristor	one way	negative gate current (a third to a fifth of anode current)	2.5–6 kV, 1–6 kA	largely replaced by IGBT and IGCT
IGCT	one way	hard-driven gate (whole anode current diverted)	4.5–6.5 kV, 2–6 kA	medium-voltage drives, interties
Light-triggered thyristor	one way	current zero	8 kV, 3–5 kA	HVDC valves, static VAR compensators

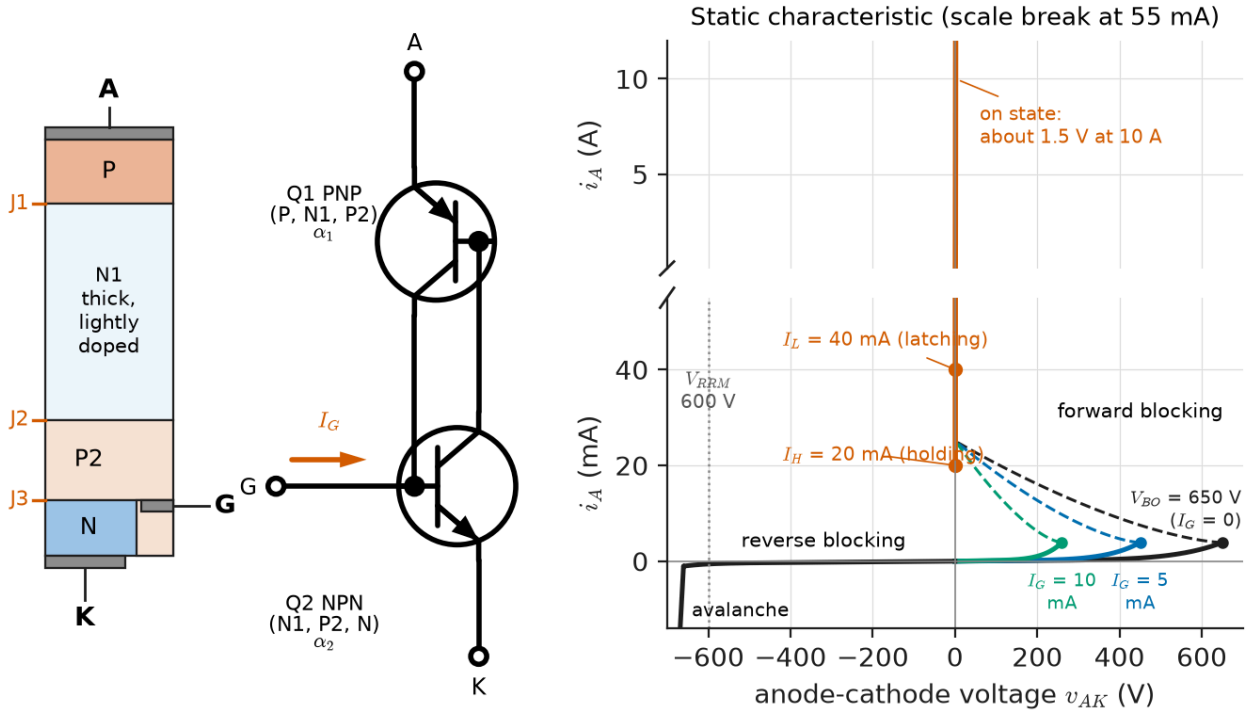


Figure 5.1 — Thyristor PNPN structure, its two-transistor model, and the static current-voltage characteristic.

5.2 SCRs

Characteristics and gate

The silicon-controlled rectifier (SCR) is the basic thyristor, and the name is often used for the whole family. Its static characteristic (Figure 5.1) has three branches: reverse blocking up to the repetitive peak reverse voltage V_{RRM}, forward blocking up to the repetitive peak off-state voltage V_{DRM}, and the on-state branch reached by gate triggering. On the on-state branch it is modelled, like a power diode, by a threshold voltage and a slope resistance, $v_T = V_{T0} + r_T i_T$ (for a 12 A part, about 0.85 V and 30 mΩ).

The gate–cathode junction is a PN diode. The datasheet gives the **gate trigger current** I_{GT} and **voltage** V_{GT} that guarantee firing at 25 °C (for a 12 A device, I_{GT} = 5–15 mA at $V_{GT} \approx 1.3$ V; sensitive-gate types fire at 50–200 µA), rising

two to three times at −40 °C, together with the **non-trigger** voltage V_{GD} (about 0.2 V at maximum temperature) that the gate must stay below in the off state. Good gate-drive practice is a pulse of two to five times I_{GT} with a fast rise, held long enough for the anode current to pass I_L, and a resistor (often 1 kΩ) from gate to cathode to divert noise and dv/dt current.

Ratings

An SCR's ratings, defined in IEC 60747-6, follow from its role as a line-commutated rectifier:

- **Average and RMS on-state current**, $I_{T(AV)}$ and $I_{T(RMS)}$. The average rating is quoted for half-sine conduction (180°) at a stated case temperature; the RMS rating is set by the internal leads and bond wires and holds at any waveform.
- **Surge current** I_{TSM} for a single 10 ms (50 Hz) or 8.3 ms (60 Hz) half-sine, typically 10–15 times $I_{T(AV)}$, and the **fusing integral** I^2t for coordination with a fuse: for a 10 ms half-sine, $I^2t = I_{TSM}^2 \times 0.005$ s.
- **Critical** di/dt at turn-on (50–200 A/µs). Conduction starts in a small region near the gate and spreads across the cathode at about 0.1 mm/µs; a current that rises faster than the conducting area concentrates its dissipation and burns a hole near the gate. Series inductance, or a hard gate pulse that turns on a larger area at once, keeps di/dt within the rating.
- **Critical** dv/dt, t_q, and the voltage ratings above.

> **Status of the principle: Average current rating versus conduction angle**
> *Derived.* The RMS-to-average ratio (form factor) of a phase-controlled half-sine follows from integrating the waveform; it is exact for sinusoidal supply and a resistive load and is the basis of every datasheet derating curve of $I_{T(AV)}$ versus conduction angle.

Worked example — Rating an SCR for phase control. A battery charger uses a TYN612-class SCR: $I_{T(RMS)} = 12$ A, $I_{T(AV)} = 8$ A, $I_{TSM} = 140$ A for 10 ms. For a half-sine of peak I_m conducting from firing angle α to π, the average and RMS currents are

$$I_{AV} = \frac{I_m(1+\cos\alpha)}{2\pi}, \qquad I_{RMS} = I_m\sqrt{\frac{\pi - \alpha + \tfrac{1}{2}\sin 2\alpha}{4\pi}}$$

At $\alpha = 0$ the form factor I_{RMS}/I_{AV} is $\pi/2 = 1.571$, so the 8 A average rating corresponds to 12.6 A RMS, slightly above the 12 A RMS limit; the RMS rating binds, and the usable average is $12/1.571 = 7.64$ A. At $\alpha = 120^\circ$ (a deeply phased-back charger near the end of charge) the form factor is 2.78 and the usable average falls to $12/2.78 = 4.32$ A, with a peak of 54 A. Narrow conduction means high peaks for a given average, so an SCR in phase control must be derated by conduction angle, exactly as the datasheet curves show. The I^2t of the device is $140^2 \times 0.005 = 98$ A^{2}s; the protecting fuse must clear a short circuit with a total I^2t below this, for example a 16 A semiconductor fuse ("aR" class) with a clearing I^2t of 60 A²s or less at the working voltage.

SCRs remain the choice wherever turn-off at the current zero is acceptable: controlled rectifiers and chargers (Chapter 31), soft starters, overvoltage crowbars, and HVDC valves stacking a hundred or more 8 kV thyristors.

5.3 TRIACs

Structure and quadrants

A TRIAC (triode for alternating current) is in effect two thyristors in antiparallel on one die, with a single gate. Its main terminals are MT1 (the reference, next to the gate) and MT2. It conducts in both directions and can be triggered by either polarity of gate current, which gives four **triggering quadrants**, defined by the polarity of MT2 and of the gate, both with respect to MT1:

Quadrant	MT2	Gate	Sensitivity
I (1+)	+	+	best
II (1−)	+	−	good
III (3−)	−	−	good
IV (3+)	−	+	poor: needs two to three times I_{GT}

Because quadrant IV is insensitive and slow, modern designs drive the gate negative (quadrants II and III), which works with a microcontroller whose supply rail is referenced to MT1, and **three-quadrant** triacs ("snubberless", "Hi-Com" or "alternistor" types) are not specified in quadrant IV at all. Removing that capability lets the manufacturer optimise the structure for commutation, the subject of the opening problem.

Commutation

With a resistive load, current and voltage pass through zero together and the triac turns off with only a gently rising voltage across it. With an inductive load, the current lags. At the current zero the supply voltage is already at $V_m \sin\varphi$, and as soon as the triac stops conducting that voltage appears across it, limited in rate only by stray capacitance. Meanwhile, charge stored during the half-cycle just ended is still being removed. If the current was falling quickly before its zero (high di/dt) the stored charge is larger; if the voltage then rises quickly (high dv/dt) the displacement current is larger. Together they can fire the half of the triac that is about to block. Datasheets therefore specify a **commutating** pair: the rate of fall of current $(dI/dt)_c$ in A/ms that the device survives while a given reapplied $(dV/dt)_c$ in V/µs appears.

Worked example — Commutation of a fan-motor triac. The opening's motor draws 6 A RMS at a power factor of 0.6 from 230 V, 50 Hz. Its current slope at the zero crossing is

$$\left(\frac{dI}{dt}\right)_c = \omega\sqrt{2}I_{RMS} = 314.2 \times 8.49 = 2.67\ \text{A/ms}$$

and the voltage that reappears at the current zero is $V_m \sin\varphi = 325 \times 0.8 = 260$ V, applied through the wiring and the triac's capacitance in a few microseconds. A standard four-quadrant 8 A triac rated for, say, 2–3 A/ms at a $(dV/dt)_c$ of 10 V/µs is marginal, and spread in stored charge between parts explains the one-in-fifty failures. Two cures exist. A three-quadrant (snubberless) 8–12 A triac, with commutating ratings typically of 5–20 A/ms at 20 V/µs or more, solves it at the device level. Alternatively an RC snubber across the triac (typically 100 Ω and 47–100 nF) slows the reapplied dv/dt, at the cost of a leakage current through the capacitor when the triac is off and some dissipation.

Applications and drivers

Triacs switch AC loads up to a few kilowatts (heaters, fans, pumps, appliance motors, lamp dimmers); their on-state drop of 1.0–1.5 V costs 1–1.5 W per ampere. Isolation from the control electronics comes from an **optotriac** driven by an infrared LED. Zero-crossing optotriacs (MOC3063 class) fire only near the voltage zero, removing inrush and interference with resistive loads, but are unsuitable for phase control and for transformers, whose magnetising inrush is worst when switched at a voltage zero; random-phase types (MOC3023 class) serve phase control. Both burst-fire and phase control must meet the flicker and harmonic limits of IEC 61000-3-3 and IEC 61000-3-2.

5.4 DIACs

A bidirectional trigger

A DIAC is a two-terminal, symmetrical trigger device: in effect a triac without a gate, or an NPN structure whose two junctions break over symmetrically. Below its **breakover voltage** V_{BO} (typically 28–36 V for the DB3 class, with a symmetry tolerance of ±3 V) it passes only microamperes. At V_{BO} it switches into a negative-resistance region and its

voltage drops by the **breakback** ΔV (at least 5 V, typically 10 V), dumping charge from a timing capacitor into the triac gate as a sharp current pulse of a few hundred milliamperes. Its symmetry makes the triac fire at the same angle in each half-cycle, which avoids a DC component in the load current, and its breakback produces a clean pulse whatever the slope of the voltage that charges it.

The phase-control dimmer

The classic DIAC application is the leading-edge dimmer (Figure 5.2): an RC network across the triac charges a capacitor through a potentiometer; when the capacitor voltage reaches V_{BO} the DIAC fires the triac, which conducts for the rest of the half-cycle. Larger resistance delays the capacitor voltage and reduces its amplitude, delaying firing. Treating the network as a lossless RC phase shifter driven by the full mains (valid before the triac fires, and ignoring the capacitor's partial discharge at each firing), the capacitor voltage lags the mains by $\arctan(\omega RC)$ with amplitude $V_m/\sqrt{1+(\omega RC)^2}$, and firing occurs where it reaches V_{BO}. Listing 5.1 computes the firing angle and the RMS voltage that a resistive load receives.

```
 # Firing angle of a DIAC-triggered triac dimmer (single RC timing network)
import math

Vm, f, C, Vbo = 230*math.sqrt(2), 50.0, 100e-9, 32.0   # DIAC breakover 32 V
w = 2*math.pi*f

def firing_angle(R):
    x = w*R*C
    Vc = Vm/math.sqrt(1 + x*x)            # capacitor voltage amplitude
    lag = math.degrees(math.atan(x))      # phase lag behind the mains
    if Vc <= Vbo:
        return Vc, None                   # never reaches breakover: lamp off
    return Vc, lag + math.degrees(math.asin(Vbo/Vc))

for R in (10e3, 33e3, 68e3, 100e3, 150e3, 220e3):
    Vc, a = firing_angle(R)
    shown = f"{a:6.1f} deg" if a is not None else "   off    "
    # fraction of full-wave RMS voltage delivered to a resistive load
    if a is not None:
        ar = math.radians(a)
        frac = math.sqrt((math.pi - ar + math.sin(2*ar)/2)/math.pi)
        shown += f"   Vload = {230*frac:5.1f} V rms"
    print(f"R = {R/1e3:5.0f} k   Vc,pk = {Vc:5.1f} V   alpha = {shown}")
```

```
 R =    10 k   Vc,pk = 310.3 V   alpha =   23.4 deg   Vload = 228.4 V rms
R =    33 k   Vc,pk = 225.8 V   alpha =   54.2 deg   Vload = 212.1 V rms
R =    68 k   Vc,pk = 137.9 V   alpha =   78.3 deg   Vload = 182.2 V rms
R =   100 k   Vc,pk =  98.7 V   alpha =   91.3 deg   Vload = 160.3 V rms
R =   150 k   Vc,pk =  67.5 V   alpha =  106.3 deg   Vload = 130.8 V rms
R =   220 k   Vc,pk =  46.6 V   alpha =  125.2 deg   Vload =  90.5 V rms
```

Worked example — The dimming range of a single RC network. With $C = 100$ nF and R from 10 kΩ to 220 kΩ, the firing angle moves from 23° to 125° and the lamp voltage from 228 V to 91 V RMS. At 100 kΩ, $\omega RC = \pi$, the capacitor voltage has an amplitude of only 98.7 V and lags by 72.3°, and firing occurs at $72.3^\circ + \arcsin(32/98.7) = 91.3^\circ$. A single RC network cannot push the angle much beyond about 130° before the capacitor amplitude falls below V_{BO} and the lamp abruptly goes out: the angle-versus-resistance curve flattens and then ends. It also shows **snap-on hysteresis**: once the triac fires, the partial discharge of C in each half-cycle shifts the next charging cycle, so the lamp turns on at a higher setting than it turns off. Commercial dimmers use a second RC stage to extend the range and reduce the hysteresis. Phase-controlled dimmers also generate strong odd harmonics and are incompatible with many LED

lamps, whose capacitive input stages draw a current spike at each firing edge; trailing-edge (MOSFET or IGBT) dimmers are the modern answer.

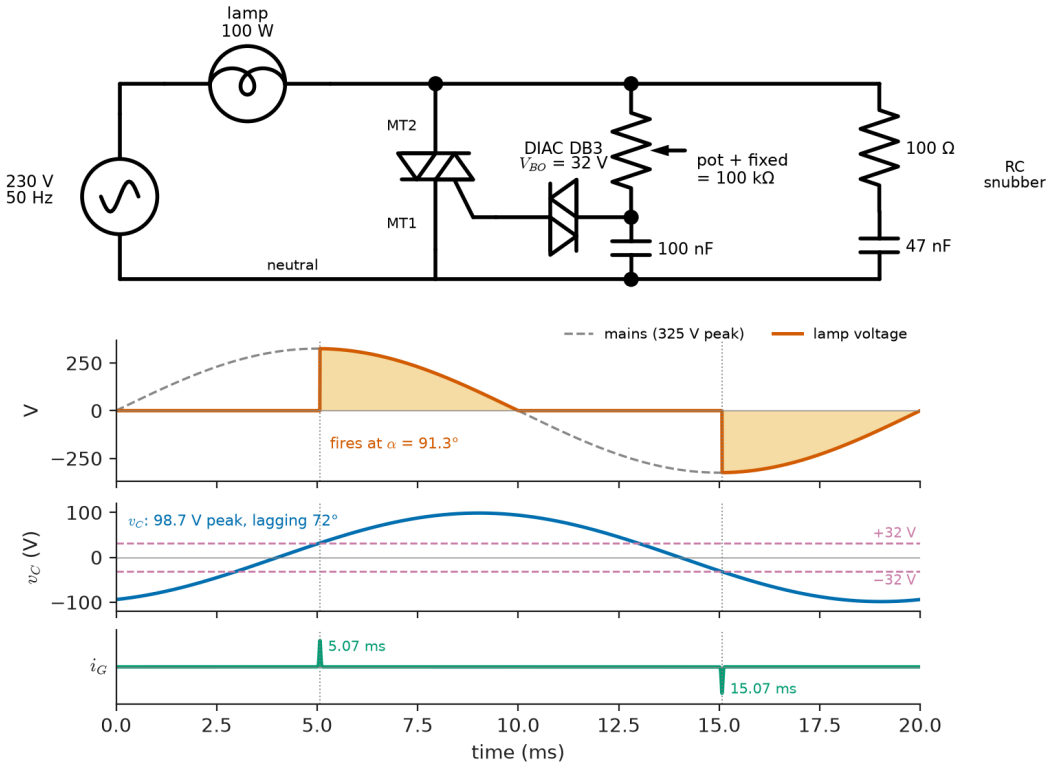


Figure 5.2 — Leading-edge DIAC-TRIAC dimmer: circuit and waveforms at a firing angle of 91 degrees.

5.5 IGBTs

Structure

The insulated-gate bipolar transistor (IGBT) combines a MOSFET's voltage-controlled gate with a bipolar transistor's conductivity modulation. Its structure (Figure 5.3) is that of a vertical n-channel power MOSFET (Section 4.10) with one change: the n+ drain substrate is replaced by a p+ **collector** layer. When the gate turns the MOS channel on, electrons flow from the emitter through the channel into the n− drift region, and the p+ collector injects holes into the drift region to meet them. The drift region fills with an electron–hole plasma whose density far exceeds its doping, and its resistance collapses. That is **conductivity modulation**, the same effect that gives a PIN diode its low forward drop, and it frees the IGBT from the unipolar limit that makes a high-voltage MOSFET's on-resistance rise as roughly the 2.5th power of its blocking voltage.

The equivalent circuit is a MOSFET driving the base of a wide-base PNP transistor. Its output characteristic therefore has a knee: no current flows until the collector junction is forward-biased (about 0.7 V), after which the voltage rises slowly with current. A useful conduction model is

$$v_{CE} = V_{CE0} + r_{CE}\, i_C$$

with $V_{CE0} \approx 0.7$–1.0 V and r_{CE} a few milliohms for a 200 A, 1200 V device.

> **Status of the principle: Threshold-plus-slope conduction model of the IGBT**
> *Approximate.* A piecewise-linear fit to the datasheet output characteristic at a given gate voltage and temperature, adequate from about 10 % to 100 % of rated current; it misrepresents the knee region at low current and must be refitted at each junction temperature.

Generations, tail current and latch-up

Early **punch-through** IGBTs had a negative temperature coefficient of $V_{CE(sat)}$, which made paralleling hazardous; **non-punch-through** devices fixed that. Today's **field-stop trench** IGBTs combine a trench gate, a thin drift region and a lightly doped n field-stop layer before the collector, giving low $V_{CE(sat)}$, a positive temperature coefficient at rated current and 5–10 µs short-circuit capability.

Two penalties come with the bipolar action. At turn-off the MOS channel closes at once, but the stored plasma must recombine or be swept out, and the collector current decays with a **tail** lasting hundreds of nanoseconds to microseconds. The tail dominates turn-off energy and keeps hard-switched silicon IGBTs mostly below about 20 kHz. And the p+ collector, n− drift, p body and n+ emitter form a parasitic **thyristor**: if lateral hole current under the emitter turns on the parasitic NPN, the structure **latches** and the gate loses control. Modern cells push the latching current far above operating currents, but the parasite still sets the limit in short circuit and in the reverse-bias safe operating area (RBSOA).

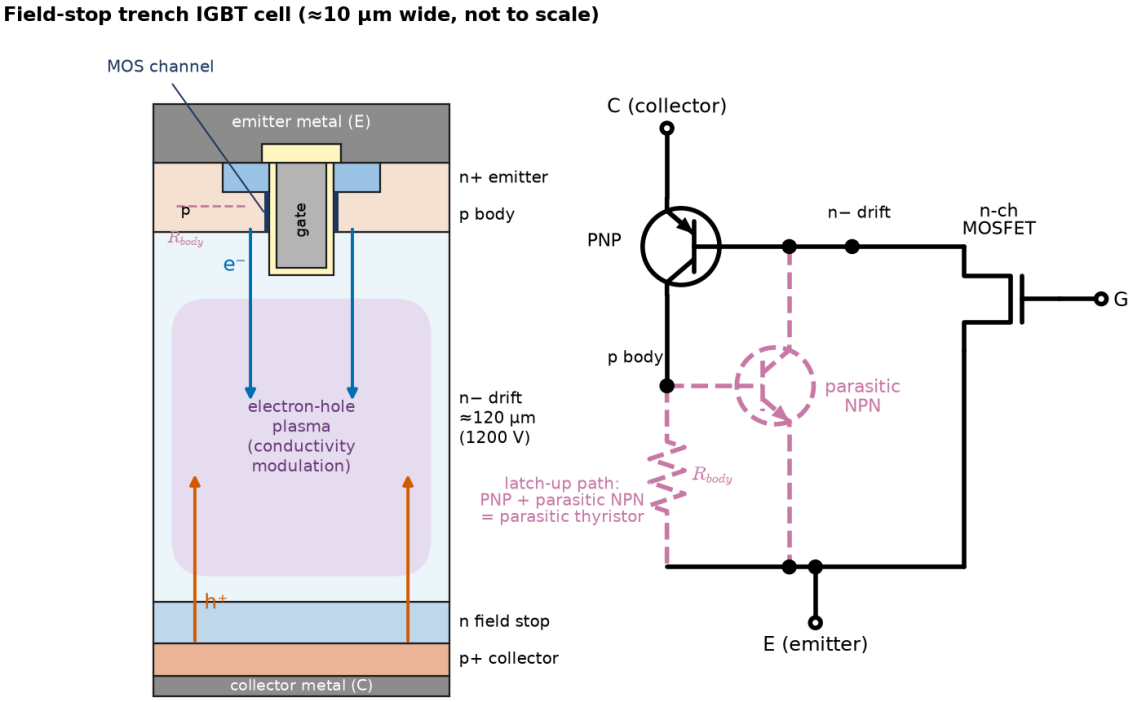


Figure 5.3 — Field-stop trench IGBT cell cross-section with carrier flows, and its equivalent circuit including the parasitic thyristor.

Ratings

An IGBT datasheet (IEC 60747-9) adds to the MOSFET's list:

- V_{CES} (600, 650, 1200, 1700, 3300, 4500, 6500 V) and V_{GES} (±20 V; drive is typically +15 V on, 0 to −15 V off).
- **Nominal current** $I_{C,nom}$ at a stated case temperature, and the repetitive peak I_{CRM} (usually $2I_{C,nom}$).
- $V_{CE(sat)}$ at $I_{C,nom}$ at 25 °C and at 125 or 150 °C: 1.45–2.0 V for 1200 V field-stop trench devices.
- Switching energies E_{on}, E_{off} at stated voltage, current, gate resistance and temperature, and for the antiparallel diode E_{rec} and Q_{rr} (the circuit use of these numbers is in Chapter 29).

- **Short-circuit withstand time** t_{sc} (5–10 µs at $V_{GE} = 15$ V and a stated DC-link voltage and T_j), within which the desaturation detector of the gate driver must act.
- RBSOA and SCSOA, and the thermal resistances $R_{th(j-c)}$ per switch and per diode.

Table 5.2 — Representative 1200 V devices for a 200 A half-bridge (typical datasheet values at rated current).

Quantity	Si field-stop trench IGBT + diode	SiC MOSFET module
Conduction at 25 °C	$V_{CE(sat)} \approx 1.75$ V	$R_{DS(on)} \approx 8$ mΩ (1.6 V at 200 A)
Conduction at 150 °C	$V_{CE(sat)} \approx 2.0$ V	$R_{DS(on)} \approx 13.5$ mΩ (2.7 V at 200 A)
Reverse conduction	antiparallel PIN diode, $V_F \approx 1.8$ V, Q_{rr} 10–30 µC	channel (synchronous) or body diode, $V_F \approx 3.5$ V, $Q_{rr} < 1$ µC
Turn-off	tail current, E_{off} 10–20 mJ at 600 V	no tail, E_{off} 1–3 mJ
Short-circuit withstand	6–10 µs	2–3 µs
Gate drive	+15 / −8 to −15 V	+15 to +18 / −3 to −5 V
Typical switching frequency	4–20 kHz	10–100 kHz

Worked example — Where an IGBT and a SiC MOSFET cross over. At 150 °C, fit the IGBT as $v_{CE} = 0.9\ \text{V} + 5.5\ \text{m}\Omega \cdot i_C$ (2.0 V at 200 A) and the SiC MOSFET as a pure resistance of 13.5 mΩ. Equal drops occur at $0.9 = (13.5 - 5.5) \times 10^{-3}\, I$, that is $I = 112.5$ A. Below it the MOSFET conducts with less drop: at 50 A it drops 0.68 V against the IGBT's 1.18 V, dissipating 34 W instead of 59 W. Above it the IGBT wins: at 200 A, 400 W against 540 W. An electric-vehicle traction inverter rated for 200 A spends most of a drive cycle at a fraction of its peak current, where the MOSFET's lack of a knee pays, and the MOSFET's switching energy is a fraction of the IGBT's at every current; this is the device-level reason behind the shift to SiC traction inverters described in *Materials Science and Engineering* Ch. 33. Figure 5.4 plots both characteristics. Industrial drives at 4–8 kHz, where switching loss matters less and cost more, remain IGBT territory.

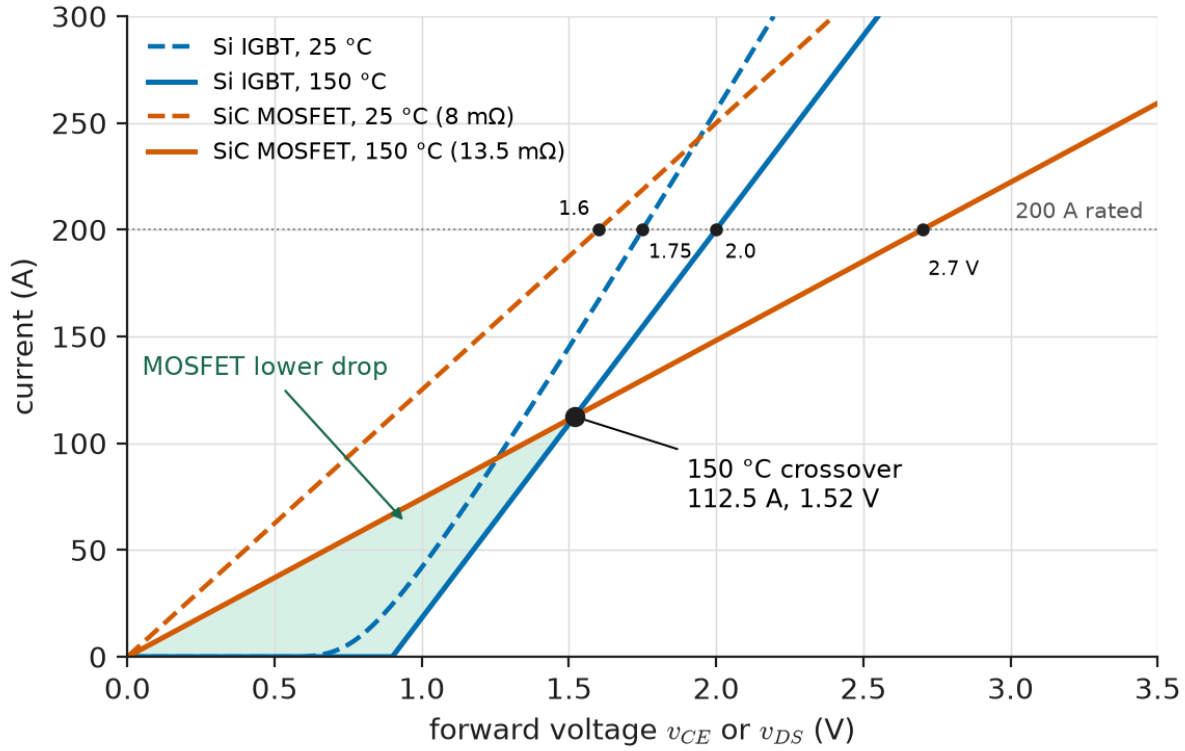


Figure 5.4 — Forward conduction characteristics of a 1200 V, 200 A IGBT and a 1200 V SiC MOSFET at 25 and 150 degrees C.

5.6 Wide-bandgap power devices

Why the material matters

The specific on-resistance of a unipolar drift region varies inversely with the cube of the critical field (*Materials Science and Engineering*, Ch. 33 §33.10). 4H-SiC and GaN have critical fields about ten times silicon's, so for the same blocking voltage a unipolar SiC or GaN device needs a drift region a tenth as thick and a hundred times more heavily doped, and its drift resistance per area is several hundred times lower. Three consequences reach the circuit: unipolar devices become practical at 650–3300 V, where silicon needs bipolar devices with stored charge; for a given $R_{DS(on)}$ the die is smaller, so capacitances and gate charge are smaller and switching faster; and intrinsic leakage stays negligible far above 200 °C, so the package, not the semiconductor, sets the temperature limit.

Silicon carbide devices

SiC Schottky diodes (650–1700 V), with only capacitive recovery charge, are covered as junction devices in Section 2.5; they are the standard boost diode of continuous-conduction PFC stages and the usual freewheeling partner of fast IGBTs.

SiC MOSFETs (650 V–3.3 kV) are vertical DMOS or trench devices like their silicon cousins, with distinctive properties that follow from the SiC/SiO_2 interface and the wide gap:

- **Gate drive.** The poor channel mobility (*Materials Science and Engineering* Ch. 33 §33.4) means $R_{DS(on)}$ keeps falling well above $V_{GS} = 10$ V, so SiC devices are driven at +15 to +20 V. Their threshold is low (2–4.5 V at 25 °C, falling to 1.5–2.5 V at 175 °C) and the Miller charge is a larger fraction of the total, so a negative off-voltage of −3 to −5 V is used to prevent false turn-on from dv/dt of 50–100 V/ns. The allowed gate range is asymmetric (for example +22/−8 V), and the threshold drifts under prolonged bias and switching, a reliability point covered by JEDEC JEP184 and AEC-Q101 qualification.

- **Temperature.** $R_{DS(on)}$ rises less with temperature than in silicon (typically 1.3–1.7 times from 25 °C to 175 °C), because the channel, whose resistance falls with temperature, is a large part of the total.
- **Body diode.** The PN body diode has a 3–4 V drop, because of the wide gap, and small reverse-recovery charge. Recombination at basal-plane dislocations can grow stacking faults that raise the drift resistance, a degradation now screened out by epitaxial processing; the usual practice is synchronous conduction through the channel, using the diode only during dead time.
- **Short circuit.** The small die heats faster than an IGBT, so the withstand time is only 2–3 μs, demanding fast desaturation or current-sense protection.

Gallium nitride devices

GaN power transistors are lateral **high-electron-mobility transistors** (HEMTs) on silicon substrates, whose channel is the polarization-induced two-dimensional electron gas at an AlGaN/GaN interface (*Materials Science and Engineering*, Ch. 33 §33.5). Since the 2DEG exists without gate bias, the basic HEMT is normally on (Section 4.3). Three ways of making a normally-off switch are in use (Figure 5.5):

- **p-GaN gate (enhancement mode)**: a p-doped GaN layer under the gate depletes the 2DEG at zero bias. The threshold is low (1.1–1.7 V) and the gate is a diode-like junction with a tight voltage range (typically −10 to +7 V absolute maximum, driven at +5 to +6 V), so gate overshoot must be controlled carefully.
- **Cascode**: a normally-on high-voltage GaN HEMT in series with a low-voltage silicon MOSFET, which provides a conventional ±20 V gate and a 4 V threshold at the cost of a package with two dies and less control of switching speed.
- **Integrated GaN ICs**, with driver and protection on the GaN die, removing gate-loop inductance.

A GaN HEMT has no body diode and no minority carriers, so its reverse recovery charge is zero. It still conducts in reverse: with the gate off, the channel turns on when the drain is pulled below the source by about the threshold plus the off-state gate bias, giving a "diode" drop of 2–3 V (more with negative gate bias), which makes dead time costly. It has no avalanche rating; instead it is specified for transient overvoltage (for example 750 V for 1 μs on a 650 V part). Charge trapping in the buffer and surface layers can raise the on-resistance for some time after high-voltage blocking (**dynamic** $R_{DS(on)}$, or current collapse), which manufacturers now specify and screen.

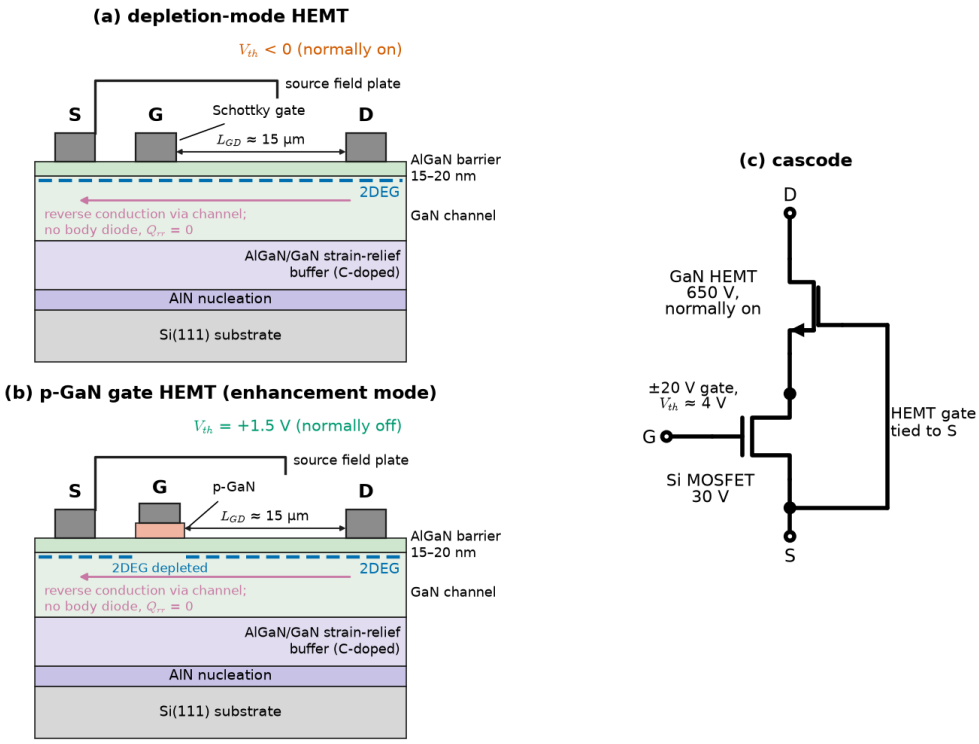


Figure 5.5 — Lateral GaN power HEMT on silicon: depletion-mode, p-GaN enhancement-mode and cascode configurations.

Table 5.3 — Representative 650 V switches near 50 mΩ (typical datasheet values).

Quantity	Si superjunction MOSFET	SiC MOSFET	GaN HEMT (p-GaN gate)
$R_{DS(on)}$ ratio, 150 °C to 25 °C	2.2–2.5	1.2–1.5	1.9–2.3
Total gate charge Q_g	70–100 nC	40–60 nC	5–7 nC
Reverse-recovery charge Q_{rr}	6–12 µC	60–150 nC	0
Gate drive	0 / +10 to +12 V	−4 / +15 to +18 V	0 or −3 / +6 V
Reverse-conduction drop (gate off)	0.9 V (body diode)	3–4 V (body diode)	2–3 V (channel)
Avalanche rating	yes	yes	no (transient overvoltage rating)
Typical role	PFC and LLC stages, soft-switched	hard-switched bridges, PFC, chargers	high-frequency adapters, server PSUs, LiDAR

> **Status of the principle: Wide-bandgap advantage in on-resistance and charge**
> *Approximate.* The material figure of merit predicts orders-of-magnitude lower drift resistance; real devices realise much less because channel, contact and substrate resistances, packaging and reliability margins add to the drift term, so the practical gain at 650 V is a factor of 5–20 in charge for equal on-resistance, not the material factor of several hundred.

Worked example — What the body diode costs in a hard-switched bridge. In a half-bridge with a 400 V DC link switching at 100 kHz, each time the upper switch turns on it must sweep out the reverse-recovery charge of the lower

device's body diode, drawing roughly $Q_{rr}V_{DC}$ of energy from the link per cycle (a first estimate; the loss mechanism is detailed in Chapter 29). With the values of Table 5.3:

- Si superjunction, $Q_{rr} = 8\ \mu$C: 8 μC × 400 V = 3.2 mJ per cycle, 320 W at 100 kHz.
- SiC, $Q_{rr} = 100$ nC: 40 µJ per cycle, 4 W.
- GaN, $Q_{rr} = 0$: none, leaving only the output-capacitance charge ($Q_{oss} = 57$ nC in a typical 50 mΩ part, so $Q_{oss}V_{DC} = 22.8$ µJ drawn from the 400 V link per cycle).

The silicon number is absurd, and that is the point: a superjunction MOSFET's body diode cannot be hard-commutated, which is why the bridgeless totem-pole PFC, which depends on exactly that, became practical only with SiC and GaN. Gate-drive power tells the same story: $Q_g V_{drive} f$ is 0.11 W for the superjunction device at 12 V and 100 kHz, 0.10 W for SiC at a 22 V swing, but only 3.5 mW for GaN at 6 V.

5.7 Optoelectronic devices

> **Boundary.**
> LEDs, photodiodes and solar cells as junction devices are covered in Chapter 2; the optics and laser physics in *Engineering Physics* Ch. 12; optical sources and receivers in fibre links in *Circuits, Signals and Communications*, Ch. 31 §31.5–31.6; emitter and detector materials in *Materials Science and Engineering* Ch. 35. This section covers the composite and specialised devices a circuit designer specifies.

Laser diodes

A laser diode is an LED with optical feedback whose output above a **threshold current** I_{th} rises with slope efficiency η_s (W/A). For the circuit designer it is a current-driven load with three hazards: its threshold rises and slope falls with temperature (constant power needs a monitor photodiode and a loop, or a thermoelectric cooler); nanosecond current or ESD transients destroy the facet; and its output may exceed the limits of IEC 60825-1, whose classes (1 to 4) dictate interlocks and labelling. Representative types: 850 and 940 nm **VCSELs** (I_{th} 0.5–2 mA; data-centre links, face-recognition and flash LiDAR), 905 nm **pulsed edge-emitters** for automotive and robot LiDAR (tens of watts peak in 2–10 ns pulses of 10–40 A, driven by GaN switches), and 1310/1550 nm DFB lasers for telecom.

Phototransistors and optocouplers

A **phototransistor** is a BJT whose collector-base junction is a large photodiode; the photocurrent is amplified by β, giving responsivities of tens to hundreds of mA per mW/cm² at the cost of speed (rise times of 5–20 µs at kilohm loads, from the Miller-multiplied junction capacitance). Paired with an infrared LED in one package across an insulating gap, it forms the **optocoupler**, the oldest means of galvanic isolation for signals.

The optocoupler's transfer is specified by its **current-transfer ratio**, CTR $= I_C/I_F$, which spreads widely (50–600 % for a PC817-class part, sold in rank bins), falls at low and high temperatures, and decreases over life as the LED's output degrades, faster at high forward current and temperature. Its isolation is rated by test voltage (5 kV RMS for one minute per UL 1577), working voltage and creepage under IEC 60747-5-5 or IEC 60747-17, and its **common-mode transient immunity** (CMTI, 15–50 kV/µs for standard optocouplers) states the dv/dt between the two sides that it tolerates without a false output.

Worked example — Designing an optocoupler link for a ten-year life. A PC817-class optocoupler with minimum CTR 50 % at $I_F = 5$ mA isolates a fault signal into a 3.3 V microcontroller input, with a pull-up R_L on the transistor side; the output must reach $V_{CE} \le 0.4$ V. Allowing a factor of 0.7 for LED degradation over ten years at the operating temperature and 0.8 for the CTR fall at −20 °C gives a design CTR of $0.50 \times 0.7 \times 0.8 = 28$ %. At $I_F = 5$ mA the guaranteed collector current is then 1.4 mA. The pull-up must satisfy $(3.3 - 0.4)/R_L \le 1.4$ mA, so $R_L \ge 2.07$ kΩ; choosing 4.7 kΩ needs only 0.62 mA and gives an overdrive of 2.3, comfortable margin. The LED resistor from a 3.3 V GPIO with $V_F \approx 1.2$ V is $(3.3 - 1.2)/5$ mA $= 420\ \Omega$, so 390 Ω (5.4 mA). The cost of the large R_L is speed: the transistor's

turn-off time at 4.7 kΩ is tens of microseconds, limiting the link to roughly 10 kbit/s. Fast links use digital-output optocouplers (6N137 class, 10 Mbit/s) or the alternatives below.

Isolation alternatives and gate-drive couplers

Digital isolators transmit across a thin silicon-dioxide or polyimide barrier capacitively (TI ISO77xx class) or through on-chip transformers (ADI iCoupler class), at 100–150 Mbit/s, with CMTI of 100–200 kV/µs and no wear-out mechanism comparable to LED degradation. **Gate-drive optocouplers and isolated gate drivers** (HCPL-3120 class at 2.5 A, and isolator-based drivers at 4–10 A with desaturation detection) carry the gate signal to IGBTs and SiC MOSFETs whose emitters swing hundreds of volts at 50–100 V/ns; CMTI is their decisive parameter. **Photovoltaic couplers** generate a floating gate voltage and, with MOSFETs, form contactless **solid-state relays** (PhotoMOS class) for test equipment and battery-pack monitoring.

Detectors with gain and imagers

Avalanche photodiodes (Section 2.6) biased beyond breakdown and quenched after each event become **single-photon avalanche diodes** (SPADs); arrays of them form silicon photomultipliers (SiPMs) for LiDAR receivers, PET scanners and particle detectors, with bias that must track the breakdown voltage's temperature drift. **CMOS image sensors** integrate a pinned photodiode, a transfer gate and a source-follower readout in each pixel (the 4T pixel), with full-well capacities of 5–50 thousand electrons and read noise of 1–3 electrons.

5.8 Compound-semiconductor devices

Why III–V and III–N transistors

Silicon CMOS and SiGe dominate integration, but where power, noise or frequency at the device level decide a design, III–V and III–N transistors win. GaAs and InP offer high electron mobility and velocity overshoot for speed; GaN offers a high critical field together with the dense 2DEG, for power at frequency (*Materials Science and Engineering*, Ch. 35 §35.8, whose Johnson figure of merit ranks them). The heterojunction is the tool in every case: a wider-gap barrier on a narrower-gap channel, either to form a high-mobility 2DEG (the HEMT) or to block holes from flowing back into the emitter (the heterojunction bipolar transistor, HBT).

The device families

GaAs pseudomorphic HEMTs (pHEMTs) serve low-noise amplifiers and antenna switches, with noise figures of 0.3–1 dB at 10 GHz. InP HEMTs give the lowest noise at any frequency and, cooled to 4 K, read out radio telescopes and superconducting quantum computers. GaN-on-SiC HEMTs, at 28–50 V and 5–10 W per millimetre of gate width, have taken over radar, base-station and satellite power amplifiers, the SiC substrate removing the heat. GaAs/InGaP HBTs are the power amplifiers of nearly every handset; SiGe HBTs, built in BiCMOS processes, power 77 GHz automotive radar transceivers; InP HBTs drive 100 Gbaud optical modulators. Table 5.4 compares them.

Table 5.4 — Representative RF transistor technologies (typical).

Technology	f_T	f_{max}	Supply	Output power density	Main uses
Si LDMOS	10–20 GHz	20–30 GHz	28–50 V	about 1 W/mm	sub-3 GHz base stations, broadcast, RF heating
GaAs pHEMT	50–150 GHz	100–250 GHz	3–8 V	0.5–1 W/mm	LNAs, switches, millimetre-wave
GaN-on-SiC HEMT	30–150 GHz	60–300 GHz	28–50 V	5–10 W/mm	radar, 5G massive-MIMO, satellite PAs
InP HEMT	500–700 GHz	1 THz class	1–2 V	low	cryogenic and sub-THz LNAs
GaAs/InGaP HBT	40–60 GHz	60–100 GHz	3.4–5 V	high current density	handset power amplifiers
SiGe HBT	300–500 GHz	400–700 GHz	1.5–3 V	low	automotive radar, optical drivers

Worked example — Why radar amplifiers moved to GaN. A 40 W X-band power-amplifier stage needs a total gate periphery of $40/5 = 8$ mm in GaN at 5 W/mm, against $40/1 = 40$ mm in GaAs at 1 W/mm. The optimum load resistance for a class-B-like stage is about $R_{opt} = (V_{DD} - V_{knee})^2/(2P_{out})$. For GaN at 28 V with a 3 V knee, $R_{opt} = 25^2/80 = 7.8\ \Omega$; for GaAs at 8 V with a 1 V knee, $7^2/80 = 0.61\ \Omega$. Matching 7.8 Ω to a 50 Ω system is an impedance ratio of 6.4; matching 0.61 Ω is a ratio of 82, which needs a lossy, narrowband multi-section network and a combiner across a 40 mm die. Smaller periphery also means smaller parasitic capacitance and wider bandwidth. The matching theory itself is in *Circuits, Signals and Communications*, Ch. 32.

GaN RF HEMTs are depletion-mode, so their negative gate bias must be applied before the drain supply (bias sequencing).

5.9 Device selection and ratings

Absolute maximum ratings and characteristics

The distinction between absolute maximum ratings, which must hold under every condition including faults, and guaranteed characteristics, which hold only at their test conditions, is set out in Section 1.8. For power devices it has a sharp consequence: selection works from the absolute maxima with margins, while losses and temperatures are computed from the maximum (not typical) characteristics at the hottest junction temperature.

Voltage class

The blocking rating must cover the highest DC or peak voltage, plus switching overshoot $L_\sigma\, di/dt$ from the commutation loop's stray inductance, plus grid and load transients, with a margin for life. For power devices that margin is set largely by **cosmic-ray-induced failure**: neutrons from cosmic-ray showers occasionally trigger a destructive local avalanche in a device blocking high voltage, at a rate that rises roughly exponentially with the applied voltage as a fraction of rating, and with altitude. Manufacturers publish failure rates (FIT, failures per 10^9 device-hours) versus DC voltage, and the usual result is a sustained DC voltage of no more than 55–70 % of the rating for silicon and similar or lower for SiC. Table 5.5 shows the resulting classes.

Table 5.5 — Typical voltage classes by application.

System	Highest working voltage	Device class
12 V automotive (ISO 16750-2 load dump suppressed to about 35 V)	16 V working, 35 V transient	40 V MOSFET
24 V industrial, 48 V telecom and server	30 V, 60 V	60 V, 80–100 V MOSFET
230 V single-phase, PFC bus	400 V DC, 450 V peak	600–650 V SJ, SiC, GaN
400 V three-phase drive	565–750 V DC	1200 V IGBT or SiC
800 V EV battery	900 V DC at full charge	1200 V SiC
690 V three-phase drive, 1500 V solar (three-level)	1100 V DC; 750 V per device	1700 V IGBT; 1200 V devices in NPC

Current, temperature and technology

Current capability is a thermal statement. The useful rating is the current at which the junction reaches its design limit, typically 25 °C below $T_{j,max}$ for long life, with the device's conduction and switching losses at that temperature and the whole thermal path of *Engineering Physics* Ch. 14 §14.7. The datasheet's continuous current at $T_C = 25$ °C is a comparison figure only. Power cycling, the repeated heating and cooling of the bond wires and solder layers, sets the life of power modules in traction and wind, and its empirical models (LESIT and CIPS fits of cycles to failure against ΔT_j and mean temperature) are part of selection for those duty cycles.

> **Status of the principle: Voltage derating for cosmic-ray robustness**
> *Empirical.* Failure rates are measured in neutron and proton beams and fitted per technology to exponential functions of applied voltage and altitude; the margins are manufacturers' and industry practice, not consequences of a closed-form law, and they differ between suppliers of the same voltage class.

With voltage class and current settled, the technology choice follows the map of Figure 5.6: silicon MOSFETs below about 200 V at any frequency; superjunction, SiC and GaN contesting 650 V, with GaN winning at the highest frequencies and lowest powers and SiC at higher powers and in hard switching; IGBTs and SiC at 1200–1700 V, with SiC wherever efficiency, frequency or size pays for its price; IGBT modules and press-pack IGBTs and IGCTs at 3.3–6.5 kV; and thyristors where line commutation suffices and power is highest.

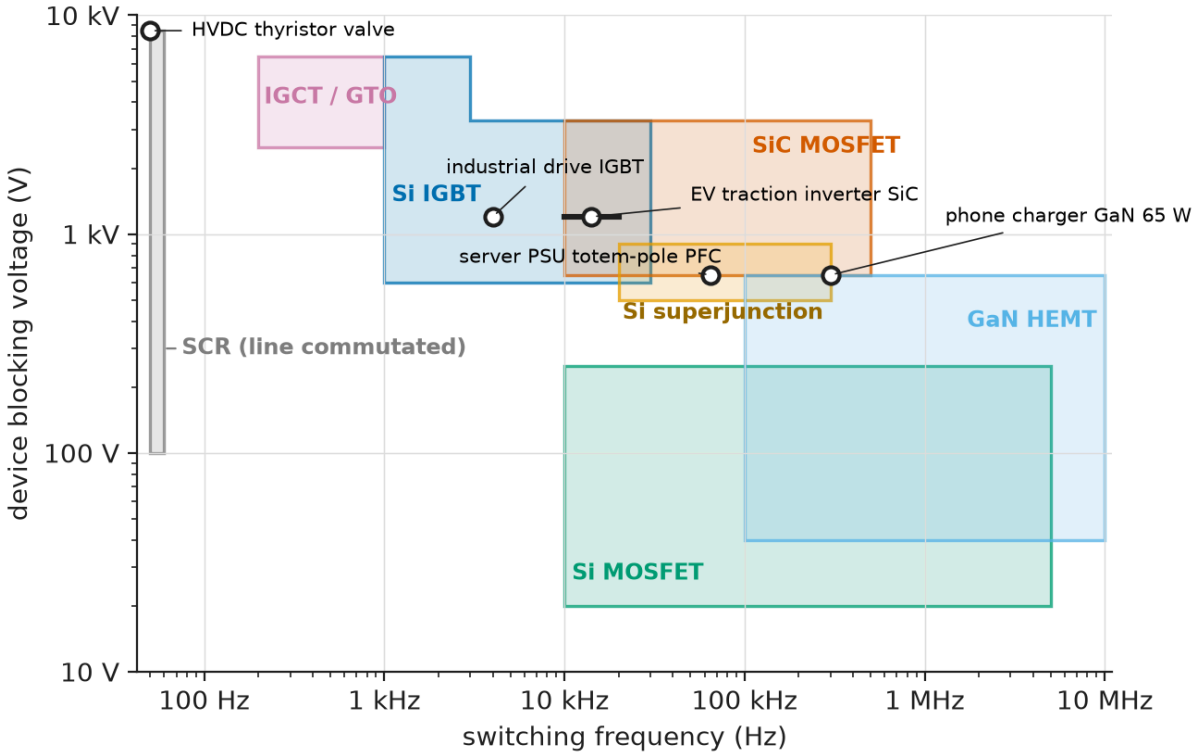


Figure 5.6 — Application map of power semiconductor technologies by blocking voltage and switching frequency.

Worked example — First-pass selection for an 800 V traction inverter. A 150 kW motor inverter runs from an 800 V battery that reaches 900 V at full charge. A commutation loop of 10 nH carrying 600 A switched in 100 ns adds $10\ \text{nH} \times 600\ \text{A}/100\ \text{ns} = 60\ \text{V}$ of overshoot, so the device sees 960 V, 80 % of a 1200 V rating at the worst instant; the sustained 900 V is 75 %, which the manufacturer's FIT curve must show to be acceptable at the vehicle's altitude profile. At full power with sinusoidal space-vector modulation, the highest linear line voltage is $800/\sqrt{2} = 566$ V RMS, so at a power factor of 0.85 the phase current is $150\ \text{kW}/(\sqrt{3} \times 566 \times 0.85) = 180$ A RMS, 255 A peak. With synchronous conduction, each switch carries roughly $180/\sqrt{2} = 127$ A RMS. A SiC module with 2.2 mΩ per switch at 25 °C, 3.5 mΩ hot (factor 1.6), dissipates about $127^2 \times 3.5\ \text{m}\Omega = 57$ W in conduction; allowing an equal amount for switching (to be checked properly with the methods of Chapter 29) and a junction-to-coolant resistance of 0.22 K/W, the junction rises 25 K above 65 °C coolant to about 90 °C. The candidate passes the first check with room for the peak-torque transient, and the detailed loss, gate-drive and short-circuit protection design can begin.

What remains open

Open question	Where it is answered
How are switching and conduction losses of IGBTs, SiC and GaN devices calculated, and how are they gated, snubbed and protected?	Chapter 29
How are thyristor-controlled and three-phase rectifiers and PWM inverters built from these devices?	Chapter 31
How do these switches appear in motor drives, battery chargers and grid-connected converters?	Chapter 32; drive control in *Electrical Energy and Machines*, Ch. 12
How are uncontrolled rectifiers for power supplies designed around diodes?	Chapter 6
How are optocoupler, photodiode and laser-driver interface circuits designed?	Chapters 8 and 26; optical links in *Circuits, Signals and Communications*, Ch. 31
How are RF amplifiers matched and characterised?	*Circuits, Signals and Communications*, Ch. 32
Why do SiC and GaN have the critical fields and defects they do?	*Materials Science and Engineering*, Ch. 33–34

Next: Chapter 6 opens Part II by putting the simplest of these devices, the diode, to work in rectifiers, filters, clippers, clampers and multipliers.

Part II — Analog Electronics

Part I described devices one at a time. This part connects them into circuits that do continuous-valued work: rectifying the mains into a DC rail, amplifying a microvolt sensor signal, filtering it, making it oscillate on purpose or keeping it from oscillating by accident, and holding a reference voltage steady. A nonlinear device, biased and wrapped in feedback, behaves like a linear building block (a gain of 100, a 1 kHz low-pass filter, a 2.500 V reference), but only within the limits of swing, bandwidth, slew rate, noise, offset, temperature and stability that the device sets. A strain-gauge amplifier that drifts with the seasons, a regulator that oscillates when a ceramic capacitor is fitted, a rectifier whose capacitor dies after two summers: each is an abstraction pushed past its limits, and each is predicted by the methods of this part.

By the end of this part you should understand

- How diodes rectify, filter, clip, clamp and multiply, and how to design a transformer–rectifier–capacitor supply from its worst-case corners rather than from its nominal values.
- How a single transistor stage amplifies, how its gain, input and output impedance follow from the small-signal model, and how stages are cascaded, biased and coupled.
- What sets an amplifier's bandwidth and distortion, and how the Miller effect, the dominant pole and nonlinearity show up in measurements.
- How far the ideal op-amp model can be trusted, and how to design the standard op-amp circuits, from instrumentation amplifiers to active filters, to a numerical specification.
- What negative and positive feedback do to gain, bandwidth, impedance and distortion, how loop gain and phase margin decide stability, and how the Barkhausen criterion turns instability into RC, LC and crystal oscillators.
- How integrated circuits build amplifiers, references and regulators from matched transistors, current mirrors and active loads, why they need frequency compensation, and how comparators, DACs and ADCs form the interface to digital logic.

Chapters

Chapter	Central question	What it delivers
Chapter 6 — Diode Circuits	How much DC, ripple and component stress does a rectifier produce, and how do diodes reshape and protect signals?	Rectifier circuits and figures of merit, ripple, capacitor-input filters simulated for peak and RMS currents, clippers and input protection, clampers, voltage multipliers, a worst-case supply design
Chapter 7 — Transistor Amplifiers	How are the gain and the input and output impedance of a transistor stage computed, and what sets its bandwidth and distortion?	Common-emitter, common-source and follower stages, small-signal analysis, multistage amplifiers, impedance levels, frequency response, distortion
Chapter 8 — Operational Amplifiers	How far can the ideal op-amp be trusted, and how are the standard op-amp circuits designed to a specification?	Ideal and practical op-amps, amplifier configurations, integrators, comparators, instrumentation amplifiers, precision rectifiers, active filters, oscillators, limitations and stability
Chapter 9 — Feedback and Oscillators	What does feedback do to an amplifier, when does it make the amplifier unstable, and how is that instability used to build oscillators?	Positive and negative feedback, the four feedback topologies, gain–bandwidth trade-off, stability margins, the Barkhausen criterion, RC, LC and crystal oscillators, waveform generators
Chapter 10 — Analog Integrated Circuits	How are amplifiers, references and regulators built from matched transistors on one die, and how do they meet the digital world?	Differential pairs, current mirrors, active loads, multistage IC amplifiers, frequency compensation, analog IC architecture, voltage references, regulators, comparators, mixed-signal interfaces

How this part connects to the rest of the book

Part II stands on Part I. The diode ratings and loss model of Chapter 2, the BJT and FET bias and small-signal models of Chapters 3 and 4, and the datasheet and thermal discipline of Chapter 1 are used here as known tools. Looking forward, the comparators and converters of Chapter 10 are the boundary with the digital electronics of Part III, and the ADC and DAC interfaces return in Chapter 26 as embedded peripherals. The rectifiers of Chapter 6 are developed into controlled and three-phase rectifiers for power conversion in Chapter 31, and the linear regulator of Chapter 10 meets its switch-mode competitor in Chapter 30.

Three sibling books carry theory this part uses without re-deriving it. Circuit laws, network theorems, phasors, Laplace-domain analysis, Bode plots, Fourier series and feedback stability (Nyquist, root locus, margins) come from *Circuits, Signals and Communications*, Chs. 2–13, 18 and 34–35. The one-section sketches of the diode, transistor, op-amp, filter and rectifier, and the thermal-resistance network behind every heatsink calculation, come from *Engineering Physics: Top Down*, Ch. 18 and Ch. 14 §14.7. The power-quality consequences of rectifier input currents, and the limits that grid codes place on them, belong to *Electrical Energy and Machines*, Ch. 25.

Chapter 6 — Diode Circuits

Part II · Analog Electronics

An industrial sensor hub takes its 12 V supply from a small linear power supply: a 230 V transformer, a bridge rectifier, a reservoir capacitor and a 7812 regulator delivering 1 A. The designer sized the transformer from the DC power, 12 W plus the regulator's headroom, and bought a 3300 µF capacitor because the ripple formula asked for about that much capacitance. Two summers later units start to fail. The transformer runs at 85 °C, and the reservoir capacitor's vent has bulged. Neither part was misused in the sense its designer understood. The bridge does not draw a sinusoid from the transformer: it draws pulses about 70° wide and four times the load current at their peak, so the secondary carries 1.8 A RMS to deliver 1 A DC, and the capacitor carries 1.5 A RMS of ripple current at 100 Hz, which heats it from inside. A rectifier is the simplest nonlinear circuit in electronics, and it is also the one in which the nonlinearity most directly sets the size, temperature and life of the parts around it. This chapter treats the diode as a circuit element, first in rectifiers and their filters, then in the waveshaping circuits that clip, clamp and multiply, and ends with a design procedure that would have kept the sensor hub alive.

> **How much DC, ripple and stress does each rectifier circuit produce, and how are those numbers computed when a capacitor makes the diode current a pulse?**
> **How do diodes reshape, limit, shift and multiply signals and protect inputs, and where do their non-ideal characteristics show? What procedure turns a load specification into a transformer, diodes, capacitor and fuse that survive mains tolerance, inrush and a ten-year life?**

Concept	Engineering destination
Half-wave rectifiers	Auxiliary and bias supplies, envelope and peak detectors, AC voltmeters
Full-wave rectifiers	Low-voltage high-current supplies, SMPS secondaries with centre-tapped windings
Bridge rectifiers	Mains input of every linear and switch-mode supply, battery chargers, LED drivers
Rectifier performance	Comparing topologies; sizing transformers and diodes
Ripple	Regulator headroom, hum in audio, ADC reference noise
Capacitor filtering	Reservoir-capacitor and inrush design; transformer and capacitor heating
Clipping circuits	ADC and MCU input protection, signal limiting, waveform shaping
Clamping circuits	DC restoration, negative gate bias for SiC drives, detector biasing
Voltage multipliers	Photomultiplier, X-ray and ionizer supplies, gate-drive and bias rails
Practical rectifier design	Linear supplies for instrumentation, industrial control and audio

Roadmap. The three rectifier circuits (6.1–6.3) → the figures of merit that compare them (6.4) → ripple (6.5) and the capacitor-input filter that dominates practice (6.6) → waveshaping with diodes: clippers (6.7), clampers (6.8) and multipliers (6.9) → a complete rectifier design (6.10).

6.1 Half-wave rectifiers

> **Boundary.**
> The average and RMS values of rectified sinusoids are tabulated in *Circuits, Signals and Communications*, Ch. 8 §8.2; the harmonic content of rectifier input current and the resulting distortion power factor are treated in Ch. 9 §9.8; RC time constants in Ch. 6. *Engineering Physics* Ch. 18 §18.8.1 sketches the half-wave and bridge averages and the full-wave ripple estimate. Rectifier diode ratings (V_{RRM}, $I_{F(AV)}$, I_{FSM}, I^2t, recovery) and the threshold–slope loss model are in Chapter 2. Controlled (thyristor) and three-phase rectifiers for power conversion belong to Chapter 31; this chapter covers uncontrolled single-phase rectifiers for power supplies and low-power diode circuits.

The circuit and the diode model

A single diode in series with a resistive load passes the half-cycles that forward-bias it (Figure 6.1a). The analysis depends on how much of the diode's nonlinearity is kept. Three models cover practice:

- the **ideal diode**, a short circuit forward and an open circuit reverse, adequate when the peak voltage is tens of volts or more;
- the **constant-drop** model, a 0.7 V source for silicon (0.3–0.5 V for Schottky) in series with an ideal diode, the standard for supply design;
- the **threshold–slope** model $v_F = V_{F0} + r_T i_F$ of Chapter 2, needed when the peak current is large enough that r_T matters, as it is in capacitor-input filters.

> **Status of the principle: Constant-drop diode model**
> *Approximate.* It replaces the exponential characteristic by a fixed voltage, valid when the current varies over less than about two decades so that the true drop changes by less than about 0.1–0.2 V; it ignores reverse recovery, junction capacitance and leakage, which matter at high frequency and in precision circuits.

With the constant-drop model, the diode conducts only while $V_m \sin \omega t > V_D$, from $\theta_0 = \arcsin(V_D/V_m)$ to $\pi - \theta_0$. For $V_m \gg V_D$ the DC output approaches the familiar

$$V_{dc} \approx \frac{V_m - V_D}{\pi}$$

which overestimates slightly at low voltage because it ignores the dead zone near the zero crossings. The diode must block the full negative peak, so its **peak inverse voltage** (PIV) is V_m with a resistive load and $2V_m$ when a reservoir capacitor holds the cathode at $+V_m$ while the anode swings to $-V_m$.

Where half-wave rectification is used

The half-wave rectifier's defects are severe at power: the output ripple is at the supply frequency, so filtering needs twice the capacitance of a full-wave circuit; the transformer secondary carries a DC component that biases the core toward saturation and raises its magnetising current; and the input current contains even harmonics and DC, which IEC 61000-3-2 effectively prohibits above small powers. It survives where these do not matter:

- **Low-power auxiliary supplies**: a few milliamperes for a standby microcontroller, often transformerless from the mains through a capacitive dropper.
- **Envelope (AM) and peak detectors.** A diode feeding a parallel RC recovers the envelope of a modulated carrier if $1/f_c \ll RC \ll 1/(2\pi f_m)$: for a 455 kHz IF carrying 5 kHz audio, RC must lie between 2.2 µs and 32 µs, and 10 kΩ with 1 nF (10 µs) is the classic choice. The same circuit with a long time constant is a peak detector, used in RF power meters and signal-strength indicators, with a Schottky diode to reduce the drop.
- **Flyback converter secondaries**, where the transformer is driven by pulses and the half-wave rectifier is natural (Chapter 30).

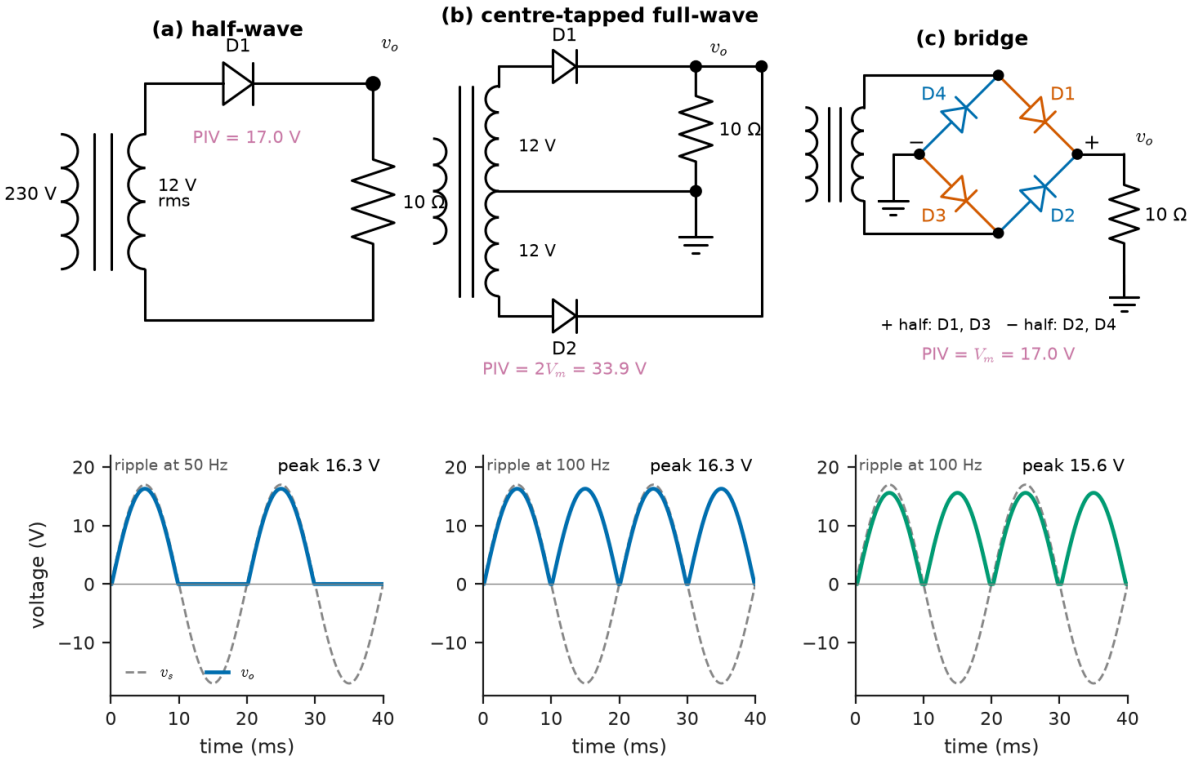


Figure 6.1 — The three single-phase rectifier circuits with resistive load and their output waveforms.

6.2 Full-wave rectifiers

The centre-tapped circuit

A transformer with a centre-tapped secondary supplies two voltages in antiphase; one diode on each end conducts on alternate half-cycles into a common load (Figure 6.1b). Every half-cycle is used, so the ripple frequency doubles to $2f$ and the DC output doubles:

$$V_{dc} \approx \frac{2(V_m - V_D)}{\pi}$$

where V_m is the peak of each half-winding. Only one diode is in the current path at a time, which saves one diode drop compared with the bridge. The price is the diode's PIV, which is $2V_m$ (strictly $2V_m - V_D$): while one diode conducts, the other sees the full voltage of the whole secondary. Each half of the secondary also carries current only half of the time, so the winding copper is used less effectively than in a bridge.

Where it wins

The saved diode drop is decisive at low output voltage. In a 5 V, 20 A supply, a 0.5 V Schottky drop is 10 % of the output; a bridge would lose 1 V, 20 %, and 20 W. Centre-tapped full-wave rectification is therefore the norm on the secondaries of push-pull, half-bridge and full-bridge switch-mode converters for low-voltage, high-current outputs (server, telecom and PC supplies), where the diodes are Schottky types or, increasingly, synchronous-rectifier MOSFETs whose drop is $I \cdot R_{DS(on)}$. It is also the standard way of producing a symmetrical dual supply: a centre-tapped secondary feeding a bridge gives $+V$ and $-V$ about the centre tap, the classic ±15 V rails of an analog instrument or audio amplifier.

6.3 Bridge rectifiers

Operation

Four diodes in a bridge rectify a single secondary winding (Figure 6.1c). On the positive half-cycle D1 and D3 conduct; on the negative, D2 and D4. The load current always flows the same way, the ripple is at $2f$, and

$$V_{dc} \approx \frac{2(V_m - 2V_D)}{\pi}$$

since two diodes are always in series. Each non-conducting diode sees the peak V_m (plus one diode drop) in reverse, half the PIV of the centre-tapped circuit, and the transformer secondary carries current in both half-cycles with no DC component.

Packages and active bridges

Bridges are sold as single components: DB and MB packages (0.5–1 A, surface mount), KBP and GBU in-line packages (2–8 A, the latter with a heatsink tab), and KBPC square packages (15–50 A, bolted to a heatsink). Their ratings are those of rectifier diodes (Chapter 2), stated for the whole bridge: V_{RRM} of 50–1000 V, $I_{F(AV)}$ for the bridge's DC output, I_{FSM} per diode, and a forward drop per diode.

At low voltage the two diode drops cost a large fraction of the output: 1.4–2 V out of a 12 V or 24 V supply. **Active bridges** replace the diodes with four MOSFETs controlled by an ideal-diode bridge controller (the LT4320 class), cutting the drop to tens of millivolts. They are used in Power-over-Ethernet powered devices, whose input polarity is unknown, and on 24 V AC control circuits and thermostat wiring, where every watt of heat in a sealed enclosure counts. The three-phase six-diode bridge, with ripple at $6f$, is the input stage of industrial drives and large supplies; it belongs, with its controlled thyristor relatives, to Chapter 31.

6.4 Rectifier performance

Figures of merit

Rectifiers are compared by a small set of ratios, all computed for an ideal diode and a resistive load unless stated otherwise:

- **DC output** V_{dc} and **RMS output** V_{rms}, from which the **form factor** V_{rms}/V_{dc} follows.
- **Ripple factor** $r = V_{ac,rms}/V_{dc} = \sqrt{(V_{rms}/V_{dc})^2 - 1}$, the ratio of the RMS of the AC part of the output to its DC value.
- **Rectification efficiency** $\eta = P_{dc}/P_{ac} = V_{dc}^2/V_{rms}^2$, the fraction of the load power that is DC.
- **Transformer utilisation factor** (TUF), the DC load power divided by the VA rating the transformer winding must have; it measures how much copper and iron a given DC output costs.
- **Peak inverse voltage** across each diode, and the **ripple frequency**.

> **Status of the principle: Rectifier figures of merit**
> *Derived.* Each ratio follows by integrating the ideal rectified waveform over a period; the values in Table 6.1 are exact for ideal diodes, sinusoidal supply and resistive load, and are only a starting point for capacitor-input rectifiers, whose current waveform is set by the filter (Section 6.6).

Table 6.1 — Rectifier figures of merit (ideal diodes, resistive load, peak V_m per winding).

Quantity	Half-wave	Centre-tapped full-wave	Bridge
V_{dc}	$V_m/\pi = 0.318V_m$	$2V_m/\pi = 0.637V_m$	$2V_m/\pi = 0.637V_m$
V_{rms}	$V_m/2$	$V_m/\sqrt{2}$	$V_m/\sqrt{2}$
Form factor	1.571	1.111	1.111
Ripple factor	1.211	0.483	0.483
Rectification efficiency	40.5 %	81.1 %	81.1 %
TUF	0.287	0.693 (average of primary 0.812 and secondary 0.574)	0.812
Diodes in conduction path	1	1	2
PIV per diode	V_m	$2V_m$	V_m
Ripple frequency	f	$2f$	$2f$

The table explains the field's habits. The bridge has the best transformer utilisation and the lowest diode PIV at the cost of a second diode drop; the centre-tapped circuit saves that drop and pays in transformer size and PIV; the half-wave circuit is poor on every count except part count.

Worked example — Half-wave versus bridge at 12 V RMS. A 12 V RMS secondary ($V_m = 16.97$ V) feeds a 10 Ω load through silicon diodes modelled as 0.7 V drops. Integrating the clipped half-sines with their dead zones:

- Half-wave: $V_{dc} = 5.06$ V, $V_{rms} = 8.04$ V, so $P_{dc} = 5.06^2/10 = 2.56$ W out of a total load power of $8.04^2/10 = 6.47$ W; $\eta = 39.5$ % against the ideal 40.5 %.
- Bridge: $V_{dc} = 9.44$ V, $V_{rms} = 10.76$ V, $P_{dc} = 8.91$ W of 11.57 W; $\eta = 77.0$ % against the ideal 81.1 %.

The bridge delivers 1.87 times the DC voltage, not 2, because it loses two drops instead of one, and its efficiency suffers more from the drops for the same reason. The simple formulas give 5.18 V and 9.91 V: 2–5 % high, because they ignore the dead zones. Both diodes must block 17.0 V; a 50 V part gives the usual margin of at least two for mains transients.

6.5 Ripple

Ripple with a resistive load

Without a filter, the rectifier output is mostly ripple: Table 6.1's ripple factors of 1.21 and 0.48 mean that the AC component is larger than or half the DC. Its spectrum (Fourier series per *Circuits, Signals and Communications* Ch. 18 §18.1) is dominated by the lowest component: for full-wave rectification, a second harmonic of amplitude $4V_m/(3\pi)$, two-thirds of the DC value, followed by the fourth harmonic at a fifth of that. Every filter is designed around the lowest ripple frequency, f for half-wave and $2f$ for full-wave, which is the main reason full-wave rectification dominates.

Ripple with a reservoir capacitor

A capacitor across the load charges to nearly the peak voltage when the diodes conduct and discharges into the load between peaks (Figure 6.2). If the load draws a constant current I_{dc} and the capacitor discharges for nearly the whole interval between charging pulses, charge balance gives the peak-to-peak ripple

$$\Delta V \approx \frac{I_{dc}}{p f C}$$

with $p = 1$ pulse per supply cycle for half-wave and $p = 2$ for full-wave rectification (the full-wave case is quoted in *Engineering Physics* Ch. 18 §18.8.1). The ripple waveform is roughly a sawtooth, so its RMS value is about $\Delta V/(2\sqrt{3})$, and the DC output is about $V_{pk} - \Delta V/2$, where V_{pk} is the peak capacitor voltage.

> **Status of the principle: Reservoir-capacitor ripple estimate**
> *Approximate.* It assumes constant load current, a ripple small compared with the peak ($\Delta V \lesssim 0.2 V_{pk}$) and discharge throughout the period $1/(pf)$; because the capacitor actually recharges during a conduction interval of 40–80° per half-cycle, the formula overestimates the ripple by 20–40 % in typical mains supplies, which makes it conservative for design but useless for predicting diode and capacitor currents.

The estimate makes the trade-offs visible. Ripple falls in proportion to capacitance and to the number of pulses; a 50 Hz half-wave supply needs twice the capacitance of a full-wave one; and a switch-mode converter's rectifier at 100 kHz needs a thousandth of the capacitance of a mains rectifier for the same ripple, which is one of the reasons switch-mode supplies are small. What it does not tell is the cost of the small ripple: the smaller the ripple, the shorter and taller the current pulses that recharge the capacitor.

6.6 Capacitor filtering

Conduction angle and peak current

The capacitor charges only while the rectified source exceeds its voltage, in a short interval before each peak. All the charge the load takes in a half-cycle, $I_{dc}/(2f)$, must flow in during that interval, so the diode current is a pulse whose height is several times the DC load current. Its shape is set not by the capacitor but by the impedance in series with the source: the transformer's winding resistance and leakage inductance, the diodes' slope resistance, and the mains impedance. A stiff source gives a narrow, tall pulse; a soft source gives a wider, lower one. The quantities that heat the parts all depend on this pulse: the RMS secondary current (transformer copper loss), the capacitor's RMS ripple current (ESR heating), and the diode RMS current (the $r_T I_{rms}^2$ term of Chapter 2's loss model). None of them is given by the ripple formula, so they are computed numerically.

Listing 6.1 time-steps a bridge rectifier feeding a reservoir capacitor and a constant-current load (a regulator is close to one). The transformer is modelled as a sinusoidal EMF behind its winding resistance referred to the secondary, and each diode as 0.8 V plus 50 mΩ.

```
 # Bridge rectifier + reservoir capacitor + constant-current load, time-stepped
import numpy as np

def bridge(Vrms=18.0, f=50.0, Rs=1.0, Vd=0.8, rd=0.05, C=3300e-6, Iload=1.0,
           cycles=40, n=4000):
    dt = 1/(f*n)
    t = np.arange(cycles*n)*dt
    vs = np.sqrt(2)*Vrms*np.sin(2*np.pi*f*t)
    vc = np.zeros_like(t); i = np.zeros_like(t)
    v = 0.0                                # capacitor starts discharged
    for k in range(len(t)):
        drive = abs(vs[k]) - 2*Vd - v      # two diodes conduct per half-cycle
        i[k] = drive/(Rs + 2*rd) if drive > 0 else 0.0
        v = max(v + (i[k] - Iload)*dt/C, 0.0)
        vc[k] = v
    s = slice((cycles-2)*n, cycles*n)      # last two cycles: steady state
    vo, io = vc[s], i[s]
    return dict(Vdc=vo.mean(), Vmin=vo.min(), ripple=vo.max()-vo.min(),
                Ipk=io.max(), Isec=np.sqrt(np.mean(io**2)),
                Icap=np.sqrt(np.mean((io - Iload)**2)),
                cond=180*np.mean(io > 0),  # conduction angle per half-cycle
                PF=np.mean(np.abs(vs[s])*io)/(Vrms*np.sqrt(np.mean(io**2))))

for line in (0.9, 1.0, 1.1):
    r = bridge(Vrms=18.0*line)
    print(f"line {line:.1f}: Vdc={r['Vdc']:5.2f} Vmin={r['Vmin']:5.2f} "
          f"ripple={r['ripple']:4.2f} V  Ipk={r['Ipk']:4.2f} Isec={r['Isec']:4.2f} "
          f"Icap={r['Icap']:4.2f} A  cond={r['cond']:4.1f} deg  PF={r['PF']:4.2f}")
print(f"constant-discharge estimate I/(2fC) = {1.0/(2*50*3300e-6):.2f} V")
```

```
 line 0.9: Vdc=17.00 Vmin=16.03 ripple=1.92 V  Ipk=3.83 Isec=1.75 Icap=1.43 A  cond=71.2 deg  PF=0.78
line 1.0: Vdc=19.40 Vmin=18.41 ripple=1.95 V  Ipk=3.97 Isec=1.78 Icap=1.47 A  cond=68.6 deg  PF=0.76
line 1.1: Vdc=21.80 Vmin=20.79 ripple=1.99 V  Ipk=4.10 Isec=1.81 Icap=1.51 A  cond=66.4 deg  PF=0.75
constant-discharge estimate I/(2fC) = 3.03 V
```

Worked example — The sensor-hub supply, simulated. For the opening's supply, an 18 V RMS open-circuit secondary behind 1.0 Ω (a 30–40 VA transformer), 3300 µF and a 1 A load, at nominal mains:

- The DC output is 19.4 V with 1.95 V peak-to-peak ripple; the constant-discharge formula predicts 3.03 V, 55 % high, because the capacitor recharges for 69° of each 180° half-cycle and so discharges for only about 62 % of it.
- The diodes conduct in pulses of 3.97 A peak, four times the load current.
- The secondary carries 1.78 A RMS to deliver 1 A DC. A transformer rated in VA for its DC output of about 19 W would be overloaded; the winding must be rated for $18 \times 1.78 = 32$ VA, 1.7 times the DC power.
- The capacitor's RMS ripple current is 1.47 A, nearly one and a half times the load current.
- The power factor at the secondary is 0.76: distortion, not displacement, since the current pulses are centred on the voltage peaks.

These four currents, not the ripple voltage, decide the transformer's VA rating, the capacitor's ripple-current rating and the diode losses, and they explain both failures of the opening. The simulation also shows how strongly the source impedance matters: with the same parts but a stiffer 0.3 Ω source, the peak diode current rises to 5.5 A, the capacitor current to 1.84 A and the secondary current to 2.10 A, while the conduction angle narrows to 50°.

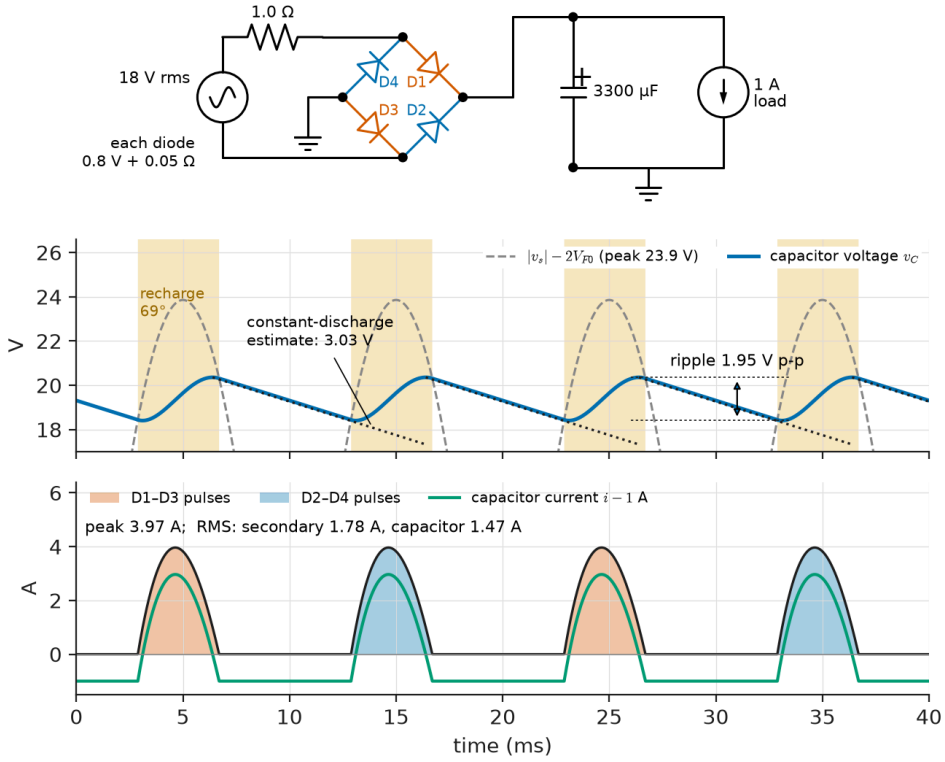


Figure 6.2 — Bridge rectifier with reservoir capacitor: output voltage, diode current pulses and capacitor current over two mains cycles.

Choosing the capacitor

An aluminium electrolytic reservoir capacitor is chosen for four ratings, not one:

1. **Capacitance**, from the ripple the load tolerates; allow for the −20 % tolerance and for the 10–20 % loss of capacitance over life.
2. **Voltage**, above the highest no-load peak at maximum mains with at least 10–20 % margin.
3. **Ripple current** at 100 or 120 Hz at the operating temperature, from simulation, since the ESR dissipation $I_{cap,rms}^2$ ESR heats the core.
4. **Life**, which electrolytic-capacitor makers specify as hours at the maximum rated temperature (2000 h at 105 °C for a general-purpose part, 5000–12 000 h for long-life types).

> **Status of the principle: Ten-degree rule for electrolytic-capacitor life**
> *Empirical.* Life roughly doubles for every 10 K reduction of core temperature below the rated maximum, an Arrhenius-type fit to the evaporation of electrolyte through the seal; it holds over roughly 40–105 °C and manufacturers add ripple-current and voltage terms of their own.

A 2000 h, 105 °C capacitor whose core runs at 65 °C has an expected life of about $2000 \times 2^{(105-65)/10} = 32\,000$ h, under four years of continuous service; if the ripple current and a hot enclosure push the core to 75 °C, the life halves again. That is the opening failure. A 10 000 h, 105 °C part at the same 65 °C would last about 1.6×10^5 h, eighteen years.

Inrush

At switch-on the capacitor is a short circuit. If the mains is applied at its peak, the initial current is limited only by the source and diode resistances: $(\sqrt{2} \times 19.8 - 1.6)/1.1 = 24$ A at high line for the sensor-hub supply. That must lie within the bridge's surge rating I_{FSM} and its I^2t (Chapter 2). In small transformer supplies the winding resistance usually suffices. In transformerless and switch-mode supplies, whose bulk capacitors charge directly from the rectified mains through milliohms, the inrush would be hundreds of amperes, so an **NTC thermistor** (5–10 Ω cold, falling to a fraction

of an ohm when hot), or a resistor bypassed by a relay or thyristor after start-up, limits it. A thermistor has a weakness: if the supply is switched off and on again before it cools, it no longer limits.

Choke-input and π filters

An inductor in series before the capacitor (the **choke-input** or LC filter) forces the rectifier current to flow over most of each half-cycle. Provided the inductance exceeds a critical value, the diode current never falls to zero, and the output tends to the average of the rectified wave, $2V_m/\pi$, rather than the peak. For a full-wave rectifier, the dominant second-harmonic ripple current, of peak $4V_m/(3\pi)/(2\omega L)$, must not exceed the DC current V_{dc}/R_L, which gives the critical inductance

$$L_c \approx \frac{R_L}{3\omega}$$

With $L > L_c$ the current is continuous, the output voltage is nearly independent of load, the peak and RMS currents fall close to the DC value, and the input power factor rises to about 0.9. The penalty is the inductor's size and weight at 50 Hz. Choke-input filters therefore survive in high-power battery chargers, valve amplifiers and passive power-factor correction of small drives, where a DC-link choke serves the same purpose (Chapter 31). The **π filter** (C–L–C or C–R–C) follows a reservoir capacitor with a second LC or RC section that attenuates the ripple by a further factor of roughly $1/(\omega_r^2 LC)$ at the ripple frequency ω_r, used where hum must be very low and a regulator is not, such as audio preamplifier rails.

6.7 Clipping circuits

Series and shunt clippers

A **clipper** (limiter) removes the part of a waveform above or below a set level. In a **shunt clipper** a diode, in series with a bias voltage V_B, is placed across the output, fed through a series resistor R: while the input is below $V_B + V_D$ the diode is off and the output follows the input; above it, the diode conducts and the output stays at about $V_B + V_D$, the excess dropping across R. A **series clipper** places the diode in the signal path, passing only the part of the waveform that forward-biases it. Two shunt branches of opposite polarity clip both sides; two Zener diodes in series, anode to anode, clip symmetrically at $\pm(V_Z + V_D)$ without bias sources.

The transfer characteristic of a clipper (Figure 6.3) is linear with unit slope in the pass region and nearly flat in the clipped region, with a slope equal to $r_d/(R + r_d)$, the divider formed by the diode's small-signal resistance and the series resistor. Because the diode's knee is exponential, not sharp, the transition is rounded over about 0.1–0.2 V, which matters for small signals. For precise clipping at small levels, the diode is placed inside an op-amp's feedback loop (the precision limiters and rectifiers of Chapter 8).

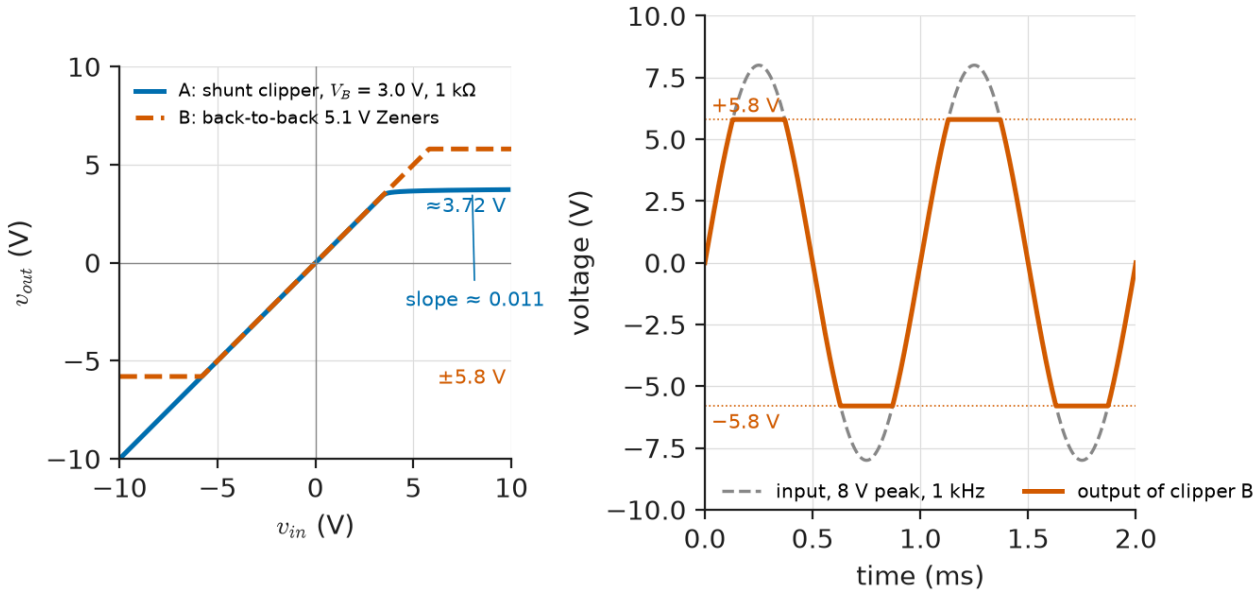


Figure 6.3 — Transfer characteristics of diode clippers: biased shunt clipper and symmetrical Zener clipper, with an input and clipped output waveform.

Input protection as a clipper

The most common clipper in modern electronics is not a waveshaper but a protection network: a series resistor and two diodes to the supply rails at the input of an ADC, a microcontroller pin or an op-amp. The input is clipped at about one diode drop above the positive rail and below ground, and the series resistor limits the fault current. Microcontroller inputs contain such diodes internally, rated for an injected current of typically ±5 mA per pin, beyond which the injected current can disturb neighbouring channels or trigger latch-up. External Schottky diodes, with a lower forward drop than the internal silicon ones, take the current first. Against fast transients (ESD, surge) the same arrangement is augmented by the TVS and steering arrays of Chapter 2.

Worked example — Protecting a 3.3 V ADC input against a 24 V miswire. A 0–3.3 V ADC input on an industrial controller must survive an accidental connection to the 24 V field supply, of either polarity. Use a 4.7 kΩ series resistor and a BAT54S-class dual Schottky to the 3.3 V rail and ground (drop about 0.3 V at a few milliamperes).

- Positive fault: $(24 - 3.3 - 0.3)/4.7\ \text{k}\Omega = 4.34$ mA flows into the 3.3 V rail. The pin sees about 3.6 V. If the board's 3.3 V regulator cannot sink current and the rest of the circuit draws less than 4.34 mA, the fault pumps the rail up; a Zener or TVS on the rail, or a minimum load, prevents it.
- Negative fault: $(24 - 0.3)/4.7\ \text{k}\Omega = 5.04$ mA through the lower diode to ground.
- Resistor dissipation: at most $24^2/4.7\ \text{k}\Omega = 0.12$ W if the diode end were shorted, so a 0.25 W part (or a pulse-rated 0805) is adequate.

The protection costs accuracy. A Schottky's reverse leakage rises steeply with temperature, from a few hundred nanoamperes at 25 °C to several microamperes at 85 °C. At 5 µA, the 4.7 kΩ resistor drops $5\ \mu\text{A} \times 4.7\ \text{k}\Omega = 23.5$ mV, which is 29 LSB of a 12-bit, 3.3 V converter (0.81 mV/LSB), varying with temperature. Low-leakage silicon diodes (BAV199 class, picoamperes typical) or a smaller series resistor with a TVS, or an op-amp buffer with its own clamp, restore accuracy. Protection, leakage and source impedance must be designed together.

6.8 Clamping circuits

The DC restorer

A **clamper** shifts a waveform's DC level without changing its shape, so that one extreme sits at a chosen voltage. The basic circuit is a series capacitor followed by a shunt diode to ground (Figure 6.4). On the first negative excursion the diode conducts and charges the capacitor to the negative peak; thereafter the capacitor holds that charge and the output is the input shifted up by $V_m - V_D$, so that its negative peak sits at $-V_D$ (about −0.6 V for silicon, −0.3 V for Schottky). Reversing the diode clamps the positive peak; adding a bias source in series with the diode clamps the extreme at $V_B \pm V_D$.

> **Status of the principle: Ideal clamper level shift**
> *Approximate.* With an ideal diode and $RC \gg T$, the capacitor charges to the peak and the waveform is shifted rigidly; with a load resistance R the capacitor discharges between peaks, so each cycle sags by about $(V_{pp} - V_D)T_{off}/(RC)$ and the diode must restore the lost charge, rounding the clamped extreme.

The design condition is that the capacitor must not discharge appreciably through the load between the diode's conduction intervals: RC must be large compared with the signal period. The fractional droop over an interval t is about $t/(RC)$.

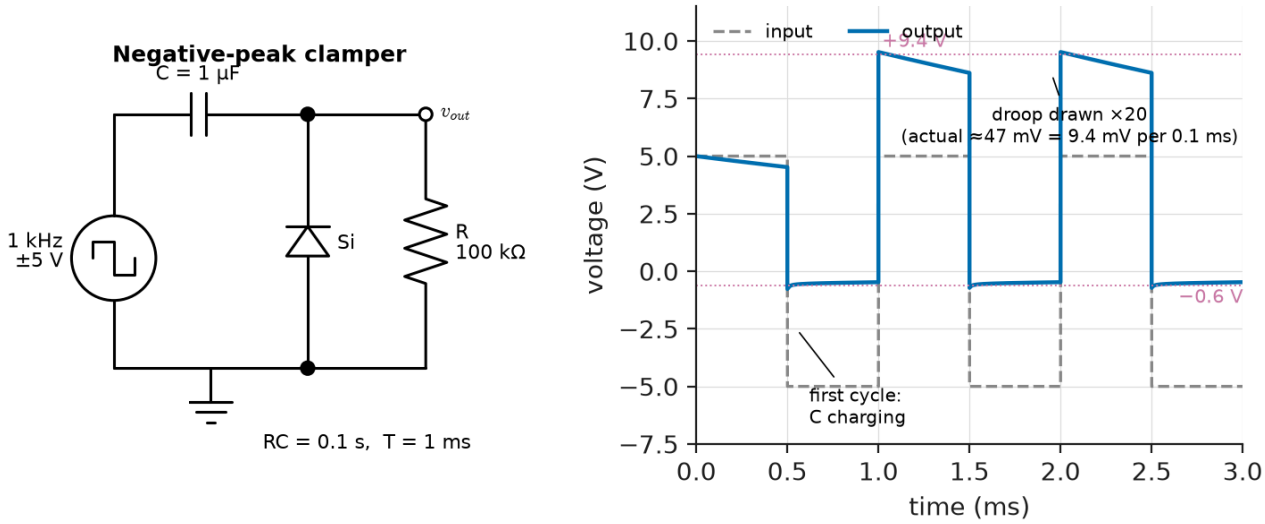


Figure 6.4 — Negative-peak diode clamper (DC restorer): circuit and input and output waveforms.

Worked example — Droop of a clamped square wave. A 1 kHz square wave of 10 V peak-to-peak is clamped by a 1 µF capacitor into a 100 kΩ load ($RC = 0.1$ s, 100 times the period). During each 0.5 ms half-cycle in which the diode is off, the output, about $V_{pp} - V_D \approx 9.4$ V above the clamp level, sags by about $9.4\ \text{V} \times 0.5\ \text{ms}/0.1\ \text{s} \approx 47$ mV, 0.5 % of the swing, and the diode restores the charge at the start of the next negative half-cycle. With a 10 nF capacitor the sag would be 5 V and the clamp would no longer work. For signals with a long, variable duty cycle, such as a video line or a data stream with long runs, the time constant must cover the longest interval between clamping events.

Where clampers are used

- **Gate drives with negative off-bias.** A gate driver supplied from a single 0–20 V rail can produce +15 V on and −5 V off for a SiC MOSFET or IGBT through a coupling capacitor and a 5.1 V Zener clamp from gate to source: the capacitor charges to the Zener voltage and shifts the drive down by that amount. The same principle sets the DC level in pulse-transformer gate drives. Its limits, the duty-cycle dependence and the start-up transient, are discussed with gate drives in Chapter 29.

- **DC restoration of AC-coupled signals**: video sync-tip clamps, optical-receiver baseline restoration, and the input of diode detectors.
- **Voltage multipliers**, which are clampers followed by peak detectors (Section 6.9).

6.9 Voltage multipliers

The half-wave doubler

A clamper followed by a peak detector doubles the voltage (Figure 6.5a). The clamper (C1, D1) shifts the input so that it swings from about 0 to $2V_m$; the peak detector (D2, C2) holds the peak, $2V_m$ less two diode drops at no load. This is the **Greinacher** or **Villard cascade** doubler. Its ripple is at the input frequency, and its output sags under load faster than a rectifier's because the charge reaches C2 through C1. The **full-wave (Delon) doubler** instead charges two series capacitors on alternate half-cycles, each to V_m; it was used in the switchable 115/230 V input stage of older PC supplies, where at 115 V a link turned the bridge into a doubler to deliver the same 325 V bus.

The Cockcroft–Walton cascade

Stacking N doubler stages gives the **Cockcroft–Walton** multiplier, with an ideal no-load output of $2NV_m$ (Figure 6.5b). Each diode and capacitor sees only $2V_m$, so a high voltage is built from low-voltage parts. Its weakness is output impedance: the load charge passes through every stage in series, and with equal capacitors C, supply frequency f and load current I, the voltage drop and peak-to-peak ripple are approximately

$$\Delta V \approx \frac{I}{fC}\left(\frac{2N^3}{3} + \frac{N^2}{2} - \frac{N}{6}\right), \qquad \delta V \approx \frac{I}{fC}\cdot\frac{N(N+1)}{2}$$

> **Status of the principle: Cockcroft–Walton droop and ripple**
> *Approximate.* It follows from charge balance through the ladder with ideal diodes, equal capacitors and a ripple small compared with the stage voltage; stray capacitance from the upper stages to ground, diode drops and capacitor tolerance make real droop larger, especially for large N.

The cube of N is the dominant fact: doubling the number of stages multiplies the droop by about eight. Cockcroft–Walton multipliers are therefore driven at tens of kilohertz from a small inverter and transformer, and kept to as few stages as the transformer insulation allows.

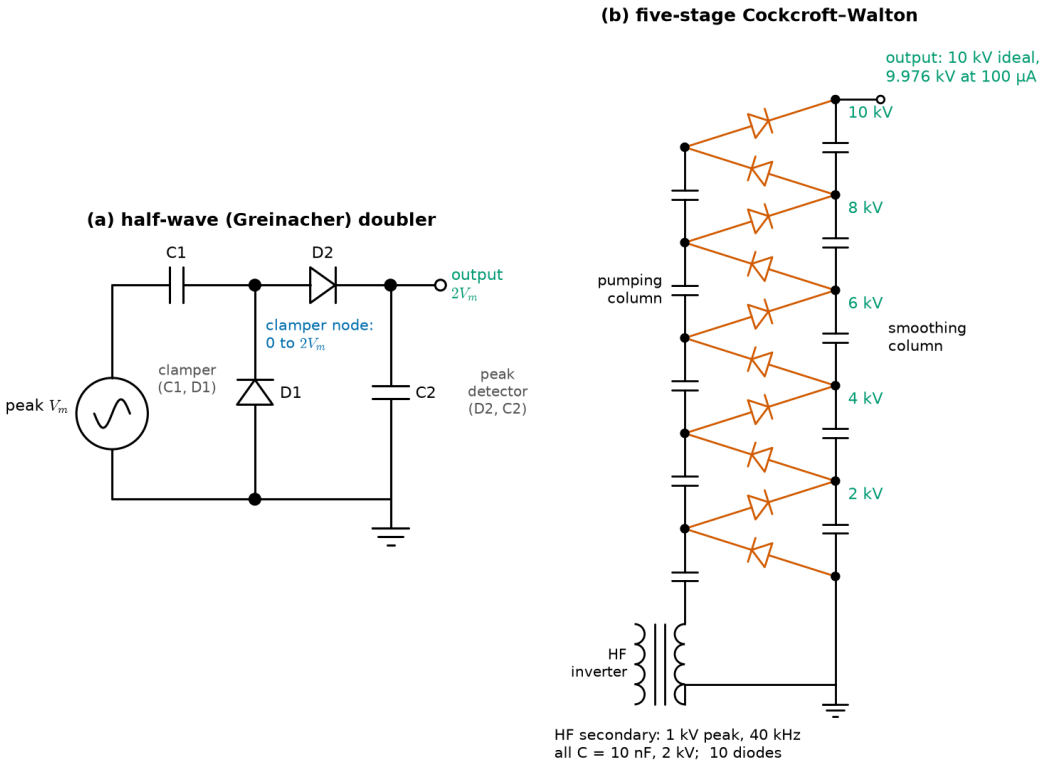


Figure 6.5 — Voltage multipliers: half-wave (Greinacher) doubler and a five-stage Cockcroft-Walton cascade.

Worked example — A 10 kV supply for a photomultiplier or ionizer. A five-stage Cockcroft–Walton multiplier is driven at $f = 40$ kHz with $V_m = 1$ kV, using 10 nF capacitors, and delivers 100 µA. The scale factor is $I/(fC) = 10^{-4}/(4 \times 10^{4} \times 10^{-8}) = 0.25$ V. The droop is $0.25 \times (83.33 + 12.5 - 0.83) = 0.25 \times 95 = 23.75$ V and the ripple $0.25 \times 15 = 3.75$ V peak-to-peak, so the output is 9.976 kV with 0.04 % ripple: excellent. Run from the 50 Hz mains instead, the same ladder would droop by $0.25 \times 800 \times 95 = 19$ kV, more than its whole output: the multiplier would deliver nothing useful. Every practical high-voltage multiplier, from X-ray generators and electrostatic precipitators to the bias supplies of photomultiplier tubes and CRT anodes, runs at high frequency for this reason. At the other end of the scale, the same diode–capacitor pumping, with transistors replacing the diodes, generates gate-drive and flash-memory programming rails inside integrated circuits (charge pumps, Chapter 30).

6.10 Practical rectifier design

A procedure

A transformer–rectifier–capacitor supply is designed backwards from the load, at the worst corners of mains voltage and temperature:

1. **Specify** the load: output voltage and current, the regulator's minimum input voltage (output plus dropout), ripple tolerance, ambient temperature and life.
2. **Choose the topology**: a bridge for most supplies; a centre-tapped full-wave rectifier with Schottky diodes for low output voltages; half-wave only for milliamperes.
3. **Choose the transformer voltage** so that the ripple trough at minimum mains (230 V −10 % per IEC 60038) stays above the regulator's minimum input, allowing for diode drops and the winding's regulation.
4. **Choose the capacitor** for that trough, then simulate (or use published design curves) to obtain the peak, RMS secondary and RMS capacitor currents.
5. **Rate the transformer** by the RMS secondary current, not the DC power.
6. **Rate the diodes or bridge**: average current, the loss $V_{F0} I_{av} + r_T I_{rms}^2$ per diode with its heatsinking, PIV with a margin of at least two, I_{FSM} and I^2t for inrush.

7. **Rate the capacitor** for voltage at maximum mains no-load, ripple current, and life at its core temperature.
8. **Check the regulator's dissipation** at maximum mains, where the input is highest.
9. **Add protection and compliance**: a primary fuse (time-lag, to ride through transformer magnetising inrush) and often a secondary fuse; a bleeder resistor across large capacitors; harmonic limits (IEC 61000-3-2, which sets no limits for most equipment of 75 W or less input power) and safety (IEC 62368-1 for ICT and audio-video equipment, IEC 61010-1 for instruments). Small RC snubbers across the bridge diodes are sometimes added to damp the ringing of the transformer's leakage inductance with the diodes' capacitance at turn-off, a source of conducted interference in sensitive receivers.

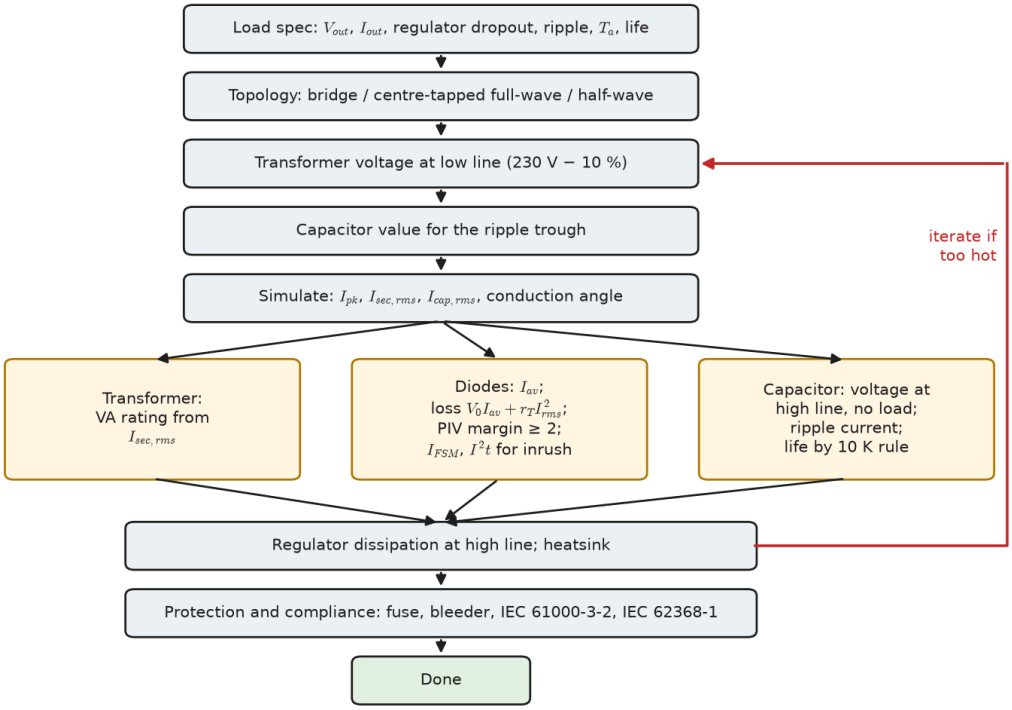


Figure 6.6 — Design procedure for a transformer-rectifier-capacitor power supply.

Worked example — Completing the sensor-hub supply. The load is a 7812-class regulator delivering 12 V at 1 A. Its dropout at 1 A is about 2 V, so the input must never fall below 14 V; the procedure is followed in Figure 6.6.

Transformer voltage. Using Listing 6.1 at 0.9 of nominal mains with 3300 µF, a 15 V secondary gives a trough of 12.5 V, below the 14 V needed, and even 16 V with 4700 µF only just reaches 14.0 V. An 18 V secondary gives a trough of 16.03 V at low line, a 2 V margin, with 3300 µF.

Transformer rating. The secondary RMS current is 1.81 A at high line, so the winding must be rated about $18 \times 1.81 = 33$ VA; a 40 VA transformer, which also has lower winding resistance, is the sensible choice. The original design's 15 VA part, rated on DC power, ran at more than twice its rated current, hence its 85 °C.

Diodes. Each diode carries an average of 0.5 A and an RMS current of $1.78/\sqrt{2} = 1.26$ A. With 0.8 V plus 50 mΩ, the loss per diode is $0.8 \times 0.5 + 0.05 \times 1.26^2 = 0.48$ W, about 1.9 W for the bridge, which a 2 A in-line bridge (KBP class, $I_{FSM} = 60$ A) dissipates at a modest temperature rise on a board, or a GBU-class bridge with a tab handles with margin. The highest no-load peak is $1.1 \times 18\sqrt{2} = 28$ V, so a 100 V bridge gives ample PIV margin against mains surges. The worst-case inrush of 24 A lies within I_{FSM}.

Capacitor. 3300 µF, 35 V (no-load peak at high line about 26.4 V after the diode drops), with a ripple-current rating of at least 1.5 A at 100 Hz and 105 °C, and a long-life (10 000 h) series, for the reasons computed in Section 6.6.

Regulator. At high line the average input is 21.8 V, so the regulator dissipates $(21.8 - 12) \times 1 = 9.8$ W. To keep its junction at 125 °C in a 50 °C enclosure requires a total junction-to-ambient resistance of $(125 - 50)/9.8 = 7.7$ K/W; with

about 5 K/W junction-to-case for a TO-220 regulator, the heatsink and interface must provide 2.7 K/W or better, using the thermal network of *Engineering Physics* Ch. 14 §14.7. The supply's efficiency is $12/(21.8 \times 1 + 1.9 +$ transformer losses), about 45 %: the price of a linear design, and the reason a switching pre-regulator or a complete switch-mode supply is chosen above a few watts (Chapter 30).

Protection and compliance. The input power is about 30 W, below the 75 W threshold of IEC 61000-3-2, so no harmonic limits apply despite the 0.76 power factor at the secondary. A time-lag primary fuse of about 250 mA rides through the transformer's magnetising inrush; a 2.2 kΩ bleeder across the capacitor discharges it within a minute of disconnection for service safety.

> **Status of the principle: Worst-corner design of rectifier supplies**
> *Derived*. Each limit (regulator headroom at low line, regulator and capacitor heating at high line, PIV and inrush at high line no-load) is a monotonic function of the supply voltage, so checking the extreme corners bounds every intermediate case; it presumes that mains tolerance, component tolerance and ambient temperature are specified, which is a design input, not a result.

The redesign costs a larger transformer and a heatsink, and its numbers come from three places: the circuit topology of Sections 6.1–6.4, the filter behaviour of Sections 6.5–6.6 computed rather than estimated, and the device ratings of Chapter 2. That combination, device ratings plus circuit computation at worst-case corners, is the method of every chapter of Part II.

What remains open

Open question	Where it is answered
How do transistors amplify the signals that these supplies power?	Chapter 7
How do op-amps make precision rectifiers and limiters that remove the diode drop?	Chapter 8
How do linear regulators and references turn the rectified, filtered voltage into a stable rail?	Chapter 10
How are thyristor-controlled and three-phase rectifiers designed, and how is their harmonic current filtered?	Chapter 31
How do switch-mode converters and charge pumps replace transformer–rectifier supplies?	Chapter 30
How do rectifier input currents distort the supply, and how is the distortion power factor measured and limited?	*Circuits, Signals and Communications*, Ch. 9; *Electrical Energy and Machines*, Ch. 25

Next: Chapter 7 turns from the two-terminal diode to the three-terminal transistor and builds single- and multistage amplifiers on the bias and small-signal models of Chapters 3 and 4.

Chapter 7 — Transistor Amplifiers

Part II · Analog Electronics

A car-audio supplier builds a discrete preamplifier for a head unit. On the bench, with a signal generator at the input and an oscilloscope probe at the output, the single common-emitter stage gives 46 dB of gain, flat to 1.5 MHz. Installed in the radio it gives 41 dB, its response falls off above 690 kHz, and a 1 V output carries 7 % second harmonic, enough to be heard on a sustained piano note. Nothing in the stage is faulty. The bench probe had a 10 MΩ input; the next stage presents 10 kΩ, so the stage was loaded. The generator had 50 Ω output resistance; the real source is a 600 Ω tuner output, so the base–collector capacitance, multiplied by the stage's own gain, now forms a low-pass filter with a resistance twelve times larger. And the 7 mV signal at the base is no longer small compared with the 26 mV thermal voltage that sets the curvature of the transistor's exponential law. Loading, the Miller effect and nonlinearity are the three things that separate a transistor amplifier on paper from one in a product, and all three are predictable from the device models of Part I.

> **How are the gain, input impedance and output impedance of a transistor stage computed, and how do they change when the stage is connected to a real source and load?**
> **What sets an amplifier's bandwidth and its distortion, and what does an engineer do to improve each?**

Concept	Engineering destination
Single-stage configurations	Choosing CE/CS for gain, CB/CG for wide band and low input impedance, CC/CD for buffering
Common-emitter amplifier	Microphone and sensor preamplifiers, RF gain blocks, discrete audio stages
Common-source amplifier	Electret microphones, JFET instrument inputs, every gain stage inside a CMOS chip
Emitter and source followers	Line drivers, ADC drivers, output stages, level shifters
Multistage amplifiers	Preamplifier chains, cascode and Darlington stages, the inside of every op-amp
Voltage, current and power gain	Link budgets, sensor-chain gain allocation, decibel bookkeeping
Input and output impedance	Loading between stages, bootstrapped inputs, matching to cables and antennas
Small-signal analysis	Hand design and simulator checks of any transistor circuit
Frequency response	Coupling-capacitor sizing, Miller effect, wideband and video amplifiers
Distortion	Audio THD specifications, intermodulation in receivers, clipping and headroom

Roadmap. The amplifier as a two-port and the three single-stage configurations (7.1) → a systematic small-signal method with a nodal solver (7.2) → the common-emitter stage, bypassed and degenerated (7.3) → the common-source stage and how it differs (7.4) → followers (7.5) → cascades, cascodes and Darlingtons (7.6) → input and output impedance as design variables (7.7) → frequency response at both ends of the band (7.8) → distortion, clipping and headroom (7.9).

7.1 Single-stage amplifiers

> **Boundary.**
> The BJT and MOSFET models used here (the exponential and square laws, the hybrid-π small-signal model with g_m, r_π, r_o, C_π, C_μ, and DC bias with load lines) are developed in Chapter 3 and Chapter 4, building on *Engineering Physics*, Ch. 18 §18.3.2–18.3.3. Two-port parameters and terminated-two-port formulas are from *Circuits, Signals and Communications*, Ch. 12; Bode plots and poles from Ch. 13. This chapter uses those models to design amplifier stages; feedback as a general principle is the subject of Chapter 9.

The amplifier as a two-port

Whatever is inside, a linear amplifier stage seen from its terminals is a two-port with an input resistance R_{in}, an output resistance R_{out} and a controlled source. Book 3 gives the full h-, y- and ABCD-parameter description (Ch. 12 §12.10); for design the **unilateral voltage-amplifier model** is almost always enough: an input resistance R_{in}, and a Thévenin output made of the open-circuit voltage gain A_{vo} times the input voltage behind R_{out}. Driven from a source v_s with resistance R_s and loaded by R_L,

$$\frac{v_o}{v_s} = \frac{R_{in}}{R_s + R_{in}}\, A_{vo}\, \frac{R_L}{R_{out} + R_L}$$

Two voltage dividers sit on either side of the intrinsic gain. Every "lost" decibel in the opening story is one of these two fractions. The model is unilateral, meaning nothing at the output affects the input; at high frequency the base–collector capacitance breaks this assumption (Section 7.8), and Book 3 shows the size of the error at low frequency to be a few per cent.

> **Status of the principle: Unilateral two-port amplifier model**
> *Approximate.* Exact for an ideal controlled source; for a transistor stage it neglects reverse transmission (through C_μ and r_μ), which is small at low frequency and becomes the Miller effect at high frequency.

Voltage, current and power gain

Three gains describe a stage, and a datasheet or a specification may quote any of them:

- **Voltage gain** $A_v = v_o/v_i$, or from the source, $A_{vs} = v_o/v_s$; open-circuit A_{vo} when $R_L \to \infty$.
- **Current gain** $A_i = i_o/i_i$. With $i_i = v_i/R_{in}$ and $i_o = v_o/R_L$, $A_i = A_v R_{in}/R_L$.
- **Power gain** $A_p = P_o/P_i = A_v A_i$, which is what an amplifier really provides: a passive transformer can step up voltage or current, but not both.

In decibels, A_v and A_i use $20\log_{10}$ and A_p uses $10\log_{10}$. The voltage-gain decibel equals the power-gain decibel only when $R_{in} = R_L$, which is true in RF systems built around 50 Ω but almost never true in a sensor chain. A common-emitter stage with $|A_v| = 137$ (42.8 dB), $R_{in} = 2.39$ kΩ and $R_L = 10$ kΩ has $|A_i| = 32.8$ (30.3 dB) and $A_p = 4515$ (36.5 dB); all three numbers are correct and they differ because the impedance levels at the two ports differ (Worked example in Section 7.3).

The three configurations

A transistor has three terminals; one is common to input and output in the small-signal circuit, which gives three basic stages for each device type. Table 7.1 collects their approximate properties, ignoring r_o except where it dominates, with $r_e = 1/g_m$ (strictly α/g_m) for the BJT.

Table 7.1 — The single-stage configurations (small-signal, midband, r_o neglected unless noted).

Stage	Voltage gain	Input resistance	Output resistance	Typical use
Common emitter (CE), R_E bypassed	$-g_m R'_C$	r_π	$R_C \parallel r_o$	Voltage gain
CE with unbypassed R_E	$\approx -R'_C/(r_e + R_E)$	$r_\pi + (\beta+1)R_E$	$\approx R_C$	Stable, linear gain
Common base (CB)	$+g_m R'_C$	$\approx r_e$	R_C	Wideband, cascode, current buffer
Common collector (CC, emitter follower)	$\approx R'_E/(r_e + R'_E)$	$(\beta+1)(r_e + R'_E)$	$r_e + R_s/(\beta+1)$	Voltage buffer
Common source (CS)	$-g_m R'_D$	gate: $\approx \infty$	$R_D \parallel r_o$	Gain, high-Z input
Common gate (CG)	$+g_m R'_D$	$\approx 1/g_m$	R_D	Wideband, cascode, RF input
Common drain (CD, source follower)	$\approx g_m R'_S/(1 + g_m R'_S)$	$\approx \infty$	$\approx 1/g_m$	Buffer, level shift

Here a prime denotes the total small-signal resistance at that node ($R'_C = R_C \parallel R_L \parallel r_o$ and so on). In the table the "$\parallel$" symbol means parallel combination. The pattern is general: the terminal where the signal enters and the terminal where it leaves decide the stage. Signal into the base or gate and out of the collector or drain gives an inverting gain stage; into the emitter or source and out of the collector or drain gives a non-inverting stage with low input resistance; into the base or gate and out of the emitter or source gives a follower with gain just below one.

The common-base and common-gate stages appear in this chapter mainly as halves of the cascode (Section 7.6) and as low-impedance inputs: a common-gate low-noise amplifier presents $1/g_m = 50\ \Omega$ to an antenna when $g_m = 20$ mS, matching it without an inductor, and a common-base stage is the classic transimpedance input for a photodiode, holding the diode's capacitance at a near-constant voltage.

Table 7.2 — Representative small-signal parameters ($\beta = 150$, $V_A = 100$ V for the BJT; overdrive $V_{ov} = 0.2$ V and $\lambda = 0.1\ \text{V}^{-1}$ for the MOSFET; 300 K).

Bias current	BJT g_m	BJT r_π	BJT r_o	MOSFET $g_m = 2I_D/V_{ov}$	MOSFET r_o
0.1 mA	3.87 mS	38.8 kΩ	1 MΩ	1 mS	100 kΩ
1 mA	38.7 mS	3.88 kΩ	100 kΩ	10 mS	10 kΩ
10 mA	387 mS	388 Ω	10 kΩ	100 mS	1 kΩ

The BJT's transconductance per ampere, $g_m/I_C = 1/V_T = 38.7\ \text{V}^{-1}$, is fixed by physics; the MOSFET's, $2/V_{ov}$, is a design choice that is always smaller in strong inversion ($10\ \text{V}^{-1}$ at 0.2 V overdrive). That single difference explains most of the contrasts between Sections 7.3 and 7.4.

7.2 Small-signal analysis

The procedure

Every linear transistor-amplifier calculation follows the same five steps. They are worth stating because most errors come from skipping one.

1. **DC analysis.** Short the signal source, open all capacitors and find the operating point (I_C, V_{CE} or I_D, V_{DS}). Check the region: $V_{CE} > V_{CE(sat)}$ (about 0.2 V) for a BJT, $V_{DS} > V_{ov}$ for a MOSFET.

2. **Parameters.** From the bias, compute $g_m = I_C/V_T$ (BJT) or $2I_D/V_{ov}$ (MOSFET), $r_\pi = \beta/g_m$, $r_o = V_A/I_C$ or $1/(\lambda I_D)$; take C_π, C_μ (or C_{gs}, C_{gd}) from the datasheet's f_T and output capacitance.
3. **Small-signal circuit.** Replace every DC voltage source by a short and every DC current source by an open; replace coupling and bypass capacitors by shorts in the midband (or keep them for low-frequency analysis); replace the transistor by its hybrid-π model.
4. **Solve.** Nodal analysis, or for simple stages the rules of Table 7.1.
5. **Check.** Is the signal small? For a BJT, the base–emitter signal must be well below V_T (Section 7.9). Does the output swing stay inside the bias headroom?

> **Status of the principle: Small-signal linearisation**
> *Approximate.* Replacing a device by the first-order Taylor expansion of its characteristic around the operating point is exact in the limit of vanishing signal; for a BJT the second-order term is $v_{be}/(2V_T)$ of the first, so the linear model is good to 1 % only for v_{be} below about 0.5 mV, and acceptable for gain and impedance (not for distortion) up to a few millivolts.

A nodal solver for transistor stages

Step 4 is ordinary nodal analysis with one extra element, the voltage-controlled current source $g_m v_\pi$, which makes the admittance matrix non-symmetric. A dozen lines of Python stamp the elements and solve at any frequency; the same function then gives the midband gain and, by sweeping frequency, the bandwidth. Listing 7.1 does this for the common-emitter stage designed in Section 7.3, including the base spreading resistance r_x and both capacitances (Figure 7.1 shows the circuit it solves).

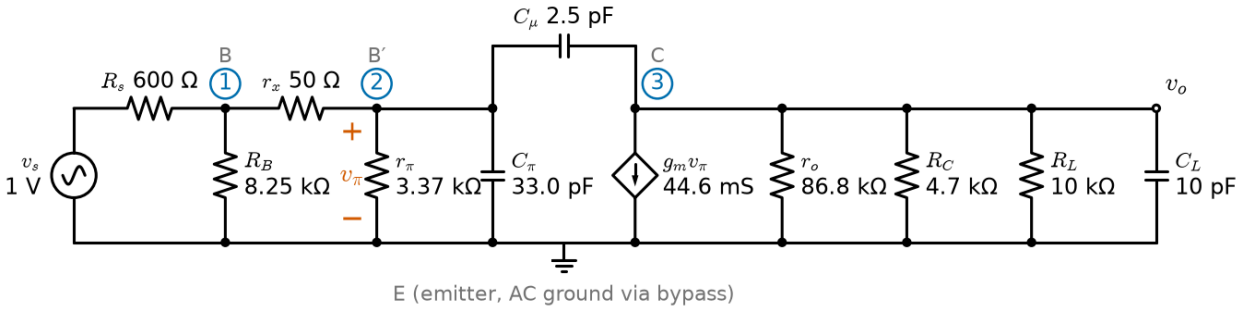


Figure 7.1 — Small-signal hybrid-pi equivalent of the common-emitter stage of Worked example 7.1, as solved by Listing 7.1.

```
 import numpy as np

# Hybrid-pi CE stage of Worked example 7.1 (2N3904, IC = 1.15 mA)
VT, beta, VA, IC = 0.02585, 150, 100.0, 1.152e-3
gm, rpi, ro = IC / VT, beta * VT / IC, VA / IC
rx, Cmu, fT = 50.0, 2.5e-12, 200e6          # base resistance, C_mu, f_T at 1 mA
Cpi = gm / (2 * np.pi * fT) - Cmu
Rs, RB, RC, RL, CL = 600.0, 8.246e3, 4.7e3, 10e3, 10e-12

def gain(f):
    """Nodal analysis: node 1 = base pin, 2 = internal base b', 3 = collector."""
    s = 2j * np.pi * f
    Y = np.zeros((3, 3), complex)
    def stamp(a, b, y):                      # admittance between nodes a, b (0 = ground)
        for n in (a, b):
            if n: Y[n-1, n-1] += y
        if a and b:
            Y[a-1, b-1] -= y; Y[b-1, a-1] -= y
    stamp(1, 0, 1/Rs + 1/RB); stamp(1, 2, 1/rx)
    stamp(2, 0, 1/rpi + s*Cpi); stamp(2, 3, s*Cmu)
    stamp(3, 0, 1/ro + 1/RC + 1/RL + s*CL)
    Y[2, 1] += gm                            # VCCS gm*v2 flowing from node 3 to ground
    i = np.array([1/Rs, 0, 0])               # Norton equivalent of a 1 V source behind Rs
    return np.linalg.solve(Y, i)[2]

f = np.logspace(3, 9, 6001)
A = np.array([abs(gain(x)) for x in f])
fH = f[np.argmax(A < A[0] / np.sqrt(2))]
print(f"Cpi = {Cpi*1e12:.1f} pF, midband |vo/vs| = {A[0]:.1f} ({20*np.log10(A[0]):.1f} dB)")
print(f"-3 dB upper corner fH = {fH/1e3:.0f} kHz")
```

Running it prints $C_\pi = 33.0$ pF, a midband gain from the source of 108.5 (40.7 dB) and an upper corner of 689 kHz. The hand analysis of Section 7.3, which ignores r_x, gives 109.9; the 1 % difference is the 50 Ω base resistance in series with the source. The solver's value is not "more correct" than the hand result in any useful sense, because β alone varies by a factor of three between parts; its value is that it confirms the hand model and handles the capacitances that hand analysis approximates (Section 7.8). A circuit simulator does exactly this, with nonlinear models re-linearised at the computed operating point.

7.3 Common-emitter amplifiers

The bypassed stage

Figure 7.2 shows the classic discrete common-emitter amplifier: a voltage-divider bias (R_1, R_2), an emitter resistor R_E that stabilises the DC operating point, a collector load R_C, coupling capacitors C_1 and C_2 and an emitter bypass capacitor C_E that removes R_E from the signal path. The bias design, with the rule that the divider current should be about ten times the base current so that β barely affects I_C, belongs to Chapter 3. The signal analysis is the subject here.

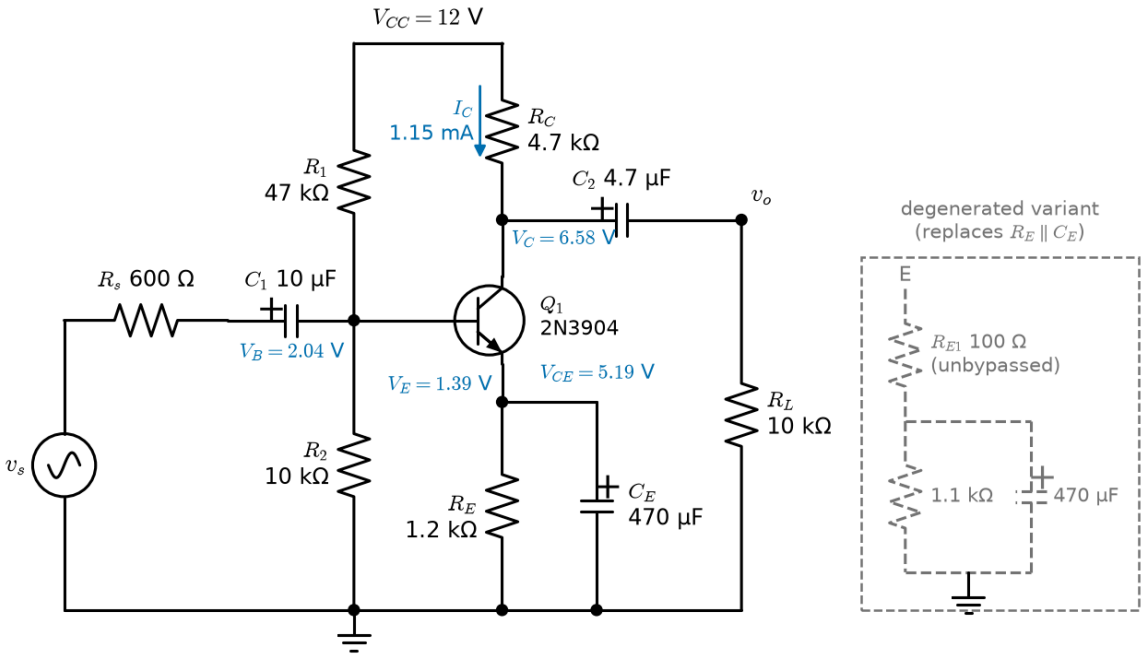


Figure 7.2 — Discrete common-emitter amplifier with divider bias and emitter bypass, with the values of Worked example 7.1.

With C_E a short at signal frequencies, the emitter is at AC ground and the stage is the textbook $A_v = -g_m R'_C$ of *Engineering Physics* §18.3.2, with $R'_C = R_C \parallel R_L \parallel r_o$. The input resistance is $R_1 \parallel R_2 \parallel r_\pi$, and the output resistance seen by the load is $R_C \parallel r_o$. The unloaded gain $-g_m(R_C \parallel r_o)$ can be written $-I_C(R_C \parallel r_o)/V_T$: the DC voltage across the load divided by 25.85 mV. A stage with 5 V across its collector resistor can never give more than about 190 without an active load, whatever transistor is used; that ceiling is why integrated amplifiers replace R_C by a current-source load (Chapter 10), and the absolute ceiling is the intrinsic gain $g_m r_o = V_A/V_T$, about 3900 for $V_A = 100$ V.

Worked example 7.1 — A 2N3904 preamplifier stage. Take Figure 7.2 with $V_{CC} = 12$ V, $R_1 = 47$ kΩ, $R_2 = 10$ kΩ, $R_E = 1.2$ kΩ (bypassed), $R_C = 4.7$ kΩ, a 2N3904 with $\beta = 150$, $V_{BE} = 0.65$ V and $V_A = 100$ V, a source with $R_s = 600$ Ω and a load of $R_L = 10$ kΩ (the next stage).

DC. The divider's Thévenin equivalent is $V_{Th} = 12 \times 10/57 = 2.105$ V behind $R_{Th} = 8.25$ kΩ. Then

$$I_B = \frac{V_{Th} - V_{BE}}{R_{Th} + (\beta + 1)R_E} = \frac{1.455}{8.25\ \text{k}\Omega + 181.2\ \text{k}\Omega} = 7.68\ \mu\text{A}, \qquad I_C = \beta I_B = 1.152\ \text{mA}$$

so $V_E = 1.39$ V, $V_C = 12 - 1.152 \times 4.7 = 6.58$ V and $V_{CE} = 5.19$ V: forward active, with room to swing.

Parameters. $g_m = 1.152/25.85 = 44.6$ mS, $r_\pi = 150/g_m = 3.37$ kΩ, $r_o = 100/1.152$ mA $= 86.8$ kΩ.

Gains. $R'_C = 4.7 \parallel 10 \parallel 86.8$ kΩ $= 3.08$ kΩ, so $A_v = -44.6$ mS $\times\ 3.08$ kΩ $= -137.5$ (42.8 dB). Unloaded, $A_{vo} = -g_m(R_C \parallel r_o) = -198.7$; the 10 kΩ load costs 3.2 dB. $R_{in} = 8.25 \parallel 3.37$ kΩ $= 2.39$ kΩ, so the source divider gives $2.39/2.99 = 0.80$ and $A_{vs} = -109.9$ (40.8 dB). The current gain is $A_i = A_v R_{in}/R_L = -32.8$ and the power gain $A_p = 4515$ (36.5 dB). The output resistance is $R_C \parallel r_o = 4.46$ kΩ.

Sensitivity. With $\beta = 100$ the bias gives $I_C = 1.124$ mA and $A_v = -134.2$; with $\beta = 300$, $I_C = 1.182$ mA and $A_v = -140.8$. The stiff bias holds the gain within ± 2.5 % over a threefold spread of β. Temperature is worse: $g_m = I_C/V_T$ falls as $1/T$ at constant current, about 0.33 % per kelvin; the bias current rises somewhat as V_{BE} falls by 2 mV/K, and the net gain still drops by about 22 % between −40 °C and +85 °C in an automotive unit (the V_T temperature coefficient and the V_{BE} drift of −2 mV/K are covered in Chapter 1).

Emitter degeneration

The bypassed stage's gain depends on g_m, which depends on bias current, temperature and the instantaneous signal (Section 7.9). Leaving part of the emitter resistance unbypassed, R_{E1} in Figure 7.2, trades gain for predictability. The emitter now follows the base, the signal current is set by $v_i/(r_e + R_{E1})$, and

$$A_v = -\frac{\beta R_C'}{r_\pi + (\beta+1)R_{E1}} \approx -\frac{R_C'}{r_e + R_{E1}}, \qquad R_{ib} = r_\pi + (\beta+1)R_{E1}$$

When $R_{E1} \gg r_e$ the gain is a resistor ratio. The stage has become a feedback amplifier: the emitter voltage, proportional to the output current, is subtracted from the input. The factor by which gain falls, $1 + g_m R_{E1}$, is the loop gain plus one, and the same factor raises the input resistance, flattens the gain against β and temperature, and reduces distortion. This series-series feedback is the first example of the topologies classified in Chapter 9.

> **Status of the principle: Degenerated CE gain** $A_v \approx -R_C'/(r_e + R_E)$
> *Approximate.* Derived from the hybrid-π model with $\beta \gg 1$ and $r_o \gg R_C'$; becomes a pure resistor ratio as $g_m R_E \gg 1$, which is the general rule that negative feedback replaces device parameters by passive ones.

Continuing Worked example 7.1 with R_E split into $R_{E1} = 100\ \Omega$ unbypassed and 1.1 kΩ bypassed (bias unchanged): $R_{ib} = 3.37 + 151 \times 0.1 = 18.5\ \text{k}\Omega$, $R_{in} = 8.25 \parallel 18.5 = 5.70\ \text{k}\Omega$, and $A_v = -150 \times 3.20\ \text{k}\Omega/18.5\ \text{k}\Omega = -26.0$, with $R_C' = R_C \parallel R_L$ because degeneration raises the collector's output resistance far above r_o (the approximation $-R_C'/(r_e + R_{E1})$ gives -26.1). From the 600 Ω source, $A_{vs} = -23.5$. For $\beta = 100$ and 300 the gain is -25.8 and -26.2, a spread of ± 0.8 %, three times tighter than the bypassed stage, and its temperature coefficient drops by the factor $1 + g_m R_{E1} = 5.5$.

Where the CE stage is used

Discrete CE stages survive where a single transistor beats an op-amp on noise, bandwidth or cost: moving-coil phono preamplifiers (a BJT at a few milliamps has a voltage noise near $1\ \text{nV}/\sqrt{\text{Hz}}$), RF gain blocks (an MMIC such as a 50 Ω Darlington gain block is a degenerated CE stage with shunt feedback), ultrasound and LiDAR receivers, and the first stage of discrete audio power amplifiers. Inside integrated circuits the CE stage with an active load is the second, high-gain stage of most bipolar op-amps.

7.4 Common-source amplifiers

Same topology, different numbers

The common-source stage is the MOSFET or JFET version of the CE stage: signal in at the gate, out at the drain, $A_v = -g_m R_D'$ (*Engineering Physics* §18.3.3). Three differences matter in design.

1. **Input resistance.** The gate draws no DC current, so R_{in} is set entirely by the bias network, and can be 10 MΩ to 1 GΩ. That is why the CS stage, usually with a JFET, is the input of piezoelectric sensors, pH electrodes and electret microphones, all of which are capacitive sources that a 2 kΩ BJT input would short out.
2. **Transconductance.** For the same current, $g_m/I_D = 2/V_{ov}$ is several times smaller than the BJT's $1/V_T$, so the gain from a given voltage drop across R_D is lower: $A_v = -2I_D R_D/V_{ov}$ against $-I_C R_C/V_T$.
3. **Linearity and threshold spread.** The square law is gentler than the exponential (Section 7.9), but the threshold voltage of discrete MOSFETs varies by ±0.5–1 V between parts, so a discrete CS stage almost always needs source degeneration or a feedback bias to set its current.

Worked example 7.2 — An integrated common-source stage. In a 0.18 µm CMOS process, an NMOS device with $\mu_n C_{ox} W/L = 2\ \text{mA/V}^2$, $V_{th} = 0.5$ V and $\lambda = 0.1\ \text{V}^{-1}$ is biased at $I_D = 200\ \mu\text{A}$ with a drain resistor $R_D = 10\ \text{k}\Omega$.

$$V_{ov} = \sqrt{\frac{2I_D}{\mu_n C_{ox} W/L}} = \sqrt{\frac{0.4\ \text{mA}}{2\ \text{mA/V}^2}} = 0.447\ \text{V}, \qquad g_m = \frac{2I_D}{V_{ov}} = 0.894\ \text{mS}$$

so $g_m/I_D = 4.47\ \text{V}^{-1}$. With $r_o = 1/(\lambda I_D) = 50\ \text{k}\Omega$, $A_v = -0.894\ \text{mS} \times (10 \parallel 50)\ \text{k}\Omega = -7.45$ (17.4 dB). The intrinsic gain is $g_m r_o = 44.7$. A BJT at the same 200 µA has $g_m = 7.74\ \text{mS}$ ($g_m/I_C = 38.7\ \text{V}^{-1}$) and an intrinsic gain $V_A/V_T \approx 3900$. Adding a 1 kΩ source resistor degenerates the MOSFET stage to $-g_m R_D/(1 + g_m R_S) = -4.72$. Designers of CMOS amplifiers live with this low gain per stage by using cascodes and active loads (Chapter 10) and by running devices at low overdrive, where g_m/I_D approaches the weak-inversion limit of about $1/(nV_T) \approx 25\ \text{V}^{-1}$.

The electret microphone: a CS stage in every phone headset

An electret capsule contains a charged diaphragm (a few picofarads) and a JFET, gate tied to the diaphragm through a gigaohm-level leakage path, so $V_{GS} \approx 0$ and $I_D \approx I_{DSS}$. For a typical capsule JFET with $I_{DSS} = 300\ \mu\text{A}$ and $V_P = -0.4$ V, $g_{m0} = 2I_{DSS}/|V_P| = 1.5$ mS. The phone or headset supplies 2 V through a 2.2 kΩ drain resistor ("plug-in power"), giving $A_v = -g_{m0} R_D = -3.3$. The stage's real job is not gain but impedance conversion: from a 5 pF source, whose impedance at 100 Hz is 320 MΩ, to a 2.2 kΩ output that can drive a cable. The JFET is chosen for its gate leakage of picoamps, which a MOSFET matches but a BJT, whose base current would be tens of nanoamps, cannot.

> **Status of the principle: Transconductance efficiency g_m/I**
> *Derived.* From the exponential law, $g_m/I_C = 1/V_T$ for a BJT at any current; from the square law, $g_m/I_D = 2/V_{ov}$ for a MOSFET in strong inversion. The square-law result fails in weak inversion and in short-channel devices with velocity saturation, where g_m/I_D is lower still at high overdrive.

7.5 Emitter and source followers

The buffer

An emitter follower (common collector) takes the signal in at the base and out at the emitter. The emitter sits one V_{BE} below the base and follows it, so the voltage gain is just under one; what the stage provides is current gain and a large impedance transformation. Looking into the base, a load R'_E in the emitter appears multiplied by $\beta + 1$; looking back into the emitter, the source resistance R_s in the base appears divided by $\beta + 1$:

$$A_v = \frac{R'_E}{r_e + R'_E}, \qquad R_{ib} = (\beta + 1)(r_e + R'_E), \qquad R_{out} = R_E \parallel \left(r_e + \frac{R_s}{\beta + 1} \right)$$

with $R'_E = R_E \parallel R_L$. The source follower is the same with $1/g_m$ in place of r_e and no β division: its input resistance is that of the gate bias network and its output resistance is $1/g_m$, typically 100 Ω to 1 kΩ for a small device, much higher than an emitter follower's few ohms. The source follower also suffers from the **body effect** in an integrated circuit: when the source rises above the substrate, the threshold rises with it and the gain falls to $g_m/(g_m + g_{mb})$, about 0.8–0.9, unless the device sits in its own well (Chapter 4).

> **Status of the principle: Impedance transformation by $\beta + 1$ in the emitter follower**
> *Derived.* KCL at the emitter with $i_e = (\beta + 1) i_b$ makes every impedance in the emitter look $\beta + 1$ times larger from the base and every impedance in the base look $\beta + 1$ times smaller from the emitter; valid while the device is active and at frequencies well below f_T/β, above which β falls and the output impedance turns inductive.

Swing limits and the follower's distortion

A follower can push current into its load through the transistor without limit (until the device's ratings), but it can only *sink* current through R_E. On the negative half-cycle the transistor's current falls; when it reaches zero the emitter can go no lower, and the output clips at a negative peak of

$$V_{o,\min} = -I_E \left(R_E \parallel R_L \right)$$

below its quiescent value. A follower biased at 4.8 mA with $R_E = 1.2\,\text{k}\Omega$ driving a 1 kΩ load can swing only 2.63 V negative, even with 5.8 V of DC headroom. The cure is more quiescent current, a current-source emitter load, or a complementary push-pull output in which an NPN follower sources and a PNP follower sinks. The push-pull pair has its own failure, crossover distortion near zero current (Section 7.9), and is the output stage of almost every op-amp and audio power amplifier.

A second, subtler limit appears with capacitive loads: an emitter follower driving a capacitor can see its output impedance become inductive above f_T/β, and the resulting resonance with the load capacitance can make the follower oscillate in the hundreds of megahertz. A 47–100 Ω base-stopper resistor kills the resonance; it is standard practice on discrete followers driving cables.

7.6 Multistage amplifiers

Cascading with loading

One stage rarely meets all of a specification. A sensor preamplifier may need 40 dB of gain, a 10 kΩ input and the ability to drive a 1 kΩ load or a cable; no single stage does all three. Stages are cascaded, and the overall gain is the product of the individual gains **each evaluated with the next stage's input resistance as its load**:

$$A_{v,\text{total}} = A_{v1}\big|_{R_L = R_{in2}} \times A_{v2}\big|_{R_L = R_{in3}} \times \cdots$$

Multiplying unloaded gains is the classic mistake. Book 3 formalises the cascade with ABCD matrices (Ch. 12 §12.8); with the unilateral model the loaded-gain rule is the same thing written by hand.

> **Status of the principle: Loaded-gain product for cascades**
> *Derived.* Follows from the unilateral two-port model applied stage by stage; exact for unilateral stages, and accurate for real transistor stages as long as reverse transmission (Section 7.8) is negligible.

The CE–CC pair

The commonest discrete cascade is a CE gain stage followed by an emitter follower, often direct-coupled to save a capacitor and the low-frequency pole it creates (Figure 7.3). The follower's input resistance, $\beta + 1$ times its emitter load, isolates the CE stage from the real load, and its low output resistance drives the load.

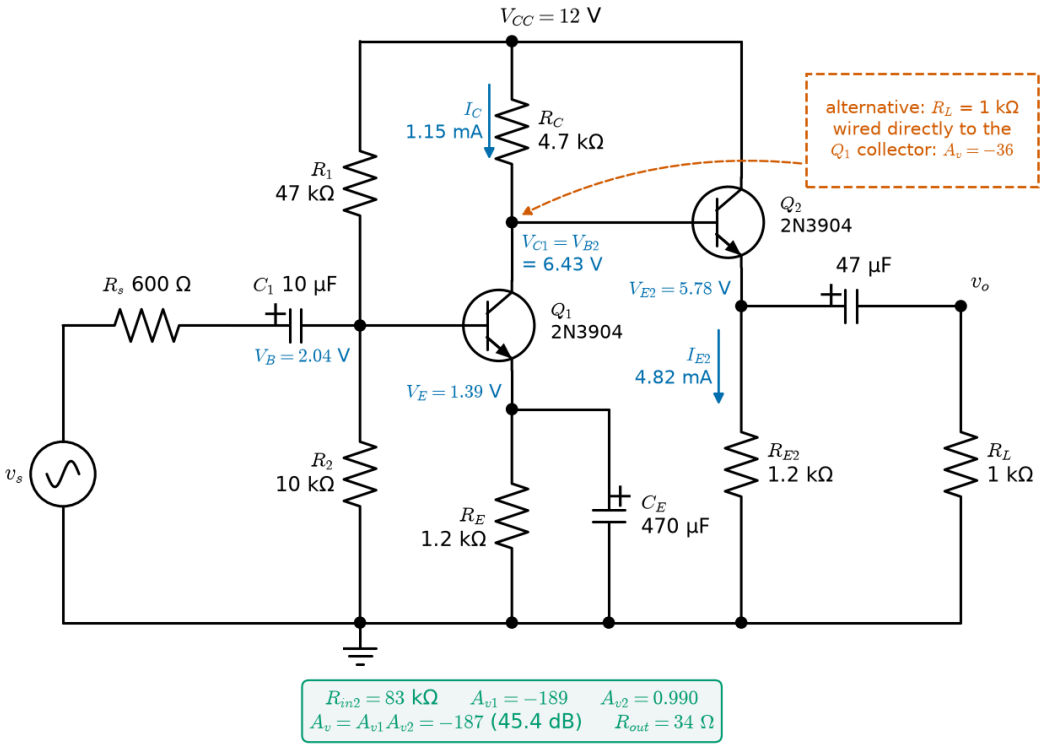


Figure 7.3 — Two-stage amplifier: the common-emitter stage of Worked example 7.1 direct-coupled to an emitter follower driving a 1 kilohm load.

Worked example 7.3 — Adding a follower to drive 1 kΩ. The CE stage of Worked example 7.1 must drive a 1 kΩ line. Connected directly, its gain would be $-g_m(R_C \parallel r_o \parallel 1\ \text{k}\Omega) = -36.4$, a loss of 14.7 dB from the unloaded value. Add a direct-coupled 2N3904 follower with $R_{E2} = 1.2$ kΩ, as in Figure 7.3.

DC. The follower's base current, $I_{E2}/(\beta + 1)$, also flows in R_C; iterating $V_{C1} = 12 - (I_{C1} + I_{B2})R_C$ with $I_{E2} = (V_{C1} - 0.65)/R_{E2}$ converges to $V_{C1} = 6.43$ V, $I_{E2} = 4.82$ mA, $I_{B2} = 31.9\ \mu$A and $V_{E2} = 5.78$ V. (The base current shifted the first stage's collector by 0.15 V, a typical direct-coupling interaction.)

Small signal. $r_{e2} = 25.85/4.82 = 5.36\ \Omega$; the follower's emitter load is $1.2 \parallel 1 = 0.545$ kΩ, so $R_{in2} = 151 \times (5.36 + 545) = 83.2$ kΩ. The first stage now sees $R_C \parallel r_o \parallel R_{in2} = 4.22$ kΩ and gives $A_{v1} = -188.6$. The follower gives $A_{v2} = 545/(5.36 + 545) = 0.990$. The total is -186.8 (45.4 dB), five times the directly loaded value. The output resistance is $R_{E2} \parallel [r_{e2} + (R_C \parallel r_o)/151] = 1.2\ \text{k}\Omega \parallel 34.9\ \Omega = 33.9\ \Omega$.

Limit. The follower clips negative at $I_{E2}(R_{E2} \parallel R_L) = 2.63$ V peak, so the cascade's useful output is about 2.6 V peak; a larger swing needs more follower current or a push-pull output.

Cascode, Darlington and differential pair

Three two-transistor combinations recur so often that they are treated as single "super-devices".

- **Cascode** (CE or CS followed by CB or CG, stacked on one current). The lower device sets g_m; the upper device holds the lower device's collector at a fixed voltage and passes its current to the output. The input sees almost no voltage gain across C_μ, which removes the Miller effect (Section 7.8), and the output resistance rises to about βr_o (BJT) or $g_m r_o^2$ (MOSFET), which raises the achievable gain with an active load. Cascodes are the standard RF low-noise amplifier, video amplifier and CMOS op-amp gain stage.
- **Darlington** (two followers in cascade, collectors joined). The current gain is about $\beta_1\beta_2$, 10 000 or more, at the price of a $2V_{BE}$ drop, a saturation voltage near 0.9 V and slow turn-off; it survives in relay and solenoid drivers (ULN2003 arrays) and in some linear regulators. The **Sziklai** (complementary feedback) pair gives similar gain with one V_{BE} and is common in audio output stages.

- **Differential pair** (two emitter- or source-coupled devices on a tail current). It amplifies the difference between two inputs and rejects their common part, and it is the input stage of every op-amp and comparator. Its analysis, with current-mirror loads, belongs to Chapter 10.

7.7 Input and output impedance

Impedance as a design variable

The terminated two-port formula of Section 7.1 shows that a stage's input and output resistances are not secondary details but halves of the gain calculation. The design rule depends on what is being transferred.

- **Voltage transfer** (sensor chains, audio, instrumentation): make $R_{in} \gg R_s$ and $R_{out} \ll R_L$, typically by a factor of 10 or more ("bridging"). A line output of 100 Ω into a 10 kΩ line input is standard audio practice.
- **Current transfer** (photodiodes, current-output DACs, current loops): the reverse, $R_{in} \ll R_s$ and $R_{out} \gg R_L$. This is why the common-base and transimpedance stages are used for photodiodes.
- **Power transfer and reflection-free lines** (RF, video, long cables): $R_{in} = R_s^*$ and $R_{out} = R_L^*$, usually 50 Ω or 75 Ω, so that power is maximised and the line is terminated (Book 3 Ch. 14).

Table 7.3 — Typical source and load impedances an amplifier meets.

Interface	Source or load impedance	Design consequence
Piezo accelerometer, hydrophone	0.5–5 nF capacitive (≈ 300 kΩ–3 MΩ at 100 Hz)	JFET/CMOS input or charge amplifier
Electret microphone capsule	≈ 5 pF before its JFET, 2.2 kΩ after	Internal JFET source follower/CS stage
Dynamic microphone	150–600 Ω	Low-noise BJT input, transformer optional
Thermocouple	5–100 Ω	Low voltage noise and offset matter, not R_{in}
Photodiode	Current source with 1–100 pF	Transimpedance input (Chapter 9)
Audio line in/out	10 kΩ in / 100 Ω out	Bridging, not matching
RF and video	50 Ω / 75 Ω	Matched input and output
SAR ADC input	Switched 10–40 pF sampling capacitor	Low-R_{out} driver plus RC charge reservoir (Chapter 26)

Raising input impedance: bootstrapping

A discrete stage's input resistance is often limited not by the transistor but by its bias divider: in Worked example 7.1 the 8.25 kΩ divider is in parallel with an 18.5 kΩ base. **Bootstrapping** feeds the output of a follower back to the bottom of a resistor in the bias path through a capacitor. If the resistor R_B connects the base to a node that moves with gain $A \approx 0.99$, the signal current through it is $(1 - A)v_i/R_B$ and it looks like $R_B/(1 - A)$, a hundred times larger. Bootstrapped followers give megohm inputs from a single BJT; the same trick, applied to a cable shield ("guard driving"), cancels cable capacitance in pH and electrometer inputs.

Measuring impedance

Input resistance is measured by inserting a known series resistor R_x and observing the drop in output: if the output halves, $R_{in} = R_x$. Output resistance is measured by loading the output with R_x at small signal: if the output falls to half its open-circuit value, $R_{out} = R_x$. Both measurements must be done at small signal and in the midband; a large signal changes g_m and clips, and above the corner frequency the impedances are complex.

7.8 Frequency response

Two ends of the band

An amplifier's gain is flat only over its **midband**. At low frequencies, coupling and bypass capacitors stop behaving as shorts; at high frequencies, the transistor's internal capacitances stop behaving as opens. Both effects are poles of the transfer function and are drawn with the Bode asymptotes of Book 3 Ch. 13 §13.3. Figure 7.4 shows the complete response of the stage in Worked example 7.1.

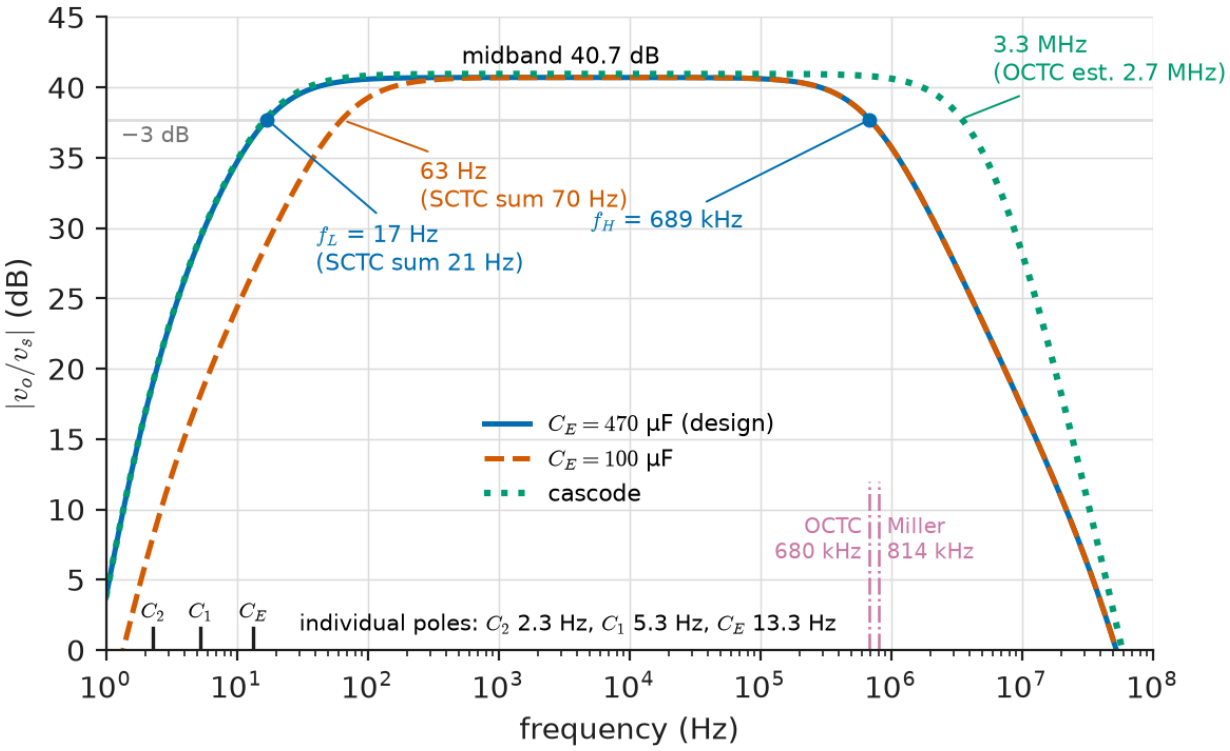


Figure 7.4 — Magnitude response of the 2N3904 common-emitter stage from 1 Hz to 100 MHz, with the low-frequency poles of the coupling and bypass capacitors and the Miller-limited upper corner.

Low-frequency poles: short-circuit time constants

Each coupling or bypass capacitor contributes a pole at $1/(2\pi RC)$, where R is the resistance it sees with the signal source shorted and the other capacitors treated as shorts. When the poles are well separated the lower corner is close to the largest one; in general the **short-circuit time-constant** method gives

$$f_L \approx \sum_k \frac{1}{2\pi R_k C_k}$$

The emitter bypass capacitor sees the smallest resistance by far, because it looks into the emitter, where everything in the base circuit is divided by $\beta + 1$:

$$R_{C_E} = R_E \parallel \frac{r_\pi + (R_s \parallel R_1 \parallel R_2)}{\beta + 1}$$

That is why bypass capacitors in discrete audio stages are hundreds of microfarads while coupling capacitors are a few.

> **Status of the principle: Short-circuit time-constant estimate of f_L**
> *Approximate.* Exact for a single low-frequency pole; with several poles it overestimates f_L slightly (the sum is the coefficient of a polynomial, not the root) and is accurate within about 10–20 % unless two poles coincide.

Worked example 7.4 — Sizing the capacitors and finding the bandwidth. For the stage of Worked example 7.1:

Low end. C_1 sees $R_s + R_{in} = 0.6 + 2.39 = 2.99$ kΩ; C_2 sees $(R_C \parallel r_o) + R_L = 14.46$ kΩ; C_E sees $1.2\ \text{k}\Omega \parallel (3.37 + 0.56)\ \text{k}\Omega/151 = 25.4\ \Omega$. With $C_1 = 10\ \mu$F, $C_2 = 4.7\ \mu$F and $C_E = 100\ \mu$F the poles are 5.3 Hz, 2.3 Hz and 62.6 Hz, and $f_L \approx 70$ Hz: bass is lost. Raising C_E to 470 µF moves its pole to 13.3 Hz and f_L to 21.0 Hz. By the same sum a 20 Hz design would need $C_E \approx 510\ \mu$F, but the sum overestimates: the exact response (Figure 7.4) has its corner at 17 Hz with 470 µF and 63 Hz with 100 µF. Degeneration helps here too: with 100 Ω unbypassed, C_E sees roughly 100 Ω more and can be four to five times smaller.

High end, Miller estimate. The 2N3904 has $C_\mu \approx 2.5$ pF at this bias (C_{ob} is 4 pF maximum on the datasheet) and $f_T \approx 200$ MHz at 1 mA (300 MHz at 10 mA), so $C_\pi = g_m/(2\pi f_T) - C_\mu = 33.0$ pF. C_μ connects the base to a collector that swings $-g_m R'_C = -137.5$ times the base voltage, so the current through it is $(1 + 137.5)$ times what it would be to ground: it looks like

$$C_M = C_\mu(1 + g_m R'_C) = 2.5\ \text{pF} \times 138.4 = 346\ \text{pF}$$

from the base. The total input capacitance is $C_\pi + C_M = 379$ pF, ten times C_π. It is driven through $R_{sig} = r_\pi \parallel (r_x + R_s \parallel R_B) = 516\ \Omega$, so

$$f_H \approx \frac{1}{2\pi R_{sig}(C_\pi + C_M)} = 814\ \text{kHz}$$

High end, open-circuit time constants. Summing the time constants each capacitor sees with the others open, $\tau_\pi = R_{sig}C_\pi = 17.0$ ns, $\tau_\mu = [R_{sig}(1 + g_m R'_C) + R'_C]C_\mu = 186$ ns and $\tau_L = R'_C C_L = 30.8$ ns for a 10 pF load and wiring capacitance; $f_H \approx 1/(2\pi \sum \tau) = 680$ kHz. Listing 7.1's exact nodal solution gives 689 kHz.

The numbers show where the bandwidth goes: 80 % of the total time constant is C_μ, a 2.5 pF capacitance multiplied by the gain. Reducing R_s from 600 Ω to 50 Ω triples f_H to 2.2 MHz (Listing 7.1 with `Rs = 50`), and removing the 10 kΩ load as well gives the 1.5 MHz, 46 dB bench result of the opening paragraph: the bench and the product differ only in their terminations. A cascode, which holds the lower transistor's collector at nearly constant voltage so that the Miller multiplier falls from 138 to about 2, raises the estimate to about 2.7 MHz with the same transistors and gain (the exact solution gives 3.3 MHz).

> **Status of the principle: Miller approximation**
> *Approximate.* Replaces a bridging capacitance by an input capacitance $C(1 - A_v)$ using the midband gain; it ignores the right-half-plane zero at g_m/C_μ and the output-side loading, and is accurate when the input pole is dominant, as it is here; the open-circuit time-constant sum is the safer estimate when it is not.

Gain–bandwidth trade in a single stage

For a CE stage limited by the Miller input pole, $f_H \propto 1/(1 + g_m R'_C)$ roughly, so the product of gain and bandwidth stays nearly constant as R_C is changed. The degenerated stage shows the same trade by a different route: its lower gain reduces the Miller multiplication and its feedback widens the bandwidth. For a well-designed stage the gain–bandwidth product is a fraction of f_T, a few tens of megahertz for a 2N3904 and several gigahertz for an RF transistor such as a BFR92 ($f_T \approx 5$ GHz). The general form of this trade, $A \cdot B \approx$ constant under feedback, and its use in op-amp design are developed in Chapter 9.

Table 7.4 — Representative discrete small-signal transistors (typical datasheet values).

Device	Type	β or I_{DSS}	f_T	C_μ / C_{rss}	Typical role
2N3904 / 2N3906	NPN / PNP	100–300	300 / 250 MHz	2–4 pF	General-purpose discrete stages
BC547B / BC557B	NPN / PNP	200–450	300 / 150 MHz	2–4 pF	Audio, consumer
BFR92A	NPN RF	50–150	5 GHz	0.4 pF	VHF/UHF gain blocks
J201	N-JFET	0.2–1 mA	—	1–3 pF	High-impedance inputs, electret-like stages
BF862	N-JFET	10–25 mA	700 MHz (approx.)	1.9 pF	Low-noise hydrophone and RF front ends
2N7000	N-MOSFET	V_{th} 0.8–3 V	—	5 pF	Switching; poor linear amplifier without degeneration

7.9 Distortion

Harmonic distortion from the exponential law

A BJT's collector current is $I_C\, e^{v_{be}/V_T}$. With $v_{be} = \hat{V}\cos\omega t$, the power series

$$i_c = I_C\left[\frac{\hat{V}}{V_T}\cos\omega t + \frac{1}{2}\left(\frac{\hat{V}}{V_T}\right)^2\cos^2\omega t + \frac{1}{6}\left(\frac{\hat{V}}{V_T}\right)^3\cos^3\omega t + \cdots\right]$$

shows that the second harmonic is $\hat{V}/(4V_T)$ of the fundamental and the third about $(\hat{V}/V_T)^2/24$. **Harmonic distortion** HD_n is the ratio of the n-th harmonic to the fundamental; **total harmonic distortion** is $\mathrm{THD} = \sqrt{\sum_{n\ge 2}\mathrm{HD}_n^2}$. Equivalent results for the MOSFET square law give $\mathrm{HD}_2 = \hat{V}/(4V_{ov})$ and, ideally, no third harmonic; the much larger V_{ov} (0.2–0.5 V against 25.85 mV) is why MOSFET and JFET stages sound "cleaner" at the same input level, and why they need more input for the same output. For the stage of Worked example 7.2, a 50 mV gate signal gives $\mathrm{HD}_2 = 0.05/(4\times 0.447) = 2.8\ \%$.

> **Status of the principle:** $\mathrm{HD}_2 = \hat{v}_{be}/(4V_T)$ **for a bypassed BJT stage**
> *Derived.* Taylor expansion of the exponential collector law with a sinusoidal base–emitter voltage; valid for $\hat{v}_{be}$ up to about V_T, ignoring r_o nonlinearity, Early effect and high-level injection, which add distortion at large collector swings.

The output level of a bypassed CE stage therefore comes with its distortion attached. For 1 V peak out of Worked example 7.1 ($|A_v| = 137.5$), $\hat{v}_{be} = 7.3$ mV and $\mathrm{HD}_2 \approx 7\ \%$. Degeneration helps because the base–emitter junction sees only a fraction $1/(1+g_mR_{E1})$ of the input, and its distortion is then corrected by the same factor:

$$\mathrm{HD}_2 \approx \frac{\hat{v}_{be}}{4V_T}\cdot\frac{1}{1+g_mR_{E1}}, \qquad \hat{v}_{be} = \frac{\hat{v}_i}{1+g_mR_{E1}}$$

The second factor is the general feedback result of Chapter 9: distortion produced inside a loop is divided by one plus the loop gain.

Worked example 7.5 — Distortion of the bypassed and degenerated stages. Listing 7.2 solves the large-signal emitter-loop equation $v_i = v_{be} + i_cR_{E1}$ exactly at each sample by Newton iteration and takes the FFT of the output.

```
import numpy as np

VT, IC = 0.02585, 1.152e-3          # bias of Worked example 7.1
RLp = 3.084e3                       # RC || RL || ro

def harmonics(v_in_pk, RE1, N=4096):
    """Large-signal CE stage: solve v_in = v_be + i_c*RE1 (AC parts) by Newton."""
    t = np.arange(N) / N
    v_in = v_in_pk * np.sin(2 * np.pi * t)
    x = np.zeros(N)                                 # x = AC part of v_be
    for _ in range(50):
        ic = IC * np.exp(x / VT)                    # total collector current
        f = x + (ic - IC) * RE1 - v_in
        x -= f / (1 + ic * RE1 / VT)
    vo = -(IC * np.exp(x / VT) - IC) * RLp          # AC output voltage
    H = np.abs(np.fft.rfft(vo)) / (N / 2)
    return H[1], H[2] / H[1], H[3] / H[1]

for RE1, vin in [(0.0, 7.3e-3), (100.0, 38.5e-3)]:
    a1, h2, h3 = harmonics(vin, RE1)
    print(f"RE1 = {RE1:5.0f} ohm, vin = {vin*1e3:4.1f} mV: "
          f"vo1 = {a1:.2f} V, HD2 = {100*h2:.2f} %, HD3 = {100*h3:.3f} %")
```

The output is:

```
RE1 =     0 ohm, vin =  7.3 mV: vo1 = 1.01 V, HD2 = 7.04 %, HD3 = 0.331 %
RE1 =   100 ohm, vin = 38.5 mV: vo1 = 0.97 V, HD2 = 1.28 %, HD3 = 0.084 %
```

For the same 1 V output, the bypassed stage gives 7.0 % second harmonic, matching $\hat{v}_{be}/(4V_T) = 7.06\,\%$; the degenerated stage gives 1.28 %, against the formula's 1.25 %, an improvement by the factor $1 + g_m R_{E1} = 5.5$. The price is the gain: 38.5 mV of input instead of 7.3 mV, so the preceding stage must deliver five times more, and its distortion matters in turn. A preamplifier for a moving-coil phono cartridge (signals of 0.2–0.5 mV) can run bypassed; a line-level stage cannot. Figure 7.5 shows the two output waveforms and spectra.

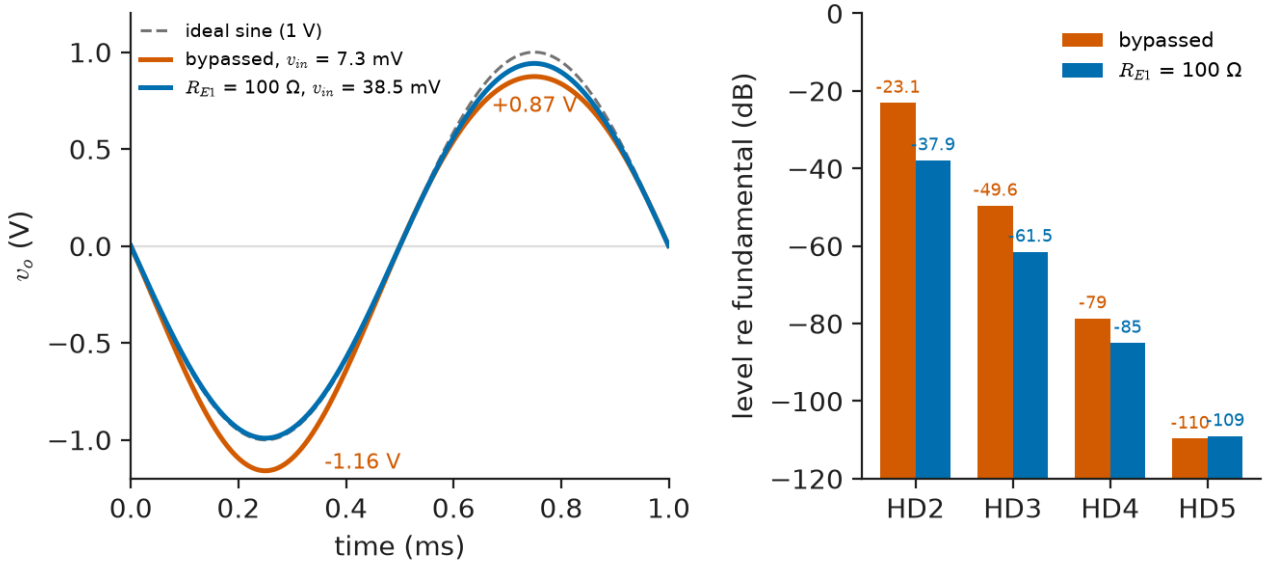


Figure 7.5 — Output waveform and harmonic spectrum of the CE stage at 1 V peak output, bypassed and with 100 ohm emitter degeneration.

Intermodulation

A single tone reveals harmonics; two tones reveal **intermodulation**. Driving a nonlinearity with f_1 and f_2 produces second-order products at $f_2 \pm f_1$ and third-order products at $2f_1 - f_2$ and $2f_2 - f_1$. The third-order products are the dangerous ones in a receiver, because when f_1 and f_2 are two strong adjacent channels, $2f_1 - f_2$ falls in the band next to them, where a weak wanted signal may be, and no filter can remove it after the stage. For small signals the two-tone third-order product is three times the single-tone HD_3 at the same per-tone amplitude, and it rises 3 dB for every 1 dB of input; RF designers summarise this by the third-order intercept point, whose system use belongs to Book 3 Ch. 32. In audio, intermodulation between a bass note and a cymbal is more audible than the harmonics of either, which is why SMPTE and CCIF intermodulation tests accompany THD in amplifier specifications.

Clipping and headroom

Every stage has a hard ceiling set by its bias. In the CE stage of Worked example 7.1 the collector can fall until the transistor saturates, about $V_{CE} - 0.2\ \mathrm{V} = 5.0\ \mathrm{V}$ below its quiescent point; it can rise only until the transistor cuts off, which with the load AC-coupled happens when the signal current equals I_C, at $I_C(R_C \parallel R_L) = 1.152\ \mathrm{mA} \times 3.20\ \mathrm{k\Omega} = 3.68\ \mathrm{V}$. The stage clips first on the positive output peak, asymmetrically: the classic asymmetric clipping of a single-ended stage. Maximum symmetric swing is obtained by placing the quiescent point at the centre of the **AC load line** rather than the DC one, which here would mean a larger I_C or smaller R_C. **Headroom** is the ratio of the clipping level to the normal operating level, typically 12–20 dB in audio, and more in instrumentation where a fault transient must not saturate the chain.

Crossover distortion in push-pull stages

A complementary push-pull follower (Section 7.5) biased at zero quiescent current (class B) has a dead band of about ± 0.6 V in which neither transistor conducts. The resulting **crossover distortion** is worst at low signal levels, rich in high-order harmonics and therefore audible far below the THD figure that would be acceptable for low-order distortion. Class AB biasing, a quiescent current of tens of milliamps set by a V_{BE}-multiplier between the bases, makes the two halves overlap; emitter resistors of 0.1–0.47 Ω stabilise the quiescent current against thermal runaway (the V_{BE} drift of -2 mV/K with the output devices' heatsink temperature, Chapter 3). Global negative feedback then reduces the residue by the loop gain at the relevant frequency, which falls at high frequency and is the reason amplifier THD rises above a few kilohertz (Chapter 9).

Table 7.5 — Distortion and its remedies.

Mechanism	Signature	Remedy
Exponential v_{be} law	Strong 2nd harmonic, asymmetric waveform	Degeneration, differential pair (cancels even orders), feedback
Square law	2nd harmonic	Larger V_{ov}, differential pair
Early effect, C_μ voltage dependence	Rising distortion at large collector swing and high frequency	Cascode, lower swing per stage
Clipping	Flat-topped peaks, all harmonics	Bias at AC load-line centre, more supply, gain staging
Crossover (class B)	Notch at zero crossing, high-order harmonics	Class AB bias, feedback
Thermal (self-heating at signal rate)	Low-frequency distortion, memory effects	Larger devices, thermal coupling of pairs

What remains open

Open question	Where it is answered
How are op-amps used as ideal-looking building blocks, and when does that ideal fail?	Chapter 8 (Operational Amplifiers)
What is negative feedback in general, which topology raises or lowers which impedance, and why can feedback make an amplifier oscillate?	Chapter 9 (Feedback and Oscillators)
How are differential pairs, current mirrors and active loads combined into integrated amplifiers, and how is the Miller effect used deliberately for compensation?	Chapter 10 (Analog Integrated Circuits)
How are the BJT and MOSFET bias points, f_T and capacitances obtained from device physics and datasheets?	Chapter 3 and Chapter 4
What noise does a transistor stage add, and how is noise figure computed?	*Circuits, Signals and Communications*, Ch. 29; op-amp noise in Chapter 8
How do transistors behave when used as switches rather than amplifiers, in logic and in power converters?	Chapter 33 and Chapter 29
How are amplifiers matched and characterised with S-parameters at microwave frequencies?	*Circuits, Signals and Communications*, Ch. 32

Next: Chapter 8 packages dozens of the transistors of this chapter into the operational amplifier and shows how feedback around it turns a high, poorly controlled gain into precise amplifiers, filters, comparators and signal processors.

Chapter 8 — Operational Amplifiers

Part II · Analog Electronics

An electric-vehicle battery pack measures its current through a 100 μΩ shunt in the negative bus. At the full 500 A the shunt develops 50 mV; at a 1 A parking load, 100 μV. The first prototype amplifies the shunt voltage with a general-purpose LM358 in a textbook gain-of-50 stage. On the bench, with a precision source, it reads correctly at 500 A. In the car, the battery-management system's coulomb counter, which integrates current to estimate state of charge, drifts by a fifth of the pack's capacity per hour while the car is parked. The amplifier is not faulty: its 2 mV input offset is the shunt voltage of a 20 A current that does not exist, and integrating 20 A for an hour on a 100 Ah pack is 20 % of its charge. A zero-drift op-amp with 10 μV offset reduces the phantom current to 0.1 A; a dedicated current-sense amplifier with laser-trimmed resistors adds the common-mode rejection needed when the shunt moves to the high side of a 400 V bus. The ideal op-amp of the textbook got the gain right. Everything else in the design was decided by the op-amp's departures from ideal.

> **How far can a circuit designer trust the two "golden rules" of the ideal op-amp, and which datasheet parameter breaks them first in a given application?**
> **How are the standard op-amp circuits (amplifiers, integrators, comparators, instrumentation amplifiers, rectifiers, filters and oscillators) designed to a numerical specification?**

Concept	Engineering destination
Ideal and practical op-amps	Choosing a part: offset, bias current, bandwidth, slew rate, noise, supply
Inverting and non-inverting amplifiers	Sensor gain stages, buffers, ADC drivers, audio preamplifiers
Summing and difference amplifiers	Mixers, DAC outputs, current-shunt sensing, level shifting
Integrators and differentiators	Charge amplifiers, Rogowski-coil integrators, PID controllers, ramp generators
Comparators	Zero-crossing and overcurrent detectors, window monitors, PWM generation
Instrumentation amplifiers	Load cells, strain gauges, ECG and EEG front ends, thermocouples
Precision rectifiers	AC voltmeters, envelope detectors, AGC loops
Active filters	Anti-aliasing and reconstruction filters, audio crossovers, vibration analysers
Oscillators	Test-tone sources, relaxation timers, sensor excitation
Limitations and stability	Rail-to-rail swing, full-power bandwidth, noise budgets, capacitive loads

Roadmap. The ideal model and the error sources of a real op-amp (8.1) → inverting and non-inverting stages with their real errors (8.2) → summing and difference amplifiers and the CMRR of resistor networks (8.3) → integrators and differentiators in practice (8.4) → comparators and hysteresis (8.5) → instrumentation amplifiers (8.6) → precision rectifiers (8.7) → active-filter design (8.8) → op-amp oscillator circuits (8.9) → the limits that decide every design: swing, slew, noise and stability (8.10).

8.1 Ideal and practical op-amps

Boundary.
The ideal op-amp, its two golden rules and the standard configurations with their gain formulas ($-R_f/R_1$, $1+R_f/R_1$, summing, integrator, differentiator) are stated in *Engineering Physics*, Ch. 18 §18.4.1–18.4.2, with the gain-bandwidth product, slew rate and offset sketched in §18.4.3; filter approximations and the Sallen–Key section are in §18.5.3–18.5.4. The closed-loop gain formula $A/(1+A\beta)$ and loop gain come from *Circuits, Signals and Communications*, Ch. 34 §34.2; the active-filter network interface from Ch. 13 §13.8. Feedback topologies, stability margins in op-amp circuits and oscillator theory belong to Chapter 9; the transistor circuits inside the op-amp to Chapter 10. This chapter designs with op-amps.

The ideal model and why it works

An op-amp is a differential amplifier with an open-loop gain A of 10^5 to 10^7 at DC. With negative feedback through a network that returns a fraction β of the output to the inverting input, the closed-loop gain is

$$G = \frac{A}{1+A\beta} = \frac{1}{\beta}\cdot\frac{1}{1+1/(A\beta)}$$

(Book 3 §34.2). When the loop gain $A\beta$ is large, $G \to 1/\beta$ and the differential input voltage v_o/A is negligible, which is the virtual short. The golden rules are therefore not a separate assumption; they are the limit of large loop gain. The quantity $1/\beta$, the closed-loop gain the circuit would have with an ideal op-amp *seen from the non-inverting input*, is called the **noise gain**. It sets how much of every input-referred error appears at the output and, because A falls with frequency, how much bandwidth the circuit gets. The noise gain is $1+R_f/R_1$ for both the non-inverting stage and the inverting stage, even though the inverting stage's signal gain is $-R_f/R_1$.

Status of the principle: The ideal op-amp golden rules
Approximate. Virtual short and zero input current hold when the loop gain $A\beta \gg 1$ and the input bias current is negligible against the currents in the feedback network; they fail as $A(f)$ falls at high frequency, when the output saturates or slews, and whenever the op-amp is used without negative feedback.

The error model

A real op-amp is best described by adding error sources to an ideal one (Figure 8.1). Each source is referred to the input and multiplied by the noise gain (or by the feedback resistance, for currents) to find its output effect.

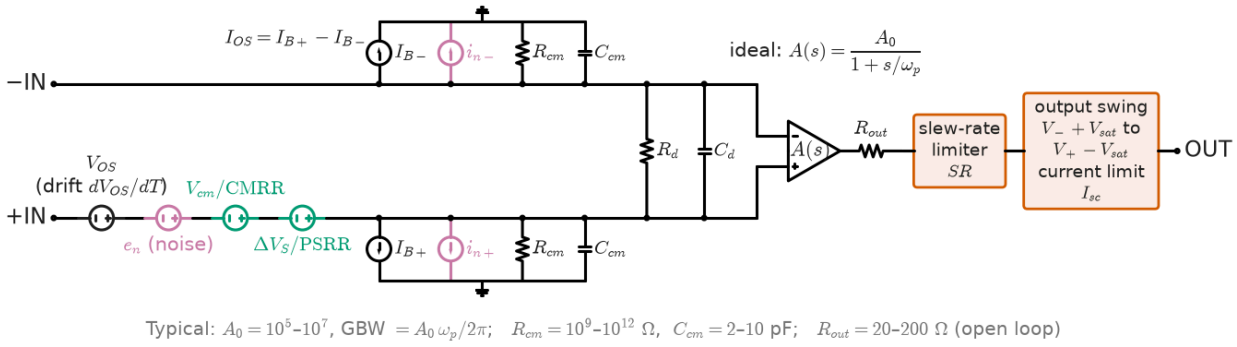


Figure 8.1 — Error model of a practical op-amp: an ideal differential amplifier with the input-referred error sources that a datasheet specifies.

Table 8.1 — Representative op-amps (typical datasheet values at 25 °C; maxima are higher, check the current datasheet).

Part	Input	GBW	Slew rate	V_{OS}	V_{OS} drift	I_B	e_n at 1 kHz	Supply
LM358	Bipolar	0.7–1 MHz	0.3 V/µs	2 mV	7 µV/°C	20–45 nA	40 nV/√Hz	3–32 V
TL072	JFET	3 MHz	13 V/µs	3 mV	18 µV/°C	65 pA	18 nV/√Hz	7–36 V
OP07	Bipolar precision	0.6 MHz	0.3 V/µs	30 µV	0.3 µV/°C	1.2 nA	10 nV/√Hz	±3 to ±18 V
NE5532	Bipolar low noise	10 MHz	9 V/µs	0.5 mV	—	200 nA	5 nV/√Hz	±5 to ±20 V
OPA1612	Bipolar audio	40 MHz	27 V/µs	0.1 mV	1 µV/°C	60 nA	1.1 nV/√Hz	±2.25 to ±18 V
OPA2188	CMOS zero-drift	2 MHz	0.8 V/µs	6 µV	0.03 µV/°C	160 pA	8.8 nV/√Hz	4–36 V
AD8065	JFET high speed	145 MHz	180 V/µs	0.4 mV	1 µV/°C	2 pA	7 nV/√Hz	5–24 V
MCP6001	CMOS rail-to-rail	1 MHz	0.6 V/µs	≤ 4.5 mV (max)	2 µV/°C	1 pA	28 nV/√Hz	1.8–6 V

The table spans four orders of magnitude in offset, five in bias current and two in bandwidth. No part is best at everything: bipolar inputs give low voltage noise and offset but large bias current; JFET and CMOS inputs give picoamp bias current but more voltage noise and offset; zero-drift (chopper or auto-zero) parts give microvolt offsets with almost no drift, at the cost of switching ripple and modest bandwidth; high-speed parts give bandwidth at the cost of supply current and DC precision. Choosing an op-amp means finding which error dominates the application, which is what the rest of the chapter does.

> **Status of the principle: Single-pole op-amp model** $A(f) \approx \mathrm{GBW}/(jf)$
> *Approximate*. Internally compensated op-amps are designed so that one pole dominates from a few hertz up to near the unity-gain frequency; the model gives closed-loop bandwidth GBWβ for noise gains above about 2 to 3, and fails near unity gain, where the second pole sets phase margin (Chapter 9), and for large signals, where slew rate dominates.

8.2 Inverting and non-inverting amplifiers

The basic stages and their real impedances

The inverting stage's gain is $-R_f/R_1$ and its input resistance is exactly R_1, because the inverting input is a virtual ground. That is a design constraint: a gain of -100 with a 10 kΩ input needs a 1 MΩ feedback resistor, whose bias-current error and thermal noise may be unacceptable. The **T-network** obtains high gain from moderate resistors: replacing R_f by R_2 and R_3 in series with R_4 from their junction to ground gives

$$G = -\frac{R_2 + R_3 + R_2 R_3 / R_4}{R_1}$$

so $R_1 = 10\ \mathrm{k\Omega}$, $R_2 = R_3 = 100\ \mathrm{k\Omega}$ and $R_4 = 1\ \mathrm{k\Omega}$ give $G = -1020$ without a 10 MΩ resistor; the price is a noise gain larger than the signal gain, because the T-network attenuates feedback.

The non-inverting stage's gain is $1 + R_f/R_1$ and its input impedance is the op-amp's common-mode input impedance, 10^9 to $10^{12}\ \Omega$ in parallel with a few picofarads, which is why it is the default buffer for high-impedance sensors. Its weakness is that the input common-mode voltage equals the signal, so the input stage must accept the full signal range: a single-supply op-amp whose common-mode range ends 1.5 V below the positive rail cannot buffer a signal that approaches it. **Rail-to-rail input** parts use two complementary input pairs to cover the whole supply, with a crossover region near one rail where offset shifts by hundreds of microvolts.

Single-supply circuits move the signal reference to mid-supply: an inverting stage on 3.3 V references its non-inverting input to a 1.65 V reference (buffered, or a divider with a decoupling capacitor), and AC coupling at the input removes the DC level of the source.

Finite gain, offset and bias current

From the closed-loop formula, the fractional gain error due to finite open-loop gain is about $1/(A\beta)$. Input offset V_{OS} appears at the output multiplied by the noise gain $1/\beta$. Input bias current I_B flowing in the feedback resistor R_f adds $I_B R_f$; if the non-inverting input is fed through a resistance equal to $R_1 \parallel R_f$, the two bias currents produce equal and opposite errors and only the offset current I_{OS} remains, giving $I_{OS} R_f$. The compensation resistor helps bipolar parts (where $I_{OS} \ll I_B$) and does nothing for CMOS parts, whose picoamp bias currents are dominated by ESD-diode leakage that doubles every 10 °C.

> **Status of the principle: Output error of a feedback stage** $V_{o,err} = V_{OS}/\beta + I_{OS} R_f$
> *Derived.* Superposition of the input-referred error sources through the linear closed-loop circuit; offsets add algebraically with unknown signs, so worst-case budgets add magnitudes and statistical budgets add squares.

Worked example 8.1 — Error budget of a gain-of-101 sensor stage. A thermocouple or bridge signal is amplified by a non-inverting stage with $R_f = 100$ kΩ and $R_1 = 1$ kΩ ($G = 1/\beta = 101$), first with an OP07 (typical $A_0 = 4 \times 10^5$, $V_{OS} = 30\ \mu$V, drift 0.3 µV/°C, $I_B = 1.2$ nA, $I_{OS} = 0.5$ nA, GBW 0.6 MHz).

Gain error at DC. $1/(A\beta) = 101/(4 \times 10^5) = 0.025$ %, so $G = 100.97$.

Gain error at 1 kHz. The open-loop gain has fallen to $\mathrm{GBW}/f = 600$ with a −90° phase, so $A\beta = -j5.94$ and $G = 101/(1 + j0.168)$: magnitude 99.6 (−1.4 %) and a phase lag of 9.6°. A 0.025 % amplifier at DC is a 1.4 % amplifier at 1 kHz; the closed-loop bandwidth is 0.6 MHz/101 = 5.9 kHz.

Offset. $V_{OS}/\beta = 30\ \mu\mathrm{V} \times 101 = 3.03$ mV at the output; without bias compensation $I_B R_f = 0.12$ mV; with a 990 Ω resistor in the non-inverting lead, $I_{OS} R_f = 0.05$ mV. Over a 45 °C rise the drift adds $0.3\ \mu\mathrm{V}/^\circ\mathrm{C} \times 45 \times 101 = 1.36$ mV.

The same stage with a TL072. The 3 mV typical offset becomes 303 mV at the output, 100 times the OP07's; the bandwidth improves to 3 MHz/101 = 29.7 kHz and the bias-current error vanishes. For a 10 mV full-scale sensor signal (1.01 V output), the TL072 stage has a 30 % offset error that must be calibrated out, and its 18 µV/°C drift makes calibration hold only over a few degrees. The OP07, or a zero-drift OPA2188 (6 µV, 0.03 µV/°C), is the right choice; the TL072 is the right choice for an AC-coupled audio or piezo stage, where offset is irrelevant and picoamp bias current matters.

8.3 Summing and difference amplifiers

Summing amplifiers

An inverting stage with several input resistors sums currents into its virtual ground: $v_o = -R_f \sum v_k/R_k$ (*Engineering Physics* §18.4.2). Because the summing node is held at ground, the inputs do not interact; each source sees only its own resistor. That isolation is the reason the summer is the core of audio mixing consoles, of analog computers, and of the output stage of current-steering and R-2R DACs, where binary-weighted currents are summed and converted to voltage

(Chapter 26). The noise gain is $1 + R_f/(R_1 \parallel R_2 \parallel \cdots)$, larger than any single channel's gain: a 16-channel mixer with unity gain per channel has a noise gain of 17, so each added channel raises the output noise even when its input is silent.

A non-inverting summer (inputs through equal resistors to the non-inverting input) averages rather than sums, and its inputs interact through the shared node; it is used where an inversion is unwelcome and the sources are low-impedance.

The difference amplifier and its CMRR

The four-resistor difference amplifier (R_1 and R_2 on the inverting side, R_3 and R_4 on the non-inverting side, Figure 8.2a) gives

$$v_o = \frac{R_2}{R_1}(v_2 - v_1) \qquad \text{when} \qquad \frac{R_4}{R_3} = \frac{R_2}{R_1}$$

It is the simplest way to measure a voltage that rides on a large common-mode level, such as the voltage across a current shunt in a supply rail. The difficulty is the "when". If the ratios differ, part of the common-mode voltage leaks to the output. For resistors of tolerance t the worst-case common-mode rejection ratio of the network alone is

$$\mathrm{CMRR} \approx \frac{1 + G}{4t}$$

with $G = R_2/R_1$. A gain-of-1 stage with 1 % resistors has a CMRR of only 50 (34 dB); the op-amp's own CMRR of 80–120 dB is irrelevant beside that. The source resistances count as part of R_1 and R_3, so an unequal source impedance destroys CMRR as surely as a resistor mismatch, and the two inputs present different impedances (R_1 on the inverting side, $R_3 + R_4$ on the other), which loads unbalanced sources differently.

> **Status of the principle: Difference-amplifier CMRR** $\approx (1 + G)/(4t)$
> *Approximate.* First-order worst-case result for four resistors each off by up to $\pm t$ in the most damaging directions, with an ideal op-amp; statistical (random) mismatch gives about twice this value, and the op-amp's own CMRR combines with the network's as a parallel sum of their reciprocals.

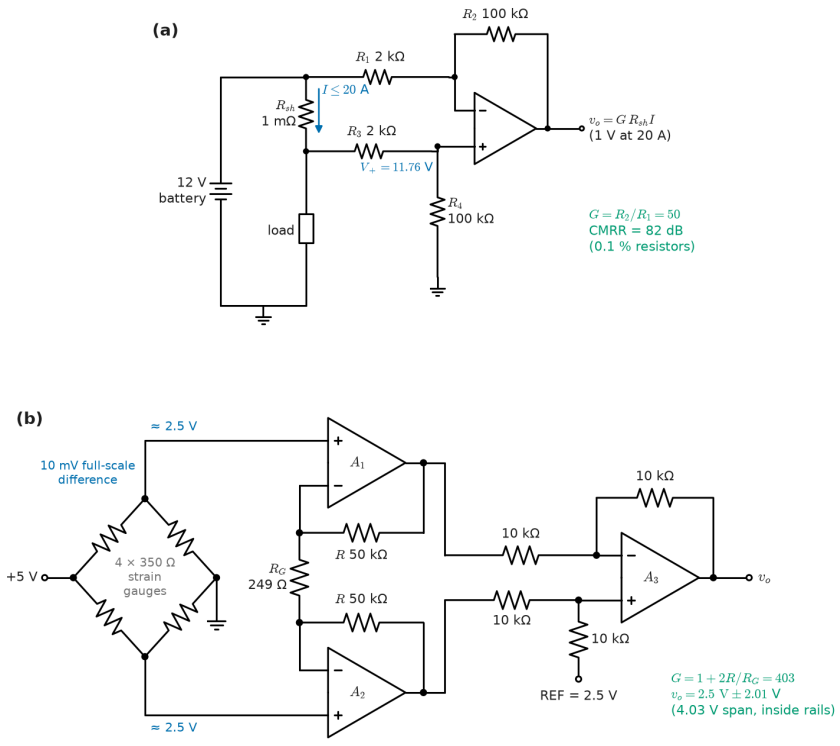


Figure 8.2 — (a) Four-resistor difference amplifier across a 1 milliohm high-side shunt; (b) three-op-amp instrumentation amplifier reading a load-cell bridge.

Worked example 8.2 — A high-side current sense with discrete resistors. A 12 V automotive load draws up to 20 A through a 1 mΩ shunt in the positive rail, giving 20 mV full scale. A difference amplifier with $G = 50$ ($R_1 = R_3 =$ 2 kΩ, $R_2 = R_4 = 100$ kΩ) gives a 1 V full-scale output referred to ground (Figure 8.2a).

CMRR. With 0.1 % resistors, CMRR $= 51/0.004 = 12\,750$ (82.1 dB). The 12 V common mode therefore appears as an input-referred error of $12/12\,750 = 0.94$ mV, which on a 1 mΩ shunt is a phantom 0.94 A: 4.7 % of full scale, varying with battery voltage between 9 V and 16 V. With 0.01 % resistors the CMRR rises to 102 dB and the error to 94 µV (94 mA).

Common-mode range. The non-inverting input sits at $12 \times R_4/(R_3 + R_4) = 11.76$ V, so the op-amp must be powered from at least about 13–14 V, or use a rail-to-rail input part on the 12 V rail itself, or the resistor ratio must attenuate the common mode further. A load-dump transient of 40 V or more on the automotive rail (ISO 16750-2) adds a survival requirement.

The integrated answer. Dedicated current-sense amplifiers (an INA240, for example, with −4 V to 80 V common-mode range) integrate laser-trimmed, thermally matched resistor networks and reach CMRR well above 120 dB, with rejection of the fast common-mode steps of a PWM-switched shunt. For a shunt on a 400 V traction bus, galvanic isolation is required, and an isolated amplifier or isolated delta-sigma modulator replaces the difference amplifier altogether.

8.4 Integrators and differentiators

The practical integrator

The ideal inverting integrator, $v_o = -\frac{1}{RC}\int v_i\,dt$, has infinite gain at DC, so any offset is integrated until the output saturates: an op-amp with 1 mV of offset in an integrator with $RC = 300$ µs ramps at 3.3 V/s and hits the rail in a few seconds, before considering bias current, which charges the capacitor at I_B/C. Three practical cures exist, used in different applications:

- **A resistor across the capacitor** ($R_f \parallel C$) gives a finite DC gain $-R_f/R$ and turns the integrator into a first-order low-pass with corner $1/(2\pi R_f C)$. It integrates only above that corner, and it introduces magnitude and phase errors near it.
- **A reset switch** across the capacitor (an analog switch or MOSFET) zeroes the integrator at the start of each measurement: used in charge-integrating ADCs, photodiode integrators in CT scanners and X-ray detectors, and the dual-slope converter of a bench multimeter.
- **An outer loop** that holds the integrator output near zero, as in a PI controller, where the integrator is supposed to have infinite DC gain because the loop removes the offset by acting on the plant.

A special case is the **charge amplifier** for piezoelectric sensors: the sensor delivers a charge q, the integrator capacitor converts it to $v_o = -q/C_f$, independent of the sensor and cable capacitance (which sits at the virtual ground). An accelerometer giving 10 pC/g into $C_f = 1$ nF produces 10 mV/g whatever the length of its cable; a large R_f (often 1–10 GΩ) sets the low-frequency limit.

Worked example 8.3 — A Rogowski-coil integrator. A Rogowski coil around a busbar produces $v = M\,di/dt$; a coil with $M = 0.3$ µH gives $\omega M I = 94$ mV rms for 1000 A rms at 50 Hz. An integrator must restore $v_o \propto i$, scaled to 1 V rms at 1000 A: $v_o = (M/RC)\,i$, so $RC = MI/v_o = 0.3\ \mu\text{H} \times 1000\ \text{A}/1\ \text{V} = 300$ µs, for example $R = 30$ kΩ, $C = 10$ nF (C0G or film, never X7R, whose capacitance moves with voltage and temperature).

A DC-stabilising $R_f = 3$ MΩ gives a DC gain of 100 (an op-amp with 1 mV offset sits at 0.1 V output instead of the rail) and a corner at $1/(2\pi R_f C) = 5.3$ Hz. At 50 Hz this corner costs a 0.56 % magnitude error and a 6.1° phase lead. For protection relays that is acceptable; for revenue-grade energy metering (IEC 62053-22 class 0.2S), where a 0.1° phase error between current and voltage matters at low power factor, it is not, and practical meters either correct the phase digitally or digitise the coil voltage and integrate in software after the ADC.

> **Status of the principle: Practical integrator** $H(s) = -\dfrac{R_f/R}{1 + sR_fC}$
>
> *Derived.* Nodal analysis of the inverting stage with feedback impedance $R_f \parallel (1/sC)$ and an ideal op-amp; it integrates with phase error $\arctan[1/(\omega R_f C)]$ above the corner, and is further limited at high frequency by the op-amp's gain-bandwidth product.

The differentiator, and why it is rarely used

The ideal differentiator, $v_o = -RC\,dv_i/dt$, has a gain rising at 20 dB/decade without limit. Two consequences make it unusable as drawn. It amplifies high-frequency noise in proportion to frequency. And it is a stability problem: the feedback network's β falls with frequency while the op-amp's gain falls too, so the loop gain approaches −180° and the circuit forms a lightly damped resonance near $\sqrt{\mathrm{GBW}/(2\pi RC)}$: about 69 kHz for $R = 10$ kΩ, $C = 10$ nF and a 3 MHz op-amp, where it rings or oscillates. The practical differentiator adds a resistor in series with the input capacitor and a small capacitor across the feedback resistor, so it differentiates only over a chosen band and rolls off above it. In control systems the same limit appears as the derivative filter of a PID controller (Book 3 Ch. 35).

8.5 Comparators

Comparator versus op-amp

A comparator answers one question: is v_+ above v_-? An op-amp without feedback does answer it, slowly. Its compensation capacitor, which makes it stable with feedback, limits its slew rate, and its output stage recovers from saturation in microseconds; an LM358 used open-loop takes 10–30 µs to switch its output. A **comparator** has no compensation, an output stage designed to saturate cleanly (often open-drain or open-collector, for wired-OR and level translation), and a specified propagation delay that falls with input overdrive.

Table 8.2 — Comparators and an op-amp used as one (typical propagation delays).

Part	Output	Propagation delay	Supply	Use
LM358 (op-amp, open loop)	Push-pull	10–30 µs	3–32 V	Slow thresholds only
LM393	Open collector	0.3–1.3 µs (overdrive dependent)	2–36 V	Battery monitors, general purpose
TLV3201	Push-pull	≈ 40 ns	2.7–5.5 V	Zero-crossing, PWM, encoders
TLV3501	Push-pull	≈ 4.5 ns	2.7–5.5 V	Fast pulse detection, clock recovery
ADCMP600	LVTTL/LVCMOS	≈ 3.5 ns	2.5–5.5 V	LiDAR time-of-flight, oscilloscope triggers

Comparator internals (latch, hysteresis cells, output drivers) are covered in Chapter 10. Applications include overcurrent trip circuits in motor drives (a comparator on the shunt amplifier output can disable the gate drivers within 100 ns, far faster than a microcontroller interrupt), zero-crossing detectors for triac phase control, window comparators that flag under- and overvoltage on a battery cell, and the PWM comparator of an analog converter controller, which compares an error voltage with a sawtooth (Chapter 30).

Hysteresis: the Schmitt trigger

A comparator fed with a slow, noisy signal near its threshold switches many times as the noise crosses back and forth, producing a burst of edges where one was wanted: a speed sensor reads ten times the true speed, a counter overflows. **Hysteresis** fixes this by positive feedback from the output to the non-inverting input, so the threshold moves away from the input each time the output switches (Figure 8.3).

For a non-inverting Schmitt trigger with a reference V_{ref} at the inverting input, the input through R_1 to the non-inverting input and feedback R_2 from a rail-to-rail output (0 or V_{DD}),

$$V_{TH+} = V_{ref}\left(1+\frac{R_1}{R_2}\right), \qquad V_{TH-} = V_{ref}\left(1+\frac{R_1}{R_2}\right) - V_{DD}\frac{R_1}{R_2}, \qquad \Delta V = V_{DD}\frac{R_1}{R_2}$$

> **Status of the principle: Schmitt-trigger thresholds**
> *Derived.* Superposition at the non-inverting input with the output at one rail or the other; assumes rail-to-rail output and negligible input current; with an open-drain output, the pull-up resistor adds to R_2 in the high state and the thresholds shift.

Worked example 8.4 — A noise-immune threshold for a tachometer. A conditioned Hall-sensor signal on a 3.3 V supply crosses mid-supply with ±50 mV of noise. A TLV3201 with hysteresis of 0.2 V centred at 1.65 V is required. Then $R_1/R_2 = 0.2/3.3 = 0.0606$, satisfied by $R_1 = 10\ \text{k}\Omega$ and $R_2 = 165\ \text{k}\Omega$ (E96). The centre of the hysteresis band is $V_{ref}(1 + R_1/R_2) - V_{DD}R_1/(2R_2)$; setting it to 1.65 V gives $V_{ref} = 1.65$ V exactly (a property of placing the band at $V_{DD}/2$). The thresholds are 1.75 V and 1.55 V: the ±50 mV noise is half the margin on either side, and each crossing produces one clean edge.

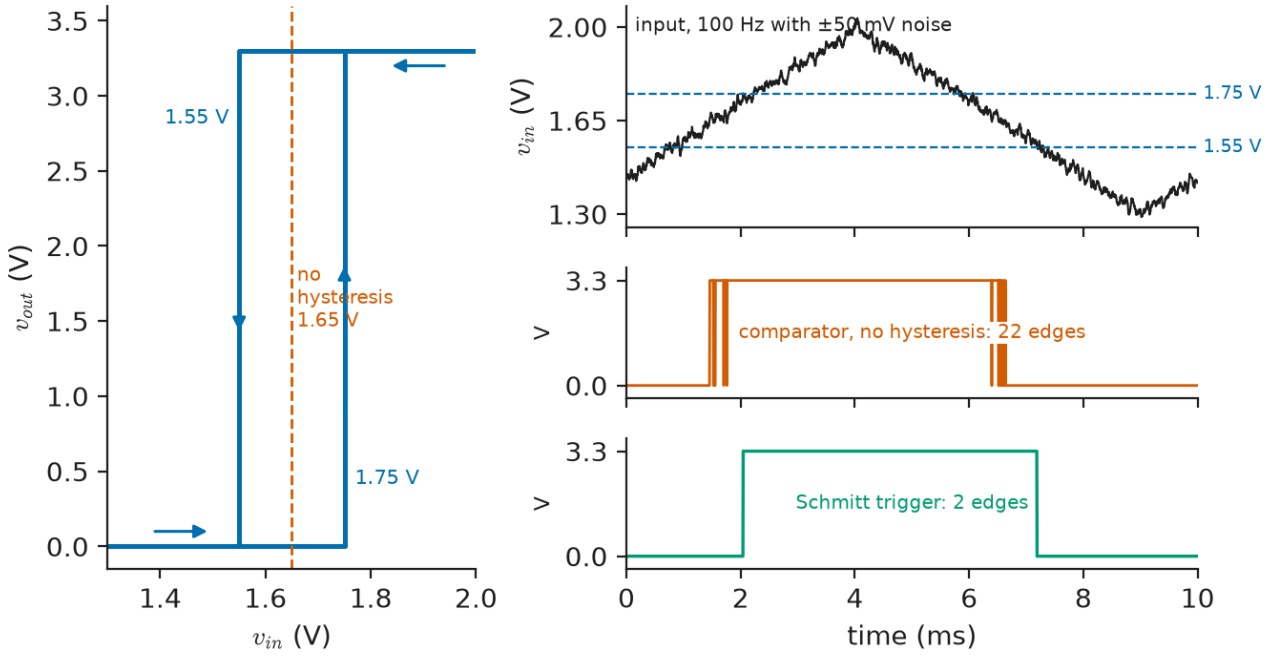


Figure 8.3 — Schmitt trigger of Worked example 8.4: transfer characteristic with hysteresis and its response to a noisy input, compared with a comparator without hysteresis.

Hysteresis is not free: it delays switching by the time the input takes to cross half the hysteresis band, which for a sinusoid adds a phase lag, and it sets a minimum detectable amplitude. Comparators driven from fast signals also need care with layout, since the output edge couples back to the input through ground and stray capacitance and can make even a hysteresis-free comparator oscillate during a slow transition.

8.6 Instrumentation amplifiers

Why the difference amplifier is not enough

The difference amplifier of Section 8.3 has a CMRR limited by resistor matching, input impedances of only kilohms (unequal on the two inputs), and a gain that can be changed only by changing two matched resistors at once. A strain-gauge bridge, an ECG electrode pair or a thermocouple needs the opposite: very high, equal input impedances, high CMRR and gain set by one resistor. The **instrumentation amplifier** (in-amp) provides all three.

The three-op-amp in-amp

In the classic topology (Figure 8.2b) two non-inverting buffers share a single gain resistor R_G between their inverting inputs, followed by a unity-gain difference amplifier. The virtual shorts put the full differential input across R_G; the current v_d/R_G flows through both feedback resistors R, so the first stage's differential gain is $1 + 2R/R_G$. A common-mode input moves both inverting inputs together, no current flows in R_G, and the first stage passes the common mode at unity gain. The difference stage then rejects it. The overall gain is

$$G = \left(1 + \frac{2R}{R_G}\right)\frac{R_{\text{out},2}}{R_{\text{out},1}}$$

and, because the differential signal is amplified before the difference stage while the common mode is not, **the CMRR rises in proportion to gain**: a part with 80 dB at $G = 1$ typically reaches 100–120 dB at $G = 100$.

> **Status of the principle: Three-op-amp in-amp gain $1 + 2R/R_G$ and gain-proportional CMRR**
> *Derived*. Virtual-short analysis of the input stage with ideal op-amps; the CMRR improvement holds while the first stage's common-mode gain is unity and its op-amps are not saturated, and degrades at high frequency as the matching of the two input channels' gain and phase deteriorates.

Integrated in-amps set R internally and specify the gain equation: $G = 1 + 49.4\,\text{k}\Omega/R_G$ for the AD620 and INA826 classes, $G = 1 + 100\,\text{k}\Omega/R_G$ for the zero-drift INA333. The single most common in-amp design error is not gain or noise but **common-mode range**: the internal nodes of the first stage swing to $V_{cm} \pm Gv_d/2$, so a high gain with a common mode near a rail saturates an internal amplifier while the output looks merely wrong. Manufacturers publish a "diamond plot" of allowed output swing against common-mode voltage for each gain and supply; a design is checked against it, not against the input-range line on the first page of the datasheet.

Worked example 8.5 — A load-cell front end. A 2 mV/V load cell excited at 5 V gives 10 mV at full load, riding on a common mode of 2.5 V. An INA333 on a 5 V supply with its REF pin at 2.5 V (from a buffered reference) is to produce a span of about 4 V for a 5 V ADC. The gain needed is 400; $R_G = 100\,\text{k}\Omega/399 = 250.6\,\Omega$, rounded to 249 Ω (E96), gives $G = 402.6$ and a full-scale span of 4.03 V, 2.5 ± 2.01 V at the output for a bipolar load, within the INA333's rail-to-rail output. A CMRR of 100 dB at this gain turns the 2.5 V common mode into $2.5/10^5 = 25\ \mu$V of input-referred error: 0.25 % of full scale, but constant, so it is removed by the tare (zero) calibration of the scale. What tare cannot remove is drift; the INA333's 25 µV maximum offset drifts at 0.1 µV/°C, 0.001 % of full scale per kelvin. Exciting the bridge from the ADC's reference makes the measurement **ratiometric**, so the excitation voltage's own drift cancels (Chapter 26). In a weighing system to OIML R76 class III, the remaining error budget goes to the load cell itself.

Medical front ends add requirements that the in-amp alone cannot meet. An ECG amplifier sees electrode offsets of up to ±300 mV (IEC 60601-2-25 requires tolerance of this), so its in-amp runs at a gain of only 5–10 before a high-pass filter; a "right-leg drive" amplifier feeds the inverted common-mode voltage back into the patient to reduce 50/60 Hz pickup; and RFI filters at the inputs prevent radio-frequency signals from being rectified in the input transistors into DC offsets.

8.7 Precision rectifiers

The superdiode

A silicon diode does not conduct until its anode is about 0.6 V above its cathode (Chapter 6), so a diode rectifier cannot rectify a 100 mV signal at all. Placing the diode inside an op-amp's feedback loop divides its threshold by the open-loop gain: in the **superdiode** half-wave rectifier, the op-amp drives the diode's anode and the output (the cathode, with a load to ground) is fed back to the inverting input. For positive inputs the loop closes through the conducting diode and the output follows the input; for negative inputs the diode is reverse-biased, the loop is open, and the output is held at zero by the load. The effective forward drop is V_D/A.

The weakness is speed. During the negative half-cycle the op-amp, with no feedback, saturates at its negative rail; when the input turns positive it must slew back up through the diode drop before the loop closes. Even before saturation, at frequency f the dead zone is $V_D/A(f) = V_D f/\mathrm{GBW}$. A TL072 (3 MHz) at 10 kHz has $A = 300$ and a dead zone of 0.6 V/300 = 2 mV, a 20 % error on a 10 mV signal.

The improved half-wave and full-wave circuits

The **improved half-wave rectifier** is an inverting stage with two diodes: one in series with the feedback resistor (the output path) and one directly from output to inverting input. In one polarity the first diode conducts and the stage is an inverter of gain $-R_f/R_1$; in the other the second diode conducts and holds the op-amp output one diode drop from virtual ground, so the op-amp never saturates and recovers in tens of nanoseconds instead of microseconds. Adding a second op-amp that sums the half-wave output (with weight 2) and the input (with weight 1) gives the **precision full-wave rectifier**, or absolute-value circuit (Figure 8.4).

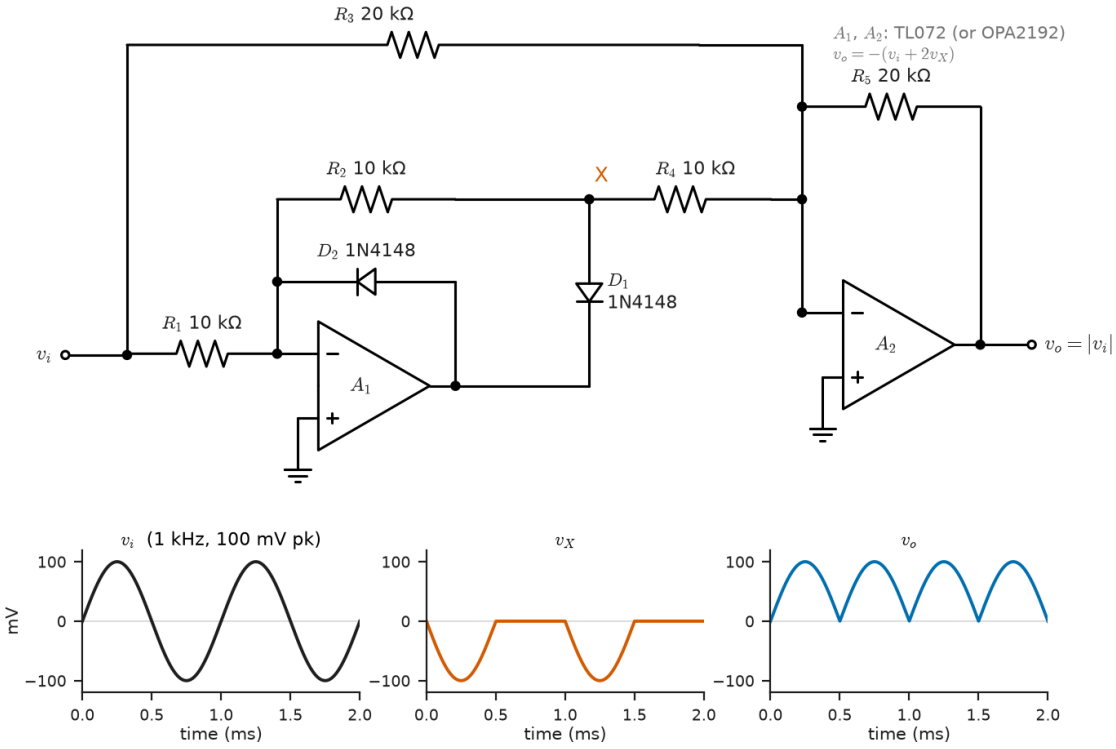


Figure 8.4 — Two-op-amp precision full-wave rectifier (absolute-value circuit) with input and output waveforms.

> **Status of the principle: Superdiode dead zone $V_D/A(f)$**
> *Approximate.* Follows from the loop forcing the op-amp output to $v_i + V_D$ with differential input $(v_i + V_D)/A$; the real high-frequency error is larger, because recovery from saturation and the slew through $2V_D$ at each zero crossing add a delay independent of signal level.

Applications and true RMS

Precision rectifiers are used in average-responding AC voltmeters, envelope detectors for ultrasound and radio, AGC loops and peak detectors. An average-responding meter is calibrated to read the RMS of a sine (scale factor $1.111 = \pi/(2\sqrt{2})$ times the average of the rectified waveform), and it misreads every other waveform: a square wave reads 11 % high, a triangle wave 3.8 % low, and the distorted current of a rectifier-input load can be 30–40 % in error. **True-RMS converters** (log-antilog circuits such as the AD737, or delta-sigma computing converters such as the LTC1966) compute $\sqrt{\overline{v^2}}$ directly; today most meters digitise and compute RMS in firmware.

8.8 Active filters

Choosing a topology

The filter approximations (Butterworth, Chebyshev, Bessel, elliptic) and the decomposition of an n-th order filter into second-order sections, each with its own f_0 and Q, belong to *Engineering Physics* §18.5.3 and Book 3 §13.7–13.8, which also gives the unity-gain Sallen–Key equations and the source-impedance pitfall. The circuit designer's task is to choose a topology for each section, compute component values, and check the result against the op-amp's limits.

Table 8.3 — Second-order active-filter sections.

Topology	Op-amps per section	Sign of gain	Strengths	Weaknesses
Sallen–Key (unity gain)	1 (as follower)	Non-inverting	Few parts, low noise gain (1), good for low Q	Stopband "leakage" at high frequency through the op-amp output impedance; capacitor ratio grows as $4Q^2$
Sallen–Key (gain K)	1	Non-inverting	Equal components possible ($Q = 1/(3-K)$)	Q very sensitive to K near 3
Multiple feedback (MFB)	1	Inverting	Good stopband, low sensitivity, natural with inverting stages	Higher noise gain, capacitor ratio $\geq 4Q^2(1+K)$
State-variable / KHN	3	Low-, high- and band-pass outputs	Independent tuning of f_0 and Q, high Q	Three op-amps, more power
Tow–Thomas biquad	3	Band-pass, low-pass	High Q, easy tuning	Three op-amps
Switched-capacitor	Integrated	—	Corner set by clock, no precision RC needed	Needs anti-alias filter and clock; 1 Hz–100 kHz

For the MFB low-pass with input resistor R_1, feedback resistor R_2 from the output to the internal node, capacitor C_1 from that node to ground, R_3 from the node to the inverting input and C_2 from the output to the inverting input, nodal analysis gives

$$H(s) = -\frac{R_2/R_1}{s^2 C_1 C_2 R_2 R_3 + s C_2\left(R_2 + R_3 + \dfrac{R_2 R_3}{R_1}\right) + 1}$$

so $\omega_0 = 1/\sqrt{R_2 R_3 C_1 C_2}$, the DC gain is $-K = -R_2/R_1$, and $Q = 1/[\omega_0 C_2 (R_2 + R_3(1+K))]$. Real resistor values exist only if $C_1/C_2 \geq 4Q^2(1+K)$, which is the design constraint: pick C_2, pick C_1 from a standard value above the bound, and solve a quadratic for R_3.

Design procedure and component choice

Practical rules decide whether a filter meets its specification in production:

1. **Impedance level.** Resistors between about 1 kΩ and 100 kΩ: below, the op-amp's output current and the source impedance matter; above, thermal noise, bias current and stray capacitance matter.
2. **Capacitors.** C0G/NP0 ceramic (up to tens of nanofarads) or polypropylene film. Class-2 ceramics (X7R, X5R) lose 20–80 % of their capacitance with DC bias and change with temperature, and their voltage-dependent capacitance produces distortion of order 0.01–0.1 % in an audio filter.
3. **Order of sections.** In a cascade, put the low-Q section first so that the high-Q section's resonant peak does not clip the first stage; for noise, the opposite order is sometimes chosen.
4. **Op-amp bandwidth.** Keep GBW $\geq 100\,Q\,f_0$ (Book 3 §13.8); below that, the realised Q rises and high-Q sections can oscillate. The slew rate must cover the largest signal at the highest passband frequency (Section 8.10).
5. **Tolerances.** With 1 % resistors and 5 % capacitors, the pole frequency varies by about ±3 % and high-Q sections vary in Q by more; a Monte Carlo run in a simulator, or the few lines of Python below with random perturbations, confirms the spread.

> **Status of the principle: MFB low-pass realisability** $C_1/C_2 \geq 4Q^2(1+K)$
> *Derived.* The discriminant of the quadratic for R_3 from the ω_0 and Q equations must be non-negative; it holds for an ideal op-amp, and the finite gain-bandwidth product raises the realised Q above the design value.

Worked example 8.6 — A fourth-order anti-aliasing filter for a vibration analyser. A machine-condition monitor samples accelerometer signals at 51.2 kS/s and analyses up to 10 kHz. A fourth-order Butterworth low-pass at $f_c = 10$ kHz, built from two unity-gain MFB sections, precedes the ADC. The sections need $Q = 0.5412$ and $Q = 1.3066$, both with $f_0 = f_c$. Listing 8.1 designs them with $C_2 = 1$ nF, chooses C_1 above the bound $8Q^2 C_2$ (2.34 nF and 13.7 nF), rounds resistors to E96 and evaluates the realised response.

```
import numpy as np

def mfb_lowpass(f0, Q, K, C2, C1):
    """Unity-sign MFB low-pass: returns R1, R2, R3 for gain -K, pole pair (f0, Q)."""
    w0 = 2 * np.pi * f0
    if C1 / C2 < 4 * Q**2 * (1 + K):
        raise ValueError("C1/C2 too small for this Q and gain")
    P = 1 / (w0**2 * C1 * C2)                # R2*R3
    S = 1 / (w0 * Q * C2)                    # R2 + (1+K)*R3
    R3 = (S - np.sqrt(S**2 - 4 * (1 + K) * P)) / (2 * (1 + K))
    R2 = P / R3
    return R2 / K, R2, R3

def e96(r):
    """Round a resistance to the nearest E96 (1 %) value."""
    base = np.round(np.logspace(0, 2, 97)[:-1], 2)
    d = 10 ** np.floor(np.log10(r))
    return d * base[np.argmin(abs(base - r / d))]

def H(f, R1, R2, R3, C1, C2):
    s = 2j * np.pi * f
    return -(R2 / R1) / (s**2 * C1 * C2 * R2 * R3 + s * C2 * (R2 + R3 + R2 * R3 / R1) + 1)

fc, C2 = 10e3, 1e-9
sections = [(0.5412, 3.3e-9), (1.3066, 15e-9)]      # Butterworth Q values, chosen C1
parts = []
for Q, C1 in sections:
    R = [e96(r) for r in mfb_lowpass(fc, Q, 1.0, C2, C1)]
    parts.append((*R, C1, C2))
    print(f"Q = {Q}: C1 = {C1*1e9:.1f} nF, R1, R2, R3 = " + ", ".join(f"{r/1e3:.2f}k" for r in R))
for f in (1e3, 10e3, 20e3, 41.2e3):             # 41.2 kHz aliases onto 10 kHz at 51.2 kS/s
    g = np.prod([H(f, *p) for p in parts])
    print(f"f = {f/1e3:5.1f} kHz: {20*np.log10(abs(g)):7.2f} dB")
```

The output:

```
 Q = 0.5412: C1 = 3.3 nF, R1, R2, R3 = 22.60k, 22.60k, 3.32k
Q = 1.3066: C1 = 15.0 nF, R1, R2, R3 = 7.87k, 7.87k, 2.15k
f =   1.0 kHz:   -0.00 dB
f =  10.0 kHz:   -2.95 dB
f =  20.0 kHz:  -23.98 dB
f =  41.2 kHz:  -49.03 dB
```

Two inverting sections give a non-inverting overall gain of +1. After rounding, the response is −2.95 dB at 10 kHz (ideal −3.01 dB) and −24.0 dB at 20 kHz, as a fourth-order Butterworth should be (Figure 8.5). The op-amp needs GBW $\geq 100 \times 1.31 \times 10\ \text{kHz} = 1.3\ \text{MHz}$; a 10 MHz NE5532 or an OPA1612 leaves margin, an LM358 does not.

The last line is the engineering conclusion. A tone at 41.2 kHz, the frequency that aliases onto 10 kHz at 51.2 kS/s (Book 3 Ch. 19), is attenuated by only 49 dB. A 16-bit system would want about 96 dB. Reaching that with an analog filter alone needs an eighth-order elliptic design with tight components; the modern answer is to oversample with a delta-sigma ADC and do the sharp filtering digitally, leaving the analog filter a gentle job at a much higher frequency (Chapter 26).

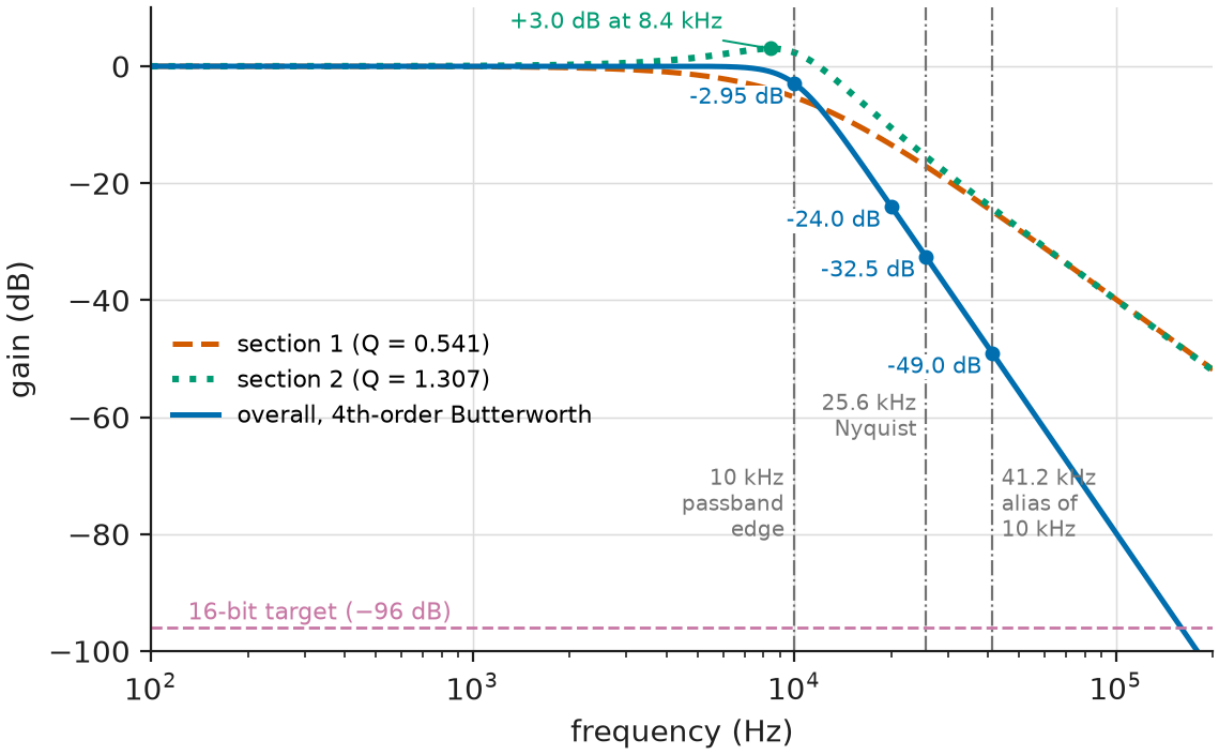


Figure 8.5 — Magnitude response of the fourth-order MFB Butterworth anti-aliasing filter of Worked example 8.6 and of its two sections.

8.9 Oscillators

> **Boundary.**
> The conditions for oscillation (the Barkhausen criterion), amplitude stabilisation, frequency stability and the RC, LC and crystal oscillator families are developed in Chapter 9. This section gives the two op-amp oscillator circuits most often built and their design equations.

The Wien-bridge oscillator

An op-amp non-inverting stage of gain $1 + R_f/R_1$ with a Wien network (series R–C from the output to the non-inverting input, parallel R–C from that input to ground) oscillates at

$$f_0 = \frac{1}{2\pi RC}$$

where the network returns one third of the output in phase. Oscillation needs a gain of exactly 3 ($R_f = 2R_1$) in steady state and slightly more at start-up, so every practical Wien oscillator has an amplitude-control element (antiparallel diodes, a JFET, a thermistor or a lamp) in the gain network. With $R = 15.8\ \text{k}\Omega$ and $C = 10$ nF, $f_0 = 1007$ Hz. The amplitude and distortion that result from the choice of control element are computed in Chapter 9.

The relaxation oscillator

A Schmitt trigger made from a comparator or op-amp (thresholds $\pm\beta V_{sat}$ with $\beta = R_2/(R_1 + R_2)$ from the output divider), whose output charges a capacitor through R that feeds its inverting input, toggles each time the capacitor reaches a threshold. The capacitor voltage is a sequence of exponential segments and the output is a square wave of period

$$T = 2RC \ln \frac{1+\beta}{1-\beta}$$

For $\beta = 0.5$, $T = 2RC \ln 3 = 2.197RC$. With $C = 10$ nF and $R = 45.3$ kΩ (E96), $f = 1/T = 1005$ Hz (Figure 8.6). The frequency depends on β and RC but not on the supply, provided the output saturates symmetrically; rail-to-rail CMOS outputs approach this, and an LM358, whose output does not reach its upper rail, gives an asymmetric duty cycle. At high frequency the slew rate and the comparator's delay add to each half-period, so op-amp relaxation oscillators are used below about 100 kHz; above that, a dedicated comparator or a 555-type timer is used (Chapter 9).

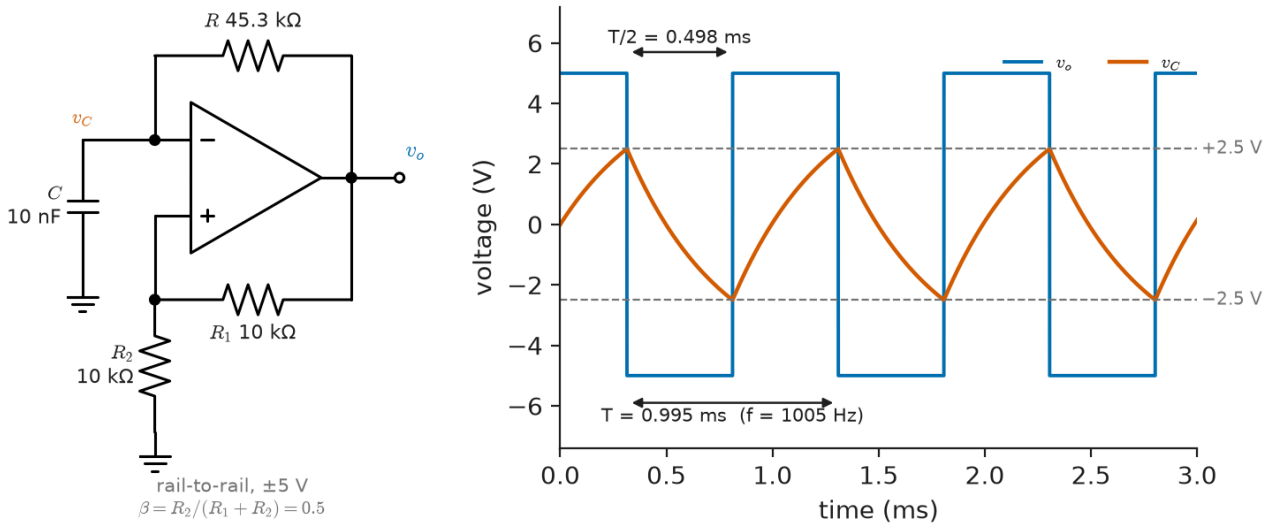


Figure 8.6 — Op-amp relaxation oscillator: capacitor voltage and output square wave for beta = 0.5, R = 45.3 kilohm, C = 10 nF.

A useful variant replaces the RC by an integrator, giving a linear triangle wave and a square wave of constant frequency: the function generator of Chapter 9. A **quadrature oscillator**, two integrators and an inverter in a loop, gives sine and cosine outputs for resolver excitation and phase-sensitive detection.

8.10 Limitations and stability

Output swing and current

An op-amp's output cannot reach its supply rails. Classic bipolar output stages stop 1–2 V short (an LM358's output swings to within about 1.5 V of its positive rail but to within millivolts of the negative); **rail-to-rail output** stages using common-source or common-emitter output devices reach within 10–100 mV, depending on load current. Output current is limited to 10–40 mA in most general-purpose parts, with internal short-circuit protection; driving a 600 Ω headphone or a long cable may need a dedicated buffer. The usable output range is what a design budget uses, and on a 3.3 V supply the difference between a 3.2 V and a 2.0 V swing is the difference between using an ADC's full range and losing 4 dB of dynamic range.

Slew rate and full-power bandwidth

The internal compensation capacitor must be charged by a finite current, which limits the output rate of change to the slew rate SR. A sinusoid of peak V_p at frequency f needs $2\pi f V_p \le SR$, so the **full-power bandwidth** is $f_{FP} = SR/(2\pi V_p)$ (*Engineering Physics* §18.4.3). Slew limiting is nonlinear: a slew-limited sine turns into a triangle, and its distortion appears suddenly above f_{FP} rather than gradually.

Worked example 8.7 — Slew rate and noise in an audio stage. *Slew.* A line driver must deliver 10 V peak to 20 kHz. It needs $SR \ge 2\pi \times 20\ \text{kHz} \times 10\ \text{V} = 1.26\ \text{V}/\mu\text{s}$. An LM358 (0.3 V/µs) has a full-power bandwidth of $0.3\ \text{V}/\mu\text{s}/(2\pi \times 10\ \text{V}) = 4.8$ kHz and turns the top octaves into triangle waves; a TL072 (13 V/µs) or NE5532 (9 V/µs)

passes with a factor of 7 or more in hand, which matters because slew-induced distortion begins well below the hard limit.

Noise. A dynamic microphone (200 Ω source) feeds a non-inverting stage of gain 101 ($R_f = 10\ \text{k}\Omega$, $R_1 = 100\ \Omega$, so $R_f \parallel R_1 = 99\ \Omega$). Uncorrelated noise densities add in power (Book 3 Ch. 29); referred to the input,

$$e_{ni}^2 = e_n^2 + i_n^2 \left(R_s^2 + (R_f \parallel R_1)^2\right) + 4kT \left(R_s + R_f \parallel R_1\right)$$

The source alone contributes $\sqrt{4kTR_s} = 1.82\ \text{nV}/\sqrt{\text{Hz}}$ at 300 K. With an OPA1612 ($e_n = 1.1\ \text{nV}/\sqrt{\text{Hz}}$, $i_n = 1.7\ \text{pA}/\sqrt{\text{Hz}}$) the total is 2.51 nV/√Hz, or 0.36 µV rms over 20 kHz (36 µV at the output) and a noise figure of 2.8 dB. An NE5532 (5 $\text{nV}/\sqrt{\text{Hz}}$, 0.7 $\text{pA}/\sqrt{\text{Hz}}$) gives 5.48 nV/√Hz and 9.6 dB; a TL072 (18 $\text{nV}/\sqrt{\text{Hz}}$) gives 18.1 nV/√Hz, 2.6 µV rms and 20.0 dB. For a 200 Ω source the bipolar parts win; for a 10 MΩ piezo source, where $i_n R_s$ dominates, the JFET part would win by a large factor. The rule is to match the op-amp's noise resistance e_n/i_n (650 Ω for the OPA1612, 1.8 MΩ for the TL072) to the source.

> **Status of the principle: Addition of uncorrelated noise sources in power**
> *Derived.* The variance of a sum of uncorrelated random signals is the sum of their variances; it applies to the thermal noise of different resistors and the op-amp's voltage and current noise, which are physically independent, and fails for correlated sources (the same current noise flowing through two resistors).

Offset drift, zero-drift amplifiers and their side effects

Where DC precision matters (shunts, thermocouples, bridges, battery monitoring), zero-drift op-amps remove offset by chopping or auto-zeroing the input stage at tens to hundreds of kilohertz. The residual offset is a few microvolts with nanovolt-per-kelvin drift, and 1/f noise disappears. In exchange, the chopping frequency and its harmonics appear as ripple at the output and in the input current, and an input signal near the chopping frequency can alias to DC. They are the right parts for low-frequency precision; for signals approaching their chopping frequency a precision bipolar part is safer.

Common-mode limits, phase reversal and protection

Exceeding the common-mode input range of some older JFET-input op-amps, including the TL07x family near its negative rail, causes **phase reversal**: the output jumps to the opposite rail, and a servo or control loop built around it locks up at full drive. Inputs driven beyond the rails conduct through ESD diodes; series resistors limit the current to the datasheet's few milliamps. CMRR and PSRR fall with frequency (typically 20 dB/decade above 1–10 kHz), so supply ripple from a switching regulator at 500 kHz passes almost unattenuated; every op-amp needs a local 100 nF ceramic decoupling capacitor at its supply pins, and precision circuits add filtering.

Stability in practice

Internally compensated op-amps are stable as followers with resistive loads; three common circuit features change that, and each is analysed with loop gain and phase margin in Chapter 9:

- **Capacitive load.** The op-amp's open-loop output resistance and a load capacitance (a cable, a MOSFET gate, an ADC input or a reference decoupling capacitor) add a pole inside the loop. Symptoms are overshoot and ringing on a step and peaking near the crossover frequency. Cures: an isolation resistor of 10–100 Ω between the output and the load, with feedback taken before it; an op-amp specified for capacitive loads; or a dual-feedback network.
- **Input capacitance with large feedback resistors.** The input capacitance (op-amp, photodiode, cable) and R_f add a pole in the feedback path. A small capacitor C_f across R_f restores phase margin; the transimpedance amplifier of Chapter 9 is the standard example.
- **Low noise gain with decompensated op-amps.** Parts designed for speed (stable only at noise gains above, say, 5 or 10) oscillate as followers.

The diagnostics are the same in every case: a square-wave test at small amplitude that shows overshoot above about 20 % means less than about 45° of phase margin, and any sustained ringing means the circuit is one component tolerance away from oscillation.

What remains open

Open question	Where it is answered
How does feedback change gain, impedance and bandwidth for each feedback topology, and how are phase margins of op-amp circuits computed and designed?	Chapter 9 (Feedback and Oscillators)
What conditions make a circuit oscillate, how is its amplitude controlled, and how are LC and crystal oscillators and function generators built?	Chapter 9
What transistor circuits are inside an op-amp and a comparator, and how is internal compensation designed?	Chapter 10 (Analog Integrated Circuits)
How is an op-amp front end matched to an ADC, and how are ratiometric bridges and delta-sigma converters used?	Chapter 26 (Embedded Interfaces)
How are sampling rate and anti-aliasing requirements derived?	*Circuits, Signals and Communications*, Ch. 19
How are the Butterworth, Chebyshev and elliptic prototypes and their pole positions obtained?	*Engineering Physics*, Ch. 18 §18.5.3; *Circuits, Signals and Communications*, Ch. 13
How is a comparator used as the PWM modulator of a converter controller?	Chapter 30 and Chapter 31

Next: Chapter 9 steps back from particular circuits to feedback itself: the four topologies, the trade between gain and bandwidth, the stability of op-amp loops with real loads, and the deliberate instability that makes oscillators.

Chapter 9 — Feedback and Oscillators

Part II · Analog Electronics

Two failures land on the same engineer's desk in one week. The first is a sensor buffer in a process-control cabinet: a unity-gain op-amp stage that has worked for years starts to oscillate at 5 MHz when a customer installs it with 10 m of coaxial cable, and the radiated interference upsets a nearby radio modem. The second is a smart electricity meter whose 16 MHz crystal oscillator fails to start in about one unit in five hundred when the meter is installed outdoors in a Scandinavian winter. The first circuit was meant to be stable and became an oscillator; the second was meant to be an oscillator and refused. Both failures have the same cause. A feedback loop either returns a signal smaller than it sent, with a phase that cannot reinforce it, or it does not; the cable added phase lag and turned a stable loop into an oscillator, and the cold raised the crystal's loss resistance until the oscillator's loop gain fell below one. This chapter treats feedback as a single idea used in both directions: negative feedback to make amplifiers precise, wide-band and stable, and positive feedback, arranged to satisfy one condition at one frequency, to make oscillators.

> **How does negative feedback change an amplifier's gain, bandwidth and impedances, and what decides whether a feedback amplifier is stable with a real load?**
> **What conditions make a circuit oscillate at a well-defined frequency and amplitude, and how are RC, LC and crystal oscillators and waveform generators designed?**

Concept	Engineering destination
Positive and negative feedback	Precision amplifiers, Schmitt triggers, latches, oscillators
Feedback topologies	Voltage buffers, transimpedance photodiode receivers, 4–20 mA transmitters, current amplifiers
Gain and bandwidth	Choosing op-amps and stage counts for a gain-bandwidth target; distortion versus frequency
Stability	Op-amps driving cables and ADC inputs, photodiode amplifiers, regulators
Barkhausen criterion	The design condition of every oscillator; amplitude control
RC oscillators	Audio test sources, sensor excitation, low-frequency timing
LC oscillators	RF local oscillators, VCOs in PLL synthesisers, inductive proximity sensors
Crystal oscillators	MCU and processor clocks, Ethernet, USB and radio references, real-time clocks
Waveform generators	Function generators, 555 timers, PWM sawtooths, DDS synthesisers

Roadmap. Feedback in both signs (9.1) → the four topologies and what each does to impedances (9.2) → the gain–bandwidth trade and what feedback does and does not fix (9.3) → stability of real op-amp loops: capacitive loads and photodiode amplifiers (9.4) → the oscillation condition (9.5) → RC oscillators (9.6) → LC oscillators (9.7) → crystal oscillators (9.8) → waveform generators (9.9).

9.1 Positive and negative feedback

Boundary.
The closed-loop formula $T = A/(1 + A\beta)$, loop gain, sensitivity $S = 1/(1 + L)$, disturbance rejection and the costs of feedback are developed in *Circuits, Signals and Communications*, Ch. 34 §34.2; the Nyquist criterion and gain and phase margins in §34.8–34.9; frequency-domain signs of marginal stability, the margin–damping table and the phase-shift oscillator's factor of 29 in Ch. 13 §13.9; the two-port interconnection rules behind the feedback topologies in Ch. 12 §12.8; resonators and Q in Ch. 13 §13.4–13.5. Feedback across physical domains is the theme of *Engineering Physics*, Ch. 21. This chapter applies those results to electronic circuits and does not re-derive them. The op-amp circuits themselves are in Chapter 8; internal op-amp compensation is in Chapter 10.

Negative feedback: what it buys

An amplifier of forward gain A with a feedback network of transmission β, subtracting the fed-back signal at the input, has closed-loop gain $A/(1 + A\beta)$, which tends to $1/\beta$ when the loop gain $T = A\beta$ is large (Book 3 §34.2). For electronic circuits the consequences are worth listing with their sizes, because each is a design tool:

1. **Desensitivity.** A fractional change in A appears in the closed-loop gain divided by $1 + T$. An op-amp whose open-loop gain varies by a factor of three between parts and temperatures yields closed-loop gain set by two resistors to 0.01 %.
2. **Bandwidth extension.** Where A has a dominant pole, the closed-loop pole moves out by $1 + T_0$ (Section 9.3).
3. **Distortion reduction.** Distortion generated inside the loop is divided by $1 + T$ at the frequency of the distortion product: the degenerated transistor stage of Chapter 7 showed a factor of 5.5, and an audio power amplifier with 60 dB of loop gain at 1 kHz reduces its output stage's crossover distortion a thousandfold.
4. **Impedance control.** Feedback multiplies or divides the input and output impedances by $1 + T$, in a direction set by the topology (Section 9.2).

What feedback does not do is equally important. It does not reduce the noise or offset of the input stage, which sits at the point where the error is measured and is indistinguishable from the signal; it reduces only disturbances entering *after* the comparison point. And it is subject to stability (Section 9.4): a loop with lag can return the correction too late.

Positive feedback: what it is used for

With the sign reversed, the closed-loop gain becomes $A/(1 - A\beta)$, larger than A, and infinite when $A\beta = 1$. In electronics positive feedback is used deliberately in four ways:

- **Hysteresis** (the Schmitt trigger of Chapter 8): a loop gain above one at DC makes the circuit bistable, with two thresholds.
- **Regeneration and latching:** the cross-coupled inverters of an SRAM cell or a comparator's latch (Chapter 10, Chapter 14) use a loop gain greater than one to resolve a small difference into a full logic level within a short time.
- **Negative resistance:** a positive-feedback circuit can present a negative input resistance, used to cancel the loss of a resonator (Section 9.7) or the leakage of a sample-and-hold.
- **Oscillation:** a loop gain of exactly one, with zero net phase, at one frequency (Section 9.5).

Many practical circuits contain both: a Wien-bridge oscillator has a negative-feedback gain network and a positive-feedback frequency-selective network, and a Howland current source balances the two to obtain a high output impedance.

Status of the principle: Effects of negative feedback scale with $1 + T$
Derived. Gain sensitivity, distortion, dominant-pole bandwidth and the impedances at the sensing and mixing ports change by the factor $1 + T(j\omega)$ at each frequency; the benefit vanishes wherever the loop gain has fallen near one, which for every real amplifier happens at high frequency.

9.2 Feedback topologies

Sensing and mixing

A feedback amplifier is classified by how the feedback network **samples** the output (in shunt with it, sensing voltage; or in series with it, sensing current) and how it **mixes** the fed-back signal with the input (in series, adding voltages; or in shunt, adding currents). The four combinations stabilise four kinds of amplifier. Book 3 §12.8 shows that each corresponds to adding one set of two-port parameters (h, y, z or g) of the amplifier and the feedback network; here the interest is in what each does in a circuit (Figure 9.1).

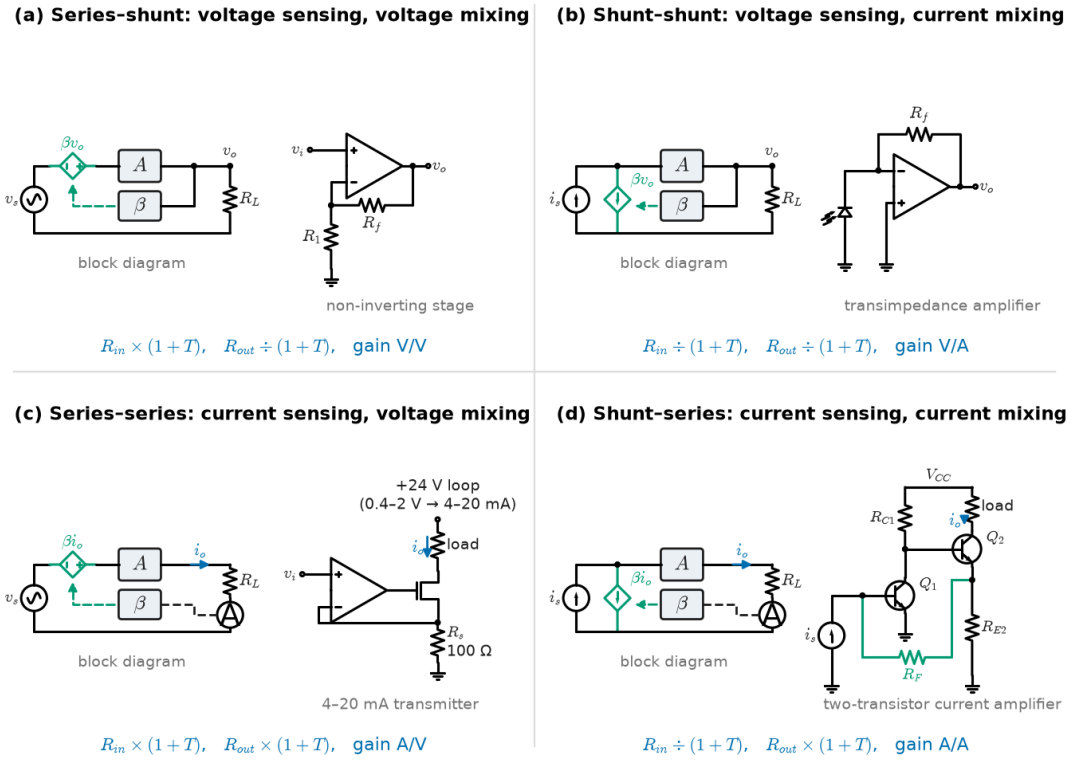


Figure 9.1 — The four feedback topologies, each shown as a block diagram and as its common op-amp or transistor realisation.

Table 9.1 — The four feedback topologies.

Topology (sensing–mixing)	Two-port sum	Stabilised gain	Input impedance	Output impedance	Typical circuit
Series–shunt (voltage–voltage)	h	Voltage gain v_o/v_i	$\times (1+T)$	$\div (1+T)$	Non-inverting op-amp stage, follower
Shunt–shunt (voltage–current)	y	Transimpedance v_o/i_i	$\div (1+T)$	$\div (1+T)$	Photodiode TIA, inverting stage
Series–series (current–voltage)	z	Transconductance i_o/v_i	$\times (1+T)$	$\times (1+T)$	V-to-I converter, 4–20 mA transmitter, degenerated CE stage
Shunt–series (current–current)	g	Current gain i_o/i_i	$\div (1+T)$	$\times (1+T)$	Current amplifier, feedback current mirror

The rule behind the impedance columns is simple: **series connection at a port multiplies the impedance there by $1 + T$; shunt connection divides it.** Sensing the output voltage in shunt makes the output a better voltage source (lower impedance); sensing the output current in series makes it a better current source (higher impedance). Mixing in series makes the input draw less current (higher impedance); mixing in shunt makes the input a virtual ground (lower impedance).

Blackman's impedance formula

The two-port method requires the feedback network to be separable and the loading of each network on the other to be accounted for, which becomes awkward in transistor circuits where the "feedback network" shares elements with the amplifier. A more general result, **Blackman's formula**, gives the impedance at any port of a circuit containing a controlled source:

$$Z = Z_0 \, \frac{1 + T_{sc}}{1 + T_{oc}}$$

where Z_0 is the impedance with the controlled source set to zero, and T_{sc} and T_{oc} are the loop gains (return ratios) with the port short-circuited and open-circuited. For a shunt-connected port, shorting it kills the feedback ($T_{sc} = 0$) and $Z = Z_0/(1 + T_{oc})$; for a series port, opening it kills the feedback ($T_{oc} = 0$) and $Z = Z_0(1 + T_{sc})$. Table 9.1 is the special case, and the formula handles every intermediate case without identifying a topology.

> **Status of the principle: Blackman's impedance formula**
> *Derived.* Follows from superposition applied to a linear circuit with one controlled source, by computing the port impedance as the source's gain varies; exact for linear circuits, and it reduces to the $(1 + T)$ rules of Table 9.1 when the feedback is fully killed by shorting or opening the port.

Three circuits

The follower's output impedance is inductive. For a voltage follower ($\beta = 1$, series–shunt) with an open-loop output resistance $R_o = 50\ \Omega$ and $A(f) \approx \mathrm{GBW}/(jf)$ with $\mathrm{GBW} = 10$ MHz, the closed-loop output impedance is $R_o/(1 + A)$: 5 mΩ at 1 kHz where $A = 10^4$, but 4.98 Ω at 1 MHz where $|A| = 10$. Because $1/A$ rises in proportion to frequency, $Z_{out} \approx j\omega R_o/(2\pi\,\mathrm{GBW})$ is an inductance $L_{eq} = R_o/(2\pi\,\mathrm{GBW}) = 0.80\ \mu\mathrm{H}$. Connect a 1 nF cable and the inductance resonates with it at $1/(2\pi\sqrt{L_{eq}C}) = 5.6$ MHz: this is the 5 MHz oscillation of the opening paragraph, seen from the output port (Section 9.4 sees the same thing from the loop gain).

A 4–20 mA transmitter is series–series feedback. An op-amp drives a MOSFET whose source current flows through a sense resistor $R_s = 100\ \Omega$, and the sense voltage is fed back to the inverting input. The loop forces $i_o = v_i/R_s$, so 0.4–2.0 V gives 4–20 mA, the industrial current-loop standard (IEC 60381-1). The output impedance at the drain is raised from the MOSFET's r_o (say 50 kΩ) by roughly the loop gain; with $T = 10^4$ at low frequency it exceeds 500 MΩ, so a change of loop supply from 10 V to 30 V changes the current by 0.04 μA. That high impedance is the reason current loops ignore wire resistance and ground offsets across a plant.

The transimpedance amplifier is shunt–shunt feedback. A photodiode's current enters the inverting input of an op-amp with feedback resistor R_f; $v_o = -i_i R_f$ and the input impedance is $R_f/(1 + A)$, nearly zero, so the photodiode's capacitance is held at constant voltage and does not slow the response. The stability of this circuit, which looks harmless, is the subject of Worked example 9.3.

9.3 Gain and bandwidth

The single-pole trade

When the open-loop gain has one dominant pole, $A(s) = A_0/(1 + s/\omega_p)$, closing the loop with feedback β gives a closed-loop gain $A_0/(1 + A_0\beta)$ and a closed-loop pole at $\omega_p(1 + A_0\beta)$. The product of the two is $A_0\omega_p$, the **gain-bandwidth**

product, independent of β: the gain given up is returned as bandwidth (*Engineering Physics* §18.4.3). With an op-amp, the gain that counts is the **noise gain** $1/\beta$, not the signal gain (Chapter 8), so

$$f_{-3\,\mathrm{dB}} \approx \beta\,\mathrm{GBW}, \qquad t_r \approx \frac{0.35}{f_{-3\,\mathrm{dB}}}$$

An inverting stage of gain -1 has a noise gain of 2 and therefore half the bandwidth of a follower.

> **Status of the principle: Constant gain-bandwidth product under feedback**
> *Approximate.* Exact for a single-pole forward path; for real op-amps it holds for noise gains above about 2–3, where the closed-loop crossover lies well below the second pole; near unity gain the second pole produces peaking and the bandwidth can exceed GBWβ.

Cascading stages for gain and bandwidth

When one stage cannot provide the gain at the required bandwidth, the gain is split. n identical stages each of gain $G^{1/n}$ have individual bandwidths $\mathrm{GBW}/G^{1/n}$, and n coincident poles shrink the overall bandwidth by $\sqrt{2^{1/n}-1}$.

Worked example 9.1 — 60 dB of gain from 10 MHz op-amps. An ultrasound receiver needs a gain of 1000 with the widest possible bandwidth, using op-amps with $\mathrm{GBW} = 10$ MHz.

Stages	Gain per stage	Bandwidth per stage	Overall bandwidth
1	1000	10 kHz	10 kHz
2	31.6	316 kHz	204 kHz
3	10	1.0 MHz	510 kHz
4	5.62	1.78 MHz	774 kHz
5	3.98	2.51 MHz	969 kHz

Two stages give twenty times the bandwidth of one; three give fifty. Beyond three or four the returns diminish while noise, offset, power and the risk of peaking in low-gain stages grow. Three stages of 20 dB is the usual answer, with the first stage chosen for noise and the last for output swing and slew rate.

Distortion and loop gain versus frequency

Because the loop gain falls with frequency, every benefit of Section 9.1 shrinks with frequency. In an audio power amplifier whose loop gain crosses unity at 1 MHz with a single-pole roll-off, the loop gain is 1000 (60 dB) at 1 kHz but only 50 (34 dB) at 20 kHz, so crossover distortion products are suppressed twenty times less at the top of the audio band. That is why amplifier THD is specified at 1 kHz *and* at 20 kHz, why the second figure is always worse, and why designers push loop gain up with two-pole compensation schemes that hold more gain in the audio band while still crossing unity safely.

9.4 Stability

Reading op-amp stability from the Bode plot

Book 3 §34.9 defines the phase margin at the loop-gain crossover and relates it to overshoot and peaking (the table in §13.9: 45° gives about 23 % overshoot, 60° about 9 %, and $\zeta \approx \mathrm{PM}/100^\circ$). For op-amp circuits there is a convenient graphical form: plot the open-loop gain $|A|$ and the noise gain $1/|\beta|$ on the same axes. Since $|T| = |A|/|1/\beta|$, the loop crosses over where the two curves intersect, and the phase margin depends on the difference of their slopes there, the **rate of closure**:

- 20 dB/decade (a -20 dB/decade A meeting a flat $1/\beta$): phase margin 45–90°, stable.
- 40 dB/decade (either A falling at -40 dB/decade, or $1/\beta$ rising at $+20$ dB/decade where A falls at -20): phase margin near 0–20°, marginal or unstable.

> **Status of the principle: Rate-of-closure stability rule**
> *Approximate.* Follows from Bode's gain–phase relation for minimum-phase loops: each 20 dB/decade of slope difference corresponds to about 90° of phase; reliable when poles and zeros are separated by at least a decade from the crossover, and to be confirmed by computing the margin when they are not.

Internally compensated op-amps are designed so that $|A|$ falls at -20 dB/decade down to unity gain, with their second pole just above it; the internal compensation that achieves this is covered in Chapter 10. The external circuit can still spoil the loop in two common ways: a pole added in the forward path at the output (a capacitive load), or a zero added to $1/\beta$ by capacitance at the inverting input.

Capacitive loads

The op-amp's open-loop output resistance R_o and a load capacitance C_L add a pole at $1/(2\pi R_o C_L)$ inside the loop, after the amplifier and before the feedback tap.

Worked example 9.2 — An op-amp follower driving a cable. Model the op-amp of the opening paragraph by $A_0 = 10^5$, GBW $= 10$ MHz, a second pole at 60 MHz and $R_o = 50\ \Omega$, connected as a follower ($\beta = 1$). Computing the loop gain $A(j\omega)/(1 + j\omega R_o C_L)$ numerically:

Load	Added pole	Crossover	Phase margin
None	—	9.9 MHz	80.7°
100 pF	31.8 MHz	9.5 MHz	64.5°
1 nF (10 m of 100 pF/m coax)	3.18 MHz	5.2 MHz	26.5°
10 nF	318 kHz	1.8 MHz	8.5°

With the cable, the phase margin falls to 26.5°, and the closed-loop response peaks by 6.8 dB at 5.3 MHz with a 47 % overshoot on a small step: the circuit is not oscillating on the bench, but a little extra phase lag from temperature, a longer cable or a reactive termination finishes the job. The standard cure is an **isolation resistor** R_{iso} between the op-amp output and the cable, with the feedback taken from the op-amp side. The loop gain becomes $A(1 + sR_{iso}C_L)/[1 + s(R_o + R_{iso})C_L]$: the pole moves down but a zero follows it, and above the zero the load is isolated.

R_{iso}	Phase margin	Load-side corner $1/(2\pi R_{iso}C_L)$
10 Ω	40.8°	15.9 MHz
22 Ω	53.7°	7.2 MHz
47 Ω	69.7°	3.4 MHz
100 Ω	79.3°	1.6 MHz

With 47 Ω the margin returns to 70° and the peaking disappears; the cost is a low-pass corner of 3.4 MHz at the load, a small DC error if the load draws current, and, for a cable, the bonus of an approximate back-termination. When DC accuracy at the load is critical, a dual-feedback network (DC feedback from the load, AC feedback from the op-amp output through a small capacitor) gives both. Figure 9.2 shows the loop gains.

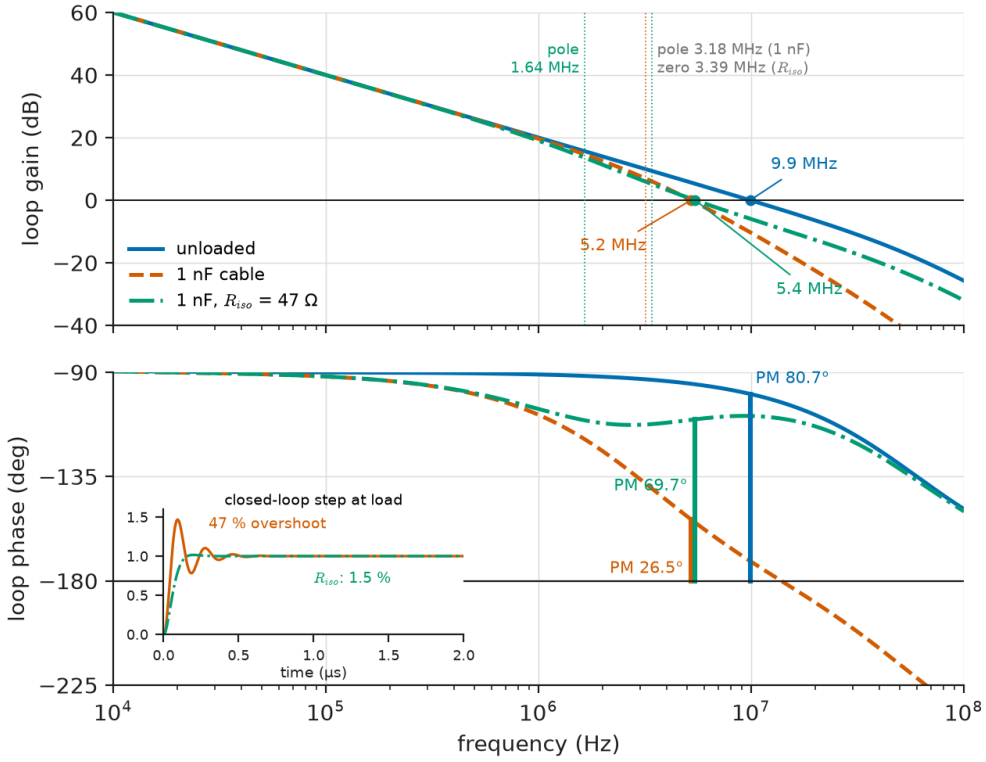


Figure 9.2 — Loop gain of an op-amp follower (GBW 10 MHz, second pole 60 MHz, Ro = 50 ohm) unloaded, with a 1 nF cable load, and with a 47 ohm isolation resistor.

Input capacitance and the transimpedance amplifier

Capacitance from the inverting input to ground, C_{in} (the op-amp's input capacitance plus a photodiode, a cable, or stray), forms a pole with R_f in the feedback network. The noise gain $1/\beta = 1 + sR_fC_{in}$ then rises at 20 dB/decade from $1/(2\pi R_fC_{in})$, meeting the op-amp's falling gain at 40 dB/decade. The fix is a feedback capacitor C_f across R_f, which flattens $1/\beta$ at $1 + C_{in}/C_f$ above $1/(2\pi R_fC_f)$. For about 45° of phase margin with a single-pole op-amp, C_f is placed so that the noise gain flattens where it meets $|A|$:

$$C_f \approx \sqrt{\frac{C_{in}}{2\pi R_f \,\mathrm{GBW}}}, \qquad f_{-3\,\mathrm{dB}} \approx \sqrt{\frac{\mathrm{GBW}}{2\pi R_f C_{in}}}$$

> **Status of the principle: TIA feedback-capacitor rule**
> *Approximate.* Places the noise-gain zero at the intersection with a single-pole open-loop gain, giving about 45° of margin when $C_f \ll C_{in}$; the second op-amp pole, the op-amp's own input capacitance and the parasitic capacitance of R_f (0.05–0.2 pF for an 0603 resistor) shift the optimum, so the value is a starting point for computation or measurement.

Worked example 9.3 — A photodiode receiver for an optical encoder. A photodiode with 10 pF of capacitance plus 5 pF of op-amp input capacitance ($C_{in} = 15$ pF) feeds a TIA with $R_f = 100$ kΩ (100 mV/µA) built with a 10 MHz op-amp (second pole at 60 MHz). The rule gives $C_f = 1.55$ pF and a bandwidth near 1 MHz. Listing 9.1 computes the loop gain and the closed-loop transimpedance.

```
 import numpy as np

# Photodiode TIA: op-amp A(s) = A0 / ((1 + s/wp1)(1 + s/wp2)), feedback Rf || Cf
A0, GBW, fp2 = 1e5, 10e6, 60e6            # DC gain, gain-bandwidth, second pole
Rf, Cin = 100e3, 15e-12                    # feedback resistor, photodiode + input capacitance

def loop_gain(f, Cf):
    s = 2j * np.pi * f
    A = A0 / ((1 + s * A0 / (2 * np.pi * GBW)) * (1 + s / (2 * np.pi * fp2)))
    Zf = Rf / (1 + s * Rf * Cf)
    beta = 1 / (1 + s * Cin * Zf)          # divider Zf against 1/(s*Cin)
    return A * beta

def margins(Cf):
    f = np.logspace(3, 9, 200001)
    L = loop_gain(f, Cf)
    i = np.argmax(np.abs(L) < 1)           # first crossing of 0 dB
    pm = 180 + np.degrees(np.angle(L[i]))
    T = Rf / (1 + 2j*np.pi*f*Rf*Cf) * L / (1 + L)   # transimpedance, V/A
    peak = 20 * np.log10(np.abs(T).max() / Rf)
    f3 = f[np.argmax(np.abs(T) < Rf / np.sqrt(2))]
    return f[i], pm, peak, f3

Cf_45 = np.sqrt(Cin / (2 * np.pi * Rf * GBW))
for Cf in (0.0, Cf_45, 2 * Cf_45):
    fc, pm, peak, f3 = margins(Cf)
    print(f"Cf = {Cf*1e12:4.2f} pF: crossover {fc/1e6:5.2f} MHz, PM = {pm:5.1f} deg, "
          f"peaking {peak:4.1f} dB, -3 dB at {f3/1e6:4.2f} MHz")
```

The output:

```
 Cf = 0.00 pF: crossover  1.03 MHz, PM =   4.9 deg, peaking 21.3 dB, -3 dB at 1.60 MHz
Cf = 1.55 pF: crossover  1.22 MHz, PM =  53.2 deg, peaking  1.0 dB, -3 dB at 1.23 MHz
Cf = 3.09 pF: crossover  1.78 MHz, PM =  75.0 deg, peaking -0.0 dB, -3 dB at 0.65 MHz
```

Without C_f the circuit has 4.9° of margin and a 21 dB resonant peak near 1 MHz: it rings violently on every edge from the encoder disc and oscillates with any extra capacitance. With the computed 1.55 pF the margin is 53° (a little more than the 45° the single-pole rule promises), the peaking is 1 dB and the bandwidth 1.23 MHz. Doubling C_f buys 75° of margin but halves the bandwidth to 0.65 MHz, because above $1/(2\pi R_f C_f)$ the feedback capacitor, not the resistor, sets the transimpedance. The designer's choice is between speed and ringing; for a digital encoder signal, about 60° is the usual target. A faster receiver needs either a smaller photodiode, a higher-GBW op-amp ($f_{-3\,\mathrm{dB}}$ grows only as $\sqrt{\mathrm{GBW}}$), or a bootstrap that removes the photodiode capacitance from the input; LiDAR receivers at hundreds of megahertz use dedicated TIA ICs.

Other sources of instability

- **Decompensated op-amps** (stable only above a stated noise gain, such as 5 or 10) offer more bandwidth for high-gain stages but oscillate as followers or in inverting stages of low gain.
- **Conditional stability:** a loop with more than 180° of phase lag below crossover is stable at its design gain but oscillates if saturation or a fault reduces its gain (Book 3 §34.9); this occurs in high-order loops such as some regulators and audio amplifiers with nested compensation.
- **Supply paths:** without local decoupling, a shared supply impedance couples an output stage back into an input stage and a multistage amplifier oscillates through its rails.

- **Converter control loops** use the same margins with the converter's LC filter in the loop; their compensation belongs to Chapter 30 and Book 3 Ch. 35.

9.5 Barkhausen criterion

The oscillation condition

An oscillator is a feedback loop designed to fail the stability test at exactly one frequency. In the language of Book 3, its closed-loop poles sit on the $j\omega$ axis at $\pm j\omega_0$; equivalently, a signal at ω_0 travelling round the loop returns with the same amplitude and phase. With the forward gain A and frequency-selective feedback β connected for positive feedback, the **Barkhausen criterion** is

$$A(j\omega_0)\,\beta(j\omega_0) = 1$$

that is, $|A\beta| = 1$ and $\angle A\beta = 0^\circ$ (or a multiple of 360°). The phase condition sets the frequency; the magnitude condition sets the gain needed.

> **Status of the principle: Barkhausen criterion**
> *Derived*. The condition for a pair of closed-loop poles of a linear loop to lie on the imaginary axis; it is necessary for steady oscillation of a linear loop but not sufficient for a practical oscillator, which must start (requiring $|A\beta| > 1$ initially, poles in the right half-plane) and then settle (requiring a nonlinearity that reduces the effective gain to exactly one at the final amplitude).

Start-up and amplitude control

A loop that satisfies the criterion exactly has poles exactly on the axis; any component drift moves them into the left half-plane, and the oscillation dies, or into the right, and it grows. Practical oscillators are therefore designed with a small-signal loop gain of 1.2 to 3 (or, in crystal oscillators, 5 to 20) so that they start reliably from noise, and with a mechanism that lowers the gain as the amplitude grows until the *average* loop gain at the fundamental is exactly one. The mechanism determines amplitude stability and distortion:

Amplitude-control mechanism	How it works	Typical distortion	Used in
Hard limiting (supply rails, transistor cut-off)	Gain collapses during part of the cycle	1–10 %	Simple LC and relaxation oscillators, MCU crystal drivers
Soft clipping (diodes in the gain network)	Gain falls smoothly as diodes conduct	1–5 %	Low-cost Wien and phase-shift oscillators
Automatic gain control (JFET or multiplier, with a rectifier and filter)	Slow loop holds amplitude to a reference	0.01–0.1 %	Audio test oscillators, sensor excitation
Thermal (lamp, thermistor)	Resistance rises with RMS power, time constant of seconds	0.001–0.01 %	Classic laboratory audio oscillators

The slow mechanisms give low distortion because they adjust gain over many cycles rather than within one; their weakness is amplitude "bounce" after a frequency change.

Frequency stability

How firmly the loop holds its frequency depends on how fast the phase of β changes with frequency near ω_0. If a component change or the amplifier adds a phase error $\Delta\phi$, the loop shifts to the frequency where β's phase cancels it: $\Delta\omega/\omega_0 = -\Delta\phi/(\omega_0\, d\phi/d\omega)$. For a resonator of quality factor Q, $\omega_0\, d\phi/d\omega = 2Q$ (Book 3 §13.5), so

$$\frac{\Delta f}{f_0} \approx -\frac{\Delta \phi}{2Q}$$

A 1° (17.5 mrad) phase error shifts an RC oscillator with an effective Q of 1/3 by 2.6 %, an LC oscillator with $Q = 100$ by 87 ppm, and a crystal with $Q = 50\,000$ by 0.17 ppm. The same Q appears in the oscillator's **phase noise**: in Leeson's model, the noise sidebands close to the carrier fall as $1/(2Q)^2$ times the amplifier's noise, so a high-Q resonator is the first requirement of a clean reference (Book 3 Ch. 32 treats phase noise at the system level).

> **Status of the principle: Frequency stability $\Delta f/f_0 \approx -\Delta\phi/(2Q)$**
> *Approximate.* Linearises the phase of a second-order resonator near its centre frequency; it ranks oscillator families correctly and is quantitative for small phase errors, but ignores amplitude-to-frequency conversion through nonlinear capacitances, which dominates in some VCOs.

9.6 RC oscillators

The Wien-bridge oscillator

The Wien network (series R–C feeding parallel R–C, Chapter 8) has transmission

$$\beta(j\omega) = \frac{1}{3 + j\left(\omega RC - \dfrac{1}{\omega RC}\right)}$$

which is real and equal to 1/3 at $\omega_0 = 1/RC$. A non-inverting amplifier of gain 3 satisfies the Barkhausen criterion there. Because the network's phase slope at ω_0 is $-2/3$ rad per unit of normalised frequency, its effective Q is 1/3; a Wien oscillator's frequency is therefore only as stable as its resistors and capacitors (CoG capacitors and 0.1 % metal-film resistors give about 50–100 ppm/°C) and the amplifier's phase shift, which at f_0 adds an error $\approx 3f_0/\text{GBW}$ radians: an argument for $\text{GBW} \geq 100 f_0$.

The gain of exactly 3 must be held by an amplitude-control element (Figure 9.3). The simplest is a pair of antiparallel diodes across part of the feedback resistance: at small amplitude the gain is $1 + (R_a + R_b)/R_1 > 3$ and the oscillation grows; as the peak voltage across R_b approaches a diode drop, the diodes shunt R_b and the gain falls.

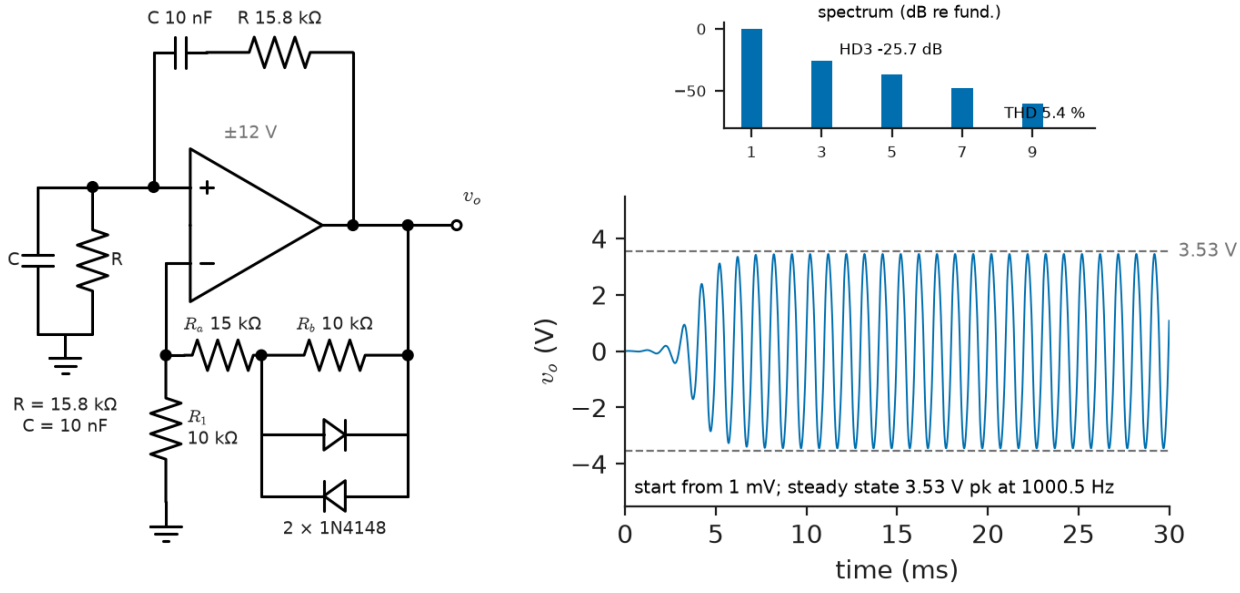


Figure 9.3 — Wien-bridge oscillator with diode amplitude stabilisation, and its simulated output.

Worked example 9.4 — Amplitude and distortion of a diode-stabilised Wien oscillator. With $R = 15.8\ \text{k}\Omega$ and $C = 10$ nF ($f_0 = 1007.3$ Hz), $R_1 = 10\ \text{k}\Omega$, $R_a = 15\ \text{k}\Omega$ and $R_b = 10\ \text{k}\Omega$ (small-signal gain 3.5), the amplitude is where the diodes' average conduction brings the gain to 3, a nonlinear problem. Listing 9.2 integrates the circuit with an ideal op-amp and Shockley-law 1N4148 diodes, then measures frequency and THD in the steady state.

```
 import numpy as np
from scipy.integrate import solve_ivp
from scipy.optimize import brentq

R, C = 15.8e3, 10e-9                          # Wien network: f0 = 1/(2 pi R C)
R1, Ra, Rb = 10e3, 15e3, 10e3                 # gain network; Rb shunted by 1N4148 pair
Is, nVT = 2.7e-9, 1.9 * 0.02585               # diode saturation current, n*V_T

def v_out(vp):
    """Ideal op-amp: v- = vp, current vp/R1 flows through Ra and (Rb || diodes)."""
    i = vp / R1
    vb = brentq(lambda v: v / Rb + 2 * Is * np.sinh(v / nVT) - i, -2, 2)
    return vp + i * Ra + vb

def rhs(t, x):
    vcs, vp = x                               # series-C voltage, parallel-C voltage (= v+)
    i = (v_out(vp) - vcs - vp) / R            # current in the series R-C arm
    return [i / C, (i - vp / R) / C]

f0 = 1 / (2 * np.pi * R * C)
T = 1 / f0
sol = solve_ivp(rhs, [0, 150 * T], [0.0, 1e-3], max_step=T / 400, rtol=1e-8, atol=1e-10)
t = np.linspace(100 * T, 150 * T, 40000)                 # steady state
vo = np.array([v_out(v) for v in np.interp(t, sol.t, sol.y[1])])
up = np.flatnonzero((vo[:-1] < 0) & (vo[1:] >= 0))       # rising zero crossings
tz = t[up] - vo[up] * (t[up + 1] - t[up]) / (vo[up + 1] - vo[up])
f_osc = (len(tz) - 1) / (tz[-1] - tz[0])
n = 4096                                                 # resample exactly 40 cycles
ts = tz[0] + np.arange(n) * 40 / f_osc / n
spec = np.abs(np.fft.rfft(np.interp(ts, t, vo))) * 2 / n
thd = np.sqrt(np.sum(spec[80:440:40] ** 2)) / spec[40]
print(f"f0 = {f0:.1f} Hz, oscillates at {f_osc:.1f} Hz, "
      f"{spec[40]:.2f} V peak, THD = {100 * thd:.2f} %")
```

It prints a frequency of 1000.5 Hz (0.7 % below $1/(2\pi RC)$, the harmonics' contribution to the loop phase), an amplitude of 3.53 V peak and a THD of 5.4 %, almost all third harmonic. Changing the gain network to $R_a = 16\ \text{k}\Omega$, $R_b = 5.6\ \text{k}\Omega$ (small-signal gain 3.16, less excess) gives 1005.5 Hz, 4.12 V and 2.8 %: less excess gain means the diodes need to act less, with less distortion, at the price of a slower and less certain start. The amplitude depends on the diode drop and therefore on temperature, at about −0.3 %/°C. For a test oscillator needing 0.01 % THD, the diodes are replaced by a JFET in the gain network, controlled by a rectifier–filter loop with a time constant of tens of cycles, or by a lamp or thermistor whose thermal time constant averages over many cycles.

Phase-shift and quadrature oscillators

The **RC phase-shift oscillator** uses three RC sections to produce 180° at $f_0 = 1/(2\pi\sqrt{6}RC)$, where the ladder attenuates by 1/29, and an inverting amplifier of gain at least 29 (Book 3 §13.9). A single inverting stage suffices, but its frequency is hard to tune and its output is distorted.

The **quadrature oscillator**, two integrators and an inverter in a loop (or an integrator, an inverting integrator and a small positive-feedback adjustment), oscillates at $1/(2\pi RC)$ with two outputs 90° apart. Its output after the second

integrator is filtered twice and is cleaner than the Wien's; it excites resolvers and LVDTs in aerospace position sensing, where a sine and a cosine reference are needed for demodulation.

RC oscillators cover roughly 1 Hz to 1 MHz. Above that, the amplifier's phase shift and the small resistors and capacitors needed make LC resonators the better choice; below about 1 Hz, digitally synthesised sine waves (Section 9.9) are easier.

9.7 LC oscillators

The negative-resistance view

An LC tank with inductor loss resistance R_s oscillates freely at $1/(2\pi\sqrt{LC})$ while its stored energy decays; to sustain the oscillation, an active circuit must supply exactly the energy lost each cycle. Viewed from the tank, the active circuit is a **negative resistance**, and the oscillation condition is that its magnitude equal (and at start-up exceed) the tank's loss resistance. This is the Barkhausen criterion written at one port, and it is how RF designers think about oscillators: the gain element is a one-port with $\mathrm{Re}\,Z < 0$ at the frequency of interest.

Colpitts, Hartley and Clapp

The common LC topologies differ in how the tank is tapped to provide the 180° phase reversal needed around an inverting transistor (Figure 9.4):

- **Colpitts:** a capacitive divider (C_1, C_2) with the inductor across both. For a transistor of transconductance g_m connected across the divider (ignoring its own capacitances and r_π), the impedance it presents in series with the inductor has a real part

  $$R_{neg} = -\frac{g_m}{\omega^2 C_1 C_2}$$

  so oscillation starts when $g_m/(\omega^2 C_1 C_2) > R_s$. The frequency is set by L and the series combination of C_1 and C_2.
- **Hartley:** an inductive divider (a tapped coil) with a capacitor across it; easy to tune with one variable capacitor, and common in inductive-proximity sensors.
- **Clapp:** a Colpitts with a small capacitor C_3 in series with the inductor. When $C_3 \ll C_1, C_2$ it alone sets the frequency, and the transistor's voltage-dependent capacitances, which sit across C_1 and C_2, barely affect it: better frequency stability.

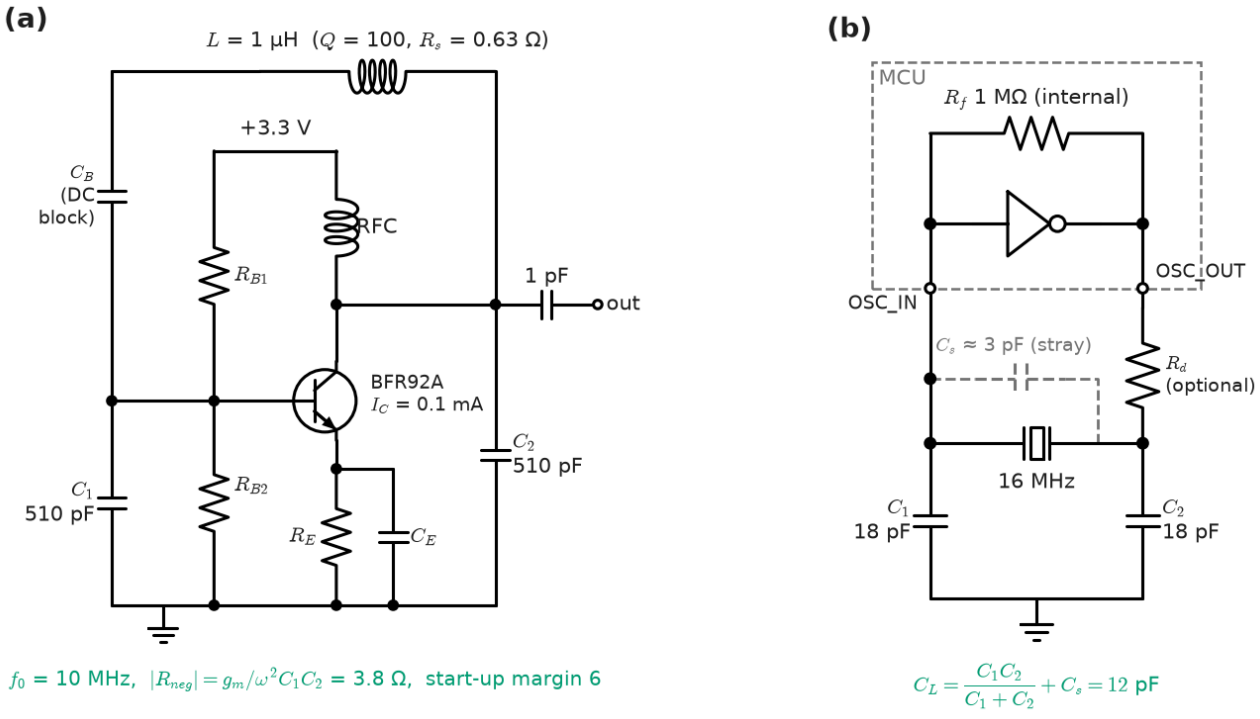


Figure 9.4 — (a) Common-emitter Colpitts oscillator of Worked example 9.5 at 10 MHz; (b) Pierce crystal oscillator as used in microcontrollers.

Worked example 9.5 — Start-up margin of a 10 MHz Colpitts. A 10 MHz oscillator uses $L = 1\ \mu\text{H}$ with $Q = 100$, so $R_s = \omega L/Q = 62.8\ \Omega/100 = 0.63\ \Omega$. The capacitance for resonance is $1/(\omega^2 L) = 253$ pF, obtained as $C_1 = C_2 = 507$ pF in series (510 pF parts). Start-up needs

$$g_m > R_s\,\omega^2 C_1 C_2 = 0.63 \times (6.28 \times 10^7)^2 \times (507\ \text{pF})^2 = 0.64\ \text{mS}$$

which a BJT provides at only 16.5 µA. At $I_C = 0.1$ mA, $g_m = 3.87$ mS and $|R_{neg}| = 3.8\ \Omega$, a start-up margin of 6. At 1 mA the margin would be 61, and the oscillation would grow until the transistor cut off for most of each cycle, with harmonic-rich output and a frequency pulled by the transistor's changing capacitances. A margin of 3 to 10, with amplitude limited gently (by emitter degeneration, or an AGC loop in a precision source), is the usual design. The transistor's own capacitances (a few picofarads) add to C_1 and C_2 and lower the frequency by a few tenths of a per cent, which is trimmed or, in a Clapp design, swamped.

> **Status of the principle: Colpitts negative resistance** $-g_m/(\omega^2 C_1 C_2)$
> *Approximate.* Derived for an ideal transconductor across the capacitive divider, neglecting r_π, r_o and the device capacitances; it gives the start-up condition to within a factor that a 3–10× margin absorbs, and it is the same expression used for the Pierce crystal oscillator.

VCOs and integrated LC oscillators

Replacing part of the tank capacitance by a varactor diode (Chapter 2) makes a **voltage-controlled oscillator**, whose gain K_{VCO} (MHz per volt) and tuning range are set by the varactor's capacitance ratio and the fixed capacitance around it. In RF integrated circuits the gain element is usually a cross-coupled transistor pair, whose negative resistance is $-2/g_m$ across the tank, with on-chip inductors of Q 10–20. A VCO alone drifts by hundreds of ppm per degree; it becomes a precise frequency source only inside a phase-locked loop that locks it to a crystal reference, a type-2 feedback loop whose phase margin decides spur suppression and settling (Book 3 §34.9). Every phone, Wi-Fi radio and GNSS receiver generates its local oscillators this way.

9.8 Crystal oscillators

The quartz resonator

A quartz crystal is a piezoelectric mechanical resonator: an applied voltage strains it, and its mechanical resonance appears electrically as a very high-Q series RLC branch (motional inductance L_m, capacitance C_m and resistance R_m, often called ESR) in parallel with the electrode capacitance C_0. This is the Butterworth–Van Dyke model. The motional elements have values no lumped component could reach: henries to kilohenries of inductance and femtofarads of capacitance.

Table 9.2 — Representative crystal parameters (typical).

Crystal	f	L_m	C_m	R_m (ESR)	C_0	Q
Tuning fork (watch, RTC)	32.768 kHz	≈ 7.9 kH	≈ 3 fF	30–90 kΩ	1–1.5 pF	30 000–100 000
AT-cut fundamental (MCU clock)	16 MHz	≈ 10 mH	≈ 10 fF	20–60 Ω	3–5 pF	25 000–80 000
AT-cut third overtone	100 MHz	≈ 5–10 mH	≈ 0.3 fF	40–100 Ω	3–5 pF	50 000–100 000

With $L_m = 9.9$ mH, $C_m = 10$ fF and $R_m = 40\ \Omega$ near 16 MHz ($f_s = 15.996$ MHz), $Q = \omega L_m / R_m \approx 25\,000$. The branch is series-resonant at $f_s = 1/(2\pi\sqrt{L_m C_m})$; above f_s it is inductive and resonates with C_0 at the antiresonance $f_a \approx f_s(1 + C_m/2C_0)$, here 26.7 kHz (1670 ppm) above f_s. Between f_s and f_a the crystal looks like an inductor whose value changes enormously over a tiny range (Figure 9.5); an oscillator that uses it as the inductor of a Colpitts circuit is held to that range.

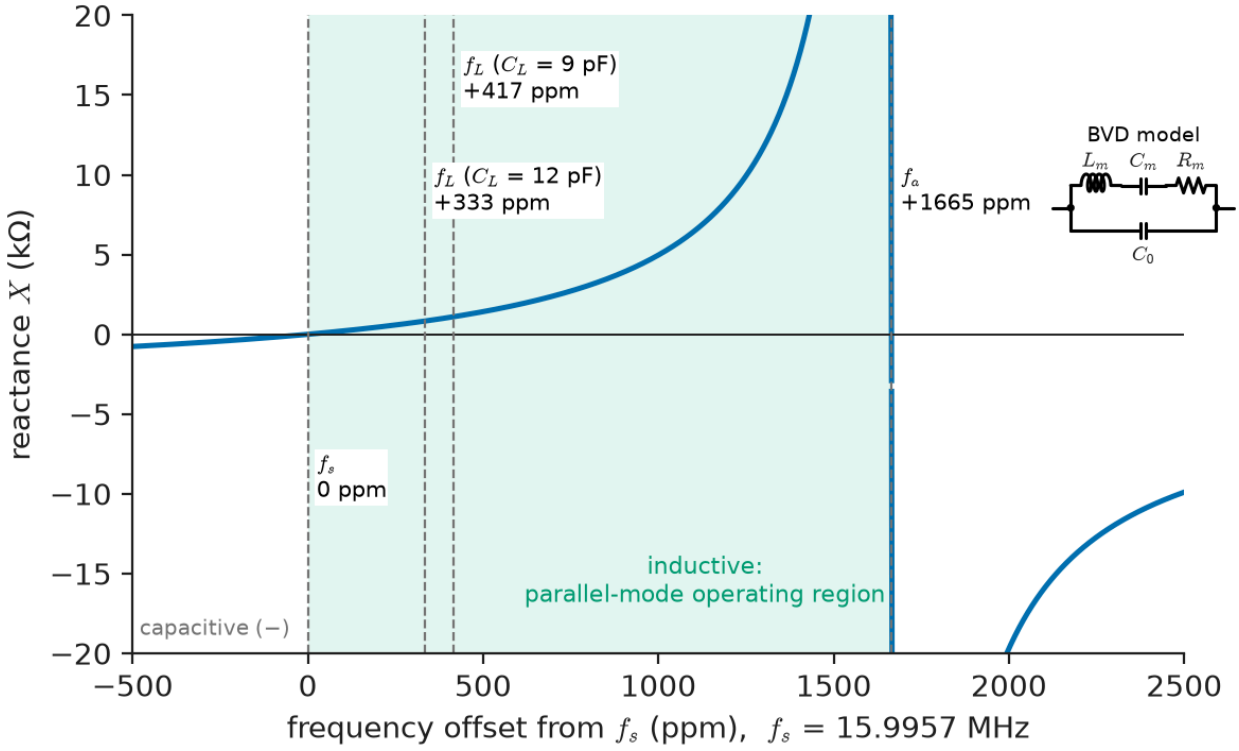


Figure 9.5 — Reactance of a 16 MHz AT-cut crystal (Lm = 9.9 mH, Cm = 10 fF, Rm = 40 ohm, Co = 3 pF) near resonance, with the series, load and antiresonant frequencies.

Load capacitance and pulling

A crystal for a Pierce oscillator is specified at a **load capacitance** C_L, the capacitance the oscillator circuit presents across it, and it oscillates at

$$f_L \approx f_s \left(1 + \frac{C_m}{2(C_0 + C_L)}\right)$$

The load seen by the crystal in a Pierce circuit (Figure 9.4b) is the series combination of the two load capacitors plus the stray capacitance of pins and traces: $C_L = C_1 C_2/(C_1 + C_2) + C_s$. Differentiating gives the **pulling sensitivity** $-f_s C_m / [2(C_0 + C_L)^2]$ per farad.

> **Status of the principle: Crystal load-capacitance pulling**
> *Approximate.* From the Butterworth–Van Dyke model, linearised for $C_m \ll C_0 + C_L$; accurate to a few per cent for fundamental-mode crystals within their pullable range, and the basis of trimming and of voltage-controlled crystal oscillators (VCXOs).

Worked example 9.6 — Load capacitors for an Ethernet crystal. A 16 MHz crystal ($C_m = 10$ fF, $C_0 = 3$ pF, $R_m = 40\ \Omega$ maximum) is specified at $C_L = 12$ pF and ±20 ppm tolerance; the board adds $C_s \approx 3$ pF.

Correct capacitors. $C_1 = C_2 = 2(C_L - C_s) = 18$ pF gives $C_L = 9 + 3 = 12$ pF.

A common mistake. Fitting 12 pF capacitors "because the crystal says 12 pF" gives $C_L = 6 + 3 = 9$ pF. The frequency rises by $\frac{C_m}{2}\left(\frac{1}{C_0 + 9\ \text{pF}} - \frac{1}{C_0 + 12\ \text{pF}}\right) = 83$ ppm. The sensitivity at 12 pF is 22 ppm/pF. Added to ±20 ppm initial tolerance, ±20 ppm over temperature and a few ppm of ageing, the clock is out of the ±100 ppm that IEEE 802.3 allows an Ethernet reference; links work on the bench with a good partner and fail with a partner at the other end of its tolerance.

Start-up margin. The oscillator's critical transconductance for a Pierce circuit is approximately $g_{m,crit} = 4 R_m \omega^2 (C_0 + C_L)^2 = 4 \times 40 \times (1.005 \times 10^8)^2 \times (15\ \text{pF})^2 = 0.36$ mA/V. A microcontroller oscillator with $g_m = 5$ mA/V has a margin of 13.7, above the factor of 5 that vendors recommend. Every term of $g_{m,crit}$ explains a field failure: a cheaper crystal with 80 Ω ESR halves the margin, larger load capacitors cut it as $(C_0 + C_L)^2$, and at −40 °C some crystals show an ESR increase ("drive-level dependence" or a spurious mode) that, in the opening paragraph's meter, pushed one unit in five hundred below a margin of one. The fix was a crystal with a lower maximum ESR and a firmware check of the oscillator-ready flag with a fallback to the internal RC oscillator.

Pierce oscillators in microcontrollers

Almost every microcontroller, processor and radio chip contains a Pierce oscillator: a CMOS inverter biased into its linear region by a large internal feedback resistor, the crystal from its output to its input, and the two load capacitors to ground. It is a Colpitts circuit with the crystal as the inductor, the inverter as the transconductor and C_1, C_2 as the capacitive divider. The inverter's gain is often programmable (drive strength), to be matched to the crystal's ESR and maximum **drive level** (the power dissipated in R_m, typically limited to 10–300 µW for MHz crystals and below 1 µW for tuning forks, beyond which the crystal ages faster or fractures). Start-up takes thousands of cycles because the high Q makes the amplitude grow slowly: 1–5 ms for a 16 MHz crystal and 0.3–2 s for a 32.768 kHz watch crystal, which is why MCUs boot on an internal RC oscillator and switch to the crystal later (Chapter 17, Chapter 25).

Temperature, compensation and the oscillator family

The quartz cut sets the temperature behaviour. The AT cut has a cubic frequency–temperature curve with a turning point near 25 °C, holding ±10–20 ppm from −40 °C to +85 °C. The 32.768 kHz tuning fork has a parabolic curve, $\Delta f/f \approx -0.034\ \text{ppm}/^\circ\text{C}^2 \times (T - 25\ ^\circ\text{C})^2$: at −20 °C it is 68.9 ppm slow, and a real-time clock left outdoors loses 6 s per day. Smart meters, which must keep time to within seconds per month, measure temperature and correct the RTC digitally.

Table 9.3 — Frequency sources and their stability (typical, over industrial temperature range).

Source	Stability	Typical use
On-chip RC oscillator	±1–2 % (trimmed), ±0.1 % with calibration	MCU boot clock, UART at low rates
Ceramic resonator	±0.3–0.5 %	Low-cost MCU clocks, appliances
MEMS oscillator	±10–50 ppm	Shock-resistant clocks, consumer devices
Crystal oscillator (XO)	±20–100 ppm total	Processors, Ethernet, USB, CAN
TCXO (temperature-compensated)	±0.5–2.5 ppm	GNSS receivers, cellular radios, smart meters
OCXO (oven-controlled)	±1–50 ppb	Base stations, test equipment, network timing
Rubidium atomic standard	$\approx 10^{-11}$, low drift	Telecom timing, GNSS-disciplined references

Interface standards set the floor: USB full speed tolerates ±2500 ppm (crystal-less designs recover the clock from bus traffic), Ethernet ±100 ppm, and cellular and GNSS receivers need TCXO-class references because their search windows scale with frequency error.

9.9 Waveform generators

The triangle–square generator

The relaxation oscillator of Chapter 8 produces exponential segments. Replacing its RC by an integrator gives linear ramps: an integrator drives a non-inverting Schmitt trigger, whose output feeds back to the integrator's input (Figure 9.6). The Schmitt output is a square wave of $\pm V_{sat}$; the integrator ramps at $V_{sat}/(RC)$ between the Schmitt thresholds $\pm V_{sat} R_1/R_2$, so the triangle's amplitude is set by a resistor ratio and

$$f = \frac{R_2}{4 R_1 R C}$$

independent of V_{sat}, because the ramp rate and the thresholds scale together.

> **Status of the principle: Triangle–square generator frequency $f = R_2/(4R_1RC)$**
> *Derived.* Time for a linear ramp of slope V_{sat}/RC to traverse the hysteresis band $2V_{sat}R_1/R_2$, twice per period; assumes symmetric saturation and an ideal integrator, and fails at high frequency where comparator delay and slew rate add to each half-period.

Worked example 9.7 — A 1 kHz function generator. A square wave of ±10 V and a triangle of ±5 V at 1 kHz are required. The triangle amplitude fixes $R_1/R_2 = 0.5$ (say $R_1 = 10$ kΩ, $R_2 = 20$ kΩ); then $f = R_2/(4R_1RC) = 1/(2RC)$ gives $RC = 0.5$ ms, for example $R = 50$ kΩ (49.9 kΩ E96) and $C = 10$ nF. The frequency is tuned over a decade by a potentiometer that scales the voltage applied to the integrator, or over three decades by switching C. A sine output follows from a **sine shaper**: a diode-resistor network that progressively reduces the slope of the triangle near its peaks, or a differential pair driven into its tanh region, gives 0.5–2 % THD from a well-adjusted triangle, which is how the classic function-generator ICs (ICL8038, XR-2206) made their sine outputs. Making the integrator's input current proportional to a control voltage gives a VCO with a linear frequency-voltage law, used for frequency modulation and swept measurements.

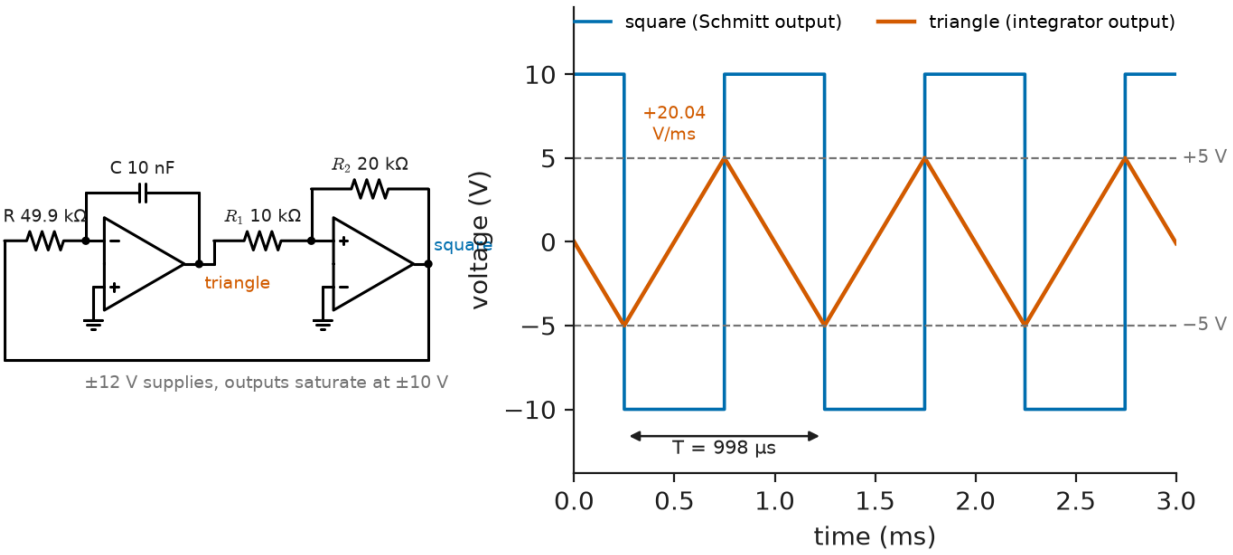


Figure 9.6 — Triangle-square function generator of Worked example 9.7 and its waveforms.

The 555 timer

The 555, introduced in 1972 and still made in bipolar and CMOS versions (NE555, TLC555, LMC555), packages two comparators with thresholds at $\tfrac{1}{3}V_{CC}$ and $\tfrac{2}{3}V_{CC}$, a flip-flop and a discharge transistor. In the astable connection a capacitor charges through $R_A + R_B$ and discharges through R_B between the two thresholds, so

$$f = \frac{1}{\ln 2\,(R_A + 2R_B)C} \approx \frac{1.44}{(R_A + 2R_B)C}, \qquad D = \frac{R_A + R_B}{R_A + 2R_B}$$

With $R_A = 1\ \text{k}\Omega$, $R_B = 6.8\ \text{k}\Omega$ and $C = 10$ nF, $f = 9.88$ kHz with a 53 % duty cycle. Because both thresholds are fractions of V_{CC}, the frequency is nearly independent of supply voltage, typically within 0.1 %/V. The 555 drives buzzers, LED flashers and charge pumps; where accuracy matters it has given way to microcontroller timers (Chapter 17).

Sawtooth, PWM and digital synthesis

A **sawtooth** generator (a capacitor charged by a constant current and reset by a switch at each clock pulse) provides the ramp against which an analog PWM controller compares its error voltage; the comparator output's duty cycle is proportional to the error (Chapter 30), and a triangular carrier does the same in inverter modulators (Chapter 31).

Direct digital synthesis (DDS) has replaced analog function generators in most instruments. An N-bit phase accumulator adds a tuning word M every clock cycle; its top bits address a sine table whose output drives a DAC and a reconstruction filter. The output frequency is $f_{out} = M f_{clk}/2^N$: a 28-bit accumulator on a 25 MHz clock, as in the AD9833, tunes in 0.093 Hz steps, with phase-continuous frequency changes and exact sine, triangle and square shapes. The analog design moves to the DAC and the reconstruction filter, whose requirements follow from sampling theory (Book 3 Ch. 19), and the DDS clock must itself come from a crystal (Section 9.8): its stability is the output's stability.

What remains open

Open question	Where it is answered
How is an op-amp compensated internally, with Miller capacitance and pole splitting, so that it is stable as a follower?	Chapter 10 (Analog Integrated Circuits)
How are compensators designed systematically to meet margin, bandwidth and overshoot targets?	*Circuits, Signals and Communications*, Ch. 35
How is the feedback loop of a switching converter closed and compensated around its LC filter?	Chapter 30 (DC–DC Converters)
How does a microcontroller select, start and monitor its clock sources, and how are timers used to generate waveforms?	Chapter 17 and Chapter 25
How do clock jitter and phase noise limit ADC resolution and communication links?	Chapter 26; *Circuits, Signals and Communications*, Ch. 32
How are clocks distributed and crossed between domains in digital systems?	Chapter 14 (Sequential Digital Systems)
Why can the Nyquist plot, and not only the Bode margins, be needed for loops with several crossovers?	*Circuits, Signals and Communications*, Ch. 34 §34.8–34.9

Next: Chapter 10 opens the op-amp and the comparator to show how differential pairs, current mirrors, active loads, compensation capacitors, references and regulators are combined on one chip.

Chapter 10 — Analog Integrated Circuits

Part II · Analog Electronics

An industrial pressure transmitter has passed every bench test for two years when the purchasing department replaces the 4.7 µF tantalum capacitor on the output of its 3.3 V low-dropout regulator with a cheaper and smaller 10 µF ceramic. The next production batch fails final test: the regulator output carries a 30 mV oscillation at a few hundred kilohertz, and the 12-bit ADC that digitises the bridge sensor shows a noise floor ten times worse than before. While the engineers chase that, a field report arrives from a desert installation: the same transmitter reads 0.35 % off every afternoon and correct every morning. Nothing in the sensor is wrong. The 2.5 V shunt reference on the ADC drifts by 50 ppm/°C, and the 70 °C swing between a cold night and a sun-heated enclosure moves every reading by fourteen codes. Neither failure can be understood from the ideal models of the previous chapters; both need the inside of the chip: the pairs, mirrors and compensation capacitors of an op-amp, the bandgap of a reference, the loop of a regulator, the comparators and capacitor arrays of a converter.

> **How are the amplifiers, references and regulators that analog design takes for granted built from matched transistors on one die, and what in that construction sets their offset, gain, bandwidth, drift and stability?**
> **How do comparators, DACs and ADCs turn those same building blocks into the interface between the analog world and the digital logic of Part III?**

Concept	Engineering destination
Differential amplifiers	Input stage of every op-amp, comparator and instrumentation amplifier; offset and CMRR budgets
Current mirrors	Biasing whole chips from one reference current; current DACs; LED and laser drivers
Active loads	High gain from a single stage at low supply voltage
Multistage integrated amplifiers	Reading an op-amp datasheet: gain, swing, output drive and quiescent current
Frequency compensation	Why an op-amp is stable at unity gain, why it oscillates with a capacitive load, slew rate
Analog IC architecture	Choosing among bipolar, JFET, CMOS and chopper op-amps; rail-to-rail behaviour; trimming
Voltage references	ADC and DAC accuracy over temperature; sensor excitation; calibration standards
Regulators	Clean local supplies for ADCs, oscillators and sensors; LDO stability and dissipation
Comparators	Zero-crossing and over-current detection, ADC decision elements, metastability
Mixed-signal interfaces	DAC and ADC architectures inside microcontrollers, data-acquisition and audio chips

Roadmap. The differential pair (10.1) → current mirrors that bias it (10.2) and load it (10.3) → the two-stage op-amp built from them (10.4) → making it stable (10.5) → the architectural choices of real parts (10.6) → bandgap and buried-Zener references (10.7) → linear regulators built around a reference and an amplifier (10.8) → comparators and latches (10.9) → DAC and ADC architectures (10.10).

10.1 Differential amplifiers

Boundary.
The large-signal BJT and MOSFET laws, the hybrid-π model, $g_m = I_C/V_T$, $r_o = V_A/I_C$ and the square-law model with channel-length modulation are developed in Chapter 3 and Chapter 4; single-stage gain calculations in Chapter 7; the op-amp as a circuit element in Chapter 8. *Engineering Physics*, Ch. 18 §18.4, sketches the ideal op-amp and names its limitations (GBW, slew rate, offset); this chapter explains where each of them comes from inside the chip.

Why integrated circuits are differential

An integrated circuit cannot make accurate resistors (±20 % absolute is normal), large capacitors or coupling capacitors. What it makes extremely well is **matched** devices: two transistors drawn side by side see the same doping, oxide and temperature, so their parameters track to 0.1 % even when each is uncertain by 20 %. Analog IC design is therefore built on ratios and differences, and the differential pair, which responds to the *difference* of two inputs and rejects what is common to both, is its first building block.

The long-tailed pair

Two matched transistors Q_1 and Q_2 with joined emitters (or sources) are fed from a tail current I_{EE} (Figure 10.1). With a differential input $v_{id} = v_{B1} - v_{B2}$, the exponential BJT law divides the tail current as

$$i_{C1} - i_{C2} = \alpha I_{EE} \tanh\!\left(\frac{v_{id}}{2V_T}\right)$$

The pair is linear only for small inputs: the tanh deviates by 1 % from a straight line at $v_{id} = 9.0$ mV, and 90 % of the current has switched at $v_{id} = 76$ mV (at 300 K, $V_T = 25.85$ mV). That is why an op-amp needs negative feedback, which keeps v_{id} in the microvolt range, and why the same circuit makes a fast comparator or current switch (Section 10.9). A MOS pair is linear over roughly $\pm\sqrt{2}\,V_{ov}$, typically ±0.1 to ±0.3 V, with less transconductance per ampere.

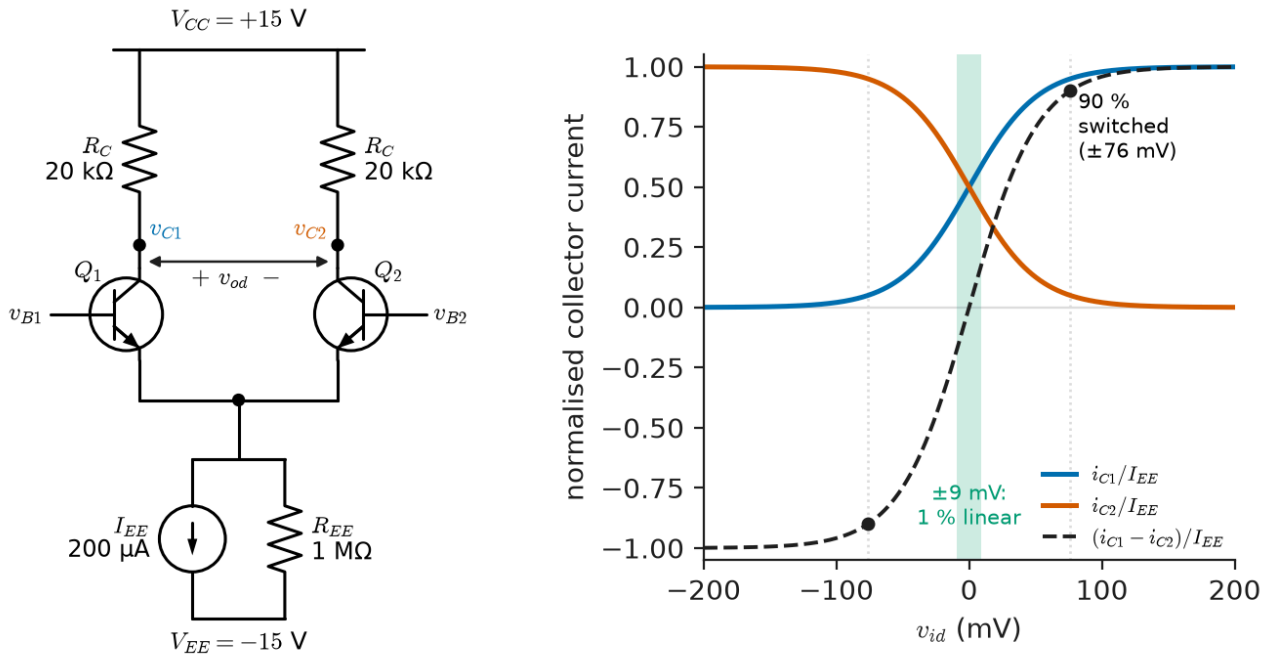


Figure 10.1 — BJT long-tailed pair with resistive loads and its large-signal transfer characteristic.

Differential and common-mode gain

For small signals each transistor has $g_m = I_C/V_T$ with $I_C = I_{EE}/2$. A differential input splits equally, $\pm v_{id}/2$, across the two base-emitter junctions, and the emitter node does not move: it is a **virtual ground** for differential signals. The differential gain to the differential output is

$$A_d = \frac{v_{od}}{v_{id}} = g_m R_C$$

and half of that to either single-ended output. A common-mode input v_{ic} moves both bases together; the emitters follow, and the only thing that changes the collector currents is the current that flows into the finite output resistance R_{EE} of the tail source. Each half then sees an emitter resistance of $2R_{EE}$, and the single-ended common-mode gain is approximately

$$A_{cm} \approx -\frac{R_C}{2R_{EE}}$$

The **common-mode rejection ratio** compares the single-ended differential gain with this:

$$\mathrm{CMRR} = \frac{A_d/2}{|A_{cm}|} \approx g_m R_{EE}$$

> **Status of the principle: Differential-pair small-signal gains**
> *Approximate.* Linearised hybrid-π analysis of perfectly matched devices for $|v_{id}| \ll 2V_T$; real CMRR is lower, because mismatch in R_C or g_m converts common-mode input directly into differential output, and it falls with frequency as the capacitance across the tail source shunts R_{EE}.

Worked example — Gains and CMRR of a 200 µA pair. With $I_{EE} = 200$ µA, $R_C = 20$ kΩ and a tail source of $R_{EE} = 1$ MΩ (Figure 10.1): $I_C = 100$ µA and $g_m = 100\,\mu\text{A}/25.85\,\text{mV} = 3.87$ mS. The differential gain is $A_d = 3.87\,\text{mS} \times 20\,\text{k}\Omega = 77.4$ (37.8 dB), and 38.7 to one collector. The common-mode gain is $20\,\text{k}\Omega/2\,\text{M}\Omega = 0.010$, so the ideal CMRR is $38.7/0.010 = 3870$, or 71.8 dB, and $g_m R_{EE} = 3870$ agrees. If the collector resistors mismatch by 1 %, a 1 V common-mode step gives 10 mV at each collector differing by 0.1 mV, equivalent through $A_d = 77.4$ to 1.3 µV of differential input (118 dB). Here the tail impedance limits CMRR; in a real op-amp with a cascoded tail (Section 10.2) and a mirror load (Section 10.3) matching does, and sets datasheet CMRR of 80 to 120 dB.

Offset voltage

If the two transistors are not identical, a non-zero input is needed to balance the output. For BJTs the main cause is a mismatch $\Delta I_S/I_S$ in saturation current (emitter area and base doping), which gives

$$V_{OS} \approx V_T \ln\!\left(1 + \frac{\Delta I_S}{I_S}\right)$$

a 1 % mismatch causes 0.26 mV. Load-resistor mismatch adds $V_T\,\Delta R_C/R_C$. For MOS pairs, threshold mismatch dominates, and it scales with device area according to **Pelgrom's rule**:

$$\sigma(\Delta V_{TH}) = \frac{A_{VT}}{\sqrt{WL}}$$

with A_{VT} of roughly 3 to 5 mV·µm for 0.18 to 0.35 µm processes and about 1 to 2 mV·µm for the thinner oxides of advanced nodes. A pair with $WL = 1\ \mu\text{m}^2$ and $A_{VT} = 3.5$ mV·µm has $\sigma = 3.5$ mV; a hundred times the area brings it down to 0.35 mV. Untrimmed CMOS op-amps therefore have offsets of 1 to 10 mV and bipolar ones 0.5 to 3 mV. A BJT pair's offset drifts as V_{OS}/T, about 3.3 µV/°C per millivolt, so trimming the offset also nulls most of the drift.

> **Status of the principle: Pelgrom mismatch scaling**
> *Empirical.* Fitted to measured mismatch across many processes: random local variations average over the gate area, so the standard deviation falls as $1/\sqrt{WL}$; the rule fails for very large devices separated by long distances, where wafer-scale gradients (handled by common-centroid layout) dominate.

10.2 Current mirrors

The basic mirror

A diode-connected transistor Q_1 carrying I_{REF} sets a V_{BE} (or V_{GS}); a matched Q_2 with the same V_{BE} carries the same current. This **current mirror** (Figure 10.2a) replaces the resistor in analog ICs: one reference current is mirrored to every stage, with ratios set by emitter area or W/L.

Two errors limit a bipolar mirror. The base currents of both transistors are taken from I_{REF}, so

$$\frac{I_{OUT}}{I_{REF}} = \frac{1}{1 + 2/\beta}$$

about 2 % low for $\beta = 100$. And Q_2's collector voltage differs from Q_1's, so the Early effect changes its current by $(V_{CE2} - V_{BE})/V_A$; the output resistance is just $r_o = V_A/I_{OUT}$. MOS mirrors have no gate current, but the same $r_o = 1/(\lambda I_D)$ limitation, and in addition a V_{TH} mismatch that produces a current error $2\,\Delta V_{TH}/V_{ov}$: a 2 mV threshold mismatch at $V_{ov} = 0.2$ V is a 2 % error. Precision current mirrors therefore run their transistors at large overdrive, which costs voltage headroom.

Better mirrors

Each improvement trades headroom or complexity for accuracy or output resistance (Figure 10.2, Table 10.1).

- **Emitter-follower-augmented (β-helper) mirror**: a third transistor supplies the base currents; the error falls to about $2/\beta^2$.
- **Widlar source**: an emitter resistor R_E in the output device produces small currents from a sensible reference without megohm resistors. The output current satisfies $I_{OUT} R_E = V_T \ln(I_{REF}/I_{OUT})$.
- **Wilson mirror**: feedback through a third transistor cancels the base-current error to about $2/\beta^2$ and raises the output resistance to about $\beta r_o/2$.
- **Cascode mirror**: a second, stacked mirror holds the output transistor's collector (or drain) voltage constant. Output resistance rises to about βr_o for BJTs and to $g_m r_o^2$ for MOSFETs, at the cost of one extra V_{BE} or V_{GS} of headroom; the **wide-swing cascode** recovers most of that headroom with an extra bias branch and is the default in low-voltage CMOS.

Table 10.1 — Current-mirror options (typical values, BJT $\beta = 100$, $V_A = 100$ V; MOS $g_m = 1$ mS, $r_o = 100$ kΩ at the stated current).

Mirror	Ratio error	Output resistance	Minimum output voltage
Basic BJT	$2/\beta \approx 2\%$	$r_o = 1$ MΩ at 100 µA	$V_{CE(sat)} \approx 0.2$ V
β-helper BJT	$2/\beta^2 \approx 0.02\%$	r_o	≈ 0.2 V
Wilson BJT	$2/\beta^2$	$\approx \beta r_o/2 = 50$ MΩ	$V_{BE} + V_{CE(sat)} \approx 0.9$ V
Basic MOS	$2\Delta V_{TH}/V_{ov}$, e.g. 2 %	$r_o = 100$ kΩ	$V_{ov} \approx 0.2$ V
Cascode MOS	as basic	$g_m r_o^2 = 10$ MΩ	$V_{TH} + 2V_{ov} \approx 0.9$ V
Wide-swing cascode MOS	as basic	$\approx g_m r_o^2$	$2V_{ov} \approx 0.4$ V

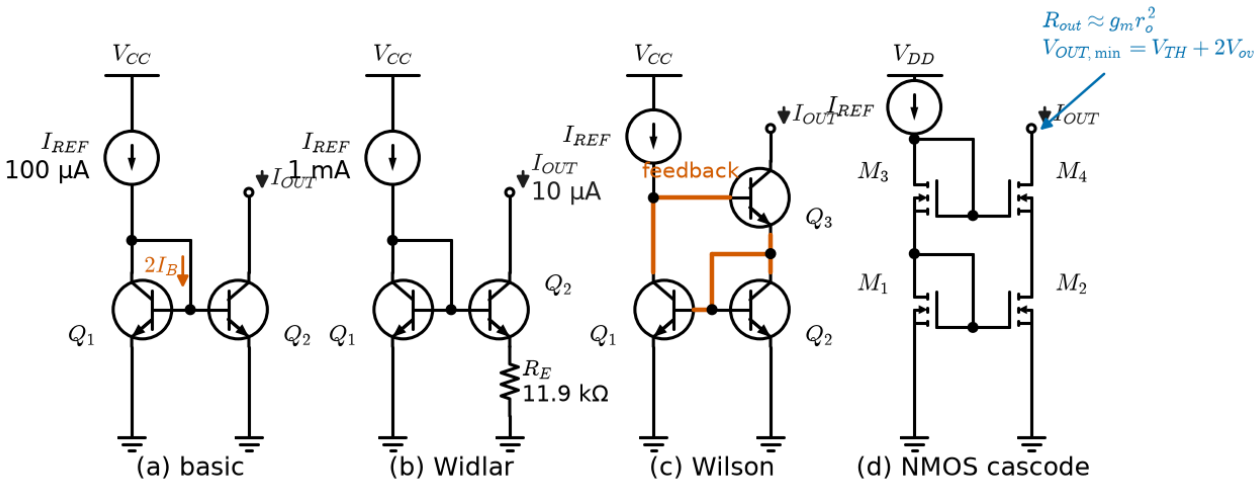


Figure 10.2 — Four integrated current mirrors: basic, Widlar, Wilson and MOS cascode.

Worked example — A 10 µA bias current from a 1 mA reference. A chip has a 1 mA reference and needs 10 µA for a low-power input stage. A basic mirror would need a 1:100 emitter-area ratio, or a 100 times larger reference resistor; a Widlar source does it with one resistor. Solving $I_{OUT}R_E = V_T \ln(I_{REF}/I_{OUT})$ for R_E gives $R_E = 25.85\ \text{mV} \times \ln(100)/10\ \mu\text{A} = 11.9\ \text{k}\Omega$, small enough to integrate. The emitter degeneration also raises the output resistance from $r_o = 100\ \text{V}/10\ \mu\text{A} = 10\ \text{M}\Omega$ to $r_o(1 + g_m R'_E)$, where $g_m = 0.387\ \text{mS}$ and $R'_E = R_E \parallel r_\pi = 11.9\ \text{k}\Omega \parallel 258\ \text{k}\Omega = 11.4\ \text{k}\Omega$: $R_{out} = 10\ \text{M}\Omega \times (1 + 4.40) = 54\ \text{M}\Omega$. The output current is nearly PTAT, which keeps the $g_m = I_C/V_T$ of the pair it biases constant with temperature.

> **Status of the principle: Current mirror**
> *Approximate.* Relies on matched devices at equal V_{BE} or V_{GS} and equal temperature; accuracy is limited by base current, Early effect, threshold mismatch and thermal gradients (a power output stage a millimetre away can shift a mirror by several percent), all of which the improved topologies reduce but none remove.

10.3 Active loads

Why resistive loads run out of gain

With a resistor load, $g_m R_L = I_C R_L / V_T$: the gain is the DC drop across the load divided by 25.85 mV, only 77 for a 2 V drop, which low-voltage ICs cannot afford. An **active load** (a current source) carries the bias current with little DC drop but presents its high r_o to the signal. The stage gain becomes the transistor's **intrinsic gain**:

$$A_v = -g_m \left(r_{o,n} \parallel r_{o,p}\right)$$

For a BJT, $g_m r_o = V_A/V_T$, about 3900 for $V_A = 100$ V and independent of current. For a MOSFET in saturation, $g_m r_o = 2/(\lambda V_{ov})$: about 100 for $\lambda = 0.1\ \text{V}^{-1}$ and $V_{ov} = 0.2$ V. Short-channel devices in advanced nodes have intrinsic gains of only 5 to 20, which is why those processes need cascodes or more stages to build an op-amp.

Table 10.2 — Intrinsic gain of integrated transistors (typical).

Device	Typical V_A or λ	Intrinsic gain $g_m r_o$
Vertical NPN, analog bipolar process	$V_A = 50$–150 V	2000–6000
Lateral PNP, analog bipolar process	$V_A = 30$–80 V	1000–3000
NMOS, $L = 0.18$ µm, 0.18 µm process	$\lambda \approx 0.2\ \text{V}^{-1}$	30–70
NMOS, minimum L, 28 nm process	—	5–15

The differential pair with a current-mirror load

The most widely used analog building block is the differential pair loaded by a current mirror (the input stage of Figure 10.3). The mirror copies Q_1's signal current to the output node, where it adds to Q_2's opposite signal current. Both halves of the differential signal reach the single-ended output, so the stage converts differential to single-ended without wasting half the gain, and its transconductance is the full g_m of one transistor:

$$A_d = g_m \left(r_{o2} \parallel r_{o4} \right)$$

With $I_C = 100$ µA, $V_{A,npn} = 100$ V and $V_{A,pnp} = 50$ V, $r_{o2} = 1$ MΩ, $r_{o4} = 500$ kΩ, and $A_d = 3.87\ \text{mS} \times 333\ \text{k}\Omega = 1290$ (62 dB) from a single stage, against 39 for the resistively loaded single-ended output of Section 10.1. The mirror also makes the common-mode response much smaller, because a common-mode input moves both collector currents equally and the mirror subtracts them.

The high-impedance output node is fragile: any DC imbalance pushes it towards a rail and any load below a megohm destroys the gain, so it is always followed by a high-impedance stage, and the whole amplifier needs global feedback to set its operating point.

10.4 Multistage integrated amplifiers

The two-stage architecture

Almost every general-purpose op-amp, bipolar or CMOS, has the structure of Figure 10.3:

1. **Input stage**: differential pair with active load; provides the transconductance G_{m1}, CMRR, low offset and low noise. Its output resistance R_1 is high.
2. **Gain stage**: a common-emitter (often a Darlington) or common-source transistor with an active load; gain $-G_{m2} R_2$, with R_2 also high. The compensation capacitor C_c is connected across it (Section 10.5).
3. **Output stage**: a class-AB complementary emitter follower (bipolar) or common-source pair (CMOS rail-to-rail), biased by a V_{BE} multiplier or translinear loop so that both devices conduct a small quiescent current and crossover distortion is suppressed; it gives low output resistance and the load current, with short-circuit current limiting.
4. **Bias network**: a reference current and mirrors that set every stage's current, so that g_m, slew rate and quiescent current all scale together.

The open-loop DC gain is the product of the first two stages, $A_0 = G_{m1} R_1 \, G_{m2} R_2$, typically 10^4 to 10^6 (80 to 120 dB); the output stage has a gain near one.

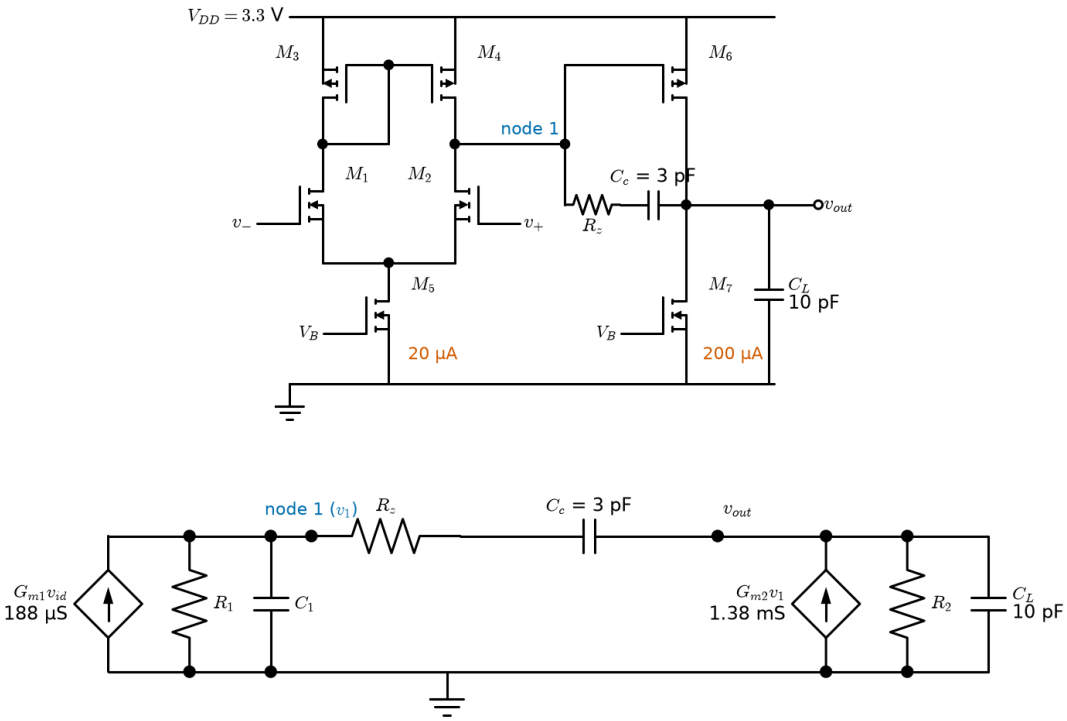


Figure 10.3 — Two-stage CMOS op-amp with Miller compensation and its small-signal model.

A classic bipolar example: the 741

The 741 (1968) follows this pattern with bipolar devices. NPN emitter followers driving lateral PNP common-base transistors form an input stage that tolerates ±30 V differential inputs and has $G_{m1} \approx 0.19$ mS at 19 µA; a Darlington second stage carries a 30 pF on-chip Miller capacitor; a protected class-AB follower drives the output. Its datasheet figures (gain 200 000, GBW 1 MHz, slew rate 0.5 V/µs) follow from those few numbers (Section 10.5). Modern parts keep the architecture and change the devices: JFET inputs for picoampere bias currents (TL072), CMOS for rail-to-rail operation at 1.8 to 5.5 V (AD8605), bias-cancelled bipolar inputs for low offset and noise (OP07, OPA1612). Every headline datasheet parameter maps to one block: offset and CMRR to input-pair matching, bias current to the input device, A_0 to the two stages' intrinsic gains, GBW and slew rate to C_c, noise to the input pair's g_m, swing to the output topology.

10.5 Frequency compensation

> **Boundary.**
> Loop gain, phase margin and the Nyquist and Bode stability criteria are the subject of *Circuits, Signals and Communications*, Ch. 34 §34.8–34.9, and of Chapter 9 of this book; Bode-plot construction is in Book 3 Ch. 13 §13.3. Here they are used to choose the compensation network inside the amplifier.

Why an op-amp needs compensation

Each high-impedance node of Figure 10.3 forms a pole with its capacitance: node 1 with R_1C_1, the output with R_2C_L. Uncompensated, the phase reaches $-180°$ while the gain is still far above one (dashed curve of Figure 10.4), and as a unity-gain follower, where the loop gain equals $A(j\omega)$, the amplifier oscillates. A general-purpose op-amp must be stable down to unity gain, so its response is shaped to fall below one before the second pole adds significant phase.

Miller (pole-splitting) compensation

A capacitor C_c across the second stage appears at node 1 multiplied by the stage gain (the Miller effect, Chapter 7), giving a very low-frequency dominant pole, and at the same time the feedback through C_c makes the second stage behave like a low output impedance at high frequency, pushing the output pole up. Analysis of the two-node model in Figure 10.3 gives, when the poles are widely separated,

$$p_1 \approx \frac{1}{G_{m2} R_1 R_2 C_c}, \qquad p_2 \approx \frac{G_{m2} C_c}{C_1 C_2 + C_c (C_1 + C_2)} \approx \frac{G_{m2}}{C_L}, \qquad z = +\frac{G_{m2}}{C_c}$$

(in rad/s; C_2 is the total output capacitance, dominated by C_L). The unity-gain frequency, which equals the gain-bandwidth product, is

$$\omega_u = A_0 p_1 = \frac{G_{m1}}{C_c}$$

independent of the high-impedance resistances and of G_{m2}. That is the reason GBW is one of the most reproducible op-amp parameters. The zero z lies in the right half-plane: at high frequency the signal fed forward through C_c opposes the inverted signal from the second stage, so the zero adds phase *lag*. It is removed by a series **nulling resistor** R_z: with $R_z = 1/G_{m2}$ the zero moves to infinity, and with larger R_z it moves into the left half-plane, where it can be placed on top of p_2.

> **Status of the principle: Two-pole Miller-compensated op-amp model**
> *Approximate.* Assumes two dominant nodes, $p_2 \gg p_1$ and $G_{m2} R_2 \gg 1$; it ignores the output stage and mirror poles (a third pole near the mirror's g_m/C), which in practice set the upper limit to the achievable GBW and phase margin.

Slew rate

A large step at the input switches the whole tail current I_{tail} into one side of the pair. The input stage then acts as a current source charging C_c, and the output, held by the second stage as an integrator, ramps at

$$SR = \frac{I_{tail}}{C_c}$$

Combining with $\omega_u = G_{m1}/C_c$ gives $SR = \omega_u \, I_{tail}/G_{m1}$. For a BJT pair $G_{m1} = I_{tail}/(2V_T)$, so $SR = 2V_T \omega_u$: a bipolar op-amp with 1 MHz GBW slews at only $2 \times 0.02585 \times 2\pi \times 10^6 = 0.32$ V/µs unless its input pair is degenerated with emitter resistors or replaced by JFET or MOS devices, whose G_{m1}/I_{tail} is three to ten times smaller. That is why the TL072 (JFET) slews at 13 V/µs with 3 MHz GBW while the bipolar LM358 manages 0.3 V/µs with 1 MHz.

For the 741: $G_{m1} = 0.19$ mS and $C_c = 30$ pF give $\omega_u/2\pi = 0.19\ \text{mS}/(2\pi \times 30\ \text{pF}) = 1.0$ MHz, and $SR = 19\ \mu\text{A}/30\ \text{pF} = 0.63$ V/µs, close to the datasheet's typical 0.5 V/µs; its emitter-follower–common-base input cascade halves G_{m1} to $I_{tail}/(4V_T)$, which is why it slews twice as fast as the simple estimate above. Above the **full-power bandwidth** $f_{FP} = SR/(2\pi V_{pk})$ (*Engineering Physics* §18.4.3) large sinusoids distort.

Worked example — Compensating a two-stage CMOS op-amp. Design target: GBW 10 MHz, phase margin at least 60° into $C_L = 10$ pF, for an ADC driver on a 3.3 V supply (Figure 10.3).

1. *Second-stage transconductance.* Place p_2 at 2.2 times the GBW so that it contributes $\arctan(1/2.2) = 24.4^\circ$ of lag at crossover: $G_{m2} = 2.2 \times 2\pi \times 10\ \text{MHz} \times 10\ \text{pF} = 1.38$ mS, reached with about 200 µA in M6.
2. *Compensation capacitor.* A common rule takes $C_c \geq 0.22 C_L$; choose $C_c = 3$ pF.
3. *Input transconductance.* $G_{m1} = 2\pi \times 10\ \text{MHz} \times 3\ \text{pF} = 188$ µS.
4. *The right-half-plane zero.* Without R_z, $z/2\pi = G_{m2}/(2\pi C_c) = 73$ MHz, only 7.3 times the GBW. A numerical evaluation of the two-pole-one-zero model with $A_0 = 80$ dB gives crossover at 9.3 MHz and a phase margin of 59.9°; without the zero it would be 67.3°. The zero has cost 7.4°.
5. *Nulling resistor.* $R_z = 1/G_{m2} = 723\ \Omega$ removes the zero and restores 67°. Choosing $R_z = (C_L + C_c)/(G_{m2} C_c) = 3.1$ kΩ instead places a left-half-plane zero on p_2 and cancels it, in principle giving close to 90°, in practice limited

by the mirror pole and by C_L uncertainty.

6. *Slew rate.* With $I_{tail} = 20$ µA, $SR = 20\ \mu\text{A}/3\ \text{pF} = 6.7$ V/µs, a full-power bandwidth of 530 kHz at 2 V peak.

The same numbers explain a classic field failure, an op-amp driving a long cable: $p_2 \propto 1/C_L$, so at $C_L = 50$ pF this design's phase margin falls below 30° and the step response rings. The fix is an isolation resistor of 20 to 100 Ω in series with the output, outside the loop.

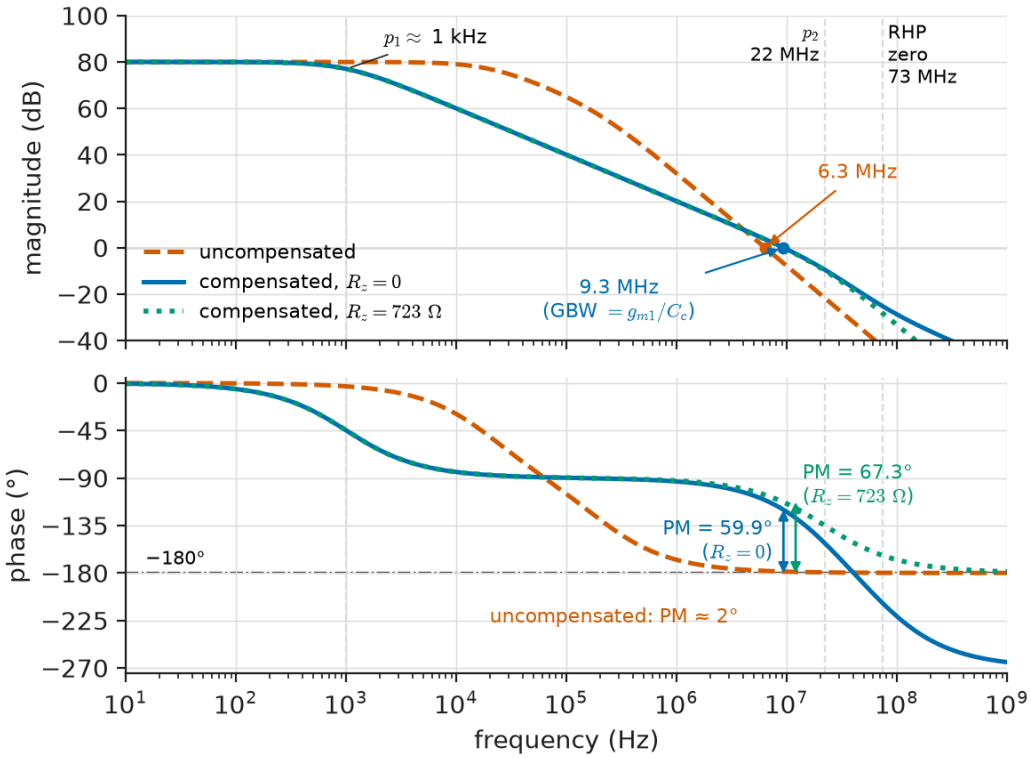


Figure 10.4 — Open-loop gain and phase of the two-stage op-amp before and after Miller compensation.

Other compensation styles

Decompensated op-amps (OPA637) use a smaller C_c and are stable only above a minimum gain such as 5, for five times the GBW. **Nested Miller** compensation serves three-stage low-voltage CMOS amplifiers. **Current-feedback** amplifiers set bandwidth with the feedback resistor and slew at thousands of volts per microsecond. An **LDO regulator** (Section 10.8) is the same problem with a load capacitor chosen by the user.

10.6 Analog IC architecture

Processes

The process decides which devices exist. **Bipolar** gives high g_m per ampere, low 1/f noise and good V_{BE} matching: precision and low-noise op-amps and references. **BiFET** adds JFETs for picoampere inputs (TL072). **CMOS** gives low-voltage, rail-to-rail, micropower parts integrated with logic, at the cost of offset and 1/f noise that chopping cures. **BCD** (bipolar-CMOS-DMOS) puts precision, logic and 40 to 100 V power devices on one die for power management and automotive ICs, and **SOI** serves high voltage, 175 to 225 °C operation and radiation tolerance.

Matching by layout and correction by trimming

Matched devices are drawn as **common-centroid** arrays (ABBA, cross-coupled quads) so that linear gradients cancel, with **dummy** devices at array ends and power devices on the thermal symmetry line of the input pair. What layout

cannot fix is trimmed: **laser trimming** of thin-film resistors, **Zener zapping**, or **digital trim** in fuses or EEPROM, bringing bipolar offsets to 10 to 100 µV and references to ±0.02 to ±0.1 %. **Chopper** amplifiers modulate the input to tens of kilohertz, amplify it there and demodulate it, moving offset and 1/f noise up to the chopping frequency where a filter removes them; **auto-zero** amplifiers sample and subtract their own offset. Parts such as the OPA333 reach a few microvolts and 0.02 µV/°C, with residual ripple at the chopping frequency.

Input and output stages for low supplies

An NMOS input pair stops working within about 1.1 V of the negative rail and a PMOS pair near the positive rail. **Rail-to-rail input** stages put both in parallel; in the crossover region, typically 1 to 1.5 V below the positive rail, g_m changes by up to 2:1 and offset by hundreds of microvolts, distorting a buffer whose input sweeps through it. Zero-crossover designs (OPA365) run one pair from an internal charge pump. **Rail-to-rail outputs** use common-source devices that swing to within $I \cdot R_{DS(on)}$ of the rails: tens of millivolts at 1 mA, hundreds at 20 mA.

Two traps catch users: **phase reversal** in some older JFET and bipolar parts (output jumps to the wrong rail when the input exceeds its common-mode range, turning negative feedback positive), and **input clamp diodes** that conduct heavily when an op-amp is misused as a comparator (Section 10.9).

Table 10.3 — Representative op-amp families (typical values at 25 °C).

Part (input technology)	Supply	V_{OS}	Drift	GBW	Slew rate	Noise at 1 kHz	I_Q per amplifier
LM358 (bipolar)	3–32 V	2 mV	7 µV/°C	0.7–1 MHz	0.3 V/µs	40 nV/√Hz	0.35 mA
TL072 (JFET)	±5 to ±18 V	3 mV	18 µV/°C	3 MHz	13 V/µs	18 nV/√Hz	1.4 mA
AD8605 (CMOS, RRIO)	2.7–5.5 V	20 µV	1 µV/°C	10 MHz	5 V/µs	8 nV/√Hz	1.2 mA
OPA333 (CMOS chopper)	1.8–5.5 V	2 µV	0.02 µV/°C	0.35 MHz	0.16 V/µs	55 nV/√Hz	17 µA
OPA1612 (bipolar, audio)	±2.25 to ±18 V	0.1 mV	1 µV/°C	40 MHz	27 V/µs	1.1 nV/√Hz	3.6 mA
LT1028 (bipolar, ultra-low noise)	±5 to ±18 V	10 µV	0.2 µV/°C	75 MHz	15 V/µs	0.85 nV/√Hz	7.4 mA

Noise falls with input-stage current (LT1028 against OPA333: 400:1 in current for 65:1 in noise density), speed costs current, and precision is bought with trimming or with chopping, which costs bandwidth.

10.7 Voltage references

What a reference must do

Every ADC compares its input with a reference; every DAC scales its output by one; every regulator and many sensor excitations are built on one. A reference is judged by **initial accuracy** (±0.02 % to ±2 %), **temperature coefficient** (0.05 to 100 ppm/°C), **long-term drift** (ppm per 1000 hours), **thermal hysteresis** (the shift after a temperature cycle), **noise** (µV peak-to-peak in 0.1–10 Hz) and line and load regulation. The temperature coefficient is normally specified by the **box method**:

$$TC = \frac{V_{max} - V_{min}}{V_{nom}\,(T_{max} - T_{min})} \times 10^{6} \ \text{ppm/°C}$$

which bounds the total excursion over the range without implying that the drift is linear.

The bandgap principle

> **Boundary.**
> The band gap of silicon (1.12 eV at room temperature) and its origin are in *Engineering Physics*, Ch. 6 §6.5–6.6; the temperature dependence of V_{BE} at constant current (about -2 mV/K) is in Chapter 3. This section uses both.

A forward-biased V_{BE} falls with temperature, about -2 mV/K: it is **complementary to absolute temperature** (CTAT). The difference between the V_{BE} of two transistors run at a current-density ratio n is

$$\Delta V_{BE} = V_T \ln n = \frac{kT}{q} \ln n$$

which is exactly **proportional to absolute temperature** (PTAT), with a slope of $(k/q)\ln n = 86.2\ \mu\text{V/K} \times \ln n$, independent of process parameters. A **bandgap reference** adds the two with a weight K chosen so that the slopes cancel:

$$V_{REF} = V_{BE} + K\,\Delta V_{BE}$$

When the cancellation is right, V_{REF} comes out close to the silicon band gap extrapolated to absolute zero, $E_{g0}/q \approx 1.205$ V (plus a small correction), which is where the circuit gets its name. The **Brokaw cell** (Figure 10.5) implements this with two transistors of emitter-area ratio 8:1, a resistor R_1 across which ΔV_{BE} appears, a resistor R_2 that scales it, and an amplifier that forces equal collector currents; $V_{REF} = V_{BE2} + 2(R_2/R_1)\Delta V_{BE}$, and only the resistor *ratio* matters, which is well controlled on chip. An amplifier and divider after the core give the standard outputs of 1.25, 2.048, 2.5, 3.0, 4.096 and 5.0 V.

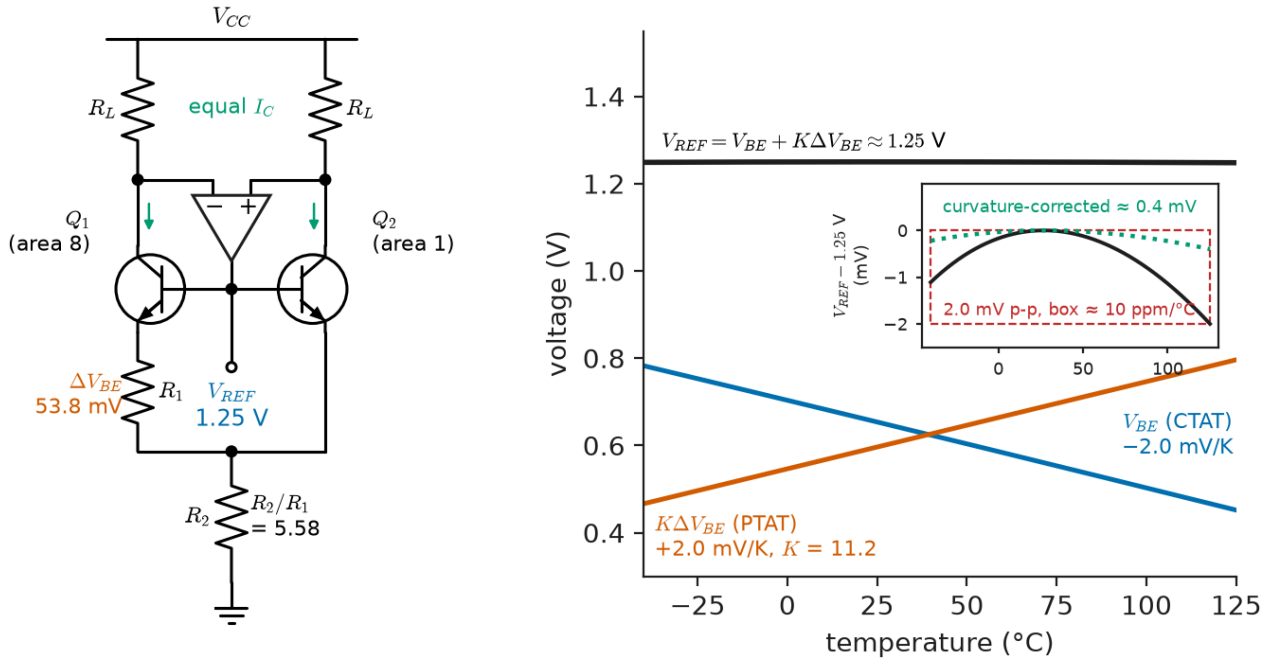


Figure 10.5 — Brokaw bandgap cell and the cancellation of CTAT and PTAT temperature slopes.

Worked example — Designing a bandgap and budgeting its error in an ADC. At 300 K a transistor at the chosen current has $V_{BE} = 0.650$ V with a slope of -2.0 mV/K. With a current-density ratio $n = 8$, $\Delta V_{BE} = 25.85\ \text{mV} \times \ln 8 = 53.8$ mV, with a slope of $86.2\ \mu\text{V/K} \times \ln 8 = 0.179$ mV/K. Cancellation requires $K = 2.0/0.179 = 11.16$, and then $V_{REF} = 0.650 + 11.16 \times 0.0538 = 1.250$ V. In the Brokaw cell, $K = 2R_2/R_1$, so $R_2/R_1 = 5.58$.

Now put a reference in front of the transmitter's 12-bit ADC from the opening. One LSB is $1/4096 = 244$ ppm of full scale. A 2.5 V shunt reference with a 50 ppm/°C box specification can move by $50 \times 70 = 3500$ ppm, 8.75 mV, over a 0 to 70 °C range: 14.3 LSB, which is the afternoon error the field engineers saw. A precision series reference with 3 ppm/°C

moves 210 ppm, 0.86 LSB, over the same range. For a 16-bit system (15 ppm per LSB) only 1 to 2 ppm/°C references will do, or a ratiometric measurement in which sensor excitation and ADC share the reference so that its drift cancels.

> **Status of the principle: Bandgap voltage reference**
> *Approximate.* First-order cancellation of the linear CTAT slope of V_{BE} by a PTAT term; the residual is a parabolic curvature from the nonlinear temperature dependence of V_{BE}, typically 10 to 20 ppm/°C uncorrected and 1 to 5 ppm/°C with curvature correction, and the result is further limited by resistor-ratio drift, package stress and amplifier offset.

Zener and buried-Zener references, and shunt versus series

A **buried Zener** (a base-emitter breakdown near 6 to 7 V formed below the surface, its small positive tempco cancelled by a forward V_{BE}) gives the lowest noise and long-term drift of any semiconductor reference: the heated LTZ1000 reaches 0.05 ppm/°C in 8.5-digit multimeters and calibrators, at the cost of a supply above 10 V. A **shunt** reference (TL431, LM4040) sinks whatever current a series resistor supplies, like a precision Zener; the TL431 is the error amplifier of most isolated mains supplies. A **series** reference (REF5025, ADR4525) is a small regulator with better accuracy, drift and noise.

Table 10.4 — Representative voltage references (typical or grade-limit values).

Part	Type	Output	Initial accuracy	Temperature coefficient	Noise 0.1–10 Hz	Supply current
TL431	Shunt bandgap, adjustable	2.495–36 V	±0.5 to ±2 %	20–50 ppm/°C	—	1 mA min
LM4040	Shunt bandgap	2.048–10 V	±0.1 to ±1 %	up to 100 ppm/°C	35 µV rms (10 Hz–10 kHz)	60 µA min
REF3030	Series bandgap	3.0 V	±0.2 %	50 ppm/°C max	33 µV p-p	50 µA
REF5025	Series bandgap, precision	2.5 V	±0.05 %	3 ppm/°C max	7.5 µV p-p	0.8 mA
ADR4525	Series bandgap, precision	2.5 V	±0.02 %	2 ppm/°C max	1.25 µV p-p	0.7 mA
LTZ1000	Buried Zener, heated	7.2 V	— (calibrated)	0.05 ppm/°C	1.2 µV p-p	5 mA + heater

10.8 Regulators

> **Boundary.**
> Switching regulators (buck, boost, buck-boost) are the subject of Chapter 30; the rectifier and reservoir capacitor in front of a linear regulator are in Chapter 6; thermal-resistance networks from junction to ambient are in *Engineering Physics*, Ch. 14 §14.7. This section covers the linear regulator as an analog integrated circuit.

The linear regulator loop

A linear regulator is an op-amp circuit: a reference V_{REF}, an error amplifier, a **pass element** in series between input and output, and a feedback divider R_1, R_2. The loop forces

$$V_{OUT} = V_{REF}\left(1 + \frac{R_1}{R_2}\right)$$

and the pass element drops the difference $V_{IN} - V_{OUT}$ at the full load current. That fact governs everything about linear regulators: their efficiency is at most

$$\eta = \frac{V_{OUT}\, I_{OUT}}{V_{IN}\,(I_{OUT} + I_Q)} \le \frac{V_{OUT}}{V_{IN}}$$

and their dissipation is $P_D = (V_{IN} - V_{OUT})I_{OUT} + V_{IN} I_Q$. In return they are quiet, fast and reject input ripple, so precision ADCs, PLLs and RF front ends have one close by, often after a switching converter.

Pass elements and dropout

The **dropout voltage** is the smallest $V_{IN} - V_{OUT}$ at which the loop still regulates; it is set by the pass element.

- **Standard (NPN Darlington)** regulators (LM7805, LM317) need $V_{BE} + V_{BE} + V_{CE(sat)}$ of the driver, about 1.5 to 2.5 V. The output is an emitter follower, so the output impedance is low without a capacitor, and they are stable with almost any load.
- **Quasi-LDO** (one NPN with a PNP driver, as in the LM1117) drops 1.0 to 1.3 V.
- **LDO** regulators use a PNP or, now almost always, a PMOS pass device as a common-emitter or common-source stage that can be driven fully on. Dropout is $I_{OUT} R_{DS(on)}$ for PMOS: 250 mV at 600 mA in a typical SOT-23 LDO means $R_{DS(on)} \approx 0.42\ \Omega$.

The price of a low dropout is a high-impedance output. A common-source pass device driving the load capacitor forms a pole $p_o = 1/[2\pi(R_L \parallel r_o)C_{OUT}]$ that moves with load current and with the user's choice of capacitor, inside the same loop as the error amplifier's own pole. Older PNP LDOs relied on the **ESR zero** $f_z = 1/(2\pi R_{ESR} C_{OUT})$ of a tantalum output capacitor to restore phase near crossover, and their datasheets specify a stable ESR window, typically from about 0.1 Ω to a few ohms. That is the transmitter failure of the opening: a 4.7 µF tantalum with 1 Ω ESR puts the zero at 34 kHz, inside the loop bandwidth; a 10 µF ceramic with 5 mΩ ESR puts it at 3.2 MHz, far above crossover, and the loop loses its phase margin. Modern LDOs use internal compensation (pole-zero tracking, an internal zero, or a buffer driving the large pass-device gate) and are specified as stable with any ceramic capacitor above a minimum value, typically 1 µF; when an old design is cost-reduced, the regulator must be changed with the capacitor.

Thermal design

The dissipation must leave through the package. With junction-to-ambient thermal resistance θ_{JA}, $T_J = T_A + P_D \theta_{JA}$, and the absolute limit is the junction rating (125 or 150 °C), with thermal shutdown typically near 165 °C.

Worked example — An LDO on a 5 V and on a 12 V rail. A sensor board needs 3.3 V at 300 mA. An LDO in a SOT-223 package with $\theta_{JA} = 60$ K/W on a modest copper area, $I_Q = 0.5$ mA, operates at up to $T_A = 85$ °C.

- From 5 V: $P_D = (5 - 3.3) \times 0.3 + 5 \times 0.0005 = 0.51$ W; $\Delta T = 0.51 \times 60 = 31$ K; $T_J = 116$ °C, below 125 °C. Efficiency $3.3 \times 0.3 / (5 \times 0.3005) = 66\%$. Acceptable.
- From 12 V: $P_D = (12 - 3.3) \times 0.3 + 12 \times 0.0005 = 2.62$ W; $\Delta T = 157$ K; T_J would be 242 °C. The regulator enters thermal shutdown and cycles; efficiency would have been 27 %.

The maximum allowed dissipation at 85 °C is $(125 - 85)/60 = 0.67$ W, which caps the input at $3.3 + 0.67/0.3 = 5.5$ V. From 12 V the right architecture is a buck converter (Chapter 30) to about 3.8 V followed by the LDO, which then dissipates 0.15 W and still filters the converter's ripple.

Power-supply rejection, noise and protection

PSRR is the loop gain at work: 60 to 80 dB at 1 kHz, falling to 20 to 40 dB near 1 MHz, where switching ripple lies, and collapsing near dropout. **Output noise** is the reference noise times $1 + R_1/R_2$; low-noise LDOs (TPS7A47, LT3045) filter the reference to reach 1 to 5 µV rms, which matters for PLL supplies. Protection includes **foldback current limiting**, **thermal shutdown** and, in automotive parts, reverse-battery and load-dump tolerance (ISO 16750-2).

Table 10.5 — Representative linear regulators (typical values).

Part	Pass element	Output	I_{OUT} max	Dropout	I_Q	Notes
LM7805	NPN Darlington	5 V	1.5 A	2 V	5 mA	Stable without output capacitor
LM1117	Quasi-LDO	1.8–5 V, adj.	0.8 A	1.2 V at 0.8 A	5 mA	Needs tantalum ESR window (older versions)
AP2112	PMOS	1.2–3.3 V	0.6 A	0.25 V at 0.6 A	55 µA	Ceramic-stable
TPS7A02	PMOS	0.8–5 V	0.2 A	0.27 V at 0.2 A	25 nA	Wearables, energy harvesting
TPS7A47	NPN, low noise	1.4–34 V	1 A	0.3 V at 1 A	0.6 mA	4 µV rms, PSRR 80 dB at 1 kHz

10.9 Comparators

> **Boundary.**
> Comparator *circuits* — threshold detectors, Schmitt triggers with external hysteresis, window comparators, relaxation oscillators — are in Chapter 8 and Chapter 9. This section is about what is inside a comparator IC and inside the comparators of an ADC.

Why a comparator is not an op-amp

A comparator runs open loop and must switch as fast as possible. It has **no compensation capacitor**, its stages **recover quickly from saturation** (an op-amp used as a comparator may take tens of microseconds), and its output is **logic-compatible**: open-drain (LM393) for wired-OR fault lines and level translation, push-pull CMOS for speed, LVDS for nanosecond timing. Many include a few millivolts of **internal hysteresis**. Delay depends on the **overdrive** beyond the threshold, so datasheets quote it at, say, 5 mV and 100 mV; the change of delay with overdrive (*dispersion*) is a timing error in zero-crossing detectors.

Table 10.6 — Representative comparators (typical values).

Part	Output	Supply	Propagation delay	Supply current	Typical use
LM393	Open collector	2–36 V	1.3 µs (5 mV overdrive), 0.3 µs (large)	0.4 mA per comparator	Supervisors, level detectors, legacy industrial
TLV3201	Push-pull	2.7–5.5 V	40 ns	40 µA	Over-current trip, zero crossing
LMV7219	Push-pull	2.7–5 V	7 ns	1.1 mA	Optical receivers, time-of-flight
ADCMP600	Push-pull, rail-to-rail	2.5–5.5 V	3.5 ns	4 mA	Clock recovery, fast PWM current limits

In the cycle-by-cycle current limit of an inverter or SMPS, the total trip delay (comparator, logic, gate driver) must beat the short-circuit withstand time: 1 to 3 µs for IGBTs, a few hundred nanoseconds for SiC and GaN (Chapter 29).

Latched comparators and metastability

Inside an ADC, a comparator must produce a full logic decision from a sub-millivolt input in a fraction of a clock period. The **latched (regenerative) comparator** does this with a small preamplifier followed by a cross-coupled pair of inverters, reset during one clock phase and released during the next. Once released, positive feedback makes any initial imbalance v_0 grow exponentially,

$$v(t) = v_0\, e^{t/\tau}, \qquad \tau \approx \frac{C}{g_m}$$

where C is the capacitance of each latch node and g_m the transconductance of the latch transistors. The time to reach a logic level V_L is $t = \tau \ln(V_L/v_0)$: with $\tau = 20$ ps, resolving a 1 mV input to 1 V takes $20\text{ ps} \times \ln 1000 = 138$ ps. An input closer to the threshold takes longer, and an input exactly at it never resolves within a finite time. This is **metastability**: the probability that a latch is still undecided after time t falls as $e^{-t/\tau}$ but never reaches zero. In a flash ADC it causes occasional large errors called sparkle codes; in digital logic it is the reason why a signal from another clock domain must pass through a synchroniser (Chapter 14). The comparator and the flip-flop are the same circuit seen from two sides of the analog-digital boundary.

> **Status of the principle: Exponential regeneration of a latch**
> *Approximate.* Linearised small-signal model of the cross-coupled pair around its balance point, valid while both transistors are in their high-gain region; it fails once the output approaches a rail, where the response saturates, and it ignores input-referred noise, which in practice randomises decisions within a band of tens to hundreds of microvolts around the threshold.

10.10 Mixed-signal interfaces

> **Boundary.**
> Sampling, aliasing, aperture jitter and the sample-and-hold are in *Circuits, Signals and Communications*, Ch. 19 §19.5 and §19.7; quantization noise, SQNR $= 6.02N + 1.76$ dB, ENOB, oversampling and noise shaping in Book 3 Ch. 19 §19.8 and Ch. 21 §21.2. Those results are used here as the specification that the converter circuits must meet. How a microcontroller configures, triggers and reads its converters is in Chapter 17 and Chapter 26.

Static accuracy: offset, gain, DNL and INL

Beyond ideal quantization, a converter with LSB $V_{REF}/2^N$ has **offset** and **gain** errors (removed by two-point calibration), **differential nonlinearity** (DNL, each code width's deviation from 1 LSB) and **integral nonlinearity** (INL, the accumulated deviation from a straight line). DNL above -1 LSB makes a DAC **monotonic** and an ADC free of **missing codes**; in a control loop a non-monotonic DAC causes limit cycles and a missing code a dead band.

DAC architectures

- **Resistor string**: 2^N equal resistors and a switch tree; inherently monotonic, practical to 8 to 12 bits. Microcontroller DACs, digital potentiometers.
- **R-2R ladder** (Figure 10.6, left): two resistor values, each leg's current halved at each node. In **current mode** (AD5450) the legs switch between an op-amp's summing node and ground; in **voltage mode** (DAC8830) between V_{REF} and ground. Laser-trimmed thin film reaches 14 to 16 bits.
- **Current steering**: unit current sources switched by differential pairs into a load; GS/s DACs for radar and 5G transmitters.
- **Sigma-delta DAC**: a digital noise-shaping modulator drives a 1-bit DAC and an analog filter; audio.

The critical transition of a binary-weighted DAC is the **major carry** at mid-scale, $011\ldots1 \rightarrow 100\ldots0$, where the MSB element must equal all the others plus one LSB. An MSB error ε makes that step wrong by about $2^{N-1}\varepsilon$ LSB, and unequal switching instants produce a **glitch impulse**. **Segmentation** (thermometer coding of the top 3 to 6 bits with equal elements) cures both.

ADC architectures

- **Flash**: $2^N - 1$ latched comparators compare the input with taps of a resistor ladder at once, and an encoder converts the thermometer code to binary. Fastest (1 to 10 GSa/s at 6 to 8 bits), but comparators, input capacitance and power double per bit: 255 for 8 bits. Metastable comparators cause the sparkle codes of Section 10.9; bubble-correcting encoders limit them.
- **Pipeline**: low-resolution stages each digitise a few bits, subtract them with a DAC and amplify the residue for the next; 10 to 16 bits at 10 MSa/s to GSa/s, with several clocks of latency.
- **Successive approximation (SAR)**: one comparator and an N-bit **charge-redistribution** capacitor DAC, which doubles as the sample-and-hold, perform a binary search, one bit per clock (Figure 10.6, right). Power scales with speed, so SAR dominates 8 to 18 bits: every microcontroller ADC, battery monitors, motor-control current sensing.
- **Sigma-delta** ($\Sigma\Delta$): a loop filter, a 1-bit comparator and a 1-bit DAC in feedback, heavily oversampled and followed by a decimation filter that removes the shaped noise (Book 3 §21.2). A two-level DAC is perfectly linear, so 16 to 24 bits need no matching; the decimation filter adds tens of samples of delay, which matters in control loops. Audio, metering, weigh scales, bridge sensors.
- **Dual-slope integrating**: integrate the input for a fixed time, de-integrate with the reference while counting. Component values cancel and a 20 ms multiple rejects 50 Hz; 1 to 100 Sa/s at 4.5 to 6.5 digits, in multimeters.

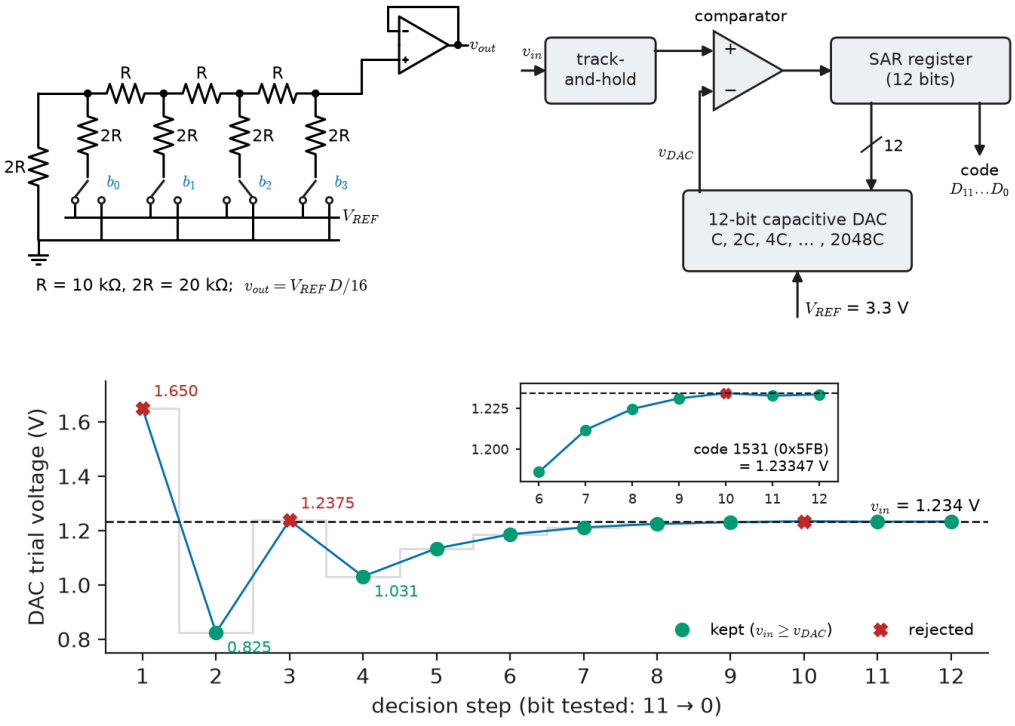


Figure 10.6 — R-2R ladder DAC and the successive-approximation ADC with its binary search.

Table 10.7 — ADC architectures compared (typical ranges, 2026 commercial parts).

Architecture	Resolution	Sample rate	Latency	Typical application
Flash	6–8 bits	1–10 GSa/s	1 clock	Oscilloscopes, sub-ADCs, optical links
Pipeline	10–16 bits	10 MSa/s–3 GSa/s	5–20 clocks	Software radio, ultrasound, radar
SAR	8–18 bits	10 kSa/s–50 MSa/s	1 sample	MCU ADCs, motor control, BMS, multiplexed DAQ
Sigma-delta	16–24 bits	10 Sa/s–5 MSa/s output	10–100 output samples	Audio, metering, weigh scales, ECG
Dual-slope integrating	12–22 bits	1–100 Sa/s	1 conversion	Multimeters, panel meters

The SAR conversion as an algorithm

The binary search of a SAR converter is a few lines of code. The listing models an ideal 12-bit SAR with a 3.3 V reference: at each step the next bit is set tentatively, the DAC output is compared with the held input, and the bit is kept if the input is at least as large.

```
VREF, N = 3.3, 12

def sar_convert(vin, vref=VREF, nbits=N, verbose=False):
    code = 0
    for bit in reversed(range(nbits)):
        trial = code | (1 << bit)              # set the bit under test
        vdac = trial * vref / 2**nbits          # capacitive-DAC output
        if vin >= vdac:                         # comparator decision
            code = trial                        # keep the bit
        if verbose:
            print(f"bit {bit:2d}: DAC {vdac:.5f} V -> {'keep' if code & (1 << bit) else 'clear'}")
    return code

c = sar_convert(1.234, verbose=True)
print(f"code {c} (0x{c:03X}), reconstructed {c * VREF / 2**N:.5f} V")
```

Running it for $v_{in} = 1.234$ V prints twelve decisions — the first trial at mid-scale, 1.650 V, is rejected; the second, 0.825 V, kept; and so on — and ends with code 1531 (0x5FB), whose lower code edge is 1.23347 V, within one LSB (0.806 mV) below the input. Each of the N decisions must settle the DAC to half an LSB, $\ln(2^{13}) = 9.0$ time constants at 12 bits, which limits SAR speed.

Worked example — How good must the SAR's capacitors and the R-2R resistors be? Two circuit limits follow from the numbers above.

Thermal noise of the sampling capacitor. The sampled input carries kT/C noise with RMS value $\sqrt{kT/C}$ (Book 3 treats the sampling, this is the circuit's contribution). For a 16-bit SAR with a 5 V range, one LSB is 76.3 µV and the ideal quantization noise $\Delta/\sqrt{12} = 22.0$ µV rms. Requiring kT/C noise to be no larger than that gives $C \geq kT/(22.0\ \mu\text{V})^2 = 8.5$ pF at 300 K; with 30 pF the noise is 11.8 µV rms. The driving op-amp (Section 10.5) must charge these tens of picofarads within the acquisition time.

Ladder matching. In a 10-bit R-2R DAC whose MSB leg is 0.1 % off, the mid-scale step error is about $2^9 \times 0.001 = 0.51$ LSB, so DNL stays above -1 LSB and the DAC is monotonic. A 12-bit DAC with 0.05 % resistors has a mid-scale error of about $2^{11} \times 0.0005 = 1.02$ LSB, enough for a non-monotonic step in the worst case: 12 bits and beyond need laser-trimmed resistors, segmentation or self-calibration.

> **Status of the principle: Converter accuracy is a matching problem**
> *Derived*. In every Nyquist-rate architecture the bit weights are ratios of matched elements (resistors, capacitors or current sources), so linearity at N bits requires matching to about 2^{-N} of the major element; sigma-delta converters escape this by using a two-level DAC, at the cost of oversampling and digital filtering.

What remains open

Open question	Where it is answered
How do comparator outputs and converter codes become logic signals, numbers and arithmetic?	Chapter 11 (codes, two's-complement and offset-binary formats) and Chapter 12
How is metastability of a latch handled in synchronous digital design?	Chapter 14, Chapter 15
How does a microcontroller configure, trigger and read its SAR ADC and DAC, and how are external converters interfaced over SPI and I²C?	Chapter 17, Chapter 26
What does the digital side do with the samples, and how does finite word length limit it?	*Circuits, Signals and Communications*, Ch. 21
How are switching regulators designed when a linear regulator dissipates too much?	Chapter 30
How are matched devices, capacitor arrays and references drawn and verified on silicon, and how do CMOS transistors behave as switches?	Chapter 33, Chapter 34
How are mixed-signal blocks partitioned against processors and FPGAs at system level?	Chapter 35

Next: Chapter 11 opens Part III by asking what the codes leaving an ADC actually mean — how numbers, signs, fractions and floating-point values are represented in binary hardware, and how they overflow.

Part III — Digital Electronics

Part II ended where analog circuits hand their results to logic: a comparator decides, an ADC delivers a code. This part builds the digital half of electronics from that boundary. A bit is an abstraction imposed on a voltage, valid only while the voltage stays outside the undefined band and the clock leaves time for every signal to settle. Inside those limits digital design offers exact repeatability, cascading without accumulated noise, and designs that can be specified completely and verified on every input. Outside them come the characteristic digital failures: a field read with the wrong sign or byte order, an overflow that turns full forward into full reverse, a glitch that clocks a register, a synchroniser that fails once a month. This part develops the representations, logic, building blocks and timing rules that make synchronous digital systems work at first power-up and keep working.

By the end of this part you should understand

- How integers, signed values, fixed-point fractions and IEEE 754 floating-point numbers are represented in bits, what each costs in range, resolution and hardware, and how overflow is detected, saturated or exploited.
- How requirements become Boolean equations and truth tables, how De Morgan's laws, Karnaugh maps and minimisation algorithms reduce them to gates, and what logic levels and noise margins a real gate family guarantees.
- How adders, multiplexers, decoders, encoders, comparators, ALUs and code converters are organised, and how their structure trades logic for propagation delay.
- How latches, flip-flops, registers, counters and shift registers store state, how finite-state machines are encoded, and how setup, hold, clock-domain crossings and resets constrain them.
- How a complete digital system goes from specification through state diagrams to an implementation, how glitches and asynchronous inputs are handled, and how test benches verify it before hardware.

Chapters

Chapter	Central question	What it delivers
Chapter 11 — Digital Number Systems	How are numbers of every kind held in a fixed number of bits, and what happens at the edges of the representation?	Binary, octal and hexadecimal with byte order; sign-magnitude, offset binary and two's complement; arithmetic cost on real processors; the overflow rule and saturation; fixed-point scaling and word growth; IEEE 754 and AI floating-point formats in hardware; Boolean variables as flags and register fields
Chapter 12 — Boolean Algebra and Logic Gates	How are requirements turned into simplified gate networks with guaranteed voltage levels?	Boolean laws as rewriting rules, gate set and symbols, logic families and noise margins, truth tables and canonical forms, De Morgan and bubble pushing, Karnaugh maps with don't-cares and hazards, multi-output combinational design, Quine–McCluskey and synthesis-tool minimisation
Chapter 13 — Combinational Digital Systems	How are the standard combinational blocks organised, and how long do they take to produce a valid output?	Ripple, lookahead and prefix adders, subtractors, multiplexers and barrel shifters, demultiplexers, encoders, decoders, comparators, ALUs with status flags, Gray and BCD code converters, propagation and contamination delay and critical paths
Chapter 14 — Sequential Digital Systems	How is memory added to logic, and what timing rules must a synchronous circuit obey?	Latches and flip-flops, registers, counters, shift registers and LFSRs, Moore and Mealy state machines, state encoding, setup and hold constraints and maximum clock frequency, synchronisers and metastability MTBF, reset strategies
Chapter 15 — Digital System Design	How is a complete digital system taken from specification to verified hardware?	Design methodology, state diagrams and state tables, hardware timing on boards and chips, glitches, synchronisation of asynchronous inputs, memory elements in designs, logic implementation choices and verification with test benches

How this part connects to the rest of the book

Part III stands on Parts I and II: the comparators and converters of Chapter 10 are the analog side of every digital input, and the MOSFET switch of Chapter 4 lies beneath every gate, whose transistor-level design is deferred to Chapter 33. Part IV assembles these blocks into processors, memories and programmable logic described in HDLs; Part V programs them, Chapter 21 returning to integer types, bitwise operators and fixed point in C; Part VI builds embedded and real-time systems on them; and Part VIII returns to logic at chip scale, with synthesis and timing analysis in Chapter 34.

Three sibling books supply theory used here without re-derivation. Boolean algebra and functional completeness, and the floating-point error model, come from *The Mathematics Universe*, Ch. 29 and Ch. 22. Finite-word-length effects in digital filters, sampling and quantisation, and transmission-line behaviour of fast board traces come from *Circuits, Signals and Communications*, Ch. 19, 21 and 14. The carrier physics of the transistors that switch underneath every gate comes from *Engineering Physics: Top Down*, Ch. 6 and Ch. 18.

Chapter 11 — Digital Number Systems

Part III · Digital Electronics

On 25 February 1991 a Patriot air-defence battery in Dhahran failed to intercept an incoming Scud missile. The battery had been running for about 100 hours. Its software counted time in tenths of a second and converted the count to seconds by multiplying by 0.1 held in a 24-bit fixed-point register, but 0.1 has no finite binary expansion, and the stored constant was short by about 9.5×10^{-8}. After 3.6 million ticks the computed time was 0.34 s behind, enough for the radar's range gate to look more than half a kilometre away from the target. Five years later the first Ariane 5 launcher destroyed itself 37 seconds after lift-off when its inertial reference software converted a 64-bit floating-point horizontal-velocity value into a 16-bit signed integer; the value exceeded 32 767, the conversion raised an unhandled operand error, and both redundant units shut down. Neither failure was a logic error in the usual sense. Each was a number that did not fit the bits chosen to hold it. Every value in a digital system — an ADC code, a motor current, a timestamp, a coefficient, a flag — lives in a fixed number of bits, and the representation decides what range it covers, how finely it resolves, what happens when it overflows and how much hardware and time arithmetic on it costs.

> **How are integers, signed values, fractions and real numbers represented in a fixed number of bits, and what does each representation cost in range, resolution and hardware?**
> **What happens at the edges — overflow, wrap-around, rounding and conversion between formats — and how does an engineer detect and contain it?**

Concept	Engineering destination
Binary, octal, decimal and hexadecimal	Register maps, memory dumps, bus analysers, configuration words
Signed number representation	ADC output codes (offset binary, two's complement), sensor and protocol fields
Two's complement	Every integer ALU, signed sensor data, wrap-safe timer arithmetic
Binary arithmetic	Adder, multiplier and divider cost; cycle counts on microcontrollers
Overflow	Status flags, saturating arithmetic, timer wrap, launch and safety failures
Fixed-point arithmetic	Motor control, power metering and filters on MCUs and FPGAs without floating point
Floating-point representation	IEEE 754 FPUs in processors, GPUs and AI accelerators; format choice and conversion
Boolean variables	Flags, bit fields, control and status registers, active-low signals

Roadmap. Positional number systems and the hexadecimal view of hardware (11.1) → the ways of encoding a sign (11.2) → two's complement, the one that won (11.3) → what addition, multiplication and division cost in hardware (11.4) → overflow and how to contain it (11.5) → fixed-point fractions (11.6) → IEEE 754 floating point (11.7) → single bits as Boolean variables (11.8).

11.1 Binary, octal, decimal and hexadecimal

> **Boundary.**
> The real and complex number systems as mathematics are in *The Mathematics Universe*, Ch. 1 §1.1; floating-point error analysis in Ch. 22 §22.1; fixed- and floating-point arithmetic as a source of error in signal processing in *Circuits, Signals and Communications*, Ch. 21 §21.4–21.6. This chapter takes the hardware view: bit patterns, their encodings and what circuits and processors do with them.

Why two states

A digital circuit represents information by voltages that are deliberately driven far towards one of two levels. Two levels make the widest possible noise margin for a given supply (Chapter 12 puts numbers on it), make the switching transistor either fully on or fully off where it dissipates least, and let any regenerating gate restore a degraded signal. Multi-level signalling exists where bandwidth is scarce — four-level PAM4 in 400G Ethernet and PCIe 6.0, and three or four bits per cell in NAND Flash — but the price is a smaller margin per level and error-correcting codes to recover from it. Inside logic, the unit is the **bit**.

Positional notation

A string of digits $d_{n-1} \ldots d_1 d_0 . d_{-1} d_{-2} \ldots$ in base b has the value

$$x = \sum_{i} d_i \, b^i$$

Base 2 uses digits 0 and 1; each binary digit position doubles in weight. Groups of bits have names: 4 bits a **nibble**, 8 bits a **byte**, and a **word** whatever the processor's natural width is (16 bits for the MSP430, 32 for a Cortex-M, 64 for a server CPU). The leftmost bit is the **most significant bit** (MSB), the rightmost the **least significant bit** (LSB).

Engineers read binary through two shorthand bases. **Hexadecimal** (base 16, digits 0–9 and A–F) packs exactly four bits into one digit, so a 32-bit register is eight hex digits and every digit boundary is a nibble boundary: `0x4002_0014` is the address of the GPIOA output data register in an STM32F4, and `0xA5` reads at once as `1010 0101`. **Octal** (base 8) packs three bits per digit; it survives in Unix file permissions (`chmod 754`), in some aviation transponder (Mode A "squawk") codes, and in older minicomputer documentation. **Decimal** remains the interface to people, and conversion to and from it is the expensive part of every display, log file and user interface.

Table 11.1 — Common unsigned word widths and their ranges.

Width	Hex digits	Unsigned range	Typical use
1 bit	—	0–1	Flag, GPIO pin, Boolean variable
8 bits	2	0–255	Byte; UART character; 8-bit MCU register; pixel channel
10–12 bits	3	0–1023, 0–4095	MCU ADC result
16 bits	4	0–65 535	Timer counter, Modbus register, 16-bit audio sample
24 bits	6	0–16 777 215	Sigma-delta ADC result, RGB colour
32 bits	8	0–4 294 967 295	Cortex-M register, IPv4 address, millisecond tick counter
64 bits	16	$0 – 1.8 \times 10^{19}$	Server register, nanosecond timestamp, file size

Converting between bases

Integer conversion from decimal to base b is **repeated division**: the remainders, read in reverse order, are the digits. Fractions convert by **repeated multiplication**: multiply by b, the integer part is the next digit, keep the fraction. The listing implements both and decodes a two-byte field as a protocol analyser would.

```
def to_base(n, base, digits="0123456789ABCDEF"):
    """Unsigned integer to a string in any base, by repeated division."""
    out = ""
    while True:
        n, r = divmod(n, base)
        out = digits[r] + out
        if n == 0:
            return out

def frac_to_bin(x, nbits):
    """Fraction 0 <= x < 1 to binary digits, by repeated multiplication."""
    bits = ""
    for _ in range(nbits):
        x *= 2
        bits += "1" if x >= 1 else "0"
        x -= int(x)
    return bits

def to_signed(u, n):
    """Interpret an n-bit pattern u as two's complement."""
    return u - (1 << n) if u & (1 << (n - 1)) else u

print(to_base(2026, 2), to_base(2026, 8), to_base(2026, 16))
print("0.1 =", "0." + frac_to_bin(0.1, 24), "...")

payload = bytes([0x2E, 0xFF])             # CAN data bytes 0 and 1
raw = payload[0] | (payload[1] << 8)      # little-endian 16-bit word
print(f"raw 0x{raw:04X} unsigned {raw} signed {to_signed(raw, 16)}"
      f" -> {to_signed(raw, 16) * 0.1:.1f} degC")
```

It prints `11111101010 3752 7EA` for 2026, the repeating pattern `0.0001100110011001100110011001...` for 0.1, and `raw 0xFF2E unsigned 65326 signed -210 -> -21.0 degC`. The second line is the root of the Patriot failure (Section 11.6): a fraction whose denominator in lowest terms contains a factor 5 never terminates in binary, so 0.1, 0.01 and 0.2 are always approximations in binary hardware. The third line anticipates Section 11.3.

Byte order

A multi-byte value stored in byte-addressed memory or sent over a byte-oriented link must put its bytes in some order. **Little-endian** stores the least significant byte at the lowest address; x86, Arm Cortex-M and RISC-V (by default) are little-endian, and so is CAN data in most automotive DBC definitions ("Intel" byte order). **Big-endian** stores the most significant byte first; it is **network byte order** for IP, TCP and UDP headers, Modbus registers and many sensor ICs on SPI and I²C ("Motorola" order in DBC files). A 32-bit value 0x12345678 is stored as `78 56 34 12` in little-endian memory and sent as `12 34 56 78` in network order. Endianness errors produce wrong values, sometimes obviously wrong (a temperature of +1203.1 °C instead of −21.0 °C when the bytes below are read big-endian) and sometimes plausible; firmware should convert explicitly at the boundary of each interface.

Worked example — Reading a CAN temperature field. A battery-management CAN message carries a cell temperature in bytes 0–1, little-endian, signed, with a scale of 0.1 °C per bit, and a bus analyser shows the bytes `2E FF`. Assembled little-endian, the word is $0\text{xFF2E} = 65\,326$ as an unsigned number. Its MSB is 1, so as a 16-bit two's-complement number it is $65\,326 - 65\,536 = -210$, and the temperature is −21.0 °C, a plausible winter value. Read big-endian (`0x2EFF` $= 12\,031$) it would be +1203.1 °C; read little-endian but unsigned, 6532.6 °C. One field definition, three interpretations: the bits carry no meaning without the format, which is why automotive and industrial protocols publish their field definitions (DBC, EDS and Modbus register maps) as precisely as their electrical layers.

Units of data

Memory sizes are powers of two and line rates are powers of ten. The IEC prefixes of IEC 80000-13 keep them apart: 1 KiB $= 1024$ bytes, 1 MiB $= 2^{20}$ bytes, while 1 kB $= 1000$ bytes and 1 Mbit/s $= 10^6$ bit/s. A "64 GB" memory card holds 64×10^9 bytes, which an operating system reporting in binary units shows as 59.6 GiB. Small discrepancies of this kind are harmless; a buffer sized as 1000 bytes when the DMA engine transfers 1024 is not.

11.2 Signed number representation

An n-bit pattern has 2^n values. Representing signed numbers means deciding which patterns mean negative values. Four encodings are in use, each for a reason (Table 11.2).

Sign-magnitude uses the MSB as the sign and the remaining $n-1$ bits as the magnitude. It is the natural human notation and survives where magnitude and sign are processed separately: the sign bit of IEEE 754 floating point (Section 11.7), some sensor formats, and the sign-plus-magnitude command of an H-bridge motor driver (direction pin plus PWM duty). It has two zeros ($+0$ and -0), and an adder must compare magnitudes before deciding whether to add or subtract them.

Ones' complement negates a number by inverting every bit. It also has two zeros. It is extinct in processors but survives in one important place: the 16-bit Internet checksum of IPv4, UDP and TCP headers is a ones'-complement sum, chosen because it is independent of byte order and easy to update incrementally in routers.

Offset binary (excess-K) stores $x + K$ as an unsigned number, usually with $K = 2^{n-1}$. The most negative value is all zeros, zero is $100\ldots0$, and the most positive value is all ones. It is the native output of a bipolar ADC: a converter measuring $\pm V_{REF}$ produces code 0 at $-V_{REF}$, mid-scale at 0 V and full scale near $+V_{REF}$, which is just a unipolar converter with its input shifted. The exponent field of IEEE 754 is also offset binary (with $K = 127$ in binary32), which makes floating-point numbers sort correctly when compared as integers.

Two's complement gives the MSB a negative weight, -2^{n-1}. It has one zero, an asymmetric range from -2^{n-1} to $2^{n-1}-1$, and the overwhelming advantage that the same adder circuit handles signed and unsigned numbers. Every modern integer ALU uses it, and since C23 and C++20 it is the only signed integer representation the C and C++ standards allow.

Table 11.2 — Four 4-bit signed encodings.

Pattern	Unsigned	Sign-magnitude	Ones' complement	Offset binary ($K = 8$)	Two's complement
0111	7	+7	+7	−1	+7
0001	1	+1	+1	−7	+1
0000	0	+0	+0	−8	0
1111	15	−7	−0	+7	−1
1001	9	−1	−6	+1	−7
1000	8	−0	−7	0	−8
Range	0 to 15	−7 to +7	−7 to +7	−8 to +7	−8 to +7

Offset binary and two's complement differ only in the MSB: inverting the MSB converts one into the other. A firmware driver for a bipolar ADC that delivers offset binary therefore converts its result to a signed integer with a single XOR, and many converters (the ADS1115, most audio codecs) offer a configuration bit that selects two's-complement output directly.

11.3 Two's complement

Arithmetic modulo 2^n

An n-bit register is a counter modulo 2^n: adding 1 to $11\ldots1$ gives $00\ldots0$ with a carry out that the register discards. Two's complement assigns to each pattern the representative of its residue class that lies in $[-2^{n-1}, 2^{n-1}-1]$. For an n-bit pattern $b_{n-1}\ldots b_0$ the value is

$$x = -b_{n-1}2^{n-1} + \sum_{i=0}^{n-2} b_i 2^i$$

Because the pattern for $-x$ is the one that, added to x, gives $2^n \equiv 0$, negation is

$$-x = \overline{x} + 1$$

(invert every bit, add one), since $x + \overline{x} = 2^n - 1$. Figure 11.1 shows the 4-bit patterns arranged on a wheel: moving clockwise adds, moving counter-clockwise subtracts, and the same wheel serves unsigned numbers (0 to 15) and signed numbers (−8 to +7). The only difference between the two interpretations is where the wheel is cut: between 15 and 0 for unsigned, between +7 and −8 for signed. Crossing the cut is overflow (Section 11.5).

> **Status of the principle: Two's-complement arithmetic**
> *Derived.* Follows from arithmetic modulo 2^n: addition, subtraction and the low n bits of multiplication give the correct result whenever the true result lies in $[-2^{n-1}, 2^{n-1}-1]$, using exactly the same circuit as unsigned arithmetic; outside that range the result is wrong by a multiple of 2^n.

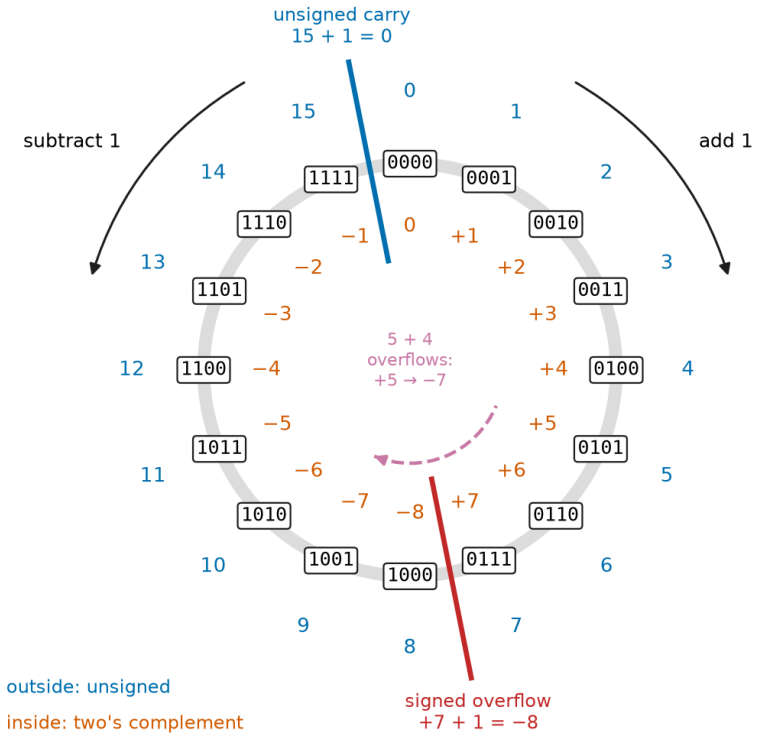


Figure 11.1 — The 4-bit number wheel: one set of patterns read as unsigned and as two's-complement values.

Sign extension and shifts

Widening a two's-complement number to more bits copies its sign bit into the new upper bits: -3 is `1101` in 4 bits and `1111 1101` in 8 bits. Zero extension, appropriate for unsigned numbers, would turn it into $+13$. Processors have both

forms: Arm's `SXTB` / `SXTH` and `UXTB` / `UXTH`, and the load instructions `LDRSB` / `LDRSH` (sign-extending) and `LDRB` / `LDRH` (zero-extending). A 12-bit two's-complement ADC result placed in a 16-bit register must be sign-extended before use; forgetting it is a classic bug that turns small negative readings into values near +4000.

Shifting left by k multiplies by 2^k for both interpretations. Shifting right divides by 2^k, but there are two right shifts: **logical** (fill with zeros, correct for unsigned) and **arithmetic** (fill with copies of the sign bit, correct for signed). An arithmetic right shift rounds towards $-\infty$: $-7 \gg 1 = -4$, while C integer division truncates towards zero, $-7/2 = -3$. In C, right-shifting a negative signed integer is implementation-defined (arithmetic on every mainstream compiler); left-shifting a negative value or shifting into the sign bit is undefined behaviour, so portable code uses unsigned types for bit manipulation.

Wrap-around as a tool

Modular arithmetic is often exactly what is wanted. A free-running 32-bit millisecond tick counter wraps every 2^{32} ms $=$ 49.7 days; a Windows 95 timer bug famously hung machines after that interval, and the same bug recurs in embedded firmware written as `if (now >= start + timeout)`. The wrap-safe form computes the elapsed time as an unsigned difference, which is correct across the wrap because subtraction modulo 2^{32} gives the true difference whenever it is less than 2^{32}:

```
 #include <stdbool.h>
#include <stdint.h>

/* True once at least timeout_ms have elapsed since start_ms.
   Correct across the 2^32 wrap of the tick counter, provided the
   interval being measured is shorter than 49.7 days. */
static inline bool timed_out(uint32_t now_ms, uint32_t start_ms,
                             uint32_t timeout_ms)
{
    return (uint32_t)(now_ms - start_ms) >= timeout_ms;
}
```

The same property makes phase accumulators in direct digital synthesis and encoder position counters work: a 16-bit encoder count can wrap freely as long as the controller only uses differences between successive readings that are smaller than half the range.

11.4 Binary arithmetic

Addition and subtraction

Binary addition proceeds column by column exactly as decimal addition does, with a carry of weight 2 into the next column. One column takes two operand bits and a carry in and produces a sum bit and a carry out; this is the full adder, whose logic Chapter 12 minimises and whose fast carry-lookahead forms Chapter 13 develops. Subtraction is addition of the negation, $A - B = A + \overline{B} + 1$: an adder with an inverter on one input and the carry-in set to 1. In unsigned subtraction the final carry out is the inverse of a borrow: carry 1 means $A \geq B$. Arm processors keep that convention (the C flag after `SUBS` or `CMP` is "not borrow"), while x86 sets its carry flag on borrow; compilers hide the difference, assembly programmers and simulator writers must not.

Multiplication

The product of an n-bit and an m-bit number needs $n + m$ bits. A multiplier forms n partial products (each operand shifted and ANDed with one bit of the other) and sums them; hardware multipliers compress the partial products with trees of full adders (Wallace or Dadda trees) and use Booth recoding to halve their number, so a 32 × 32-bit multiply

completes in one clock cycle on a Cortex-M3/M4 and in three to four pipeline stages on a server CPU. Signed multiplication differs from unsigned only in the upper half of the product: the low n bits of the product are the same for both interpretations, which is why C's `int` multiplication and `unsigned` multiplication compile to the same instruction while the "long multiply" instructions (`SMULL` , `UMULL`) are distinct.

Division

Division is the expensive operation. Hardware dividers produce one to a few quotient bits per cycle (restoring, non-restoring or SRT algorithms), so a 32-bit divide takes 2 to 12 cycles on a Cortex-M3/M4 (data-dependent), 20 to 40 cycles on larger cores for 64-bit operands, and on cores without a divider (Cortex-M0, 8-bit AVR) a software routine of 100 or more cycles. Division by a constant is replaced by multiplication by a scaled reciprocal and a shift: for any 16-bit unsigned x, $\lfloor x/10 \rfloor = (x \times 0\text{xCCCD}) \gg 19$ exactly, which compilers generate automatically. Division by a power of two is a shift — but for signed numbers only after adding $2^k - 1$ to negative operands, so that the result truncates towards zero as C requires.

Table 11.3 — Integer arithmetic cost on representative processors (typical cycle counts).

Operation	8-bit AVR (ATmega328P)	Cortex-M0+	Cortex-M4	Notes
8-bit add	1	1	1	—
32-bit add	4	1	1	AVR: add plus three add-with-carry
8 × 8 multiply	2	1	1	—
32 × 32 → 32 multiply	about 40 (library)	1 or 32 (implementation option)	1	—
32 × 32 → 64 multiply	about 80	about 40 (library)	1	`UMULL` / `SMULL` on M4
32 / 32 divide	about 600 (library)	about 50–150 (library)	2–12	M4 `UDIV` / `SDIV` early-terminate

The table explains much of embedded firmware style: on an 8-bit core, 32-bit arithmetic is a library call, and on any core without a hardware divider a control loop that divides every cycle wastes much of its budget. The engineering responses are wider multiply instead of divide, precomputed reciprocals, and power-of-two scale factors.

11.5 Overflow

Carry versus overflow

An n-bit adder produces an n-bit result and a carry out. For unsigned operands the result is wrong exactly when the carry out is 1: the true sum exceeds $2^n - 1$. For two's-complement operands the carry out means nothing on its own; the result is wrong when the true sum lies outside $[-2^{n-1}, 2^{n-1} - 1]$. That can only happen when both operands have the same sign (the sum of a positive and a negative number always lies between them), and it shows itself as a result of the opposite sign. Processors therefore keep two flags, C (carry) and V (overflow), and the program's choice of conditional branch decides which interpretation applies: unsigned comparisons test C, signed comparisons test V together with N (negative).

The overflow rule

Let c_{n-1} be the carry into the sign position and c_n the carry out of it. Consider the sign bits a and b of the operands and s of the result, $s = a \oplus b \oplus c_{n-1}$, $c_n = ab + c_{n-1}(a \oplus b)$.

- If $a = b = 0$ (both non-negative): $c_n = 0$, and $s = c_{n-1}$. Overflow (a negative-looking result) occurs exactly when $c_{n-1} = 1$, i.e. when $c_{n-1} \neq c_n$.
- If $a = b = 1$ (both negative): $c_n = 1$, and $s = c_{n-1}$. Overflow (a non-negative result) occurs exactly when $c_{n-1} = 0$, again when $c_{n-1} \neq c_n$.
- If $a \neq b$: $c_n = c_{n-1}$, and overflow is impossible.

Hence

$$V = c_n \oplus c_{n-1}$$

one XOR gate on the carry chain, which is how every ALU computes the flag (Figure 11.2, and the ALU of Chapter 13). An equivalent form that software can evaluate without access to the carries is $V = (a \cdot b \cdot \overline{s}) + (\overline{a} \cdot \overline{b} \cdot s)$: same operand signs, different result sign.

> **Status of the principle: Signed overflow rule $V = c_n \oplus c_{n-1}$**
> *Derived.* A case analysis of the sign-bit full adder in two's-complement arithmetic; it holds for addition and for subtraction performed as addition of the complement, and says nothing about unsigned overflow, which is the carry c_n (or its inverse, the borrow, in subtraction).

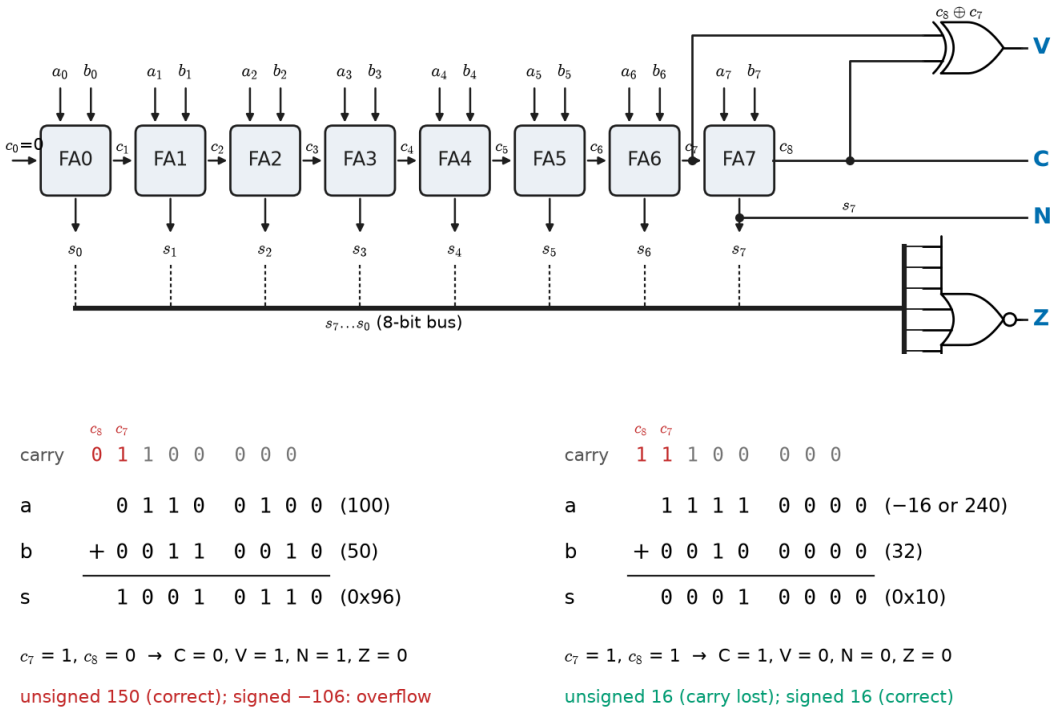


Figure 11.2 — An 8-bit ripple-carry adder with carry, overflow, negative and zero flag logic, and two example additions.

Worked example — Four 8-bit additions and their flags. Using the adder of Figure 11.2:

Operands (hex)	Result	C	V	Unsigned reading	Signed reading
0x64 + 0x32	0x96	0	1	100 + 50 = 150, correct	100 + 50 = −106, overflow
0xF0 + 0x20	0x10	1	0	240 + 32 = 16, carry lost	−16 + 32 = 16, correct
0x80 + 0x80	0x00	1	1	128 + 128 = 0, carry lost	−128 + (−128) = 0, overflow
0x7F + 0x01	0x80	0	1	127 + 1 = 128, correct	127 + 1 = −128, overflow

The same eight bits are right in one interpretation and wrong in the other; only the program knows which interpretation it meant. The last line is the most common overflow in practice: a signed counter or integrator one step past its maximum jumps to its most negative value.

Containing overflow

There are four engineering responses, and choosing among them is part of the design of every arithmetic path.

1. **Make it impossible** by sizing the word: a sum of M values of n bits needs $n + \lceil \log_2 M \rceil$ bits, a product of n- and m-bit values $n + m$ bits. This is the guard-bit argument that Book 3 §21.5 applies to DSP accumulators; in control firmware it means computing a 16-bit × 16-bit PI-controller product in 32 bits and accumulating the integral in 32 or 64 bits.
2. **Saturate**: clamp the result to the largest or smallest representable value. Saturation keeps the sign right and limits the error, the correct behaviour for physical quantities such as a PWM duty, a controller output or an audio sample. Cortex-M4 and M7 provide `QADD`, `QSUB` and `SSAT` / `USAT` instructions; in portable C:

```
#include <stdint.h>

/* Signed 16-bit addition that saturates instead of wrapping. */
static inline int16_t sat_add16(int16_t a, int16_t b)
{
    int32_t s = (int32_t)a + (int32_t)b;   /* exact: cannot overflow in 32 bits */
    if (s > INT16_MAX) return INT16_MAX;
    if (s < INT16_MIN) return INT16_MIN;
    return (int16_t)s;
}
```

3. **Wrap deliberately**, where modular arithmetic is the intended behaviour: timers, sequence numbers, phase accumulators (Section 11.3).
4. **Detect and handle**: test the V or C flag (or use the compiler built-ins `__builtin_add_overflow` in GCC and Clang) and take a defined action. In C, signed integer overflow is undefined behaviour; the compiler may assume it never happens and optimise accordingly, so a check written after the fact (`if (a + b < a)`) can be deleted. Unsigned overflow is defined to wrap.

Worked example — Overflow failures in the field. The Ariane 5 failure of 1996 was case 4 done wrongly: a conversion from a 64-bit float to a 16-bit signed integer overflowed because the new launcher's trajectory produced horizontal velocities that the Ariane 4 software had never seen, and the protection that would have caught the value had been omitted for that variable to keep processor load low. Other overflows are wrap-arounds with long fuses. The GPS week number is a 10-bit field counting weeks since January 1980; it wraps every 1024×7 days $= 19.6$ years, and receivers that did not handle the rollovers of August 1999 and April 2019 reported dates two decades in the past. A signed 32-bit count of seconds since 1970 (`time_t` on older Unix systems) reaches $2^{31} - 1$ on 19 January 2038, about 68 years after its epoch; embedded systems with 30-year service lives, such as utility meters and industrial controllers installed today, must use 64-bit time. Each of these is a sizing decision made under the belief that the range would never be reached.

11.6 Fixed-point arithmetic

Integers with an implied scale

A fixed-point number is an integer k with an agreed scale factor: the value is $k \times 2^{-f}$, where f is the number of fractional bits. The binary point exists only in the programmer's head and in the documentation; the hardware sees integers and uses the ordinary integer adder and multiplier. In the **Q**m**.**n notation (sign excluded from m in the convention of Book

3), a 16-bit Q15 number covers $[-1, 1 - 2^{-15}]$ in steps of 2^{-15}, and a 32-bit number with 16 fractional bits (Q15.16 in this convention, widely called Q16.16) covers $[-32\,768,\ 32\,768 - 2^{-16}]$ in steps of 1.53×10^{-5}. Fixed point gives **constant absolute resolution** over its whole range, the property that matches ADC samples, which are themselves fixed-point numbers.

> **Boundary.**
> Q15 and Q31 as DSP formats, guard bits, rounding modes and the noise produced by fixed-point roundoff in filters are developed in *Circuits, Signals and Communications*, Ch. 21 §21.5–21.6. This section covers fixed point as a hardware representation: scaling physical quantities into words, the word growth of each operation and the cost of getting the scaling wrong.

The rules of word growth

Each operation has a fixed effect on the format:

- **Addition and subtraction** require the same scale on both operands (align the binary points by shifting) and grow the result by one integer bit.
- **Multiplication** of $Qm_1.n_1$ by $Qm_2.n_2$ gives $Q(m_1 + m_2 + 1).(n_1 + n_2)$ in a double-width word (with this book's sign convention, the product of two signed numbers has one redundant sign bit except for $(-1) \times (-1)$). The product of two Q15 numbers is a Q30 number in 32 bits; it is returned to Q15 by an arithmetic right shift of 15, preferably with rounding (add 2^{14} first).
- **Division** shrinks fractional bits and is avoided, as in Section 11.4.
- **Conversion** between formats is a shift; between a physical quantity and its fixed-point code, a multiplication by a scale factor fixed at design time.

Getting the scale right is a design task, done once per signal and written down: which physical value corresponds to full scale, how many integer bits are needed for the worst-case transient, and what resolution the application needs.

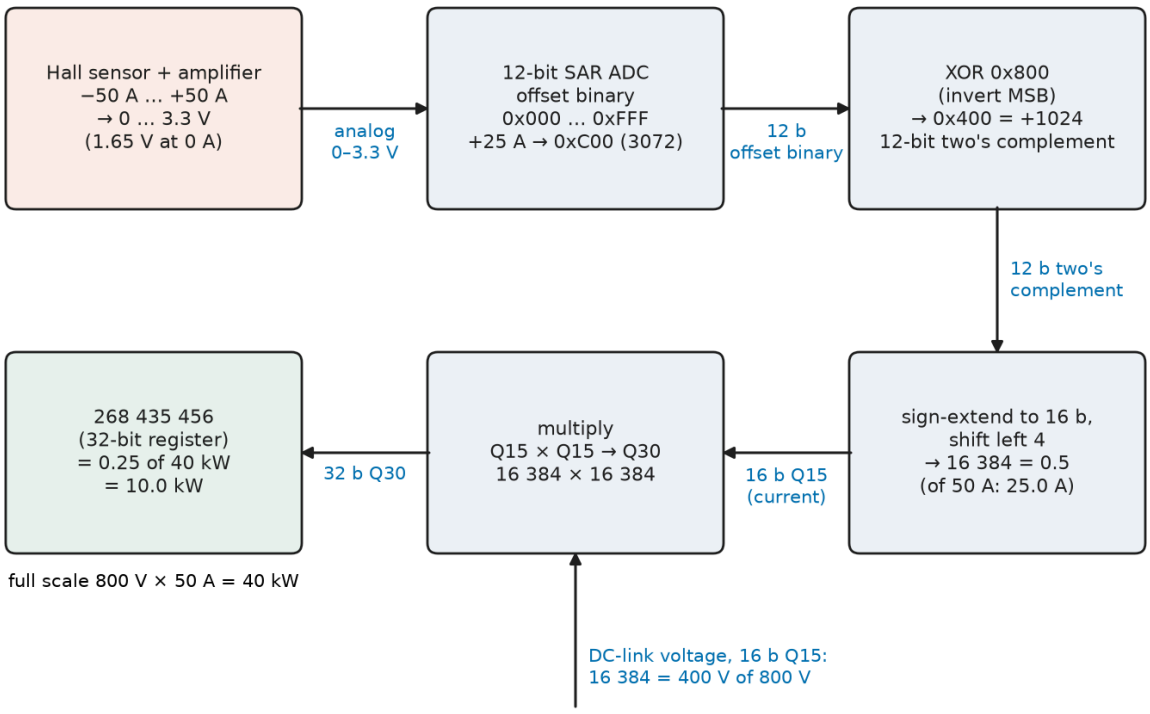


Figure 11.3 — Fixed-point data path from a bipolar current sensor through a 12-bit ADC to Q15 current and power values.

Worked example — Motor current and power in fixed point. An EV auxiliary inverter measures phase current with a Hall sensor whose amplifier maps -50 A to $+50$ A onto 0 to 3.3 V, read by a 12-bit SAR ADC (Figure 11.3). The ADC output is offset binary: code 2048 at 0 A, one LSB $= 100\ \text{A}/4096 = 24.4$ mA.

1. *ADC code to signed integer.* A reading of 0xC00 $= 3072$ becomes, after XOR with 0x800, 0x400 $= +1024$ as a 12-bit two's-complement number, $1024 \times 24.4\ \text{mA} = +25.0$ A.
2. *Signed integer to Q15.* Sign-extending to 16 bits and shifting left by 4 gives 16 384, which in Q15 is 0.5 of full scale, $0.5 \times 50 = 25.0$ A. The low four bits are zero: the ADC's resolution, not the word, limits the precision. The Q15 LSB is $50\ \text{A}/32\,768 = 1.53$ mA.
3. *Power.* The DC-link voltage is held in Q15 with 800 V full scale; 400 V is 16 384. The product $16\,384 \times 16\,384 = 268\,435\,456$ is Q30 in a 32-bit register: $268\,435\,456/2^{30} = 0.25$ of the product's full scale, $800\ \text{V} \times 50\ \text{A} = 40$ kW, so $P = 10.0$ kW, which agrees with 400×25.

No floating point was needed, every step is one or two single-cycle instructions on any 32-bit MCU, and the scales are fixed in a design document rather than recomputed at run time. The price is discipline: a later change of sensor range from ±50 A to ±100 A silently halves every computed current unless every scale is updated together.

The Patriot clock revisited

The Patriot weapon-control computer held 1/10 as a 24-bit fixed-point fraction with 23 bits after the binary point, by chopping (truncating) the expansion of Section 11.1:

$$0.1 \approx 0.00011001100110011001100_2, \qquad \varepsilon = 0.1 - \frac{\lfloor 0.1 \times 2^{23} \rfloor}{2^{23}} = 9.54 \times 10^{-8}$$

Every tick of the tenth-of-a-second counter was converted with that constant, so the error in the computed time grew linearly with the count: after 100 hours, 3.6×10^6 ticks, it reached $3.6 \times 10^6 \times 9.54 \times 10^{-8} = 0.343$ s. A Scud travels at about 1676 m/s, so the predicted position was off by about 575 m, outside the range gate. Part of the software had been updated to use a more precise conversion and part had not, so two time values that should have cancelled no longer did. The lessons are representation lessons: decimal constants are inexact in binary; truncation is biased and its error accumulates linearly; and a long-running system must either keep time as an exact integer count of ticks or bound its running time.

> **Status of the principle: Fixed-point resolution and range**
> *Derived.* A B-bit word with f fractional bits has a constant absolute resolution 2^{-f} and a range of 2^{B-f}; every operation's effect on the integer and fractional bit counts follows from integer arithmetic, so overflow and resolution can be guaranteed at design time from the signal ranges alone.

11.7 Floating-point representation

The format

> **Boundary.**
> The IEEE 754 encoding (sign, biased exponent, hidden bit, subnormals, infinities and NaNs), machine epsilon and the standard model of rounding error are set out in *The Mathematics Universe*, Ch. 22 §22.1; floating point as a DSP arithmetic, its relative-error behaviour and catastrophic cancellation in *Circuits, Signals and Communications*, Ch. 21 §21.5–21.6. This section is about the formats as hardware: what fields an FPU decodes, which formats processors and accelerators implement, and what conversion and exceptions do in a real system.

A floating-point number is a sign, an exponent and a significand: $x = (-1)^s \times 1.f \times 2^{E-\text{bias}}$, with the exponent field E stored in offset binary and the leading 1 of the significand implied. IEEE 754 (first edition 1985, current edition IEEE

754-2019) fixes the field widths, the encodings of zero, subnormals, infinities and NaNs, and — the property that makes results portable — requires the basic operations to be **correctly rounded**: the result is the exact result rounded according to the current rounding mode (round to nearest, ties to even, by default; also towards zero, $+\infty$ and $-\infty$). The listing decodes the fields of a few values the way an FPU's unpacking stage does.

```
 import struct

FORMATS = {"binary16": (">e", ">H", 5, 10), "binary32": (">f", ">I", 8, 23)}

def fields(x, fmt="binary32"):
    """Return (hex word, sign, biased exponent, fraction, class) of x."""
    ffmt, ifmt, ebits, fbits = FORMATS[fmt]
    w = struct.unpack(ifmt, struct.pack(ffmt, x))[0]
    s = w >> (ebits + fbits)
    e = (w >> fbits) & ((1 << ebits) - 1)
    f = w & ((1 << fbits) - 1)
    if e == (1 << ebits) - 1:
        cls = "NaN" if f else "infinity"
    elif e == 0:
        cls = "subnormal" if f else "zero"
    else:
        cls = "normal"
    return hex(w), s, e, hex(f), cls

for x in (3.3, -118.625, 0.1, 1e-40, float("inf")):
    print(f"{x!r:>10} binary32 {fields(x)}")
print(f"{3.3!r:>10} binary16 {fields(3.3, 'binary16')}")
print("stored binary32 value of 3.3:", struct.unpack(">f", struct.pack(">f", 3.3))[0])
```

The output shows 3.3 as `0x40533333` (biased exponent 128, i.e. 2^{1}), -118.625 as `0xC2ED4000` (exact, because $118.625 = 1110110.101_{2}$ terminates), 0.1 as `0x3DCCCCCD` (rounded up in its last bit), 10^{-40} as a subnormal with exponent field 0, infinity as `0x7F800000`, and 3.3 in binary16 as `0x429A`, which is really 3.30078. The binary32 value actually stored for 3.3 V is 3.2999999523 V: harmless in a voltage display, but enough to make an equality test `if (v == 3.3f)` in firmware fail when `v` was computed rather than assigned.

Formats in hardware

Table 11.4 — Floating-point formats implemented in hardware (IEEE 754-2019 binary formats plus industry formats for machine learning).

Format	Sign/exponent/fraction bits	Bias	Largest finite value	Relative step 2^{-p+1}	Where implemented
binary64 (double)	1/11/52	1023	1.8×10^{308}	2.2×10^{-16}	Server and desktop CPUs, Cortex-M7 (optional), Cortex-A, scientific GPUs
binary32 (single)	1/8/23	127	3.4×10^{38}	1.2×10^{-7}	Cortex-M4F/M33/M7 FPUs, DSPs (SHARC, C67x), all GPUs
binary16 (half)	1/5/10	15	65 504	9.8×10^{-4}	GPUs, Arm v8.2-A, image and audio pipelines, AI inference
bfloat16	1/8/7	127	3.4×10^{38}	7.8×10^{-3}	AI training accelerators, recent CPUs and GPUs
FP8 E4M3	1/4/3	7	448	0.125	AI accelerators (OCP 8-bit floating-point specification)
FP8 E5M2	1/5/2	15	57 344	0.25	AI accelerators (gradients)

The table shows the trade between **range** (exponent bits) and **precision** (fraction bits) at each width. bfloat16 is binary32 with the bottom 16 fraction bits dropped: it keeps binary32's range, so a neural network trained in binary32 can be converted by truncating words, and its 8-bit significand is enough for weights and activations; binary16 has three more bits of precision but overflows at 65 504, which large activations and gradients exceed. FP8 formats push the same idea to inference and training on accelerators that process tens of thousands of multiply-accumulates per cycle, where halving the word halves memory traffic and roughly quarters multiplier area. Figure 11.4 shows how the spacing of representable values grows with magnitude in four of these formats.

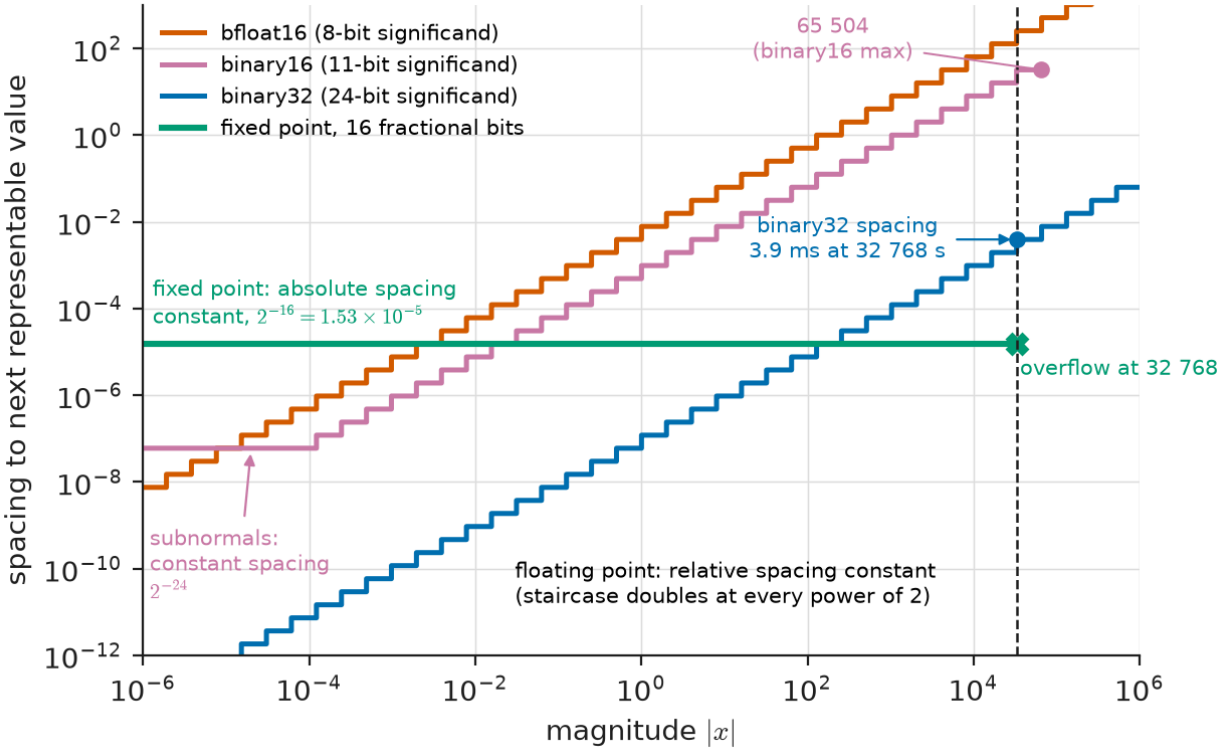


Figure 11.4 — Spacing of representable values (ulp) versus magnitude for binary16, bfloat16, binary32 and a 32-bit fixed-point format with 16 fractional bits.

What an FPU does and costs

An FPU unpacks the fields, aligns significands by shifting the smaller operand right by the exponent difference, adds or multiplies the significands, normalises the result, rounds it and repacks it, setting exception flags (invalid operation, division by zero, overflow, underflow, inexact) in a status register — FPSCR on Arm. On a Cortex-M4F a binary32 add or multiply takes one cycle, a fused multiply-accumulate three, and a divide or square root 14 cycles; binary64 operations on the same core run in a software library at tens of cycles each, a fact that a single unsuffixed constant in C (`x * 0.5` instead of `x * 0.5f`) silently triggers by promoting the expression to double. Subnormal arithmetic is slow on many cores (handled by microcode or traps); embedded and audio code commonly enables **flush-to-zero** mode, which replaces subnormal results by zero at the cost of IEEE conformance near underflow.

The most dangerous operation is **conversion to integer**. When a floating-point value is out of range for the target integer, IEEE 754 signals invalid operation; Arm's `VCVT` instruction then saturates to the largest or smallest integer, x86's `CVTTSS2SI` returns the "integer indefinite" value -2^{31}, Ada (the language of the Ariane 5 software) raised an exception, and C declares the behaviour undefined. Firmware that converts a computed float into a PWM compare register, an array index or a 16-bit DAC code must clamp it first.

Worked example — A binary32 timestamp in a data logger. A vibration logger on a wind turbine keeps elapsed time in a binary32 variable, adding 1 ms (`0.001f`) at every sample. Near t the spacing of binary32 values is $\mathrm{ulp}(t) = 2^{\lfloor \log_2 t \rfloor - 23}$, and each addition rounds $t + 0.001$ to the nearest representable value. The listing shows what one tick actually adds at several elapsed times.

```
 import numpy as np

dt = np.float32(0.001)                      # 1 ms tick
for t in (1.0, 100.0, 3600.0, 16384.0, 32768.0):
    t32 = np.float32(t)
    ulp = np.spacing(t32)                   # gap to the next binary32 value
    step = float((t32 + dt) - t32)          # what one tick actually adds
    print(f"t = {t:8.0f} s  ulp = {ulp * 1e3:9.5f} ms  tick adds {step * 1e3:7.4f} ms")
```

At 1 s the tick adds 1.0000 ms; at 3600 s, where the spacing is 0.244 ms, it adds 0.9766 ms, 2.3 % short; between 16 384 s and 32 768 s, where the spacing is 1.953 ms, every tick rounds up to a full spacing and the clock runs at almost twice real time; at 32 768 s the spacing is 3.906 ms, a 1 ms tick is less than half of it, and time stops. Simulating the full accumulation from zero, the variable reads 3530.2 s after one real hour (3.6 million ticks), and it freezes at 32 768 s after 25 151 s (just under 7 hours) of real time. The fix is a representation choice, not more precision: count ticks in a 64-bit integer, which is exact for 584 million years at 1 ms, and convert to floating point only for display or for computing differences. Figure 11.4 marks the point where the binary32 spacing reaches 3.9 ms.

> **Status of the principle: Floating-point spacing**
> *Derived.* With p significand bits, the representable values between 2^e and 2^{e+1} are equally spaced by 2^{e-p+1}, so the absolute spacing doubles at each power of two while the relative spacing stays between 2^{-p} and 2^{-p+1}; this holds for normal numbers of every IEEE 754 binary format and fails in the subnormal range, where the spacing is constant.

Choosing a representation

The choice between integer, fixed point and floating point is made per signal and per processor:

- **Integer** for counts, indices, time in ticks, addresses and anything that must be exact.
- **Fixed point** where the range is known, resolution must be uniform, and the hardware has no FPU or the datapath is in an FPGA or ASIC where every bit costs area: motor control on Cortex-M0+, power meters, hearing aids, FPGA filters.
- **binary32** where dynamic range is large or unknown and an FPU exists: control loops on Cortex-M4F and M7, sensor fusion, audio effects.

- **binary64** for accumulations over long times or many terms, for geometry and navigation (a binary32 latitude in degrees near 90° is quantised in steps of 7.6×10^{-6}°, about 0.85 m) and for scientific computation.
- **binary16, bfloat16, FP8** for storage and throughput where error is tolerated statistically, as in neural-network inference (Chapter 35).

Decimal floating-point formats (decimal64 and decimal128 in IEEE 754-2019), which represent 0.1 exactly, exist for financial calculation and are implemented in hardware on some mainframe processors; engineering hardware almost never needs them.

11.8 Boolean variables

One bit, two meanings

The smallest number system has two values. A **Boolean variable** takes the values 0 and 1, interpreted as false and true; Chapter 12 develops the algebra of such variables and the gates that compute with them, and *The Mathematics Universe*, Ch. 29 §29.3, the logic behind it. In hardware a Boolean variable is one wire or one flip-flop. Its electrical meaning is a convention: in **active-high** (positive) logic a high voltage means true; in **active-low** logic a low voltage means true, and the signal name carries a marker — $\overline{\text{RESET}}$, `nCS`, `CS_N`, `/IRQ`. Active-low signals are common because open-drain outputs can only pull low (so several devices can share one interrupt or reset line, any of them asserting it), because a reset line held low by a pull-down or a discharged capacitor during power-up is safe by default, and for historical reasons in TTL, whose inputs float high. Confusing the two conventions inverts a function, so a schematic or HDL module must state the polarity of every control signal.

Flags, bit fields and registers

A processor's natural unit is a word, so Boolean variables are packed. A **status register** collects flags such as N, Z, C and V (Section 11.5); a peripheral's **control register** packs enable bits, mode fields of several bits and interrupt enables into one 32-bit word (Figure 11.5). Firmware manipulates individual bits with masks:

- set bit k: `reg |= (1u << k)`;
- clear bit k: `reg &= ~(1u << k)`;
- toggle bit k: `reg ^= (1u << k)`;
- test bit k: `(reg >> k) & 1u`;
- write a field of width w at position p: `reg = (reg & ~(mask << p)) | ((value & mask) << p)` with `mask = (1u << w) - 1`.

The read-modify-write sequence behind `|=` is three instructions and is not atomic: an interrupt that changes another bit of the same register between the read and the write loses its change. Microcontrollers provide two remedies in hardware: **set/reset registers**, such as the STM32's GPIO bit set/reset register (BSRR), in which writing a 1 to bit k sets output k and writing a 1 to bit $k + 16$ clears it while zeros leave other pins alone; and **bit-banding** on Cortex-M3/M4, which maps every bit of a region onto its own word address.

```
#include <stdint.h>

/* STM32F4 GPIOA registers (addresses from the reference manual; illustrative). */
#define GPIOA_IDR   (*(volatile uint32_t *)0x40020010u)  /* input data      */
#define GPIOA_BSRR  (*(volatile uint32_t *)0x40020018u)  /* bit set/reset   */

#define LED_PIN     5u
#define BUTTON_PIN  13u

static inline void led_on(void)  { GPIOA_BSRR = 1u << LED_PIN; }          /* atomic set   */
static inline void led_off(void) { GPIOA_BSRR = 1u << (LED_PIN + 16u); }  /* atomic reset */

static inline int button_pressed(void)
{
    return ((GPIOA_IDR >> BUTTON_PIN) & 1u) == 0u;   /* active-low input */
}
```

The `volatile` qualifier tells the compiler that every access must actually happen, because the register can change, or must be written, independently of the program. The button function shows active-low logic in software: a pressed button pulls the pin to ground, so "pressed" is the Boolean value true when the bit is 0.

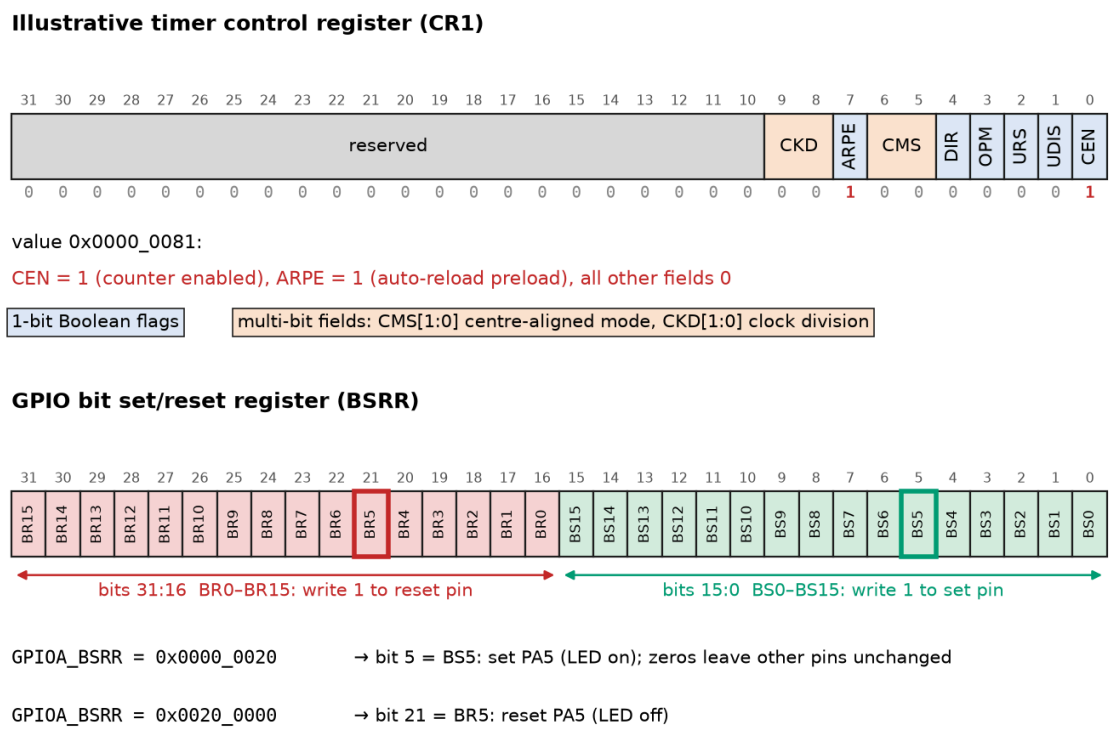


Figure 11.5 — A 32-bit peripheral control register as a set of Boolean flags and multi-bit fields, and the BSRR set/reset layout.

Boolean variables in languages and memory

C has had `_Bool` since C99 (spelled `bool` with `<stdbool.h>`, and a keyword in C23); C++ has `bool`. Each occupies at least one byte, because a byte is the smallest addressable unit, so an array of 1000 flags costs 1000 bytes unless packed into a bit array of 125 bytes, a trade between memory and the masking instructions needed for every access. Any non-zero integer converts to `true`. In hardware description languages a single-bit signal (`logic` in SystemVerilog, `std_logic` in VHDL) has more than two values in simulation — unknown (X) and high-impedance (Z) at least, nine values in VHDL — so that a simulator can show an uninitialised flip-flop or a floating bus, states that two-valued Boolean algebra cannot represent (Chapter 19).

> **Status of the principle: Two-valued Boolean variable**
> *Fundamental.* By definition a Boolean variable takes exactly two values, and every digital representation in this chapter is a vector of them; the physical signal that carries it is two-valued only outside the transition times and inside the noise margins of the logic family, which is where the digital abstraction ends (Chapters 12 and 14).

What remains open

Open question	Where it is answered
How are Boolean variables combined, simplified and built into gates?	Chapter 12
How are adders, subtractors, comparators, ALUs and code converters (Gray, BCD) built and how fast are they?	Chapter 13
How are numbers stored in registers and counted by counters?	Chapter 14
How do instruction sets, ALUs and status flags appear to a programmer?	Chapter 16, Chapter 17
How are fixed-point and floating-point arithmetic written in HDL and mapped to FPGA DSP blocks?	Chapter 18, Chapter 19
How do C data types, integer promotion and bitwise operators map onto these representations?	Chapter 21
How does rounding error accumulate in long numerical computations?	Chapter 24; *The Mathematics Universe*, Ch. 22
How does finite word length limit digital filters and FFTs?	*Circuits, Signals and Communications*, Ch. 21
How are low-precision formats exploited in AI accelerators?	Chapter 35

Next: Chapter 12 takes the single bit of Section 11.8 and builds the algebra and the gates that compute with it.

Chapter 12 — Boolean Algebra and Logic Gates

Part III · Digital Electronics

An electric-vehicle battery pack connects to its 400 V DC link through three switching elements: a main negative contactor, a precharge relay with a series resistor that charges the inverter's capacitors gently, and a main positive contactor that closes only when the capacitors are within a few percent of pack voltage. Close the main positive contactor too early and an inrush current of thousands of amperes welds its contacts; leave the precharge relay closed and its resistor burns. The rules are simple to say — close nothing unless the driver requests it, the high-voltage interlock loop is intact and no fault is present; precharge first; then switch over — and the battery-management controller enforces them in software. But a certification engineer reviewing the design asks for an independent hardware path that blocks the contactors whatever the software does, built from a handful of gates whose behaviour can be proved correct for every combination of inputs. To build it, the rules must become equations, the equations must be simplified, and the simplified equations must become gates with known voltage levels and delays. That is the subject of this chapter: Boolean algebra used as an engineering tool for designing logic.

> **How are requirements turned into Boolean equations and truth tables, and how are those equations simplified into the smallest or fastest gate network?**
> **What does a real logic gate guarantee — which functions, which voltage levels and noise margins, which delays — and where does the two-valued abstraction stop?**

Concept	Engineering destination
Boolean algebra	Writing and transforming interlock, decoding and control equations
Logic gates	Glue logic, safety interlocks, level translation, standard cells and FPGA LUTs
Truth tables	Specifications, exhaustive verification, test vectors, LUT contents
Boolean simplification	Fewer gates, fewer transistors, shorter critical paths
Karnaugh maps	Hand minimisation of small functions with don't-cares; hazard checks
De Morgan's laws	NAND/NOR implementations, active-low signals, bubble pushing in schematics
Combinational logic	Decoders, voters, interlocks, arithmetic blocks (Chapter 13)
Logic minimization	The algorithms inside synthesis tools for FPGAs and ASICs

Roadmap. The algebra as a set of rules for rewriting equations (12.1) → the gates that implement it and the electrical levels that make them reliable (12.2) → the truth table as specification (12.3) → De Morgan's laws and active-low design (12.4) → algebraic simplification and cost (12.5) → Karnaugh maps (12.6) → the combinational design procedure, worked on the contactor interlock (12.7) → algorithmic minimization as performed by synthesis tools (12.8).

12.1 Boolean algebra

> **Boundary.**
> Propositional logic, the laws of Boolean algebra as identities, the count of 2^{2^n} functions of n variables and the proof that every Boolean function has a sum-of-products formula (so that NAND alone is functionally complete) are in *The Mathematics Universe*, Ch. 29 §29.3. This chapter uses those results as design tools. The transistor-level construction of gates is in Chapter 33; binary numbers and the Boolean variable as a bit in Chapter 11.

Variables, operators and notation

A Boolean variable takes the values 0 and 1 (Section 11.8). Three operators generate the algebra: complement (NOT, $\overline{A}$), product (AND, $A \cdot B$ or AB) and sum (OR, $A + B$). By convention AND binds tighter than OR, so $AB + C$ means $(A \cdot B) + C$. Exclusive OR, $A \oplus B = A\overline{B} + \overline{A}B$, is 1 when its inputs differ and is the addition of bits modulo 2; its complement, XNOR, is the equality test. An expression written as an OR of AND terms is a **sum of products** (SOP); as an AND of OR terms, a **product of sums** (POS). Each AND term of an SOP is an **implicant** of the function, and each variable appearance, complemented or not, is a **literal**.

The laws as rewriting rules

Engineers use the laws of Table 12.1 as rewriting rules, in both directions: to shorten an equation (fewer gates), to factor it (fewer levels or less fan-in), or to put it into a form that matches the available gates.

Table 12.1 — Laws of Boolean algebra and their duals.

Law	Form	Dual form
Identity	$A + 0 = A$	$A \cdot 1 = A$
Null (dominance)	$A + 1 = 1$	$A \cdot 0 = 0$
Idempotence	$A + A = A$	$A \cdot A = A$
Complement	$A + \overline{A} = 1$	$A \cdot \overline{A} = 0$
Involution	$\overline{\overline{A}} = A$	—
Commutative	$A + B = B + A$	$AB = BA$
Associative	$(A + B) + C = A + (B + C)$	$(AB)C = A(BC)$
Distributive	$A(B + C) = AB + AC$	$A + BC = (A + B)(A + C)$
Absorption	$A + AB = A$	$A(A + B) = A$
Simplification	$A + \overline{A}B = A + B$	$A(\overline{A} + B) = AB$
Adjacency (combining)	$AB + A\overline{B} = A$	$(A + B)(A + \overline{B}) = A$
Consensus	$AB + \overline{A}C + BC = AB + \overline{A}C$	$(A + B)(\overline{A} + C)(B + C) = (A + B)(\overline{A} + C)$
De Morgan	$\overline{A + B} = \overline{A}\,\overline{B}$	$\overline{AB} = \overline{A} + \overline{B}$

Two features distinguish this algebra from ordinary arithmetic. OR distributes over AND ($A + BC = (A + B)(A + C)$), which has no counterpart for numbers. And every law has a **dual**, obtained by swapping AND with OR and 0 with 1; proving one form proves the other. The **adjacency** law is the engine of all minimisation: two product terms that differ in exactly one variable, complemented in one and not in the other, merge into one term without that variable. Karnaugh maps (Section 12.6) and the Quine–McCluskey algorithm (Section 12.8) are two systematic ways of finding every such merge.

The **consensus** term BC in $AB + \overline{A}C + BC$ is logically redundant, but not always useless: Section 12.6 shows that it suppresses a glitch when A changes, which is why a designer may deliberately keep a term the algebra says can be removed.

> **Status of the principle: Boolean algebra**
> *Fundamental.* The algebra of two-valued logic, fixed by its postulates (Huntington's set: closure, identities, commutativity, distributivity, complements); every law of Table 12.1 follows from them, and any two expressions are equal exactly when they agree on all 2^n input combinations.

Positive and negative logic

Boolean algebra knows nothing about voltages. A gate whose output is high only when both inputs are high is an AND gate in **positive logic** (high = 1). Read in **negative logic** (low = 1), the same circuit outputs 1 when either input is 1: it is an OR gate. The duality of Table 12.1 is the algebraic statement of this fact. Real designs mix the two conventions, with active-low signals such as $\overline{\text{CS}}$, $\overline{\text{RESET}}$ and $\overline{\text{IRQ}}$ (Section 11.8) alongside active-high data, and Section 12.4 shows how to keep the algebra straight.

12.2 Logic gates

The gate set

A **logic gate** is a circuit whose output, after a short delay, is a fixed Boolean function of its present inputs. Figure 12.1 shows the standard gates in the two symbol sets of IEEE Std 91-1984 and IEC 60617-12: the **distinctive-shape** symbols used in most schematics, and the **rectangular** symbols with a qualifying character (& for AND, ≥1 for OR, =1 for XOR, 1 for a buffer) found in European and industrial documentation and on PLC function-block diagrams. A small circle (bubble) on an input or output denotes inversion.

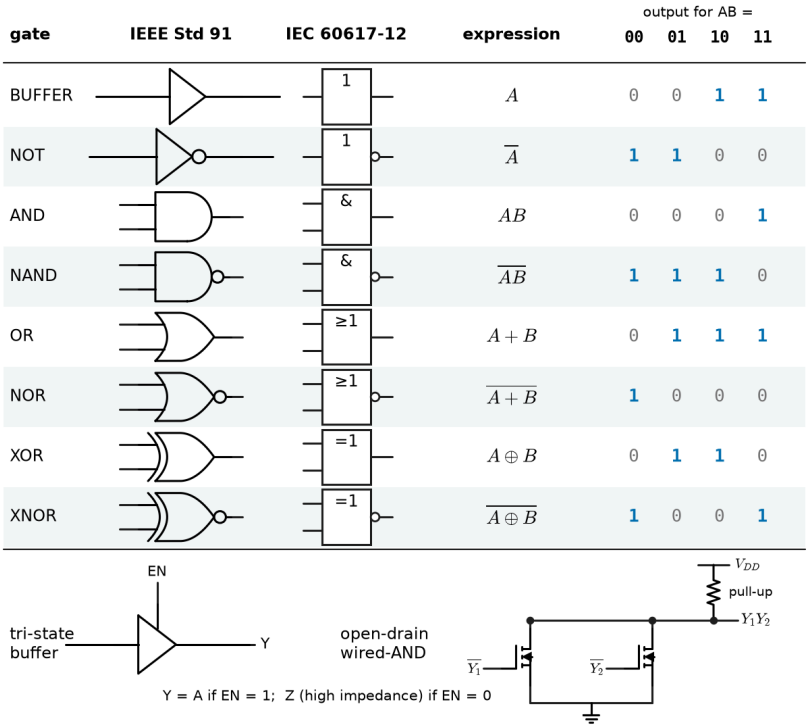


Figure 12.1 — Standard logic gates in distinctive-shape and IEC rectangular symbols with their two-input truth tables.

Some gates matter for reasons beyond their function:

- **NAND and NOR** are each functionally complete: any Boolean function can be built from NAND gates alone, or NOR gates alone. In CMOS they are also the cheapest gates, four transistors for a two-input version against six for AND or OR, which are a NAND or NOR followed by an inverter (Chapter 33). CMOS designs are therefore NAND/NOR-centric, and De Morgan's laws (Section 12.4) translate AND-OR equations into them.
- **XOR and XNOR** compute parity, comparison and the sum bit of addition. They are costly in CMOS (8 to 12 transistors) and do not simplify on Karnaugh maps, so arithmetic and error-detection logic is designed with them explicitly.
- **Buffers** restore signal levels and drive large loads: clock trees, long PCB traces, LED arrays.

- **Tri-state buffers** add a third output condition, high impedance (Z), so that several drivers can share a bus line with only one enabled at a time; two enabled drivers fighting each other short the supply through their output transistors.
- **Open-drain (open-collector) outputs** can only pull low; a pull-up resistor supplies the high level. Several such outputs on one line form a **wired-AND** (the line is high only if no output pulls it low), which is how I²C, interrupt and fault lines are shared, and the pull-up can go to a different supply, giving free level translation.
- **Schmitt-trigger inputs** have two thresholds separated by a hysteresis of a few hundred millivolts, so that slow or noisy edges from RC networks, switches and long cables produce one clean transition instead of a burst (Chapter 8 treats the circuit).

Gates are bought as **discrete logic** (the 74-series families: the 74HC00 quad two-input NAND, or single-gate parts such as the 74LVC1G08 in a 1.5 mm package), used inside **programmable logic** (CPLD macrocells, FPGA look-up tables, Chapter 18), or placed as **standard cells** in an ASIC (Chapter 34). Discrete gates remain the right choice for a few functions that must work before or without software: power sequencing, reset generation, watchdog and safety interlocks, and glue between chips from different voltage domains.

Logic levels and noise margins

A gate turns a range of input voltages into a Boolean value. Its datasheet guarantees four numbers (Figure 12.2): any input above V_{IH} is read as 1 and any input below V_{IL} as 0; the output is above V_{OH} when it drives 1 and below V_{OL} when it drives 0, at a stated load current. The **noise margins** are the voltage a noise source may add to a valid output before the next input misreads it:

$$NM_H = V_{OH} - V_{IH}, \qquad NM_L = V_{IL} - V_{OL}$$

Inputs between V_{IL} and V_{IH} are undefined: the gate may read either value, may oscillate, and in CMOS draws a large through-current when held there, which is why unused CMOS inputs must never float.

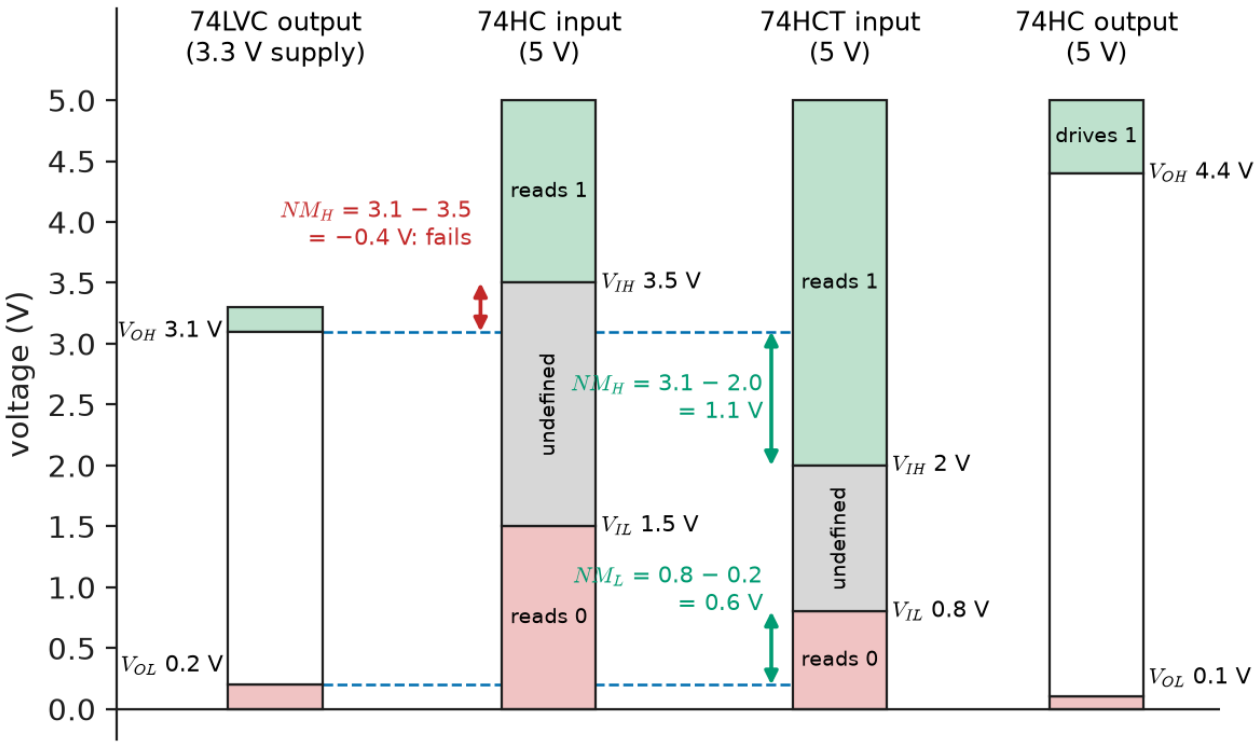


Figure 12.2 — Input and output voltage levels and noise margins of 5 V HC, 5 V HCT and 3.3 V LVC logic, and the failed 3.3 V to 5 V HC interface.

Table 12.2 — Representative logic families (typical datasheet limits at the stated supply; output levels at small load current).

Family	V_{CC}	V_{IH} min	V_{IL} max	V_{OH} min	V_{OL} max	NM_H	NM_L	t_{pd} typical	Output drive
74LS (bipolar TTL)	5 V	2.0 V	0.8 V	2.7 V	0.5 V	0.7 V	0.3 V	9 ns	0.4 / 8 mA
74HC (CMOS)	5 V	3.5 V	1.5 V	4.4 V	0.1 V	0.9 V	1.4 V	8 ns	±5.2 mA
74HCT (CMOS, TTL inputs)	5 V	2.0 V	0.8 V	4.4 V	0.1 V	2.4 V	0.7 V	10 ns	±5.2 mA
74LVC (CMOS, 5 V tolerant)	3.3 V	2.0 V	0.8 V	3.1 V	0.2 V	1.1 V	0.6 V	3 ns	±24 mA
74AUP (low power)	1.8 V	1.17 V	0.63 V	1.7 V	0.1 V	0.53 V	0.53 V	4 ns	±1.9 mA

The 3.3 V levels follow the LVTTL/LVCMOS interface standard JEDEC JESD8C; the 1.8 V levels ($V_{IH} = 0.65V_{CC}$, $V_{IL} = 0.35V_{CC}$) follow JESD8-7. Margins of several hundred millivolts look generous until they are compared with what a fast edge does on a board: ground bounce when many outputs switch together, crosstalk from a neighbouring trace, and ringing on an unterminated line (*Circuits, Signals and Communications*, Ch. 14) can each consume hundreds of millivolts.

Worked example — Can a 3.3 V microcontroller drive 5 V logic? A 3.3 V MCU (output stage comparable to 74LVC, $V_{OH} \geq 3.1$ V at light load) must drive the enable input of a 5 V relay-driver board built with 74HC logic. Against the 74HC input, $NM_H = 3.1 - 3.5 = -0.4$ V: negative. A high output lies inside the undefined band; the board may work on the bench and fail on a warm day or with a lower supply. Replacing the input gate with a 74HCT part (TTL-compatible thresholds) gives $NM_H = 3.1 - 2.0 = 1.1$ V and $NM_L = 0.8 - 0.2 = 0.6$ V, both comfortable, and the 74HCT output still swings to 4.4 V for the rest of the 5 V board. In the other direction, a 5 V HC output into a 3.3 V MCU input exceeds the input's absolute maximum rating (typically $V_{DD} + 0.3$ V or +0.5 V) unless the pin is specified 5 V tolerant; the input protection diode then conducts into the 3.3 V rail. The remedies are a 5 V tolerant buffer (74LVC inputs accept 5.5 V at a 3.3 V supply), a dual-supply translator (74LVC1T45), or an open-drain output with a pull-up to 3.3 V.

> **Status of the principle: The digital abstraction (logic levels and noise margins)**
> *Approximate.* A signal is treated as a Boolean value only while it lies outside the undefined band and after transitions have settled; the abstraction fails during transitions, with negative noise margins, with inputs left floating, and when a bistable element samples a value inside the band (metastability, Chapter 14).

Delay, fan-out and power in one paragraph

A gate's output follows its inputs after a **propagation delay** t_{pd}, a few nanoseconds for discrete CMOS and tens of picoseconds for a standard cell in an advanced node; the delay grows with load capacitance and falls with supply voltage. **Fan-out**, the number of inputs one output can drive, is limited for CMOS not by DC current (inputs draw microamps) but by the delay that the added input capacitance causes. A CMOS gate draws almost no static current and dissipates $P \approx (C_{pd} + C_L)V_{CC}^2 f$ when switching; for a 74HC gate with power-dissipation capacitance $C_{pd} = 22$ pF and a 15 pF load switching at 1 MHz from 5 V, about 0.93 mW. Delay definitions and their timing consequences are in Chapter 13 (Section 13.9); their physical origin, together with noise margins derived from the inverter transfer curve, in Chapter 33.

12.3 Truth tables

From requirement to table

A **truth table** lists the output of a function for every combination of its inputs: 2^n rows for n inputs. It is the complete and unambiguous specification of a combinational function, which makes it the natural bridge between a requirement written in words and an implementation in gates, and the natural set of test vectors for verifying that implementation.

Writing the table forces decisions that prose leaves open. For the contactor interlock of the opening, the inputs are K (driver requests high voltage, key on), I (high-voltage interlock loop closed: every HV connector mated and every service cover in place), F (fault reported by the battery-management system or the insulation monitor) and P (precharge complete: DC-link voltage above 95 % of pack voltage, from a comparator). The outputs are N (main negative contactor), R (precharge relay), M (main positive contactor) and W (warning lamp). The requirements, rewritten as Boolean statements, are:

- High voltage may be connected only if K, I and not F: call this condition $T = K I \overline{F}$.
- $N = T$: the negative contactor closes whenever connection is allowed.
- $R = T\overline{P}$: precharge while the link is not yet charged.
- $M = TP$: the main positive contactor only after precharge, which also opens the precharge relay.
- $W = F + K\overline{I}$: warn on a fault, or when the driver requests high voltage with the interlock open.

The listing writes the specification directly from these statements, builds an implementation from NAND gates only (derived in Section 12.4), and checks the two against each other on all 16 input combinations, printing the truth table.

```
 from itertools import product

def spec(K, I, F, P):
    """Contactor sequencing written directly from the requirements."""
    ok = K and I and not F                  # request, interlock closed, no fault
    return {"N": ok,                        # main negative contactor
            "R": ok and not P,              # precharge relay
            "M": ok and P,                  # main positive contactor
            "W": F or (K and not I)}        # warning lamp

def nand(*x):
    return int(not all(x))

def gates(K, I, F, P):
    """NAND-only implementation obtained with De Morgan's laws."""
    nF, nP, nI = nand(F), nand(P), nand(I)  # NAND used as an inverter
    T = nand(nand(K, I, nF))                # shared term K.I.F'
    return {"N": T, "R": nand(nand(T, nP)), "M": nand(nand(T, P)),
            "W": nand(nF, nand(K, nI))}     # F + K.I' = NAND(F', NAND(K, I'))

print(" K I F P | N R M W")
for K, I, F, P in product((0, 1), repeat=4):
    s, g = spec(K, I, F, P), gates(K, I, F, P)
    assert all(int(s[k]) == g[k] for k in s), (K, I, F, P)
    print(f" {K} {I} {F} {P} | {g['N']} {g['R']} {g['M']} {g['W']}")
print("NAND implementation matches the specification in all 16 rows")
```

Only two of the sixteen rows close anything: $KIFP = 1100$ gives $N = R = 1$ (precharging) and 1101 gives $N = M = 1$ (running). W is 1 in the eight rows with $F = 1$ and in the two further rows with $K = 1$, $I = 0$, $F = 0$. The assertion passes in all rows. For four inputs an exhaustive check is trivial; for the 64-bit operands of an adder it is impossible (2^{128} rows), which is why larger blocks are verified by formal equivalence checking instead (Chapter 19, Chapter 34).

Minterms, maxterms and canonical forms

Each row of a truth table corresponds to one **minterm**, the AND of all n variables, each complemented if it is 0 in that row; minterm m_i is 1 only in row i (row index = the input pattern read as a binary number). Any function is the OR of the minterms of the rows where it is 1, its **canonical sum of products**, written compactly as $F = \sum m(\dots)$. Dually, each row has a **maxterm** M_i, the OR of all variables complemented if they are 1 in that row, which is 0 only in row i; the AND of the maxterms where $F = 0$ is the **canonical product of sums**, $F = \prod M(\dots)$. In the contactor example, $M = KI\overline{F}P = m_{13}$, a single minterm, and $W = \sum m(2, 3, 6, 7, 8, 9, 10, 11, 14, 15)$.

Canonical forms exist for every function and are unique, so they make a reliable starting point — and are usually far from minimal: a canonical SOP of a function that is 1 in half its rows has 2^{n-1} terms of n literals each.

Don't-care conditions

Some input combinations can never occur, or their output does not matter. A BCD digit uses only codes 0 to 9 of its 16 four-bit patterns; a one-hot state register never has two bits set; an output that is ignored while an enable is off may take any value. These rows are **don't-cares**, written X or d, and listed as $\sum d(\dots)$. A minimiser may set each to 0 or 1, whichever gives the simpler result, and they are often the largest single source of simplification (Section 12.6). The engineering condition is that the "impossible" input really is impossible, or harmless: a don't-care assigned to 1 in an interlock is a safety decision, not just an optimisation.

12.4 De Morgan's laws

The laws and their use

De Morgan's laws state that the complement of a sum is the product of the complements, and vice versa:

$$\overline{A + B + C + \cdots} = \overline{A} \cdot \overline{B} \cdot \overline{C} \cdots, \qquad \overline{A \cdot B \cdot C \cdots} = \overline{A} + \overline{B} + \overline{C} + \cdots$$

In gate terms: a NAND gate is an OR gate with inverted inputs, and a NOR gate is an AND gate with inverted inputs. Both pictures describe the same transistors; which one a designer draws is a matter of making the schematic say what the circuit means.

> **Status of the principle: De Morgan's laws**
> *Derived.* Follow from the complement and distributive postulates of Boolean algebra (or by checking the four rows of a two-input truth table) and extend to any number of variables by induction; they hold for every logic family, because they concern the Boolean function, not the circuit.

Two-level NAND and NOR logic

Any SOP expression becomes a two-level NAND–NAND circuit. By involution and De Morgan,

$$F = AB + CD = \overline{\overline{AB + CD}} = \overline{\overline{AB} \cdot \overline{CD}}$$

so the AND gates become NAND gates and the OR gate becomes a NAND gate whose inputs are the NAND outputs. Dually, any POS expression becomes NOR–NOR. Since NAND is the cheapest CMOS gate, SOP-then-NAND–NAND is the classic route from a minimised equation to silicon, and it preserves the two-level delay of the SOP. The warning-lamp output is an example: $W = F + K\overline{I} = \overline{\overline{F} \cdot \overline{K\overline{I}}}$, a NAND of $\overline{F}$ and NAND$(K, \overline{I})$, as the listing of Section 12.3 implements.

Bubble pushing and active-low signals

Bubble pushing is De Morgan's law applied graphically (Figure 12.3): moving a bubble through a gate swaps AND with OR and inverts every input and output. Pairs of bubbles on the two ends of a wire cancel. The method lets a designer

convert AND–OR logic into NAND–NAND by inserting bubble pairs, and, more importantly, read mixed-polarity schematics. In **mixed-logic** convention, a gate symbol is chosen to match the active level of its signals: a NAND gate whose inputs are active-low request lines and whose output is active-high is drawn as an OR with bubbled inputs, because "either request asserted" is what it means.

Active-low signals arise naturally in memory and peripheral decoding. A chip-select that must be asserted (low) when address lines $A_{15} = 1$ and $A_{14} = 1$ and $A_{13} = 0$ is

$$\overline{\mathrm{CS}} = \overline{A_{15} A_{14} \overline{A_{13}}}$$

a single three-input NAND with one inverted input. Its output is low exactly in the 8 KiB block 0xC000–0xDFFF of a 64 KiB address space, the classic decoding job of a 74HC138 decoder or a PLD in an 8-bit system (Chapter 13, Section 13.5).

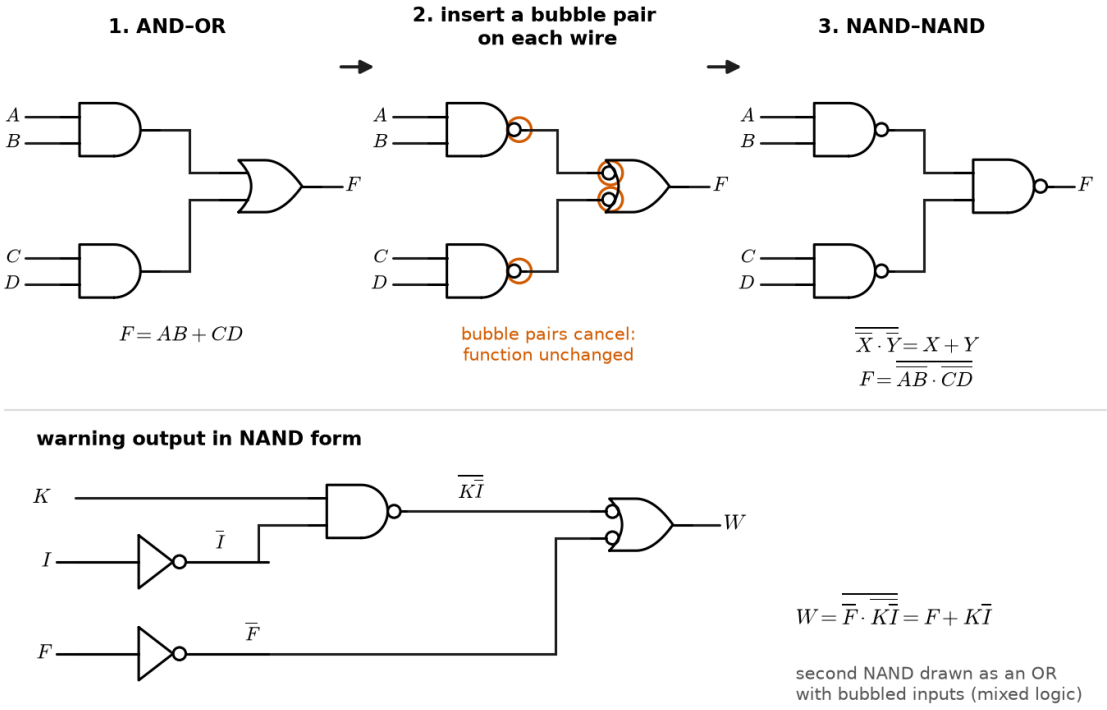


Figure 12.3 — Converting AND-OR logic to NAND-NAND by bubble pushing, and the warning-lamp output W = F + K NOT I in NAND form.

12.5 Boolean simplification

What "simpler" means

A Boolean function has infinitely many equivalent expressions; the designer wants the one that is cheapest by the measure that matters:

- **Literal count** and **term count** of a two-level SOP or POS, which set the number and fan-in of gates in a two-level implementation and the size of a PLA.
- **Transistor count** or **cell area** in CMOS: a two-input NAND is 4 transistors, an AND 6, an XOR 8 to 12, an AND-OR-INVERT (AOI21) cell 6.
- **Logic depth**, the number of gate levels on the longest path, which with the gates' delays sets the propagation delay (Chapter 13).
- In an FPGA, the number of **look-up tables** and their depth: a 6-input LUT implements any function of six inputs at the same cost, so literal counts barely matter (Section 12.8).

These measures conflict. Factoring an SOP into a multi-level expression usually saves literals and area but adds levels and delay; flattening a multi-level expression to two levels does the reverse. Synthesis tools trade one against the other under a timing constraint (Chapter 34).

Algebraic simplification

The laws of Table 12.1 simplify small expressions by inspection. Take the function that is 1 for minterms 1, 3, 5, 6 and 7 of A, B, C:

$$F = \overline{A}\,\overline{B}C + \overline{A}BC + A\overline{B}C + ABC + AB\overline{C}$$

The first four terms merge pairwise by adjacency: $\overline{A}\,\overline{B}C + \overline{A}BC = \overline{A}C$ and $A\overline{B}C + ABC = AC$, and then $\overline{A}C + AC = C$. The last term combines with ABC (reused, by idempotence): $ABC + AB\overline{C} = AB$. So $F = C + AB$: from 15 literals in five three-input AND gates and a five-input OR to 3 literals in one AND and one OR. The weakness of algebra is that nothing tells the designer which merges to try or when the result is minimal; reusing ABC was the step that made the difference. Karnaugh maps make the merges visible, and the Quine–McCluskey algorithm makes them systematic.

Worked example — Factoring for gate count versus delay. The expression $F = AC + AD + BC + BD$ (8 literals, two levels: four AND2 and one OR4) factors to $F = (A + B)(C + D)$ (4 literals: two OR2 and one AND2, still two levels). In CMOS the factored form is three NOR2 gates, $F = \overline{\overline{A + B} + \overline{C + D}}$, 12 transistors, against 24 for the SOP built as NAND–NAND (four NAND2 gates and a NAND4). Here factoring wins on every measure because the function happens to be a product of sums; in general the factored form trades area for depth, and the choice depends on whether the path is timing-critical.

12.6 Karnaugh maps

The map

A **Karnaugh map** arranges the 2^n cells of a truth table in a grid whose rows and columns are labelled in Gray-code order (00, 01, 11, 10), so that cells that are geometrically adjacent — including across the edges, which wrap around — differ in exactly one variable. The adjacency law then becomes visual: two adjacent 1s merge into one term with one variable eliminated, a block of four into a term with two variables eliminated, a block of 2^k into a term with k eliminated. The Gray code of the labels is the same one that Chapter 13 uses for encoders (Chapter 13, Section 13.8).

The procedure for a minimal SOP:

1. Fill the map with 1s, 0s and Xs (don't-cares) from the truth table.
2. Find every **prime implicant**: a rectangular block of 2^k cells containing only 1s and Xs that cannot be enlarged.
3. Select the **essential** prime implicants, those that alone cover some 1.
4. Cover the remaining 1s with as few and as large prime implicants as possible. Xs may be covered or not, as convenient; they never need covering.
5. Read each block as a product term: the variables that are constant over the block, complemented where they are 0.

Grouping the 0s instead gives a minimal POS. Maps are practical for up to four variables, workable for five or six (as two or four 4-variable maps stacked), and useless beyond — at which point the algorithm of Section 12.8 takes over.

> **Status of the principle: Karnaugh-map minimisation**
> *Derived*. Gray-code adjacency makes every block of 2^k cells a valid application of the adjacency law; a cover by prime implicants that includes all essential primes and the fewest others gives a minimal two-level SOP, minimal in literal and term count but not necessarily in multi-level area or in delay, and silent on hazards unless they are checked separately.

Example 1: the full adder, a majority voter and a parity function

The carry out of a one-bit full adder (inputs A, B and carry in C_{in}) is 1 when at least two inputs are 1: $C_{out} = \sum m(3,5,6,7)$. On the three-variable map (Figure 12.4, left), the 1 in cell 111 is adjacent to each of the other three, giving three overlapping pairs:

$$C_{out} = AB + AC_{in} + BC_{in}$$

This is also the **majority function** of three inputs, the voter of **triple modular redundancy** (TMR): three identical channels — sensors, processors, or flip-flops in a radiation-tolerant FPGA — feed a voter whose output follows the majority, masking any single faulty channel. If each channel fails independently with probability p over a mission, the voted output fails only when two or more channels fail, with probability $3p^2 - 2p^3$; for $p = 10^{-4}$ that is 3.0×10^{-8}, more than three orders of magnitude better than one channel, provided the voter itself is reliable (it is the single point of failure, so aerospace designs triplicate voters too).

The sum bit $S = \sum m(1,2,4,7)$ is the opposite case: its 1s form a checkerboard, no two adjacent, so no merge is possible and the minimal SOP is the canonical one with four three-literal terms. The function is $S = A \oplus B \oplus C_{in}$, the three-input parity, built far more cheaply from two XOR gates. Parity functions are the worst case for two-level logic (an n-input parity needs 2^{n-1} terms) and the reason XOR gates are a primitive. Chapter 13 builds adders from these two results (Chapter 13, Section 13.1).

Example 2: a seven-segment decoder with don't-cares

A BCD-to-seven-segment decoder drives segment a (the top bar) for the digits 0, 2, 3, 5, 6, 7, 8 and 9, and not for 1 and 4; inputs 10 to 15 never occur in BCD. With inputs A (MSB), B, C, D:

$$a = \sum m(0,2,3,5,6,7,8,9) + \sum d(10,11,12,13,14,15)$$

Worked example — Minimising segment a. On the four-variable map (Figure 12.4, right) the don't-cares fill the bottom two rows ($A = 1$) except for cells 8 and 9, which are 1s.

- The whole $A = 1$ half (cells 8–15, eight cells of 1s and Xs) is one prime implicant: A.
- The column $C = 1$ (cells 2, 3, 6, 7, 10, 11, 14, 15) is another: C.
- Cells 5, 7, 13, 15 form the block BD, covering the 1 in cell 5, which nothing else covers.
- The four corners, cells 0, 2, 8, 10, are adjacent by wrap-around and form $\overline{B}\,\overline{D}$, covering the 1 in cell 0.

Each of the four is essential (cell 9 needs A, cell 3 needs C, cell 5 needs BD, cell 0 needs $\overline{B}\,\overline{D}$), so

$$a = A + C + BD + \overline{B}\,\overline{D}$$

Six literals, against 32 in the canonical SOP of the eight minterms. Setting the don't-cares to 0 instead would force the terms $\overline{A}C$, $\overline{A}BD$, $\overline{A}\,\overline{B}\,\overline{D}$ and $A\overline{B}\,\overline{C}$: 11 literals. The don't-cares halved the logic. In practice a 74HC4511 or a few FPGA LUTs do this job, but the same minimisation runs inside the synthesis tool that maps them.

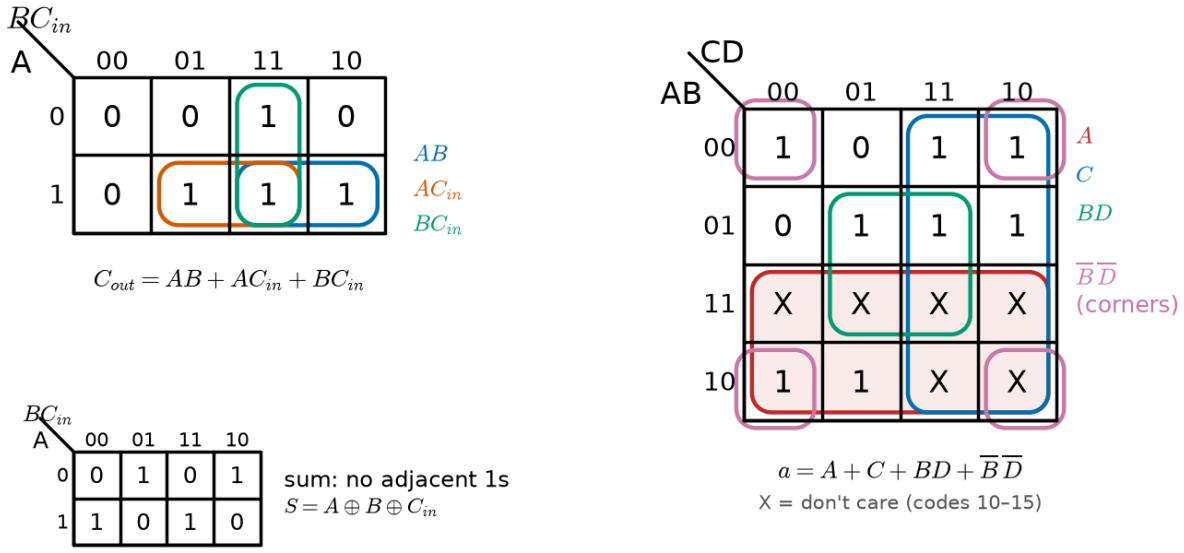


Figure 12.4 — Karnaugh maps for the full-adder carry (majority) and for segment a of a BCD-to-seven-segment decoder with don't-cares.

Hazards: when the minimal cover glitches

A minimal cover can **glitch**. Take $F = AB + \overline{A}C$ with $B = C = 1$: the output should stay 1 while A changes, because one term or the other is always 1. But if the inverter that produces $\overline{A}$ is slower than the direct path, there is a moment after A falls when AB has already turned off and $\overline{A}C$ has not yet turned on, and the output dips to 0 for about one gate delay: a **static-1 hazard**. On the map, the hazard appears wherever two adjacent 1s are covered by different, non-overlapping loops. Adding the consensus term BC, a loop bridging the two (redundant by the consensus law of Table 12.1), holds the output at 1 during the transition. In synchronous designs such glitches are normally harmless, because a register samples the output only after it has settled; they matter where a combinational output drives a clock, an asynchronous reset, a latch enable or an external device directly. Chapter 15 treats glitches and hazard-free design (Section 15.5).

12.7 Combinational logic

Definition and design procedure

A **combinational** circuit is one whose outputs, once settled, depend only on its present inputs: it has no memory and no feedback loops. Every combinational circuit is a set of Boolean functions, and every set of Boolean functions can be built as a combinational circuit. (A circuit with feedback, whose output depends also on its history, is **sequential**; Chapter 14.) The design procedure is the one this chapter has been assembling:

1. **Specify**: name the inputs and outputs, their polarity (active-high or active-low) and their meaning; write the requirements as Boolean statements.
2. **Tabulate**: build the truth table, marking don't-cares — and confirming that each one is truly impossible or harmless.
3. **Minimise**: map, algebra or tool; for multiple outputs, look for shared terms.
4. **Map to gates**: convert to the available primitives (NAND–NAND, NOR–NOR, a standard-cell library, LUTs), choosing the polarity of intermediate signals by bubble pushing.
5. **Verify**: compare the implementation with the specification on every input (as in Section 12.3) or by formal equivalence, then check timing (Chapter 13) and hazards where outputs drive asynchronous inputs.

Multiple outputs and shared terms

A circuit with several outputs is minimised as a whole. Minimising each output separately may miss a term that serves two of them. In the contactor interlock the term $T = KI\overline{F}$ appears in N, R and M, so it is built once and fanned out: $N = T$, $R = T\overline{P}$, $M = TP$. Separately, R and M would each need a four-input AND; shared, they need one three-input AND and two two-input ANDs. Multi-output minimisation is part of every synthesis tool (Section 12.8).

Worked example — The contactor interlock in hardware. Mapped to NAND gates (Figure 12.5), the interlock uses one three-input NAND with an inverter to form T; two two-input NANDs with inverters for R and M (or, since relay drivers are often active-low, the NAND outputs directly as $\overline{R}$ and $\overline{M}$); two two-input NANDs for W; and three inverters for $\overline{F}$, $\overline{I}$ and $\overline{P}$. That is eleven gates, which fit in three quad or hex 74HC packages, or a single small CPLD. The exhaustive check of Section 12.3 is the verification. Three engineering details complete the design, and none of them is Boolean algebra:

- **Fail-safe polarity.** Each input is wired so that a broken wire, a missing connector or a lost supply reads as the unsafe value: I is a current loop that reads 0 when open, F is active-high from a fault output that reads 1 when its source loses power (via a pull-up), and the contactor coils are energised to close, so loss of logic power opens them.
- **Break-before-make.** At the instant P rises, R falls and M rises; with unequal gate delays both could briefly be on (harmless here, the precharge resistor is bypassed for nanoseconds) or both briefly off (a contactor drops out). Contactor mechanics take milliseconds, so nanosecond glitches cannot move them, but a microcontroller input reading these signals could see them; Chapter 15 treats this as a glitch and synchronisation problem.
- **Independence.** The hardware path sits in series with the microcontroller's own enable outputs, so either can open the contactors and both must agree to close them. That diverse redundancy, not the gates themselves, is what a functional-safety assessment under ISO 26262 credits.

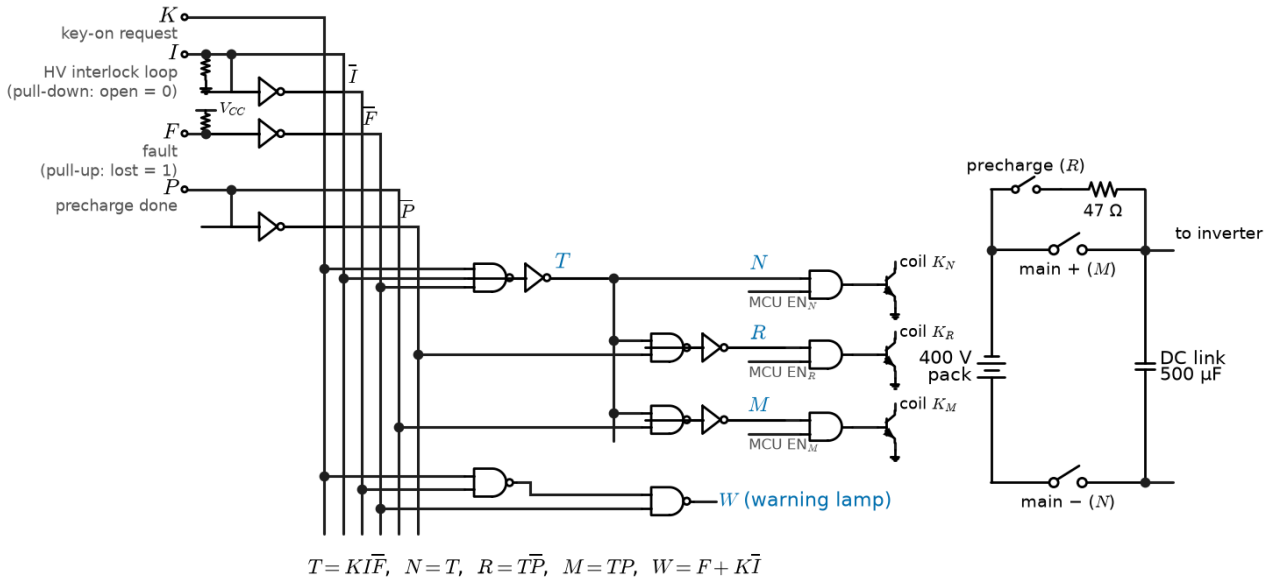


Figure 12.5 — Hardware interlock for an EV battery pack's contactors, implemented in NAND gates.

Where combinational logic lives

Small combinational functions appear as discrete gates in interlocks, power sequencing and level translation; as PLDs and CPLDs where a few dozen functions must change without a board respin; as LUTs in FPGAs, where any function of six inputs costs one LUT; and as standard cells in ASICs, where synthesis builds millions of them from HDL (Chapter 19). The large, structured combinational blocks — adders, multiplexers, decoders, comparators and ALUs — are the subject of Chapter 13; they are designed from the same principles, but by structure (a carry chain, a mux tree) rather than by minimising one large truth table.

12.8 Logic minimization

The Quine–McCluskey algorithm

Karnaugh maps rely on the eye and stop at about six variables. The **Quine–McCluskey** (QM) method does the same job by tabulation: list the minterms and don't-cares as binary strings; repeatedly merge pairs that differ in one bit (writing a dash for the eliminated variable), as the adjacency law allows; the terms that never merge are the prime implicants. A **covering** step then selects primes: essential ones first, and then a minimum set for the remaining minterms, exactly (Petrick's method) or greedily. The listing is a compact implementation, run on segment a of Section 12.6.

```
 from itertools import combinations

def combine(a, b):
    """Merge two implicants (strings of 0/1/-) that differ in one fixed bit."""
    diff = [i for i in range(len(a)) if a[i] != b[i]]
    if len(diff) == 1 and "-" not in (a[diff[0]], b[diff[0]]):
        i = diff[0]
        return a[:i] + "-" + a[i + 1:]
    return None

def covers(imp, m, n):
    return all(c in "-" + b for c, b in zip(imp, format(m, f"0{n}b")))

def minimise(ones, dcs, n):
    terms = {format(m, f"0{n}b") for m in ones + dcs}
    primes = set()
    while terms:                                    # Quine-McCluskey merging
        merged, used = set(), set()
        for a, b in combinations(sorted(terms), 2):
            c = combine(a, b)
            if c:
                merged.add(c); used.update((a, b))
        primes |= terms - used
        terms = merged
    cover, todo = [], set(ones)                     # essential primes first,
    while todo:                                     # then greedy on the rest
        ess = [p for p in sorted(primes) for m in sorted(todo)
               if [q for q in primes if covers(q, m, n)] == [p]]
        best = ess[0] if ess else max(sorted(primes), key=lambda p: sum(covers(p, m, n) for m in todo))
        cover.append(best)
        todo = {m for m in todo if not covers(best, m, n)}
    return sorted(primes), cover

def to_sop(imp, names="ABCD"):
    return "".join(v if c == "1" else v + "'" for v, c in zip(names, imp) if c != "-") or "1"

# Segment a of a BCD-to-7-segment decoder: on for 0,2,3,5,6,7,8,9; 10-15 never occur
primes, cover = minimise([0, 2, 3, 5, 6, 7, 8, 9], list(range(10, 16)), 4)
print("prime implicants:", [to_sop(p) for p in primes])
print("a =", " + ".join(to_sop(p) for p in cover))
```

It prints the four prime implicants, `prime implicants: ['C', "B'D'", 'BD', 'A']`, and the cover `a = C + B'D' + BD + A`, the result found by hand (the covering loop scans `sorted(primes)`, so the order does not depend on Python's set hashing). Called as `minimise([3, 5, 6, 7], [], 3)` it returns the three carry terms of the full adder. The merging step is

exact; the greedy covering step is a heuristic that can miss the true minimum on functions with cyclic prime-implicant tables, which is where Petrick's method or integer programming is needed.

> **Status of the principle: Exact two-level minimisation**
> *Derived.* Every minimal SOP consists of prime implicants (Quine's theorem), so generating all primes and solving the covering problem exactly gives a guaranteed minimum; the covering problem is NP-hard, and the number of primes can grow as fast as about $3^n/n$, so exact minimisation is limited to functions of roughly 15 to 20 inputs.

Heuristics and multi-level synthesis

Real designs have hundreds of inputs and outputs, far beyond exact methods. **Espresso**, developed at Berkeley in the 1980s, minimises two-level logic heuristically by repeatedly expanding implicants to primes, removing redundant ones and reshaping the cover; it reaches results within a few percent of minimal on most functions in a fraction of the time, and it remains the two-level engine inside PLD fitters. Modern synthesis is mostly **multi-level**: tools such as the open-source ABC represent logic as and-inverter graphs, restructure them by algebraic factoring, rewriting and resubstitution, and then perform **technology mapping**: covering the graph with the cells of a standard-cell library (each with its area, delay and power) or with K-input LUTs. For an FPGA with 6-input LUTs, the cost is the number of LUTs and the depth of the LUT network, not the literal count, so a function that would need dozens of gates may cost one LUT, and a 7-input function costs at least two. **Binary decision diagrams** (BDDs), canonical graph representations of Boolean functions, underpin equivalence checking: two netlists are equivalent if their reduced ordered BDDs, or their SAT-solver miters, agree (Chapter 34).

Table 12.3 — Minimization methods and where they are used.

Method	Exact or heuristic	Practical size	Where used
Algebraic manipulation	Neither (by inspection)	A few variables	Hand design, reading equations
Karnaugh map	Exact for small cases	4–6 variables	Hand design with don't-cares, hazard checks
Quine–McCluskey with Petrick's method	Exact	About 15–20 inputs	Teaching, small PLD functions, verification of heuristics
Espresso	Heuristic	Hundreds of inputs and outputs	PLA and CPLD fitting
Multi-level synthesis and technology mapping (ABC, commercial tools)	Heuristic	Millions of gates	FPGA and ASIC synthesis

What the designer still decides

Automatic minimisation removes the drudgery, not the decisions. The designer still chooses which inputs are don't-cares and whether that is safe, which outputs share logic, where the timing-critical paths are, whether a hazard matters, and which polarity each signal has. In the contactor interlock the eleven gates are not the hard part; the fail-safe wiring, the independence from software and the treatment of transitions are. That division of labour — algebra and tools for the function, engineering judgement for its electrical and temporal behaviour — continues through the rest of Part III.

What remains open

Open question	Where it is answered
How are adders, multiplexers, decoders, comparators and ALUs built from gates, and how long do they take to settle?	Chapter 13
How is memory added to combinational logic: latches, flip-flops, counters and state machines?	Chapter 14
How are glitches, hazards and asynchronous inputs handled in a complete design?	Chapter 15
How are logic functions mapped onto FPGA LUTs and CPLD macrocells?	Chapter 18
How is combinational logic described in Verilog and VHDL and synthesised?	Chapter 19
How are gates built from CMOS transistors, and where do noise margins, delay and power come from?	Chapter 33
How do synthesis, technology mapping and equivalence checking work at chip scale?	Chapter 34
How are long transmission-line traces between gates terminated to protect noise margins?	*Circuits, Signals and Communications*, Ch. 14; Chapter 15

Next: Chapter 13 assembles gates into the standard combinational building blocks — adders, multiplexers, decoders, comparators and ALUs — and introduces the propagation delays that limit how fast they compute.

Chapter 13 — Combinational Digital Systems

Part III · Digital Electronics

An engine-bay body controller passes every bench test at 25 °C. Its 16-bit microcontroller reads a memory-mapped analog front end through an external bus, and a 74HC138 three-to-eight decoder turns the upper address lines into chip selects. In the climatic chamber at 105 °C, one board in twenty starts returning corrupted sensor readings. Nothing is broken: the decoder is slower when hot, the chip select now arrives a few nanoseconds later, and the memory it enables no longer has its data on the bus before the processor samples it. The logic is right; the timing is not. A combinational circuit is a Boolean function only after its outputs have settled, and an engineer who designs adders, multiplexers, decoders and ALUs must know both the function and how long it takes.

> **How are the standard combinational building blocks (adders, multiplexers, decoders, encoders, comparators, ALUs, code converters) organised, and how do their structures trade logic for speed? How long does a combinational block take to produce a valid output, what changes that time, and how is it read from a datasheet or an FPGA timing report?**

Concept	Engineering destination
Adders and subtractors	ALUs, address arithmetic, DSP accumulators, PWM timers, FPGA carry chains
Multiplexers	Data selection, bus sharing, barrel shifters, FPGA look-up tables
Demultiplexers	Strobe and enable distribution, time-division outputs, LED scanning
Encoders	Interrupt priority, keypad scanning, flash-ADC thermometer decoding
Decoders	Memory and peripheral address decoding, 7-segment and row drivers
Comparators	Address matching, limit and window checks in protection logic, sorting
Arithmetic logic units	Processor datapaths, DSP blocks, flags for branch decisions
Code converters	Gray-code encoders and FIFO pointers, BCD displays, line codes
Timing and propagation delay	Bus timing budgets, FPGA timing closure, worst-case corners

Roadmap. Arithmetic first, because it sets the critical path of most systems: adders and subtractors (13.1) → the selection blocks: multiplexers (13.2), demultiplexers (13.3), encoders (13.4) and decoders (13.5) → comparison and arithmetic combined: comparators (13.6) and ALUs (13.7) → code converters (13.8) → what all of them have in common, propagation delay and how it is budgeted (13.9).

13.1 Adders and subtractors

> **Boundary.**
> Binary and two's-complement representation, overflow and the arithmetic rules are in Chapter 11; Boolean algebra, gates, Karnaugh maps and minimisation in Chapter 12. This chapter uses those results to build blocks; transistor-level gate design and the RC origin of gate delay belong to Chapter 33.

Half and full adders

Adding two one-bit numbers A and B gives a sum bit and a carry bit. The **half adder** computes

$$S = A \oplus B, \qquad C = A \cdot B$$

Adding a carry-in C_{in} as well needs a **full adder**:

$$S = A \oplus B \oplus C_{in}, \qquad C_{out} = AB + C_{in}(A \oplus B)$$

Both follow directly from the truth table of binary addition (Chapter 12 minimises it on a Karnaugh map). The carry expression is the key to everything that follows, so it is worth naming its two parts. The **generate** signal $G = AB$ says that this bit position produces a carry regardless of the carry-in. The **propagate** signal $P = A \oplus B$ says that it passes an incoming carry through. Then

$$C_{out} = G + P\,C_{in}$$

> **Status of the principle: Full-adder equations and the generate–propagate form**
> *Derived.* They follow from the truth table of one-bit binary addition and Boolean algebra; they are exact for any logic family.

The ripple-carry adder and its delay

Chaining n full adders, carry-out to carry-in, gives an n-bit **ripple-carry adder** (Figure 13.1). It is the smallest adder (one full adder per bit) and the slowest. In the worst case a carry generated in bit 0 must travel through every stage; adding 1 to 0xFFFF is the classic example. If P and G take one gate delay t_g and each carry stage is an AND–OR of two gate delays, the last sum bit settles after about

$$t_{ripple} \approx 2n\,t_g$$

so the delay grows linearly with the word length. A 64-bit ripple adder is roughly eight times slower than an 8-bit one.

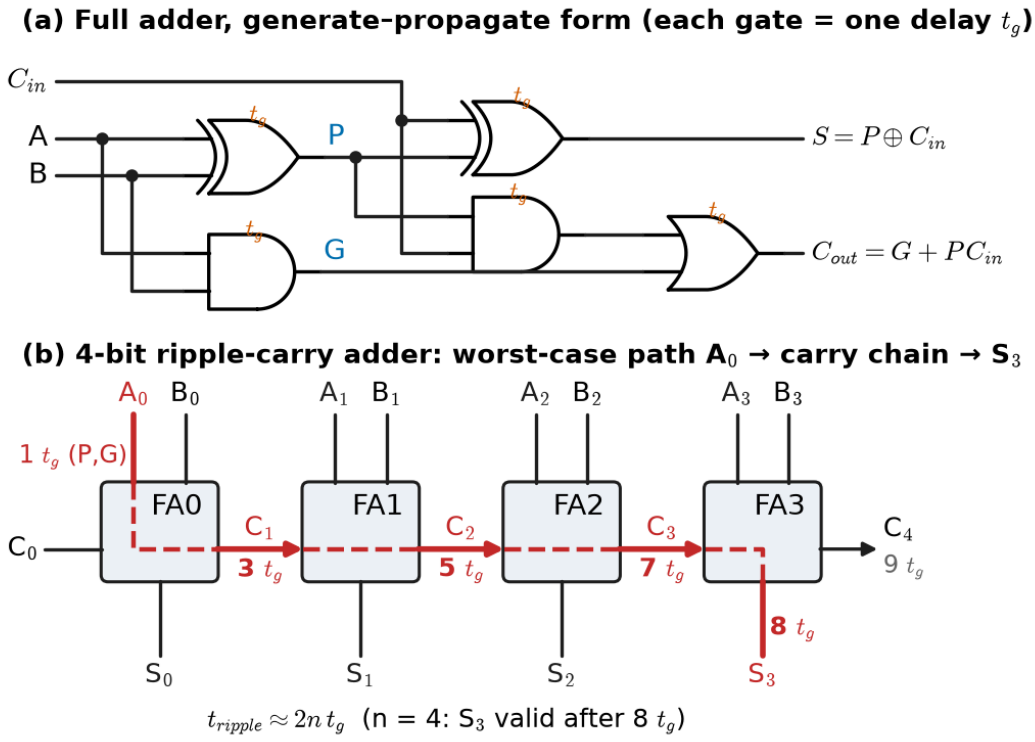


Figure 13.1 — Full adder in generate-propagate form and a 4-bit ripple-carry adder with its critical path.

Faster carries: lookahead and parallel prefix

The generate–propagate form can be expanded so that a carry depends directly on the inputs rather than on the previous carry. For a 4-bit group,

$$C_4 = G_3 + P_3G_2 + P_3P_2G_1 + P_3P_2P_1G_0 + P_3P_2P_1P_0C_0$$

which is a two-level AND–OR expression: every carry in the group is available after the same few gate delays. This is the **carry-lookahead adder** (CLA). Its cost is the growing fan-in of the AND and OR terms, so practical designs stop at groups of four and compute group generate and propagate signals $G_{3:0}$ and $P_{3:0}$ that feed a second level of lookahead.

Taken to its conclusion, lookahead becomes a **parallel-prefix adder**. The operator $(G, P) \circ (G', P') = (G + PG',\ PP')$ is associative, so the carries for all n bits can be computed with a tree of depth $\log_2 n$. Different trees trade depth, wiring and fan-out: **Kogge–Stone** has the minimum depth and fan-out of 2 but many cells and long wires; **Brent–Kung** uses about $2\log_2 n - 1$ levels and far fewer cells; **Sklansky**, **Han–Carlson** and **Ladner–Fischer** sit between them. Two other common structures avoid a full tree: the **carry-select adder** computes each upper block twice, once for carry-in 0 and once for carry-in 1, and picks the right answer with a multiplexer when the real carry arrives; the **carry-skip adder** lets a carry bypass a block whose bits all propagate.

The listing compares the depth and cell count of these structures in the same unit-gate model.

```
 import math

def adder_cost(n):
    """(worst-case gate delays, carry cells) in the unit-gate model."""
    L = int(math.log2(n))
    return {
        "ripple": (2 * n, n),
        "brent-kung": (2 * (2 * L - 1) + 2, 2 * n - 2 - L),
        "kogge-stone": (2 * L + 2, n * L - n + 1),
    }

print(f"{'n':>3}" + "".join(f"{k:>14}" for k in adder_cost(8)))
for n in (8, 16, 32, 64):
    cells = (str(d) + "tg/" + str(c) for d, c in adder_cost(n).values())
    print(f"{n:>3}" + "".join(f"{s:>14}" for s in cells))
```

```
  n        ripple    brent-kung   kogge-stone
  8        16tg/8       12tg/11        8tg/17
 16       32tg/16       16tg/26       10tg/49
 32       64tg/32       20tg/57      12tg/129
 64      128tg/64      24tg/120      14tg/321
```

Each prefix level is counted as an AND–OR pair (two gate delays) plus one level for P, G and one for the final XOR. At 32 bits Kogge–Stone is about five times faster than ripple carry and costs four times as many carry cells; Brent–Kung recovers most of the speed for less than twice the cells. Figure 13.2 plots the trend. Real silicon is less tidy, because long wires and high fan-out slow the tree adders, but the ranking holds, and it is why every high-speed processor ALU uses a prefix adder while small microcontrollers and FPGAs often use ripple carry.

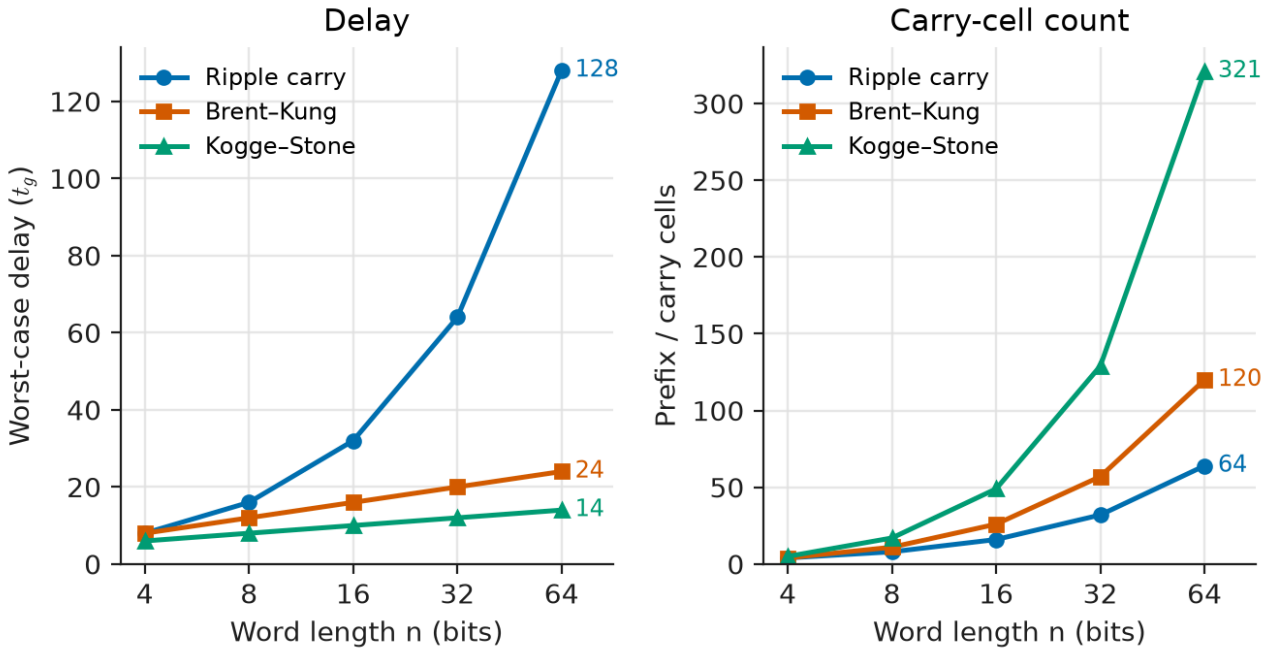


Figure 13.2 — Worst-case delay and carry-cell count of ripple, Brent-Kung and Kogge-Stone adders versus word length in the unit-gate model.

> **Status of the principle: Unit-gate delay model of adders**
> *Approximate.* Counts every gate as one equal delay and ignores fan-out and wire load; it ranks architectures correctly but underestimates the delay of wide, wire-heavy trees such as Kogge–Stone by tens of percent in real layouts.

Adders in FPGAs

An FPGA does not build adders from general logic. Each logic slice contains a dedicated **carry chain**: a multiplexer per bit that selects either the local generate or the incoming carry, wired directly to its neighbour without passing through the programmable interconnect. In a Xilinx 7-series device a CARRY4 primitive covers 4 bits; Intel ALMs and Lattice slices have equivalents. Although this is structurally a ripple adder, the per-bit delay is only tens of picoseconds, so a 32-bit add easily runs above 300 MHz. FPGA architecture and the LUT itself are the subject of Chapter 18.

Worked example — a 16-bit adder from 74HC283 parts and a 32-bit adder in an FPGA. A legacy test fixture adds two 16-bit counts with four cascaded 74HC283 4-bit adders. At 4.5 V and 50 pF, representative typical delays are 20 ns from an A or B input to C_4, 20 ns from C_0 to C_4 and 22 ns from C_0 to a sum output. The worst path enters the first package at A_0, ripples through the carries of the next two and ends at a sum bit of the fourth:

$$t_{16} = 20 + 20 + 20 + 22 = 82 \text{ ns} \quad \text{(typical)}$$

The guaranteed maximum at 25 °C is about twice the typical figure and rises another 25 % at 85 °C, so the design must allow roughly 200 ns. Inside each 74HC283 the four bits use internal lookahead, which is why one package takes 20 ns instead of four times a single full adder.

The same function in an Artix-7 FPGA (speed grade −1), as a 32-bit adder across eight CARRY4 primitives, has representative delays of 0.5 ns from the LUT through the first carry block, about 0.11 ns for each of the six middle blocks and 0.35 ns from the last block to its output:

$$t_{32} \approx 0.5 + 6 \times 0.11 + 0.35 = 1.51 \text{ ns}$$

Adding about 0.6 ns of routing at each end gives roughly 2.7 ns, comfortably inside a 4 ns (250 MHz) clock period. The FPGA adder is twice as wide and about 30 times faster than the discrete one at its typical rating.

Subtraction

Subtraction is addition of the two's complement: $A - B = A + \overline{B} + 1$ (Chapter 11). An **adder–subtractor** therefore needs only an XOR on each B input, controlled by a mode bit M that also drives the carry-in:

$$B'_i = B_i \oplus M, \qquad C_0 = M$$

With $M = 0$ the circuit adds; with $M = 1$ it inverts B and adds 1, so it subtracts. The carry-out then means "no borrow" for unsigned operands, and signed overflow is detected as $V = C_n \oplus C_{n-1}$, the rule derived in Chapter 11. A separate **full subtractor** with a borrow chain exists in textbooks and in a few serial arithmetic units, but nearly every real design uses the adder–subtractor because it shares the adder's carry structure.

13.2 Multiplexers

The selection function

A **multiplexer** (mux) routes one of 2^k data inputs to its output under the control of k select lines. A 2:1 mux is

$$Y = \overline{S}\, D_0 + S\, D_1$$

and a 4:1 mux is $Y = \overline{S_1}\,\overline{S_0} D_0 + \overline{S_1} S_0 D_1 + S_1 \overline{S_0} D_2 + S_1 S_0 D_3$. Larger multiplexers are built as trees of smaller ones: an 8:1 mux is two 4:1 muxes and a 2:1 mux, with the most significant select line driving the last stage. The delay from the data inputs is a few gate delays; the delay from the select lines is usually longer, because a select change must decode and then gate.

Representative parts are the 74HC157 (quad 2:1), 74HC153 (dual 4:1) and 74HC151 (8:1 with complementary outputs). The 74HC4051 and 74HC4053 look similar but are *analog* multiplexers, CMOS transmission gates with an on-resistance of 50 to 100 Ω that pass voltages in both directions; they select sensor channels ahead of an ADC rather than logic levels.

A multiplexer is a universal function

Any Boolean function of k variables can be written by **Shannon expansion** about one variable,

$$f(x_1, \ldots, x_k) = \overline{x_1}\, f(0, x_2, \ldots, x_k) + x_1\, f(1, x_2, \ldots, x_k)$$

which is exactly a 2:1 mux selected by x_1. Repeating the expansion shows that a 2^k:1 mux whose data inputs are tied to the constants of the truth table implements *any* k-input function. This is literally how an FPGA look-up table works: a 6-input LUT is 64 bits of configuration memory feeding a 64:1 multiplexer tree whose select lines are the six logic inputs. It is also why mux-based logic design (pass-transistor and transmission-gate muxes in CMOS) appears in Chapter 33.

> **Status of the principle: Shannon expansion**
> *Derived*. An identity of Boolean algebra, true for every function; it underlies multiplexer-based logic, LUTs and binary decision diagrams used in synthesis tools.

Muxes versus tri-state buses

Several sources can share one destination either through a mux or through **tri-state drivers** on a common wire, with exactly one driver enabled at a time. Board-level buses (the data bus of an external SRAM, a 74LVC245 transceiver) still use tri-states, because the wire runs between packages. Inside ASICs and FPGAs internal tri-state buses have essentially disappeared: two drivers briefly enabled together during a select change cause contention current, and a bus with no driver floats. Synthesis tools convert internal tri-state descriptions to multiplexers.

Barrel shifters

A **barrel shifter** shifts or rotates an n-bit word by any amount from 0 to $n-1$ in one combinational pass. It is built from $\log_2 n$ layers of 2:1 multiplexers; layer j either passes its input or shifts it by 2^j positions. A 32-bit shifter therefore needs 5 layers and $5 \times 32 = 160$ two-input multiplexers, and its delay is five mux delays regardless of the shift amount. Barrel shifters serve floating-point normalisation (Chapter 11), the shift operands of Arm instructions, and bit-field extraction in network packet processors.

13.3 Demultiplexers

A **demultiplexer** does the reverse of a multiplexer: it sends one input to one of 2^k outputs selected by k address lines, holding the others inactive. Output i is

$$Y_i = D \cdot m_i(S_{k-1}, \ldots, S_0)$$

where m_i is the i-th minterm of the select lines. That is a decoder with its enable used as the data input, and in practice the two are the same part: the 74HC138 three-to-eight decoder becomes a 1:8 demultiplexer when the data signal drives one of its enable pins. Its outputs are active low, so the data appears inverted on the selected line unless it drives an active-low enable.

Typical uses are distribution rather than computation:

- **Strobe distribution.** One write strobe from a processor is routed to the latch enable of one of eight output registers, selected by the address.
- **Multiplexed displays and keypads.** A demultiplexer drives one row of an LED matrix or keypad at a time, cycling fast enough (typically 100 Hz to 1 kHz per row) for persistence of vision or for scanning.
- **Analog demultiplexing.** The same analog switches as Section 13.2 route one DAC output to several sample-and-hold capacitors in a multichannel bias generator.

The engineering hazard of a demultiplexer is not the function but its outputs during a select change. If the select lines do not change at exactly the same instant, the outputs can briefly pass through an unintended code: switching the address from 011 to 100 may momentarily produce 111 or 000. A short pulse on the wrong output is harmless if it drives a data line that is sampled later, and destructive if it drives a latch enable or a clock. Section 15.5 develops these glitches and how synchronous design removes them.

13.4 Encoders

Binary and priority encoders

An **encoder** converts a one-hot input (exactly one of 2^k lines active) into a k-bit code. For an 8-to-3 encoder each output bit is the OR of the inputs whose index has a 1 in that position:

$$Y_0 = I_1 + I_3 + I_5 + I_7, \quad Y_1 = I_2 + I_3 + I_6 + I_7, \quad Y_2 = I_4 + I_5 + I_6 + I_7$$

If two inputs are active at once, a plain encoder outputs the OR of their codes, which is meaningless. Real systems rarely guarantee one-hot inputs, so the useful block is the **priority encoder**, which outputs the index of the highest-priority active input and a "valid" flag that distinguishes "input 0 active" from "nothing active". The 74HC148 is the standard 8-to-3 part, with active-low inputs and enable-in and enable-out pins for cascading to 16 or more lines.

Where encoders appear

- **Interrupt controllers.** The Arm NVIC and every programmable interrupt controller contain priority-encoding logic that picks the highest-priority pending request; Chapters 16 and 17 describe the architecture.

- **Flash ADCs.** A flash converter compares the input with $2^N - 1$ reference levels at once, giving a **thermometer code** (all comparators below the input read 1). A logic stage finds the 1-to-0 boundary and encodes its position into N bits. A comparator that fires out of order (a "bubble") would corrupt a plain encoder; practical designs use three-input bubble-correction gates or convert through Gray code first. Mixed-signal converter interfaces are in Chapter 10.
- **Leading-zero counting.** Floating-point normalisation and integer square-root routines need the position of the most significant 1, which is a priority encoder; processors provide it as a CLZ instruction.
- **Keypads.** A 16-key pad is scanned with a demultiplexer on the rows and a priority encoder on the columns.

13.5 Decoders

The n-to-2^n decoder

A **decoder** turns a k-bit code into 2^k outputs, exactly one of which is active: output i is the minterm m_i of the inputs, ANDed with an enable. Because every minterm is available, a decoder followed by an OR gate implements any function of its inputs, a sum of minterms (Chapter 12). Decoders are cascaded through their enables: two 74HC138s with complementary enables driven by a fourth address bit make a four-to-sixteen decoder.

Address decoding

The decoder's main job on a circuit board is **address decoding**: turning the high-order address lines of a processor bus into chip-select signals, so that each memory or peripheral occupies its own range of the memory map (Chapter 17). Inside every memory chip, row and column decoders do the same thing on a much larger scale; a 1 Mbit SRAM array with 1024 rows needs a 10-to-1024 row decoder, usually built from 2-to-4 or 3-to-8 **predecoders** whose outputs are ANDed at each row, which keeps the fan-in of each gate small.

Worked example — will a 74HC138 meet the bus timing at 105 °C? The opening problem, with representative numbers. A processor's external bus runs a 100 ns read cycle. The address is valid 8 ns after the cycle starts, and the processor samples data 10 ns before the cycle ends, at 90 ns. A 74HC138 at 4.5 V decodes the address into the chip select of a 55 ns SRAM (access time from chip select $t_{ACS} = 55$ ns, which is longer than its address access time, so chip select is the limiting path).

The decoder's typical delay is about 13 ns, and with it the data would be valid at $8 + 13 + 55 = 76$ ns, inside the window. But the design must use the guaranteed maximum, which for this class of part is about 30 ns at 25 °C and 38 ns at 85 °C (the automotive-grade part's figure at 105–125 °C is higher still):

$$t_{data} = 8 + 38 + 55 = 101 \text{ ns} > 90 \text{ ns}$$

The design fails by 11 ns at 85 °C even before considering the higher temperature and board delays. It passed on the bench because typical parts at room temperature are three times faster than their guaranteed limit. Replacing the decoder with a 74LVC138 at 3.3 V, whose maximum delay is about 7 ns, gives

$$t_{data} = 8 + 7 + 55 = 70 \text{ ns}, \qquad \text{slack} = 90 - 70 = 20 \text{ ns}$$

which leaves room for trace delay and temperature. The alternative is to add one wait state to the bus cycle. Either way, the decision is made from maximum delays, never typical ones.

Display and driver decoders

A **BCD-to-7-segment decoder** maps a 4-bit decimal digit to the seven segments a–g of a display; the 74HC4511 includes a latch and segment drivers. It is a code converter as much as a decoder (Section 13.8). Larger-scale relatives drive the row lines of LCD and OLED panels (gate drivers) and the word lines of memories.

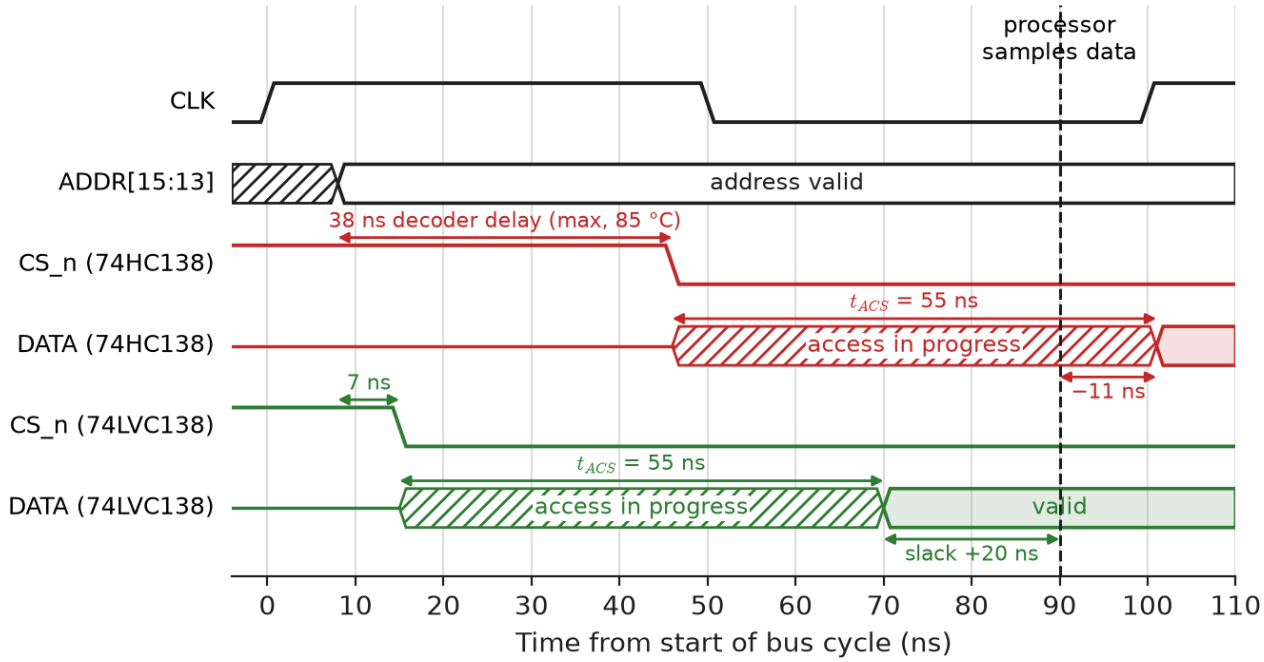


Figure 13.3 — Read cycle of an external bus with a 74HC138 address decoder, comparing the maximum decoder delay with that of a 74LVC138.

13.6 Comparators

Equality

Two n-bit words are equal when every bit pair matches. Each bit uses an XNOR, $E_i = \overline{A_i \oplus B_i}$, and the results are ANDed:

$$(A = B) = \prod_{i=0}^{n-1} \overline{A_i \oplus B_i}$$

The AND of n terms is built as a tree of depth $\lceil \log_2 n \rceil$ two-input gates (or $\lceil \log_4 n \rceil$ four-input ones), so a 32-bit equality check takes one XNOR delay plus five AND levels. Equality comparators are everywhere: cache tag matching (Chapter 16), address match in an I²C or CAN controller's acceptance filter, breakpoint comparators in a debugger, and the compare-match unit of every microcontroller timer, where a counter equal to a compare register toggles a PWM output.

Magnitude

Deciding $A > B$ requires finding the most significant bit where the words differ. Scanning from the MSB,

$$(A > B) = A_{n-1}\overline{B_{n-1}} + E_{n-1}A_{n-2}\overline{B_{n-2}} + E_{n-1}E_{n-2}A_{n-3}\overline{B_{n-3}} + \cdots$$

which has the same chain-of-products structure as the carry lookahead of Section 13.1, and is accelerated the same way. The 74HC85 is a 4-bit magnitude comparator with $A > B$, $A = B$ and $A < B$ cascade inputs, so that several packages compare longer words (in a slow ripple fashion).

In a processor or FPGA the cheapest magnitude comparator is a subtractor. Compute $A - B$ and read the flags: for unsigned operands $A \geq B$ exactly when the carry-out is 1 (no borrow); for signed two's-complement operands $A < B$ exactly when $N \oplus V = 1$, the sign of the result corrected for overflow. This is why compare instructions (`CMP` in Arm, `SLT` in RISC-V) are subtractions that discard the result and keep the flags. Mixing the two interpretations is a classic bug: the 8-bit pattern $1000\,0000_2$ is greater than $0111\,1111_2$ as unsigned (128 > 127) and less as signed (−128 < 127).

Window comparators in protection logic

Digital protection logic often checks that a measured value lies inside a window, $L \le X \le H$, using two magnitude comparators and an AND gate. In a motor inverter's FPGA, the phase current from a 12-bit ADC is compared every sample with overcurrent limits; a value outside the window latches a fault and disables the gate drivers within a few clock cycles, faster than any software path. The comparators themselves are combinational, but the fault latch is sequential (Chapter 14), and the analog front end that produces X sets the real response time.

13.7 Arithmetic logic units

Structure

An **arithmetic logic unit** (ALU) combines an adder–subtractor, bitwise logic and often a shifter behind an output multiplexer selected by an operation code (Figure 13.4). For an 8-bit ALU, a typical operation set is ADD, SUB, AND, OR, XOR, NOT, pass-through and shifts, selected by a 3- or 4-bit opcode. All functional units compute in parallel on every operation; the multiplexer chooses which result to keep. This wastes energy on the unused units, which is why low-power designs gate the operand inputs of idle units, but it keeps the delay equal to the slowest unit plus one mux.

The historical reference point is the 74181 (1970), a 4-bit TTL ALU with 16 logic and 16 arithmetic functions and lookahead outputs for cascading with a 74182 carry-lookahead unit. Minicomputers of the 1970s built their datapaths from these; today the same function is a few hundred cells inside a processor core or a handful of LUTs in an FPGA.

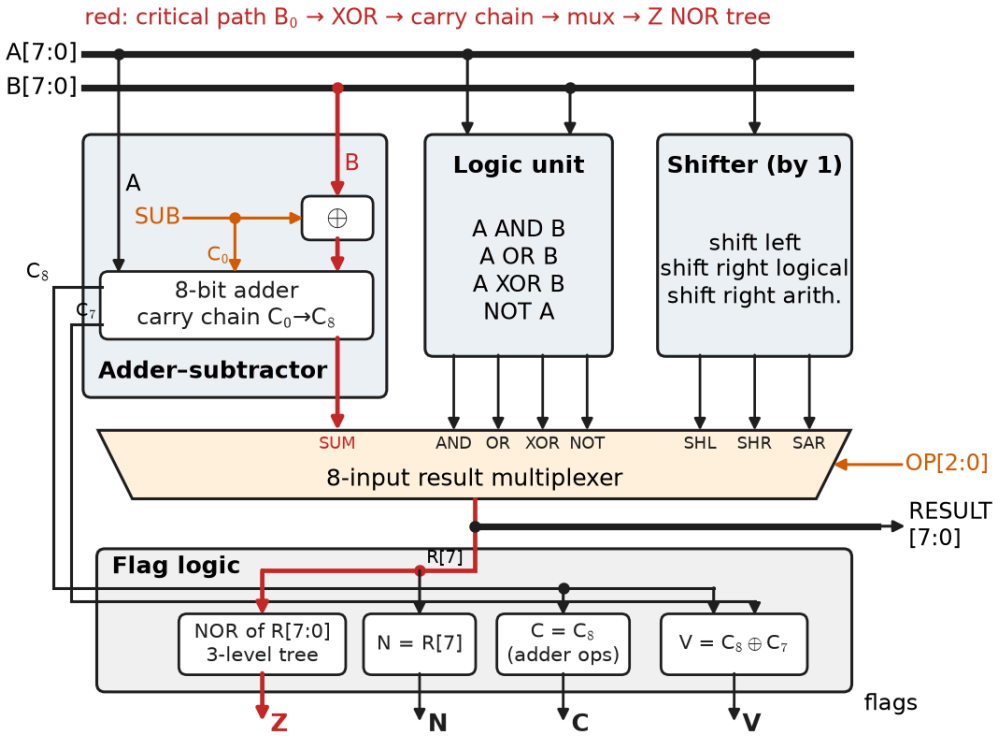


Figure 13.4 — An 8-bit ALU built from an adder-subtractor, a logic unit and a shifter behind a result multiplexer, with flag generation.

Status flags

Most ALUs produce four flags from each result:

Flag	Meaning	Logic
Z (zero)	Result is all zeros	NOR of all result bits
N (negative)	Result is negative as two's complement	Result MSB
C (carry)	Unsigned carry-out, or no-borrow on subtract	Adder carry-out C_n
V (overflow)	Signed result out of range	$C_n \oplus C_{n-1}$

The flags feed conditional branches, multi-word arithmetic (the carry flag chains 32-bit additions into 64-bit ones) and saturation logic. The zero flag is often the critical path of an ALU, because it waits for the slowest sum bit and then passes through a $\log_2 n$-deep NOR tree.

> **Status of the principle: Overflow rule** $V = C_n \oplus C_{n-1}$
> *Derived.* A consequence of two's-complement arithmetic (Chapter 11): signed overflow occurs exactly when the carry into the sign bit differs from the carry out of it. It holds for addition and for subtraction performed as addition of the complement.

Where ALUs live

The ALU at the heart of a processor datapath, and how the control unit drives its opcode, is the subject of Chapter 16. Outside processors, small special-purpose ALUs appear in DSP blocks (an FPGA DSP48 slice combines a 25×18 multiplier with a 48-bit ALU), in network switches (header field arithmetic), in motor-control timers (dead-time insertion as an add and compare), and in safety controllers, which run two ALUs in lockstep and compare their results every cycle to detect hardware faults.

13.8 Code converters

Gray code

In the **reflected binary Gray code**, consecutive values differ in exactly one bit. Conversion from binary B to Gray G is a single layer of XOR gates,

$$G_i = B_i \oplus B_{i+1}, \qquad G_{n-1} = B_{n-1}$$

and the reverse is a chain, each binary bit being the XOR of all Gray bits at and above it:

$$B_i = G_{n-1} \oplus G_{n-2} \oplus \cdots \oplus G_i$$

so Gray-to-binary has a delay that grows with n, like a ripple carry, unless built as an XOR prefix tree.

The single-bit-change property matters wherever a value is read while it is changing:

- **Absolute position encoders.** An optical encoder disc read by n photodetectors cannot make all tracks change at exactly the same angle.
- **Asynchronous FIFO pointers.** A counter in one clock domain read by logic in another (Chapter 14) can be sampled mid-transition; with Gray code the sampled value is either the old or the new count, never a wild value.
- **Karnaugh maps.** The ordering of rows and columns in a K-map (Chapter 12) is a Gray code, for the same reason: adjacent cells differ in one variable.

Worked example — why a robot joint encoder uses Gray code. A 10-bit absolute encoder on a robot wrist reports 1024 positions per revolution, 0.352° each. Between position 511 ($01\,1111\,1111_2$) and 512 ($10\,0000\,0000_2$) all ten bits change. The tracks are read by separate photodiodes whose edges are offset by mechanical tolerance; suppose they are spread over 0.05° of rotation. At 60 rpm the shaft turns 360° per second, so the transition is spread over

$$\frac{0.05^\circ}{360^\circ/\mathrm{s}} = 139\ \mu\mathrm{s}$$

and a controller sampling every 125 µs (an 8 kHz servo loop) is certain to take at least one sample inside the window. That sample could read any of the $2^{10} = 1024$ codes, from 0 to 1023, a position error of up to half a revolution that the servo loop would treat as real and react to violently. In Gray code, consecutive positions differ in exactly one bit by construction; the listing checks the 511-to-512 transition and the round trip for all 1024 codes.

```
def bin2gray(b):
    return b ^ (b >> 1)

def gray2bin(g):
    b = 0
    while g:
        b ^= g
        g >>= 1
    return b

for v in (511, 512):
    print(v, format(v, "010b"), format(bin2gray(v), "010b"))
print("bits differing:", bin(bin2gray(511) ^ bin2gray(512)).count("1"))
assert all(gray2bin(bin2gray(v)) == v for v in range(1024))
```

```
511 0111111111 0100000000
512 1000000000 1100000000
bits differing: 1
```

Only the MSB of the Gray code changes. The worst error from a mid-transition sample falls from half a revolution to one count, 0.352°, which the servo loop's filtering absorbs. The controller converts Gray to binary with the XOR chain before using the position.

BCD and display codes

Binary-coded decimal stores each decimal digit in four bits. It survives in displays, real-time clock chips (whose time registers are BCD), utility meters and some financial arithmetic. Converting binary to BCD is not a simple wiring: the **double-dabble** algorithm shifts the binary number left one bit at a time into the BCD digits, adding 3 to any digit that is 5 or more before each shift, so that it carries correctly into the next decade. A 16-bit binary value needs 16 shift steps and five BCD digits. In combinational form it unrolls into a triangular array of add-3 cells; in a microcontroller it is a short loop. BCD to 7-segment is a pure look-up (the 74HC4511 of Section 13.5).

Thermometer, one-hot and line codes

Other converters in common use:

- **Thermometer to binary**, the flash-ADC encoder of Section 13.4, and the reverse, **binary to thermometer**, used by segmented DACs whose most significant bits drive $2^k - 1$ equal unit elements so that every code step switches one more element (good differential linearity).
- **One-hot to binary** (an encoder) and **binary to one-hot** (a decoder), used at the boundary between state machines encoded one way and logic that expects another (Chapter 14).
- **Line codes** such as 8b/10b and 64b/66b, which map data words to code words with guaranteed transitions and DC balance for serial links (PCIe, USB 3, Ethernet). They are look-up-table converters with a running-disparity state; their purpose belongs to communications, covered in *Circuits, Signals and Communications*, Ch. 27.

13.9 Timing and propagation delay

Defining the delays

Every combinational block takes time to respond. A datasheet or timing model defines it by four quantities measured between the 50 % points of input and output edges (Figure 13.5):

- t_{PLH}, the delay for an output rising from low to high, and t_{PHL} for an output falling. They differ because the pull-up and pull-down transistors differ.
- **Propagation delay** t_{pd}: the maximum time from an input change to a *stable, valid* output. Static timing uses the larger of t_{PLH} and t_{PHL}, at the slowest conditions.
- **Contamination delay** t_{cd}: the minimum time from an input change until the output *starts* to change. Datasheets for standard logic rarely give it; FPGA and ASIC timing models always do, because hold-time checks (Chapter 14) depend on it.
- **Transition time** t_t (rise and fall, usually 10–90 %), which depends on load capacitance and affects the delay of the next stage.

Between t_{cd} and t_{pd} the output is undefined: it may hold its old value, move to the new one, or pass through intermediate values (glitches). A combinational block behaves as a Boolean function only *after* t_{pd}.

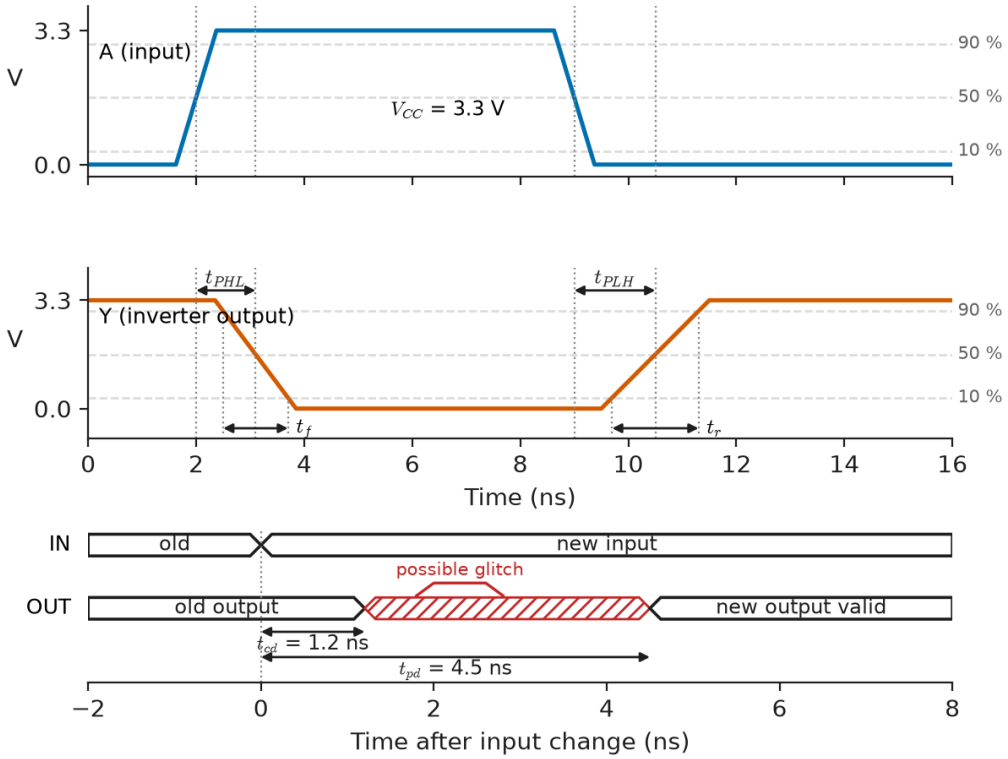


Figure 13.5 — Definitions of propagation delays, contamination delay and transition times for an inverting gate and for a multi-input block.

> **Status of the principle: The propagation-delay abstraction**
> *Approximate.* Replacing continuous analog edges by two delays, t_{cd} and t_{pd}, measured at 50 % thresholds is valid while edges are fast compared with the delays and the interconnect is short; it breaks down with slow input edges, crosstalk, ringing on long traces and supply droop, where the delay depends on waveform detail.

What sets the delay

A gate's delay is, to a first approximation, the time its output resistance takes to charge the load capacitance, an RC time constant (*Circuits, Signals and Communications*, Ch. 6 §6.5); the transistor-level origin is developed in Chapter 33. From the user's side, four factors move it:

1. **Supply voltage.** Lower V_{CC} means less drive current and slower switching. A 74HC00 has a typical t_{pd} of about 45 ns at 2 V, 9 ns at 4.5 V and 8 ns at 6 V.
2. **Temperature.** Carrier mobility falls as temperature rises, so CMOS slows when hot: the 74HC00's guaranteed maximum is about 18 ns at 25 °C, 23 ns up to 85 °C and 27 ns up to 125 °C (at 4.5 V).
3. **Load capacitance.** Each picofarad of load adds delay through the output resistance. Datasheets specify delay at 15 pF or 50 pF loads; delays at other loads scale roughly linearly.
4. **Process.** Parts from different wafers differ. "Typical" means a nominal process at 25 °C and nominal supply; "maximum" covers the slow process corner, lowest supply and highest temperature in the stated range.

These are the **PVT corners** (process, voltage, temperature) that FPGA and ASIC timing analysis checks automatically and that a board designer must check by hand from the datasheet's maximum column.

Table 13.1 — Representative propagation delays (typical values; check the specific datasheet).

Element	Conditions	t_{pd} typical	t_{pd} maximum
74HC00 NAND	4.5 V, 50 pF, 25 °C	9 ns	18 ns (23 ns to 85 °C)
74HC00 NAND	2.0 V, 50 pF, 25 °C	45 ns	90 ns
74AHC00 NAND	5 V, 50 pF	5 ns	8–9 ns
74LVC1G00 NAND	3.3 V, 50 pF	2.5 ns	4.5 ns
74AUP1G00 NAND	1.8 V, 15 pF	3–4 ns	8 ns
74HC157 2:1 mux, data to Y	4.5 V, 50 pF	11–14 ns	30 ns
74HC138 decoder, A to Y	4.5 V, 50 pF	13 ns	30 ns (38 ns to 85 °C)
74HC283 4-bit adder, A to Σ	4.5 V, 50 pF	18–22 ns	40–45 ns
FPGA LUT6 (28 nm, mid speed grade)	internal	0.10–0.12 ns	—
FPGA local route (28 nm)	internal	0.2–1 ns	—
FPGA carry per 4 bits (28 nm)	internal	about 0.1 ns	—
iCE40 LUT4 (40 nm, low power)	internal	0.4–0.5 ns	—
CMOS FO4 inverter delay	28 nm / 7 nm	12–18 ps / 5–8 ps	—

The table shows three regimes. Discrete logic (5 to 50 ns per block) is dominated by package and board capacitance; a signal crossing two packages and a trace costs more than any logic it performs. FPGA logic (about 0.1 ns per LUT) is dominated by programmable routing, which typically accounts for 50 to 70 % of a path's delay. ASIC logic (picoseconds per gate) is increasingly dominated by wire resistance and capacitance. The FPGA numbers are representative of Xilinx 7-series and Intel Cyclone-class devices as reported by their timing tools; they vary by speed grade and device.

Critical path and timing budget

The **critical path** of a combinational network is the input-to-output path with the largest sum of delays. Static timing analysis finds it without simulation by adding delays along every path, the method used by FPGA tools and ASIC sign-off (Chapters 19 and 34). Two complications matter in practice:

- **False paths.** A structural path that can never be sensitised (for example, through both the "add" and "shift" units of an ALU when the output mux selects only one) is excluded by the designer, or the tool overestimates the delay.
- **Data-dependent delay.** The ripple adder of Section 13.1 takes $2n$ gate delays only for inputs that propagate a carry across every bit; most additions settle much sooner. Synchronous design must budget for the worst case anyway, which is why asynchronous and self-timed circuits, which signal completion, remain an active research topic.

Worked example — derating delay for supply, temperature and load. A 74HC00 in an industrial input module drives four gate inputs and 15 cm of PCB trace, a total load of about 85 pF, at $V_{CC} = 4.5$ V, specified to 85 °C. The datasheet maximum is 23 ns at 50 pF over −40 to 85 °C. Comparing typical delays at 15 pF and 50 pF loads across the family suggests that the output resistance in the delay sense is about 85 Ω, so each added picofarad costs roughly $0.69 \times 85\ \Omega \approx 0.06$ ns. The extra 35 pF adds

$$\Delta t \approx 0.69 \times 85\ \Omega \times 35\ \text{pF} = 2.1\ \text{ns}$$

giving a worst-case delay of about 25 ns, nearly three times the 9 ns typical figure printed on the first page of the datasheet. If the module were redesigned for a 3.3 V supply, a 74HC part would slow by roughly half again (its delay rises steeply below 4.5 V), whereas a 74LVC1G00 would deliver 4.5 ns maximum at 50 pF; at 85 pF the same derating approach, with the LVC part's lower output resistance of roughly 25 Ω, adds about 0.6 ns. The lesson is that the datasheet's headline delay is a typical value at light load and room temperature; the design value is the maximum, derated for the actual load.

Delay is a chain, timing is a budget

Combinational delay is one term in a larger equation. Once the outputs of a block feed a register, the clock period must cover the register's own clock-to-output delay, the combinational delay and the next register's setup time, with margin for clock skew and jitter. That inequality, and its counterpart for hold time that uses t_{cd}, are developed in Section 14.8; glitches in the window between t_{cd} and t_{pd} are the subject of Section 15.5; and the board-level version, where traces add delay and reflections (*Circuits, Signals and Communications*, Ch. 14 §14.5) add ringing, is worked in Section 15.4.

What remains open

Open question	Where it is answered
How are results held between operations, and how fast can a register-to-register path be clocked?	Chapter 14 (registers, setup and hold, timing constraints)
What happens when combinational outputs glitch, and how does a design tolerate it?	Chapter 15, Section 15.5
How does an ALU fit into a processor datapath under control of an instruction decoder?	Chapter 16
How are LUTs, carry chains and routing organised inside an FPGA?	Chapter 18
How are combinational blocks described in Verilog and VHDL and synthesised?	Chapter 19
Where does gate delay come from at transistor level, and how is it sized?	Chapter 33
How are chip-level critical paths found and closed across PVT corners?	Chapter 34
How do long traces turn edges into reflections and ringing?	*Circuits, Signals and Communications*, Ch. 14

Next: Chapter 14 adds memory to logic, building latches, flip-flops, registers, counters and state machines, and turns the propagation delays of this chapter into the setup, hold and clock-period constraints that every synchronous system must meet.

Chapter 14 — Sequential Digital Systems

Part III · Digital Electronics

An e-bike motor controller passes ten thousand bench power cycles and then, in the field, refuses to start about once in every few thousand switch-ons. The rider cycles the power and it works. The fault is traced to the commutation state machine in the controller's FPGA: it is one-hot encoded, and on the bad power-ups two of its state bits are found set at once, a state that has no exit. The reset line from the supply supervisor is asynchronous, and when its release happens to land within a few hundred picoseconds of a clock edge, some flip-flops leave reset on that edge and others on the next. Nothing in the logic equations is wrong. The circuit has memory, memory has timing, and timing has rules that combinational logic never needed: setup and hold windows, metastability, clock domains and reset release.

> **How are latches, flip-flops, registers, counters and shift registers built from gates and feedback, and how are they combined into finite-state machines?**
> **What timing rules must a synchronous circuit obey, how fast can it be clocked, and what happens when a signal crosses between unrelated clocks or a reset is released at the wrong moment?**

Concept	Engineering destination
Latches	Address latches on multiplexed buses, latch-based ASIC timing, accidental latches in HDL
Flip-flops	Every synchronous register in processors, FPGAs and controllers
Registers	Pipeline stages, peripheral registers, register files
Counters	Timers, PWM generators, frequency dividers, event counting
Shift registers	GPIO expansion, serial links, PRBS test patterns, CRC
Finite-state machines	Protocol controllers, motor commutation, sequencing and interlocks
State encoding	Speed and area of FSMs, recovery from illegal states
Timing constraints	Maximum clock frequency, hold fixing, timing closure
Clock-domain considerations	Synchronizers, MTBF, asynchronous FIFOs, multi-clock SoCs
Reset strategies	Reliable start-up, brown-out recovery, FPGA and ASIC reset trees

Roadmap. Memory from feedback: latches (14.1) and flip-flops (14.2) → the standard sequential blocks: registers (14.3), counters (14.4), shift registers (14.5) → the general structure, finite-state machines (14.6), and how their states are encoded (14.7) → the rules that keep them correct: timing constraints (14.8), clock-domain crossings and metastability (14.9) and reset strategies (14.10).

14.1 Latches

> **Boundary.**
> Gates, Boolean algebra and minimisation are from Chapter 12, and propagation and contamination delay from Chapter 13, Section 13.9. The transistor-level construction of latches and flip-flops (transmission-gate latches, keepers, sense-amplifier flip-flops) belongs to Chapter 33; this chapter works at the gate and block level.

The SR latch

Two cross-coupled NOR gates form the simplest memory, the **SR latch** (Figure 14.1, top). With $S = R = 0$ each gate's output holds the other's input, and the pair has two stable states, $Q = 1$ or $Q = 0$. Raising S forces $Q = 1$; raising R forces $Q = 0$. Its next-state behaviour is

$$Q^{+} = S + \overline{R}\,Q, \qquad SR = 0$$

The constraint $SR = 0$ is real: with both inputs high both outputs go low, and if both inputs then fall together, the latch races to an unpredictable state, an early encounter with metastability (Section 14.9). The NAND version has active-low inputs, $\overline{S}$ and $\overline{R}$. SR latches survive in switch debouncers (a changeover switch driving $\overline{S}$ and $\overline{R}$ directly removes contact bounce) and inside every flip-flop.

The D latch

Adding an enable E (often called the clock or gate) and a single data input gives the **D latch**:

$$Q^{+} = E\,D + \overline{E}\,Q$$

While $E = 1$ the latch is **transparent**: Q follows D after a propagation delay. When E falls, Q holds the last value of D. Like any storage element, it needs D stable for a setup time before E falls and a hold time after.

> **Status of the principle: Bistable storage by positive feedback**
> *Fundamental.* Two inverting stages in a loop with loop gain above 1 at the switching point have exactly two stable equilibria and one unstable one; every static memory element, from an SR latch to an SRAM cell, is this circuit. The unstable equilibrium is the origin of metastability.

Where latches are used, and where they are a mistake

The 74HC573 is an octal transparent latch. Its classic job is demultiplexing an address/data bus: an 8051-class microcontroller drives the low address byte on port 0, pulses ALE (address latch enable), and the 74HC573 holds that byte while the same pins carry data. High-performance ASICs use latch-based (two-phase) timing deliberately, because a transparent latch lets a slow logic stage borrow time from the next one. In FPGA design, by contrast, a latch is almost always an accident: an HDL `if` without an `else` in combinational code infers one (Chapter 19), and FPGA timing tools analyse latches poorly. Synthesis warnings about inferred latches should be treated as errors unless the latch was intended.

14.2 Flip-flops

Edge triggering

A **flip-flop** samples its input only at a clock edge. The standard construction is **master–slave**: two D latches in series with opposite enables. While the clock is low the master is transparent and the slave holds; when the clock rises the master closes, capturing D, and the slave opens, passing the captured value to Q. The result changes only on the rising edge:

$$Q^{+} = D \quad \text{at each active clock edge}$$

Figure 14.1 contrasts the two behaviours: the latch output follows every change of D while enabled; the flip-flop output changes once per cycle.

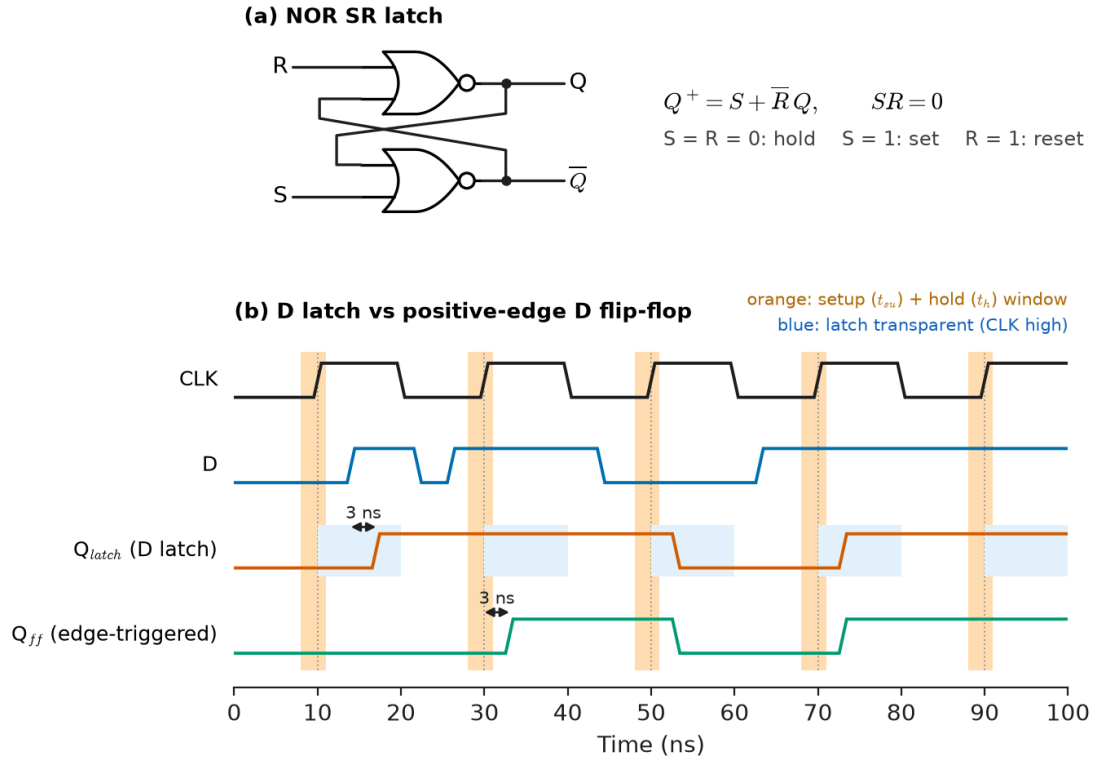


Figure 14.1 — NOR SR latch, and the response of a D latch and a positive-edge D flip-flop to the same data and clock waveforms.

The other flip-flop types are D flip-flops with logic in front:

Type	Characteristic equation	Typical use
D	$Q^+ = D$	Everything synchronous
D with enable	$Q^+ = EN \cdot D + \overline{EN} \cdot Q$	Registers that load only on command
T (toggle)	$Q^+ = T \oplus Q$	Counters, frequency dividers
JK	$Q^+ = J\overline{Q} + \overline{K}Q$	Legacy designs (74HC109, 74HC112)

> **Status of the principle: Flip-flop characteristic equations**
> *Derived.* Each follows from the D flip-flop and the combinational logic in front of it; they describe behaviour at clock edges only, and only when setup and hold times are met.

The timing parameters

A flip-flop is characterised by three times measured from the active clock edge:

- **Setup time** t_{su}: D must be stable this long *before* the edge.
- **Hold time** t_h: D must remain stable this long *after* the edge.
- **Clock-to-output delay** t_{cq} (maximum, for setup analysis) and its minimum or contamination value t_{ccq} (for hold analysis).

Asynchronous set and clear inputs add **recovery** and **removal** times, the setup and hold equivalents for releasing a reset (Section 14.10).

Table 14.1 — Representative flip-flop timing (typical values; check the specific datasheet or timing model).

Element	Conditions	t_{cq}	t_{su}	t_h	Max toggle rate
74HC74	4.5 V, 50 pF	15 ns typ, 44 ns max to 85 °C	20–25 ns min	0–5 ns	25 MHz guaranteed, about 70 MHz typ
74LVC1G79 / 1G80	3.3 V, 50 pF	2–4.5 ns	1–2 ns	0.3–1 ns	about 175 MHz
FPGA slice flip-flop (28 nm)	internal	0.3–0.45 ns	0.03–0.1 ns	0.1–0.3 ns	above 600 MHz (clock-tree limited)
ASIC standard-cell flip-flop (7 nm)	nominal corner	30–50 ps	10–30 ps	5–20 ps	multi-GHz

The "minimum" setup time on a standard-logic datasheet is a *requirement* on the user: the data must arrive at least that early. The 74HC74's typical maximum toggle rate is nearly three times its guaranteed figure, the same typical-versus-maximum gap seen with combinational parts in Chapter 13.

Clock enables and clock gating

A register that should load only sometimes is built with an enable multiplexer in front of D (the second row of the table), not by gating its clock with an AND gate. A gated clock adds delay and skew to that branch of the clock tree and glitches if the enable changes while the clock is high. ASICs do gate clocks to save dynamic power, typically 20–40 % of a block's clock power, but only through **integrated clock-gating cells** that latch the enable while the clock is low. FPGAs provide dedicated clock-enable pins on every flip-flop and global clock buffers with glitch-free enables.

14.3 Registers

A **register** is a group of flip-flops sharing a clock, holding one word. The 74HC574 (octal D flip-flop with tri-state outputs) and 74HC273 (octal D with common clear) are the discrete versions; in an FPGA every logic cell contains one or two flip-flops, so a 32-bit register costs 32 of the tens of thousands available.

Registers appear in three roles:

- **Storage and interface.** Every microcontroller peripheral is controlled through memory-mapped registers: a timer's compare register, a UART's baud-rate divider, a GPIO output register. Their bit-level layout is the subject of Chapters 17 and 21.
- **Pipelining.** Inserting a register in the middle of a long combinational path splits it into two shorter ones, each of which can then run at a higher clock rate, at the cost of one cycle of latency. A 32-bit multiply-accumulate unit in an FPGA DSP slice has optional pipeline registers at its inputs, after the multiplier and at its output; enabling all three raises its maximum clock from roughly 200 MHz to over 450 MHz in a mid-range device.
- **Register files.** A processor's general-purpose registers are an array of registers with two or more read ports (multiplexers selecting a register by number) and one or more write ports (a decoder steering the write enable). A RISC-V core with 32 registers of 32 bits and two read ports needs 992 flip-flops (register x0 is hardwired to zero) and two 32:1 multiplexers per bit (Chapter 16).

A register can also hold a word *serially*, one bit per clock, which is the shift register of Section 14.5.

14.4 Counters

Ripple counters

The simplest counter is a chain of toggle flip-flops in which each stage is clocked by the output of the previous one: a **ripple** (asynchronous) counter. Each stage divides its input frequency by 2, so an n-stage ripple counter divides by 2^n

with no logic at all. The 74HC393 contains two 4-bit ripple counters, and the 74HC4060 is a 14-stage ripple counter with a built-in crystal oscillator, the classic way to derive 2 Hz from a 32.768 kHz watch crystal ($2^{14} = 16\,384$).

The cost is timing. Bit k changes only after bit $k-1$ has changed, so the outputs settle one after another, and the count passes through false intermediate values. Going from 0111 to 1000 the outputs read 0111 → 0110 → 0100 → 0000 → 1000 as the toggle ripples up, each step one flip-flop delay long. Logic that decodes the count (for example, "count = 0000") sees a glitch. Ripple counters are therefore used as frequency dividers and prescalers whose output is a single clock, never as counters whose value is decoded.

Synchronous counters

In a **synchronous** counter every flip-flop shares the same clock and each bit toggles when all lower bits are 1:

$$T_k = Q_0 Q_1 \cdots Q_{k-1}$$

All outputs change together, one t_{cq} after the clock edge, so the count is never wrong once settled. The AND chain is a carry chain, and it limits the clock rate exactly as in the adder of Section 13.1; the 74HC163 (4-bit synchronous binary counter with synchronous clear and load) brings out a ripple-carry output and two enables, ENP and ENT, so that wider counters use lookahead between packages. **Modulo-N** counting is done by synchronously loading or clearing the counter when it reaches $N-1$: a 74HC163 that clears at count 9 is a decade counter. Up/down counters (74HC191, 74HC193) add a direction input that switches between the AND chain of the true outputs and that of the complements.

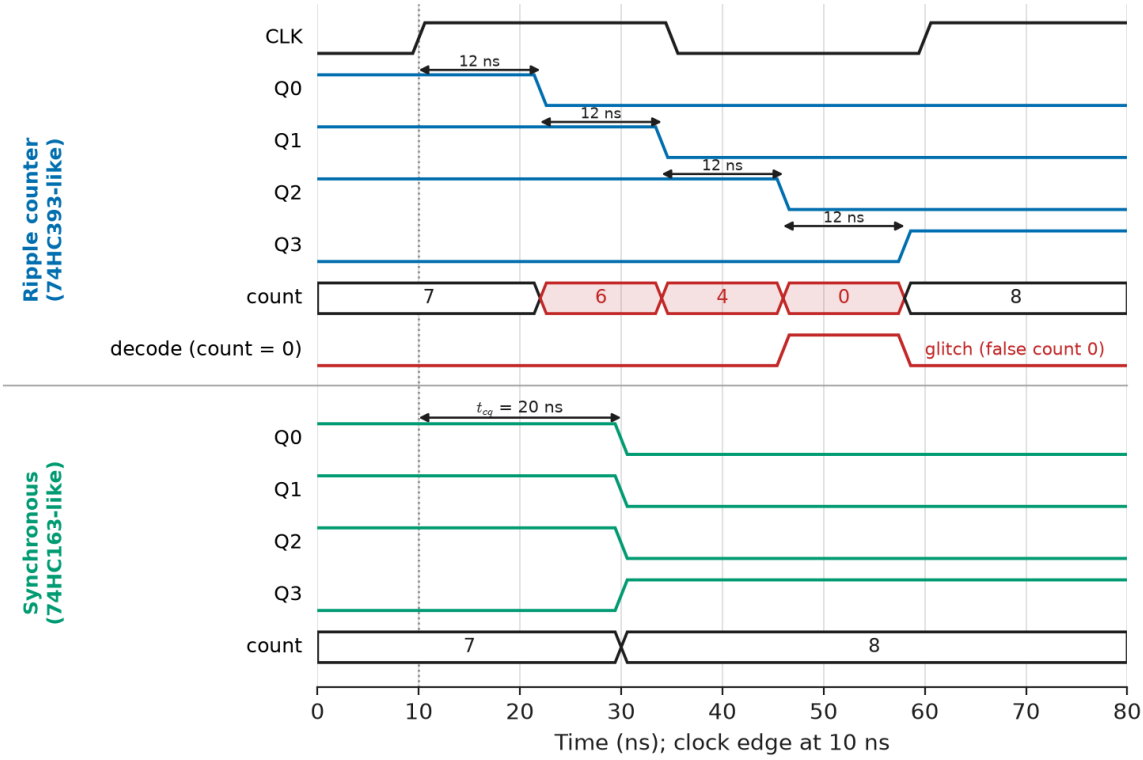


Figure 14.2 — Transition of a 4-bit counter from 7 to 8: outputs of a ripple counter settling in sequence versus a synchronous counter.

Worked example — when is a ripple counter's value valid? An 8-bit ripple counter built from two 4-bit sections with about 12 ns per stage at 4.5 V counts pulses from a flow meter. The worst-case transition, 0111 1111 to 1000 0000, ripples through all eight stages:

$$t_{settle} = 8 \times 12\ \text{ns} = 96\ \text{ns}$$

A processor that reads the counter at an arbitrary moment can catch it mid-ripple. If input pulses arrive no faster than once per 10 µs, reading twice and accepting the value only when two consecutive reads agree is enough, because the counter is stable for over 99 % of the time. A synchronous 74HC163 chain settles within one t_{cq} of about 20 ns typical,

but the reading problem returns in another form whenever the counting clock is unrelated to the reader's clock, which is a clock-domain crossing (Section 14.9) and is solved with Gray-coded counters.

Counters in practice

Every microcontroller timer is a synchronous counter with compare and capture registers (Chapter 17). A PWM output is a counter compared against a duty register; a 16-bit timer clocked at 64 MHz gives a PWM frequency of $64\ \text{MHz}/2^{16} = 977$ Hz at full resolution, or 20 kHz with a period of 3200 counts, about 11.6 bits. **Johnson counters** (a shift register with inverted feedback) produce $2n$ states with n flip-flops and glitch-free decoding, used for multiphase clock generation, and **prescalers** in frequency synthesisers are high-speed counters, often dual-modulus (N or $N+1$) to give fine frequency steps.

14.5 Shift registers

Serial and parallel conversion

A **shift register** is a chain of D flip-flops, each feeding the next, all sharing a clock. On each edge the stored word moves one position. Two variants carry most of the traffic:

- **Serial-in, parallel-out** (SIPO). The 74HC595 shifts in one bit per clock and has a separate storage register that updates all eight outputs at once on a latch strobe, so the outputs never show the shifting. Three microcontroller pins (data, shift clock, latch) then drive any number of outputs by daisy-chaining packages; at a typical 10 MHz SPI clock, 32 outputs refresh in 3.2 µs. Its typical shift-clock limit is about 100 MHz at 4.5 V, with a guaranteed figure near 25 MHz.
- **Parallel-in, serial-out** (PISO). The 74HC165 loads eight inputs at once and shifts them out, the input-expansion counterpart, used for reading DIP switches and limit switches.

Serial interfaces are built on the same structure: the shift register in an SPI peripheral, a UART's transmit and receive registers, and the serialiser/deserialiser (SerDes) of a multi-gigabit link. Their protocols belong to Chapter 26.

Linear-feedback shift registers

Feeding the XOR of selected stages back into the input gives a **linear-feedback shift register** (LFSR). If the feedback taps correspond to a *primitive* polynomial over GF(2), the register cycles through all $2^n - 1$ non-zero states before repeating, a **maximal-length sequence**. LFSRs generate pseudo-random bit sequences for testing serial links (PRBS7, PRBS15, PRBS31 in ITU-T O.150 and in every SerDes test mode), scramble data to guarantee transitions, spread the spectrum of clocks, and compute cyclic redundancy checks, where the message is fed through an LFSR whose final state is the CRC. They are also the cheapest counters in hardware when only the sequence length matters, since they need no carry chain.

```
def lfsr_period(taps, n, seed=1):
    """Fibonacci LFSR: feedback = XOR of tapped bits (1-based from the left)."""
    state, steps = seed, 0
    while True:
        fb = 0
        for t in taps:
            fb ^= (state >> (n - t)) & 1
        state = ((state >> 1) | (fb << (n - 1))) & ((1 << n) - 1)
        steps += 1
        if state == seed:
            return steps

print("PRBS7  x^7+x^6+1  :", lfsr_period((7, 6), 7))
print("       x^7+x^5+1  :", lfsr_period((7, 5), 7))
print("PRBS15 x^15+x^14+1:", lfsr_period((15, 14), 15))
```

```
 PRBS7  x^7+x^6+1  : 127
       x^7+x^5+1  : 93
PRBS15 x^15+x^14+1: 32767
```

The PRBS7 and PRBS15 polynomials give the full $2^7 - 1 = 127$ and $2^{15} - 1 = 32\,767$ states. The polynomial $x^7 + x^5 + 1$ factors over GF(2) as $(x^2 + x + 1)(x^5 + x^4 + x^3 + x + 1)$, so it is not primitive, and from this seed the register cycles through only 93 states. Tap selection is taken from published tables, never guessed.

> **Status of the principle: Maximal-length LFSR sequences**
> *Derived.* A consequence of the algebra of polynomials over GF(2): an n-stage LFSR has period $2^n - 1$ exactly when its feedback polynomial is primitive. The sequence is deterministic; it is "random" only in its balance and autocorrelation properties.

14.6 Finite-state machines

The general sequential circuit

Counters and shift registers are special cases of the **finite-state machine** (FSM): a state register of k flip-flops, combinational **next-state logic** that computes the next state from the present state and the inputs, and combinational **output logic** (Figure 14.3). On each clock edge the register loads the next state. Two variants differ in where the outputs come from:

- In a **Moore machine** the outputs depend only on the present state. They change only after a clock edge and are free of combinational paths from inputs, which makes them glitch-free if registered and easy to time.
- In a **Mealy machine** the outputs depend on the present state and the present inputs. They respond within the same cycle, often with fewer states, but an input glitch can pass straight through to an output, and a Mealy output feeding another Mealy machine can create a long combinational path or even a combinational loop between the two.

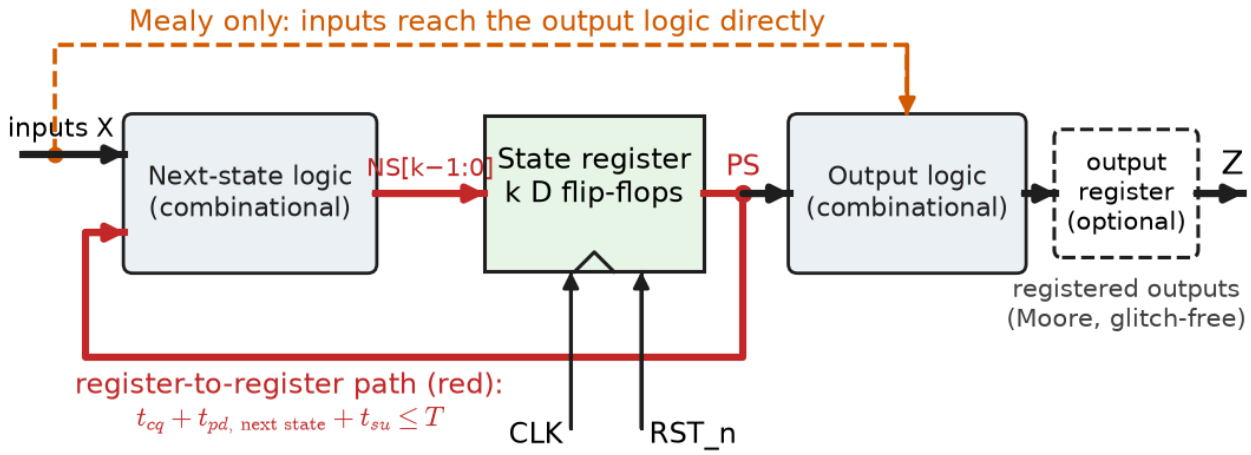


Figure 14.3 — Structure of a synchronous finite-state machine in Moore and Mealy forms.

The design procedure, from a specification through a state diagram and state table to logic, is the subject of Chapter 15. What matters here is that every FSM is a register plus combinational logic, so everything in this chapter's timing sections applies to it: its clock period is set by the longest path from the state register through the next-state logic and back.

Why FSMs are everywhere

An FSM is the hardware form of a sequence: wait for an event, act, wait for the next. Examples include the commutation sequencer of a brushless-DC motor controller (six states selected by Hall sensors), the bus controller of an I²C peripheral (start, address, acknowledge, data, stop), the cache controller of a processor (idle, compare tag, write back, allocate), the interlock logic of a medical X-ray generator (door closed, armed, exposing, cooling) and the training sequence of a DDR memory controller. Software implements the same idea with a `switch` statement over a state variable (Chapter 28); hardware FSMs are used when the reaction must be deterministic to the clock cycle, or when the logic must keep working while the processor is busy or crashed.

14.7 State encoding

The choices

A machine with N states needs at least $\lceil \log_2 N \rceil$ flip-flops, but the minimum is not always best. Common encodings:

Encoding	Flip-flops for N states	Next-state logic	Typical use
Binary	$\lceil \log_2 N \rceil$	Dense, deep	Small ASIC FSMs, area-critical designs
Gray	$\lceil \log_2 N \rceil$	Similar to binary	Sequential state flows; low switching activity
One-hot	N	Shallow, one flip-flop per state	FPGAs (flip-flops are cheap, LUT depth is costly)
Johnson	$\lceil N/2 \rceil$	Simple decoding	Counters and sequencers with fixed cyclic order

In a **one-hot** machine each state has its own flip-flop, and the next-state logic for a state is the OR of the conditions on its incoming transitions; the output decode for a state is a single wire. The logic per flip-flop is small, fits in one LUT, and is fast. Synthesis tools for FPGAs choose one-hot automatically for machines above a handful of states unless told otherwise, and ASIC tools usually choose binary or Gray.

Worked example — encoding a 10-state FSM. A packet-header parser in an FPGA network interface has 10 states.

- Binary encoding uses $\lceil \log_2 10 \rceil = 4$ flip-flops. Each next-state bit is a function of the 4 state bits and several inputs; with, say, 4 input conditions, each bit depends on 8 signals, more than a 6-input LUT holds, so each next-state function needs two LUT levels. There are $2^4 - 10 = 6$ unused codes.
- One-hot encoding uses 10 flip-flops. A state with three incoming transitions needs the three predecessor bits and their conditions, typically 6 or fewer signals, one LUT level. The unused codes number $2^{10} - 10 = 1014$.

At about 0.11 ns per LUT and 0.5 ns per route in a 28 nm FPGA, saving one LUT level and its route saves roughly 0.6 ns, which at a 4 ns clock period is 15 % of the cycle. The price is six more flip-flops, which in a device with tens of thousands of them is negligible, and many more illegal codes.

Illegal states and safe machines

Unused codes are not a theoretical worry. A single-event upset (a particle strike flipping a flip-flop, a real rate in avionics, automotive and data-centre equipment), a supply glitch or a badly released reset (Section 14.10) can put the register into an unused code. What happens next depends on the next-state logic for that code, which synthesis optimised assuming it never occurs. A one-hot machine that finds itself with two bits set, or none, typically stays there: the lock-up in this chapter's opening problem.

A **safe** state machine defines a transition from every unused code to a known recovery state, or at least detects the illegal condition (for one-hot, "not exactly one bit set") and forces a reset. Synthesis tools offer a "safe" FSM attribute that adds this logic, sometimes with Hamming-distance-3 encodings that correct single-bit errors. Functional-safety standards (ISO 26262 for road vehicles, IEC 61508 for industrial control) expect such detection for state machines in safety functions.

14.8 Timing constraints

The setup constraint

Consider a path from a launching flip-flop through combinational logic to a capturing flip-flop, both on the same clock of period T (Figure 14.4). Data leaves the launching flip-flop t_{cq} after the clock edge, passes through logic with maximum delay t_{pd}, and must arrive t_{su} before the *next* edge at the capturing flip-flop. If the capture clock arrives t_{skew} later than the launch clock (positive skew helps setup) and the clock edges wander by up to t_{jit}, the constraint is

$$t_{cq} + t_{pd} + t_{su} \le T + t_{skew} - t_{jit}$$

The largest t_{pd} over all register-to-register paths sets the maximum clock frequency, $f_{max} = 1/T_{min}$. The difference between the two sides is the **setup slack**; timing tools report it per path, and a negative value is a **setup violation**, which shows up as wrong data at high clock rates and is cured by a slower clock.

The hold constraint

The second constraint concerns the *same* edge. When the clock rises, the launching flip-flop starts changing its output after its contamination delay t_{ccq}, and the change reaches the capturing flip-flop after the logic's contamination delay t_{cd}. It must not arrive before the capturing flip-flop's hold time has passed:

$$t_{ccq} + t_{cd} \ge t_h + t_{skew}$$

Here positive skew *hurts*. The hold constraint does not contain T, so a hold violation cannot be cured by slowing the clock; the chip fails at every frequency. It is fixed by adding delay to the short path (buffers in an ASIC, routing detours in an FPGA, which the tools do automatically) or by reducing skew.

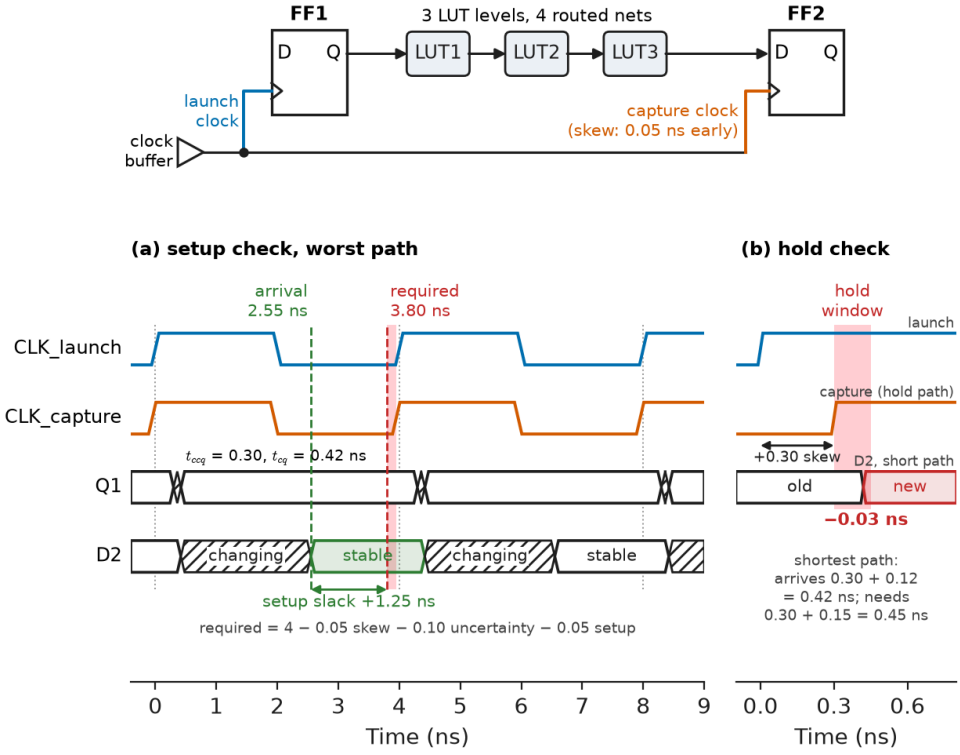


Figure 14.4 — Register-to-register path with the setup and hold constraints at a 250 MHz clock.

> **Status of the principle: Setup and hold constraints of synchronous logic**
> *Derived.* Direct consequences of the flip-flop's sampling window and the maximum and minimum delays of the logic between flip-flops; they are exact given correct delay bounds. Their accuracy rests on the delay models, which are approximate (Section 13.9), so tools add margins for on-chip variation and clock uncertainty.

Worked example — the maximum clock of a 74HC data path. A legacy accumulator clocks a 74HC74 register through a 74HC283 adder back into the register. Worst-case values at 4.5 V over −40 to 85 °C are $t_{cq} = 44$ ns, adder $t_{pd} \approx 50$ ns, $t_{su} = 25$ ns, and 2 ns of adverse clock skew between packages:

$$T_{min} = 44 + 50 + 25 + 2 = 121 \text{ ns}, \qquad f_{max} = 8.3 \text{ MHz}$$

With typical values (15 ns, 20 ns, about 6 ns, no skew) the sum is 41 ns and the circuit runs at 24 MHz on the bench. A design that is clocked at 20 MHz because it "works" fails in production on slow parts or at temperature. The hold check is easy here: the 74HC74's hold time is near zero and its minimum clock-to-output delay is several nanoseconds, so any path is long enough.

Worked example — setup and hold slack in an FPGA at 250 and 400 MHz. A path in a 28 nm FPGA has three LUT levels. Representative values from the timing report: $t_{cq} = 0.42$ ns, three LUTs at 0.11 ns each, four routed nets averaging 0.45 ns, $t_{su} = 0.05$ ns, clock uncertainty (jitter and margin) 0.10 ns, and a capture clock arriving 0.05 ns earlier than the launch clock:

$$t_{path} = 0.42 + 3(0.11) + 4(0.45) + 0.05 + 0.10 + 0.05 = 2.75 \text{ ns}$$

At 250 MHz ($T = 4$ ns) the setup slack is $+1.25$ ns. At 400 MHz ($T = 2.5$ ns) it is -0.25 ns: the path fails. Routing, not logic, is 65 % of the delay, so the usual cures are a pipeline register after the second LUT, packed into the same slice so that no routed net is added (two paths of about 1.7 ns and 1.6 ns), placement constraints that keep the logic close together, or one fewer logic level by restructuring.

For hold, take the shortest path in the same design: a flip-flop driving another directly through one short net. With $t_{ccq} = 0.30$ ns and a minimum route of 0.12 ns, data arrives 0.42 ns after the edge. The capturing flip-flop has $t_h = 0.15$

ns and, on this path, its clock arrives 0.30 ns *late* because of the clock-tree topology:

$$\text{hold slack} = (0.30 + 0.12) - (0.15 + 0.30) = -0.03 \text{ ns}$$

The router fixes this automatically by detouring the net by a few tens of picoseconds; in an ASIC, the hold-fixing step inserts delay buffers. In a design with many clocks or hand-built clock gating, large skews can create hold violations that no amount of slowing down can fix.

Clock distribution

The constraints show why clocks are distributed through special networks. FPGAs have dedicated global clock trees with skew of tens to a few hundred picoseconds across the die; ASICs use balanced H-trees, meshes or clock-tree synthesis. On boards, a clock distributed to several chips through a fan-out buffer (for example a CDCLVC1104) has output-to-output skew of 50–100 ps, but trace length differences add about 6–7 ps per millimetre of FR-4. Clock jitter, from the oscillator and from supply noise in PLLs, subtracts directly from the period; a 100 MHz crystal oscillator has a few picoseconds of RMS jitter, a PLL output tens.

14.9 Clock-domain considerations

Metastability

Every flip-flop assumes that its input meets setup and hold. A signal from outside the clock domain, such as a push button, a sensor's data-ready line, or data from a circuit clocked by a different oscillator, will sooner or later change inside the sampling window. The flip-flop's internal bistable loop is then left near its unstable balance point (Section 14.1) and may take an unbounded time to fall to 0 or 1. During that time its output can sit at an intermediate voltage, oscillate, or resolve late. This is **metastability**. It cannot be prevented by any circuit; it can only be made rare.

Near the balance point the loop behaves as a linear amplifier with positive feedback, so the voltage difference between its nodes grows exponentially with a time constant τ set by the loop's transconductance and capacitance. The probability that the flip-flop is still unresolved a time t_r after the clock edge therefore falls as $e^{-t_r/\tau}$ (Figure 14.5). Combined with the rate at which data transitions land in the window, this gives the **mean time between failures** of a synchronizer:

$$\text{MTBF} = \frac{e^{t_r/\tau}}{T_0 \, f_{clk} \, f_{data}}$$

where f_{clk} is the sampling clock frequency, f_{data} is the rate of data transitions, T_0 is an effective window width, and t_r is the **resolution time** allowed: the time between the clock edge and the moment the output is next sampled, minus that next element's setup time. Failure events are rare and independent, so the time to the first failure is exponentially distributed and reliability over a period t is $e^{-t/\text{MTBF}}$ (*The Mathematics Universe*, Ch. 19 §19.4).

> **Status of the principle: Synchronizer MTBF formula**
> *Approximate*. The exponential resolution follows from linearising the bistable loop about its unstable point; τ and T_0 are fitted to measurements of each flip-flop and vary with supply voltage and temperature. The formula is accurate for resolution times of many τ, but is not valid at very short t_r, and τ can grow sharply at low supply voltage.

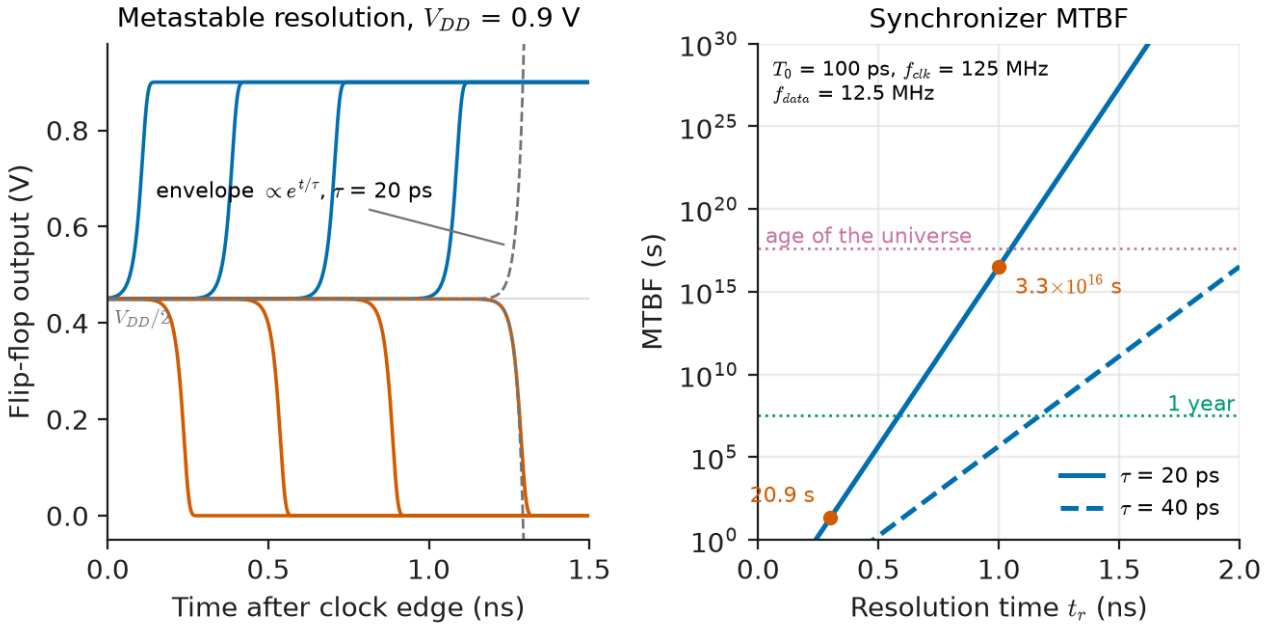


Figure 14.5 — Metastable resolution of a flip-flop and synchronizer MTBF versus resolution time.

Table 14.2 — Order-of-magnitude metastability parameters.

Technology	τ	Comment
5 V standard logic of the 1980s–90s	hundreds of ps	Measured per family in vendor application notes
28 nm FPGA flip-flop, nominal supply	10–30 ps	Some vendors publish MTBF calculators
Advanced-node ASIC flip-flop, nominal supply	5–15 ps	Rises strongly at low supply and low temperature
Dedicated synchronizer cell	below typical τ	High-gain cells offered in standard-cell libraries

Worked example — MTBF of a one-flip-flop and a two-flip-flop synchronizer. An FPGA in an industrial sensor hub samples an asynchronous data-valid line that toggles at up to 12.5 MHz with a 125 MHz clock (8 ns period). Take $\tau = 20$ ps and $T_0 = 100$ ps as representative of a 28 nm flip-flop, $t_{cq} = 0.40$ ns and $t_{su} = 0.10$ ns.

```
 import math

def mtbf(t_res, tau, t0, f_clk, f_data):
    """Synchronizer MTBF in seconds: exp(t_res/tau) / (T0 * f_clk * f_data)."""
    return math.exp(t_res / tau) / (t0 * f_clk * f_data)

tau, t0 = 20e-12, 100e-12          # representative 28 nm flip-flop
f_clk, f_data = 125e6, 12.5e6      # 8 ns clock, data toggling at 12.5 MHz
T, t_cq, t_su = 1 / f_clk, 0.40e-9, 0.10e-9
year = 3.156e7

cases = {
    "1 FF, 0.3 ns slack": 0.30e-9,
    "1 FF, 1.0 ns slack": 1.00e-9,
    "2 FF, 0.3 ns route": T - t_cq - t_su - 0.30e-9,
}
for name, t_res in cases.items():
    m = mtbf(t_res, tau, t0, f_clk, f_data)
    print(f"{name:20s} t_res={t_res*1e9:5.2f} ns  MTBF={m:9.3g} s = {m/year:9.3g} yr")
print("gain per extra 100 ps:", round(math.exp(100e-12 / tau)))
```

```
 1 FF, 0.3 ns slack   t_res= 0.30 ns  MTBF=     20.9 s =  6.63e-07 yr
1 FF, 1.0 ns slack   t_res= 1.00 ns  MTBF= 3.32e+16 s =  1.05e+09 yr
2 FF, 0.3 ns route   t_res= 7.20 ns  MTBF=1.42e+151 s =  4.5e+143 yr
gain per extra 100 ps: 148
```

If the asynchronous signal feeds a single flip-flop whose output goes straight into logic with only 0.3 ns of slack left in the cycle, the circuit fails about every 21 seconds. With 1 ns of slack the figure jumps to 10^9 years, because every extra 100 ps of resolution time multiplies the MTBF by $e^5 = 148$. That sensitivity cuts both ways: if a low supply or a hot corner doubles τ to 40 ps, the 1 ns case falls to $e^{25}/(1.56 \times 10^5) = 4.6 \times 10^5$ s, about 5.3 days. And a fleet multiplies the failure rate: a million units each containing 20 such synchronizers at 10^9 years apiece see a failure somewhere every $1.05 \times 10^9/(2 \times 10^7) \approx 52$ years, but at 5.3 days apiece the fleet sees millions of failures per day.

The two-flip-flop synchronizer gives the first flip-flop an almost full clock period, 7.2 ns, to resolve before the second samples it, and its MTBF is beyond any physical meaning. That is why it is the universal rule: an asynchronous single-bit signal passes through two (at high clock rates or low voltages, three) flip-flops in series, placed close together with no logic between them, before any logic uses it. FPGA tools have attributes (for example `ASYNC_REG` in Xilinx tools) that keep synchronizer flip-flops adjacent and tell the timing and MTBF analysis what they are.

Crossing more than one bit

A two-flip-flop synchronizer handles one bit. Synchronising each bit of a multi-bit bus separately is a classic error: the bits resolve independently, so on a transition the receiving side can capture some bits from the old value and some from the new, a word that was never sent. The standard techniques are:

- **Gray-coded counters.** If the value changes by at most one count per source clock, Gray-code it (Section 13.8) so that only one bit changes, synchronise each bit, and convert back. A mis-sampled transition yields the old or new value, never a wild one.
- **Handshake (request–acknowledge).** The sender holds the data stable, toggles a request line; the receiver synchronises the request, captures the (stable) data, and returns a synchronised acknowledge. It costs several clock cycles per transfer in each domain and suits slow control words.
- **Pulse synchronisers.** A single-cycle pulse in a fast domain can be missed entirely by a slow domain. It is converted to a level toggle, the toggle synchronised, and an edge detector in the destination domain regenerates a pulse.

- **Asynchronous FIFOs.** For streams of data, a dual-port memory is written in one domain and read in the other. The write and read pointers are kept in Gray code and each is synchronised into the opposite domain to compute "full" and "empty". Because a synchronised pointer is a few cycles old, full and empty are pessimistic (safe), never optimistic.

Worked example — sizing an asynchronous FIFO. A camera interface writes bursts of 256 pixels at 100 MHz into a FIFO read by a processing pipeline that consumes one pixel per cycle at 80 MHz. During a burst of $256/100\ \text{MHz} = 2.56\ \mu\text{s}$ the reader removes $2.56\ \mu\text{s} \times 80\ \text{MHz} = 204.8$ pixels, so the backlog peaks at

$$256\left(1 - \frac{80}{100}\right) = 51.2 \text{ words}$$

The pointer synchronisers add about two to three cycles of latency in each direction, during which the full flag lags reality, so the minimum depth is about 57 words. Gray-coded pointers need a power-of-two depth, so the design uses 64 words. If the reader can stall, the FIFO must instead cover the longest stall, or the writer must obey back-pressure.

Clock-domain discipline

A multi-clock design (a typical SoC has a dozen clocks: processor, bus, memory controller, each peripheral, a 32.768 kHz real-time clock) must list every crossing and assign each a technique. CDC verification tools check the structure statically, flagging crossings without synchronizers and multi-bit buses synchronised bit by bit, because simulation alone will not reveal metastability. The timing paths through synchronizers are declared asynchronous or constrained with maximum-delay limits rather than analysed as ordinary paths. The HDL form of a synchronizer is shown in Chapter 15, Section 15.6.

14.10 Reset strategies

Power-on reset and supervisors

A digital system must start in a known state. At power-up, flip-flops wake in random states and the supply ramps through voltages at which logic is not yet valid. A **supply supervisor** (for example a TPS3839 or MAX809-class part) holds the reset line asserted until V_{CC} has been above its threshold for a timeout, typically 0.2–240 ms depending on the part, and reasserts it if the supply dips below the threshold (brown-out). Microcontrollers integrate power-on and brown-out reset circuits with thresholds around 1.7–2.9 V selectable for a 3.3 V system, and a watchdog timer that resets the device if software stops servicing it (Chapter 27). An RC network on a reset pin is a weaker substitute: it does not reassert on a short dip.

Synchronous and asynchronous reset

Inside the logic, the reset reaches flip-flops in one of two ways:

- A **synchronous reset** is a data input: the flip-flop clears on the next clock edge while reset is asserted, through logic in front of D. It needs a running clock, adds logic to the data path, and is timed like any other signal, so it is immune to short glitches on the reset line.
- An **asynchronous reset** acts directly on the flip-flop's clear input, without a clock. It works when the clock is stopped or not yet stable, which matters at power-up and in safety outputs that must go inactive immediately.

The asynchronous form is safe to *assert* at any time, but dangerous to *release* asynchronously. Releasing reset is like an asynchronous data change: if it occurs within a flip-flop's **recovery** time before a clock edge or its **removal** time after it, that flip-flop may leave reset on this edge, the next edge, or go metastable. Across a large design with a reset network spanning the chip, some flip-flops then start one cycle before others. In a counter that costs one count; in a one-hot FSM it can leave two state bits set, which is exactly the e-bike controller's fault.

Asynchronous assert, synchronous deassert

The standard solution combines both: a **reset synchronizer** of two flip-flops whose asynchronous clears are driven by the external reset and whose data input is tied to logic 1 (Figure 14.6). When the external reset asserts, both flip-flops clear at once and the internal reset asserts immediately, without a clock. When the external reset releases, the internal reset stays asserted until a 1 has shifted through both flip-flops, so it releases synchronously with the clock, two cycles later, and with the same metastability protection as a data synchronizer. The internal reset's release is then timed by static timing analysis as a recovery/removal check across the whole reset tree.

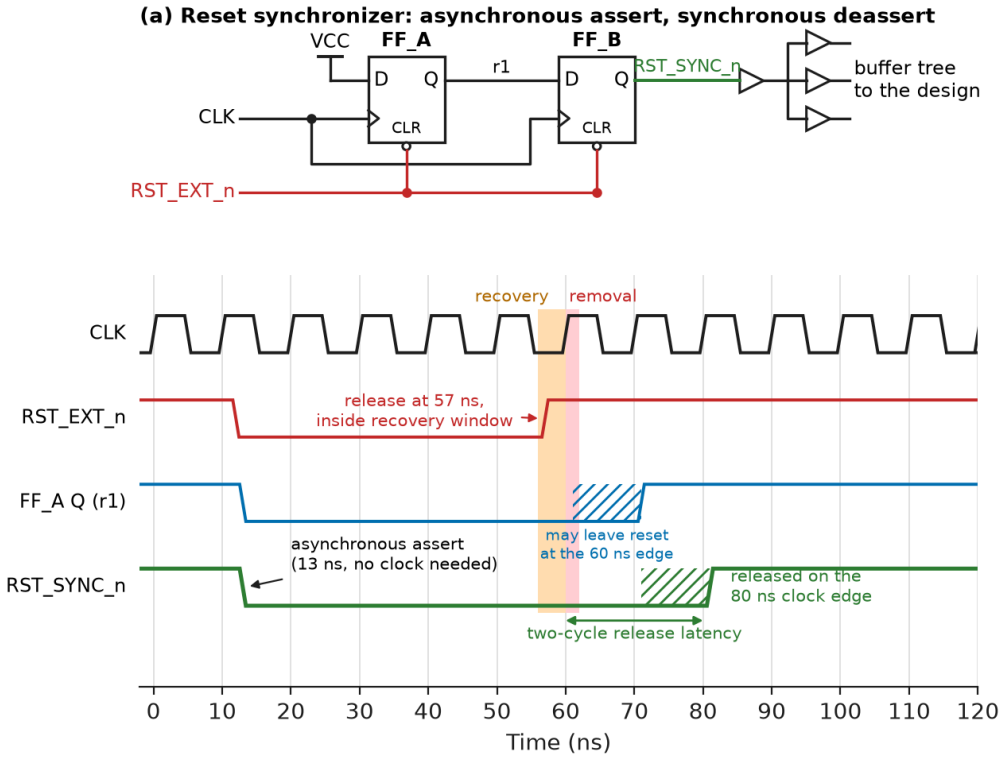


Figure 14.6 — Reset synchronizer with asynchronous assertion and synchronous deassertion, and its waveforms.

Each clock domain needs its own reset synchronizer, and systems that must release domains in order (for example, memory controller before processor) sequence the synchronizer outputs. In a design with a high-fanout reset, the synchronised reset is distributed through a buffered tree or a dedicated global network, and sometimes pipelined through extra registers so that its timing can be met at full clock rate.

Which flip-flops need a reset

Not every flip-flop does. Control state (FSM state registers, valid bits, counters that define protocol behaviour) must be reset. Datapath registers whose contents are overwritten before use (pipeline data, FIFO storage, DSP delay lines) usually need not be. Leaving them unreset saves routing and lets FPGA tools pack logic into shift-register LUTs and DSP blocks that have no reset pins. FPGAs also initialise every flip-flop to its configured value at the end of configuration through a global set/reset (GSR); for many FPGA designs that start once after configuration, that initial value plus a synchronised reset for the control logic is the recommended practice, and vendor guides advise against using the GSR network itself as a functional reset because its release is not synchronised to user clocks.

> **Status of the principle: Asynchronous-assert, synchronous-deassert reset**
> *Derived.* Assertion needs no clock and is harmless at any time; deassertion through a synchronizer turns the reset release into a synchronous event obeying recovery and removal constraints, with the MTBF of a two-flip-flop synchronizer. It does not protect against glitches on the external reset line, which need a filter or a supervisor with a clean output.

What remains open

Open question	Where it is answered
How is an FSM designed from a specification through state diagrams and state tables?	Chapter 15, Sections 15.1–15.3
How are synchronizers, reset synchronizers and FSMs written in Verilog and verified?	Chapter 15, Sections 15.6 and 15.9; Chapter 19
How are board-level clock and I/O timing budgets drawn up?	Chapter 15, Section 15.4
How are registers, register files and pipelines organised in a processor?	Chapter 16
How do microcontroller timers, counters and watchdogs use these blocks?	Chapter 17; Chapter 27
How are latches and flip-flops built from CMOS transistors, and what sets τ?	Chapter 33
How are timing constraints closed across corners on a full chip, including clock-tree synthesis?	Chapter 34

Next: Chapter 15 turns these elements into a design method: from specification to state diagram, state table and logic, with board-level timing budgets, glitch control, synchronization in HDL and a self-checking test bench.

Chapter 15 — Digital System Design

Part III · Digital Electronics

The interlock controller of an industrial laser cutter is a four-state machine in a small FPGA. It passed every functional simulation and a week of factory acceptance tests. Three months after installation, a service engineer reports that the laser occasionally fires a single shot when a chiller fault trips, with the door open. The state machine never enters its FIRE state; the logic equations are correct. The laser-enable output is decoded combinationally from two state bits, and when a fault forces the state from ARMED (01) to FAULT (10), both bits change and for about half a nanosecond the decoder sees 11, which is FIRE. The laser driver's input is edge-triggered and latches that pulse into a 2 ms shot. Every block in the design was right; the *design* was not. This chapter is about the method that prevents such faults: specifying behaviour precisely, turning it into state diagrams and tables, implementing it in hardware that respects timing, and verifying it with test benches before it reaches a machine.

> **How is a digital system taken from a written specification to verified hardware, and which rules make the result robust?**
> **How do glitches, asynchronous inputs and timing budgets break designs whose logic is correct, and how are they designed out and tested for?**

Concept	Engineering destination
Design methodology	Requirements to bitstream for FPGA and ASIC projects, safety-certified hardware
State diagrams	Specifying controllers, protocol engines and interlocks unambiguously
State tables	Deriving and checking next-state and output logic
Hardware timing	Internal timing closure and board-level I/O budgets
Glitches	Glitch-free outputs to lasers, gate drivers, clocks and resets; glitch power
Synchronization	Push buttons, sensors and data-ready lines entering a clocked design
Memory elements	Choosing flip-flops, LUT RAM, block RAM or external memory
Logic implementation	Choosing between discrete logic, CPLD, FPGA, ASIC and a microcontroller
Verification and test benches	Self-checking simulation, coverage, assertions, sign-off

Roadmap. The overall method (15.1) → specification as state diagrams (15.2) and state tables leading to logic (15.3) → the physical constraints on that logic: hardware timing (15.4), glitches (15.5) and synchronization of the outside world (15.6) → implementation choices: memory elements (15.7) and the technology that carries the logic (15.8) → proving that it works: verification and test benches (15.9).

15.1 Design methodology

> **Boundary.**
> Combinational blocks and propagation delay are from Chapter 13; flip-flops, FSM structure, state encoding, setup and hold, metastability and reset synchronizers from Chapter 14. Hardware description languages are covered in depth in Chapter 19; the Verilog here is kept to the few constructs needed to show the method, in Verilog-2001.

From requirement to hardware

A digital design project moves through a sequence of representations, each more concrete than the last, with a check at every step (Figure 15.1):

1. **Requirements.** What the system must do, in the user's terms: "the laser must not emit while the door is open", "the encoder interface must count at 2 MHz edge rate", "the system must recover from any single fault without a power cycle".
2. **Specification.** The requirements made precise and testable: every input and output with its timing, every operating mode, the response to each fault, the reset behaviour, the clock frequencies and interfaces.
3. **Architecture.** The partition into a **datapath** (registers, adders, multiplexers, memories that move and transform data) and a **control unit** (one or more FSMs that sequence the datapath), plus clock domains and their crossings.
4. **Behavioural description.** State diagrams (Section 15.2) or algorithmic state machine (ASM) charts for the controllers; block diagrams with widths for the datapath.
5. **Register-transfer-level (RTL) code** in an HDL, which says what each register loads on each clock edge.
6. **Functional verification** in simulation against the specification (Section 15.9).
7. **Synthesis and implementation.** Logic synthesis maps the RTL to gates or LUTs (Section 15.8), placement and routing put them on the die, and static timing analysis checks every path (Section 15.4).
8. **Hardware bring-up and validation** in the real system, with the measurements that simulation cannot give: signal integrity, power, temperature.

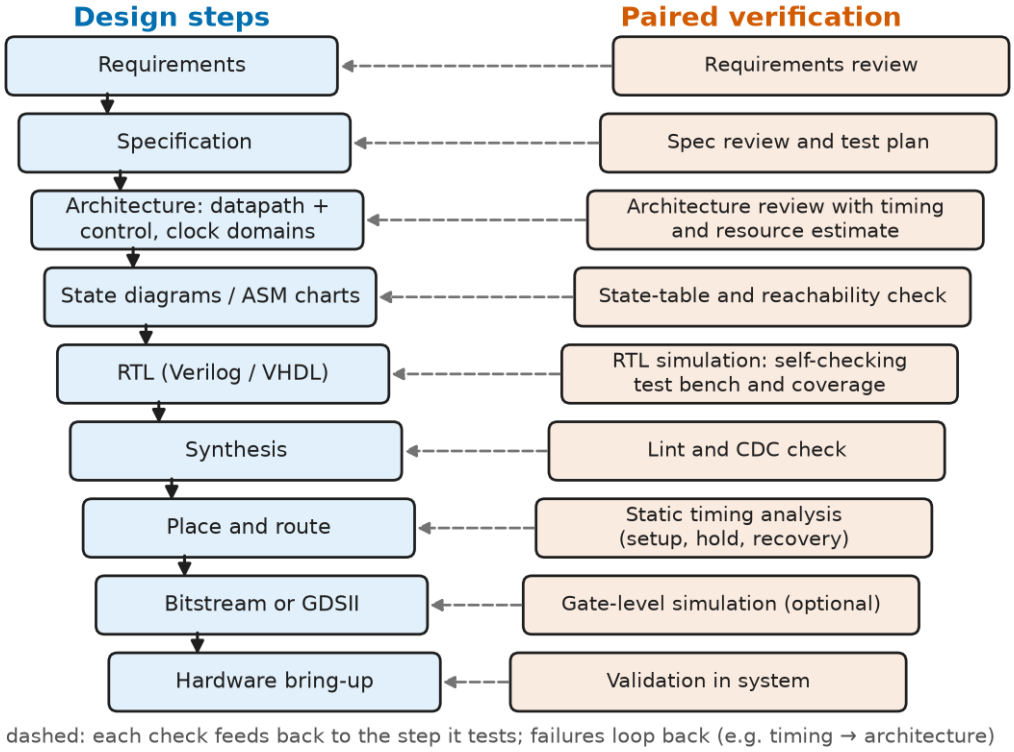


Figure 15.1 — Digital design flow from requirements to validated hardware, with the verification activity paired with each step.

The flow is iterative, not a waterfall: a timing failure after routing may send the designer back to the architecture to add a pipeline stage. What makes it a *method* is that each representation is checked against the one above it before work continues, and that the checks are planned at the same time as the design. Safety-critical sectors formalise this: DO-254 for airborne electronic hardware, ISO 26262 for road vehicles and IEC 61508 for industrial safety functions all require traceability from each requirement to the design element that implements it and the test that verifies it.

The synchronous design rules

Most robustness problems in digital hardware come from breaking a short list of rules. They follow from the timing analysis of Chapter 14, which is valid only when they hold:

- **One clock edge per domain.** All flip-flops in a domain are clocked by the same edge of the same clock, distributed through the clock network, never through logic.
- **No gated or derived clocks in logic.** Use clock enables, or dedicated clock-management blocks (PLLs, clock buffers with enables).
- **Every asynchronous input is synchronised once** (Section 15.6), and every clock-domain crossing uses a defined technique (Section 14.9).
- **Reset released synchronously** in each domain (Section 14.10).
- **No combinational loops and no unintended latches.**
- **Outputs that drive edge-sensitive or safety-critical inputs come straight from flip-flops** (Section 15.5).

> **Status of the principle: Synchronous design discipline**
> *Derived.* If every storage element samples on one clock edge and every path meets setup and hold, the circuit's behaviour is fully described by its state table regardless of glitches between edges; the rules above are exactly the conditions under which static timing analysis can guarantee that. Breaking any one of them puts behaviour outside what the tools can check.

15.2 State diagrams

Notation

A **state diagram** draws an FSM as a directed graph. Each state is a circle (or rounded box) labelled with its name and, for a Moore machine, the outputs asserted in that state. Each transition is an arrow labelled with the input condition under which it is taken at the next clock edge; for a Mealy machine, arrow labels also carry outputs, written condition/output. A transition back to the same state can be drawn as a loop or left implicit ("otherwise, stay").

For the laser interlock, the specification gives four synchronised inputs, D (door closed), K (key switch on), F (fire request) and E (fault from the chiller or beam monitor), and four states (Figure 15.2):

- **SAFE**: laser off. Go to ARMED when $D \cdot K$.
- **ARMED**: ready lamp on. Go to SAFE if the door opens or the key turns off; go to FIRE on F.
- **FIRE**: laser enabled. Return to ARMED when F is released; go to FAULT if the door opens or the key turns off while firing, because that indicates misuse or a failed switch.
- **FAULT**: fault lamp on, laser off. Leave to SAFE only when the fault has cleared and the key has been turned off, which forces a deliberate acknowledgement.
- From every state, $E = 1$ forces FAULT.

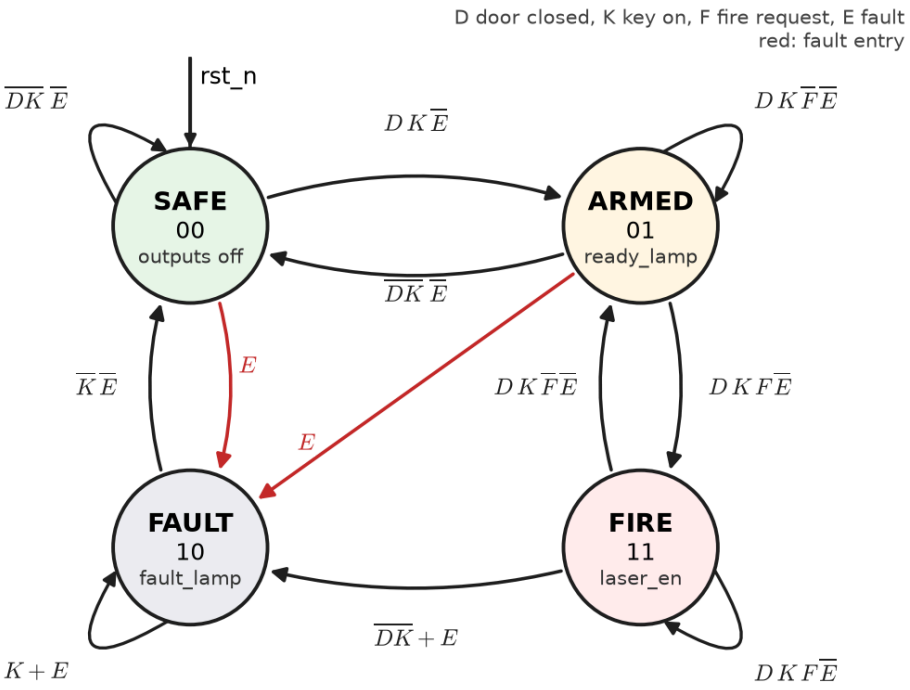


Figure 15.2 — Moore state diagram of the laser interlock controller with Gray-coded state assignment.

Completeness and exclusivity

A state diagram is a specification, and two checks catch most of its errors:

- **Completeness.** For each state, the conditions on all outgoing arcs (including the self-loop) must OR to 1, so that every input combination has a defined next state.
- **Exclusivity.** For each state, the conditions on different arcs must be pairwise disjoint (their AND is 0), so that no input combination selects two next states.

In FIRE, for example, the arcs are $E + \overline{DK}$ (to FAULT), $\overline{E}\,DK\,\overline{F}$ (to ARMED) and $\overline{E}\,DK\,F$ (stay). Their OR is $E + \overline{DK} + \overline{E}\,DK = 1$, and every pair is disjoint. A diagram that fails exclusivity is ambiguous; the HDL will silently resolve it by statement order, perhaps not as intended. These checks are mechanical, which is why the next section turns the diagram into a table.

ASM charts (algorithmic state machine charts) express the same information as a flowchart: a state box with its Moore outputs, decision diamonds for inputs, and conditional output boxes for Mealy outputs. They are easier to read than state diagrams when a state tests several inputs in sequence, and they map naturally onto `if – else` chains in RTL.

15.3 State tables

From diagram to table

A **state table** lists, for every present state and every input combination, the next state and the outputs. It is the state diagram in a form that can be checked and turned into equations. Writing input conditions instead of all $2^4 = 16$ combinations keeps it compact:

Present state	Condition	Next state	laser_en	ready_lamp	fault_lamp
SAFE (00)	E	FAULT	0	0	0
SAFE (00)	$\overline{E}\,D\,K$	ARMED	0	0	0
SAFE (00)	$\overline{E}\,\overline{D\,K}$	SAFE	0	0	0
ARMED (01)	E	FAULT	0	1	0
ARMED (01)	$\overline{E}\,\overline{D\,K}$	SAFE	0	1	0
ARMED (01)	$\overline{E}\,D\,K\,F$	FIRE	0	1	0
ARMED (01)	$\overline{E}\,D\,K\,\overline{F}$	ARMED	0	1	0
FIRE (11)	$E + \overline{D\,K}$	FAULT	1	0	0
FIRE (11)	$\overline{E}\,D\,K\,\overline{F}$	ARMED	1	0	0
FIRE (11)	$\overline{E}\,D\,K\,F$	FIRE	1	0	0
FAULT (10)	$\overline{E}\,\overline{K}$	SAFE	0	0	1
FAULT (10)	$E + K$	FAULT	0	0	1

The outputs depend only on the present state, as befits a Moore machine; a Mealy machine would have output entries that differ between rows of the same state.

State assignment and next-state equations

With the Gray-ordered assignment SAFE = 00, ARMED = 01, FIRE = 11, FAULT = 10 (state bits $S_1 S_0$), the table becomes two Boolean functions of six variables. Minimising them, by Karnaugh map or by a tool (Chapter 12), gives

$$S_1^{+} = E + K S_1 \overline{D} + K S_1 \overline{S_0} + S_1 S_0 \overline{K} + D K F S_0$$

$$S_0^{+} = \overline{E}\,D\,K\,(S_0 + \overline{S_1})$$

and the outputs are `laser_en` $= S_1 S_0$, `ready_lamp` $= \overline{S_1} S_0$ and `fault_lamp` $= S_1 \overline{S_0}$. Hand minimisation of six-variable functions is error-prone, so the equations are checked exhaustively against the table, which takes 64 cases:

```
 from itertools import product

SAFE, ARMED, FIRE, FAULT = 0b00, 0b01, 0b11, 0b10

def spec(s, d, k, f, e):
    """Next state read from the state diagram (Figure 15.2)."""
    if e:
        return FAULT
    if s == SAFE:
        return ARMED if d and k else SAFE
    if s == ARMED:
        return SAFE if not (d and k) else (FIRE if f else ARMED)
    if s == FIRE:
        return FAULT if not (d and k) else (FIRE if f else ARMED)
    return SAFE if not k else FAULT              # s == FAULT

def equations(s1, s0, d, k, f, e):
    nd, nk, ns0 = 1 - d, 1 - k, 1 - s0
    n1 = e | (k & s1 & nd) | (k & s1 & ns0) | (s1 & s0 & nk) | (d & k & f & s0)
    n0 = (1 - e) & d & k & (s0 | (1 - s1))
    return (n1 << 1) | n0

bad = [v for v in product((0, 1), repeat=6)
       if spec(2 * v[0] + v[1], *v[2:]) != equations(*v)]
print("mismatches:", len(bad), "of", 2 ** 6)
```

```
 mismatches: 0 of 64
```

The same idea, comparing an implementation against an independent reference model over many inputs, is the core of the test bench in Section 15.9.

Worked example — what the state table says about cost and speed. S_1^+ depends on all six variables S_1, S_0, D, K, F, E and S_0^+ on five. In an FPGA with 6-input LUTs, each next-state function fits in exactly one LUT, so the whole controller costs two LUTs and two flip-flops for the state, plus three flip-flops for registered outputs (Section 15.5) with their decode absorbed into LUTs: about five LUTs and five flip-flops. Its register-to-register path is one LUT level, roughly $0.42 + 0.11 + 2 \times 0.45 + 0.05 \approx 1.5$ ns in a 28 nm device, so it could run above 600 MHz; in practice it runs at the system's 100 MHz clock with 8.5 ns of slack. In discrete 74HC logic, S_1^+ alone needs five product terms (a 4-input AND, three 3-input ANDs and a 5-input OR), roughly four packages for the whole machine, with a register-to-register path of about three gate delays plus the flip-flop: some 70 ns worst case at 4.5 V, still ample for a controller whose inputs change in milliseconds. The state table makes the cost visible before any code is written.

State minimisation

Two states are **equivalent** if, for every input sequence, they produce the same output sequence; equivalent states can be merged. Formal minimisation (partitioning states into equivalence classes) matters in automatically generated machines and in protocol controllers derived from long specifications, where duplicated states creep in. For hand-designed controllers of a few states, a review of the table usually finds redundancies directly. Minimising states is not always desirable: an extra state that makes outputs registered and glitch-free, or that separates two fault causes for diagnosis, is worth more than a flip-flop.

15.4 Hardware timing

Inside the device

Static timing analysis (Section 14.8) checks every register-to-register path inside an FPGA or ASIC automatically, once the designer has told the tool the clock frequencies. The designer's job is to supply correct **constraints**: clock definitions, the relation between clocks (synchronous or asynchronous), and the timing of the I/O pins relative to external devices. Unconstrained paths are not checked at all, and an asynchronous crossing wrongly declared synchronous produces impossible requirements. The constraint languages (SDC and its vendor variants) are covered in Chapter 19.

At the board level

Paths that leave one chip and enter another are timed by hand or with the I/O constraints. The budget adds the output delay of the sending device, the trace delay (about 6–7 ps per millimetre in FR-4; *Circuits, Signals and Communications*, Ch. 14 §14.4), and the input setup and hold requirements of the receiver, for both the clock path and the data path.

Worked example — reading a parallel ADC from an FPGA. An FPGA in an ultrasound front end clocks a 12-bit, 65 MSPS parallel-output ADC and captures its data on the next edge of the same internal clock (system-synchronous capture), as in Figure 15.3. Representative values:

Element	Max	Min
FPGA internal clock edge to clock-output pin	3.5 ns	2.5 ns
Clock trace to ADC, 10 cm at 6.0 ps/mm	0.6 ns	0.6 ns
ADC clock-to-data-valid t_{OD}	7.0 ns	3.0 ns
Data trace back to FPGA, 10 cm	0.6 ns	0.6 ns
FPGA input setup / hold at pin	2.0 ns / 0.5 ns	—

At 65 MHz the period is 15.38 ns. Data launched by one edge arrives at the FPGA, at the latest, $3.5 + 0.6 + 7.0 + 0.6 = 11.7$ ns later, and must be there 2.0 ns before the next edge:

$$\text{setup slack} = 15.38 - 2.0 - 11.7 = 1.68 \text{ ns}$$

The earliest arrival is $2.5 + 0.6 + 3.0 + 0.6 = 6.7$ ns, far beyond the 0.5 ns hold requirement, so hold is safe. The interface works at 65 MHz with modest margin. At 100 MHz (10 ns period) the setup slack would be $10 - 2.0 - 11.7 = -3.7$ ns: the round trip of clock out and data back is longer than the period allows. The fixes are architectural rather than logical: capture with a PLL-generated clock phase-shifted by about 4–6 ns to centre the data window, or use the ADC's own data-clock output to capture the data (source-synchronous capture), which cancels the outbound path and both traces to first order. High-speed ADCs abandon parallel CMOS outputs altogether for LVDS or JESD204 serial links for this reason.

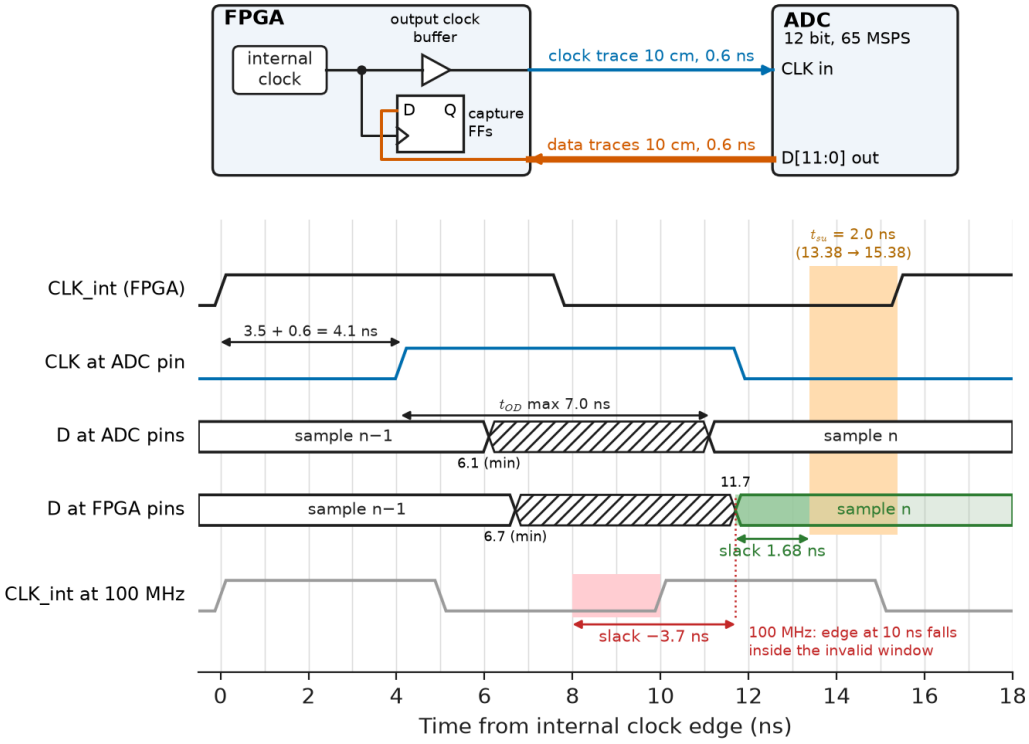


Figure 15.3 — System-synchronous capture of a parallel ADC by an FPGA at 65 MHz, with the timing budget.

> **Status of the principle: Board-level timing budget**
> *Approximate.* Adds worst-case datasheet delays and a constant per-millimetre trace delay; valid when traces are terminated so that the receiver sees a clean first edge. It ignores reflections, crosstalk and simultaneous-switching noise, which on fast edges shift the effective arrival time and must be checked with signal-integrity simulation.

15.5 Glitches

Static and dynamic hazards

A combinational output that should stay constant while an input changes may briefly pulse to the wrong value, because different paths from the changing input to the output have different delays. This is a **static hazard**: static-1 if the output should stay 1 and dips to 0, static-0 if it should stay 0 and pulses to 1. A **dynamic hazard** is an output that should change once but changes three or more times. A pulse produced by a hazard is a **glitch**.

The textbook case is the 2:1 multiplexer $Y = A\overline{S} + BS$ with $A = B = 1$. When S falls from 1 to 0, the term BS turns off at once, but the term $A\overline{S}$ turns on only after the inverter's delay. For that delay both terms are 0 and Y dips (Figure 15.4). On a Karnaugh map the two product terms are adjacent groups that do not overlap; adding the **consensus term** AB, which covers the boundary between them, holds Y at 1 throughout:

$$Y = A\overline{S} + BS + AB$$

A 74HC04 inverter at 4.5 V has a typical delay of about 7 ns, so the unprotected mux can produce a glitch of that order; inside an FPGA the same structure produces glitches of 0.1 to 1 ns, set by routing differences. Adding consensus terms removes static hazards for single-input changes in two-level logic, but synthesis tools remove "redundant" terms and map logic into LUTs whose internal structure the designer does not control, so hazard-free logic cannot be relied on in an FPGA.

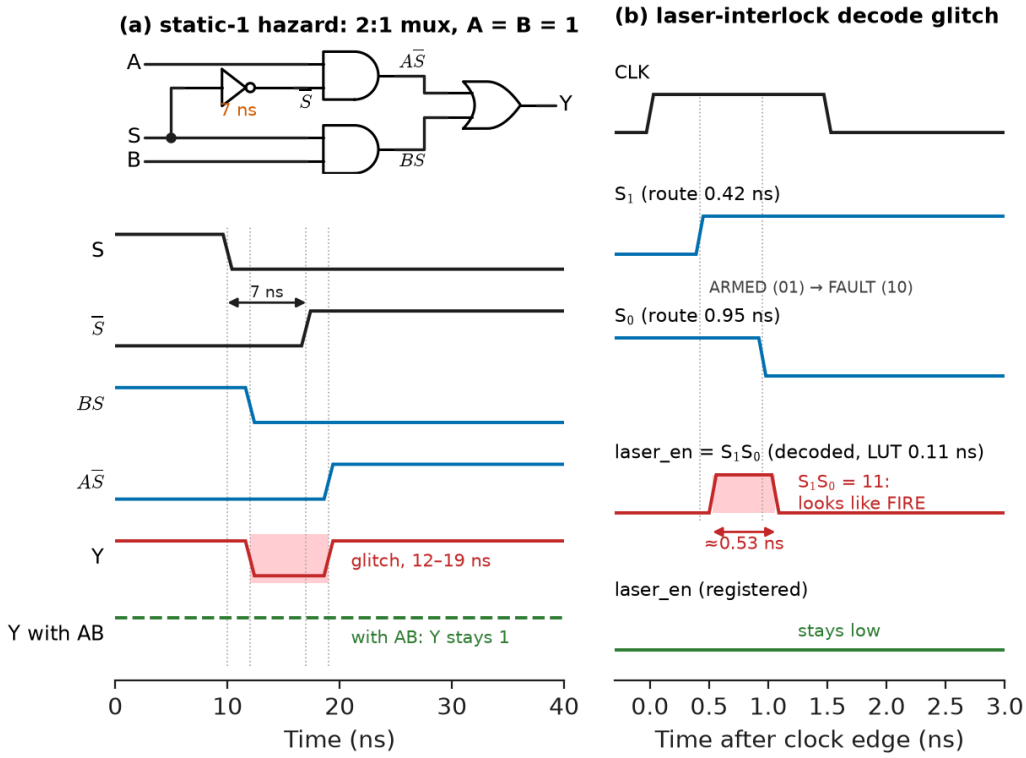


Figure 15.4 — Static-1 hazard in a 2:1 multiplexer, and the decode glitch that fired the laser when the state changed from 01 to 10.

Glitches in synchronous design

In a synchronous design glitches on data paths are harmless by construction: they are over before the next clock edge, and the timing analysis guarantees that the capturing flip-flop sees only the settled value. Glitches matter in three situations:

1. **The signal is used as a clock, an asynchronous reset or set, or a latch enable.** Any of these acts on a glitch immediately. This is why decoded signals must never drive clocks or resets (Section 15.1), and why the counter decode of Section 14.4 must come from a synchronous counter.
2. **The signal leaves the chip to an edge-sensitive or fast-acting device.** A laser driver, a gate driver for a power MOSFET or IGBT bridge (where a glitch can turn on both switches of a leg and short the DC link; Chapter 29), a stepper-motor driver's step input, a chip select, or a pyrotechnic actuator in an automotive airbag controller.
3. **Power.** Every glitch charges and discharges node capacitance. In arithmetic datapaths, especially array multipliers, glitching commonly accounts for 10 to 40 % of dynamic power; pipelining and balanced paths reduce it (Chapter 33 develops dynamic power).

The cure for the first two is the one the laser controller lacked: **register the output**. A flip-flop output changes once per clock, one t_{cq} after the edge, with no glitch. The interlock's outputs are therefore computed from the *next* state and loaded into output flip-flops on the same edge as the state register, so they change in step with the state with no extra cycle of latency (Listing in Section 15.6). An alternative for Moore outputs is an encoding in which each output is a state bit itself (one-hot, or an output-encoded assignment), which removes the decode entirely.

Worked example — how wide was the laser glitch? In the original design, a fault in ARMED (01) sends the machine to FAULT (10). The two state flip-flops are clocked together, but their outputs reach the laser-enable LUT through different routes: S_1 arrives 0.42 ns after the clock edge and S_0's falling edge 0.95 ns after. Between those times the LUT sees $S_1S_0 = 11$ and, after its own 0.11 ns delay and an output path of a few nanoseconds that shifts but does not shorten the pulse, drives the pin high for about $0.95 - 0.42 \approx 0.53$ ns. An I/O buffer with a 1–2 ns slew-rate-limited edge may round that into a runt pulse of a volt or two, which may or may not cross the driver's input threshold depending on temperature, board and part, hence a fault seen once every few weeks. A registered output removes it completely. Gray-

coding the transition (making ARMED → FAULT a one-bit change) would also remove this particular glitch, but not the general hazard, which is why the rule is to register.

15.6 Synchronization

The input boundary

Every input from outside the clock domain is asynchronous: switches, sensor comparators, interrupt lines, signals from another board with its own oscillator. Section 14.9 showed that each such input must pass through a synchronizer, and the reset through a reset synchronizer. The synthesisable Verilog for both is short:

```
 // Two-flip-flop synchronizer for independent single-bit signals (not a bus).
module sync2 #(parameter W = 1) (
    input  wire         clk,
    input  wire         rst_n,
    input  wire [W-1:0] d_async,
    output wire [W-1:0] q
);
    (* ASYNC_REG = "TRUE" *) reg [W-1:0] s1, s2;
    always @(posedge clk or negedge rst_n)
        if (!rst_n) {s2, s1} <= {2*W{1'b0}};
        else        {s2, s1} <= {s1, d_async};
    assign q = s2;
endmodule

// Reset synchronizer: asynchronous assertion, synchronous release.
module reset_sync (
    input  wire clk,
    input  wire rst_ext_n,
    output wire rst_n
);
    (* ASYNC_REG = "TRUE" *) reg r1, r2;
    always @(posedge clk or negedge rst_ext_n)
        if (!rst_ext_n) {r2, r1} <= 2'b00;
        else            {r2, r1} <= {r1, 1'b1};
    assign rst_n = r2;
endmodule
```

The parameter `W` lets one instance synchronise several *unrelated* bits (door switch, key switch, fault line); it must never be used for the bits of a counter or data word, for the reasons given in Section 14.9. The `ASYNC_REG` attribute (Xilinx syntax; other vendors have equivalents) keeps the two flip-flops adjacent and marks them for MTBF analysis.

The interlock FSM in RTL

With its inputs synchronised and its reset released synchronously, the interlock controller of Sections 15.2–15.3 is written as a state register, a combinational next-state block and registered Moore outputs:

```
module interlock (
    input  wire clk, rst_n,                         // rst_n from reset_sync
    input  wire door_closed, key_on, fire_req, fault, // from sync2
    output reg  laser_en, ready_lamp, fault_lamp
);
    localparam [1:0] SAFE = 2'b00, ARMED = 2'b01, FIRE = 2'b11, FAULT = 2'b10;
    reg [1:0] state, next;

    always @* begin
        next = state;                               // default: stay, no latch
        if (fault) next = FAULT;
        else case (state)
            SAFE:  if (door_closed && key_on)        next = ARMED;
            ARMED: if (!(door_closed && key_on))     next = SAFE;
                   else if (fire_req)                next = FIRE;
            FIRE:  if (!(door_closed && key_on))     next = FAULT;
                   else if (!fire_req)               next = ARMED;
            FAULT: if (!key_on)                      next = SAFE;
            default:                                 next = FAULT;
        endcase
    end

    always @(posedge clk or negedge rst_n)
        if (!rst_n) begin
            state <= SAFE;
            {laser_en, ready_lamp, fault_lamp} <= 3'b000;
        end else begin
            state      <= next;
            laser_en   <= (next == FIRE);               // registered: glitch-free
            ready_lamp <= (next == ARMED);
            fault_lamp <= (next == FAULT);
        end
endmodule
```

Three details carry the lessons of this chapter. The default assignment `next = state` at the top of the combinational block guarantees that `next` is assigned on every path, so no latch is inferred (Section 14.1). The outputs are decoded from `next` and stored in flip-flops on the same edge as `state`, so they are glitch-free and aligned with the state. The `default` branch is unreachable with two state bits, but it states the designer's intent that any unexpected code goes to the safe state, the habit that matters once encodings have unused codes (Section 14.7). Chapter 19 covers the coding styles, blocking and non-blocking assignments and synthesis semantics behind these choices.

Debouncing and edge detection

A synchronised mechanical input is clean in the metastability sense but still bounces: contacts typically chatter for 1 to 5 ms when operated, and worn or large switches for up to 20 ms. A **debouncer** accepts a new level only after the synchronised input has been stable for longer than the bounce time.

Worked example — debouncing the fire button. With a 50 MHz system clock and a required stable time of 10 ms, a counter that resets on every change of the synchronised input and updates the debounced output when it reaches its limit must count

$$10\ \text{ms} \times 50\ \text{MHz} = 500\,000\ \text{cycles}$$

which needs a 19-bit counter ($2^{19} = 524\,288$). Alternatively, a shared 1 kHz tick from a prescaler lets each debouncer use a 4-bit counter that requires 10 consecutive equal samples; with dozens of inputs this saves hundreds of flip-flops. The 10 ms delay is imperceptible to an operator and long compared with any bounce.

After synchronising and debouncing, an **edge detector** (the signal ANDed with the complement of its value one clock earlier) produces a one-cycle pulse on each press. The interlock specification would benefit from it: as written, holding the fire button while turning the key moves the machine from SAFE to ARMED and, one clock later, to FIRE. Requiring a fresh press (the rising edge of F) to enter FIRE is a specification change, and it is exactly the kind of gap that verification against the requirements, rather than against the designer's own state diagram, exposes.

Synchronise once

An asynchronous input must be synchronised in *one* place, and the synchronised version distributed. Feeding the raw signal to two synchronizers, or to two state machines that each sample it, lets them resolve differently on the same edge, so one part of the design believes the door closed a cycle before the other does. The same applies to a value read by several clock domains: each domain synchronises it once and shares the result internally.

15.7 Memory elements

A design stores state in several kinds of element, and the choice affects area, speed, power and how the timing is analysed.

Table 15.1 — Storage options in an FPGA-based design (representative of 28 nm mid-range devices).

Element	Capacity per unit	Read behaviour	Typical use
Flip-flop	1 bit	Immediate (t_{cq})	State, control, pipeline registers, anything needing reset
Shift-register LUT	32 bits per LUT as a delay line	Addressable tap	Delay lines, small FIFOs without reset
Distributed (LUT) RAM	64 bits per LUT6	Asynchronous read	Small register files, coefficient tables under a few hundred bits per word lane
Block RAM	18 or 36 kbit per block, dual port	Synchronous read, 1–2 cycles latency	FIFOs, frame lines, large tables, packet buffers
UltraRAM / large on-chip RAM	288 kbit per block (UltraScale+)	Synchronous	Megabit-scale buffers
External SRAM, DRAM, flash	Mbit to Gbit	Tens of ns to µs, interface-limited	Frame stores, data logging, program storage

Two rules of thumb follow. First, use flip-flops for anything that defines behaviour after reset, and RAM for bulk data whose contents need not be reset; block RAM has no per-word reset. Second, respect the synchronous read of block RAM: the data appears one clock after the address (two with the optional output register for speed), so a design that expects combinational read must be restructured, typically by issuing the address a cycle earlier. Synthesis tools infer RAMs from HDL arrays written in their recognised templates (Chapter 19).

Worked example — where to put a sine table. A direct digital synthesiser in a motor-drive test rig stores one cycle of a sine wave as 1024 samples of 16 bits, 16 384 bits. As distributed RAM it occupies $16\,384/64 = 256$ LUTs, about 1 % of a 20 000-LUT device, and reads combinationally. As block RAM it fits in a single 18 kbit block, costs no logic, and reads with one cycle of latency, which the phase accumulator absorbs by being one cycle ahead. Using quarter-wave symmetry (storing 0 to 90° and deriving the rest by mirroring and negation) cuts the table to 256 samples and 4096 bits. The block RAM is the default choice; the distributed version wins only when block RAMs are exhausted or combinational read is essential.

The internal organisation of SRAM, DRAM and flash, their interfaces, and the FPGA memory architecture belong to Chapter 18; the transistor-level latch and SRAM cell to Chapter 33.

15.8 Logic implementation

The technology choices

The same state table can be built in very different technologies:

Table 15.2 — Implementation technologies for digital logic (typical values).

Technology	Logic capacity	Typical internal speed	Unit cost	Development cost	Where it fits
Discrete 74HC/74LVC	A few gates per package	5–50 ns per gate	Cents per package	Very low	Glue logic, level shifting, a few gates on a board
CPLD / small PLD	32–500 macrocells	5–10 ns pin to pin	About 1–5 US dollars	Low	Instant-on glue logic, address decoding, simple FSMs
FPGA	Thousands to millions of LUTs	100–500 MHz designs	A few to thousands of US dollars	Moderate	Custom interfaces, parallel processing, prototypes, low-volume products
ASIC	Millions to billions of gates	GHz	Lowest at high volume	Very high (masks, verification)	High volume, lowest power, highest speed
Microcontroller firmware	Limited by CPU time	µs-scale reaction	Below 1 to a few US dollars	Low	Sequencing where microsecond latency is acceptable

The choice turns on volume, latency, power, flexibility and certification. The interlock controller could run in firmware on the machine's main microcontroller, with a response time of tens of microseconds; it is put in hardware because a safety function should not depend on software that might hang, and because certification of a small FSM is far simpler than certification of a processor and its code. A CPLD or a small flash-based FPGA, which is live as soon as power is applied, is the usual home for such a function. Comparisons of processors and FPGAs for computation, and the partitioning between hardware and software, are the subject of Chapter 35.

What synthesis does

Logic synthesis converts RTL into a network of library cells: it extracts the FSMs and may re-encode them (Section 14.7), optimises the Boolean logic (the minimisation of Chapter 12, at scale), maps it onto the target's primitives (LUTs, carry chains, DSP blocks and block RAMs in an FPGA; standard cells in an ASIC) and sizes or replicates logic to meet timing. **Technology mapping** for LUTs is a covering problem: any function of up to k inputs costs one k-input LUT, regardless of its complexity, so the cost of logic in an FPGA is set by the number of inputs per function rather than by gate count. That is why one-hot encoding and wide-but-shallow logic suit FPGAs, and why the interlock's six-input next-state function costs exactly one LUT.

Synthesis is also where intent can be lost. It removes logic it proves redundant (including deliberately redundant hazard-cover terms and duplicated safety logic, unless told to keep them), it may merge equivalent registers, and it can turn an unintended `if` without `else` into a latch. Reading the synthesis report (inferred latches, FSM encodings chosen, removed registers, resource use) is part of the design method, not an afterthought.

15.9 Verification and test benches

What verification must show

Verification shows that the design does what the specification says, under all the conditions the specification allows. In industry it takes more effort than design, often 60 to 70 % of a digital project. Its main tools are:

- **Simulation with a test bench.** A test bench is HDL code (or a Python or SystemVerilog environment) that is not synthesised: it generates clocks, resets and stimuli, drives the design under test (DUT), and checks the outputs.
- **Self-checking.** The test bench compares the DUT's outputs with expected values from a **reference model** (an independent description of the specification) and counts mismatches. A test bench that only produces waveforms for a human to inspect does not scale and does not catch regressions.
- **Coverage.** Code coverage (lines, branches) shows what the tests exercised; **functional coverage** shows which specified behaviours were exercised: here, every state, every one of the 12 arcs of Figure 15.2 (eight transitions and four self-loops), and the fault input asserted in each state.
- **Assertions.** Properties that must hold at all times, checked during simulation and, with formal tools, for all possible inputs: "laser_en implies door_closed was true on the previous cycle"; "the state is never FIRE immediately after FAULT". SystemVerilog Assertions (SVA) and PSL express them; Chapter 19 introduces both.
- **Formal verification.** Model checkers prove assertions exhaustively over all reachable states. For a small safety FSM, this is practical and gives a stronger guarantee than any amount of simulation.

A self-checking test bench

The listing drives the interlock through a directed sequence that covers the main path and the two fault entries, and compares the registered outputs with expected values after each clock edge.

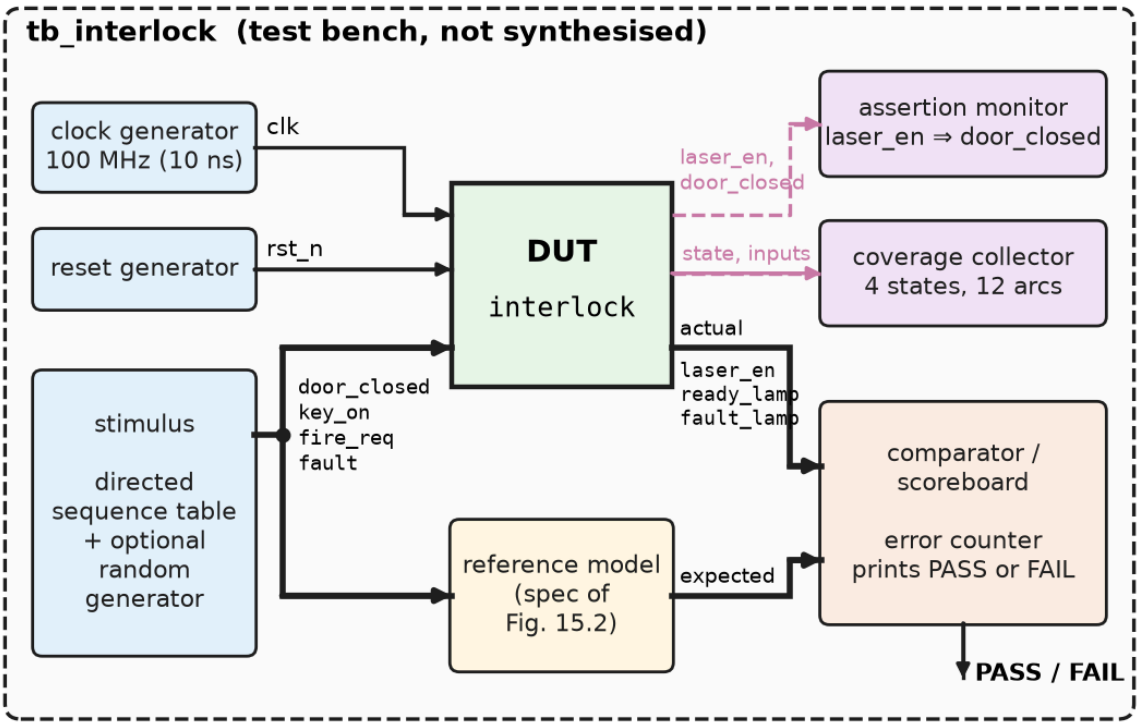


Figure 15.5 — Structure of a self-checking test bench around the interlock controller.

```
`timescale 1ns/1ps
module tb_interlock;
    reg  clk = 1'b0, rst_n = 1'b0;
    reg  door = 1'b0, key = 1'b0, fire = 1'b0, flt = 1'b0;
    wire laser, ready, fault_l;
    integer errors = 0;

    interlock dut (.clk(clk), .rst_n(rst_n), .door_closed(door), .key_on(key),
                   .fire_req(fire), .fault(flt), .laser_en(laser),
                   .ready_lamp(ready), .fault_lamp(fault_l));

    always #5 clk = ~clk;                                   // 100 MHz

    task step(input d, input k, input f, input e, input [2:0] expected);
        begin                                       // expected = {laser, ready, fault}
            {door, key, fire, flt} = {d, k, f, e};
            @(posedge clk); #1;
            if ({laser, ready, fault_l} !== expected) begin
                errors = errors + 1;
                $display("t=%0t in=%b%b%b%b got=%b expected=%b",
                         $time, d, k, f, e, {laser, ready, fault_l}, expected);
            end
        end
    endtask

    initial begin
        repeat (2) @(posedge clk);
        @(negedge clk) rst_n = 1'b1;                        // release away from edge
        step(1, 0, 0, 0, 3'b000);                           // SAFE
        step(1, 1, 0, 0, 3'b010);                           // key on: ARMED
        step(1, 1, 1, 0, 3'b100);                           // fire: FIRE
        step(0, 1, 1, 0, 3'b001);                           // door opens: FAULT
        step(1, 1, 0, 0, 3'b001);                           // held while key on
        step(1, 0, 0, 0, 3'b000);                           // key off: SAFE
        step(1, 1, 1, 1, 3'b001);                           // fault has priority
        if (errors == 0) $display("PASS");
        else             $display("FAIL: %0d errors", errors);
        $finish;
    end
endmodule
```

The task `step` applies one input vector, waits for the clock edge, and checks the outputs 1 ns later, after the registered outputs have settled in zero-delay RTL simulation. The reset is released on a falling edge so that its release cannot race with the rising edge at which the flip-flops sample. The comparison uses `!==`, which treats unknown (`x`) values as mismatches, so an uninitialised register is caught rather than passed. The expected values here are written by hand; a stronger test bench computes them from a reference model, such as a behavioural model of Figure 15.2 in the test bench or the Python `spec` function of Section 15.3 exported as a vector file, and adds constrained-random stimuli with the coverage of the 12 arcs measured rather than assumed.

> **Status of the principle: Simulation-based verification**
> *Approximate.* A passing simulation shows correct behaviour only for the stimuli applied and only within the simulation model, which in RTL ignores delays, glitches and metastability; coverage metrics measure how much of the specified behaviour was exercised, and formal proof, static timing analysis and CDC checks cover what simulation cannot.

What the test bench does not see

The opening fault would have passed this test bench. RTL simulation has zero delays, so the decoded output in the original design never glitched in simulation; the glitch existed only in the routed hardware. Verification therefore combines several checks, each covering a different gap: functional simulation and formal proof for behaviour, static timing analysis for setup and hold (Section 14.8), CDC analysis for crossings (Section 14.9), lint rules that flag combinational outputs driving pins and unregistered decode of state bits, and, last, validation of the assembled system with an oscilloscope or logic analyser on the outputs that matter. Gate-level simulation with back-annotated delays can reveal glitches, but it is slow and only as good as its stimuli, so design rules (register the output) are a better defence than hoping to observe the fault. The full verification methodology, SystemVerilog test benches, assertions, and FPGA implementation and timing constraints are developed in Chapter 19; hardware-in-the-loop testing of embedded systems in Chapter 28.

What remains open

Open question	Where it is answered
How does this method scale to a processor, with a datapath and instruction-driven control unit?	Chapter 16
How are memories and FPGA fabrics organised, and how are RAMs inferred?	Chapter 18
How are Verilog, SystemVerilog and VHDL used for synthesis, constraints, assertions and full verification environments?	Chapter 19
How are state machines written in firmware, and how are embedded systems tested in the loop?	Chapter 28
How are glitches, dynamic power and latches analysed at transistor level?	Chapter 33
How is an ASIC taken through synthesis, physical design, timing and sign-off?	Chapter 34
When should a function be in hardware, and when in software?	Chapter 35

Next: Chapter 16 opens Part IV by assembling registers, ALUs, multiplexers and control state machines into the datapath and control unit of a computer.

Part IV — Computer Architecture and Digital Hardware

Part III built digital circuits from gates and flip-flops: adders, multiplexers, counters, state machines, all clocked and timed. This part assembles them into the machines that run almost every modern product — processors, memories and programmable logic — and into the languages used to design them. An instruction set lets software be written once and run on circuits that change every few years, but the abstraction leaks. A motor-control loop misses deadlines because of cache misses, a data logger dies because its flash wears out, an interrupt arrives late because of a priority set one line wrong, and an FPGA design fails timing because its routing is congested. This part explains the hardware beneath the instruction set and the netlist well enough to predict those failures with numbers.

By the end of this part you should understand

- How an instruction set (RISC-V as the example) defines the contract between software and hardware, and how a datapath, a control unit, registers and an ALU implement it.
- How pipelining raises throughput, what data, control and structural hazards cost, and how to estimate execution time with the performance equation.
- Why memory is a hierarchy, how caches are organised and why they miss, and how to compute average memory access time and its effect on CPI.
- How buses, interrupts, polling and DMA move data between processors, memory and devices, and when each is appropriate.
- How a Cortex-M microcontroller executes code, maps memory and peripherals, handles nested interrupts, and drives timers, GPIO and serial peripherals at register level — and when a microprocessor is needed instead.
- How SRAM, DRAM and flash cells store bits, and what that implies for speed, density, refresh, endurance and retention, and how memories are interfaced.
- How FPGAs implement logic with LUTs, configurable logic blocks and programmable interconnect, how CPLDs differ, and when spatial hardware accelerates a computation.
- How hardware is described in Verilog, SystemVerilog and VHDL, simulated, synthesised, constrained and verified.

Chapters

Chapter	Central question	What it delivers
Chapter 16 — Computer Architecture	How does an instruction set become a working, fast processor, and how is its performance estimated?	Performance equation, RISC-V formats and encodings, registers, ALUs, single-cycle and pipelined datapaths with hazards, control units, memory hierarchy and AMAT, caches, buses, interrupts, polling, interrupt and DMA I/O
Chapter 17 — Microprocessors and Microcontrollers	How does a microcontroller core execute code, reach its peripherals and respond to events, and when is a microprocessor needed instead?	Cortex-M programmer's model, wait states, memory map and vector table, Thumb-2 addressing modes, NVIC stacking and tail-chaining, timers and PWM with dead time, GPIO and atomic access, serial peripheral blocks, clocks, DMA and triggers, MCU versus MPU
Chapter 18 — Memory and Programmable Logic	How are bits stored and logic made programmable, and what limits speed, endurance and reliability?	SRAM and DRAM cells, DRAM timing and refresh, ROM and NOR/NAND flash with wear calculations, parallel, DDR and serial interfaces, FPGA architecture, LUTs, CLBs, routing, CPLDs, acceleration and Amdahl's law, applications
Chapter 19 — Hardware Description Languages	How is digital hardware described, simulated, synthesised and verified in a language?	HDL fundamentals, Verilog/SystemVerilog and VHDL, combinational and sequential descriptions, simulation semantics, synthesis, timing constraints, test benches, the FPGA implementation flow and hardware verification

How this part connects to the rest of the book

Part IV stands on Part III: number representation (Chapter 11), Boolean algebra and logic minimisation (Chapter 12), adders, multiplexers and ALUs (Chapter 13), flip-flops, timing constraints and synchronisers (Chapter 14) and digital system design (Chapter 15) are used here as known building blocks. Looking forward, Part V programs these machines, Chapter 21 relying on the memory map, `volatile` access and cache behaviour of Chapters 16 and 17. Part VI builds embedded and real-time systems on them: Chapter 25 chooses and boots microcontrollers, Chapter 26 develops the interface protocols whose peripheral blocks appear in Chapter 17, and Chapter 27 turns the interrupt latencies of this part into schedulability analysis. The timers of Chapter 17 drive the converters of Part VII, and Part VIII designs the chips themselves, ending with processor–FPGA partitioning in Chapter 35.

Two sibling books carry theory this part uses without re-deriving it. The tunnelling physics behind flash memory and the magnetic tunnel junction behind MRAM come from *Engineering Physics: Top Down*, Ch. 4 and Ch. 14. Transmission-line behaviour of fast buses and DDR interfaces, and the digital-filter and FFT algorithms that processors and FPGAs accelerate, come from *Circuits, Signals and Communications*, Ch. 14 and Chs. 22–23. The drive-control laws that run on the microcontrollers of Chapter 17 belong to *Electrical Energy and Machines*, Ch. 10–12.

Chapter 16 — Computer Architecture

Part IV · Computer Architecture and Digital Hardware

A team porting a motor-control loop from an old 100 MHz microcontroller to a new 1.2 GHz application processor expects the 20 kHz current loop to run ten times faster. It runs barely twice as fast on average, and once every few hundred cycles it takes four times longer than usual and misses its deadline. Nothing is wrong with the compiler or the code. The new chip spends most of its time waiting: for instructions and data that are not in its caches, for a branch whose direction it guessed wrongly, for a bus shared with a display controller that has just started a burst. The instruction set promised that every `lw` and `add` does exactly what the manual says; it never promised how long either would take. That time is decided by the organisation underneath the instruction set — the datapath, the pipeline, the caches, the buses and the way interrupts and I/O reach the processor — and that organisation is the subject of this chapter.

> **What does an instruction set promise to software, and how does a datapath and control unit keep that promise in hardware?**
> **Why does a processor's speed depend so much on its pipeline and memory hierarchy, and how can it be estimated with numbers before code is run? How do buses, interrupts and DMA connect a processor to memory and to the outside world?**

Concept	Engineering destination
Computer organisation	Reading a block diagram of any SoC, MCU or server; locating bottlenecks
Instruction sets	Compiler targets, assembly debugging, choosing RISC-V, Arm or x86 parts
CPU datapaths	Cycle-time and CPI estimates, understanding pipeline stalls in profiles
Control units	Instruction decoding in soft cores, microcode updates, custom extensions
Registers	Calling conventions, context-switch cost, interrupt stacking
ALUs	Arithmetic throughput, flag and comparison semantics, DSP extensions
Memory hierarchy	Data layout for real-time loops, sizing on-chip memory, server performance
Caches	AMAT estimates, worst-case execution time, conflict misses in buffers
Buses	AXI, AHB, APB and PCIe bandwidth budgets, SoC interconnect choice
Interrupts	Latency budgets for control loops and communication stacks
Input/output architecture	Polling, interrupt and DMA design for sensors, networks and storage

Roadmap. The parts of a computer and the performance equation that ties them together (16.1) → the instruction set as a contract, using RISC-V (16.2) → the registers and ALU that execute it (16.3–16.4) → the single-cycle and pipelined datapaths (16.5) and the control unit that drives them (16.6) → why memory is a hierarchy (16.7) and how caches work (16.8) → buses (16.9) → interrupts and exceptions (16.10) → input/output architecture, polling, interrupts and DMA (16.11).

16.1 Computer organisation

The stored-program machine

Every general-purpose computer built since the late 1940s follows the same plan (Figure 16.1). A **processor** (central processing unit, CPU) fetches instructions from **memory**, decodes them, reads operands from a small set of **registers** or from memory, operates on them in an **arithmetic logic unit** (ALU), and writes results back. The instructions themselves sit in the same kind of memory as the data: this is the **stored-program** idea, and it is what lets a compiler write a program, a bootloader load it and an operating system run it. **Input/output** (I/O) devices — timers, converters, network controllers, storage — exchange data with the processor and memory over **buses** and signal events with **interrupts**.

Two arrangements of memory are common. In a **von Neumann** organisation, instructions and data share one memory and one bus, so a load competes with the next instruction fetch. In a **Harvard** organisation they have separate paths. Practically every modern processor is a *modified* Harvard machine: one unified address space and main memory, but separate level-1 instruction and data caches (or, in a microcontroller, separate instruction and data buses into a bus matrix, Chapter 17), so that a fetch and a load can proceed in the same clock cycle.

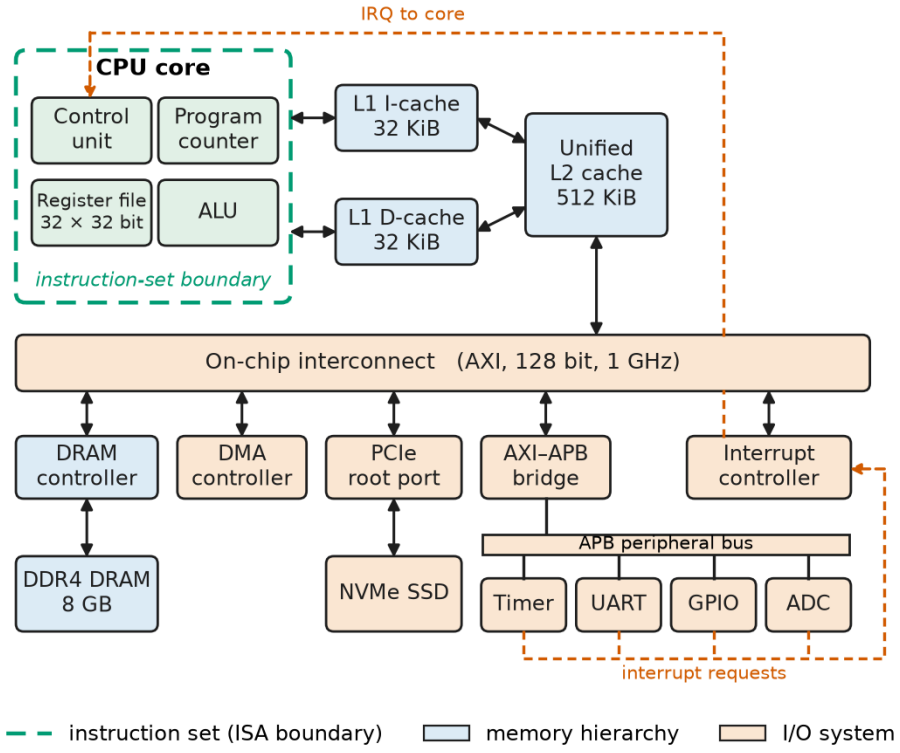


Figure 16.1 — Organisation of a stored-program computer as it appears in a modern system-on-chip.

Architecture, organisation, implementation

It pays to separate three levels:

- **Instruction-set architecture** (ISA): what the programmer and compiler see — instructions, registers, data types, addressing, memory model, exceptions. RV32I, Armv7-M and x86-64 are ISAs.
- **Microarchitecture** (organisation): how one design implements the ISA — pipeline depth, cache sizes, branch predictor, number of execution units. A Cortex-M0+ and a Cortex-A78 implement related Arm ISAs with very different organisations.
- **Implementation**: the circuits, process node, clock frequency and physical layout (Part VIII).

Software is portable across microarchitectures because the ISA is a contract. Its *timing* is not, because timing belongs to the organisation.

The performance equation

The time a program takes is the product of three factors:

$$T_{\text{CPU}} = N_{\text{instr}} \times \text{CPI} \times T_c$$

where N_{instr} is the number of instructions executed, CPI the average clock cycles per instruction and T_c the clock period. The ISA and compiler set N_{instr}; the microarchitecture sets CPI; the implementation and microarchitecture together set T_c. Most design arguments in this chapter are trade-offs among the three: a deeper pipeline shortens T_c but raises CPI through hazards; a richer instruction set lowers N_{instr} but may lengthen T_c; a bigger cache lowers CPI but may add a cycle to every hit.

> **Status of the principle: Processor performance equation ("iron law")**
> *Derived*. It is an identity from the definitions of CPI and clock period; it is exact for one run, and its usefulness lies in separating the factors that the compiler, the microarchitecture and the circuit designer control.

IPC (instructions per cycle) is 1/CPI; superscalar cores reach 2–4 on favourable code. MIPS ratings are meaningless across ISAs; for embedded work the useful benchmark is the cycle count of the real loop, measured with the processor's cycle counter.

16.2 Instruction sets

The ISA as a contract

An ISA specifies the architectural **state** (PC, registers, control registers, memory); the **instructions**, their encodings and effects; the **data types** (two's-complement integers, Chapter 11, and optionally IEEE 754 floating point); the **addressing** of memory (Chapter 17 compares addressing modes); the **memory model** (endianness, alignment, the order in which other processors and devices see loads and stores); and the **privilege and exception** behaviour (Section 16.10).

RISC and CISC

Instruction sets fall into two broad families. **CISC** (complex instruction set computer) ISAs such as x86 have variable-length instructions (1 to 15 bytes on x86-64), allow memory operands in arithmetic instructions and offer complex operations in one instruction. **RISC** (reduced instruction set computer) ISAs — RISC-V, Arm, MIPS, Power — are **load-store** architectures: only loads and stores touch memory, arithmetic works on registers, instructions have a few fixed formats, and the register file is large (16 to 32 registers). The distinction has blurred — x86 cores translate instructions into RISC-like micro-operations internally, and Thumb-2 and RISC-V's compressed extension add 16-bit encodings for density — but a load-store ISA remains easier to pipeline, verify and extend, which is why nearly every new embedded design uses one.

RISC-V as the running example

RISC-V is an open ISA, defined by RISC-V International, with a small base and optional standard extensions. RV32I, the 32-bit base integer ISA, has about forty instructions, 32 registers of 32 bits (`x0` always reads zero) and a 32-bit PC. Extensions add integer multiply and divide (M), atomics (A), single- and double-precision floating point (F, D), 16-bit compressed encodings (C) and vector operations (V). A microcontroller core such as an RV32IMC part in a wireless sensor chip implements a few of them; a Linux-capable application core implements RV64GC (G = IMAFD plus a few system extensions).

Every RV32I instruction is 32 bits long and uses one of six formats (Figure 16.2). The register fields `rs1`, `rs2` and `rd` are always in the same bit positions, so the register file can be read before the instruction is fully decoded, and the sign bit of every immediate is always bit 31.

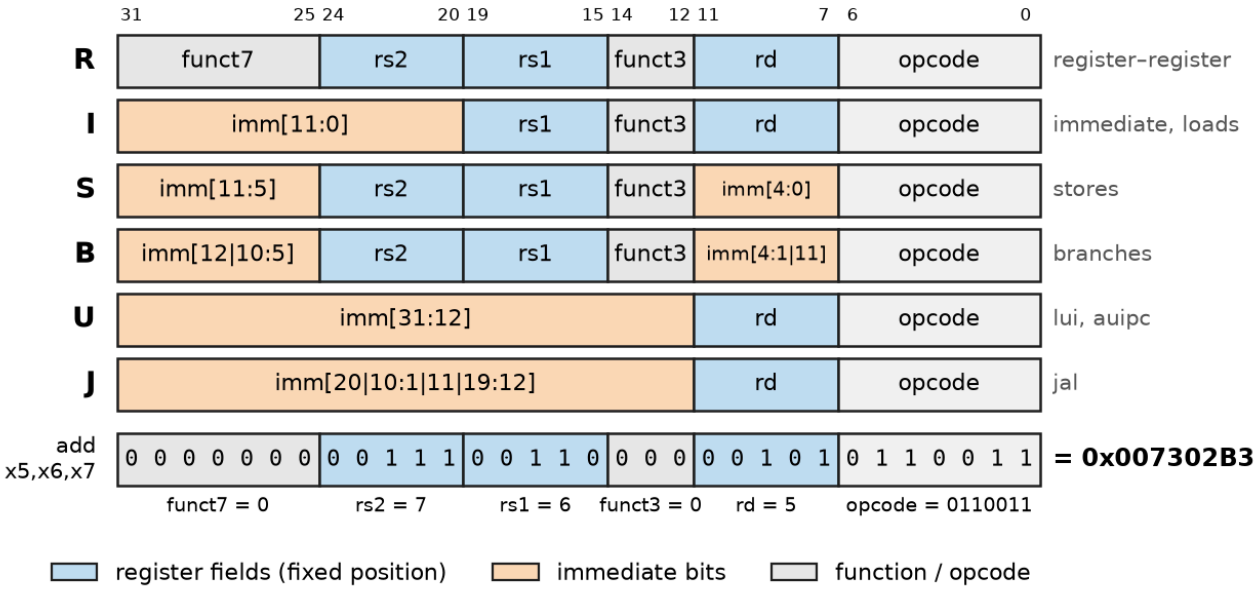


Figure 16.2 — The six RV32I instruction formats with field positions.

Worked example — encoding two instructions. The R-type instruction `add x5, x6, x7` has opcode 0110011, `funct3` = 000, `funct7` = 0000000, `rd` = 5, `rs1` = 6 and `rs2` = 7. Concatenating the fields from bit 31 down gives 0000000 00111 00110 000 00101 0110011 = `0x007302B3`. The I-type load `lw x5, 8(x10)` has opcode 0000011, `funct3` = 010 (word), `rd` = 5, `rs1` = 10 and a 12-bit immediate of 8: 000000001000 01010 010 00101 0000011 = `0x00852283`. A disassembler, a debugger's memory view and a logic analyser on an instruction bus all show these hexadecimal words; reading one by hand is a routine debugging skill when a core has jumped into data.

Instruction classes

Class	RV32I examples	Effect
Arithmetic and logic, register	`add`, `sub`, `and`, `or`, `xor`, `sll`, `srl`, `sra`, `slt`, `sltu`	`rd ← rs1 op rs2`
Arithmetic and logic, immediate	`addi`, `andi`, `ori`, `xori`, `slli`, `slti`	`rd ← rs1 op imm12` (sign-extended)
Upper immediate	`lui`, `auipc`	Build 32-bit constants and PC-relative addresses
Loads	`lb`, `lh`, `lw`, `lbu`, `lhu`	`rd ← mem[rs1 + imm12]`
Stores	`sb`, `sh`, `sw`	`mem[rs1 + imm12] ← rs2`
Conditional branches	`beq`, `bne`, `blt`, `bge`, `bltu`, `bgeu`	`if (rs1 cmp rs2) PC ← PC + imm13` (±4 KiB)
Jumps	`jal`, `jalr`	`rd ← PC+4; PC ← target` (±1 MiB for `jal`)
System	`ecall`, `ebreak`, `fence`, CSR access	Traps, ordering, control registers

RISC-V has no condition-code flags: a branch compares two registers itself, whereas Arm branches on N, Z, C and V flags set by earlier instructions (Chapter 17).

A short function shows the style. It sums an array of 32-bit integers, with the pointer in `a0` (`x10`) and the count in `a1` (`x11`), returning the sum in `a0` under the standard calling convention:

```
 # int32_t sum(const int32_t *p, uint32_t n)   -- RV32I
sum:    li    t0, 0             # running sum
        beqz  a1, done
loop:   lw    t1, 0(a0)         # load element
        add   t0, t0, t1
        addi  a0, a0, 4         # next element
        addi  a1, a1, -1
        bnez  a1, loop
done:   mv    a0, t0
        ret
```

`li`, `beqz`, `bnez`, `mv` and `ret` are **pseudo-instructions** that the assembler expands into `addi`, `beq`, `bne` and `jalr` with `x0`. The loop body is five instructions, so for n elements $N_{\text{instr}} \approx 5n$. How many cycles each iteration takes depends on the pipeline, and Section 16.5 returns to this loop to count them.

16.3 Registers

The register file

Registers are the fastest storage in the machine: a 32-entry, 32-bit register file is about 1 kbit of multi-ported SRAM or flip-flops, read in a fraction of a clock cycle. A single-issue RISC-V core needs two read ports (for `rs1` and `rs2`) and one write port (for `rd`); a four-wide superscalar core needs about eight read and four write ports, which makes its register file one of the largest and hottest blocks on the die. Register `x0` is not storage at all: reads return zero and writes are discarded, which turns `addi rd, x0, imm` into "load immediate" and `add rd, rs, x0` into "move" without extra opcodes.

More architectural registers mean fewer spills to the stack but wider register fields and more state to save on a context switch; RV32E halves the file to 16 registers for the smallest cores.

Software conventions

The hardware treats `x1` to `x31` alike; the **application binary interface** (ABI) gives them roles so that separately compiled functions interoperate:

Register	ABI name	Role	Saved by
`x0`	`zero`	Constant 0	—
`x1`	`ra`	Return address	Caller
`x2`	`sp`	Stack pointer	Callee
`x5` – `x7`, `x28` – `x31`	`t0` – `t6`	Temporaries	Caller
`x8` – `x9`, `x18` – `x27`	`s0` – `s11`	Saved registers (`s0` = frame pointer)	Callee
`x10` – `x17`	`a0` – `a7`	Arguments and return values	Caller

The split between caller-saved and callee-saved registers decides how many registers an interrupt handler written in C must save: the hardware or the handler's prologue must preserve every caller-saved register, because the interrupted

code did not expect a call. That is 16 registers on RV32I, and it is why interrupt entry costs tens of cycles (Section 16.10; Chapter 17 shows how Cortex-M stacks eight of them in hardware).

Special registers

Beside the PC, RISC-V has a separate space of **control and status registers** (CSRs), accessed with `csrr` and `csrw`. Those that matter most in embedded work are `mstatus` (global interrupt enable), `mie` and `mip` (interrupt enable and pending bits), `mtvec` (trap vector base), `mepc` (address to return to after a trap), `mcause` (why the trap happened) and the counters `mcycle` and `minstret`, which count cycles and retired instructions and are the honest way to measure CPI on real code.

16.4 ALUs

What the ALU computes

The ALU is the combinational block that executes the arithmetic and logic part of every instruction and computes addresses for loads, stores and branches. Its circuit building blocks — adders, carry-lookahead and prefix structures, shifters, comparators and the multiplexer that selects among them — are designed in Chapter 13. At the architecture level the ALU is a black box with two 32-bit operands, a control field and a 32-bit result, plus status outputs such as "result is zero" for branch evaluation.

For RV32I the ALU must support:

ALU control	Operation	Used by
ADD	$a + b$	`add`, `addi`, loads, stores (address), `jalr`
SUB	$a - b$	`sub`, branches (compare)
AND, OR, XOR	bitwise	logic instructions
SLT, SLTU	$a < b$ signed or unsigned, result 0 or 1	`slt`, `sltu`, `blt`, `bltu`
SLL, SRL, SRA	shift by $b[4{:}0]$	shift instructions

Branch comparisons are done by subtraction: `beq` tests whether $a - b$ is zero, `blt` tests the sign of $a - b$ corrected for overflow (signed less-than is $N \oplus V$ in flag terms), and `bltu` tests the borrow. The two's-complement facts behind these tests are in Chapter 11.

Speed and extensions

The adder's carry chain is usually the ALU's critical path: in a 28 nm process a 32-bit prefix adder settles in about 100–150 ps, and with operand and result multiplexing the execute stage takes 150–250 ps (the examples below use 200 ps). A 32×32-bit multiplier is pipelined over 1–3 cycles, while division is iterative and takes 2 to 35 cycles depending on core and operands — which matters when a control loop divides by a variable. DSP-oriented ISAs add saturating arithmetic, multiply-accumulate and SIMD operations on packed 8- and 16-bit data (Chapter 17 shows the Cortex-M4 versions).

16.5 CPU datapaths

The single-cycle datapath

The simplest way to implement RV32I is a **single-cycle** datapath, in which every instruction completes in one long clock cycle (Figure 16.3). The PC addresses the instruction memory; the instruction's register fields address the register file; an

immediate generator sign-extends the immediate; the ALU computes; the data memory is read or written; and a multiplexer selects what is written back to `rd` — the ALU result, the loaded data or PC + 4. A separate adder computes the branch target PC + imm, and a multiplexer chooses the next PC.

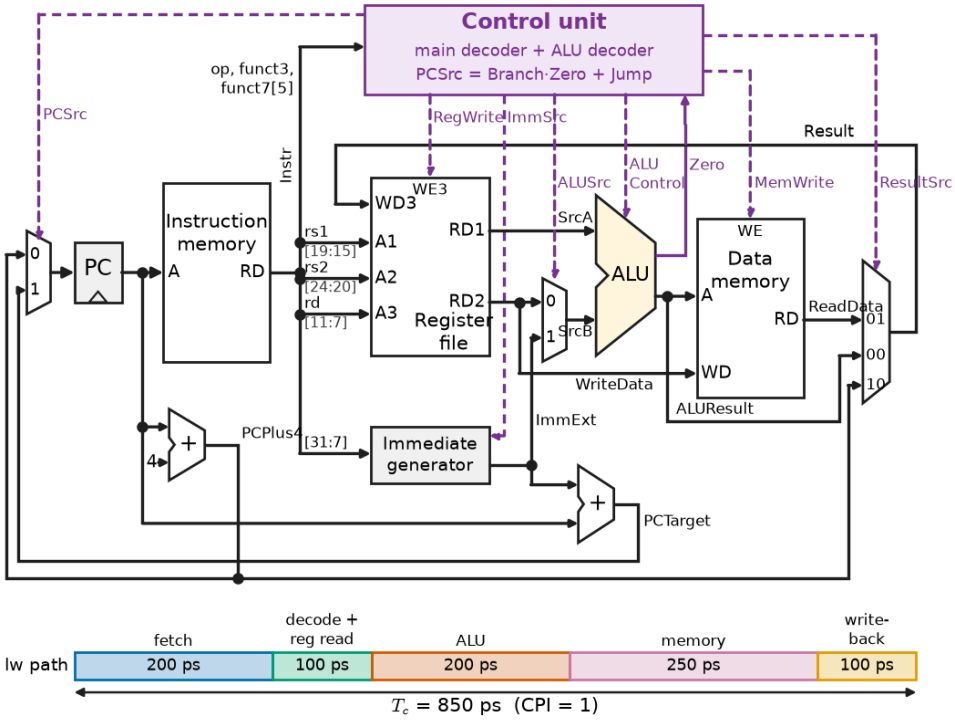


Figure 16.3 — Single-cycle RV32I datapath with control signals.

The clock period must accommodate the slowest instruction, the load, whose path runs through every block. With representative delays of 200 ps for instruction fetch, 100 ps for decode and register read, 200 ps for the ALU, 250 ps for the data-memory access and 100 ps for write-back, $T_c = 850$ ps ($f_c \approx 1.18$ GHz) and CPI = 1. An `add` uses only about 600 ps of that period; the rest is wasted. The single-cycle design is useful as a reference model and appears in small soft cores and teaching designs, but no performance design uses it.

Pipelining

A **pipelined** datapath splits the same hardware into stages separated by pipeline registers and starts a new instruction every cycle. The classic RISC pipeline has five stages:

1. **IF** — fetch the instruction at PC, compute PC + 4.
2. **ID** — decode, read registers, generate the immediate.
3. **EX** — ALU operation, branch comparison and target.
4. **MEM** — data-memory read or write.
5. **WB** — write the result to the register file.

At any moment five instructions are in flight (Figure 16.4). The clock period is now set by the slowest *stage* plus the overhead of the pipeline register (clock-to-Q delay plus setup time, Chapter 14), and the ideal CPI is 1.

> **Status of the principle: Ideal pipeline speedup**
> *Approximate.* A k-stage pipeline approaches a k-fold throughput gain only if the stages are perfectly balanced, the register overhead is negligible and no hazards occur; in practice the gain is limited by the slowest stage, by register overhead (typically 20–50 ps per stage) and by stall cycles.

Hazards

Overlapping instructions creates three kinds of **hazard**, situations in which the next instruction cannot proceed in the next cycle.

- **Structural hazards** — two instructions need the same hardware in the same cycle. A single memory port shared by IF and MEM is the classic case; separate instruction and data caches remove it.
- **Data hazards** — an instruction needs a result that an earlier instruction has not yet written. In `add t0, t0, t1` followed by `addi t2, t0, 1`, the second instruction reads `t0` in ID while the first is still in EX. **Forwarding** (bypassing) routes the ALU result from the EX/MEM or MEM/WB pipeline register straight back to the ALU input, removing the stall. Forwarding cannot help a **load-use hazard**: data loaded in MEM is not available until the end of that stage, so an instruction that uses it in the very next cycle must wait one bubble. A **hazard unit** detects the dependence by comparing register numbers and stalls IF and ID for one cycle.
- **Control hazards** — the pipeline fetches the instructions after a branch before knowing whether the branch is taken. If the branch is resolved in EX, two wrongly fetched instructions must be flushed when it is taken: a two-cycle penalty. Cores reduce the cost by resolving branches earlier and by **branch prediction**: a static rule (predict not taken, or predict backward branches taken), or a dynamic predictor that keeps a 2-bit saturating counter per branch, indexed by PC bits, together with a branch-target buffer. Dynamic predictors in application cores are correct on 95–99 % of branches in typical code; the remaining mispredictions cost 10–20 cycles in deep pipelines.

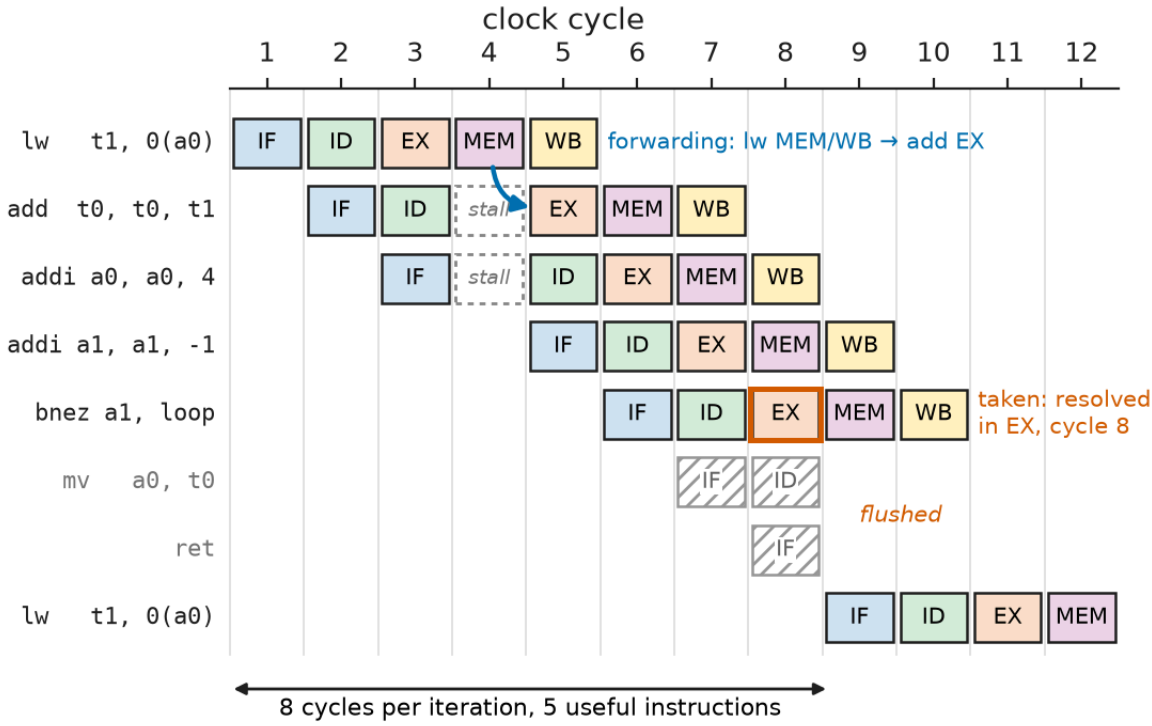


Figure 16.4 — Five-stage pipeline diagram of the sum loop showing a load-use bubble and a taken-branch flush.

Worked example — single-cycle versus pipelined. A benchmark compiled for RV32I executes 10^9 instructions with the mix: loads 25 %, stores 10 %, ALU operations 45 %, conditional branches 15 % and jumps 5 %. Take the stage delays above, a pipeline-register overhead of 30 ps, and these hazard statistics: 40 % of loads are followed immediately by a dependent instruction (one bubble each); branches are predicted not taken, are resolved in EX and are taken 60 % of the time (two-cycle penalty); jumps are resolved in ID (one-cycle penalty).

The single-cycle machine runs at $T_c = 850$ ps with CPI = 1: $T = 10^9 \times 1 \times 850\ \text{ps} = 0.850$ s.

The pipelined machine's clock is set by the 250 ps memory stage plus overhead, $T_c = 280$ ps ($f_c = 3.57$ GHz in this idealised model). Its CPI is

$$\text{CPI} = 1 + 0.25 \times 0.40 \times 1 + 0.15 \times 0.60 \times 2 + 0.05 \times 1 = 1 + 0.10 + 0.18 + 0.05 = 1.33$$

and $T = 10^9 \times 1.33 \times 280\ \text{ps} = 0.372$ s. The speedup is $0.850/0.372 = 2.28$, against an ideal of $850/280 = 3.04$ for a hazard-free pipeline and 5 for perfectly balanced stages with no overhead. The three losses are visible separately: the unbalanced stages cost $850/(5 \times 250)$, the register overhead $250/280$, and the hazards $1/1.33$. This model still assumes every memory access hits in one cycle; Section 16.8 removes that assumption, with large consequences.

The array-sum loop of Section 16.2 shows the same effects in code. In the five-stage pipeline each iteration takes five instruction slots, one load-use bubble (the `add` uses `t1` immediately after `lw`), and a two-cycle flush for the taken `bnez`: eight cycles. Moving one `addi` between the `lw` and the `add` removes the bubble (seven cycles), and a predictor that learns the loop branch removes the flush (five cycles). Compilers perform exactly this **instruction scheduling**; the example shows why two functionally identical binaries can differ by 60 % in speed.

Beyond the five-stage pipeline

Real cores go further: **deeper pipelines** (10–20 stages) raise the clock but lengthen the branch penalty; **superscalar** cores issue two to eight instructions per cycle; **out-of-order** cores keep a window of 100–600 instructions, execute each when its operands are ready and retire in program order, hiding miss latency behind independent work. A Cortex-M4 uses a three-stage pipeline and executes in order; a Cortex-A78 or a high-end RISC-V application core is out-of-order with a dozen or more stages. The deeper and more speculative the core, the less predictable its timing — which is why hard real-time controllers are still built on short in-order pipelines (Chapter 27).

16.6 Control units

Hardwired control

The **control unit** turns each instruction's opcode and function fields into the multiplexer selects, write enables and ALU control lines of the datapath. In a RISC core it is **hardwired**: a combinational **main decoder** driven by the 7-bit opcode, plus an **ALU decoder** driven by `funct3`, bit 5 of `funct7` and a 2-bit `ALUOp` from the main decoder. For the single-cycle datapath of Figure 16.3 and a representative RV32I subset:

Instruction	Opcode	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Branch	ALUOp	Jump
`lw`	0000011	1	I	imm	0	memory	0	00 (add)	0
`sw`	0100011	0	S	imm	1	—	0	00 (add)	0
R-type	0110011	1	—	reg	0	ALU	0	10 (funct)	0
`beq`	1100011	0	B	reg	0	—	1	01 (sub)	0
I-type ALU	0010011	1	I	imm	0	ALU	0	10 (funct)	0
`jal`	1101111	1	J	—	0	PC+4	0	—	1

The next-PC select is $\text{PCSrc} = (\text{Branch} \cdot \text{Zero}) + \text{Jump}$. In the pipelined datapath the same decoder runs in ID and the control bits travel down the pipeline registers with the instruction, each stage consuming its own bits; the hazard unit and the forwarding multiplexers are added as separate small control blocks.

The decoder is small — a few hundred gates for RV32I — and fast. Because RISC-V reserves opcode space for custom instructions, a soft-core or ASIC designer can add a decoder entry and an execution unit for a domain-specific operation (a CRC step, a fixed-point multiply-accumulate) without breaking compatibility (Chapter 35).

Microprogrammed control

The alternative, dominant in CISC machines from the 1960s to the 1980s, is **microprogrammed** control: each machine instruction triggers a sequence of **microinstructions** in a control ROM, each setting the datapath controls for one cycle. Microcode makes complex ISAs tractable and patchable; x86 processors still use it for rare complex instructions and ship **microcode updates** to work around errata and security flaws, at the cost of multi-cycle execution.

> **Status of the principle: Hardwired versus microprogrammed control**
> *Empirical.* Neither is universally better: hardwired control wins when the instruction set is simple and regular, microcode when it is large and irregular; the industry's convergence on hardwired control for common instructions and microcode for rare ones reflects decades of measured instruction frequencies, not a theorem.

16.7 Memory hierarchy

The memory wall

A processor needs, every cycle, one instruction and often one data word. A 3 GHz core issuing two loads per cycle asks for 24 GB/s of data with a latency it can hide of a few cycles, about 1 ns. Main memory is DRAM (Chapter 18), whose random-access latency is 50–100 ns: 150–300 cycles. This gap between processor and DRAM speed, which grew for decades because logic speed improved faster than DRAM latency, is the **memory wall**. No single memory technology is simultaneously large, fast, cheap and low-power, so computers use a **hierarchy** (Table 16.1): small, fast, expensive memories near the processor and large, slow, cheap ones further away.

Table 16.1 — Representative memory hierarchy (typical values, mid-2020s).

Level	Typical capacity	Latency	Bandwidth per core or device	Technology
Registers	32–200 × 64 bit	0 cycles	several operands per cycle	Flip-flops, multi-port SRAM
L1 cache (I and D)	32–64 KiB each	3–5 cycles (about 1 ns)	64–128 B/cycle	6T SRAM
L2 cache	256 KiB–2 MiB	10–16 cycles (3–5 ns)	32–64 B/cycle	SRAM
L3 cache (shared)	8–128 MiB	35–70 cycles (10–20 ns)	about 20–30 B/cycle	SRAM
Main memory	8 GB–2 TB	60–120 ns	25.6 GB/s per DDR4-3200 channel	DDR4/DDR5 DRAM
Solid-state storage	0.25–30 TB	20–100 µs (read)	3–14 GB/s (PCIe NVMe)	NAND flash
Hard disk	1–30 TB	5–10 ms	0.2–0.3 GB/s	Magnetic
MCU on-chip SRAM	16 KiB–2 MiB	0–1 wait states	1 word/cycle	SRAM
MCU on-chip flash	64 KiB–4 MiB	0–7 wait states, hidden by prefetch buffers	1 line per access	NOR flash

Locality

A hierarchy works only because programs do not access memory at random. They show:

- **Temporal locality** — a location used now is likely to be used again soon (loop counters, the instructions of a loop body, a filter's coefficients).
- **Spatial locality** — locations near one just used are likely to be used soon (sequential instructions, array elements, structure fields).

Keeping recently used blocks of neighbouring addresses in a small fast memory therefore captures most accesses. On typical code, a 32 KiB L1 data cache satisfies 90–98 % of loads.

> **Status of the principle: Locality of reference**
> *Empirical.* Observed across decades of program traces (often summarised as "90 % of execution time is spent in 10 % of the code"); it is a property of how programs are written, not a law, and it fails for pointer-chasing, hash-table and streaming workloads larger than the cache, where the hierarchy gives little help.

Average memory access time

The figure of merit of a hierarchy is the **average memory access time**:

$$\mathrm{AMAT} = t_{\mathrm{hit}} + m \times t_{\mathrm{miss}}$$

where t_{hit} is the hit time of the first level, m its miss rate (misses per access) and t_{miss} the miss penalty. For more levels the formula nests: the miss penalty of L1 is the AMAT of L2, with L2's **local** miss rate (misses of L2 divided by accesses that reach L2).

> **Status of the principle: Average memory access time**
> *Derived.* It is the expectation of access time over hit and miss outcomes; it is exact as an average but hides the distribution, so it says nothing about worst-case latency, which is what a hard real-time deadline depends on.

Worked example — AMAT and its effect on CPI. A 2 GHz application core has a 1-cycle L1 hit time (included in the pipeline), L1 miss rates of 2 % for instructions and 5 % for data, an L2 with 10-cycle hit time and 20 % local miss rate, and DRAM at 80 ns = 160 cycles. The L1 miss penalty is $t_{\mathrm{L2}} + m_{\mathrm{L2}}\, t_{\mathrm{DRAM}} = 10 + 0.20 \times 160 = 42$ cycles. The data-side AMAT is $1 + 0.05 \times 42 = 3.1$ cycles. With a base CPI of 1.2 and 0.35 data accesses per instruction (one fetch per instruction),

$$\mathrm{CPI} = 1.2 + 1 \times 0.02 \times 42 + 0.35 \times 0.05 \times 42 = 1.2 + 0.84 + 0.735 = 2.78$$

The memory system has more than doubled the CPI. Halving the L2 miss rate to 10 % would cut the miss penalty to 26 cycles and the CPI to 2.18, a 28 % speedup with no change to the core. In the motor-control port of the opening, this is the mechanism: the loop ran from DRAM-backed caches shared with other code, and the occasional slow iteration was the one that found its instructions evicted.

16.8 Caches

Organisation

A **cache** holds copies of memory in fixed-size **lines** (blocks), typically 32 or 64 bytes. Each line carries a **tag** identifying which memory block it holds, a **valid** bit, and for write-back caches a **dirty** bit. A memory address is split into three fields (Figure 16.5):

- the **offset** selects the byte within the line ($\log_2 B$ bits for line size B);

- the **index** selects a **set** ($\log_2 S$ bits for S sets);
- the **tag** is the rest, compared with the stored tags of that set.

With C bytes of capacity and **associativity** W (lines per set), $S = C/(BW)$. A **direct-mapped** cache has $W = 1$: each block can live in exactly one line. A **fully associative** cache has one set: a block can live anywhere, at the cost of comparing every tag. **Set-associative** caches (2- to 16-way) are the compromise used almost everywhere.

Worked example — address fields. A 32 KiB, 4-way set-associative L1 data cache with 64-byte lines serves a 32-bit address space. Offset: $\log_2 64 = 6$ bits. Sets: $32768/(64 \times 4) = 128$, so the index is 7 bits. Tag: $32 - 7 - 6 = 19$ bits. The address `0x2000_1A44` has offset `0x04`, index $(\texttt{0x2000_1A44} \gg 6)$ & `0x7F` $=$ `0x69` $= 105$, and tag `0x2000_1A44 >> 13` = `0x10000`. The tag store is 128×4 entries of 19 bits plus valid and dirty bits, about 10.5 kbit, or 4 % overhead on the 256 kbit data array.

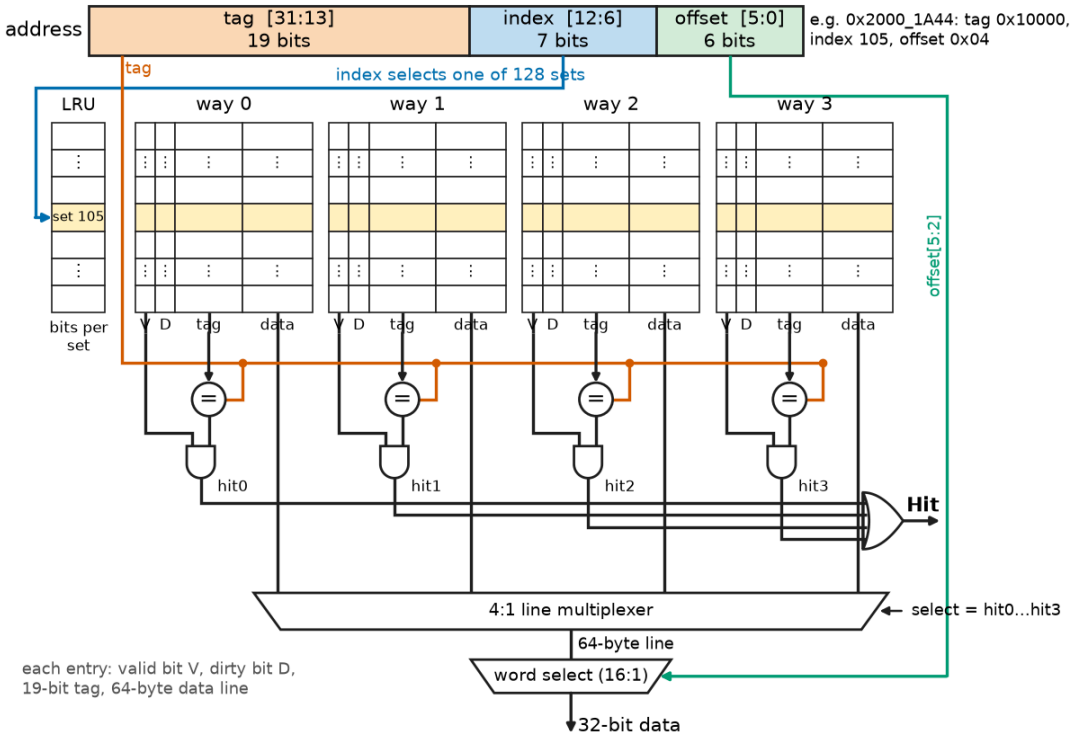


Figure 16.5 — Lookup in a 32 KiB 4-way set-associative cache with 64-byte lines.

Placement, replacement and writes

On a miss the cache fetches the whole line from the next level and must choose a victim within the set. **Least-recently-used** (LRU) replacement is exact for 2-way sets and approximated (pseudo-LRU trees) for higher associativity; random replacement is simpler and almost as good for large caches, and has the advantage for real-time analysis of not depending on access history.

Writes need a policy for two questions:

- **Hit policy.** *Write-through* updates the cache and the next level on every store, usually through a small write buffer; *write-back* updates only the cache and writes a dirty line back when it is evicted. Write-back saves bus bandwidth and is the norm for L1 and L2 data caches.
- **Miss policy.** *Write-allocate* fetches the line on a store miss; *no-write-allocate* sends the store straight to the next level. Write-back caches normally allocate.

Why misses happen

Misses fall into three classes, the **three Cs**:

1. **Compulsory** — the first access to a block; reduced by larger lines and prefetching.
2. **Capacity** — the working set does not fit; reduced only by a larger cache or a smaller working set.
3. **Conflict** — blocks that map to the same set evict each other although the cache has room; reduced by higher associativity or by changing data placement.

Conflict misses are the ones an embedded programmer causes and can fix. The simulator below replays the address trace of a dot product of two 1024-element `float32` arrays through a 4 KiB cache with 32-byte lines:

```
 from collections import OrderedDict

def miss_rate(trace, size=4096, line=32, ways=1):
    """Set-associative cache with LRU replacement; returns miss fraction."""
    n_sets = size // (line * ways)
    sets = [OrderedDict() for _ in range(n_sets)]
    misses = 0
    for addr in trace:
        block = addr // line
        s, tag = sets[block % n_sets], block // n_sets
        if tag in s:
            s.move_to_end(tag)          # hit: mark most recently used
        else:
            misses += 1
            if len(s) == ways:
                s.popitem(last=False)   # evict least recently used
            s[tag] = True
    return misses / len(trace)

# dot product of two 1024-element float32 arrays placed 4 KiB apart
A, B, N = 0x2000_0000, 0x2000_1000, 1024
trace = [x for i in range(N) for x in (A + 4*i, B + 4*i)]
for ways in (1, 2, 4):
    print(f"{ways}-way 4 KiB cache: miss rate = {miss_rate(trace, ways=ways):.3f}")
B = 0x2000_1020                          # pad b[] by one 32-byte line
trace = [x for i in range(N) for x in (A + 4*i, B + 4*i)]
print(f"1-way, b[] padded: miss rate = {miss_rate(trace, ways=1):.3f}")
```

It prints miss rates of 1.000 (direct-mapped), 0.125 (2- and 4-way) and 0.125 (direct-mapped with `b[]` padded by one line). Because the arrays start exactly one cache size apart, `a[i]` and `b[i]` map to the same line and evict each other on every access, although the cache is nowhere near full. With two ways, or a 32-byte offset, only the compulsory misses remain, one per eight elements. Power-of-two buffers — DMA ping-pong buffers, FFT arrays, image rows — are the usual culprits.

> **Status of the principle: Miss rate versus cache size**
> *Empirical.* For many workloads the miss rate falls roughly as $C^{-1/2}$ (the "square-root rule"), and doubling associativity from 1 to 2 gives about the same benefit as doubling size; both are fitted trends that depend strongly on the program, as the example's jump from 100 % to 12.5 % shows.

Caches in multicore and real-time systems

When several cores have private caches, a store by one must invalidate or update copies held by others. **Coherence protocols** such as MESI (each line Modified, Exclusive, Shared or Invalid), implemented by snooping a shared bus or

through a directory, keep the copies consistent in hardware; their cost appears as extra latency when cores share a line, and as **false sharing** when unrelated variables sit in one line. Parallel-programming consequences are treated in Chapter 24.

For hard real-time code the worst case — every access a miss after an interrupt has evicted the loop — must still meet the deadline. Embedded processors therefore offer **tightly coupled memories** (TCMs) or locked cache ways: SRAM at L1 speed whose contents the programmer controls. A Cortex-M7 runs its 20 kHz control ISR from instruction TCM and its communication stack from cached flash, so the loop's timing does not depend on what the stack did last.

16.9 Buses

What a bus does

A **bus** (more generally an **interconnect**) carries transactions between **initiators** (masters: CPU cores, DMA controllers, GPUs) and **targets** (slaves: memory controllers, peripherals). Each transaction carries an address, a direction, data and handshake signals. A bus is defined by its **width and clock**, which set peak bandwidth $\mathrm{BW} = \text{width} \times f \times (\text{transfers per clock})$; its **protocol** (whether transactions are pipelined, split, reordered or burst); and its **arbitration** among initiators (fixed priority, round-robin or bandwidth-guaranteeing quality of service).

A shared bus serialises all transfers; a **crossbar** or **bus matrix** lets independent initiator–target pairs transfer at once; large SoCs use **networks-on-chip** that packetise transactions over routed links.

On-chip buses: AMBA

Arm's AMBA family is the de facto standard for on-chip interconnect, used also in RISC-V and FPGA SoCs:

- **APB** (Advanced Peripheral Bus): unpipelined, two cycles per transfer, low power and small; used for register access to timers, UARTs and GPIO.
- **AHB / AHB-Lite** (Advanced High-performance Bus): pipelined address and data phases, bursts, single-cycle transfers; the main bus of Cortex-M microcontrollers.
- **AXI** (Advanced eXtensible Interface): five independent channels (read address, read data, write address, write data, write response), multiple outstanding transactions, out-of-order completion by transaction ID; used in application processors and FPGA SoCs. AXI-Stream drops addresses entirely for flowing data between accelerators.

Every channel uses a **valid/ready handshake**: the source asserts `VALID` when it has data, the destination asserts `READY` when it can accept, and a transfer occurs on each clock edge where both are high. The handshake makes back-pressure and clock-rate independence simple and is the single most reused protocol idea in digital hardware.

Off-chip and system buses

Bus	Width or lanes	Signalling rate	Peak bandwidth (one direction)	Typical use
APB at 100 MHz	32 bit	2 cycles per transfer	200 MB/s	Peripheral registers
AHB-Lite at 168 MHz	32 bit	1 transfer/cycle	672 MB/s	Cortex-M4 MCU main bus
AXI4 at 250 MHz	128 bit	1 transfer/cycle	4.0 GB/s	FPGA SoC, accelerator port
DDR4-3200	64 bit	3200 MT/s	25.6 GB/s	Main memory (JEDEC JESD79-4)
PCIe 3.0 ×4	4 lanes	8 GT/s, 128b/130b	3.94 GB/s	NVMe SSD
PCIe 4.0 ×16	16 lanes	16 GT/s, 128b/130b	31.5 GB/s	GPU, accelerator card
USB 2.0 high speed	1 pair	480 Mbit/s	about 40 MB/s practical	Peripherals (USB 2.0 spec)

PCIe and USB are packet-switched serial links rather than buses in the old sense, but they present the same abstraction to software: memory-mapped transactions to a device's address window. The electrical side of high-speed interconnect (reflections, termination, controlled impedance) belongs to transmission-line theory.

> **Boundary.**
> Signal integrity of fast buses — critical length, characteristic impedance and termination — is developed in *Circuits, Signals and Communications*, Ch. 14 §14.10; the DRAM interface timing is in Chapter 18 and the embedded serial links (UART, SPI, I²C, CAN, USB, Ethernet) in Chapter 26.

Address phases, turnarounds, arbitration and DRAM page misses typically cut sustained bandwidth to 50–80 % of peak, so a bus is comfortably sized when its average load stays below about 60 % of peak.

16.10 Interrupts

Exceptions, traps and interrupts

The processor's normal flow — fetch the instruction at PC, execute, advance — is broken by three kinds of events, which RISC-V calls **traps**:

- **Exceptions** — synchronous, caused by the instruction itself: illegal instruction, misaligned or faulting access, `ecall` (system call), `ebreak` (breakpoint).
- **Interrupts** — asynchronous, caused by something outside the instruction stream: a timer, a peripheral, another core.
- **Faults and resets** — bus errors, watchdog expiry, power-on.

All are handled the same way. On a trap in machine mode, a RISC-V hart (hardware thread) completes or cancels the current instruction, saves the PC of the interrupted instruction in `mepc`, writes the cause into `mcause` (most significant bit set for interrupts; code 7 = machine timer, 11 = machine external, 3 = software), copies the interrupt-enable bit `MIE` into `MPIE` and clears `MIE` so that the handler is not interrupted, and jumps to the address in `mtvec` (either one common

handler or, in vectored mode, a table indexed by cause). The handler saves the registers it uses, services the event, restores them and executes `mret`, which restores `MIE` and returns to `mepc`.

> **Status of the principle: Precise exceptions**
> *Fundamental.* By definition of the ISA contract, a trap must appear to occur between two instructions — all earlier instructions complete, no later one has changed architectural state; pipelined and out-of-order cores must flush and retire in order to honour this, and it is what makes an interrupted program resumable.

Interrupt controllers and priority

A core has few interrupt inputs; an interrupt controller multiplexes many sources onto them, assigns priorities and records which are pending. In RISC-V systems a **CLINT** (or ACLINT) provides the machine timer (`mtime`, `mtimecmp`) and software interrupts, and a **PLIC** (platform-level interrupt controller) arbitrates up to 1023 external sources with programmable priorities and per-hart thresholds. Arm Cortex-M cores integrate the **NVIC**, which adds hardware register stacking and nested preemption by priority (Chapter 17); application cores use the GIC.

Interrupts may be **level-sensitive** (the request stays asserted until the handler clears its cause in the peripheral) or **edge-triggered** (a pulse sets a pending bit). A level interrupt whose cause is not cleared re-enters the handler forever; an edge interrupt that arrives while its pending bit is already set is lost. Both failure modes appear regularly in field returns.

Interrupt latency

Interrupt latency is the time from the request at the controller to the first instruction of the handler; **response time** extends it to the point where the handler has done its useful work. It is the sum of (1) synchronising the request into the core clock domain (2–3 cycles, Chapter 14); (2) completing or abandoning the current instruction; (3) pipeline flush and vector fetch; (4) state saving, in hardware or in the handler's prologue; and (5) any time interrupts are **disabled** by other code or masked by a higher-priority handler. Items 1–4 total 12 cycles on a Cortex-M3/M4 with zero-wait-state memory and roughly 20–40 cycles on a small RISC-V core whose handler saves 16 registers in software. Item 5 is under the programmer's control and often dominates: a library that disables interrupts for 50 µs while it updates a linked list adds 50 µs to every other interrupt's worst case. The scheduling consequences — priorities, preemption thresholds and response-time analysis — are the subject of Chapter 27.

16.11 Input/output architecture

Addressing devices

A processor talks to a device through the device's **registers**: control registers it writes, status registers it reads, data registers it reads or writes. Two ways of addressing them exist:

- **Memory-mapped I/O**: device registers occupy addresses in the normal address space and are accessed with ordinary loads and stores. RISC-V and Arm use this exclusively; Chapter 17 shows the memory map of a microcontroller.
- **Port-mapped I/O**: a separate address space reached with special instructions (`in`, `out` on x86). It survives on PCs for legacy devices.

Device registers are not memory. Reading a status register may clear a flag; writing the same value twice may send two bytes; the compiler must neither cache nor reorder such accesses, which is why C declares them `volatile` and why caches mark device regions as **uncacheable** and **strongly ordered**. RISC-V's `fence` instruction and Arm's `DMB` / `DSB` barriers enforce ordering where the memory model would otherwise allow reordering.

Polling, interrupts and DMA

Three mechanisms move data between a device and memory:

1. **Programmed I/O with polling**: the CPU repeatedly reads a status register until the device is ready, then transfers a word. Simple, deterministic and wasteful; good for fast devices that are almost always ready, and for short waits at boot.
2. **Interrupt-driven I/O**: the device raises an interrupt when it is ready; the handler transfers the data. The CPU is free between events, but each event costs the interrupt overhead.
3. **Direct memory access** (DMA): a DMA controller, itself a bus initiator, transfers blocks between the device and memory without the CPU, and interrupts only when a block (or half a block, for double buffering) is complete. Scatter-gather DMA engines follow linked **descriptors** in memory, each naming a buffer address and length, so that a network controller can fill a ring of packet buffers indefinitely.

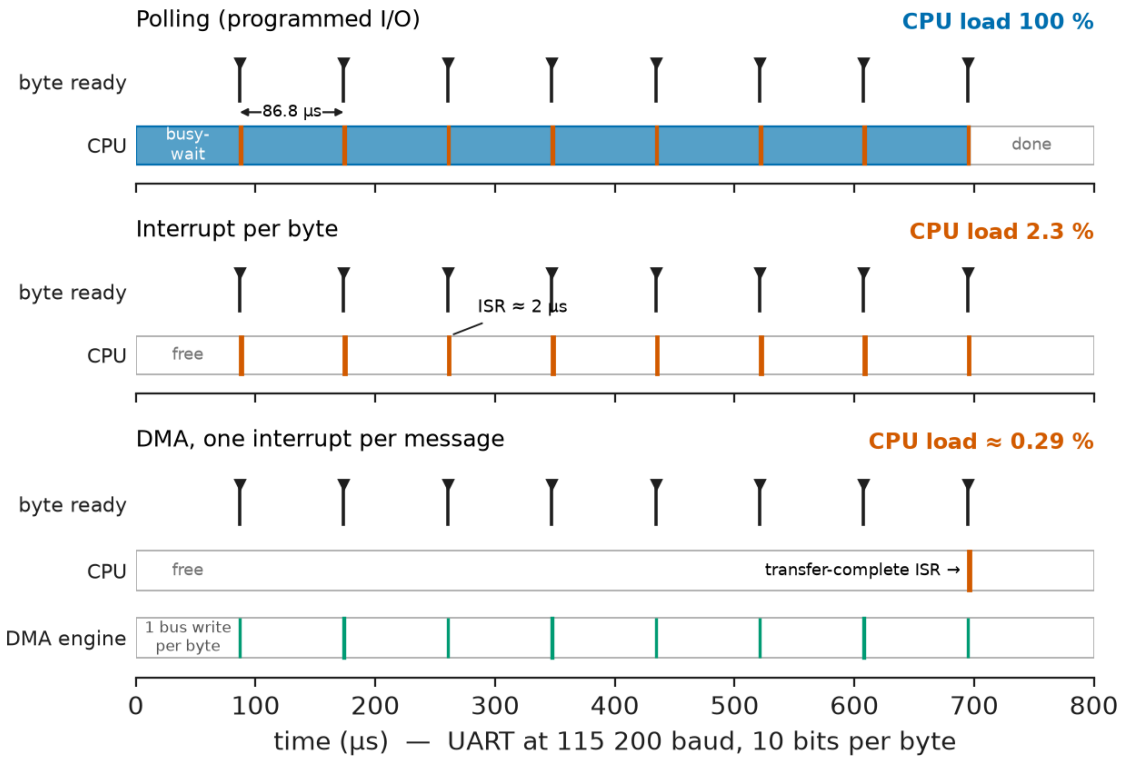


Figure 16.6 — CPU activity for polling, interrupt-driven and DMA transfer of the same 8-byte message.

Worked example — choosing polling, interrupts or DMA. A 100 MHz microcontroller receives a UART stream at 115 200 baud (10 bits per byte including start and stop bits: 11 520 bytes/s) and also a 100 Mbit/s Ethernet stream (12.5 MB/s) at full load. A receive interrupt costs 200 cycles including entry, handler and exit (Figure 16.6).

- UART with one interrupt per byte: $11\,520 \times 200 / 10^8 = 2.3\ \%$ of the CPU. Acceptable, and simpler than DMA; polling would cost 100 % of the CPU while waiting.
- Ethernet with one interrupt per byte: 12.5×10^6 interrupts per second would need 2.5×10^9 cycles per second — 25 times the CPU's capacity. Even programmed I/O at the bare minimum of one load and one store per byte would take 8 cycles per byte on a 100 MHz core that has 8 cycles per byte available in total.
- Ethernet with DMA and one interrupt per 1500-byte frame: $12.5 \times 10^6 / 1500 = 8333$ frames/s; at 500 cycles per frame interrupt the CPU load is $8333 \times 500 / 10^8 = 4.2\ \%$.

The rule that emerges is general: per-word interrupts suit event rates up to roughly a few tens of kilohertz on a microcontroller; beyond that, transfers must be batched by DMA (or by hardware FIFOs) so that the CPU pays once per block.

DMA and caches

The DMA engine reads and writes main memory, not the CPU's cache. Before a DMA transmit, dirty lines holding the buffer must be **cleaned** (written back); after a DMA receive, stale lines must be **invalidated** before the CPU reads the buffer. Getting this wrong produces the classic intermittent bug of a Cortex-M7 or Cortex-A system: packets corrupted only when the cache is on and only for some buffer alignments. The remedies are cache maintenance on line-aligned buffers, non-cacheable buffer regions, or hardware **I/O coherence** in which the interconnect snoops the caches; an **IOMMU** additionally confines each device to its own buffers.

A modern system arranges its devices by bandwidth, much as memory is arranged: low-speed peripherals on APB, DMA-capable controllers on AHB or AXI, and high-bandwidth devices on PCIe with their own DMA engines and message-signalled interrupts (MSI-X). In an industrial controller, an EtherCAT controller DMAs process data into SRAM every 250 µs and raises one interrupt, a real-time core runs the control law from TCM, and a Linux core writes logs to an NVMe drive — every layer of this chapter in one data path.

What remains open

Open question	Where it is answered
How do adders, multiplexers and comparators inside the ALU work at gate level?	Chapter 13
What sets the flip-flop overhead and maximum clock frequency of a pipeline stage?	Chapter 14 (timing constraints); Chapter 34 (static timing analysis)
How does a real microcontroller core — Cortex-M — execute, stack registers and handle nested interrupts?	Chapter 17
How are SRAM, DRAM and flash cells built, and what are DRAM timing parameters?	Chapter 18
How is a datapath described in an HDL and synthesised to an FPGA soft core?	Chapter 19
How do caches, alignment and data layout appear to a C programmer?	Chapter 21
How do coherence, false sharing and memory bandwidth limit parallel programs?	Chapter 24
How are UART, SPI, I²C, CAN, USB and Ethernet signalled and debugged?	Chapter 26
How are interrupt priorities, response times and worst-case execution times analysed for deadlines?	Chapter 27
What do SRAM and register-file transistors cost in power and area?	Chapter 33
When should a function move from the processor to an accelerator, and how is data movement modelled?	Chapter 35

Next: Chapter 17 takes these ideas into the processor engineers meet most often in embedded products, the Arm Cortex-M microcontroller, with its memory map, NVIC, timers, GPIO and serial peripherals.

Chapter 17 — Microprocessors and Microcontrollers

Part IV · Computer Architecture and Digital Hardware

A field-oriented motor drive in an e-bike runs its current loop in a timer interrupt at 20 kHz on a 168 MHz Cortex-M4. In the lab it is flawless. In the field, under a particular combination of throttle and Bluetooth traffic, the motor emits a click every few seconds and the phase current shows a single distorted PWM period. The cause turns out to be three facts about the microcontroller rather than about the control law: the Bluetooth stack's UART handler had been given the same interrupt priority as the current loop, so it could delay it by up to 40 µs; the ADC was started by software inside the interrupt instead of by the timer, so its sampling instant moved with the interrupt latency; and a flag in a GPIO register was updated with a read-modify-write that the UART handler occasionally interrupted. Each fix — a priority change, a timer-to-ADC trigger, a write to an atomic set/reset register — is one line of code, but finding it requires knowing how the processor core, its interrupt controller and its peripherals are wired together. This chapter supplies that knowledge for the processor engineers meet most often, the Arm Cortex-M microcontroller.

> **How does a microcontroller core execute instructions, address memory and peripherals, and respond to interrupts — and with what latency?**
> **How are timers, GPIO and serial peripherals organised at register level, and how are they combined so that hardware, not software, sets the timing? When is a microcontroller the right choice, and when does a design need a microprocessor running an operating system?**

Concept	Engineering destination
Processor architecture	Choosing a core (M0+, M4, M7, M33, RISC-V), estimating cycle budgets
Instruction execution	Cycle counts of control loops, wait states, running code from RAM
Memory maps	Linker scripts, register access, fault debugging, bootloaders
Addressing modes	Reading compiler output, efficient array and structure access
Interrupt handling	Latency budgets, priority assignment, nested and tail-chained handlers
Timers and counters	PWM for converters and motors, input capture, encoders, time bases
GPIO	Pin configuration, atomic bit operations, electrical limits of pins
Serial interfaces	UART, SPI and I²C peripheral blocks, baud-rate setting, DMA service
Peripheral integration	Clock trees, DMA, timer-triggered ADCs, pin multiplexing, low-power modes
Microprocessor versus microcontroller	Part selection for drives, instruments, gateways and HMIs

Roadmap. The Cortex-M core and its registers (17.1) → how it executes instructions and why wait states matter (17.2) → the memory map that places code, data and peripherals (17.3) → addressing modes (17.4) → exception and interrupt handling in the NVIC (17.5) → timers and counters (17.6) → GPIO (17.7) → serial interface blocks (17.8) → integrating peripherals with clocks, DMA and triggers (17.9) → microprocessor versus microcontroller (17.10).

17.1 Processor architecture

From core to chip

A **microcontroller** (MCU) puts a processor core, its program memory, data memory and a set of peripherals on one die, so that a single chip with a crystal and a regulator is a complete computer. Most 32-bit MCUs sold today use one of

Arm's **Cortex-M** cores, licensed to vendors such as ST, NXP, Microchip, Renesas, Infineon, TI, Nordic and Silicon Labs, who add their own memories and peripherals; RISC-V cores (Chapter 16) are growing in the same space with the same organisation. The vendor's chip, not Arm's core, sets most of what the engineer configures: clock tree, memory sizes, peripheral registers and pin assignments.

Table 17.1 — Cortex-M family (typical implementations).

Core	Architecture	Pipeline	Typical clock	Notable features	Typical use
Cortex-M0+	Armv6-M	2 stage	32–64 MHz	Smallest, about 12 000 gates; 32-cycle or single-cycle multiply option	Sensors, wearables, battery tools
Cortex-M3	Armv7-M	3 stage	50–120 MHz	Thumb-2, hardware divide, bit-banding option	General industrial control
Cortex-M4	Armv7E-M	3 stage	80–200 MHz	DSP instructions, single-cycle MAC, optional single-precision FPU	Motor control, audio, power conversion
Cortex-M7	Armv7E-M	6 stage, dual issue	200–1000 MHz	Caches, TCMs, optional double-precision FPU	High-end drives, HMI, audio
Cortex-M33	Armv8-M	3 stage	64–160 MHz	TrustZone security, DSP option	IoT nodes, secure devices
Cortex-M55/M85	Armv8.1-M	4–7 stage	200–1000 MHz	Helium vector extension	Embedded machine learning

The running example of this chapter is an STM32F4-class device: a Cortex-M4 with FPU at 168 MHz, 1 MiB of on-chip flash, 192 KiB of SRAM, two DMA controllers and the usual timers, ADCs and serial ports (Figure 17.1). Register addresses and bit positions in the listings follow that family and are marked illustrative; another vendor's part has the same structure under different names.

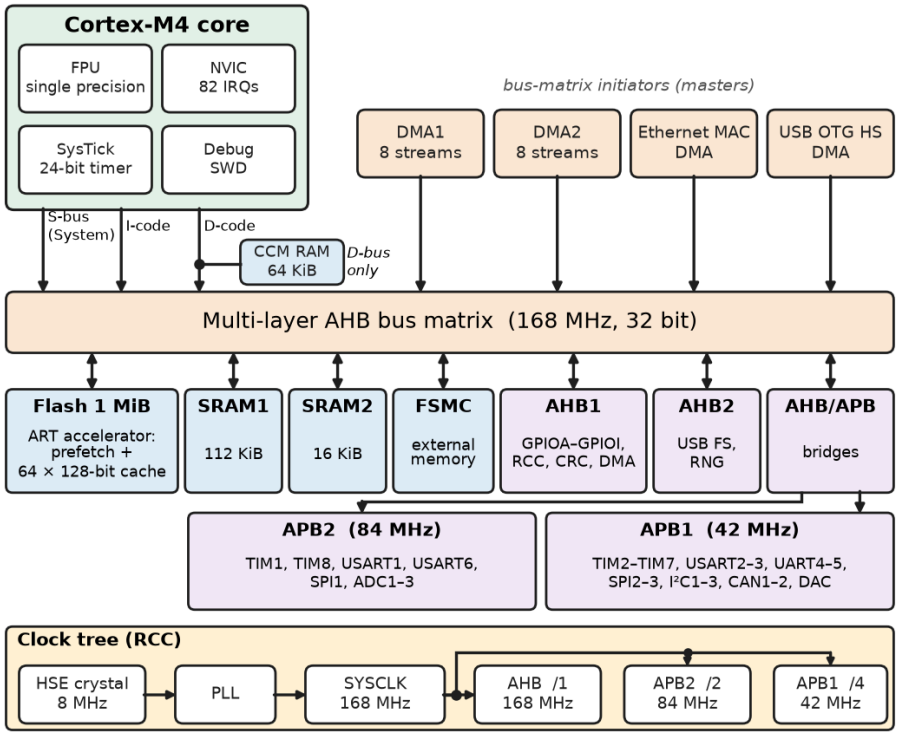


Figure 17.1 — Internal organisation of a Cortex-M4 microcontroller of the STM32F4 class.

The Cortex-M programmer's model

The Cortex-M core is a 32-bit load-store machine executing the **Thumb-2** instruction set, a mix of 16- and 32-bit encodings chosen for code density (typically 25–30 % smaller than fixed 32-bit encodings). Its registers are:

Register	Role
R0–R12	General purpose; R0–R3 carry arguments and results (AAPCS calling convention), R12 is a scratch register
R13 (SP)	Stack pointer, banked: **MSP** (main, used by handlers and after reset) and **PSP** (process, used by RTOS tasks)
R14 (LR)	Link register: return address of a call, or an `EXC_RETURN` code inside an exception handler
R15 (PC)	Program counter
xPSR	Program status: flags N, Z, C, V, Q (saturation), GE (SIMD), the Thumb bit and the active exception number
PRIMASK, FAULTMASK, BASEPRI	Interrupt masking: disable all, disable all but NMI, or mask below a priority level
CONTROL	Selects MSP or PSP in thread mode, privilege level, FPU context active
S0–S31, FPSCR	Single-precision floating-point registers (with FPU)

Two operating modes exist: **thread mode** for normal code and **handler mode** for exceptions, which always runs privileged on the MSP. An RTOS runs its tasks in thread mode on the PSP, so that each task has its own stack and the handlers share the main one (Chapter 27).

> **Boundary.**
> The ISA concepts used here — load-store design, register files, pipelines, caches, buses, precise exceptions — are developed in Chapter 16 with RISC-V as the example. This chapter adds what an MCU adds: the memory map, the NVIC and the peripherals. Signalling and protocol details of UART, SPI, I²C and CAN belong to Chapter 26; task scheduling and interrupt-priority analysis to Chapter 27.

17.2 Instruction execution

The three-stage pipeline

The Cortex-M3 and M4 use a three-stage pipeline: **fetch**, **decode** and **execute** (address generation, ALU, load/store and write-back all happen in execute). A 32-bit fetch brings in one 32-bit or two 16-bit instructions. There is a simple branch speculation: the decode stage fetches the branch target speculatively, so that a taken branch costs one to three extra cycles to refill the pipeline. Most data-processing instructions take one cycle; a load takes two cycles, but consecutive independent loads pipeline to one cycle each.

Table 17.2 — Representative Cortex-M4 instruction timings (zero-wait-state memory).

Instruction class	Cycles
ALU operation, `MOV`, shift, compare	1
`MUL`, `MLA` (32×32 → 32), `SMLAL` (→ 64), `SMLAD` (dual 16×16 MAC)	1
`UDIV`, `SDIV`	2–12 (data dependent)
`LDR` single / pipelined sequence	2 / 1 each
`LDM`, `STM`, `PUSH`, `POP` of n registers	$1 + n$
Taken branch	2–4
FPU add, multiply, multiply-accumulate (single precision)	1–3
FPU divide, square root	14

The DSP extension of Armv7E-M matters in signal-processing loops: `SMLAD` performs two 16×16-bit multiplies and adds both products to a 32-bit accumulator in one cycle, and saturating instructions such as `QADD` clip instead of wrapping, which is what a PI controller's integrator wants. A 64-tap Q15 FIR filter written with `SMLAD` and packed coefficient loads runs at about 1 to 1.3 cycles per tap; written in plain C with 32-bit arithmetic it takes 3 to 4.

Wait states and the flash bottleneck

On-chip flash is slower than the core. Embedded NOR flash reads in about 25–35 ns, while the 168 MHz clock period is 5.95 ns, so the flash controller must insert **wait states**: on STM32F4 parts, 5 wait states (6 cycles per access) at 168 MHz and 2.7–3.6 V. Straight-line code would then run at a fraction of the core's speed. Vendors hide the latency with a wide flash interface (128 bits, four to eight instructions per access), a **prefetch buffer** that fetches the next line while the current one executes, and a small instruction cache (ST's "ART accelerator": 64 lines of 128 bits) that holds loops. Branch-heavy code, large switch statements and interrupt vectors that are not cached still pay the full penalty.

The engineering consequences are concrete. A cycle budget measured with the cache warm can grow by 20–50 % after a context switch or a burst of unrelated code; time-critical routines are therefore often copied to SRAM and run from there, where every fetch is zero-wait-state. Not every RAM can do this: on STM32F4 the 64 KiB core-coupled memory (CCM) is wired only to the D-bus and cannot hold code, while on a Cortex-M7 the instruction TCM exists precisely for this purpose.

Worked example — cycle budget of a current-control interrupt. A field-oriented controller on the 168 MHz Cortex-M4 runs once per 20 kHz PWM period. The period is $168 \times 10^6 / 20 \times 10^3 = 8400$ cycles. The handler's body — Clarke and Park transforms, two PI regulators with saturation, inverse Park, space-vector modulation, written in single-precision float — measures 1500 cycles with the cycle counter (`DWT->CYCCNT`) in SRAM. Exception entry takes 12 cycles and exit about 10. The CPU load of the loop is

$$\frac{20 \times 10^3 \times (1500 + 12 + 10)}{168 \times 10^6} = 0.181 = 18.1\ \%$$

and the interrupt entry latency is $12 / 168\ \text{MHz} = 71.4$ ns. Run from flash with a cold cache, the body could take 1800–2200 cycles, raising the load to 22–26 %; the margin left for the speed loop, communications and diagnostics shrinks accordingly. The calculation also fixes the ADC timing: the current samples must be taken at the PWM centre, so they must be triggered by the timer in hardware (Section 17.9), not by the handler, whose start time varies with whatever else the core was doing.

17.3 Memory maps

The Cortex-M address space

A Cortex-M core sees one flat 4 GiB address space, divided by the architecture into fixed regions with fixed default attributes (Figure 17.2):

Address range	Region	Contents	Default attributes
`0x0000_0000` – `0x1FFF_FFFF`	Code	Flash, boot ROM, alias of boot memory at 0	Executable, cacheable
`0x2000_0000` – `0x3FFF_FFFF`	SRAM	On-chip RAM; bit-band region in the first 1 MiB	Executable, cacheable
`0x4000_0000` – `0x5FFF_FFFF`	Peripheral	Vendor peripheral registers; bit-band region in the first 1 MiB	Device, never executed
`0x6000_0000` – `0x9FFF_FFFF`	External RAM	External memory controller (SRAM, SDRAM)	Executable, cacheable
`0xA000_0000` – `0xDFFF_FFFF`	External device	Memory-mapped external devices	Device
`0xE000_0000` – `0xE00F_FFFF`	Private peripheral bus	NVIC (`0xE000_E100`), SysTick (`0xE000_E010`), System Control Block (`0xE000_ED00`), debug	Strongly ordered
`0xE010_0000` – `0xFFFF_FFFF`	Vendor system	Vendor-specific	Device

Within these regions the vendor places its memories and peripherals. On the STM32F4 example, flash starts at `0x0800_0000` (and is aliased at `0x0000_0000` when the chip boots from flash), SRAM at `0x2000_0000`, CCM RAM at `0x1000_0000`, GPIO port A at `0x4002_0000`, the reset and clock controller (RCC) at `0x4002_3800`, TIM1 at `0x4001_0000` and USART1 at `0x4001_1000`. Each peripheral occupies a block of 1 KiB or more, and its registers sit at fixed offsets from the block's base address.

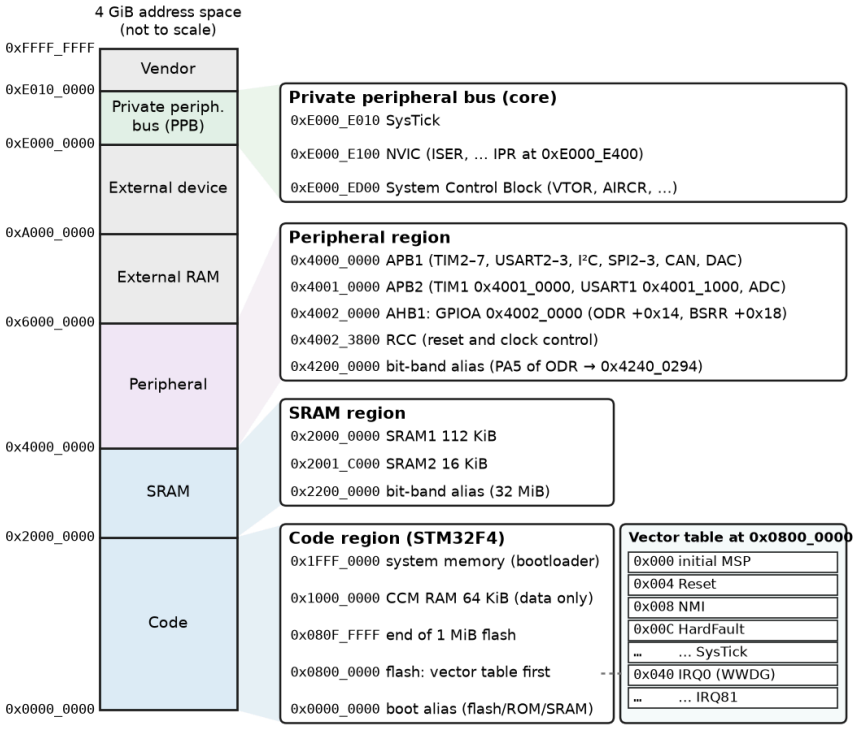


Figure 17.2 — Cortex-M memory map with the STM32F4 example placements.

The vector table and reset

The first words of the boot memory form the **vector table**. Word 0 is the initial value of the main stack pointer; word 1 is the address of the reset handler; the following words are the addresses of the fault handlers, the system exceptions and then one entry per vendor interrupt (82 on the STM32F407). On reset the core loads SP and PC from the first two words and starts executing — no boot code is needed to set up a stack before the first C function. Addresses in the table have bit 0 set to mark Thumb code. The **VTOR** register in the System Control Block relocates the table, which is how a bootloader at the start of flash hands control to an application placed higher up (Chapter 25).

Linker scripts and the map file

The memory map is made concrete for software by the **linker script**, which declares the memory regions (`FLASH ORIGIN = 0x08000000, LENGTH = 1024K`; `RAM ORIGIN = 0x20000000, LENGTH = 128K`) and places sections in them: code and constants (`.text`, `.rodata`) in flash, initialised data (`.data`) in RAM with its initial values stored in flash and copied by the startup code, zero-initialised data (`.bss`) in RAM, then heap and stack. The map file the linker emits is the first place to look when RAM overflows or a function is unexpectedly slow because it was placed in flash.

Bit-banding

The Cortex-M3 and M4 optionally map each bit of the first megabyte of the SRAM and peripheral regions to a whole 32-bit word in an **alias region**, so that a single store sets or clears one bit atomically, without a read-modify-write. The alias address is

$$A_{\text{alias}} = A_{\text{alias base}} + 32\,(A_{\text{byte}} - A_{\text{region base}}) + 4\,b$$

with alias bases `0x2200_0000` (SRAM) and `0x4200_0000` (peripherals) and b the bit number.

Worked example — register and bit-band addresses. The output data register `ODR` of GPIO port A is at offset `0x14` from `0x4002_0000`, i.e. `0x4002_0014`; the bit-set/reset register `BSRR` is at `0x4002_0018`. For bit 5 of `ODR` (pin PA5), the byte offset into the peripheral region is `0x2_0014`, and the alias address is $\texttt{0x4200_0000} + 32 \times \texttt{0x2_0014} + 4 \times 5 =$

`0x4240_0294`. Writing 1 to that word sets PA5; writing 0 clears it. The Cortex-M7 and the Armv8-M cores dropped bit-banding; GPIO blocks provide set/reset registers instead (Section 17.7), which work on every core and are the portable choice.

17.4 Addressing modes

Load-store addressing in Thumb-2

As a load-store architecture, Armv7-M computes addresses only in load and store instructions, but it offers more **addressing modes** than RISC-V's single base-plus-offset form. Each saves instructions in a common pattern:

```
@ Armv7-M (Thumb-2) addressing modes, GNU assembler syntax
        ldr   r1, =0x40020014      @ PC-relative literal: 32-bit constant from a pool
        movw  r1, #0x0014          @ ...or build it from two 16-bit halves
        movt  r1, #0x4002
        ldr   r2, [r1]             @ register indirect
        ldr   r2, [r1, #4]         @ base + immediate offset
        ldr   r2, [r1, #4]!        @ pre-indexed: r1 = r1 + 4, then load
        ldr   r2, [r1], #4         @ post-indexed: load, then r1 = r1 + 4
        ldr   r2, [r1, r3, lsl #2] @ base + scaled index: a[i] for 32-bit a
        ldrh  r2, [r1, #2]         @ halfword, zero-extended
        ldrsb r2, [r1, #1]         @ byte, sign-extended
        ldmia r1!, {r4-r7}         @ load four words, then r1 = r1 + 16
        push  {r4-r7, lr}          @ store multiple to the descending stack
```

Mode	Effective address	Typical C source
Immediate (data processing)	— (operand in instruction)	`x + 10`, constants up to a rotated 8-bit pattern
PC-relative literal	PC + offset	Large constants, peripheral base addresses
Register indirect	R_n	`*p`
Base + immediate offset	$R_n + \text{imm}$	`s->field`, `p[3]`
Pre-indexed with write-back	$R_n \leftarrow R_n + \text{imm}$, then access	`*++p`
Post-indexed	access R_n, then $R_n \leftarrow R_n + \text{imm}$	`*p++` in loops
Base + scaled register	$R_n + (R_m \ll s)$	`a[i]` with variable `i`
Multiple (`LDM` / `STM` / `PUSH` / `POP`)	consecutive words	Structure copy, function prologue and epilogue

Immediate operands in data-processing instructions are limited: Thumb-2 encodes a constant as an 8-bit value rotated or replicated across the word, so `0xFF00` fits but `0x1234` does not; such constants come from `MOVW` / `MOVT` pairs or a literal pool. Post-indexing makes the array sum of Chapter 16 a four-instruction loop (`ldr r2, [r0], #4`; `add`; `subs`; `bne`), one fewer than RV32I, and `subs` sets the Z flag that `bne` tests, so no separate compare is needed.

Where modes matter to the engineer

Addressing modes are the compiler's business in normal code, but three situations bring them to the engineer's attention. When reading a disassembly to find why a loop is slow, the presence or absence of post-indexed loads and `LDM` sequences shows whether the compiler recognised the access pattern. When structures overlay peripheral registers (Section 17.7), every field access becomes a base-plus-offset load from one base register, which is efficient — whereas computing each register address separately from macros costs a literal-pool load per access. And alignment matters: `LDR`

and `STR` of words tolerate unaligned addresses on Armv7-M at extra cost, but `LDM`, `STM` and `LDRD` fault on them, which is a classic HardFault when a packed communication buffer is cast to a structure pointer.

17.5 Interrupt handling

Exceptions on Cortex-M

Cortex-M treats every interruption of normal flow as an **exception** with a number and a vector:

Number	Exception	Priority
1	Reset	−3 (fixed, highest)
2	NMI (non-maskable interrupt; e.g. clock-failure detection)	−2 (fixed)
3	HardFault	−1 (fixed)
4–6	MemManage, BusFault, UsageFault	Configurable
11	SVCall (supervisor call, `SVC` instruction)	Configurable
14	PendSV (pendable service call, used for RTOS context switches)	Configurable
15	SysTick (24-bit system timer)	Configurable
16 and up	IRQ0, IRQ1, … vendor peripheral interrupts	Configurable

The **nested vectored interrupt controller** (NVIC), part of the core, handles up to 240 (Armv7-M) external interrupts. For each it has enable, pending and active bits and a priority field of up to 8 bits, of which the vendor implements typically 3 or 4 (8 or 16 levels; 4 bits on STM32F4), left-aligned in the byte. **Lower numbers mean higher priority.** A configurable split divides the field into **preemption priority** (which decides whether one handler can interrupt another) and **subpriority** (which only orders pending interrupts of equal preemption priority).

Exception entry and return

When an interrupt is accepted, the hardware does what a RISC-V handler's prologue does in software (Chapter 16), and more:

1. **Stacking**: it pushes eight registers — R0–R3, R12, LR, PC and xPSR — onto the current stack. These are exactly the registers the AAPCS calling convention does not require a C function to preserve, so the handler can be an ordinary C function with no special attribute.
2. **Vector fetch** in parallel with stacking, over the separate instruction bus.
3. **Mode change**: handler mode, MSP; LR is loaded with an `EXC_RETURN` value such as `0xFFFF_FFF9` (return to thread mode, main stack) or `0xFFFF_FFFD` (thread mode, process stack).
4. The first handler instruction executes **12 cycles** after the request on a Cortex-M3/M4 with zero-wait-state memory.

Return happens when the handler loads an `EXC_RETURN` value into the PC, typically through `BX LR` or `POP {..., PC}`: the hardware recognises the special value, unstacks the eight registers and resumes. With the FPU active, the frame grows to 26 words (adding S0–S15 and FPSCR); **lazy stacking** reserves the space but saves the floating-point registers only if the handler itself uses the FPU, so integer-only handlers keep the 12-cycle latency.

Three optimisations make back-to-back interrupts cheap (Figure 17.3):

- **Tail-chaining**: if another interrupt is pending when a handler returns, the core skips the unstack-and-restack and enters the next handler in 6 cycles.

- **Late arrival**: if a higher-priority interrupt arrives during the stacking of a lower one, the core uses the same stack frame and vectors to the higher-priority handler instead.
- **Pop preemption**: an interrupt arriving during unstacking abandons the unstack and enters the new handler.

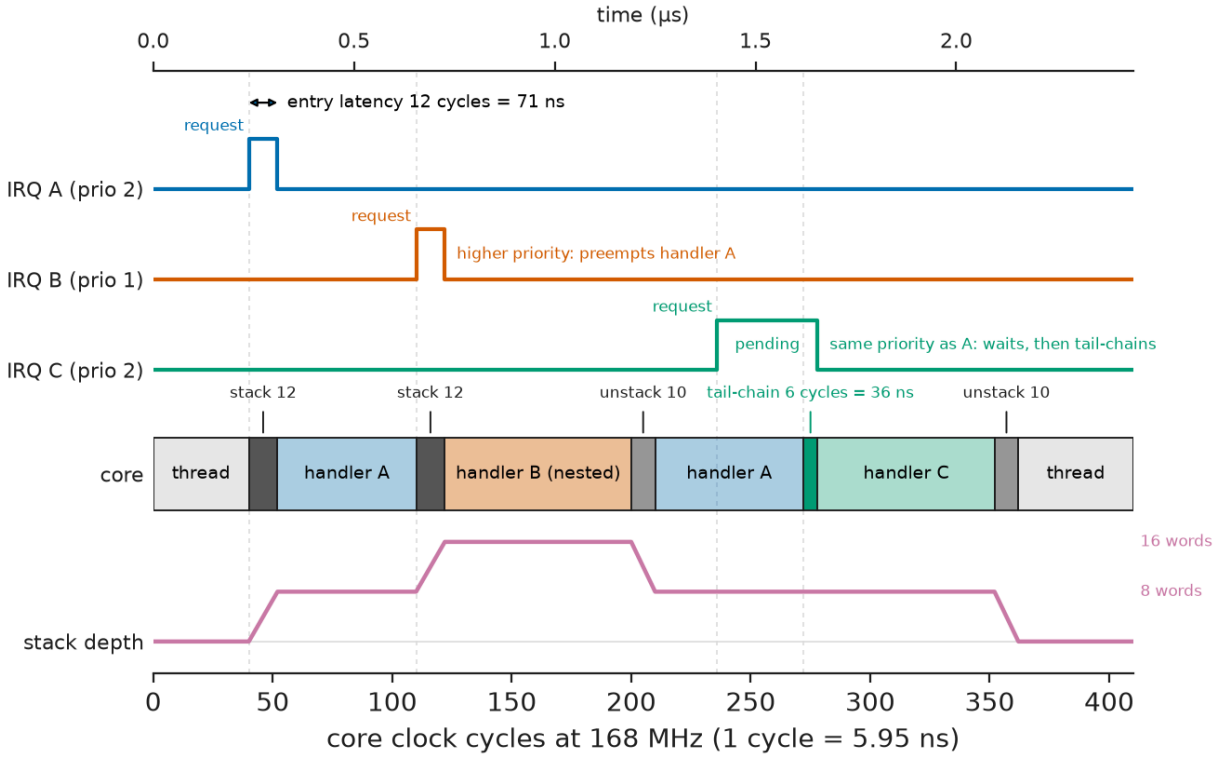


Figure 17.3 — Cortex-M4 exception timing: entry, preemption by a higher priority, and tail-chaining.

> **Status of the principle: Cortex-M exception latency**
> *Empirical.* The 12-cycle entry and 6-cycle tail-chain figures are Arm's specified values for the Cortex-M3/M4 with zero-wait-state memory and no bus contention; flash wait states on the vector fetch, a stalled bus, an in-progress multi-cycle instruction or floating-point stacking add cycles, so a measured worst case is required for a hard deadline.

Priorities, masking and critical sections

Priority assignment follows from deadlines, not importance: the shortest-deadline, shortest-handler events get the highest priorities (rate-monotonic reasoning, Chapter 27). In the e-bike drive of the opening, the correct order is PWM/current-loop interrupt highest, encoder and overcurrent next, communication lowest — and the Bluetooth UART, which had been set to the same preemption level as the current loop, could no longer delay it once moved down.

Code that shares data with a handler needs **critical sections**. `CPSID I` (setting PRIMASK) disables all configurable interrupts and is the blunt tool; writing **BASEPRI** masks only interrupts at or below a chosen priority, so that the current-loop interrupt stays live while lower-priority code updates a shared structure. Every critical section adds its length to the worst-case latency of the interrupts it masks (Chapter 16, Section 16.10).

Configuring an interrupt in C

The listing configures TIM1 for centre-aligned PWM at 20 kHz with dead time and enables its update interrupt at preemption priority 1. Register names and offsets follow the STM32F4 TIM1 block and are illustrative:

```
#include <stdint.h>                          /* C11; illustrative STM32F4-style layout */
typedef struct {
    volatile uint32_t CR1, CR2, SMCR, DIER, SR, EGR, CCMR1, CCMR2, CCER,
                      CNT, PSC, ARR, RCR, CCR1, CCR2, CCR3, CCR4, BDTR;
} TIM_TypeDef;
#define TIM1          ((TIM_TypeDef *)0x40010000u)
#define RCC_APB2ENR   (*(volatile uint32_t *)0x40023844u)
#define NVIC_ISER0    (*(volatile uint32_t *)0xE000E100u)
#define NVIC_IPR      ((volatile uint8_t *)0xE000E400u)
#define TIM1_UP_IRQn  25u

void pwm_init(void)
{
    RCC_APB2ENR |= 1u << 0;                  /* clock TIM1 (168 MHz timer clock) */
    TIM1->PSC   = 0u;
    TIM1->ARR   = 4200u;                     /* centre-aligned: 168 MHz / (2 x 4200) = 20 kHz */
    TIM1->RCR   = 1u;                        /* one update event per PWM period */
    TIM1->CCMR1 = (6u << 4) | (1u << 3);     /* channel 1: PWM mode 1, preload */
    TIM1->CCR1  = 2100u;                     /* 50 % duty */
    TIM1->CCER  = (1u << 0) | (1u << 2);     /* enable CH1 and complementary CH1N */
    TIM1->BDTR  = (1u << 15) | 0x94u;        /* main output enable, dead time 168 ticks */
    TIM1->DIER  = 1u << 0;                   /* update interrupt */
    TIM1->EGR   = 1u << 0;                   /* load preloaded registers */
    TIM1->CR1   = (1u << 7) | (1u << 5) | (1u << 0); /* ARR preload, centre mode 1, start */
    NVIC_IPR[TIM1_UP_IRQn] = 1u << 4;        /* priority 1 in the upper 4 bits */
    NVIC_ISER0 = 1u << TIM1_UP_IRQn;
}

void TIM1_UP_TIM10_IRQHandler(void)
{
    TIM1->SR = ~1u;                          /* clear UIF (bits are write-0-to-clear) */
    /* read latest ADC results, run current loop, write new CCR1..CCR3 */
}
```

The handler's name must match the symbol in the vendor's vector table; the linker then places its address in entry 16 + 25. Clearing the flag first matters: the interrupt is level-sensitive at the NVIC, so a handler that returns with `UIF` still set re-enters immediately, and because the write to the peripheral travels over the bus, clearing it at the very end can let the handler return before the flag has actually cleared — a classic cause of double-triggered interrupts.

17.6 Timers and counters

Timer anatomy

A microcontroller timer is a counter with programmable clock, period and comparison logic (Figure 17.4). Its parts are:

- a **prescaler** dividing the timer clock by $\mathrm{PSC} + 1$;
- a **counter** (`CNT`, 16 or 32 bits) counting up, down or up-down (centre-aligned);
- an **auto-reload register** (`ARR`) setting the period: the counter wraps after $\mathrm{ARR} + 1$ counts in up-counting mode;
- several **capture/compare channels** (`CCRx`), each usable as an output compare (toggle or set a pin, or generate PWM, when `CNT` matches) or as an input capture (latch `CNT` when a pin edge arrives);
- **update events** and interrupt or DMA requests on overflow, compare match or capture;
- **trigger outputs** to start other timers, ADC conversions or DMA transfers in hardware.

The update frequency of an up-counting timer is

$$f_{\text{update}} = \frac{f_{\text{timer}}}{(\text{PSC}+1)(\text{ARR}+1)}$$

and in centre-aligned mode the period is 2 ARR timer ticks.

> **Status of the principle: Timer period relation**
> *Derived.* It follows from counting clock edges; it is exact in the digital domain, so frequency accuracy is entirely that of the timer clock — a crystal at 20–50 ppm or an internal RC oscillator at 1–3 % — and the resolution is one tick.

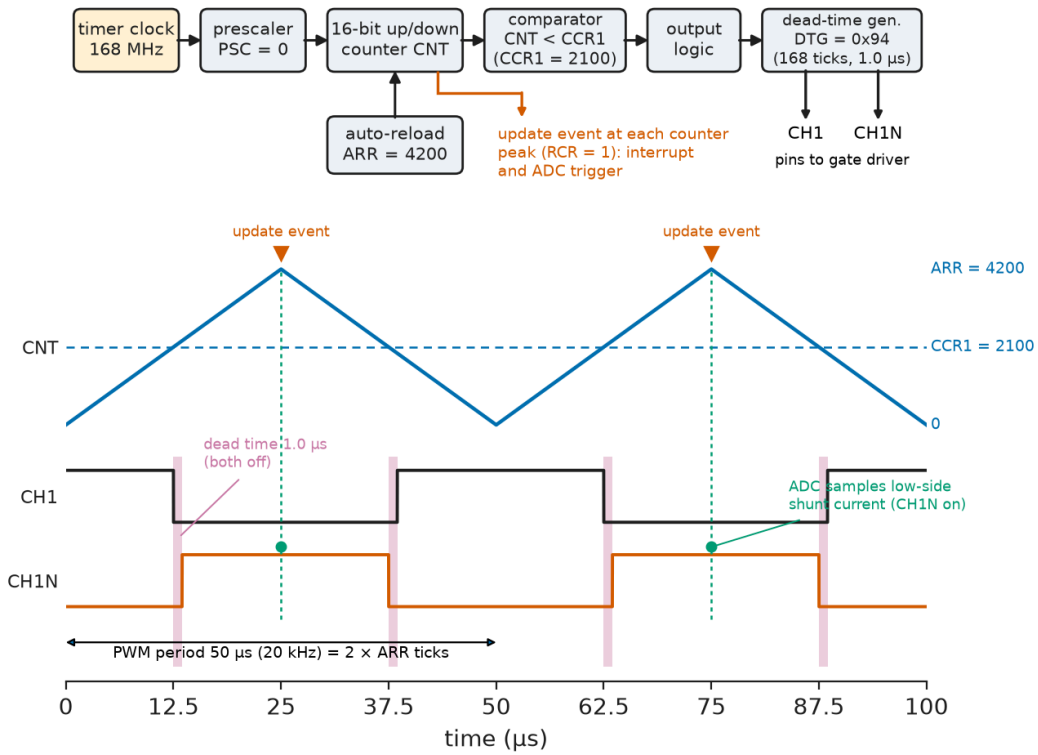


Figure 17.4 — Timer structure and centre-aligned PWM generation with complementary outputs and dead time.

Timer modes and their uses

Mode	How it works	Application
Time base	Update interrupt every $(\text{PSC}+1)(\text{ARR}+1)$ ticks	1 kHz scheduler tick, sampling clock
Output compare	Pin toggles or pulses on match	Stepper-motor pulses, ultrasonic burst timing
PWM (edge-aligned)	Output high while $\text{CNT} < \text{CCR}$	LED dimming, fan control, DC-DC converters
PWM (centre-aligned)	Up-down counting; symmetric pulses	Three-phase inverters (lower harmonics, natural ADC sample point)
Complementary outputs with dead time	High-side and low-side signals with programmable gap	Half-bridge and inverter gate drive (Chapter 32)
Break input	Hardware forces outputs to a safe state on a fault pin	Overcurrent shutdown in under 1 µs, independent of software
Input capture	`CNT` latched on pin edge	Frequency, period and pulse-width measurement; ultrasonic echo
Encoder interface	Counter clocked by quadrature A/B signals	Motor position and speed
One-pulse	Single pulse of programmed delay and width	Triac firing, sensor triggering

Worked example — PWM resolution, dead time and a time base. The TIM1 configuration of Section 17.5 uses the 168 MHz timer clock with $\text{PSC} = 0$. In centre-aligned mode at 20 kHz, $\text{ARR} = 168 \times 10^6 / (2 \times 20 \times 10^3) = 4200$, so duty cycle is set in steps of $1/4200$: a resolution of $\log_2 4200 = 12.0$ bits. An edge-aligned 20 kHz PWM would have $\text{ARR}+1 = 8400$ (13.0 bits) but no symmetric sampling point. The gate driver needs 1.0 µs of dead time: 168 ticks of the 5.95 ns clock. STM32-style dead-time registers encode values above 127 ticks with a coarser step: the code `10xxxxxx` means $(64 + \text{DTG}[5{:}0]) \times 2$ ticks, so $\text{DTG}[5{:}0] = 20$ gives $(64+20)\times 2 = 168$ ticks and the register value `0x94`. For a 1 kHz scheduler tick on a 32-bit general-purpose timer clocked at 84 MHz, $\text{PSC} = 83$ gives a 1 MHz count and $\text{ARR} = 999$ gives the 1 kHz update. A 16-bit counter at 1 MHz overflows every 65.5 ms; the 32-bit one would run 4295 s at 1 MHz, or 51.1 s at the full 84 MHz, before wrapping — long enough to timestamp events without overflow handling in most protocols.

SysTick and watchdogs

Every Cortex-M core contains **SysTick**, a 24-bit down-counter with its own exception, intended as the operating-system tick; because it is part of the core, RTOS ports use it without vendor-specific code. Vendors add a **real-time clock** on a 32.768 kHz crystal, running in the lowest power modes, and **watchdog timers**: an independent watchdog on its own RC oscillator resets the chip if software fails to refresh it in time, and a window watchdog also resets it if software refreshes too early, catching code that is looping in the wrong place. A watchdog is only useful if the refresh is placed where it proves that the whole system, not just one loop, is alive (Chapter 25).

17.7 GPIO

The pin as a configurable circuit

A **general-purpose input/output** (GPIO) pin is a small configurable circuit (Figure 17.5): an output driver that can be push-pull or open-drain with selectable slew rate; optional pull-up and pull-down resistors (typically 30–50 kΩ); an input path with a Schmitt trigger and a synchroniser into the bus clock domain; an analog switch to the ADC; and a

multiplexer that hands the pin to a peripheral as an **alternate function**. Each pin is controlled by fields in its port's registers:

Register (STM32F4 offsets)	Function
`MODER` (0x00), 2 bits per pin	Input, general-purpose output, alternate function, analog
`OTYPER` (0x04), 1 bit	Push-pull or open-drain
`OSPEEDR` (0x08), 2 bits	Output slew rate (low to very high)
`PUPDR` (0x0C), 2 bits	None, pull-up, pull-down
`IDR` (0x10)	Input levels (read only)
`ODR` (0x14)	Output latch
`BSRR` (0x18)	Write-only: bits 0–15 set, bits 16–31 reset the corresponding `ODR` bits
`AFR[0]`, `AFR[1]` (0x20, 0x24), 4 bits	Alternate-function number 0–15

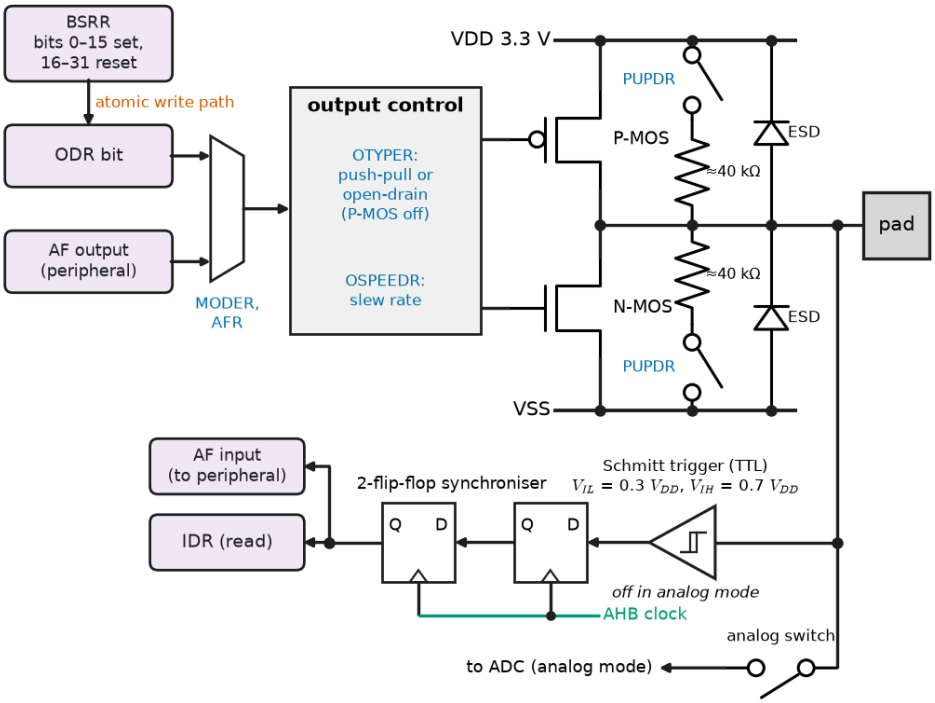


Figure 17.5 — Structure of a microcontroller GPIO pin.

Atomic access and the read-modify-write hazard

Setting one pin by `GPIOA->ODR |= 1u << 5;` compiles to a load, an OR and a store. If an interrupt handler changes another bit of the same `ODR` between the load and the store, the main code's store writes back the old value of that bit and silently undoes the handler's change — the third fault of the e-bike drive. Writing to `BSRR` is a single store that affects only the bits written as 1, so no other bit can be lost:

```
#include <stdint.h>                          /* C11; illustrative STM32F4-style layout */
typedef struct {
    volatile uint32_t MODER, OTYPER, OSPEEDR, PUPDR, IDR, ODR, BSRR, LCKR;
    volatile uint32_t AFR[2];
} GPIO_TypeDef;
#define GPIOA        ((GPIO_TypeDef *)0x40020000u)
#define RCC_AHB1ENR  (*(volatile uint32_t *)0x40023830u)

void status_led_init(void)
{
    RCC_AHB1ENR |= 1u << 0;                  /* clock GPIO port A */
    (void)RCC_AHB1ENR;                       /* read back before first access */
    GPIOA->MODER   = (GPIOA->MODER & ~(3u << 10)) | (1u << 10);  /* PA5 output */
    GPIOA->OTYPER &= ~(1u << 5);             /* push-pull */
    GPIOA->OSPEEDR &= ~(3u << 10);           /* lowest slew rate */
}

static inline void led_set(void)   { GPIOA->BSRR = 1u << 5; }
static inline void led_clear(void) { GPIOA->BSRR = 1u << (5u + 16u); }
static inline uint32_t button(void) { return (GPIOA->IDR >> 0) & 1u; }  /* PA0 */
```

The configuration writes are still read-modify-write sequences and belong in initialisation, before interrupts are enabled. The dummy read after enabling the clock makes sure the enable has reached the peripheral before its registers are written, a documented requirement on several families.

Electrical limits

A GPIO pin is a CMOS output, not a power driver. Representative limits for a 3.3 V MCU are listed below; the interface circuits that respect them (LED and relay drivers, level shifters, input protection) are in Chapter 26.

Parameter	Typical value (3.3 V supply)
Output current per pin, guaranteed levels	±8 mA with $V_{OH} \geq V_{DD} - 0.4$ V, $V_{OL} \leq 0.4$ V
Absolute maximum per pin / total per package	±25 mA / 120–150 mA
Input thresholds (CMOS/TTL compatible)	$V_{IL} \leq 0.3V_{DD}$, $V_{IH} \geq 0.7V_{DD}$, hysteresis about 0.1–0.2 V
Pull resistors	30–50 kΩ
Input leakage	±1 µA
Output edge rate at highest speed setting, 30 pF	2–4 ns, toggle up to 50–100 MHz
5 V tolerance	Only on pins marked FT, and only when not in analog mode

The slew-rate setting deserves attention: the fastest setting gives nanosecond edges whose harmonics radiate and couple into analog inputs, so it should be used only where the signal needs it (SPI at tens of MHz, external memory) and the lowest setting elsewhere.

17.8 Serial interfaces

Serial peripherals as register blocks

Microcontrollers talk to sensors, memories, radios and other processors through serial peripherals. At the architecture level each is a block of registers on the APB bus with a shift register, a baud-rate generator, status flags, interrupt and

DMA requests, and pins assigned through the GPIO alternate-function multiplexer.

Peripheral	Signals	Clocking	Typical MCU rate	Key registers
USART/UART	TX, RX (optional RTS, CTS)	Asynchronous, baud generator	9.6 kbit/s–10 Mbit/s	`BRR` (baud), `CR1` – `CR3`, `SR`, `DR`
SPI	SCK, MOSI, MISO, NSS	Synchronous, master provides clock	1–50 MHz	`CR1` (mode, prescaler), `SR`, `DR`
I²C	SDA, SCL (open-drain)	Synchronous, multi-master	100 kHz, 400 kHz, 1 MHz	`CR1`, `CR2`, `CCR`, `OAR`, `SR1`, `SR2`, `DR`
CAN (bxCAN, FDCAN)	TX, RX to transceiver	Asynchronous, bit timing segments	125 kbit/s–1 Mbit/s (5–8 Mbit/s CAN FD data)	Bit-timing, mailboxes, acceptance filters
USB device/host	D+, D–	Packet based, 48 MHz clock	12 Mbit/s (FS), 480 Mbit/s (HS)	Endpoint buffers, descriptors

> **Boundary.**
> Frame formats, timing diagrams, arbitration, electrical levels and troubleshooting of UART, SPI, I²C, CAN, USB and Ethernet are developed in Chapter 26 (with ISO 11898 for CAN and the USB 2.0 specification). This section covers only the peripheral blocks and how the processor services them.

Baud-rate generation

A UART divides its kernel clock to produce the bit rate. With 16× oversampling (each bit sampled 16 times, the middle samples voted),

$$\text{USARTDIV} = \frac{f_{\text{ck}}}{16\,\text{baud}}$$

and STM32-style parts store USARTDIV as a 12-bit mantissa and a 4-bit fraction in `BRR`.

Worked example — baud-rate error. USART1 on the APB2 bus has $f_{\text{ck}} = 84$ MHz. For 115 200 baud, $\text{USARTDIV} = 84 \times 10^6 / (16 \times 115\,200) = 45.573$. The mantissa is 45, the fraction $0.573 \times 16 = 9.17$ rounds to 9, and $\text{BRR} = (45 \ll 4) + 9 =$ `0x2D9`. The actual rate is $84 \times 10^6 / (16 \times 45.5625) = 115\,226$ baud, an error of +0.023 %. On USART2 (APB1, 42 MHz) the same calculation gives a mantissa of 22, fraction 13 and 115 068 baud, −0.11 %. Both are far inside the roughly ±2 % total mismatch an asynchronous link tolerates between transmitter and receiver (Chapter 26) — but an MCU running from its internal RC oscillator at ±1 % across temperature uses half that budget before the divider error is counted, which is why reliable UART links run from a crystal.

Servicing serial peripherals

A peripheral raises a request when its receive register is full (`RXNE`) or its transmit register is empty (`TXE`), and the core services it by polling, by interrupt or by DMA (Chapter 16, Section 16.11). At 115 200 baud one byte arrives every 86.8 µs, so an interrupt per byte is cheap. The usual structure is a **ring buffer** between the handler and the main code:

```
 #include <stdint.h>
#define USART1_SR  (*(volatile uint32_t *)0x40011000u)   /* illustrative */
#define USART1_DR  (*(volatile uint32_t *)0x40011004u)
#define SR_RXNE    (1u << 5)
#define SR_ORE     (1u << 3)

static volatile uint8_t rx_buf[256];
static volatile uint8_t rx_head, rx_tail;    /* 8-bit indices wrap at 256 */

void USART1_IRQHandler(void)
{
    uint32_t sr = USART1_SR;
    if (sr & (SR_RXNE | SR_ORE)) {
        uint8_t c = (uint8_t)USART1_DR;      /* SR then DR read clears RXNE and ORE */
        uint8_t next = (uint8_t)(rx_head + 1u);
        if (next != rx_tail) {               /* drop the byte if the buffer is full */
            rx_buf[rx_head] = c;
            rx_head = next;
        }
    }
}

int uart_getc(void)                          /* called from main code; -1 if empty */
{
    if (rx_tail == rx_head) return -1;
    uint8_t c = rx_buf[rx_tail];
    rx_tail = (uint8_t)(rx_tail + 1u);
    return c;
}
```

The handler is the only writer of `rx_head` and the main code the only writer of `rx_tail`; each 8-bit index is read and written with single instructions, so no critical section is needed on a single-core MCU. The overrun flag must be cleared too: on these parts an overrun that is not cleared keeps the interrupt asserted forever. Synchronisation in the general case — multiple writers, multicore parts, RTOS queues — is the subject of Chapter 27. For SPI flash reads, audio streams and anything above a few hundred kilobytes per second, DMA replaces the per-byte handler: an SPI at 21 MHz moves 2.6 MB/s, one byte every 381 ns, far too fast for interrupts.

17.9 Peripheral integration

Clock tree and power gating

Every peripheral on an MCU is fed from a **clock tree** in the reset and clock controller: an external crystal (HSE, 8–25 MHz) or internal RC oscillator feeds a PLL that produces the system clock (168 MHz here), which is divided for the AHB bus (168 MHz), the high-speed APB2 (84 MHz) and the low-speed APB1 (42 MHz); timers on an APB bus with a divider other than 1 receive twice the APB clock. Each peripheral has an enable bit; after reset almost everything is off, and a register write to a peripheral whose clock is disabled is silently ignored — the most common "the peripheral does nothing" bug. Disabling unused clocks also saves power: dynamic CMOS power scales with switched capacitance and frequency (Chapter 33).

Table 17.3 — Representative power modes of 32-bit MCUs (typical, 25 °C).

Mode	What runs	High-performance M4 at 168 MHz	Ultra-low-power M4/M33 class	Wake-up time
Run	Core and enabled peripherals	40–90 mA (about 250–500 µA/MHz)	30–110 µA/MHz	—
Sleep	Core clock stopped, peripherals run	15–60 mA	10–40 µA/MHz	a few cycles
Stop (deep sleep)	All high-speed clocks off, SRAM retained	0.3–0.5 mA	1–5 µA	5–20 µs
Standby / shutdown	Only RTC and backup domain	2–4 µA	0.03–0.5 µA	50 µs–1 ms (reset)

DMA, triggers and event routing

The integration that matters most for real-time behaviour is the ability of peripherals to start one another without the CPU. On the STM32F4, two DMA controllers with eight streams each move data between peripherals and memory, and timers can trigger ADC conversions directly. A motor drive therefore chains: TIM1 reaches the centre of its PWM period → its trigger output starts an injected ADC conversion of two phase currents and the DC-link voltage → the ADC's end-of-conversion raises the interrupt in which the control law runs (or a DMA transfer stores the results). The sampling instant is fixed to the timer within one ADC clock, independent of interrupt latency — the second fix in the e-bike drive. Newer families generalise this with an event matrix that routes any peripheral's event to any other's trigger input.

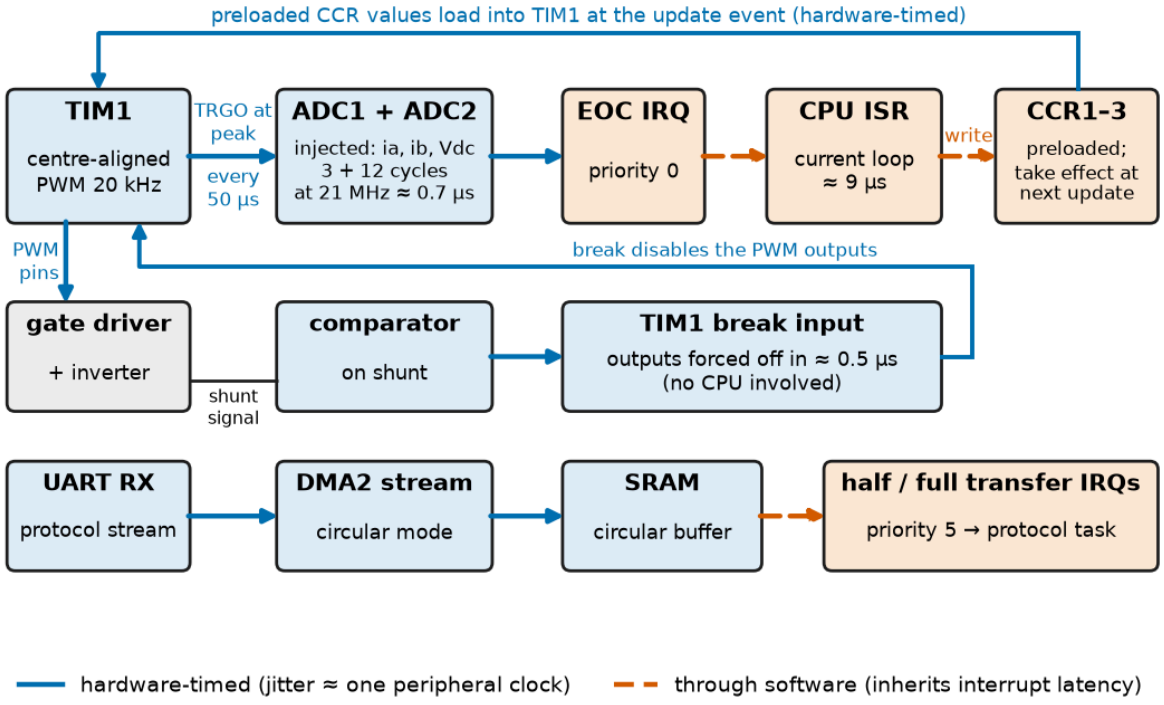


Figure 17.6 — Hardware-triggered peripheral chain of a motor-control MCU.

The same thinking applies across the chip (Figure 17.6). An overcurrent comparator wired to the timer's break input turns the inverter off in under a microsecond whether or not the software has crashed. A UART receiving a protocol stream uses circular DMA, with interrupts at half and full buffer, so the CPU handles data in blocks. **Pin multiplexing** ties it together: a 100-pin package exposes perhaps 140 peripheral signals, each pin offering up to 16 alternate functions, and assigning them is an early, constraining step of board design that vendor configuration tools automate.

> **Status of the principle: Hardware timing over software timing**
> *Empirical.* Experience across embedded designs shows that events tied to hardware triggers (timer-to-ADC, comparator-to-break, DMA) have jitter of one peripheral clock, while the same events started in software inherit interrupt latency and its variation, typically 0.1–50 µs; it holds as long as the trigger chain is configured correctly and the CPU only consumes results.

17.10 Microprocessor versus microcontroller

Two kinds of chip

A **microprocessor** (MPU), in the embedded sense, is a processor chip that relies on external memory: an application core (Cortex-A, a RISC-V application core, x86) with a memory-management unit, caches, a DRAM controller and high-speed interfaces, booting an operating system such as Linux from flash or eMMC into external DRAM. A **microcontroller** is a self-contained computer with on-chip flash and SRAM, running bare-metal code or an RTOS directly from flash, with deterministic timing and fast wake-up. The boundary has blurred — Cortex-M7 "crossover" MCUs reach 1 GHz, and some MPUs embed a Cortex-M core for real-time tasks — but the design questions that decide between them are stable.

Attribute	Microcontroller (Cortex-M4 class)	Microprocessor / application SoC (quad Cortex-A53 class)
Clock, performance	48–200 MHz, about 1.25 DMIPS/MHz	1–2 GHz, 2–3 DMIPS/MHz per core, several cores
Memory	64 KiB–2 MiB flash, 16 KiB–1 MiB SRAM on chip	External DDR3/DDR4/LPDDR4, 256 MB–8 GB; eMMC or NAND storage
Memory protection	MPU (8–16 regions), no virtual memory	MMU with virtual memory, caches, IOMMU
Software	Bare metal or RTOS (FreeRTOS, Zephyr), single image	Linux or Android, processes, file systems, drivers
Boot time	Under 1 ms to `main()`	2–30 s to application
Interrupt latency	12 cycles (under 0.1 µs), deterministic	1–10 µs typical, tens of µs worst case under Linux; PREEMPT_RT improves the tail
Power	1–100 mW active, µW asleep	0.5–5 W active, tens of mW in suspend
Board	2–4 layer PCB, single supply	6–10 layer PCB, PMIC with several rails, DRAM routing rules
Typical uses	Motor drives, converters, sensors, wearables, automotive ECUs	HMIs, gateways, cameras, vision, network appliances

Choosing

The decision follows from requirements, not from performance alone:

- **Hard real-time control** (current loops at 10–100 kHz, protection, safety functions) wants a microcontroller or a dedicated real-time core, because its timing can be bounded (Chapter 27).
- **Rich software** — a graphical interface, a TCP/IP stack with TLS and many connections, a file system, a camera pipeline, a Python runtime — wants an MPU with Linux.
- **Power and start-up**: a battery sensor that wakes for 2 ms every second and sleeps at 2 µA can only be an MCU.

- **Cost and board complexity**: an MPU needs external DRAM, a power-management IC and a denser PCB; the bill of materials is typically several times that of an MCU design.

Many products combine both. An EV charger or an industrial servo drive runs its power stage from a Cortex-M or DSP-class controller and its user interface, networking and logging from a Cortex-A running Linux, linked by SPI, UART or shared memory; heterogeneous SoCs (an application cluster plus a Cortex-M core on one die) put the pair on one chip. Where neither is fast or deterministic enough — a 1 MHz control loop, a radar front end, a custom protocol at gigabits per second — the computation moves into programmable logic, the subject of Chapter 18, and the partitioning decision itself is treated in Chapters 25 and 35.

What remains open

Open question	Where it is answered
How are pipelines, caches and buses designed and analysed in general?	Chapter 16
How are flash and SRAM built, and how is external memory interfaced?	Chapter 18
How are C pointers, `volatile`, bit operations and peripheral drivers written well?	Chapter 21
How is a microcontroller chosen, booted, updated and its firmware structured?	Chapter 25
How do UART, SPI, I²C, CAN, USB and Ethernet work on the wire, and how are they debugged?	Chapter 26
How are interrupt priorities and RTOS task priorities assigned so that deadlines are provably met?	Chapter 27
How do timer PWM and dead time translate into gate drive and inverter behaviour?	Chapters 29 and 32
How are field-oriented and other drive control laws designed?	*Electrical Energy and Machines*, Ch. 12
What sets the dynamic and leakage power of the chip in each mode?	Chapter 33
When should a control loop or filter move from the MCU to an FPGA?	Chapter 35

Next: Chapter 18 opens the memories that every processor depends on — SRAM, DRAM and flash — and the programmable logic that takes over where a processor is too slow or too unpredictable.

Chapter 18 — Memory and Programmable Logic

Part IV · Computer Architecture and Digital Hardware

A data logger on a wind-turbine pitch controller works perfectly for nineteen months and then begins to lose its configuration at random. The cause is not a software bug: the firmware writes a 64-byte status record to two 16 KiB sectors of the microcontroller's internal flash every ten seconds, and those sectors, rated for 10 000 erase cycles, have simply worn out. In the same turbine, an FPGA that generates the PWM for a 2 MW converter misbehaves now and then — across a fleet of turbines on a high plateau, about one unexplained trip a month — because a cosmic-ray neutron flips a bit of the configuration memory that defines its logic, and the design has no scrubbing. And in the control room's server, a database query runs three times slower on a machine with twice the DRAM, because the extra modules forced the memory channel to a lower speed grade. Memories and programmable logic are often treated as commodities that simply hold bits or implement a netlist. They are circuits with timing, wear, refresh, upset rates and interface rules, and every one of those faults follows from how the cells and arrays are built. This chapter builds them up from the cell to the system.

> **How do SRAM, DRAM and flash store a bit, and what do their cell structures imply for speed, density, refresh, endurance and retention?**
> **How is a memory connected to a processor, and what limits the bandwidth and latency actually achieved? How does an FPGA implement arbitrary logic with look-up tables and programmable interconnect, and when is it worth using one to accelerate a computation?**

Concept	Engineering destination
SRAM and DRAM	On-chip memory sizing, DRAM timing and refresh, server memory configuration
ROM and Flash	Firmware storage, data logging, wear levelling, SSD and eMMC behaviour
Memory interfaces	External SRAM and SDRAM on MCUs, DDR layout, QSPI/OSPI flash, HBM
FPGA architecture	Selecting an FPGA, reading utilisation reports, power and configuration
LUTs	Mapping logic to resources, estimating LUT counts, timing of logic levels
Configurable logic blocks	Carry chains, distributed RAM, shift registers, packing efficiency
Programmable interconnects	Routing delay, congestion, why FPGA clock rates are lower than ASICs
CPLDs	Glue logic, power sequencing, instant-on board management
Hardware acceleration	Throughput and latency gains, Amdahl's limit, data-movement costs
FPGA applications	Telecom, data centres, motor control, imaging, aerospace, prototyping

Roadmap. Static and dynamic RAM cells and DRAM organisation and timing (18.1) → read-only and flash memory (18.2) → how memories connect to processors (18.3) → the FPGA as an architecture (18.4) built from LUTs (18.5), configurable logic blocks (18.6) and programmable interconnect (18.7) → CPLDs (18.8) → hardware acceleration (18.9) and the applications that use it (18.10).

18.1 SRAM and DRAM

The 6T SRAM cell

A **static RAM** cell stores a bit in two cross-coupled CMOS inverters, a bistable latch (Chapter 14), accessed through two NMOS pass transistors gated by a **word line** (Figure 18.1a). The six-transistor (6T) cell holds its state as long as power is applied, with no refresh. To read, both **bit lines** are precharged high and the word line is raised; the side of the cell storing 0 pulls its bit line down slowly through the access and pull-down transistors, and a **sense amplifier** detects a differential of 50–150 mV and resolves it to full logic levels long before the bit line has swung fully. To write, drivers force the bit lines to complementary full levels and overpower the cell's pull-up transistors.

The cell must be stable when read and writable when written, and these demands conflict: the pull-down transistor must be stronger than the access transistor (cell ratio about 1.5–2.5), so that reading does not flip the cell, while the access transistor must be stronger than the pull-up (pull-up ratio below about 1) so that a write succeeds. The margin between them, the **static noise margin**, shrinks with supply voltage and with transistor variability, which is why SRAM sets the minimum operating voltage of most digital chips and why large SRAM arrays use assist circuits and redundant rows.

SRAM is the fastest memory that can be built in a logic process, and the densest that needs no special process steps, so it is used for register files, caches, MCU RAM and FPGA block RAM. Its weaknesses are area and leakage. A 6T cell occupies roughly 120–150 F^2 (with F the real minimum feature pitch, about 12–13 nm in a "5 nm-class" process, not the node name): about 0.021 µm² there, so 1 MiB of raw cells takes about 0.18 mm² before sense amplifiers, decoders and redundancy roughly double it. SRAM bit-cell scaling has nearly stopped below 5 nm, which is one reason large caches are moving to separate stacked dies. In a large SoC, SRAM leakage can be a substantial fraction of standby power (Chapter 33).

The 1T1C DRAM cell

A **dynamic RAM** cell is one access transistor and one storage capacitor (Figure 18.1b): charge on the capacitor represents the bit. The cell is about 6 F^2 in modern DRAM processes, roughly twenty times denser than SRAM, which is why main memory is DRAM. The price is that reading is destructive and the charge leaks away.

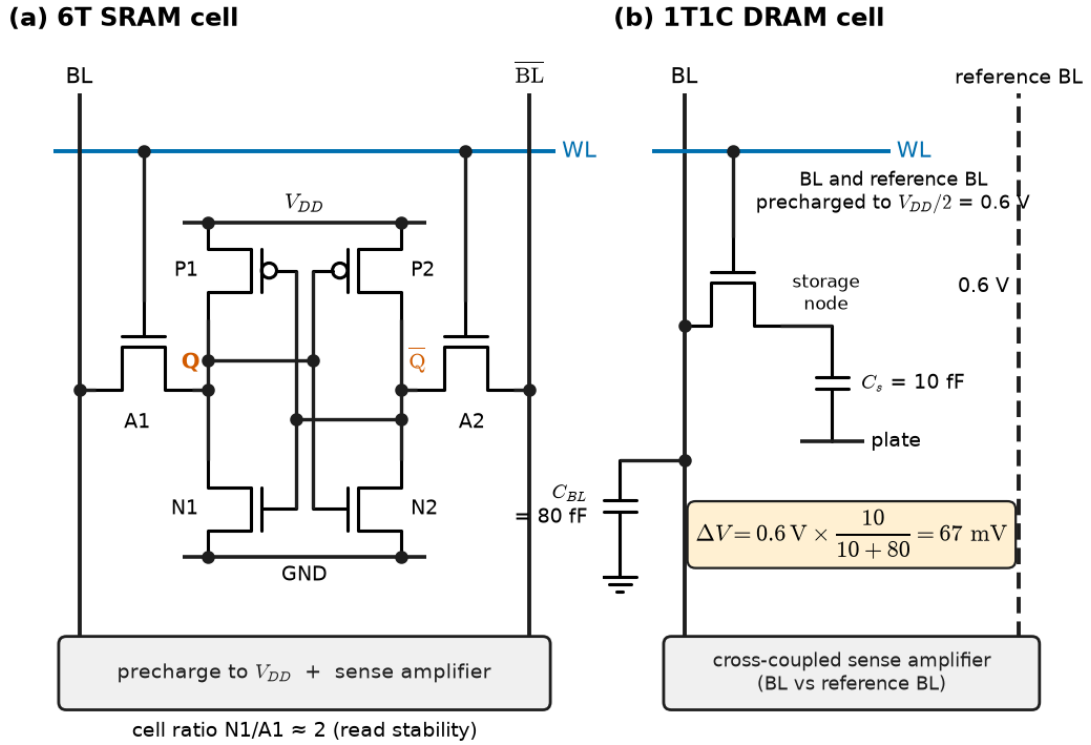


Figure 18.1 — (a) Six-transistor SRAM cell and (b) one-transistor one-capacitor DRAM cell with bit-line sensing.

To read, the bit line is precharged to $V_{DD}/2$ and the word line opens the access transistor; the cell's charge is shared with the much larger bit-line capacitance. Charge conservation gives the bit-line change

$$\Delta V = \pm \frac{V_{DD}}{2}\,\frac{C_s}{C_s + C_{BL}}$$

> **Status of the principle: DRAM charge-sharing signal**
> *Derived.* It follows from charge conservation between two capacitors; it neglects the access transistor's threshold drop (removed by boosting the word line above V_{DD}), coupling between adjacent bit lines and the charge already lost to leakage, all of which reduce the real signal.

Worked example — DRAM read signal and retention. A DRAM with $V_{DD} = 1.2$ V has $C_s = 10$ fF and $C_{BL} = 80$ fF. A stored 1 (1.2 V) shared with a bit line at 0.6 V gives $\Delta V = 0.6 \times 10/90 = 66.7$ mV, which a cross-coupled sense amplifier amplifies to full levels — and in doing so rewrites the cell, restoring the charge the read destroyed. If the sensing circuits need the cell to retain at least 0.9 V (0.3 V of droop) after one 64 ms refresh interval, the total leakage from the 10 fF node must stay below $10\ \text{fF} \times 0.3\ \text{V}/64\ \text{ms} = 47$ fA. Holding a few femtoamperes per cell over billions of cells, at 85 °C, is what makes DRAM processes so specialised: deep-trench or tall stacked capacitors, very-low-leakage access transistors, and a refresh interval that halves at high temperature.

DRAM organisation

A DRAM chip is organised hierarchically: **banks** (16 in DDR4, in four bank groups; 32 in DDR5), each an array of **rows** (pages) of, typically, 1–2 KiB per chip, with a **row buffer** of sense amplifiers per bank. Access is by command:

1. **ACTIVATE** (ACT) opens a row: its entire contents are sensed into the row buffer. The time until a column can be read is t_{RCD}.
2. **READ** or **WRITE** accesses a column of the open row. Data appear after the **CAS latency** CL and stream out as a **burst** (8 transfers in DDR4, 16 in DDR5).
3. **PRECHARGE** (PRE) closes the row and restores the bit lines, taking t_{RP}. A row must stay open at least t_{RAS}, so the minimum row cycle is $t_{RC} = t_{RAS} + t_{RP}$.
4. **REFRESH** (REF) commands, issued on average every $t_{REFI} = 7.8$ µs (64 ms over 8192 commands, halved above 85 °C), each block the device for t_{RFC} (about 350 ns for an 8 Gbit DDR4 die).

A **row hit** (the wanted row is already open) costs only CL; a **row miss** costs $t_{RP} + t_{RCD} + CL$. Memory controllers reorder requests to maximise row hits and spread traffic across banks so that one bank's activate overlaps another's data transfer (Figure 18.2).

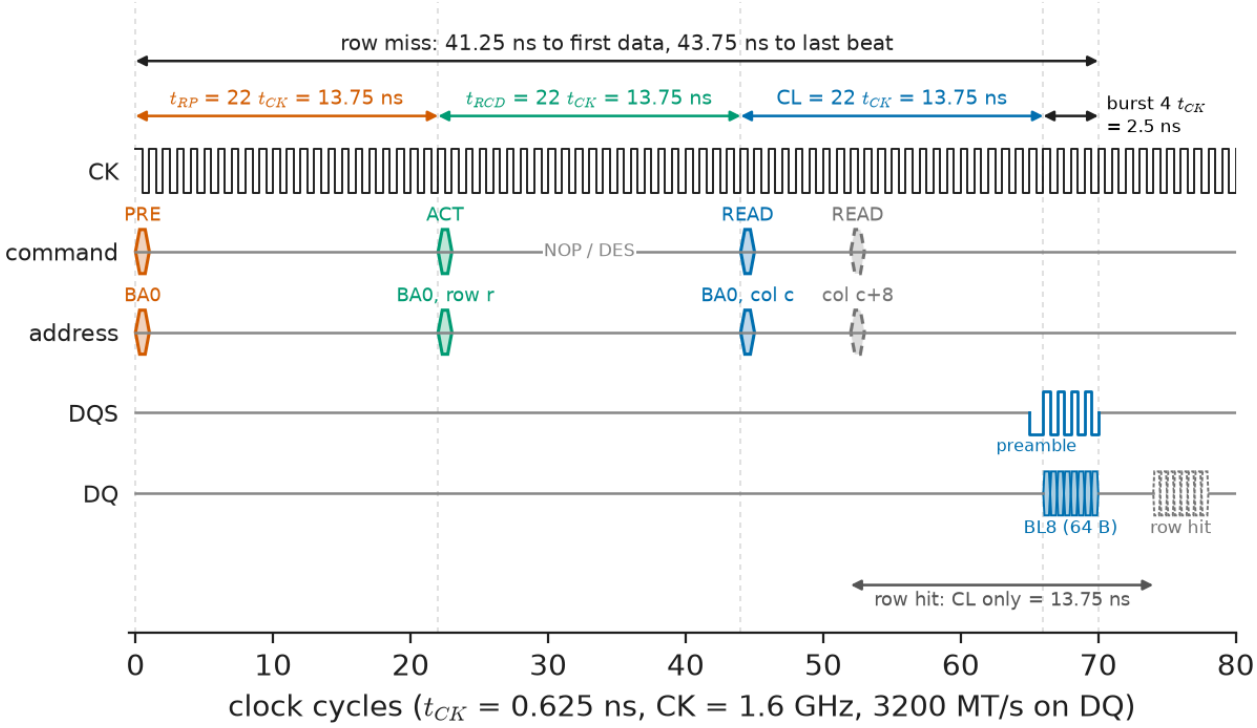


Figure 18.2 — DDR4-3200 read with a row miss: precharge, activate, read and burst.

Table 18.1 — Representative memory technologies (typical values).

Technology	Cell	Density driver	Access time	Volatile	Endurance (write cycles)	Standards and typical parts
On-chip SRAM	6T	Logic process	0.3–2 ns	Yes	Unlimited	Caches, MCU RAM, FPGA BRAM
Async SRAM chip	6T	4–32 Mbit per chip	8–55 ns	Yes	Unlimited	10 ns 8 Mbit parts for MCU expansion
DDR4 SDRAM	1T1C	8–16 Gbit per die	13–15 ns CL, 40–50 ns row miss	Yes	Unlimited	JEDEC JESD79-4, 1.2 V, up to 3200 MT/s
DDR5 SDRAM	1T1C	16–32 Gbit per die	14–17 ns CL	Yes	Unlimited	JEDEC JESD79-5, 1.1 V, 4800–8400 MT/s
LPDDR4X/LPDDR5	1T1C	Mobile, low power	similar to DDR	Yes	Unlimited	JEDEC JESD209-4/-5
HBM3	1T1C, stacked dies	16–24 GB per stack	similar to DDR	Yes	Unlimited	JEDEC JESD238, 1024-bit interface, about 819 GB/s per stack
NOR flash	Floating gate	1 Mbit–2 Gbit	10–100 ns read	No	10^4–10^5	SPI/QSPI/OSPI parts, MCU embedded flash
NAND flash	Floating gate or charge trap, 3D	0.5–2 Tbit per die	25–100 µs page read	No	10^3–10^5 (QLC to SLC)	ONFI, eMMC (JESD84), UFS, NVMe SSDs
EEPROM	Floating gate, byte erase	1 kbit–4 Mbit	100 ns–1 µs read, 3–5 ms write	No	10^5–10^6	I²C/SPI configuration memories
FRAM	Ferroelectric capacitor	4 kbit–16 Mbit	50–100 ns	No	10^{12}–10^{14}	Data loggers, metering
MRAM	Magnetic tunnel junction	1 Mbit–1 Gbit, embedded	10–35 ns	No	10^8 to effectively unlimited	Embedded NVM at 22–28 nm

Worked example — DDR4 latency, bandwidth and refresh overhead. A DDR4-3200 module with timings 22-22-22 (CL-t_{RCD}-t_{RP}) has a 1.6 GHz clock, $t_{CK} = 0.625$ ns, and transfers on both edges. $CL = 22 \times 0.625 = 13.75$ ns, and so are t_{RCD} and t_{RP}. A row hit delivers its first data after 13.75 ns, a row miss after $3 \times 13.75 = 41.25$ ns, and the 8-beat burst of a 64-byte cache line adds $4\,t_{CK} = 2.5$ ns, for 43.75 ns. The peak bandwidth of a 64-bit channel is $3200 \times 10^6 \times 8\ \text{B} = 25.6$ GB/s. Refresh steals $t_{RFC}/t_{REFI} = 350\ \text{ns}/7.8125\ \mu\text{s} = 4.5\ \%$ of the time, twice that above 85 °C. These figures explain the server of the opening: populating two modules per channel forced the controller from DDR4-3200 to DDR4-2666 for signal-integrity reasons, cutting peak bandwidth by 17 % — and a memory-bound query lost more than that, because row-miss latency in nanoseconds barely improves with speed grade while queueing under load grows.

> **Status of the principle: DRAM retention and the 64 ms refresh interval**
> *Empirical.* Cell retention times follow a wide statistical distribution measured on production DRAM; JEDEC fixes 64 ms (32 ms above 85 °C) so that the weakest cells of a qualified die survive with margin, and phenomena such as variable retention time and row-hammer disturbance (repeated activation of one row leaking charge from its neighbours) are handled by additional mitigation, not by the basic interval.

18.2 ROM and Flash

Read-only and one-time-programmable memory

Mask ROM stores bits in the presence or absence of a transistor or contact, defined by a lithography mask: the densest and most robust read-only storage, used for boot ROMs and fixed tables, but changeable only by a new mask set. **One-time-programmable** (OTP) memories use antifuses or eFuses blown by a high current or voltage after manufacture; they store calibration trims, device keys and security configuration. Erasable memories descend from the **EPROM**, whose floating-gate cells were erased by ultraviolet light through a quartz window, and the **EEPROM**, erasable electrically byte by byte at the cost of two transistors per cell. Small serial EEPROMs (I²C or SPI, 1 kbit to 4 Mbit, 10^6 write cycles, 3–5 ms per page write) remain the standard place for configuration and calibration data.

The floating-gate transistor

Flash memory stores charge on a **floating gate** (or, in most modern NAND, in a silicon-nitride **charge-trap** layer) between the MOSFET channel and the control gate, insulated by oxide all round (Figure 18.3). Electrons on the floating gate raise the transistor's threshold voltage: a cell with a high threshold reads as 0 (programmed), one with a low threshold as 1 (erased). Charge is moved through the tunnel oxide by **Fowler–Nordheim tunnelling** under a field of about 10 MV/cm (around 15–20 V across the stack), or injected as hot electrons from the channel in NOR cells. Because the oxide is about 7–10 nm thick, the stored charge stays for years when no field is applied.

> **Boundary.**
> Quantum tunnelling through a thin barrier, the physics behind flash program and erase and behind gate-oxide leakage, is treated in *Engineering Physics*, Ch. 4 §4.4; the magnetic tunnel junction used in MRAM in Ch. 14 §14.9. This section uses only the engineering consequences: high programming voltages, block erase, finite endurance and finite retention.

Each program/erase cycle damages the tunnel oxide slightly, creating traps that capture charge and shift the threshold distributions; eventually the programmed and erased states can no longer be distinguished reliably. Retention also falls with temperature and with accumulated cycles. Storing 2, 3 or 4 bits per cell (MLC, TLC, QLC) by placing 4, 8 or 16 threshold levels within the same voltage window multiplies density but narrows the margins, so endurance falls by roughly an order of magnitude per extra bit.

> **Status of the principle: Flash endurance and retention ratings**
> *Empirical.* Ratings such as 10 000 cycles with 20-year retention at 85 °C, or 3000 cycles for TLC NAND, come from accelerated stress tests and Arrhenius extrapolation (JEDEC JESD47 and JESD218); real wear-out is gradual and statistical, and ratings hold only with the specified ECC, temperature and erase-block usage.

NOR and NAND

The two flash architectures differ in how cells are connected in the array (Figure 18.3):

- **NOR flash** connects each cell between a bit line and ground, like a NOR gate's parallel transistors. Any word can be read randomly in 10–100 ns, so code can **execute in place** (XIP) directly from it. Programming is by word or page (a 256-byte page in serial NOR takes about 0.4–3 ms); erasure is by sector (4 KiB in serial NOR, 16–128 KiB in MCU embedded flash), taking 20 ms to 2 s. Endurance is 10^4–10^5 cycles. Microcontroller program memory and serial boot flash are NOR.
- **NAND flash** connects 32 to over 100 cells in series in a string, like a NAND gate's series transistors, removing most contacts and making the cell far smaller; 3D NAND stacks more than 200 layers of cells vertically. Reads are by page (4–16 KiB) in 25–100 µs, programs by page in 200 µs–2 ms, and erasure by block (several MiB in 3D NAND) in 2–10 ms. Raw NAND has bit errors from the start and develops more with wear, so it requires strong

error-correcting codes (BCH or LDPC), **bad-block management** and a **flash translation layer** (FTL) that maps logical sectors to physical pages, never rewrites a page in place, and garbage-collects partly valid blocks. SSDs, eMMC, UFS and SD cards contain NAND plus a controller that does all this.

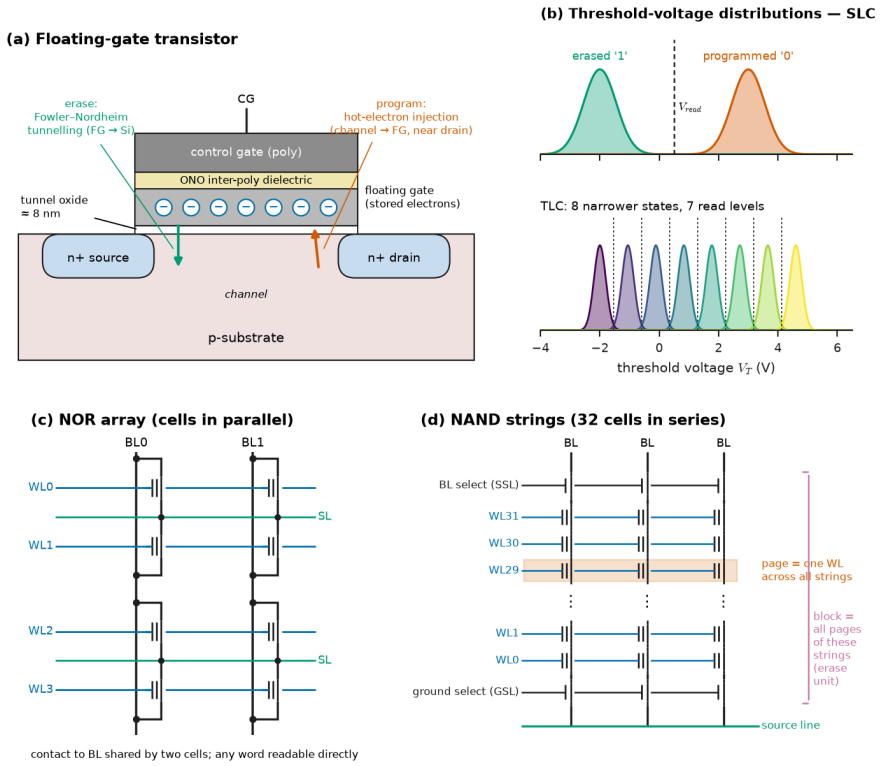


Figure 18.3 — Floating-gate cell and NOR versus NAND array organisation.

Living with finite endurance

The data logger of the opening shows the arithmetic every firmware engineer must do.

Worked example — flash wear in a data logger. The logger writes one 64-byte record every 10 s, 8640 records per day, into two 16 KiB sectors of MCU flash rated for 10 000 erase cycles, filling one sector while the other holds the previous data. Each sector holds $16384/64 = 256$ records, so the pair is erased $8640/256 = 33.75$ times per day, 16.9 times per sector. The sectors reach their rating after $10\,000/16.9 = 593$ days, 1.6 years — matching the nineteen-month failure. Spreading the log over four sectors doubles the life to 3.2 years, still short of a 20-year turbine. Moving the log to an external 16 Mbit SPI NOR flash rated for 100 000 cycles, with 4 KiB sectors and wear levelling across 256 sectors (1 MiB), gives $8640/64 = 135$ sector erases per day, 0.53 per sector, and a wear-out time of about 520 years: endurance is no longer the limit, and data retention and the controller's power-fail behaviour become the design issues. FRAM, with 10^{12} or more cycles and no erase, removes the problem entirely for small logs.

Two further rules follow from the cell physics. An erase or program interrupted by power loss leaves a sector in an undefined state, so logs and configuration stores use two copies, sequence numbers and checksums, and verify after writing. And programming flash on an MCU usually stalls instruction fetch from the same flash bank for milliseconds — long enough to miss many control-loop deadlines unless the code doing the write runs from RAM or the device has a second bank (Chapter 25).

18.3 Memory interfaces

Asynchronous parallel memory

The oldest interface is the asynchronous parallel bus: address lines, a bidirectional data bus, and active-low chip enable (`CE#`), output enable (`OE#`) and write enable (`WE#`). Timing is set by the memory's parameters — address access time t_{AA}, output-enable time, write-pulse width — and the processor's external memory controller is programmed with matching numbers of clock cycles for address setup, data phase and hold. Microcontrollers use such controllers (ST's FSMC/FMC, NXP's SEMC) for external SRAM, NOR flash, display controllers and FPGAs that present a memory-like interface.

Worked example — external SRAM timing on a 168 MHz MCU. A 10 ns asynchronous SRAM (8 Mbit, 16-bit data) is connected to the external memory controller of the Cortex-M4 of Chapter 17, clocked at $f_{HCLK} = 168$ MHz ($t_{HCLK} = 5.95$ ns). The data phase must cover the SRAM's 10 ns access time plus about 3 ns of input setup at the MCU and 1 ns of board propagation: 14 ns, so it needs $\lceil 14/5.95 \rceil = 3$ cycles. With one cycle of address setup, a read takes 4 cycles, 23.8 ns, and transfers 2 bytes: a sustained 84 MB/s for back-to-back reads. That is ample for a frame buffer of a 480×272 display at 16 bits per pixel and 60 Hz (15.7 MB/s), but code executed from the external SRAM would run about four times slower than from internal flash with its accelerator — so the external memory holds data, not the control loop.

Synchronous DRAM and DDR

SDRAM replaced the asynchronous interface with a clocked command interface (Section 18.1). **DDR** SDRAM transfers data on both clock edges and uses a **source-synchronous** strobe, DQS, sent with each byte of data so that the receiver samples with a clock that has travelled the same path. At 3200 MT/s a bit lasts 312 ps, comparable with the propagation delay of 5 cm of PCB trace, so DDR interfaces depend on controlled-impedance routing, on-die termination, length matching within bytes, a fly-by clock and command topology across the chips of a module, and **training** at power-up: the controller sweeps its sampling delays for each byte lane, finds the centre of the data eye, and stores the result. DDR5 adds on-die ECC, two independent 32-bit subchannels per module and decision-feedback equalisation in the receivers. A memory interface at these rates is a transmission-line design problem first and a logic problem second.

> **Boundary.**
> Reflections, characteristic impedance, termination and the critical length of interconnects are developed in *Circuits, Signals and Communications*, Ch. 14 (§14.10 for high-frequency interconnects); this chapter takes them as given.

Serial memory interfaces

Most non-volatile memory in embedded products is now attached serially, trading pins for latency:

Interface	Data lines	Typical clock	Throughput	Typical use
SPI NOR (single)	1 in, 1 out	50–133 MHz	6–16 MB/s	Boot flash, configuration, logs
Quad SPI (QSPI)	4 bidirectional	100–133 MHz, SDR or DDR	50–100 MB/s	XIP code, FPGA configuration
Octal SPI / xSPI (JESD251), HyperBus	8 bidirectional, DDR	166–200 MHz	330–400 MB/s	XIP on crossover MCUs, PSRAM
eMMC (JESD84)	8, DDR	200 MHz (HS400)	up to 400 MB/s	Managed NAND in industrial and automotive SoCs
UFS 3.1/4.0	Serial lanes (M-PHY)	11.6–23.2 Gb/s per lane	2–4 GB/s	Phones, automotive infotainment
NVMe over PCIe 4.0 ×4	4 lanes	16 GT/s	about 7 GB/s	Servers, PCs, edge AI boxes

A QSPI controller in **memory-mapped mode** makes external flash appear in the processor's address space, so code can execute from it through a cache; the first access to a line costs a command and address phase of several hundred nanoseconds, so cache behaviour dominates performance. At the high end, **HBM** stacks DRAM dies on a silicon interposer next to the processor with a 1024-bit interface per stack, reaching about 819 GB/s per HBM3 stack (6.4 Gb/s per pin) at much lower energy per bit than DDR — the memory of GPUs, AI accelerators and high-end FPGAs.

18.4 FPGA architecture

Logic you can rewrite

A **field-programmable gate array** (FPGA) is a chip whose logic function is defined after manufacture by loading a configuration. Instead of fixed gates and wires, it contains a regular array of programmable logic blocks, a programmable routing network, and programmable I/O, together with hard blocks — memories, multipliers, clock managers, transceivers and sometimes processors — embedded in the fabric (Figure 18.4). The configuration, a **bitstream** of a few megabits to hundreds of megabits, sets every look-up table's contents and every routing switch.

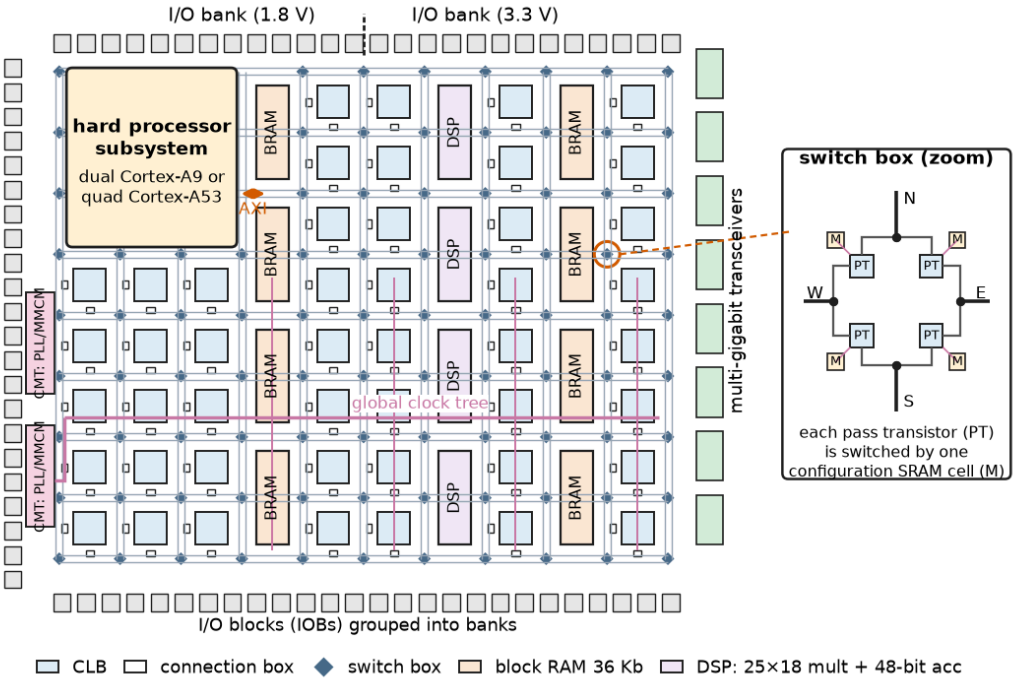


Figure 18.4 — Island-style FPGA architecture with hard-block columns.

Configuration technologies

How the configuration is stored decides much of an FPGA's behaviour:

Technology	How a switch is set	Volatile	Strengths	Weaknesses	Examples
SRAM-based	SRAM cell drives pass transistor or mux	Yes; loaded at power-up	Densest, fastest, latest processes, unlimited reprogramming	External configuration flash, boot time 10–500 ms, configuration upsets from radiation	AMD (Xilinx) 7-series, UltraScale+; Intel Cyclone, Agilex; Lattice ECP5
Flash-based	Floating-gate switch	No	Instant-on, low static power, configuration immune to upsets	Older processes, lower density	Microchip PolarFire, IGLOO2
Antifuse	One-time fused link	No (OTP)	Highest radiation tolerance, security	Not reprogrammable	Microchip RTAX space FPGAs

SRAM-based FPGAs dominate by volume and capability. They load their bitstream at power-up from a QSPI flash or a processor, which means the system has a configuration time, a dependence on an external part, and a security question (bitstreams can be encrypted with AES-256 and authenticated). They also have a reliability question: the configuration SRAM is a very large memory, and a single-event upset in it changes the circuit itself, not just a data value. At sea level a large SRAM FPGA sees configuration upsets at a rate of the order of one per several years per device; a fleet of hundreds of devices therefore sees them every month or so, and the neutron flux rises several-fold on a mountain and by two orders of magnitude at aircraft altitudes; safety and aerospace designs use **configuration scrubbing** (continuous readback with ECC correction), triple modular redundancy of critical logic, or flash and antifuse parts. The turbine converter of the opening needed exactly this.

A representative device range

Device (family)	Logic	Memory	DSP	Other hard blocks	Typical use
Lattice iCE40 UltraPlus UP5K	5280 4-input LUTs	1 Mbit SPRAM, 120 kbit block RAM	8 multiply-accumulate blocks	I²C, SPI, RGB LED drivers; static power about 100 µW	Sensor hubs, wearables, glue
AMD Artix-7 XC7A35T	20 800 6-input LUTs, 41 600 flip-flops	50 × 36 kbit block RAM (1.8 Mbit)	90 DSP48E1	6.6 Gb/s transceivers (some packages), MMCM/PLL	Motor control, industrial I/O, video bridges
AMD Artix-7 XC7A100T	63 400 LUTs, 126 800 flip-flops	135 × 36 kbit (4.9 Mbit)	240 DSP48E1	as above	Instrumentation, software-defined radio
AMD Zynq UltraScale+ MPSoC (mid-range)	100 000–500 000 LUTs	5–35 Mbit block RAM plus UltraRAM	700–2500 DSP slices	Quad Cortex-A53, dual Cortex-R5F, GPU, DDR4 controller	ADAS, 5G radio, machine vision
Microchip PolarFire MPF300	about 300 000 logic elements (4-LUT plus flip-flop)	about 20 Mbit	about 900 18×18 math blocks	12.7 Gb/s transceivers, flash configuration	Low-power and radiation-sensitive systems

18.5 LUTs

A memory that computes

The basic logic element of almost every FPGA is the **look-up table** (LUT): a small SRAM of 2^k configuration bits addressed by k logic inputs, whose selected bit is the output (Figure 18.5). Any Boolean function of k inputs is a truth table of 2^k entries (Chapter 12), so a k-input LUT implements every one of the 2^{2^k} such functions with identical delay: $2^{16} = 65\,536$ functions for a 4-LUT and $2^{64} \approx 1.8 \times 10^{19}$ for a 6-LUT. In hardware the LUT is a tree of 2-to-1 multiplexers with the configuration bits at its leaves and the inputs driving the select lines.

> **Status of the principle: Universality of the look-up table**
> *Fundamental.* A Boolean function of k variables is completely defined by its 2^k-entry truth table, so a 2^k-bit memory addressed by the variables realises any such function; the only limits are k and the number of LUTs, never the function's form.

The synthesis tool's technology mapper cuts the logic network into pieces of at most k inputs and computes each LUT's contents, the **INIT** value. The listing reproduces that last step for two functions:

```
def lut_init(f, k):
    """Truth table of a k-input function packed as a LUT INIT word.
    Bit i of INIT holds f(inputs) where input j is bit j of the index i."""
    init = 0
    for i in range(2**k):
        bits = [(i >> j) & 1 for j in range(k)]
        if f(*bits):
            init |= 1 << i
    return init

def lut_eval(init, *inputs):
    """What the hardware does: the inputs form an address into the INIT bits."""
    addr = sum(b << j for j, b in enumerate(inputs))
    return (init >> addr) & 1

f = lambda a, b, c, d: (a & b) | (c ^ d)          # a 4-input function
init = lut_init(f, 4)
print(f"LUT4 INIT = 16'h{init:04X}")
assert all(lut_eval(init, a, b, c, d) == f(a, b, c, d)
           for a in (0, 1) for b in (0, 1) for c in (0, 1) for d in (0, 1))

# majority of five inputs in one LUT6 (sixth input unused)
maj5 = lambda a, b, c, d, e, _: int(a + b + c + d + e >= 3)
print(f"LUT6 INIT = 64'h{lut_init(maj5, 6):016X}")
print("distinct 6-input functions:", 2**2**6)
```

It prints `LUT4 INIT = 16'h8FF8`, `LUT6 INIT = 64'hFEE8E880FEE8E880` and the 20-digit count 2^{64}. The 4-input value can be checked by hand: $c \oplus d = 1$ for indices 4–11 (bits `0x0FF0`), and $a \cdot b = 1$ with $c \oplus d = 0$ adds indices 3 and 15 (`0x8008`). A 5-input majority needs a multi-level network of gates; in a 6-LUT it costs one LUT and one logic level, and so does any other function of up to six inputs.

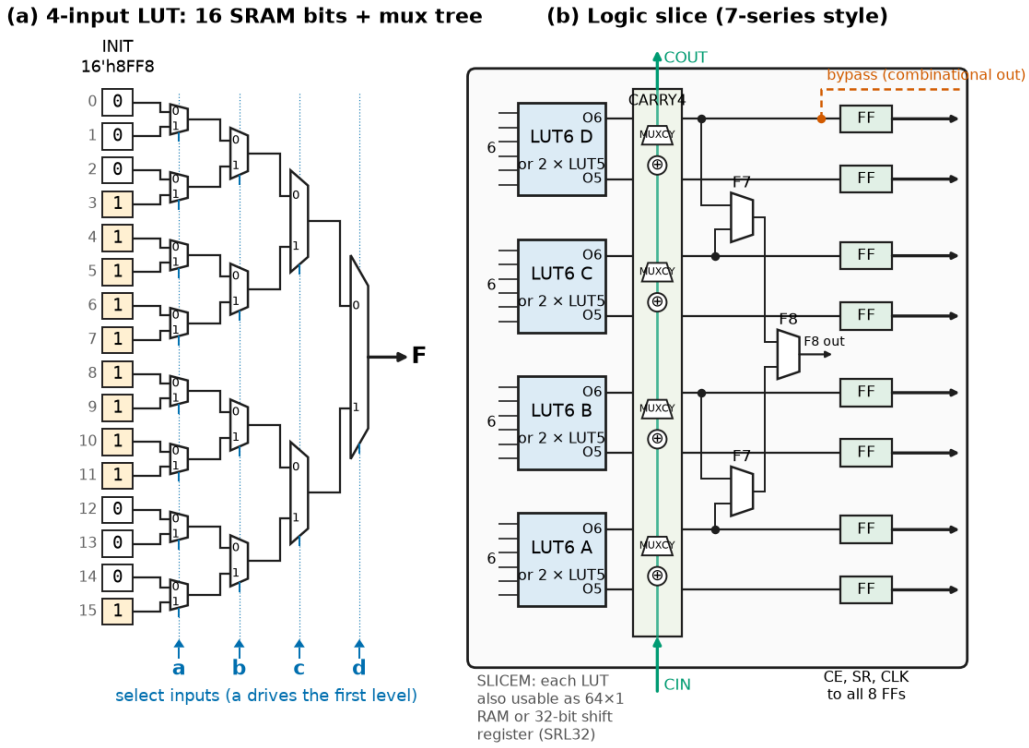


Figure 18.5 — A 4-input LUT as a multiplexer tree, and a logic slice built from 6-input LUTs.

LUT size and logic depth

LUT size is an architectural trade-off. A larger LUT absorbs more logic per level, so fewer levels and fewer slow routing hops lie on the critical path, but its area grows as 2^k and most of its bits go unused for simple functions. Studies and two decades of products settled on 4-input LUTs for low-cost and low-power parts and 6-input LUTs, often **fracturable** into two 5-input LUTs sharing inputs, for high-performance families. A useful rule of thumb for estimating an FPGA design's speed is the number of LUT levels on its critical path: each level costs a LUT delay of roughly 0.1–0.2 ns plus a routing delay of 0.3–1 ns in a 28 nm-class device, so a path of four to six levels closes at 200–300 MHz while a path of twelve does not reach 100 MHz.

18.6 Configurable logic blocks

Inside a CLB

LUTs are grouped with flip-flops and arithmetic support into **configurable logic blocks** (CLBs; Intel calls the corresponding unit a logic array block of **adaptive logic modules**, ALMs). In AMD 7-series devices a CLB contains two **slices**, each with:

- four 6-input LUTs, each usable as one 6-input or two 5-input functions;
- eight flip-flops (two per LUT), with clock enable and synchronous or asynchronous set/reset;
- a dedicated **carry chain** running vertically through the slices for fast addition, subtraction, counters and comparators;
- wide multiplexers that combine LUT outputs into 7- and 8-input functions;
- in about a quarter to a half of the slices (SLICEM), LUTs that can also act as **distributed RAM** (64×1 bit per LUT) or **shift registers** (up to 32 stages per LUT).

The carry chain is the reason an FPGA adds quickly: a 32-bit adder uses one LUT per bit to form the propagate signal and the hard carry logic to ripple the carry through dedicated wiring at about 10–30 ps per bit, instead of through general

routing at hundreds of picoseconds per hop. Its cost is that the adder must be placed in a vertical column.

The flip-flop-rich structure (two per LUT) makes **deep pipelining** cheap: inserting registers between LUT levels costs almost nothing in area and is the standard way to raise an FPGA design's clock rate, at the cost of latency.

Hard blocks beside the CLBs

Logic that would be inefficient in LUTs is provided as hardened blocks in dedicated columns:

- **Block RAM** (BRAM): 18–36 kbit true dual-port synchronous SRAMs with configurable width (36 kbit as 32k×1 up to 512×72), optional output registers, ECC and FIFO logic; AMD UltraScale+ adds 288 kbit UltraRAM blocks. A design's memory is inferred from HDL templates — the short Verilog-2001 module below maps to one block RAM because its read is registered:

```
 // Simple dual-port RAM, 1024 x 16, registered read (Verilog-2001)
module sdp_ram #(parameter AW = 10, parameter DW = 16) (
    input  wire          clk,
    input  wire          we,
    input  wire [AW-1:0] waddr,
    input  wire [DW-1:0] wdata,
    input  wire [AW-1:0] raddr,
    output reg  [DW-1:0] rdata
);
    reg [DW-1:0] mem [0:(1<<AW)-1];
    always @(posedge clk) begin
        if (we) mem[waddr] <= wdata;
        rdata <= mem[raddr];      // one-cycle read latency -> block RAM
    end
endmodule
```

Making the read combinational (declaring `rdata` as a `wire` driven by `assign rdata = mem[raddr];`) would instead infer distributed LUT RAM, which suits small, fast memories but would consume 256 LUTs for this size. HDL coding styles and inference rules are developed in Chapter 19. The other hard blocks are:

- **DSP slices**: a pre-adder, a 25×18 (7-series) or 27×18 (UltraScale+) multiplier and a 48-bit accumulator with pattern detection, running at 400–700 MHz; filters, FFTs, matrix arithmetic and control laws use these, not LUTs. Intel's variable-precision DSP blocks also support single-precision floating point.
- **Clock management**: PLLs and mixed-mode clock managers synthesise and phase-shift clocks; a dedicated low-skew global clock network distributes them.
- **I/O**: banks of pins supporting LVCMOS, LVDS, SSTL and other standards with programmable drive, termination and delay; SerDes for source-synchronous interfaces; multi-gigabit transceivers (6.6–58 Gb/s) for PCIe, Ethernet and JESD204 converter links.
- **Processor subsystems**: SoC FPGAs (AMD Zynq, Intel Agilex SoC, Microchip PolarFire SoC with RISC-V cores) integrate Arm or RISC-V processors with caches and DDR controllers, linked to the fabric by AXI ports — the platform for co-design (Chapter 35).

18.7 Programmable interconnects

Routing architecture

In an island-style FPGA the CLBs sit in a sea of routing (Figure 18.4). **Routing channels** of wire segments run horizontally and vertically between them; **connection boxes** connect CLB pins to channel wires; **switch boxes** at the intersections connect wire segments to one another. Every connection point is a pass transistor or multiplexer controlled

by a configuration bit. Segments come in several lengths — single (one CLB), double, quad or hex, and long lines spanning the chip — so that the router can use short wires for local connections and long ones for distant signals with fewer switches in series.

Routing dominates the FPGA. It typically occupies 70–90 % of the fabric area and contributes half or more of a typical path's delay, and it is the main reason an FPGA implementation of a circuit runs 3–5 times slower, uses 10–30 times more area and draws 5–15 times more dynamic power than a standard-cell ASIC of the same process. The router's job is a large combinatorial search: assign every net of the design to a path of segments without conflicts, while meeting timing constraints. **Congestion**, where demand for wires in a region exceeds supply, is a common reason a design fails to route or misses timing even with LUTs to spare; utilisation above about 70–80 % makes it likely.

> **Status of the principle: Rent's rule for interconnect demand**
> *Empirical*. The number of external connections T of a block of G logic cells follows $T \approx tG^{p}$ with Rent exponent p of about 0.5–0.75 for typical logic; FPGA architects size channel widths to this observed scaling, which is why highly connected designs such as large crossbars route poorly and regular, pipelined ones route well.

Clocks, timing and the design flow

Dedicated clock networks keep skew low across the die, and clock-region structures limit how many distinct clocks can reach one area. Because routing delay is decided only after placement and routing, FPGA timing is known only after the full implementation flow — synthesis, technology mapping to LUTs, packing into CLBs, placement, routing, static timing analysis and bitstream generation. The flow and its timing constraints are the subject of Chapter 19; the same steps for ASICs are in Chapter 34.

18.8 CPLDs

Sum-of-products logic with predictable timing

A **complex programmable logic device** (CPLD) descends from the PAL and GAL devices of the 1980s, which implement logic as a programmable AND array feeding fixed OR gates — directly the sum-of-products form of Chapter 12. A CPLD combines several such **function blocks**, each with a programmable AND array, product-term allocators and **macrocells** (an OR gate, an XOR for polarity control, a flip-flop and output enable), connected through a central **programmable interconnect matrix** (Figure 18.6). Configuration is stored in on-chip EEPROM or flash, so the device is **instant-on** at power-up.

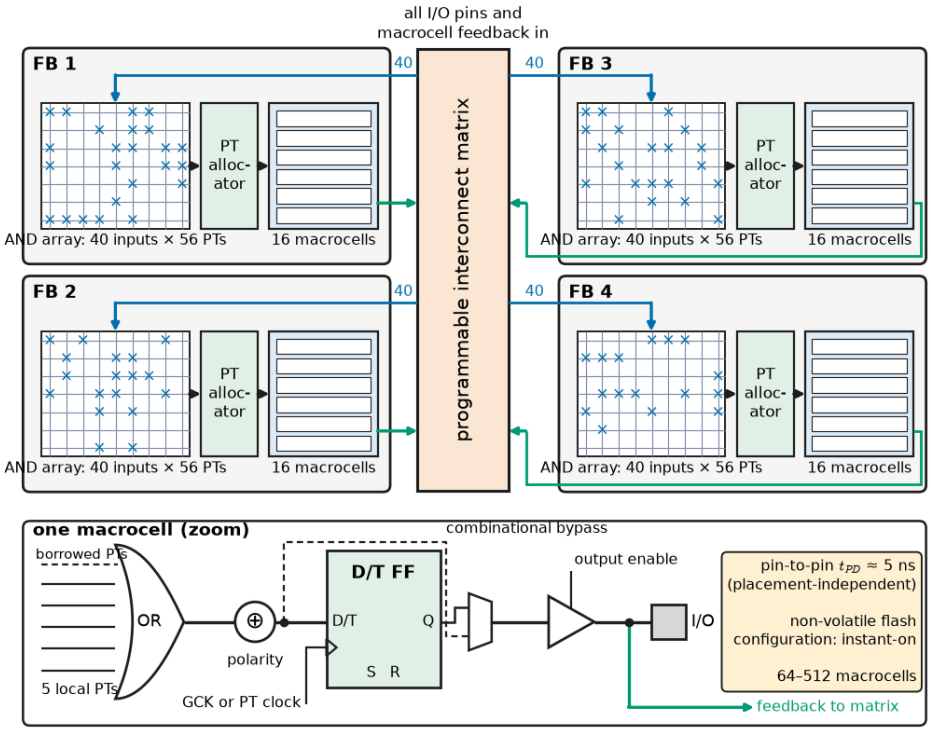


Figure 18.6 — CPLD architecture: function blocks of product-term macrocells linked by a global interconnect matrix.

Because every signal passes through the same interconnect matrix and one level of AND-OR logic, the pin-to-pin delay is nearly independent of placement and known from the datasheet before the design is built (typically 4–10 ns). This **predictable timing**, combined with instant-on operation, low static current and 3.3 V or 5 V tolerant I/O, is what CPLDs are for. Their limitation is capacity: classic CPLDs offer 32 to 512 macrocells, and the product-term architecture scales poorly beyond that.

Feature	CPLD	FPGA
Logic element	Product terms and macrocells	LUTs and flip-flops
Capacity	32–512 macrocells (up to a few thousand LUTs in flash-based "CPLD-class" parts)	Thousands to millions of LUTs
Configuration	Non-volatile, instant-on	Mostly SRAM, loaded at power-up (flash-based exceptions)
Timing	Fixed, placement-independent	Depends on placement and routing
Registers	One per macrocell	Two per LUT
Typical uses	Glue logic, address decoding, power sequencing, reset and board management	Data paths, DSP, interfaces, processors, acceleration

Many devices now marketed as CPLDs — Intel MAX 10, Lattice MachXO2/MachXO3 — are small LUT-based FPGAs with on-chip flash, keeping the instant-on and single-chip properties with FPGA-style logic. Their dominant job on modern boards is **board management**: sequencing the dozen power rails of a server or FPGA board in the right order within milliseconds of power-up, holding processors in reset until clocks are stable, multiplexing debug and boot-mode pins, and monitoring fault signals — functions that must work before any processor runs.

18.9 Hardware acceleration

Spatial versus temporal computing

A processor computes **temporally**: one ALU (or a few) is reused for every operation in sequence, and the cost of flexibility is paid in instruction fetch, decode, register-file access and control on every operation — typically 10–100 times the energy of the arithmetic itself. An FPGA or ASIC computes **spatially**: each operation of the algorithm becomes its own piece of hardware, connected by wires in the shape of the dataflow, and all of them work at once. The advantage comes from three sources, in order of importance: **parallelism** (many operations per clock), **pipelining** (a new input every clock while earlier ones are still in flight) and **specialisation** (exactly the word widths, memory organisation and control needed, no more).

Worked example — FIR filter on an MCU and an FPGA. A 64-tap FIR filter on 16-bit samples needs 64 multiply-accumulates (MACs) per output. The 168 MHz Cortex-M4 with its DSP instructions achieves about 1.2 cycles per tap, about 140 million MACs per second, so it can filter at most $168 \times 10^6 / (64 \times 1.2) = 2.19$ million samples per second — and only by spending its entire time on the filter. An Artix-7 implementation with one DSP48 slice per tap, fully pipelined at 200 MHz, accepts one sample per clock: 200 million samples per second, $64 \times 200 \times 10^6 = 12.8 \times 10^9$ MACs per second, about 90 times the MCU, using 64 of the XC7A100T's 240 DSP slices and leaving the rest of the device free. If the sample rate is only 1 MS/s, the FPGA can instead **time-multiplex** one DSP slice across all 64 taps at 64 MHz; spatial hardware can always be traded back for time.

Amdahl's law and data movement

Acceleration helps only the part of the work that is accelerated. If a fraction p of the original run time is sped up by a factor s, the overall speedup is

$$S = \frac{1}{(1 - p) + p/s}$$

and it can never exceed $1/(1-p)$, however large s becomes.

> **Status of the principle: Amdahl's law**
> *Derived.* It follows from splitting run time into an accelerated and an unaccelerated part; it assumes the parts do not overlap and that acceleration adds no overhead, so real speedups are lower once data transfer and synchronisation between processor and accelerator are counted.

Worked example — accelerating an FFT-based spectrum analyser. Profiling shows that 80 % of a vibration-analysis application's time on an embedded processor is spent in FFTs and 20 % in control, logging and communication. An FPGA FFT core runs the FFTs 20 times faster. The overall speedup is $1/(0.2 + 0.8/20) = 4.17$, and even an infinitely fast FFT would give only $1/0.2 = 5$. If moving each data block to the FPGA and back over a bus takes time equal to 5 % of the original run time, the speedup falls to $1/(0.2 + 0.04 + 0.05) = 3.45$. The FPGA has made the FFT almost free; the remaining 20 % and the data movement now decide performance — the central lesson of hardware–software co-design.

Data movement deserves its own emphasis. Moving a 32-bit word from off-chip DRAM costs of the order of 0.5–1 nJ, three to four orders of magnitude more than a 32-bit integer add in the same process; an accelerator that streams data from on-chip block RAM through a pipeline and keeps intermediate results local saves both time and energy, while one that fetches every operand from DRAM gains little. Choosing what to accelerate, how to partition the system between processor and FPGA, and how to model the data flow is the subject of Chapter 35; here it is enough that the architecture of Sections 18.4–18.7 — thousands of LUTs, hundreds of DSP slices and hundreds of independent block RAMs — is what makes spatial computation possible.

18.10 FPGA applications

FPGAs earn their place where volumes are too low for an ASIC, where standards or algorithms change after shipment, where timing must be deterministic to nanoseconds, or where interfaces are too fast or too unusual for a processor:

Domain	Typical FPGA function	Why an FPGA
Telecom	5G radio-unit digital front ends, beamforming, O-RAN fronthaul, packet processing	Throughput, evolving standards, field updates
Data centres	SmartNICs, network and storage offload, search and database acceleration, compression and encryption	Line-rate processing, reconfiguration per workload
Industrial control and drives	Multi-axis motor control, PWM with ns resolution, EtherCAT and other real-time Ethernet slaves, protection logic	Deterministic timing, parallel loops at 100 kHz–1 MHz
Power electronics	Modulation and protection for multilevel and modular multilevel converters, hardware-in-the-loop simulators	Hundreds of synchronised gate signals, sub-µs protection
Test and measurement	Oscilloscope and logic-analyser acquisition, triggering, protocol analysers	Gigasample data rates, custom triggers
Medical imaging	Ultrasound beamforming, CT and MRI data acquisition, PET coincidence detection	Many parallel channels, low latency
Automotive	ADAS sensor fusion, camera and radar interfacing, prototyping of ECU silicon	Parallel sensor streams, flexibility before ASIC
Aerospace and defence	Radiation-tolerant on-board processing, radar and electronic warfare, satellite payloads	Reconfigurability in orbit, flash/antifuse upset immunity
ASIC prototyping and emulation	Multi-FPGA systems running a chip's RTL before tape-out	Software development months before silicon (Chapter 34)
Finance and broadcast	Low-latency trading, video format conversion and compression	Nanosecond latency, many formats

Worked example — choosing a device for a converter controller. A modular multilevel converter for an HVDC link has 6 arms of 40 submodules each: 240 submodule half-bridges, each needing two gate signals with 100 ns timing resolution and a capacitor-voltage measurement, all updated at 10 kHz, with fault shutdown within 2 µs. The 480 gate signals synchronised to 100 ns, the 240 parallel measurements and the hard shutdown requirement rule out a single processor. A mid-range SRAM FPGA handles the timing and sorting in fabric, with a soft or hard processor for the outer control loops; since the converter station sits at high altitude and must run for decades, the design adds configuration scrubbing and triplicated protection logic, or moves to a flash-based FPGA. The same reasoning — count the parallel channels, the timing resolution and the reliability requirement before counting operations per second — selects FPGAs across the domains in the table.

What remains open

Open question	Where it is answered
How do latches, flip-flops and synchronous design underpin SRAM, registers and FPGA timing?	Chapter 14
How do caches and memory hierarchies use SRAM and DRAM?	Chapter 16
How does a microcontroller use its embedded flash, SRAM and external memory controller?	Chapter 17
How are FPGA designs written, simulated, constrained and implemented in Verilog and VHDL?	Chapter 19
How do firmware updates, bootloaders and non-volatile storage layouts handle power loss?	Chapter 25
How is wear-levelled storage used through file systems and data formats?	Chapter 24
How do the SRAM cell, sense amplifier and CMOS leakage behave at transistor level?	Chapter 33
How are memories compiled and placed in an ASIC, and how is static timing analysed?	Chapter 34
When should a computation move from a processor to an FPGA, and how is performance modelled?	Chapter 35
How does tunnelling physics set flash program voltages and oxide wear?	*Engineering Physics*, Ch. 4 §4.4
How are DDR and transceiver traces designed as transmission lines?	*Circuits, Signals and Communications*, Ch. 14

Next: Chapter 19 turns to the languages in which programmable logic and ASICs are described — Verilog, SystemVerilog and VHDL — and to the simulation, synthesis and timing flow that turns a description into a working FPGA.

Chapter 19 — Hardware Description Languages

Part IV · Computer Architecture and Digital Hardware

A servo-drive team moves the quadrature-encoder counter and PWM generator of a robot joint from a microcontroller into a small FPGA. Every RTL simulation passes. On the bench the joint creeps by a few encoder counts per minute, and during an EMC burst test it occasionally jumps by thousands of counts. Three lines of code are to blame, and none is a typing error. The encoder inputs reach a counter without a synchroniser, and no timing report flags them because the pins carry no constraint at all. A blocking assignment in a clocked block has merged two pipeline registers into one. And an `if` without an `else` in a block meant to be combinational has made the synthesiser build a latch. The text described hardware the engineers did not intend, and the simulator agreed with the text rather than with the silicon. This chapter is about writing HDL so that text, simulation and hardware mean the same thing, and about proving that they do.

> **How does a textual description become flip-flops, look-up tables and wires, and which language constructs describe which hardware?**
> **How do simulation, synthesis, timing constraints, test benches and assertions together establish that the hardware does what the design intends?**

Concept	Engineering destination
HDL fundamentals (concurrency, RTL abstraction)	Reading and writing any FPGA or ASIC design; IP reuse
SystemVerilog	Industry default for FPGA/ASIC design and verification
VHDL	Aerospace, defence, rail and European industrial FPGA projects; legacy IP
Combinational descriptions	Decoders, arithmetic, saturation logic, DSP datapaths without latches
Sequential descriptions	Registers, counters, FSMs, block-RAM buffers, encoder and PWM interfaces
Simulation semantics	Avoiding race conditions and simulation–synthesis mismatch
Synthesis	Utilisation, inference of DSP and RAM blocks, reading synthesis reports
Timing constraints (SDC/XDC)	Closing timing on ADC, DAC and memory interfaces; clock-domain crossings
Test benches	Self-checking regression suites, reference models, random stimulus
FPGA implementation	Place and route, pin planning, timing closure, on-chip debug, bitstreams
Hardware verification	Assertions, coverage, formal checks; DO-254 and ISO 26262 evidence

Roadmap. HDLs and the design flow (19.1) → SystemVerilog (19.2) and VHDL (19.3) → combinational (19.4) and sequential (19.5) descriptions → simulation (19.6) and synthesis (19.7) → timing constraints (19.8) → test benches (19.9) → FPGA implementation (19.10) → assertions, coverage and formal verification (19.11).

19.1 HDL fundamentals

Describing structure and behaviour, not a sequence of steps

A hardware description language describes a circuit: blocks that all exist at once, wires between them and registers that change on clock edges. A C program is a sequence of operations; an HDL description is a set of concurrent processes,

each reacting whenever one of its inputs changes. Two `always` blocks written one after the other are two pieces of hardware working in parallel.

HDLs serve two purposes. For **design**, a description must be **synthesisable**: a tool must be able to map it onto gates, flip-flops, look-up tables (LUTs), block RAMs and DSP slices. For **verification**, the language must also express delays, file I/O, random numbers and assertions about time, none of which is hardware; much of SystemVerilog exists only for test benches.

Descriptions are written at several levels of abstraction:

- **Behavioural**: algorithmic code with delays, for models and test benches.
- **Register-transfer level (RTL)**: registers updated on clock edges with combinational logic between them, the synchronous methodology of Chapter 15; almost all design is done here.
- **Structural/gate level**: instances of library cells, produced by synthesis.

> **Status of the principle: The RTL abstraction**
> *Approximate.* Treating a design as registers that update simultaneously on a clock edge, with ideal combinational logic in between, is exact only when every register-to-register path meets setup and hold time, every input is synchronous to the capturing clock and there are no combinational loops; it fails at unsynchronised inputs (metastability), at unconstrained or failing paths, and across clock domains, which is why timing constraints (Section 19.8) and synchronisers (Section 19.5) are part of the design, not afterthoughts.

The design flow

The same RTL feeds two streams (Figure 19.1). The **verification stream** simulates it against a test bench, checks assertions and measures coverage. The **implementation stream** synthesises it, maps it onto the device or cell library, places, routes and checks timing against constraints. Formal equivalence checking shows that the netlist still implements the RTL, and static timing analysis shows that it runs at the intended clock.

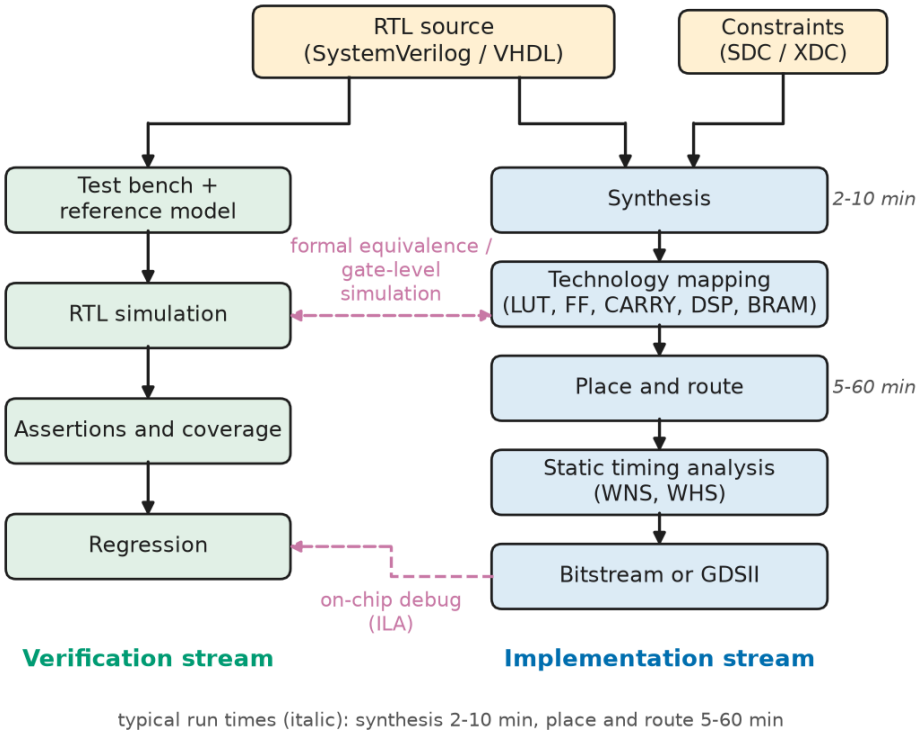


Figure 19.1 — The HDL design flow: one RTL source feeding a verification stream and an implementation stream that meet at equivalence and timing checkpoints.

The languages in use

Verilog and VHDL date from the 1980s; SystemVerilog absorbed Verilog in 2009, adding a type system, interfaces and a verification language. Higher-level options (Table 19.1) generate RTL, but the RTL and timing reports they produce must still be read.

Table 19.1 — Hardware description languages and their typical use.

Language	Standard	Typical use
Verilog	IEEE 1364-2005 (merged into 1800)	Legacy IP, simple FPGA designs, netlists
SystemVerilog	IEEE 1800-2017 (revised 2023)	FPGA and ASIC design; UVM verification; assertions (SVA)
VHDL	IEEE 1076-2008 (revised 2019)	Aerospace, defence, rail, medical FPGAs; many European companies
SystemC / C++ for HLS	IEEE 1666; vendor HLS tools	Algorithmic accelerators, transaction-level models
Chisel, Amaranth, SpinalHDL	Open-source, generate Verilog	Processor generators, research, open hardware

This chapter uses SystemVerilog as the main language and gives the VHDL equivalent of the essential constructs. The design rules (what infers a register, what infers a latch, what a constraint means) are the same in both.

19.2 SystemVerilog

Modules, ports and the 4-state logic type

The unit of design is the **module**, a block with named ports. The workhorse data type is `logic`, a **4-state** type whose bits take the values 0, 1, X (unknown) and Z (high impedance). X appears in simulation for uninitialised registers and conflicting drivers; it does not exist in hardware. Two-state types (`bit`, `int`) simulate faster and serve test benches and loop counters.

Vectors are declared with a range, `logic [11:0] duty;`, and literals carry a width and base: `12'd2048`, `8'hA5`. Unsized literals are 32-bit signed integers, and `'0` or `'1` fill every bit of the target. **Width and sign rules** cause many bugs: an expression is evaluated at the width of its widest operand (and of the target), and it is signed only if *every* operand is signed, so adding the unsigned constant `1'b1` to a signed value makes the whole expression unsigned and breaks comparisons of negative numbers.

Procedural blocks with intent

SystemVerilog replaced Verilog's single `always` with three blocks that state the designer's intent, so that tools can check it:

- `always_comb` for combinational logic. It is sensitive to everything it reads, executes once at time zero, and tools warn or error if it would infer storage.
- `always_ff @(posedge clk)` for flip-flops. Only one event control is allowed, and variables it assigns may not be assigned anywhere else.
- `always_latch` for the rare intentional latch.

Continuous assignments, `assign y = a & b;`, describe combinational logic in a single expression.

Packages, user types and parameters

Listing 19.1 shows a **package** of shared types, an **enumerated type** and a **packed struct** that bundles a command into one vector. The module is a PWM carrier for a motor-drive inverter leg: a 12-bit counter clocked at 100 MHz wraps every 4096 cycles, giving a PWM frequency of 100 MHz/4096 = 24.4 kHz, above the audible range.

```
package motor_pkg;
  localparam int PWM_W = 12;                      // duty resolution, bits
  typedef enum logic [1:0] {IDLE, RUN, BRAKE, FAULT} mode_t;
  typedef struct packed {
    logic             enable;
    mode_t            mode;
    logic [PWM_W-1:0] duty;                       // 0..4095 -> 0..100 %
  } pwm_cmd_t;
endpackage

module pwm_gen
  import motor_pkg::*;
(
  input  logic     clk,
  input  logic     rst_n,                         // asynchronous, active low
  input  pwm_cmd_t cmd,
  output logic     pwm_out
);
  logic [PWM_W-1:0] cnt;

  always_ff @(posedge clk or negedge rst_n)
    if (!rst_n) cnt <= '0;
    else        cnt <= cnt + 1'b1;                // wraps modulo 2**PWM_W

  assign pwm_out = cmd.enable && (cmd.mode == RUN) && (cnt < cmd.duty);
endmodule
```

Listing 19.1 — A PWM carrier using a package, an enumerated type and a packed struct (SystemVerilog-2017).

The enum makes `cmd.mode == RUN` self-documenting; the packed struct is still a 15-bit vector to the synthesiser. Parameters (`#(parameter int W = 16)`) make a module generic; they are fixed at **elaboration**, when the hierarchy is built, and cost no hardware.

Modules are instantiated with named port connections (`.clk(clk)`, or `.clk` when names match); `generate` loops replicate instances, such as the three legs of an inverter, and **interfaces** bundle the signals of a bus into one port.

19.3 VHDL overview

VHDL descends from Ada and is **strongly typed**: a `std_logic_vector` cannot be added to an `integer` without explicit conversion, and widths must match exactly. VHDL is therefore more verbose and catches at compile time many errors that Verilog accepts silently. Its logic type `std_logic` (IEEE 1164) has nine values; 'U' marks a signal never assigned, which helps find missing resets.

A VHDL design unit has two parts: an **entity**, declaring the interface, and an **architecture**, describing the implementation. Concurrent statements in the architecture are the equivalent of `assign`; a `process` with a sensitivity list is the equivalent of an `always` block. Listing 19.2 is the PWM carrier of Listing 19.1 in VHDL-2008, reduced to an enable and a duty input.

```
 library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity pwm_gen is
  generic (WIDTH : positive := 12);
  port (
    clk     : in  std_logic;
    rst_n   : in  std_logic;
    enable  : in  std_logic;
    duty    : in  unsigned(WIDTH-1 downto 0);
    pwm_out : out std_logic
  );
end entity pwm_gen;

architecture rtl of pwm_gen is
  signal cnt : unsigned(WIDTH-1 downto 0);
begin
  carrier : process (clk, rst_n)
  begin
    if rst_n = '0' then
      cnt <= (others => '0');
    elsif rising_edge(clk) then
      cnt <= cnt + 1;                -- numeric_std "+" wraps modulo 2**WIDTH
    end if;
  end process carrier;

  pwm_out <= '1' when enable = '1' and cnt < duty else '0';
end architecture rtl;
```

Listing 19.2 — The PWM carrier in VHDL-2008.

VHDL distinguishes **signals**, which update after the process suspends (like non-blocking `<=`), from **variables**, which update immediately (like blocking `=`, written `:=`), so it has fewer of the races of Section 19.6. Arithmetic uses `signed` and `unsigned` from `numeric_std`; the non-standard `std_logic_arith` package of older code should not be used.

Table 19.2 — Equivalent constructs in SystemVerilog and VHDL.

Purpose	SystemVerilog	VHDL-2008
Design unit	`module ... endmodule`	`entity` + `architecture`
Generic constant	`parameter int W = 16`	`generic (W : positive := 16)`
Basic logic type	`logic` (4-state)	`std_logic` (9-state)
Arithmetic vectors	`logic signed [15:0]`	`signed(15 downto 0)`
Combinational process	`always_comb`	`process (all)`
Clocked process	`always_ff @(posedge clk)`	`if rising_edge(clk) then` in a process
Deferred update	`q <= d;` (non-blocking)	signal assignment `q <= d;`
Immediate update	`x = a + b;` (blocking)	variable assignment `x := a + b;`

Mixed-language designs are routine: FPGA tools and commercial simulators compile both languages together.

19.4 Combinational logic descriptions

Expressions, always_comb and the no-latch rule

Combinational logic is written as continuous assignments or `always_comb` blocks using `if`, `case` and arithmetic. The synthesiser turns it into LUTs (any function of up to six inputs per LUT6, Section 18.5) or library gates, after the Boolean minimisation of Chapter 12.

The one rule that matters most: **in a combinational block, every output must be assigned on every path through the block.** If a path leaves an output unassigned, the language says the variable keeps its old value, and keeping a value requires storage, so the synthesiser infers a latch:

```
always_comb begin            // BAD: y keeps its value when sel == 2 or 3
  if      (sel == 2'd0) y = a;
  else if (sel == 2'd1) y = b;
end
```

An `always_comb` block like this produces a lint warning or error in every modern tool; the same code in Verilog-2001 `always @*` produced a silent latch. The fixes are to assign a **default** at the top of the block (`y = '0;`), to complete every `if` with an `else`, or to give every `case` a `default` branch.

> **Status of the principle: Latch inference from incomplete assignment**
> *Derived.* It follows from the semantics of procedural assignment: a variable that is not assigned on some path through a block retains its previous value, and the only hardware that retains a value without a clock edge is a level-sensitive latch; a latch in an FPGA fabric is timed poorly, glitches with its enable and usually indicates a coding error.

Saturating arithmetic: a complete example

Listing 19.3 is a signed saturating adder, the basic element of fixed-point DSP datapaths (filters, PI controllers, mixers) where wrap-around on overflow would turn a large positive value into a large negative one. The representation and overflow rules are those of two's-complement arithmetic from Chapter 11 (Sections 11.3 and 11.5); the HDL point is how they are written.

```
module sat_add #(parameter int W = 16) (
  input  logic signed [W-1:0] a, b,
  output logic signed [W-1:0] y,
  output logic                sat          // result was clipped
);
  localparam logic signed [W-1:0] MAXV = {1'b0, {(W-1){1'b1}}};
  localparam logic signed [W-1:0] MINV = {1'b1, {(W-1){1'b0}}};
  logic signed [W:0] sum;                  // one guard bit

  always_comb begin
    sum = a + b;                           // both operands sign-extended to W+1
    sat = (sum[W] != sum[W-1]);            // guard bit differs from sign bit
    if (!sat)        y = sum[W-1:0];
    else if (sum[W]) y = MINV;             // negative overflow
    else             y = MAXV;             // positive overflow
  end
endmodule
```

Listing 19.3 — A parameterised signed saturating adder (SystemVerilog-2017).

Because `sum` is $W+1$ bits wide and every operand is signed, the addition is done at $W+1$ bits with sign extension and cannot overflow; overflow of the W-bit result shows up as disagreement between guard bit and sign bit. Every path assigns `y` and `sat`, so no latch is inferred. In a Xilinx 7-series device a 16-bit instance needs about 17 LUTs on a carry chain plus about 16 LUTs of output multiplexing.

case, priority and parallel logic

An `if`–`else if` chain describes **priority** logic (a chain of multiplexers); a `case` statement describes selection among exclusive values (a balanced multiplexer). SystemVerilog's `unique case` asserts that exactly one item matches and `priority case` that at least one does; simulation checks the claim, so the qualifiers are safe replacements for the old `full_case` and `parallel_case` pragmas, which changed the hardware without telling the simulator.

Arithmetic, multipliers and constants

Adders map onto dedicated carry chains, multipliers onto **DSP slices** (a 25×18-bit multiplier in 7-series DSP48E1, 27×18-bit in UltraScale DSP48E2, each with a 48-bit post-adder), and constant multiplications reduce to shifts and adds. Division by a non-power of two synthesises into a large, slow array and is normally replaced by multiplication by a reciprocal or an iterative divider. A sine table for a numerically controlled oscillator is written as a constant array and becomes LUT logic or a block-RAM ROM.

19.5 Sequential logic descriptions

Registers, enables and resets

A flip-flop is an `always_ff` block with **non-blocking** assignments (`<=`); an `if (en)` around an assignment becomes a clock enable. Reset style is a design decision whose consequences belong to Chapter 14 (Section 14.10); in code:

- **Asynchronous reset**: `always_ff @(posedge clk or negedge rst_n) if (!rst_n) q <= '0; else q <= d;` The reset acts immediately; its de-assertion must be synchronised to the clock.
- **Synchronous reset**: `always_ff @(posedge clk) if (rst) q <= '0; else q <= d;` The reset is just another data input and is timed like one.

Xilinx recommends synchronous, active-high resets applied only to registers that need them (control state, not wide datapaths), because large reset nets consume routing and prevent use of the DSP and RAM internal registers.

Synchronisers and a finite-state machine

The quadrature encoder of the opening problem is a good example. An incremental encoder produces two square waves, A and B, 90° apart; the order of their edges gives the direction, and counting every edge gives four counts per line. A 2500-line encoder at 6000 rpm (100 rev/s) produces $2500 \times 4 \times 100 = 1.0 \times 10^{6}$ counts per second, an edge every 100 cycles of a 100 MHz clock: easy, provided the inputs are sampled properly.

Each asynchronous input passes through a **two-flip-flop synchroniser** before any logic uses it (metastability and its mean time between failures belong to Section 14.9). The decoder is a four-state machine (Figure 19.2) whose state is the last sampled {A, B}; when B leads A the state steps 00, 01, 11, 10 and the position counts up. Section 15.6 coded an FSM with a separate state register and next-state block; Listing 19.4 uses the compact one-block form, which suits a machine whose next state is the current input and whose outputs are actions on transitions.

```
module quad_decoder #(parameter int CW = 32) (
  input  logic          clk,
  input  logic          rst,              // synchronous, active high
  input  logic          enc_a, enc_b,     // asynchronous encoder inputs
  output logic [CW-1:0] position,
  output logic          error             // sticky: both inputs changed at once
);
  (* ASYNC_REG = "TRUE" *) logic [1:0] a_sync, b_sync;
  always_ff @(posedge clk) begin          // two-flip-flop synchronisers
    a_sync <= {a_sync[0], enc_a};
    b_sync <= {b_sync[0], enc_b};
  end

  logic [1:0] ab;
  assign ab = {a_sync[1], b_sync[1]};

  typedef enum logic [1:0] {S00 = 2'b00, S01 = 2'b01, S11 = 2'b11, S10 = 2'b10} qstate_t;
  qstate_t state;                         // last sampled {A,B}

  always_ff @(posedge clk) begin
    if (rst) begin
      state <= qstate_t'(ab);  position <= '0;  error <= 1'b0;
    end else begin
      unique case ({state, ab})           // {old, new}
        {S00,2'b01}, {S01,2'b11}, {S11,2'b10}, {S10,2'b00}: position <= position + 1'b1;
        {S00,2'b10}, {S10,2'b11}, {S11,2'b01}, {S01,2'b00}: position <= position - 1'b1;
        {S00,2'b11}, {S11,2'b00}, {S01,2'b10}, {S10,2'b01}: error    <= 1'b1;
        default: ;                        // no change
      endcase
      state <= qstate_t'(ab);
    end
  end
endmodule
```

Listing 19.4 — Quadrature decoder with input synchronisers (SystemVerilog-2017).

The `ASYNC_REG` attribute makes Xilinx tools place the synchroniser flip-flops together and leave them unoptimised. The `unique case` with `default` covers all 16 combinations: four with no change, four forward, four reverse and four illegal double transitions, which occur only if edges arrive faster than the clock samples or noise corrupts an input, and which set the sticky `error` flag for the drive's fault logic.

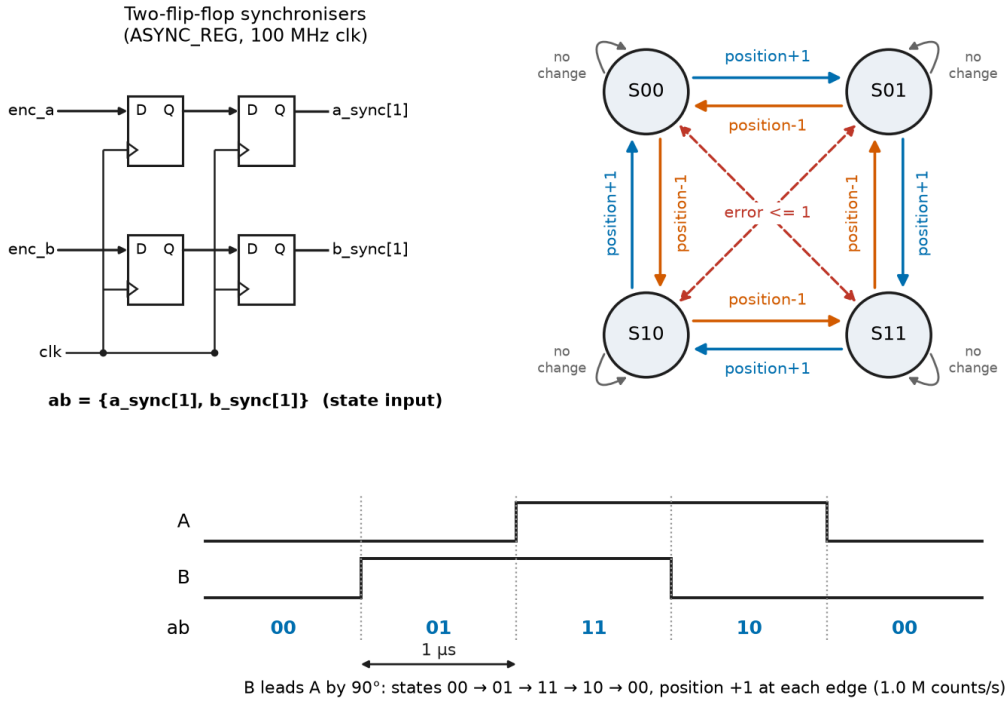


Figure 19.2 — Quadrature decoder: input synchronisers and the four-state machine whose transitions count up, count down or flag an error.

Counters, shift registers and memories

Counters and shift registers (`sr <= {sr[N-2:0], din};`) follow the same pattern; a long unreset shift register maps onto SRL primitives (32 bits in one Xilinx LUT).

Memories are **inferred** from an array and a template the synthesiser recognises. Listing 19.5 is a simple dual-port RAM for an ADC sample buffer.

```
module sample_ram #(parameter int AW = 10, parameter int DW = 16) (
  input  logic          clk,
  input  logic          we,
  input  logic [AW-1:0] waddr, raddr,
  input  logic [DW-1:0] wdata,
  output logic [DW-1:0] rdata
);
  logic [DW-1:0] mem [2**AW];             // 1024 x 16 bits = 16 kb

  always_ff @(posedge clk) begin
    if (we) mem[waddr] <= wdata;
    rdata <= mem[raddr];                  // synchronous read -> block RAM
  end
endmodule
```

Listing 19.5 — Block-RAM inference template (SystemVerilog-2017).

A synchronous read is essential: block RAMs register their address, so an asynchronous read (`assign rdata = mem[raddr];`) forces the tool to build the 16 kb array from distributed LUT RAM, several hundred LUTs, instead of half of one 36 kb block RAM.

19.6 Simulation

Event-driven execution

An HDL simulator keeps a **time wheel** of scheduled events. At each simulation time it runs every process whose inputs changed and schedules the updates they produce, repeating until nothing changes; only then does time advance. Each round at the same time is a **delta cycle**.

SystemVerilog divides each time step into ordered **regions** (IEEE 1800-2017 clause 4). The ones that matter for RTL are:

1. **Active**: processes run, blocking assignments (`=`) update immediately, continuous assignments are evaluated, and the right-hand sides of non-blocking assignments are evaluated.
2. **NBA (non-blocking assignment) update**: the left-hand sides of all non-blocking assignments (`<=`) are updated together, which may trigger more Active events.
3. **Observed and Reactive**: concurrent assertions are evaluated and test-bench programs run.
4. **Postponed**: the values are final; the $strobe and $monitor system tasks print here.

The order in which the simulator runs processes inside the Active region is **not defined** by the standard. Any code whose result depends on that order is a **race**.

Blocking versus non-blocking

This is why clocked logic uses non-blocking assignments. Consider a three-stage shift register:

```
 always_ff @(posedge clk) begin   // correct: q3 gets d after three clocks
   q1 <= d;
   q2 <= q1;
   q3 <= q2;
end

always @(posedge clk) begin      // wrong: blocking assignments in this order
   q1 = d;
   q2 = q1;                      // reads the value just written
   q3 = q2;                      // q3 = d after one clock
end
```

With non-blocking assignments every right-hand side is sampled in the Active region and all registers update together in the NBA region, so each stage receives the old value of the stage before it, exactly as flip-flops sharing a clock edge do. With blocking assignments each line sees the new value of the previous line and the three "stages" collapse into one: simulation shows q_3 following d after one clock, and synthesis, reading the same semantics, builds one register (Figure 19.3). Reversing the statement order happens to work, but correctness that depends on statement order is fragile.

Worse, two blocks that exchange values through blocking assignments (`a = b;` in one, `b = a;` in the other, both on `posedge clk`) give results that depend on which block the simulator runs first, while synthesis builds two flip-flops that swap correctly; with `<=` the swap is deterministic.

Worked example — a pipeline register that disappeared. In the opening's encoder interface, the position passed through a two-stage pipeline to align it with a timestamp from a block RAM of 2-cycle latency. Written with blocking assignments, the two stages became one register, and the position arrived one 10 ns cycle before its timestamp. At 1.0 M counts/s the position changes once every 100 cycles, so about 1 sample in 100 paired a position with the previous count's timestamp, and the velocity estimator saw spurious one-count steps: the observed drift. Rewriting the block with `<=` restored the register, confirmed by 32 more flip-flops in the utilisation report.

Status of the principle: Non-blocking assignment for clocked logic
Derived. From the scheduling semantics of IEEE 1800: non-blocking assignments sample all right-hand sides before any left-hand side changes, which reproduces the behaviour of edge-triggered registers sharing a clock and makes the result independent of process order; the companion rules are blocking assignments in combinational blocks, never both kinds to one variable, and no variable assigned from two blocks.

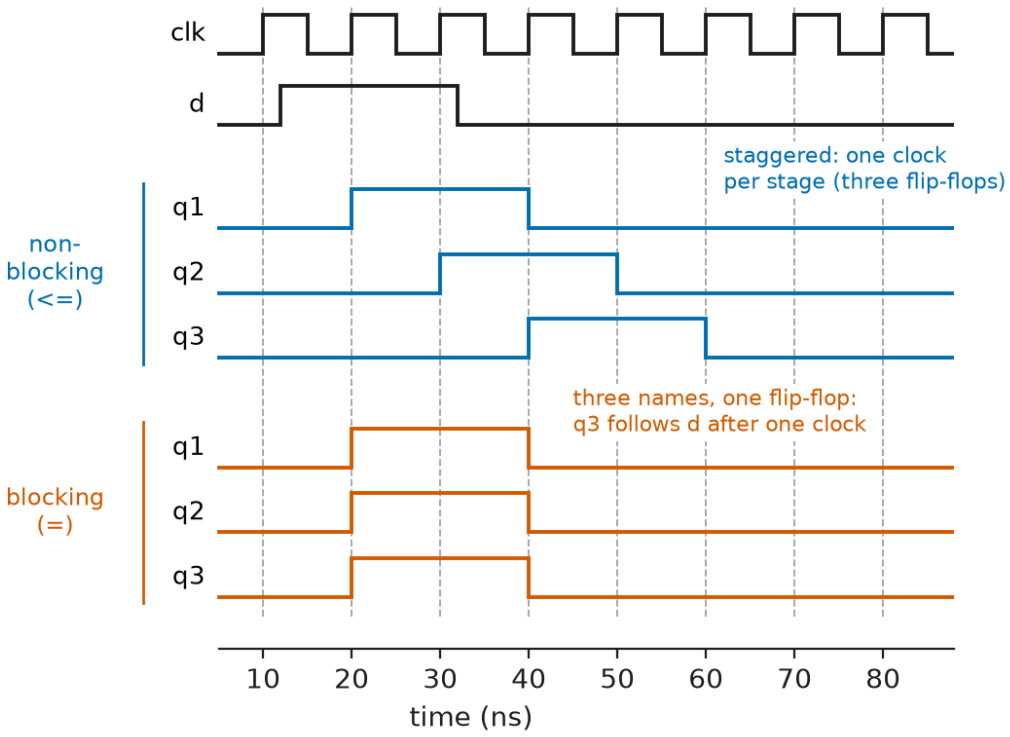


Figure 19.3 — A three-stage shift register written with non-blocking and with blocking assignments: simulated waveforms.

X propagation and other simulator effects

Four-state simulation can mislead in both directions. **X-optimism**: `if (sel) y = a; else y = b;` with `sel = X` takes the `else` branch and hides an uninitialised select line. **X-pessimism**: some gate-level constructs propagate X where hardware would be fine. Reset control registers, avoid X as a don't-care in simulated RTL, and assert that control signals are never X (the SVA function $isunknown).

Delays written as `#5` are simulation-only and ignored by synthesis; RTL should contain none. Initial values (`logic [7:0] cnt = 8'd0;`) are honoured by FPGA synthesis, because FPGA flip-flops power up from the bitstream to a defined value, but not by ASIC synthesis.

Commercial simulators (Siemens Questa, Synopsys VCS, Cadence Xcelium) support the full language, UVM, assertions and coverage; FPGA vendors bundle adequate simulators (Vivado XSim); open-source Icarus Verilog and GHDL serve teaching and CI, and Verilator compiles synthesisable SystemVerilog to cycle-based, two-state C++ that runs 10 to 100 times faster on large designs.

19.7 Synthesis

What the synthesiser does

Synthesis translates RTL into a netlist of target primitives in four broad steps:

1. **Elaboration**: parameters are resolved, generate loops unrolled and the hierarchy built.
2. **Inference**: language patterns are recognised as hardware: `always_ff` blocks become flip-flops with enables and resets, arrays with the right access pattern become RAMs, `*` becomes a DSP slice, `case` on an enum becomes an FSM that the tool may re-encode (one-hot, Gray or binary, Chapter 14).
3. **Optimisation**: constant propagation, removal of logic with no observable output, sharing of common subexpressions, and optionally **retiming**, which moves registers across combinational logic to balance path delays.
4. **Technology mapping**: logic is packed into LUTs, carry chains, multiplexers and dedicated blocks (FPGA) or into standard cells chosen for speed and area (ASIC, Chapter 34).

The output is a netlist and, as important, the **reports**: utilisation, inferred primitives and warnings about latches, multiple drivers and removed logic. A warning that a register "has no load and was removed" usually means a bug.

Worked example — reading a utilisation report. The servo FPGA is an Artix-7 XC7A35T (20 800 LUT6, 41 600 flip-flops, 90 DSP48E1, 50 block RAMs of 36 kb; the device range is tabulated in Section 18.4), holding three PWM generators, two 32-bit quadrature decoders, a 1024-sample buffer and a PI current controller with two 16×16 multiplications per axis. Expected primitives:

- PWM: three 12-bit counters and comparators: about 36 flip-flops and fewer than 72 LUTs.
- Decoders: per axis 4 synchroniser, 2 state, 32 position and 1 error flip-flops (39) and about 40 LUTs: 78 flip-flops and 80 LUTs for two axes.
- Buffer: one RAMB18 (half a RAMB36) holds 1024 × 16 = 16 384 bits exactly.
- Controller: four DSP48E1 slices.

The report matched within a few LUTs per block, plus the 32 flip-flops restored in Section 19.6. The design used under 2 % of the LUTs, which is typical: control FPGAs are chosen for I/O, determinism and latency, not logic capacity.

Simulation–synthesis mismatch

A synthesiser reads the same language as a simulator, but it interprets only the synthesisable subset and makes assumptions the simulator does not. The classic mismatches are:

Table 19.3 — Common causes of simulation–synthesis mismatch.

Cause	Simulation sees	Hardware does	Prevention
Incomplete sensitivity list (`always @(a)` reading `b`)	Output ignores changes of `b`	Combinational function of `a` and `b`	`always_comb` or `@*`
Blocking assignments in clocked blocks	Order-dependent registers	Registers per synthesis semantics	`<=` in `always_ff`
`full_case` / `parallel_case` pragmas	X or priority behaviour	Parallel logic with don't-cares	`unique` / `priority` case
Assigning X as don't-care	X propagates or is optimistic	Arbitrary 0/1 chosen by optimiser	Defined defaults; assertions
`#` delays and `initial` blocks in RTL	Timed behaviour	Delays ignored; initial values FPGA-only	No delays in RTL
Asynchronous feedback loops	Oscillation or settling	Unpredictable, untimed	Synchronous design only

> **Status of the principle: Simulation–synthesis equivalence**
> *Approximate.* RTL simulation predicts the synthesised netlist's behaviour only for code within the synthesisable subset written without races, incomplete sensitivity or X-dependent decisions, and only at clock-cycle resolution; it says nothing about whether the netlist meets timing, which static timing analysis decides, and formal equivalence checking or gate-level simulation is used where the guarantee must be explicit.

19.8 Timing constraints

What the tools need to know

Synthesis and place-and-route optimise for timing, and static timing analysis (STA) signs it off, but only against **constraints**: clock periods, how external devices launch and capture data at the pins, and which paths need special treatment. A path with no constraint is not timed at all; the opening problem's encoder pins were such paths. The register-to-register condition $T_{clk} \geq t_{cq} + t_{logic} + t_{su} - t_{skew}$ and its hold counterpart are developed in Chapter 14 (Section 14.8), the ASIC sign-off view in Chapter 34; here the subject is how to state constraints.

The language is **SDC** (Synopsys Design Constraints), a set of Tcl commands used by ASIC tools and by Intel Quartus; AMD Vivado uses **XDC**, which is SDC plus Xilinx physical properties (pins, I/O standards, placement). Table 19.4 lists the commands that cover most designs.

Table 19.4 — Core SDC/XDC timing commands.

Command	Meaning
`create_clock -period T [get_ports clk]`	A primary clock entering a pin (or a GT/PLL output)
`create_generated_clock`	A divided or forwarded clock derived from another (automatic for PLL/MMCM outputs in Vivado)
`set_input_delay -clock c -max/-min d`	External launch: data arrive at the pin d after the clock edge at the pin
`set_output_delay -clock c -max/-min d`	External capture: the external device needs data d before (max) the clock, and holds after (min, often negative)
`set_false_path`	Path not timed (static configuration, synchronised asynchronous input)
`set_multicycle_path N`	Path allowed N clock periods (with a matching hold adjustment)
`set_clock_groups -asynchronous`	Clocks with no phase relationship; paths between them not timed
`set_max_delay -datapath_only`	Bound on a CDC path (e.g. Gray-coded FIFO pointers) without clock skew

Input and output delays

Section 15.4 budgeted a board-level ADC interface by hand; constraints hand the same budget to the tool, which then checks it on every bit, at every corner, with routed delays. An **input delay** is the time after the clock edge at the FPGA's clock pin at which data arrive at the data pin, as seen from outside. For a system-synchronous interface, where one oscillator clocks both an external device and the FPGA, it is built from the device's clock-to-output time and the board delays (Figure 19.4):

$$t_{in,\max} = t_{co,\max} + t_{d,\max} + t_{clk,dev} - t_{clk,fpga}, \qquad t_{in,\min} = t_{co,\min} + t_{d,\min} + t_{clk,dev} - t_{clk,fpga}$$

where t_d is the data trace delay and $t_{clk,dev}$, $t_{clk,fpga}$ the clock trace delays (FR-4 stripline: about 6.5–7 ps/mm). The FPGA needs the rest of the period to bring the data to the capturing register and meet setup.

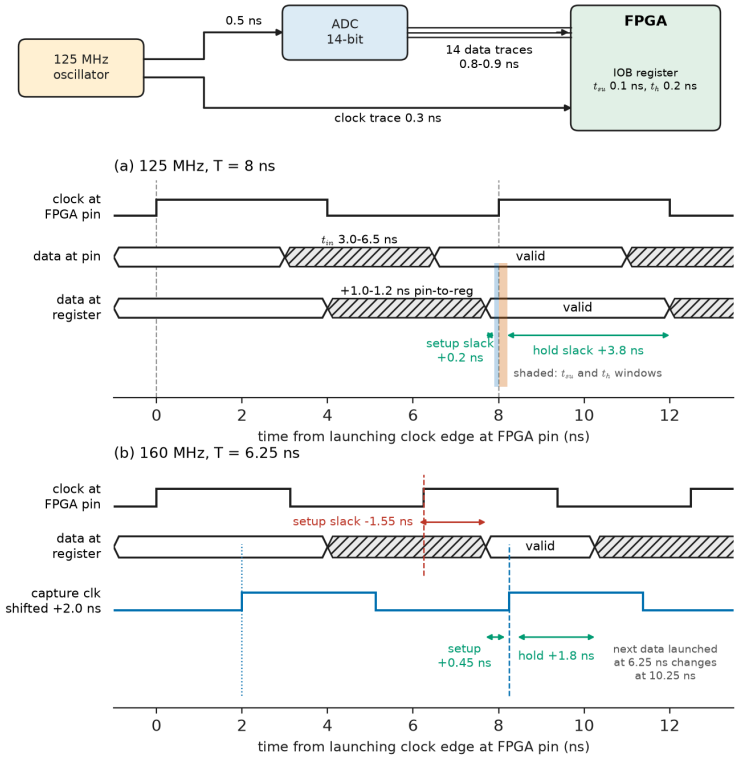


Figure 19.4 — System-synchronous ADC-to-FPGA interface: input delay built from clock-to-output and board delays, and the setup and hold slack at the capturing register.

Worked example — constraining a 14-bit ADC interface. A software-defined-radio receiver digitises at 125 MS/s with a 14-bit parallel-CMOS ADC whose clock-to-output time is 2.0–5.4 ns; unlike the forwarded-clock interface of Section 15.4, one oscillator drives the ADC (0.5 ns trace) and the FPGA clock pin (0.3 ns trace), and the data traces are 0.8–0.9 ns.

- Input delays: $t_{in,\max} = 5.4 + 0.9 + 0.5 - 0.3 = 6.5$ ns and $t_{in,\min} = 2.0 + 0.8 + 0.5 - 0.3 = 3.0$ ns.
- Inside the FPGA, with the capture clock compensated by an MMCM so that it arrives at the input register in phase with the clock pin, the pin-to-register delay through the input buffer is 1.0–1.2 ns, setup time 0.1 ns, hold time 0.2 ns.
- Setup slack at 125 MHz: $8.0 - 6.5 - 1.2 - 0.1 = +0.2$ ns. Hold slack: $3.0 + 1.0 - 0.2 = +3.8$ ns. The interface works, with little setup margin.
- At 160 MS/s ($T = 6.25$ ns) the setup slack becomes $6.25 - 6.5 - 1.2 - 0.1 = -1.55$ ns: the data are not yet valid when the edge arrives.
- Shifting the capture clock 2.0 ns later with the MMCM's fine phase shift moves the setup check later and the hold check with it: setup slack $6.25 + 2.0 - 6.5 - 1.2 - 0.1 = +0.45$ ns, hold slack $3.0 + 1.0 - 0.2 - 2.0 = +1.8$ ns. Both are positive and the interface closes.

Listing 19.6 states these numbers as constraints; placing the first register in the I/O block (`IOB TRUE`) makes the pin-to-register delay identical for all 14 bits.

```
 # 125 MHz system-synchronous ADC interface (illustrative port names)
create_clock -name sys_clk -period 8.000 [get_ports clk_125]
set_input_delay  -clock sys_clk -max 6.5 [get_ports {adc_d[*] adc_ovr}]
set_input_delay  -clock sys_clk -min 3.0 [get_ports {adc_d[*] adc_ovr}]
set_property IOB TRUE [get_cells {adc_d_q_reg[*]}]

# DAC capturing on the same clock: tsu = 2.0 ns, th = 0.5 ns
set_output_delay -clock sys_clk -max  2.0 [get_ports {dac_d[*]}]
set_output_delay -clock sys_clk -min -0.5 [get_ports {dac_d[*]}]

# Asynchronous push-button through a synchroniser: do not time
set_false_path -from [get_ports btn_rst_n]

# Gray-coded FIFO pointers between unrelated 125 MHz clocks
create_clock -name eth_rxc -period 8.000 [get_ports rgmii_rxc]
set_max_delay -datapath_only 8.0 -from [get_clocks eth_rxc] -to [get_clocks sys_clk]
```

Listing 19.6 — XDC constraints for the receiver (Vivado; the same timing commands are valid SDC).

Exceptions are claims

Every exception is a claim by the designer that a path does not need normal timing, and the tool believes it without proof: a false path on a crossing without a synchroniser hides a real failure. Exceptions should be few, written narrowly (`-from` and `-to` specific registers), commented and reviewed like code; clock-domain-crossing checks (`report_cdc` in Vivado) verify structurally that each crossing has a synchroniser.

> **Status of the principle: Static timing analysis as timing sign-off**
> *Approximate.* STA checks every constrained path exhaustively against worst-case (and best-case) delay models at the characterised process, voltage and temperature corners, so it needs no test vectors; its verdict is only as good as the constraints, and it cannot detect an unconstrained path, an incorrect exception, metastability at an unsynchronised input or a functional error.

19.9 Test benches

Structure of a self-checking test bench

A **test bench** is HDL (or Python, C++) code that instantiates the design under test (DUT), drives its inputs, observes its outputs and decides automatically whether they are correct. Section 15.9 introduced the idea with a directed test of a small FSM; a professional test bench adds four elements:

- **Stimulus generation**: directed tests for the cases the specification names, plus constrained-random tests for those it does not.
- **A reference model**: an independent implementation of the intended behaviour, written at a higher level (integer arithmetic in SystemVerilog, a Python or C model, a MATLAB golden output file).
- **A checker or scoreboard**: compares DUT outputs with the reference, cycle by cycle or transaction by transaction, and counts errors.
- **A verdict**: a single PASS or FAIL line and a non-zero exit status, so that a regression script or continuous-integration job can run hundreds of tests unattended.

A test bench whose waveforms must be inspected by eye is not self-checking and does not scale.

Listing 19.7 tests the saturating adder of Listing 19.3: six directed corner cases at the saturation boundaries, then 100 000 random pairs, every result checked against an integer reference model.

```
module tb_sat_add;
  localparam int W = 16;
  localparam int MAXV = 2**(W-1) - 1, MINV = -(2**(W-1));
  logic signed [W-1:0] a, b, y;
  logic                sat;
  int                  errors = 0, checks = 0;

  sat_add #(.W(W)) dut (.a, .b, .y, .sat);

  function automatic int ref_sat(int s);            // reference model
    return (s > MAXV) ? MAXV : (s < MINV) ? MINV : s;
  endfunction

  task automatic check(int x, int z);
    a = x;  b = z;
    #1;                                             // combinational settle
    checks++;
    if (y !== ref_sat(x + z) || sat !== (x + z != ref_sat(x + z))) begin
      errors++;
      $error("a=%0d b=%0d: got y=%0d sat=%b, expected %0d",
             x, z, y, sat, ref_sat(x + z));
    end
  endtask

  initial begin
    check(MAXV, 1);    check(MINV, -1);   check(MAXV, MAXV);
    check(MINV, MINV); check(0, 0);       check(-1, 1);
    repeat (100_000)
      check(int'($urandom_range(0, 2**W - 1)) + MINV,
            int'($urandom_range(0, 2**W - 1)) + MINV);
    if (errors == 0) $display("PASS: sat_add, %0d checks", checks);
    else             $fatal(1, "FAIL: %0d of %0d checks", errors, checks);
    $finish;
  end
endmodule
```

Listing 19.7 — Self-checking test bench with directed and random stimulus (SystemVerilog-2017).

The reference model works in 32-bit integers, where the sum cannot overflow, so it is independent of the DUT's guard-bit trick; the 4-state `!==` comparison also fails on an X or Z output, which `!=` would let pass. The DUT is combinational, so the task waits 1 time unit for it to settle; a clocked DUT is driven and sampled through a **clocking block** that applies inputs just after the edge and samples outputs just before it, avoiding test-bench races.

How much stimulus is enough

Worked example — exhaustive or random? The 16-bit saturating adder has two 16-bit inputs, $2^{32} = 4.29 \times 10^{9}$ input pairs. At 10^{6} checks per second they take 4295 s, about 72 minutes: exhaustive testing is feasible once. A 32-bit version has $2^{64} = 1.8 \times 10^{19}$ pairs, 5.8×10^{5} years at the same rate. For realistic datapaths confidence must come from corner cases, random stimulus measured by coverage (Section 19.11) and formal proofs.

Random stimulus, files and Python

SystemVerilog **classes** with `rand` variables and `constraint` blocks let a solver generate legal but varied transactions (packet lengths of 64 to 1518 bytes, weighted toward the extremes), reproducibly from a seed. For DSP blocks the expected output is often computed in Python or MATLAB and read from files with the $readmemh or $fscanf system tasks. **cocotb** writes the whole test bench in Python, driving Icarus, Verilator or a commercial simulator, so the reference model can use NumPy directly (Chapter 22).

Large verification environments use the **Universal Verification Methodology** (UVM, IEEE 1800.2), a class library that standardises drivers, monitors, sequencers and scoreboards; it is the norm in ASIC teams and overkill for a small FPGA design.

19.10 FPGA implementation

From netlist to bitstream

The FPGA architecture (LUTs, CLBs, routing, block RAM, DSP slices, clock networks) is the subject of Chapter 18. The implementation flow maps the synthesised netlist onto it in stages, each with a report worth reading (Figure 19.5):

1. **Optimisation** of the mapped netlist.
2. **Placement** of every primitive on a physical site, guided by timing.
3. **Physical optimisation**: replicating high-fan-out drivers, retiming.
4. **Routing** through the programmable interconnect, which typically contributes 50–70 % of a path's delay.
5. **Timing analysis** on routed delays, reporting **WNS** (worst negative slack, setup), **TNS** (total negative slack), and **WHS**/**THS** for hold. Timing is met when WNS and WHS are both non-negative at all corners.
6. **Bitstream generation**, loaded by JTAG in development and from SPI/QSPI flash in the product, usually encrypted and authenticated, with a fallback image for safe field updates.

Before implementation, the **I/O plan** assigns every port to a package pin with an I/O standard (`LVCMOS33`, `LVDS_25`, `SSTL15` for DDR3), consistent with bank voltages, clock-capable pins and differential pairs; it is done together with the PCB designer.

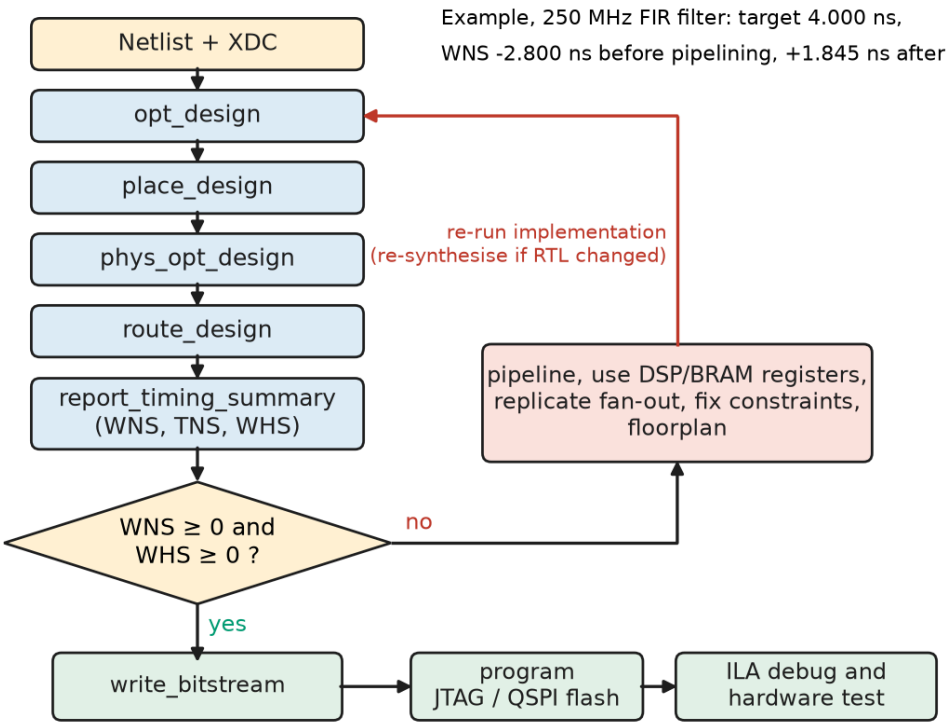


Figure 19.5 — FPGA implementation flow from synthesised netlist to programmed device, with the timing-closure loop.

Closing timing

When WNS is negative, first correct the constraints, then restructure the logic (pipeline, use DSP and RAM internal registers), reduce fan-out, and only then use tool directives and floorplanning.

Worked example — pipelining a filter multiply–accumulate. A 250 MHz FIR filter for a radar front end (the filter design is from *Circuits, Signals and Communications*, Ch. 23) had one register before and one after each multiply–add. Critical path: clock-to-output 0.35 ns, unregistered DSP48E1 multiplier 3.9 ns, post-adder 1.5 ns, routing 1.0 ns, setup 0.05 ns; total 6.8 ns, $f_{max} = 147$ MHz, WNS $= 4.000 - 6.8 = -2.8$ ns. Adding one register stage between multiplier and post-adder lets synthesis absorb the three register levels into the slice's A/B, `MREG` and `PREG` registers, so the path is limited by the slice's fully pipelined maximum frequency, about 464 MHz (2.155 ns) in the slowest Artix-7 speed grade: WNS $= 4.000 - 2.155 = +1.845$ ns. The cost is latency, two stages (8 ns) instead of one (6.8 ns): irrelevant in a streaming filter, but each sample of delay inside a feedback loop costs phase margin (*Circuits, Signals and Communications*, Ch. 35 §35.8).

On-chip debug and power

Integrated logic analysers (Xilinx ILA, Intel SignalTap) insert capture memory and trigger logic into the design and read waveforms over JTAG at the real clock rate, at the cost of block RAM and some timing margin. The **power report** estimates static power (leakage, strongly temperature dependent) and dynamic power from toggle rates; dynamic power follows the $\alpha C V^2 f$ law of Chapter 33.

19.11 Hardware verification

Why verification dominates

A bug found in simulation costs minutes; in the lab, hours; in a shipped product, a field update or a recall; in an ASIC, a respin costing millions. Industry surveys report that verification takes more than half the effort of a digital project. It is

planned: a **verification plan** lists each feature, how it will be checked and the coverage goal that defines "done". DO-254 (airborne hardware) and ISO 26262 (automotive) make this evidence mandatory.

Assertions

An **assertion** is an executable statement of a property. **Immediate assertions** (`assert (fifo_count <= DEPTH);`) check a condition in procedural code. **Concurrent assertions** (SVA) describe behaviour over clock cycles, checked in simulation and provable by formal tools, using implications `|->` (same cycle) and `|=>` (next cycle), delays `##n` and sampled-value functions such as $past, $rose and $isunknown. VHDL designs express the same properties in PSL (IEEE 1850), which VHDL-2008 accepts embedded in the source. Listing 19.8 binds properties to the quadrature decoder without editing its source.

```
module quad_decoder_props (
  input logic        clk, rst,
  input logic [1:0]  ab,
  input logic [31:0] position,
  input logic        error
);
  default clocking cb @(posedge clk); endclocking
  default disable iff (rst);

  // position moves by at most one count per clock
  a_step: assert property (
    position == $past(position) || position == $past(position) + 32'd1 ||
    position == $past(position) - 32'd1)
    else $error("position jumped to %0d", position);

  a_sticky:  assert property (error |=> error);          // error never clears
  a_known:   assert property (!$isunknown(ab));           // synchronised inputs defined
  a_double:  assert property ((ab == ~$past(ab)) |=> error); // double step flags

  c_up:      cover property (position == $past(position) + 32'd1);
  c_down:    cover property (position == $past(position) - 32'd1);
  c_wrap:    cover property ($past(position) == '1 && position == '0);
endmodule

bind quad_decoder quad_decoder_props u_props (.clk, .rst, .ab, .position, .error);
```

Listing 19.8 — Concurrent assertions and cover properties bound to the quadrature decoder (SystemVerilog-2017).

`a_step` encodes the requirement behind the synchronisers: the position changes by at most one count per clock. `a_sticky` checks that a fault flag clears only on reset; `a_double` that a jump of {A, B} to its complement sets the flag on the next clock. The `cover` properties check nothing; they record that counting up, counting down and wrapping actually happened, guarding against tests that pass because they never exercised the design. `bind` attaches the checker to every `quad_decoder` instance, connecting to the internal `ab` by name, so verification code stays out of the RTL.

Coverage

Code coverage (statement, branch, toggle, FSM state and transition) is collected automatically; it shows what was *not* exercised, not that the design is correct. **Functional coverage** is written from the verification plan as covergroups with coverpoints and crosses.

```
covergroup cg_sat_add with function sample(int x, int z, bit s);
  cp_a:   coverpoint x { bins lo = {MINV}; bins neg = {[MINV+1:-1]}; bins zero = {0};
                         bins pos = {[1:MAXV-1]}; bins hi = {MAXV}; }
  cp_b:   coverpoint z { bins lo = {MINV}; bins neg = {[MINV+1:-1]}; bins zero = {0};
                         bins pos = {[1:MAXV-1]}; bins hi = {MAXV}; }
  cp_sat: coverpoint s;
  x_ab:   cross cp_a, cp_b;               // 25 operand-class combinations
endgroup
```

Listing 19.9 — Functional coverage for the saturating adder, sampled from the `check` task of Listing 19.7 (SystemVerilog-2017).

Uniform random stimulus hits the `lo` and `hi` bins with probability 2^{-16} per sample, so their cross bins need the directed corner cases or weighted constraints.

Worked example — how many random tests close coverage? If a covergroup has B bins that random stimulus hits with equal probability, the expected number of bins covered after n tests is

$$E[\text{covered}] = B\left[1 - \left(1 - \frac{1}{B}\right)^{n}\right]$$

and the expected number of tests to cover every bin is $B\,H_B \approx B(\ln B + 0.577)$, with H_B the B-th harmonic number (the coupon-collector result). For $B = 256$ (every value of an 8-bit opcode field), 100 tests cover 82.9 bins on average (32 %), 256 tests cover 162 (63 %), and 95 % coverage needs $n = \ln 0.05 / \backslash\mathrm{n}(255/256) = 765.4$, so 766 tests. Full coverage needs on average $256\,H_{256} = 1568$ tests, twice as many: the last few bins cost as much as all the others (Figure 19.6). Real bins are not equally likely, so the tail is longer still, and the last holes are closed with directed tests or adjusted constraints.

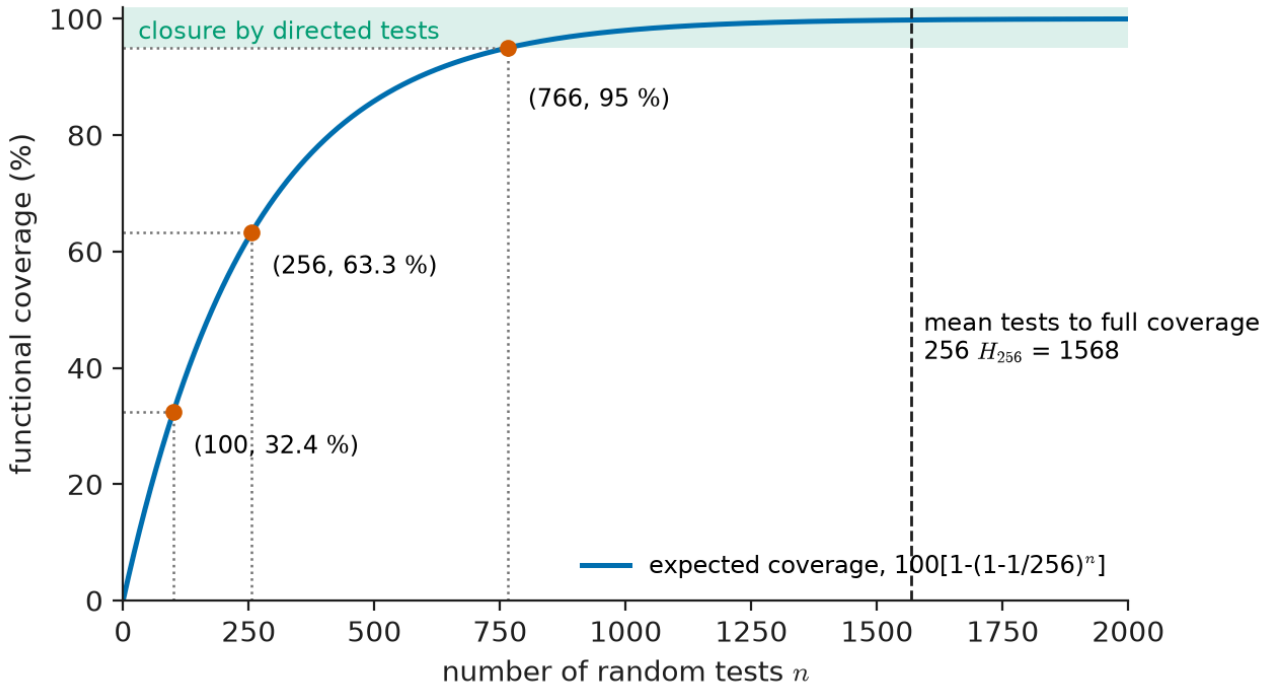


Figure 19.6 — Functional coverage versus number of random tests for 256 equally likely bins.

> **Status of the principle: Random-test coverage growth**
> *Approximate.* The coupon-collector model assumes independent tests and equally likely bins; real stimulus distributions are skewed and correlated, so it gives a lower bound on effort, and its practical lesson (coverage growth slows sharply near 100 %, and the remaining holes need targeted stimulus) is borne out empirically in every verification project.

Formal verification

Formal property checking uses SAT and model-checking engines to prove that an assertion holds for *every* input sequence from reset, or to produce a counterexample trace. For control logic (arbiters, FIFO pointers, protocol FSMs, the decoder above) it finds corner cases no random test would reach. Proofs may be **bounded** (k cycles from reset) or complete; large datapaths make proofs expensive, so formal methods are applied to blocks. **Equivalence checking** proves that RTL and netlist, or two netlists, implement the same function. SymbiYosys brings formal checking to FPGA projects; Cadence Jasper, Synopsys VC Formal and Siemens Questa Formal dominate ASIC work.

Table 19.5 — Verification methods and what they find.

Method	Finds	Misses	Typical cost
Directed tests	Specified behaviour, known corner cases	Unforeseen interactions	Low per test, high to write many
Constrained random + coverage	Unforeseen combinations within constraints	Cases the constraints exclude	Moderate; needs reference model
Assertions in simulation	Protocol and invariant violations, close to the cause	Unexercised behaviour	Low; reusable in formal
Formal property checking	All violations of a property, with a trace	Properties nobody wrote; large datapaths	High expertise; small blocks
Equivalence checking	Synthesis and ECO errors	Errors present in the RTL	Low, automated
Lab test with ILA	Board-level, timing and interface issues	Rare states, poor observability	High per bug

The methods are complementary: a typical drive FPGA runs random regression with coverage on every commit, proves its handshake and FIFO properties formally, checks clock-domain crossings structurally and finishes with hardware-in-the-loop tests (Chapter 28).

What remains open

Open question	Where it is answered
What are the setup, hold and metastability limits that the constraints and synchronisers protect?	Chapter 14 (Sequential Digital Systems)
How are FSMs designed from state diagrams and checked with simple test benches?	Chapter 15 (Digital System Design)
What are LUTs, CLBs, routing, block RAM and DSP slices inside the FPGA?	Chapter 18 (Memory and Programmable Logic)
How are the FIR and IIR filters implemented here designed, and how does word length affect them?	*Circuits, Signals and Communications*, Ch. 23
How do UART, SPI, I²C and CAN interfaces, often written in HDL, work at the protocol level?	Chapter 26 (Embedded Interfaces)
How is hardware-in-the-loop testing of a complete FPGA- or MCU-based controller organised?	Chapter 28 (Embedded System Design)
How do RTL synthesis, placement, routing, STA and verification work for ASICs?	Chapter 34 (VLSI Design)
When should a function go into FPGA logic rather than software, and how is the system verified as a whole?	Chapter 35 (Hardware–Software Co-Design)

Next: Chapter 20 opens Part V by turning from hardware described in concurrent processes to software executed one instruction at a time, starting from the programming fundamentals every engineer uses in C and Python.

Part V — Programming and Computational Systems

Parts III and IV built the machine: logic gates, registers, datapaths, memories, processors and programmable logic, ending with hardware described in a language. Part V turns to the software that runs on that machine and the software engineers write around it. An electronics engineer today programs constantly: firmware in C or C++ for the microcontroller inside a product, Python for acquiring and analysing measurements and automating a test rig, and numerical code for simulating a circuit, a filter or a control loop before it is built. These programs have finite word lengths, memory and time, and their failures (an accumulator that stops counting, a pointer that outlives its buffer, a result that cannot be reproduced) are as real as a resistor that overheats. This part treats programming as an engineering discipline, with the same habit as the rest of the book: state the model, know where it holds, and measure the consequence.

By the end of this part you should understand

- How an algorithm is specified and argued correct, and how variables, types, control flow, functions and data structures map onto the memory and time of a real machine.
- Where data live (static storage, stack, heap), how big each region must be on a microcontroller, and why firmware usually avoids dynamic allocation.
- How to organise code into modules, handle errors and faults deliberately, debug systematically and build test suites that run on every change.
- How C and C++ expose pointers, structure layout, bits, memory-mapped registers and the compiler-linker toolchain, and how embedded coding standards restrict them for safety-critical work.
- How Python, with NumPy, SciPy and Matplotlib, acquires, processes, plots and archives engineering data, and how to make a computational experiment reproducible.
- How to analyse the cost of an algorithm, choose a data structure for a task, and apply searching, sorting, numerical and graph algorithms to engineering problems.
- How scientific computing environments, matrix computation, data formats, parallelism, profiling, numerical precision, testing and version control fit together in a professional simulation workflow.

Chapters

Chapter	Central question	What it delivers
20 — Programming Fundamentals	How does an engineer turn a calculation or control rule into a correct, robust program?	Algorithms and invariants, data types and their limits, control flow, functions, ring buffers, the stack/heap memory model, modules, error handling, debugging and testing, in C and Python
21 — C/C++ for Engineering	How do C and C++ map onto memory, registers and the toolchain, and how are they made safe for firmware?	Integer promotions and undefined behaviour, pointers, struct layout, memory pools, bit manipulation, `volatile` and register maps, classes, RAII, interfaces, linking and start-up code, MISRA-style subsets
22 — Python for Engineering	How is Python used to acquire, analyse, visualise and automate engineering data?	NumPy arrays, data processing, plotting, SciPy, signal analysis, instrument control and data acquisition, automation scripts, file formats, reproducible experiments
23 — Algorithms and Data Structures	How is an algorithm's cost analysed, and which structure and algorithm suit a given engineering task?	Complexity analysis, lists, stacks, queues, trees, graphs, hash tables, searching and sorting, numerical and graph algorithms, selection criteria with engineering applications
24 — Scientific Computing	How are large numerical studies built, run, checked and shared reliably?	Computing environments, matrix computation, scientific data formats, simulation workflows, parallelism, profiling, numerical precision, reproducibility, software testing, version control and collaboration

How this part connects to the rest of the book

Part V stands on the hardware of Parts III and IV. The number representations of Chapter 11 explain why a `float` accumulator freezes and why integer counters wrap; the processor, memory hierarchy and memory map of Chapters 16 and 17 explain what a pointer, a stack frame or a cache-friendly loop really is; and the HDL of Chapter 19 is the hardware counterpart of this software, with the same test discipline. Part V in turn feeds Part VI directly: the C, C++, memory, interrupt-safety and testing practice of Chapters 20 and 21 becomes the firmware architecture, interface drivers, real-time scheduling and hardware-in-the-loop validation of Chapters 25–28.

The mathematics behind the numerical work belongs to *The Mathematics Universe*: floating-point error analysis, root finding, interpolation, quadrature and ODE solvers in Ch. 22, optimisation in Ch. 23, and numerical linear algebra, iterative solvers and Monte Carlo methods in Ch. 24. This part uses those results and adds word lengths, memory budgets, libraries, data formats and workflows. The signal-processing algorithms that Python analyses and C implements (sampling, the DFT and FFT, digital filter design) are owned by *Circuits, Signals and Communications*, Part V, and the discrete controllers coded in firmware by its Ch. 35.

Chapter 20 — Programming Fundamentals

Part V · Programming and Computational Systems

The battery-management controller of an electric scooter estimates state of charge by **coulomb counting**: every millisecond it adds $I\,\Delta t$ to a running total of charge. On a 30-minute lab test the estimate agrees with a reference meter to 2 %. In the field, riders on long trips report that the gauge falls too fast after about four and a half hours and then freezes, while the pack keeps discharging until the undervoltage cut-out stops the scooter without warning. The algorithm is right and the sensor calibrated. The fault is one word, `float`: a 32-bit floating-point accumulator that, above 32 768 A·s, can no longer add 0.002 A·s correctly, and above 65 536 A·s cannot add it at all. Programming for engineering is the craft of writing code that is right not only in the test that was run but for every input, duration and failure the product will meet. This chapter sets out the fundamentals, language-neutrally, with examples in C (the language of firmware) and Python (the language of analysis, test and automation).

> **How does an engineer turn a calculation or a control rule into a correct, efficient program, and what does the machine actually do with variables, functions and data?**
> **How are programs organised, made robust against bad inputs and failures, debugged and tested so that they stay correct as they grow?**

Concept	Engineering destination
Algorithms	Lookup-and-interpolate calibration, filters, control laws, search and sort
Variables and data types	Choosing integer widths, fixed or floating point, avoiding overflow and lost precision
Control flow	Hysteresis controllers, superloops, state-driven firmware
Functions	Reusable, testable units; interfaces between firmware layers
Arrays and data structures	Sample buffers, ring buffers for ADC and UART data, records for telemetry
Memory	Stack and heap budgets on microcontrollers; leaks and dangling pointers
Debugging	Finding faults systematically with debuggers, logging, sanitisers and bisection
Modular programming	Drivers, libraries and packages that teams can share and replace
Error handling	Sensor faults, timeouts, safe states in industrial and medical equipment
Testing	Unit and regression tests, coverage, evidence for IEC 61508, ISO 26262, DO-178C

Roadmap. Algorithms and their correctness (20.1) → variables and types (20.2) → control flow (20.3) → functions (20.4) → arrays and data structures (20.5) → memory (20.6) → modules (20.7) → error handling (20.8) → debugging (20.9) → testing (20.10).

20.1 Algorithms

From a calculation to a procedure

An **algorithm** is a finite sequence of well-defined steps that transforms inputs into outputs and terminates. Engineers meet algorithms everywhere: the difference equation of a digital filter, the update of a PI controller, the conversion of an ADC count to a temperature, the search for a free slot in a schedule. Writing one down precisely, before coding, separates the questions "what should be computed?" and "how does this language express it?".

Take a thermistor input on a battery pack. A 10 kΩ NTC thermistor ($B = 3950$ K) sits under a 10 kΩ pull-up resistor feeding a 12-bit ADC. The resistance–temperature law is strongly nonlinear, and evaluating a logarithm on a small microcontroller costs hundreds of cycles, so the firmware stores a **calibration table**: ADC counts at every 5 °C from −40 to +125 °C, 34 points, decreasing from 3996 counts at −40 °C to 142 counts at +125 °C. To convert a reading, the algorithm must (1) reject readings outside the table, (2) find the two table entries that bracket the reading, and (3) interpolate linearly between them.

Step (2) can be done by scanning the table from one end, up to 33 comparisons, or by **binary search**: compare the reading with the middle entry, discard the half that cannot contain it, and repeat. Each comparison halves the interval, so a table of n entries needs at most $\lceil \log_2(n-1) \rceil$ comparisons, here 6. The cost of algorithms as functions of problem size and the choice among them are the subject of Chapter 23; the mathematics of algorithmic cost is in *The Mathematics Universe*, Ch. 24 §24.1. Figure 20.1 shows the procedure as a flowchart and Listing 20.1 as Python.

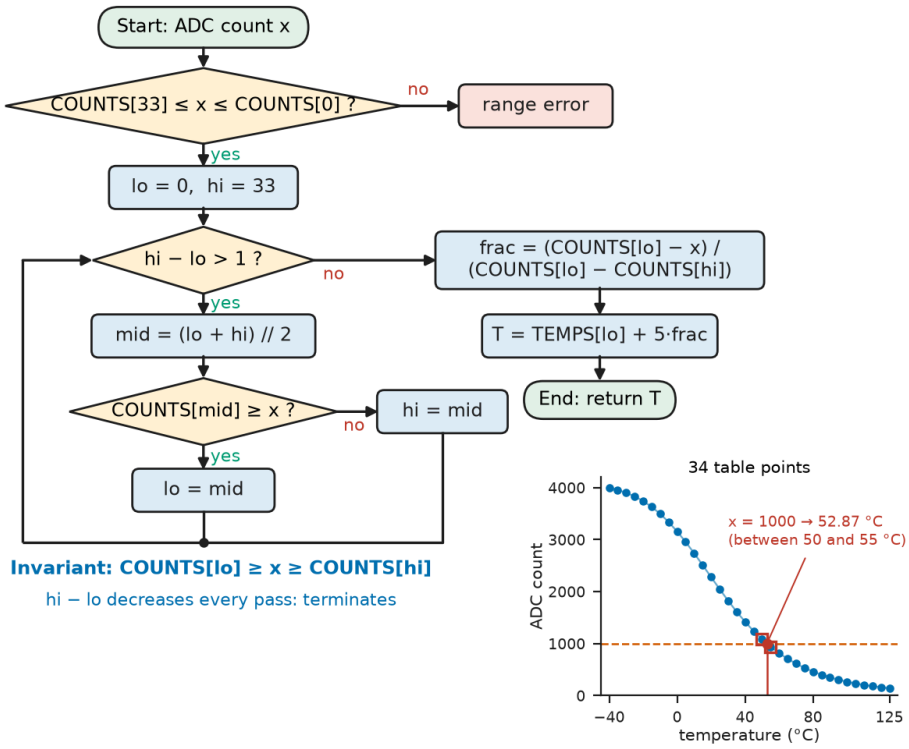


Figure 20.1 — Binary search with linear interpolation for converting a thermistor ADC count to temperature, with the loop invariant marked.

```
 import math

# Calibration table: 10 kOhm NTC (B = 3950 K) under a 10 kOhm pull-up, 12-bit ADC.
TEMPS = list(range(-40, 126, 5))                    # deg C, 34 points
def adc_of(t_c, r25=10e3, beta=3950.0, r_pu=10e3):
    r = r25 * math.exp(beta * (1 / (t_c + 273.15) - 1 / 298.15))
    return round(4095 * r / (r + r_pu))
COUNTS = [adc_of(t) for t in TEMPS]                 # strictly decreasing

def adc_to_temp(x):
    """Return temperature in deg C for ADC count x by binary search + interpolation."""
    if not COUNTS[-1] <= x <= COUNTS[0]:
        raise ValueError(f"ADC count {x} outside calibrated range")
    lo, hi = 0, len(COUNTS) - 1
    # invariant: COUNTS[lo] >= x >= COUNTS[hi]
    while hi - lo > 1:
        mid = (lo + hi) // 2
        if COUNTS[mid] >= x:
            lo = mid
        else:
            hi = mid
    frac = (COUNTS[lo] - x) / (COUNTS[lo] - COUNTS[hi])
    return TEMPS[lo] + frac * (TEMPS[hi] - TEMPS[lo])

if __name__ == "__main__":
    for x in (3000, 2048, 1000, 300):
        print(x, round(adc_to_temp(x), 2))
```

Listing 20.1 — Table lookup by binary search with linear interpolation (Python 3; saved as `ntc.py` for the tests of Section 20.10).

Running it prints 3.88, 25.0, 52.87 and 95.66 °C for counts of 3000, 2048, 1000 and 300. The closed-form thermistor law gives 3.92, 24.99, 52.80 and 95.64 °C, so the table with 5 °C spacing is accurate to better than 0.1 °C at these points; over the whole range the worst error, found by the test of Section 20.10, is 0.16 °C, well inside the ±1 °C tolerance of the thermistor itself.

Correctness: invariants and termination

How do we know the loop is right? Not by trying values, but by an argument. The comment states a **loop invariant**: a condition true before the loop, preserved by every iteration, and giving the result when combined with the exit condition. The range check makes it true initially; each iteration moves `lo` or `hi` to `mid` only on the side that keeps it true; at exit `hi - lo == 1`, so the two entries bracket x, which is what interpolation needs. **Termination** follows because `hi - lo` is a positive integer that strictly decreases.

> **Status of the principle: Loop invariants and termination**
> *Derived.* Proof by induction on the iterations: if an invariant holds initially and each iteration preserves it, it holds at exit, and a strictly decreasing non-negative integer measure guarantees exit; the argument is about the algorithm, so it does not cover the language's arithmetic limits (overflow, rounding), which must be checked separately (Section 20.2).

20.2 Variables and data types

A variable is named storage with a type

A **variable** is a name bound to storage; its **type** says how many bits it occupies, what they represent and which operations are allowed. In C, types are **static**: declared, checked by the compiler and mapped directly onto machine words. In Python, types are **dynamic**: a name refers to an object that carries its type at run time. C gives control and predictable cost; Python gives convenience and safety, at a cost of one to two orders of magnitude in speed and memory for scalar code.

Table 20.1 — Common data types in C (on 32-bit Arm Cortex-M and 64-bit x86/Arm hosts) and Python.

Type	Size	Range or precision	Typical use
`uint8_t` / `int8_t`	1 byte	0 to 255 / −128 to 127	Bytes on a bus, small counters, status flags
`uint16_t` / `int16_t`	2 bytes	0 to 65 535 / −32 768 to 32 767	ADC samples, Q15 fixed-point values
`uint32_t` / `int32_t`	4 bytes	0 to 4.29×10^9 / $\pm 2.15 \times 10^9$	Tick counters, addresses on 32-bit MCUs, Q31 values
`uint64_t` / `int64_t`	8 bytes	0 to 1.8×10^{19} / $\pm 9.2 \times 10^{18}$	Energy and charge accumulators, timestamps in ns
`float` (IEEE 754 binary32)	4 bytes	about 7 significant digits	DSP and control on MCUs with single-precision FPU
`double` (binary64)	8 bytes	about 16 significant digits	Host analysis; accumulators; Python's `float`
`bool`	1 byte (C)	`true` / `false`	Flags, conditions
`char` arrays / `str`	1 byte per character (C, ASCII/UTF-8)	—	Messages, commands, file names
Python `int`	variable	unbounded (arbitrary precision)	Counters, bit manipulation on the host

In C the plain types `int` and `long` have implementation-defined widths (`int` is 16 bits on an AVR and 32 bits on Cortex-M and x86-64), so embedded code uses the fixed-width types of `<stdint.h>`. Python's integers never overflow, which is convenient on the host but means that Python code modelling firmware must apply word-length limits explicitly, for example with NumPy's fixed-width types (Chapter 22).

The binary representations behind these types (two's complement, fixed point, the IEEE 754 formats) are the subject of Chapter 11 (Sections 11.3, 11.6 and 11.7); the error model of floating-point arithmetic, with machine epsilon $\varepsilon_M = 2^{-23}$ for binary32 and 2^{-52} for binary64, is in *The Mathematics Universe*, Ch. 22 §22.1. Here the question is the programmer's: which type to choose, and what happens at its limits.

Floating point: when adding stops working

A floating-point number has a fixed number of significant bits, so the spacing between representable values (the **ulp**, unit in the last place) grows with magnitude. For binary32 with value in $[2^e, 2^{e+1})$ the spacing is 2^{e-23}. Adding a small increment δ to a large total S gives a result rounded to the nearest multiple of the ulp of S:

- if δ is much larger than the ulp, the error per addition is a small fraction of δ;
- if δ is about one ulp, each addition rounds to one or two ulps, an error of tens of percent;
- if δ is less than half an ulp, $S + \delta$ rounds back to S, and the total stops changing.

> **Status of the principle: Rounding in floating-point accumulation**
> *Derived.* From round-to-nearest in IEEE 754: each addition $S + \delta$ is exact to within half an ulp of the result, so the relative error per step is about $\mathrm{ulp}(S)/(2\delta)$, which grows without bound as S grows while δ stays fixed; it holds for any binary floating-point format, with the threshold moving to larger S as precision increases.

Worked example — the coulomb counter. The scooter's controller adds $\delta = 2\ \mathrm{A} \times 1\ \mathrm{ms} = 0.002$ A·s every millisecond to a binary32 total. The ulp in $[2^{15}, 2^{16}) = [32\,768,\ 65\,536)$ A·s is $2^{15-23} = 2^{-8} = 0.003\,906$ A·s, so δ is 0.512 ulp and each addition rounds up to one full ulp: the count runs at $0.003\,906/0.002 = 1.95$ times the true rate. In $[2^{16}, 2^{17})$ the ulp is 0.007 81 A·s, δ is 0.256 ulp, rounds to zero, and the count freezes. Between 2^{11} and 2^{15} A·s the increment rounds to a whole number of ulps (8, 4, 2 or 1) that is 2.3 % below its true value. Listing 20.2 simulates eight hours at 2 A.

```
 import numpy as np

dt, current = 1e-3, 2.0                 # 1 ms tick, 2 A constant discharge
n = 8 * 3600 * 1000                     # 8 hours of ticks
step32 = np.full(n, current * dt, dtype=np.float32)
q32 = np.cumsum(step32, dtype=np.float32)    # sequential float32 accumulation
q64 = np.cumsum(np.full(n, current * dt), dtype=np.float64)

for hours in (1, 4, 5, 6, 7, 8):
    k = hours * 3600 * 1000 - 1
    true = current * (k + 1) * dt
    print(f"{hours} h: true {true:8.0f} A*s  float32 {q32[k]:8.0f}  "
          f"float64 {q64[k]:10.3f}")
```

Listing 20.2 — Simulating a coulomb counter with single and double precision (Python 3, NumPy).

The output (true charge, binary32, binary64): after 1 h, 7200, 7060 and 7200.000 A·s (binary32 already 1.9 % low); after 4 h, 28 800, 28 154; after 5 h, 36 000, 37 603; after 6 h, 43 200, 51 665; after 7 and 8 h the binary32 count is stuck at 65 536 A·s against a true 50 400 and 57 600 A·s. It crosses 32 768 A·s at 4.66 h (true 33 525 A·s) and reaches 65 536 A·s at 6.99 h (true 50 302 A·s): a 30 % overestimate, then a frozen gauge, exactly the field complaint (Figure 20.2). The binary64 total is still correct to 10^{-5} A·s after 10 h.

Two fixes are standard. On an MCU with only a single-precision FPU, accumulate in an **integer** of fixed resolution: charge in units of 1 mA·ms in an `int64_t` is exact, never loses an increment and, at 2 A, would take $9.2 \times 10^{18}/2000$ ms, about 146 000 years, to overflow. On a host or a processor with a double-precision FPU, use `double`. Either way the general rule is to choose the type for the *largest* value and the *smallest* increment the variable will ever see, not for the values in the first test.

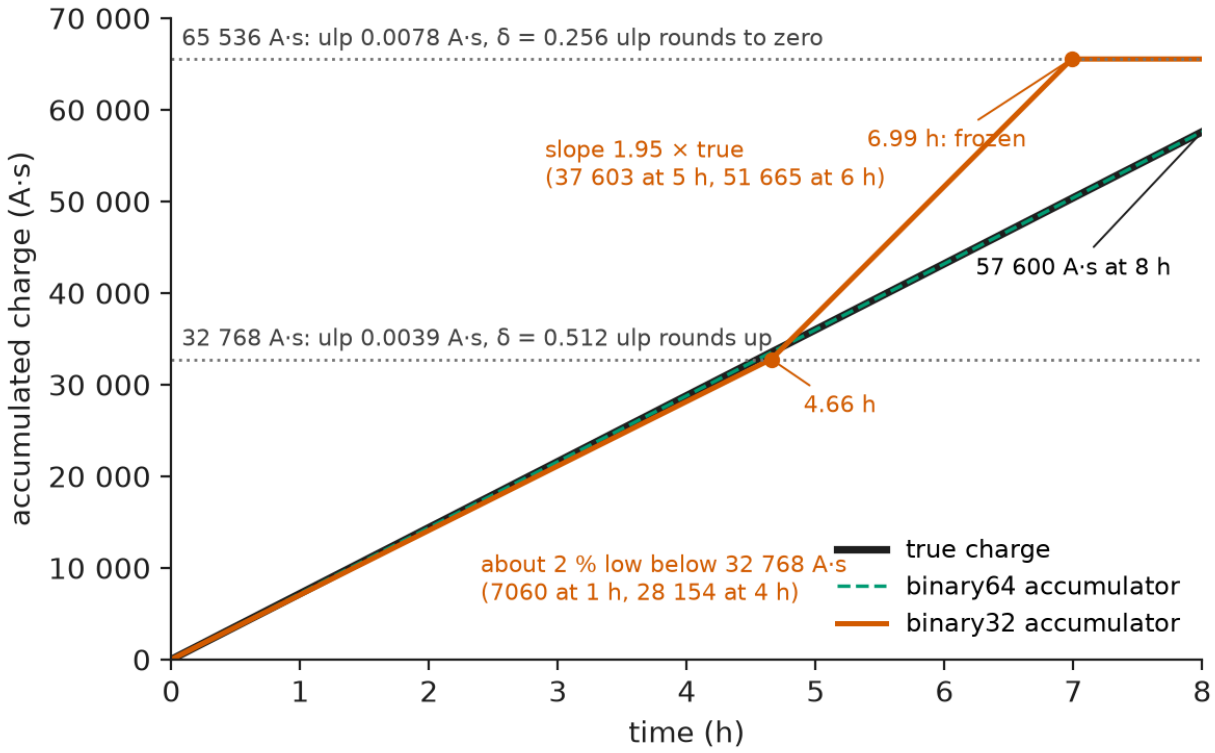


Figure 20.2 — Coulomb counter adding 0.002 A s per millisecond: true charge, binary32 and binary64 accumulators over eight hours.

Integers: overflow and wrap-around

Fixed-width integers have the opposite failure: they are exact until they overflow. Unsigned arithmetic in C is defined to wrap modulo 2^N; signed overflow is **undefined behaviour**, meaning the compiler may assume it never happens and optimise accordingly (Chapter 21). The classic case is the millisecond tick counter of an embedded system, a `uint32_t` incremented by a timer interrupt. It wraps after 2^{32} ms $= 49.7$ days, which is why some devices fail on day 50 of uptime. The cure is to compare **elapsed times**, computed by unsigned subtraction, which remains correct across one wrap:

```
#include <stdbool.h>
#include <stdint.h>

extern volatile uint32_t g_ms_ticks;           /* incremented by a 1 kHz timer ISR */

bool timeout_expired(uint32_t start_ms, uint32_t timeout_ms)
{
    return (uint32_t)(g_ms_ticks - start_ms) >= timeout_ms;   /* correct across wrap */
}
/* WRONG: (g_ms_ticks >= start_ms + timeout_ms) fails when the sum wraps past 2^32 */
```

If `start_ms` is 4 294 967 000 and the counter has wrapped to 704, the difference is $704 - 4\,294\,967\,000 + 2^{32} = 1000$ ms, the correct elapsed time, whereas the sum `start_ms + timeout_ms` wraps to a small number and the "wrong" comparison would declare the timeout expired immediately.

Conversions

Mixing types triggers **conversions**: integer division truncates (`7 / 2` is 3 in C; Python's `/` gives a float), float-to-integer conversion truncates toward zero (and is undefined in C when out of range), and narrowing keeps the low bits. Every conversion should be deliberate and reviewed for range.

20.3 Control flow

Sequence, selection and iteration

Every computable procedure can be written with three control structures: **sequence** (do this, then that), **selection** (`if` / `else`, `switch` / `match`) and **iteration** (`while`, `for`). Programs built only from these, each with one entry and one exit, are **structured**, and they can be read and reasoned about block by block, which is why structured programming replaced free use of `goto` in the 1970s.

> **Status of the principle: Structured program theorem**
> *Derived.* Böhm and Jacopini (1966) showed that any flowchart program can be rewritten with sequence, selection and iteration alone, using extra Boolean variables if needed; it guarantees expressiveness, not readability, and practical rules (MISRA C, for example) still permit early `return`, `break` and a forward `goto` to a single error exit where they make code clearer.

Loops are **counted** (`for`), **conditional** (`while`, possibly zero times) or run at least once (`do ... while` in C); `break` leaves early and `continue` skips to the next iteration. `&&` / `and` and `||` / `or` evaluate **left to right with short-circuit**: in `if (p != NULL && p->ready)` the pointer is dereferenced only if valid.

State as control flow: a thermostat with hysteresis

A heater controlled by "on below the set point, off above it" chatters on and off at every small noise excursion. Adding **hysteresis**, a band within which the output keeps its previous state, makes the behaviour depend on history, and history is stored in a state variable. Listing 20.3 is the C function called once per control period.

```
 typedef enum { HEATER_OFF, HEATER_ON } heater_state_t;

heater_state_t thermostat_step(heater_state_t s, float temp_c,
                               float setpoint_c, float band_c)
{
    switch (s) {
    case HEATER_OFF:
        if (temp_c < setpoint_c - 0.5f * band_c) {
            s = HEATER_ON;
        }
        break;
    case HEATER_ON:
        if (temp_c > setpoint_c + 0.5f * band_c) {
            s = HEATER_OFF;
        }
        break;
    default:
        s = HEATER_OFF;                 /* corrupted state value: fail safe */
        break;
    }
    return s;
}
```

Listing 20.3 — A two-state hysteresis controller (C11).

With a set point of 37.0 °C and a 0.4 °C band (an incubator), the heater switches on below 36.8 °C and off above 37.2 °C. The `default` branch handles a state value that should be impossible, for instance after a memory upset, by choosing the safe output. This is a two-state **finite-state machine**, the abstraction of Chapter 15 expressed as control flow; larger firmware state machines belong to Chapter 28.

The superloop

The simplest embedded program is a **superloop**: initialise, then repeat forever: read inputs, compute, write outputs, possibly wait for the next tick. Its timing is the sum of the execution times of everything in the loop, so one slow branch (a flash write, a blocking serial read) delays everything else. Interrupts, schedulers and real-time operating systems, which relax this constraint, belong to Chapter 27.

20.4 Functions

Abstraction with a contract

A **function** packages a computation behind a name, parameters and a result. A good function has a **contract**: preconditions on inputs, postconditions on the result, and stated side effects. Functions are the unit of reuse, of testing and of reasoning: once `adc_to_temp` is trusted, its callers can be read without reading its body.

A **pure function** depends only on its arguments and has no side effects; it always gives the same output for the same input, which makes it trivially testable and safe to call from anywhere. Functions that read hardware, change global state or perform I/O are **impure**, and the art of structuring a program is to keep the impure parts thin and push the computation into pure functions.

How arguments are passed

In C every argument is passed **by value**: the function receives a copy. To let a function modify the caller's data, or to avoid copying a large structure, the caller passes a **pointer**, and `const` on the pointed-to type promises that the function will not modify it:

```
#include <stddef.h>

float mean(const float *x, size_t n)        /* const: x[] is read only */
{
    float sum = 0.0f;
    for (size_t i = 0; i < n; i++) {
        sum += x[i];
    }
    return (n > 0u) ? sum / (float)n : 0.0f;
}

void scale(float *x, size_t n, float gain)  /* modifies the caller's array */
{
    for (size_t i = 0; i < n; i++) {
        x[i] *= gain;
    }
}
```

Python passes **object references** by value: the function receives a reference to the same object. Rebinding the parameter name inside the function does not affect the caller, but mutating a mutable object (a list, a dict, a NumPy array) does. A related trap is that default argument values are evaluated once, when the function is defined:

```
def scale_in_place(samples, gain):
    for i in range(len(samples)):        # mutates the caller's list
        samples[i] *= gain

def log_event(msg, log=[]):              # BUG: one default list shared by all calls
    log.append(msg)
    return log

def log_event_fixed(msg, log=None):
    if log is None:
        log = []                         # a fresh list on every call
    log.append(msg)
    return log

x = [1.0, 2.0, 3.0]
scale_in_place(x, 0.5)
print(x)
print(log_event("overtemp"), log_event("undervolt"))
print(log_event_fixed("overtemp"), log_event_fixed("undervolt"))
```

Listing 20.4 — Argument passing and the mutable-default trap (Python 3).

It prints `[0.5, 1.0, 1.5]` (the caller's list was modified), then `['overtemp', 'undervolt']` twice, because both calls appended to the same hidden list, and finally `['overtemp']` and `['undervolt']` from the corrected version.

Scope, recursion and the cost of a call

Variables declared inside a function are **local**: they exist only while the function runs and are invisible outside it. **Global** variables are visible everywhere, which makes them convenient and dangerous: any function may change them, so reasoning about one function requires reading all of them. Firmware limits globals to hardware state and data shared with interrupts, and gives them file scope (`static` in C) wherever possible.

Each call creates a **stack frame** (Section 20.6) for the return address, saved registers and locals. **Recursion** adds a frame per level, so its stack depth depends on the input, which is why safety-critical rules forbid it (Chapter 23 uses it for trees and divide-and-conquer). A call costs a few cycles on a Cortex-M and tens of nanoseconds in CPython, and compilers **inline** small functions, so in C a function boundary is almost never a reason to write longer functions.

20.5 Arrays and data structures

Arrays

An **array** stores elements of one type contiguously, so element i is at base $+\, i \times$ size and is reached in constant time. In C the index starts at 0 and **nothing checks the bounds**: `x[n]` for an n-element array touches whatever lies next in memory, the most common serious C bug (Section 20.9). Python lists check bounds and hold references to arbitrary objects, at a cost in memory and speed; NumPy arrays (Chapter 22) store raw numbers contiguously like C. Two-dimensional data are row-major in C and NumPy and column-major in Fortran and MATLAB; traversing in storage order is several times faster because consecutive accesses share cache lines (Chapter 16).

Records

A **record** groups values of different types under one name: a `struct` in C, a `dataclass` in Python, such as a solar-inverter telemetry sample with timestamp, DC voltage, AC power and status. Records make interfaces explicit; Chapter 21 shows how field order changes a struct's size.

The ring buffer

The single most useful data structure in embedded and real-time code is the **ring buffer** (circular FIFO). A producer, typically an interrupt service routine storing ADC samples or received UART bytes, writes at the **head**; a consumer, the main loop or a task, reads at the **tail**; both indices wrap around a fixed array (Figure 20.3). No data are moved and no memory is allocated, so every operation takes constant time. Section 17.8 used one with 8-bit indices for UART reception; Listing 20.5 generalises it with free-running 32-bit indices and a power-of-two size, so the fill level is simply `head - tail` and the array index is a bit mask.

```
 #include <stdbool.h>
#include <stdint.h>

#define RB_SIZE 256u                          /* must be a power of two */

typedef struct {
    int16_t           data[RB_SIZE];
    volatile uint32_t head;                   /* written only by the producer */
    volatile uint32_t tail;                   /* written only by the consumer */
} ring_t;

bool rb_put(ring_t *rb, int16_t v)            /* producer, e.g. ADC interrupt */
{
    if (rb->head - rb->tail == RB_SIZE) {
        return false;                         /* full: sample dropped, count it */
    }
    rb->data[rb->head & (RB_SIZE - 1u)] = v;
    rb->head++;
    return true;
}

bool rb_get(ring_t *rb, int16_t *v)           /* consumer, e.g. main loop */
{
    if (rb->head == rb->tail) {
        return false;                         /* empty */
    }
    *v = rb->data[rb->tail & (RB_SIZE - 1u)];
    rb->tail++;
    return true;
}
```

Listing 20.5 — A single-producer, single-consumer ring buffer (C11).

Because each index is written by only one side, and a 32-bit aligned store is a single, indivisible operation on a 32-bit processor, this buffer is safe between one interrupt and one main loop on a single-core microcontroller without disabling interrupts. On multi-core processors, or with compilers that reorder memory accesses, memory barriers or C11 atomics are also needed; concurrency and race conditions are treated in Chapter 27, and `volatile` in Chapter 21.

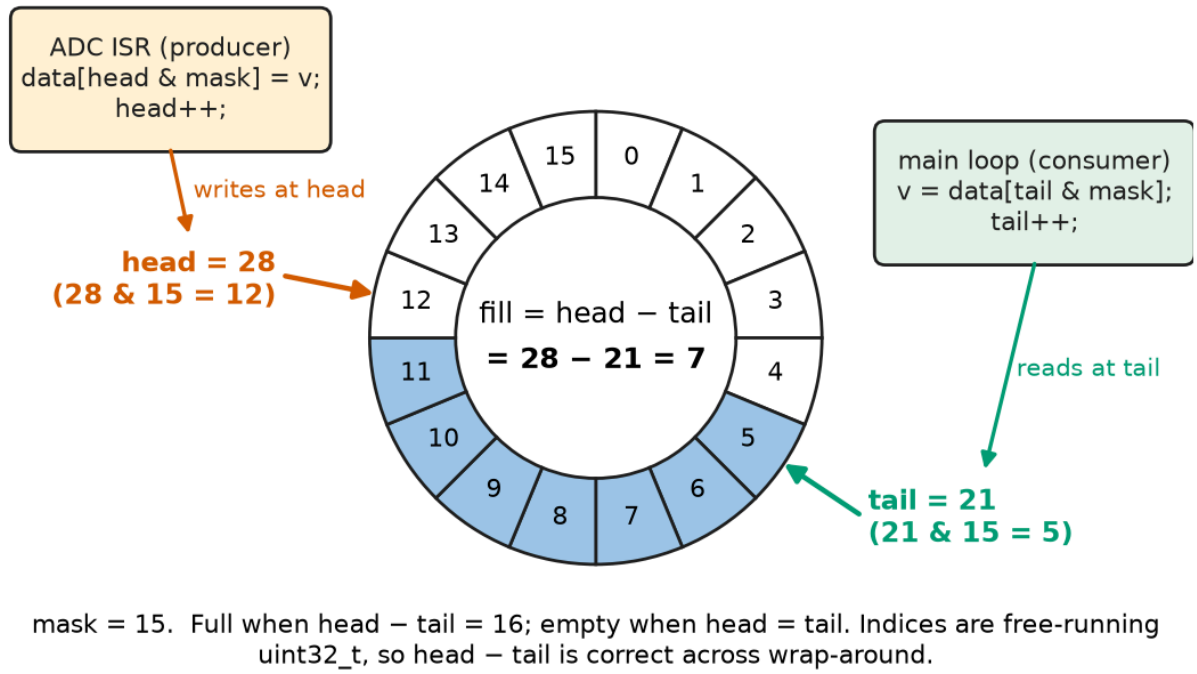


Figure 20.3 — A 16-slot ring buffer between an ADC interrupt (producer) and the main loop (consumer), with free-running indices.

Worked example — sizing a sample buffer. A vibration monitor on an industrial pump samples three accelerometer axes at 1 kHz with 16-bit samples, and must ride through the main loop being blocked for up to 2 s while it writes a log block to flash. The buffer needs $1000 \times 3 \times 2$ bytes $\times\ 2$ s $= 12\,000$ bytes; the next power of two in samples is 8192 samples (16 384 bytes), 8 % of the 192 KB of SRAM on an STM32F407-class MCU. If 16 KB were too much, the remedy would be to make the flash write non-blocking, not to shrink the buffer and lose data.

Other structures

Linked lists, stacks, queues, trees, graphs and hash tables, and the algorithms that use them, are the subject of Chapter 23. Python provides the most important of them built in: `list` (dynamic array), `dict` (hash table), `set`, `collections.deque` (double-ended queue) and `heapq` (priority queue).

20.6 Memory

The memory model of a running program

Every variable lives somewhere, and where it lives decides how long it lives and how much of it there can be. A compiled C program, on a microcontroller or under an operating system, divides its memory into regions (Figure 20.4):

- **Text (code)**: the machine instructions, and **read-only data** (`.rodata`) such as constant tables and string literals. On a microcontroller these stay in flash and execute in place.
- **Static data**: variables that exist for the whole run. Initialised ones (`.data`) are copied from flash to RAM at start-up; zero-initialised ones (`.bss`) are cleared at start-up. Globals and `static` locals live here.
- **Stack**: a region that grows and shrinks with function calls. Each call pushes a frame with return address, saved registers and local variables; each return pops it. Allocation is a single subtraction from the stack pointer, so it is fast and deterministic, but the size is fixed when the program is built.
- **Heap**: a region from which blocks are allocated and released explicitly at run time (`malloc` / `free` in C, `new` / `delete` in C++), or implicitly for every object in Python.

```c
#include <stdint.h>
#include <stdlib.h>

const int16_t sine_lut[256] = { 0, 804, 1608 /* ... */ };  /* .rodata: flash */
uint32_t error_count = 0;            /* .bss: zeroed at start-up */
static float gain = 1.25f;           /* .data: initial value copied from flash */

float *make_buffer(size_t n)
{
    float *p = malloc(n * sizeof *p);    /* heap: lives until free(p) */
    return p;                            /* may be NULL: the caller must check */
}

int16_t *broken(void)
{
    int16_t local[16];                   /* stack: dies when the function returns */
    return local;                        /* BUG: dangling pointer (compilers warn) */
}
```

Listing 20.6 — Where C variables live (C11).

> **Status of the principle: Stack discipline**
> *Fundamental.* By definition of the call mechanism, frames are created and destroyed in last-in, first-out order, so a local variable's lifetime ends when its function returns; any pointer to it held beyond that point refers to memory that will be reused by the next call, which is the root of every dangling-stack-pointer bug.

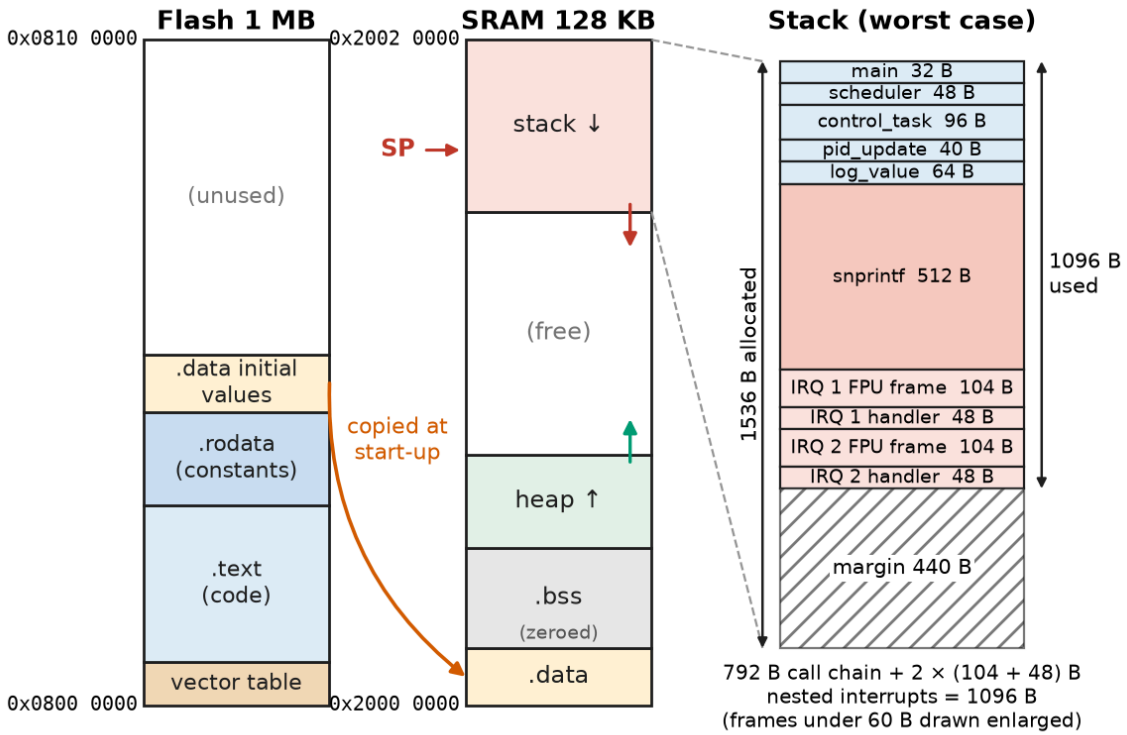


Figure 20.4 — Memory layout of a program on a Cortex-M4 microcontroller and the stack frames of its deepest call chain.

Stack and heap on a microcontroller

On a PC the operating system gives each thread a stack of a megabyte or more and grows the heap on demand. On a microcontroller the whole SRAM may be 20 KB to 1 MB, the stack is a fixed region set in the linker script, and nothing stops it from growing into the static data below it. A **stack overflow** then silently corrupts variables, and the symptom appears far from the cause.

Worked example — a stack budget. A motor controller's deepest call chain is `main` (32 bytes of frame) → `scheduler` (48) → `control_task` (96) → `pid_update` (40) → `log_value` (64) → `snprintf` (512, a formatted-print routine is often the largest single user of stack). That is 792 bytes. Interrupts use the same stack on a bare-metal Cortex-M: each one pushes a hardware frame of 104 bytes when the floating-point unit is in use (26 words) and its handler needs 48 bytes of locals, and two priority levels can nest, adding $2 \times (104 + 48) = 304$ bytes. The worst case is $792 + 304 = 1096$ bytes; with a 25 % margin, 1370 bytes, so the stack is set to 1.5 KB. The figures for each function come from the compiler (GCC's `-fstack-usage` writes them to `.su` files) and the result is checked on the running system by filling the stack with a known pattern at start-up and inspecting how much was overwritten after a long test ("stack painting").

The heap brings two further risks. A **leak** grows until allocation fails, typically after days. **Fragmentation** leaves free memory in pieces too small for a request although the total is ample. Allocation time also varies. Most safety-critical firmware therefore allocates statically or only at initialisation (memory pools, Chapter 21).

Managed memory in Python

Python allocates every object on its heap and frees it automatically, by reference counting plus a cycle collector, so dangling pointers cannot occur. The costs are memory (a Python `int` occupies 28 bytes, a `float` 24, against 4 or 8 in C) and occasional collection pauses, one reason Python runs on the test bench while firmware is C and C++; MicroPython brings a subset of Python to microcontrollers with tens of kilobytes of RAM or more.

20.7 Modular programming

Interfaces and implementations

A program of a few hundred lines can live in one file. A product's firmware, often 50 000 to 500 000 lines, cannot: it is divided into **modules**, each with a single responsibility, a published **interface** and a hidden **implementation**. Other modules depend only on the interface, so the implementation can be fixed, optimised or replaced (a different sensor, a new MCU) without touching them.

In C a module is a pair of files. The **header** (`.h`) declares the types and functions that form the interface; the **source** (`.c`) defines them, and marks internal helpers and data `static` so that they are invisible outside the file. A PID controller module might publish:

```
 /* pid.h - interface of the PID controller module */
#ifndef PID_H
#define PID_H

typedef struct {
    float kp, ki, kd;             /* gains */
    float integ, prev_err;        /* internal state */
    float out_min, out_max;       /* actuator limits, used for anti-windup */
} pid_ctrl_t;

void  pid_init(pid_ctrl_t *c, float kp, float ki, float kd,
               float out_min, float out_max);
float pid_update(pid_ctrl_t *c, float setpoint, float measured, float dt_s);

#endif /* PID_H */
```

Listing 20.7 — A C module interface (C11).

The include guard prevents double definition. The discrete PID law itself comes from *Circuits, Signals and Communications*, Ch. 35 §35.2 and §35.8; the module only packages it. Because state lives in a `pid_ctrl_t` passed by pointer, not in globals, one module serves the current, speed and position loops of a drive and can be tested on a PC. Declaring only `typedef struct pid_ctrl pid_ctrl_t;` in the header (an **opaque type**) hides the fields completely.

In Python a module is a `.py` file and a **package** is a directory of modules; names beginning with an underscore are internal by convention. A test-automation package for a power-supply production line might be laid out as:

```
psu_test/
    __init__.py          # package marker, public API re-exports
    instruments.py       # SCPI drivers: DMM, electronic load, oscilloscope
    limits.py            # pass/fail limits loaded from a versioned file
    sequences.py         # test steps: no-load voltage, line and load regulation
    report.py            # CSV and PDF reports
tests/
    test_limits.py       # unit tests run without instruments
```

Coupling, cohesion and layers

Good modules have **high cohesion** and **low coupling**. Firmware is usually layered, drivers at the bottom, then a hardware abstraction layer, middleware and application logic, with dependencies pointing downward (Chapter 25, Chapter 28); separate compilation and linking are covered in Chapter 21 and version control in Chapter 24.

> **Status of the principle: Information hiding**
> *Empirical.* Parnas (1972) argued, and decades of software-engineering experience confirm, that decomposing a system so that each module hides one design decision likely to change makes change cheaper and errors more local; it is a design heuristic, not a law, and its benefit depends on predicting correctly which decisions will change.

20.8 Error handling

Errors are part of the specification

A sensor disconnects, a serial reply never arrives, a file is missing, a user types a negative flow rate. These are not bugs; they are inputs the program must handle. The design question is, for each failure, **who detects it, who decides what**

to do, and what the safe behaviour is. A dosing pump in a water-treatment plant that loses its flow sensor should stop dosing; a ventilator that loses a pressure reading should alarm and fall back to a conservative mode; a logging script that loses one sample may simply record a gap.

Status codes in C

C has no exceptions. Functions report failure through their return value, conventionally `0` for success and a negative or enumerated code for each failure, with results returned through pointer arguments:

```
 #include <stdint.h>
#include "fault.h"               /* fault_report(), FAULT_SUPPLY_ADC */
#include "supply.h"              /* supply_filter_update() */

typedef enum { ADC_OK = 0, ADC_TIMEOUT, ADC_RANGE } adc_status_t;

adc_status_t adc_read_mv(uint8_t channel, int32_t *out_mv);

void sample_supply(void)
{
    int32_t mv;
    adc_status_t st = adc_read_mv(3u, &mv);
    if (st != ADC_OK) {
        fault_report(FAULT_SUPPLY_ADC, (uint32_t)st);   /* log, count, maybe trip */
        return;
    }
    supply_filter_update(mv);
}
```

The weakness of status codes is that nothing forces the caller to check them; compiler attributes (`__attribute__((warn_unused_result))` in GCC and Clang, `[[nodiscard]]` in C23 and C++17) and static analysers catch ignored returns.

Exceptions in Python

Python signals errors by **raising exceptions**, objects that propagate up the call stack until a `try` / `except` block handles them; if none does, the program stops with a traceback. Exceptions separate the code that detects an error from the code that knows how to respond, and they cannot be silently ignored. Listing 20.8 reads a 4–20 mA pressure transmitter through a flaky serial link, retrying timeouts and treating an out-of-range loop current as a hard fault.

```
 import random

class SensorError(Exception):
    """Base class for failures of the pressure transmitter."""
class SensorTimeout(SensorError):
    pass
class SensorRangeError(SensorError):
    pass

def read_raw(rng):                       # stand-in for a serial query to a real device
    r = rng.random()
    if r < 0.5:
        raise TimeoutError("no reply within 50 ms")
    return 4.0 + 16.0 * rng.random() if r < 0.95 else 2.1   # 4-20 mA loop; 2.1 mA = broken wire

def read_pressure_bar(rng, retries=3):
    """Return pressure in bar from a 4-20 mA, 0-10 bar transmitter, or raise SensorError."""
    for _ in range(retries):
        try:
            ma = read_raw(rng)
        except TimeoutError as exc:
            last = exc                   # transient: try again
            continue
        if not 3.8 <= ma <= 20.5:        # NAMUR NE 43 signal limits
            raise SensorRangeError(f"loop current {ma:.1f} mA out of range")
        return (ma - 4.0) / 16.0 * 10.0
    raise SensorTimeout(f"{retries} attempts failed") from last

rng = random.Random(7)
for k in range(8):
    try:
        print(f"sample {k}: {read_pressure_bar(rng):5.2f} bar")
    except SensorRangeError as exc:
        print(f"sample {k}: FAULT ({exc}); valve to safe position")
    except SensorError as exc:
        print(f"sample {k}: no data ({exc}); hold last value")
```

Listing 20.8 — Exceptions, retries and fault classes (Python 3).

With this seed, samples 0–2 and 4–6 return 0.72, 3.66, 0.37, 1.24, 9.48 and 3.97 bar; sample 3 fails three attempts and is reported as missing; sample 7 reads 2.1 mA, below the 3.8 mA limit that NAMUR NE 43 sets for a valid signal, and is reported as a sensor fault. The hierarchy lets the caller hold the last value on a transient loss but drive the valve safe on a broken wire, and `raise ... from` keeps the original timeout for the log. Context managers (`with open(...) as f:`) close files and instrument sessions even when an exception occurs; C++ uses destructors (RAII, Chapter 21).

Assertions are not error handling

An **assertion** (`assert(n > 0)`) checks a condition that can be false only if the program itself is wrong. It must not be used for conditions that can legitimately occur, such as bad input, because assertions are often compiled out (`NDEBUG`, `python -O`). In deployed firmware a failed internal check typically logs the fault and lets the watchdog reset the system (Chapter 27).

Table 20.2 — Error-handling mechanisms.

Mechanism	Languages	Strength	Weakness
Status/return codes	C, C++, Rust (`Result`), Go	Explicit, zero overhead, deterministic	Easily ignored; clutters code
`errno` -style global	C standard library	Compact interface	Must be read immediately; not reentrant without care
Exceptions	Python, C++, Java	Cannot be ignored; separates detection from handling	Hidden control paths; often banned in hard real-time C++
Assertions	All	Catches programming errors early	Not for expected failures; often disabled in release
Sentinel values (NaN, −1)	All	Propagate through computations	Silently spread if never checked

---

20.9 Debugging

A method, not a mood

A **bug** is a difference between what the program does and what it should do. Debugging is the scientific method applied to code: observe the failure, form a hypothesis about its cause, design an experiment that could refute it, run the experiment, and repeat until the cause is found and the fix is verified. The first step is always to make the failure **reproducible**, ideally as an automated test (Section 20.10), because a failure that cannot be reproduced cannot be shown to be fixed.

Tools

- **Logging and print statements**: the simplest observation tool. Structured logs with timestamps and levels (Python's `logging` module; a ring-buffered event log in firmware) let a failure in the field be reconstructed afterwards. In real-time code, printing changes timing and can hide or create the bug, so firmware logs to RAM and transmits later.
- **Debuggers**: `gdb` , `pdb` , and the debuggers in IDEs let the engineer stop at a **breakpoint**, step through statements, inspect variables and the call stack, and stop when a variable changes (**watchpoint**). On a microcontroller the debugger connects through the processor's debug port (SWD or JTAG, with a probe such as a J-Link or ST-LINK), and the Cortex-M debug unit provides hardware breakpoints and data watchpoints without changing the code.
- **Sanitisers and checkers**: compilers instrument code to detect errors at run time: AddressSanitizer (out-of-bounds access, use after free), UndefinedBehaviorSanitizer (signed overflow, bad shifts), Valgrind on Linux. They run on the host, which is one strong reason to make firmware modules compile and run on a PC.
- **Static analysis**: tools that find bugs without running the code (compiler warnings at their highest level, `clang-tidy` , `cppcheck` , commercial analysers), and type checkers such as `mypy` for Python.
- **Bisection**: when a bug appeared between a working and a broken version, testing the version halfway between and repeating finds the change that introduced it. `git bisect` automates this.

A typical bug that sanitisers catch instantly and humans find slowly is the off-by-one write:

```
 float history[8];
for (int i = 0; i <= 8; i++) {      /* BUG: <= writes history[8], one past the end */
    history[i] = 0.0f;
}
```

On a microcontroller the ninth write silently overwrites whatever variable the linker placed after `history`; compiled with `-fsanitize=address` on a PC, the program stops at that line with a report naming the array and the offset.

Worked example — bisecting a regression. A drive's firmware passed its acceptance test at release 4.1 and fails at 4.2, 500 commits later, and each build-flash-test cycle takes 6 minutes. Testing commits one by one could take 500 cycles, 50 hours; bisection needs at most $\lceil \log_2 500 \rceil = 9$ cycles, 54 minutes, to find the commit that introduced the failure, provided every commit builds and can be tested.

Table 20.3 — Common bug classes and how they are usually found.

Bug class	Typical symptom	Usually found by
Off-by-one, out-of-bounds index	Corrupted neighbour variable, crash later	Sanitisers, bounds-checked tests, code review
Uninitialised variable	Behaviour changes between builds or power cycles	Compiler warnings, static analysis, Valgrind
Integer overflow, wrong type	Fails only for large values or long uptime (day 49.7)	Boundary-value tests, UBSan, review of types
Floating-point precision loss	Slow drift, frozen accumulator	Long-duration and large-value tests
Race condition	Rare, timing-dependent corruption	Code review, stress tests, thread sanitiser (Chapter 27)
Stack overflow, memory leak	Random crash after hours or days	Stack painting, heap statistics, endurance tests
Misunderstood requirement	Code does what was written, not what was needed	Reviews, acceptance tests with the customer

20.10 Testing

What testing can and cannot show

Testing runs the program on chosen inputs and compares the results with expected outputs. As the saturating-adder example of Chapter 19 showed for hardware, the input space of any useful function is far too large to test exhaustively, so tests sample it, and sampling can show the presence of bugs but not their absence. The art is to choose tests that are likely to reveal bugs, and to automate them so that they run on every change.

> **Status of the principle: Testing reveals the presence, not the absence, of faults**
> *Derived.* A finite set of test inputs drawn from an input space of 2^{32} or more values leaves almost all inputs untested, so a passing test suite bounds nothing in general; confidence comes from combining tests with structure (small pure functions whose input classes can be argued), coverage measurement, reviews and, for critical code, formal methods.

Levels of testing

Tests are organised in levels that mirror the design (Figure 20.5):

- **Unit tests** check one function or module in isolation, on a PC, in milliseconds. Hardware dependencies are replaced by **test doubles**: a fake ADC driver that returns scripted values, a mock that records the commands sent to it.
- **Integration tests** check that modules work together: the control loop with the real filter and PID modules, or the firmware with the real sensor driver on a development board.

- **System tests** check the complete product against its requirements, often on a **hardware-in-the-loop** rig that simulates the plant (Chapter 28).
- **Regression tests** are the accumulated tests of all levels, rerun automatically on every change so that a fixed bug stays fixed.

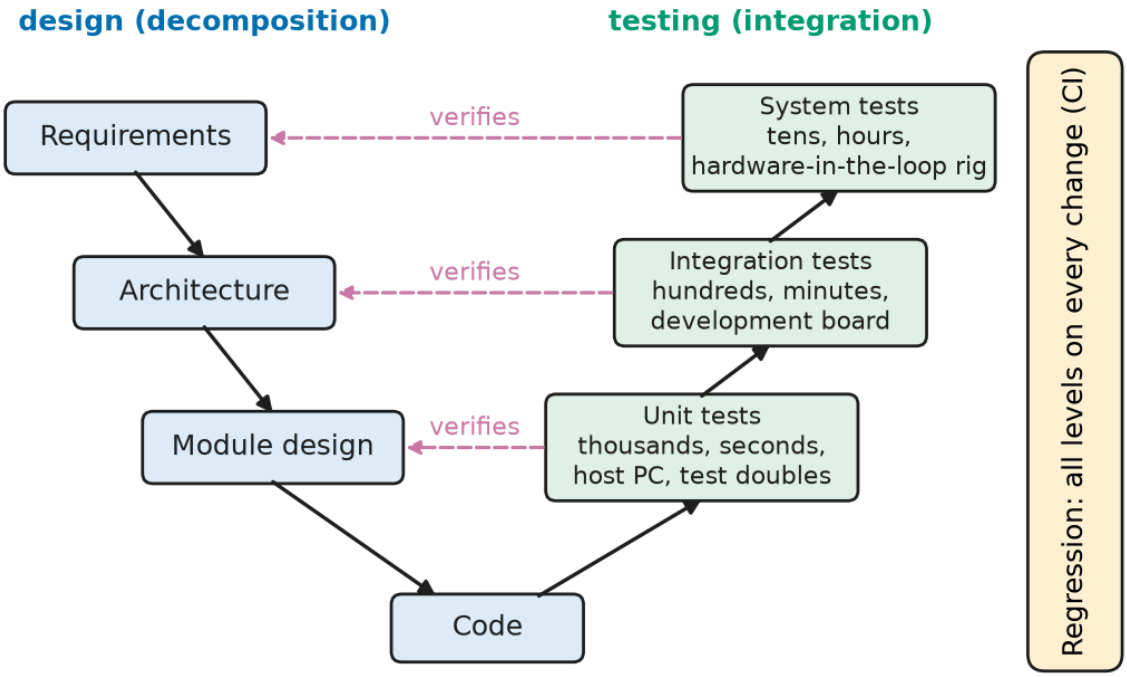


Figure 20.5 — The V-model: design levels matched by test levels.

Choosing test cases

Two classical techniques choose inputs systematically. **Equivalence partitioning** divides the input domain into classes that the program should treat alike (readings inside the table, readings above it, readings below it) and tests at least one value from each. **Boundary-value analysis** adds tests at and just beyond each boundary, because off-by-one and comparison errors cluster there: the first and last table entries, and one count outside each. Listing 20.9 applies both to `adc_to_temp` with the `pytest` framework, using the closed-form thermistor law as an independent reference.

```
 import math
import pytest
from ntc import adc_to_temp, COUNTS, TEMPS

def reference(x):                       # closed-form beta model, independent of the table
    r = 10e3 * x / (4095 - x)
    return 1 / (1 / 298.15 + math.log(r / 10e3) / 3950) - 273.15

@pytest.mark.parametrize("i", range(len(TEMPS)))
def test_table_points_exact(i):         # every calibration point maps back to its temperature
    assert adc_to_temp(COUNTS[i]) == pytest.approx(TEMPS[i])

@pytest.mark.parametrize("x", range(COUNTS[-1], COUNTS[0] + 1, 7))
def test_interpolation_error(x):        # whole range: within 0.5 deg C of the model
    assert abs(adc_to_temp(x) - reference(x)) < 0.5

@pytest.mark.parametrize("x", [COUNTS[0] + 1, COUNTS[-1] - 1, 0, 4095])
def test_out_of_range_rejected(x):      # boundary values just outside the table
    with pytest.raises(ValueError):
        adc_to_temp(x)

def test_monotonic():                   # hotter sensor -> lower count -> higher temperature
    temps = [adc_to_temp(x) for x in range(COUNTS[-1], COUNTS[0] + 1)]
    assert all(a >= b for a, b in zip(temps, temps[1:]))
```

Listing 20.9 — Unit tests with equivalence classes, boundary values and a reference model (Python 3, pytest; run with `python -m pytest`).

`pytest` generates 590 test cases from these functions (34 table points, 551 interpolation points, 4 out-of-range values, one monotonicity check) and runs them in under a second; all pass, and the worst interpolation error over the range is 0.16 °C. For C firmware the same tests are written with Unity, CppUTest or GoogleTest and compiled for the host, so they run in continuous integration without hardware.

Coverage and the safety standards

Code coverage measures which parts of the code the tests executed: **statement coverage** (every statement ran), **branch coverage** (every branch of every decision was taken both ways) and **modified condition/decision coverage** (MC/DC: every condition in a decision has been shown to affect the outcome independently). Coverage does not show that the code is right; it shows which code has never been tested at all, which is where to look next.

Safety standards set coverage requirements by criticality. DO-178C, for airborne software, requires statement coverage at Level C, adds decision coverage at Level B and MC/DC at Level A (software whose failure could be catastrophic). ISO 26262-6, for road vehicles, recommends statement, branch and MC/DC coverage increasingly strongly from ASIL A to ASIL D, and IEC 61508-3 does the same across safety integrity levels for industrial systems.

Worked example — MC/DC for an interlock. A motor-drive interlock trips when `overtemp || (overcurrent && !inhibit) || estop`. It has $N = 4$ conditions, so exhaustive testing of the decision needs $2^4 = 16$ combinations. MC/DC needs, for each condition, a pair of tests differing only in that condition and giving different outcomes; such pairs can share tests, and the minimum is $N + 1 = 5$ tests. One set is: all false (no trip); `overtemp` alone (trip); `estop` alone (trip); `overcurrent` alone (trip); `overcurrent` and `inhibit` (no trip). The first test pairs with each of the next three to show the independent effect of `overtemp`, `estop` and `overcurrent`, and the last pairs with the fourth to show the effect of `inhibit`. For a decision with 10 conditions the saving is 1024 tests against 11, which is why MC/DC, not exhaustive testing, is the aviation standard's criterion.

Testing as a habit

Tests are most valuable written early and run constantly: a test written before a bug fix proves it detects the bug, and running the suite on every commit lets a team change code confidently. Testing numerical software is treated in Chapter 24, firmware validation on target hardware in Chapter 28.

What remains open

Open question	Where it is answered
How are two's-complement, fixed-point and IEEE 754 numbers represented in bits?	Chapter 11 (Digital Number Systems)
What exactly do pointers, structs, `volatile`, the compiler and the linker do in C and C++?	Chapter 21 (C/C++ for Engineering)
How are arrays, data files, plots and instruments handled efficiently in Python?	Chapter 22 (Python for Engineering)
How is an algorithm's cost analysed, and when should a tree, hash table or graph be used?	Chapter 23 (Algorithms and Data Structures)
How are numerical results kept accurate, reproducible and under version control?	Chapter 24 (Scientific Computing); *The Mathematics Universe*, Ch. 22
How is firmware structured into drivers, abstraction layers and boot stages?	Chapter 25 (Embedded-System Architecture)
How do interrupts and tasks share data safely, and what do RTOS schedulers guarantee?	Chapter 27 (Real-Time Computing)
How is firmware debugged on target and validated with hardware-in-the-loop rigs?	Chapter 28 (Embedded System Design)

Next: Chapter 21 takes the language of firmware, C, and its successor C++, down to the level of pointers, memory layout, registers, objects, compilation and linking.

Chapter 21 — C/C++ for Engineering

Part V · Programming and Computational Systems

A motor-controller board for an electric forklift passes every bench test with its debug build. The production build, compiled with optimisation turned on, hangs within a second of power-up. The debugger shows the processor spinning forever in a loop that waits for a flag, `while (!adc_done) { }`, although the ADC's end-of-conversion interrupt fires and sets the flag on time. Nothing is wrong with the ADC, the interrupt or the loop's logic. The optimiser, seeing that nothing inside the loop changes `adc_done`, read it once into a register and never looked at memory again. One keyword, `volatile`, tells the compiler that the variable can change behind its back. The episode is typical of C and C++ in engineering: the languages give direct control of memory, bits and hardware, and in exchange they hold the programmer to a precise contract with the compiler, the processor and the linker. This chapter sets out that contract.

> **How do pointers, structures, memory allocation and bit operations map onto the memory and registers of a real machine?**
> **How do C++ classes and object-oriented design help build safe firmware, and what do compilation, linking and the embedded coding standards demand?**

Concept	Engineering destination
C fundamentals (types, promotions, undefined behaviour)	Portable, optimiser-safe arithmetic in firmware and drivers
Pointers	Buffers, DMA descriptors, callback tables, interrupt vectors
Structures	Telemetry records, protocol frames, register maps, configuration blocks
Memory management	Static allocation and pools in firmware; leak-free host tools
Bitwise operations	Register fields, flags, packed sensor data, CRCs, fixed-point maths
Peripheral-oriented programming	Memory-mapped registers, `volatile`, atomic set/reset, ISR-shared data
C++ classes and RAII	Drivers that cannot be misused; locks and interrupt guards that always release
Object-oriented design	Sensor and actuator interfaces, host testing with test doubles
Compilation and linking	Linker scripts, start-up code, flash and RAM budgets, mixed C/C++ builds
Embedded-oriented C/C++	MISRA C/C++, AUTOSAR C++14, CERT C; evidence for IEC 61508, ISO 26262

Roadmap. The C model of types and arithmetic (21.1) → pointers (21.2) and structures (21.3) → memory management (21.4) → bitwise operations (21.5) → talking to peripherals (21.6) → C++ classes and RAII (21.7) → object-oriented design (21.8) → how source becomes a flash image (21.9) → the restricted subsets used in embedded and safety-critical work (21.10).

> **Boundary.**
> Language-neutral fundamentals (types, control flow, functions, the stack and heap, testing) are in Chapter 20; the processor, memory map and peripherals at architecture level in Chapter 17 (Sections 17.3, 17.5 and 17.7). Firmware architecture, drivers, interfaces and real-time behaviour belong to Chapters 25–28, so peripheral examples here are kept short.

21.1 C fundamentals

The C model

C, standardised as ISO/IEC 9899 (C99, C11, C17, C23), is a compiled language whose types map almost one-to-one onto machine words and whose operations map onto a handful of instructions. Each `.c` file is a **translation unit**, preprocessed (macros, `#include`, conditional compilation), compiled to an object file and linked with others (Section 21.9). There is no run-time system checking array bounds, null pointers or arithmetic overflow; that is what makes C fast and predictable on an 8 KB microcontroller, and what makes its errors silent.

Engineering code uses the fixed-width types of `<stdint.h>` (`uint8_t`, `int32_t`), `<stdbool.h>` for `bool`, `const` for data that must not change (and that the linker can place in flash), `enum` for named constants, and `static inline` functions in preference to function-like macros, which evaluate their arguments as text and so can evaluate them twice.

Integer promotions and the usual arithmetic conversions

C never does arithmetic on types narrower than `int`. Operands of type `char`, `short`, `uint8_t` or `uint16_t` are first **promoted** to `int` (on a 32-bit target), and when two operands still differ, the **usual arithmetic conversions** bring them to a common type, which is unsigned if the unsigned operand's rank is at least that of the signed one. Listing 21.1 shows three consequences that regularly reach production firmware.

```
 #include <stdint.h>
#include <stdio.h>

int main(void)
{
    uint8_t  a = 200, b = 100;
    uint8_t  sum8 = a + b;              /* int 300, converted back to uint8_t: 44 */
    int      wide = a + b;              /* promoted to int: 300 */

    uint16_t x = 50000, y = 50000;
    uint32_t bad  = x * y;              /* int * int = 2.5e9 > INT_MAX: undefined */
    uint32_t good = (uint32_t)x * y;    /* unsigned arithmetic: 2 500 000 000 */

    int32_t  neg = -1;
    uint32_t one = 1u;
    if (neg < one) { puts("expected"); } else { puts("surprise: -1 > 1u"); }

    printf("%u %d %lu\n", (unsigned)sum8, wide, (unsigned long)good);
    (void)bad;
    return 0;
}
```

Listing 21.1 — Integer promotion pitfalls (C11, 32-bit `int`).

The first case wraps silently: $200 + 100 = 300$ is computed in `int`, and storing it in a `uint8_t` keeps $300 \bmod 256 = 44$. The second is worse: two `uint16_t` values promote to *signed* `int`, and $50\,000^2 = 2.5 \times 10^9$ exceeds $2^{31} - 1 = 2\,147\,483\,647$, which is signed overflow and therefore undefined; on a 16-bit AVR, where `int` is 16 bits, the same line is well-defined unsigned arithmetic that wraps. The third prints "surprise": comparing `int32_t` with `uint32_t` converts -1 to $4\,294\,967\,295$. Compiling with `-Wall -Wextra -Wconversion -Wsign-compare` flags all three.

> **Status of the principle: Integer promotions and usual arithmetic conversions**
> *Derived*. They follow from the conversion rules of ISO C (§6.3.1), so the result type of every expression is decidable at compile time; the outcome depends on the target's width of `int`, which is why fixed-width types and explicit casts are needed for code that must behave identically on 16-bit and 32-bit processors.

Undefined, unspecified and implementation-defined behaviour

The C standard deliberately leaves some behaviour open. **Implementation-defined** behaviour must be documented by the compiler (the width of `int`, right-shifting a negative number). **Unspecified** behaviour may differ between evaluations (the order in which function arguments are evaluated). **Undefined behaviour** (UB) places no requirement at all: the compiler may assume it never happens and optimise on that assumption, so a signed-overflow check written as `if (x + 1 < x)` can be deleted entirely.

Table 21.1 — Common undefined and implementation-defined behaviour in C.

Construct	Category	Typical consequence
Signed integer overflow	Undefined	Overflow checks removed by the optimiser
Shift by ≥ width, or left-shift of a negative value	Undefined	Different results at different optimisation levels
Out-of-bounds array access, null dereference	Undefined	Silent corruption, hard fault on Cortex-M
Reading an uninitialised automatic variable	Undefined (in practice indeterminate)	Behaviour changes between builds
Accessing an object through an incompatible pointer type	Undefined (strict aliasing)	Stale values after optimisation
Right shift of a negative signed value	Implementation-defined	Arithmetic shift on GCC/Clang/Arm compilers
Width of `int`, signedness of `char`	Implementation-defined	16-bit `int` on AVR and MSP430; `char` unsigned on Arm

> **Status of the principle: Undefined behaviour**
> *Fundamental*. By definition of the language: the standard imposes no requirement on a program that executes undefined behaviour, so no test can prove such a program correct, because a different compiler version or optimisation level may change its behaviour; the defence is to make UB impossible by construction, with sanitisers (Chapter 20) and static analysis as the check.

21.2 Pointers

Addresses, dereference and arithmetic

A **pointer** holds the address of an object and has the type of what it points to. `&x` takes an address, `*p` reaches the object, and `p + k` points *k elements* further on, advancing by *k* times the element size. An array name used in an expression **decays** to a pointer to its first element, so `buf[i]` is defined as `*(buf + i)`. Forming a pointer one past the end of an array is legal (it marks the end of a loop); dereferencing it, or forming any pointer beyond it, is undefined.

```
 #include <stddef.h>
#include <stdint.h>

int32_t sum_window(const int16_t *p, size_t n)
{
    int32_t acc = 0;
    const int16_t *end = p + n;          /* one past the last element: legal */
    while (p < end) {
        acc += *p++;                     /* read, then advance by sizeof(int16_t) */
    }
    return acc;
}

typedef void (*cmd_handler_t)(const uint8_t *args, size_t len);
void cmd_reset(const uint8_t *args, size_t len);
void cmd_set_speed(const uint8_t *args, size_t len);

static const cmd_handler_t handlers[] = { cmd_reset, cmd_set_speed };

void dispatch(uint8_t opcode, const uint8_t *args, size_t len)
{
    if (opcode < sizeof handlers / sizeof handlers[0]) {
        handlers[opcode](args, len);     /* indirect call through a table in flash */
    }
}
```

Listing 21.2 — Pointer arithmetic and a table of function pointers (C11).

Worked example — pointer arithmetic on a sample buffer. A buffer `int16_t buf[64]` is placed by the linker at address 0x2000 0100. The call `sum_window(&buf[10], 8)` passes $p = \texttt{0x20000100} + 10 \times 2 = \texttt{0x20000114}$ and computes `end` $= p + 8 \times 2 = \texttt{0x20000124}$; the loop reads the eight 16-bit samples at 0x2000 0114 to 0x2000 0122. Had the same buffer been passed as `const uint8_t *`, `p + 8` would advance only 8 bytes. The compiler scales arithmetic by the pointed-to type, which is why pointer types must match the data, and why casting a `uint8_t *` from a byte stream to `uint32_t *` is dangerous: the address may be misaligned (a fault on Cortex-M0 and on some Cortex-M3/M4 instructions) and it breaks the strict-aliasing rule. Copying with `memcpy` into a properly typed variable is the portable way, and compilers turn it into a single load when alignment allows.

Figure 21.1 shows these relationships. `const` placement matters: `const int16_t *p` is a pointer to read-only data (the function promises not to write the samples), while `int16_t *const p` is a pointer that cannot itself be moved. The `restrict` qualifier promises that a pointer is the only way the function reaches its object, letting the compiler vectorise loops such as filters without fear of overlap.

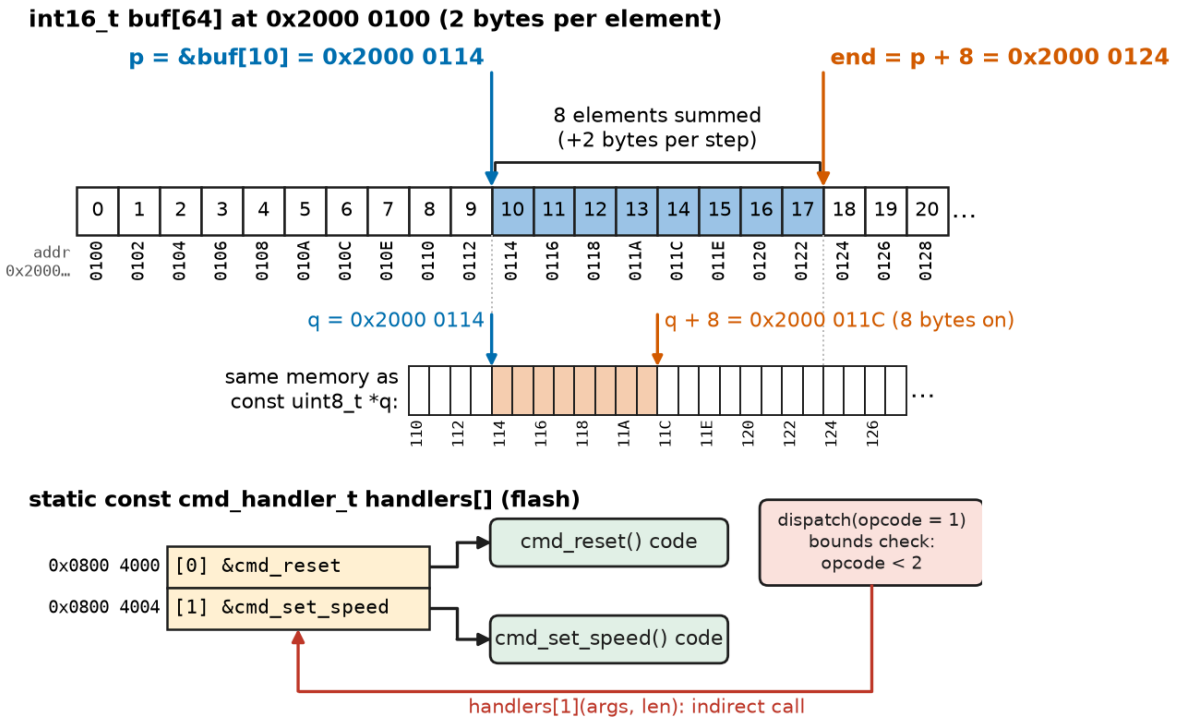


Figure 21.1 — Pointers into a sample buffer: addresses, element scaling and a function-pointer table.

Function pointers

A **function pointer** stores the address of code. Tables of them implement command dispatchers (Listing 21.2), state machines and driver operations; the Cortex-M interrupt vector table is itself an array of function pointers at the start of flash (Section 17.5). Declaring the table `static const` places it in flash, where a stray write cannot redirect it, and the bounds check before the indirect call is essential: an unchecked opcode from a serial port would otherwise jump to an arbitrary address.

> **Status of the principle: Pointer arithmetic is scaled by the element type**
> *Derived.* From the definition of array subscripting in ISO C (`E1[E2]` is `*((E1)+(E2))`) and of pointer addition, which advances by multiples of the pointed-to object's size; the guarantee holds only within one array object (plus one past its end), which is why arithmetic that crosses between separate objects is undefined even when the addresses happen to be adjacent.

21.3 Structures

Layout, alignment and padding

A `struct` lays its members out in declaration order, but each member must sit at an address that is a multiple of its **alignment** (normally its size for scalar types: 4 for `uint32_t` and `float`, 8 for `double` under both the Arm AAPCS and the x86-64 System V ABI). The compiler inserts **padding** to satisfy this, and pads the end so that arrays of the struct keep every element aligned. Field order therefore changes the size.

```
#include <assert.h>                     /* static_assert (C11) */
#include <stdint.h>

typedef struct {                        /* fields in the order they were thought of */
    uint8_t  valid;
    uint32_t timestamp;
    uint8_t  channel;
    float    voltage;
    uint16_t flags;
    double   energy;
} rec_bad_t;                            /* 32 bytes on Cortex-M (AAPCS) and x86-64 */

typedef struct {                        /* largest alignment first */
    double   energy;
    uint32_t timestamp;
    float    voltage;
    uint16_t flags;
    uint8_t  valid;
    uint8_t  channel;
} rec_good_t;                           /* 24 bytes: 20 of data + 4 of tail padding */

static_assert(sizeof(rec_good_t) == 24, "unexpected padding in rec_good_t");
```

Listing 21.3 — The same telemetry record in two field orders (C11).

Python's `ctypes` module lays out structures by the platform's C rules, so it can check such layouts on a host without a cross-compiler:

```
import ctypes as C

class Bad(C.Structure):          # field order as first written
    _fields_ = [("valid", C.c_uint8), ("timestamp", C.c_uint32),
                ("channel", C.c_uint8), ("voltage", C.c_float),
                ("flags", C.c_uint16), ("energy", C.c_double)]

class Good(C.Structure):         # largest alignment first
    _fields_ = [("energy", C.c_double), ("timestamp", C.c_uint32),
                ("voltage", C.c_float), ("flags", C.c_uint16),
                ("valid", C.c_uint8), ("channel", C.c_uint8)]

for s in (Bad, Good):
    offs = {name: getattr(s, name).offset for name, _ in s._fields_}
    print(s.__name__, C.sizeof(s), C.alignment(s), offs)
print("1000 records:", 1000 * C.sizeof(Bad), "vs", 1000 * C.sizeof(Good), "bytes")
```

Listing 21.4 — Checking struct layout with `ctypes` (Python 3).

Worked example — the cost of field order. The script prints for `Bad` a size of 32 bytes with offsets valid 0, timestamp 4, channel 8, voltage 12, flags 16, energy 24, and for `Good` 24 bytes with energy 0, timestamp 8, voltage 12, flags 16, valid 18, channel 19. The record holds $1 + 4 + 1 + 4 + 2 + 8 = 20$ bytes of data, so the first order wastes 12 bytes (37.5 % of the struct) and the second 4. A data logger on a battery-storage system that keeps 1000 records in RAM uses 32 000 bytes in the first layout and 24 000 in the second, a saving of 8 KB that on a 64 KB microcontroller decides whether the logger fits (Figure 21.2). The `static_assert` in Listing 21.3 turns an accidental layout change into a compile error.

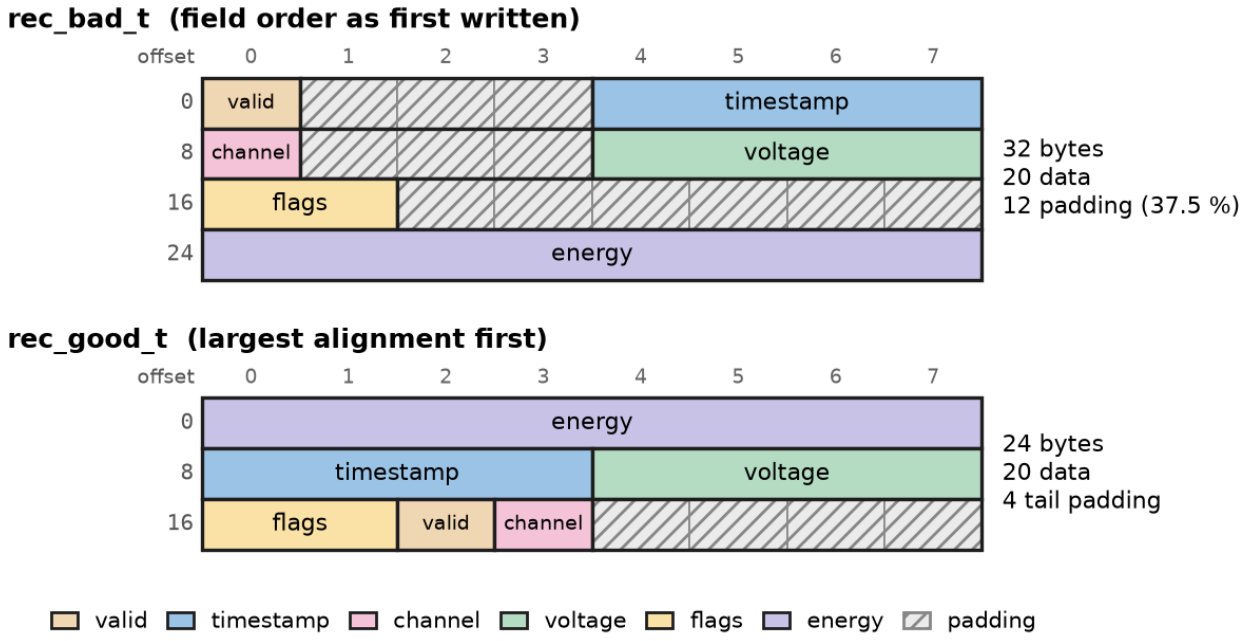


Figure 21.2 — Byte layout of the same telemetry record in two field orders, showing alignment padding.

Packed structs, unions, bit-fields and byte order

Compilers offer `__attribute__((packed))` (GCC, Clang) or `#pragma pack` to remove padding, typically to match a wire format. Packed members may be misaligned, so every access becomes several byte loads on cores without unaligned access, and taking a member's address yields a pointer the compiler cannot use safely. **Unions** overlay members on the same storage; reading a member other than the one last written is allowed in C (it reinterprets the bytes) but not in C++. **Bit-fields** (`unsigned mode : 2;`) look ideal for register and protocol fields, but their order within a word, their padding and whether a plain `int` bit-field is signed are implementation-defined, so portable code uses masks and shifts (Section 21.5).

Multi-byte values also depend on **byte order**. Arm Cortex-M and x86 are little-endian (least significant byte at the lowest address); network protocols, CAN signals in Motorola format and many sensors are big-endian. Reading a 16-bit big-endian value from a received byte array is written explicitly, `(uint16_t)((rx[0] << 8) | rx[1])`, which is correct on any host. Overlaying a struct directly on a received frame is fast but non-portable: it depends on padding, packing, alignment and byte order at once.

21.4 Memory management

Storage duration

Every C object has a **storage duration**. **Static** objects (globals and `static` locals) exist for the whole run and are zero- or value-initialised before `main`. **Automatic** objects (locals) live on the stack until their block ends. **Allocated** objects come from `malloc`, `calloc` or `realloc` and live until `free`. C++ adds `new` and `delete`, which call constructors and destructors. Chapter 20 described where each lives in memory; the C programmer's task is to make the lifetime of every pointer no longer than the lifetime of the object it points to.

The allocation bugs

Manual allocation brings a family of bugs that no compiler catches:

- **Leak**: an allocated block whose last pointer is lost. A host tool that leaks 100 bytes per measurement cycle runs for days; firmware that does the same fails after a predictable number of cycles.
- **Use after free** and **double free**: the block has been returned and perhaps reused, so writes corrupt another object or the allocator's own data.
- **Buffer overflow and over-read**: writing or reading past the end of a block. The Heartbleed vulnerability in OpenSSL (CVE-2014-0160) was an over-read: a length field taken from the network was trusted, and up to 64 KB of server memory, including private keys, was returned to the requester.
- **Unchecked failure**: `malloc` returns `NULL` when memory is exhausted, and code that does not check dereferences it.

On a host, AddressSanitizer and Valgrind find most of these. On a microcontroller the additional problems are fragmentation and non-deterministic allocation time (Chapter 20), so the usual rule, codified in MISRA C (Section 21.10), is no dynamic allocation after initialisation.

> **Status of the principle: Static allocation for embedded systems**
> *Empirical.* Field experience in safety-critical and long-running systems shows that heap fragmentation and leak-induced failures appear after hours to months and are rarely reproduced in testing, while statically sized memory makes the worst case visible at link time; the rule trades flexibility for analysability and is relaxed in systems with an MMU, an operating system and fault isolation.

Fixed-block memory pools

When a design does need run-time allocation, for example message buffers that are produced by one task and freed by another, a **fixed-block pool** gives constant-time allocation with no fragmentation. All blocks have the same size, and free blocks are chained through their own storage.

```
 #include <stddef.h>
#include <stdint.h>

#define POOL_BLOCKS 32u
#define BLOCK_SIZE  64u                       /* bytes: one CAN FD payload */

typedef union block {
    union block *next;                        /* used while the block is free */
    uint8_t      data[BLOCK_SIZE];            /* used while it is allocated */
} block_t;

static block_t  pool[POOL_BLOCKS];
static block_t *free_list;

void pool_init(void)
{
    for (size_t i = 0; i < POOL_BLOCKS - 1u; i++) {
        pool[i].next = &pool[i + 1u];
    }
    pool[POOL_BLOCKS - 1u].next = NULL;
    free_list = &pool[0];
}

void *pool_alloc(void)                        /* O(1); NULL when exhausted */
{
    block_t *b = free_list;
    if (b != NULL) {
        free_list = b->next;
    }
    return b;
}

void pool_free(void *p)                       /* O(1); p must come from pool_alloc */
{
    block_t *b = p;
    b->next = free_list;
    free_list = b;
}
```

Listing 21.5 — A fixed-block memory pool (C11).

The pool's 2048 bytes are reserved at link time, so its memory cost appears in the build's RAM budget (Section 21.9). If the pool is shared between an interrupt and a task, `pool_alloc` and `pool_free` must run inside a critical section (Section 21.7, Chapter 27). RTOS kernels provide the same structure as memory partitions or block pools.

21.5 Bitwise operations

Operators and idioms

C's bitwise operators `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` and `>>` act on each bit of an integer and are the everyday tools of register programming, flags, packed data and checksums. Four idioms cover most uses, with `mask` a constant having ones in the bits concerned:

Table 21.2 — Bit-manipulation idioms.

Operation	Expression	Example (bit 5)
Set bits	`reg = reg | mask;`	`flags |= 1u << 5;`
Clear bits	`reg = reg & ~mask;`	`flags &= ~(1u << 5);`
Toggle bits	`reg = reg ^ mask;`	`flags ^= 1u << 5;`
Test a bit	`(reg & mask) != 0u`	`if (flags & (1u << 5))`

Shifts need care: shifting by the type's width or more is undefined, left-shifting a negative value is undefined, and `1 << 31` overflows a signed `int`. Writing the constant as unsigned (`1u << 31`, `1UL << n`) avoids the last two.

Fields within a word

Peripheral registers and protocol words pack several fields into one integer. Listing 21.6 defines field helpers and two functions used with real hardware: extracting a 12-bit ADC result from the two bytes an SPI converter sends, and multiplying two Q15 fixed-point numbers (the Q format itself is defined in Chapter 11, Section 11.6).

```
#include <stdint.h>

#define FIELD_MASK(w, pos)          (((1UL << (w)) - 1UL) << (pos))
#define FIELD_GET(reg, w, pos)      (((reg) >> (pos)) & ((1UL << (w)) - 1UL))
#define FIELD_SET(reg, w, pos, val) \
    (((reg) & ~FIELD_MASK(w, pos)) | (((uint32_t)(val) << (pos)) & FIELD_MASK(w, pos)))
/* valid for field widths 1..31 */

static inline uint16_t adc_from_spi(uint8_t b0, uint8_t b1)
{
    return (uint16_t)(((uint16_t)(b0 & 0x0Fu) << 8) | b1);   /* 4 junk bits, 12 data */
}

static inline int16_t q15_mul(int16_t a, int16_t b)
{
    int32_t p = (int32_t)a * b;          /* Q30 product, exact in 32 bits */
    return (int16_t)(p >> 15);           /* back to Q15; arithmetic shift assumed */
}
```

Listing 21.6 — Field helpers, ADC byte unpacking and Q15 multiplication (C11).

Worked example — unpacking and scaling. A 12-bit SPI ADC on a 3.3 V reference returns the bytes 0x0A and 0xBC, the first carrying four leading don't-care bits. `adc_from_spi` masks the high byte to 0x0A & 0x0F = 0x0A, shifts it to 0x0A00 and ORs in 0xBC: 0x0ABC = 2748 counts, or $2748/4096 \times 3.3 = 2.214$ V. In a motor controller's Q15 datapath, a gain of 0.75 (0x6000 = 24 576) times a current of −0.5 (0xC000 = −16 384) gives the exact Q30 product −402 653 184; shifting right by 15 gives −12 288, which is $-12\,288/32\,768 = -0.375$ in Q15, as expected. The one input pair that fails is $(-1) \times (-1)$: +1 is not representable in Q15, and the result wraps to −1 unless saturated, which is why DSP instruction sets (the Cortex-M4 `QADD` and `SSAT` instructions) provide saturating arithmetic. Setting the 2-bit mode field of GPIO pin 12 to "output" (01) is `FIELD_SET(moder, 2, 24, 1u)`, which clears bits 25:24 and sets bit 24.

Other common bit tasks are counting set bits (`__builtin_popcount`, a single instruction on many cores), finding the highest set bit (`__builtin_clz`, the Cortex-M `CLZ` instruction, used for priority schedulers), byte reversal for endianness (`__builtin_bswap32`, `REV`) and cyclic redundancy checks for frames, computed bitwise, by table lookup or by a hardware CRC unit.

21.6 Peripheral-oriented programming

Memory-mapped registers in C

On a microcontroller, peripherals are controlled through **registers** at fixed addresses in the memory map (Section 17.3). Writing to such an address configures a timer or starts a conversion; reading may return a status that changes by itself, or clear a flag as a side effect. Section 17.7 showed the standard C idiom on a GPIO port: a struct whose members mirror the register block, overlaid on the block's base address. This section looks at the language mechanics behind that idiom, using an illustrative ADC block (Figure 21.3).

```
 #include <assert.h>
#include <stddef.h>
#include <stdint.h>

typedef struct {                          /* illustrative ADC register block */
    volatile uint32_t       CR;           /* 0x00 control: START bit 1, CH bits 12:8 */
    volatile uint32_t       SR;           /* 0x04 status: EOC bit 0, OVR bit 1; write 1 to clear */
    const volatile uint32_t DR;           /* 0x08 result, read only */
    uint32_t                RESERVED0;    /* 0x0C gap in the map */
    volatile uint32_t       SMPR;         /* 0x10 sample time */
} adc_regs_t;
static_assert(offsetof(adc_regs_t, SMPR) == 0x10u, "ADC register map mismatch");

#define ADC1            ((adc_regs_t *)0x40012000UL)   /* base address: illustrative */
#define ADC_SR_EOC      (1UL << 0)
#define ADC_SR_OVR      (1UL << 1)
#define ADC_CR_START    (1UL << 1)
#define ADC_CR_CH_MASK  (0x1FUL << 8)

uint16_t adc_convert(uint32_t ch)
{
    ADC1->CR = (ADC1->CR & ~ADC_CR_CH_MASK) | ((ch << 8) & ADC_CR_CH_MASK);
    ADC1->CR |= ADC_CR_START;
    while ((ADC1->SR & ADC_SR_EOC) == 0u) { }      /* re-read on every pass */
    uint16_t v = (uint16_t)(ADC1->DR & 0x0FFFu);
    ADC1->SR = ADC_SR_EOC;                         /* clear EOC only */
    return v;
}
```

Listing 21.7 — A register-map struct with layout check, read-only and write-1-to-clear registers (C11; register names, layout and address illustrative).

Four details carry the weight. Every member is `volatile`, so each access in the source becomes exactly one bus access. The result register is `const volatile`: it changes on its own, but the program may not write it, and the compiler rejects an accidental `ADC1->DR = 0;`. The reserved word keeps later members at the right offsets, and the `static_assert` on `offsetof` turns a missing or mistyped member into a compile error instead of a write to the wrong register; vendor headers avoid the problem by being generated from the silicon description (CMSIS-SVD files). Finally, the status register uses **write-1-to-clear** (W1C) semantics, common for interrupt and error flags.

Worked example — a write-1-to-clear trap and the cost of polling. Suppose a conversion finishes while an earlier overrun is still flagged, so `SR` reads 0x3 (EOC and OVR set). The tempting `ADC1->SR |= ADC_SR_EOC;` reads 0x3, ORs in 0x1 and writes 0x3 back, which clears *both* flags: the overrun, which the fault handler should have seen, is silently lost. Writing the mask alone, `ADC1->SR = ADC_SR_EOC;` (0x1), clears only EOC, because zeros written to a W1C register have no effect. The polling loop itself costs time: a conversion of 15 sampling cycles plus 12 conversion cycles at a 36 MHz ADC clock takes $27/36\ \text{MHz} = 0.75\ \mu\text{s}$, which a 168 MHz core spends as $0.75\ \mu\text{s} \times 168\ \text{MHz} = 126$ cycles of busy-

waiting. For one reading that is acceptable; for a 20 kHz control loop the conversion is triggered by a timer and collected by interrupt or DMA instead (Chapters 25–27), and a production driver also bounds the wait with a timeout (Section 20.2).

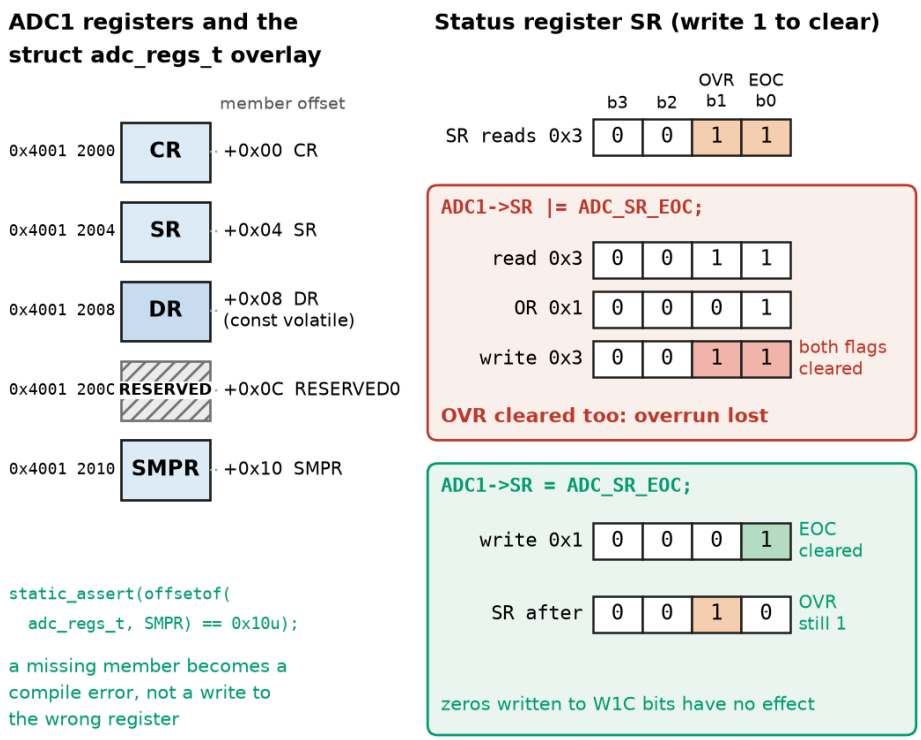


Figure 21.3 — A memory-mapped register block overlaid by a C struct, and the write-1-to-clear trap.

volatile

The qualifier tells the compiler that each access is an observable event: it must perform every read and write, in program order relative to other volatile accesses, and may not keep the value in a register or merge two writes. Without it, the optimiser deletes the second of two writes to the same register and hoists a status read out of a polling loop. Variables shared with interrupt handlers need it for the same reason, which is the failure of the opening problem:

```
 #include <stdbool.h>
#include <stdint.h>
#include "adc_regs.h"                   /* adc_regs_t, ADC1, ADC_SR_EOC: Listing 21.7 */

static volatile uint16_t adc_result;
static volatile bool adc_done;           /* written by the ISR, read by main */

void ADC_IRQHandler(void)
{
    adc_result = (uint16_t)(ADC1->DR & 0x0FFFu);
    ADC1->SR = ADC_SR_EOC;
    adc_done = true;
}

uint16_t adc_wait(void)
{
    while (!adc_done) { }                /* re-reads memory on every iteration */
    adc_done = false;
    return adc_result;
}
```

> **Status of the principle: What volatile guarantees**
> *Approximate.* `volatile` guarantees that the compiler performs each access, in order with respect to other volatile accesses; it does not make a read-modify-write atomic, does not order non-volatile accesses around it, and does not stop the processor or a bus from reordering or buffering accesses, so it is sufficient for registers and simple flags on a single-core microcontroller but not for synchronisation between cores, for which C11 atomics and memory barriers are required.

Read-modify-write and tearing

Any `|=` or `&=` on a register or shared variable is a load, an operation and a store, and an interrupt between load and store can lose an update. Section 17.7 showed the hardware cures for GPIO pins (set/reset registers, bit-banding); W1C registers are another, and where none exists the sequence runs with interrupts briefly disabled (Section 21.7). Variables wider than the bus **tear**: a 64-bit timestamp read in two 32-bit halves on a Cortex-M can combine the low half of one value with the high half of the next. Interrupt-safe design and RTOS synchronisation belong to Chapter 27; complete drivers built on these registers to Chapter 25 and Chapter 28; the bus protocols to Chapter 26.

21.7 C++ classes

Why C++ in engineering

C++ (ISO/IEC 14882; C++17 and C++20 are the common targets in embedded work) is close to a superset of C that adds **classes**, **templates**, **references**, **namespaces**, `constexpr` evaluation at compile time and a large standard library. Its design rule is the **zero-overhead principle**: a feature you do not use costs nothing, and a feature you use costs no more than hand-written C doing the same job. Used with discipline, C++ lets firmware make errors impossible to write instead of merely documenting them: a driver class whose constructor configures the peripheral cannot be used unconfigured, and a strongly typed `Millivolts` value cannot be passed where `Milliamps` is expected.

A **class** groups data (members) with the functions that operate on them (member functions) and controls access: `private` members are reachable only from the class's own functions, so its **invariants** (a ring buffer's fill level never exceeds its capacity) can be guaranteed by a few functions instead of the whole program. **Constructors** establish the invariant when an object is created, and the **destructor** runs automatically when it goes out of scope. `const` member

functions promise not to modify the object, and **references** (`T&`) give pass-by-reference without the null and arithmetic hazards of pointers.

Templates

A **template** is code parameterised by types or constants, instantiated by the compiler for each combination used. Listing 21.8 turns the C ring buffer of Chapter 20 into a type-safe class template whose capacity is checked at compile time.

```
 #include <array>
#include <cstddef>
#include <optional>

template <typename T, std::size_t N>
class RingBuffer {
    static_assert(N > 0 && (N & (N - 1)) == 0, "N must be a power of two");
public:
    bool push(const T& v) noexcept {
        if (head_ - tail_ == N) return false;          // full
        buf_[head_ & (N - 1)] = v;
        ++head_;
        return true;
    }
    std::optional<T> pop() noexcept {
        if (head_ == tail_) return std::nullopt;       // empty
        T v = buf_[tail_ & (N - 1)];
        ++tail_;
        return v;
    }
    std::size_t size() const noexcept { return head_ - tail_; }
private:
    std::array<T, N> buf_{};
    std::size_t head_ = 0;
    std::size_t tail_ = 0;
};

RingBuffer<float, 256> vibration_samples;              // storage is static: no heap
```

Listing 21.8 — A fixed-capacity ring buffer as a class template (C++17).

`RingBuffer<float, 256>` and `RingBuffer<CanFrame, 32>` are separate classes generated from one source, each as efficient as a hand-written C version; `std::optional` makes "no value" explicit instead of relying on a separate status return. Used between an interrupt and the main loop, the indices would be `std::atomic<std::size_t>`.

RAII

Resource acquisition is initialisation (RAII) ties a resource to an object's lifetime: the constructor acquires it, the destructor releases it, and because C++ runs destructors on every exit from a scope (normal return, early `return`, `break`, exception), release cannot be forgotten. Files (`std::fstream`), memory (`std::unique_ptr`) and locks (`std::lock_guard`) all work this way. In firmware the classic use is the interrupt-disabling critical section:

```
#include <cstdint>

extern "C" std::uint32_t irq_save();          // saves PRIMASK, then disables IRQs
extern "C" void irq_restore(std::uint32_t);   // restores the saved PRIMASK

class CriticalSection {
public:
    CriticalSection() noexcept : saved_(irq_save()) {}
    ~CriticalSection() { irq_restore(saved_); }
    CriticalSection(const CriticalSection&) = delete;            // not copyable
    CriticalSection& operator=(const CriticalSection&) = delete;
private:
    std::uint32_t saved_;
};

struct MotorCmd { float torque_nm; float speed_limit_rpm; };
MotorCmd g_cmd;                               // read by the 20 kHz control ISR

void set_command(const MotorCmd& c)
{
    CriticalSection cs;                       // interrupts off from here ...
    g_cmd = c;                                // both fields updated together
}                                             // ... restored here, on every path
```

Listing 21.9 — An RAII critical-section guard (C++17).

Because the guard restores the *saved* interrupt state rather than unconditionally enabling interrupts, guards nest correctly. The equivalent C code needs a matching `irq_restore` before every `return`, and a maintenance change that adds an early return can leave interrupts disabled for good.

> **Status of the principle: The zero-overhead principle**
> *Approximate.* With optimisation enabled, classes, inline member functions, references, templates, `constexpr` and RAII compile to code as small and fast as equivalent C, which can be checked in the generated assembly; the principle does not extend to exceptions and RTTI (tables and non-deterministic timing), to virtual calls (an indirection, Section 21.8), to heap-allocating library containers, or to template instantiation across many types, which costs flash.

21.8 Object-oriented design

Encapsulation, inheritance and polymorphism

Object-oriented design organises a program around objects that combine state and behaviour behind interfaces. Its three mechanisms are **encapsulation** (hiding state behind member functions, Section 21.7), **inheritance** (a derived class extends a base class) and **polymorphism** (code written against a base-class interface works with any derived class). In engineering the most valuable use is the **interface**: an abstract class that names what a component does, with each hardware variant and each test double implementing it.

```
 #include <cstdint>
#include <optional>

class TemperatureSensor {                          // interface: what, not how
public:
    virtual ~TemperatureSensor() = default;
    virtual std::optional<float> read_celsius() = 0;
};

class Ntc final : public TemperatureSensor {       // thermistor on an ADC channel
public:
    explicit Ntc(std::uint8_t adc_channel) : ch_(adc_channel) {}
    std::optional<float> read_celsius() override;  // ADC + table lookup (Ch. 20)
private:
    std::uint8_t ch_;
};

class Tmp117 final : public TemperatureSensor {    // digital sensor on I2C
public:
    explicit Tmp117(std::uint8_t i2c_addr) : addr_(i2c_addr) {}
    std::optional<float> read_celsius() override;  // 7.8125 mdegC per LSB
private:
    std::uint8_t addr_;
};

class FakeSensor final : public TemperatureSensor { // test double for host tests
public:
    std::optional<float> next;
    std::optional<float> read_celsius() override { return next; }
};

class OvertempMonitor {                            // depends only on the interface
public:
    OvertempMonitor(TemperatureSensor& s, float limit_c) : s_(s), limit_(limit_c) {}
    bool trip() {                                  // fail safe: no reading -> trip
        const auto t = s_.read_celsius();
        return !t || *t > limit_;
    }
private:
    TemperatureSensor& s_;
    float limit_;
};
```

Listing 21.10 — An abstract sensor interface, two implementations and a test double (C++17).

`OvertempMonitor` neither knows nor cares which sensor it has. On the product it receives an `Ntc` or a `Tmp117`, chosen by board variant; in a host unit test it receives a `FakeSensor` whose `next` value the test sets to 84.9, 85.1 or "no reading" to check every branch of `trip()` without hardware. This **dependency injection** (passing the dependency in, instead of letting the class create it) is what makes object-oriented firmware testable. **Composition** (a monitor *has* a sensor) is preferred to deep inheritance hierarchies, which couple classes tightly; the SOLID design principles give names to these habits.

What a virtual call costs

A class with virtual functions gets a **vtable**, a table of function pointers in flash, one per class; each object carries a hidden **vptr** to its class's vtable. A virtual call loads the vptr, loads the function address from the table and branches

indirectly (Figure 21.4).

Worked example — virtual calls in a control loop. On a Cortex-M4 each polymorphic object grows by one 4-byte vptr, and each class's vtable occupies 4 bytes per virtual function plus a two-word header (offset-to-top and an RTTI pointer, placed before the slot the vptr points to); under the Itanium C++ ABI that GCC and Clang follow on Arm, a virtual destructor takes two slots (complete-object and deleting). A virtual call costs roughly 3 to 5 cycles more than a direct call (two loads and an indirect branch that refills the pipeline) and, more importantly, prevents inlining. A drive's 20 kHz current loop making six virtual calls per period spends about $6 \times 5 \times 20\,000 = 600\,000$ extra cycles per second, 0.36 % of a 168 MHz processor: negligible. The same calls inside a per-sample loop over a 256-tap filter would not be. Where the variant is fixed at build time, **static polymorphism** with templates (`OvertempMonitor<Ntc>`) gives the same decoupling with direct, inlinable calls.

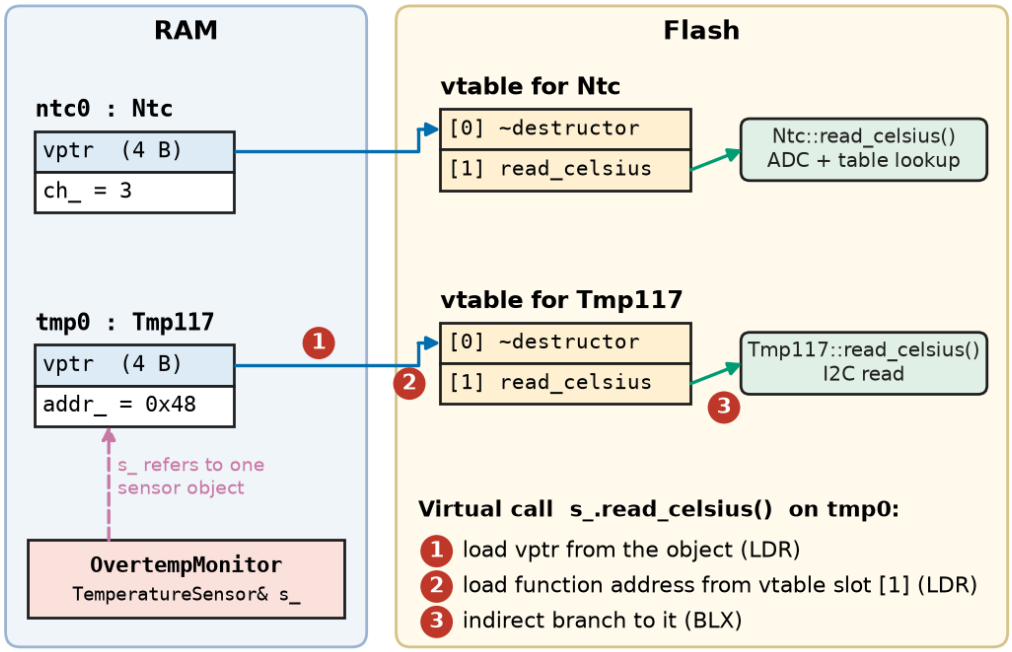


Figure 21.4 — Virtual dispatch: objects with vptrs, per-class vtables in flash, and the steps of a virtual call.

21.9 Compilation and linking

From source to image

A C or C++ build has four stages (Figure 21.5). The **preprocessor** expands `#include` and macros. The **compiler** translates each translation unit into assembly, optimising it (`-O0` for debugging, `-O2` or `-Os` for release). The **assembler** produces an **object file** containing machine code in named **sections** and a **symbol table** listing what the file defines and what it needs. The **linker** combines object files and libraries, resolves every undefined symbol to a definition, assigns final addresses according to a **linker script**, and writes an executable (ELF), from which a raw binary or Intel HEX file is extracted for flashing.

```
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard \
    -std=c11 -O2 -Wall -Wextra -Wconversion -ffunction-sections -fdata-sections \
    -c pid.c -o pid.o
arm-none-eabi-gcc -mcpu=cortex-m4 -mthumb -mfpu=fpv4-sp-d16 -mfloat-abi=hard \
    -T stm32f407.ld -Wl,--gc-sections -Wl,-Map=app.map \
    startup.o main.o pid.o -o app.elf
arm-none-eabi-size app.elf
   text    data     bss     dec     hex filename
  23456     512    6144   30112    75a0 app.elf
```

The standard sections are `.text` (code), `.rodata` (constants, counted in `text` by the size tool), `.data` (initialised variables) and `.bss` (zero-initialised variables). `-ffunction-sections` with `--gc-sections` lets the linker discard unused functions. Undefined-symbol errors at link time mean a declaration had no definition; multiple-definition errors mean a variable was defined, not just declared, in a header included by several files.

The linker script and start-up code

On a microcontroller the linker script says where memory is and where each section goes:

```
MEMORY
{
  FLASH (rx)  : ORIGIN = 0x08000000, LENGTH = 1024K
  RAM   (rwx) : ORIGIN = 0x20000000, LENGTH = 128K
}
_estack = ORIGIN(RAM) + LENGTH(RAM);            /* initial stack pointer */

SECTIONS
{
  .isr_vector : { KEEP(*(.isr_vector)) } > FLASH
  .text       : { *(.text*) *(.rodata*) } > FLASH
  _sidata = LOADADDR(.data);                    /* initial values, in flash */
  .data       : { _sdata = .; *(.data*) _edata = .; } > RAM AT > FLASH
  .bss        : { _sbss = .; *(.bss*) *(COMMON) _ebss = .; } > RAM
}
```

`.data` has two addresses: it **runs** in RAM but its initial values are **loaded** in flash. Before `main`, the reset handler must copy them across and clear `.bss`, which is what makes C's guarantee about initialised globals true on bare metal:

```
#include <stdint.h>

extern uint32_t _sidata, _sdata, _edata, _sbss, _ebss;   /* from the linker script */
extern int main(void);
void SystemInit(void);                    /* clocks, FPU enable: vendor code */

void Reset_Handler(void)
{
    const uint32_t *src = &_sidata;
    for (uint32_t *dst = &_sdata; dst < &_edata; ) {
        *dst++ = *src++;                  /* copy .data initial values to RAM */
    }
    for (uint32_t *dst = &_sbss; dst < &_ebss; ) {
        *dst++ = 0u;                      /* zero .bss */
    }
    SystemInit();
    (void)main();
    for (;;) { }                          /* main must not return */
}
```

Listing 21.11 — Linker-script excerpt and reset handler for a Cortex-M4 (GNU ld; C11).

Worked example — reading the size report. The size output above gives text 23 456, data 512 and bss 6144 bytes. Flash holds code, constants and the initial values of `.data`: $23\,456 + 512 = 23\,968$ bytes, 2.3 % of 1 MB. RAM holds `.data` and `.bss`: $512 + 6144 = 6656$ bytes, to which the linker script adds the stack (1.5 KB from the budget of Chapter 20) and any heap. The 2048-byte pool of Listing 21.5 and the 1032-byte ring buffer of Listing 21.8 are inside the 6144 bytes of `.bss`, visible by name in `app.map`. Watching these four numbers in continuous integration catches a change that silently adds 40 KB of `printf` floating-point support.

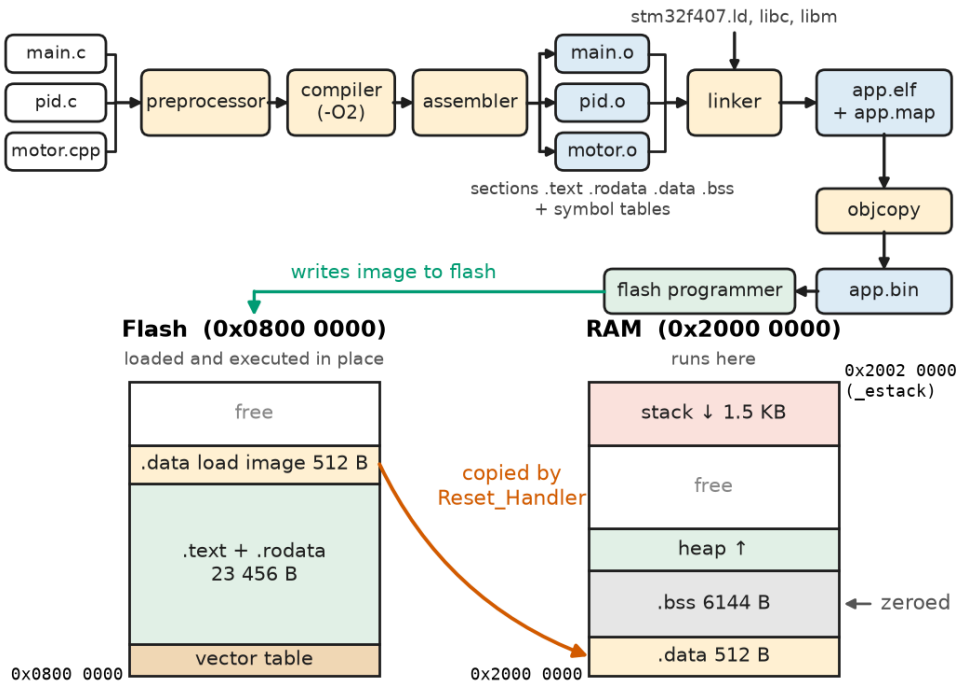


Figure 21.5 — The C/C++ build pipeline and where each output section is loaded and runs on a microcontroller.

Mixing C and C++, and libraries

C++ compilers **mangle** names to encode parameter types (so overloaded functions can coexist), so a C++ file calling a C function, or providing an interrupt handler that the C start-up code names, must declare it `extern "C"`, as in Listing 21.9. **Static libraries** (`.a`) are archives of object files from which the linker takes only what is referenced; **shared libraries** (`.so`, `.dll`) are linked at load time and are used on embedded Linux but not on bare-metal MCUs. **Link-time optimisation** (`-flto`) lets the compiler inline and discard across translation units, often shrinking firmware by 10–20 %. C++ adds the **one-definition rule**: an inline function or template must be defined identically in every translation unit that uses it.

21.10 Embedded-oriented C/C++

Why restrict the language

C and C++ are large languages with undefined behaviour, implicit conversions and features whose cost or timing is hard to predict. For firmware whose failure can injure people or damage equipment, industry uses **coding standards** that restrict the languages to a subset that is easier to analyse, review and test, and the functional-safety standards (IEC 61508 for industrial systems, ISO 26262 for road vehicles, IEC 62304 for medical device software, DO-178C for airborne software) expect such a standard to be applied and its deviations justified.

- **MISRA C** (MISRA C:2012 with amendments, consolidated as MISRA C:2023) is the most widely used, in automotive, industrial, medical and aerospace firmware.
- **MISRA C++:2023** covers C++17, superseding MISRA C++:2008 and absorbing the **AUTOSAR C++14** guidelines used for adaptive automotive platforms.
- **CERT C** and **CERT C++** focus on security (memory safety, input validation); **JSF AV C++** was written for the F-35 programme.

Table 21.3 — Typical embedded coding rules (MISRA C:2012 numbering, paraphrased) and their rationale.

Rule	Requirement	Rationale
Dir 4.6	Use typedefs of explicit size and signedness instead of `int`, `short`	Behaviour identical across 16- and 32-bit targets
Rules 10.1–10.8	No implicit conversions between essential type categories or to narrower types	Prevents the promotion surprises of Listing 21.1
Rule 14.4	Conditions of `if` and loops must be essentially Boolean	No `if (x = 5)` or accidental integer tests
Rule 15.5 (advisory)	A single point of exit at the end of a function	Easier review; often deviated in favour of guard clauses
Rule 16.4	Every `switch` has a `default`	Corrupted or new values handled deliberately
Rule 17.2	No recursion	Stack depth bounded and computable
Rule 21.3	No `malloc` / `free` from `<stdlib.h>`	No fragmentation or allocation failure at run time
Rule 8.13 (advisory)	Pointers point to `const` whenever possible	Read-only intent checked by the compiler

The rules are enforced by **static analysers** (PC-lint Plus, Polyspace, Helix QAC, Parasoft, and the open-source cppcheck and clang-tidy with MISRA-oriented checks), by compiler warnings treated as errors (`-Wall -Wextra -Wconversion -`

`Werror`), and by review. A project records each deviation with a justification; compliance means no *undocumented* violations, not zero violations.

> **Status of the principle: Restricted language subsets for safety-critical code**
> *Empirical.* The rules distil failure experience: each targets a construct that has repeatedly caused field defects or defeated analysis (implicit conversions, unbounded recursion, dynamic memory, undefined behaviour); evidence that following them reduces defect density is strong for the core memory- and type-safety rules and weaker for stylistic ones, which is why the standards classify rules as mandatory, required or advisory.

The embedded C++ subset

Embedded C++ projects keep the features that cost nothing at run time and disable or avoid those that do.

Table 21.4 — Cost of C++ features on a microcontroller and common embedded practice.

Feature	Run-time cost	Typical practice
Classes, inline member functions, references, namespaces	None beyond equivalent C	Use freely
`constexpr`, `static_assert`, `enum class`	None (compile time)	Use freely
Templates	Flash per instantiation	Use; watch code size
Virtual functions	vptr per object, vtable per class, indirect call	Use where run-time variation is needed
Exceptions	Unwind tables (often tens of KB), non-deterministic timing	Disabled (`-fno-exceptions`)
RTTI (`dynamic_cast`, `typeid`)	Type information per class	Disabled (`-fno-rtti`)
Heap containers (`std::vector`, `std::string`, `std::map`)	Dynamic allocation	Avoid after start-up; `std::array`, fixed-capacity libraries such as ETL
`iostream`	100 KB or more of flash	Avoid; use a small formatted-print routine

With exceptions disabled, errors are reported by return values, `std::optional` or an `expected`-style result type (`std::expected` in C++23), and the compiler options must be consistent across all libraries. The resulting subset, classes, templates, RAII, `constexpr` and carefully used virtual functions over statically allocated objects, gives firmware stronger type checking and fewer resource leaks than C at the same code size and speed, which is why it now appears in motor drives, automotive ECUs and medical devices alongside C.

Habits that matter more than rules

Coding standards catch constructs; engineering judgement still decides the design. Four habits carry most of the value: enable every warning and keep the build warning-free; give every variable the smallest scope and the most restrictive type and qualifier (`const`, `static`, fixed width) that works; make illegal states unrepresentable (enums instead of integers, classes with invariants); and compile the hardware-independent modules for a PC, where unit tests, sanitisers and static analysers run on every commit before the code ever reaches a target (Chapter 20, Chapter 28).

What remains open

Open question	Where it is answered
How are two's-complement and Q-format numbers represented and scaled?	Chapter 11 (Digital Number Systems)
How do the processor's memory map, interrupt controller and peripherals work at architecture level?	Chapter 17 (Microprocessors and Microcontrollers)
How are the same algorithms written for analysis and automation on a host?	Chapter 22 (Python for Engineering)
How is firmware structured into layers, drivers, boot loaders and update mechanisms?	Chapter 25 (Embedded-System Architecture)
How do UART, SPI, I²C and CAN drivers use these register techniques?	Chapter 26 (Embedded Interfaces)
How are data shared safely between interrupts and tasks, and how do RTOS primitives work?	Chapter 27 (Real-Time Computing)
How are device drivers, state machines and hardware-in-the-loop validation organised in a product?	Chapter 28 (Embedded System Design)
When does a computation move from C on a processor into FPGA logic?	Chapter 35 (Hardware–Software Co-Design)

Next: Chapter 22 turns to Python, the language engineers use to acquire, process, plot and automate around the hardware and firmware of this chapter.

Chapter 22 — Python for Engineering

Part V · Programming and Computational Systems

A power-supply validation lab has to qualify a new 48 V to 12 V, 240 W bus converter for a telecom rack. The test plan asks for efficiency, output ripple and load-step response at 20 load points, three input voltages (36, 48 and 60 V) and five ambient temperatures: 300 operating points, each needing two minutes of thermal settling and an oscilloscope capture. Done by hand, the job takes two weeks, and the first review finds a column of input currents pasted one row out of step. A script of about 200 lines runs the whole matrix overnight: it programs the load and the supply, reads the power analyser, pulls each waveform from the scope as binary data, stores everything with its metadata and plots the results before the engineers arrive. When a thermocouple fails at 3 a.m., the script turns the supply off, logs the fault and stops; it does not record two hours of nonsense. The language almost every lab uses for that script is Python. Python wins not by speed but because a short program can drive instruments, crunch arrays, plot and leave a record someone else can rerun.

> **How does an engineer use Python and its numerical stack to turn raw measurements into trustworthy numbers and plots?**
> **How are instruments and microcontrollers driven from Python, and how are the resulting data stored so that the experiment can be repeated?**

Concept	Engineering destination
Python fundamentals	Test scripts, calculators for component selection, glue between tools
Numerical arrays (NumPy)	ADC records, oscilloscope captures, sensor logs, matrix models
Data processing (pandas)	Characterisation sweeps, production-test logs, fleet telemetry
Plotting and visualisation	Datasheet curves, test reports, design reviews
Scientific libraries (SciPy)	Sensor calibration fits, root finding for trip points, ODE models of circuits
Signal analysis (scipy.signal)	THD and noise measurement, digital filtering of logged data, vibration spectra
Data acquisition	Bench automation over USB-TMC, LAN and GPIB; MCU data streams over UART
Automation scripts	Overnight sweeps, burn-in racks, production test stations
File formats	CSV for interchange, JSON for configuration, HDF5 for large waveform sets
Reproducible experiments	Audit trails in qualification, regulated (medical, automotive) test records

Roadmap. The language itself, seen from an engineer's desk (22.1) → NumPy arrays and why they are fast (22.2) → tabular data with pandas (22.3) → plots that survive a design review (22.4) → the SciPy toolbox (22.5) → signal analysis with scipy.signal (22.6) → talking to instruments and microcontrollers (22.7) → scripts that run unattended and fail safe (22.8) → file formats (22.9) → experiments that can be repeated (22.10).

22.1 Python fundamentals

> **Boundary.**
> Programming concepts in general are developed in Chapter 20 and C/C++ in Chapter 21. The mathematics behind the numerical routines used here belongs to *The Mathematics Universe*, Ch. 22; the DFT and digital filters to *Circuits, Signals and Communications*, Ch. 22–23 (see §22.6).

What kind of language Python is

Python is an interpreted, dynamically typed language. CPython compiles source to bytecode for a virtual machine; every value is a heap object with a type pointer and a reference count, and every operation is dispatched at run time. That makes Python pleasant to write (no declarations, no compile step, an interactive prompt for querying an instrument line by line) and makes a pure-Python loop slow, about 50–100 ns per simple operation against roughly 0.3 ns for a compiled add on a 3 GHz core. The working rule follows directly: *Python is the conductor, not the orchestra.* The heavy arithmetic runs inside compiled libraries (NumPy, SciPy, LAPACK, FFTW-class FFTs), and Python code decides what to compute, in what order, and what to do with the result.

The built-in types an engineer meets first behave as follows:

Type	Engineering use	Behaviour to remember
`int`	Register values, counts, codes	Arbitrary precision: never overflows (unlike `numpy.int16`, §22.2)
`float`	Physical quantities	IEEE 754 binary64, about 16 significant digits; `0.1 + 0.2 != 0.3`
`complex`	Phasors, impedances	Engineering `j`: `z = 3 + 4j`, `abs(z) == 5.0`
`str`, `bytes`	SCPI commands; raw serial data	Distinct types; serial ports and sockets deliver `bytes`
`dict`, `set`	Configuration, look-up by name	Hash tables with average $O(1)$ access (Chapter 23)

The floating-point behaviour is the IEEE 754 binary64 format, whose layout, unit roundoff $u = 2^{-53} \approx 1.1 \times 10^{-16}$ and rounding model are set out in *The Mathematics Universe*, Ch. 22 §22.1; the bit-level formats themselves are covered in Chapter 11 of this book. In test scripts this means: never compare floats with `==` (use `math.isclose` with a tolerance), and never step a sweep by repeated float addition; compute `v = v0 + k*dv` from an integer counter so the 300th point is where the plan says.

Functions, data classes and readable engineering code

Engineering scripts outlive their authors. Functions with docstrings and type hints, `dataclass` records and f-strings that print numbers with units keep them readable. Listing 22.1 uses all three in a small component-selection tool.

```
 from dataclasses import dataclass
from itertools import product

E24 = [1.0, 1.1, 1.2, 1.3, 1.5, 1.6, 1.8, 2.0, 2.2, 2.4, 2.7, 3.0,
       3.3, 3.6, 3.9, 4.3, 4.7, 5.1, 5.6, 6.2, 6.8, 7.5, 8.2, 9.1]

@dataclass(frozen=True)
class Divider:
    r_top: float      # ohm
    r_bot: float      # ohm
    def ratio(self) -> float:
        return self.r_bot / (self.r_top + self.r_bot)

def decade(values, lo, hi):
    """All series values between lo and hi ohms."""
    return [round(v * 10**k) for k in range(8) for v in values if lo <= v * 10**k <= hi]

def best_divider(v_in, v_out, lo=10e3, hi=1e6):
    target = v_out / v_in
    pairs = (Divider(a, b) for a, b in product(decade(E24, lo, hi), repeat=2))
    return min(pairs, key=lambda d: abs(d.ratio() - target))

d = best_divider(v_in=30.0, v_out=3.3)
print(d, f"ratio={d.ratio():.5f}  error={(d.ratio()*30/3.3 - 1)*100:+.2f} %")
print(f"Vout at 30 V: {30*d.ratio():.4f} V, load on sensor: {30/(d.r_top+d.r_bot)*1e6:.1f} uA")
```

```
 Divider(r_top=130000, r_bot=16000) ratio=0.10959  error=-0.37 %
Vout at 30 V: 3.2877 V, load on sensor: 205.5 uA
```

Listing 22.1 — Choosing an E24 resistor pair for a sensor divider by exhaustive search.

Worked example — a 30 V sensor input for a 3.3 V ADC. An industrial controller reads a 0–24 V transmitter signal, with headroom to 30 V, on a microcontroller ADC whose reference is 3.3 V. The divider must map 30 V to 3.3 V, a ratio of 0.1100, with total resistance between 10 kΩ and 1 MΩ so that the source is lightly loaded and the ADC's sampling capacitor can still be charged through a buffer. Listing 22.1 searches all $49^2 = 2401$ ordered pairs of E24 values in that range and picks $R_{top} = 130\ \text{k}\Omega$, $R_{bot} = 16\ \text{k}\Omega$. The ratio is $16/146 = 0.10959$, so 30 V gives 3.288 V, 0.37 % below full scale. That is a safe direction: the ADC never saturates on a 30 V input. The divider draws $30/146\ \text{k}\Omega = 205.5\ \mu\text{A}$, well within the few milliamperes a voltage-output transmitter can drive. With 1 % resistors the worst-case ratio error is about $\pm 2 \times 1\,\% \times (1 - 0.1096) = \pm 1.8\,\%$, five times the selection error, so the gain is calibrated at production test anyway. The brute-force search is $O(n^2)$, but with 2401 candidates it takes milliseconds; a cleverer algorithm would be wasted effort (Chapter 23).

Packages and environments

Engineering work relies on a small core of packages: **NumPy**, **SciPy**, **pandas**, **Matplotlib**, **h5py**, **pyserial** and **PyVISA**. Each project gets its own virtual environment (`python -m venv .venv`, or conda) with pinned versions, so that a library upgrade cannot silently change last month's results (§22.10, Chapter 24).

> **Status of the principle: Interpreter cost and vectorisation**
> *Empirical.* The 50–100 ns per interpreted operation and the 100–1000× gain of compiled array code are measured properties of CPython on current processors; the ratio varies with operation, array size and version, but never reverses for large arrays.

22.2 Numerical arrays

The ndarray: one block of typed memory

NumPy's `ndarray` is what makes Python usable for engineering: one contiguous block of raw numbers of a single **dtype** (`int16`, `float32`, `float64`, `complex128`, ...), plus a header giving its **shape** and **strides**, the bytes to step to move one index along each axis. A record of 10 000 samples from four ADC channels stored as `int16` with shape `(10000, 4)` occupies exactly 80 000 bytes, and its strides are `(8, 2)`: 8 bytes to the next sample, 2 bytes to the next channel (Figure 22.1). Slicing (`a[:, 2]`, `a[::10]`) creates a **view** with different strides over the same memory, not a copy. That is fast, but writing into a view modifies the original.

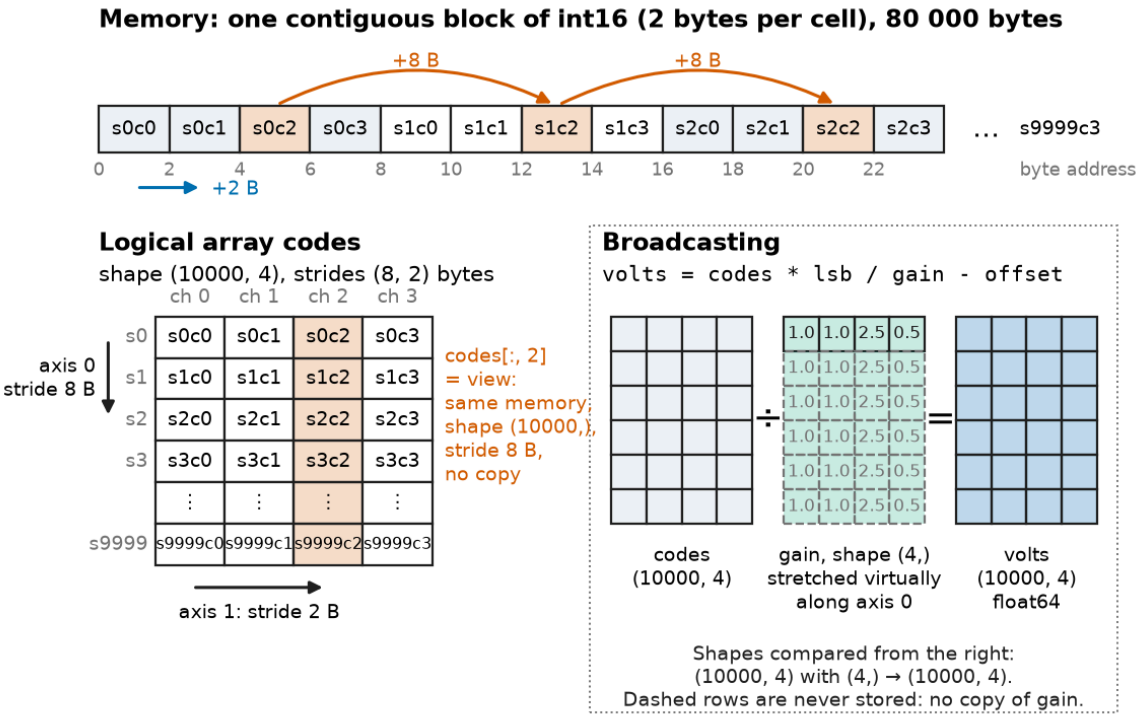


Figure 22.1 — An int16 ADC record as one block of memory: shape, strides, a column view, and broadcasting of per-channel gain.

Whole-array operations (`+`, `*`, `np.sin`, `np.sum`, `@`) run as compiled loops over that memory. On the machine used for this chapter, the sum of squares of 10^6 doubles takes 104 ms as a Python loop and 0.2 ms as the dot product `x @ x`, a factor of about 500. Why the gain is so large, and why it still cannot turn an $O(N^2)$ algorithm into an $O(N \log N)$ one, is analysed in *The Mathematics Universe*, Ch. 24 §24.1 (vectorisation, BLAS levels and arithmetic intensity). Chapter 24 of this book takes the hardware side.

Broadcasting

When two arrays of different shapes meet in an operation, NumPy **broadcasts** them. Axes are compared from the right; an axis of length 1 (or a missing axis) is stretched to match, without copying. Converting a multi-channel record from ADC codes to volts with a different gain and offset per channel is then one line, as Listing 22.2 shows: an array of shape `(10000, 4)` combined with vectors of shape `(4,)`.

```
 import numpy as np

fs, n_ch = 100_000, 4                      # 100 kS/s per channel, 4 channels
rng = np.random.default_rng(1)
t = np.arange(10_000) / fs                 # 0.1 s record
# Simulated 16-bit ADC: +/-10 V full scale, codes -32768..32767
v_true = np.stack([9.5 * np.sin(2*np.pi*50*t + k) for k in range(n_ch)], axis=1)
codes = np.clip(np.round(v_true / 10 * 32768 + rng.normal(0, 2, v_true.shape)),
                -32768, 32767).astype(np.int16)          # shape (10000, 4)

lsb = 10 / 32768                                         # V per code
gain = np.array([1.0, 1.0, 2.5, 0.5])                    # V/V of each front end
offset = np.array([0.0, 0.012, -0.004, 0.0])             # V, from calibration
volts = codes * lsb / gain - offset                      # broadcasting: (N,4) op (4,)

print(codes.dtype, codes.shape, codes.strides, codes.nbytes, "bytes")
print("peak ch0:", volts[:, 0].max().round(4), "V")
print("sum of squares, int16 :", (codes[:, 0] * codes[:, 0]).sum())   # wraps!
print("sum of squares, int64 :", (codes[:, 0].astype(np.int64)**2).sum())
print("RMS ch0:", np.sqrt(np.mean(volts[:, 0]**2)).round(4), "V")
```

```
 int16 (10000, 4) (8, 2) 80000 bytes
peak ch0: 9.501 V
sum of squares, int16 : 787652
sum of squares, int64 : 4845254214852
RMS ch0: 6.7175 V
```

Listing 22.2 — Converting a four-channel 16-bit ADC record to volts by broadcasting, and the integer-overflow trap.

Dtypes and the overflow trap

Python's own `int` never overflows; NumPy's fixed-width integers do, silently, by two's-complement wraparound, exactly as the C types of Chapter 21 do. In Listing 22.2 the product `codes * codes` is computed in `int16`, so every square above 32 767 wraps, and the "sum of squares" is garbage: 787 652 instead of 4.85×10^{12}.

Worked example — sizing the accumulator for an RMS computation. A 16-bit code has magnitude at most 2^{15}, so its square is at most $2^{30} \approx 1.07 \times 10^{9}$. An `int32` accumulator (maximum $2^{31} - 1 \approx 2.15 \times 10^{9}$) therefore overflows after only *two* full-scale samples. Summing $N = 10^{4}$ squares needs up to $10^{4} \times 2^{30} = 1.07 \times 10^{13}$, about 44 bits, so `int64` is required, as in the last line of the listing. Converted to volts, the RMS of channel 0 is 6.7175 V, which matches $9.5/\sqrt{2} = 6.7175$ V. The same arithmetic sizes the 40-bit accumulators of fixed-point DSPs and the 64-bit accumulators in FPGA power meters (Chapters 11 and 18). Dtype choice also sets memory: four channels at 1 MS/s stored as `int16` produce 8 MB/s, or 28.8 GB per hour; converted carelessly to `float64` the same hour occupies 115.2 GB. Keep raw codes as integers on disk and convert to physical units only for the slice being analysed.

22.3 Data processing

Tables, not arrays

Measurement campaigns produce *tables*: each row an operating point, each column a quantity with its own unit and type. **pandas** provides the `DataFrame`, named NumPy columns sharing a row index, with reading and writing of CSV, Excel and Parquet, selection, derived columns, grouping, pivoting, joins and time-series resampling.

Listing 22.3 processes a sweep of the opening's bus converter. A stand-in for the logged file is built from a loss model, $P_{loss} = 0.8 + 0.02 V_{in} + 0.03 I_o + 0.008 I_o^2$ W, plus measurement noise; the script then reads the CSV, derives power, efficiency and loss, pivots, and finds the best point at each input voltage.

```
 import io
import numpy as np
import pandas as pd

# --- stand-in for a logged CSV: 48 V -> 12 V converter, 3 input voltages x 20 loads
rng = np.random.default_rng(3)
rows = []
for vin in (36.0, 48.0, 60.0):
    for iout in np.linspace(1, 20, 20):
        p_loss = 0.8 + 0.02 * vin + 0.03 * iout + 0.008 * iout**2      # W, loss model
        vout = 12.0 - 0.004 * iout
        iin = (vout * iout + p_loss) / vin
        rows.append((vin, iin * (1 + rng.normal(0, 2e-4)), vout, iout))
csv_text = pd.DataFrame(rows, columns=["vin", "iin", "vout", "iout"]).to_csv(index=False)

df = pd.read_csv(io.StringIO(csv_text))                        # normally: pd.read_csv("sweep.csv")
df["pin"] = df.vin * df.iin
df["pout"] = df.vout * df.iout
df["eff"] = df.pout / df.pin
df["ploss"] = df.pin - df.pout

table = df.pivot_table(index="iout", columns="vin", values="eff")
print(table.loc[[2.0, 5.0, 10.0, 20.0]].round(4))
best = df.loc[df.groupby("vin")["eff"].idxmax(), ["vin", "iout", "eff", "ploss"]]
print(best.round(3).to_string(index=False))
```

```
 vin     36.0    48.0    60.0
iout
2.0   0.9375  0.9280  0.9196
5.0   0.9698  0.9660  0.9622
10.0  0.9779  0.9764  0.9745
20.0  0.9782  0.9775  0.9762
 vin  iout   eff  ploss
36.0  15.0 0.980  3.731
48.0  17.0 0.978  4.465
60.0  18.0 0.977  5.022
```

Listing 22.3 — Reducing a converter efficiency sweep with pandas: derived columns, a pivot table and a group-wise maximum.

Worked example — where is the efficiency peak, really? With a loss of the form $P_0 + aI + bI^2$ and an output power of nearly $V_o I$, the efficiency is greatest where the loss per watt delivered, $P_0/I + a + bI$, is smallest. That is at $I^* = \sqrt{P_0/b}$, where the fixed loss equals the conduction loss. At $V_{in} = 36$ V, $P_0 = 0.8 + 0.72 = 1.52$ W and $I^* = \sqrt{1.52/0.008} = 13.8$ A. At 48 V, $P_0 = 1.76$ W and $I^* = 14.8$ A; at 60 V, $P_0 = 2.00$ W and $I^* = 15.8$ A. The table, however, reports 15, 17 and 18 A. Near its maximum the efficiency curve is flat: at 60 V, taking $P_{out} = 12 I$ as Listing 22.4 and Figure 22.2 do, the model gives 97.696 % (the 97.70 % peak) at 15.8 A and 97.679 % at 18 A, a difference of 0.017 percentage points (the 4 mV/A output droop in Listing 22.3 lowers both by about 0.012 points). That is comparable to the 0.02 % measurement noise placed on the input current. The `idxmax` of a noisy, flat curve is dominated by the noise. The right report fits the loss model (a three-parameter linear least-squares problem, §22.5) and quotes I^* from the fitted coefficients with an uncertainty. Real power analysers have 0.02–0.05 % basic accuracy, so this is the regime of real efficiency testing.

A few habits keep tabular work honest. Put units in column names (`iout_A`, `temp_C`), because pandas stores numbers, not units. Store one row per observation and pivot only for display. Decide explicitly what a missing value (`NaN`) means; a disconnected thermocouple must not quietly become part of an average. Beyond a few gigabytes, read in chunks or store in Parquet or HDF5 (§22.9).

22.4 Plotting and visualization

The figure as an engineering document

A plot in a test report is evidence. It should show the quantity against the variable that matters, with units on both axes, ranges that do not hide the region of interest, and a legend that survives black-and-white printing. **Matplotlib** is the standard library. Its object-oriented interface separates the **figure** (the page) from one or more **axes** (plotting areas), each configured explicitly. Listing 22.4 produces the efficiency curves of the opening example (Figure 22.2).

```
 import numpy as np
import matplotlib
matplotlib.use("Agg")                          # no window: render straight to files
import matplotlib.pyplot as plt

iout = np.linspace(0.5, 20, 200)
fig, ax = plt.subplots(figsize=(6, 3.5), layout="constrained")
for vin, style in [(36, "-"), (48, "--"), (60, ":")]:
    eff = 12 * iout / (12 * iout + 0.8 + 0.02 * vin + 0.03 * iout + 0.008 * iout**2)
    ax.plot(iout, 100 * eff, style, label=f"Vin = {vin} V")
ax.set(xlabel="Output current (A)", ylabel="Efficiency (%)", ylim=(85, 100), xlim=(0, 20))
ax.grid(True, alpha=0.3); ax.legend(loc="lower right")
fig.savefig("efficiency.svg")                  # vector for reports; PNG at dpi=200 for slides
print(len(ax.lines), "curves saved")
```

```
 3 curves saved
```

Listing 22.4 — A report-quality efficiency plot rendered headless and saved as a vector file.

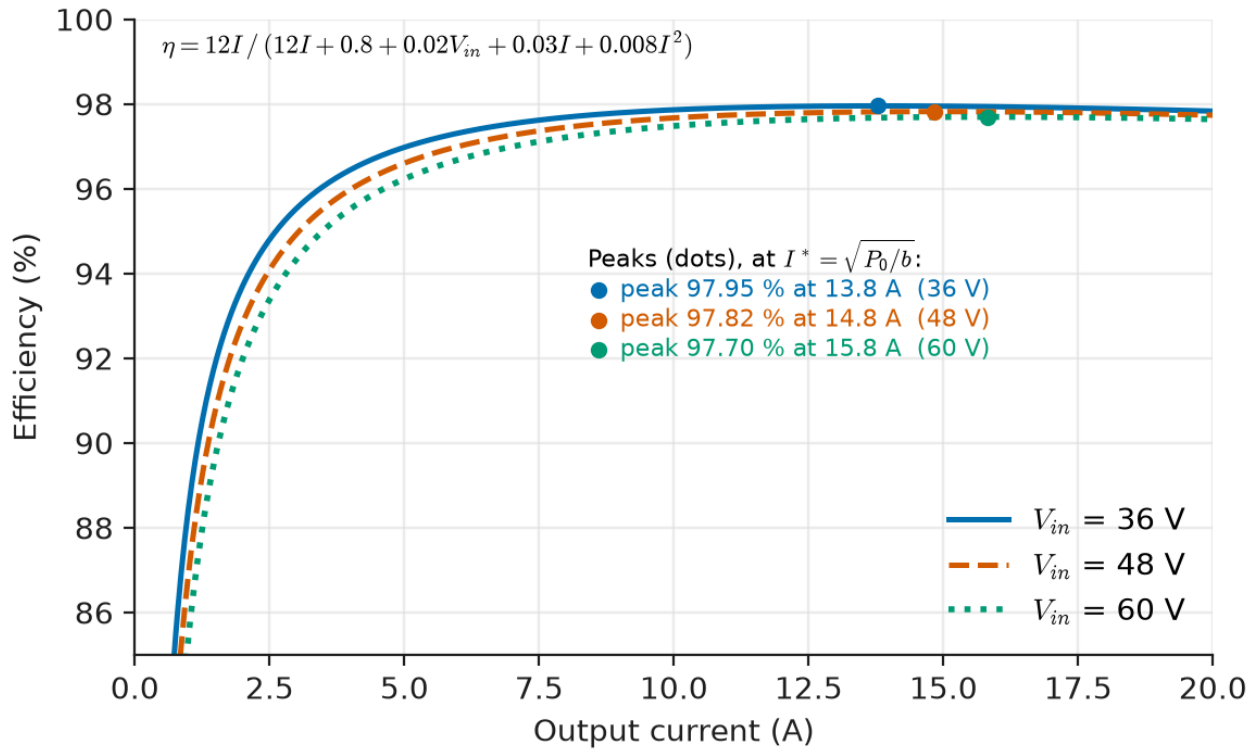


Figure 22.2 — Efficiency of the 48 V to 12 V bus converter versus output current at three input voltages, from the loss model of Section 22.3.

The `Agg` back end renders without a display, which is what an overnight script on a headless test PC needs. Interactive work in a Jupyter notebook uses the default back end instead.

Choices that matter

Situation	Recommended practice
Frequency response, spectra, noise density	Log frequency axis (`ax.semilogx`), magnitude in dB; never a linear axis spanning decades
Efficiency or loss curves	Linear axes, efficiency axis not starting at 0 if differences of 0.1 % matter, but say so
Many traces of a sweep	One colour map ordered by the swept variable (`viridis`), or small multiples (`plt.subplots(2, 3)`)
Waveforms with 10^6 points	Decimate or plot min/max envelopes per pixel column; plotting every point is slow and adds nothing
Report and paper output	Vector format (SVG, PDF) for line art; PNG at 200–300 dpi for raster; embed fonts
Colour	Colour-blind-safe palettes; distinguish series by line style as well as colour
Reproducibility	Generate every figure from a script reading stored data, never by hand-editing

Two failures recur in design reviews: the autoscaled axis that makes a 0.05 % efficiency change look like a cliff, and the plot whose data lived only in a notebook session. Both are cured by treating plotting code as ordinary code, kept with the data it reads.

22.5 Scientific libraries

The SciPy toolbox

SciPy wraps long-established Fortran and C libraries (LAPACK, QUADPACK, MINPACK, ODEPACK, ARPACK, SuperLU). The engineer picks the routine, poses the problem well and checks the answer. The theory of each method, with its error and convergence behaviour, is in *The Mathematics Universe*, Ch. 22 (root finding §22.3–22.4, interpolation §22.5, least squares §22.6, quadrature §22.8, ODEs §22.9) and Ch. 24 (linear algebra §24.3, iterative solvers §24.4, sparse matrices §24.5, parameter estimation §24.7).

Subpackage	Key routines	Typical engineering task
`scipy.linalg`	`solve`, `lu_factor`, `cholesky`, `eig`, `svd`, `expm`	Nodal analysis, state-space models, fitting
`scipy.sparse`, `.sparse.linalg`	CSR/CSC matrices, `spsolve`, `cg`, `eigsh`	Power-grid IR drop, FEM thermal models (Chapter 24)
`scipy.optimize`	`brentq`, `root`, `least_squares`, `curve_fit`, `minimize`, `linprog`	Trip points, operating points, model fitting, design optimisation
`scipy.integrate`	`solve_ivp`, `quad`, `cumulative_trapezoid`	Circuit and thermal transients, energy from power logs
`scipy.interpolate`	`CubicSpline`, `make_interp_spline`, `RegularGridInterpolator`	Look-up tables, resampling irregular logs
`scipy.stats`	Distributions, tests, `linregress`	Production-test statistics, C_{pk}, outlier screening

Calibration and a trip point

Listing 22.5 fits a sensor model to calibration data and then solves the model for a design value. An NTC thermistor follows the Steinhart–Hart equation $1/T = A + B\ln R + C(\ln R)^3$ (T in kelvin), which is *linear* in A, B, C, so four calibration points give a linear least-squares problem for `numpy.linalg.lstsq`. Two nested root-finding problems are then solved by Brent's method (`brentq`), a safeguarded bracketing method that cannot diverge once a sign change is bracketed.

```
 import numpy as np
from scipy.optimize import brentq

# Calibration of a 10 kOhm NTC in a stirred oil bath
T_C = np.array([0.0, 25.0, 50.0, 85.0])               # deg C
R = np.array([32_650.0, 10_000.0, 3_602.0, 1_070.0])  # ohm

# Steinhart-Hart: 1/T = A + B ln R + C (ln R)^3, linear in A, B, C -> least squares
L = np.log(R)
M = np.column_stack([np.ones_like(L), L, L**3])
(A, B, C), res, *_ = np.linalg.lstsq(M, 1 / (T_C + 273.15), rcond=None)
print(f"A={A:.6e}  B={B:.6e}  C={C:.6e}")

def temp_C(r):
    l = np.log(r)
    return 1 / (A + B * l + C * l**3) - 273.15

print("fit residuals (mK):", np.round((temp_C(R) - T_C) * 1e3, 1))

# Over-temperature trip: NTC to ground, 10 kOhm pull-up to 3.3 V, comparator at 0.60 V
v_node = lambda T: 3.3 * r_of_T(T) / (10e3 + r_of_T(T))
r_of_T = lambda T: brentq(lambda r: temp_C(r) - T, 50.0, 1e6)   # invert the fit
T_trip = brentq(lambda T: v_node(T) - 0.60, 0.0, 150.0)
print(f"trip temperature = {T_trip:.2f} C, R_ntc = {r_of_T(T_trip):.0f} ohm")
```

```
 A=1.130968e-03  B=2.338158e-04  C=8.892946e-08
fit residuals (mK): [-0.9  3.9 -4.7  1.8]
trip temperature = 63.15 C, R_ntc = 2222 ohm
```

Listing 22.5 — Steinhart–Hart calibration by linear least squares and an over-temperature trip point by nested root finding.

Worked example — the trip point of a battery-pack thermal cut-out. A 10 kΩ NTC on an e-bike pack sits under a 10 kΩ pull-up from 3.3 V, and a comparator at 0.60 V disconnects the pack. The trip occurs when $3.3R/(10\ \text{k}\Omega + R) = 0.60$, that is at $R = 0.60 \times 10/2.70 = 2.222\ \text{k}\Omega$. The fitted Steinhart–Hart model puts that resistance at 63.15 °C. The fit residuals of a few millikelvin show that the model reproduces the calibration to far better than the ±1 % tolerance of the pull-up resistor. That tolerance alone moves the trip by roughly ± 0.3 °C; the comparator offset and the thermistor's own ±1 % R_{25} tolerance are of the same order, so the design is specified as 63 ± 1 °C. The firmware alternative, reading the node with an ADC and evaluating the model, is a look-up-table problem treated in Chapter 23.

How library routines fail

SciPy routines rarely crash on a bad problem; they return something. `curve_fit` started far from the answer converges to a local minimum. `solve_ivp` with its default explicit Runge–Kutta method on a stiff circuit (time constants of nanoseconds and seconds together) takes millions of tiny steps. `quad` on a switching waveform reports convergence while stepping over its edges. The defences: check status flags (`sol.success`), test on a case with a known answer, use an implicit method (`"Radau"` , `"LSODA"`) for stiff problems (Ch. 22 §22.9), and pass known breakpoints to integrators.

22.6 Signal analysis

> **Boundary.**
> The DFT and its interpretation (bins, amplitude scaling, leakage, windows, zero padding and Welch averaging) are developed in *Circuits, Signals and Communications*, Ch. 22 §22.3–22.8; digital filter structures and design in Ch. 23; sampling and aliasing in Ch. 19. This section uses those results and concentrates on doing the computation correctly with NumPy and `scipy.signal`.

From theory to library calls

From Book 3: a tone with an integer number of cycles in the record falls in one bin, bin k lies at $f_k = k f_s / N$, a window spreads each tone over its main lobe, and Welch averaging trades resolution for variance. The library calls map onto those ideas as follows; most practical measurement errors occur at exactly these points:

Task	Call	What to get right
Spectrum of a real record	`np.fft.rfft(x * w)`	Single-sided; scale by $2/\sum w$ for peak amplitude (Book 3 §22.3, coherent gain)
Window	`signal.windows.hann(N, sym=False)`	`sym=False` (periodic) for spectral analysis, symmetric for FIR design
Power spectral density	`signal.welch(x, fs, nperseg=...)`	`scaling="density"` gives V²/Hz, `"spectrum"` gives V² per bin; always pass `fs`
IIR filter design	`signal.butter(..., output="sos")`	Use second-order sections, not `(b, a)`, above order 2
Zero-phase filtering of stored data	`signal.sosfiltfilt`	Non-causal: fine offline, impossible in real time
Rate change	`signal.resample_poly`, `signal.decimate`	Built-in anti-alias filtering; never just take every Mth sample
Peaks and events	`signal.find_peaks(x, prominence=...)`	Use prominence, not just height, on noisy data

Measuring harmonic distortion

Listing 22.6 measures the total harmonic distortion of an audio amplifier output captured at 48 kHz (spectrum in Figure 22.3). The record is chosen **coherently**: $N = 4800$ samples at 48 kHz give 10 Hz bins, so the 1 kHz test tone sits exactly in bin 100 and every harmonic in an exact bin. A periodic Hann window then confines each tone to three bins ($k-1$, k, $k+1$), and summing the power over those three bins captures each tone completely. The ratio of harmonic power to fundamental power does not depend on window scaling, because both are windowed alike.

```
 import numpy as np
from scipy import signal

fs, N, f0 = 48_000, 4_800, 1_000.0          # coherent: 1 kHz falls exactly in bin 100
n = np.arange(N)
rng = np.random.default_rng(7)
x = (1.0 * np.sin(2*np.pi*f0*n/fs)          # 1 V peak fundamental
     + 1e-3 * np.sin(2*np.pi*2*f0*n/fs)     # 2nd harmonic, -60 dBc
     + 3e-3 * np.sin(2*np.pi*3*f0*n/fs)     # 3rd harmonic, -50.5 dBc
     + rng.normal(0, 1e-4, N))              # noise, 100 uV RMS

w = signal.windows.hann(N, sym=False)       # periodic Hann for spectral analysis
X = np.fft.rfft(x * w)
P = np.abs(X)**2                             # power per bin (window-weighted)
k0 = round(f0 * N / fs)

def tone_power(k):                           # Hann main lobe spans k-1..k+1
    return P[k-1:k+2].sum()

harm = [tone_power(h * k0) for h in range(2, 11)]
thd = np.sqrt(sum(harm) / tone_power(k0))
print(f"bin spacing {fs/N:.1f} Hz, fundamental in bin {k0}")
print(f"THD = {100*thd:.4f} %  ({20*np.log10(thd):.2f} dB)")
print("H2, H3 (dBc):", [round(float(10*np.log10(h / tone_power(k0))), 2) for h in harm[:2]])
```

```
 bin spacing 10.0 Hz, fundamental in bin 100
THD = 0.3164 %  (-49.99 dB)
H2, H3 (dBc): [-60.03, -50.45]
```

Listing 22.6 — THD from a coherently sampled, Hann-windowed record.

Worked example — THD and noise floor of an audio power stage. The synthetic capture contains a 1 V peak fundamental, a second harmonic of 1 mV (−60 dBc) and a third of 3 mV (−50.5 dBc). The expected THD is $\sqrt{(10^{-3})^2 + (3\times10^{-3})^2}/1 = \sqrt{10^{-5}} = 0.3162\,\%$, or −50.0 dB. The script reports 0.3164 %; the extra 0.0002 % is noise power falling in the harmonic bins. The individual harmonics read −60.03 and −50.45 dBc. For the noise floor, `signal.welch(x, fs=fs, nperseg=1024)` returns a PSD whose level corresponds to $10^{-4}/\sqrt{24\,000} = 0.65\ \mu\mathrm{V}/\sqrt{\mathrm{Hz}}$. Two easy mistakes would have spoiled the measurement. A 1 kHz tone recorded with $N = 4096$ samples lies at bin 85.33, not on a bin. With a rectangular window its leakage skirt (Book 3 §22.5) is still about −49 dB at the second-harmonic frequency, 85 bins away, and would bury the −60 dBc second harmonic. And reading the per-bin noise floor of a 4800-point FFT as if it were the SNR overstates the dynamic range by the FFT processing gain of Book 3 §22.8, here $10\log_{10}(2400/1.5) = 32$ dB. AES17 fixes the measurement bandwidth for this reason.

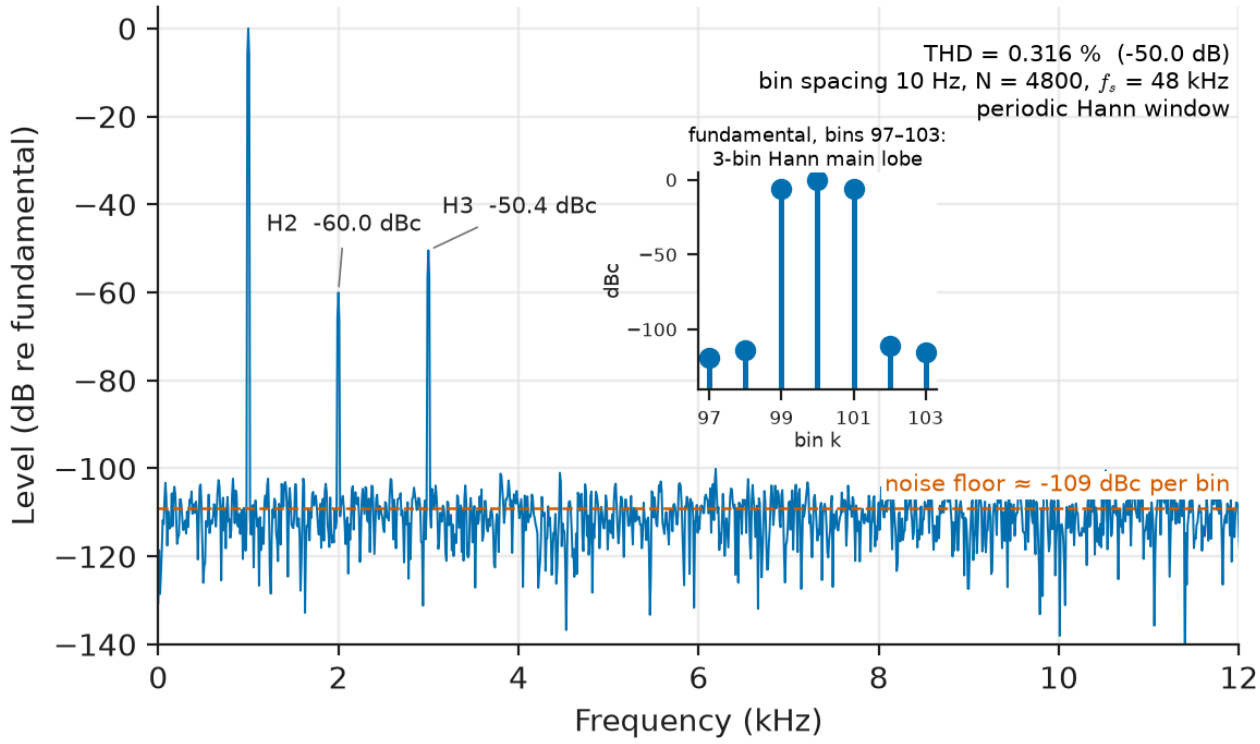


Figure 22.3 — Hann-windowed spectrum of the coherently sampled 1 kHz amplifier test signal: fundamental, second and third harmonics, and the noise floor.

Filters: second-order sections or nothing

Listing 22.7 designs an eighth-order Butterworth low-pass at 20 Hz for data sampled at 10 kHz, the kind of filter used to extract a slowly varying temperature or strain signal from a fast ADC stream. It then implements it in two ways.

```
 import numpy as np
from scipy import signal

fs = 10_000
b, a = signal.butter(8, 20, fs=fs)                  # 8th-order low-pass, fc = 20 Hz
sos = signal.butter(8, 20, fs=fs, output="sos")
print("max |pole|, (b,a) form :", np.abs(np.roots(a)).max())
print("max |pole|, SOS form   :", np.abs(signal.sos2zpk(sos)[1]).max())

t = np.arange(20_000) / fs
x = np.sin(2*np.pi*5*t) + 0.5*np.sin(2*np.pi*400*t)  # 5 Hz wanted, 400 Hz interference
y_ba = signal.lfilter(b, a, x)
y_sos = signal.sosfiltfilt(sos, x)                   # zero-phase, forward-backward
print("peak |y| from (b,a)     :", f"{np.abs(y_ba).max():.3g}")
print("peak |y| from sosfiltfilt:", f"{np.abs(y_sos[5000:15000]).max():.4f}")
```

```
 max |pole|, (b,a) form : 1.0061473105002674
max |pole|, SOS form   : 0.9975514848369335
peak |y| from (b,a)     : 2.79e+23
peak |y| from sosfiltfilt: 1.0000
```

Listing 22.7 — The same eighth-order Butterworth filter as one polynomial and as second-order sections.

The polynomial form fails outright. Its poles cluster within 0.0025 of $z = 1$, and rounding the coefficients to binary64 pushes one pole outside the unit circle: the output grows to 10^{23}. SciPy prints a `BadCoefficients` warning, which an unattended script ignores. The SOS form keeps every pole at radius 0.9976 and passes the 5 Hz signal at unity gain while removing the 400 Hz interference. This is the coefficient-sensitivity result of Book 3 Ch. 23 §23.3 and §23.9, seen in double precision on a PC. In the `float32` or Q15 arithmetic of a microcontroller (Chapter 28) it bites at much lower orders.

> **Status of the principle: Cascade (SOS) implementation of IIR filters**
> *Derived*. The roots of a high-order polynomial are far more sensitive to coefficient perturbations than the roots of its quadratic factors, so a cascade of second-order sections preserves the designed pole positions under rounding where the direct form cannot; it holds for any finite precision, including binary64.

`sosfiltfilt` runs the filter forward and backward, cancelling the phase response: ideal for stored records that must stay time-aligned, but not what a real-time controller sees. To predict firmware behaviour, including group delay, use `sosfilt` with state carried between blocks.

22.7 Data acquisition

Two worlds: instruments and embedded targets

A test station talks to **bench instruments** (supplies, loads, DMMs, scopes, power analysers), which accept SCPI (Standard Commands for Programmable Instruments) text commands over a standard transport, and to **embedded targets** (an MCU board, a battery-management unit), which stream data over a serial link in a firmware-defined format. Figure 22.4 shows the two software stacks side by side.

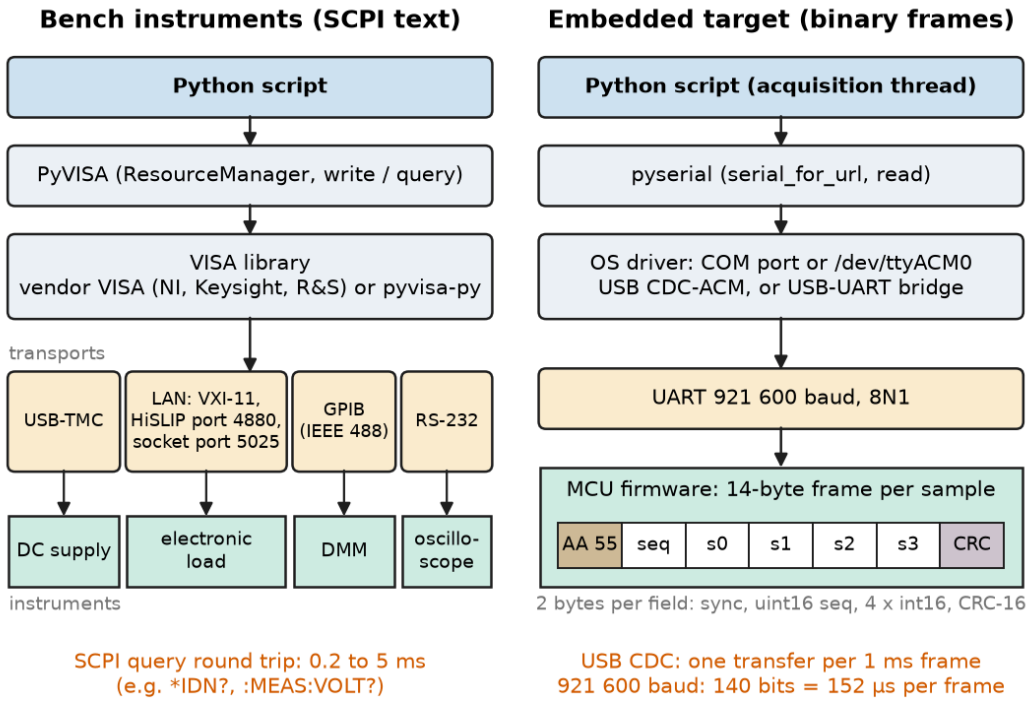


Figure 22.4 — Software stacks for data acquisition from Python: PyVISA to SCPI instruments, and pyserial to a microcontroller.

Instruments: VISA and SCPI

VISA (Virtual Instrument Software Architecture, IVI Foundation) gives one API over every transport. An instrument is named by a **resource string** such as `USB0::0x2A8D::0x1102::MY61001234::INSTR` (USB-TMC), `TCPIP0::192.168.1.20::hislip0::INSTR` (HiSLIP over LAN) or `GPIB0::12::INSTR`. **PyVISA** wraps a vendor VISA library or its own pure-Python back end (`pyvisa-py`). Commands follow **IEEE 488.2** (common commands such as `*IDN?`, `*RST`, `*OPC?`, `*ESR?`) and SCPI's hierarchical syntax (`:SOURce:VOLTage:LEVel 3.3`, shortened to `:VOLT 3.3`). Queries end in `?` and return text.

Listing 22.8 runs against PyVISA's simulated back end (package `pyvisa-sim`), which models a SCPI power supply, so it executes without hardware. Changing `"@sim"` to `""` and the resource string to a real address is all that is needed on the bench.

```
import pyvisa

rm = pyvisa.ResourceManager("@sim")       # "@sim" = simulated back end; use "" for NI/Keysight/R&S VISA
psu = rm.open_resource("USB::0x1111::0x2222::0x2468::INSTR",
                       read_termination="\n", write_termination="\n", timeout=2000)
print(psu.query("*IDN?"))                  # IEEE 488.2 identification
psu.write("*RST")
psu.write(":VOLT:IMM:AMPL 3.300")          # SCPI: set output to 3.3 V
psu.write("OUTP 1")
v = float(psu.query(":VOLT:IMM:AMPL?"))
print(f"set point read back: {v:.3f} V, output = {psu.query('OUTP?')}")
psu.write("OUTP 0")
psu.close()
```

```
SCPI,MOCK,VERSION_1.0
set point read back: 3.300 V, output = 1
```

Listing 22.8 — Programming a SCPI power supply through PyVISA (simulated back end).

Waveforms should not travel as text. An oscilloscope returns a 10-million-point record as an **IEEE 488.2 definite-length block**: `#`, one digit giving the number of length digits, the length, then raw bytes (`#8` followed by `20000000` and 20 MB of `int16` data). PyVISA's `query_binary_values(..., datatype="h", container=numpy.array)` parses the header and returns an array; the scope's preamble query supplies the volts-per-code, offset and sample interval. ASCII transfer of the same record is about six times larger. Typical throughputs are 0.2–1 MB/s for GPIB, 5–30 MB/s for USB-TMC and 20–100 MB/s for HiSLIP or raw sockets on gigabit LAN, with 0.2–5 ms latency per query.

Robust instrument code follows a few rules. Query `*IDN?` first and record the reply. Set a timeout longer than the slowest operation. Synchronise with `*OPC?` after configuration rather than with `time.sleep`. Read the error queue (`:SYST:ERR?`) after every block of commands, because a mistyped SCPI command is rejected silently and the script carries on with the old setting.

Embedded targets: pyserial and framed binary data

A microcontroller streaming ADC data over a UART or a USB CDC virtual COM port appears to Python as a serial port, opened with **pyserial**. The stream is just bytes. The firmware and the script must agree on a **frame**: a synchronisation pattern, a sequence counter to detect lost frames, the payload in a fixed binary layout (little-endian on Arm Cortex-M and most MCUs), and a checksum to reject corrupted frames. The UART electrical layer and framing are the subject of Chapter 26. Listing 22.9 is the receiving side. It uses pyserial's `loop://` URL, which echoes writes back as reads, so the parser can be tested without a board, including a deliberately corrupted frame.

```
 import struct, binascii
import serial                                   # pyserial

SYNC = b"\xAA\x55"
BODY = struct.Struct("<H4h")                     # seq (uint16) + 4 x int16 samples
FRAME_LEN = 2 + BODY.size + 2                    # sync + body + CRC-16 = 14 bytes

def make_frame(seq, samples):                    # what the MCU firmware sends
    body = BODY.pack(seq, *samples)
    return SYNC + body + struct.pack("<H", binascii.crc_hqx(body, 0xFFFF))

def read_frames(port, n):
    buf, out, bad = bytearray(), [], 0
    while len(out) < n:
        buf += port.read(64)
        while (i := buf.find(SYNC)) >= 0 and len(buf) - i >= FRAME_LEN:
            frame = bytes(buf[i:i + FRAME_LEN])
            body, (crc,) = frame[2:-2], struct.unpack("<H", frame[-2:])
            if binascii.crc_hqx(body, 0xFFFF) == crc:
                out.append(BODY.unpack(body)); del buf[:i + FRAME_LEN]
            else:
                bad += 1; del buf[:i + 1]        # resynchronise on the next sync word
    return out, bad

# loop:// echoes writes back, so this runs without hardware; use "COM5" or "/dev/ttyACM0" for a board
with serial.serial_for_url("loop://", baudrate=921_600, timeout=0.1) as port:
    port.write(make_frame(1, (100, -200, 300, -400)))
    corrupted = bytearray(make_frame(2, (1, 2, 3, 4))); corrupted[6] ^= 0x10
    port.write(bytes(corrupted) + make_frame(3, (5, 6, 7, 8)))
    frames, bad = read_frames(port, 2)
print(frames, "CRC failures:", bad)
```

```
[(1, 100, -200, 300, -400), (3, 5, 6, 7, 8)] CRC failures: 1
```

Listing 22.9 — Receiving CRC-checked binary frames from a microcontroller with pyserial (tested on the loopback port).

The format string `"<H4h"` is the contract with the firmware: little-endian, an unsigned 16-bit sequence number and four signed 16-bit samples, the layout of a packed C `struct` on the target (Chapter 21). The checksum is CRC-16/CCITT (initial value 0xFFFF). After a bad frame the parser drops one byte and searches again, so a bit error costs one frame; the sequence jump from 1 to 3 records the loss.

Worked example — will the UART keep up? A battery-test board must stream four channels at 2 kS/s each, one frame of Listing 22.9 per sample instant. A frame is 14 bytes, and with 8N1 framing each byte costs 10 bit times, so a frame is 140 bits. At 2000 frames/s the link must carry 280 kbit/s. A 115 200-baud link carries at most $115\,200/140 = 823$ frames/s and fails by a factor of 2.4; at 921 600 baud the capacity is 6583 frames/s and the utilisation $280\,000/921\,600 = 30\,\%$. On the PC, the serial driver buffers typically 4–16 kB; at 28 kB/s a script that stalls for half a second while plotting loses data. Acquisition therefore runs in its own thread or process, feeding a queue, and plotting runs elsewhere. Native USB CDC (around 1 MB/s in practice) removes the baud-rate limit; the firmware-side buffering is a ring-buffer problem (Chapter 23).

Dedicated DAQ cards and PXI systems come with vendor Python APIs (such as `nidaqmx`) that configure hardware-timed sampling: the sample clock lives on the card and Python only collects buffers. Above a few hundred samples per second this is the rule, because software-timed Python loops jitter by milliseconds.

22.8 Automation scripts

Anatomy of an unattended test

An automation script almost always has the structure of Figure 22.5: load the configuration, identify instruments, reach a safe initial state, loop over operating points (set, settle, measure, check, store), and always end in a safe state, whatever happened.

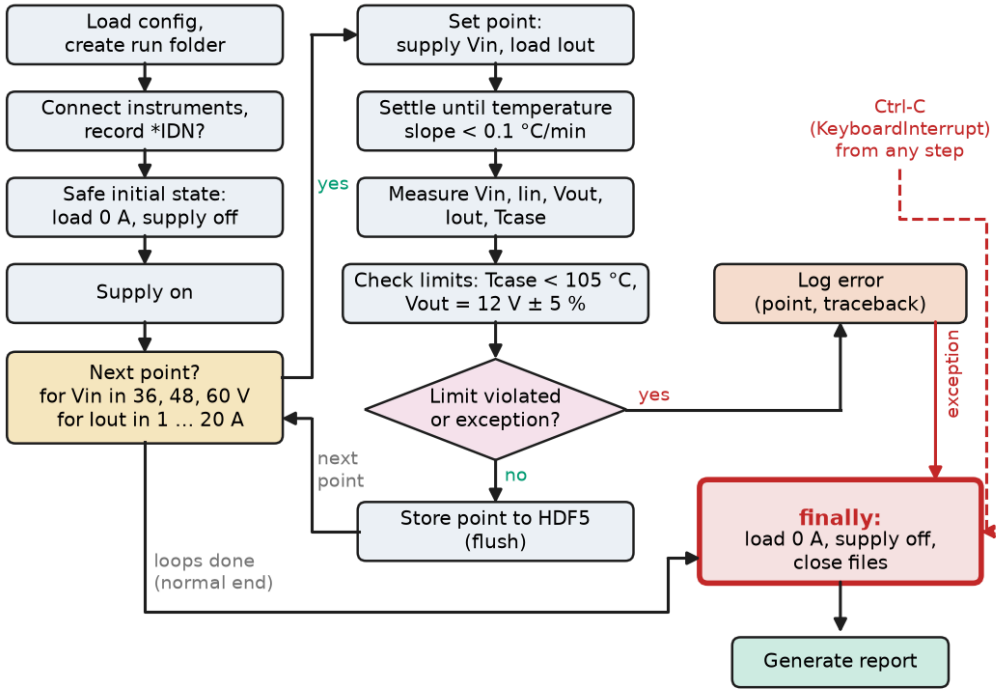


Figure 22.5 — Structure of an unattended characterisation sweep with a guaranteed safe-state exit path.

Listing 22.10 shows the skeleton with stand-in instruments; the load faults at 20 A, as a real electronic load does when its heatsink overheats. The `finally` block runs on success, exception or keyboard interrupt, so the supply is always turned off.

```
 import logging, time
logging.basicConfig(level=logging.INFO, format="%(levelname)s %(message)s")
log = logging.getLogger("sweep")

class FakeSupply:                                  # stands in for a PyVISA instrument class
    def __init__(self): self.v, self.on = 0.0, False
    def set_voltage(self, v): self.v = v
    def output(self, on): self.on = on; log.info("supply output %s", "ON" if on else "OFF")

class FakeLoad:
    def __init__(self): self.i = 0.0
    def set_current(self, i):
        if i > 15: raise RuntimeError("load reports over-temperature")   # injected fault
        self.i = i

def run_sweep(supply, load, vins, iouts, settle_s=0.0):
    results = []
    try:
        supply.output(True)
        for vin in vins:
            supply.set_voltage(vin)
            for i in iouts:
                load.set_current(i)
                time.sleep(settle_s)                # thermal settling in a real rig
                results.append((vin, i))            # ...measure and store here
    except Exception as exc:
        log.error("sweep aborted at Vin=%.0f V, I=%.1f A: %s", supply.v, i, exc)
    finally:                                        # runs on success, error or Ctrl-C
        load.set_current(0.0)
        supply.output(False)
    return results

res = run_sweep(FakeSupply(), FakeLoad(), vins=[36, 48], iouts=[5, 10, 20])
print(len(res), "points recorded:", res)
```

```
 INFO supply output ON
ERROR sweep aborted at Vin=36 V, I=20.0 A: load reports over-temperature
INFO supply output OFF
2 points recorded: [(36, 5), (36, 10)]
```

Listing 22.10 — Skeleton of an automated sweep: the `finally` block guarantees the safe state.

Design rules for automation

- **Software is not a safety interlock.** The `finally` block protects the experiment, not people. Hazardous energy (a 400 V EV pack, a laser) needs hardware interlocks and over-voltage and over-current limits set in the instruments themselves, so that a crashed PC cannot leave the DUT powered. IEC 61010-1 governs the test equipment; the script is only a second layer.
- **Configuration apart from code.** Operating points, limits and addresses live in a versioned JSON or YAML file (§22.9), so the plan changes without editing Python.
- **Instrument classes.** Wrap each instrument model in a class with methods named for what they do (`set_current`). Swapping a load vendor then changes one class, and a simulated class tests the script itself, as here.
- **Settling by criterion.** Wait until the case temperature changes by less than, say, 0.1 °C/min, with a timeout, not a fixed two minutes.

- **Log, do not print; offer a dry run.** `logging` timestamps every event into the test record, and an `argparse` flag such as `--dry-run` walks the plan with simulated instruments before an overnight run.
- **Other tools as subprocesses.** `subprocess.run(["ngspice", "-b", "buck.cir"], check=True)` runs a simulator in batch mode; Chapter 24 builds simulation workflows on this.

> **Status of the principle: Fail-safe exit of automated tests**
> *Fundamental.* By the definition of the language, a `finally` clause executes whenever control leaves its `try` block, whether normally or by an exception; this guarantees the software return to a safe state for every failure the interpreter survives, but not for a power cut, an operating-system crash or a hung driver, which only hardware protection covers.

22.9 File formats

Choosing a format

Engineering data come in three shapes: small structured **configuration and metadata**, moderate **tables** of results, and large **arrays** of waveforms or images. No single format is best for all three.

Format	Shape	Strengths	Weaknesses	Python
CSV	Tables	Universal, human-readable, opens in any tool	No types or units; ~3–4× larger than binary; slow; float precision lost if written with too few digits	`pandas.read_csv`, `csv`
JSON	Configuration, metadata	Nested structure, human-readable, standard (RFC 8259)	No comments; inefficient for arrays; no NaN in strict JSON	`json`
NPY / NPZ	Arrays	Exact binary copy of an ndarray; fast	Python/NumPy-specific; minimal metadata	`np.save`, `np.load`
HDF5	Arrays, hierarchies, metadata	Self-describing, chunked, compressed, partial reads, multi-gigabyte files, language-neutral	Single-writer by default; a corrupted file can be lost if not closed	`h5py`, `pandas.HDFStore`
Parquet	Large tables	Columnar, compressed, typed, fast filtering	Not human-readable	`pandas.read_parquet`
Touchstone (.sNp)	S-parameters	Industry standard for RF and signal-integrity data	Frequency-domain only	`scikit-rf`

Domain formats such as Touchstone, COMTRADE, VCD and NetCDF are surveyed in Chapter 24.

The same data, four ways

Listing 22.11 writes one second of a two-channel 1 MS/s oscilloscope capture (voltage and current of a converter switch node) as CSV, NPY and compressed HDF5, with its metadata in JSON and inside the HDF5 file, and compares the sizes.

```
 import json, os
import numpy as np
import pandas as pd
import h5py

fs, n = 1_000_000, 1_000_000                      # 1 s of 2-channel scope data at 1 MS/s
rng = np.random.default_rng(0)
t = np.arange(n) / fs
v = (0.5 * np.sin(2 * np.pi * 10e3 * t) + 0.01 * rng.standard_normal(n)).astype(np.float32)
i = (2.0 * np.sin(2 * np.pi * 10e3 * t - 0.3)).astype(np.float32)
meta = {"fs_Hz": fs, "probe": "10:1", "dut": "SN 2311-044", "scope": "MSO 4-series"}

pd.DataFrame({"t": t, "v": v, "i": i}).to_csv("cap.csv", index=False)
np.save("cap.npy", np.stack([v, i]))
with open("cap.json", "w") as fh:
    json.dump(meta, fh, indent=2)
with h5py.File("cap.h5", "w") as f:
    d = f.create_dataset("scope/ch", data=np.stack([v, i]), chunks=(2, 65_536),
                         compression="gzip", compression_opts=4, shuffle=True)
    d.attrs.update(meta)                          # metadata travels with the data
    d.attrs["units"] = "V, A"

for name in ["cap.csv", "cap.npy", "cap.h5", "cap.json"]:
    print(f"{name:9s} {os.path.getsize(name)/1e3:9.1f} kB")
with h5py.File("cap.h5") as f:                    # read back only 1000 samples of ch 1
    print(f["scope/ch"][1, 5000:6000].max(), dict(f["scope/ch"].attrs)["probe"])
```

```
 cap.csv     32004.0 kB
cap.npy      8000.1 kB
cap.h5       3150.4 kB
cap.json        0.1 kB
1.9997995 10:1
```

Listing 22.11 — One capture stored as CSV, NPY and gzip-compressed HDF5 with attached metadata.

Worked example — storage for a week of vibration monitoring. The NPY file is exactly the raw data, $2 \times 10^6 \times 4$ bytes = 8.0 MB. The CSV is four times larger and carries no units or sample rate. The HDF5 file, shuffled and gzip-compressed, is 3.15 MB, mostly because the noise-free current channel compresses well. Now scale to a condition-monitoring system on a wind-turbine gearbox with eight accelerometers at 25.6 kS/s in `float32`. That is $8 \times 25\,600 \times 4 = 819$ kB/s, or 70.8 GB per day and about 495 GB per week as raw binary. As CSV the week would approach 2 TB and take hours to parse. As chunked HDF5 with one dataset per channel and day, the ten minutes around an alarm are read from a 70 GB file in well under a second, because only the chunks holding those samples are touched. The format decides whether the data are usable at all.

Precision and portability details

- **CSV round trip.** A `float64` needs 17 significant digits to survive text exactly; loggers that write six digits truncate silently.
- **Layout.** A raw `.bin` dump from an MCU records neither byte order nor type, so its `numpy.dtype` (`np.dtype("<i2")`) must be stored beside it. HDF5 and NPY record both.
- **Time stamps.** Store absolute times in UTC (ISO 8601, or `int64` nanoseconds since the epoch) and t_0 and f_s as metadata, not ten million `float64` time values.
- **Closing files.** An HDF5 file left open by a crash can be corrupted; the `with` block closes it on exceptions, the file counterpart of the `finally` rule of §22.8.

22.10 Reproducible computational experiments

> **Boundary.**
> The numerical side of reproducibility (levels of reproducibility, seeded and independent random streams, floating-point non-associativity, tolerance-based testing of numerical results) is treated in *The Mathematics Universe*, Ch. 24 §24.9. This section adds what a measurement-driven experiment needs on top; Chapter 24 extends both to large simulation campaigns, testing and version control.

What must be recorded

A measured efficiency of 97.8 % means something only with the conditions that produced it. A reproducible experiment records, automatically: **the plan** (configuration file and its hash); **the equipment** (every `*IDN?` string, calibration dates, probes, fixture); **the DUT and environment** (serial number, hardware and firmware revision, ambient temperature, operator); **the software** (Git commit, Chapter 24; package versions); and **the raw data**, immutable, with a cryptographic hash that exposes any alteration or mix-up.

Listing 22.12 is a minimal pattern for the first, fourth and fifth items. Each run gets its own folder, named by UTC time and a hash of the configuration, holding a manifest and a `raw` subfolder.

```
 import hashlib, json, platform, sys
from datetime import datetime, timezone
from importlib.metadata import version
from pathlib import Path
import numpy as np

def sha256(path):
    with open(path, "rb") as f:
        return hashlib.file_digest(f, "sha256").hexdigest()

def start_run(config: dict, root="runs") -> Path:
    tag = hashlib.sha256(json.dumps(config, sort_keys=True).encode()).hexdigest()[:8]
    run = Path(root) / f"{datetime.now(timezone.utc):%Y%m%dT%H%M%SZ}_{tag}"
    (run / "raw").mkdir(parents=True)
    manifest = {"config": config, "config_hash": tag, "python": sys.version.split()[0],
                "platform": platform.platform(),
                "packages": {p: version(p) for p in ("numpy", "scipy", "pandas", "h5py")}}
    (run / "manifest.json").write_text(json.dumps(manifest, indent=2))
    return run

cfg = {"dut": "SN 2311-044", "vin_V": [36, 48, 60], "iout_A": [1, 20, 20], "seed": 42}
run = start_run(cfg)
data = np.random.default_rng(cfg["seed"]).normal(size=1000)
np.save(run / "raw" / "noise.npy", data)
print(run.name[-8:], sha256(run / "raw" / "noise.npy")[:16])
```

```
 a1fea18f 50b0d735b2bae064
```

Listing 22.12 — A run folder with a configuration hash, an environment manifest and a hash of the raw data.

The output is identical on every run: the tag `a1fea18f` depends only on the plan (serialised with sorted keys) and the data hash only on the seeded generator. A changed plan gets a different tag, so results from two versions of a test plan cannot be confused. In a laboratory accredited to ISO/IEC 17025, or in automotive (ISO 26262) and medical (IEC

62304) development, this kind of traceability is not optional: test software must be validated and its outputs traceable to equipment, procedure and data.

Notebooks, scripts and the single command

Jupyter notebooks are excellent for exploration and the most common source of irreproducible results: cells run out of order, variables survive from deleted cells, and the notebook that "produced Figure 3" no longer does when run from the top. The discipline is simple. Move anything that produces a reported number into a version-controlled module; restart and run all before recording a result; treat raw data as read-only, with every table, figure and report regenerable by one command (`python make_report.py runs/20251014T221500Z_a1fea18f`); and pin the environment (`pip freeze > requirements.txt`).

> **Status of the principle: Reproducibility of a measurement pipeline**
> *Approximate.* With recorded configuration, equipment, software versions and immutable raw data, a re-analysis reproduces derived results exactly and a re-measurement reproduces them within the measurement uncertainty; the guarantee fails for anything not recorded (an unlogged probe change, a fixture swap, an instrument's internal firmware update).

The opening's 200-line script is reproducible in this sense: a colleague who doubts the 97.8 % peak can see what measured it, rerun the analysis on untouched raw data, and rerun the measurement itself.

What remains open

Open question	Where it is answered
How are the algorithms behind sorting, searching, look-up tables and graph problems chosen and analysed?	Chapter 23
How do matrix libraries, parallel execution, profiling, numerical precision, testing and version control scale Python work up to large simulations?	Chapter 24
How are UART, SPI, I²C, CAN and USB links that deliver data to the PC designed and debugged at the electrical and protocol level?	Chapter 26
How does firmware buffer, timestamp and log data before it reaches the host?	Chapter 28
What are the error and convergence properties of the root finders, fitters and integrators that SciPy wraps?	*The Mathematics Universe*, Ch. 22 and Ch. 24
How are FIR and IIR filters designed to a specification, and how do windows trade leakage against resolution?	*Circuits, Signals and Communications*, Ch. 22 and Ch. 23

Next: Chapter 23 looks underneath the library calls used here, at the algorithms and data structures (lists, queues, trees, hash tables, sorting, searching and graph algorithms) whose cost decides whether an engineering program finishes in milliseconds or in hours.

Chapter 23 — Algorithms and Data Structures

Part V · Programming and Computational Systems

A layout team's sign-off script checks a 2-million-pin post-layout netlist for pins that appear on no net. It is twenty lines of clear Python and it works on the test board. On the real chip it runs all night. The script keeps the connected pins in a list, and `pin in connected` searches that list from the start every time: about $3n^2/8 \approx 1.5 \times 10^{12}$ string comparisons, seven hours at 17 ns each. Changing one line, so that the pins are held in a set, makes each look-up a hash-table probe, and the whole check runs in about a tenth of a second. Nothing about the processor, the language or the netlist changed. What changed was the **data structure**, and with it the **algorithm** the program actually executed. The same lesson appears across the field: a UART driver that drops bytes because its buffer is a list that is shifted on every read; a median filter that fits a 1 kHz control loop but not a 20 kHz one; a router that must recompute shortest paths across a thousand links in milliseconds; a timing analyser that must find the slowest of 10^9 paths without enumerating them.

> **How is the cost of an algorithm predicted before it is run, and how does the choice of data structure set that cost?**
> **Which classic structures and algorithms (lists, queues, trees, hash tables, sorting, searching, graph traversal, shortest paths) solve which engineering problems, and where do they fail?**

Concept	Engineering destination
Complexity analysis	Will the netlist check, the log parser or the control-loop filter finish in time?
Arrays and linked lists	Sample buffers, RTOS ready lists, memory pools
Stacks and queues	UART ring buffers, call stacks, event-driven simulators, timer wheels
Trees and graphs	Routing tables, file systems, spatial queries in PCB layout, netlists
Hash tables	Symbol tables, net-name look-up, CAN-ID dispatch, deduplication
Searching and sorting	Calibration look-up tables, median filters, production-test ranking
Numerical algorithms	CORDIC in FPGAs, running statistics, fixed-iteration solvers in firmware
Graph algorithms	OSPF routing, net extraction, static timing analysis, floating-node checks
Algorithm selection	Matching worst case, memory and determinism to the platform

Roadmap. How cost is counted and what it means on real hardware (23.1) → contiguous and linked sequences (23.2) → stacks, queues, ring buffers and priority queues (23.3) → trees and graphs as data (23.4) → hash tables (23.5) → searching and sorting (23.6) → numerical algorithms as implemented (23.7) → graph algorithms (23.8) → choosing among them (23.9) → engineering applications (23.10).

23.1 Complexity analysis

> **Boundary.**
> Asymptotic notation (O, Ω, Θ), the master theorem for divide-and-conquer recurrences, flop counting and the empirical scaling exponent $p = \ln(T_2/T_1)/\ln(n_2/n_1)$ are developed in *The Mathematics Universe*, Ch. 24 §24.1, together with Horner's rule and the cost of the DFT against the FFT. This section uses them and adds what matters when the algorithm runs on a particular machine under a deadline.

What is being counted

Complexity analysis counts the **basic operations** an algorithm performs (comparisons, memory accesses, arithmetic operations) as a function of the input size n, and the **memory** it needs beyond its input. Three counts answer different engineering questions:

- **Worst case**: the maximum over all inputs of size n; the only acceptable measure for hard real-time code (Chapter 27) and for safety arguments under ISO 26262 or DO-178C.
- **Average case**: the mean over a stated input distribution; it governs throughput.
- **Amortised cost**: the worst-case total of a *sequence* of m operations divided by m. An occasional expensive step averages out, but can still blow a deadline (§23.2).

The cost of one basic operation is not a constant either. On a desktop processor an L1-cache hit costs about 1 ns, a main-memory access about 80–100 ns, and an interpreted Python operation 50–100 ns; on a Cortex-M4 at 168 MHz a single-cycle SRAM access costs 6 ns, but flash with wait states costs several cycles unless the accelerator cache hits. An algorithm with fewer operations but scattered memory accesses can lose to one with more operations on contiguous data. Big-O hides constants by design; engineering decisions keep them.

Budgets: from complexity to time

Table 23.1 converts complexity classes into times for a compiled language at roughly 10^8 simple operations per second with a realistic memory mix (an interpreted language is 10–100 times slower).

n	$\log_2 n$	n	$n \log_2 n$	n^2	n^3	2^n
10^2	70 ns	1 µs	7 µs	100 µs	10 ms	4×10^{14} years
10^4	130 ns	100 µs	1.3 ms	1 s	2.8 h	—
10^6	200 ns	10 ms	0.2 s	2.8 h	317 years	—
10^9	300 ns	10 s	5 min	317 years	—	—

Table 23.1 — Time at 10^8 operations per second. Netlists, log files and sample records routinely reach 10^6–10^9 elements.

The table explains most of the "it worked on the test case" failures in engineering software. An $O(n^2)$ step that takes 100 µs on a 100-net test circuit takes three hours on a million-net chip. The cure is rarely a faster computer: it is a structure or algorithm that drops the exponent.

> **Status of the principle: Asymptotic dominance**
> *Derived.* For any constants, $c_1 n^a$ eventually exceeds $c_2 n^b$ when $a > b$, and any exponential exceeds any polynomial; the crossover size depends on the constants, which is why small inputs can favour the asymptotically worse algorithm.

Worked example — a median filter inside a control loop. A motor-drive current loop on a Cortex-M4 at 168 MHz rejects ADC glitches with a running median over the last w samples. At a 1 kHz loop rate the budget is $168 \times 10^6 / 10^3 = 168\,000$ cycles per period, of which the filter may take 10 %, about 16 800 cycles. Sorting a fresh copy of the window by insertion sort costs at worst $w(w-1)/2$ compare-and-swap steps. For $w = 9$ that is 36 steps, roughly 200 cycles at about 5 cycles each: trivial. For $w = 64$ it is 2016 steps, about 10 000 cycles: still inside the 1 kHz budget but 60 % of the filter's share. Move the same filter into the 20 kHz inner current loop and the whole period is 8400 cycles, so the 64-sample sort alone exceeds it. Two changes rescue it. A **sliding** median keeps the window sorted and, per new sample, deletes the oldest value and inserts the newest by shifting: $O(w)$, at most about 64 moves, a few hundred cycles. Or a much smaller window ($w = 5$, which a fixed 9-comparator sorting network handles in constant time) suffices to reject single-sample glitches. The choice is set by worst-case cycles, not by average speed, because the loop deadline is hard.

23.2 Arrays and linked lists

Contiguous arrays

An **array** stores n elements of equal size in consecutive memory. Element i is at address base $+ \, i \cdot s$, so indexing is $O(1)$, and a sequential scan moves through memory in order. That order is what caches and prefetchers are built for: a 64-byte cache line brings in 16 `float32` samples at once, and the hardware prefetcher fetches the next lines before they are needed. Inserting or deleting in the middle costs $O(n)$, because the elements after the gap must move. NumPy arrays (Chapter 22), C arrays and `std::vector` (Chapter 21) are arrays in this sense.

A **dynamic array** (Python `list`, C++ `std::vector`) adds capacity management. When an append finds the array full, it allocates a block larger by a factor r and copies everything. Since the capacities form a geometric series, the total copying over n appends is about $n/(r-1)$ element moves, so each append costs $O(1)$ amortised. A Python list stores *pointers* to objects and grows by about 12.5 % plus a small constant: watching `sys.getsizeof` during appends shows capacities of 4, 8, 16, 24, 32, 40, … pointers.

Worked example — the price of a growth factor. With $r = 2$ (common in C++ libraries) the copies total about n, one per element; with CPython's $r \approx 1.125$ they total about $n/0.125 = 8n$. Measured over 200 000 appends, CPython performed 72 reallocations and 8.1 copies per element, as predicted. The amortised cost is still $O(1)$, but the *largest* single append copied 180 440 pointers at once, about 1.4 MB of memory traffic. In a soft real-time data logger that is a visible hiccup; in a hard real-time loop, dynamic growth is simply not allowed. Embedded firmware therefore preallocates fixed-size arrays at start-up (Chapter 25), and the MISRA C guidelines restrict dynamic memory allocation after initialisation.

Linked lists

A **linked list** stores each element in a node that points to the next (and, in a doubly linked list, the previous) node. Inserting or removing a node whose address is known is $O(1)$: two or four pointer updates, no shifting. Finding the i-th element is $O(n)$, and a traversal jumps around memory, one likely cache miss per node. On a desktop that makes a linked-list scan 10–50 times slower than an array scan of the same length (Figure 23.1).

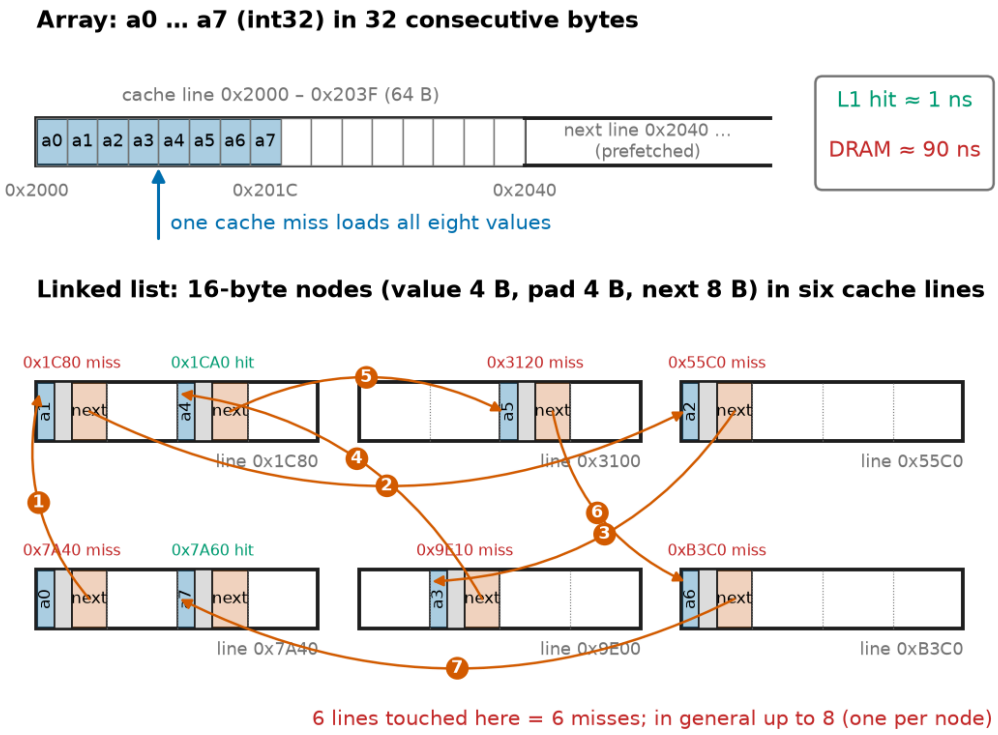


Figure 23.1 — An array and a linked list holding the same eight 32-bit values, drawn against 64-byte cache lines.

Linked lists earn their place where elements move between collections more often than they are scanned. An RTOS keeps tasks on **intrusive** doubly linked lists, with the links embedded in each task control block, so moving a task to the ready list is a constant-time splice with no allocation (FreeRTOS, Zephyr and Linux all do this). Memory pools thread a **free list** through their free blocks, giving $O(1)$, fragmentation-free allocation of packet or CAN-frame buffers.

Operation	Array	Dynamic array	Singly linked list	Doubly linked list
Index i	$O(1)$	$O(1)$	$O(n)$	$O(n)$
Append at end	— (fixed)	$O(1)$ amortised	$O(1)$ with tail pointer	$O(1)$
Insert/delete at front	$O(n)$	$O(n)$	$O(1)$	$O(1)$
Delete a known node	$O(n)$	$O(n)$	$O(n)$ (needs predecessor)	$O(1)$
Scan, cache behaviour	Excellent	Excellent	Poor	Poor
Memory overhead per element	0	0–12 % slack	1 pointer	2 pointers

---

23.3 Stacks and queues

Stacks

A **stack** is a last-in, first-out (LIFO) collection with $O(1)$ `push` and `pop`. The processor's call stack holds return addresses, saved registers and locals, with a hardware stack pointer (Chapter 16); stacks also parse expressions, implement undo in a schematic editor and drive depth-first search (§23.8). The embedded hazard is **stack overflow**: a 1 kB task stack, three nested interrupts and a 600-byte `printf` silently overwrite adjacent memory. Worst-case depth is computed by static analysis of the call graph, a longest-path problem on a DAG (§23.8), and checked at run time with a guard pattern or the MPU (Chapter 27).

Queues and ring buffers

A **queue** is first-in, first-out (FIFO): `enqueue` at the tail, `dequeue` at the head. On a fixed memory budget the standard implementation is the **ring buffer** (circular buffer): an array of N slots with a head index where the producer writes and a tail index where the consumer reads, both wrapping at N (Figure 23.2). With N a power of two, the wrap is a bit mask (`index & (N-1)`) instead of a division or a branch, and free-running unsigned counters make "full" (head − tail = N) and "empty" (head = tail) unambiguous without wasting a slot. With one writer of `head` (a UART interrupt) and one of `tail` (a task), it needs no lock, provided each index update is one atomic store that the compiler cannot reorder (Chapter 27).

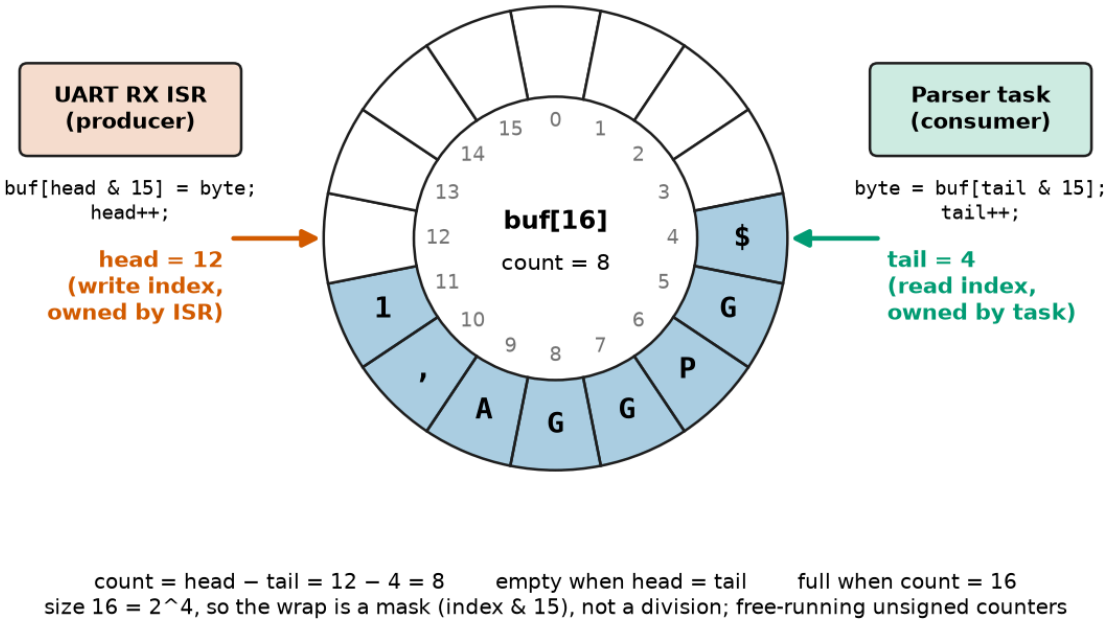


Figure 23.2 — A 16-slot ring buffer between a UART receive interrupt (producer) and a parsing task (consumer).

Listing 23.1 implements it with the same logic as the C firmware version (Chapter 26).

```
class RingBuffer:
    """Single-producer/single-consumer byte FIFO, size a power of two (as in UART firmware)."""
    def __init__(self, size_log2):
        self.buf = bytearray(1 << size_log2)
        self.mask = (1 << size_log2) - 1
        self.head = self.tail = 0              # free-running counters, never reset
        self.dropped = 0
    def put(self, byte):                       # called from the RX interrupt
        if self.head - self.tail == len(self.buf):
            self.dropped += 1                  # full: count the overrun, keep old data
            return
        self.buf[self.head & self.mask] = byte
        self.head += 1
    def get(self):                             # called from the main loop or a task
        if self.head == self.tail:
            return None                        # empty
        byte = self.buf[self.tail & self.mask]
        self.tail += 1
        return byte
    def __len__(self):
        return self.head - self.tail

rb = RingBuffer(4)                             # 16 bytes
for b in b"GPS,$GPGGA,123519,4807.038,N":      # 28-byte burst arrives before the task runs
    rb.put(b)
print(len(rb), "bytes buffered,", rb.dropped, "dropped")
print(bytes(rb.get() for _ in range(len(rb))))
```

```
 16 bytes buffered, 12 dropped
b'GPS,$GPGGA,12351'
```

Listing 23.1 — A power-of-two ring buffer; a 28-byte burst overruns a 16-byte buffer.

Worked example — sizing a UART receive buffer. A GNSS receiver streams NMEA sentences at 921 600 baud (8N1), 92 160 bytes/s at most. The parsing task runs every 10 ms but, under worst-case load from higher-priority tasks, may be delayed by up to 15 ms. The buffer must therefore absorb $92\,160 \times (10 + 15) \times 10^{-3} = 2304$ bytes without overrun. The next power of two is 4096 bytes. At 115 200 baud the same timing needs 288 bytes, so 512. The `dropped` counter of Listing 23.1 turns a silent overrun into a diagnosable fault. DMA reception removes the per-byte interrupt but not the sizing rule.

> **Status of the principle: Ring-buffer sizing**
> *Derived.* A FIFO between a producer of peak rate R and a consumer whose worst-case service interval is T never overflows if its capacity is at least RT; the bound is exact for constant-rate bursts and conservative otherwise, and it fails if T is not a true worst case.

Priority queues and event-driven simulation

A **priority queue** returns the smallest key first. The usual implementation is a **binary heap**, an array in which element i is no larger than its children $2i + 1$ and $2i + 2$; insert and remove-min are $O(\log n)$. Priority queues schedule software timers, merge sorted logs and drive every **event-driven simulator**, including Verilog and VHDL simulators (Chapter 19). Listing 23.2 is the core of a gate-level one: pop the earliest event, update the net, schedule each fan-out gate's new output after its delay.

```
import heapq

# Event-driven logic simulation: (time_ps, sequence, net, new_value)
delay = {"inv": 15, "nand": 22}                 # ps, illustrative gate delays
fanout = {"a": [("n1", "nand")], "b": [("n1", "nand"), ("nb", "inv")],
          "n1": [("y", "inv")], "nb": [], "y": []}
value = {"a": 1, "b": 0, "n1": 1, "nb": 1, "y": 0}

def evaluate(net):
    if net == "n1": return 1 - (value["a"] & value["b"])
    if net == "nb": return 1 - value["b"]
    if net == "y":  return 1 - value["n1"]

events, seq = [], 0
heapq.heappush(events, (100, seq, "b", 1))     # input b rises at t = 100 ps
while events:
    t, _, net, v = heapq.heappop(events)        # always the earliest pending event
    if value[net] == v:
        continue                                # no change, no propagation
    value[net] = v
    print(f"t={t:4d} ps  {net:>2} -> {v}")
    for out, gate in fanout[net]:
        seq += 1
        heapq.heappush(events, (t + delay[gate], seq, out, evaluate(out)))
```

```
 t= 100 ps   b -> 1
t= 115 ps  nb -> 0
t= 122 ps  n1 -> 0
t= 137 ps   y -> 1
```

Listing 23.2 — An event-driven gate-level simulator built on a binary-heap priority queue.

The sequence number breaks ties so that equal-time events run in scheduling order and the run is deterministic. The work is proportional to the number of *changes*, not gates times time steps, which is why simulating a mostly idle million-gate design is fast. Real simulators also model inertial delay, cancelling pulses shorter than a gate delay by removing superseded events.

23.4 Trees and graphs

Trees

A **tree** is a hierarchy of nodes, each with one parent except the root: file systems, Linux device trees, an SoC's design hierarchy (Chapter 34), parse trees of HDL source. As *data structures*, trees keep a collection ordered with $O(\log n)$ search, insertion and deletion:

- A **binary search tree** keeps smaller keys left and larger right. Unbalanced, it degrades to a linked list when keys arrive sorted, as time stamps do; **balanced** variants (red–black, AVL) guarantee $O(\log n)$ height. C++ `std::map` is a red–black tree.
- A **B-tree** stores hundreds of keys per node, so 10^9 keys need only 3–4 levels, one flash or disk page read each: file systems and databases such as SQLite.
- A **trie** branches on successive bits of the key. Routers use compressed tries (or TCAM hardware) for **longest-prefix match** of each packet against about a million IPv4 prefixes in tens of nanoseconds.

- **Spatial trees** (k-d trees, quadtrees, R-trees) let a design-rule checker find all shapes near a given one in $O(\log n + k)$ instead of comparing all $O(n^2)$ pairs of millions of polygons.

Graphs and their representations

A **graph** $G = (V, E)$ is a set of vertices joined by edges, directed or not, possibly weighted. Graphs are the natural model of a circuit (nodes and branches), a netlist (cells joined by nets), a communication network (routers and links), a power grid (buses and lines), a state machine (states and transitions, Chapter 14) and a task dependency set (a build system or a firmware boot sequence). A netlist is really a **hypergraph**, because a net joins many pins, and is usually stored as a bipartite graph of cells and nets.

There are two standard representations. An **adjacency matrix** is an $n \times n$ array with entry (i, j) nonzero when an edge joins i to j: $O(1)$ edge queries, $O(n^2)$ memory. **Adjacency lists** store, for each vertex, the list of its neighbours: $O(n + m)$ memory for m edges, and iteration over a vertex's neighbours in time proportional to its degree. Packed into two arrays, one of offsets and one of neighbour indices, adjacency lists are exactly the compressed sparse row (CSR) format of sparse matrices described in *The Mathematics Universe*, Ch. 24 §24.5, where the graph of a circuit matrix also governs fill-in during factorisation.

Worked example — storing a netlist graph. A mixed-signal block has $n = 10^5$ nodes and $m = 2 \times 10^5$ branches, an average of four connections per node, typical of circuits. An adjacency matrix needs $n^2 = 10^{10}$ entries: 1.25 GB even at one bit per entry, 10 GB at one byte, almost all zeros. In CSR form each undirected edge appears twice, so the neighbour array holds 4×10^5 32-bit indices (1.6 MB) and the offset array $10^5 + 1$ entries (0.4 MB): about 2 MB in total, 5000 times smaller than the bit matrix. Traversing every edge costs $O(n + m) = 3 \times 10^5$ steps against $O(n^2) = 10^{10}$ for scanning the matrix. For engineering graphs, which are sparse with bounded average degree, adjacency lists are the default.

23.5 Hash tables

Constant-time look-up by key

A **hash table** stores key–value pairs in an array of M buckets and places each key at index $h(k) \bmod M$, where the **hash function** h scrambles the key into an integer. Look-up computes $h(k)$, goes to the bucket and compares the few keys found there. If the hash spreads keys evenly, the expected number of comparisons depends only on the **load factor** $\alpha = n/M$, not on n, so insertion, deletion and look-up are $O(1)$ on average. Python's `dict` and `set`, C++ `std::unordered_map`, compiler symbol tables, netlist name look-up, and the CAN message dispatch table in a vehicle gateway all rely on this.

Two keys that land in the same bucket **collide**, and there are two remedies (Figure 23.3). **Separate chaining** keeps a small list per bucket. **Open addressing** stores everything in the array and, on a collision, probes other slots in a fixed sequence (linear probing tries $h, h+1, h+2, \dots$); it is more cache-friendly but degrades sharply as $\alpha \to 1$. CPython's `dict` uses open addressing with a perturbed probe sequence and resizes when it is two-thirds full.

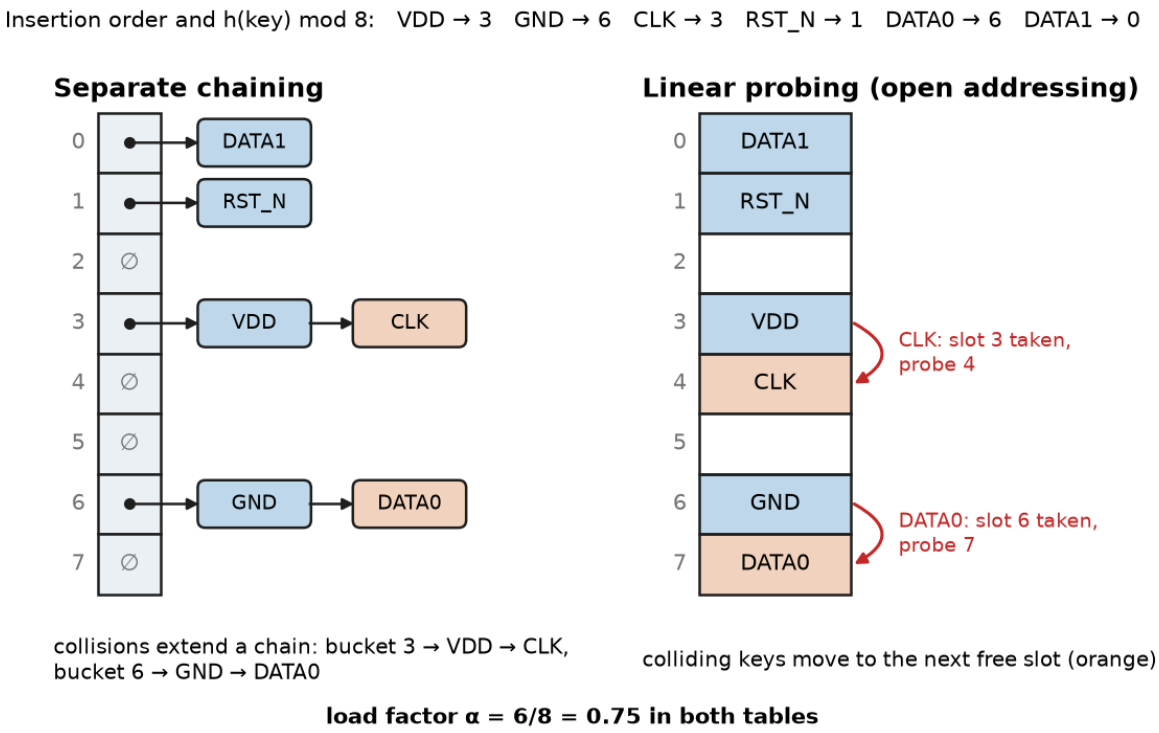


Figure 23.3 — Two ways to resolve hash collisions for net names: separate chaining and linear probing.

Load factor α	Linear probing, successful search	Linear probing, unsuccessful search	Chaining, successful search
0.50	1.5	2.5	1.25
0.75	2.5	8.5	1.38
0.90	5.5	50.5	1.45

Table 23.2 — Expected probes per operation under uniform hashing (Knuth's formulas: $\frac{1}{2}(1+\frac{1}{1-\alpha})$ and $\frac{1}{2}(1+\frac{1}{(1-\alpha)^2})$ for linear probing, $1+\alpha/2$ for chaining).

Table 23.2 shows why every practical table resizes before it is full. At 90 % load a miss under linear probing costs 50 probes, and every insertion is a miss. Resizing doubles M and reinserts everything, an $O(n)$ step that, as with dynamic arrays, is $O(1)$ amortised but long in the worst case.

> **Status of the principle: Expected O(1) hash-table operations**
> *Derived.* Under the assumption of uniform hashing, the expected cost of each operation is a function of the load factor alone; it fails when keys are adversarial or the hash function is poor (all keys in one bucket give $O(n)$), which is why hash tables are avoided where a worst-case bound is required.

Worked example — the overnight netlist check. Listing 23.3 repeats the opening problem at a smaller scale: 20 000 pin names, half of them connected, checked against a list and against a set.

```
 import random, time

random.seed(1)
pins = [f"U{random.randrange(5000)}.{random.randrange(1, 257)}" for _ in range(20_000)]
connected_list = pins[::2]                 # pins that appear on some net
connected_set = set(connected_list)

def unconnected(container):
    t0 = time.perf_counter()
    missing = [p for p in pins if p not in container]
    return len(missing), time.perf_counter() - t0

n_l, t_l = unconnected(connected_list)
n_s, t_s = unconnected(connected_set)
print(f"list: {n_l} unconnected in {t_l:.2f} s")
print(f"set : {n_s} unconnected in {t_s*1e3:.2f} ms  (speed-up ~{t_l/t_s:,.0f}x)")
```

```
 list: 9934 unconnected in 2.58 s
set : 9934 unconnected in 1.18 ms  (speed-up ~2,185x)
```

Listing 23.3 — Membership tests against a list ($O(n)$ each) and a set ($O(1)$ expected each); times from the machine used for this chapter.

With $n = 20\,000$ pins and $n/2$ in the list, the misses scan the whole list and the hits half of it on average: about $\frac{n}{2} \cdot \frac{n}{2} + \frac{n}{2} \cdot \frac{n}{4} = 3n^2/8 = 1.5 \times 10^8$ comparisons, which at the measured 2.6 s is about 17 ns per comparison including loop overhead. The set does 20 000 hash probes in 1.2 ms, 60 ns each. Scaling to the opening's 2×10^6 pins multiplies the list version by $100^2 = 10^4$, to about seven hours, and the set version by 100, to about 0.12 s. The algorithmic speed-up grows linearly with n; at 20 000 pins it is about 2000, at two million about 200 000.

Table hashes (SipHash for Python strings, multiplicative hashes for integers) are chosen for speed and spread; a CRC (Chapter 26) detects transmission errors; a cryptographic hash such as SHA-256 (Chapter 22's run manifests, Git in Chapter 24) resists deliberate collisions.

23.6 Searching and sorting

Searching

Linear search examines elements in turn: $O(n)$, no preconditions, and fastest of all for n below a few dozen, because it streams through cache with no branches mispredicted. **Binary search** on a sorted array halves the interval at each comparison: at most $\lceil \log_2(n+1) \rceil$ comparisons, 12 for 4096 entries and 20 for a million. Hash tables answer "is k present?"; binary search also answers "where would k go?", which is exactly what an interpolating look-up table needs.

Listing 23.4 is the firmware pattern for converting an NTC thermistor's ADC reading to temperature, run here in Python. It tabulates the ADC code every 5 °C from −40 to 120 °C (33 entries, generated from the Steinhart–Hart fit of Chapter 22 with a 10 kΩ pull-up and a 12-bit ADC), finds the bracketing entries by binary search and interpolates linearly.

```
 import bisect, math

A, B, C = 1.130968e-3, 2.338158e-4, 8.892946e-8       # Steinhart-Hart fit from Chapter 22
def r_ntc(T):                                          # invert S-H by bisection on ln R
    lo, hi = math.log(10.0), math.log(1e7)
    for _ in range(60):
        mid = 0.5 * (lo + hi)
        if A + B*mid + C*mid**3 > 1/(T + 273.15): hi = mid
        else: lo = mid
    return math.exp(mid)
code_of = lambda T: 4095 * r_ntc(T) / (10e3 + r_ntc(T))     # 12-bit ADC, 10 k pull-up

temps = list(range(120, -45, -5))                      # 33 entries, code ascending
codes = [code_of(T) for T in temps]

def lookup(code):                                      # O(log n) search + linear interpolation
    k = bisect.bisect_right(codes, code)
    k = min(max(k, 1), len(codes) - 1)
    c0, c1, t0, t1 = codes[k-1], codes[k], temps[k-1], temps[k]
    return t0 + (t1 - t0) * (code - c0) / (c1 - c0)

print(f"code 1234 -> {lookup(1234):.2f} C")
fine = [T / 10 for T in range(-400, 1201)]
err = max(abs(lookup(code_of(T)) - T) for T in fine)
print(f"max interpolation error, -40..120 C: {err:.3f} C over {len(codes)} entries")
```

```
 code 1234 -> 45.34 C
max interpolation error, -40..120 C: 0.168 C over 33 entries
```

Listing 23.4 — Binary search with linear interpolation in a 33-entry thermistor table.

Worked example — three ways to linearise a thermistor in firmware. (a) A direct table indexed by the 12-bit code holds 4096 entries; as `int16` in units of 0.01 °C it occupies 8 KB of flash and needs one memory read, $O(1)$. (b) The 33-entry table of Listing 23.4 occupies 132 bytes (two 16-bit values per entry) and needs at most $\lceil \log_2 34 \rceil = 6$ comparisons plus one multiply and one divide; its worst interpolation error is 0.17 °C, well inside a ±1 % thermistor's ±0.5 °C at 25 °C. (c) Evaluating Steinhart–Hart needs a natural logarithm and a reciprocal in floating point, a few hundred cycles on a Cortex-M4F with FPU and a few thousand with software floating point on a Cortex-M0+. On a 32 KB-flash Cortex-M0+, option (b) wins; on an FPGA with spare block RAM, (a); on an application processor, (c) costs nothing worth measuring.

The interpolation step is the piecewise-linear interpolant whose error bound, $\tfrac{1}{8}h^2 \max|f''|$ for spacing h, is derived in *The Mathematics Universe*, Ch. 22 §22.5. Halving the spacing to 2.5 °C would cut the error about fourfold, to roughly 0.04 °C.

Sorting

Sorting underlies binary search, median filters, merging logs by time stamp and finding duplicate net names. Comparison sorts cannot beat $\Omega(n \log n)$ comparisons in the worst case (there are $n!$ orderings and each comparison halves the possibilities at best, $\log_2 n! \approx n \log_2 n$).

Algorithm	Average	Worst	Extra memory	Stable	Where it is used
Insertion sort	$O(n^2)$	$O(n^2)$	$O(1)$	Yes	$n \lesssim 20$; nearly sorted data; inner step of hybrids
Merge sort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	External sorting of files larger than RAM
Heapsort	$O(n \log n)$	$O(n \log n)$	$O(1)$	No	Guaranteed bound in place; embedded, safety code
Quicksort	$O(n \log n)$	$O(n^2)$	$O(\log n)$	No	C `qsort`; introsort in C++ `std::sort` falls back to heapsort
Timsort	$O(n \log n)$	$O(n \log n)$	$O(n)$	Yes	Python `sorted`, Java objects; exploits existing runs
Counting / radix sort	$O(n + k)$	$O(n + k)$	$O(n + k)$	Yes	Small integer keys: 12-bit ADC codes, 11-bit CAN IDs
Sorting network	$O(\log^2 n)$ depth	Same	—	No	Fixed small n in FPGA or SIMD, data-independent timing

Table 23.3 — Sorting algorithms and their engineering niches.

Three properties beyond speed decide the choice. **Stability** keeps equal keys in their original order, so a log sorted by serial number and then stably by temperature stays in serial order within each temperature. **Worst case** rules out plain quicksort in hard real-time code. **Data-independent timing** (sorting networks) matters in FPGAs and cryptographic code. When keys are small integers, counting sort breaks the $n \log n$ barrier: a histogram of 10^6 12-bit ADC codes, then a cumulative sum, gives the median and any percentile in $O(n + 4096)$ with no comparisons at all.

23.7 Numerical algorithms

> **Boundary.**
> The mathematics of numerical algorithms (floating-point error, root finding by bisection and Newton–Raphson, interpolation, quadrature, ODE solvers, stability and convergence) is in *The Mathematics Universe*, Ch. 22; dense, iterative and sparse linear algebra and running statistics (Welford's algorithm) in Ch. 24 §24.1–24.5. The FFT and Goertzel algorithms are in *Circuits, Signals and Communications*, Ch. 22 §22.4. This section is about turning those methods into code that runs inside fixed budgets of cycles, memory and word length.

From method to implementation

A numerical method on paper becomes a different object in firmware or an FPGA. Five implementation rules recur:

1. **Bound every loop.** A Newton iteration needs a hard iteration cap and a bisection fallback, because a bad initial guess or a NaN can stop convergence; a real-time task iterates at most k times and reports failure.
2. **Prefer recurrences to recomputation.** A moving average kept as a running sum costs two additions per sample, exact in integers; an exponential filter $y \leftarrow y + (x - y)/2^k$ costs a subtract, a shift and an add.
3. **Match the algorithm to the arithmetic.** Without a divider or FPU, shift-and-add algorithms such as CORDIC (below) beat ones with fewer but costlier operations.
4. **Scale for fixed point.** Every intermediate needs a word length that neither overflows nor loses resolution (Chapter 11; *Circuits, Signals and Communications*, Ch. 21 §21.5).
5. **Use a library when one exists.** CMSIS-DSP, vendor FPGA IP, LAPACK and FFTW encode decades of tuning and testing.

CORDIC: rotation by shifts and adds

The **CORDIC** algorithm (COordinate Rotation DIgital Computer, Volder 1959) computes sine, cosine, arctangent, vector magnitude and, in variants, hyperbolic functions, logarithms and square roots, using only additions, subtractions, bit shifts and a small table of constants. It rotates a vector by a sequence of fixed angles $\theta_i = \arctan 2^{-i}$, choosing the direction of each micro-rotation to drive the residual angle z toward zero:

$$x_{i+1} = x_i - d_i\, 2^{-i} y_i, \qquad y_{i+1} = y_i + d_i\, 2^{-i} x_i, \qquad z_{i+1} = z_i - d_i \arctan 2^{-i}, \qquad d_i = \operatorname{sign}(z_i).$$

Each micro-rotation also lengthens the vector by $\sqrt{1 + 2^{-2i}}$, so after N steps the vector is scaled by $\prod_i \sqrt{1 + 2^{-2i}} \approx 1.6468$. Starting from $x_0 = K = 1/1.6468 = 0.607253$ cancels it. Listing 23.5 implements the rotation mode in integer arithmetic, as an FPGA or a DSP without FPU would.

```
 import math

N, FRAC = 16, 18                                  # 16 iterations, 18 fraction bits (32-bit int)
ATAN = [round(math.atan(2.0**-i) * (1 << FRAC)) for i in range(N)]   # ROM table
K = 1.0
for i in range(N):
    K /= math.sqrt(1 + 2.0**(-2 * i))             # CORDIC gain correction, 0.607253

def cordic_sincos(theta):                         # theta in radians, |theta| <= pi/2
    x, y = round(K * (1 << FRAC)), 0              # start on the x axis, pre-scaled by K
    z = round(theta * (1 << FRAC))
    for i in range(N):                            # only shifts, adds and a table look-up
        if z >= 0:
            x, y, z = x - (y >> i), y + (x >> i), z - ATAN[i]
        else:
            x, y, z = x + (y >> i), y - (x >> i), z + ATAN[i]
    return y / (1 << FRAC), x / (1 << FRAC)

print(f"K = {K:.6f}")
worst = max(max(abs(s - math.sin(t)), abs(c - math.cos(t)))
            for t in [k * 0.001 for k in range(-1570, 1571)]
            for s, c in [cordic_sincos(t)])
s, c = cordic_sincos(math.radians(30))
print(f"sin 30 = {s:.5f}, cos 30 = {c:.5f}, worst error = {worst:.2e} = 2^{math.log2(worst):.1f}")
```

```
 K = 0.607253
sin 30 = 0.50001, cos 30 = 0.86603, worst error = 5.92e-05 = 2^-14.0
```

Listing 23.5 — CORDIC sine and cosine with 16 iterations in 32-bit integer arithmetic.

Worked example — CORDIC in a motor-control FPGA. A field-oriented controller (Chapter 32; control theory in *Electrical Energy and Machines*) needs $\sin\theta$ and $\cos\theta$ of the rotor angle for the Park transform every 50 µs. With 16 iterations and 18 fraction bits, Listing 23.5 gives a worst error of $5.9 \times 10^{-5} \approx 2^{-14}$ over $\pm\pi/2$, about 14 correct bits. That fits the rule of thumb of one bit per iteration, since the last table angle is $\arctan 2^{-15} = 3.05 \times 10^{-5}$ rad, and is finer than a 12-bit position sensor's $2\pi/4096 = 1.5 \times 10^{-3}$ rad by a factor of 25. Unrolled and pipelined in an FPGA, the 16 stages need 48 adders of about 20 bits and produce one sine–cosine pair per clock with 16 cycles of latency: at 100 MHz, 160 ns, a negligible part of the 50 µs period. On a Cortex-M4F, CMSIS-DSP's table-based `arm_sin_cos_f32` is the software alternative; on a Cortex-M0+ without FPU, a CORDIC loop is often fastest. Other quadrants are folded into $\pm\pi/2$ by symmetry first.

> **Status of the principle: CORDIC convergence**
> *Derived.* Because $\arctan 2^{-i} \le \sum_{j>i} \arctan 2^{-j} + \arctan 2^{-(N-1)}$, any angle within the convergence range (± 1.743 rad) can be reduced to a residual below $\arctan 2^{-(N-1)}$, so the angular error falls by about one bit per iteration until the word length's rounding error dominates.

23.8 Graph algorithms

Traversal: BFS and DFS

Breadth-first search (BFS) visits vertices in order of their distance in edges from a source, using a FIFO queue; **depth-first search** (DFS) follows each branch as deep as possible, using a stack or recursion. Both are $O(n+m)$ with adjacency lists, and between them they answer the structural questions engineers ask of netlists and networks:

- **Connectivity.** SPICE rejects a **floating node** (no DC path to ground, such as a node joined only through capacitors) because its nodal matrix is singular; a BFS from ground over DC-conducting branches finds such nodes in linear time.
- **Shortest paths in hops.** The **Lee algorithm** for maze routing in PCB and IC layout is a BFS over the routing grid, expanding a wavefront from the source pin to the target.
- **Cycles and topological order.** A combinational network must be acyclic; a loop without a register is a design error that synthesis flags (Chapter 19). An acyclic graph has a **topological order**, every vertex after all its predecessors, found by DFS or Kahn's algorithm in $O(n+m)$.

Static timing analysis as a longest path

The maximum clock frequency of a synchronous design is set by its slowest register-to-register path (Chapters 14 and 34). Enumerating paths is hopeless, since their number grows exponentially with logic depth. But the **arrival time** at each gate output is the maximum over its inputs of their arrival times, plus the gate delay. Processing gates in topological order computes every arrival time in one pass, $O(n+m)$, and the critical path is recovered by remembering which input set each maximum. Listing 23.6 does this for a six-gate network (Figure 23.4).

```
 from graphlib import TopologicalSorter

# Gate-level netlist as a DAG: gate -> (delay_ps, list of driving gates/inputs)
gates = {"g1": (25, ["a", "b"]), "g2": (18, ["b", "c"]), "g3": (32, ["g1", "g2"]),
         "g4": (20, ["g2", "d"]), "g5": (27, ["g3", "g4"]), "g6": (15, ["g4"])}
arrival = {"a": 0, "b": 40, "c": 0, "d": 10}          # input arrival times, ps
outputs, period = ["g5", "g6"], 120                    # clock period, ps

order = TopologicalSorter({g: set(ins) for g, (_, ins) in gates.items()}).static_order()
worst_in = {}
for n in order:                                        # every node after all its fan-in
    if n in gates:
        delay, ins = gates[n]
        worst_in[n] = max(ins, key=lambda i: arrival[i])
        arrival[n] = arrival[worst_in[n]] + delay
for o in outputs:
    path, n = [o], o
    while n in worst_in:
        n = worst_in[n]; path.append(n)
    print(f"{o}: arrival {arrival[o]:3d} ps, slack {period - arrival[o]:4d} ps, "
          f"path {' -> '.join(reversed(path))}")
```

```
g5: arrival 124 ps, slack   -4 ps, path b -> g1 -> g3 -> g5
g6: arrival  93 ps, slack   27 ps, path b -> g2 -> g4 -> g6
```

Listing 23.6 — Arrival times, slack and critical path by one pass in topological order.

Worked example — finding the timing violation. Input `b` arrives late, at 40 ps (it comes from a slow register in another block). Gate `g1` sees inputs at 0 and 40 ps, so its output arrives at $40 + 25 = 65$ ps; `g2` at $40 + 18 = 58$ ps; `g3` at $\max(65, 58) + 32 = 97$ ps; `g4` at $\max(58, 10) + 20 = 78$ ps; `g5` at $\max(97, 78) + 27 = 124$ ps; `g6` at $78 + 15 = 93$ ps. Against a 120 ps clock period (8.33 GHz in this illustrative example; scale the delays by 10 for a 833 MHz block) output `g5` has slack $120 - 124 = -4$ ps: the design fails timing, and the critical path is `b → g1 → g3 → g5`. Speeding up `g4` would gain nothing, because it is not on that path; upsizing `g3` by 5 ps would fix it. Commercial static timing analysers do exactly this pass, with delays that depend on load capacitance and input slew, on graphs of 10^8–10^9 timing arcs, followed by a backward pass of required times to give every node its slack (Chapter 34).

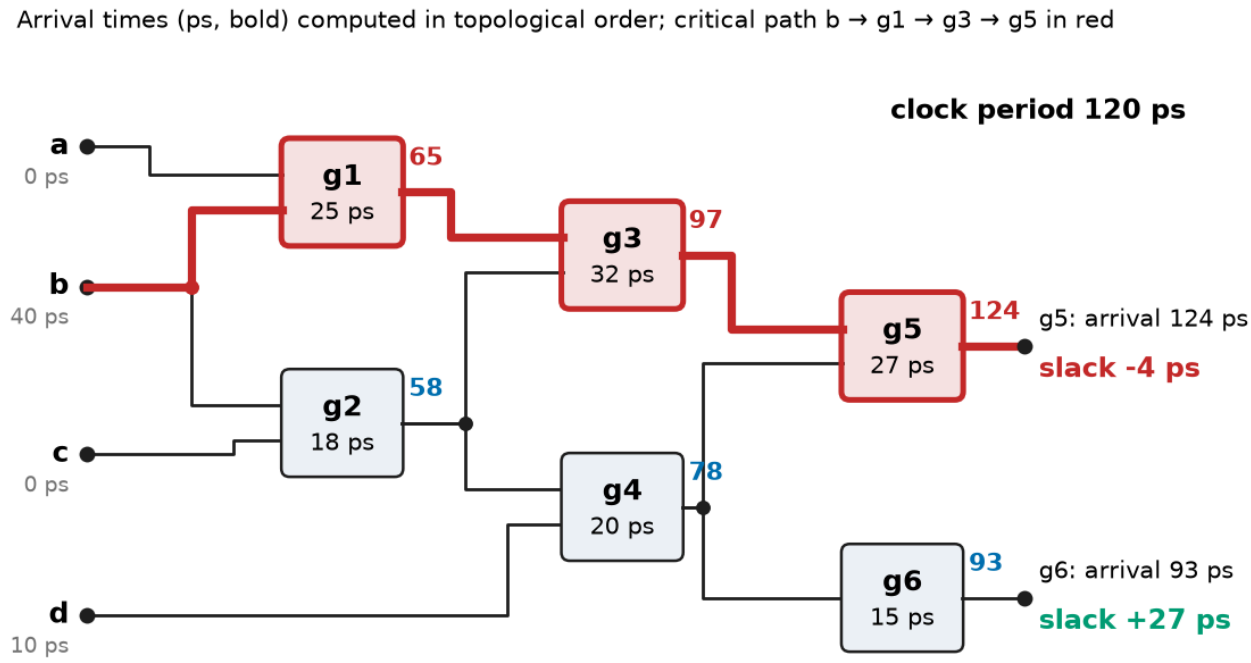


Figure 23.4 — Six-gate combinational network with arrival times computed in topological order; the critical path to g5 is highlighted.

Shortest weighted paths: Dijkstra

When edges carry non-negative weights (link costs, wire lengths, delays), **Dijkstra's algorithm** finds the cheapest path from a source to every vertex. It keeps a priority queue of tentative distances, repeatedly settles the closest unsettled vertex, and relaxes its outgoing edges. With a binary heap the cost is $O((n + m) \log n)$. Link-state routing protocols (OSPF in enterprise and data-centre networks, IS-IS in service-provider backbones) run Dijkstra on every router: each router floods its link costs, every router builds the same graph, and each computes its own shortest-path tree. Listing 23.7 does so for six routers with OSPF costs derived from a 10 Gb/s reference bandwidth, so that a 10 Gb/s link costs 1, a 1 Gb/s link 10 and a 100 Mb/s link 100 (Figure 23.5).

```
 import heapq

# OSPF-style costs: reference 10 Gb/s -> 10G link = 1, 1G = 10, 100M = 100
links = [("R1", "R2", 10), ("R1", "R3", 1), ("R2", "R4", 10), ("R3", "R4", 10),
         ("R3", "R5", 100), ("R4", "R6", 10), ("R5", "R6", 10), ("R2", "R5", 10)]
graph = {}
for u, v, c in links:                                   # undirected adjacency list
    graph.setdefault(u, []).append((v, c))
    graph.setdefault(v, []).append((u, c))

def dijkstra(graph, src):
    dist, prev = {src: 0}, {}
    pq = [(0, src)]
    while pq:
        d, u = heapq.heappop(pq)
        if d > dist[u]:
            continue                                    # stale entry: a shorter path was found
        for v, c in graph[u]:
            if d + c < dist.get(v, float("inf")):
                dist[v], prev[v] = d + c, u
                heapq.heappush(pq, (d + c, v))
    return dist, prev

dist, prev = dijkstra(graph, "R1")
for dst in sorted(dist):
    path, n = [dst], dst
    while n in prev:
        n = prev[n]; path.append(n)
    print(f"{dst}: cost {dist[dst]:3d}  via {' -> '.join(reversed(path))}")
```

```
 R1: cost   0  via R1
R2: cost  10  via R1 -> R2
R3: cost   1  via R1 -> R3
R4: cost  11  via R1 -> R3 -> R4
R5: cost  20  via R1 -> R2 -> R5
R6: cost  21  via R1 -> R3 -> R4 -> R6
```

Listing 23.7 — Dijkstra's algorithm with a binary heap computing an OSPF shortest-path tree.

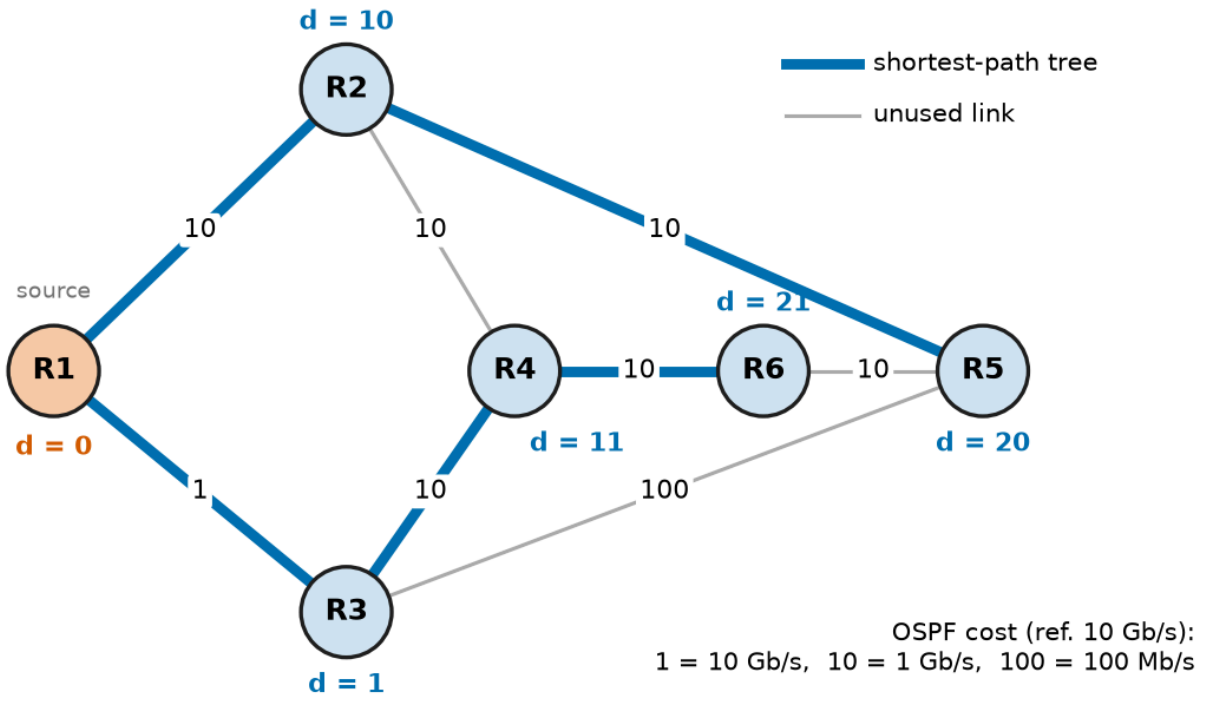


Figure 23.5 — Six-router network with OSPF link costs and the shortest-path tree computed by Dijkstra from R1.

Worked example — rerouting after a link failure. From R1, traffic to R6 takes R1→R3→R4→R6 at cost $1 + 10 + 10 = 21$, and to R5 takes R1→R2→R5 at cost 20, avoiding the direct 100 Mb/s link R3–R5 (cost 101 by that route). If the 1 Gb/s link R3–R4 fails, every router recomputes: R4 is now reached via R2 at $10 + 10 = 20$, and R6 either via R4 ($20 + 10 = 30$) or via R5 ($20 + 10 = 30$), an equal-cost tie that OSPF can split across both paths (equal-cost multipath). A data-centre fabric of 1000 switches and 20 000 links needs about $(n + m)\log_2 n \approx 2 \times 10^5$ heap operations, well under a millisecond on a switch CPU, so link-state protocols reconverge in tens of milliseconds. With negative weights, Bellman–Ford ($O(nm)$) replaces Dijkstra.

> **Status of the principle: Correctness of Dijkstra's algorithm**
> *Derived.* With non-negative edge weights, the closest unsettled vertex cannot be reached more cheaply through any unsettled vertex, so its tentative distance is final when it is removed from the queue; a single negative edge breaks this argument.

Union–find and spanning trees

The **union–find** (disjoint-set) structure maintains a partition of elements into groups, with `find(x)` returning a group's representative and `union(a, b)` merging two groups. With path compression and union by rank, a sequence of m operations runs in $O(m\,\alpha(n))$, where the inverse Ackermann function $\alpha(n)$ is below 5 for any n that fits in the universe. Layout extraction uses it to turn millions of overlapping copper shapes and vias into electrical nets (§23.10). **Kruskal's algorithm** sorts edges by weight and adds each one that joins two different union–find groups, producing a **minimum spanning tree**: the cheapest set of edges connecting all vertices. Placement tools use spanning trees (and their Steiner-tree refinements, which may add junction points) to estimate the wire length of each net before routing.

23.9 Algorithm selection

The questions to ask

Choosing an algorithm is an engineering trade-off among measurable properties, not a search for the "best" one. Before writing code, answer these questions:

1. **How large is n, now and in five years?** A netlist tool written for 10 000 nets will meet a billion; an MCU calibration table will always hold 33 entries.
2. **Worst case or average?** Hard real-time code, safety functions and adversarial input (network packets) need worst-case bounds; batch analysis can use average-case winners such as quicksort and hash tables.
3. **What does memory cost?** On a 32 KB MCU, $O(n)$ extra memory may be impossible; on a server, trading memory for time is usually right.
4. **Static or dynamic data? Is order needed?** Built-once tables favour sorted arrays and binary search; frequent inserts favour balanced trees or hash tables; range queries need a sorted structure.
5. **Does a tested library already do it?** Python's `sorted`, `bisect`, `heapq`, `dict`, `graphlib`; C++ `<algorithm>`; NetworkX; CMSIS-DSP.

Problem	Small n or MCU	Large n, average case	Hard worst-case bound
Membership by key	Linear search in array	Hash table	Sorted array + binary search, balanced tree
Ordered map with inserts	Sorted array	Balanced tree, B-tree	Balanced tree
Sorting	Insertion sort, sorting network	Timsort, introsort	Heapsort, merge sort
Median or percentile	Sorting network; insertion into sorted window	Quickselect $O(n)$ average	Counting sort (small keys), median of medians
FIFO between ISR and task	Ring buffer	Ring buffer	Ring buffer, sized by RT
Scheduling by time	Sorted list of timers	Binary heap	Timer wheel ($O(1)$ per tick)
Shortest path	BFS (unweighted)	Dijkstra with heap	Dijkstra (bounded graph size)
Function evaluation	Table + interpolation, CORDIC	Library math	Table, CORDIC (fixed iteration count)

Table 23.4 — Typical choices by regime.

Where the asymptotics mislead

Complexity classes guide the choice; measurement confirms it. Three effects regularly reverse naive predictions. **Constants and cache**: below a few hundred elements a linear scan of an array often beats a hash table or tree, which is why library sorts switch to insertion sort below 16–32 elements. **Interpreter overhead**: a vectorised $O(n \log n)$ NumPy call beats a pure-Python $O(n)$ loop for any practical n (Chapter 22). **Input structure**: logs are nearly sorted (Timsort exploits this), and netlists have a few huge nets (power, clock) among millions of two-pin nets, so the *average* degree hides the worst. Profiling (Chapter 24) shows which effect dominates.

23.10 Engineering applications

Net extraction from layout

A layout's copper consists of millions of polygons on several layers, joined through vias. Extraction must decide which pins are electrically connected. Each overlap between shapes, found with a spatial index (§23.4), becomes a `union`; afterwards, each union–find group is a net. Listing 23.8 shows the final step on a small board fragment and flags single-pin nets, which are either intentional test points or open circuits.

```
 parent = {}
def find(x):                                    # root of x's set, with path halving
    parent.setdefault(x, x)
    while parent[x] != x:
        parent[x] = parent[parent[x]]
        x = parent[x]
    return x
def union(a, b):
    parent[find(a)] = find(b)

# Copper connections extracted from a layout: (pin or via, pin or via)
segments = [("U1.3", "V7"), ("V7", "R4.1"), ("R4.2", "V9"), ("V9", "C2.1"),
            ("C2.2", "GND"), ("U1.7", "GND"), ("R4.2", "U1.5"), ("J1.1", "V12")]
pins = ["U1.3", "U1.5", "U1.7", "R4.1", "R4.2", "C2.1", "C2.2", "J1.1", "J1.2"]
for a, b in segments:
    union(a, b)

nets = {}
for p in pins:
    nets.setdefault(find(p), []).append(p)
for k, (root, members) in enumerate(sorted(nets.items(), key=lambda kv: kv[1])):
    flag = "  <-- single-pin net" if len(members) == 1 else ""
    print(f"net {k}: {members}{flag}")
```

```
 net 0: ['J1.1']  <-- single-pin net
net 1: ['J1.2']  <-- single-pin net
net 2: ['U1.3', 'R4.1']
net 3: ['U1.5', 'R4.2', 'C2.1']
net 4: ['U1.7', 'C2.2']
```

Listing 23.8 — Union–find turns extracted copper connections into nets and flags single-pin nets.

Connector J1 is reported as two single-pin nets: pin 1 reaches a via (V12) that goes nowhere, and pin 2 is not connected at all. Comparing the extracted nets with the schematic's netlist, a **layout-versus-schematic** (LVS) check, is itself a graph problem: deciding whether two netlists are the same circuit up to renaming is graph isomorphism, solved in practice by iterative partition refinement of device and net "signatures" (Chapter 34).

More of the field

Domain	Problem	Structures and algorithms
Automotive gateway	Route CAN frames between buses by 11- or 29-bit identifier	Hash table or direct-indexed table (2048 entries for 11-bit IDs); hardware acceptance filters as bit masks (ISO 11898, Chapter 26)
Battery management	Choose which of 96 series cells to bleed for balancing	Partial selection of the k highest voltages, $O(n \log k)$ with a heap
Telecom	Longest-prefix match on every packet	Compressed tries, TCAM hardware
Instrumentation	Trigger on a pattern in a 1 GS/s stream	Finite automaton in FPGA logic; ring buffer of pre-trigger samples
Power systems	Islanding detection after breaker trips	Connected components of the bus–line graph
EDA	Logic simulation, timing, placement, routing	Event queues, topological order, spanning and Steiner trees, maze routing
Embedded RTOS	Ready list, delayed tasks, software timers	Intrusive linked lists, priority bitmaps, timer wheels or heaps

The thread through all of them is the one in the opening: the data structure fixes which operations are cheap, the algorithm decides how many of them are needed, and the platform (an interrupt deadline, a flash budget, a cache line, a billion-element netlist) decides which costs matter.

What remains open

Open question	Where it is answered
How are large matrix computations, parallel execution, profiling and floating-point precision handled at scale?	Chapter 24
How are ring buffers, linked lists and memory pools written in C with interrupts and DMA?	Chapters 21 and 26
How do schedulers use priority queues, ready lists and timer wheels, and how are worst-case execution times bounded?	Chapter 27
How do synthesis, placement, routing and static timing analysis tools apply these algorithms to real chips?	Chapter 34
How are algorithms mapped onto FPGA hardware, and when does a hardware sorter or CORDIC pay off?	Chapters 18 and 35
What are the mathematical foundations of complexity, recurrences and sparse-matrix graphs?	*The Mathematics Universe*, Ch. 24
How do the FFT and Goertzel algorithms work, and how are digital filters structured for efficiency?	*Circuits, Signals and Communications*, Ch. 22 and Ch. 23

Next: Chapter 24 scales these ideas up to scientific computing: the environments, matrix libraries, data formats, parallel hardware, profiling tools, precision choices, tests and version-control workflows with which engineering simulations are built, run and trusted.

Chapter 24 — Scientific Computing

Part V · Programming and Computational Systems

A power-electronics team runs a Monte Carlo study of a 48 V to 12 V converter: 4000 component draws at five temperatures, each a 40-second transient simulation. On the lead engineer's laptop it takes nine hours. A colleague parallelises it across the 32 cores of a workstation and gets a speed-up of three, not thirty: every worker process also started eight BLAS threads, and 256 threads fought over 32 cores. When the run finally finishes, the yield is 91.2 %; a rerun on the build server gives 90.7 %, and nobody can say which is right, because the random seed, the library versions and two edited model files were never recorded. A month later a "small cleanup" of the post-processing script silently changes the efficiency numbers in the design report, and no test notices. None of these failures is a physics or circuit problem. Each is a failure of the computing practice around the engineering: matrix libraries, parallel execution, profiling, numerical precision, reproducibility, testing and version control. This chapter collects that practice.

> **How are large engineering computations organised so that they run fast on real hardware: matrix libraries, sparse solvers, parallel cores and GPUs, profiling?**
> **How are their results made trustworthy: precision choices, recorded environments, automated tests and version-controlled collaboration?**

Concept	Engineering destination
Numerical computing environments	Choosing between Python, MATLAB, Julia and compiled code for a team and a task
Matrix computation	Circuit and power-grid solvers, state-space models, FEM thermal and field models
Scientific data formats	Exchanging S-parameters, waveforms, disturbance records and simulation output
Simulation workflows	Corner, sweep and Monte Carlo campaigns; batch SPICE; co-simulation
Parallel computing fundamentals	Speeding up yield studies, EM solvers and design-space exploration
Performance profiling	Finding the 10 % of code that takes 90 % of the time before optimising
Numerical precision	float32 versus float64 on GPUs, phase accumulators, simulator tolerances
Reproducibility	Qualification reports that can be regenerated and audited
Software testing	Catching regressions in analysis scripts, models and firmware tools
Version control and collaboration	Shared code, models and data across a team with review and CI

Roadmap. The environments (24.1) → dense and sparse matrix computation on real hardware (24.2) → data formats for exchange and archive (24.3) → simulation workflows (24.4) → parallel computing and its limits (24.5) → profiling (24.6) → precision as a design choice (24.7) → reproducibility at campaign scale (24.8) → testing (24.9) → Git and collaborative workflows (24.10).

24.1 Numerical computing environments

> **Boundary.**
> The mathematics of numerical computation (floating-point arithmetic, conditioning and stability, the cost and accuracy of LU, QR, SVD and eigenvalue algorithms, iterative and sparse solvers, Monte Carlo, reproducibility and the testing of numerical code) is in *The Mathematics Universe*, Ch. 22 and Ch. 24. Python as a laboratory tool is Chapter 22 of this book, and algorithms and data structures Chapter 23. This chapter covers the engineering practice of computing at scale.

An engineering computation lives in an **environment**: a language, its numerical libraries, an editor or notebook, and the tooling for packages, tests and builds. All the mainstream environments call the same compiled kernels underneath (BLAS and LAPACK for linear algebra, FFTW-class FFTs, SUNDIALS-class ODE solvers), so for large array operations their speed is similar. They differ in licensing, ecosystem, speed of user-written loops, and fit to the engineering domain.

Environment	Strengths	Weaknesses	Typical EEE use
Python (NumPy, SciPy, pandas, Jupyter)	Free; huge ecosystem; instrument control, ML, web, automation in one language	Interpreted loops slow; packaging needs discipline	Test automation, data analysis, ML, scripting EDA tools
MATLAB / Simulink	Polished toolboxes (control, signal, power systems, HDL Coder, code generation for MCUs); industry standard in control and automotive	Licence cost; closed source; loops historically slow (now JIT-compiled)	Control design, model-based design, Simulink to embedded C
Julia	Compiled (LLVM JIT) loops as fast as C with MATLAB-like syntax; strong differential-equation and optimisation packages	Smaller ecosystem; first-run compilation latency	Large ODE/DAE models, power-system dynamics, optimisation
C, C++, Fortran with libraries (Eigen, LAPACK, PETSc, Trilinos)	Full control and speed; HPC standard	Slow to write; memory errors	Production solvers (SPICE engines, FEM codes), embedded targets

Mixed environments are the norm: Python calls compiled cores through **Cython**, **pybind11** or **f2py**, and **Numba** JIT-compiles numerical functions. A loop that runs 10^8 times must run in compiled code; almost nothing else needs to.

24.2 Matrix computation

BLAS, LAPACK and their implementations

Nearly every engineering simulation ends in linear algebra: a circuit's nodal matrix at every Newton step, a power-flow Jacobian, a finite-element stiffness matrix. The standard interfaces are **BLAS** (Basic Linear Algebra Subprograms) and **LAPACK** (Linear Algebra PACKage). BLAS defines three levels of kernels, vector–vector, matrix–vector and matrix–matrix; why only level 3 can reuse data in cache, and the blocked algorithms that recast LU and QR in level-3 form, are explained in *The Mathematics Universe*, Ch. 24 §24.1, with the flop counts of each factorisation in §24.3. LAPACK names its routines by a code: a precision letter (`s`, `d`, `c`, `z` for single, double, complex single, complex double), a matrix type (`ge` general, `po` symmetric positive definite, `sy` symmetric, `gb` banded, `tr` triangular) and an operation (`sv` solve, `trf` factor, `trs` solve with factors, `ev` eigenvalues, `svd`). So `dgesv` solves a general double-precision system by LU, `dpotrf` computes a Cholesky factor, and `zgeev` finds eigenvalues of a complex matrix. NumPy's `linalg.solve` calls `dgesv`.

Implementations (**OpenBLAS**, the NumPy default; **Intel oneMKL**; **AOCL-BLIS**; **Apple Accelerate**; **cuBLAS** on GPUs) tune the kernels to each processor, can differ by a factor of two on the same hardware, and are multithreaded by default.

Peak, bandwidth and the roofline

A processor's **peak floating-point rate** is cores × clock × floating-point operations per cycle per core. A core with AVX2 and two 256-bit fused multiply–add (FMA) units does $2 \times 4 \times 2 = 16$ double-precision flops per cycle; a server core with two 512-bit FMA units does 32. The laptop used for this chapter, a four-core Core i7-1165G7 at a 2.8 GHz base clock with 16 double-precision flops per cycle per core, has a nominal peak of $4 \times 2.8 \times 16 = 179$ GFLOP/s. Its memory delivers roughly 47 GB/s in practice. The **roofline model** combines the two: a kernel that performs I flops per byte moved from memory (its **arithmetic intensity**) can run no faster than

$$P_{\text{attainable}} = \min\left(P_{\text{peak}},\ I \cdot B_{\text{mem}}\right).$$

> **Status of the principle: Roofline bound**
> *Approximate.* A kernel cannot exceed either the arithmetic peak or the rate at which memory delivers its operands; the bound ignores cache reuse effects beyond the chosen memory level, latency limits and instruction overheads, so measured performance lies below the roof, often well below it for irregular access.

Listing 24.1 measures three kernels on that laptop.

```
 import time
import numpy as np

def gflops(f, flops, reps=5):
    f(); best = min(timed(f) for _ in range(reps))     # warm up, then best of reps
    return flops / best / 1e9, best
def timed(f):
    t0 = time.perf_counter(); f(); return time.perf_counter() - t0

rng = np.random.default_rng(0)
n = 3000
A, B = rng.standard_normal((n, n)), rng.standard_normal((n, n))
x, b = rng.standard_normal(n), rng.standard_normal(n)
for name, f, fl in [("dgemv  y = A x", lambda: A @ x, 2 * n**2),
                    ("dgemm  C = A B", lambda: A @ B, 2 * n**3),
                    ("dgesv  A x = b", lambda: np.linalg.solve(A, b), 2 * n**3 / 3)]:
    g, t = gflops(f, fl)
    print(f"{name}: {t*1e3:8.2f} ms  {g:6.1f} GFLOP/s")
```

```
 dgemv  y = A x:     1.54 ms    11.7 GFLOP/s
dgemm  C = A B:   617.59 ms    87.4 GFLOP/s
dgesv  A x = b:   487.47 ms    36.9 GFLOP/s
```

Listing 24.1 — Measured rates of matrix–vector product, matrix–matrix product and LU solve (OpenBLAS, four-core laptop; numbers vary by machine and run).

Worked example — reading the roofline. The matrix–vector product reads each of the n^2 doubles of A once (8 bytes) and does 2 flops with it, so $I = 0.25$ flop/byte. The roofline predicts at most $0.25 \times 47 = 11.7$ GFLOP/s, which is what the measurement shows: DGEMV is memory-bound, and a faster processor with the same memory would not speed it up. The matrix–matrix product performs $2n^3 = 5.4 \times 10^{10}$ flops on $3n^2$ doubles (216 MB), an intensity of $2n^3/(24n^2) = n/12 = 250$ flop/byte, far to the right of the ridge point $179/47 \approx 3.8$ flop/byte (Figure 24.1). It is compute-bound and reaches 87 GFLOP/s, about half the nominal peak. The shortfall is normal for a laptop that lowers

its clock under sustained vector load. LU with partial pivoting achieves 37 GFLOP/s at this size, because its panel factorisation steps are level-2 operations; the fraction rises with n. The practical consequences are general. A sparse matrix–vector product (about 0.1 to 0.2 flop/byte in CSR double precision), an FFT (about 1 flop/byte) and an explicit time-stepping stencil are bandwidth-bound everywhere. Their speed is improved by touching memory less (blocking, fusing operations, using `float32`), not by adding cores beyond the point where the memory bus saturates.

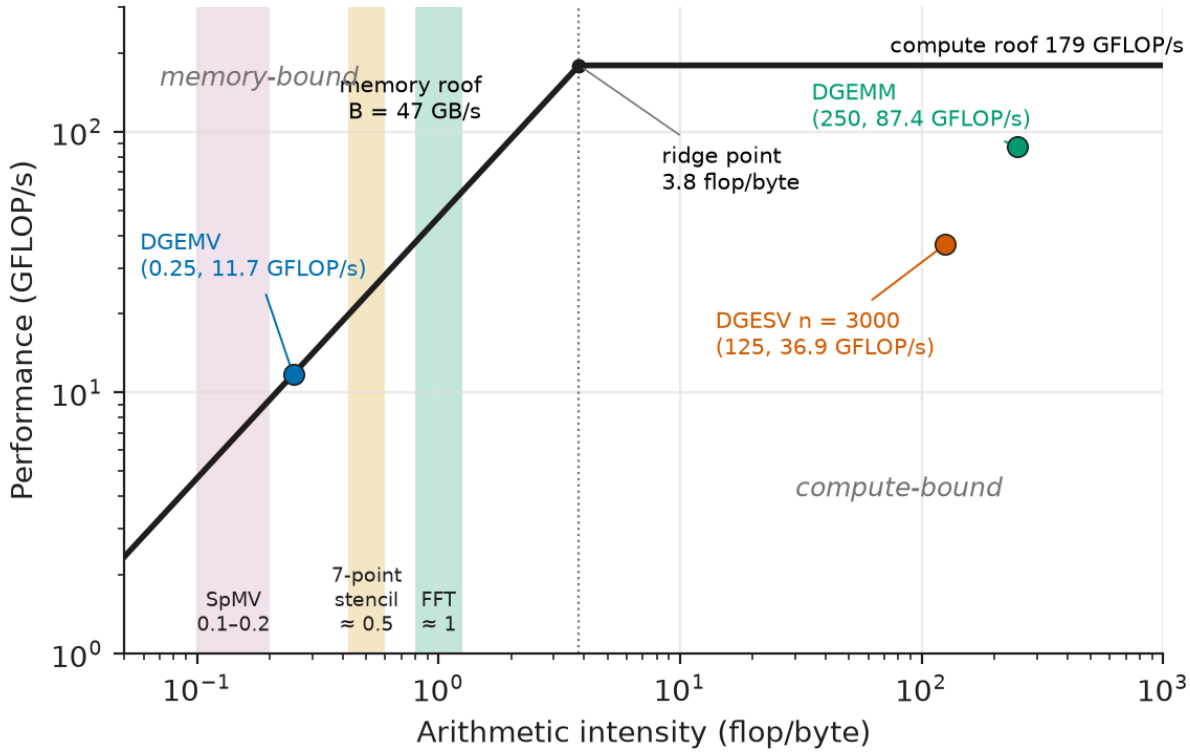


Figure 24.1 — Roofline of the four-core laptop used in this chapter with the measured kernels of Listing 24.1.

Factor once, solve many

The most valuable matrix-computation habit in engineering simulation is to separate **factorisation** from **solution**. LU costs about $\frac{2}{3}n^3$ flops; each subsequent solve with the stored factors costs $2n^2$ (*The Mathematics Universe*, Ch. 24 §24.3, cost summary). A linear circuit integrated with a fixed time step, a frequency sweep at one frequency with many excitations, a sensitivity analysis and an adjoint computation all solve the same matrix with many right-hand sides.

Worked example — a fixed-step transient of a linear network. A linear RC interconnect model with $n = 1000$ nodes is integrated for 1000 time steps with backward Euler at a fixed step, so the system matrix $G + C/h$ never changes. Calling `solve` at every step costs $1000 \times \frac{2}{3} \times 10^9 = 6.7 \times 10^{11}$ flops. Calling `scipy.linalg.lu_factor` once and `lu_solve` at each step costs $\frac{2}{3} \times 10^9 + 1000 \times 2 \times 10^6 = 2.7 \times 10^9$ flops, 250 times less. At the 37 GFLOP/s measured above, that is 18 s against 0.07 s. SPICE engines go one step further. A nonlinear circuit changes its matrix *values* at every Newton iteration but not its *sparsity pattern*, so the ordering and symbolic factorisation are computed once and only the numeric factorisation is repeated (*The Mathematics Universe*, Ch. 24 §24.5).

Sparse matrices in practice: on-chip IR drop

Engineering matrices are sparse: a node connects to a handful of neighbours. Storage formats (CSR, CSC, COO), fill-reducing orderings and the choice between sparse direct and preconditioned iterative solvers are developed in *The Mathematics Universe*, Ch. 24 §24.4–24.5. Listing 24.2 applies them to a problem every chip team solves before tape-out: the static **IR drop** of an on-chip power grid. The top-metal VDD mesh of a 3 mm × 3 mm die is modelled as a 300 × 300 grid of nodes at 10 µm pitch with 0.1 Ω per segment. Flip-chip VDD bumps every 300 µm hold their nodes at 0.80 V, and the logic draws 4 A spread uniformly plus an extra 1 A in a 300 µm × 300 µm hot spot.

```
 import numpy as np
import scipy.sparse as sp
from scipy.sparse.linalg import spsolve, cg

k, R_seg, vdd = 300, 0.1, 0.80              # 300 x 300 grid, 10 um pitch, 0.1 ohm per segment
n = k * k
idx = np.arange(n).reshape(k, k)
pads = np.zeros((k, k), bool); pads[15::30, 15::30] = True     # VDD bumps every 300 um
I = np.full((k, k), 4.0 / n)                                    # 4 A spread uniformly
I[100:130, 180:210] += 1.0 / 900                                # +1 A hot spot, 300 x 300 um

# Conductance (graph Laplacian) of the mesh, assembled in COO form
g = 1 / R_seg
right = (idx[:, :-1].ravel(), idx[:, 1:].ravel()); down = (idx[:-1, :].ravel(), idx[1:, :].ravel())
rows = np.concatenate([right[0], down[0]]); cols = np.concatenate([right[1], down[1]])
G = sp.coo_matrix((np.full(rows.size, -g), (rows, cols)), shape=(n, n))
G = (G + G.T).tocsr()
G = G - sp.diags(np.asarray(G.sum(axis=1)).ravel())            # row sums zero: KCL
# Pads held at VDD: eliminate them, solve for the drop d = VDD - v at the free nodes
free = ~pads.ravel()
Gff = G[free][:, free].tocsc()
d = np.zeros(n)
d[free] = spsolve(Gff, I.ravel()[free])
it = []
d_cg, info = cg(Gff, I.ravel()[free], rtol=1e-10, maxiter=5000, callback=lambda xk: it.append(1))
print(f"unknowns {Gff.shape[0]}, nonzeros {Gff.nnz}, density {Gff.nnz/Gff.shape[0]**2:.1e}")
r, c = np.unravel_index(d.argmax(), (k, k))
print(f"worst IR drop {d.max()*1e3:.1f} mV ({100*d.max()/vdd:.1f} % of VDD) at node ({r}, {c})")
print(f"CG: {len(it)} iterations, max difference {np.abs(d_cg - d[free]).max()*1e6:.3f} uV")
```

```
 unknowns 89900, nonzeros 447900, density 5.5e-05
worst IR drop 19.4 mV (2.4 % of VDD) at node (118, 194)
CG: 778 iterations, max difference 0.000 uV
```

Listing 24.2 — Static IR drop of a 300 × 300 on-chip power grid by sparse direct and conjugate-gradient solves.

Worked example — IR-drop sign-off. The grid has 89 900 unknown node voltages (90 000 nodes less 100 bump nodes) and five nonzeros per row; only 5.5×10^{-5} of the matrix is nonzero. Stored densely it would need $89\,900^2 \times 8$ bytes = 64.7 GB, and LU would take $\frac{2}{3}n^3 = 4.8 \times 10^{14}$ flops, about 1.7 hours at 80 GFLOP/s. The sparse direct solve takes a few seconds on the laptop, and conjugate gradients, valid because the conductance matrix is symmetric positive definite, agrees to well under a microvolt after 778 unpreconditioned iterations. The worst drop, 19.4 mV or 2.4 % of the 0.80 V supply, sits inside the hot spot, midway between two bumps (Figure 24.2); away from it the uniform load alone dips only about 2.7 mV between bumps. A typical static budget is 2–3 % of VDD, with a similar allowance for dynamic droop, so this design is marginal: the fix is a bump, or a wider top-metal strap, over the hot block. Production tools solve the same equations for 10^8–10^9 nodes with multigrid-preconditioned CG. The physics is Kirchhoff's current law, *Circuits, Signals and Communications*, Ch. 2; the engineering is in the sparse linear algebra.

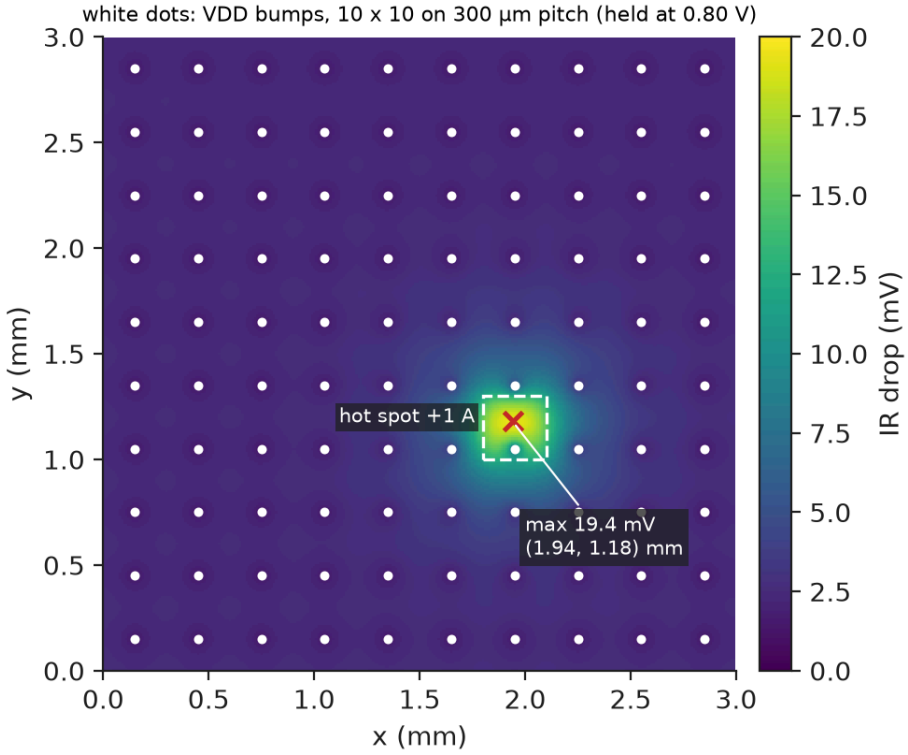


Figure 24.2 — Static IR-drop map of the 300 by 300 on-chip VDD grid of Listing 24.2, with the hot spot and bump locations.

24.3 Scientific data formats

What a scientific format must carry

A data file is useful only if another program, years later, can read it correctly: the numbers in a defined binary layout (type, byte order, shape), the **metadata** that gives them meaning (units, sample rate, axes, reference impedance), and efficient partial access to large datasets. Formats that carry their own structure and metadata are **self-describing**. Chapter 22 compared CSV, JSON, NPY and HDF5 for laboratory data; Table 24.1 adds the formats an EEE engineer meets in exchange with tools, vendors and other disciplines.

Format	Domain	Standard or owner	Content and notes
HDF5	General scientific	The HDF Group	Hierarchical groups, chunked compressed arrays, attributes; parallel I/O via MPI; basis of MATLAB v7.3 `.mat` and NetCDF-4
NetCDF-4 with CF conventions	Climate, weather, ocean, increasingly power-system resource data	Unidata; CF conventions	Gridded data with named dimensions and coordinate variables; wind and solar resource datasets
Parquet / Arrow	Large tables	Apache	Columnar, compressed, typed; fleet telemetry, production-test databases
Touchstone (`.sNp`) v1.1 / v2.0	RF and signal integrity	IBIS Open Forum	*N*-port S-, Y- or Z-parameters versus frequency, reference impedance in the header
IBIS / IBIS-AMI	I/O buffer models	IBIS Open Forum (ANSI/EIA-656)	Driver and receiver I–V and V–t tables; SerDes equalisation models
SPICE raw	Circuit simulation output	De facto (Berkeley SPICE, ngspice, LTspice variants)	Header plus binary or ASCII vectors of node voltages and branch currents
VCD, FST	Digital waveforms	IEEE 1364 (VCD); GTKWave (FST)	Value changes with time stamps; VCD is text and huge, FST compressed
COMTRADE	Power-system disturbance records	IEEE C37.111 / IEC 60255-24	Relay and fault-recorder waveforms: configuration file plus binary data
EDF / EDF+	Biomedical signals	Open specification	Multichannel EEG, ECG, polysomnography; DICOM for imaging
GDSII, OASIS	IC layout	De facto; SEMI P39 (OASIS)	Hierarchical polygons per layer; the tape-out deliverable

Table 24.1 — Data formats in EEE scientific and engineering computing.

Three practical rules cover most of the pitfalls. **Prefer the domain standard** over a private format: a Touchstone file opens in every RF tool, and a COMTRADE record in every protection-analysis package. **Never lose the metadata**: a Touchstone file whose header says `# GHZ S MA R 50` has its frequency unit, parameter type, number format and reference impedance in that one line, and a script that ignores it will misread every value. **Check binary layout explicitly** when reading foreign files: byte order (little-endian on x86 and Arm, big-endian in some older instrument formats and in network protocols), integer width, and whether arrays are stored row-major (C, NumPy) or column-major (Fortran, MATLAB).

24.4 Simulation workflows

From one simulation to a campaign

A single simulation answers a single question. Engineering decisions need **campaigns**: parameter sweeps for design-space exploration, **corner analysis** at process, voltage and temperature extremes (an IC is signed off at slow and fast process corners, ±10 % supply and −40 to 125 °C junction temperature), **Monte Carlo** analysis for yield (statistical method in *The Mathematics Universe*, Ch. 24 §24.6), **design of experiments** to fit response surfaces with few runs, and optimisation loops that call the simulator hundreds of times. A workflow turns those campaigns into a repeatable pipeline (Figure 24.3).

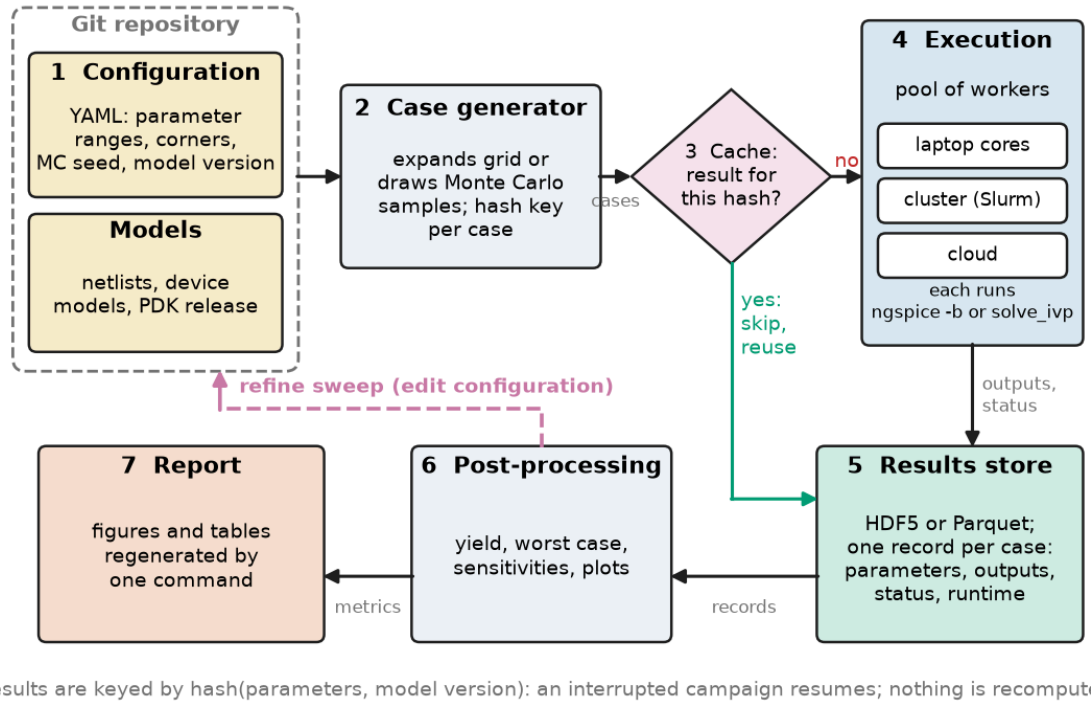


Figure 24.3 — A simulation campaign as a pipeline: configuration, case generation, cached parallel execution, post-processing and reporting, all under version control.

Ingredients of a robust workflow

- **One versioned configuration file** per campaign: parameter ranges, corners, seeds, solver options, model versions.
- **Deterministic cases.** Each case is defined by its parameters and can be rerun alone; random draws use recorded seeds, one stream per case (*The Mathematics Universe*, Ch. 24 §24.9).
- **External simulators in batch mode** (`ngspice -b netlist.cir -r out.raw`), launched by `subprocess` with a timeout, captured logs and a checked exit status.
- **Caching by content.** Results are stored under a hash of the parameters *and* the model version, so an interrupted campaign resumes and a post-processing change reruns nothing.
- **Failure as data.** Record non-converged cases and report the failure rate; 3 % failures concentrated at one corner is itself a finding.
- **Workflow tools** beyond a laptop: Make or Snakemake, cluster schedulers (Slurm), and the FMI co-simulation standard to couple, say, a Simulink controller with a Modelica thermal model.

Listing 24.3 shows content-addressed caching around an LC output-filter simulation, a buck converter's output stage starting up into a 1.2 Ω load. The first pass simulates eight cases; the second finds all eight in the cache.

```
 import hashlib, json, itertools
from pathlib import Path
import numpy as np
from scipy.integrate import solve_ivp

MODEL_VERSION = "lc-filter-v2"                # bump when the model code changes
CACHE = Path("cache"); CACHE.mkdir(exist_ok=True)

def simulate(p):                              # LC output filter of a buck, 12 V input step
    L, C, R_esr, R_load = p["L"], p["C"], p["esr"], p["R"]
    def f(t, y):                              # y = [iL, vC], 12 V input step
        iL, vC = y
        vo = (vC * R_load + iL * R_esr * R_load) / (R_load + R_esr)
        return [(12.0 - vo) / L, (iL - vo / R_load) / C]
    sol = solve_ivp(f, (0, 2e-3), [0.0, 0.0], method="Radau", max_step=1e-6, rtol=1e-8)
    return {"overshoot_pct": 100 * (sol.y[1].max() / 12.0 - 1)}

def cached(p):
    key = hashlib.sha256(json.dumps([MODEL_VERSION, p], sort_keys=True).encode()).hexdigest()
    file = CACHE / f"{key[:16]}.json"
    if file.exists():
        return json.loads(file.read_text()), False
    out = simulate(p)
    file.write_text(json.dumps({"params": p, "result": out}))
    return {"params": p, "result": out}, True

grid = {"L": [4.7e-6, 10e-6], "C": [100e-6, 220e-6], "esr": [0.005, 0.03], "R": [1.2]}
for attempt in (1, 2):
    runs = [cached(dict(zip(grid, v))) for v in itertools.product(*grid.values())]
    print(f"pass {attempt}: {sum(new for _, new in runs)} simulated, "
          f"{sum(not new for _, new in runs)} from cache")
worst = max(runs, key=lambda r: r[0]["result"]["overshoot_pct"])[0]
print("worst overshoot", round(worst["result"]["overshoot_pct"], 1), "% at", worst["params"])
```

```
 pass 1: 8 simulated, 0 from cache
pass 2: 0 simulated, 8 from cache
worst overshoot 78.2 % at {'L': 4.7e-06, 'C': 0.00022, 'esr': 0.005, 'R': 1.2}
```

Listing 24.3 — A parameter sweep with content-addressed result caching.

The worst case is the expected one: with $L = 4.7\ \mu\text{H}$ and $C = 220\ \mu\text{F}$ the characteristic impedance $\sqrt{L/C} = 0.146\ \Omega$ is far below the 1.2 Ω load, so the filter is very lightly damped ($Q \approx 8$) and a hard start-up overshoots by 78 %. That is why converters soft-start (Chapter 30). The solver choice matters too: `Radau`, an implicit method, handles the stiff combination of a 5 mΩ ESR and a 1.2 Ω load without the tiny steps an explicit method would need.

Worked example — budgeting a Monte Carlo corner campaign. A converter controller is to be verified at 5 process corners and 3 temperatures, with 1000 Monte Carlo samples at each, so $5 \times 3 \times 1000 = 15\,000$ transient simulations of 40 s each: 6×10^5 CPU-seconds, or 167 hours on one core. On a 32-core workstation with perfect scaling that is 5.2 hours, an overnight run. Two refinements change the plan. First, Monte Carlo error scales as $1/\sqrt{N}$ (*The Mathematics Universe*, Ch. 24 §24.6). Estimating a yield near 99 % to $\pm 0.3\,\%$ at 95 % confidence needs $N \approx 1.96^2 \times 0.99 \times 0.01 / 0.003^2 \approx 4200$ samples per condition, not 1000, so the campaign must either grow fourfold or use variance reduction. Second, if the same netlist runs with only parameter changes, a simulator that reuses its symbolic factorisation and a warm-start operating point can cut per-run time by a third. Budgeting before launching is part of the workflow.

24.5 Parallel computing fundamentals

Levels of parallelism

Modern hardware is parallel at every level, and each level has its own programming model and cost (Table 24.2).

Level	Hardware	Programming model	Typical granularity	Main cost
SIMD (vector)	AVX2 (4 doubles), AVX-512 (8 doubles), Arm NEON and SVE	Compiler auto-vectorisation, NumPy and BLAS kernels	One instruction	Data layout and alignment
Multicore, shared memory	4–128 cores per socket	Threads (OpenMP, C++ threads, threaded BLAS); in Python, processes because of the GIL	Loop iterations, matrix blocks	Synchronisation, memory bandwidth
Multi-process, one node	Same cores	`multiprocessing`, `concurrent.futures`	Independent tasks (simulation cases)	Process start-up, data copying
Distributed memory	Cluster nodes over InfiniBand or Ethernet	MPI	Domain-decomposed subproblems	Communication latency and bandwidth
Accelerators	GPUs with thousands of lanes	CUDA, HIP, OpenCL; CuPy, PyTorch, JAX from Python	Large data-parallel arrays	PCIe transfers, FP64 throughput

Table 24.2 — Levels of parallel computing. Python's global interpreter lock (GIL) lets only one thread execute Python bytecode at a time, so CPU-bound pure-Python code is parallelised with processes; NumPy and BLAS release the GIL inside their kernels, and the optional free-threaded build of CPython (3.13 and later) removes it.

Amdahl's and Gustafson's laws

If a fraction s of a program's run time is inherently serial and the rest divides perfectly among N processors, the speed-up is bounded by **Amdahl's law**:

$$S_{\text{Amdahl}}(N) = \frac{1}{s + (1 - s)/N} \le \frac{1}{s}.$$

> **Status of the principle: Amdahl's law**
> *Derived.* Follows from splitting a fixed workload into a serial part and a perfectly parallel part; it is an upper bound, since real parallel parts also pay for communication, synchronisation and load imbalance.

Amdahl's law assumes a *fixed* problem. Engineers usually do the opposite: given more processors, they run more Monte Carlo samples or a finer mesh in the same time. If the serial part stays constant while the parallel part grows with N, the **scaled speed-up** is given by **Gustafson's law**,

$$S_{\text{Gustafson}}(N) = N - s\,(N - 1),$$

where s is now the serial fraction of the run time on the parallel machine (Figure 24.4).

> **Status of the principle: Gustafson's law**
> *Derived.* Follows from holding the run time fixed and scaling the parallel workload with N; it holds when the problem can grow, as Monte Carlo sample counts and mesh sizes can, and fails for fixed-size problems.

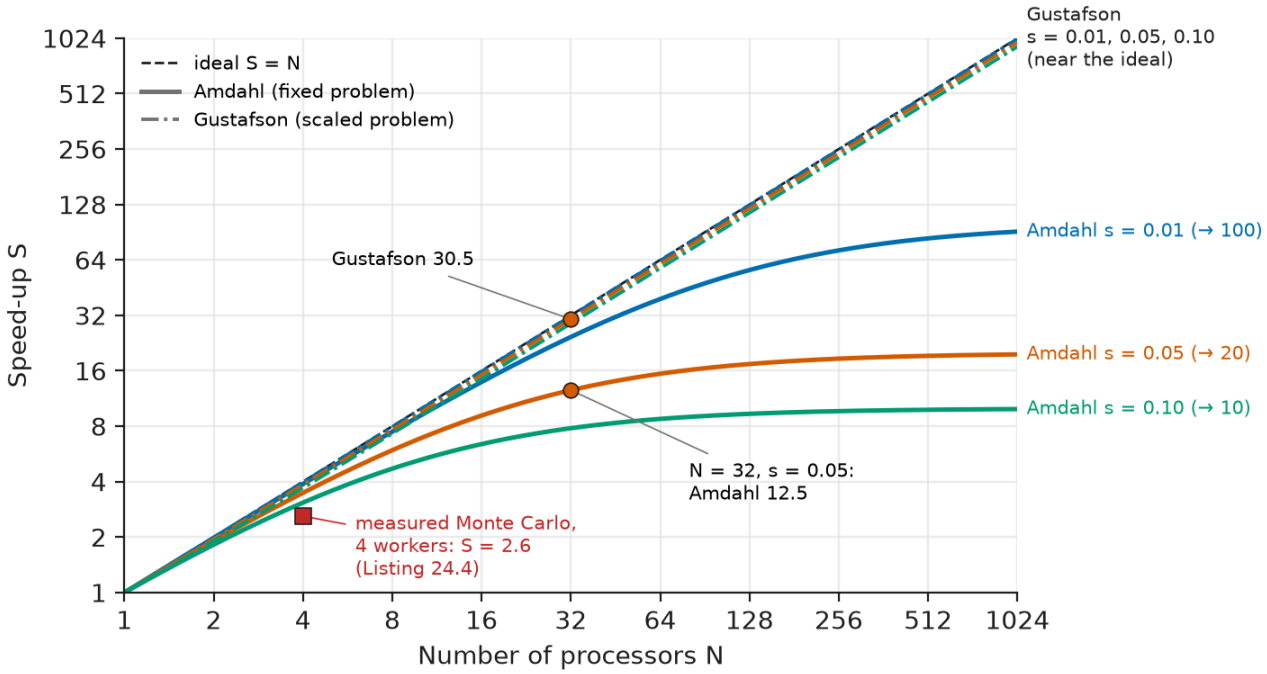


Figure 24.4 — Speed-up versus number of processors under Amdahl's law (fixed problem) and Gustafson's law (scaled problem) for serial fractions of 1, 5 and 10 percent.

Worked example — what 32 cores buy. A yield study spends 5 % of its single-core time in serial work: reading the netlist, generating cases, merging results. With $N = 32$ cores and a fixed 15 000-run campaign, Amdahl gives $S = 1/(0.05 + 0.95/32) = 12.5$, not 32. Even infinitely many cores could not beat $1/0.05 = 20$. If the team instead keeps the run time fixed and lets each core run more samples, with the serial part a constant 5 % of the parallel run time, Gustafson gives $S = 32 - 0.05 \times 31 = 30.5$: the 32-core run does 30.5 times the work of a one-core run in the same time. The two laws describe two decisions. For a deadline on a fixed design question, attack the serial fraction first (parallel case generation, parallel I/O). For better statistics, scale the workload.

Embarrassingly parallel campaigns in Python

Sweeps and Monte Carlo studies are **embarrassingly parallel**: cases are independent, so they can run on separate processes with no communication. Listing 24.4 computes the yield of a second-order RC low-pass filter (1 % resistors, 5 % capacitors) whose cut-off must lie within 2 % of 1 kHz, with 32 million samples in 32 chunks. Each chunk receives its own random stream spawned from one recorded seed, so the answer is identical whatever the number of workers.

```
 import time
from concurrent.futures import ProcessPoolExecutor
import numpy as np

N_CHUNKS, PER_CHUNK = 32, 1_000_000            # 32 million Monte Carlo samples in total

def chunk_yield(seed):
    """Fraction of 2nd-order RC filters (1 % R, 5 % C) with fc within 1 kHz +/- 2 %."""
    rng = np.random.default_rng(seed)
    R = 1.0e3 * (1 + 0.01 * rng.uniform(-1, 1, (PER_CHUNK, 2)))
    C = 159.15e-9 * (1 + 0.05 * rng.uniform(-1, 1, (PER_CHUNK, 2)))
    fc = 1 / (2 * np.pi * np.sqrt(R.prod(axis=1) * C.prod(axis=1)))
    return np.count_nonzero(np.abs(fc - 1e3) <= 20)

def run(workers):
    seeds = np.random.SeedSequence(20251025).spawn(N_CHUNKS)   # one stream per chunk
    t0 = time.perf_counter()
    with ProcessPoolExecutor(max_workers=workers) as pool:
        passed = sum(pool.map(chunk_yield, seeds))
    return passed / (N_CHUNKS * PER_CHUNK), time.perf_counter() - t0

if __name__ == "__main__":                     # required for process pools on Windows/macOS
    for w in (1, 4):
        y, t = run(w)
        print(f"{w} worker(s): yield = {y:.6f}   ({t:.2f} s)")
```

```
 1 worker(s): yield = 0.633043   (7.86 s)
4 worker(s): yield = 0.633043   (3.02 s)
```

Listing 24.4 — A process-parallel Monte Carlo yield estimate whose result does not depend on the number of workers (timings from one run; they vary with machine load).

The yield is identical to the last digit with one or four workers, because the work is divided into a fixed number of chunks, each with its own stream, rather than into "one chunk per worker". The speed-up on the four-core laptop is $7.86/3.02 = 2.6$. Process start-up, the serial reduction, and the drop of the turbo clock when all cores are busy account for the rest; Amdahl's law with $s \approx 0.18$ reproduces it.

The oversubscription trap and the GPU transfer trap

The opening's workstation shows the commonest parallel bug in scientific Python. Each worker process calls NumPy, and NumPy's BLAS starts one thread per core by default. With 32 workers on 32 cores, 1024 threads compete; the cores spend their time switching contexts and thrashing caches. The fix is to set `OMP_NUM_THREADS=1` (and `OPENBLAS_NUM_THREADS` or `MKL_NUM_THREADS`) in the environment of each worker, or to use `threadpoolctl`, so that parallelism happens at exactly one level.

GPUs offer 10–100 times a CPU's throughput on data-parallel work, but data must cross the PCIe bus: at about 25 GB/s (PCIe 4.0 ×16), 1 GB takes 40 ms, time in which a data-centre GPU could perform some 10^{12} flops. A GPU pays off when data stay resident across many operations (FDTD field solvers, neural-network training) and loses when short kernels move data back and forth.

24.6 Performance profiling

Measure before optimising

Intuition about where a program spends its time is usually wrong. A **profiler** measures it. Deterministic profilers (Python's `cProfile`) record every function call; **sampling profilers** (`py-spy`, Linux `perf`, Intel VTune, AMD uProf, NVIDIA Nsight for GPUs) interrupt the program periodically and record where it is, with little overhead, and can be attached to a running process. **Line profilers** (`line_profiler`) attribute time to individual lines, and memory profilers (`tracemalloc`, `memray`) to allocations. The output is often shown as a **flame graph**: stacked bars for the call hierarchy, width proportional to time.

Listing 24.5 profiles a two-stage post-processing pipeline: parse 300 000 lines of a logger's text file, then compute a windowed spectrum.

```
 import cProfile, pstats
import numpy as np

def make_log(n):                                # stand-in for a 300 000-line logger file
    t = np.arange(n) / 10e3
    v = np.sin(2 * np.pi * 50 * t)
    return [f"{a:.6f},{b:.6f}" for a, b in zip(t, v)]

def parse(lines):                               # pure-Python parsing: the suspect
    return np.array([float(s.split(",")[1]) for s in lines])

def spectrum(v):
    return np.abs(np.fft.rfft(v * np.hanning(v.size)))

def pipeline(lines):
    v = parse(lines)
    return spectrum(v).argmax()

lines = make_log(300_000)
prof = cProfile.Profile()
peak = prof.runcall(pipeline, lines)
st = pstats.Stats(prof)
total = st.total_tt
for name in ("parse", "spectrum"):
    ct = next(v[3] for k, v in st.stats.items() if k[2] == name)   # cumulative time
    print(f"{name:9s} {100 * ct / total:5.1f} % of run time")
print("peak bin:", peak)
```

```
 parse      74.4 % of run time
spectrum   25.6 % of run time
peak bin: 1500
```

Listing 24.5 — Profiling a post-processing pipeline with cProfile (shares from one run; they vary noticeably between runs on a busy machine).

Worked example — what optimising the hot spot can buy. The profile attributes about three-quarters of the run time to parsing and a quarter to the FFT. The peak lands in bin 1500, which is 50 Hz at a bin spacing of $10\,000/300\,000 = 0.033$ Hz, as it should. Amdahl's law applies to optimisation as well as to parallelism. If parsing is made infinitely fast, the run time falls to the remaining 26 %, a speed-up of at most $1/0.256 = 3.9$. Optimising the FFT, the part an engineer might instinctively suspect, could gain at most $1/0.744 = 1.34$. Replacing the Python parsing loop with pandas' C-coded CSV reader cut parsing time about fourfold on the same machine, from roughly 0.39 s to 0.09 s, for a

total speed-up of about 2.3, close to what the profile predicted. The broader rule, sometimes quoted as the 90/10 rule, is that a small fraction of the code dominates the run time.

> **Status of the principle: The 90/10 rule of program hot spots**
> *Empirical.* Across many programs, a small fraction of the code (often around 10 %) accounts for most of the execution time (often around 90 %); the exact split varies widely, which is why it must be measured for each program rather than assumed.

Benchmarking hygiene

Timing measurements are noisy. Warm up caches, JIT compilers and FFT plans before timing; repeat and report the minimum or the median with spread; use `time.perf_counter` or `timeit`; and remember that laptops change clock with temperature and load. Code already at the memory-bandwidth roof (§24.2) gets faster only by moving less data. Profile at realistic sizes: the hot spot of a 1000-node test case may not be the hot spot at 10^6 nodes.

24.7 Numerical precision

> **Boundary.**
> The IEEE 754 formats, unit roundoff, the standard model of floating-point arithmetic, cancellation, and compensated and pairwise summation are in *The Mathematics Universe*, Ch. 22 §22.1–22.2; bit-level formats, fixed point and binary arithmetic are in Chapter 11 of this book; finite-word-length effects in DSP are in *Circuits, Signals and Communications*, Ch. 21. This section treats precision as an engineering design choice.

Precision costs and buys

Choosing a floating-point format trades accuracy against memory, bandwidth and arithmetic throughput:

Format	Unit roundoff	Bytes	Relative throughput (CPU SIMD)	Relative throughput (GPU)	Typical engineering use
binary64 (double)	1.1×10^{-16}	8	1	1/64 (consumer) to 1/2 (data-centre) of FP32	Circuit and field simulation, linear algebra, accumulation
binary32 (single)	6.0×10^{-8}	4	2	1	Large waveform sets, real-time DSP on FPU-equipped MCUs, GPU simulation
bfloat16	3.9×10^{-3}	2	—	2–16 on tensor units	Neural-network training and inference
binary16 (half)	4.9×10^{-4}	2	—	2–16 on tensor units	Inference, image processing
FP8 (E4M3, E5M2)	6.3×10^{-2}, 1.3×10^{-1}	1	—	2× FP16	Inference on recent accelerators

Halving the word length doubles the numbers per cache line and per byte of memory bandwidth, so a bandwidth-bound kernel (§24.2) runs about twice as fast in `float32` even on a CPU. On GPUs the difference is far larger: consumer graphics cards execute double precision at 1/32 or 1/64 of their single-precision rate, while data-centre accelerators keep double precision at about half the single-precision rate, at a large price premium.

Mixed precision: fast arithmetic, accurate answers

Mixed-precision iterative refinement gets double-precision accuracy from single-precision factorisation. Factor A in `float32` (the $O(n^3)$ part), solve, then repeatedly compute the residual $r = b - Ax$ in `float64` (an $O(n^2)$ step) and correct x by solving with the single-precision factors. Each step reduces the error by about $\kappa(A)\,u_{32}$, so the method converges whenever $\kappa(A) \ll 1/u_{32} \approx 10^7$. Listing 24.6 demonstrates it on a matrix with condition number 10^4.

```
 import numpy as np
from scipy.linalg import lu_factor, lu_solve

rng = np.random.default_rng(5)
n = 500
Q, _ = np.linalg.qr(rng.standard_normal((n, n)))
A = Q @ np.diag(np.logspace(0, 4, n)) @ Q.T            # condition number 1e4
x_true = rng.standard_normal(n)
b = A @ x_true

lu32 = lu_factor(A.astype(np.float32))                 # O(n^3) work in single precision
x = lu_solve(lu32, b.astype(np.float32)).astype(np.float64)
for it in range(4):
    err = np.linalg.norm(x - x_true) / np.linalg.norm(x_true)
    print(f"iteration {it}: relative error {err:.1e}")
    r = b - A @ x                                       # residual in double precision, O(n^2)
    x += lu_solve(lu32, r.astype(np.float32))           # correction from the float32 factors
x64 = np.linalg.solve(A, b)
print(f"float64 LU for comparison: {np.linalg.norm(x64 - x_true)/np.linalg.norm(x_true):.1e}")
```

```
 iteration 0: relative error 1.2e-04
iteration 1: relative error 1.3e-08
iteration 2: relative error 1.4e-12
iteration 3: relative error 3.1e-13
float64 LU for comparison: 3.4e-13
```

Listing 24.6 — Mixed-precision iterative refinement: float32 factorisation, float64 residuals.

The single-precision solution starts with a relative error of 1.2×10^{-4}, consistent with $\kappa u_{32} = 10^4 \times 6 \times 10^{-8} = 6 \times 10^{-4}$. Each refinement step multiplies the error by roughly 10^{-4}, and after three steps the result matches the full double-precision solve. On hardware where single precision is many times faster (GPUs, tensor units), this is how linear solvers and the HPL-MxP benchmark reach double-precision accuracy at mixed-precision speed.

> **Status of the principle: Convergence of mixed-precision iterative refinement**
> *Derived.* With residuals computed in the higher precision, each correction step contracts the error by a factor of order $\kappa(A)\,u_{\text{low}}$, so refinement converges to working accuracy when $\kappa(A)\,u_{\text{low}} \ll 1$ and fails when the low-precision factorisation is itself too inaccurate.

Precision traps in engineering code

Worked example — the phase of a long simulation. A system-level model of a 1 MHz clock evaluates $\sin(2\pi f t)$ with t stored as a time in seconds. At $t = 1000$ s, the spacing of `float32` numbers near 1000 is $2^{-14} \approx 6.1 \times 10^{-5}$ s. Rounding t to it introduces up to 3.05×10^{-5} s of error, which at 1 MHz is $2\pi \times 10^6 \times 3.05 \times 10^{-5} = 192$ rad of phase: the output is noise. In `float64` the spacing near 1000 is 1.14×10^{-13} s and the phase error at most 3.6×10^{-7} rad, adequate. The robust fix, used in every numerically controlled oscillator (NCO) in FPGAs and DDS chips, is to never form the large product. Accumulate phase modulo 2π, or better as an unsigned integer phase accumulator that wraps exactly.

A 32-bit accumulator clocked at 100 MHz has frequency resolution $10^8/2^{32} = 0.023$ Hz, with no drift however long it runs.

Other traps recur. **Subnormal numbers** can slow some processors 10–100-fold when a decaying filter state approaches zero; flush-to-zero modes trade gradual underflow for speed. **Simulator tolerances** are precision settings in disguise. SPICE's defaults (`reltol=1e-3`, `abstol=1e-12` A, `vntol=1e-6` V) set the accuracy of every Newton iteration and time step; a 0.1 % relative tolerance is adequate for most analog work and inadequate for, say, 16-bit ADC linearity or oscillator phase noise, where it must be tightened by orders of magnitude. **Accumulators** need more precision than their operands: an energy meter summing 10^9 `float32` power samples loses everything below about 10^{-7} of the running total, so it accumulates in double or in a wide integer.

24.8 Reproducibility

> **Boundary.**
> Levels of reproducibility (repeatability, reproducibility, replicability), seeded and independent random streams, floating-point non-associativity in parallel reductions, version pinning and metadata are treated in *The Mathematics Universe*, Ch. 24 §24.9; the laboratory side (instrument records, run folders, hashed raw data) in Chapter 22 §22.10.

Reproducibility at campaign scale

A team campaign adds three problems to the single-script reproducibility of Chapter 22.

Environments that can be rebuilt. A `requirements.txt` with exact versions is the minimum. A **lock file** (from `pip-tools`, `uv`, Poetry or `conda-lock`) pins every transitive dependency with hashes. A **container image** (Docker on workstations and cloud, Apptainer/Singularity on HPC clusters) freezes the operating system libraries, compilers and BLAS as well. For commercial tools the equivalent is recording the simulator version, PDK release and licence-server feature versions, since a "minor" simulator update can change convergence defaults.

Parallel non-determinism. Threaded BLAS and parallel reductions sum in an order that depends on thread count and scheduling, so the last bits of results vary between runs (*The Mathematics Universe*, Ch. 24 §24.9). Where bitwise repeatability is required, for regression debugging or audit, fix the thread count and enable deterministic modes, such as Intel MKL's conditional numerical reproducibility (`MKL_CBWR`) or the deterministic-algorithm flags of GPU libraries. Otherwise compare results with tolerances derived from the error analysis. Listing 24.4 shows the other half of the discipline: the decomposition of random streams must not depend on the number of workers.

Provenance from raw data to report. Every number in a design report should trace to a commit, a configuration, an environment and a set of inputs, through run manifests (Chapter 22), workflow records (§24.4) and CI (§24.9). Archives deposit code, environment and data together under a persistent identifier (a DOI), following the FAIR principles: findable, accessible, interoperable, reusable.

> **Status of the principle: Reproducibility of a computational campaign**
> *Approximate*. With pinned environments, recorded seeds and parameters, version-controlled code and immutable inputs, a rerun reproduces results bitwise on the same platform and within the numerical-error tolerance across platforms; anything unrecorded (an unpinned dependency, a hand-edited file, a licence-server tool update) breaks the guarantee.

24.9 Software testing

What to test in engineering code

Engineering software fails quietly. A sign error in a loss model produces plausible efficiency curves; a unit slip between mV and V makes a margin look ten times larger; a refactored post-processing script changes a reported number. Tests are small programs that check other programs automatically, and they are written at several levels:

Level	What it checks	Example in EEE work
Unit test	One function against known values and invariants	Parallel-resistance helper, unit conversions, CRC routine, CORDIC accuracy
Integration test	Several components together	Netlist generator plus simulator plus parser on a reference circuit
Regression test	Output unchanged from a trusted reference within tolerance	Efficiency table of the validated converter model
Property-based test	Invariants over many generated inputs	KCL residual zero, passivity, $Z = Z^{\mathsf{T}}$ for reciprocal networks
Hardware-in-the-loop test	Software against real or emulated hardware	Firmware with a plant simulator in real time (Chapter 28)

How to set numerical tolerances, from $\kappa\varepsilon$ for linear algebra to Ch^p for discretisations and $z \cdot \mathrm{SE}$ for Monte Carlo, and how to test convergence order with manufactured solutions, are covered in *The Mathematics Universe*, Ch. 24 §24.9. The engineering additions are the tooling and the habit.

pytest in practice

pytest is the standard Python test runner. Any function named `test_*` in a file named `test_*.py` is a test, and plain `assert` statements are its checks. `pytest.approx` compares floats with a tolerance, `pytest.mark.parametrize` runs one test over a table of cases, and `pytest.raises` checks that bad input is rejected. Listing 24.7 tests the divider helper used in Chapter 22's worked example.

```
 import math
import numpy as np
import pytest

def parallel(*r):                                  # code under test (normally imported)
    if any(x <= 0 for x in r):
        raise ValueError("resistances must be positive")
    return 1.0 / sum(1.0 / x for x in r)

def divider_out(v_in, r_top, r_bot, r_load=math.inf):
    r_eff = r_bot if math.isinf(r_load) else parallel(r_bot, r_load)
    return v_in * r_eff / (r_top + r_eff)

@pytest.mark.parametrize("r, expected", [((1e3, 1e3), 500.0), ((3e3, 6e3), 2e3),
                                         ((1e3, 1e3, 1e3), 1e3 / 3)])
def test_parallel_known_values(r, expected):       # exact references
    assert parallel(*r) == pytest.approx(expected, rel=1e-12)

def test_parallel_properties():                    # invariants on random inputs
    rng = np.random.default_rng(0)
    for a, b in rng.uniform(1.0, 1e6, (1000, 2)):
        p = parallel(a, b)
        assert p <= min(a, b) and p == pytest.approx(parallel(b, a), rel=1e-15)

def test_divider_loading():                        # regression of a design value (Ch. 22)
    assert divider_out(30.0, 130e3, 16e3) == pytest.approx(3.2877, abs=1e-4)
    assert divider_out(30.0, 130e3, 16e3, r_load=1e6) < divider_out(30.0, 130e3, 16e3)

def test_rejects_bad_input():
    with pytest.raises(ValueError):
        parallel(1e3, 0.0)
```

```
 $ python -m pytest -q test_divider.py
......                                                                   [100%]
6 passed in 0.70s
```

Listing 24.7 — A pytest module with exact-value, property, regression and error-handling tests.

The four kinds of test catch different bugs. The known-value cases catch formula errors. The property test checks, over a thousand random pairs, that a parallel combination is never larger than its smallest resistor and is symmetric; it catches errors no hand-picked case would. The regression test pins the 3.2877 V design value of Chapter 22, so a "cleanup" that changes it fails immediately, and its second assertion checks the direction of an engineering effect (loading lowers the output). The error test ensures that a zero resistance is rejected instead of producing a division by zero deep inside a sweep.

Continuous integration

Tests help only if they run. **Continuous integration** (CI: GitHub Actions, GitLab CI, Jenkins) runs the suite on every proposed change on a clean machine built from the pinned environment, and a change that breaks a test cannot be merged. CI also measures coverage (`pytest-cov`), runs linters (`ruff` , `mypy`) and regenerates key figures; for firmware it cross-compiles, runs tests in an emulator (QEMU, Renode) and, in mature teams, on a hardware-in-the-loop rack (Chapter 28).

24.10 Version control and collaborative workflows

Git's model

Git is the version-control system used for nearly all code, and increasingly for models, documentation and design files. Its model is simple and worth understanding. A **commit** is a snapshot of the whole project tree, plus metadata (author, date, message) and pointers to its parent commit or commits. Each commit is named by a cryptographic hash of its content and history (SHA-1, with SHA-256 repositories now supported), so a commit hash identifies the exact state of every file, and history cannot be altered without changing every later hash. A **branch** is just a movable name for a commit; **merging** creates a commit with two parents; a **tag** names a release (`v1.4.0`). Every clone holds the full history, so work continues offline and every copy is a backup.

```
 git switch -c fix/loss-model-units        # new branch for one change
git add src/loss_model.py tests/test_loss_model.py
git commit -m "Fix mW/W slip in switching-loss term; add regression test"
git push -u origin fix/loss-model-units   # then open a pull request for review
git tag -a v1.4.0 -m "Model used for design review DR-3"   # after merge, on main
git rev-parse --short HEAD                # the hash recorded in every result manifest
```

Team workflows

Most engineering teams use a variant of **trunk-based development with short-lived branches** (Figure 24.5). The `main` branch is always in a working, tested state. Each change, whether a fix, a feature or a model update, is made on its own branch, pushed, and proposed as a **pull request** (merge request). CI runs the tests; at least one colleague **reviews** the change, reading the diff, asking questions and checking engineering assumptions as well as code. Only then is it merged. Releases used for design reviews, qualification or production test are tagged, so a report can cite exactly `v1.4.0` or commit `a1fea18` .

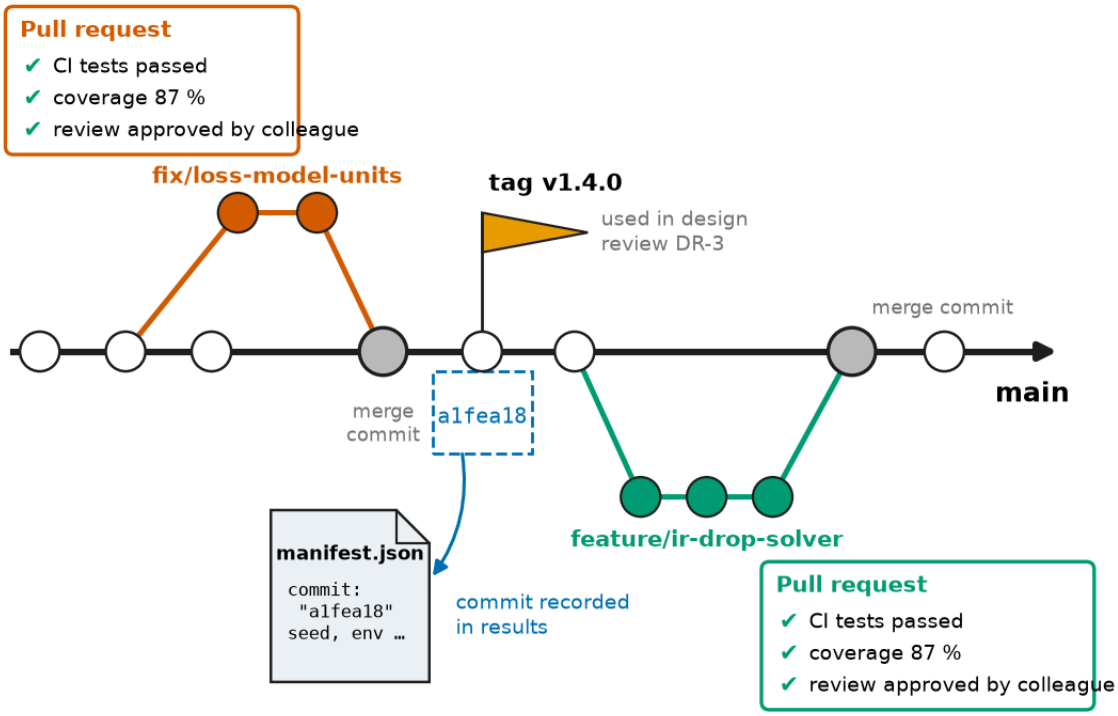


Figure 24.5 — Trunk-based Git workflow with short-lived feature branches, pull-request review, CI and tagged releases.

Good practice keeps the history useful. **Small, focused commits** with messages that say *why* make review possible and let `git bisect` find a bad commit by binary search, $O(\log n)$ test runs for n commits (Chapter 23). A `.gitignore` keeps build products and caches out. **Large binary data** go to **Git LFS** or DVC, which keep only pointers in Git. **Notebooks** are stripped of outputs before commit (`nbstripout`). **EDA files** diff well when text-based (KiCad); binary formats are versioned but reviewed in their own tools.

Worked example — finding a regression with bisect. The design report's efficiency at 20 A was 97.75 % in release `v1.2.0` and is 97.31 % at the head of `main` , 120 commits later; no one remembers changing the loss model. A script that recomputes the 20 A efficiency and exits with failure if it differs from 97.75 % by more than 0.01 points serves as the test. `git bisect start main v1.2.0` followed by `git bisect run python check_eff.py` tests the midpoint commit, keeps the half that contains the change, and repeats. It finds the culprit in at most $\lceil \log_2 120 \rceil = 7$ runs, against up to 120 for a linear search. The offending commit turns out to be a "unit cleanup" that converted a switching-loss coefficient from µJ to mJ in one place but not another. The fix, with a regression test that pins 97.75 % (as in Listing 24.7), goes through a pull request like any other change.

> **Status of the principle: Content-addressed version history**
> *Fundamental.* Because each commit's name is a cryptographic hash of its content and ancestry, a commit hash identifies one exact project state; altering any past file changes every later hash, so recorded hashes make results traceable to the code that produced them, as long as the hash function resists collisions.

The practices of this chapter reinforce one another: version control records the code, pinned environments and seeds the conditions, tests and CI guard against silent change, profiling directs optimisation, and the right kernels, formats and parallelism make the computation feasible. With them, the opening's converter team gets one yield number that anyone can regenerate, scaled across 32 cores with one level of parallelism.

What remains open

Open question	Where it is answered
How do embedded targets constrain computation: memory, power, fixed point and real-time deadlines?	Chapters 25 and 27
How is firmware tested on host, emulator and hardware-in-the-loop rigs?	Chapter 28
When should a computation move from a processor to an FPGA or accelerator, and how is the data movement modelled?	Chapter 35
How do EDA tools apply sparse solvers, graph algorithms and parallel computing to timing, power and IR-drop sign-off?	Chapter 34
How are floating-point and fixed-point formats laid out in bits, and how does binary arithmetic overflow?	Chapter 11
What are the error analysis, conditioning and convergence results behind the solvers used here?	*The Mathematics Universe*, Ch. 22 and Ch. 24
How do finite word length and quantisation affect DSP algorithms?	*Circuits, Signals and Communications*, Ch. 21 and Ch. 23

Next: Chapter 25 opens Part VI by taking computation into the device itself: embedded-system architecture, where the processor, memory, power budget, firmware structure and boot process are chosen together around a product's real-time job.

Part VI — Embedded and Real-Time Systems

Parts IV and V built a processor and taught it to run programs. This part puts that processor inside a product — a water meter, a motor drive, a flight controller, an EV battery pack, an infusion pump — where it must sense and act on the physical world through pins and buses, meet deadlines set by physics, live within kilobytes of memory and microamps of current, and keep working unattended for a decade or more. The abstraction this part builds is the **task**: a piece of firmware with a period, a deadline and a worst-case execution time, which can be scheduled, analysed and proved to meet its timing. Like every abstraction in the book, it holds only inside limits — electrical limits at the pins, bit-timing limits on the buses, energy budgets from the battery, and timing limits that interrupts, shared resources and priority inversion can break. The four chapters move from the architecture of an embedded system, through its interfaces and its timing, to the engineering practice that ties them into a product that can be built, tested and maintained.

By the end of this part you should understand

- How an embedded system is modelled and partitioned between peripherals and software, and how a microcontroller is chosen against peripheral, compute, memory, energy, security and supply requirements.
- How memory and energy budgets are written and met: linker sections, stack bounds, Flash endurance, power modes and the average-current model of battery life.
- How superloop, interrupt-driven and RTOS-based firmware differ in worst-case response, how a device gets from its reset vector to `main`, and how bootloaders install signed updates without ever leaving the device unbootable.
- The electrical behaviour of GPIO and the bit-level timing of UART, SPI, I²C and CAN, and the fundamentals of USB and Ethernet, with the numbers that the standards set.
- How converters and sensors are interfaced without losing accuracy, and how a failing link is diagnosed with an oscilloscope and logic analyser.
- How interrupt latency and task response times are bounded, how cyclic, rate-monotonic and EDF scheduling are analysed, and how response-time analysis proves schedulability.
- Why races, deadlock and priority inversion occur, and how critical sections, mutexes with priority inheritance, queues, watchdogs and fault handlers contain them.
- How firmware is layered, made low-power, debugged on the target and validated with hardware-in-the-loop testing.

Chapters

Chapter	Central question	What it delivers
25 — Embedded-System Architecture	How is an embedded system decomposed, sized and structured, and how does it boot and update itself?	System models, hardware–software partitioning, MCU selection, memory and energy budgets, firmware architectures, driver overview, the boot sequence and secure A/B firmware updates
26 — Embedded Interfaces	What electrical and timing limits govern the links between a microcontroller and the world?	GPIO electrical design, UART tolerance budgets, SPI round-trip timing, I²C pull-up sizing and bus recovery, CAN bit timing and bus load, USB and Ethernet fundamentals, converter and sensor interfacing, troubleshooting methods
27 — Real-Time Computing	How is it proved that every deadline will be met, and what breaks the proof?	Task model and WCET, interrupt latency, cyclic, RMS and EDF scheduling with response-time analysis, timers, races and synchronisation, RTOS selection, priority inversion and inheritance, latency and jitter, watchdogs and fault handling
28 — Embedded System Design	How are modules, drivers and state machines engineered into firmware that can be tested, maintained and trusted?	Hardware abstraction, modularity, robust drivers, state machines, low-power practice, data logging, communication stacks, on-target debugging, hardware-in-the-loop testing and firmware validation

How this part connects to the rest of the book

This part rests on Part IV: the memory maps, interrupt controllers, timers and peripheral blocks of Chapter 17, the caches and buses of Chapter 16, and the Flash and SRAM of Chapter 18. It uses the C and C++ of Chapter 21 throughout, and the logic levels, synchronisers and state machines of Part III at every pin. The converter architectures of Chapter 10 and the sensor amplifiers of Chapter 8 sit behind its analog interfaces. Looking forward, the controllers of Part VII's power converters and motor drives (Chapters 29–32) are real-time embedded systems of exactly this kind, and Chapter 35 returns to partitioning, accelerators and real-time constraints across processors and FPGAs.

Among the sibling books, *Circuits, Signals and Communications* supplies the transmission-line theory behind bus termination (Ch. 14), sampling, quantization and jitter (Ch. 19), real-time DSP budgets (Ch. 21), CRCs (Ch. 28) and the cost of loop delay in digital control (Ch. 35). *Electrical Energy and Machines* supplies battery ratings (Ch. 16) and the drive-control algorithms that run on these processors (Ch. 12).

Chapter 25 — Embedded-System Architecture

Part VI · Embedded and Real-Time Systems

A utility orders two hundred thousand ultrasonic water meters. Each must measure flow every second, report a reading over LTE-M four times a day, run for fifteen years on one lithium cell sealed in a potted housing, and accept firmware updates over the air because the metering standard will change during its life. The first prototype meets every functional requirement on the bench and would have flattened its battery in under five years in the field: the radio was left listening for ten seconds after each report, the microcontroller idled in a mode that kept its high-speed oscillator running, and nobody had written down an energy budget. A later pilot batch survived the battery test but bricked three hundred meters when a mains-powered gateway rebooted halfway through an update. None of this is a coding bug in the ordinary sense. It is architecture: which work goes into hardware and which into software, which microcontroller, how much memory and energy each activity may spend, how the firmware is structured, how the processor gets from reset to `main`, and how a new image replaces the old one without ever leaving the device unable to boot.

> **How is an embedded system decomposed into hardware, firmware and peripherals, and how are its microcontroller, memory and energy chosen against measurable budgets?**
> **Which firmware structure fits which timing problem, and how does a device boot and update itself safely in the field?**

Concept	Engineering destination
Embedded-system models	Specifying sensor nodes, motor controllers, ECUs and medical devices as reactive, time-bounded systems
Hardware–software partitioning	Deciding what timers, DMA, CRC and crypto blocks do so the CPU does not have to
Microcontroller selection	Choosing core, memory, peripherals, package and supply for a product and its life cycle
Memory constraints	Linker budgets, stack sizing, Flash endurance and data logging that survives fifteen years
Power constraints	Average-current budgets, power modes and battery life of duty-cycled devices
Firmware architecture	Superloop, interrupt-driven and RTOS-based designs and their worst-case response
Peripheral drivers	Register access, interrupt and DMA drivers, buffering between ISRs and tasks
Boot processes	Reset vector to `main`, clock and memory initialisation, multi-stage boot chains
Firmware updates	Bootloaders, A/B slots, signed images, rollback and secure OTA updates

Roadmap. What an embedded system is and how it is modelled (25.1) → the system-level split between hardware and software (25.2) → choosing the microcontroller (25.3) → budgeting memory (25.4) and energy (25.5) → structuring the firmware (25.6) and its drivers (25.7) → how the device starts (25.8) and how it replaces its own firmware (25.9).

25.1 Embedded-system models

What makes a system embedded

An **embedded system** is a computer built into a product whose purpose is not computing: a meter, a pacemaker, a brake controller, a drone, a protection relay, a server's baseboard management controller. Four properties follow from that purpose and drive every decision in this part of the book.

1. **Dedicated function.** The software is fixed at design time and updated rarely; the hardware is sized for that function and no more, because the bill of materials is multiplied by production volume.
2. **Reactive and time-bounded.** The system responds to events in the physical world (a pulse from a flow sensor, a CAN frame, a fault current) within deadlines set by physics rather than by user patience. Chapter 27 develops the timing theory.
3. **Resource-constrained.** Memory is kilobytes to a few megabytes, energy may come from a coin cell or a harvester, and the processor runs at tens to hundreds of megahertz.
4. **Unattended and long-lived.** Nobody presses reset on a buried meter or a satellite. The system must recover from faults on its own and run for ten to twenty-five years.

Models used to specify embedded behaviour

Designers describe embedded systems with a small set of models, each suited to a class of behaviour:

- **Sense–compute–actuate (dataflow).** Samples arrive periodically, are filtered and processed, and drive an output: a motor-current loop, a digital power supply, an audio pipeline. The model is a chain of blocks with rates and latencies; its timing is analysed as in Book 3, *Circuits, Signals and Communications*, Ch. 21 §21.3, and its control consequences as in Ch. 35 §35.8.
- **Event-driven (reactive).** The system waits, an event arrives, it changes state and acts: a door controller, a protocol stack, a user interface. The natural model is a finite-state machine (Chapter 14 for the hardware form, Chapter 28 for firmware state machines).
- **Time-triggered versus event-triggered.** Time-triggered designs run every activity at a pre-planned instant (a cyclic schedule, a FlexRay slot) and are easy to verify but waste capacity; event-triggered designs are efficient and harder to bound. Most products mix both.
- **Layered.** Hardware, board support, drivers, middleware and application, each hiding the one below (Chapter 28).

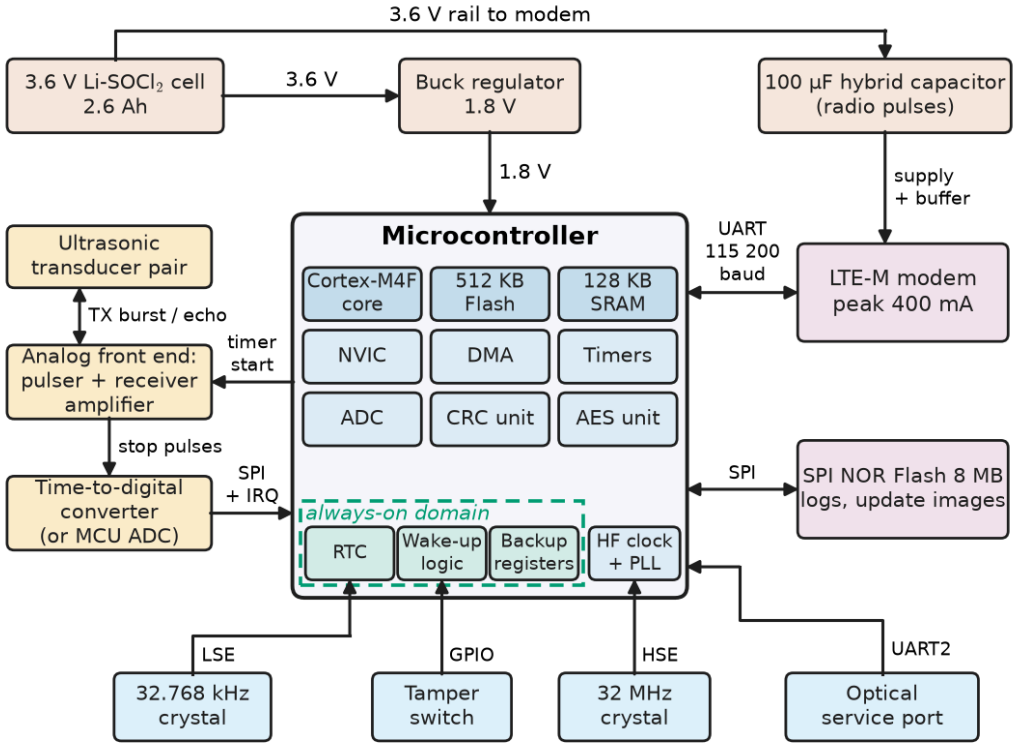


Figure 25.1 — Generic architecture of a battery-powered embedded sensor node, using the ultrasonic water meter as the example.

Figure 25.1 shows the meter of the opening drawn this way. Every block on it has a budget — cost, board area, current in each mode, time per activity — and the rest of the chapter is about setting and meeting those budgets.

The range of embedded platforms

Table 25.1 — Representative classes of embedded systems (typical values).

Class	Example	Processor	Memory	Power	Software
Tiny sensor or actuator	Smoke detector, toy, battery charger	8/16-bit or Cortex-M0+, 1–48 MHz	4–64 KB Flash, 0.5–8 KB RAM	0.5 µA sleep, 1–5 mA active	Superloop, interrupts
Connected sensor node	Water meter, asset tracker, wearable	Cortex-M4F/M33, 32–128 MHz	256 KB–1 MB Flash, 64–256 KB RAM	1–5 µA sleep, 5–20 mA active	RTOS, radio stack
Real-time controller	Motor drive, digital PSU, flight controller	Cortex-M4F/M7, DSP, 100–600 MHz	0.5–2 MB Flash, 0.1–1 MB RAM	0.1–1 W	RTOS or bare metal, hard deadlines
Safety ECU	Brake, airbag, steering controller	Lockstep Cortex-R5/TriCore, 200–400 MHz	2–8 MB Flash	1–3 W	AUTOSAR OS, ISO 26262
Embedded Linux system	Gateway, HMI panel, camera	Cortex-A53/A72, 1–2 GHz	256 MB–4 GB DRAM, eMMC	1–10 W	Linux, often with a companion MCU

25.2 Hardware–software partitioning

The question at system level

Every function in a product can be implemented in software on the CPU, in an on-chip peripheral configured by software, in an external chip, or in custom logic (an FPGA or ASIC). **Hardware–software partitioning** is the assignment of functions to these resources so that deadlines, energy and cost are all met. At the system level treated here the question is qualitative and practical: which peripherals does the chosen microcontroller have, and which jobs should be handed to them? Chapter 35 returns to partitioning quantitatively, with performance models, accelerators and FPGAs.

The guiding observation is that a microcontroller's CPU is a scarce, serial resource, while its peripherals run in parallel and often at much lower energy per operation. A timer generating PWM costs the CPU nothing per cycle; the same waveform produced by toggling a pin in software costs the CPU everything. Good partitioning therefore moves to hardware every activity that is (a) periodic and fast, (b) bit-level and timing-critical, or (c) bulk data movement.

Table 25.2 — Typical system-level partitioning in a microcontroller design.

Function	In software (CPU)	In hardware (peripheral)	Usual choice
PWM for a motor or LED	Pin toggling in a timed loop	Timer compare channels, dead-time unit	Hardware
Serial byte framing	Bit-banging GPIO	UART, SPI, I²C controllers	Hardware, software only for spare channels
Sample capture at fixed rate	Polling ADC in a loop	Timer-triggered ADC with DMA to RAM	Hardware
CRC of a frame or image	Table-driven loop, about 5–10 cycles/byte	CRC unit, about 1 cycle/word	Hardware for long data
AES, SHA-256, ECDSA	Software library	Crypto accelerator, key storage	Hardware when available (also protects keys)
Control law (PID, observer)	Firmware	CORDIC/FMAC units, FPGA	Software, unless the loop rate exceeds about 100 kHz
Protocol logic, state machines	Firmware	Rarely	Software

> **Status of the principle: Offloading periodic and bit-level work to peripherals**
> *Approximate*. The saving holds when the peripheral can complete the job autonomously; it shrinks when the peripheral needs frequent CPU servicing (small DMA buffers, one interrupt per byte) or when configuring it costs more than the job itself.

Worked example — Per-sample interrupts versus DMA. A vibration monitor on a 64 MHz Cortex-M4 samples an accelerometer channel at 100 kSa/s. In the first design every conversion raises an interrupt whose handler reads the result and stores it in a buffer, costing about 60 cycles including entry and exit. The CPU load is

$$\frac{100\,000 \times 60}{64 \times 10^6} = 9.4\ \%$$

before any processing, and every other interrupt in the system now sees up to a microsecond of extra latency 100 000 times a second. In the second design a timer triggers the ADC, DMA moves each result into a 512-sample circular buffer, and the CPU is interrupted only at the half-full and full marks, once every 256 samples, or 391 times a second. At about 200 cycles per such interrupt the load falls to $391 \times 200/64 \times 10^6 = 0.12\ \%$, nearly eighty times less, and the sampling instants are now set by the timer rather than by interrupt latency, which removes sampling jitter as well (Book 3, Ch. 19 §19.5).

Functions outside the microcontroller

Partitioning also decides what stays off the MCU: analog front ends, radios, gate drivers and independent safety monitors such as an external watchdog (Section 27.11). A function stays off-chip when it needs a different process technology (high voltage, RF, precision analog), separate certification, or independence from the processor's failure modes.

25.3 Microcontroller selection

Selection criteria

The microcontroller binds firmware, tools and board layout for the product's life. The criteria, roughly in the order they eliminate candidates:

1. **Peripherals and pins.** Count every interface the product needs (Chapter 26): UARTs, SPI, I²C, CAN FD, USB, Ethernet MAC, ADC channels and resolution, timer channels for PWM and capture, and the GPIO left over. A missing CAN FD controller or too few PWM channels with complementary outputs eliminates a part immediately.
2. **Compute.** Estimate the cycles per second of the heaviest work (control loops, filtering, protocol stacks, cryptography) and add a margin of at least 2 for growth over the product's life. A floating-point unit (Cortex-M4F/M7/M33F) or DSP instructions matter for control and signal processing.
3. **Memory.** Flash for code, constants and possibly two firmware images (Section 25.9); RAM for buffers, stacks and the heap of any network stack (Section 25.4).
4. **Energy.** Active current per megahertz and, for battery devices, sleep current with the RTC running and RAM retained (Section 25.5).
5. **Security.** Secure boot support, TrustZone-M (Armv8-M), a hardware crypto engine, a true random-number generator, read-out protection and key storage — increasingly required by regulation such as the EU Cyber Resilience Act and by standards such as IEC 62443 for industrial devices.
6. **Qualification and supply.** Temperature grade (commercial 0–70 °C, industrial −40–85 °C, automotive AEC-Q100 Grade 1 −40–125 °C), longevity commitments (10–15 years from some vendors), second sources, price at volume.
7. **Ecosystem.** Compilers, debuggers, RTOS ports, middleware, team experience.

Representative parts

Table 25.3 — Representative microcontrollers (typical datasheet values; check current datasheets before designing).

Part (core)	Clock	Flash / RAM	Active current	Lowest retention mode	Notable features
ATmega328P (AVR, 8-bit)	20 MHz	32 KB / 2 KB	about 0.2 mA/MHz at 1.8 V	0.1 µA power-down	1.8–5.5 V, simple, very robust I/O
MSP430FR5969 (16-bit)	16 MHz	64 KB FRAM / 2 KB	about 100 µA/MHz	0.4 µA LPM3 with RTC	FRAM: fast, low-energy, high-endurance writes
STM32L073 (Cortex-M0+)	32 MHz	192 KB / 20 KB	about 76 µA/MHz	0.86 µA Stop with RTC and RAM	6 KB EEPROM, LCD driver, USB FS
nRF52840 (Cortex-M4F)	64 MHz	1 MB / 256 KB	about 52 µA/MHz (DC/DC)	1.5–3 µA System ON with RTC	BLE 5, 802.15.4, USB, crypto cell
STM32F407 (Cortex-M4F)	168 MHz	1 MB / 192 KB	about 40 mA at 168 MHz	about 2 µA Standby	Ethernet MAC, 2×CAN, USB OTG HS, 3 ADCs
STM32H743 (Cortex-M7)	480 MHz	2 MB / 1 MB	about 100–250 mA	a few µA Standby	Caches, double-precision FPU, FD-CAN, 16-bit ADCs
RP2040 (2× Cortex-M0+)	133 MHz	external QSPI / 264 KB	about 20–25 mA	about 0.2 mA dormant	Programmable I/O state machines, low cost
ESP32-C3 (RISC-V)	160 MHz	external or in-package 4 MB / 400 KB	20–30 mA CPU, about 300 mA Wi-Fi TX peaks	about 5 µA deep sleep	Wi-Fi and BLE, secure boot, flash encryption

Active current per megahertz spans a factor of five between families and sleep current a factor of a thousand, so a meter and a drive need different chips even when both use a Cortex-M4. The most powerful part is rarely the right one: the H7's caches bring timing variability (Section 27.10) and its standby current is beyond a coin cell.

25.4 Memory constraints

Where the bytes go

Chapter 17 introduced the memory map of a microcontroller and Chapter 18 the physics and organisation of SRAM and Flash. Here the concern is the budget. A linked firmware image is divided into sections that the linker places in Flash or RAM (Chapter 21 for compilation and linking):

- `.text` — machine code (Flash).
- `.rodata` — constant tables, strings (Flash).
- `.data` — initialised variables: stored in Flash, copied to RAM at boot (Section 25.8), so they cost both.
- `.bss` — zero-initialised variables (RAM only).
- **Stack(s)** — one main stack for interrupts and the startup code, plus one per task under an RTOS (RAM).
- **Heap** — dynamic allocation, if allowed (RAM).

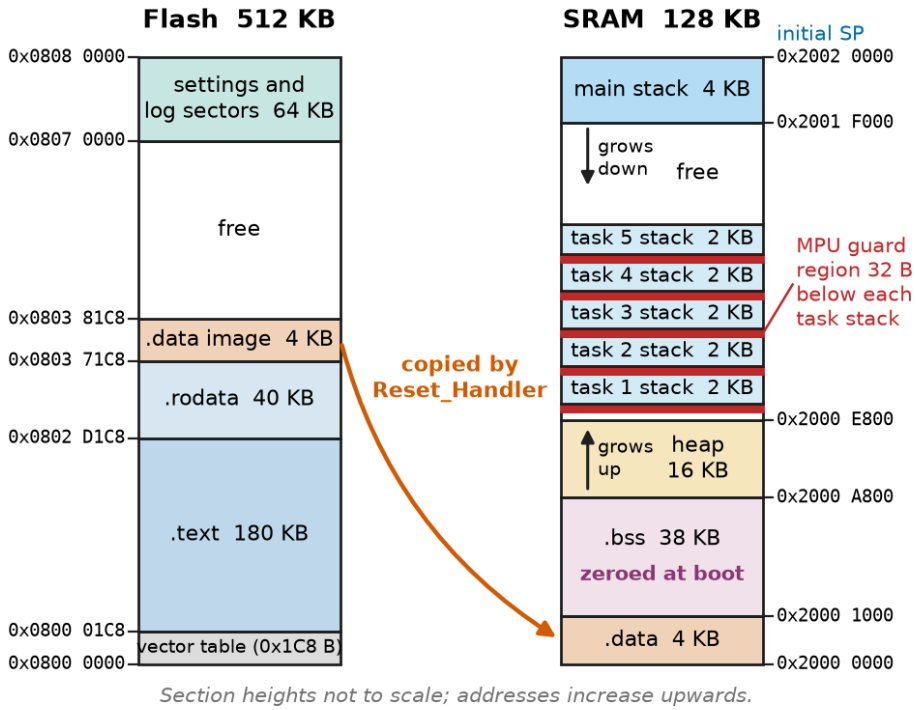


Figure 25.2 — Flash and SRAM layout of a Cortex-M firmware image after linking, with the boot-time copy of .data.

Figure 25.2 shows a typical layout. The build tools report section sizes after every link; a continuous-integration job should fail the build when Flash or RAM use crosses an agreed threshold (commonly 80 % at first release), because the product must leave room for features and security fixes over a decade.

Static allocation and the heap

Embedded coding standards (MISRA C, AUTOSAR) restrict dynamic allocation after initialisation: `malloc` often has unbounded execution time, fragmentation can make an allocation fail after weeks of uptime, and an unattended device has no good recovery. The alternatives are static buffers, fixed-size block pools (constant-time, no fragmentation) and allocation only at start-up.

Stack sizing

Stack overflow is the most common silent memory failure in embedded firmware, because it corrupts whatever lies below the stack without any immediate symptom. A stack must hold the deepest call chain of its task plus everything interrupts

can push on top of it:

$$S_{\text{task}} \geq S_{\text{calls,max}} + S_{\text{locals}} + \sum_{\text{nesting levels}} S_{\text{exception frame}}$$

On a Cortex-M, each exception entry pushes 8 words (32 bytes) of basic frame, or 26 words (104 bytes) when the floating-point context is stacked as well; with RTOS-based designs the interrupt frames land on the main stack rather than on each task stack. Static analysis of the call graph (the compiler's `-fstack-usage` output, or a dedicated tool) gives a bound when there is no recursion and no function pointer whose target set is unknown; measurement by **stack painting** (filling the stack with a known pattern at boot and later finding the high-water mark) confirms it. A guard region at the bottom of each stack, enforced by the memory protection unit, turns a silent overflow into an immediate fault (Section 27.11).

> **Status of the principle: Static stack bound**
> *Derived.* The bound from the call graph is exact for programs without recursion, unbounded loops of calls or unresolved indirect calls; measurement alone only shows the deepest path that the test happened to exercise.

Non-volatile memory and its endurance

Embedded systems store configuration, calibration and logs in non-volatile memory, and each technology has a different write model:

Table 25.4 — Non-volatile storage for embedded data (typical values).

Technology	Erase unit	Write time	Endurance (cycles)	Retention
On-chip NOR Flash (MCU)	2–128 KB page/sector	10–100 µs per word; 20 ms–2 s per erase	10 000 (some pages 100 000)	10–20 years
External SPI NOR Flash	4 KB sector, 64 KB block	0.5–3 ms per 256 B page; 45–400 ms per 4 KB sector erase	100 000	20 years
Emulated or true EEPROM	Byte or word	3–5 ms	100 000–1 000 000	20–40 years
FRAM (on-chip or SPI)	Byte, no erase	Bus speed (ns per byte)	10^{12}–10^{14}	10–100 years
SD card / eMMC (managed NAND)	Managed internally	Variable, 1–100+ ms stalls	Depends on internal wear levelling	Varies; power-fail behaviour vendor-specific

The constraint that bites is endurance multiplied by write frequency. Flash cannot overwrite a bit from 0 to 1 without erasing a whole page, so a log or a frequently updated setting must be spread across many pages (**wear levelling**) and written as appended records rather than in place.

Worked example — Will the log wear out the Flash? The water meter appends a 64-byte record (timestamp, volume, temperature, status, CRC) once a minute to its on-chip Flash, which has 2 KB pages rated for 10 000 erase cycles.

- One page holds $2048/64 = 32$ records, so it fills every 32 minutes.
- A year has $365 \times 24 \times 60 = 525\,600$ minutes. If the log used a single page, that page would be erased $525\,600/32 = 16\,425$ times a year and would exceed its rated endurance in about seven months.
- Spread over the 64 KB settings-and-log region of Figure 25.2 (32 pages), each page is erased $16\,425/32 = 513$ times a year, and the rated 10 000 cycles last $10\,000/513 = 19.5$ years, just beyond the fifteen-year target.

The circular log holds only the last 1 024 minutes (17 hours), so long-term history goes to the head end or to the external SPI NOR. Each record carries a sequence number and CRC so that the newest valid record can be found after a power failure in mid-write.

25.5 Power constraints

The average-current budget

A battery-powered device spends almost all of its life asleep and almost all of its energy awake. Its average current is the sleep current plus the charge of each activity divided by its period:

$$\bar{I} = I_{\text{sleep}} + \sum_{k} \frac{I_k \, t_k}{T_k}$$

and its life is the usable capacity divided by $\bar{I}$, reduced by self-discharge. The CMOS physics behind the currents — dynamic power proportional to $\alpha C V^2 f$ and leakage growing with temperature — is developed in Chapter 33; here the currents are datasheet numbers and the problem is accounting.

> **Status of the principle: Charge-budget model of battery life**
> *Approximate.* It treats the battery as a fixed charge reservoir. Real cells deliver less charge at high pulse current and low temperature, and their voltage sags under load; a lithium thionyl-chloride cell that supplies 2.6 Ah at 1 mA may brown out on a 400 mA radio pulse unless a buffer capacitor carries it (Book 5, *Electrical Energy and Machines*, Ch. 16 §16.2, for capacity and rate ratings).

Power modes

Every low-power microcontroller offers a ladder of modes that trade wake-up time and retained state for current.

Table 25.5 — Power-mode ladder of a typical low-power Cortex-M MCU (representative values at 3 V, 25 °C).

Mode	What runs	What is retained	Current	Wake-up time
Run (from Flash, 32 MHz)	CPU, peripherals	Everything	2–4 mA	—
Sleep (WFI)	Peripherals, DMA; CPU clock gated	Everything	0.5–1.5 mA	1–10 cycles
Stop / deep sleep	RTC, wake-up logic, low-power UART	RAM, registers	1–5 µA	5–20 µs
Standby	RTC, backup registers	Backup domain only (RAM lost)	0.3–1 µA	50 µs–1 ms (reboot)
Shutdown	Wake-up pins only	Nothing	30–100 nA	Full boot

The ladder makes a design rule: choose the deepest mode whose wake-up time and lost state the application can afford. Waking from standby means a full boot (Section 25.8), which costs time and charge each time; for an event every second it is usually better to stop with RAM retained. Leakage roughly doubles every 10–15 °C, so a sleep current quoted at 25 °C can be eight to twenty times larger at 70 °C inside a sunlit enclosure.

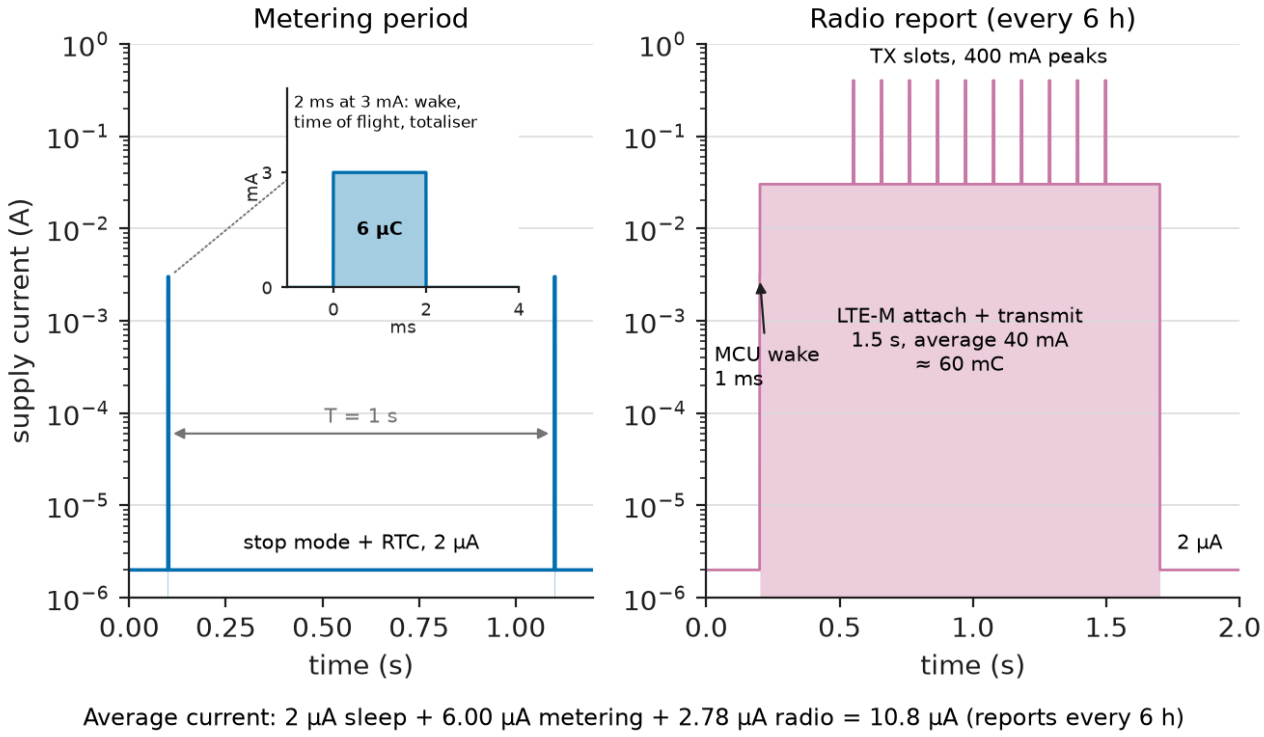


Figure 25.3 — Supply-current profile of the water meter over one metering period and over one radio report, on a logarithmic current axis.

Worked example — Will the meter last fifteen years? The meter of Figure 25.3 sleeps at 2 µA, wakes every second for 2 ms at 3 mA to measure flow, and every six hours runs its LTE-M modem for 1.5 s at an average 40 mA. Its 2.6 Ah lithium thionyl-chloride cell is derated to 80 % usable capacity, and self-discharge is taken as 1 % of nominal capacity per year. The listing computes the budget:

```
 # Average-current and lifetime budget for a duty-cycled battery node
HOURS_PER_YEAR = 8760.0

def average_current(sleep_a, events):
    """events: list of (name, current_A, duration_s, period_s)."""
    total = sleep_a
    for name, i, t, period in events:
        share = i * t / period
        total += share
        print(f"{name:<12s} {share * 1e6:7.2f} uA")
    return total

def lifetime_years(capacity_ah, i_avg_a, usable=0.8, self_discharge=0.01):
    # self-discharge as a fraction of nominal capacity per year
    i_sd = capacity_ah * self_discharge / HOURS_PER_YEAR
    return usable * capacity_ah / (i_avg_a + i_sd) / HOURS_PER_YEAR

events = [
    ("metering", 3e-3, 2e-3, 1.0),          # wake each second, 2 ms at 3 mA
    ("radio", 40e-3, 1.5, 6 * 3600.0),      # 1.5 s LTE-M burst every 6 h
]
i_avg = average_current(2e-6, events)
print(f"average      {i_avg * 1e6:7.2f} uA")
print(f"life         {lifetime_years(2.6, i_avg):7.1f} years")
```

It prints a metering share of 6.00 µA, a radio share of 2.78 µA, an average of 10.78 µA and a life of 17.3 years. Self-discharge alone is equivalent to $2.6 \times 0.01/8760 = 2.97$ µA, comparable with the radio. Three observations follow. The once-a-second measurement, not the radio, dominates, so shaving its 2 ms awake time to 1 ms (by running the CPU faster and returning to sleep sooner, "race to sleep") is worth more than any radio optimisation. Reporting hourly instead of every six hours raises the radio share to 16.7 µA and cuts the life to 8.6 years — the margin is thin, and the requirement must say how often the meter may talk. And the prototype of the opening, which listened for 10 s after each report, added another 40 mA × 10 s per report, almost seven times the transmit charge; together with an idle mode that kept the high-speed oscillator running, that is how a fifteen-year design becomes a five-year one.

Supply integrity

The brown-out reset threshold must sit above the minimum voltage for reliable Flash writes, because a write interrupted by a sagging supply corrupts its page; hence CRCs on log records, power-fail-safe updates (Section 25.9), bulk capacitance sized for radio bursts, and firmware that refuses to start an erase when the measured supply is near the threshold.

25.6 Firmware architecture

Three structures

The **firmware architecture** is the top-level structure that decides what runs when. Three structures cover nearly all small embedded systems, and each has a worst-case response time that can be stated in advance.

Superloop (round robin). `main` initialises the hardware and then calls each activity in turn forever. Interrupt handlers do the minimum (capture a byte, set a flag) and the loop does the rest:

```
#include <stdbool.h>

void board_init(void);
void sensor_sample(void);
void filter_update(void);
void protocol_handle_frame(void);
void display_refresh_if_due(void);
void sleep_until_interrupt(void);
void timer_clear_flag(void);

static volatile bool tick_10ms;          /* set by the timer ISR */
volatile bool rx_frame_ready;            /* set by the UART ISR  */

void TIM2_IRQHandler(void)
{
    timer_clear_flag();
    tick_10ms = true;
}

int main(void)
{
    board_init();
    for (;;) {
        if (tick_10ms) {
            tick_10ms = false;
            sensor_sample();             /* about 0.3 ms */
            filter_update();             /* about 0.8 ms */
        }
        if (rx_frame_ready) {
            rx_frame_ready = false;
            protocol_handle_frame();     /* up to 2 ms   */
        }
        display_refresh_if_due();        /* up to 12 ms over SPI */
        sleep_until_interrupt();         /* WFI, see text */
    }
}
```

The superloop is simple, needs no kernel and is easy to reason about. Its weakness is that every activity waits for every other: an event flagged just after its check is serviced only after a complete trip around the loop.

> **Status of the principle: Superloop worst-case response**
> *Derived.* With non-preemptive activities executed in a fixed cycle, the worst-case delay from a flag being set to its handler starting is the sum of the worst-case execution times of all activities in the loop, plus the interrupt time that steals from them.

Worked example — Is the superloop fast enough? In the listing the loop's worst case is $0.3 + 0.8 + 2 + 12 = 15.1$ ms of activities, plus interrupt overhead. A button press that must be acknowledged within 50 ms is served comfortably. A received protocol frame that must be answered within 5 ms (a Modbus slave with a tight master timeout, say) is not: the display refresh alone can hold it for 12 ms. The fixes are architectural. Splitting the display refresh into 1 ms slices bounds the loop at about 4.1 ms; moving the reply into the receive interrupt meets the deadline but makes the interrupt long; or the design moves to a preemptive scheduler.

The line `sleep_until_interrupt()` hides a classic defect. If an interrupt sets a flag after the loop has tested it but before the `WFI` instruction executes, the CPU sleeps with work pending until the next interrupt, up to 10 ms later. The standard

cure on Cortex-M is to disable interrupts with `PRIMASK`, re-check all flags, and execute `WFI` with interrupts still masked: a pending interrupt wakes the core even when masked, and it is taken as soon as interrupts are re-enabled.

Interrupt-driven (foreground/background). Time-critical work moves into prioritised interrupt handlers (the foreground) while the background loop handles the rest. The response of an interrupt-level activity is now bounded by interrupt latency and higher-priority handlers rather than by the loop (Section 27.2). The price is shared data between handlers and the loop, which must be protected (Section 27.6), and long handlers that delay everything of lower priority.

RTOS-based (preemptive multitasking). Activities become tasks with priorities; a kernel preempts a low-priority task the moment a higher-priority one becomes ready (Section 27.8). Response times are bounded by the scheduling analysis of Chapter 27, independently of how long low-priority tasks run. The price is kernel memory, a stack per task, context-switch overhead and new defects: priority inversion, deadlock and races between tasks.

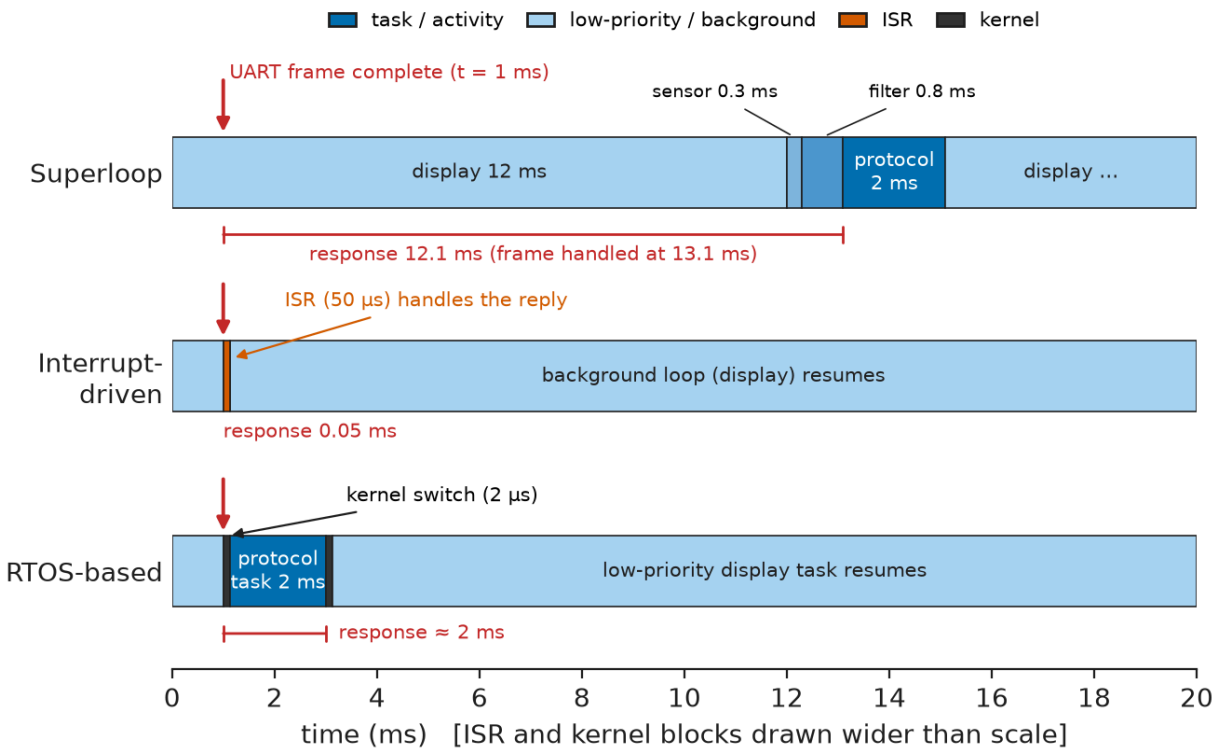


Figure 25.4 — The same event served by a superloop, an interrupt-driven design and an RTOS-based design.

Figure 25.4 compares the three on the same event. The rule is to choose the simplest structure whose worst-case response meets every deadline with margin: a superloop for a thermostat, interrupts for a controller with one fast loop, an RTOS once there are several activities with different deadlines or a network stack.

Table 25.6 — Firmware architectures compared (typical).

Property	Superloop	Interrupt-driven	RTOS-based
Worst-case response	Sum of all activities	Interrupt latency plus higher-priority ISRs	Scheduling analysis (Ch. 27)
Kernel footprint	None	None	5–20 KB Flash, 0.5–2 KB RAM plus stacks
Context-switch cost	None	Exception entry, about 12–30 cycles	Exception entry plus kernel, about 100–300 cycles
Typical defects	Starvation by long activities	Long ISRs, shared-data races	Priority inversion, deadlock, stack overflow
Typical use	Simple appliances, toys, chargers	Single-loop controllers, small sensors	Connected devices, multi-loop controllers

25.7 Peripheral drivers

What a driver is

A **peripheral driver** is the code that turns a block of memory-mapped registers into a service the rest of the firmware can use: "send these bytes", "start a conversion every 100 µs", "tell me when a CAN frame with this ID arrives". Chapter 17 describes the peripherals and their registers at the architecture level; Chapter 28 treats driver design, hardware abstraction and testability as disciplines. This section gives the overview that the architecture needs: the layers, the three service models and the shared-data boundary.

Layers

Drivers are layered. At the bottom, a vendor or CMSIS header describes each peripheral as a C structure of `volatile` registers at a fixed address. Above it, a vendor HAL or low-level library wraps common sequences. The product's own driver adds buffering, error handling and an interface independent of the chip. Middleware (file systems, TCP/IP, USB stacks) and the application use only that interface. The `volatile` qualifier matters at the bottom layer: it forces the compiler to perform every register read and write in program order, because a status register can change on its own and a write to a data register has side effects. It does not make anything atomic (Section 27.5).

Two register-level hazards recur. A **read-modify-write** of a shared register (`GPIOA->ODR |= mask;`) interrupted between read and write silently undoes any change an interrupt handler made to another bit; atomic set/reset registers (`BSRR` on STM32, `OUTSET` / `OUTCLR` on nRF) exist for exactly this reason. And **clear-on-read** status flags mean that a debugger or a careless second read can lose an event.

Polling, interrupt and DMA drivers

Table 25.7 — The three driver service models.

Model	How it works	CPU cost	Latency	Use
Polling	Software loops on a status flag	Busy for the whole transfer	Lowest for a single short event	Start-up code, very short transfers, bootloaders
Interrupt	Peripheral raises an IRQ per byte or event	One ISR per event, tens of cycles each	Bounded by interrupt latency	Low-rate serial ports, buttons, timer events
DMA	DMA controller moves blocks; IRQ per block	One ISR per block	Block-sized buffering delay	ADC streams, SPI displays, audio, Ethernet

The interrupt model needs a buffer between the interrupt handler, which produces data at the hardware's pace, and the task, which consumes it at its own. The standard structure is a single-producer, single-consumer ring buffer in which the handler writes only the head index and the task writes only the tail index, so no lock is needed as long as each index is updated with a single aligned store:

```
 #include <stdbool.h>
#include <stdint.h>
#include "stm32f4xx.h"   /* vendor CMSIS header: USART2, USART_SR_RXNE */

#define RX_SIZE 256u                       /* power of two */
static volatile uint8_t  rx_buf[RX_SIZE];
static volatile uint16_t rx_head;          /* written only by the ISR  */
static volatile uint16_t rx_tail;          /* written only by the task */
static volatile uint32_t rx_dropped;

void USART2_IRQHandler(void)
{
    if (USART2->SR & USART_SR_RXNE) {
        uint8_t byte = (uint8_t)USART2->DR;        /* reading DR clears RXNE */
        uint16_t next = (uint16_t)((rx_head + 1u) & (RX_SIZE - 1u));
        if (next != rx_tail) {
            rx_buf[rx_head] = byte;
            rx_head = next;                        /* publish after the data */
        } else {
            rx_dropped++;                          /* overflow is counted    */
        }
    }
}

bool uart_read_byte(uint8_t *out)
{
    if (rx_tail == rx_head)
        return false;                              /* empty */
    *out = rx_buf[rx_tail];
    rx_tail = (uint16_t)((rx_tail + 1u) & (RX_SIZE - 1u));
    return true;
}
```

The power-of-two size turns wrap-around into a mask; the head advances only after the byte is stored; overflow is counted, because a counter that rises in the field is a diagnosis. The buffer must hold all bytes that can arrive during the longest time the task may be kept from running: at 115 200 baud a byte arrives every 86.8 µs, so 256 bytes cover 22 ms. On a multicore part or with a cache between the CPU and DMA, the same pattern also needs memory barriers and cache maintenance (Section 27.5).

25.8 Boot processes

From power-on to the reset vector

When power is applied, nothing in the processor is trustworthy until the supply is in range and a clock is running. The **power-on reset** (POR) and **brown-out reset** circuits hold the core in reset until the supply crosses its threshold (typically 1.6–2.7 V, with tens of millivolts of hysteresis); an internal RC oscillator starts within microseconds and clocks the first instructions. Reset can also come from an external pin, the watchdog (Section 27.11), a software request or a low-power wake-up, and a reset-cause register records which one, so that firmware can log it — the first piece of field diagnosis.

On an Arm Cortex-M the hardware then does exactly two things: it loads the main stack pointer from the first word of the vector table (address 0x0000 0000, normally aliased to the start of Flash) and the program counter from the second word, the **reset vector**. On a RISC-V microcontroller the reset address is a fixed implementation constant instead, but the principle is the same: the first instruction executed is at an address set by hardware.

From the reset vector to main

The reset handler prepares the environment that C assumes, in a fixed order:

```
#include <stdint.h>

extern uint32_t _sidata, _sdata, _edata, _sbss, _ebss, _estack; /* linker */
int main(void);
void SystemInit(void);
void Reset_Handler(void);
void Default_Handler(void);

/* GCC-specific: the linker script places .isr_vector at the start of Flash */
__attribute__((section(".isr_vector"), used))
void (* const vector_table[])(void) = {
    (void (*)(void))&_estack,   /* word 0: initial main stack pointer */
    Reset_Handler,              /* word 1: reset vector               */
    Default_Handler,            /* NMI                                */
    Default_Handler,            /* HardFault                          */
    /* ... remaining system and peripheral vectors ... */
};

void Reset_Handler(void)
{
    SystemInit();                           /* clocks, PLL, Flash wait states */
    uint32_t *src = &_sidata;
    for (uint32_t *dst = &_sdata; dst < &_edata; )
        *dst++ = *src++;                    /* copy .data from Flash to RAM */
    for (uint32_t *dst = &_sbss; dst < &_ebss; )
        *dst++ = 0u;                        /* zero .bss */
    (void)main();
    for (;;) { }                            /* main must never return */
}

void Default_Handler(void) { for (;;) { } }
```

`SystemInit` starts the external crystal, configures and locks the PLL, and — before raising the clock — sets the Flash wait states, because Flash access time does not scale with the clock: an STM32F407 at 168 MHz and 3.3 V needs 5 wait states, with a prefetch buffer and instruction cache hiding most of them. Raising the clock before the wait states is a classic cause of a board that boots in the debugger and crashes on its own. The reset handler then copies `.data` and zeroes `.bss` (Figure 25.2), runs C++ static constructors if any (`__libc_init_array` in the GNU toolchain), and calls `main`. The symbol names come from the linker script and differ between toolchains; the order does not.

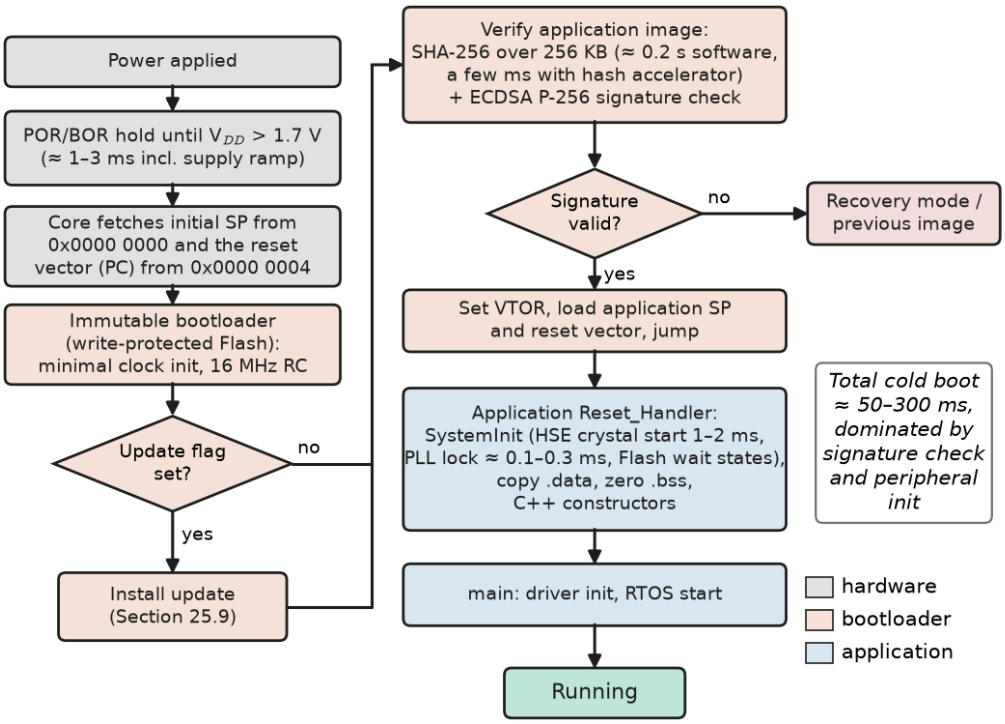


Figure 25.5 — Boot flow of a Cortex-M product with a secure first-stage bootloader, from power-on to the application's main loop, with typical durations.

Boot time

Boot time matters: an automotive ECU must answer on CAN within about 100–200 ms of wake-up, and a meter waking from standby pays each boot in charge. The contributors are the supply ramp, the crystal start-up (about 1–2 ms for an 8–25 MHz crystal, but up to one or two seconds for a 32.768 kHz watch crystal, which is why the RTC crystal is started early and left running), PLL lock (0.1–0.3 ms), image verification (Section 25.9) and peripheral and middleware initialisation, which usually dominates. Figure 25.5 shows the sequence with typical durations.

Multi-stage boot chains

Larger systems boot in stages: a Cortex-A SoC runs mask ROM, which loads a first-stage loader into on-chip SRAM; that initialises DRAM and loads U-Boot, which loads the kernel and device tree. Secure microcontrollers use a shorter chain: an immutable bootloader verifies the application before jumping to it. Each stage that checks the signature of the next extends a **chain of trust** rooted in code and keys that cannot be modified in the field.

> **Status of the principle: Chain of trust**
> *Derived.* If every stage verifies the next before executing it and the first stage and its public key are immutable, then only signed code can run; the guarantee fails if any stage has an exploitable bug, if the root key leaks, or if debug access can bypass the first stage.

25.9 Firmware updates

Why and how devices update

A connected device must be updatable: to fix defects, to close security vulnerabilities (now a regulatory obligation for many products — UNECE Regulation No. 156 for vehicle software updates, the EU Cyber Resilience Act for connected

products), and to add features. Updating is also the most dangerous thing the device ever does to itself. The design goals are that an update can be interrupted at any instant by power loss or a dropped link without bricking the device, that only authentic images are accepted, that a bad image is rolled back automatically, and that an attacker cannot reinstall an old, vulnerable version.

Bootloaders

A **bootloader** is a small program that runs after reset, decides which application image to run, and can install a new one. Three kinds are common:

- **ROM bootloaders** supplied by the chip vendor, entered through a boot pin or option byte, which accept an image over UART, USB DFU, CAN or I²C. They are invaluable in production programming and recovery, but they usually do not verify signatures and are disabled or locked in secured products.
- **Custom or open-source bootloaders** in protected Flash (MCUboot is the most widespread on Cortex-M, used by Zephyr and others) that verify and install images received by the application.
- **Multi-stage loaders** on Linux-class systems (U-Boot with A/B root file systems, or update frameworks such as SWUpdate, RAUC and Mender).

Image layout and the A/B scheme

The robust layout keeps two image slots so that a complete, verified copy of a working image always exists.

Table 25.8 — Flash partitioning of a 1 MB MCU for A/B updates (illustrative).

Region	Size	Contents
Bootloader	32 KB	Immutable, write-protected; holds public key hash and verification code
Slot 0 (primary)	448 KB	Image that runs
Slot 1 (secondary)	448 KB	Downloaded candidate image
Scratch	32 KB	Temporary sector for power-fail-safe swapping
Settings and log	64 KB	Configuration, calibration, wear-levelled log

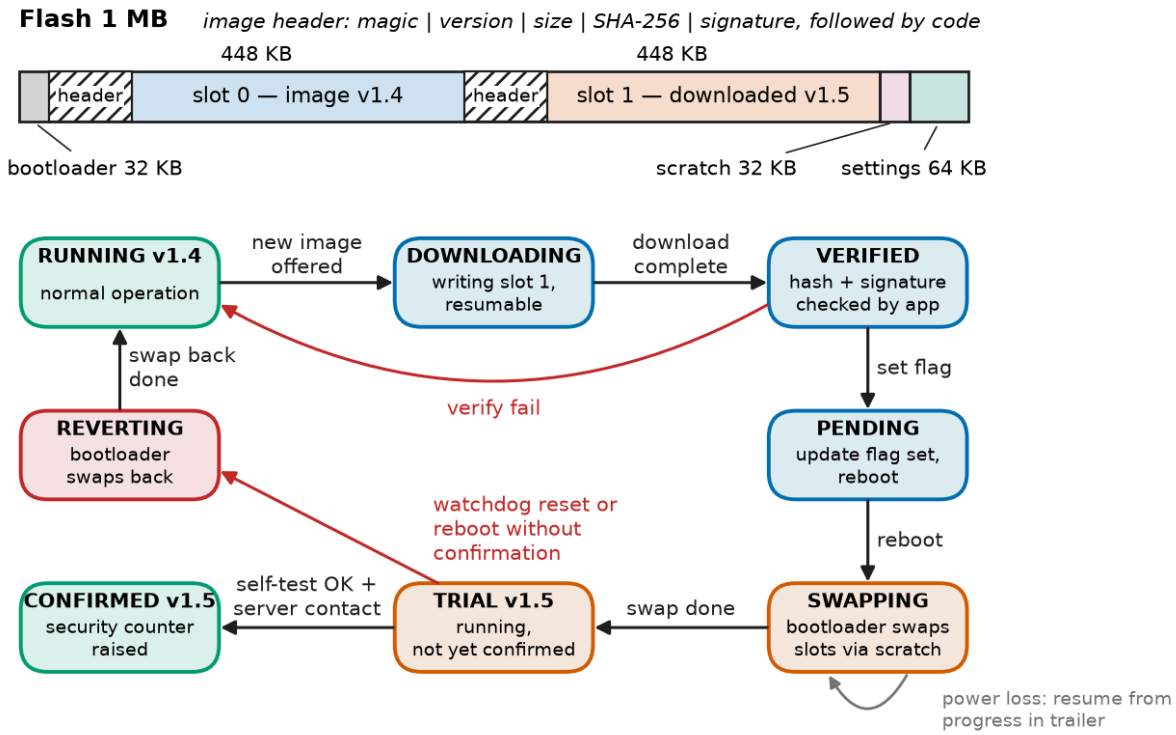


Figure 25.6 — Image slots and the update state machine of a power-fail-safe A/B firmware update with automatic rollback.

The sequence (Figure 25.6) runs as follows. The application downloads the new image into slot 1 while the old image keeps running, writing it in chunks that can be resumed after a dropped connection. It verifies the hash and signature and sets an update-pending flag. On reboot the bootloader verifies the image again — it trusts nothing the application says — and swaps the slots sector by sector through the scratch sector, recording its progress in a trailer so that a power failure mid-swap resumes where it stopped. The new image boots in **trial** state; if it passes its self-tests and reaches its server it **confirms** itself, and otherwise the next reset, whether a watchdog timeout or a deliberate reboot, makes the bootloader swap the old image back. Designs with execute-in-place from either slot avoid the swap (the bootloader simply jumps to the newer valid slot) at the cost of position-independent or twice-linked images; designs short of Flash use a single slot plus an external SPI Flash for the download, keeping the external copy until the new image is confirmed.

The bootloader's decision logic is short, and every line of it is load-bearing:

```
 #include <stdbool.h>
#include <stdint.h>

typedef struct {
    uint32_t magic;          /* marks a header; MCUboot uses 0x96f3b83d   */
    uint32_t version;        /* monotonic security version, anti-rollback */
    uint32_t image_size;     /* bytes of code following the header        */
    uint8_t  sha256[32];     /* hash of header fields and code            */
    uint8_t  signature[64];  /* ECDSA P-256 (r, s) over the hash          */
} image_header_t;

bool image_valid(const image_header_t *h);  /* magic, size, hash, signature */
bool update_pending(void);
bool trial_unconfirmed(void);
uint32_t security_counter(void);            /* in one-time-programmable bits */
void swap_slots_resumable(void);
void mark_trial(void);
void recovery_mode(void);                   /* never returns */
void jump_to_image(const image_header_t *h);

void boot_main(const image_header_t *slot0, const image_header_t *slot1)
{
    if (update_pending() && image_valid(slot1) &&
        slot1->version >= security_counter()) {
        swap_slots_resumable();             /* new image into slot 0  */
        mark_trial();                       /* app must confirm it     */
    } else if (trial_unconfirmed()) {
        swap_slots_resumable();             /* revert to the old image */
    }
    if (!image_valid(slot0))
        recovery_mode();                    /* never run a bad image   */
    jump_to_image(slot0);                   /* set VTOR and MSP, branch */
}
```

`jump_to_image` writes the image's vector-table address into the `VTOR` register, loads the main stack pointer from the image's first word and branches to its reset vector — exactly what the hardware does at reset (Section 25.8), repeated in software. Interrupts must be disabled and every peripheral the bootloader used returned to its reset state first, or the application inherits a half-configured system.

Securing the update

Authenticity. Images are signed by the manufacturer with a private key kept in a hardware security module; the device holds only the public key (or its hash) in immutable storage and verifies an ECDSA P-256 or Ed25519 signature over the image hash. A CRC detects accidental corruption but provides no security, because anyone can compute one.

Confidentiality. If the firmware contains intellectual property or secrets, the image is also encrypted (AES with a device or fleet key), and the device decrypts it while installing.

Anti-rollback. A monotonic **security counter**, stored in one-time-programmable fuses or a protected counter, records the minimum acceptable version. Raising it only after the new image confirms itself prevents an attacker from reinstalling a signed but vulnerable old image, without preventing rollback of a faulty update to the immediately previous version when the counter was not raised.

Manifest and transport. The device fetches a signed **manifest** describing the image (version, size, hash, applicability to the hardware revision, dependencies), as standardised by the IETF SUIT architecture (RFC 9019), then the image itself over TLS or DTLS, BLE, CAN (UDS-based flashing in vehicles) or a local service port.

Worked example — The cost of one over-the-air update. The meter receives a 256 KB image over LTE-M.

- At an effective application throughput of 100 kbit/s, the download takes $256 \times 1024 \times 8 / 100\,000 = 21$ s. At an average modem current of 150 mA during the transfer, it consumes $21 \times 0.15 = 3.1$ C, or 0.87 mAh, about 0.03 % of the cell's 2.6 Ah. A yearly update is affordable; a design that retries a failed download from the beginning a hundred times in a poor-coverage cellar is not, which is why downloads must resume from the last good chunk.
- Writing the image erases and programs 256 KB of Flash; with 2 KB pages at about 20 ms per page erase that is 128×20 ms $= 2.6$ s of erase time, during which code executing from the same Flash bank may stall. Dual-bank parts, or code that runs from RAM during the write, avoid stalling time-critical interrupts.
- Verification hashes the image with SHA-256: at about 50 cycles per byte in software on a 64 MHz Cortex-M4 it takes $262\,144 \times 50/64 \times 10^{6} = 0.2$ s, plus a signature check of the order of 0.1–0.2 s in software or milliseconds with a crypto accelerator. Doing it twice (application, then bootloader) is affordable and removes any reliance on the application's honesty.
- The swap through a scratch sector costs roughly three erases per sector per update; 10 000-cycle Flash permits thousands of updates.

> **Status of the principle: Power-fail-safe update**
> *Derived.* If at every instant at least one complete, verified image exists in non-volatile memory and the bootloader can determine from persistent state which one to run, an interruption at any point leaves the device bootable; the property fails if the bootloader itself is updated non-atomically or if Flash writes can be corrupted by brown-out without detection.

The bootloader is therefore minimal, almost never updated in the field, and tested harder than any other code in the product. The pilot meters of the opening bricked because their single-slot design erased the running image before the new one had been completely received.

What remains open

Open question	Where it is answered
How do GPIO, UART, SPI, I²C, CAN, USB, Ethernet and converter interfaces behave at the electrical and bit level?	Chapter 26
How are interrupt latency, task scheduling, priority inversion and watchdogs analysed and designed?	Chapter 27
How are hardware abstraction layers, driver modularity, state machines, low-power techniques and firmware validation engineered?	Chapter 28
How are peripherals, memory maps and interrupt controllers organised inside the microcontroller?	Chapter 17
How do SRAM, Flash and memory interfaces work at circuit level?	Chapter 18
How are compilation, linking and embedded-oriented C written?	Chapter 21
Where does CMOS dynamic and leakage power come from, and how does it scale?	Chapter 33
How is partitioning between processor, FPGA and accelerator quantified?	Chapter 35
How are batteries rated and managed at system level?	*Electrical Energy and Machines*, Ch. 16

Next: Chapter 26 follows the signals out of the microcontroller's pins — GPIO electrical limits, the serial buses and their bit timing, converter interfaces and sensors — and shows how to find out why a link that should work does not.

Chapter 26 — Embedded Interfaces

Part VI · Embedded and Real-Time Systems

An agricultural-robot prototype runs flawlessly in the lab and fails in the field within an hour. The I²C bus to its soil sensors hangs whenever a motor starts: the supply dips, a sensor browns out halfway through sending a byte, and when it recovers it holds the data line low forever, waiting for clock pulses the master will never send. The CAN network linking its wheel modules logs error frames only on the longest harness, because the designer chose 1 Mbit/s and a sample point that the cable's round-trip delay cannot meet. A UART link to the GPS works at 9600 baud and garbles at 115 200, because the 16 MHz crystal cannot divide down to that rate within tolerance. And the strain-gauge load cell reads 2 % high on hot days, because the excitation and the ADC reference come from different regulators. Every one of these faults lives at an interface — the boundary where firmware meets electrical reality — and every one is predictable from the electrical limits and bit timing that the standards specify.

> **What electrical and timing limits govern GPIO and the common serial buses, and how are they designed in from the standards' numbers?**
> **How are converters and sensors connected without losing accuracy, and how is a failing link diagnosed with a scope and logic analyser?**

Concept	Engineering destination
GPIO electrical limits	Driving LEDs, relays and MOSFET gates; level translation; ESD-robust inputs
UART and RS-232/RS-485	Debug consoles, GPS and modem links, Modbus RTU fieldbuses
SPI	Flash memory, displays, high-rate ADCs, IMUs, isolated measurement channels
I²C	Sensor buses, PMICs, EEPROMs, board management
CAN and CAN FD	Vehicle networks, EV battery management, industrial and agricultural machines
USB fundamentals	Device firmware for HID, virtual COM ports, DFU, USB-C power
Ethernet fundamentals	Industrial controllers, gateways, automotive Ethernet, PoE devices
ADC and DAC interfaces	Accurate acquisition from on-chip and external converters
Sensor interfaces	Temperature, strain, current-loop and digital sensors in instruments and machines
Communication troubleshooting	Finding the fault on a bus with a scope, logic analyser and protocol decoder

Roadmap. The pin itself (26.1) → asynchronous serial (26.2) → the two on-board synchronous buses, SPI (26.3) and I²C (26.4) → the robust field and vehicle bus, CAN (26.5) → the two big standard interfaces, USB (26.6) and Ethernet (26.7) → analog boundaries: converters (26.8) and sensors (26.9) → finding faults (26.10).

> **Boundary.**
> Chapter 17 describes the on-chip GPIO ports, serial controllers and converters as microcontroller peripherals — registers, clocks, interrupts and DMA. This chapter treats what happens outside the pin: electrical levels, bit timing, protocols and wiring. Transmission-line behaviour of traces and cables (reflections, termination, differential pairs) is from *Circuits, Signals and Communications*, Ch. 14 §14.7 and §14.10; CRCs from Ch. 28 §28.6; line codes from Ch. 27 §27.3; sampling and quantization from Ch. 19.

26.1 GPIO

The pin as a circuit

A general-purpose I/O pin is a small CMOS circuit (Figure 26.1): a push-pull output stage whose PMOS half can be disabled for open-drain operation, optional weak pull-up and pull-down resistors, a Schmitt-trigger input buffer, and clamp diodes from the pad to each supply rail that protect the gate oxide from electrostatic discharge. Firmware sees a bit; the outside world sees a voltage source with a few tens of ohms of output resistance, an input with picofarads of capacitance and microamps of leakage, and diodes that conduct when the pin is driven beyond the rails.

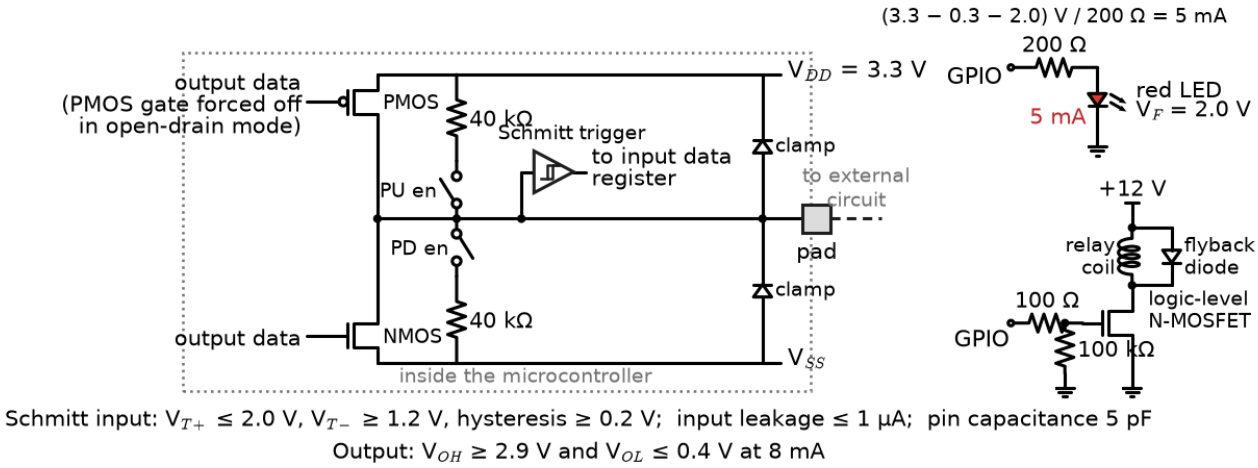


Figure 26.1 — Internal structure of a 3.3 V microcontroller GPIO pin and two typical external connections.

Electrical limits

Table 26.1 — Representative electrical characteristics of a 3.3 V microcontroller GPIO (typical datasheet values; JESD8C-style LVCMOS levels).

Parameter	Typical value	Consequence
$V_{IL,max}$ / $V_{IH,min}$	$0.3V_{DD}$ / $0.7V_{DD}$ (CMOS); some pins 0.8 V / 2.0 V (TTL-compatible)	Thresholds for level compatibility
Schmitt hysteresis	0.1–0.4 V	Clean edges from slow or noisy signals
V_{OL} at 8 mA	≤ 0.4 V	Sink capability; rises to about 1.3 V at 20 mA
V_{OH} at 8 mA	≥ V_{DD} − 0.4 V	Source capability
Current per pin (abs. max)	±20–25 mA	Exceeding it stresses metallisation
Total current per port or package	80–240 mA	Limits how many LEDs one MCU can drive
Injection current (clamp diodes)	±5 mA per pin, ±25 mA total	Overdriven pins can power the chip through the clamps
Internal pull-up / pull-down	30–50 kΩ	Too weak for fast or noisy lines
Input leakage	≤ 1 µA	Limits high-impedance dividers
Slew-rate setting	2–100 MHz edge classes	Faster edges radiate more and ring more
ESD robustness	2 kV HBM (JEDEC JS-001) at chip level	System pins need IEC 61000-4-2 protection (±8 kV contact)

Three consequences come up constantly. First, **level compatibility**: a 3.3 V CMOS output (high ≥ 2.9 V) drives a 5 V TTL-threshold input (high ≥ 2.0 V) safely but not a 5 V CMOS input ($0.7 \times 5 = 3.5$ V), which needs a translator such as a 74HCT buffer or a dedicated level shifter; a 5 V signal into a non-5-V-tolerant 3.3 V pin forward-biases the upper clamp diode and must be limited below the injection current, or the chip may be back-powered through its I/O. Second, **load current**: an LED with a 2.0 V forward drop at 5 mA from a pin whose output drops about 0.3 V at that current needs $(3.3 - 0.3 - 2.0)/5\ \text{mA} = 200\ \Omega$; anything heavier than a few milliamps — relays, motors, long cables — gets a transistor or driver, with a flyback diode across inductive loads. Third, **external exposure**: a pin that leaves the board through a connector meets ESD, cable surges and miswiring, and needs a series resistor (100 Ω–1 kΩ), a TVS diode rated to IEC 61000-4-2 and often an RC filter, because the chip's own 2 kV HBM rating covers handling in the factory, not a user's finger.

Inputs from the real world

A mechanical switch bounces for 1–10 ms; read directly, one press looks like dozens. Debouncing is done either in hardware (an RC filter of roughly 10 ms into a Schmitt input) or in firmware (sample every 1–5 ms and accept a new state only after it has been stable for several samples). An input left floating picks up mains hum and draws shoot-through current in its input buffer; every unused pin should be configured as an analog input or driven output, which is also the lowest-current configuration.

26.2 UART

The frame

A **universal asynchronous receiver-transmitter** (UART) sends bytes without a shared clock. The line idles high (mark); each character starts with a low **start bit**, followed by 5–9 data bits sent least-significant first, an optional parity bit and one or two high **stop bits** (Figure 26.2). The common format 8N1 — eight data bits, no parity, one stop bit — takes 10 bit times per byte, so 115 200 baud carries 11 520 bytes per second and each byte takes 86.8 µs.

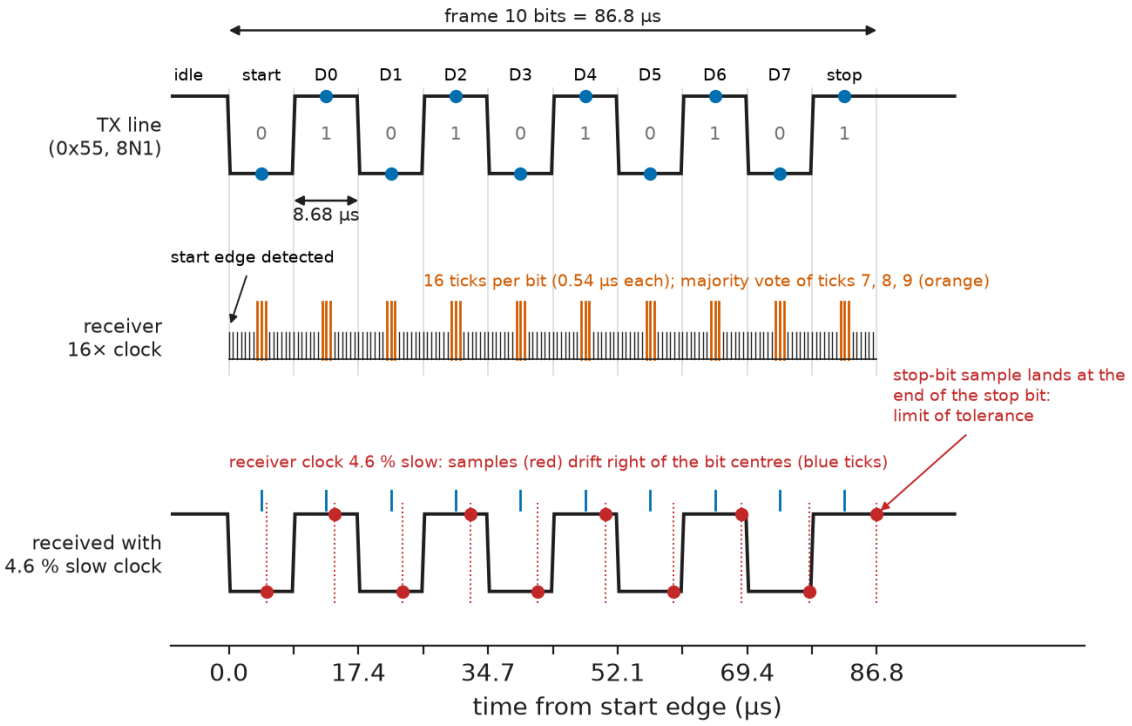


Figure 26.2 — UART 8N1 frame for the byte 0x55 at 115 200 baud, with the receiver's 16x oversampling and mid-bit sampling points, and the effect of a clock mismatch.

The receiver detects the falling edge of the start bit and then samples each bit near its centre. With **16× oversampling** it locates the start edge to within 1/16 of a bit and samples at the middle (often as a majority vote of three samples), so it can tolerate drift until the last sample — the stop bit, 9.5 bit times after the start edge for 8N1 — leaves the stop bit.

> **Status of the principle: UART clock-tolerance budget**
> *Derived.* The last sample, at 9.5 bit times for 8N1, must stay within half a bit of its nominal position less the start-edge uncertainty, so the combined transmitter–receiver rate error must satisfy $|\Delta f/f| < (0.5 - 1/16)/9.5 = 4.6\ \%$ with 16× oversampling and 3.9 % with 8×; edge distortion from slow cables and noise filtering reduce it further, which is why datasheets quote about 3.5–4 % and designers budget ±2 % per end.

Worked example — Why 115 200 baud fails on a 16 MHz AVR. The baud generator divides the peripheral clock by $16(\mathrm{UBRR}+1)$ with an integer divisor. For 115 200 baud, $16 \times 10^6/(16 \times 115\,200) = 8.68$, which rounds to 9, giving an actual rate of $16 \times 10^6/(16 \times 9) = 111\,111$ baud, an error of −3.5 %. That alone consumes most of the 4.6 % budget; a GPS module whose own oscillator is 1 % fast on the other side brings the total to 4.5 %, and characters fail intermittently. Switching to 8× oversampling (double-speed mode) gives a divisor of 17.36 → 17 and 117 647 baud, +2.1 %, which works; 9600 baud gives 104.17 → 104 and only +0.16 %, which is why it always worked. The clean fixes are a "baud-rate crystal" (14.7456 MHz divides exactly to every standard rate) or a UART with a fractional divider: an STM32 at 84 MHz with a 4-bit fractional divisor reaches 115 226 baud, +0.02 %.

Physical layers

The UART defines framing, not voltages. Three physical layers dominate:

Table 26.2 — UART physical layers (from the standards).

Layer	Signalling	Levels	Distance and rate	Topology
Logic level (TTL/CMOS)	Single-ended	0 / 3.3 V or 5 V	On-board, tens of cm; Mbit/s	Point to point
RS-232 (TIA-232-F)	Single-ended, inverted	Driver ±5 to ±15 V into 3–7 kΩ; receiver threshold ±3 V	15 m nominal at up to 20 kbit/s (2500 pF load limit); 115 kbit/s over short cables in practice	Point to point
RS-485 (TIA-485-A)	Differential, half or full duplex	Driver ≥ 1.5 V differential into 54 Ω; receiver ±200 mV; common mode −7 to +12 V	1200 m at 100 kbit/s; about 10 Mbit/s at 10–15 m	Multidrop bus, 32 unit loads, 120 Ω termination at both ends

RS-485 is the industrial workhorse: Modbus RTU, DMX512 stage lighting, building automation and many proprietary fieldbuses use it. Its differential receiver rejects the ground shifts of a factory floor, the termination resistors absorb reflections on long cables (Book 3 §14.7), and **failsafe biasing** (a pull-up on A and a pull-down on B, or receivers with built-in failsafe) defines the idle state when no driver is active, so the line does not float through the ±200 mV threshold and generate false start bits. Protocols on top use time to delimit frames: Modbus RTU ends a frame with 3.5 character times of silence, 4.0 ms at 9600 baud with its 11-bit characters — which a firmware that pauses mid-frame to service other work can break.

26.3 SPI

Signals and modes

The **Serial Peripheral Interface** is a synchronous, full-duplex bus with four wires: a clock (SCK) driven by the controller, data out (MOSI, or COPI/SDO), data in (MISO, or CIPO/SDI) and an active-low chip select (CS) per target device. Each clock cycle shifts one bit out of the controller's shift register and one bit in from the target's, so every transfer is an exchange. SPI has no formal standard; Motorola's original definition and each device's datasheet set its rules.

The clock polarity (CPOL, idle level of SCK) and phase (CPHA, which edge samples) give four **modes**:

Table 26.3 — SPI modes.

Mode	CPOL	CPHA	Clock idle	Data sampled on	Typical devices
0	0	0	Low	Rising edge	Most Flash memories, many ADCs, SD cards in SPI mode
1	0	1	Low	Falling edge	Some ADCs and DACs
2	1	0	High	Falling edge	Less common
3	1	1	High	Rising edge	Many sensors and Flash memories (mode 0 and 3 both supported)

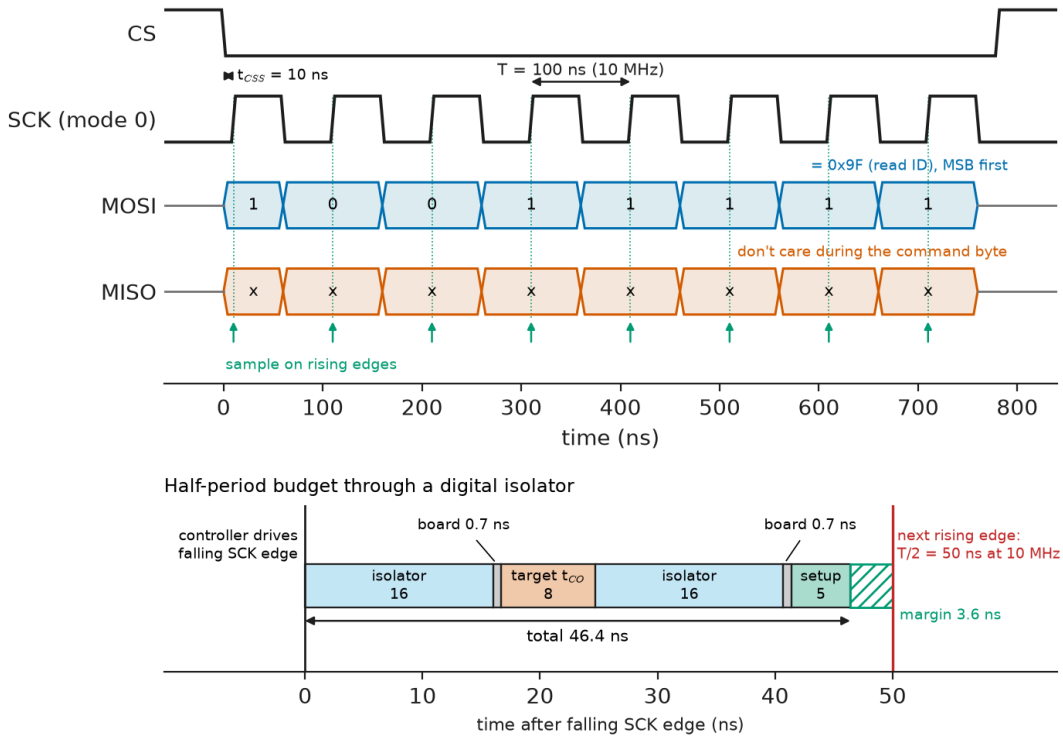


Figure 26.3 — SPI mode 0 transfer of one byte and the round-trip timing budget that limits the clock rate through a digital isolator.

The round-trip limit

SPI clocks of 50–100 MHz are common for Flash memories on short traces, but the bus has a built-in round-trip constraint (Figure 26.3). In mode 0 the target changes MISO after the falling SCK edge and the controller samples it on the next rising edge, half a period later. The clock must travel out, the target must respond, and the data must travel back within that half-period:

$$\frac{T_{SCK}}{2} \geq t_{\text{out}} + t_{CO,\text{target}} + t_{\text{back}} + t_{SU,\text{controller}}$$

> **Status of the principle: SPI round-trip timing**
> *Derived.* Follows from the sampling edge being half a period after the launching edge in modes 0–3; it is relaxed by controllers that can delay their sampling point (late sampling, or clock feedback from a returned SCK) and is unchanged by the controller's own clock rate.

Worked example — Isolated SPI to a battery-monitor ADC. A BMS reads a cell-voltage ADC through a digital isolator with a 16 ns maximum propagation delay in each direction. The ADC's clock-to-output is 8 ns, the controller's setup time 5 ns, and each board crossing 0.7 ns. The half-period must be at least $2 \times 16 + 2 \times 0.7 + 8 + 5 = 46.4$ ns, so $f_{SCK} \leq 1/(2 \times 46.4\ \text{ns}) = 10.8$ MHz. Without the isolator the same parts would run to $1/(2 \times 14.4\ \text{ns}) = 34.7$ MHz. The isolator, not the ADC, sets the speed: designers run such links at 8–10 MHz, use controllers with adjustable sample delay, or use isolators with a returned clock.

Variants and practice

Quad- and octal-SPI use 4 or 8 data lines, often double data rate, to read Flash at 50–400 MB/s and execute code in place. Practical rules: pull CS high during reset, respect each device's CS set-up and hold times, and make sure every target tri-states MISO when deselected — one that does not corrupts the whole bus.

26.4 I²C

Open-drain signalling

The **Inter-Integrated Circuit** bus (NXP UM10204) uses two open-drain lines, serial clock (SCL) and serial data (SDA), each pulled high by a resistor. Any device can pull a line low; no device ever drives it high. This **wired-AND** makes multi-controller arbitration, clock stretching and acknowledgement possible on two wires — and makes the rising edge an RC charging curve set by the pull-up and the bus capacitance.

Table 26.4 — I²C speed modes (NXP UM10204).

Mode	Max clock	Max rise time (30–70 %)	Max bus capacitance	V_{OL} / sink current	$t_{LOW,min}$ / $t_{HIGH,min}$
Standard (Sm)	100 kHz	1000 ns	400 pF	0.4 V at 3 mA	4.7 / 4.0 µs
Fast (Fm)	400 kHz	300 ns	400 pF	0.4 V at 3 mA	1.3 / 0.6 µs
Fast-mode Plus (Fm+)	1 MHz	120 ns	550 pF	0.4 V at 20 mA	0.5 / 0.26 µs
High-speed (Hs)	3.4 MHz	10–80 ns	100–400 pF	Current-source pull-up	—

The transaction

A transaction (Figure 26.4) begins with a **START** — SDA falling while SCL is high — followed by a 7-bit address and a read/write bit. Data change only while SCL is low; a change of SDA while SCL is high is a START or a **STOP** (SDA rising while SCL high). After every byte the receiver pulls SDA low during a ninth clock to **acknowledge** (ACK); leaving it high is a NACK. A **repeated START** turns a write (of a register address) into a read without releasing the bus. A target that needs time holds SCL low after the controller releases it — **clock stretching** — and the controller must wait. Reading six bytes of accelerometer data from a register at 400 kHz takes about 84 bit times including address phases and conditions, 210 µs.

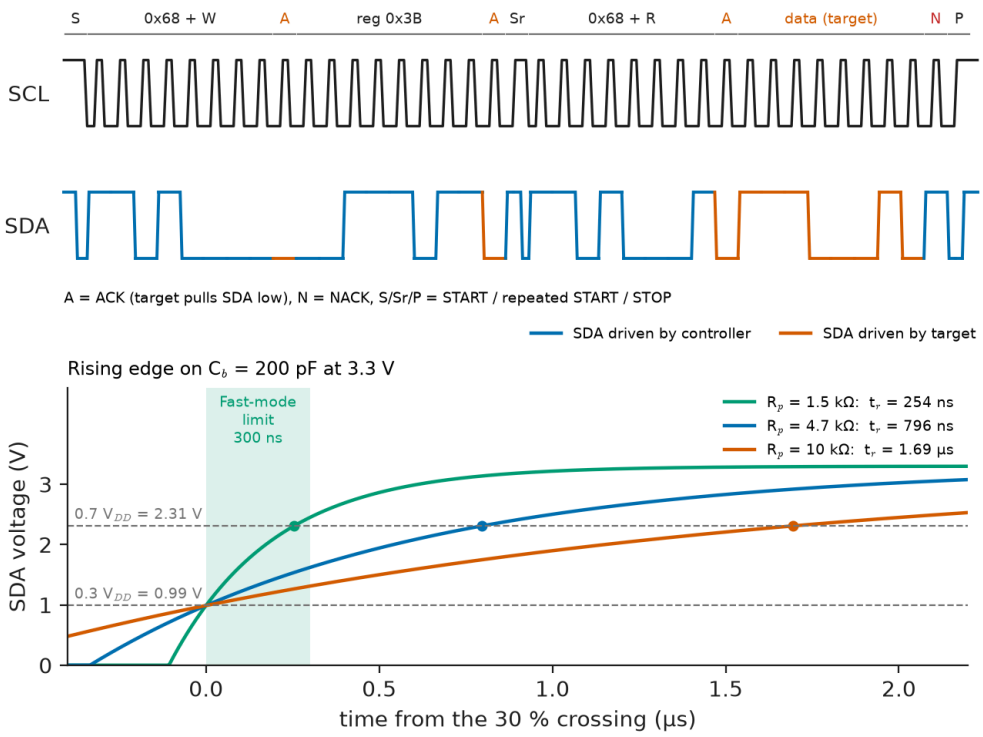


Figure 26.4 — I2C register read with repeated START, and the RC rising edge set by the pull-up resistor and bus capacitance.

Sizing the pull-up

The pull-up has a minimum and a maximum. It must be large enough that a device sinking its rated current reaches V_{OL}, and small enough that the line rises from 30 % to 70 % of V_{DD} within the mode's rise-time limit. For an RC edge that rise time is $t_r = RC\ln(0.7/0.3) = 0.8473\,RC$, so

$$R_{p,\min} = \frac{V_{DD} - V_{OL,\max}}{I_{OL}}, \qquad R_{p,\max} = \frac{t_r}{0.8473\,C_b}$$

> **Status of the principle: I²C pull-up window**
> *Derived.* From the static sink requirement and the exponential RC charge between the 30 % and 70 % thresholds of UM10204; it assumes a lumped bus capacitance and a purely resistive pull-up, and is replaced by the current-source pull-ups of Hs mode or active rise-time accelerators on long, heavily loaded buses.

Worked example — Pull-ups for a 400 kHz sensor bus. A 3.3 V bus carries an IMU, a barometer, an EEPROM and a PMIC; with traces and pins its capacitance is estimated at 200 pF. Then $R_{p,\min} = (3.3 - 0.4)/3\ \text{mA} = 967\ \Omega$ and $R_{p,\max} = 300\ \text{ns}/(0.8473 \times 200\ \text{pF}) = 1.77\ \text{k}\Omega$. A 1.5 kΩ pull-up gives $t_r = 254$ ns and 2.2 mA sink current, inside both limits. The habitual 4.7 kΩ gives 796 ns — legal at 100 kHz (limit 1000 ns, $R_{p,\max} = 5.9$ kΩ) but not at 400 kHz, where the edges arrive late, the effective high time shrinks and devices miss bits. At 400 pF, $R_{p,\max}$ falls to 885 Ω, below $R_{p,\min}$: no resistor works, and the bus must be split with a buffer or slowed down.

Bus hangs and recovery

I²C's weakness is the failure of the opening. If a target is mid-way through sending a 0 bit when the controller resets (or the target browns out and resumes in the wrong state), the target holds SDA low and waits for SCL edges. The controller sees a permanently busy bus and cannot even generate a START. UM10204 prescribes the **bus clear**: the controller toggles SCL up to nine times until the target finishes its byte and releases SDA, then issues a STOP. SMBus devices additionally reset themselves if SCL is held low for more than 25–35 ms. A robust driver runs the recovery at boot and after every timeout:

```
#include <stdbool.h>

/* Board-specific primitives: lines are open-drain with pull-ups. */
void scl_low(void);  void scl_release(void);  bool scl_is_high(void);
void sda_low(void);  void sda_release(void);  bool sda_is_high(void);
void delay_us(unsigned us);

/* Bus clear (NXP UM10204): clock until the target releases SDA,
 * at most nine pulses, then START + STOP to reset every target. */
bool i2c_bus_clear(void)
{
    sda_release();
    for (int i = 0; i < 9 && !sda_is_high(); i++) {
        scl_low();      delay_us(5);           /* 100 kHz pace */
        scl_release();  delay_us(5);
        for (int t = 0; t < 1000 && !scl_is_high(); t++)
            delay_us(1);                       /* allow clock stretching */
    }
    if (!sda_is_high())
        return false;                          /* stuck: power-cycle target */
    sda_low();      delay_us(5);               /* START (SCL high)  */
    sda_release();  delay_us(5);               /* STOP  (SCL high)  */
    return sda_is_high() && scl_is_high();
}
```

The function runs with the pins switched from the I²C peripheral to GPIO open-drain mode. If nine clocks do not free the bus, the target is latched up and only cutting its supply helps — which is why robust designs power sensors from a GPIO-switched rail or a load switch.

26.5 CAN

Physical layer

The **Controller Area Network** (ISO 11898-1 for the data link, ISO 11898-2 for the high-speed physical layer) was designed by Bosch for cars and now runs trucks, tractors, lifts, medical devices, EV battery packs and industrial machines (CANopen, J1939). It is a differential two-wire bus, CAN_H and CAN_L, terminated with 120 Ω at each physical end, so a healthy bus measures 60 Ω between the lines with power off — the first check in any CAN fault-finding. Its levels are not symmetric logic states but **dominant** and **recessive**: in the recessive state the transceiver drives nothing and both lines rest near 2.5 V; in the dominant state it pulls CAN_H to about 3.5 V and CAN_L to about 1.5 V. A dominant bit from any node overrides recessive bits from all others, the wired-AND that the protocol builds on.

Table 26.5 — ISO 11898-2 high-speed CAN physical-layer values.

Parameter	Value
Recessive differential output	−0.5 to +0.05 V (nominal 0 V, both lines about 2.5 V)
Dominant differential output	1.5–3.0 V (nominal 2.0 V)
Receiver thresholds	Recessive below 0.5 V, dominant above 0.9 V
Cable characteristic impedance; termination	120 Ω nominal; 120 Ω at each end
Propagation delay of cable	About 5 ns/m
Transceiver loop delay (TXD to RXD)	Typically 100–255 ns
Classical CAN bit rates and practical bus length	1 Mbit/s about 40 m; 500 kbit/s about 100 m; 250 kbit/s about 250 m; 125 kbit/s about 500 m

Arbitration, the frame and bit stuffing

Every node may start transmitting when the bus is idle. Each transmits its identifier bit by bit while reading the bus back; a node that sends recessive and reads dominant has lost arbitration and becomes a receiver, without destroying the winner's frame. The lowest identifier wins, so the identifier is also the message priority — a property that makes CAN analysable as a fixed-priority real-time system (Section 27.3).

A classical base-format data frame (CAN 2.0A) contains a start-of-frame bit, an 11-bit identifier, RTR, IDE and reserved bits, a 4-bit data length code, 0–8 data bytes, a 15-bit CRC with delimiter, an acknowledge slot in which every correct receiver drives dominant, and a 7-bit end of frame, followed by 3 bits of interframe space. To keep receivers synchronised, the transmitter inserts a complementary **stuff bit** after every five identical bits from the start of frame to the end of the CRC. Error handling is built in: every node checks the CRC, the bit stream, the stuffing and the frame format, and a node that sees an error destroys the frame with an error flag so that all nodes discard it and the transmitter retries. Transmit and receive error counters move a persistently faulty node from **error-active** to **error-passive** (counter above 127) and finally **bus-off** (transmit counter above 255), so that a broken node silences itself instead of jamming the network.

Bit timing

Each bit is divided into **time quanta** (tq) of the CAN controller's prescaled clock and split into segments (Figure 26.5): a 1-tq synchronisation segment in which edges are expected, a propagation segment, and two phase segments with the **sample point** between them. Phase segments lengthen or shorten (by up to the synchronisation jump width) to resynchronise on each recessive-to-dominant edge.

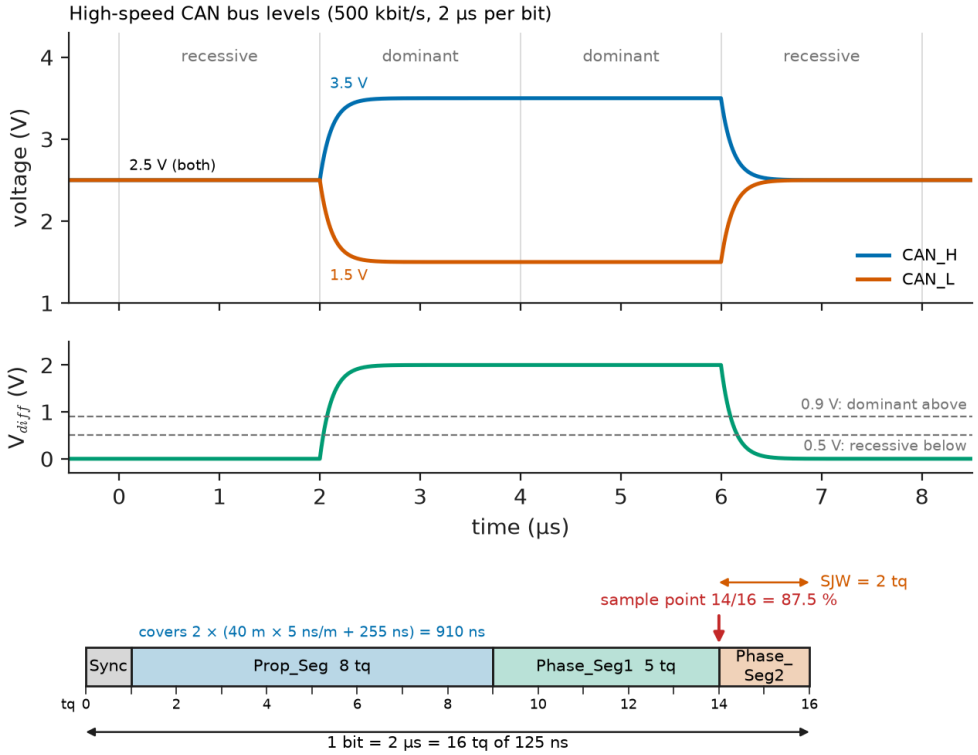


Figure 26.5 — High-speed CAN bus levels and the segmentation of one bit at 500 kbit/s.

The propagation segment must cover the round trip of a dominant bit between the two most distant nodes, because during arbitration a node must see the other end's dominant bit before it samples:

$$t_{\mathrm{PROP}} \geq 2\,(t_{\mathrm{cable}} + t_{\mathrm{loop}})$$

with t_{loop} the transceiver's TXD-to-RXD delay. This is why bit rate and bus length trade against each other.

> **Status of the principle: CAN bit-rate–length trade-off**
> *Derived.* Arbitration requires every node to sample a bit after the farthest node's response has arrived, so the round-trip delay must fit inside the bit before the sample point; the classical length figures assume 5 ns/m cable and fast transceivers, and CAN FD escapes the limit only in its data phase, after arbitration is over.

Worked example — Bit timing for a 40 m tractor harness at 500 kbit/s. The controller runs from 40 MHz with a prescaler of 5, so $t_q = 125$ ns and the 2 µs bit has 16 tq. The harness is 40 m (200 ns one way) and the transceiver loop delay is at most 255 ns, so $t_{\mathrm{PROP}} \geq 2(200 + 255) = 910$ ns, which needs $910/125 = 7.3$, rounded up to 8 tq. With Sync = 1, Prop = 8, Phase1 = 5 and Phase2 = 2 tq, the sample point is at $(1 + 8 + 5)/16 = 87.5\ \%$, the value recommended for automotive networks (SAE J2284, CiA 601). With these transceivers the same harness at 1 Mbit/s would need 910 ns of a 1000 ns bit before the sample point, leaving no room for the phase segments: the robot of the opening logged errors precisely on its longest cable.

Frame length, bus load and CAN FD

Because the frame length depends on stuffing, timing analysis uses the worst case. The listing computes both and the bus load of a set of periodic messages:

```
 # Classical CAN 2.0A frame length and bus load (ISO 11898-1)
from math import floor

def can_frame_bits(n_bytes, worst_stuffing=True):
    """Bits on the wire for a standard (11-bit ID) data frame, incl. 3-bit IFS."""
    stuffed_region = 34 + 8 * n_bytes          # SOF ... end of CRC field
    fixed = stuffed_region + 13                # CRC delim, ACK, EOF, IFS
    stuff = floor((stuffed_region - 1) / 4) if worst_stuffing else 0
    return fixed + stuff

def bus_load(messages, bit_rate):
    """messages: list of (count, n_bytes, period_s)."""
    bits_per_s = sum(k * can_frame_bits(n) / p for k, n, p in messages)
    return bits_per_s / bit_rate

ecu = [(16, 8, 0.010), (40, 8, 0.100)]          # body-network example
print("frame bits:", can_frame_bits(8, False), "nominal,",
      can_frame_bits(8), "worst case")
print(f"bus load at 500 kbit/s: {bus_load(ecu, 500e3):.1%} (worst case)")
```

An 8-byte frame is 111 bits nominally and 135 bits with worst-case stuffing, 222–270 µs at 500 kbit/s. Sixteen messages every 10 ms and forty every 100 ms load the bus to 54.0 % in the worst case — within the 50–70 % that integrators typically allow, leaving headroom for diagnostics and error retransmissions. With 8 bytes carrying only 64 of 135 bits, classical CAN is less than half efficient; **CAN FD** (ISO 11898-1:2015) keeps arbitration at the nominal rate but switches to 2–5 Mbit/s (up to 8 in short networks) for the data phase, carries up to 64 bytes, and uses 17- or 21-bit CRCs. CAN XL extends payloads to 2048 bytes and data rates to about 20 Mbit/s.

26.6 USB fundamentals

Topology and signalling

USB is a **host-centred, polled** bus: the host schedules every transaction and devices speak only when asked. USB 2.0 defines low speed (1.5 Mbit/s), full speed (12 Mbit/s) and high speed (480 Mbit/s) over one twisted pair, D+ and D−, of 90 Ω differential impedance, up to 5 m per cable segment. Full- and low-speed use 3.3 V single-ended-capable signalling; high speed uses 400 mV current-mode differential signalling with 45 Ω terminations to ground on each line. Data are NRZI-encoded with a stuffed 0 after six consecutive 1s so that the receiver's clock recovery always sees transitions (compare the line codes of Book 3 §27.3). Bit-rate accuracy must be within ±0.25 % for full speed and ±0.05 % for high speed (±1.5 % for low speed), which is why full-speed devices without a crystal must trim their RC oscillator against the host's 1 ms start-of-frame packets — the "crystal-less USB" of many microcontrollers.

A device announces itself by a pull-up on D+ (full speed, 1.5 kΩ to 3.3 V) or D− (low speed); high-speed devices attach as full-speed and then negotiate with a chirp sequence. The host resets the device, reads its **descriptors** (device, configuration, interface, endpoint, string), assigns an address and selects a configuration — **enumeration**. Device classes standardise the descriptors and protocols so no custom host driver is needed: CDC-ACM (virtual serial port), HID (keyboards, game controllers, many custom instruments), MSC (mass storage), audio, and DFU (firmware update, Section 25.9).

Transfers and throughput

Table 26.6 — USB 2.0 transfer types and maximum payload rates.

Transfer	Guarantee	Full-speed maximum	High-speed maximum	Typical use
Control	Reliable, reserved bandwidth share	64-byte packets	64-byte packets	Enumeration, configuration
Interrupt	Bounded latency (polling interval)	64 B per 1 ms frame = 64 kB/s	3 × 1024 B per 125 µs microframe = 24.6 MB/s	HID, status reports
Bulk	Reliable, uses leftover bandwidth	19 × 64 B per frame = 1.216 MB/s	13 × 512 B per microframe = 53.2 MB/s	Mass storage, CDC data
Isochronous	Guaranteed bandwidth, no retry	1023 B per frame	3 × 1024 B per microframe	Audio, video

Worked example — Is a full-speed HID fast enough for a data logger? An instrument streams 4 channels × 2 bytes at 10 kSa/s, 80 kB/s. A full-speed HID interrupt endpoint carries at most one 64-byte report per 1 ms frame, 64 kB/s, so it cannot keep up. A full-speed CDC bulk endpoint offers up to 1.216 MB/s on an otherwise idle bus (practically 0.8–1 MB/s), ample margin; a high-speed device is unnecessary. If the product needs driverless HID for corporate IT reasons, the data must be decimated or packed into fewer samples per second.

Power

USB also supplies power. A USB 2.0 device may draw 100 mA before configuration and up to 500 mA after (900 mA for USB 3.x); VBUS is 4.75–5.25 V at the host connector. USB Type-C adds 1.5 A or 3 A at 5 V, advertised by the source's pull-up on the CC pin and read by the sink's 5.1 kΩ pull-down; USB Power Delivery negotiates higher voltages over CC — up to 20 V at 5 A (100 W), and with PD 3.1 extended power range up to 48 V at 5 A (240 W). Missing 5.1 kΩ resistors on CC are the classic reason a Type-C gadget charges from a USB-A adapter but not from a C-to-C cable.

26.7 Ethernet fundamentals

PHY, MAC and the media-independent interface

Ethernet (IEEE 802.3) splits into a **MAC**, which frames data and computes the CRC, and a **PHY**, which encodes bits onto the cable. Microcontrollers such as the STM32F4/F7/H7 contain the MAC and connect to an external PHY over **MII** (4 bits at 25 MHz for 100 Mbit/s), **RMII** (2 bits at a 50 MHz reference clock) or, for gigabit, **RGMII** (4 bits double data rate at 125 MHz); a two-wire management interface (MDIO/MDC) reads the PHY's link status and configuration registers. The PHY drives the twisted pairs through **magnetics** that provide the galvanic isolation required by 802.3 (1500 V rms) and common-mode rejection.

Table 26.7 — Ethernet physical layers in embedded systems.

PHY	Standard	Encoding	Pairs and reach	Typical use
10BASE-T	802.3 clause 14	Manchester, 10 Mbaud	2 pairs, 100 m	Legacy, low-power devices
100BASE-TX	802.3 clause 25	4B5B + MLT-3, 125 Mbaud	2 pairs Cat 5, 100 m	Industrial controllers, gateways
1000BASE-T	802.3 clause 40	PAM-5 on 4 pairs, 125 Mbaud each	4 pairs Cat 5e, 100 m	Cameras, embedded Linux
100BASE-T1	802.3bw	PAM-3, 66.7 Mbaud	1 pair, 15 m	Automotive Ethernet
10BASE-T1S	802.3cg	Differential Manchester, multidrop	1 pair, 25 m, 8+ nodes	Automotive and building zones replacing CAN segments

The frame and its overhead

An Ethernet frame carries a 7-byte preamble and 1-byte start delimiter, 6-byte destination and source addresses, a 2-byte EtherType (or VLAN tag first), 46–1500 bytes of payload, and a 4-byte CRC-32 (Book 3 §28.6), followed by at least 12 byte times of interframe gap. The minimum frame of 64 bytes (from addresses to CRC) therefore occupies 84 byte times on the wire.

Worked example — Throughput and packet rate at 100 Mbit/s. A full-size frame with 1500 payload bytes occupies $1500 + 38 = 1538$ byte times, so the payload efficiency is $1500/1538 = 97.5\,\%$; a UDP application, after 20 bytes of IP and 8 bytes of UDP header, sees $1472/1538 = 95.7$ Mbit/s at most. At the other extreme, minimum-size frames arrive at up to $100 \times 10^6/(84 \times 8) = 148\,810$ per second — one every 6.7 µs. A Cortex-M7 that takes an interrupt and walks a TCP/IP stack for every frame cannot sustain that, which is why embedded MACs use DMA descriptor rings, interrupt coalescing and hardware filtering of unwanted multicast and broadcast traffic, and why an industrial device must survive a broadcast storm without missing its control deadlines (IEC 62443 robustness testing includes exactly this).

Embedded Ethernet in practice

Power over Ethernet delivers power on the same cable: 802.3af supplies 15.4 W at the source (12.95 W at the device), 802.3at 30 W (25.5 W), and 802.3bt up to 90 W (71.3 W), enough for cameras, access points and small controllers. **Time-Sensitive Networking** (IEEE 802.1AS, 802.1Qbv) and fieldbuses such as EtherCAT and PROFINET IRT give Ethernet bounded latency, with control cycles down to 100 µs or less.

26.8 ADC and DAC interfaces

What the converter assumes

> **Boundary.**
> Quantization, SQNR and ENOB (SQNR $= 6.02N + 1.76$ dB) are from *Circuits, Signals and Communications*, Ch. 19 §19.8; jitter in §19.5; anti-aliasing filters in §19.9; oversampling and dither in Ch. 21 §21.2. The one-paragraph physics sketch is in *Engineering Physics*, Ch. 18 §18.7.2. Converter architectures (SAR, pipeline, sigma–delta) belong to Chapter 10 of this book, and the on-chip ADC as a peripheral to Chapter 17. This section covers the interface: what the circuit outside the pin must do for the converter to deliver its datasheet accuracy.

Most microcontroller ADCs are successive-approximation (SAR) converters with a switched-capacitor input. During the acquisition (sampling) time the pin is connected through a switch resistance R_{SW} to a sampling capacitor C_S of a few

picofarads; then the switch opens and the conversion runs on the stored charge. The external source must charge C_S to within half an LSB during the acquisition window (Figure 26.6).

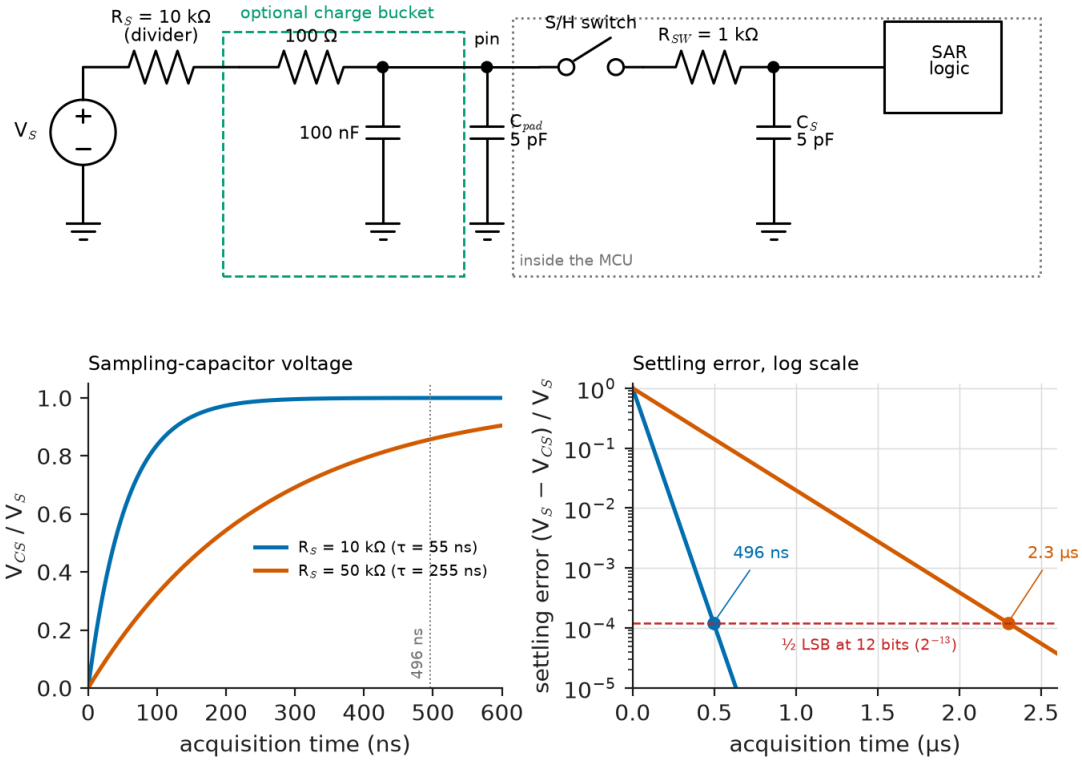


Figure 26.6 — Equivalent input circuit of a microcontroller SAR ADC with source, charge-bucket filter and sampling capacitor, and the settling of the sampled voltage.

$$t_{\text{acq}} \geq (R_S + R_{SW})\, C_S\, \ln\!\left(2^{N+1}\right)$$

> **Status of the principle: SAR acquisition-time requirement**
> *Approximate.* Treats the input as a single RC charging from a fully discharged (worst-case) capacitor to within half an LSB; real inputs add pin capacitance and charge sharing, so datasheet formulas include extra terms, and an external capacitor changes the problem from settling to charge sharing.

Worked example — Can a 10 kΩ divider drive a 12-bit SAR? With $C_S = 5$ pF, $R_{SW} = 1$ kΩ and a source resistance of 10 kΩ (a battery-voltage divider of 20 kΩ and 20 kΩ), $\tau = 11\ \text{k}\Omega \times 5\ \text{pF} = 55$ ns and $\ln(2^{13}) = 9.01$, so the acquisition time must be at least 496 ns — 18 cycles of a 36 MHz ADC clock, which is available. A 100 kΩ/100 kΩ divider (50 kΩ source), chosen to save battery current, needs 2.3 µs, longer than many default settings, and the reading is then low by an amount that depends on the previous channel's voltage — a fault that looks like crosstalk. The usual cure is a **charge-bucket** capacitor at the pin: 100 nF supplies the sampling charge with a droop of only $5\ \text{pF}/100\ \text{nF} \times 4096 = 0.2$ LSB, and is recharged by the high-impedance source between conversions — fine for slowly varying signals, but it turns the input into a low-pass filter (here with a time constant of 5 ms) and limits the usable sample rate on that channel.

Reference, ground and ratiometric measurement

A converter measures the input relative to its reference; every error in V_{REF} appears directly in the result. An MCU's supply used as reference may be ±2 % and noisy; a dedicated reference (an REF3030, ±0.2 %, 50 ppm/°C) improves absolute accuracy. For resistive sensors — potentiometers, thermistor dividers, strain-gauge bridges — the better solution is **ratiometric** measurement: excite the sensor from the same voltage that serves as the ADC reference, so that excitation drift cancels. The load cell of the opening, excited from one regulator and measured against another, inherits

the difference of their temperature coefficients. Analog and digital grounds should meet at one point near the converter, with return currents of switching loads kept away from the reference and input paths.

Table 26.8 — Representative converter interfaces.

Converter	Type	Interface	Typical use
MCU on-chip 12-bit SAR, 1–5 MSa/s	SAR	Internal bus, DMA	General acquisition; ENOB about 10–11 bits
ADS1115	16-bit ΔΣ, 860 Sa/s, PGA	I²C	Slow sensors, battery monitors
ADS8681	16-bit SAR, 1 MSa/s, ±12 V input	SPI	Industrial analog inputs
AD7124-8	24-bit ΔΣ, PGA, excitation sources	SPI	RTDs, thermocouples, bridges
MCP4725	12-bit DAC	I²C	Setpoints, bias adjustment
Timer PWM + RC filter	1-bit DAC	GPIO	LED dimming, slow setpoints

DAC outputs and PWM as a DAC

On-chip DACs are usually resistor-string or R-2R types with an output buffer of limited drive (a few kilohms minimum load) and settling times of a few microseconds; the reconstruction filter and its consequences are covered in Book 3 §19.6. Where no DAC is available, a PWM output followed by an RC low-pass filter is a cheap substitute: the average of a PWM signal of duty cycle D between 0 and V_{DD} is DV_{DD}, and a first-order filter with $\tau \gg T_{PWM}$ leaves a ripple of about $V_{DD}D(1-D)T_{PWM}/\tau$ peak to peak. With 20 kHz PWM from a 3.3 V pin into 10 kΩ and 1 µF ($\tau = 10$ ms), the ripple at 50 % duty is $3.3 \times 0.25 \times 50\ \mu\text{s}/10\ \text{ms} = 4.1$ mV, about 5 LSB of a 12-bit scale, but the filter's corner is 16 Hz and it takes about 50 ms (5τ) to settle — acceptable for a setpoint, useless for a waveform. Resolution is limited by the timer: a 16-bit timer at 20 kHz needs a 1.3 GHz clock; an 84 MHz timer gives only 4200 steps, about 12 bits.

26.9 Sensor interfaces

Sensor output types

Sensors reach the microcontroller in a handful of electrical forms, each with its own interface circuit:

Table 26.9 — Common sensor interfaces (typical values).

Output form	Example sensors	Signal level	Interface
Ratiometric voltage	Potentiometers, Hall position sensors, pressure sensors	0.5–4.5 V on 5 V supply	ADC with supply as reference
Bridge (mV/V)	Strain-gauge load cells, pressure dies	1–3 mV/V full scale	Instrumentation amplifier (Ch. 8) or ΔΣ ADC with PGA
Thermocouple	Type K, J, T	About 41 µV/°C (type K)	Cold-junction compensation, high-gain ADC
Resistance (RTD, thermistor)	Pt100/Pt1000 (IEC 60751), NTC 10 kΩ	0.385 Ω/°C for Pt100; NTC strongly nonlinear	Current excitation, 3- or 4-wire connection
Current loop	4–20 mA transmitters (process industry)	4 mA = 0 %, 20 mA = 100 %; fault below 3.6 or above 21 mA (NAMUR NE 43)	Shunt resistor (e.g. 150 Ω → 0.6–3.0 V)
Digital bus	MEMS IMUs, humidity, pressure, ToF sensors	I²C or SPI	Sections 26.3–26.4
Pulse, frequency, PWM	Flow meters, Hall speed sensors, encoders	Logic level	Timer capture, encoder mode (Ch. 17)
Automotive digital	SENT (SAE J2716), PSI5	Single-wire nibble encoding, 3 µs ticks typical	Timer capture or dedicated peripheral
Industrial smart sensor	IO-Link (IEC 61131-9)	24 V, 4.8/38.4/230.4 kbaud	Transceiver plus UART

The 4–20 mA loop survives because a current is immune to wire resistance, and its live zero both powers two-wire transmitters and distinguishes "zero reading" from "broken wire".

Resistance thermometers and lead resistance

A platinum RTD follows the IEC 60751 Callendar–Van Dusen equation; for $T \geq 0$ °C, $R = R_0(1 + AT + BT^2)$ with $A = 3.9083 \times 10^{-3}$ /°C and $B = -5.775 \times 10^{-7}$ /°C², giving 100.00 Ω at 0 °C and 138.51 Ω at 100 °C for a Pt100 — a mean sensitivity of 0.385 Ω/°C.

Worked example — Measuring a Pt100 to 0.01 °C. A Pt100 is excited with 1 mA and read by a 24-bit ΔΣ converter (AD7124 class) with a 2.5 V reference and a PGA gain of 16.

- Sensitivity: $1\ \text{mA} \times 0.385\ \Omega/^\circ\text{C} = 0.385$ mV/°C.
- Even at 16-bit effective resolution the LSB is $2.5\ \text{V}/2^{16}/16 = 2.38$ µV, or $2.38/385 = 0.0062$ °C per LSB, so the converter is not the limit.
- Self-heating: $I^2R = 1\ \text{mA}^2 \times 138.5\ \Omega = 0.14$ mW at 100 °C, which with a typical self-heating coefficient of a few tenths of a degree per milliwatt in still air is already several hundredths of a degree — the reason excitation is kept at 1 mA or less.
- Lead resistance: with a 2-wire connection, two 0.5 Ω leads add 1 Ω, read as $1/0.385 = 2.6$ °C of error, and their copper changes resistance with ambient temperature. A 3-wire connection cancels matched leads; a 4-wire (Kelvin) connection sends the excitation down one pair and senses the voltage on the other, eliminating them.

The excitation current should come from the same reference as the ADC (ratiometric operation, with a precision reference resistor in series), so that excitation drift cancels. Here, as in most precision measurements, the interface — leads, excitation, reference, self-heating — dominates the error budget, not the bits.

Digital sensors

A MEMS IMU or environmental sensor contains its own ADC, filters and FIFO behind I²C or SPI registers. The interface problems become timing and integrity: read the FIFO in bursts at a watermark interrupt, timestamp samples at the interrupt rather than when a task reads them, and after any brown-out read back the identity and configuration registers.

26.10 Communication troubleshooting

Instruments

Interface faults are found with two instruments used together. The **oscilloscope** shows the analog truth: levels, rise times, ringing, noise, ground shifts. The **logic analyser** captures many digital channels over long periods and decodes them into protocol transactions. Look at the analog signal first and the decoded data second, because a decoder shows only what its idealised thresholds saw.

Table 26.10 — Instrument requirements for interface debugging (rules of thumb).

Measurement	Requirement
Scope bandwidth for edges of rise time t_r	Scope rise time is about $0.35/f_{BW}$; keep it below one third of t_r
Logic-analyser timing capture	Sample at 4–10× the highest toggle rate (e.g. ≥ 100 MSa/s for a 10–25 MHz SPI)
Probing fast edges	Ground spring of a few mm instead of a 15 cm ground lead (which rings at about 100–300 MHz)
Differential buses (CAN, RS-485, USB)	Differential probe, or two channels with math subtraction
Current, inrush and sleep current	Current probe or a shunt with a high-dynamic-range power analyser

Worked example — Reading what the scope says. On the robot's sensor bus the scope shows SDA rising from 0.99 V to 2.31 V in 450 ns with the 4.7 kΩ pull-ups fitted. The bus capacitance is therefore $450\ \text{ns}/(0.8473 \times 4.7\ \text{k}\Omega) = 113$ pF; the rise time violates the 300 ns limit at 400 kHz, and the pull-up window of Section 26.4 at 113 pF gives $R_{p,\max} = 3.1$ kΩ, so 2.2 kΩ is a sound choice. A second measurement on a CAN transceiver's 2 ns edges taken with a 100 MHz scope (rise time $0.35/100\ \text{MHz} = 3.5$ ns) reads $\sqrt{2^2 + 3.5^2} = 4.0$ ns, twice the real value — the measurement reports the scope, not the circuit. A 500 MHz scope (0.7 ns) reads 2.1 ns.

A systematic method

Most interface faults fall into a few classes, and checking them in order from the physical layer upwards finds nearly all of them quickly:

Table 26.11 — Common interface faults and their signatures.

Symptom	Likely cause	Check
Nothing received at all	TX/RX swapped, wrong pins muxed, peripheral clock not enabled, CS never asserted	Scope on each pin; peripheral and clock registers
UART garbage, framing errors	Baud mismatch, wrong format, inverted RS-232 levels	Measure a bit width on the scope (8.68 µs at 115 200)
SPI data shifted by one bit or all 0xFF	Wrong CPOL/CPHA, MISO not driven, CS timing	Compare edges with the datasheet mode diagram
I²C NACK on every address	Wrong 7-bit versus 8-bit address convention, missing pull-ups, device in reset	Decode address byte; measure idle levels
I²C bus stuck low	Target holding SDA after a reset or brown-out	Bus clear (Section 26.4); power-cycle target
CAN error frames, nodes going bus-off	Missing or extra termination (not 60 Ω), bit timing mismatch, stub too long, ground shift	Resistance between CAN_H and CAN_L; sample point of every node
Errors only on long cables or hot days	Timing margin consumed by delay or clock drift	Compute round-trip and tolerance budgets
Intermittent errors when a motor runs	Ground bounce, coupled noise, supply dip resets a device	Scope on supply and ground during events; trigger on error
Works on the bench, not in the enclosure	ESD or EMC events, unterminated long cables, temperature	IEC 61000-4-2/-4-4 testing, thermal chamber

Two habits make these checks fast. Expose the error counters — CAN error counts, I²C NACKs and timeouts, UART framing and overrun counts, ring-buffer drops (Section 25.7) — because a counter that rises in the field points at the failing layer. And trigger the scope on the error itself (a GPIO toggled by the error handler) rather than on random traffic: only a capture triggered on the I²C timeout, with the supply rail on a second channel, showed the robot's brown-out.

What remains open

Open question	Where it is answered
How are GPIO ports, serial controllers, timers and ADCs organised as microcontroller peripherals?	Chapter 17
How do SAR, pipeline and sigma–delta converters work inside, and how are references and comparators built?	Chapter 10
How are bridge and sensor signals amplified before conversion?	Chapter 8
How are CAN messages and interrupt-driven drivers scheduled to meet deadlines?	Chapter 27
How are drivers for these interfaces structured, tested and validated, and how are communication stacks layered?	Chapter 28
How do reflections, termination and differential pairs behave on traces and cables?	*Circuits, Signals and Communications*, Ch. 14
How do sampling, quantization, jitter and anti-aliasing limit an acquisition system?	*Circuits, Signals and Communications*, Ch. 19
How do CRCs and line codes protect and shape the bit streams of these buses?	*Circuits, Signals and Communications*, Ch. 27 and Ch. 28

Next: Chapter 27 turns from the wire to time itself — interrupt latency, scheduling, shared data and priority inversion — and shows how to prove that every one of these interfaces is serviced before its deadline.

Chapter 27 — Real-Time Computing

Part VI · Embedded and Real-Time Systems

Days after landing on Mars on 4 July 1997, the Pathfinder lander began resetting itself, losing data each time. Nothing was broken. A low-priority meteorological task held a mutex protecting a shared information bus; a high-priority bus-management task blocked waiting for it; and a medium-priority communications task, needing neither, preempted the low-priority task and kept it from ever releasing the mutex. A timer noticed that the bus-management task had missed its deadline and reset the computer. Engineers reproduced the fault on a ground replica, enabled the priority-inheritance option of the operating system's mutex, and uploaded the fix. Nearly thirty years later the same pattern still appears in drone flight controllers that wobble when the SD-card logger is busy, in motor drives that trip when a diagnostic task floods a shared SPI bus, and in medical pumps whose alarms arrive late under heavy display activity. A real-time system is not a fast system. It is one whose timing can be predicted and proved, and this chapter is about the models, analyses and mechanisms that make the proof possible.

> **How are interrupt latency and task response times bounded, and how do scheduling policies and analysis prove that every deadline will be met?**
> **What goes wrong when tasks share data and resources — races, deadlock, priority inversion — and how do synchronisation, RTOS services and watchdogs contain it?**

Concept	Engineering destination
Real-time constraints	Specifying deadlines for control loops, protection, airbags, audio and communications
Interrupts	Bounding the latency from a hardware event to the code that serves it
Scheduling	Choosing cyclic, rate-monotonic or EDF scheduling and proving schedulability
Timers	Periodic activation, timeouts, timestamps and drift-free timing
Concurrency	Structuring firmware as tasks and handlers that share a processor
Race conditions	Finding and removing timing-dependent defects that appear once a week
Synchronisation	Critical sections, mutexes, semaphores, queues and lock-free buffers
Real-time operating systems	Selecting and configuring FreeRTOS, Zephyr and certified kernels
Task priorities	Priority assignment, priority inversion, inheritance and ceiling protocols
Latency and jitter	Keeping control loops and sampling instants predictable
Watchdogs and fault handling	Detecting hangs and faults and returning the system to a safe state

Roadmap. What "real-time" demands (27.1) → the first source of delay, interrupts (27.2) → scheduling policies and their analysis (27.3) → the timers that drive them (27.4) → concurrency (27.5), the races it causes (27.6) and synchronisation to prevent them (27.7) → the RTOS that packages it all (27.8) → priorities and priority inversion (27.9) → latency and jitter end to end (27.10) → what to do when everything else fails (27.11).

> **Boundary.**
> Interrupt controllers, vector tables and hardware timers as processor and microcontroller mechanisms are developed in Chapters 16 and 17; firmware structures (superloop, interrupt-driven, RTOS-based) were introduced in Section 25.6. The consequences of loop delay for control stability are from *Circuits, Signals and Communications*, Ch. 35 §35.8, and the throughput and latency budgets of signal-processing chains from Ch. 21 §21.3.

27.1 Real-time constraints

Deadlines and their kinds

A **real-time system** is one whose correctness depends on when results are produced as well as on their values. The timing requirement is expressed as a **deadline**, and deadlines are classified by what a miss costs:

- **Hard** — a miss is a failure, possibly dangerous: airbag deployment, a protection relay's trip decision, a motor drive's overcurrent shutdown, fly-by-wire control.
- **Firm** — a late result is worthless but not harmful, and occasional misses are tolerated: a video frame, a sensor sample in a monitoring system.
- **Soft** — a late result loses value gradually: user-interface updates, logging, network throughput.

Most products contain all three, and the design task is to guarantee the hard deadlines while giving the rest what is left.

Table 27.1 — Representative real-time requirements (typical values).

Application	Activity	Period or deadline	Kind
Digital power supply	Voltage/current loop	1–20 µs	Hard
Engine control	Injection and ignition timing at 6000 rpm	1° of crank angle = 27.8 µs	Hard
Motor drive	Current loop	50–125 µs (8–20 kHz)	Hard
Motor drive	Speed and position loops	0.1–1 ms	Hard
Flight controller	Attitude loop	0.25–4 ms	Hard
Audio processing	64-sample block at 48 kSa/s	1.33 ms	Firm
Industrial PLC	I/O scan	1–10 ms	Hard or firm
Vehicle ABS/ESC	Brake control loop	5–10 ms	Hard
Airbag controller	Crash detection to firing decision	About 10–30 ms	Hard
Protection relay	Trip decision	About one mains cycle (20 ms at 50 Hz)	Hard
HMI display	Screen refresh	16–50 ms	Soft

The task model

Real-time analysis abstracts each activity as a **task** τ_i that is released repeatedly and must complete each job before its deadline (Figure 27.1). The standard parameters are:

- T_i — period (or minimum inter-arrival time for sporadic tasks driven by events);
- C_i — **worst-case execution time** (WCET) of one job;
- D_i — relative deadline, often equal to T_i (implicit deadline);
- J_i — release jitter, the variation in when a job becomes ready;
- R_i — worst-case **response time**, from release to completion, which the analysis computes and compares with D_i;
- $U_i = C_i / T_i$ — utilisation, the fraction of the processor the task needs.

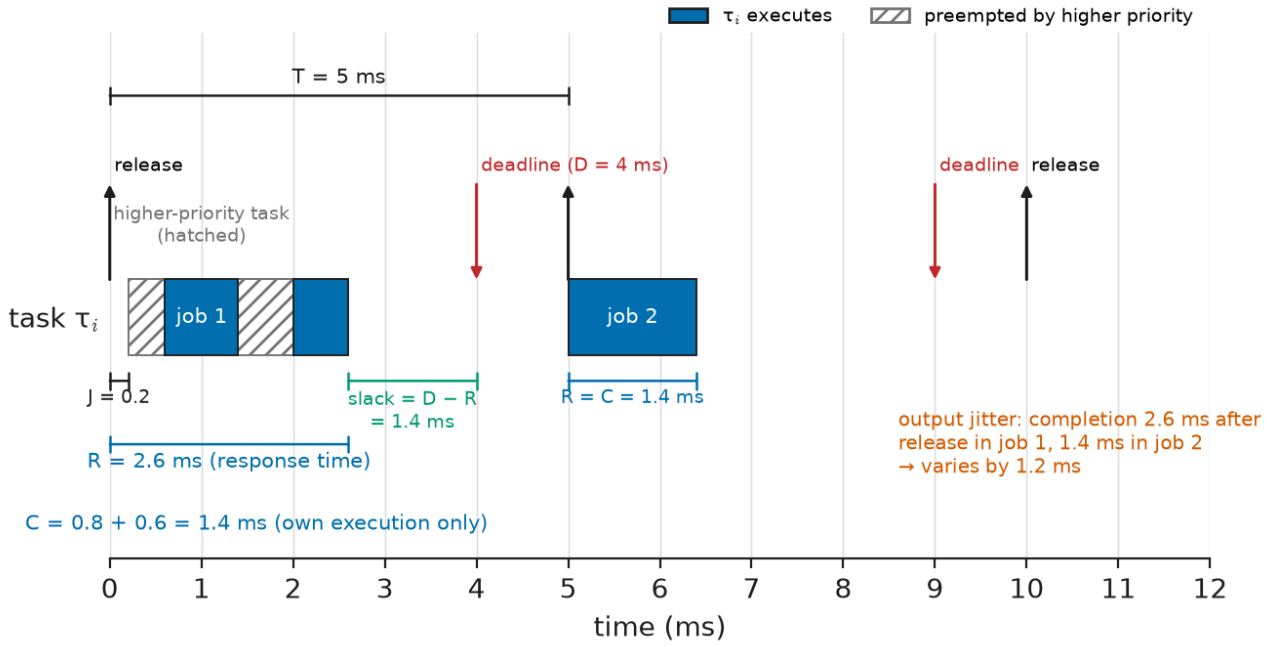


Figure 27.1 — Timing parameters of a periodic real-time task.

Worst-case execution time

Every analysis in this chapter needs C_i, and obtaining a safe value is harder than it sounds. **Measurement** — running the code under stress with a cycle counter or a GPIO pin on a scope — finds the longest path that the tests exercised, not necessarily the longest that exists. **Static analysis** tools compute a bound from the machine code and a model of the processor's pipeline and caches; they are standard in avionics (DO-178C) and automotive (ISO 26262) work, but the bound can be pessimistic by tens of percent on processors with caches and branch prediction. Practice combines both: measure, add a margin (20–50 % is common for non-certified work), and keep the timing-critical code on simple paths in tightly coupled memory, where execution time hardly varies.

> **Status of the principle: Worst-case execution time**
> *Empirical.* A measured WCET is a sample maximum and can be exceeded by an untested path; a statically computed bound (an approximation from a processor model) is safe only if the processor model (caches, pipeline, bus contention) is correct.

27.2 Interrupts

Anatomy of interrupt latency

Interrupt latency is the time from a hardware event to the first instruction of its handler. On an Arm Cortex-M3/M4/M7 the hardware part is fixed and short: the NVIC recognises the request, the core finishes or abandons the current instruction, pushes eight registers (R0–R3, R12, LR, PC, xPSR) onto the stack while fetching the vector, and begins the handler 12 cycles after the request with zero-wait-state memory — 71 ns at 168 MHz. With floating-point context stacked the entry grows to about 30 cycles, though **lazy stacking** defers the extra 18 words until the handler first touches the FPU. **Tail-chaining** runs a pending handler after the current one in 6 cycles without unstacking and restacking, and **late arrival** lets a higher-priority request that arrives during stacking take over the vector fetch.

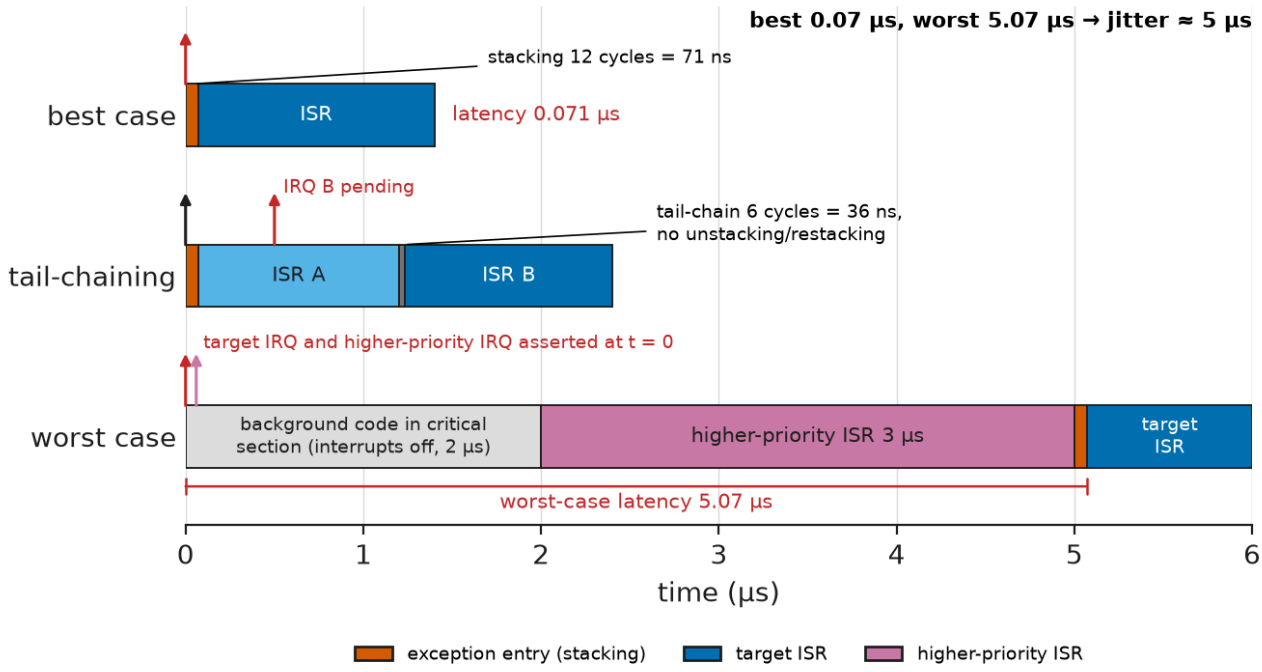


Figure 27.2 — Interrupt latency on a Cortex-M4 at 168 MHz: best case, tail-chaining, and the worst case with a critical section and a higher-priority handler.

The software part is usually much larger and is the designer's responsibility (Figure 27.2):

$$L_{\max} = t_{\text{entry}} + t_{\text{masked,max}} + \sum_{j \in \text{hp}} C_j$$

where $t_{\text{masked,max}}$ is the longest stretch during which interrupts at this priority are masked (critical sections, the RTOS kernel's own critical sections), and the sum covers higher-priority handlers that may run first.

> **Status of the principle: Interrupt latency bound**
> *Derived.* Assumes each higher-priority interrupt fires at most once during the interval and that bus masters (DMA) and Flash wait states are folded into t_{entry}; with bursty sources or heavy DMA contention, each term must be expanded as in the response-time analysis of Section 27.3.

Worked example — Worst-case latency of an encoder interrupt. A servo drive on a 168 MHz Cortex-M4 serves an encoder-index interrupt. The entry costs 12 cycles (71 ns). The longest interrupt-disabled critical section in the firmware is 2 µs (a 64-bit statistics update in a driver), and a higher-priority PWM-fault handler of 3 µs may be pending at the same moment. The worst-case latency is $0.071 + 2 + 3 = 5.07$ µs, against a best case of 71 ns. At 3000 rpm with a 10 000-line encoder (40 000 counts per revolution after quadrature), one count lasts 0.5 µs, so a position captured in software after 5 µs is ten counts stale — which is why the position is latched by the timer hardware at the index edge (Chapter 17) and the interrupt only reads it later. Lesson: interrupts bound *when software runs*; hardware capture bounds *when the event is recorded*.

Priorities and design rules

The NVIC gives each interrupt a programmable priority (4 bits, 16 levels, on most Cortex-M4 parts); a higher-priority request preempts a running lower-priority handler. Critical sections can mask only the interrupts at or below a level with `BASEPRI`, leaving the most urgent handlers — overcurrent shutdown, for instance — untouched; FreeRTOS uses exactly this, so handlers above `configMAX_SYSCALL_INTERRUPT_PRIORITY` are never delayed by the kernel but must not call kernel services.

The design rules follow from the latency formula. Keep handlers short: acknowledge the hardware, move data to a buffer, signal a task, return. Do the work in a task, where the scheduler, not the interrupt priority, decides who waits:

```
 #include <stdint.h>
#include "FreeRTOS.h"
#include "task.h"
#include "stm32f4xx.h"            /* vendor CMSIS header: ADC1 */

void run_controller(uint16_t sample);

static TaskHandle_t control_task;
static volatile uint16_t latest_sample;

void ADC_IRQHandler(void)          /* priority below the syscall limit */
{
    BaseType_t woken = pdFALSE;
    latest_sample = (uint16_t)ADC1->DR;            /* read clears EOC   */
    vTaskNotifyGiveFromISR(control_task, &woken);  /* signal the task   */
    portYIELD_FROM_ISR(woken);                     /* switch on exit    */
}

static void control_loop(void *arg)
{
    (void)arg;
    for (;;) {
        (void)ulTaskNotifyTake(pdTRUE, portMAX_DELAY);  /* wait for sample */
        run_controller(latest_sample);
    }
}

void control_start(void)
{
    xTaskCreate(control_loop, "ctrl", 256, NULL,
                configMAX_PRIORITIES - 1, &control_task);
}
```

The handler costs a few hundred cycles; `portYIELD_FROM_ISR` requests a context switch at exception exit when the notified task outranks the interrupted one, so the controller starts within microseconds. Direct-to-task notifications are the lightest FreeRTOS signalling mechanism, faster and smaller than a semaphore.

27.3 Scheduling

Cyclic executive

The oldest real-time scheduler builds the whole timetable offline. A **cyclic executive** divides time into **minor frames** of fixed length f, driven by a timer interrupt, and repeats a **major cycle** equal to the hyperperiod (least common multiple) of the task periods. Each frame runs a fixed list of task jobs to completion.

Worked example — A cyclic schedule. Three tasks have (C, T) of $(1, 10)$, $(3, 20)$ and $(4, 40)$ ms. The hyperperiod is 40 ms. A minor frame of $f = 10$ ms satisfies the usual conditions: it is at least the longest C (4 ms), it divides the hyperperiod, and $2f - \gcd(f, T_i) \le D_i$ for every task ($20 - 10 = 10 \le 10$, ≤ 20, ≤ 40), which guarantees a full frame between each release and its deadline. The four frames run $\{\tau_1, \tau_2\}$ (4 ms), $\{\tau_1, \tau_3\}$ (5 ms), $\{\tau_1, \tau_2\}$ (4 ms) and $\{\tau_1\}$ (1 ms), with the slack used for background work. The schedule is trivially verified by inspection, has zero scheduling overhead beyond the frame interrupt, and is fully deterministic — which is why cyclic executives still run flight-control

and railway-signalling computers. Its weakness is brittleness: a new task or a changed period can force the whole table to be rebuilt, and sporadic events must wait for a frame that polls them.

Fixed-priority preemptive scheduling and RMS

Most RTOSs use **fixed-priority preemptive scheduling**: each task has a static priority, and the highest-priority ready task always runs. For independent periodic tasks with implicit deadlines, **rate-monotonic scheduling** (RMS) — the shorter the period, the higher the priority — is optimal among fixed-priority assignments: if any fixed assignment meets all deadlines, RMS does. Liu and Layland (1973) proved a sufficient utilisation test:

$$U = \sum_{i=1}^{n} \frac{C_i}{T_i} \le n\left(2^{1/n} - 1\right)$$

The bound falls from 1 for one task to 0.828 for two, 0.780 for three, 0.757 for four and 0.743 for five, approaching $\ln 2 = 0.693$ for many tasks (Figure 27.3).

> **Status of the principle: Liu–Layland utilisation bound**
> *Derived.* Proved for independent, preemptible, periodic tasks with deadlines equal to periods, zero overhead and no blocking on one processor; it is sufficient but not necessary — task sets above the bound are often schedulable, and the exact answer comes from response-time analysis.

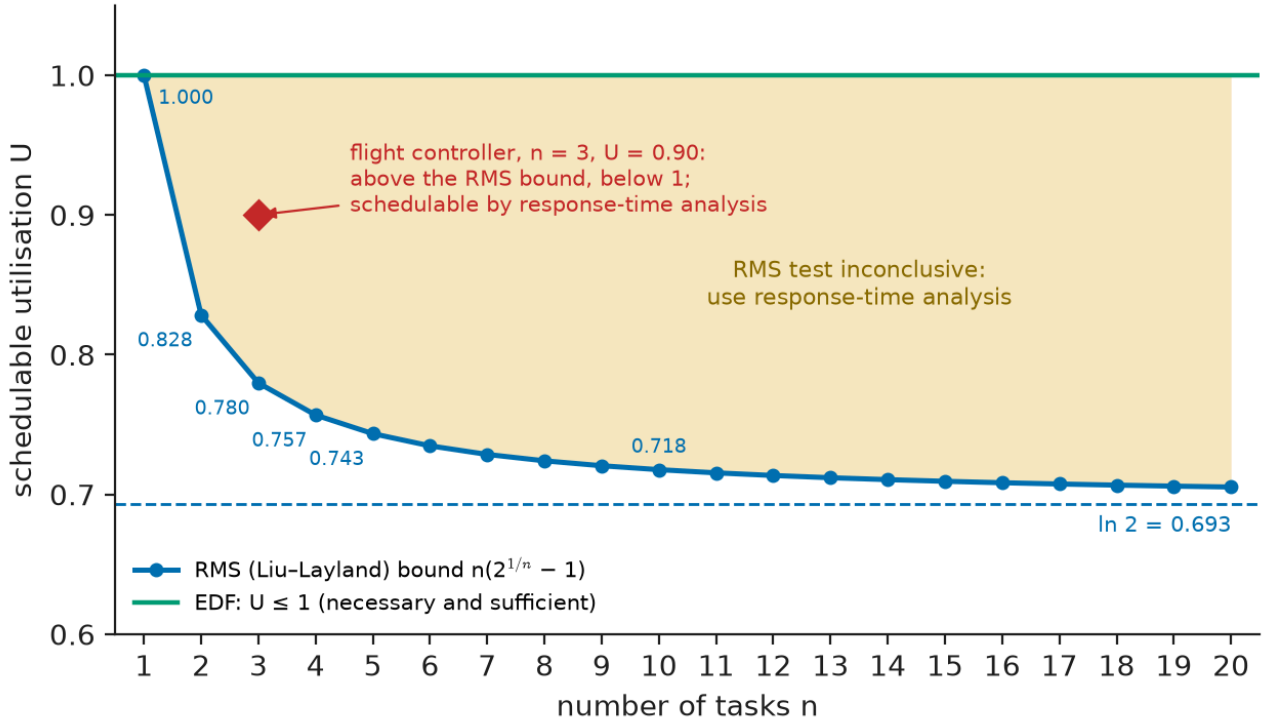


Figure 27.3 — Schedulable utilisation versus number of tasks for rate-monotonic and EDF scheduling.

A tighter sufficient test, the **hyperbolic bound** of Bini and Buttazzo, requires $\prod_i (U_i + 1) \le 2$; it accepts every task set that Liu–Layland accepts and some more.

Earliest deadline first

Earliest-deadline-first (EDF) scheduling assigns priorities dynamically: at every instant the ready job with the nearest absolute deadline runs. For the same task model EDF is optimal among all scheduling policies, and its test is exact:

$$U = \sum_i \frac{C_i}{T_i} \leq 1$$

> **Status of the principle: EDF schedulability**
> *Derived.* For independent preemptible tasks with implicit deadlines on one processor, $U \leq 1$ is necessary and sufficient; with deadlines shorter than periods a processor-demand test replaces it.

EDF uses the processor fully, but it is less common in commercial RTOSs because priorities change at run time, the implementation needs a deadline-ordered ready queue, and under overload it fails badly — a single overrunning job can make many tasks miss in a domino effect, whereas fixed priorities protect the high-priority tasks and sacrifice the low ones. Linux offers EDF as `SCHED_DEADLINE`; Zephyr offers an optional deadline scheduler.

Response-time analysis

The exact test for fixed priorities is **response-time analysis** (RTA). A task's worst-case response occurs when it is released together with all higher-priority tasks (the critical instant). Its response time is then its own execution, plus blocking B_i by lower-priority tasks holding shared resources (Section 27.9), plus the interference of every higher-priority task released during the window:

$$R_i = C_i + B_i + \sum_{j \in hp(i)} \left\lceil \frac{R_i + J_j}{T_j} \right\rceil C_j$$

The equation is solved by iteration, starting from $R_i = C_i + B_i$ and stopping when the value repeats (the task is schedulable if $R_i \leq D_i$) or exceeds D_i.

> **Status of the principle: Response-time analysis**
> *Derived.* Exact for fixed-priority preemptive scheduling of independent sporadic tasks on one processor with $D_i \leq T_i$, given correct C_i, B_i and J_i; kernel overheads (context switches, tick handler) must be added to C_i or modelled as extra tasks, and the analysis does not cover multicore interference through shared caches and buses.

Worked example — Is the flight controller schedulable? A flight controller runs an attitude loop ($C = 1$ ms, $T = 4$ ms), a state estimator ($C = 2$, $T = 5$) and a telemetry-and-logging task ($C = 3$, $T = 12$), with rate-monotonic priorities in that order. The listing applies the tests:

```
 # Response-time analysis for fixed-priority preemptive scheduling
from math import ceil

def response_times(tasks):
    """tasks: list of (name, C, T, D, B) sorted by priority, highest first.
    Returns the worst-case response time of each task, or None if R > D."""
    results = []
    for i, (name, c, t, d, b) in enumerate(tasks):
        r = c + b
        while True:
            r_next = c + b + sum(ceil(r / tj) * cj
                                 for _, cj, tj, _, _ in tasks[:i])
            if r_next == r or r_next > d:
                break
            r = r_next
        results.append((name, r_next if r_next <= d else None))
    return results

flight = [  # name, C, T, D, B  (ms)
    ("attitude", 1.0, 4.0, 4.0, 0.0),
    ("estimator", 2.0, 5.0, 5.0, 0.0),
    ("logging", 3.0, 12.0, 12.0, 0.0),
]
u = sum(c / t for _, c, t, _, _ in flight)
n = len(flight)
print(f"U = {u:.3f}, Liu-Layland bound = {n * (2 ** (1 / n) - 1):.3f}")
for name, r in response_times(flight):
    print(f"{name:<10s} R = {r}")
```

The utilisation is $0.25 + 0.4 + 0.25 = 0.900$, above the three-task bound of 0.780, and the hyperbolic product $1.25 \times 1.4 \times 1.25 = 2.19$ exceeds 2: both sufficient tests are inconclusive. RTA gives $R = 1$, 3 and 10 ms, all within their deadlines of 4, 5 and 12 ms, so the set is schedulable. For the logging task the iteration runs $R = 3 \to 6 \to 9 \to 10 \to 10$: at $R = 10$ ms the window contains three attitude jobs and two estimator jobs, $3 + 3 \times 1 + 2 \times 2 = 10$. Figure 27.4 shows the resulting schedule; the same model is used for CAN messages, whose identifiers are fixed priorities (Section 26.5), with non-preemptive frame transmission entering as blocking.

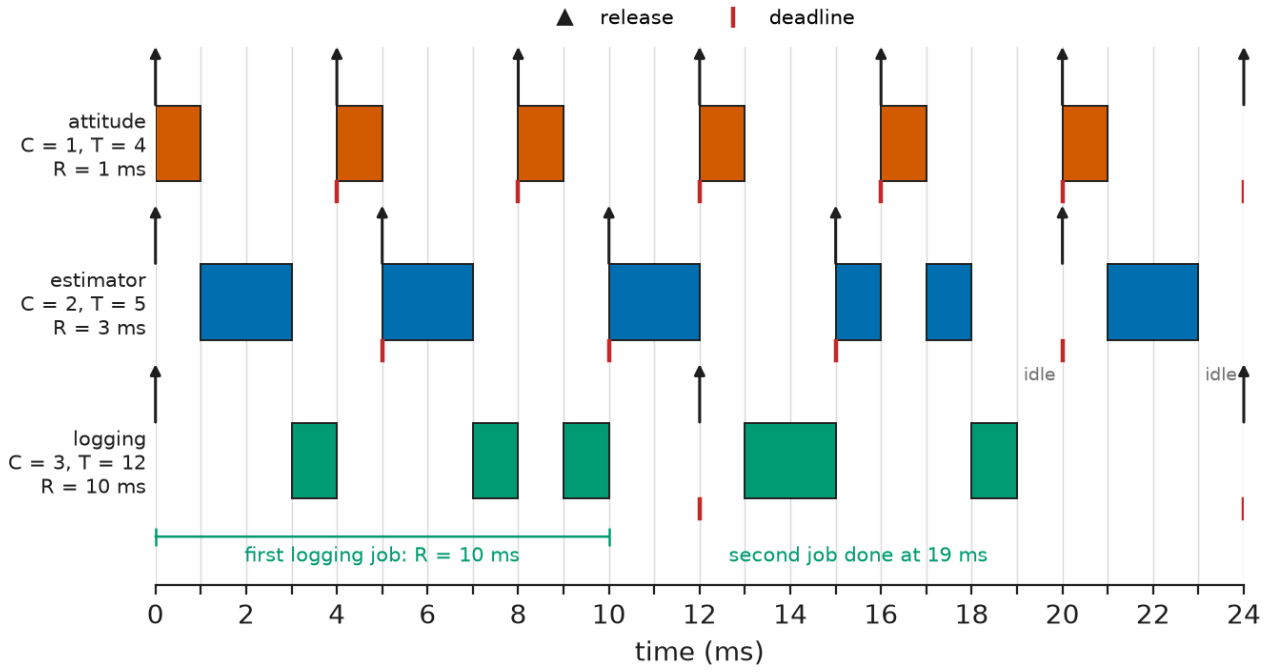


Figure 27.4 — Rate-monotonic schedule of the flight-controller task set over the first 24 ms.

27.4 Timers

Periodic activation and the system tick

Hardware timers (Chapter 17) provide the time base of every real-time system in three roles. A **periodic timer interrupt** releases tasks: the RTOS **tick**, typically 1 kHz from the Cortex-M SysTick or a low-power timer, advances the kernel's notion of time, wakes delayed tasks and enforces time slices. **Hardware-triggered events** start ADC conversions, update PWM duty cycles and capture inputs without software in the timing path (Section 27.10). **Free-running counters** timestamp events and measure execution time; the Cortex-M DWT cycle counter (`DWT->CYCCNT`) counts core clocks and is the standard tool for measuring WCET.

The tick sets the granularity of every software timeout. A delay of n ticks requested at a random moment between ticks lasts between $n-1$ and n tick periods: `vTaskDelay(10)` at 1 kHz waits 9–10 ms. Periodic tasks must therefore not be written as "work, then delay T", because the period then becomes T plus the work plus scheduling delay and drifts; they should wait until an absolute time, `vTaskDelayUntil(&last_wake, period)` in FreeRTOS or `k_timer`-based periods in Zephyr, so that each release is computed from the previous release rather than from when the task happened to finish.

Tickless idle removes the periodic tick when no task is ready: the kernel programs a low-power timer for the next timeout and sleeps until then, which saves the energy of a thousand wake-ups per second — essential for the battery devices of Section 25.5.

Clock accuracy and drift

All of these times are only as good as the oscillator. A crystal of ±20 ppm accumulates up to $20 \times 10^{-6} \times 86\,400 = 1.7$ s of error per day, 52 s per month. A 32.768 kHz tuning-fork crystal used for the RTC has a parabolic temperature curve, about -0.034 ppm/°C² away from its 25 °C turnover, so at −20 °C or +70 °C it runs $0.034 \times 45^2 = 69$ ppm slow, 6 s per day. Devices that must keep time for months either calibrate the RTC against temperature, discipline it from a network (NTP, IEEE 1588 PTP, GNSS or cellular time), or use a temperature-compensated oscillator. Distributed control (synchronised sampling in a substation, coordinated motion axes) depends on such synchronisation to the microsecond.

> **Status of the principle: Tick-granularity error**
> *Fundamental.* A software timeout measured in ticks cannot resolve times finer than one tick and is uncertain by up to one tick at its start, whatever the processor speed; finer timing needs a hardware timer channel.

27.5 Concurrency

Tasks, handlers and shared state

A **concurrent** program consists of activities whose executions overlap in time: tasks preempting one another, interrupt handlers preempting tasks, DMA engines writing memory in parallel, and on multicore parts true parallelism. Concurrency is how a single microcontroller meets several deadlines at once; it is also the source of the hardest defects in embedded software, because the interleaving of operations differs from run to run.

Three properties decide whether shared data is safe:

1. **Atomicity.** An operation is atomic if no other activity can observe it half-done. On a 32-bit Cortex-M an aligned 8-, 16- or 32-bit load or store is atomic; a 64-bit variable, a structure, or a read-modify-write such as `count++` (load, add, store) is not.
2. **Visibility and ordering.** The compiler may keep a variable in a register, merge or reorder accesses, unless told otherwise. `volatile` forces each access to happen in program order with respect to other volatile accesses, which is what device registers need — but it does not make `count++` atomic, and it does not order non-volatile accesses around it. C11 atomics (`<stdatomic.h>`) and compiler barriers express the intent properly.
3. **Hardware ordering.** On cores with write buffers, caches or multiple bus masters (Cortex-M7, multicore parts, DMA), memory barriers (`DMB` , `DSB`) and cache maintenance are needed so that, for example, a DMA engine sees a buffer's contents before it sees the descriptor that points to it.

Reentrancy

A function is **reentrant** if it can be safely called again while a previous call is in progress — from an interrupt or another task. Functions that use only local variables and their arguments are reentrant; those that use static or global state, non-reentrant library calls (`strtok` , `rand` , many `printf` implementations, `malloc` without locking) or hardware peripherals without protection are not. Library documentation for embedded C runtimes (newlib, picolibc) states which functions need locking hooks under an RTOS.

27.6 Race conditions

Definition and anatomy

A **race condition** exists when the result of a computation depends on the relative timing of concurrent activities. The classic embedded case is a variable shared between an interrupt handler and a task, where one of them performs a non-atomic sequence on it. The **race window** is the span of instructions during which an interruption corrupts the result. Races are pernicious because the window is short, so the failure is rare, irreproducible under the debugger and often blamed on hardware.

> **Status of the principle: Race-window failure rate**
> *Approximate.* If accesses of rate f_a open a window of length w and the conflicting interrupt arrives at rate f_i, independently of the access, the expected failure rate is about $f_a f_i w$; correlated timing (both driven by the same timer) can make the race either impossible or certain, which is why it must be removed by design rather than estimated away.

Worked example — The sensor driver that hangs about twice a day. A task updates a shared 32-bit status word ten times a second with a read-modify-write that takes 4 instructions (24 ns at 168 MHz). An interrupt handler that sets a different bit in the same word fires 100 times a second. The chance that the handler lands inside a given window is $100 \times 24\ \text{ns} = 2.4 \times 10^{-6}$; with ten windows a second the failure rate is $10 \times 2.4 \times 10^{-6} = 2.4 \times 10^{-5}$ per second, one lost bit on average every 11.6 hours. In a lab test of an afternoon the bug probably never appears; in a fleet of ten thousand devices it appears about twenty thousand times a day. When the lost bit is a "transfer complete" flag, the driver waits forever — the intermittent hang that the field reports and the lab cannot reproduce.

Torn reads of wide values

A second common race is a **torn read**: reading a value wider than the bus, or composed of two registers, while it changes. A 64-bit timestamp built from a 32-bit hardware counter and a software overflow count is the textbook example:

```
 #include <stdint.h>
#include "stm32f4xx.h"            /* vendor CMSIS header: TIM5 (32-bit) */

static volatile uint32_t overflow_count;

void TIM5_IRQHandler(void)          /* counter wrapped from 0xFFFFFFFF to 0 */
{
    TIM5->SR = ~TIM_SR_UIF;         /* clear the update flag */
    overflow_count++;
}

/* Broken: a wrap between the two reads gives an error of 2^32 counts. */
uint64_t now_broken(void)
{
    uint32_t hi = overflow_count;
    uint32_t lo = TIM5->CNT;
    return ((uint64_t)hi << 32) | lo;
}

/* Correct when called with interrupts enabled below the TIM5 priority:
 * repeat until the high word is unchanged across the low-word read. */
uint64_t now(void)
{
    uint32_t hi, lo;
    do {
        hi = overflow_count;
        lo = TIM5->CNT;
    } while (hi != overflow_count);
    return ((uint64_t)hi << 32) | lo;
}
```

The broken version returns a time about 2^{32} counts — 51 s at 84 MHz — too early whenever the wrap and its handler fall between its two reads. The corrected version re-reads until consistent and needs no lock, but its comment matters: called from code that masks the timer interrupt, the counter can wrap while the overflow handler is still pending, and only checking the timer's pending flag fixes that case. The other common race patterns are check-then-act (testing a flag, then acting on it after it may have changed), the lost wake-up of Section 25.6 (sleeping just after an interrupt set the flag), and double-buffer swaps signalled before the data are complete.

27.7 Synchronisation

Critical sections

The simplest protection makes a sequence atomic by preventing preemption during it. On a single-core MCU, disabling interrupts (`__disable_irq()`, or raising `BASEPRI` to mask only interrupts that share the data) does this for a few instructions. Every critical section adds directly to the interrupt latency of Section 27.2 and must be short and bounded — microseconds, never a loop over a buffer of unknown length or a call to a blocking function. Hardware support often removes the need: atomic set/clear registers for GPIO and status words (Section 25.7), exclusive load/store instructions (`LDREX` / `STREX` on Armv7-M) behind C11 `atomic_fetch_or`, and the single-producer, single-consumer ring buffer that needs no lock at all.

Kernel synchronisation objects

An RTOS provides objects that block a task instead of spinning or masking interrupts:

Table 27.2 — RTOS synchronisation objects.

Object	Purpose	Ownership	Use from ISR	Typical use
Mutex	Mutual exclusion of a resource	Owned by the locking task; priority inheritance	No	Shared bus, file system, display
Binary semaphore	Signal an event	No owner	Give only	ISR-to-task signal (or a task notification)
Counting semaphore	Count events or free resources	No owner	Give only	Pool of buffers, pending events
Queue / message buffer	Pass data by copy, with blocking	—	Send and receive (non-blocking)	Sensor samples, commands, log records
Event group / flags	Wait for combinations of events	—	Set only	Start-up synchronisation, multi-source waits
Task notification	Lightweight signal or value to one task	—	Yes	Fastest ISR-to-task signal in FreeRTOS

The distinction between a mutex and a binary semaphore matters. A mutex has an owner, may only be released by that owner, and implements priority inheritance (Section 27.9); a semaphore has no owner and is the right tool for signalling. Using a binary semaphore to protect a resource silently gives up inheritance — a common route to a Pathfinder-like inversion. Queues are often the best synchronisation of all: if tasks communicate only by passing messages and each piece of state has one owner, there is nothing to lock.

Deadlock

A **deadlock** occurs when tasks wait for each other in a cycle: task A holds the SPI mutex and waits for the I²C mutex while task B holds the I²C mutex and waits for SPI. It requires four conditions together — mutual exclusion, hold-and-wait, no preemption of resources, and a circular wait — and is prevented by breaking any one. The practical rules are to acquire multiple locks always in one global order, to avoid holding a lock while waiting for anything else, and to use timeouts on every blocking call so that a deadlock becomes a detected, logged error rather than a silent hang. The priority-ceiling protocols of Section 27.9 prevent deadlock by construction.

27.8 Real-time operating systems

What a kernel provides

A **real-time operating system** provides preemptive priority scheduling, the synchronisation objects of Section 27.7, timers and delays, and memory management for task stacks — with the guarantee that every kernel operation has a bounded execution time. It differs from a general-purpose OS less in its features than in that bound: no operation may take time proportional to the number of tasks or the amount of memory in use unless that number is fixed at design time.

On a Cortex-M the context switch uses two exceptions. The tick or a kernel call decides that a switch is needed and pends **PendSV**, the lowest-priority exception, which runs once all other handlers have finished, saves the remaining registers of the outgoing task on its stack, stores its stack pointer, selects the next task and restores its context. The whole switch takes roughly 100–300 cycles, about 1–2 µs at 168 MHz, and it never delays a higher-priority interrupt.

Table 27.3 — Representative real-time kernels (typical values).

Kernel	Licence and origin	Footprint	Notable features
FreeRTOS	MIT; maintained by AWS	5–10 KB Flash, about 1 KB RAM plus stacks	The most widely ported MCU kernel; SAFERTOS derivative certified to IEC 61508 SIL 3
Zephyr	Apache 2.0; Linux Foundation	8 KB to several hundred KB	Device tree, drivers, BLE, networking, MCUboot integration, memory protection
Eclipse ThreadX	MIT (formerly Azure RTOS)	2–20 KB	Fast, pre-certified to IEC 61508, ISO 26262 and others
AUTOSAR Classic OS	Specification (OSEK-based); several vendors	Statically configured	Automotive ECUs, ISO 26262 ASIL D implementations
VxWorks	Commercial (Wind River)	Hundreds of KB	Aerospace and defence, DO-178C; flew on Pathfinder and later Mars missions
QNX Neutrino	Commercial (BlackBerry)	Microkernel, MB-scale	Automotive cockpits and safety systems, POSIX, ISO 26262 ASIL D
Linux PREEMPT_RT	GPL; mainline since kernel 6.12 (2024)	Full Linux	Worst-case latencies of tens of microseconds on tuned hardware

Configuration and memory protection

Real-time behaviour depends on configuration as much as on the kernel. Static allocation of all tasks, stacks and queues (FreeRTOS `configSUPPORT_STATIC_ALLOCATION`, Zephyr's compile-time definitions) removes the heap from the timing and failure picture. Stack-overflow checking and the memory protection unit turn corruption into faults (Section 27.11). Tracing tools (SEGGER SystemView, Percepio Tracealyzer, Zephyr's tracing) record every context switch and kernel call with cycle timestamps and are the fastest way to see priority inversions and overruns that the analysis did not predict. For mixed-criticality systems, a multicore part often runs an RTOS on a Cortex-M or Cortex-R core for the hard loops and Linux on Cortex-A cores for networking and user interface, communicating through shared memory and mailboxes.

27.9 Task priorities

Assigning priorities

For independent periodic tasks, rate-monotonic order (shorter period, higher priority) is optimal when $D_i = T_i$, and **deadline-monotonic** order (shorter relative deadline, higher priority) is optimal when $D_i \le T_i$. Real systems deviate

for good reasons — a short-period but soft task (a display animation) should not outrank a longer-period hard one (a protection check) — and the analysis of Section 27.3 then checks the actual assignment. Priorities should reflect deadlines, not importance: a crucial but slow task at top priority makes every fast task wait for it.

Priority inversion

Priority inversion occurs when a high-priority task waits for a lower-priority one. Some inversion is unavoidable: if a low-priority task holds a mutex the high-priority task needs, the high-priority task must wait for the critical section to finish — bounded inversion. The danger is **unbounded inversion**: while the low-priority task holds the mutex, any number of medium-priority tasks, which need no resource at all, can preempt it and extend the high-priority task's wait indefinitely. This is the Pathfinder failure of the opening (Figure 27.5).

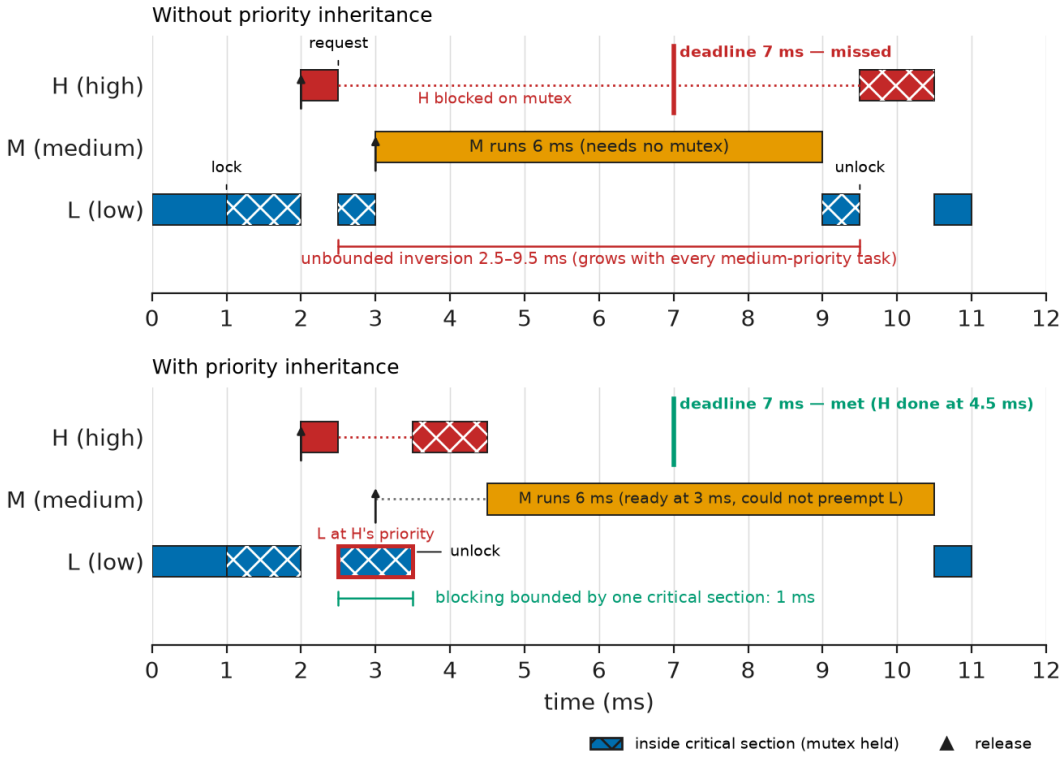


Figure 27.5 — Unbounded priority inversion and its correction by priority inheritance.

Priority inheritance and priority ceiling

Two protocols bound the inversion:

- **Priority inheritance** (PIP): when a high-priority task blocks on a mutex, the holder temporarily inherits the waiter's priority, so medium-priority tasks cannot preempt it; it returns to its own priority on release. FreeRTOS, Zephyr, ThreadX and POSIX (`PTHREAD_PRIO_INHERIT`) mutexes implement it.
- **Priority ceiling** (PCP, and its simpler immediate form, IPCP, used by AUTOSAR OS as `GetResource` and by POSIX as `PTHREAD_PRIO_PROTECT`): each resource has a ceiling equal to the highest priority of any task that uses it, and a task that locks the resource runs at that ceiling immediately.

> **Status of the principle: Blocking bounds of the inheritance and ceiling protocols**
> *Derived.* Under priority inheritance a task can be blocked at most once by each lower-priority task (or once per resource), and chained blocking is possible; under the priority-ceiling protocols a task is blocked at most once, for the duration of one critical section of a lower-priority task, and deadlock cannot occur. Both assume critical sections do not suspend for other reasons.

The bound enters response-time analysis as the term B_i: the longest critical section of any lower-priority task on a resource whose ceiling is at least the priority of τ_i.

Worked example — How long may the logger hold the SPI bus? In the flight controller of Section 27.3 the logging task writes to an SD card on the same SPI bus that the attitude and estimator tasks use to read the IMU, protected by a mutex with priority inheritance. If the logger holds the bus for one 1.5 ms block write, both higher-priority tasks can be blocked by up to $B = 1.5$ ms. RTA then gives the attitude task $R = 1 + 1.5 = 2.5$ ms (deadline 4 ms, met), but the estimator $R = 2 + 1.5 + \lceil 4.5/4 \rceil \times 1 = 5.5$ ms, which misses its 5 ms deadline. Limiting each locked SD transfer to 1.0 ms (smaller blocks, or releasing the mutex between sectors) gives $R = 2 + 1 + \lceil 4/4 \rceil \times 1 = 4$ ms, schedulable. Without inheritance no finite bound exists at all. The better architecture removes the shared bus — the IMU on its own SPI port, or the SD card served by DMA from a queue owned by one task — so that $B = 0$.

27.10 Latency and jitter

Definitions

Latency is a delay from a cause to an effect: interrupt latency (Section 27.2), scheduling latency (from a task becoming ready to it running), and end-to-end latency (from a sensor sample to the actuator output). **Jitter** is the variation of a latency, or of the instants of a periodic activity, from one occurrence to the next. Release jitter disturbs when a task starts; output jitter disturbs when its result takes effect. For control and signal processing, jitter often matters more than average latency, because a constant delay can be designed into a controller while a varying one cannot.

Table 27.4 — Sources of latency and jitter in embedded systems (typical magnitudes).

Source	Typical magnitude	Mitigation
Interrupt entry (Cortex-M)	12–30 cycles	None needed
Interrupt-disabled critical sections	1–50 µs if careless	Short sections, `BASEPRI`, lock-free structures
Higher-priority handlers and tasks	Their execution times	Priority design, RTA
Flash wait states, caches, branch prediction	Tens of percent of execution time	Run critical code from tightly coupled RAM; lock caches
DMA and bus contention	Tens to hundreds of ns per access	Bus-matrix planning, separate RAM banks
Tick granularity	Up to one tick (1 ms)	Hardware timers for fine timing
General-purpose Linux	Milliseconds worst case	PREEMPT_RT, CPU isolation, or a companion MCU

Jitter in control loops

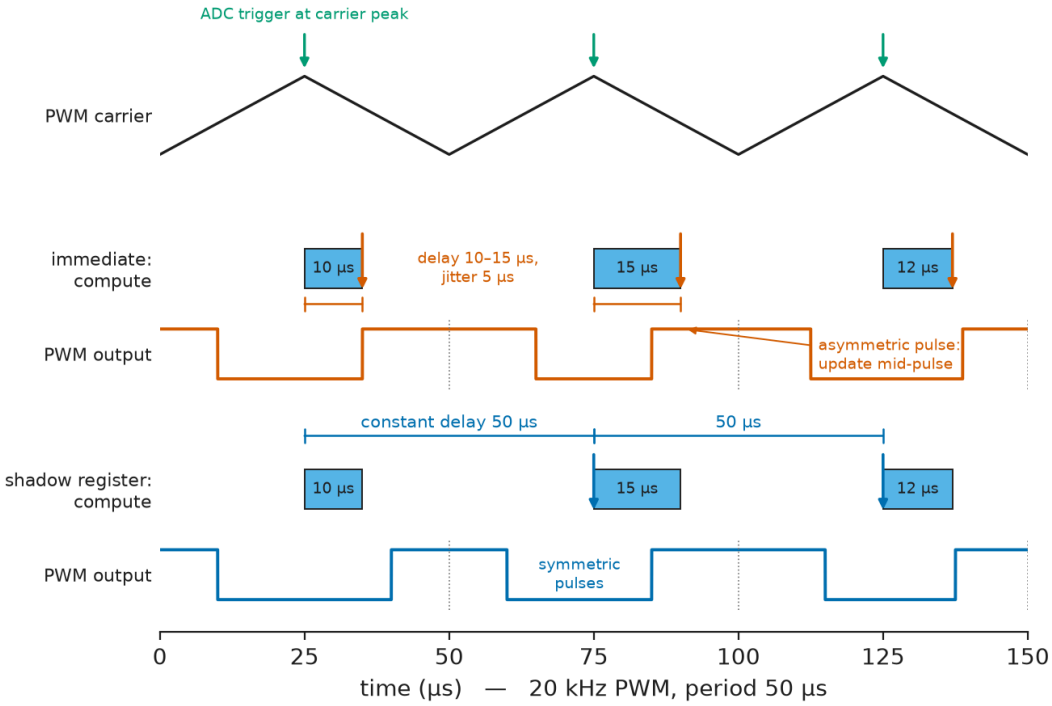
A digital controller assumes that it samples at exactly kT and updates its output at a fixed delay after each sample. Book 3, Ch. 35 §35.8 shows that a delay τ_d costs phase $\omega_c \tau_d$ at the crossover frequency. A delay that varies by $\Delta\tau$ therefore modulates the phase margin by $\omega_c \Delta\tau$ from one cycle to the next and injects noise proportional to the signal's slope, as sampling jitter does in a converter (Book 3 §19.5).

Worked example — Fixed delay or minimum delay? A motor drive's current loop runs at 20 kHz ($T = 50$ µs) with a crossover of 1 kHz. The control computation takes 10–15 µs depending on the path through the code.

- If the PWM duty cycle is written as soon as the computation finishes, the delay is 10–15 µs: average phase loss about $360^\circ \times 1000 \times 12.5\ \mu\text{s} = 4.5^\circ$, varying by $360^\circ \times 1000 \times 5\ \mu\text{s} = 1.8^\circ$ from cycle to cycle — and an update that lands in the middle of a PWM period produces an asymmetric pulse.

- If the duty cycle is written into the timer's shadow register and loaded at the next carrier peak, one full PWM period after the sample, the delay is a constant 50 µs: 18° of phase loss, but zero jitter and clean pulses.

Industrial drives almost always choose the second (Figure 27.6), sampling the current with a timer-triggered ADC at the carrier peak, the centre of the PWM period; loading the shadow register at the next valley instead halves the delay to 25 µs (9°) if the computation fits. The pattern generalises: let hardware fix the instants of sampling and actuation, and let software only have to finish in time. This is also the idea of the **logical execution time** model used in automotive and avionics frameworks, where each task reads its inputs at release and publishes its outputs exactly at its deadline.



Phase lag at the 1 kHz crossover: immediate update 4.5° ± 0.9° (varies every cycle); shadow register 18° (constant)

Figure 27.6 — Jitter in a 20 kHz current loop: immediate update versus shadow-register update at the PWM period boundary.

Measuring latency and jitter

The simplest instrument is a GPIO pin set at the start and cleared at the end of an ISR or task, watched on an oscilloscope in persistence mode or a logic analyser over hours: the spread of the edges is the jitter, and the scope's histogram gives its distribution. The DWT cycle counter records maxima in firmware. On Linux, `cyclictest` measures scheduling latency; a tuned PREEMPT_RT system on a modern Arm or x86 processor typically shows worst cases of tens of microseconds over long runs, against milliseconds for a standard kernel under load. Worst cases must be measured under the worst load — all interrupts active, all DMA streams running, the flash being written, the network flooded — because that is when they occur.

27.11 Watchdogs and fault handling

Watchdog timers

A **watchdog** is a counter that resets the processor unless software restarts ("kicks") it before it expires. It is the last line of defence against hangs: an infinite loop, a deadlock, a task starved by a priority error, a peripheral that never completes, a corrupted program counter. Microcontrollers offer two kinds. An **independent watchdog** runs from its own low-speed RC oscillator, so it keeps counting even if the main clock fails; once started it often cannot be stopped by software. A **window watchdog** resets the processor if it is kicked too late *or too early*, catching code that runs too fast

(skipped work, a tight loop that kicks the dog). Safety-related designs add an **external watchdog** chip with its own oscillator, which also catches a processor whose clock or supply has failed in a way the internal watchdog shares.

A watchdog helps only if the kick means the whole system is healthy. Kicking from a timer interrupt proves only that the timer interrupt runs; kicking from the idle task proves that something lets the idle task run. The robust pattern is a supervisor that kicks only when every monitored task has reported progress:

```
 #include <stdint.h>
#include "FreeRTOS.h"
#include "task.h"

#define N_MONITORED 4u
static volatile uint32_t progress[N_MONITORED];  /* bumped by each task */
static uint32_t last_seen[N_MONITORED];

void health_checkin(unsigned id) { progress[id]++; }
void watchdog_kick(void);                    /* e.g. write 0xAAAA to IWDG->KR */
void fault_log_stalled(unsigned id);         /* record in no-init RAM */

static void supervisor_task(void *arg)
{
    (void)arg;
    TickType_t wake = xTaskGetTickCount();
    for (;;) {
        vTaskDelayUntil(&wake, pdMS_TO_TICKS(100));
        int all_alive = 1;
        for (unsigned i = 0; i < N_MONITORED; i++) {
            uint32_t now = progress[i];
            if (now == last_seen[i]) {           /* no progress in 100 ms */
                fault_log_stalled(i);
                all_alive = 0;
            }
            last_seen[i] = now;
        }
        if (all_alive)
            watchdog_kick();                /* only a healthy system kicks */
    }
}
```

Each monitored task calls `health_checkin` at least once per 100 ms from its main loop; a task with a longer period checks in from a timeout path. A single late check-in costs one missed kick, which the watchdog's longer timeout absorbs; a task that stays stalled stops the kicks for good and the watchdog resets the system, with the stalled task's identity already logged.

Worked example — Choosing the watchdog timeout. An STM32F4's independent watchdog runs from an internal RC oscillator specified between 17 and 47 kHz (nominal 32 kHz). With a prescaler of 32 and a count of 1000, the timeout is $32 \times 1000/32\ \text{kHz} = 1.0$ s nominally, but anywhere from $32\,000/47\ \text{kHz} = 0.68$ s to $32\,000/17\ \text{kHz} = 1.88$ s across parts and temperature. The minimum, 0.68 s, must exceed the longest legitimate interval between kicks with margin. The supervisor kicks every 100 ms, comfortably inside it — until a firmware update erases a 128 KB Flash sector, which takes about 1–2 s and stalls code executing from the same Flash bank. The watchdog fires in the middle of the update. The fixes are to run the erase from RAM while kicking between operations, to erase in smaller sectors, or to use a dual-bank part; lengthening the timeout to cover the erase would also make the watchdog useless as a guard for a 100 ms control loop. The maximum, 1.88 s, is the longest time the system may be hung before recovery, and must be acceptable to the application's safety analysis.

Faults and safe states

A watchdog catches hangs; processor **fault exceptions** catch illegal operations. On Cortex-M the HardFault, MemManage, BusFault and UsageFault exceptions fire on invalid memory accesses, MPU violations, undefined instructions, division by zero (when enabled) and unaligned accesses; the configurable fault status registers (CFSR, HFSR) and the fault address registers say what happened, and the stacked program counter says where. A fault handler in production firmware should:

1. capture the stacked registers, the fault status registers and the reset cause into a **no-init RAM** region (not cleared by the startup code of Section 25.8) or a reserved Flash page;
2. drive outputs to a **safe state** — PWM off, valves closed, heater off — directly through registers, without relying on drivers or the RTOS that may be corrupted;
3. reset the system, and on the next boot report the stored record and count consecutive faults, entering a degraded or service mode instead of resetting forever.

Stack overflow, the most common cause of mysterious faults, is detected by MPU guard regions below each task stack (an overflow faults immediately at the offending instruction) or, more weakly, by the RTOS checking a canary pattern at each context switch. Safety standards build these layers into requirements: IEC 60730 Annex H (Class B) for household appliance controls requires periodic CPU-register, RAM and Flash self-tests and an independent clock check; IEC 61508 and ISO 26262 call for diagnostic coverage figures that combine watchdogs, memory protection, lockstep cores and plausibility checks.

> **Status of the principle: Watchdog coverage**
> *Approximate.* A watchdog detects failures that stop the kick, so its coverage is only as good as the condition that gates the kick; faults that leave the kicking path alive — wrong but timely outputs, a stuck sensor — need plausibility checks, redundancy and independent monitoring.

The Pathfinder engineers were lucky in one respect: the watchdog-style timer that detected the missed deadline reset the system into a safe, communicating state every time, and the logs it preserved let them diagnose a priority inversion from more than a hundred million miles away. Designing that outcome in — bounded timing, proven schedules, and a recovery path that leaves evidence — is what distinguishes a real-time system from a fast one.

What remains open

Open question	Where it is answered
How do interrupt controllers, vector tables and hardware timers work inside the processor?	Chapters 16 and 17
How are superloop, interrupt-driven and RTOS-based firmware structures chosen, and how does the device boot?	Chapter 25
How are CAN, SPI, I²C and UART links timed at the bit level?	Chapter 26
How are state machines, drivers, low-power modes, hardware-in-the-loop tests and firmware validation engineered around these timing guarantees?	Chapter 28
How do caches, pipelines and memory hierarchies affect execution time?	Chapter 16
How are real-time constraints met when work is split between processors, FPGAs and accelerators?	Chapter 35
How much loop delay can a feedback controller tolerate?	*Circuits, Signals and Communications*, Ch. 35
How are motor-drive current and speed loops designed and tuned?	*Electrical Energy and Machines*, Ch. 12

Next: Chapter 28 assembles the pieces of this part into a design method — hardware abstraction, modular drivers, state machines, low-power design, logging, debugging and hardware-in-the-loop validation.

Chapter 28 — Embedded System Design

Part VI · Embedded and Real-Time Systems

A utility deploys 5,000 battery-powered soil-moisture sensors with LoRaWAN radios, each specified for ten years on one lithium–thionyl-chloride AA cell. Fourteen months later a fifth of them are dead. The hardware is fine. Two firmware decisions killed them: a sensor driver that busy-waits on an I²C acknowledge, so a sensor browning out in frost keeps the processor out of its 2 µA stop mode for minutes, and a logger that rewrites one sector of internal flash on every sample until it wears out, starting a reset loop that transmits at the highest spreading factor after every reboot. Each module passed its unit tests. What failed was design: an abstraction that hid time, a driver without a timeout, a state machine with no way out of a fault, an energy budget nobody wrote down, and a validation plan that never ran the device cold, on a dying battery, for a month.

> **How is firmware structured so that drivers, protocols and application logic can be developed, replaced and tested independently, without hiding the timing and power behaviour that matter on real hardware?**
> **How does an engineer budget energy, store data safely, find faults on a running target and prove that the firmware meets its requirements before it ships?**

Concept	Engineering destination
Hardware abstraction	Porting firmware across MCU families; host-side unit testing with fake hardware
Firmware modularity	Teams working in parallel; reuse of drivers and stacks across products
Device drivers	Non-blocking, timeout-bounded access to sensors, radios, memories and converters
State machines	Application control logic, protocol handlers, chargers, motor and power sequencing
Low-power design	Battery life of IoT nodes, wearables, medical implants and asset trackers
Data logging	Event recorders, flight and vehicle data loggers, metering, black boxes
Communications	Framing, integrity and retry layers on top of UART, CAN, radio and Ethernet links
Debugging	SWD/JTAG, breakpoints, watchpoints, SWO and ETM trace, fault analysis
Hardware-in-the-loop testing	ECUs, BMS and inverter controllers tested against simulated plants
Firmware validation	Unit, integration and system test; static analysis; coverage; safety standards

Roadmap. Layers and interfaces (28.1) → modules built on them (28.2) → drivers that never block or hang (28.3) → control logic as explicit state machines (28.4) → energy budgets and low-power practice (28.5) → logging that survives power loss and wear (28.6) → communication stacks (28.7) → debugging on the target (28.8) → hardware-in-the-loop testing (28.9) → validation against requirements and standards (28.10).

28.1 Hardware abstraction

> **Boundary.**
> Chapter 25 sets out firmware architectures (super-loop, event-driven, RTOS-based), peripheral drivers at the architecture level, boot and update; Chapter 26 owns the interface protocols themselves; Chapter 27 owns interrupts, scheduling and synchronisation. Chapter 21 covers the C and C++ used here. This chapter is the design practice that sits on top of all four.

Layers and what each one knows

Chapter 25 (§25.7) introduced the layering of drivers. In design practice the layers, and what each is allowed to know, are fixed explicitly (Figure 28.1):

1. **Register definitions** — the vendor's generated device header; never edited.
2. **Hardware abstraction layer (HAL)** — drivers for the MCU's *own* peripherals, offering operations such as "transfer these bytes on this bus" instead of register writes.
3. **Board support package (BSP)** — pin assignments, which SPI instance the radio is on, clock sources. A new board with the same MCU changes only this layer.
4. **Device drivers** — for *external* chips (sensor, radio, SPI flash, gate-driver IC), written against HAL interfaces, never registers.
5. **Services** — protocol stacks, file systems, logger, power manager, command shell.
6. **Application** — the product's behaviour, usually state machines (Section 28.4).

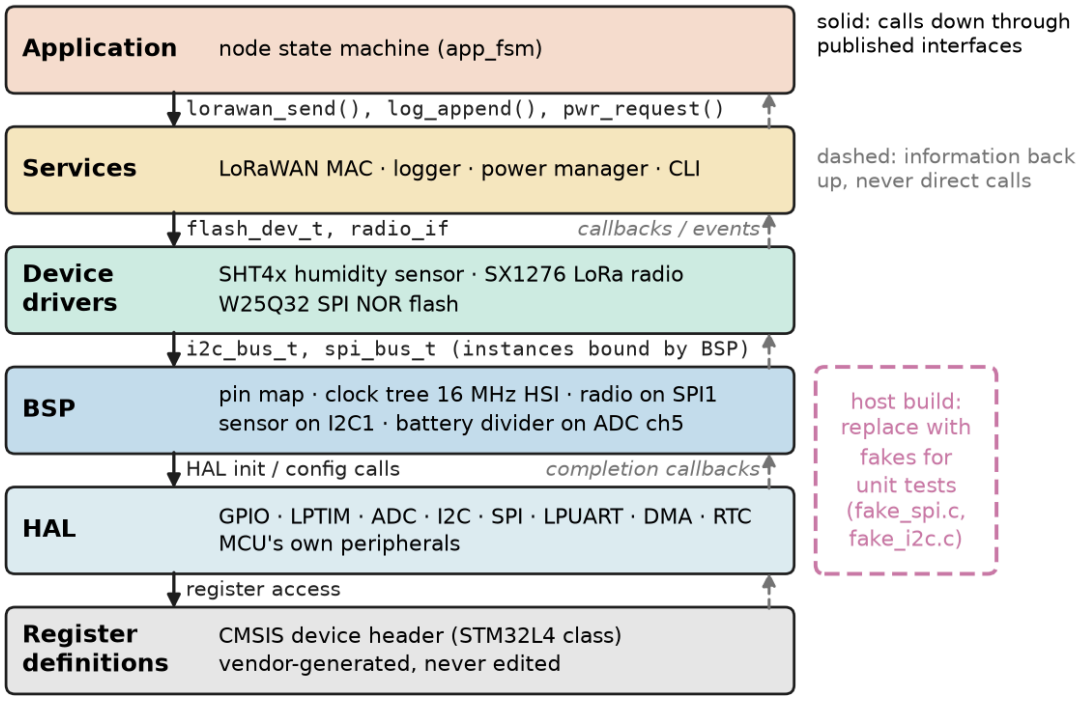


Figure 28.1 — Layered firmware architecture of a battery-powered LoRaWAN sensor node, with the interfaces each layer exposes.

The dependency rule is simple: a layer calls only downward through published interfaces, and information comes back up only through return values, callbacks or queued events. A device driver that includes the MCU's register header has broken the rule, and that driver will not move to the next product.

Binding an interface to an implementation

An interface can be bound to its implementation at three moments, each with a different cost:

Table 28.1 — Ways to bind a HAL interface (Cortex-M4 class, GCC -O2).

Binding	Mechanism	Run-time cost	Flexibility	Typical use
Compile time	Macros, `static inline` functions in a per-target header	None; fully inlined	One target per build	GPIO set/clear, register-level LL drivers
Link time	Same header, different `.c` file linked per target or for the host test build	One direct call, about 4–6 cycles	One implementation per symbol per image	Most HAL modules; unit-test fakes
Run time	`struct` of function pointers (a C "vtable")	One indirect call, about 5–8 cycles; no inlining	Several implementations in one image	Two SPI buses with different drivers; plug-in sensors

The listing shows a run-time-bound SPI interface and a fake that a host-side unit test links in place of the real port. The interface is *transaction-shaped* (one call per chip-select framed transfer), asynchronous (completion arrives through a callback), and it reports errors in a type every driver understands.

```
 /* spi_bus.h -- capability-level SPI interface (C11) */
#include <stddef.h>
#include <stdint.h>

typedef enum { BUS_OK = 0, BUS_TIMEOUT, BUS_NACK, BUS_BUSY, BUS_ERROR } bus_status_t;
typedef void (*bus_done_cb)(void *user, bus_status_t st);

typedef struct spi_bus spi_bus_t;
struct spi_bus {
    bus_status_t (*transfer)(spi_bus_t *bus, uint8_t cs,
                             const uint8_t *tx, uint8_t *rx, size_t n,
                             bus_done_cb done, void *user);
    void *impl;                          /* port-specific state */
};

extern spi_bus_t spi1_bus;               /* defined in spi_stm32l4.c */

/* test/fake_spi.c -- linked into the host build instead of the real port */
static uint8_t fake_rx[16];
static bus_status_t fake_transfer(spi_bus_t *bus, uint8_t cs, const uint8_t *tx,
                                  uint8_t *rx, size_t n, bus_done_cb done, void *user)
{
    (void)bus; (void)cs; (void)tx;
    for (size_t i = 0; rx != NULL && i < n && i < sizeof fake_rx; ++i) rx[i] = fake_rx[i];
    if (done != NULL) done(user, BUS_OK);
    return BUS_OK;
}
spi_bus_t fake_bus = { fake_transfer, NULL };
```

Where abstraction leaks

A HAL that hides *what* a peripheral does is useful; one that hides *how long it takes* or *what it costs in power* is dangerous. The leaks that bite:

- **Time.** A blocking call that spins up to its timeout keeps the core at full current and starves other work; the interface must make blocking visible or remove it.
- **DMA and memory.** DMA buffers may need alignment, DMA-reachable RAM and, on cached Cortex-M7 cores, cache maintenance around each transfer; an interface that accepts any pointer invites silent corruption.
- **Power.** A peripheral clock left enabled by a HAL `init` call costs tens of microamps in stop mode; the HAL must expose suspend and resume.

- **Interrupt context.** Callbacks usually run in interrupt context; the interface contract must say so (Chapter 27).

Vendor HALs (STM32Cube, NXP MCUXpresso) are quick to start with but tie code to one family; the Zephyr device model is portable but brings its own kernel and build system. A common compromise is the vendor's low-level layer underneath a thin, product-owned interface like the one above.

28.2 Firmware modularity

The module as the unit of design

In C, a **module** is a header that declares the interface and a source file that holds the implementation and its `static` state. Nothing outside the module touches that state directly. Four habits keep modules independent:

- **Opaque handles.** The header declares `typedef struct logger logger_t;` without its body, so callers cannot depend on fields.
- **Dependency injection.** A driver receives its bus (`spi_bus_t *`) at initialisation instead of naming `spi1_bus` , so one driver serves two instances and a test injects a fake.
- **Events, not calls, upward.** A lower module posts an event or invokes a registered callback; it never calls application code.
- **Static allocation.** Memory is allocated at build time or initialisation, never in steady state (MISRA C:2012 Directive 4.12), so heap fragmentation cannot fail the device after weeks.

The sensor node splits into `app_fsm` (behaviour and retry policy), `sht4x` and `sx127x` (sensor and radio drivers), `lorawan` (MAC layer), `logger` , `power` (sleep-mode selection) and `bsp` , each depending only on the interfaces below it.

Shared resources

Two drivers on one I²C bus, or a logger and a radio on one SPI bus, need an owner for the bus. A **bus manager** serialises transactions: drivers queue requests, and the manager runs them one at a time, with a mutex under an RTOS or a queue in an event-driven system (Chapter 27). Without it, a radio interrupt that starts an SPI transfer in the middle of a flash program corrupts both.

28.3 Device drivers

Anatomy of a driver

A device driver for an external chip does five things: **probe** (confirm the expected part is present, from an ID or serial-number register), **configure**, **operate**, **manage power** (enter the lowest-power state and wake), and **recover** (detect a stuck bus, a missing acknowledge or a bad CRC, report it, and restore a known state, for instance with the I²C bus-clear procedure of Chapter 26). It also documents its **contract**: from which context each function may be called, whether it is re-entrant, and its worst-case duration and energy per operation.

Blocking, non-blocking and DMA

Chapter 25 (§25.7) compares polling, interrupt and DMA service models for the MCU's own peripherals. For an external device the same choice appears one level up: a blocking driver is simplest and wrong for most products, because while it waits for an 8.3 ms humidity conversion the CPU does nothing useful at several milliamps. The better shape is a **non-blocking driver with a completion callback**: it starts an operation and returns, a timer or bus interrupt advances it, and a callback reports the result, so the driver is itself a small state machine; any heavy processing is deferred from interrupt to task level (Chapter 27).

The listing drives a Sensirion SHT4x humidity sensor without ever blocking. It sends the high-repeatability measurement command, arms a 10 ms one-shot timer (the datasheet maximum conversion time is about 8.3 ms), reads six bytes, checks the two CRC-8 values and converts the raw words to milli-degrees Celsius and milli-percent relative humidity. Every step can fail, and every failure returns the driver to `S_IDLE` and reports to the caller; the soft timer reuses the bus-completion callback type so that one function, `step`, advances the machine.

```
 /* sht4x.c -- non-blocking SHT4x driver (C11, illustrative HAL names) */
#include <stdbool.h>
#include <stdint.h>
#include "i2c_bus.h"     /* i2c_write_async(), i2c_read_async(), bus_status_t */
#include "soft_timer.h"  /* timer_oneshot_ms(ms, bus_done_cb, user)           */

#define SHT4X_ADDR      0x44u
#define SHT4X_MEAS_HIGH 0xFDu                /* high repeatability */

typedef void (*sht_cb)(bool ok, int32_t m_degC, int32_t m_RH);
typedef enum { S_IDLE, S_CMD, S_WAIT, S_READ } sht_state_t;
static struct { sht_state_t st; uint8_t buf[6]; sht_cb cb; } s;

static uint8_t crc8(const uint8_t *p, int n)  /* poly 0x31, init 0xFF */
{
    uint8_t c = 0xFF;
    while (n--) {
        c ^= *p++;
        for (int i = 0; i < 8; ++i)
            c = (c & 0x80u) ? (uint8_t)((c << 1) ^ 0x31u) : (uint8_t)(c << 1);
    }
    return c;
}

static void fail(void) { s.st = S_IDLE; s.cb(false, 0, 0); }

static void step(void *user, bus_status_t r)
{
    (void)user;
    if (r != BUS_OK) { fail(); return; }
    switch (s.st) {
    case S_CMD:  s.st = S_WAIT; timer_oneshot_ms(10, step, NULL); break;
    case S_WAIT:
        s.st = S_READ;
        if (i2c_read_async(SHT4X_ADDR, s.buf, 6, step, NULL) != BUS_OK) fail();
        break;
    case S_READ: {
        if (crc8(&s.buf[0], 2) != s.buf[2] || crc8(&s.buf[3], 2) != s.buf[5]) { fail(); return; }
        int64_t t  = ((int64_t)s.buf[0] << 8) | s.buf[1];
        int64_t rh = ((int64_t)s.buf[3] << 8) | s.buf[4];
        s.st = S_IDLE;
        s.cb(true, (int32_t)(-45000 + 175000 * t / 65535),
                   (int32_t)(-6000 + 125000 * rh / 65535));
        break;
    }
    default: fail(); break;
    }
}

bool sht4x_start(sht_cb cb)
{
    static const uint8_t cmd = SHT4X_MEAS_HIGH;
    if (s.st != S_IDLE) return false;
    s.cb = cb; s.st = S_CMD;
    if (i2c_write_async(SHT4X_ADDR, &cmd, 1, step, NULL) != BUS_OK) { s.st = S_IDLE; return false; }
    return true;
}
```

The arithmetic is done in 64 bits because $175000 \times 65535 \approx 1.15 \times 10^{10}$ overflows a 32-bit integer: in 32-bit arithmetic the temperature conversion wraps for every raw value above about 12,270 (about −12 °C) and the humidity conversion above about 27 % RH, a classic bug that a test at one fixed condition can miss. Note also what the driver does not do: it never waits, and the bus layer below it owns the timeout on each transfer.

Worked example — DMA buffer for a 921.6 kbaud UART. Chapter 25 sized an interrupt-driven ring buffer for 115,200 baud; at eight times the rate the per-byte interrupt itself becomes the problem. A GNSS receiver streams to an MCU at 921,600 baud, 8N1, so 10 bits per byte and $921600/10 = 92{,}160$ bytes/s. The worst-case delay before the task that drains the buffer runs, measured on the target under full load, is 5 ms.

- Bytes arriving in that time: $92160 \times 0.005 = 461$. A 1024-byte circular DMA buffer with half- and full-transfer interrupts gives 512 bytes per half, enough.
- Interrupt rate with DMA: one per 512 bytes, $92160/512 = 180$ interrupts/s.
- Interrupt rate with a per-byte receive interrupt: 92,160/s. At about 1 µs per interrupt (entry, handler and exit at 168 MHz), that is $92160 \times 1\ \mu\text{s} = 92$ ms of every second, 9.2 % of the CPU, before any parsing.

An idle-line interrupt delivers short bursts promptly. Size buffers from the line rate and a *measured* worst-case service latency, never from the average message size.

28.4 State machines

Why explicit state machines

Chapters 14 and 15 build finite-state machines in hardware. In firmware the same idea organises control logic: a battery charger, a protocol handler, a power-up sequence for an FPGA's rails. The alternative, Boolean flags tested in `if` statements, has 2^n implicit states for n flags, most of them unintended and untested. An explicit machine names every state, every event and every transition, and a reviewer can ask of each state the two questions that would have saved the sensor fleet: *what ends this state if the expected event never arrives?* and *what happens to each event this state does not expect?*

Event-driven, run-to-completion

Firmware state machines are usually **event-driven**: interrupts and drivers post events (timer expired, measurement done, packet received) into a queue, and a dispatcher hands one event at a time to the machine, which runs its transition to completion before taking the next.

> **Status of the principle: Run-to-completion event processing**
> *Derived.* If each event is handled atomically from the machine's point of view, the machine is never observed between states and no locking is needed inside it; this holds only while the worst-case handler time, multiplied by the queue depth, stays below the minimum event inter-arrival interval, otherwise the queue overflows and events are lost.

Figure 28.2 shows the node's machine. Every waiting state has a timeout, and a fault state with a bounded retry count prevents the reset-and-transmit loop of the opening paragraph.

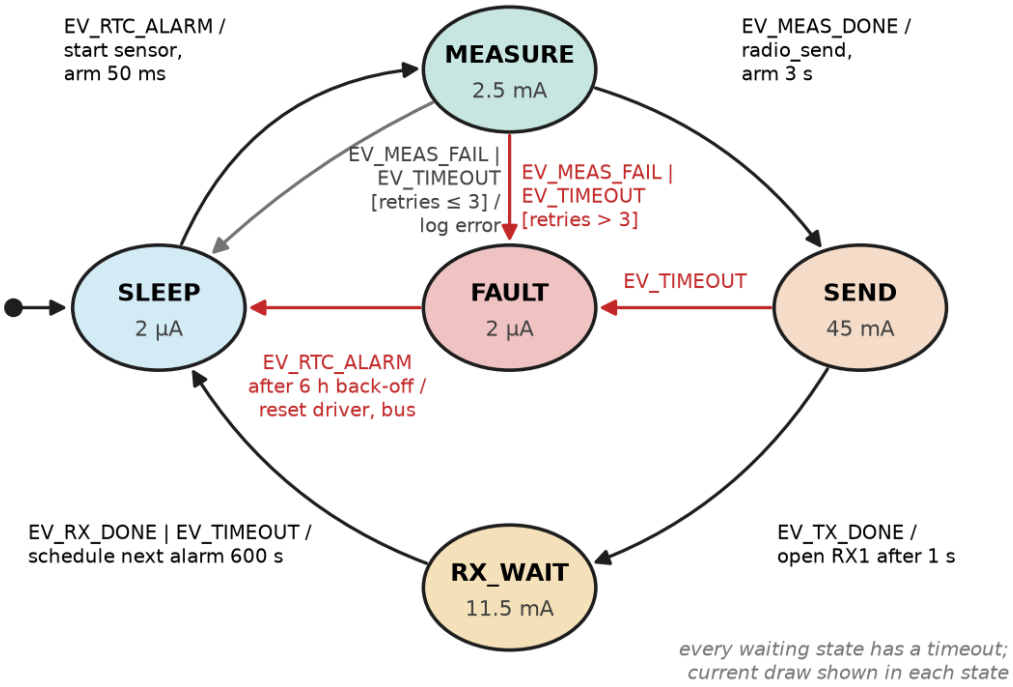


Figure 28.2 — Event-driven state machine of the sensor node, with a timeout on every waiting state.

Implementations

A **switch on the state** is compact and readable up to about ten states. A **transition table** (state × event → action, next state) is data, so it can be generated from a model and checked mechanically. **State functions** (a pointer to the current state's handler) underlie hierarchical frameworks such as QP, which implement UML statecharts with entry and exit actions and nested states, so that every operational state can inherit one "battery critical" transition.

```
 /* app_fsm.c -- switch-based node state machine (C11) */
#include "node_services.h"  /* sensor_start(), radio_send(), ERR_SENSOR, ... */

typedef enum { ST_SLEEP, ST_MEASURE, ST_SEND, ST_RX_WAIT, ST_FAULT } state_t;
typedef enum { EV_RTC_ALARM, EV_MEAS_DONE, EV_MEAS_FAIL,
               EV_TX_DONE, EV_RX_DONE, EV_TIMEOUT } event_t;

static state_t state = ST_SLEEP;
static unsigned retries;

void app_dispatch(event_t ev)
{
    switch (state) {
    case ST_SLEEP:
        if (ev == EV_RTC_ALARM) { sensor_start(); timeout_arm_ms(50); state = ST_MEASURE; }
        break;
    case ST_MEASURE:
        if (ev == EV_MEAS_DONE) {
            retries = 0; radio_send(); timeout_arm_ms(3000); state = ST_SEND;
        } else if (ev == EV_MEAS_FAIL || ev == EV_TIMEOUT) {
            log_error(ERR_SENSOR);
            if (++retries > 3) { fault_enter(); state = ST_FAULT; }
            else               { alarm_next_s(600); state = ST_SLEEP; }
        }
        break;
    case ST_SEND:
        if (ev == EV_TX_DONE)      { rx1_open_after_ms(1000); state = ST_RX_WAIT; }
        else if (ev == EV_TIMEOUT) { radio_reset(); fault_enter(); state = ST_FAULT; }
        break;
    case ST_RX_WAIT:
        if (ev == EV_RX_DONE || ev == EV_TIMEOUT) { radio_sleep(); alarm_next_s(600); state = ST_SLEEP; }
        break;
    case ST_FAULT:
        if (ev == EV_RTC_ALARM) { bus_recover(); retries = 0; alarm_next_s(600); state = ST_SLEEP; }
        break;
    }
    /* events not listed for a state are ignored deliberately; count them */
}
```

The comment at the end matters: ignoring an unexpected event must be a decision, logged in a diagnostic counter, not an accident. A transition table makes the decision visible as an explicit "ignore" cell.

Checking a machine

Unit tests that drive every (state, event) pair and assert the next state give complete **transition coverage**; for protocol machines talking across a lossy link, model checkers (SPIN, TLA+, UPPAAL) find deadlocks and livelocks that tests rarely hit.

28.5 Low-power design

> **Boundary.**
> Chapter 25 (§25.5) sets up the average-current budget $\bar{I} = I_{\text{sleep}} + \sum_k I_k t_k / T_k$, the charge-reservoir model of battery life with its limits, and the MCU power-mode ladder; Chapter 33 gives the CMOS physics behind the currents. This section uses the budget as a design tool: where the charge goes in a radio node, which board-level leaks spoil it, and which firmware practices make it come true.

The energy budget as a design tool

The budget is a table of phases, currents and measured durations, written before the firmware and checked against a power profiler (Nordic PPK2, Joulescope, Otii) that resolves 100 nA to 100 mA with microsecond timing. A multimeter on a shunt cannot: its burden voltage browns out the device at 45 mA radio peaks, and its averaging hides the pulses that dominate the budget.

Table 28.2 — Representative currents of the parts around a sleeping MCU (typical datasheet values at 3 V, 25 °C; MCU modes are in Table 25.5).

Block	Mode	Current
LoRa transceiver (SX127x class)	Sleep	0.2 µA
	RX	10–12 mA
	TX +14 dBm (module, incl. MCU)	about 45 mA
Digital humidity sensor (SHT4x class)	Idle / measuring	0.08 µA / 0.3–0.5 mA
Nano-power LDO (TPS7A02 class)	Quiescent	25 nA
10 kΩ pull-up held low at 3.3 V	—	330 µA
1 MΩ + 1 MΩ battery divider at 3.6 V	—	1.8 µA

The last two rows are the traps. A pull-up resistor on a line that a sleeping peripheral holds low, or a battery-measurement divider left permanently connected, can cost more than the whole sleeping MCU. Good boards switch the divider with a MOSFET, use the MCU's internal pull-ups only while needed, and leave no input floating (a floating CMOS input near mid-rail can draw tens of microamps through its input stage).

Worked example — ten years on one AA cell? The node wakes every $T = 600$ s, measures for 30 ms at 2.5 mA, transmits an 11-byte payload (24 bytes with the 13-byte LoRaWAN header) at 45 mA, and opens two receive windows of about eight symbols each at 11.5 mA; it sleeps at 2.0 µA in between. The cell is a 2.6 Ah Li-SOCl₂ AA, derated to 80 % usable for temperature and pulse load, with 1 % per year self-discharge. LoRa time on air follows from the spreading factor (Semtech's formula, in the listing):

```
import math

def lora_toa(payload, sf, bw=125e3, cr=1, n_pre=8):
    """LoRa time on air (s), explicit header, CRC on (Semtech AN1200.13)."""
    de = 1 if (sf >= 11 and bw == 125e3) else 0
    t_sym = 2**sf / bw
    n = 8 + max(math.ceil((8*payload - 4*sf + 44) / (4*(sf - 2*de))) * (cr + 4), 0)
    return (n_pre + 4.25 + n) * t_sym, t_sym

def budget(sf, period=600.0, i_sleep=2.0e-6, cap_ah=2.6, usable=0.8,
           self_dis=0.01):
    t_tx, t_sym = lora_toa(24, sf)               # 11-byte payload + 13 B MAC
    phases = {"measure": (2.5e-3, 0.030),        # (current A, duration s)
              "transmit": (45e-3, t_tx),
              "rx windows": (11.5e-3, 2*8*t_sym)}
    q = {k: i*t for k, (i, t) in phases.items()}
    t_active = sum(t for _, t in phases.values())
    q["sleep"] = i_sleep * (period - t_active)
    i_avg = sum(q.values()) / period
    i_sd = self_dis * cap_ah / 8760              # self-discharge as a current
    years = cap_ah*usable / (i_avg + i_sd) / 8760
    return t_tx, q, i_avg, years

for sf in (7, 10):
    t_tx, q, i_avg, yrs = budget(sf)
    parts = ", ".join(f"{k} {v*1e6:.0f} uC" for k, v in q.items())
    print(f"SF{sf}: ToA {t_tx*1e3:.1f} ms; per cycle {parts}")
    print(f"      average {i_avg*1e6:.2f} uA -> {yrs:.1f} years")
```

```
SF7: ToA 61.7 ms; per cycle measure 75 uC, transmit 2776 uC, rx windows 188 uC, sleep 1200 uC
      average 7.07 uA -> 23.7 years
SF10: ToA 370.7 ms; per cycle measure 75 uC, transmit 16681 uC, rx windows 1507 uC, sleep 1199 uC
      average 32.44 uA -> 6.7 years
```

At SF7 the arithmetic gives 23.7 years, so the cell's 10–15-year shelf life is the real limit. At SF10, which adaptive data rate assigns to nodes far from a gateway, the transmit charge rises sixfold and the life falls to 6.7 years: the specification is missed by radio physics, not firmware. Two more numbers explain the failed fleet. A sensor that never acknowledges, polled in a busy loop at 2 mA, costs $2\ \text{mA} \times 600\ \text{s} = 1.2\ \text{C}$ per period if it hangs for a whole period, nearly 300 times the normal 4.2 mC; a few days of this per winter consume years of budget. And a reset loop that transmits at SF12 (1.48 s on air) every few seconds empties the cell in weeks.

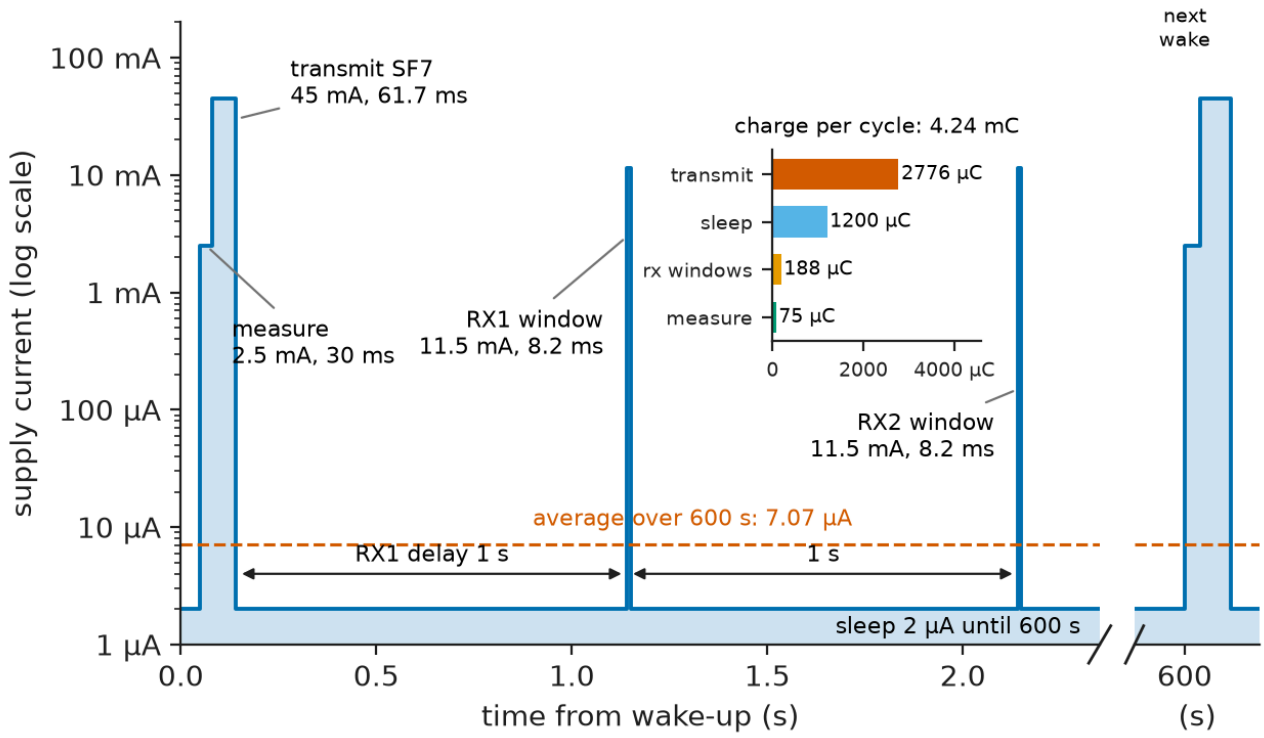


Figure 28.3 — Current profile of one 600 s reporting cycle of the sensor node at SF7, on a logarithmic current axis.

Firmware practices that make the budget true

- **Race to sleep.** At about 100 µA/MHz the charge per operation barely depends on clock frequency, while every extra millisecond awake adds peripheral and regulator current; so run fast and sleep. The exception is a core whose supply voltage scales with frequency, where energy per cycle falls as V^2 (Chapter 33).
- **A power manager chooses the sleep mode.** Drivers *request* the shallowest mode they tolerate (a receiving UART needs its clock); the manager enters the deepest mode compatible with all requests. No driver executes the sleep instruction itself.
- **Clock and pin hygiene.** Gate peripheral clocks after use, put unused pins in analogue mode, and never back-power an unpowered chip through its protection diodes.
- **Tickless idle.** Replace the 1 kHz RTOS tick by a low-power timer programmed for the next due event (Chapter 27).
- **Measure across corners.** Leakage roughly doubles every 10–15 K; a 2 µA stop current at 25 °C may be 10–20 µA at 60 °C.

28.6 Data logging

What a logger must guarantee

A lift's event recorder, a drone's flight-data logger, an energy meter's billing register and an infusion pump's history share four requirements: **integrity** (a record is fully correct or recognisably absent), **power-fail safety** (power loss at any instant leaves the log readable), **endurance** (the medium outlives the product), and **ordering** (sequence numbers and timestamps).

Chapter 25 (§25.4, Table 25.4) compares the non-volatile media and shows why a log on flash must be appended and spread over many pages, since a bit can be programmed from 1 to 0 but returns to 1 only when a whole sector is erased. This section designs such a log on an external SPI NOR flash (W25Q class: 4 KB sectors, 256-byte pages, 100,000 cycles,

page program 0.7 ms typical and 3 ms maximum, sector erase 45 ms typical and 400 ms maximum) so that it also survives power loss at any instant.

An append-only log with sequence numbers and CRCs

The robust structure (Figure 28.4) is a circular array of sectors, each starting with a small header (magic number, erase count) and filled with fixed-size records appended in order. Each record carries a 32-bit sequence number and a CRC over its contents. One sector ahead of the write position is always kept erased, so that the slow erase never happens at the moment a record must be stored. At boot, the logger scans the records and takes the head to be just after the valid record with the highest sequence number; a record torn by a power failure fails its CRC and is ignored, and everything before it remains intact.

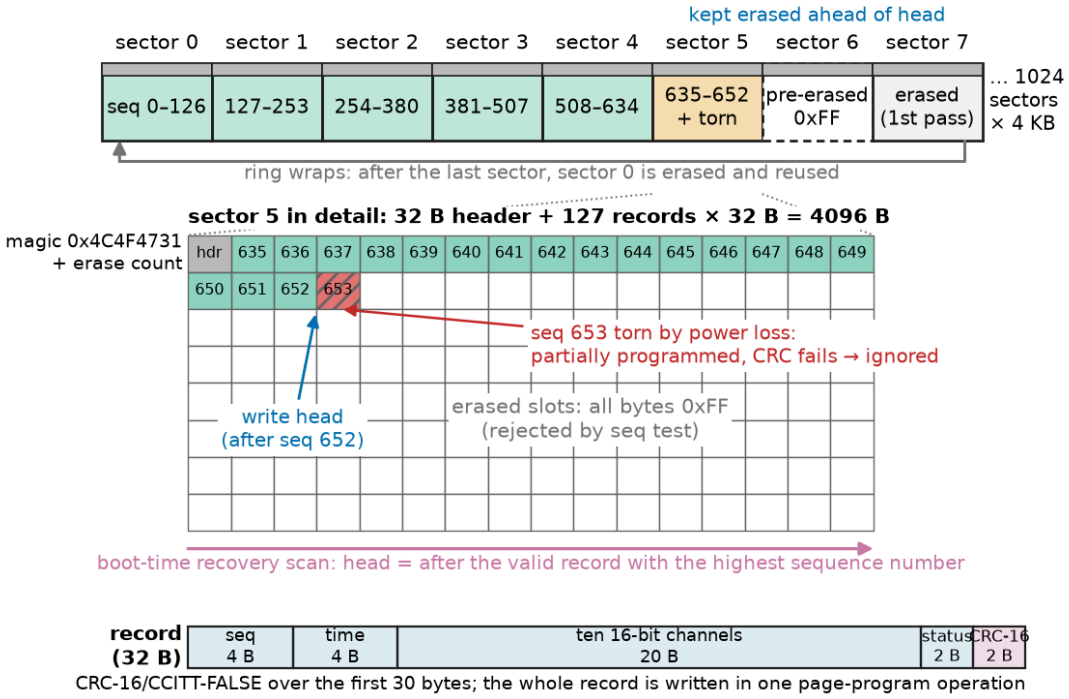


Figure 28.4 — Circular append-only log on a 4 MB SPI NOR flash: sector headers, records with sequence numbers and CRCs, the pre-erased sector and a torn record after power loss.

```
 /* log.c -- append and recovery for 32-byte records (C11, illustrative flash API) */
#include <stdbool.h>
#include <stddef.h>
#include <stdint.h>
#include "flash_dev.h"   /* flash_read(), flash_program(); SECTOR_SIZE, HDR_SIZE, LOG_SIZE */

typedef struct {
    uint32_t seq;
    uint32_t time_s;
    int16_t  ch[10];
    uint16_t status;
    uint16_t crc;          /* CRC-16/CCITT-FALSE over the first 30 bytes */
} log_rec_t;
_Static_assert(sizeof(log_rec_t) == 32, "record must be 32 bytes");

static uint32_t head, next_seq;

uint16_t crc16_ccitt(const uint8_t *p, size_t n)
{
    uint16_t c = 0xFFFFu;
    while (n--) {
        c ^= (uint16_t)(*p++ << 8);
        for (int i = 0; i < 8; ++i)
            c = (c & 0x8000u) ? (uint16_t)((c << 1) ^ 0x1021u) : (uint16_t)(c << 1);
    }
    return c;
}

static bool rec_valid(const log_rec_t *r)
{
    return r->seq != 0xFFFFFFFFu &&
           r->crc == crc16_ccitt((const uint8_t *)r, offsetof(log_rec_t, crc));
}

void log_recover(void)                      /* called once at boot */
{
    bool found = false;
    for (uint32_t a = 0; a < LOG_SIZE; a += sizeof(log_rec_t)) {
        if (a % SECTOR_SIZE == 0) a += HDR_SIZE;       /* skip sector header */
        log_rec_t r;
        flash_read(a, &r, sizeof r);
        if (rec_valid(&r) && (!found || (int32_t)(r.seq - next_seq) >= 0)) {
            found = true; next_seq = r.seq + 1u; head = a + sizeof r;
        }
    }
}
```

Three details carry the robustness. The comparison `(int32_t)(r.seq - next_seq) >= 0` survives sequence-number wrap-around. The record's CRC is computed by the writer before programming and the record is programmed in a single page-program operation, so a power failure leaves it either erased (all oxFF, rejected by the `seq` test) or torn (rejected by the CRC). And the C and Python CRC routines are checked against the same catalogue value, which turns the CRC into a testable interface between firmware and the PC tool that reads the logs:

```
def crc16_ccitt_false(data: bytes) -> int:
    crc = 0xFFFF
    for b in data:
        crc ^= b << 8
        for _ in range(8):
            crc = ((crc << 1) ^ 0x1021) if crc & 0x8000 else (crc << 1)
            crc &= 0xFFFF
    return crc

assert crc16_ccitt_false(b"123456789") == 0x29B1   # catalogue check value
print(hex(crc16_ccitt_false(bytes(range(30)))))    # a 30-byte test payload
```

```
0x3554
```

The printed value becomes a test vector for the C unit test of `crc16_ccitt`.

Worked example — will the log wear out? A pump-station logger stores one 32-byte record every 10 s.

- *SPI NOR, 4 MB, 1024 sectors of 4 KB, 100,000 cycles.* Each sector holds $(4096 - 32)/32 = 127$ records. The ring fills in $1024 \times 127 \times 10 = 1.30 \times 10^6$ s, 15.1 days, so each sector is erased once per 15.1 days and the endurance limit is reached after 15.05×10^5 days, about 4,100 years. Retention (20 years), not wear, limits this design.
- *The naive design* that erases and rewrites one sector to update a "latest value" record every 10 s reaches 100,000 cycles in 10^6 s, 11.6 days, even on the SPI NOR part.

Worked example — hold-up for a page program. To guarantee that a record write which has started also finishes, the supply must hold up through the worst-case page-program time of 3 ms at a total load of 25 mA, while the rail falls from 3.3 V to the flash's 2.7 V minimum: $C = It/\Delta V = 0.025 \times 0.003/0.6 = 125\ \mu$F. A sector erase (400 ms worst case) cannot be held up economically, which is exactly why the design pre-erases a sector ahead and relies on CRCs and sequence numbers rather than on finishing every operation.

FAT on an SD card is convenient for PC readout but not power-fail safe, because its tables are updated in place; littlefs and similar file systems use copy-on-write metadata and wear levelling instead. SD cards also draw 50–100 mA bursts during internal housekeeping, which belong in the energy and hold-up budgets.

28.7 Communications

> **Boundary.**
> The electrical and link-level behaviour of UART, SPI, I²C, CAN, USB and Ethernet belongs to Chapter 26. The theory of error-detecting codes, including the CRC's detection guarantees, is in *Circuits, Signals and Communications*, Ch. 28 §28.6: an r-bit CRC detects every burst error of length up to r and lets through about 2^{-r} of random error patterns. This section is about building a firmware stack from those pieces.

The layers of an embedded stack

A link that carries a byte stream (UART, RS-485, a USB CDC port, a TCP socket) needs the firmware to provide what the physical link does not:

1. **Framing** — byte stuffing (SLIP, HDLC) or **COBS** (at most one extra byte per 254) makes the frame delimiter unambiguous, so a receiver resynchronises after any corruption.
2. **Integrity** — CRC-16 for frames up to a few hundred bytes, CRC-32 beyond.
3. **Reliability** — sequence numbers, acknowledgements, timeouts and retransmission, where every message matters.

4. **Message encoding** — packed C structures (fast, but tied to compiler packing and endianness), CBOR, or Protocol Buffers via nanopb, which lets a newer peer add fields an older one skips.
5. **Application protocol** — Modbus, CANopen, MQTT, LoRaWAN, a BLE GATT service, or a product command set.

Links that already frame and check (CAN, Ethernet, LoRa) start at layer 3 or 4. Security is a further layer: TLS or DTLS through Mbed TLS costs 60–100 KB of flash and 20–40 KB of RAM on a Cortex-M4, plus seconds of CPU per handshake, all of which belong in the code-size and energy budgets. Every layer must also decide what happens when the next one is slower: the protocol states which messages may be dropped (periodic telemetry, superseded by the next sample) and which may not (alarms, configuration, billing data).

Worked example — stop-and-wait over RS-485. A data concentrator pulls logs from field units over RS-485 at 115,200 baud (10 bits per byte). Each frame carries 128 payload bytes plus 8 bytes of header and CRC; each acknowledgement is 8 bytes; each line turnaround (driver enable and receiver processing) takes 1 ms.

- Frame time $136 \times 10/115200 = 11.81$ ms; acknowledgement 0.69 ms; cycle $11.81 + 0.69 + 2 \times 1 = 14.50$ ms.
- Efficiency $= (128 \times 10/115200)/14.50 = 0.766$; throughput $128/0.0145 = 8.83$ kB/s against a raw 11.52 kB/s.
- On a noisy line with bit-error rate 10^{-5}, the frame-error probability is $1 - (1 - 10^{-5})^{1088} = 1.08\ \%$, and the retransmissions lower efficiency only slightly, to 0.758.

The turnaround, not the errors, costs most. A sliding-window protocol that keeps several frames in flight, or larger frames, recovers most of the 23 % lost; but larger frames raise the frame-error probability and the time to resend one, so frame size is chosen from the measured error rate.

28.8 Debugging

The on-chip debug infrastructure

Arm Cortex-M devices carry CoreSight debug logic reached through a **debug access port**. Two physical interfaces reach it:

- **JTAG** (IEEE 1149.1: TCK, TMS, TDI, TDO), which daisy-chains devices and adds **boundary scan** to find opens and shorts on assembled boards.
- **SWD** (Serial Wire Debug: SWCLK and bidirectional SWDIO), the same access with two pins; a third pin, SWO, carries trace.

Through it a probe (J-Link, ST-LINK, CMSIS-DAP, TRACE32) halts and steps the core, accesses memory while running and programmes flash. The resources inside are limited:

Table 28.3 — Debug resources of a Cortex-M4 and what they are for.

Resource	Capacity (typical)	Intrusiveness	Use
Hardware breakpoints (FPB)	6 instruction + 2 literal comparators	Halts core	Breakpoints in flash code
Watchpoints (DWT)	4 comparators	Halts core or emits trace	Find who corrupts a variable
DWT cycle counter `CYCCNT`	32 bits at core clock	None	Execution-time measurement
ITM stimulus ports over SWO	32 ports, 1-, 2- or 4-byte payloads	Microseconds per message	printf-style and event tracing
ETM instruction trace	4-bit trace port, or on-chip ETB buffer	None	Reconstruct every instruction executed
Memory access while running	Through the debug port	Small bus contention	Live variable watch, SEGGER RTT logging
Semihosting	—	Halts core on every call	Host file I/O in early bring-up only

Two warnings. Semihosting asks the debugger for service through a breakpoint instruction, so without a debugger attached it causes a HardFault. And the "debug in stop mode" option keeps clocks running so that the probe stays connected, making measured sleep current wrong by milliamps; measure power with the probe disconnected.

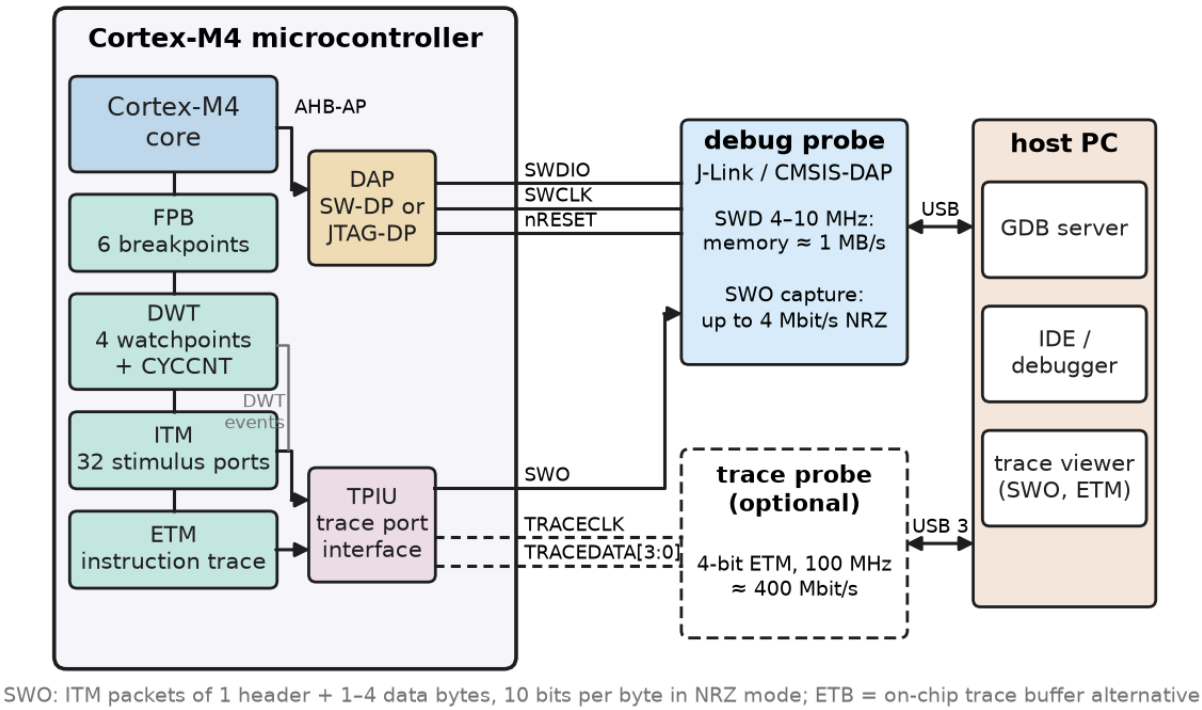


Figure 28.5 — Debug and trace paths of a Cortex-M4 microcontroller from the core to the host PC.

Fault analysis

On a HardFault the Cortex-M hardware has already pushed eight registers, including the faulting PC, onto the active stack, and the configurable fault status register (CFSR) records the cause. A handler that saves these into retained RAM turns a mysterious field reset into a line number:

```
 /* fault.c -- capture the stacked frame on HardFault (Cortex-M3/M4/M7, GCC) */
#include <stdint.h>

#define SCB_CFSR  (*(volatile uint32_t *)0xE000ED28u)
#define SCB_HFSR  (*(volatile uint32_t *)0xE000ED2Cu)
#define SCB_BFAR  (*(volatile uint32_t *)0xE000ED38u)

typedef struct { uint32_t r0, r1, r2, r3, r12, lr, pc, xpsr, cfsr, hfsr, bfar, magic; } crash_t;
__attribute__((section(".noinit"))) volatile crash_t crash;   /* survives reset */

void hardfault_c(const uint32_t *f)
{
    crash.r0 = f[0]; crash.r1 = f[1]; crash.r2 = f[2]; crash.r3 = f[3];
    crash.r12 = f[4]; crash.lr = f[5]; crash.pc = f[6]; crash.xpsr = f[7];
    crash.cfsr = SCB_CFSR; crash.hfsr = SCB_HFSR; crash.bfar = SCB_BFAR;
    crash.magic = 0xDEADFA17u;
    __asm volatile ("dsb");
    *(volatile uint32_t *)0xE000ED0Cu = 0x05FA0004u;          /* AIRCR: system reset */
    for (;;) { }
}

__attribute__((naked)) void HardFault_Handler(void)
{
    __asm volatile (
        "tst   lr, #4      \n"   /* EXC_RETURN bit 2: which stack was in use? */
        "ite   eq          \n"
        "mrseq r0, msp     \n"
        "mrsne r0, psp     \n"
        "b     hardfault_c \n");
}
```

At the next boot the application finds `crash.magic`, logs the record (Section 28.6) and reports it; on the bench, `arm-none-eabi-addr2line -e app.elf 0x<pc>` maps the saved PC to a source line. Stack overflow, the most common cause, is better caught at once by the MPU guard regions and bounded in advance by the stack analysis of Chapter 25 (§25.4).

Timing on the target

Two cheap instruments answer most timing questions. Toggling a spare GPIO at the entry and exit of an interrupt and watching it on a logic analyser shows latency, duration and jitter together. The DWT cycle counter, enabled by setting TRCENA in DEMCR (`0xE000EDFC`, bit 24) and CYCCNTENA in `DWT_CTRL` (`0xE0001000`, bit 0), reads elapsed core cycles from `0xE0001004` with no intrusion; at 168 MHz it wraps after 25.6 s.

Worked example — can SWO keep up with a control-loop trace? A 20 kHz motor-current loop logs two 32-bit values per cycle through ITM. Each ITM packet is one header byte plus four data bytes, 5 bytes, and SWO in UART (NRZ) mode sends 10 bits per byte.

- At SWO 4 Mbit/s: $4 \times 10^6 / (5 \times 10) = 80{,}000$ packets/s capacity, against $2 \times 20{,}000 = 40{,}000$ needed. It fits, at 50 % load.
- At 2 Mbit/s: capacity 40,000 packets/s, exactly the demand. Any other ITM traffic overflows the ITM FIFO, and the trace shows overflow packets and gaps.

The remedy, when the trace exceeds the link, is to log at a lower rate, log 16-bit values (3-byte packets), or capture into a RAM buffer and dump it afterwards, which is what SEGGER RTT and the ETB do.

28.9 Hardware-in-the-loop testing

From model to hardware

Embedded control software is verified on a ladder of increasingly real configurations: **model-in-the-loop** (algorithm model against plant model), **software-in-the-loop** (production C compiled for the host, against the plant model), **processor-in-the-loop** (production code on the target processor, exchanging data with the simulation, not in real time), and **hardware-in-the-loop** (HIL): the real, unmodified controller board, connected through its own connectors to a real-time simulator that emulates sensors, actuators, buses and the physical plant (Figure 28.6).

HIL is where the electrical interfaces, the timing and the fault handling are tested together. An automotive battery-management system on a HIL bench sees emulated cell voltages from programmable cell simulators (accurate to about 1 mV), emulated NTC thermistors as programmable resistances, a CAN bus on which the rest of the vehicle is simulated (restbus simulation), and an insulation-fault emulator. An inverter controller sees emulated phase-current sensor signals and an emulated encoder, computed from a motor and power-stage model.

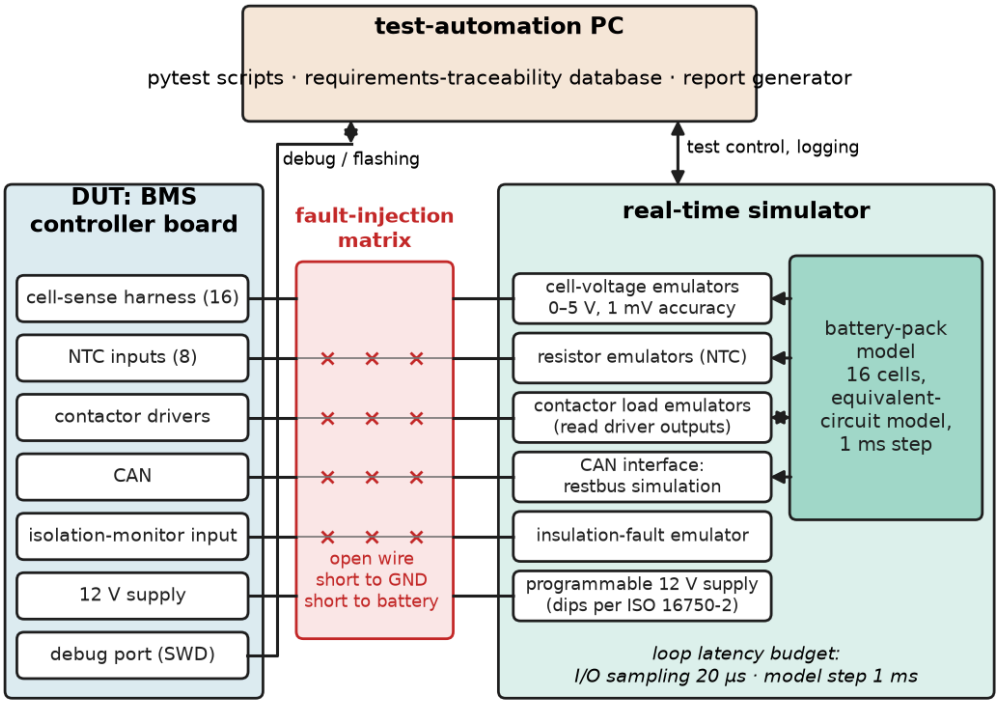


Figure 28.6 — Hardware-in-the-loop bench for a battery-management controller.

Real-time fidelity

The simulator must compute the plant fast enough that its step size and I/O latency do not change the closed-loop behaviour being tested. A useful rule is that the total delay (input sampling, one model step, output update) should add no more than a few degrees of phase at the bandwidth of the fastest loop under test, since a delay τ costs $360^\circ f\tau$ of phase at frequency f.

> **Status of the principle: Loop delay as phase lag**
> *Derived.* A pure delay τ has the transfer function $e^{-s\tau}$, with unit magnitude and phase $-360^\circ f\tau$ (*Circuits, Signals and Communications*, Ch. 34 §34.8); it is exact for a fixed delay, while a simulator with step-time jitter adds further error that grows with frequency.

Worked example — choosing a HIL platform for an inverter controller. The controller runs a 10 kHz current loop with 1 kHz closed-loop bandwidth.

- An FPGA-based simulator with a 1 µs model step and 2 µs of I/O conversion has a 3 µs loop delay: $360 \times 1000 \times 3 \times 10^{-6} = 1.1^\circ$ at 1 kHz. Negligible.
- A CPU-based simulator with a 50 µs step and 50 µs of I/O and scheduling has 100 µs delay: 36° at 1 kHz, enough to turn a loop with 50° of phase margin into one with 14°. The test would show oscillations that the real motor never has.

A battery pack, with electrochemical time constants of seconds, is well served by the 1 ms CPU step; a power stage switching at tens of kilohertz needs an FPGA solver with sub-microsecond steps to reproduce current ripple. Matching the simulator to the fastest physics under test is the first HIL design decision.

Fault injection and automation

HIL earns its cost in faults that are dangerous or impossible to create on real hardware: a cell sense wire breaking at full charge, an encoder losing a channel at 6,000 rpm, a CAN node going bus-off mid-sequence, a supply dip shaped as in ISO 16750-2. A relay matrix opens and shorts lines under script control; the scripts (pytest, dSPACE AutomationDesk, NI TestStand) are traced to requirements and run unattended against every new build.

28.10 Firmware validation

Verification and validation

Verification asks whether the firmware was built right (does it meet its specification?); **validation** asks whether the right firmware was built (does the product do what users and regulators need?). In the V-model used by safety standards, each specification level on the way down (system requirements, software requirements, architecture, detailed design) has a matching test level on the way up (system test on HIL and real hardware, software integration test, unit test), and every requirement is traced to the tests that verify it.

Unit, integration and system tests

The layering of Section 28.1 pays for itself here. Drivers and application logic that depend only on interfaces compile on the PC and run against fakes under a unit-test framework (Unity, CppUTest, GoogleTest), fast and under sanitizers that catch out-of-bounds accesses and undefined behaviour. Integration tests run on the target; system tests on the HIL bench and in environmental chambers. Building on Chapters 20 and 24, the embedded continuous-integration server also builds every variant, flashes a board farm and runs the HIL suites.

Static analysis and coding rules

Many embedded C failures (an uninitialised variable, an out-of-range index, a signed overflow, a missing `volatile`) are invisible in normal testing, so the code is analysed statically, in layers from cheap to rigorous: compiler warnings (`-Wall -Wextra -Wconversion`) as errors; checkers such as clang-tidy and cppcheck; a tool-checked coding standard (MISRA C:2012/2023 in automotive and industrial work, CERT C for security); sound abstract-interpretation analysers (Polyspace Code Prover, Astrée) that prove the absence of overflow and division by zero over all inputs; and worst-case stack and execution-time analysis (Section 28.8; Chapter 27).

Coverage and the safety standards

Coverage measures how much of the code the tests exercised. **Statement coverage** asks whether every statement ran; **branch coverage** whether every decision went both ways; **modified condition/decision coverage (MC/DC)** whether each condition in a decision was shown to affect the outcome independently.

Table 28.4 — Software assurance levels and coverage expectations (summary).

Standard	Domain	Levels (lowest to highest)	Structural coverage at the highest level
IEC 61508-3	Industrial functional safety	SIL 1–4	MC/DC highly recommended at SIL 4
ISO 26262-6	Road vehicles	ASIL A–D	MC/DC highly recommended at ASIL D
IEC 62304	Medical-device software	Class A, B, C	Unit verification required for class C; coverage set by the manufacturer's process
DO-178C	Airborne software	DAL E–A	MC/DC required at DAL A

Worked example — MC/DC for an interlock. A gate-driver enable decision in an inverter is `enable = armed && !(overtemp || overcurrent)`, three conditions. Exhaustive testing needs $2^3 = 8$ cases; MC/DC needs at least $N + 1 = 4$. One minimal set, written as (armed, overtemp, overcurrent) → enable:

```
 T1: (1, 0, 0) -> 1    baseline
T2: (0, 0, 0) -> 0    differs from T1 only in armed       -> armed shown independent
T3: (1, 1, 0) -> 0    differs from T1 only in overtemp    -> overtemp shown independent
T4: (1, 0, 1) -> 0    differs from T1 only in overcurrent -> overcurrent shown independent
```

Each condition flips the outcome while the others are held fixed. For a decision with 10 conditions, MC/DC needs at least 11 tests instead of 1,024, which is why it is affordable as a requirement for the most critical software.

> **Status of the principle: Structural coverage as evidence**
> *Empirical.* High coverage correlates with fewer escaped defects in practice, but coverage shows only that code was executed, not that its behaviour was checked; missing code (an absent timeout, an unhandled event) has no coverage to miss, which is why coverage supplements requirements-based testing and review rather than replacing them.

Robustness beyond functional tests

Products that pass every functional test still fail from conditions nobody scripted. Four activities close the gap: **fault injection** (corrupt memory, drop interrupts, force bus errors and watchdog expiries, and check that the safe state is reached); **fuzzing** of protocol parsers on the host with a coverage-guided fuzzer such as libFuzzer; **soak testing** for weeks across temperature, on a dying battery, with power cut at random instants to exercise the logger's recovery; and **EMC immunity** testing to IEC 61000-4-2, -4-4 and -4-5, after which the firmware must restart cleanly and its fault-capture path must record what happened.

The sensor fleet of the opening paragraph would have been saved by any one of these: a soak test in a freezer would have shown the hung sensor, a power-interruption test the log wear, and an energy budget checked against a profiler the cost of SF12 after every reboot.

What remains open

Open question	Where it is answered
How are interrupts, priorities, RTOS tasks and watchdogs designed so that deadlines are met?	Chapter 27
How are firmware images booted, authenticated and updated in the field?	Chapter 25
What do UART, SPI, I²C, CAN, USB and Ethernet look like on the wire, and how are their faults diagnosed?	Chapter 26
How does a microcontroller drive the switches of a power converter, and what does the gate driver need from it?	Chapter 29
How is a converter's feedback loop implemented in firmware, and how do sampling and computation delay limit it?	Chapter 30
When should a function move from firmware into an FPGA or a hardware accelerator, and how is the whole system verified?	Chapter 35
How are CRCs and error-correcting codes designed and analysed?	*Circuits, Signals and Communications*, Ch. 28
How are control loops designed and their margins specified?	*Circuits, Signals and Communications*, Ch. 34–35

Next: Chapter 29 leaves the microcontroller and turns to the switches it commands, the power semiconductors whose switching and conduction losses, gate drive and protection decide whether a converter is efficient and survives.

Part VII — Power Electronics

Every electronic system draws its energy through a converter: a processor's 1.0 V rail, a phone charger, an electric car's 400 V inverter, a pump drive, a solar inverter. This part builds those converters. Its abstraction is the **switching converter**: semiconductor switches that are either fully on or fully off, alternating tens of thousands to millions of times a second, with inductors and capacitors that average the chopped waveform into the DC or AC voltage the load needs, at efficiencies of 95–99 %. Like every abstraction in the book, it holds only inside limits: the switching and conduction losses of real devices, the junction temperature those losses produce, the overshoot that stray inductance adds at every edge, the time a device survives a short circuit, and the frequency at which the averaged model stops describing what the circuit does. The four chapters move from the switch itself, through DC–DC and AC conversion, to the drives, chargers and grid converters in which power electronics meets machines and the power system.

By the end of this part you should understand

- Why converters switch, how the commutation cell underlies every topology, and how Si MOSFETs, IGBTs, SiC MOSFETs and GaN HEMTs compare in on-state loss, switching energy, recovery charge and ruggedness.
- How switching and conduction losses are computed from datasheet values, how they set the junction temperature through steady and transient thermal impedances, and how the electrothermal loop is closed.
- How gates are driven and isolated, and how layout, snubbers, desaturation detection and soft turn-off protect fast switches.
- How the buck, boost and buck–boost converters set their conversion ratios, how their inductors and capacitors are sized for ripple and transients, what changes in discontinuous conduction, and where their losses go across the load range.
- What the averaged small-signal model is, why the boost has a right-half-plane zero, and how crossover and compensation are chosen.
- How rectifiers and PWM inverters convert between AC and DC, what harmonics they produce, and how their filters, ratings, protection and cooling are chosen.
- How these converters become motor drives, battery chargers, renewable-energy interfaces and grid-connected converters.

Chapters

Chapter	Central question	What it delivers
29 — Power-Electronic Fundamentals	How much power does a switch lose, how hot does it run, and how is it driven and protected?	The commutation cell, a four-technology comparison with datasheet numbers, switching and conduction loss models, junction-temperature design with Z_{th}, gate drivers, isolation and bootstrap supplies, snubbers, short-circuit protection and the switching-frequency trade-off
30 — DC–DC Converters	How are the basic DC–DC converters designed, and what does their feedback loop see?	Buck, boost and buck–boost in CCM and DCM, waveforms and component stresses, inductor and capacitor sizing with DC-bias derating, ripple and input filters, loss budgets and light-load modes, averaged models, Type III compensation and digital control
31 — AC–DC and DC–AC Conversion	How do rectifiers and inverters convert between AC and DC, and what do they cost the supply and the load?	Diode and thyristor rectifiers with overlap, inverter legs and dead time, sine-triangle and space-vector PWM, harmonic spectra, L and LCL filters, switching strategies, semiconductor selection, converter-level protection and power-cycling thermal design
32 — Motor Drives and Power Conversion	How are converters assembled into drives, chargers and grid interfaces?	Drive power stages for DC and AC machines, variable-frequency drives, regenerative operation, battery chargers, renewable-energy converters and grid-connected converter hardware

How this part connects to the rest of the book

This part rests on Part I: the diode recovery and Schottky diodes of Chapter 2, the gate charge, ratings and electrothermal behaviour of power MOSFETs in Chapter 4, and the thyristors, IGBTs and wide-bandgap devices of Chapter 5. The error amplifiers and references of Chapters 8 and 10 sit inside converter controllers, and the PWM timers and real-time firmware of Chapters 17 and 25–28 generate the switching signals and close the loops.

Among the sibling books, *Engineering Physics* gives the one-section sketch of rectifiers and switching converters that this part extends (Ch. 18 §18.8) and the thermal-resistance networks it uses (Ch. 14 §14.7); *Materials Science and Engineering* explains why SiC and GaN outperform silicon (Ch. 33). *Circuits, Signals and Communications* supplies the transient analysis, Fourier methods and the control theory (margins, loop shaping, digital control) on which converter design depends (Ch. 34, Ch. 35). *Electrical Energy and Machines* owns what lies beyond the converter's terminals: the machines and their drive control (V/f, FOC, DTC) in Ch. 10–12, energy storage in Ch. 16, and harmonic limits, grid codes and renewable grid integration in Ch. 25 and Ch. 27. This part stops at the converter hardware and its own loops.

Chapter 29 — Power-Electronic Fundamentals

Part VII · Power Electronics

A team redesigns one leg of a 10 kW, 400 V battery inverter, replacing its 650 V IGBTs switching at 20 kHz with SiC MOSFETs at 100 kHz so that the filter inductor shrinks to a fifth. The first prototype destroys a pair of transistors within a second of power-up: the fast voltage edge of one switch, coupled through the gate–drain capacitance of the other, lifts its gate above threshold and the leg shoots through. The second prototype survives but rings at 100 MHz with 200 V of overshoot on every turn-off and fails its conducted-emission test. The third runs, but its heatsink is 20 K hotter than the spreadsheet promised, because the spreadsheet used the on-resistance at 25 °C. None of these is a device failure. Each is a failure to treat the switch as what it is: a component with a finite transition time, parasitic capacitances and inductances around it, a gate that must be driven and protected, and a junction whose temperature closes a loop with its own losses. This chapter builds those models.

> **How much power does a semiconductor switch dissipate as it conducts and as it switches, and how do Si MOSFETs, IGBTs, SiC MOSFETs and GaN HEMTs compare with real datasheet numbers? How are the junction temperature, the gate drive, the snubbers, the protection and the switching frequency chosen so that the switch is efficient and survives?**

Concept	Engineering destination
Power switching	Every converter of Chapters 30–32: chargers, server supplies, inverters, drives
Semiconductor switch characteristics	Choosing between Si MOSFET, IGBT, SiC and GaN for a voltage class and frequency
Switching losses	Loss budgets from datasheet E_{on}, E_{off}, Q_{rr} and E_{oss}
Conduction losses	RMS-current design, synchronous rectification, dead-time loss
Thermal constraints	Heatsink and interface selection, junction-temperature margin, pulse overloads
Gate driving	Drive voltages, gate resistors, Miller clamps, isolation, bootstrap supplies
Snubbers	Overshoot and ringing control, EMI, turn-off loss transfer
Protection	Desaturation detection, soft turn-off, active clamping, UVLO
Switching-frequency trade-offs	Size versus efficiency, EMI bands, control bandwidth, minimum on-time

Roadmap. Why converters switch, and the commutation cell (29.1) → what distinguishes the four switch technologies (29.2) → switching losses (29.3) and conduction losses (29.4), combined in a device comparison → the junction temperature they produce (29.5) → driving the gate (29.6) → taming the parasitics with layout and snubbers (29.7) → protecting the switch (29.8) → choosing the switching frequency (29.9).

> **Boundary.**
> The rectifier and switching-converter sketch of *Engineering Physics*, Ch. 18 §18.8, is the starting point, and its thermal-resistance network, Ch. 14 §14.7, is used without re-derivation. Device structures and ratings come from this book: diode reverse recovery from Chapter 2 (§2.2) and SiC Schottky diodes (§2.5); the gate-charge curve, $t_v \approx Q_{gd}/I_G$, the electrothermal equilibrium of a conducting MOSFET, avalanche and SOA ratings from Chapter 4 (§4.9–4.10); IGBT, SiC and GaN device physics and ratings from Chapter 5. The material reasons behind wide-bandgap performance are in *Materials Science and Engineering*, Ch. 33 §33.4–33.5 and §33.10. The first- and second-order transients behind inductive kicks and ringing are from *Circuits, Signals and Communications*, Ch. 6 and Ch. 7.

29.1 Power switching

Why converters switch

A linear regulator controls power by dropping the unwanted voltage across a transistor in its active region. Delivering 20 A at 200 V from a 400 V bus that way dissipates $(400 - 200) \times 20 = 4$ kW in the transistor to deliver 4 kW to the load: 50 % efficiency at best, and a heatsink the size of a suitcase. A switch that is either fully on (current, almost no voltage) or fully off (voltage, almost no current) dissipates almost nothing in either state. If it alternates rapidly, an inductor and a capacitor average the chopped voltage into the wanted DC or AC value; *Engineering Physics* §18.8 sketches the result ($V_{out} = DV_{in}$ for the buck), and Chapter 30 designs it. The same 4 kW conversion with real switches loses 20–60 W.

> **Status of the principle: The lossless ideal switch**
> *Fundamental.* An ideal switch has zero voltage when on and zero current when off, so the product $v\,i$ is zero in both states and it dissipates no power; every loss in a real switch comes from a non-zero on-state voltage, a finite transition time during which voltage and current coexist, stored charge and capacitance that must be charged or discharged, and leakage.

The commutation cell

Almost every converter, from a 5 W phone charger to a 5 MW wind turbine, is built from one repeated unit, the **commutation cell** (Figure 29.1a): two switches (or a switch and a diode) connected in series across a stiff voltage V_{dc} held by a capacitor, with their midpoint, the **switch node**, feeding an inductor that carries a nearly constant current I_L through a switching period. When the upper switch turns on, the current commutates from the lower device to the upper; when it turns off, the current commutates back. The buck, boost and buck–boost converters of Chapter 30 and the inverter legs of Chapter 31 differ only in where the cell's terminals are connected.

The cell defines the hardest-working condition a switch faces, **clamped inductive switching** (Figure 29.1b). At turn-on, the switch must first take over the full inductor current while its voltage is still clamped at V_{dc} by the conducting diode; only when it carries all of I_L (plus the diode's reverse-recovery current) can the diode block and the voltage fall. At turn-off the order reverses: the voltage must rise all the way to V_{dc} before the diode can forward-bias and take the current, so the full current flows at rising voltage. In both transitions voltage and current overlap, and the energy dissipated is roughly $\frac{1}{2} V_{dc} I_L$ times the overlap time. This **hard switching** is the default; resonant and soft-switching converters (zero-voltage or zero-current switching) arrange for one of the two quantities to be near zero during the transition, at the cost of circulating current and narrower operating ranges.

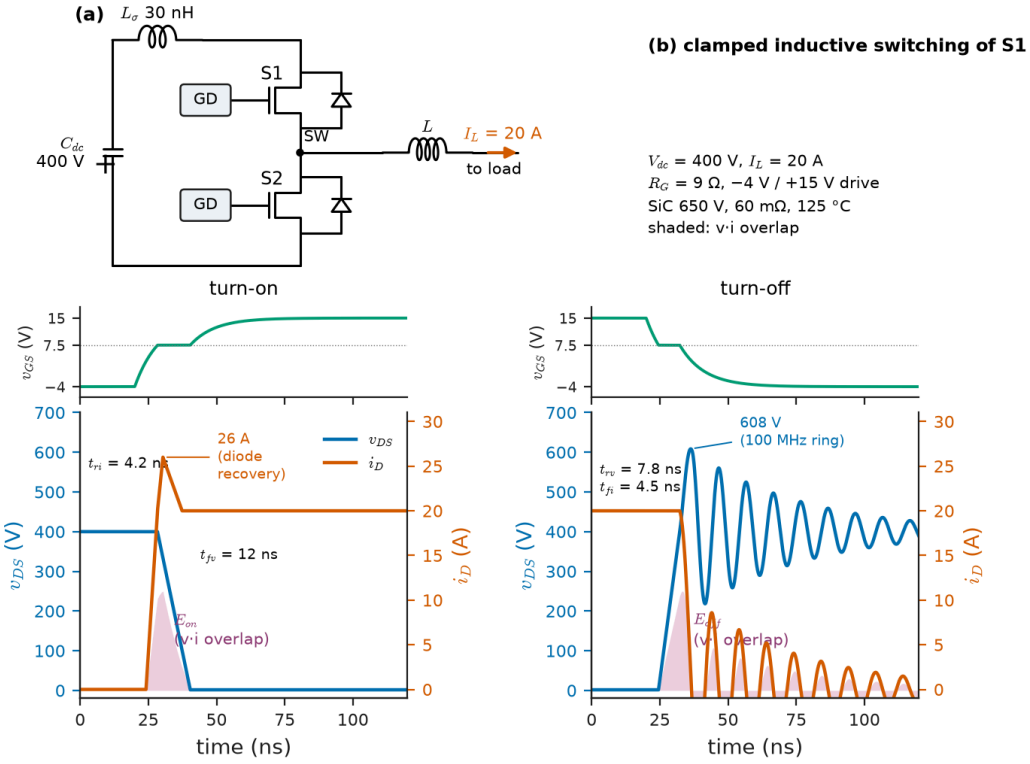


Figure 29.1 — (a) The commutation cell: a half-bridge across a 400 V capacitor with an inductor carrying 20 A from the switch node. (b) Clamped inductive turn-on and turn-off waveforms of the upper switch S1, with the overlap energy shaded.

29.2 Semiconductor switch characteristics

What a power switch is asked to do

A designer reads a power-semiconductor datasheet for seven properties: **blocking voltage** (with margin for overshoot and cosmic-ray derating; Chapter 31 gives the classes), thermally limited **current**, **on-state characteristic**, **switching energies**, **gate** levels and charge, **reverse conduction**, and **ruggedness** (avalanche, short-circuit withstand). Chapter 5 explains the values; here they are design inputs.

Two on-state models cover all devices. A unipolar device (Si MOSFET, SiC MOSFET, GaN HEMT) is a resistance:

$$v_{DS} = R_{DS(on)}(T_j)\, i_D$$

A bipolar or conductivity-modulated device (IGBT, PN diode, thyristor) has a threshold plus a slope resistance, fitted to the datasheet curve at the operating temperature:

$$v_{CE} = V_{CE0} + r_{CE}\, i_C, \qquad v_F = V_{F0} + r_F\, i_F$$

> **Status of the principle: Piecewise-linear on-state models**
> *Approximate.* Straight-line fits to the datasheet output characteristic at the stated gate drive and junction temperature; accurate to a few percent over the current range used for the fit, they fail at very low current (where the IGBT's exponential knee matters) and must be refitted at each temperature, since $R_{DS(on)}$ rises 1.3–2.2 times from 25 °C to 150 °C while V_{CE0} falls slightly and r_{CE} rises.

Four technologies at 650 V

The 650 V class, used for 400 V DC links in PFC stages, single-phase inverters, server supplies and 400 V EV systems, is the one place where all four technologies compete directly. Table 29.1 gives representative values for devices of similar die cost and roughly 40–50 mΩ or 40 A class.

Table 29.1 — Representative 650 V switches (typical datasheet values; switching energies at 400 V, 20 A, 125 °C in a hard-switched half-bridge with the recommended gate resistor).

Property	Si superjunction MOSFET (fast-diode type)	Si trench field-stop IGBT (with copack diode)	SiC MOSFET	GaN enhancement-mode HEMT
On-state at 25 °C	45 mΩ	$V_{CE(sat)}$ 1.5 V at 20 A	60 mΩ	50 mΩ
On-state at 125 °C	95 mΩ (×2.1)	$V_{CE0} = 0.95$ V, $r_{CE} = 30$ mΩ	81 mΩ (×1.35)	90 mΩ (×1.8)
Gate drive (on / off)	+10 to 12 V / 0 V	+15 V / 0 to −8 V	+15 to 18 V / −3 to −5 V	+6 V / 0 to −3 V
Total gate charge Q_g	90 nC	95 nC	60 nC	6 nC
E_{on} (incl. opposite device's recovery)	300 µJ	350 µJ	90 µJ	35 µJ
E_{off}	25 µJ	250 µJ (tail current)	25 µJ	8 µJ
Reverse-recovery charge Q_{rr}	0.6 µC	1 µC	0.1 µC	0
Reverse-conduction drop	0.9 V body diode	1.5 V copack diode	3–4 V body diode	2–3 V plus off-bias
Short-circuit withstand	Not rated	5–10 µs	2–3 µs	Under 1 µs
Typical hard-switched frequency	20–100 kHz (PFC, LLC)	2–30 kHz	20–200 kHz	100 kHz–1 MHz

Three features stand out. The IGBT's turn-off energy is ten times larger than the others', because after the channel turns off, the minority carriers stored in its drift region must recombine, producing a **tail current** that flows at full voltage; this caps IGBTs at tens of kilohertz. The superjunction MOSFET has low switching energy as a switch but a body diode with large reverse-recovery charge, which is why standard SJ devices are used where their body diode never conducts in hard commutation (boost PFC with a SiC diode, LLC converters with soft switching) and fast-diode variants are needed for bridges. SiC and GaN combine low switching energy with little or no recovery charge, at the price of fast edges that demand careful gate drive and layout (Sections 29.6–29.7).

The product $R_{DS(on)}Q_g$ is a **figure of merit**: the charge moved per cycle for a given conduction loss. From Table 29.1 it is about 4 Ω·nC for the SJ MOSFET, 3.6 for SiC and 0.3 for GaN. At 40–150 V the contest is between trench Si MOSFETs (Table 4.5) and GaN: a 5 mΩ, 100 V GaN device needs about 10 nC of gate charge against about 50 nC for silicon, which is why 48 V converters at 0.5–1 MHz increasingly use GaN.

29.3 Switching losses

Where the energy goes

At each hard-switched transition, four mechanisms dissipate energy in the switch:

1. **Current–voltage overlap.** During turn-on the current rises at full voltage, then the voltage falls at full current; during turn-off, the reverse.
2. **Reverse recovery** of the opposite diode (Chapter 2, §2.2): the switch turning on carries the load current plus the recovery current while still supporting most of V_{dc}, adding roughly $Q_{rr}V_{dc}$ per turn-on, mostly in the switch.
3. **Output capacitance.** At hard turn-on the switch discharges its own C_{oss} through its channel, dissipating the stored energy E_{oss} (the datasheet gives E_{oss} at the bus voltage, because C_{oss} is strongly nonlinear), and it also charges the opposite device's C_{oss}, drawing that charge from the bus through its own channel.
4. **Gate-drive energy** $Q_g \Delta V_{GS}$ per cycle, dissipated in the driver and gate resistors, not in the channel (Section 29.6).

The overlap intervals follow from the gate-charge picture of Chapter 4 (§4.9). With a driver swinging from V_{off} to V_{on} through a total gate resistance R_G (external plus internal), input capacitance C_{iss}, threshold V_{th} and plateau V_{pl}:

$$t_{ri} \approx R_G C_{iss} \ln \frac{V_{on} - V_{th}}{V_{on} - V_{pl}}, \qquad t_{fv} \approx \frac{Q_{gd} R_G}{V_{on} - V_{pl}}$$

$$t_{rv} \approx \frac{Q_{gd} R_G}{V_{pl} - V_{off}}, \qquad t_{fi} \approx R_G C_{iss} \ln \frac{V_{pl} - V_{off}}{V_{th} - V_{off}}$$

$$E_{on} \approx \tfrac{1}{2} V_{dc} I_L \left(t_{ri} + t_{fv}\right) + Q_{rr} V_{dc} + E_{oss}, \qquad E_{off} \approx \tfrac{1}{2} V_{dc} I_L \left(t_{rv} + t_{fi}\right)$$

> **Status of the principle: Overlap model of switching energy**
> *Approximate.* Assumes linear current and voltage ramps, a constant plateau, a lumped Q_{gd} and negligible stray inductance; it predicts trends with gate resistance, voltage and current well but is typically within a factor of 1.5–2 of measurement, and it overestimates the turn-off loss of fast unipolar devices, whose channel can turn off before the voltage rises, letting the load current charge C_{oss} losslessly.

Worked example — estimating a SiC MOSFET's switching energy from its gate charge. A 650 V, 60 mΩ SiC MOSFET switches 20 A at 400 V. At 125 °C its datasheet gives $C_{iss} \approx 1.0$ nF, $Q_{gd} \approx 10$ nC at 400 V, $V_{th} \approx 3.0$ V and a plateau of about 7.5 V at 20 A. The driver swings from −4 V to +15 V through $R_G = 9\ \Omega$ (5 Ω external plus 4 Ω internal).

- Turn-on: $t_{ri} = 9 \times 1.0\ \text{nF} \times \ln(12/7.5) = 4.2$ ns; $t_{fv} = 10\ \text{nC} \times 9/7.5 = 12.0$ ns. Overlap energy $\tfrac{1}{2} \times 400 \times 20 \times 16.2\ \text{ns} = 64.9$ µJ. Adding $Q_{rr}V_{dc} = 0.1\ \mu\text{C} \times 400 = 40$ µJ and $E_{oss} \approx 10$ µJ gives $E_{on} \approx 115$ µJ.
- Turn-off: $t_{rv} = 10\ \text{nC} \times 9/11.5 = 7.8$ ns; $t_{fi} = 9 \times 1.0\ \text{nF} \times \ln(11.5/7) = 4.5$ ns; $E_{off} \approx \tfrac{1}{2} \times 400 \times 20 \times 12.3\ \text{ns} = 49$ µJ.
- The edges imply $di/dt \approx 20/4.2 = 4.7$ A/ns and $dv/dt \approx 33$ V/ns at turn-on, 51 V/ns at turn-off.

The datasheet, measured in a double-pulse test, gives $E_{on} \approx 90$ µJ and $E_{off} \approx 25$ µJ. The turn-on estimate is 28 % high, a typical agreement; the turn-off estimate is twice the measurement, for the reason in the status box. Use the model to see how energies scale with R_G and drive voltage, and use measured energies for the loss budget.

Datasheet switching energies and how to scale them

Datasheet E_{on}, E_{off} and E_{rr} are measured by the **double-pulse test** (IEC 60747-8 for MOSFETs, IEC 60747-9 for IGBTs): a first pulse ramps an inductor's current to the test value, a short off-time captures turn-off, and a second pulse captures turn-on at that current. They hold only for the stated voltage, current, temperature, gate resistor, drive voltages, fixture loop inductance and diode. Three scaling rules extend them:

- **Voltage:** roughly proportional to V_{dc} (exponent 1 to 1.4 for IGBTs).
- **Current:** roughly proportional to I for IGBTs; for fast unipolar devices E_{on} has a constant part (E_{oss}, $Q_{rr}V$) that does not shrink at light load.
- **Temperature:** IGBT and Si-diode energies rise 30–60 % from 25 °C to 125 °C; SiC MOSFET energies change by about 10 %.

The switching loss is then $P_{sw} = (E_{on} + E_{off}) f_{sw}$ for each switch at constant current; in an inverter, where the current varies sinusoidally, the energies are averaged over the output cycle (Chapter 31, §31.8).

29.4 Conduction losses

RMS for resistors, average plus RMS for thresholds

Conduction loss is the time average of $v\,i$ over a switching period. For a unipolar device, it depends only on the RMS current through it; for a threshold device, also on the average:

$$P_{cond,MOS} = I_{rms}^2\, R_{DS(on)}(T_j), \qquad P_{cond,IGBT} = V_{CE0}\, I_{avg} + r_{CE}\, I_{rms}^2$$

For a switch carrying a trapezoidal current of mean value I and peak-to-peak ripple ΔI for a fraction D of the period, $I_{avg} = DI$ and $I_{rms}^2 = D\left(I^2 + \Delta I^2/12\right)$; the ripple term is small at the usual ripple ratios (Chapter 30). Because the on-resistance depends on junction temperature, and junction temperature on the loss, conduction losses must be evaluated at the operating temperature: the electrothermal loop of Chapter 4 (§4.10), which for a combined conduction and switching loss is solved iteratively in Section 29.5.

Reverse conduction and dead time

In a bridge, the lower device conducts in the **third quadrant** (current from source to drain) while the upper device is off. A MOSFET whose channel is turned on in the third quadrant behaves as the same resistor, which is **synchronous rectification**: at 20 A a 5 mΩ channel drops 0.1 V where a Schottky diode would drop 0.5 V. But both devices must be off for a short **dead time** at each transition to avoid shoot-through (Chapter 31, §31.2), and during it the current flows through the body diode or its equivalent. That is cheap for a silicon MOSFET (0.9 V) and expensive for SiC (3–4 V, because of its wide bandgap) and GaN (2–3 V plus the magnitude of any negative off-bias, because the HEMT conducts in reverse by turning its own channel on).

Worked example — dead-time loss with a SiC body diode. A SiC half-bridge at 20 A uses 100 ns of dead time at each of two transitions per period, with a body-diode drop of 3.5 V. The dead-time loss per period is $20 \times 3.5 \times 2 \times 100\ \text{ns} = 14\ \mu\text{J}$: 0.7 W at 50 kHz, 1.4 W at 100 kHz, about 5 % of the device's total loss at 100 kHz. Cutting the dead time to 40 ns, which the fast turn-off of SiC allows, saves 60 % of it; an antiparallel SiC Schottky diode would also take the current, at the cost of its capacitance.

Combining the losses: four technologies compared

The listing combines conduction and switching losses for one switch of a hard-switched 400 V half-bridge carrying 20 A at $D = 0.5$, with the 125 °C values of Table 29.1, and finds the highest frequency at which each device stays within a 25 W thermal budget (Section 29.5 shows where such a budget comes from).

```
 # One switch of a hard-switched 400 V half-bridge leg, 20 A, D = 0.5, Tj = 125 C
V, I, D = 400.0, 20.0, 0.5
devices = {   # on-state model at 125 C, switching energies at 400 V / 20 A / 125 C
    "Si SJ MOSFET": dict(R=0.095, Eon=300e-6, Eoff=25e-6),
    "Si IGBT":      dict(V0=0.95, r=0.030, Eon=350e-6, Eoff=250e-6),
    "SiC MOSFET":   dict(R=0.081, Eon=90e-6, Eoff=25e-6),
    "GaN HEMT":     dict(R=0.090, Eon=35e-6, Eoff=8e-6),
}
P_max = 25.0                                # W per device the heatsink allows
print(f"{'device':13s} {'Pcond':>6s} {'Esw uJ':>7s} {'20 kHz':>7s} "
      f"{'50 kHz':>7s} {'100 kHz':>8s} {'f_max kHz':>9s}")
for name, d in devices.items():
    if "R" in d:                            # MOSFET/HEMT: I_rms^2 R, square wave
        p_c = D * I**2 * d["R"]
    else:                                   # IGBT: V0 I_avg + r I_rms^2
        p_c = D * (d["V0"]*I + d["r"]*I**2)
    e_sw = d["Eon"] + d["Eoff"]
    tot = [p_c + e_sw*f for f in (20e3, 50e3, 100e3)]
    f_max = (P_max - p_c) / e_sw
    print(f"{name:13s} {p_c:6.1f} {e_sw*1e6:7.0f} {tot[0]:7.1f} "
          f"{tot[1]:7.1f} {tot[2]:8.1f} {f_max/1e3:9.1f}")
```

```
 device         Pcond  Esw uJ  20 kHz  50 kHz  100 kHz f_max kHz
Si SJ MOSFET    19.0     325    25.5    35.2     51.5      18.5
Si IGBT         15.5     600    27.5    45.5     75.5      15.8
SiC MOSFET      16.2     115    18.5    21.9     27.7      76.5
GaN HEMT        18.0      43    18.9    20.1     22.3     162.8
```

Conduction losses are similar, 15.5–19 W, because the devices were chosen for similar on-state performance; the IGBT's threshold model gives the lowest figure at this current. Switching energies differ by a factor of 14. At 20 kHz all four lie within 9 W of one another and the IGBT, cheapest per ampere, is a reasonable choice; at 100 kHz the IGBT would dissipate 75 W and the SiC and GaN devices 22–28 W. This is the quantitative content of the prototype story in the opening: moving from IGBTs at 20 kHz to SiC at 100 kHz is possible, but only for a device whose switching energy is about a fifth of the IGBT's. Figure 29.2 plots the totals against frequency.

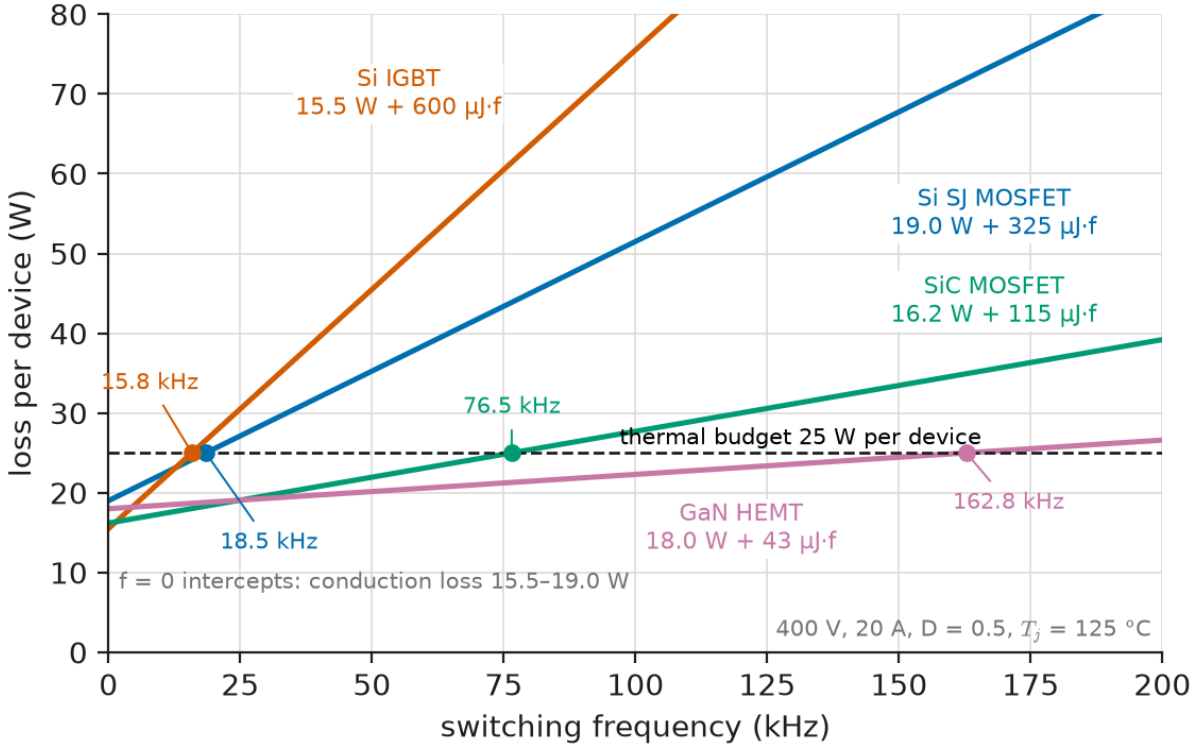


Figure 29.2 — Total loss of one switch in a hard-switched 400 V, 20 A half-bridge versus switching frequency for four 650 V technologies at 125 C.

29.5 Thermal constraints

The steady-state path

Losses become a junction temperature through the thermal-resistance network of *Engineering Physics* §14.7: junction to case $R_{th,jc}$ (from the datasheet), case to heatsink $R_{th,ch}$ (the interface), heatsink to ambient $R_{th,ha}$. Chapter 1 (§1.7) applies it to a single device and Chapter 4 (§4.10) to a conducting MOSFET. In a converter three practical points dominate.

Table 29.2 — Representative thermal resistances for discrete power packages (typical values).

Element	Range	Notes
$R_{th,jc}$, TO-247, 650 V SiC or SJ MOSFET	0.4–1.0 K/W	Lower for larger dies
$R_{th,jc}$, top-cooled GaN (e.g. 7 × 9 mm)	0.2–0.5 K/W	Heat leaves through the top pad
$R_{th,ch}$, thermal grease, non-isolated	0.1–0.2 K/W	Heatsink at switch-node potential
$R_{th,ch}$, silicone-elastomer insulating pad	0.4–1.0 K/W	Isolation to several kV; most common
$R_{th,ch}$, AlN ceramic with grease	0.2–0.3 K/W	Isolated, low resistance, brittle
$R_{th,ha}$, natural convection, 100 mm extrusion	1–3 K/W	Orientation-dependent
$R_{th,ha}$, forced air, small fan	0.2–0.8 K/W	Fan life limits reliability
$R_{th,ha}$, liquid cold plate	0.02–0.1 K/W	EV traction, server and telecom rectifiers

First, the **interface** often equals or exceeds $R_{th,jc}$: an insulating pad under a TO-247 can double the junction-to-heatsink resistance, which is why designers accept a heatsink at switch-node potential or pay for ceramic isolation. Second, **several devices share one heatsink**, so the heatsink temperature rises with the sum of their losses while each junction sits above it by its own loss times its own $R_{th,jc} + R_{th,ch}$. Third, the design target is not the absolute maximum junction temperature (150 °C for most silicon, 175 °C for many SiC and Si trench devices) but a working value 25–50 K lower, which keeps margin for overload, ambient extremes and lifetime.

Combined conduction and switching loss in a loop

Chapter 4 solved the electrothermal equilibrium of a purely conducting MOSFET in closed form. With switching losses that also depend on temperature, and two devices on one heatsink, a fixed-point iteration is simpler and shows the loop gain directly.

Worked example — junction temperature of a SiC half-bridge leg. Both switches of the 400 V, 20 A, 50 kHz leg are SiC MOSFETs ($R_{DS(on)} = 60$ mΩ at 25 °C, rising 0.35 %/K; $E_{on} + E_{off} = 105$ µJ at 25 °C, rising 0.1 %/K) with $R_{th,jc} = 0.8$ K/W and an insulating pad of 0.5 K/W, sharing a forced-air heatsink of 0.9 K/W in 50 °C air.

```
 # Electrothermal fixed point for a SiC half bridge on a shared heatsink
R25, alpha = 0.060, 0.0035        # ohm, 1/K  (1.35x at 125 C)
I, D, f = 20.0, 0.5, 50e3          # A, duty, Hz
Esw25, beta = 105e-6, 0.001        # J at 25 C, 1/K (Esw rises ~10 %/100 K)
Rjc, Rch, Rsa, Ta = 0.8, 0.5, 0.9, 50.0   # K/W, K/W, K/W (shared), C

def losses(Tj):
    pc = I**2 * R25 * (1 + alpha*(Tj - 25)) * D
    ps = Esw25 * (1 + beta*(Tj - 25)) * f
    return pc, ps

Tj = Ta
for it in range(50):
    pc, ps = losses(Tj)
    p = pc + ps
    Ths = Ta + 2*p*Rsa                  # two devices share the sink
    Tj_new = Ths + p*(Rjc + Rch)
    if abs(Tj_new - Tj) < 0.01:
        break
    Tj = Tj_new
print(f"iterations {it}: Pcond {pc:.1f} W, Psw {ps:.1f} W, "
      f"Ths {Ths:.1f} C, Tj {Tj_new:.1f} C")
# loop gain of the electrothermal feedback: dP/dTj * Rth_total
dP = I**2*R25*alpha*D + Esw25*beta*f
print(f"dP/dTj {dP:.3f} W/K, loop gain {dP*(Rjc+Rch+2*Rsa):.2f}")
```

```
 iterations 5: Pcond 15.9 W, Psw 5.7 W, Ths 88.9 C, Tj 116.9 C
dP/dTj 0.047 W/K, loop gain 0.15
```

Each device dissipates 21.6 W; the heatsink runs at 88.9 °C and the junctions at 116.9 °C, inside a 125 °C working target and 58 K below the 175 °C rating. The 25 °C losses would have been 12 W of conduction and 5.3 W of switching, 17.3 W in all, predicting a 104 °C junction: 13 K optimistic, the third prototype of the opening. The loop gain of 0.15 means the temperature feedback multiplies any loss disturbance by $1/(1 - 0.15) = 1.18$; a silicon superjunction device, whose resistance doubles over the same range, on a poorer heatsink, can approach a loop gain of 0.5, where small errors in the thermal path produce large temperature errors.

Pulses and transient thermal impedance

A junction does not reach steady state instantly: the die, solder, baseplate and heatsink are thermal capacitances (the RC network of *Engineering Physics* §14.7.3). Datasheets give the **transient thermal impedance** $Z_{th,jc}(t)$, the junction temperature rise per watt of a power step after time t, usually as a Foster network:

$$Z_{th,jc}(t) = \sum_i r_i \left(1 - e^{-t/\tau_i}\right)$$

For the SiC device above, a typical fit is $r_i = (0.05, 0.20, 0.35, 0.20)$ K/W with $\tau_i = (10\ \mu\text{s}, 0.2\ \text{ms}, 3\ \text{ms}, 30\ \text{ms})$, which sums to the steady 0.8 K/W. Evaluated, $Z_{th,jc}(1\ \text{ms}) = 0.35$ K/W and $Z_{th,jc}(10\ \text{ms}) = 0.64$ K/W.

Worked example — a 10 ms overload. A locked-rotor motor start triples the current for 10 ms. Conduction loss rises ninefold (15.9 to 143 W) and switching loss threefold (5.7 to 17 W), adding about 139 W to one device's loss. The heatsink barely moves in 10 ms, so the junction rises above its steady 116.9 °C by about $139 \times 0.64 = 89$ K, to about 206 °C: far past the 175 °C limit, and more as $R_{DS(on)}$ climbs during the pulse. Limiting the start current to twice rated adds only 53 W (conduction quadruples, switching doubles), a rise of $53 \times 0.64 = 34$ K to about 151 °C; keeping the triple current but cutting its duration to 1 ms ($139 \times 0.35 = 49$ K, to about 166 °C) also stays inside. Single-pulse Z_{th} is also what the SOA chart's pulse lines (Chapter 4, §4.10) are built from.

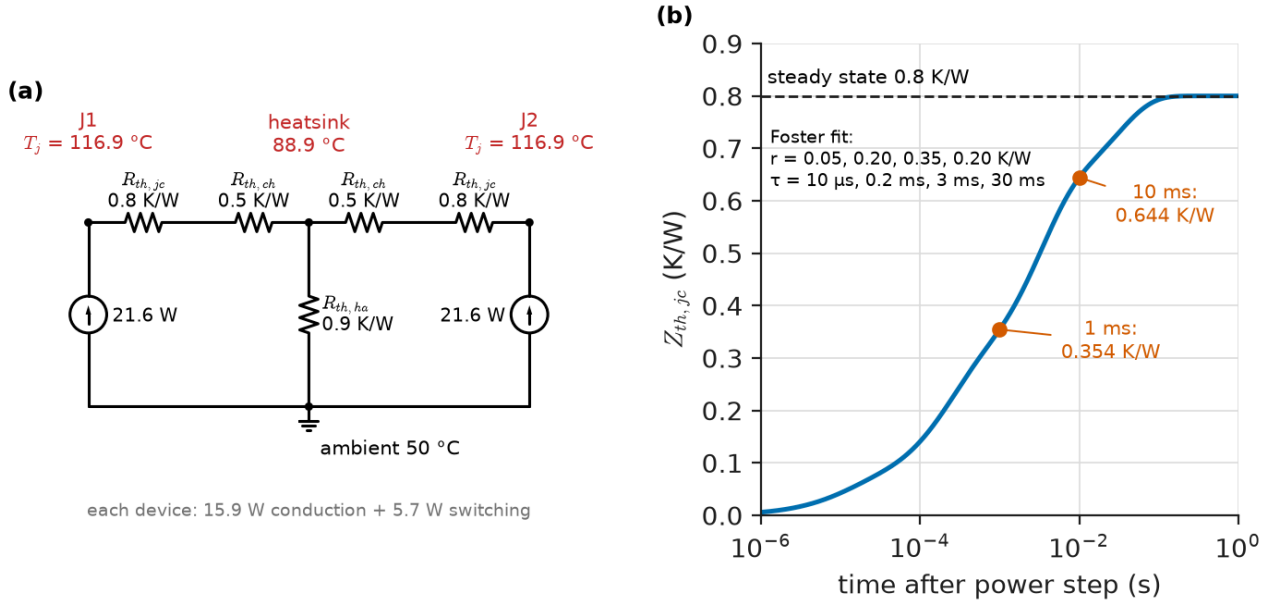


Figure 29.3 — (a) Thermal network of the SiC half-bridge leg on a shared heatsink with its solved temperatures; (b) transient thermal impedance of one device from its Foster fit.

Figure 29.3 shows the steady network of the worked example and the transient impedance used for the overload.

> **Status of the principle: Foster-network transient thermal model**
> *Approximate.* A sum of exponentials fitted to a measured junction-temperature step response; it predicts the junction temperature for any loss waveform by superposition, but its internal nodes have no physical meaning, it cannot be cascaded with a heatsink model without conversion to a Cauer network, and it assumes temperature-independent material properties.

Repeated temperature swings fatigue bond wires and solder; the resulting power-cycling lifetime, which sizes traction and wind converters, is treated in Chapter 31 (§31.9).

29.6 Gate driving

What a gate driver does

A gate driver translates a logic-level PWM signal from a controller (Chapter 17 for the timer, Chapter 28 for the firmware) into the voltage and current the power switch's gate needs, referenced to the switch's own source or emitter, which in a bridge may swing by hundreds of volts at tens of volts per nanosecond.

Table 29.3 — Gate-drive requirements by technology (typical values).

	Si MOSFET	Si IGBT	SiC MOSFET	GaN e-mode HEMT
Recommended on-voltage	10–12 V (logic level 4.5–5 V)	15 V	15–18 V	5–6 V
Off-voltage	0 V	0 V or −5 to −8 V	−3 to −5 V	0 V or −2 to −3 V
Absolute gate limits	±20 V	±20 V	+22/−8 V typical	+7/−10 V typical, device-dependent
Threshold at 125 °C	2–3 V	5–6 V	2–3 V	1.2–1.5 V
Typical edge rate	5–30 V/ns	2–10 V/ns	20–100 V/ns	50–150 V/ns
Required driver CMTI	25–50 V/ns	25–50 V/ns	at least 100 V/ns	at least 100–200 V/ns

The driver's main numbers follow directly from the gate charge:

$$P_G = Q_g\,\Delta V_{GS}\,f_{sw}, \qquad I_{G,pk} \approx \frac{\Delta V_{GS}}{R_{G,total}}$$

For the SiC device of Section 29.3 ($Q_g = 60$ nC, $\Delta V_{GS} = 19$ V, 50 kHz) the gate power is 57 mW and the peak current $19/9 = 2.1$ A; for the IGBT of Table 29.1 at 20 kHz with +15/−8 V, 95 nC × 23 × 20 kHz = 44 mW; for the GaN HEMT at 500 kHz, 6 nC × 6 × 500 kHz = 18 mW. The power is small, but it must be delivered in peaks of amperes from a supply referenced to a moving node.

Choosing the gate resistor

The external gate resistor is the main knob on switching speed (Section 29.3): a larger R_G slows the edges, lowers di/dt, dv/dt, overshoot and EMI, and raises switching loss roughly in proportion. Separate turn-on and turn-off resistors, steered by a diode or by a driver with split outputs, set the two edges independently; it is common to turn on slowly (limiting diode-recovery current and EMI) and turn off fast (limiting loss). Two layout details matter as much as the value. A **Kelvin source** pin (TO-247-4, D2PAK-7, most GaN packages) separates the gate return from the power current, removing the common-source inductance that would otherwise subtract $L_S\,di/dt$ from the gate voltage and slow the switch. And the gate loop itself must be small, because its inductance with C_{iss} forms a resonator that rings unless R_G damps it.

Miller-induced turn-on

When the upper switch of a leg turns on, the switch node rises at dv/dt, and the lower (off) device's gate–drain capacitance injects a current $C_{gd}\,dv/dt$ into its gate, which the driver must sink through its off-path resistance.

Worked example — the first prototype's shoot-through. The lower SiC MOSFET has an effective C_{gd} of about 10 pF over the transition, the upper switches at 50 V/ns, and the lower gate is held at 0 V through 9 Ω.

- Injected current: 10 pF × 50 V/ns = 0.5 A.

- Gate voltage bump: $0.5 \times 9 = 4.5$ V, well above the 2–3 V threshold at temperature. The lower device turns on partially while the upper turns on fully: shoot-through.
- Remedies: a negative off-voltage of −4 V puts the bump at +0.5 V, below threshold; an **active Miller clamp**, a transistor in the driver that shorts gate to source through about 0.5 Ω once the gate is below 2 V, reduces the bump to $0.5 \times 0.5 = 0.25$ V; a small gate–source capacitor helps at the cost of drive power.

GaN HEMTs, with thresholds near 1.3 V and a gate limit of about +7 V, are even more sensitive, which is why GaN drivers sit millimetres from the device or are integrated into the same package.

Isolation

The high-side driver of a bridge must be referenced to the switch node. Non-isolated level-shifting half-bridge driver ICs, powered by a bootstrap, serve cost-sensitive designs up to about 600–1200 V. Isolated drivers carry the signal across a barrier: optocouplers (IEC 60747-5-5) or, in most new designs, capacitive and magnetic digital isolators (IEC 60747-17), whose 30–100 ns propagation delays are shorter, better matched and more stable with age. Pulse transformers survive in thyristor firing.

The critical isolated-driver number is **common-mode transient immunity** (CMTI), the switch-node dv/dt the barrier can withstand without corrupting the output; 100 V/ns or more is needed for SiC and GaN. The isolated supply that powers the driver matters just as much: every picofarad of interwinding capacitance in its transformer carries a common-mode current $C\,dv/dt$ back into the controller ground, so a 10 pF supply at 100 V/ns injects 1 A spikes. Drivers for SiC and GaN use supplies with under 2–5 pF of barrier capacitance.

The bootstrap supply

The cheapest high-side supply is a **bootstrap** (Figure 29.4): a capacitor C_{BS} between the high-side driver's supply pin and the switch node, charged through a diode from the low-side 10–15 V rail whenever the low-side switch is on and the switch node is near ground. While the high-side switch is on, the capacitor alone powers its driver and holds its gate up.

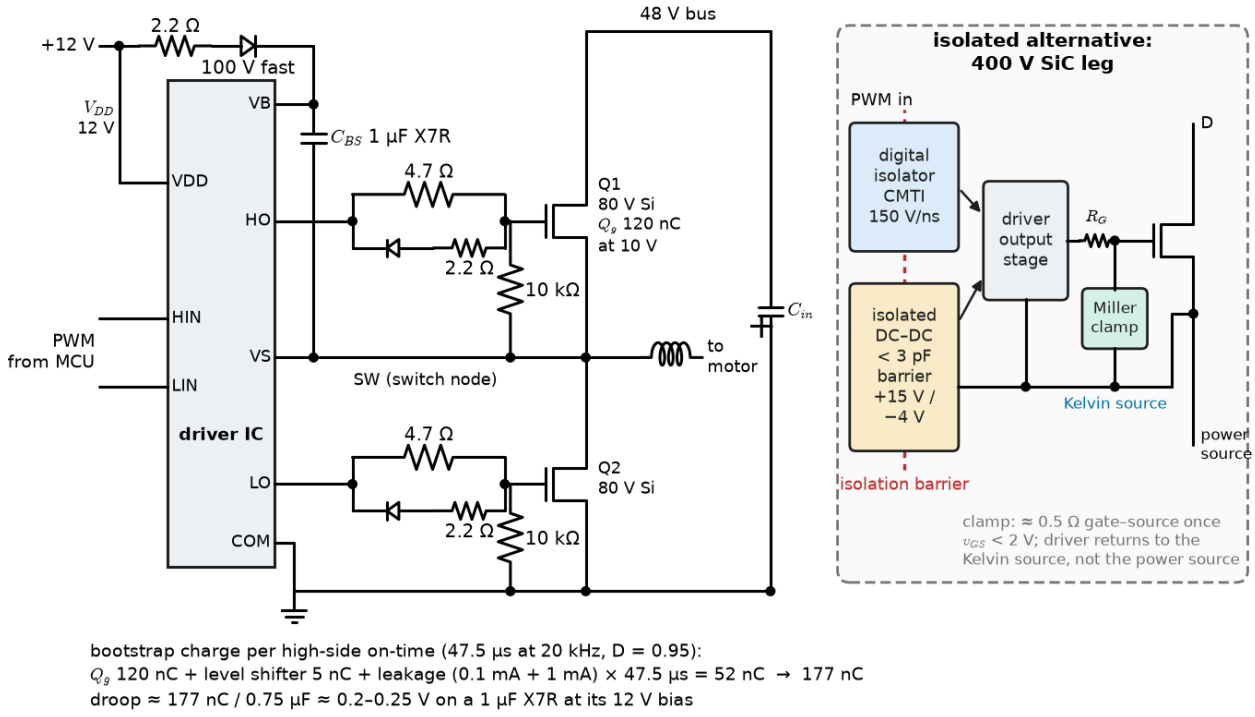


Figure 29.4 — Half-bridge gate drive: low-side driver, bootstrap high-side driver and an isolated alternative, with gate resistors, Miller clamp and Kelvin source.

Worked example — sizing a bootstrap capacitor. A 48 V motor-drive half-bridge uses 80 V silicon MOSFETs with $Q_g = 120$ nC at 10 V, switching at 20 kHz with a maximum duty of 0.95, so the longest high-side on-time is $0.95/20\ \text{kHz} = 47.5\ \mu\text{s}$. The high-side driver draws 100 µA quiescent, its level shifter consumes 5 nC per cycle, and a 10 kΩ gate–source resistor draws 1 mA at 10 V.

- Charge drawn per high-side on-time: $Q = 120 + 5 + (0.1 + 1.0)\ \text{mA} \times 47.5\ \mu\text{s} = 120 + 5 + 52 = 177$ nC.
- For at most 0.5 V of droop, $C_{BS} \geq 177\ \text{nC}/0.5\ \text{V} = 354$ nF. A 1 µF X7R capacitor rated 25 V, which still offers 0.7–0.8 µF at 12 V bias, droops by about 0.2–0.25 V.
- The 1 mA of the gate–source resistor costs nearly as much as the gate charge itself; at a low switching frequency or a long on-time it dominates. Raising that resistor to 47 kΩ, or choosing a driver with a charge pump, is the fix for long on-times.

The bootstrap has two structural limits. It cannot hold the high-side switch on indefinitely, so 100 % duty needs a charge pump or an isolated supply; and the low-side switch must be on long enough each cycle to recharge the capacitor, which with a 2.2 Ω charging resistor and 1 µF ($\tau = 2.2$ µs) means a minimum low-side time of a few microseconds. In SiC half-bridges the bootstrap can also overcharge, because the negative switch-node excursion during dead time adds to the charging voltage; a Zener clamp on V_{BS} protects the gate.

29.7 Snubbers

Parasitic inductance and ringing

Every commutation loop (the DC-link capacitor, both switches and the connections between them) has a stray inductance L_σ: 2–5 nH for a well-laid-out GaN half-bridge on a multilayer board, 10–40 nH for TO-247 devices, 15–30 nH inside a modern power module plus its busbar. At turn-off, the current through L_σ collapses at di/dt, and the switch sees an overshoot

$$\Delta V \approx L_\sigma \frac{di}{dt}$$

on top of V_{dc}. The overshoot is followed by ringing of L_σ with the switch's output capacitance: a lightly damped series RLC (the second-order circuit of *Circuits, Signals and Communications*, Ch. 7) whose frequency $f_r = 1/(2\pi\sqrt{L_\sigma C_{oss}})$ lies at 30–300 MHz and radiates efficiently. Two examples set the scale: a SiC MOSFET turning off 20 A in 5 ns through 32 nH gives $32\ \text{nH} \times 4\ \text{A/ns} = 128$ V, and a 1200 V IGBT module turning off 400 A at 5 kA/µs through 20 nH overshoots by 100 V on an 800 V bus. The first estimate is optimistic: a current fall shorter than half a ring period (5 ns at 100 MHz) also excites the $L_\sigma C_{oss}$ resonance, and the peak approaches twice $L_\sigma\, di/dt$. The SiC leg of Figure 29.5, its current falling in 4.5 ns, takes the 400 V bus to about 610 V on a 650 V device.

The first remedy is layout: decoupling capacitors (0.1–1 µF per leg) millimetres from the switches, the loop routed as overlapping planes on adjacent layers so its field cancels, and low-inductance packages. Halving L_σ halves the overshoot at no cost in loss; only then is a snubber added.

The RC damping snubber

A series RC across the switch (or across the DC link close to the devices) damps the ringing by placing a resistor equal to the loop's characteristic impedance across it. The parasitic L_σ and C_{oss} are rarely known accurately, so they are measured:

1. Measure the ringing frequency f_1 at turn-off.
2. Add a known capacitor C_a across the switch and measure the new frequency f_2. Since $f \propto 1/\sqrt{C}$, $C_p = C_a / \left[(f_1/f_2)^2 - 1\right]$ and $L_p = 1/\left[(2\pi f_1)^2 C_p\right]$.
3. Choose $R_s = \sqrt{L_p/C_p}$, the characteristic impedance, and C_s three to four times C_p so that the capacitor is a short at the ringing frequency while R_s does the damping.

4. Check the snubber loss $P_s = C_s V_{dc}^2 f_{sw}$: the capacitor is charged and discharged each cycle, and the energy $\tfrac{1}{2} C_s V^2$ is lost in R_s twice.

Worked example — damping the second prototype. The SiC leg rings at $f_1 = 100$ MHz. Adding $C_a = 240$ pF lowers it to $f_2 = 50$ MHz.

- $C_p = 240/(2^2 - 1) = 80$ pF; $L_p = 1/[(2\pi \times 10^8)^2 \times 80\ \text{pF}] = 31.7$ nH.
- $R_s = \sqrt{31.7\ \text{nH}/80\ \text{pF}} = 19.9\ \Omega$; take a 20 Ω low-inductance thick-film resistor with $C_s = 240$ pF (three times C_p), C0G dielectric, 1 kV.
- Loss: $240\ \text{pF} \times 400^2 \times 50\ \text{kHz} = 1.9$ W, dissipated in the resistor, and it rises to 3.8 W if the converter later runs at 100 kHz.

The snubber reduces the ringing to one or two cycles and cuts the radiated emissions around 100 MHz by 10–20 dB; it trims the first overshoot peak back towards the $L_\sigma\, di/dt$ estimate (from about 610 V to about 505 V in Figure 29.5) but cannot remove it, which only a smaller L_σ or a slower di/dt does. A 2 W loss in a 10 kW converter is acceptable; the same snubber on every switch of a 1 MHz GaN converter is not, which is why high-frequency designs rely on layout almost entirely.

> **Status of the principle: Lumped RC snubber design**
> *Approximate.* Treats the ringing loop as a single series $L_\sigma C_{oss}$ resonator with constant values; C_{oss} is strongly voltage-dependent, so the measurement is made at the operating bus voltage, and loops with several resonances (module, busbar, capacitor bank) may need snubbers at more than one point.

The RCD turn-off snubber and clamps

An RCD snubber (capacitor in series with a diode across the switch, a resistor across the diode) slows the voltage rise at turn-off: the load current charges C_s through the diode while the switch current falls, so voltage and current overlap less, and the energy moves from the switch to the resistor. For an IGBT with a 200 ns current fall at 20 A and 400 V, a capacitor that lets the voltage reach the bus only as the current fall ends needs $C_s \approx I\, t_f/(2V) = 5$ nF (it absorbs the charge $I t_f/2 = 2\ \mu$C); it must discharge through R_s during the minimum on-time (2 µs gives $R_s < 2\ \mu\text{s}/(3 \times 5\ \text{nF}) = 133\ \Omega$) and costs $\tfrac{1}{2} C_s V^2 f = 8$ W at 20 kHz. That is often as much as it saves, so turn-off snubbers survive mainly in thyristor circuits and high-power IGBT stacks. Two relatives remain essential: the **RCD clamp** that absorbs a flyback transformer's leakage energy (Chapter 30), and the 0.1–1 µF film **DC-link snubber capacitor** mounted on IGBT module terminals to shorten the commutation loop.

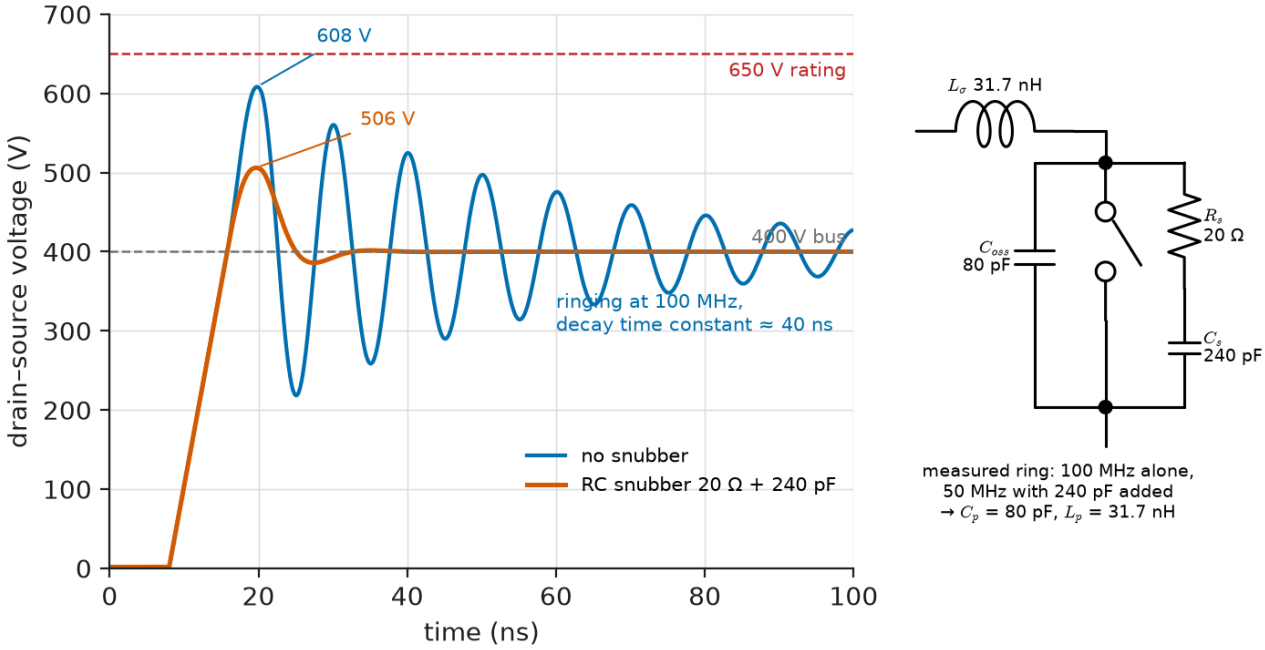


Figure 29.5 — Turn-off drain voltage of the SiC leg without and with the RC snubber, and the snubber circuit.

29.8 Protection

Chapter 31 (§31.9) covers protection at converter level: precharge, fuses, surge arresters and earth faults. This section covers the switch itself, which in a fault has microseconds to live.

Short circuit and its two forms

A **hard switching fault** (type I) occurs when a switch turns on into an existing short, for example a shorted motor cable; a **fault under load** (type II) when a short appears while the switch is already on. In both, the current rises until the device leaves its low-resistance region and limits it by its saturation characteristic, typically 4–10 times rated current, at the full bus voltage. The dissipated power is enormous: a 650 V SiC MOSFET saturating at 200 A on a 400 V bus dissipates 80 kW, and in 2 µs absorbs 80 kW × 2 μs = 160 mJ in a die of about 10 mm². The device survives only for its rated **short-circuit withstand time** (Table 29.1): 5–10 µs for IGBTs designed for drives, 2–3 µs for most SiC MOSFETs (whose smaller dies have less thermal mass), under 1 µs for GaN, at specified bus voltage, gate voltage and starting temperature.

Detection: desaturation and current sensing

Desaturation detection (Figure 29.6) senses the collector or drain voltage through a high-voltage diode while the device is commanded on. In normal conduction the voltage is 1–3 V; in a short circuit the device leaves saturation and its voltage rises toward the bus. A comparator in the driver trips when the sensed voltage exceeds a threshold (7–9 V for IGBTs, 4–6 V for SiC), after a **blanking time** that masks the voltage fall at turn-on: 1.5–3 µs for IGBTs, 0.3–1 µs for SiC. Total response, from fault to turn-off command, must fit inside the withstand time with margin. For GaN, whose withstand time is too short for desaturation sensing, current is measured directly with a shunt or a sense-FET and compared in tens of nanoseconds.

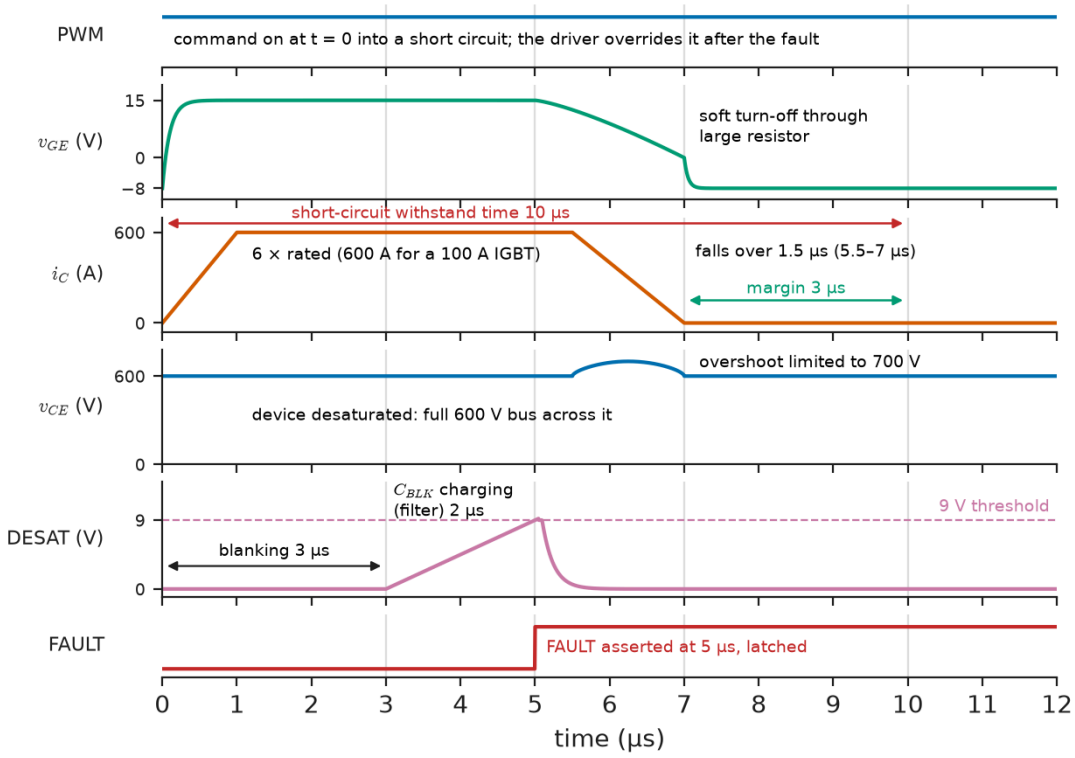


Figure 29.6 — Desaturation detection and soft turn-off of an IGBT during a hard switching fault.

Turning off a fault safely

Turning off a fault current of several hundred amperes at normal speed would produce $L_\sigma\, di/dt$ overshoot several times the normal value: 200 A collapsing in 50 ns through 30 nH adds 120 V. Drivers therefore use **soft turn-off**, discharging the gate through a larger resistor or a weak current source so that the fault current falls over 1–2 µs, or **two-level turn-off**, which first drops the gate to an intermediate voltage to reduce the saturation current before turning off. A further safeguard is **active clamping**: a chain of TVS diodes from collector to gate that turns the IGBT partially back on if the collector voltage exceeds a set level, so the device absorbs the excess energy in its linear region instead of avalanching.

> **Status of the principle: Short-circuit withstand time**
> *Empirical.* A rating found by destructive testing at stated bus voltage, gate voltage and starting junction temperature; it falls with higher bus voltage, higher gate drive and higher starting temperature, and repeated short circuits degrade the device, so the rating is a single-event limit, not a repetitive one.

The other switch-level protections

- **Undervoltage lockout (UVLO).** A gate driven at 9 V instead of 15 V leaves an IGBT or SiC MOSFET partly on, with several times its conduction loss; drivers inhibit switching until each supply exceeds 11–13 V.
- **Interlock** logic that refuses to turn on both switches of a leg, backing up the PWM dead time.
- **Gate clamps** (Zener or TVS), critical for GaN; **overtemperature** sensing by module NTC or on-die diode, with derating before tripping; and **overvoltage** prevention by clamps and slower edges, since avalanche ratings (Chapter 4, §4.10) cover only rare unclamped events.

29.9 Switching-frequency trade-offs

What rises and what falls with frequency

The switching frequency is the master design variable of a converter. Raising it:

- **Raises switching loss** in proportion, $P_{sw} = (E_{on} + E_{off}) f_{sw}$, plus gate-drive and snubber losses.
- **Shrinks the passive components.** For a given current ripple the inductance falls as $1/f_{sw}$, and so does the output capacitance for a given voltage ripple (Chapter 30); the inductor's stored energy, which sets its volume, falls with it.
- **Raises core and winding losses per unit volume.** Magnetic core loss per unit volume follows the empirical Steinmetz relation $P_v = k\, f^{\alpha} \hat{B}^{\beta}$ with $\alpha \approx 1.2$–1.6 and $\beta \approx 2.5$–3 for power ferrites, and AC winding resistance rises with skin and proximity effects; the designer reduces flux swing as frequency rises, so magnetics shrink less than $1/f$.
- **Raises the achievable control bandwidth**, since loop crossover is usually limited to a tenth to a fifth of f_{sw} (Chapter 30).
- **Shortens the minimum duty cycle** that can be resolved: a 48 V to 1 V step-down at 1 MHz needs an on-time of $1/(48 \times 1\ \text{MHz}) = 20.8$ ns, below the minimum on-time of most silicon controllers, which is one reason such conversions use two stages or GaN.

> **Status of the principle: Steinmetz core-loss relation**
> *Empirical.* A power-law fit of measured core loss per unit volume to frequency and peak flux density for sinusoidal excitation, valid over the fitted range for one material at one temperature; for the triangular flux of converter inductors, modified forms (the improved generalised Steinmetz equation) or the manufacturer's loss curves are used.

Constraints from outside the converter

Frequency is also chosen to place harmonics where they do least harm:

- **Audible noise.** Above about 20 kHz for anything near people; HVAC and appliance drives run at 8–16 kHz with randomised PWM or above 20 kHz.
- **Conducted-emission bands.** CISPR 11 and CISPR 32 limits start at 150 kHz, so a PFC stage at 65–70 kHz has its fundamental and second harmonic below the band and a smaller filter.
- **Automotive radio bands.** CISPR 25 protects the AM band (530–1,800 kHz), so automotive point-of-load converters run either below about 400 kHz or above 1.8 MHz, typically 2.1–2.2 MHz.

Table 29.4 — Typical hard-switched frequencies by application (representative values).

Application	Power	Devices	Switching frequency
Industrial drive inverter	1–500 kW	Si IGBT modules	2–16 kHz
EV traction inverter	50–300 kW	Si IGBT or SiC modules	8–30 kHz
PV string inverter	5–100 kW	SiC, Si IGBT	16–50 kHz
Server PSU PFC stage	1–3 kW	SJ MOSFET + SiC diode, GaN	65–140 kHz
USB-C laptop adapter	45–140 W	GaN	100 kHz–1 MHz
Point-of-load buck (12 V to 1 V)	10–100 W per phase	Integrated Si power stages	0.4–1.5 MHz

Worked example — the highest frequency each device can sustain. With a heatsink that allows 25 W per device (the budget used in Section 29.4), the maximum frequency is $f_{max} = (P_{max} - P_{cond})/(E_{on} + E_{off})$: 15.8 kHz for the IGBT, 18.5 kHz for the SJ MOSFET, 76.5 kHz for SiC and 163 kHz for GaN (the listing in Section 29.4). If the filter

inductor of the 10 kW inverter is 1 mH at 20 kHz for a given ripple, it can be 200 µH at 100 kHz, about a fifth of the energy storage and roughly a third of the volume once core-loss limits on flux swing are included. SiC makes 100 kHz possible only with a heatsink allowing $16.2 + 115\ \mu\text{J} \times 100\ \text{kHz} = 27.7$ W per device, slightly larger than the 25 W budget; GaN does it at 22.3 W. The choice is therefore made on total system cost and volume, with switching losses, heatsink and magnetics traded against one another, and the optimum frequency rises as switching energy falls.

What remains open

Open question	Where it is answered
How are buck, boost and buck–boost converters built from the commutation cell, and how are their inductors, capacitors and control loops designed?	Chapter 30
How are rectifier and inverter losses averaged over an AC cycle, and how are power cycling, precharge and fuses handled?	Chapter 31
How do motor drives, chargers and grid converters use these switches at system level?	Chapter 32
Why do IGBTs have tail currents, and why are SiC and GaN faster at the same voltage?	Chapter 5; *Materials Science and Engineering*, Ch. 33
How does a microcontroller generate the PWM signals, dead times and fault inputs that drive these gates?	Chapter 17, Chapter 28
How are conducted and radiated emissions of a converter limited at the installation level?	*Electrical Energy and Machines*, Ch. 25
How does heat conduct through layers and spread into a heatsink?	*Engineering Physics*, Ch. 14 §14.7

Next: Chapter 30 connects the commutation cell to an inductor and a capacitor and designs the three basic DC–DC converters, from their waveforms and component sizing to the small-signal model that their feedback loop is designed on.

Chapter 30 — DC–DC Converters

Part VII · Power Electronics

The 3.3 V rail of an industrial controller comes from a 12 V, 5 A synchronous buck converter switching at 500 kHz. On the bench it is 94.6 % efficient at full load and its ripple is a few millivolts. In the field three complaints arrive. In standby, with the board drawing 0.2 A, the converter wastes a third of a watt inside a sealed enclosure: its efficiency has fallen to 66 %. When the FPGA it feeds steps from 2.5 A to 5 A, the rail dips by more than 200 mV and the FPGA resets, because the ceramic output capacitors, 47 µF each on the label, keep only about two-thirds of that at 3.3 V of DC bias. And a sister board's boost converter, which makes 5 V from a lithium-ion cell, oscillates when the cell runs low, although it was stable on a full cell. Each complaint has a one-page explanation: fixed switching losses that do not scale with load, capacitance that depends on voltage, and a right-half-plane zero that moves with the operating point. This chapter designs the three basic converters with those explanations built in.

> **How do the buck, boost and buck–boost converters set their conversion ratios, what do their currents and voltages look like, and how are their inductors and capacitors sized for ripple, transients and efficiency?**
> **What changes at light load when the inductor current reaches zero, and what model of the converter does its feedback loop see?**

Concept	Engineering destination
Buck converter	Point-of-load supplies for processors and FPGAs, LED drivers, battery chargers, 48 V to 12 V buses
Boost converter	Power-factor correction, PV and battery interfaces, single-cell to 5 V, LED backlights
Buck–boost converter	Negative rails, USB Power Delivery, battery-powered systems whose input spans the output
Continuous and discontinuous conduction	Light-load efficiency modes, conversion-ratio changes, simpler dynamics
Converter waveforms	Component stresses, probing and debugging on the bench
Inductor and capacitor sizing	Saturation current, ripple ratio, DC-bias derating, RMS current ratings
Ripple	Output and input ripple specifications, input filters and conducted emissions
Efficiency	Loss budgets, thermal design, light-load modes
Feedback control interface	Averaged small-signal models, compensators, crossover and phase margin, digital control

Roadmap. The buck (30.1), boost (30.2) and buck–boost families (30.3) in continuous conduction → what changes in discontinuous conduction (30.4) → the waveforms that set component stresses (30.5) → inductor and capacitor sizing (30.6) and the ripple they leave (30.7) → the loss budget and efficiency across load (30.8) → the averaged model and the feedback loop (30.9).

> **Boundary.**
> *Engineering Physics*, Ch. 18 §18.8, states the ideal conversion ratios D, $1/(1-D)$ and $-D/(1-D)$ and the switching-frequency trade-off. The inductor's constitutive law and a first buck-ripple calculation are in *Circuits, Signals and Communications*, Ch. 5; the output filter as a second-order circuit in Ch. 7; Laplace, Bode and pole–zero methods in Ch. 11 and Ch. 13; stability margins, loop shaping and digital control in Ch. 34 and Ch. 35. Switch losses, gate drive and snubbers are from Chapter 29.

30.1 Buck converter

Operation and volt-second balance

The buck converter (Figure 30.1a) is the commutation cell of Chapter 29 with its switch node driving an inductor L into an output capacitor C and load. During the on-time DT_s the high-side switch connects the switch node to V_{in} and the inductor sees $V_{in} - V_o$; during the off-time $(1-D)T_s$ the low-side switch (or a freewheeling diode) connects it to ground and the inductor sees $-V_o$. In periodic steady state the inductor current returns to the same value at the end of each period, so the average inductor voltage over a period is zero (**volt-second balance**); likewise the average capacitor current is zero (**charge balance**). For the buck:

$$(V_{in} - V_o)\,DT_s - V_o(1-D)T_s = 0 \quad \Rightarrow \quad V_o = DV_{in}, \qquad I_{in} = DI_o$$

> **Status of the principle: Volt-second and charge balance**
> *Derived.* Follows from $v = L\,di/dt$ and $i = C\,dv/dt$ integrated over one period in periodic steady state; it is exact for any waveform, and the familiar ratios follow once the switches and passive components are taken as lossless and the ripple as small compared with the DC values.

The peak-to-peak inductor ripple follows from the on-interval slope:

$$\Delta I_L = \frac{(V_{in} - V_o)\,D}{f_s L} = \frac{V_o(1-D)}{f_s L}$$

Book 3 Ch. 5 evaluated this for a 10 µH inductor; Section 30.6 chooses L from it.

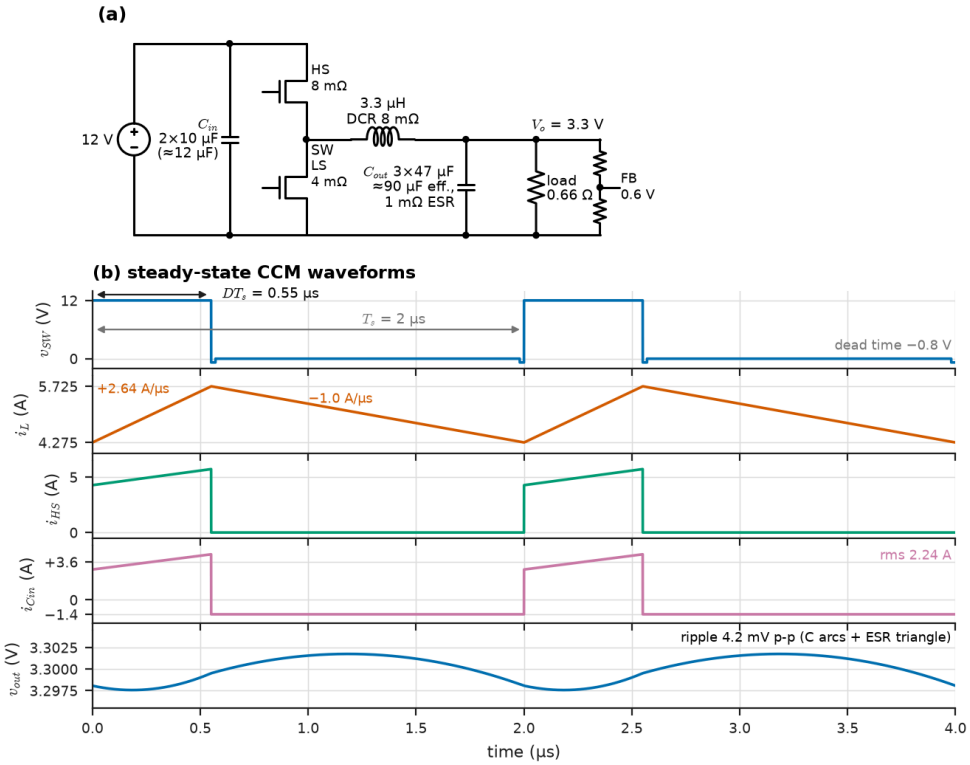


Figure 30.1 — (a) Synchronous buck converter, 12 V to 3.3 V at 5 A, 500 kHz; (b) its steady-state CCM waveforms over two switching periods.

Real switches and the duty cycle

With losses, the controller must supply a slightly larger duty cycle than V_o/V_{in}. Including the switch and inductor resistances, volt-second balance gives

$$D \approx \frac{V_o + I_o\,(R_{LS} + R_L)}{V_{in} - I_o\,(R_{HS} - R_{LS})}$$

and with a freewheeling diode of drop V_F instead of a low-side switch, $D \approx (V_o + V_F)/(V_{in} - V_{sw} + V_F)$. The difference between a diode and a synchronous switch is large at low output voltage. In the 12 V to 3.3 V, 5 A converter of the opening, a 0.45 V Schottky diode conducting for $1 - D = 72.5\ \%$ of the time dissipates $0.45 \times 5 \times 0.725 = 1.63$ W, against $0.725 \times 5^2 \times 5.2\ \text{m}\Omega = 0.09$ W for a 5 mΩ low-side MOSFET (5.2 mΩ hot). Below about 5 V of output, synchronous rectification is the norm; above about 24 V, or at a few hundred milliamperes, a diode is often cheaper and simpler.

Where bucks are used

Bucks supply nearly every processor, memory and FPGA rail: a 12 V to 1.0 V, 200 A server CPU regulator is a buck with 8–12 **interleaved** phases, each switching at 400–800 kHz with its PWM shifted by $360^\circ/N$, so that the phase ripples partly cancel at N times the switching frequency and the load current is shared. LED drivers are bucks regulating inductor current instead of output voltage; battery chargers are bucks with current and voltage loops (Chapter 32); and 48 V to 12 V intermediate-bus converters in data centres use multiphase bucks or their switched-capacitor hybrids.

30.2 Boost converter

Operation and conversion ratio

The boost (Figure 30.2a) puts the inductor on the input side. With the switch on, the inductor is connected across V_{in} and its current rises; with the switch off, the inductor current flows through the rectifier into the output at $V_o > V_{in}$, and it falls at a rate set by $V_o - V_{in}$. Volt-second balance gives

$$V_{in} D T_s + (V_{in} - V_o)(1 - D) T_s = 0 \quad \Rightarrow \quad V_o = \frac{V_{in}}{1 - D}, \qquad I_{in} = I_L = \frac{I_o}{1 - D}$$

The input current is continuous (it is the inductor current), which makes the boost the natural power-factor corrector (Chapter 31) and PV interface. The output current is pulsating: the output capacitor supplies the whole load during each on-time, so the boost needs more output capacitance than a buck of the same power (Section 30.7).

Real gain and its limit

The ideal gain diverges as $D \to 1$; real gain does not. With a total series resistance R_L in the inductor path (inductor DCR plus switch resistance) and load resistance R, volt-second balance with the resistive drop gives

$$\frac{V_o}{V_{in}} = \frac{1}{1 - D} \cdot \frac{1}{1 + \dfrac{R_L}{(1 - D)^2 R}}, \qquad \left(\frac{V_o}{V_{in}}\right)_{max} = \frac{1}{2}\sqrt{\frac{R}{R_L}} \ \text{at}\ 1 - D = \sqrt{\frac{R_L}{R}}$$

For a single-cell lithium-ion to 5 V, 2.4 A boost ($R = 2.08\ \Omega$) with $R_L = 20$ mΩ, the gain peaks at 5.1 at $D = 0.90$; beyond that, increasing D *reduces* the output, which reverses the sign of the feedback loop and latches the converter at its duty limit. The efficiency from this conduction loss alone, $1/[1 + R_L/((1 - D)^2 R)]$, is 97.4 % at $D = 0.4$ but falls to 50 % at the gain peak. Practical boosts are designed for gains of 1.2–4; higher step-up ratios use coupled inductors, transformers or cascaded stages.

Two further properties shape boost designs. There is always a path from input to output through the inductor and rectifier, so a boost cannot limit the current into a shorted output; battery-powered boosts add a disconnect switch,

which synchronous boost ICs with "true output disconnect" build into the rectifier MOSFET by switching its body connection. And the output responds to an increase in duty cycle by first *falling* (more time charging the inductor, less time delivering), a right-half-plane zero that limits the feedback loop (Section 30.9).

30.3 Buck–boost converter

The inverting buck–boost

When the input range spans the output voltage, or a negative rail is needed, the inductor is placed between the switch node and ground (Figure 30.2b). With the switch on, the inductor charges from V_{in}; with it off, the inductor discharges through the rectifier into the output capacitor, whose polarity is therefore reversed:

$$V_{in} D + V_o (1 - D) = 0 \quad \Rightarrow \quad V_o = -\frac{D}{1 - D} V_{in}, \qquad I_L = I_{in} + |I_o| = \frac{|I_o|}{1 - D}$$

The price of the flexibility is stress: the switch and rectifier block $V_{in} + |V_o|$, and the inductor carries the sum of input and output currents. Making −5 V at 1 A from 12 V needs $D = 5/17 = 0.294$, switches rated for 17 V plus overshoot, and an inductor carrying 1.42 A. Both input and output currents are pulsating, so both capacitors work hard.

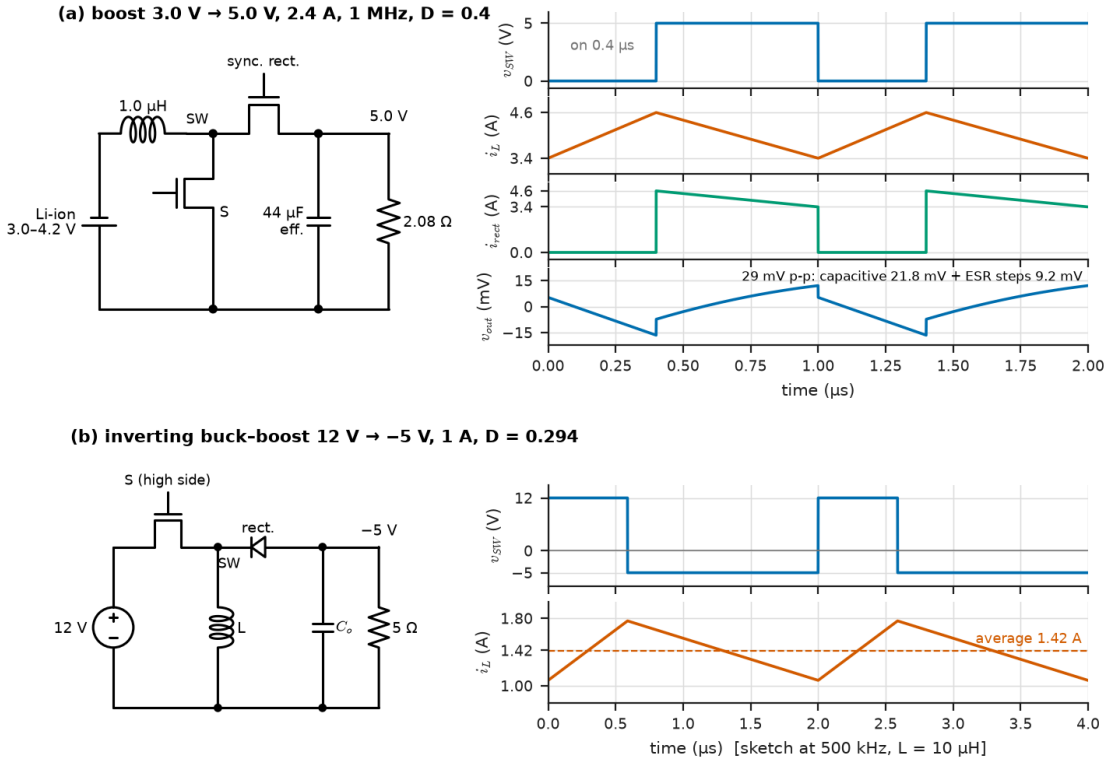


Figure 30.2 — (a) Synchronous boost converter, 3.0 V to 5.0 V at 2.4 A, 1 MHz, and its waveforms; (b) inverting buck-boost, 12 V to -5 V at 1 A, and its switch-node and inductor waveforms.

Non-inverting relatives

Several topologies give the buck–boost's flexibility without the inversion:

- **SEPIC** and **Ćuk** converters add a second inductor and a coupling capacitor; the SEPIC gives a positive output with continuous input current (automotive supplies that must work from 6 V cranking to 18 V), the Ćuk an inverted output with both currents continuous.
- The **four-switch (non-inverting) buck–boost** places a buck leg and a boost leg around one inductor and runs as a buck, a boost, or in a blended mode near $V_{in} = V_o$. It is the standard USB Power Delivery source and sink

converter, producing 5–20 V (up to 48 V with Extended Power Range) from a 2S to 4S battery at 95–97 % efficiency.

- The **flyback** replaces the inductor with a coupled inductor (a "transformer" that stores energy), adding galvanic isolation and a turns-ratio gain $V_o = \frac{N_s}{N_p}\frac{D}{1-D}V_{in}$. It dominates isolated supplies below about 100 W (phone chargers, auxiliary supplies); the energy in its leakage inductance is absorbed by the RCD clamp of Chapter 29 or recycled by an active clamp.

Table 30.1 — The three basic converters in continuous conduction (ideal components).

Property	Buck	Boost	Inverting buck–boost
Conversion ratio $M = V_o/V_{in}$	D	$1/(1-D)$	$-D/(1-D)$
Switch and rectifier voltage	V_{in}	V_o	$V_{in} + \lvert V_o\rvert$
Average inductor current	I_o	$I_o/(1-D)$	$\lvert I_o\rvert/(1-D)$
Input current	Pulsating	Continuous	Pulsating
Output current into the capacitor	Continuous (triangular ripple)	Pulsating	Pulsating
Inductor ripple ΔI_L	$V_o(1-D)/(f_s L)$	$V_{in}D/(f_s L)$	$V_{in}D/(f_s L)$
Critical $K_{crit}(D)$	$1-D$	$D(1-D)^2$	$(1-D)^2$
Small-signal control-to-output zero	LHP (capacitor ESR)	RHP and ESR	RHP and ESR

30.4 Continuous and discontinuous conduction

The boundary

With a diode rectifier, the inductor current cannot reverse. As the load falls, the average inductor current falls while the ripple ΔI_L stays the same, and when the valley reaches zero the converter enters **discontinuous conduction mode** (DCM): each period now has a third interval in which both switch and diode are off and the inductor current is zero. The boundary is where the average inductor current equals half the ripple. It is conveniently expressed with the dimensionless parameter

$$K = \frac{2L}{RT_s}$$

The converter is in continuous conduction (CCM) when $K > K_{crit}(D)$, with K_{crit} given in Table 30.1. For the 3.3 V buck ($L = 3.3$ µH, $f_s = 500$ kHz, $\Delta I_L = 1.45$ A) the boundary load is $I_{o,b} = \Delta I_L/2 = 0.725$ A; for the single-cell boost of Section 30.2 at $V_{in} = 3$ V ($L = 1$ µH, 1 MHz, $\Delta I_L = 1.2$ A), $K = 0.96$ against $K_{crit} = 0.144$ at full load, and the boundary is reached at $I_o = \tfrac{1}{2}\Delta I_L(1-D) = 0.36$ A.

The conversion ratio in DCM

In DCM the inductor current starts each period from zero, and the conversion ratio depends on load as well as duty cycle. Applying volt-second balance to the three intervals, and charge balance at the output, gives for the three converters:

$$M_{buck} = \frac{2}{1+\sqrt{1+4K/D^2}}, \qquad M_{boost} = \frac{1+\sqrt{1+4D^2/K}}{2}, \qquad M_{buck\text{–}boost} = -\frac{D}{\sqrt{K}}$$

> **Status of the principle: DCM conversion ratios**
> *Derived.* Follow from volt-second and charge balance with an inductor current that starts and ends each period at zero, for ideal switches and small output ripple; they hold only for $K < K_{crit}(D)$, and ringing of the inductor with the switch capacitances during the idle interval, ignored here, perturbs the light-load ratio slightly.

Worked example — the 3.3 V buck at 0.3 A with a diode rectifier. If the converter of the opening used diode emulation (the low-side switch turned off when its current reaches zero), then at 0.3 A the load resistance is $3.3/0.3 = 11\ \Omega$ and $K = 2 \times 3.3\ \mu\text{H} \times 500\ \text{kHz}/11 = 0.30$, below $K_{crit} = 1 - 0.275 = 0.725$: the converter is in DCM. Inverting the DCM buck ratio, the duty cycle needed for $M = 0.275$ is $D = M\sqrt{K/(1-M)} = 0.275\sqrt{0.30/0.725} = 0.177$, not 0.275. The feedback loop finds it automatically, but the plant it controls has changed (Section 30.9).

Figure 30.3 shows how the buck's conversion ratio departs from $M = D$ as K falls.

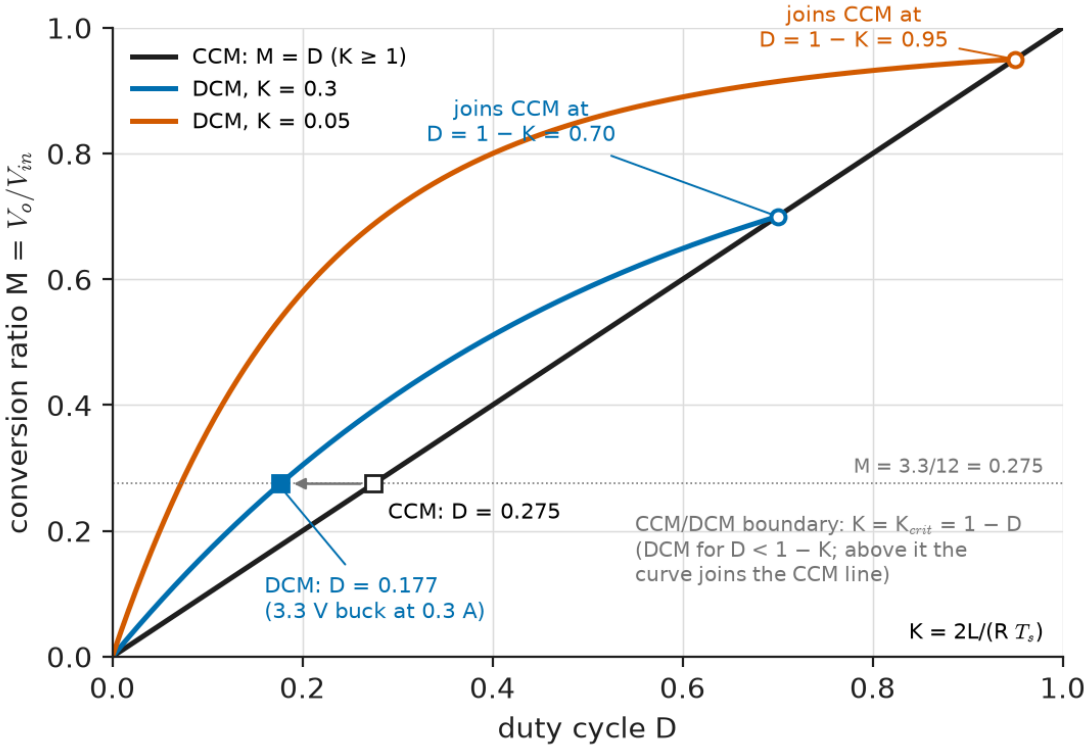


Figure 30.3 — Buck conversion ratio versus duty cycle in continuous and discontinuous conduction for several values of K = 2L/(R Ts).

Forced CCM, DCM and light-load modes

A synchronous converter can avoid DCM entirely: in **forced CCM** the low-side switch stays on even when the inductor current reverses, so $M = D$ at every load and the dynamics never change, but the ripple current circulates at zero load and the frequency-proportional losses remain (Section 30.8). Most controllers instead offer **diode emulation** (DCM) at light load, and below that **pulse-frequency modulation** or **burst mode**: the converter delivers a few pulses at full ripple amplitude and then idles, so that switching losses scale with load. The cost is variable-frequency ripple and noise, sometimes audible, which is why audio, RF and measurement equipment often specify forced CCM.

DCM is also used deliberately: boundary-conduction (critical-conduction) PFC stages up to a few hundred watts and flybacks in phone chargers run at the CCM/DCM boundary to turn their switches on at zero current, and a DCM converter has a simpler small-signal model because the inductor current no longer carries information from one period to the next.

30.5 Converter waveforms

The buck in CCM

Figure 30.1 shows the steady-state waveforms of the 12 V to 3.3 V, 5 A buck at 500 kHz, $T_s = 2$ µs, $DT_s = 0.55$ µs:

- **Switch node** v_{sw}: a rectangle between 12 V and about −0.05 V (the low-side MOSFET's $I\,R_{DS(on)}$ drop), with two short excursions to about −0.8 V during the dead times when the body diode conducts, and ringing at the edges from the commutation-loop inductance (Chapter 29, §29.7).
- **Inductor current** i_L: a triangle between 4.275 A and 5.725 A, rising at $(12 - 3.3)/3.3\ \mu\text{H} = 2.64$ A/µs and falling at $3.3/3.3\ \mu\text{H} = 1.0$ A/µs.
- **High-side switch current**: the rising part of i_L during DT_s, zero otherwise; it is the pulsating input current that the input capacitor must supply.
- **Input capacitor current**: $i_{HS} - I_{in}$, about +3.6 A during the on-time and −1.4 A during the off-time, with an RMS value of $I_o\sqrt{D(1-D)} = 2.23$ A.
- **Output capacitor current**: $i_L - I_o$, a triangle of ±0.725 A.
- **Output ripple**: the integral of the capacitor current (a train of parabolic arcs), plus the ESR drop (a triangle in phase with i_L), plus spikes at the switching edges.

The boost and buck–boost

In the boost the roles reverse: the input capacitor sees only the inductor ripple, while the output capacitor sees the rectifier's current pulses, from 0 to $I_{L,pk}$, every period. In the single-cell boost of Section 30.2 at $V_{in} = 3$ V, the inductor current runs between 3.4 A and 4.6 A, the switch node swings between 0 and 5 V, and the output capacitor must absorb pulses up to 4.6 A while the load draws a steady 2.4 A. In the buck–boost both capacitors see pulses, and the switch node swings from V_{in} to V_o (negative).

Measuring the waveforms

Converter waveforms are easy to mismeasure. A passive probe with its 15 cm ground lead forms a loop that picks up the switch node's dv/dt and the inductor's field, adding tens of millivolts of spurious "ripple"; output ripple is measured with the probe tip and barrel pressed directly across the output capacitor (or with a 50 Ω coaxial pickoff), usually with the 20 MHz bandwidth limit that ripple specifications assume. The switch node of a high-side switch is measured with a high-voltage differential or optically isolated probe, whose common-mode rejection at the edge frequencies matters more than its bandwidth. Inductor current is measured with a current probe on a short wire loop inserted for the purpose, which adds inductance to whatever loop it is placed in.

30.6 Inductor and capacitor sizing

The inductor

The inductor is chosen in four steps.

1. **Ripple ratio.** Choose the peak-to-peak ripple as a fraction $r = \Delta I_L / I_L$ of the average inductor current, typically 0.2–0.4. A smaller r means a larger, slower inductor with lower RMS and core losses; a larger r means a smaller inductor, faster transient response, more output ripple and an earlier entry into DCM. For a buck, size at maximum input voltage, where the ripple is largest; for a boost, the ripple $V_{in}D/(f_s L)$ peaks at $D = 0.5$.
2. **Inductance.** From the ripple equation, $L = V_o(1 - D)/(f_s\, r\, I_o)$ for a buck.
3. **Saturation current.** The core must not saturate at the highest peak current, which is set not by the normal peak $I_L + \Delta I_L/2$ but by the controller's current limit during start-up, overload and load steps. Ferrite cores saturate abruptly; powder-iron and metal-composite cores (molded power inductors) saturate gradually and tolerate brief overloads.

4. **Loss.** Copper loss $I_{rms}^2 R_{DC}$ plus core loss from the manufacturer's calculator (Chapter 29, §29.9), within the inductor's temperature-rise rating.

Worked example — the 3.3 V, 5 A buck. At $V_{in} = 12$ V, $D = 0.275$, $f_s = 500$ kHz and $r = 0.3$: $L = 3.3 \times 0.725/(500\ \text{kHz} \times 0.3 \times 5) = 3.19$ µH. With the standard value of 3.3 µH, $\Delta I_L = 1.45$ A, the peak is 5.725 A, and the RMS current $\sqrt{I_o^2 + \Delta I_L^2/12} = 5.02$ A, barely above the DC value. With a current limit at 7 A, choose a molded inductor rated at least 8 A saturation and 6 A for 40 K rise, with about 8 mΩ DCR: a 5 × 5 mm or 7 × 7 mm part.

The output capacitor

The output capacitor must satisfy three requirements, and the largest of the three sets its value.

- **Ripple** (Section 30.7): for a buck, $\Delta V_C \approx \Delta I_L/(8 f_s C)$ plus $\Delta I_L\, R_{ESR}$.
- **Load transient**: after a load step ΔI, the capacitor supplies the difference until the loop responds, roughly within $1/(2\pi f_c)$ for a loop crossover f_c, giving $\Delta V \approx \Delta I/(2\pi f_c C)$; and the inductor's slew rate (V_o/L for an unloading step) limits how fast the converter can respond at all.
- **Stability**: the capacitor and its ESR set the output filter's double pole and ESR zero (Section 30.9).

Table 30.2 — Output-capacitor technologies (typical values).

Type	Typical value per part	ESR	Notes
Multilayer ceramic, X5R/X7R	10–100 µF at 4–25 V	1–5 mΩ	Loses 30–70 % of capacitance at rated DC bias; piezoelectric "singing"
Polymer aluminium or tantalum	100–1,000 µF at 2.5–25 V	5–30 mΩ	Stable capacitance; bulk energy for load steps
Aluminium electrolytic	100 µF–10 mF	20–500 mΩ	Cheap bulk; life halves per 10 K rise (Section 30.8)
Film (polypropylene)	1–100 µF at 250–1,000 V	2–10 mΩ	DC links, high-voltage outputs; very long life

DC-bias derating is the trap of the opening. A 47 µF, 6.3 V X5R capacitor in a 1206 case may retain only 25–35 µF at 3.3 V, and a 22 µF, 6.3 V part in 0805 may retain less than half; the manufacturer's bias curve, not the nominal value, goes into the calculation.

Worked example — output capacitance for a 2.5 A load step. The FPGA rail must stay within ±3 % (99 mV) for a step from 2.5 A to 5 A. With a loop crossover of 50 kHz (a tenth of f_s), the needed effective capacitance is $C \geq 2.5/(2\pi \times 50\ \text{kHz} \times 0.099) = 80$ µF. Three 47 µF ceramics that retain about 30 µF each at 3.3 V give 90 µF, for a dip of $2.5/(2\pi \times 50\ \text{kHz} \times 90\ \mu\text{F}) = 88$ mV. The original design used two of them, assuming 94 µF nominal; with the 60 µF actually present the dip is about 130 mV, and with the slower loop the converter actually had, more than 200 mV.

The input capacitor

The input capacitor of a buck carries the pulsating switch current minus its average. Its RMS current $I_o\sqrt{D(1-D)}$ is largest at $D = 0.5$ ($I_o/2$) and is 2.23 A in the example; ceramics handle it easily, while an electrolytic must be chosen by its ripple-current rating. The input voltage ripple is $\Delta V_{in} \approx I_o D(1-D)/(f_s C_{in})$: 0.17 V with 12 µF effective. The input capacitor also closes the high-frequency commutation loop of Chapter 29, so a small ceramic (100 nF to 1 µF) sits directly at the switch pins regardless of the bulk capacitance.

30.7 Ripple

Output ripple and its three parts

The output ripple of a buck is the sum of three terms that peak at different instants:

$$\Delta V_o \approx \frac{\Delta I_L}{8 f_s C} + \Delta I_L \, R_{ESR} + L_{ESL} \frac{\Delta I_L}{(1-D)T_s}$$

The capacitive term assumes all the ripple current flows into an ideal capacitor and the load takes the DC; the ESR term is a triangle in phase with the inductor current; the ESL term is a small step at each slope change. For the example, $1.45/(8 \times 500\ \text{kHz} \times 90\ \mu\text{F}) = 4.0$ mV and $1.45 \times 1\ \text{m}\Omega = 1.45$ mV. The two do not simply add, because the capacitive arcs peak midway through each slope while the ESR triangle peaks at its ends; the combined waveform is 4.2 mV peak to peak. With electrolytic capacitors the ESR term dominates by an order of magnitude; with ceramics the capacitive term dominates. What usually exceeds both on the oscilloscope are the high-frequency spikes coupled from the switch-node edges, which are reduced by layout and by a small ferrite-bead or LC post-filter, not by more bulk capacitance.

> **Status of the principle: Linear-ripple approximation**
> *Approximate*. Treats the inductor current as an exact triangle (constant voltages across L during each interval) and the output voltage as constant when computing that current; valid when the ripple is a few percent of the DC values, which is the design intent, and it fails in DCM, during transients, and when the capacitor's impedance is not dominated by one of C, ESR or ESL at the switching frequency.

For the boost, the output capacitor supplies the full load during each on-time, so the capacitive ripple is $\Delta V \approx I_o D/(f_s C)$, and the ESR term sees the full rectifier pulse, $I_{L,pk} R_{ESR}$. In the single-cell boost ($I_o = 2.4$ A, $D = 0.4$, 1 MHz, 44 µF effective, 2 mΩ) these are $2.4 \times 0.4/(1\ \text{MHz} \times 44\ \mu\text{F}) = 21.8$ mV and $4.6 \times 2\ \text{m}\Omega = 9.2$ mV, several times the buck's despite the higher frequency.

Input ripple, input filters and negative resistance

The pulsating input current of a buck is the converter's main source of conducted emissions. Meeting CISPR 32 or CISPR 25 limits usually needs an input filter, a series inductor or ferrite bead with additional capacitance, giving a second-order attenuation of 40 dB/decade above its resonance. The filter brings a stability trap first identified by R. D. Middlebrook. A regulated converter draws constant power, so its incremental input resistance is negative:

$$r_{in} = \frac{dv_{in}}{di_{in}} = -\frac{V_{in}^2}{P_{in}}$$

For the 3.3 V buck at full load, $P_{in} = 16.5/0.946 = 17.4$ W and $r_{in} = -144/17.4 = -8.3\ \Omega$. If the input filter's output impedance near its resonance approaches 8.3 Ω, filter and converter together form an oscillator. The rule is to damp the filter (an electrolytic with suitable ESR across the filter capacitor, or an RC damping branch) so that its peak output impedance stays well below $|r_{in}|$ at the lowest input voltage and highest load. The same mechanism makes cascaded converters on a DC bus (a 48 V bus feeding many point-of-load bucks) a stability problem at system level.

Spread-spectrum modulation of the switching frequency (dithering by ±5–10 %) lowers the peak emission in a measurement bandwidth by spreading each harmonic, a common addition in automotive converters.

30.8 Efficiency

The loss budget

Combining the loss mechanisms of Chapter 29 with the inductor, capacitor and controller losses gives the converter's loss budget. The listing evaluates it for the 3.3 V, 5 A buck with representative parts: a 30 V high-side MOSFET of 8 mΩ

(10.5 mΩ hot) with 3 nC of switching charge, a 4 mΩ (5.2 mΩ hot) low-side MOSFET with 15 nC of body-diode Q_{rr}, 20 ns dead times, 26 nC of total gate charge at 5 V, the 3.3 µH inductor with 8 mΩ DCR and 0.1 W of core loss, and the controller's 2 mA quiescent current.

```
 import math
# 12 V -> 3.3 V, 5 A synchronous buck at 500 kHz
Vin, Vo, Io, f = 12.0, 3.3, 5.0, 500e3
D = Vo / Vin
L = 3.3e-6                                  # chosen from r = 0.3 -> 3.19 uH
dI = (Vin - Vo) * D / (f * L)               # inductor ripple, p-p
C_out, esr = 90e-6, 1e-3                    # effective at 3.3 V bias
print(f"D = {D:.3f}, dI = {dI:.2f} A, Ipk = {Io + dI/2:.3f} A")
print(f"ripple: C {dI/(8*f*C_out)*1e3:.1f} mV + ESR {dI*esr*1e3:.2f} mV")
print(f"Cin rms current {Io*math.sqrt(D*(1-D)):.2f} A")

def losses(I):
    """Loss budget (W) at load I; parts at Tj ~ 100 C."""
    i2 = I**2 + dI**2/12                    # inductor I_rms^2
    i_v, i_p = I - dI/2, I + dI/2           # valley and peak current
    return {
        "HS conduction": D * i2 * 10.5e-3,
        "LS conduction": (1 - D) * i2 * 5.2e-3,
        "HS overlap":    0.5*Vin*(max(i_v, 0)*3.75e-9 + i_p*2.31e-9)*f,
        "Coss":          0.5 * 1.0e-9 * Vin**2 * f,
        "dead time":     0.8 * (abs(i_v) + i_p) * 20e-9 * f,
        "LS Qrr":        15e-9 * Vin * f,
        "gate drive":    26e-9 * 5.0 * f,
        "L copper":      i2 * 8e-3,
        "L core":        0.10,
        "caps + PCB":    (I*math.sqrt(D*(1-D)))**2 * 3e-3 + i2 * 3e-3,
        "controller":    Vin * 2e-3,
    }

for k, v in losses(Io).items():
    print(f"  {k:14s} {v*1e3:6.1f} mW")
for I in (0.2, 0.5, 1, 2, 5):
    p = sum(losses(I).values())
    print(f"Io = {I:3.1f} A: loss {p:.3f} W, efficiency {Vo*I/(Vo*I + p)*100:.1f} %")
```

```
 D = 0.275, dI = 1.45 A, Ipk = 5.725 A
ripple: C 4.0 mV + ESR 1.45 mV
Cin rms current 2.23 A
  HS conduction    72.7 mW
  LS conduction    94.9 mW
  HS overlap       87.8 mW
  Coss             36.0 mW
  dead time        80.0 mW
  LS Qrr           90.0 mW
  gate drive       65.0 mW
  L copper        201.4 mW
  L core          100.0 mW
  caps + PCB       90.5 mW
  controller       24.0 mW
Io = 0.2 A: loss 0.337 W, efficiency 66.2 %
Io = 0.5 A: loss 0.343 W, efficiency 82.8 %
Io = 1.0 A: loss 0.367 W, efficiency 90.0 %
Io = 2.0 A: loss 0.456 W, efficiency 93.5 %
Io = 5.0 A: loss 0.942 W, efficiency 94.6 %
```

The 0.94 W total at full load gives 94.6 % efficiency. The largest single loss is the inductor's copper (0.20 W), and the MOSFET conduction losses together (0.17 W) are smaller than the switching-related ones (overlap, C_{oss}, dead time, Q_{rr} and gate drive: 0.36 W). This is typical of low-voltage converters at 0.5–1 MHz, and it is the reason for the integrated power stages (driver and both MOSFETs in one package, with a few nanohenries of loop inductance and 5–10 ns of dead time) used in server and graphics regulators.

Efficiency across load

The loss budget splits into three parts with different load dependence: losses proportional to I^2 (conduction, copper), roughly proportional to I (overlap, dead time), and independent of load at fixed frequency (gate drive, C_{oss}, Q_{rr}, core loss, controller). The fixed part, about 0.32 W here, sets the light-load efficiency: at 0.2 A the converter delivers 0.66 W and loses 0.34 W, the 66 % of the opening. Efficiency peaks where the load-dependent and fixed losses are comparable, typically at 30–60 % of rated load (Figure 30.4), and falls at both ends.

Worked example — what a light-load mode recovers. At 0.2 A, about 0.31 W of the 0.34 W loss is proportional to switching frequency. A controller that switches to pulse-frequency modulation, cutting the effective switching rate tenfold at this load, reduces that part to about 31 mW; with the 24 mW controller and a few milliwatts of conduction, the efficiency rises to roughly $0.66/(0.66 + 0.031 + 0.024 + 0.005) = 92\,\%$. The 0.3 W saved inside a sealed enclosure, multiplied across a building's worth of controllers in standby, is why light-load efficiency is regulated in external power supplies (US DOE Level VI, EU Ecodesign) and specified for most battery-powered products.

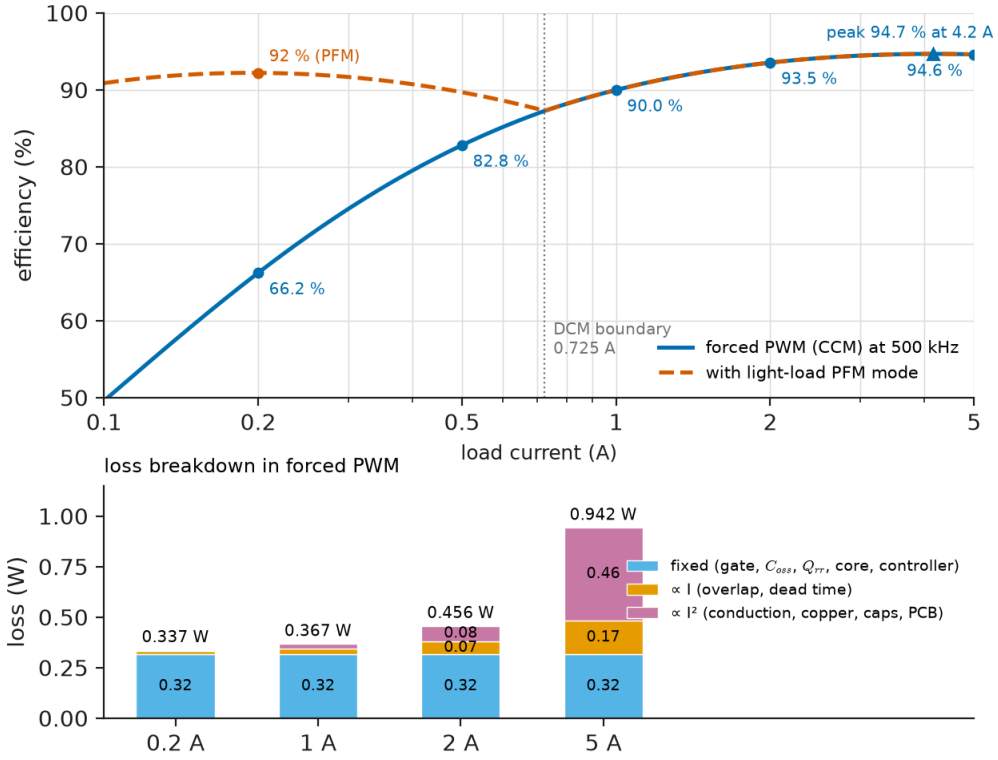


Figure 30.4 — Efficiency and loss breakdown of the 12 V to 3.3 V, 5 A buck versus load, in forced PWM and with a light-load PFM mode.

Losses outside the loss budget

The capacitors' ESR losses are small but heat the capacitors themselves, and capacitor life sets converter life. Aluminium electrolytic capacitors are the usual limit: their electrolyte evaporates at a rate that roughly doubles for every 10 K rise in core temperature, so a capacitor rated 5,000 h at 105 °C lasts about 40,000 h at 75 °C. Ceramic and film capacitors have no such wear-out mechanism, which is one reason modern converters use them wherever their values suffice.

> **Status of the principle: The 10 °C rule for electrolytic-capacitor life**
> *Empirical.* An Arrhenius-type fit to manufacturer life tests, valid between roughly 40 °C and the rated temperature; ripple current adds self-heating that must be included in the core temperature, and at low temperatures other mechanisms (seal ageing, vibration, voltage stress) dominate.

30.9 Feedback control interface

The control loop of a converter

A converter's output must stay regulated against changes in input voltage, load and component values. A controller measures the output through a divider, compares it with a reference, and adjusts the duty cycle through a pulse-width modulator (Figure 30.5). Four control architectures are common:

- **Voltage mode**: an error amplifier with a compensation network drives a comparator against a fixed sawtooth ramp of amplitude V_{ramp}; the modulator gain is $1/V_{ramp}$, often made proportional to V_{in} (input feedforward) so that loop gain does not vary with input voltage.
- **Peak current mode**: an inner loop turns the switch off when the inductor current reaches a level set by the outer voltage loop. The inductor then behaves as a current source, removing the output filter's double pole from the voltage loop, and the converter gains cycle-by-cycle current limiting. Above $D = 0.5$ the inner loop shows subharmonic oscillation unless a compensating ramp of at least half the inductor's down-slope is added.

- **Average current mode**, used in PFC and battery chargers, where the inductor current is the controlled quantity.
- **Hysteretic and constant-on-time control**, used in processor regulators for their fast response, at the cost of variable frequency.

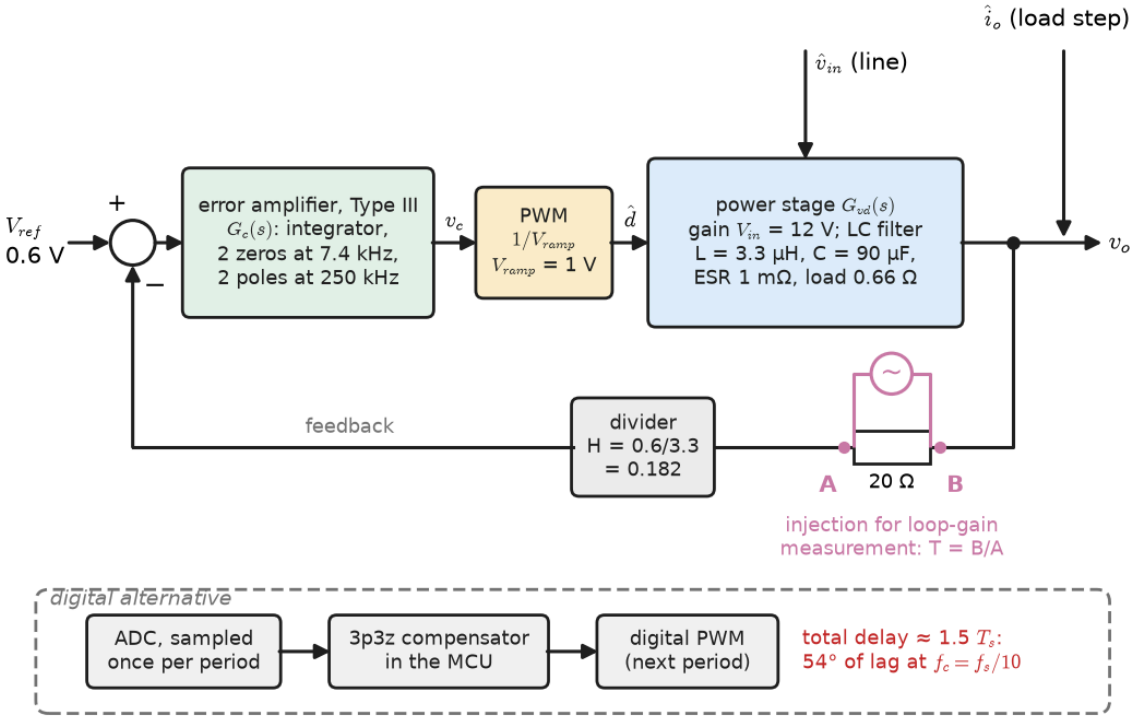


Figure 30.5 — Voltage-mode feedback loop of the buck converter with its averaged small-signal blocks.

The averaged small-signal model

The converter's state variables (inductor current, capacitor voltage) move slowly compared with the switching period, so the switches can be replaced by their averages over one period, and the resulting nonlinear averaged model linearised around the operating point (state-space averaging, or the averaged PWM-switch model). The linearisation, Laplace transformation and Bode analysis are the methods of Book 3 (Ch. 33, Ch. 11, Ch. 13); the results for duty-cycle control in CCM are:

$$G_{vd,buck}(s) = \frac{\hat{v}_o}{\hat{d}} = V_{in}\,\frac{1 + sR_{ESR}C}{1 + s\left(\dfrac{L}{R} + R_{ESR}C\right) + s^2 LC}$$

$$G_{vd,boost}(s) = \frac{V_o}{1-D}\cdot\frac{\left(1 - s\dfrac{L}{(1-D)^2 R}\right)(1 + sR_{ESR}C)}{1 + s\dfrac{L}{(1-D)^2 R} + s^2\dfrac{LC}{(1-D)^2}}$$

The buck's control-to-output response is the output LC filter driven by an amplifier of gain V_{in}: a double pole at $f_0 = 1/(2\pi\sqrt{LC})$ with quality factor $Q \approx R\sqrt{C/L}$, a $-180°$ phase drop around it, and a left-half-plane ESR zero. The boost has the same structure with an effective inductance $L/(1-D)^2$, so its resonance moves with operating point, and a **right-half-plane zero** at

$$f_{RHPZ} = \frac{R(1-D)^2}{2\pi L}$$

which adds gain like a zero but phase lag like a pole: the mathematical form of the boost's initial wrong-way response (Section 30.2). In peak current mode the buck's control-to-output response reduces to a single pole near $1/(2\pi RC)$ (2.7

kHz for the example) with a double pole at $f_s/2$ from the sampling action of the current loop; the boost's RHP zero remains in every control mode.

> **Status of the principle: Averaged small-signal converter model**
> *Approximate.* Valid for small perturbations about a CCM operating point at frequencies well below the switching frequency, about $f_s/5$ or less; it misses the sampling effects near $f_s/2$ (subharmonic oscillation in current mode), the change of structure in DCM (where the inductor pole moves near f_s and the response becomes first-order), and large-signal effects such as duty-cycle saturation during big transients.

Compensation and crossover

The compensator shapes the loop gain $T(s) = G_c(s)\,G_{vd}(s)\,H/V_{ramp}$ by the loop-shaping rules of Book 3 Ch. 35 §35.4: an integrator for zero steady-state error, crossover at the desired speed with enough phase margin (45–70° for SMPS voltage loops, Book 3 Table 35.5) and roll-off above it. Three practical constraints set the crossover frequency f_c:

- **Switching frequency**: $f_c \le f_s/10$ to $f_s/5$, where the averaged model is valid and the switching ripple is attenuated before reaching the modulator.
- **Right-half-plane zero**: $f_c \le f_{RHPZ}/5$ for boost and buck–boost, at the worst-case (lowest) f_{RHPZ}, which occurs at minimum input voltage and maximum load.
- **The LC double pole** in voltage mode: crossing above f_0 requires two compensator zeros to recover the 180° lost, the **Type III** compensator (an integrator, two zeros near f_0 and two poles at or above the ESR zero or $f_s/2$); with a large ESR zero below f_c (electrolytic output capacitors), the simpler **Type II** (integrator, one zero, one pole) suffices, and it is also the standard compensator for current mode.

Worked example — Type III compensation of the 3.3 V buck. The plant has $V_{in} = 12$ V, $V_{ramp} = 1$ V, $L = 3.3$ µH, $C = 90$ µF effective with 1 mΩ ESR (zero at 1.77 MHz, useless), full load $R = 0.66\ \Omega$, and a divider $H = 0.6/3.3$. The double pole is at $f_0 = 9.24$ kHz with $Q = 3.45$. Target $f_c = 50$ kHz, a tenth of f_s. Place both compensator zeros at $0.8 f_0 = 7.4$ kHz and both high-frequency poles at $f_s/2 = 250$ kHz, then set the integrator gain for unity loop gain at 50 kHz:

```
 import numpy as np
# Voltage-mode buck: 12 V -> 3.3 V, 5 A, 500 kHz, L = 3.3 uH, C = 90 uF, ESR 1 mOhm
Vin, Vramp, L, C, esr, R = 12.0, 1.0, 3.3e-6, 90e-6, 1e-3, 0.66
H = 0.6/3.3                                  # divider to a 0.6 V reference
fsw, fc = 500e3, 50e3
f0 = 1/(2*np.pi*np.sqrt(L*C))
s = lambda f: 2j*np.pi*f
def Gvd(f):                                  # averaged control-to-output
    x = s(f)
    return Vin*(1 + x*C*esr)/(1 + x*(L/R + C*esr) + x**2*L*C)
# Type III: integrator, two zeros near f0, two poles at fsw/2
fz1, fz2, fp1, fp2 = 0.8*f0, 0.8*f0, fsw/2, fsw/2
def Gc(f, k=1.0):
    x = s(f)
    return k*(1 + x/(2*np.pi*fz1))*(1 + x/(2*np.pi*fz2)) / (
        x*(1 + x/(2*np.pi*fp1))*(1 + x/(2*np.pi*fp2)))
T1 = lambda f, k=1.0: Gc(f, k)*Gvd(f)*H/Vramp
k = 1/abs(T1(fc))                            # set crossover at fc
f = np.logspace(2, 6, 4001)
T = T1(f, k)
ic = np.argmin(abs(abs(T) - 1)); pm = 180 + np.degrees(np.angle(T[ic]))
ph = np.unwrap(np.angle(T)); i180 = np.argmin(abs(ph + np.pi))
print(f"f0 = {f0/1e3:.2f} kHz, Q = {R*np.sqrt(C/L):.2f}, k = {k:.4g}")
print(f"crossover {f[ic]/1e3:.1f} kHz, PM {pm:.1f} deg, "
      f"GM {-20*np.log10(abs(T[i180])):.1f} dB at {f[i180]/1e3:.0f} kHz")
# extra phase lag of a digital loop with 1.5 Ts total delay sampled at fsw
print(f"digital delay lag at fc: {360*1.5*fc/fsw:.0f} deg")
```

```
 f0 = 9.24 kHz, Q = 3.45, k = 9.07e+04
crossover 50.0 kHz, PM 55.4 deg, GM 22.1 dB at 281 kHz
digital delay lag at fc: 54 deg
```

The loop crosses at 50 kHz with 55° of phase margin and 22 dB of gain margin, inside the SMPS targets, and the load-step estimate of Section 30.6 (88 mV for a 2.5 A step) assumed exactly this crossover. At light load the double pole's Q rises (the load no longer damps it), which an analog Type III tolerates because the gain near f_0 is well above unity. Figure 30.6 shows the plant, compensator and loop gain.

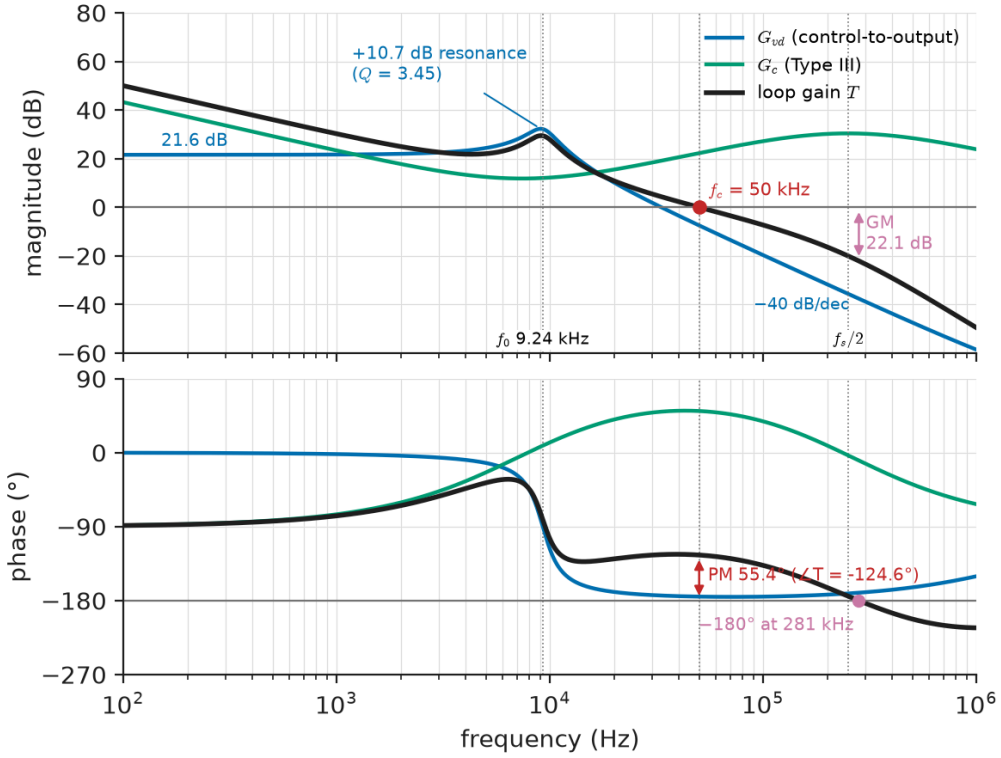


Figure 30.6 — Bode plots of the buck's control-to-output response, the Type III compensator and the loop gain, with crossover and margins marked.

Worked example — why the low-battery boost oscillates. The single-cell boost of Section 30.2 ($L = 1$ µH, $R = 2.08\ \Omega$) has its RHP zero at $f_{RHPZ} = 2.08 \times 0.6^2/(2\pi \times 1\ \mu\text{H}) = 119$ kHz at $V_{in} = 3.0$ V, but at 234 kHz on a full cell ($V_{in} = 4.2$ V, $D = 0.16$). A loop designed on a full cell with 40 kHz crossover has margin to spare there; at 3.0 V the zero has moved to three times the crossover, contributing $\tan^{-1}(40/119) = 19^\circ$ of extra lag, and the effective double pole $(1-D)/(2\pi\sqrt{LC})$ has moved down from 20.2 kHz to 14.4 kHz, adding further lag and gain. The margin vanishes and the converter oscillates near the end of discharge. The fix is to design at the worst case, $f_c \le f_{RHPZ,min}/5 \approx 24$ kHz, in peak current mode so that the double pole is gone.

Digital control

Many converters, from server supplies and EV chargers to solar inverters, close their loops in a microcontroller or DSP (TI C2000, Microchip dsPIC, Infineon XMC, STM32G4). The ADC samples the output once per switching period, usually triggered by the PWM timer at the midpoint of the on- or off-time to avoid switching noise; the compensator runs in an interrupt; and the result updates the PWM compare register for the next period. The continuous compensator is discretised by the bilinear transform or by pole–zero matching (Book 3 Ch. 35 §35.8), giving a difference equation with three poles and three zeros for a Type III design:

```
 /* cntl_3p3z.c -- Type III voltage-loop compensator in Q15, run once per PWM period (C11) */
#include <stdint.h>

typedef struct {
    int32_t b[4];          /* b0..b3, Q15: zeros                              */
    int32_t a[4];          /* a1..a3 in a[1..3], Q15: poles (a[0] unused)     */
    int32_t e[4];          /* error history e[n], e[n-1], e[n-2], e[n-3]      */
    int32_t u[4];          /* output history in u[1..3] (PWM counts)          */
    int32_t u_min, u_max;  /* duty-cycle limits in PWM counts                 */
} cntl3p3z_t;

/* u[n] = sum_{k=0..3} b_k e[n-k] + sum_{k=1..3} a_k u[n-k] */
int32_t cntl3p3z_run(cntl3p3z_t *c, int32_t ref, int32_t adc)
{
    c->e[3] = c->e[2]; c->e[2] = c->e[1]; c->e[1] = c->e[0];
    c->e[0] = ref - adc;

    int64_t acc = 0;
    for (int k = 0; k < 4; ++k) acc += (int64_t)c->b[k] * c->e[k];
    for (int k = 1; k < 4; ++k) acc += (int64_t)c->a[k] * c->u[k];

    int32_t u = (int32_t)(acc / 32768);          /* back from Q15 */
    if (u > c->u_max) u = c->u_max;              /* clamping also stops */
    if (u < c->u_min) u = c->u_min;              /* integrator wind-up  */

    c->u[3] = c->u[2]; c->u[2] = c->u[1]; c->u[1] = u;
    return u;                                    /* next PWM compare value */
}
```

Storing the *clamped* output in the history is what prevents integrator wind-up: when the duty cycle saturates during start-up or a large transient, the stored state stays at the limit instead of growing. The division by 32768 rather than a right shift keeps the rounding well defined for negative values in standard C.

The price of digital control is delay. Sampling, computation and the PWM update together add about 1.5 switching periods of delay, which costs $360^\circ \times 1.5\, f_c/f_s$ of phase at crossover: 54° at $f_c = f_s/10$, almost all of the analog design's margin. Digital loops therefore cross at $f_s/20$ to $f_s/30$, or use sampling at twice the switching frequency and computation within a fraction of a period, and the averaged model is supplemented by a discrete-time model that includes the delay. In return, digital control adds adaptive compensation, nonlinear transient response, telemetry over PMBus, and the ability to change the loop in firmware (Chapter 28).

Measuring the loop

The loop gain of a built converter is measured, not just computed. A small resistor (10–50 Ω) is inserted in the feedback path between the output and the divider; a network analyser injects a small sinusoid across it and measures the ratio of the signals on its two sides, which equals the loop gain at that frequency without opening the loop. The injected amplitude must be small enough (typically a few percent of the output) to keep the converter linear. A measured phase margin below about 30°, or a gain margin below 6 dB, at any input voltage, load and temperature corner, is a design that will oscillate somewhere in the field.

What remains open

Open question	Where it is answered
How are switching and conduction losses, gate drive, snubbers and the switching frequency chosen for each switch?	Chapter 29
How does a boost converter shape the input current of a rectifier for power-factor correction, and how do inverters reuse the buck cell?	Chapter 31
How are battery chargers, MPPT stages and bidirectional DC–DC converters for EVs and storage built and controlled?	Chapter 32
How are isolated converters (forward, full-bridge, LLC resonant) designed?	Beyond the scope of this book; the flyback of Section 30.3 is the entry point
How are compensators designed in general, and what are the theory of margins and digital control?	*Circuits, Signals and Communications*, Ch. 34, Ch. 35
How is the control interrupt scheduled and its latency bounded on a microcontroller?	Chapter 27
How are energy-storage systems and battery packs dimensioned at system level?	*Electrical Energy and Machines*, Ch. 16

Next: Chapter 31 turns from DC to AC, building rectifiers and inverters from the same switches and commutation cells, and dealing with the harmonics, filters and thermal limits that AC conversion brings.

Chapter 31 — AC–DC and DC–AC Conversion

Part VII · Power Electronics

A plating shop connects a 100 kW thyristor rectifier (250 V, 400 A) directly to its 400 V supply and three months later receives a utility letter: its power factor has fallen to 0.45 and a neighbour complains of notched voltage on the feeder. Across town, a 55 kW pump drive reprogrammed from 4 kHz to 16 kHz to quieten a hospital plant room trips on IGBT over-temperature within an hour. A 15 kW rooftop PV inverter fails the grid operator's emission check because the resonance of its output filter was left undamped. All three failures are predictable from one-page circuit models: thyristor commutation through the supply inductance, the loss and thermal budget of an inverter leg, and the transfer function of an LCL filter. This chapter builds those models and turns them into design procedures.

> **How do diode and thyristor rectifiers turn AC into DC, and what do overlap, firing angle and pulse number cost the supply?**
> **How does a PWM inverter synthesise AC from a DC link, what spectrum does it produce, and how are its filter, switches, protection and heatsink chosen?**

Concept	Engineering destination
Uncontrolled and controlled rectifiers	Drive front ends, battery chargers, electroplating and electrolysis, DC traction substations, HVDC terminals
Commutation overlap	DC voltage regulation, line notching, power factor of phase-controlled loads
Inverters (half-bridge, H-bridge, three-phase, multilevel)	UPS, PV and battery inverters, motor drives, induction heating, EV traction
PWM (sine-triangle, space-vector, discontinuous)	Timer configuration in a drive or inverter MCU, DC-link utilisation, switching-loss budget
Harmonic spectra	Filter design, EMC compliance, motor and transformer heating
L and LCL filters	Grid-connected inverters, active front ends, UPS output stages
Switching strategies	Bipolar versus unipolar PWM, selective harmonic elimination in MV drives, soft switching
Semiconductor selection	Voltage and current ratings, Si IGBT versus SiC and GaN, cosmic-ray derating
Protection and thermal design	DC-link precharge, fuses, short-circuit turn-off, junction temperature and power-cycling life

Roadmap. Rectifiers with and without phase control, including overlap (31.1) → inverter topologies and dead time (31.2) → PWM as a modulator circuit (31.3) → single- and three-phase inverter design (31.4) → the harmonic spectra both sides produce (31.5) → filters that remove them (31.6) → switching strategies (31.7) → choosing the semiconductors (31.8) → protection and thermal design (31.9).

> **Boundary.**
> The rectifier and switching-converter sketch of *Engineering Physics*, Ch. 18 §18.8, and the single-phase diode bridge with capacitor filter of Chapter 6 are the starting point; thyristor, IGBT and wide-bandgap devices are introduced in Chapter 5; switching and conduction losses, gate drive and snubbers in Chapter 29; the buck and boost cells reused here in Chapter 30. Fourier series, THD and active power with harmonics are from *Circuits, Signals and Communications*, Ch. 18 §18.1 and Ch. 9 §9.8. Harmonic limits at the point of common coupling (IEEE 519, IEC 61000-3-2/-12) are owned by *Electrical Energy and Machines*, Ch. 25 §25.3, and how a drive uses the inverter (V/f, FOC, DTC) by Ch. 12.

31.1 Controlled and uncontrolled rectifiers

From the single-phase bridge to the six-pulse bridge

Chapter 6 treated the single-phase diode bridge feeding a smoothing capacitor: the capacitor charges near the voltage peaks and the line current flows in short pulses. That circuit powers almost every product under a kilowatt. Above a few kilowatts two things change. The supply is three-phase, and the DC side usually contains enough inductance (a DC choke, a motor armature, a plating cell's leads plus a reactor) that the DC current I_d is nearly constant over a supply cycle. With a constant DC current, a rectifier becomes a set of switches that route I_d from one supply line to another, and its behaviour is set by *which* switch conducts *when*.

In the **three-phase diode bridge** (six diodes, two per phase), the upper group connects the DC positive rail to whichever line voltage is most positive and the lower group connects the negative rail to the most negative. The DC voltage is the envelope of the six line-to-line voltages, a waveform with six pulses per supply cycle and average

$$V_{d0} = \frac{3\sqrt{2}}{\pi} V_{LL} = 1.35\, V_{LL}$$

540 V from a 400 V supply, 931 V from 690 V. Each diode conducts for 120° and carries average current $I_d/3$ and RMS current $I_d/\sqrt{3}$; each line current is a quasi-square wave of 120° positive and 120° negative blocks, whose fundamental is $3/\pi = 0.955$ of its RMS value. The peak reverse voltage on each diode is the peak line voltage, $\sqrt{2}V_{LL}$: 566 V on a 400 V line, which together with the 2–2.5× margin for line surges is why 400 V bridges use 1200–1600 V diodes and 690 V bridges 2200 V.

Phase control with thyristors

Replacing the diodes by thyristors lets the controller delay each commutation by a **firing angle** α measured from the instant the diode would have started conducting (the natural commutation point: the voltage zero crossing in a single-phase bridge, 30° after the phase-voltage zero crossing in a three-phase one). A single-phase fully controlled bridge with continuous DC current gives

$$V_d = \frac{2\sqrt{2}}{\pi} V \cos\alpha = 0.90\, V \cos\alpha$$

and the three-phase fully controlled bridge

$$V_d = 1.35\, V_{LL} \cos\alpha$$

Both follow from averaging the segment of supply voltage that is connected to the DC side between successive firings. For $\alpha < 90^\circ$ the bridge rectifies; for $90^\circ < \alpha < 180^\circ$ the average voltage reverses while the current, which a thyristor passes in one direction only, does not, so power flows from the DC side to the AC supply: the converter **inverts**. Line-commutated inversion is how a thyristor DC drive regenerates and how classic HVDC terminals transfer power; its use in drives is described in *Electrical Energy and Machines*, Ch. 11 §11.1.

A **half-controlled bridge** (three thyristors, three diodes) cannot invert but freewheels at large α, improving power factor; it serves battery chargers and field supplies. A **12-pulse** converter feeds two bridges from star and delta secondaries 30° apart, cancelling the 5th and 7th harmonics.

Commutation overlap

Current cannot transfer from one thyristor to the next instantaneously: the supply and transformer have leakage inductance L_c per phase. For an **overlap angle** μ both devices conduct, the two lines are short-circuited through $2L_c$, and the DC rail sits at the mean of the two phase voltages. Integrating the commutation loop's voltage over the overlap interval, with the DC current constant, gives the overlap equation

$$\cos\alpha - \cos(\alpha+\mu) = \frac{2\omega L_c I_d}{\sqrt{2}\,V_{LL}}$$

The lost volt-seconds, repeated six times per cycle, lower the average DC voltage by

$$\Delta V_d = \frac{3\omega L_c}{\pi} I_d$$

so the loaded bridge behaves as $V_d = 1.35 V_{LL}\cos\alpha - R_{ov} I_d$ with $R_{ov} = 3\omega L_c/\pi$, a "resistance" that dissipates nothing. Overlap also carves six **commutation notches** of width μ per cycle into the terminal voltage, which upset zero-crossing detectors of other loads on a weak feeder.

> **Status of the principle: Overlap equation for a line-commutated bridge**
> *Derived.* Follows from Faraday's law applied to the commutation loop under constant DC current and sinusoidal source voltage; it fails when the DC current is discontinuous, when the source contains significant resistance or distortion, and when two commutations overlap ($\mu > 60^\circ$ in a six-pulse bridge, reached only on very weak supplies or under faults).

Overlap also limits inversion. A thyristor needs a reverse-bias interval after its current reaches zero (its turn-off time t_q, 100–400 µs for phase-control types) before forward voltage returns. The extinction angle $\gamma = 180^\circ - \alpha - \mu$ must therefore exceed ωt_q plus margin; firing circuits clamp α to about 150°. If a supply dip enlarges μ unexpectedly, the outgoing thyristor refires and the bridge short-circuits the AC supply through the DC load: **commutation failure**, the classic fault of regenerating DC drives and HVDC inverters.

Power factor of rectifiers

With a sinusoidal supply voltage, only the fundamental line current carries power (*Circuits, Signals and Communications*, Ch. 9 §9.8). The rectifier's power factor is then the product of a **distortion factor** I_1/I_{rms} (0.955 for 120° blocks, 0.5–0.7 for a capacitor-input bridge) and a **displacement factor** $\cos\varphi_1$. For a phase-controlled bridge the fundamental lags the voltage by about $\alpha + \mu/2$, so

$$\mathrm{PF} \approx 0.955\cos\!\left(\alpha + \tfrac{\mu}{2}\right)$$

The diode bridge ($\alpha = 0$) runs near 0.95 with a DC choke. A capacitor-input bridge without a choke has a displacement factor near unity but a distortion factor of 0.5–0.7. Phase control buys a variable DC voltage at the price of reactive power that does not fall when the DC power does.

Worked example — the plating rectifier. A three-phase fully controlled bridge must supply 250 V, 400 A (100 kW) to a plating line. Case A connects it to the 400 V, 50 Hz supply through a commutating reactor of $L_c = 0.3$ mH per phase.

- $V_{d0} = 1.35 \times 400 = 540.2$ V; $R_{ov} = 3 \times (2\pi \cdot 50) \times 0.3 \times 10^{-3}/\pi = 0.090\ \Omega$, so overlap costs $0.090 \times 400 = 36.0$ V.
- $\cos\alpha = (250 + 36)/540.2 = 0.529$, $\alpha = 58.0^\circ$.
- Overlap: $\cos(\alpha+\mu) = 0.529 - 2 \times 314.2 \times 0.3 \times 10^{-3} \times 400/(\sqrt{2} \times 400) = 0.529 - 0.133$, so $\mu = 8.6^\circ$.
- PF $\approx 0.955\cos(62.3^\circ) = 0.44$. The line current is $\sqrt{2/3} \times 400 = 327$ A RMS and the apparent power $\sqrt{3} \times 400 \times 327 = 226$ kVA for 100 kW.

Case B adds a 400/230 V transformer so that the bridge works near full output. With $L_c = 0.1$ mH referred to the 230 V side, $V_{d0} = 310.6$ V, the overlap drop is 12.0 V, $\alpha = 32.5^\circ$, $\mu = 7.5^\circ$, and PF ≈ 0.77; the supply now sees 130 kVA instead of 226 kVA. The rule that comes out of the numbers: size the converter transformer so that the bridge runs at α of 15–35° at rated output, and keep phase-controlled rectifiers for applications that genuinely need deep voltage control or line-commutated regeneration. Where they are kept, the reactive power and the 5th and 7th harmonic currents become a power-quality problem for the plant (*Electrical Energy and Machines*, Ch. 25 §25.8).

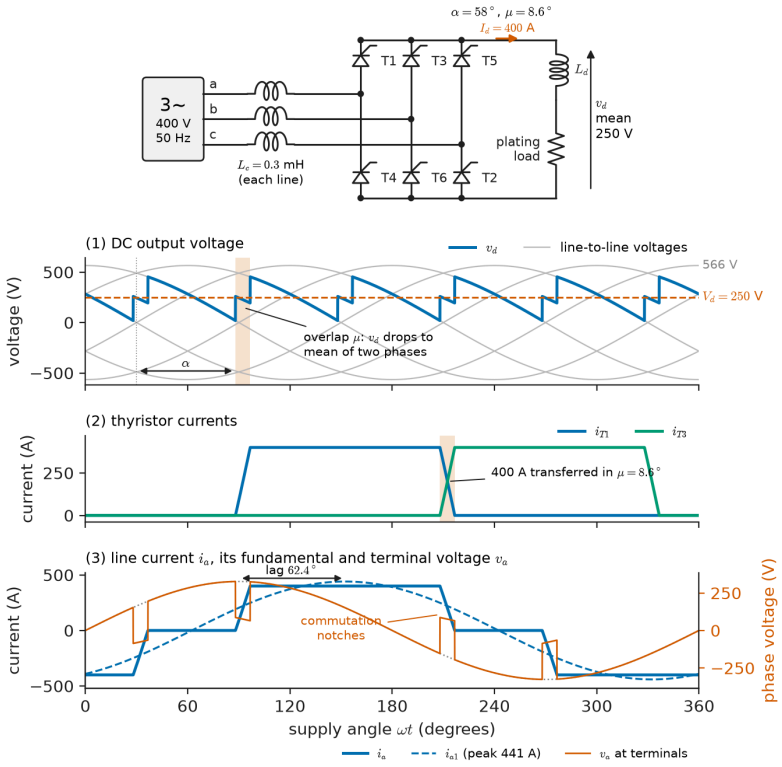


Figure 31.1 — Three-phase fully controlled thyristor bridge with commutation overlap: circuit and waveforms for the plating rectifier at alpha = 58 degrees and mu = 8.6 degrees.

Figure 31.1 shows the waveforms for case A: the notches in the DC voltage during overlap, the trapezoidal current transfer between thyristors, and the line-current fundamental lagging its phase voltage by about $\alpha + \mu/2$.

31.2 Inverters

The switching leg

Every voltage-source inverter is built from a **leg**: two switches with antiparallel diodes in series across the DC link, output at the midpoint. One switch is on at a time, so the output sits at $+V_{dc}$ or 0 whatever the current direction, the diodes carrying current that flows against the conducting transistor. The leg is the buck cell of Chapter 30 made bidirectional; the inverter varies its duty ratio sinusoidally.

Three arrangements cover most applications:

- **Half-bridge**: one leg and a split DC-link capacitor; output $\pm V_{dc}/2$. Small UPS and induction-heating stages.
- **Full bridge (H-bridge)**: two legs; output $+V_{dc}$, 0 or $-V_{dc}$. Single-phase PV, UPS and battery inverters to about 10 kW; the cell of cascaded multilevel converters.
- **Three-phase two-level bridge**: three legs, eight switching states; drives, EV traction, three-phase grid inverters.

Driven by square waves, the three-phase bridge runs in **six-step** mode: its line voltage contains only $h = 6k \pm 1$ with $\hat{V}_h = \hat{V}_1/h$ and a fundamental of $0.780 V_{dc}$ RMS, the most a two-level bridge can give, used at the top speed of traction drives. Everything else uses PWM (Section 31.3).

Voltage-source inverters dominate because a DC-link capacitor is cheaper than the DC-link inductor of a **current-source inverter**, which survives in multi-megawatt synchronous-motor drives. **Multilevel** legs give more than two output levels: the three-level **neutral-point-clamped** (NPC) leg, in which each device blocks only $V_{dc}/2$; the **T-type** leg for 400–690 V at 10–20 kHz; **flying-capacitor**, **cascaded H-bridge** and, for HVDC, **modular multilevel**

converters that stack cells. More levels mean smaller voltage steps, lower dv/dt, a smaller filter and lower-voltage devices.

Dead time

A real leg cannot switch both devices at the same instant: the outgoing device takes 0.1–1 µs to turn off (an IGBT's tail current lasts longer), and if the incoming one turns on first the DC link is shorted through the leg, a **shoot-through**. The gate driver therefore inserts a **dead time** t_d during which both gates are off (0.5–3 µs for Si IGBTs, 50–300 ns for SiC MOSFETs, 10–50 ns for GaN). During the dead time the load current itself chooses the path: positive current freewheels through the lower diode, negative current through the upper, so the leg voltage is decided by the current sign, not by the command. The result is a volt-second error once per switching edge pair, an average voltage error of

$$\Delta V \approx t_d \, f_s \, V_{dc} \, \operatorname{sgn}(i)$$

which opposes the current like a nonlinear resistance and distorts the output near current zero crossings.

> **Status of the principle: Dead-time voltage error**
> *Approximate*. Counts only the dead interval and assumes the current does not change sign within it and that the devices switch instantaneously; turn-on and turn-off delays, diode forward drop and the finite edge rate add to or partly cancel the error, and at small currents the edge itself is slowed by the device capacitances so the error is less than predicted.

Worked example — dead time at low speed. A 400 V drive with $V_{dc} = 540$ V switches at 8 kHz with 2 µs dead time: the error is $2 \times 10^{-6} \times 8000 \times 540 = 8.6$ V per leg. At 50 Hz (327 V phase peak) that is under 3 %. At 2 Hz, constant V/f asks for 16 V line, a 13 V phase peak, and the error is two-thirds of the command: the current flattens at each zero crossing and torque pulsates at six times the output frequency. Every drive therefore adds a current-sign-dependent compensation to the duty ratio (Chapter 32), and SiC inverters with 100 ns dead time have far cleaner low-speed waveforms.

31.3 PWM

The modulator as a circuit

> **Boundary.**
> The averaged leg-voltage relation of sine-triangle PWM, the linear-range limits ($V_{LL1} = 0.612\,m V_{dc}$ for sine-triangle, $0.707 V_{dc}$ with third-harmonic injection or SVPWM, $0.780 V_{dc}$ in six-step) and the space-vector hexagon are stated in *Electrical Energy and Machines*, Ch. 12 §12.2 and §12.5. This section is about how a modulator is built and what it does to the spectrum and losses.

Sine-triangle PWM compares a reference $v^*(t)$ with a triangular carrier of frequency f_s; the upper switch is on while the reference is higher. In an analogue implementation (still found in small UPS and class-D audio stages) a comparator does this continuously: **natural sampling**. In a digital controller the reference is updated once or twice per carrier period and held: **regular sampling**, symmetric if updated at one peak of the carrier, asymmetric if at both. The digital modulator is a hardware timer in **centre-aligned** (up–down) mode: a counter counts from 0 to a period value and back, a compare register per phase holds $D\times$ period, and the output is high while the counter is below the compare value. At a 168 MHz timer clock, a 10 kHz carrier needs a period of $168 \times 10^6/(2 \times 10^4) = 8400$ counts, a duty resolution of 0.012 %. The timer's dead-time generator inserts t_d in hardware on complementary outputs, and the counter's peak or valley triggers the ADC so that currents are sampled midway between switching edges, where the ripple passes through its average (Chapter 32).

Space-vector PWM in practice

The two-level three-phase bridge has eight states: six active vectors of length $2V_{dc}/3$ and two zero vectors (all upper or all lower switches on). Space-vector PWM synthesises a reference vector $\vec{v}^{*}$ of magnitude V^{*} at angle θ within a 60° sector from the two adjacent active vectors for times

$$T_1 = \sqrt{3}\,T_s \frac{V^{*}}{V_{dc}} \sin(60^\circ - \theta), \qquad T_2 = \sqrt{3}\,T_s \frac{V^{*}}{V_{dc}} \sin\theta, \qquad T_0 = T_s - T_1 - T_2$$

and splits T_0 equally between the two zero vectors, placed at the start, middle and end of the period. That split has a simpler equivalent that needs no sector logic: add to all three sinusoidal references the common-mode offset $v_{off} = -\tfrac{1}{2}(\max v^{*} + \min v^{*})$ and run ordinary centre-aligned carrier PWM on the result (**min–max injection**). The line voltages are unchanged because the offset is common to all phases; the leg duty ratios become identical to those of SVPWM. Listing 31.1 is the whole modulator.

```
 #include <stdint.h>

/* Min-max injection modulator, equivalent to symmetric SVPWM.
   va, vb, vc: phase-voltage references (V); vdc: measured DC link (V).
   Returns compare values for a centre-aligned timer counting 0..period;
   the upper switch of a leg is on while the counter is below its value. */
typedef struct { uint16_t ccr[3]; } pwm_cmp_t;

static float clampf(float x, float lo, float hi)
{
    return x < lo ? lo : (x > hi ? hi : x);
}

pwm_cmp_t svpwm_compare(float va, float vb, float vc, float vdc, uint16_t period)
{
    float vmax = va, vmin = va;
    if (vb > vmax) vmax = vb;
    if (vc > vmax) vmax = vc;
    if (vb < vmin) vmin = vb;
    if (vc < vmin) vmin = vc;

    const float voff = -0.5f * (vmax + vmin);     /* common-mode offset */
    const float v[3] = { va + voff, vb + voff, vc + voff };
    pwm_cmp_t out;
    for (int k = 0; k < 3; k++) {
        float d = clampf(0.5f + v[k] / vdc, 0.0f, 1.0f);   /* leg duty */
        out.ccr[k] = (uint16_t)(d * (float)period + 0.5f);
    }
    return out;
}
```

Dividing by the *measured* V_{dc} makes the output volt-seconds immune to DC-link sag and 300 Hz rectifier ripple; the clamp is a crude overmodulation rule. For a reference of 0.5 V_{dc} at 20° the listing gives duties 0.926, 0.370 and 0.074, whose differences 0.557 and 0.296 equal T_1/T_s and T_2/T_s from the dwell-time formulas, as they must.

> **Status of the principle: Equivalence of min–max injection and SVPWM**
> *Derived.* Both place the same active-vector times T_1, T_2 in each period and split the zero-vector time equally; the equivalence is exact for regular sampling and holds in the linear range ($V^{*} \le V_{dc}/\sqrt{3}$), beyond which both must be supplemented by an overmodulation rule.

Discontinuous PWM

Putting all of T_0 on one zero vector clamps each leg to a DC rail for a third of the cycle: **discontinuous PWM** (DPWM). Switching losses fall by about one-third, or up to half if the clamp is placed where the leg current peaks, at the price of more ripple at low modulation index; drives change to DPWM above about half speed, and traction and PV inverters use it throughout.

Unipolar and bipolar PWM in the H-bridge

An H-bridge can be modulated in two ways. **Bipolar** PWM drives the two legs complementarily, so the output jumps between $+V_{dc}$ and $-V_{dc}$ at f_s. **Unipolar** PWM compares two references of opposite sign with the same carrier (or one reference with two opposite carriers); the output steps between $+V_{dc}$ and 0 in the positive half-cycle and between $-V_{dc}$ and 0 in the negative one, and its first harmonic cluster sits at $2f_s$ because the two legs' carrier components cancel. For the same switching frequency, unipolar PWM halves the voltage step and doubles the ripple frequency, quartering the inductance needed for a given ripple (Section 31.4).

31.4 Single-phase and three-phase inverters

The single-phase inverter and its twice-line-frequency power

A single-phase inverter delivering $v = \sqrt{2}V\cos\omega t$ and $i = \sqrt{2}I\cos\omega t$ delivers instantaneous power $p = VI(1+\cos 2\omega t)$: the power pulsates between zero and twice its average at $2f$ (100 Hz). The DC source (a PV string, a battery, the DC link of a UPS) wants constant power, so the difference must be buffered by the DC-link capacitor, which absorbs and releases energy $\pm P/(2\omega)$ per half-cycle. For a peak-to-peak DC-link ripple ΔV around V_{dc},

$$C_{dc} = \frac{P}{\omega\, V_{dc}\, \Delta V}$$

and the capacitor carries a 100 Hz current of RMS value $P/(\sqrt{2}V_{dc})$ on top of its switching ripple. This buffer makes single-phase inverters bulky and limits electrolytic-capacitor life in microinverters; balanced three-phase power is constant and needs none.

Worked example — a 5 kW single-phase PV inverter stage. The H-bridge runs from a 400 V DC link, delivers 5 kW at 230 V, 50 Hz, and switches at 20 kHz.

- Output current $5000/230 = 21.7$ A RMS, 30.7 A peak.
- DC-link capacitor for 5 % (20 V peak-to-peak) ripple: $C = 5000/(314.2 \times 400 \times 20) = 1.99$ mF, carrying $5000/(\sqrt{2}\times 400) = 8.8$ A RMS at 100 Hz.
- Output inductor for a peak-to-peak ripple of 20 % of the peak current, 6.1 A. With bipolar PWM the worst-case ripple is $V_{dc}/(2Lf_s)$ (at zero output voltage), so $L = 400/(2\times 20\,000\times 6.15) = 1.63$ mH. With unipolar PWM the worst case is $V_{dc}/(8Lf_s)$ (at half the peak output), so $L = 0.41$ mH.

Unipolar PWM cuts the inductor to a quarter, but in a transformerless PV inverter it makes the array's common-mode voltage jump and drives leakage current to earth; Chapter 32 shows the topologies (H5, HERIC) that solve this.

The three-phase inverter

A three-phase two-level inverter has three design quantities: the DC-link voltage, the semiconductor current rating and the DC-link capacitor.

- **DC-link voltage.** In the linear range of SVPWM the peak line-to-line output equals V_{dc}, so a grid inverter at 400 V needs $V_{dc} \ge \sqrt{2}\times 400 = 566$ V plus the drop across its filter and control headroom, typically 650–750 V; a drive fed from its own diode bridge has only $1.35V_{LL}$ and accepts the 5 % voltage shortfall at base speed discussed in *Electrical Energy and Machines*, Ch. 12 §12.2.

- **Current rating.** From peak current (overload plus ripple) and the thermal calculation of Section 31.8.
- **DC-link capacitor.** It buffers no line-frequency power but must absorb the switching-frequency current. The RMS capacitor current for sinusoidal PWM with modulation index M (referred to $V_{dc}/2$) and load power factor $\cos\varphi$ is

$$I_{C,rms} = I_{rms}\sqrt{2M\left[\frac{\sqrt{3}}{4\pi} + \cos^2\varphi\left(\frac{\sqrt{3}}{\pi} - \frac{9M}{16}\right)\right]}$$

about 0.5–0.65 of the output phase current over the usual operating range.

> **Status of the principle: DC-link capacitor RMS current**
> *Derived.* Obtained by integrating the square of the switched input current of the bridge over a fundamental cycle, assuming sinusoidal output currents with negligible ripple, a switching frequency much higher than the output frequency and a stiff DC link; ripple, dead time and overmodulation change the result by a few per cent.

For the 55 kW drive of Section 31.8 ($I_{rms} = 100$ A, $M = 1.0$, $\cos\varphi = 0.86$) the formula gives 50.9 A. Electrolytic capacitors of 450 V cannot stand a 560 V link alone, so cans are used in series pairs with balancing resistors; with 2200 µF, 450 V cans each rated about 12 A RMS at the switching frequency and 85 °C, five parallel strings of two give 5.5 mF and 60 A of ripple capability. Capacitor life roughly doubles for every 10 °C reduction in core temperature, an empirical rule that makes the capacitor's ripple heating, not its capacitance, the usual sizing criterion. Film capacitors carry far more ripple per microfarad and dominate in EV traction inverters, where the DC link is stiffened by the battery and only 0.5–1 mF is needed.

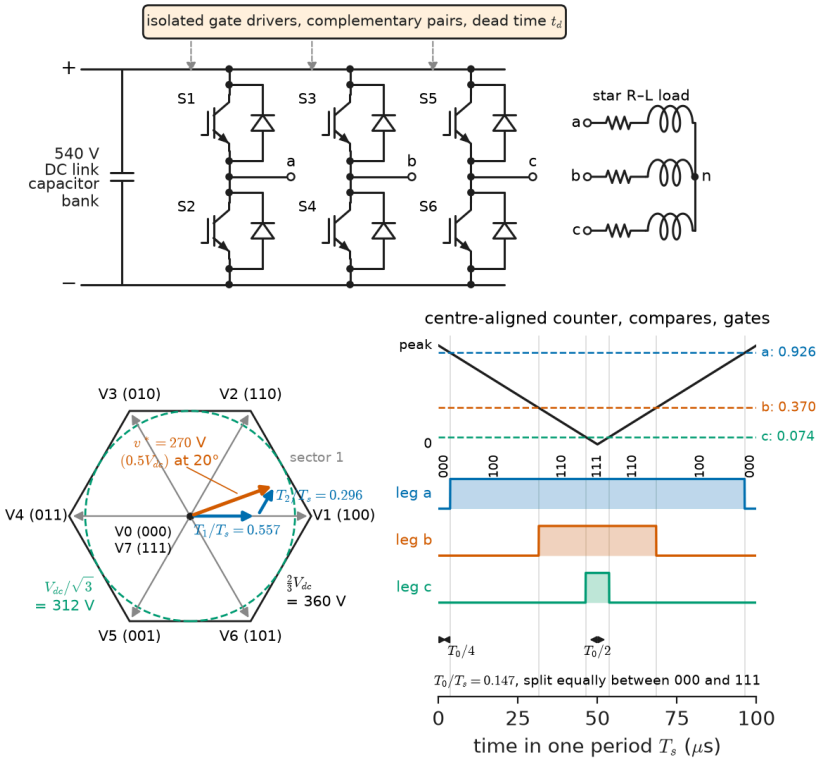


Figure 31.2 — Three-phase two-level inverter with its eight switching states and one centre-aligned SVPWM period.

Figure 31.2 collects the inverter, the hexagon of switching states and one modulation period; the symmetric state sequence 000–100–110–111–110–100–000 is the switching pattern the min–max modulator produces.

31.5 Harmonics

Rectifier-side spectra

A p-pulse rectifier with constant DC current draws the **characteristic harmonics** $h = kp \pm 1$ with $I_h = I_1/h$ ideally (20 % 5th, 14.3 % 7th for $p = 6$). Overlap reduces the higher orders; a capacitor-input bridge without a choke raises the 5th and 7th to 30–80 %; unbalance and asymmetrical firing add non-characteristic orders. Real spectra and limits are tabulated in *Electrical Energy and Machines*, Ch. 25 §25.3. The circuit levers are pulse number, DC chokes and 3–5 % AC reactors, and the PWM rectifier, the inverter of Section 31.4 run in reverse.

Inverter-side spectra

A PWM inverter's output contains the fundamental plus harmonic clusters around multiples of the carrier frequency. For naturally sampled sine-triangle PWM of one leg, a double Fourier analysis (in the time variables of carrier and reference) gives the harmonic at $mf_s + nf_1$ the peak amplitude

$$\hat{V}_{mn} = \frac{4}{m\pi}\,\frac{V_{dc}}{2}\left|J_n\!\left(m\,M\frac{\pi}{2}\right)\sin\frac{(m+n)\pi}{2}\right|$$

where J_n is the Bessel function of the first kind and M the modulation index. The odd-m clusters (around f_s, $3f_s$, ...) contain even sidebands $n = 0, \pm 2, \pm 4$; the even-m clusters (around $2f_s$) contain odd sidebands. In the line-to-line voltage of a three-phase inverter, components with n a multiple of 3, including the carrier itself, are common to all three legs and cancel; the largest remaining harmonics are the sidebands $f_s \pm 2f_1$ and $2f_s \pm f_1$.

> **Status of the principle: Bessel-function spectrum of carrier PWM**
> *Derived.* Exact for naturally sampled PWM with an ideal leg; regular sampling adds small baseband harmonics and sideband asymmetry that grow as the carrier ratio f_s/f_1 falls, and dead time adds low-order (5th, 7th) harmonics that this formula does not contain.

Listing 31.2 checks the formula by brute force for a 540 V link, $M = 0.8$ and a carrier ratio of 51 (2.55 kHz at 50 Hz), computing the FFT of the switched leg and line voltages over one fundamental period.

```
 import numpy as np
from scipy.special import jv

Vdc, M, f1, mf = 540.0, 0.8, 50.0, 51       # DC link, modulation index, carrier ratio
N = 2**18                                    # samples in one fundamental period
t = np.arange(N) / (N * f1)
carrier = 2 / np.pi * np.arcsin(np.sin(2 * np.pi * mf * f1 * t + np.pi / 2))  # triangle, +-1
ref = lambda ph: M * np.sin(2 * np.pi * f1 * t - ph)
leg = lambda ph: np.where(ref(ph) > carrier, Vdc / 2, -Vdc / 2)  # natural sampling
va, vb = leg(0.0), leg(2 * np.pi / 3)

def spectrum(v):                             # peak amplitude of each harmonic h
    return 2 * np.abs(np.fft.rfft(v)) / N

Sa, Sab = spectrum(va), spectrum(va - vb)
bessel = lambda m, n: 4 / (m * np.pi) * Vdc / 2 * abs(jv(n, m * M * np.pi / 2))
for h, mn in [(1, None), (mf, (1, 0)), (mf - 2, (1, -2)), (2 * mf - 1, (2, -1))]:
    theory = M * Vdc / 2 if mn is None else bessel(*mn)
    print(f"h={h:3d}  leg {Sa[h]:6.1f} V (theory {theory:6.1f})  line {Sab[h]:6.1f} V")
wthd = np.sqrt(sum((Sab[h] / h) ** 2 for h in range(2, 2000))) / Sab[1]
print(f"line-voltage WTHD = {100 * wthd:.3f} %")
```

Its output:

```
 h=  1  leg  216.0 V (theory  216.0)  line  374.1 V
h= 51  leg  220.9 V (theory  220.9)  line    0.0 V
h= 49  leg   59.3 V (theory   59.4)  line  102.8 V
h=101  leg   84.9 V (theory   84.9)  line  147.0 V
line-voltage WTHD = 0.982 %
```

The leg's carrier component (220.9 V) exceeds its fundamental yet vanishes in the line voltage; the surviving sidebands are $\sqrt{3}$ times the leg sidebands; and the **weighted THD** (harmonics divided by their order, a proxy for current distortion in an inductive load) is under 1 %, although the plain voltage THD of PWM is 60–100 %. Current ripple, WTHD and the spectrum near the filter resonance, not voltage THD, are an inverter's figures of merit.

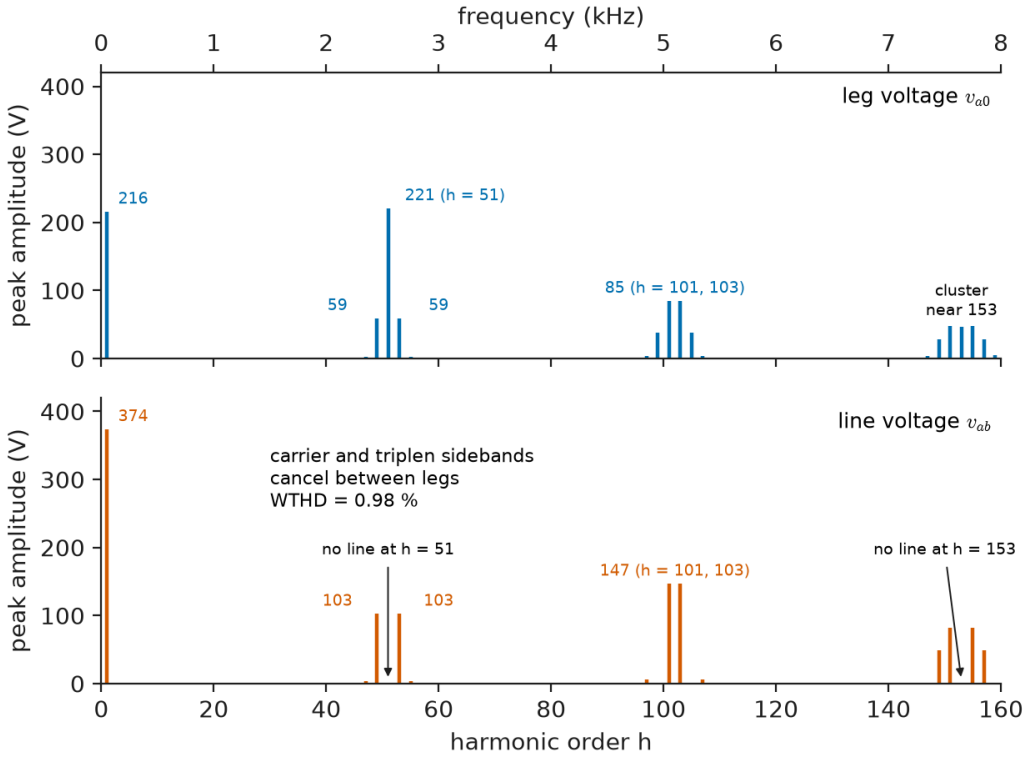


Figure 31.3 — Spectrum of naturally sampled sine-triangle PWM for Vdc = 540 V, M = 0.8 and carrier ratio 51: one leg versus the line-to-line voltage.

The two spectra are plotted in Figure 31.3. At low carrier ratios (MV drives, high-speed spindles) the carrier must be **synchronised** to the fundamental with an odd integer ratio that is a multiple of three, as 51 is here, or subharmonics appear; above a ratio of about 20 a fixed asynchronous carrier is acceptable.

31.6 Filtering

What has to be filtered and where

Filters sit at three interfaces: the **DC side** (DC-link capacitor, DC choke), the **AC side** (an L, LC or LCL network that turns PWM voltage into sinusoidal current or voltage), and the **EMI path** (common- and differential-mode filters for the 150 kHz–30 MHz limits of CISPR 11 and IEC 61800-3). The passive-filter methods and second-order resonance used below are those of *Circuits, Signals and Communications*, Ch. 13 §13.4–13.7.

The L filter

The simplest grid interface is an inductor in each phase. The inverter impresses a PWM voltage on one side and the grid holds a sinusoid on the other, so the inductor sees the difference and the ripple current is set by volt-seconds. For a three-phase leg with carrier PWM, the worst-case peak-to-peak ripple is approximately

$$\Delta I_{max} \approx \frac{V_{dc}}{6\, L\, f_s}$$

The trouble is attenuation. An L filter falls only at 20 dB per decade, so meeting a grid operator's high-frequency current limit at 10 kHz needs an inductor of 10–20 % per unit, which drops so much fundamental voltage that the DC link must rise, and which is large, lossy and expensive.

The LCL filter

Splitting the inductance into a converter-side inductor L_1 and a grid-side inductor L_2 with a capacitor C_f between them gives a third-order filter. Neglecting resistances, the transfer from converter voltage to grid current is

$$\frac{i_2}{v_1}(s) = \frac{1}{s\, L_1 L_2 C_f\, (s^2 + \omega_{res}^2)}, \qquad \omega_{res} = \sqrt{\frac{L_1 + L_2}{L_1 L_2 C_f}}$$

and the switching ripple of i_1 reaches the grid attenuated by

$$\frac{i_2}{i_1}(\omega_s) = \frac{1}{|1 - \omega_s^2 L_2 C_f|}$$

Above resonance the attenuation falls at 60 dB per decade. At resonance the undamped filter has infinite gain: any switching harmonic, grid harmonic or control action near ω_{res} excites it. The filter must be damped, either **passively** with a resistor R_d in series with C_f (cheap, costs a few watts, reduces high-frequency attenuation slightly) or **actively** by feeding back the capacitor current in the inverter's control loop (Chapter 32).

A standard design procedure, for rated power P, line voltage V, DC link V_{dc} and switching frequency f_s:

1. **Base values**: $Z_b = V^2/P$, $C_b = 1/(\omega Z_b)$.
2. **Capacitor**: $C_f \le 0.05\, C_b$, so that the filter draws no more than 5 % of rated reactive power at the fundamental.
3. **Converter-side inductor** from the allowed ripple (10–40 % of rated peak current, here 20 %) with the formula above.
4. **Grid-side inductor** from the required attenuation at f_s, keeping the total inductance below about 0.1 pu so that the voltage drop is acceptable.
5. **Check resonance**: $10 f_1 < f_{res} < f_s/2$, far enough from the grid's harmonics and from the switching frequency.
6. **Damping**: $R_d \approx 1/(3\omega_{res} C_f)$ as a starting value, then check loss and the margin of the current loop.

> **Status of the principle: LCL design rules**
> *Empirical.* The 5 % capacitor limit, the ripple fraction and the resonance window are engineering conventions that balance reactive power, losses, size and control stability; the transfer functions they rest on are exact for ideal components, but core saturation, inductance roll-off with current (powder cores lose 20–40 % at full load) and grid impedance shift the real resonance.

Worked example — an LCL filter for a 15 kW grid inverter. Three-phase, 400 V, 50 Hz, $V_{dc} = 700$ V, $f_s = 10$ kHz.

- $Z_b = 400^2/15\,000 = 10.67\ \Omega$, $C_b = 298\ \mu$F, so $C_f \le 14.9\ \mu$F; choose 15 μF per phase (star). Its reactive power is $3 \times 230.9^2 \times 2\pi 50 \times 15 \times 10^{-6} = 754$ var, 5.0 % of rating.
- Rated current $15\,000/(\sqrt{3} \times 400) = 21.7$ A RMS, 30.6 A peak. Ripple target 20 %: 6.1 A peak-to-peak, so $L_1 = 700/(6 \times 10^4 \times 6.12) = 1.91$ mH.

- Grid side: with $L_2 = 0.8$ mH, $\omega_s^2 L_2 C_f = (2\pi \times 10^4)^2 \times 0.8 \times 10^{-3} \times 15 \times 10^{-6} = 47.4$, so the attenuation is $1/46.4 = 0.0216$ and the grid ripple is 0.13 A, 0.43 % of rated peak current. Raising L_2 to 1.0 mH brings it to 0.34 %; at 0.5 mH it would be 0.70 %.
- Resonance with $L_2 = 0.8$ mH: $f_{res} = \frac{1}{2\pi}\sqrt{(2.71 \times 10^{-3})/(1.91 \times 10^{-3} \times 0.8 \times 10^{-3} \times 15 \times 10^{-6})} = 1.73$ kHz, inside the window 500 Hz to 5 kHz.
- Damping: $R_d = 1/(3 \times 2\pi \times 1731 \times 15 \times 10^{-6}) = 2.0\ \Omega$. The fundamental capacitor current (1.09 A per phase) dissipates 7 W in the three resistors, plus the ripple current's contribution of a few watts more.
- Total inductance: $\omega(L_1 + L_2) = 0.85\ \Omega$, or $0.85/10.67 = 0.080$ pu, an acceptable fundamental voltage drop.

This is the filter that the PV inverter of the opening lacked: its designer chose L_1, C_f and L_2 correctly but omitted R_d to save 10 W, and the grid's own 35th harmonic (1.75 kHz) sat on the resonance.

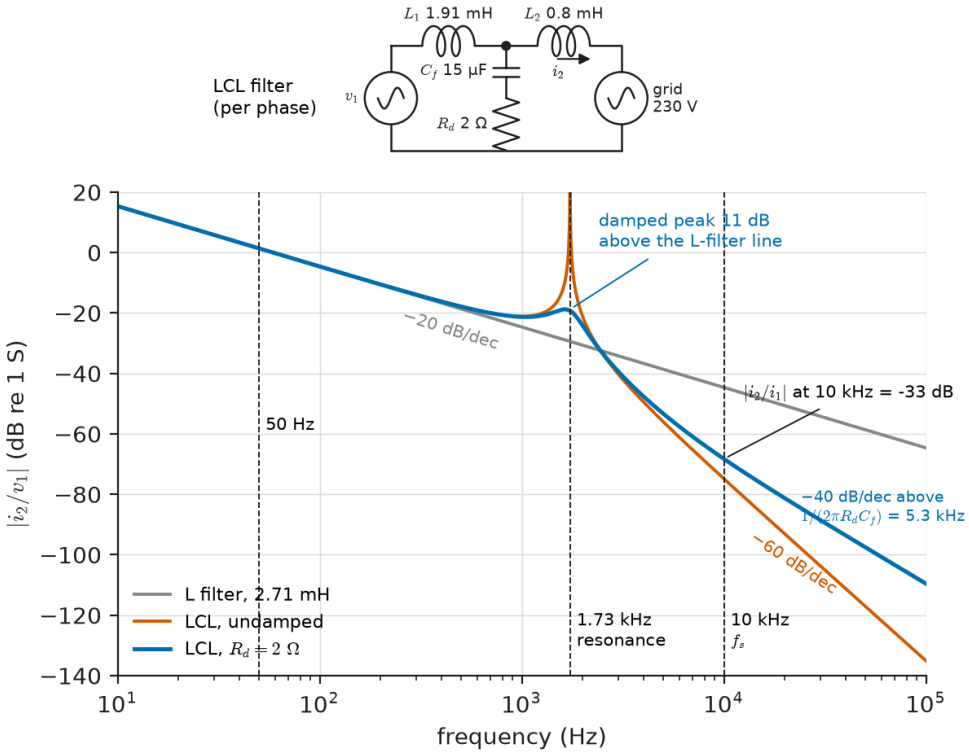


Figure 31.4 — LCL filter of the 15 kW grid inverter: circuit and grid-current transfer function, undamped and with a 2 ohm damping resistor, compared with an L filter of the same total inductance.

Figure 31.4 compares the three responses. Damping lowers the peak to 10 dB above the L-filter line but leaves only 40 dB per decade above $1/(2\pi R_d C_f) = 5.3$ kHz (−68 instead of −75 dB at 10 kHz).

A UPS or off-grid inverter needs a sinusoidal *voltage* and uses an LC filter with closed-loop voltage control. Motor drives normally rely on the motor's leakage inductance; their dv/dt and sine filters belong to Chapter 32.

31.7 Switching strategies

The modulation choice sets the converter's losses, ripple and spectrum. Table 31.1 compares the strategies in common use.

Table 31.1 — Switching strategies for AC–DC and DC–AC converters (typical).

Strategy	Device switching frequency	Strengths	Typical use
Line-commutated (phase control)	Supply frequency	Simple, robust, no switching loss	Thyristor rectifiers, DC drives, LCC HVDC
Square wave / six-step	Output frequency	Maximum voltage, minimum switching loss	Top speed of traction drives, induction heating
Selective harmonic elimination	3–15 × output frequency	Chosen low-order harmonics removed exactly	MV drives, high-power rectifiers
Sine-triangle PWM	2–20 kHz (Si), up to 100 kHz (SiC, GaN)	Simple, clean spectrum	Single-phase inverters, low-cost drives
SVPWM (min–max injection)	2–20 kHz	15 % more voltage than sine-triangle	Standard three-phase drive and grid inverter
Discontinuous PWM	Two-thirds of carrier, on average	One-third to one-half less switching loss	Traction, PV and high-speed drives
Hysteresis current control	Variable, 5–30 kHz	Fast, inherently current-limited	Simple servo drives, active filters, BLDC
Soft switching (ZVS/ZCS)	50 kHz–1 MHz	Very low switching loss and EMI	Resonant DC–DC stages, induction heating

Selective harmonic elimination (SHE) computes a few switching angles offline so that chosen harmonics are exactly zero. With three angles per quarter-cycle in a two-level leg, the fundamental can be set and the 5th and 7th removed; for $M = 0.8$ (referred to $V_{dc}/2$) the angles are 18.35°, 37.03° and 48.45°. Each device switches at seven times the output frequency (350 Hz at 50 Hz) and the 11th becomes the lowest line-voltage harmonic (the leg's triplens cancel between phases). This low switching frequency with a controlled spectrum is why SHE and its optimised relatives run MV drives whose IGCTs and 4.5 kV IGBTs cannot switch faster than a few hundred hertz.

Hysteresis control switches whenever the current touches the edge of a band around its reference; it is fast and inherently current-limiting, but its switching frequency wanders with load, which complicates filter design. **Soft switching** makes each transistor turn on at zero voltage with a resonant tank; it is standard in the isolated DC–DC stages of chargers (Chapter 32), while in hard-switched inverters SiC devices now give most of its benefit without resonant parts.

31.8 Semiconductor selection

Voltage rating

The blocking voltage must cover the highest DC-link voltage (regeneration, surges), the turn-off overshoot $L_\sigma\, di/dt$ (Chapter 29) and cosmic-ray derating: atmospheric neutrons trigger single-event burnout at a rate that rises steeply with DC voltage, so the continuous DC voltage is kept at about 55–70 % of the rated blocking voltage at sea level, less at altitude. The resulting pairings:

Table 31.2 — Device voltage classes and typical applications.

DC link (typical)	Device class	Devices	Application
12–60 V	40–150 V	Si MOSFET, GaN HEMT	Automotive auxiliaries, telecom, e-bikes, servers
350–450 V	650 V	Si superjunction MOSFET, Si IGBT, SiC MOSFET, GaN HEMT	Single-phase PV and UPS, 400 V EV traction, PFC
540–800 V	1200 V	Si IGBT, SiC MOSFET	400–480 V drives, 800 V EV traction, three-phase PV
900–1100 V	1700 V	Si IGBT, SiC MOSFET	690 V drives and wind converters, 1500 V PV (three-level)
1.8–3.6 kV	3.3–6.5 kV	Si IGBT, IGCT, SiC (emerging)	MV drives, traction at 1.5–3 kV DC, MMC cells
Line-commutated	1.6–8.5 kV	Phase-control thyristors, rectifier diodes	Rectifiers, DC drives, LCC HVDC valves

Current rating and losses in an inverter leg

The current rating follows from losses and temperature, not from the nameplate current. In a sinusoidal PWM inverter the IGBT and diode of each switch position share the current according to the modulation index and load power factor. Modelling each device's on-state characteristic as a threshold plus a resistance ($V_{CE} = V_{CE0} + r_{CE} i$, $V_F = V_{F0} + r_F i$, Chapter 29) and integrating over a fundamental cycle gives the conduction losses

$$P_{c,T} = V_{CE0}\hat{I}\left(\frac{1}{2\pi} + \frac{M\cos\varphi}{8}\right) + r_{CE}\hat{I}^2\left(\frac{1}{8} + \frac{M\cos\varphi}{3\pi}\right)$$

$$P_{c,D} = V_{F0}\hat{I}\left(\frac{1}{2\pi} - \frac{M\cos\varphi}{8}\right) + r_F\hat{I}^2\left(\frac{1}{8} - \frac{M\cos\varphi}{3\pi}\right)$$

and, scaling the datasheet switching energies linearly with current and voltage, the switching losses

$$P_{s,T} = \frac{f_s}{\pi}\left(E_{on} + E_{off}\right)\frac{\hat{I}}{I_{ref}}\frac{V_{dc}}{V_{ref}}, \qquad P_{s,D} = \frac{f_s}{\pi}E_{rec}\frac{\hat{I}}{I_{ref}}\frac{V_{dc}}{V_{ref}}$$

> **Status of the principle: Averaged inverter-leg loss model**
> *Approximate.* Assumes sinusoidal current, carrier frequency much higher than output frequency, piecewise-linear on-state curves and switching energy proportional to current and voltage at a fixed junction temperature; it underestimates loss at very low output frequency (where the junction temperature follows each half-cycle), in DPWM (where a different integral applies) and with SiC devices, whose switching energy is not linear in current.

Worked example — loss and junction temperature of a 55 kW drive inverter. A 400 V drive at full load near base speed: $V_{dc} = 560$ V, 100 A RMS ($\hat{I} = 141$ A) at $\cos\varphi = 0.86$, $M = 1.0$, $f_s = 4$ kHz. Three 1200 V, 200 A half-bridge IGBT modules sit on one forced-air heatsink with $R_{th,ha} = 0.08$ K/W in 40 °C air. Listing 31.3 applies the formulas with typical datasheet values at 125 °C.

```
import numpy as np

# Operating point: 55 kW, 400 V drive near base speed (SVPWM, M referred to Vdc/2)
Vdc, Ipk, M, pf, fs = 560.0, 141.0, 1.0, 0.86, 4e3
# 1200 V / 200 A IGBT half-bridge module, typical datasheet values at Tj = 125 C
Vce0, rce, Esw_igbt = 0.85, 5.0e-3, 33e-3     # V, ohm, J (Eon+Eoff at 600 V, 200 A)
Vf0, rf, Erec = 0.90, 4.0e-3, 14e-3           # diode
Vref, Iref = 600.0, 200.0
Rjc_t, Rjc_d, Rch, Rha, Ta = 0.15, 0.25, 0.02, 0.080, 40.0   # K/W, C

a, b = 1 / (2 * np.pi), M * pf / 8
c, d = 1 / 8, M * pf / (3 * np.pi)
P_ct = Vce0 * Ipk * (a + b) + rce * Ipk**2 * (c + d)        # IGBT conduction
P_cd = Vf0 * Ipk * (a - b) + rf * Ipk**2 * (c - d)          # diode conduction
scale = fs / np.pi * (Ipk / Iref) * (Vdc / Vref)
P_st, P_sd = Esw_igbt * scale, Erec * scale                  # switching / recovery
Pt, Pd = P_ct + P_st, P_cd + P_sd
P_inv = 6 * (Pt + Pd)
Ts = Ta + P_inv * Rha                                        # one heatsink, six switches
Tj_t = Ts + Pt * (Rch + Rjc_t)                               # Rch per switch position
Tj_d = Ts + Pd * (Rch + Rjc_d)
print(f"IGBT: cond {P_ct:.1f} W + sw {P_st:.1f} W = {Pt:.1f} W")
print(f"diode: cond {P_cd:.1f} W + rec {P_sd:.1f} W = {Pd:.1f} W")
print(f"inverter loss {P_inv:.0f} W ({100 * P_inv / 55e3:.2f} % of 55 kW)")
print(f"heatsink {Ts:.0f} C, Tj IGBT {Tj_t:.0f} C, Tj diode {Tj_d:.0f} C")
```

```
IGBT: cond 53.5 W + sw 27.6 W = 81.1 W
diode: cond 9.2 W + rec 11.7 W = 21.0 W
inverter loss 612 W (1.11 % of 55 kW)
heatsink 89 C, Tj IGBT 103 C, Tj diode 95 C
```

At 4 kHz the IGBT junction runs at 103 °C, comfortably below a 150 °C rating with margin for overload. Changing only `fs` to 16 kHz quadruples the switching losses: the IGBT loss rises to 164 W, the inverter loss to 1.32 kW, the heatsink to 146 °C and the IGBT junction to 174 °C, beyond its maximum. Holding the junction at its 4 kHz value would require halving the output current, to about 70 A peak (50 A RMS). That is the hospital pump drive of the opening, and it is why drive catalogues publish current derating against switching frequency. A 1200 V SiC MOSFET module, with switching energies a fifth to a tenth of the IGBT's and no diode recovery charge to speak of, would run the same 16 kHz with lower total loss than the IGBT at 4 kHz.

Silicon or wide bandgap

At 650–1700 V the choice between Si IGBTs and SiC MOSFETs is now economic. SiC wins where frequency, efficiency or weight pay: EV traction (3–8 % less loss over a drive cycle), PV, on-board and fast chargers. Si IGBTs stay cheaper per ampere in 2–8 kHz industrial drives, where conduction loss dominates and motors dislike fast edges. GaN HEMTs take the 650 V class at 100 kHz–1 MHz in adapters, server supplies and PFC stages. The material reasons (bandgap, critical field, thermal conductivity) are developed in *Materials Science and Engineering*, Ch. 33; the device consequences in Chapter 5.

31.9 Protection and thermal design

Protection at converter level

Chapter 29 covered protection of the individual switch: desaturation detection, active clamping, snubbers and soft turn-off. A converter adds faults that only appear at system level.

- **Inrush and DC-link precharge.** An uncharged DC link is a short circuit to the rectifier; drives insert a precharge resistor (an NTC in small units, half-controlled thyristors in some) and bypass it once the link is charged.
- **Overcurrent and short circuit.** Si IGBTs withstand a hard short circuit for about 10 µs (specified at a stated V_{CC} and T_j), SiC MOSFETs for 2–3 µs, GaN HEMTs for less than 1 µs. The gate driver's desaturation or current-sense protection must act within that time; software current limits act more slowly on overloads.
- **Shoot-through.** Hardware dead time plus a gate-driver interlock that refuses to turn on both devices of a leg.
- **DC-link overvoltage.** Brake chopper, ramp stretching or crowbar (Chapter 32).
- **Line surges.** Varistors coordinated with the IEC 61000-4-5 surge levels (1.2/50 µs, 2–4 kV line to earth for industrial equipment).
- **Fuses.** Thyristor rectifiers and the input of drives use semiconductor fuses (IEC 60269-4, classes aR and gR) whose total clearing I^2t is below the device's surge I^2t rating (given for a 10 ms half-sine), so that the fuse clears an internal short before the silicon melts.
- **Earth faults.** Summation of output currents; PV inverters also monitor insulation resistance and residual current (IEC 62109).

Worked example — precharging a 55 kW drive. The DC link of 4.7 mF charges to 565 V (the no-load peak of a 400 V supply).

- Energy stored: $\tfrac{1}{2} \times 4.7 \times 10^{-3} \times 565^2 = 750$ J. Charging through any resistor from a constant source dissipates the same energy in the resistor, so a precharge resistor must absorb 750 J in one pulse, regardless of its value.
- With $R = 47\ \Omega$ the initial current is $565/47 = 12.0$ A, the time constant $47 \times 4.7 \times 10^{-3} = 0.22$ s, and the link reaches 99 % in $5\tau = 1.1$ s, after which the bypass contactor closes.
- The resistor is therefore chosen for pulse energy (a wirewound or ceramic type rated, for example, 1 kJ for 1 s with a recovery time between pulses), not for continuous power, and the controller limits the number of mains connections per minute so that the resistor cools between them.

Thermal design

The steady-state thermal path from junction through case, interface material and heatsink to ambient is a series network of thermal resistances (*Engineering Physics*, Ch. 14 §14.7), already used in Listing 31.3. Two additional effects decide real designs.

The first is **transient thermal impedance**. Datasheets give $Z_{th,jc}(t)$ as a Foster network with time constants from about 1 ms to 1 s. At high output frequency the junction sees the average loss; at low output frequency (a heavy start at 1–2 Hz, a crane holding a load at zero speed) each device conducts for a whole half-cycle and its peak temperature rises well above the average. Drives therefore derate current below a few hertz, typically to 50–70 % at 0 Hz.

The second is **power cycling**. The junction temperature swing ΔT_j at each load cycle strains the bond wires and the solder layers, whose thermal expansion coefficients differ from silicon's, and they crack and lift after a number of cycles that falls steeply with ΔT_j. Fitted lifetime models of the LESIT type have the form

$$N_f = A\,\Delta T_j^{-\alpha} \exp\!\left(\frac{E_a}{k_B T_{j,m}}\right)$$

with $\alpha \approx 5$ and $T_{j,m}$ the mean junction temperature, so halving the swing from 80 K to 40 K multiplies life by about $2^5 = 32$. Traction, wind and lift converters are sized on power cycling rather than peak temperature; sintered-silver die attach and copper bond wires raise A by an order of magnitude.

> **Status of the principle: Power-cycling lifetime of power modules**
> *Empirical.* LESIT, CIPS and similar models are fits to accelerated test data on specific module technologies; extrapolation outside the tested range of ΔT_j, $T_{j,m}$, heating time and construction is uncertain by factors of two to ten, and mission-profile analysis with rainflow counting is used to apply them to real load histories.

Natural convection serves up to a few hundred watts, forced air reaches $R_{th,ha}$ of 0.02–0.1 K/W, and liquid cold plates (EV traction, wind, MV drives) 0.01 K/W or less. Capacitor life, power cycling and cosmic-ray derating together are why converter ratings are published as derating curves against temperature, altitude, switching frequency and output frequency.

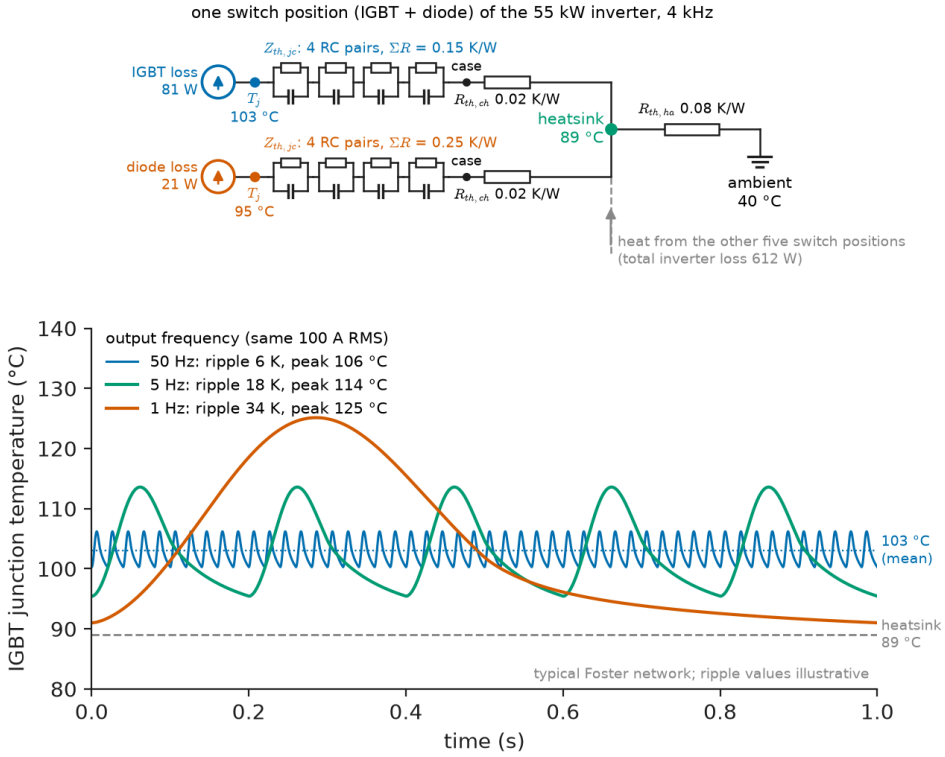


Figure 31.5 — Thermal model of one inverter switch position for the 55 kW drive: Foster network, heatsink and the junction-temperature ripple at low output frequency.

Figure 31.5 shows the thermal network and the junction-temperature ripple growing as output frequency falls.

What remains open

Open question	Where it is answered
How are switching and conduction losses, gate drive and snubbers characterised for a single device?	Chapter 29
How are the buck and boost cells inside PFC stages, chargers and MPPT stages designed?	Chapter 30
How do these converters become drives, chargers and renewable interfaces, and how are reflected waves, bearing currents and grid synchronisation handled in hardware?	Chapter 32
How does a drive controller use the inverter (V/f, FOC, DTC)?	*Electrical Energy and Machines*, Ch. 12
What harmonic limits apply at the point of common coupling, and how are plant-level filters sized?	*Electrical Energy and Machines*, Ch. 25
How is the PWM timer and ADC trigger programmed on a microcontroller, and how is the control ISR scheduled?	Chapter 17, Chapter 27
Why are SiC and GaN switches faster and more rugged at high voltage?	Chapter 5; *Materials Science and Engineering*, Ch. 33

Next: Chapter 32 puts the rectifiers, inverters and filters of this chapter to work in motor drives, battery chargers, PV and wind converters and grid-connected inverters.

Chapter 32 — Motor Drives and Power Conversion

Part VII · Power Electronics

A water utility replaces the starters of its borehole pumps with variable-frequency drives and, within eighteen months, three of the 75 kW submersible motors fail with turn-to-turn insulation breakdown, 300 m of cable below the drive. A logistics company's electric forklifts regenerate on every descent of the warehouse ramp until a firmware update raises the regenerative current, and the battery contactor starts opening on overvoltage when the pack is full. A wind farm's converters ride through a grid fault perfectly in simulation but trip in the field, because the DC-link chopper was sized for 100 ms and the grid code asked for 500 ms. None of these is a control-theory failure. They are failures of the power-electronic hardware that sits between a source and a machine or a grid: the edge rate of an inverter meeting a long cable, the receptivity of a battery meeting a regenerating motor, the energy rating of a resistor meeting a grid code. This chapter assembles the converters of Chapters 29–31 into drives, chargers and renewable converters, and follows each one to the interface where it usually fails.

> **How are rectifiers, inverters and DC–DC stages combined into motor drives, battery chargers and renewable-energy converters, and what does each interface (cable, motor, battery, PV array, grid) demand of the hardware?**
> **How are regeneration, synchronisation and current control implemented at the circuit level, and where do they break?**

Concept	Engineering destination
Drive architectures	Choosing between diode front end, active front end, common DC bus, matrix and multilevel drives
DC motor drive converters	Thyristor DC drive retrofits, automotive and servo H-bridges, battery-vehicle choppers
Induction motor drive interfaces	Cable length limits, dv/dt and sine filters, insulation and bearing protection
Variable-frequency drives	Power module, DC link, current sensing, gate supply, the control ISR, EMC filter
Converter-fed machines	PMSM safe states, BLDC commutation, switched-reluctance converters, high-speed machines
Regenerative operation	Brake choppers, active front ends, common DC buses, EV regeneration limits
Battery charging	CC–CV chargers, EV on-board chargers, DC fast chargers, portable-device chargers
Renewable-energy converters	PV MPPT boost stages, transformerless string inverters, wind back-to-back converters
Grid-connected converter fundamentals	PLL and current-loop implementation, delays, LCL damping, synchronisation sequence

Roadmap. Architectures (32.1) → DC drive converters (32.2) → what a PWM inverter does to a motor and its cable (32.3) → the hardware of a VFD (32.4) → machine-specific converter issues (32.5) → regeneration (32.6) → chargers (32.7) → PV and wind converters (32.8) → grid-connected converter hardware, PLL and current control (32.9).

> **Boundary.**
> Drive mechanics, four-quadrant operation and drive selection are owned by *Electrical Energy and Machines*, Ch. 10; DC drive control by Ch. 11; V/f, FOC, DTC and PMSM control by Ch. 12; battery cells, BMS and state estimation by Ch. 16; PV and wind resources and MPPT algorithms by Ch. 15; harmonic limits by Ch. 25; grid codes, PLL concepts and P–Q control by Ch. 27. This chapter cites those results and supplies the converter hardware; the converter circuits themselves are from Chapter 29, Chapter 30 and Chapter 31.

32.1 Power-electronic drive architectures

A drive is a chain: supply → front end → DC link (or none) → machine-side converter → cable → machine. Each link is chosen from a small menu, and the combinations that survive in practice are summarised in Table 32.1.

Table 32.1 — Power-electronic drive architectures (typical).

Architecture	Front end	Machine side	Quadrants	Typical range and use
Thyristor DC drive	6-pulse thyristor bridge (dual for 4Q)	None (DC armature)	2 or 4	5 kW–10 MW; mill retrofits, excavators
DC chopper drive	DC source or diode bridge	1-, 2- or 4-quadrant chopper	1–4	0.1–50 kW; forklifts, actuators, servo amplifiers
Standard VFD	6-pulse diode bridge, DC choke	2-level IGBT inverter, brake chopper	2 (+braking)	0.2 kW–2 MW; pumps, fans, conveyors
Regenerative VFD	Active front end (IGBT, LCL)	2-level inverter	4	5 kW–5 MW; cranes, lifts, test benches
Common DC bus	One AFE or diode bridge	Several inverters on one bus	4 (internal exchange)	Paper machines, printing lines, CNC cells
EV traction inverter	Battery (400 or 800 V)	2-level SiC or IGBT inverter	4	50–500 kW; cars, buses, trucks
MV drive	Multi-pulse transformer or AFE	3-level NPC, cascaded H-bridge, MMC	2–4	0.5–100 MW; compressors, mills, ship propulsion
Matrix converter	None (direct AC–AC, 9 bidirectional switches)	—	4	5–500 kW; niche, lifts, compact drives
LCI / CSI	Thyristor or PWM current-source rectifier	Thyristor or GTO/IGCT current-source inverter	4	5–100 MW synchronous motors

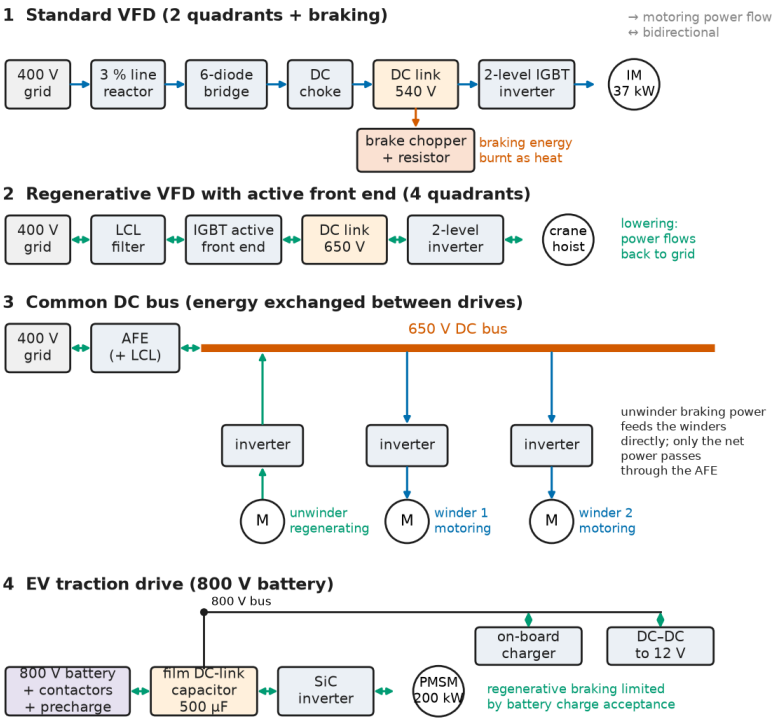


Figure 32.1 — Four drive architectures compared on one page: standard VFD, regenerative VFD, common DC bus and EV traction inverter.

The architecture is decided by three questions. *Does the load regenerate, how often and how much?* (Section 32.6). *What does the supply tolerate?* A six-pulse diode bridge draws 30–45 % current THD, acceptable for one drive on a stiff bus but not for a dozen (*Electrical Energy and Machines*, Ch. 25 §25.3). *What does the machine tolerate?* Fast edges and long cables call for filters or multilevel converters (Section 32.3). Figure 32.1 shows the four architectures that cover most low-voltage applications. The matrix converter, which promises a drive without a DC-link capacitor, has stayed niche because its output voltage is limited to 86.6 % of the input, it has no ride-through energy and its 18 switches cost more than a capacitor bank.

32.2 DC motor drives

Thyristor converters: the firing hardware

The thyristor armature converter and its averaged behaviour, power factor and four-quadrant dual-bridge arrangement are covered in *Electrical Energy and Machines*, Ch. 11 §11.1 and §11.6; the bridge equations, overlap and inversion limit in Section 31.1. What makes a thyristor drive work in hardware is its firing system.

- **Synchronisation.** Firing angles are measured from the natural commutation points of the supply. Old drives derived them from a ramp reset at each zero crossing of a filtered line voltage (the "cosine-crossing" or ramp-comparator method); modern digital drives run a PLL on the measured line voltages (Section 32.9) so that the timing survives notched, distorted and frequency-varying supplies.
- **Gate pulses.** A phase-control thyristor needs a gate current well above its I_{GT} (typically 100–200 mA for a 100–500 A device) with a fast rise (about 1 A/µs) to spread conduction across the die and avoid di/dt failure. The gate is driven through a **pulse transformer** for isolation, with a **pulse train** (for example 10 kHz bursts for 120°) rather than a single pulse, so that the thyristor still turns on if the anode current is momentarily below its latching current, as happens with an inductive load or in discontinuous conduction.
- **Double pulsing.** In a six-pulse bridge two thyristors (one upper, one lower) must conduct to start current; each thyristor is fired again 60° after its main pulse so that the bridge can start from zero current.

- **Dual-converter logic.** In a circulating-current-free dual converter, current-zero detection must confirm that the conducting bridge has stopped before the other is released, with a dead interval of a few milliseconds; a false zero detection fires both bridges into a supply short circuit.

Choppers and H-bridges

Below about 50 kW and wherever the source is DC, the armature supply is a chopper (*Electrical Energy and Machines*, Ch. 11 §11.3, for the quadrants, averaged model and ripple). The hardware is the half-bridge or H-bridge of Section 31.2 built from MOSFETs at 12–80 V and IGBTs or SiC above. In an H-bridge for a brushed motor, one leg is usually held static (selecting direction) while the other is PWM-switched, which halves the switching loss compared with switching both legs.

Worked example — an automotive H-bridge. A 12 V brushed-DC actuator (an electric parking brake or seat motor; supply 9–16 V, 14 V nominal) draws 30 A at stall-limited peak. Four 40 V MOSFETs with $R_{DS(on)} = 1.0$ mΩ at 25 °C (about 1.6 mΩ at 125 °C) switch at 20 kHz, above audibility, with 100 ns rise and fall times.

- Conduction: two MOSFETs carry the current at every instant, $2 \times 30^2 \times 1.6 \times 10^{-3} = 2.9$ W.
- Switching, one leg switching: $\tfrac{1}{2} \times 14 \times 30 \times (200 \times 10^{-9}) \times 20\,000 = 0.84$ W.
- Gate drive: with $Q_g = 100$ nC at 10 V, two gates switching give $2 \times 100 \times 10^{-9} \times 10 \times 20\,000 = 0.04$ W, negligible.

About 3.8 W in four small packages is removable through a PCB with thermal vias into the aluminium housing. The design constraints that remain are automotive ones: reverse-battery protection (a series MOSFET or a high-side driver that blocks reverse current), load-dump surges of tens of volts per ISO 16750-2 (clamped by a TVS diode), and an AEC-Q101-qualified MOSFET. Raising the switching frequency further would buy nothing: the motor's armature inductance already keeps ripple small, and switching loss would grow.

32.3 Induction motor drive interfaces

What the inverter's edges do to the motor

A PWM inverter delivers voltage pulses with rise times of 50–200 ns (IGBT) or 10–50 ns (SiC). To such edges the motor cable is a transmission line (*Circuits, Signals and Communications*, Ch. 14 §14.5, for the reflection coefficient). The cable's characteristic impedance is 30–100 Ω; the motor's high-frequency impedance is several hundred to a few thousand ohms, so the reflection coefficient at the motor is $\Gamma_m = 0.8$–0.95, while the inverter end, a low impedance, reflects with $\Gamma_s \approx -1$. Once the cable's one-way propagation time exceeds about half the rise time, the reflected wave arrives before the edge has finished and the motor terminal voltage approaches

$$\hat{V}_m \approx V_{dc}\,(1 + \Gamma_m)$$

The cable length at which this happens is the **critical length**

$$l_c \approx \frac{v\,t_r}{2}$$

with $v \approx 150$ m/µs for typical PVC and XLPE motor cables.

> **Status of the principle: Reflected-wave overvoltage**
> *Approximate.* A lossless-line model with a resistive termination; cable losses damp the ringing on very long runs, the motor's capacitance shapes the waveform, and two successive edges closer together than the ring-down time (minimum pulse width, polarity reversal) can push the peak toward $3V_{dc}$, which is why drives enforce a minimum pulse spacing.

Worked example — the borehole pumps. The drives of the opening run at $V_{dc} = 560$ V with IGBT rise times of 100 ns into 300 m of cable.

- Critical length: $150 \times 0.1/2 = 7.5$ m, so at 300 m full reflection is certain; with SiC at 20 ns it would be 1.5 m.
- Motor terminal peak with $\Gamma_m = 0.85$: $560 \times 1.85 = 1036$ V per edge, more with double pulsing. NEMA MG 1 Part 31 inverter-duty motors at 600 V and below are specified for 1600 V peak with rise times of at least 0.1 µs, and IEC TS 60034-25 gives similar limit curves; the submersible motors, rewound to an old specification, were not inverter-duty, and partial discharge in the first turns eroded the enamel.
- The same arithmetic for a 690 V drive ($V_{dc} \approx 930$ V) gives 1720 V, above the 1600 V class even for inverter-duty motors: 690 V drives on long cables need filters or reinforced insulation as a rule.
- A **dv/dt filter** (a series reactor of about 0.5–1 % with an RC network) that limits the edge to 500 V/µs stretches the rise time to $560/500 = 1.12$ µs, raising the critical length to 84 m and keeping the peak well below 1.2 kV on most cable runs; for 300 m of cable, a full **sine filter** (an LC low-pass at about one-sixth of the switching frequency) is the robust choice, at the cost of a few per cent voltage drop and of the drive's ability to identify the motor at standstill.

Common-mode voltage and bearing currents

The three phase voltages of a two-level inverter never sum to zero: their average, the **common-mode voltage** $v_{cm} = (v_{a0} + v_{b0} + v_{c0})/3$, steps by $V_{dc}/3$ (187 V on a 560 V link) at every switching edge and swings between $\pm V_{dc}/2$ during zero vectors. It drives current through every stray capacitance from winding to earth. With 5 nF of cable-plus-winding capacitance and a 100 ns edge, each 187 V step pushes a pulse of about $5 \times 10^{-9} \times 187/10^{-7} \approx 9$ A peak into the protective earth, which is why drives need careful earthing and why residual-current devices of type B (IEC 60755) with high-frequency immunity are specified upstream.

Inside the motor, the stator winding couples capacitively to the rotor (C_{wr}), the rotor to the frame (C_{rf}) and the bearings form small capacitors (C_b) while their lubricating film is intact. The shaft voltage is a capacitive divider of the common-mode voltage, set by the **bearing voltage ratio**

$$\mathrm{BVR} = \frac{C_{wr}}{C_{wr} + C_{rf} + 2C_b}$$

typically 2–10 %. With $C_{wr} = 100$ pF, $C_{rf} = 1.5$ nF and $C_b = 150$ pF, $\mathrm{BVR} = 5.3\,\%$ and a 280 V common-mode peak puts about 15 V across each bearing, enough to break down a lubricant film of a fraction of a micrometre. The discharge (**EDM bearing current**) melts microscopic craters, and after months the races show the washboard **fluting** pattern. Large motors (frame sizes above about 280, roughly 100 kW and up) also suffer **circulating bearing currents** driven by high-frequency flux around the shaft, and poorly earthed motors **rotor-earth currents** through the bearings into the driven machine.

> **Status of the principle: Bearing-voltage capacitive divider**
> *Approximate.* Treats the motor's internal coupling as lumped capacitances and the bearing as a capacitor until breakdown; film thickness varies with speed, temperature and load, so breakdown is statistical, and circulating and rotor-earth currents, which the divider does not describe, dominate in large motors.

The mitigation hierarchy is: symmetrical shielded cable with 360° bonding at both ends (gives the common-mode current a low-impedance path home); a shaft-grounding ring or brush on small and medium motors; an insulated non-drive-end bearing (or ceramic hybrid bearings) on large ones; common-mode chokes (nanocrystalline cores on the output) to slow the common-mode edges; and, at the root, three-level inverters, whose common-mode steps are half as large, or sine filters with a common-mode return path.

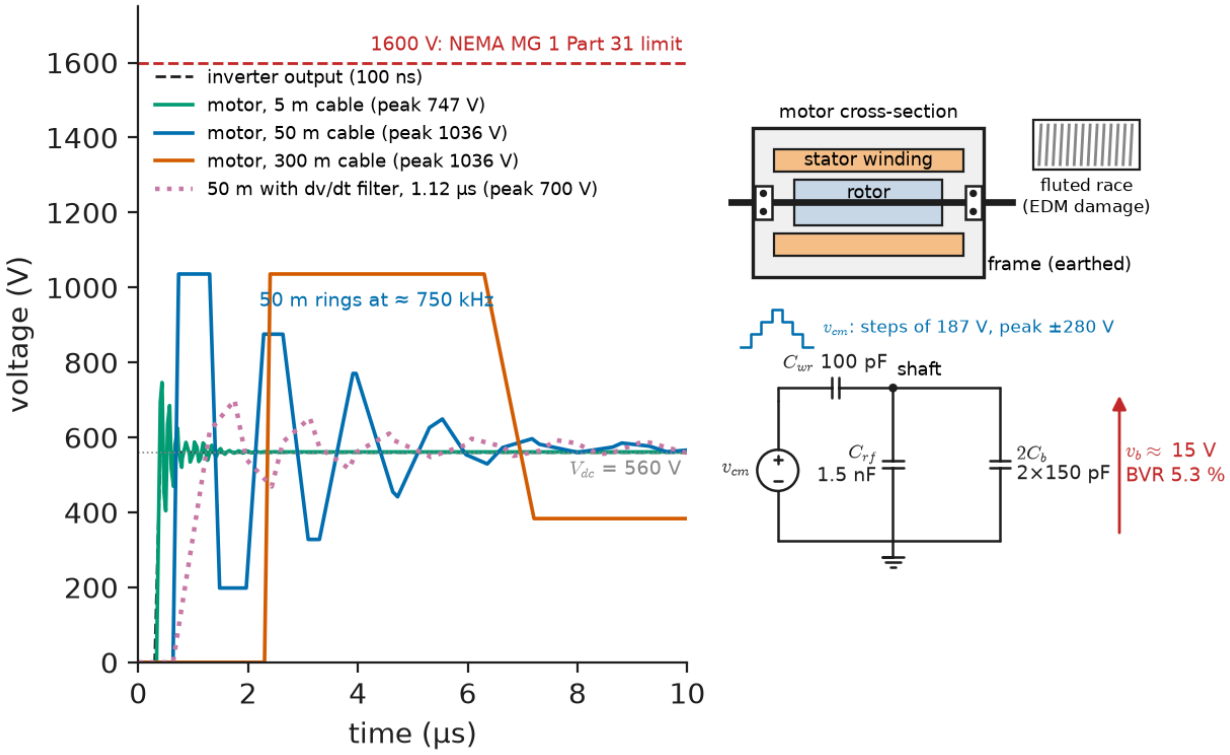


Figure 32.2 — Reflected-wave overvoltage at the motor terminals and the bearing-voltage capacitive divider.

Figure 32.2 shows both effects for the borehole pump: the terminal overshoot rising with cable length, the dv/dt-filtered edge, and the capacitive divider that places a few per cent of the common-mode voltage across the bearings.

32.4 Variable-frequency drives

The hardware

The VFD's power circuit, ratings and control modes are described in *Electrical Energy and Machines*, Ch. 12 §12.1. Opened up, a 37 kW drive contains:

- **Input stage**: EMC filter (X and Y capacitors, common-mode choke) to IEC 61800-3 category C2 or C3, varistors, a diode or half-controlled thyristor bridge, precharge resistor and bypass relay (Section 31.9), and a DC choke of 2–4 % or an AC reactor.
- **DC link**: electrolytic capacitors (series pairs, Section 31.4) of 50–150 µF/kW, or in **slim DC-link** drives film capacitors of only 2–10 µF/kW, which leaves the link voltage with 300 Hz ripple but draws a line current closer to the ideal six-pulse blocks (THD near 30 % without a choke).
- **Inverter**: a six-pack or three half-bridge IGBT modules with an NTC thermistor, on a forced-air heatsink; brake chopper IGBT in the same module.
- **Gate drive**: isolated gate drivers (optocoupler, capacitive or magnetic isolation, rated for common-mode transient immunity above 50 kV/µs), each with a desaturation detector, active Miller clamp and soft turn-off; isolated supplies at +15/−8 V for IGBTs or +18/−3 V for SiC, often a bootstrap supply for the lower devices and a multi-winding flyback for the upper ones.
- **Sensing**: two or three output currents (Hall or fluxgate sensors above about 30 A, shunts with isolated sigma–delta modulators below), the DC-link voltage through a resistive divider and isolated amplifier, heatsink temperature, and an encoder or resolver interface when fitted.
- **Control board**: a microcontroller or DSP (a Cortex-M4/M7 at 150–600 MHz, or a C2000-class DSP), with the PWM timer, ADCs triggered by the timer, fieldbus interfaces and a separate safety channel for STO that removes gate-driver power in hardware.

The control interrupt

Everything the drive does is synchronised to the PWM carrier. The timer's centre-aligned counter (Section 31.3) triggers the ADCs at its peak, where the upper switches of all legs are off for the zero vector 000: low-side shunts then carry the phase currents, and the switching ripple of an inductive load passes through its average at exactly that instant. The conversion-complete interrupt runs the control law, which belongs to the drive (*Electrical Energy and Machines*, Ch. 12 §12.3–12.8), then corrects for dead time and writes the new compare values, which the timer loads at the next period boundary. Listing 32.1 is that interrupt, reusing the modulator of Listing 31.1.

```
 #include <stdint.h>
#include "pwm.h"     /* pwm_cmp_t and svpwm_compare() from Listing 31.1 */

#define TD_S       1.0e-6f    /* dead time, s */
#define FS_HZ      8000.0f    /* switching frequency, Hz */
#define I_BAND     0.5f       /* A: current sign linearised inside this band */
#define PWM_PERIOD 10500u     /* 168 MHz timer clock / (2 x 8 kHz) */

enum { ADC_IA, ADC_IB, ADC_VDC };
extern float adc_amps(int ch);
extern float adc_volts(int ch);
extern void  control_update(float ia, float ib, float vdc,
                            float *va, float *vb, float *vc);
extern volatile uint32_t TIM1_CCR[3];   /* illustrative compare registers */

/* Volt-second correction for one leg, added to its voltage reference. */
static inline float dt_comp(float i, float vdc)
{
    float s = i / I_BAND;
    if (s > 1.0f)  s = 1.0f;
    if (s < -1.0f) s = -1.0f;
    return s * TD_S * FS_HZ * vdc;
}

void adc_pwm_peak_isr(void)          /* triggered at the counter peak */
{
    float ia = adc_amps(ADC_IA), ib = adc_amps(ADC_IB), ic = -ia - ib;
    float vdc = adc_volts(ADC_VDC);
    float va, vb, vc;

    control_update(ia, ib, vdc, &va, &vb, &vc);   /* V/f or FOC */
    va += dt_comp(ia, vdc);
    vb += dt_comp(ib, vdc);
    vc += dt_comp(ic, vdc);

    pwm_cmp_t c = svpwm_compare(va, vb, vc, vdc, (uint16_t)PWM_PERIOD);
    for (int k = 0; k < 3; k++) TIM1_CCR[k] = c.ccr[k];
}
```

The compensation term is the dead-time error of Section 31.2 with the opposite sign. Near current zero the sign is uncertain (ripple crosses zero several times per period), so the listing linearises it over a small band instead of switching abruptly, which would itself inject distortion. The whole interrupt must finish well inside one period (125 µs at 8 kHz); its timing budget, priority and jitter are real-time-system problems treated in Chapter 27. The latency from sampling to new PWM output, about one period (half a period until the new compare values load at the counter zero, plus half a period because the new duty's volt-seconds are centred in the following period), is the dominant delay of every current loop (Section 32.9).

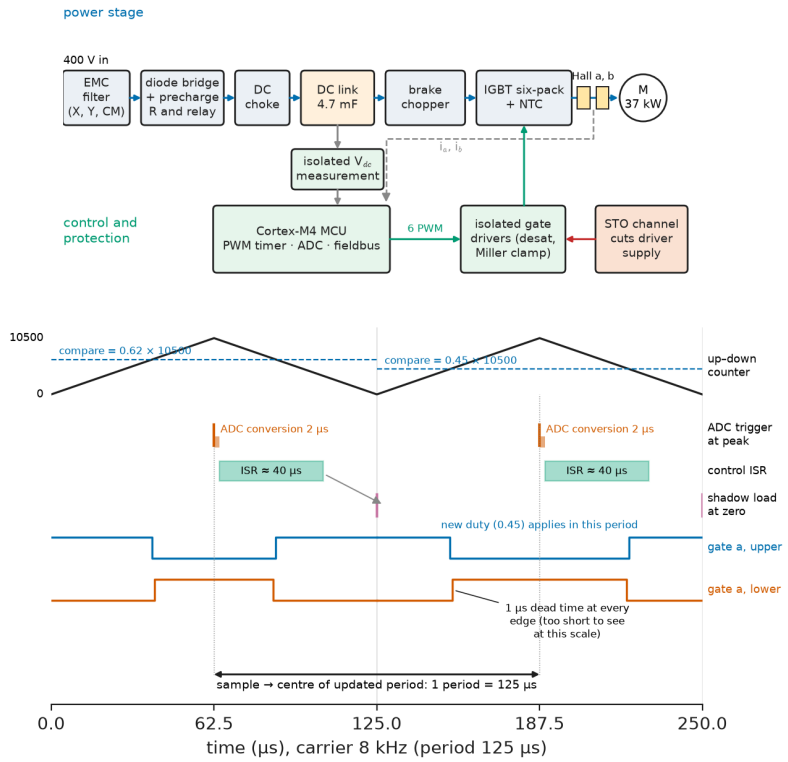


Figure 32.3 — Inside a 37 kW VFD: hardware blocks and the timing of one control period at 8 kHz.

Figure 32.3 places the blocks and the interrupt timing side by side.

Ride-through

A drive's DC link stores little energy relative to its power. When the supply dips, the inverter keeps drawing power and the link voltage falls toward the undervoltage trip.

Worked example — how long does a 37 kW drive ride through? The DC link is 4.7 mF at 540 V and the undervoltage trip is 430 V. The usable energy is $\frac{1}{2} \times 4.7 \times 10^{-3} \times (540^2 - 430^2) = 251$ J, which supplies 37 kW for only 6.8 ms. A 100 ms voltage sag, the most common disturbance on industrial networks (*Electrical Energy and Machines*, Ch. 25 §25.6), would trip the drive. **Kinetic buffering** solves it: on detecting the dip the controller lowers the output frequency slightly so that the motor regenerates from the load's inertia and holds the link at a setpoint near 480 V, trading speed for time; a fan or pump loses a few per cent of speed and rides through a sag of several hundred milliseconds. Drives on processes that cannot tolerate speed loss use a DC-link supercapacitor or battery module instead.

32.5 Converter-fed machines

Each machine type asks something specific of the converter.

Permanent-magnet synchronous machines. A PMSM generates a back-EMF whenever it turns, whether or not the inverter is switching. If the gates are turned off at high speed, the inverter's six diodes form a rectifier, and once the motor's peak line-to-line EMF exceeds the DC-link voltage, current flows uncontrolled into the link: **uncontrolled generator mode** (UCG). The alternative safe state is the **active short circuit** (ASC), which turns on all three lower (or upper) switches so that the windings are shorted; the short-circuit current settles at about ψ_{PM}/L_d, independent of speed at high speed, and the braking torque is small there.

Worked example — the safe state of a traction motor. A PMSM with 4 pole pairs, magnet flux linkage $\psi_{PM} = 0.08$ Wb (peak, per phase) and $L_d = 0.3$ mH runs from a 400 V battery to 14 000 rpm.

- The peak line-to-line EMF equals $\sqrt{3}\,\omega_e \psi_{PM}$. It reaches 400 V at $\omega_e = 400/(\sqrt{3} \times 0.08) = 2887$ rad/s, which is $2887/4 \times 60/(2\pi) = 6890$ rpm.
- At 14 000 rpm ($\omega_e = 5864$ rad/s) the EMF is 813 V peak line to line, twice the battery voltage. Opening the gates there would push a large uncontrolled current into the battery and the DC-link capacitor, and if the battery contactor is also open, the capacitor charges toward 813 V, beyond the 650 V rating of its devices.
- ASC current: $0.08/0.3 \times 10^{-3} = 267$ A peak, below the inverter's 300 A rating, so the windings and switches can carry it indefinitely at high speed.

The fault strategy is therefore speed-dependent: freewheel (gates off) below about 6000 rpm, ASC above, with a hardware path that applies ASC even if the controller fails. This is the emergency-stop problem of *Electrical Energy and Machines*, Ch. 12, seen from the converter side, and it is one of the safety goals (typically ASIL C or D under ISO 26262) of every traction inverter.

Brushless DC (BLDC) motors with trapezoidal back-EMF use **six-step commutation**: three Hall sensors (or back-EMF zero-crossing detection on the floating phase) select which two phases conduct in each 60° sector, and PWM on one switch sets the current. The hardware is the three-phase bridge with a simpler modulator; fans, pumps, drones and power tools use it by the hundred million.

Switched-reluctance motors need unipolar current and are fed from an **asymmetric half-bridge** per phase (two switches, two diodes), which cannot shoot through (each switch is in series with the winding) and gives each phase independent control; their torque ripple and acoustic noise confine them to rugged applications (mining, some appliances, a few EVs).

High-speed machines (spindles at 30 000–100 000 rpm, compressors, turbo-generators) have fundamental frequencies of 1–3 kHz. A carrier ratio of at least 10–20 then forces switching frequencies of 20–60 kHz, which is SiC territory, and their low inductance makes current ripple large unless an output filter or a multilevel inverter is used.

32.6 Regenerative operation

Where braking energy goes, and the system-level choice between dissipation and regeneration, is set out in *Electrical Energy and Machines*, Ch. 10 §10.4. Here is the hardware of each option.

The brake chopper

A brake chopper is an IGBT and a resistor in series across the DC link, with a freewheeling diode across the resistor's inductance. A comparator with hysteresis turns it on when the link rises above a threshold (750 V on a 400 V drive) and off a few tens of volts lower; some drives modulate it with PWM proportional to the excess voltage instead. Its design has three numbers: the resistance (from the peak power at the threshold voltage), the IGBT current, and the resistor's energy and duty rating.

Worked example — a crane hoist chopper. A 22 kW hoist lowers its rated load for 15 s in every 120 s cycle, with peak braking power up to 150 % (33 kW) during deceleration.

- Resistance: $R \le 750^2/33\,000 = 17\ \Omega$. With 17 Ω the chopper absorbs 33.1 kW at 750 V and carries $750/17 = 44$ A, so a 1200 V, 50–75 A IGBT (often the seventh switch in the inverter module) suffices.
- Energy per lowering: $22 \times 15 = 330$ kJ; average power $22 \times 15/120 = 2.75$ kW.
- The resistor is specified for 2.75 kW continuous *and* for a 330 kJ, 15 s pulse every 120 s; a resistor rated only for average power would overheat during each pulse, and one rated for 22 kW continuous would be eight times larger than necessary.

The active front end

An active front end is the three-phase inverter of Chapter 31 connected to the grid through an L or LCL filter and run as a **boost rectifier**: it controls its AC currents to be sinusoidal and in phase (or antiphase, when regenerating) with the grid voltage, and regulates the DC link above the peak line voltage. Because its PWM can only synthesise line-to-line voltages up to V_{dc}, a 400 V AFE needs at least 566 V on its link plus the filter drop; at 650 V it can synthesise up to $650/\sqrt{2} = 460$ V line, a 15 % margin for the inductor drop and grid overvoltage. The control is that of the grid-connected converter of Section 32.9 with an outer DC-voltage loop. An AFE costs roughly 1.5–2 times a diode front end and adds 1–1.5 % loss, so it is chosen where regeneration is frequent (lifts, cranes, centrifuges, test benches) or harmonic limits are tight (3–5 % THD).

Common DC bus and battery receptivity

A **common DC bus** connects several inverters to one link, so that a braking machine (an unwinder) feeds a motoring one (a winder) directly; only the net power passes through the front end. In battery systems the receiver is the battery, whose **charge acceptance** falls when it is full, cold or near its voltage limit (*Electrical Energy and Machines*, Ch. 16 §16.3). The forklift of the opening regenerated into a full pack: its BMS saw cell overvoltage and opened the contactor, leaving the inverter regenerating into a capacitor alone. The correct arrangement is a regenerative current limit computed from the BMS's live charge-power limit, with a brake chopper or mechanical brake taking the rest, and an inverter strategy that never regenerates into an open contactor.

32.7 Battery charging

CC–CV charging

Lithium-ion cells are charged **constant current, then constant voltage** (CC–CV): the charger supplies a set current (0.5–1C for most cells, 2–4C for fast-charge cells) until the cell voltage reaches its limit (4.2 V for NMC, 3.65 V for LFP), then holds that voltage while the current decays, and terminates when the current falls to a set fraction (C/10 to C/20). A deeply discharged cell (below about 3.0 V for NMC) is first **precharged** at C/10 so that a damaged cell is not forced, and a timer and a temperature window (typically 0–45 °C on charge) abort the charge. The cell physics and the limits come from *Electrical Energy and Machines*, Ch. 16 §16.1–16.3; the charger enforces them.

In hardware, CC–CV is two error amplifiers, one for current and one for voltage, whose outputs are combined by a **minimum selector** (diode-OR in analogue chargers, a `min()` in digital ones) before driving the PWM stage: whichever loop demands less output controls, so the transition from CC to CV is automatic and bumpless. Firmware supervises the states around it, as in Listing 32.2.

```
#include <stdbool.h>

typedef enum { CHG_PRECHARGE, CHG_CC, CHG_CV, CHG_DONE, CHG_FAULT } chg_state_t;

typedef struct {
    float v_pre, v_cv, v_abort;   /* V per cell: precharge exit, CV set, abort */
    float i_pre, i_cc, i_term;    /* A: precharge, constant current, termination */
    float t_max_s;                /* safety timer, s */
} chg_cfg_t;

/* Called every 100 ms. Returns the current limit for the power stage, whose
   inner voltage loop is always set to v_cv; the minimum selector in the
   stage makes the voltage loop take over automatically in CV. */
float chg_step(const chg_cfg_t *c, chg_state_t *s, float v, float i,
               bool temp_ok, float *t_s)
{
    *t_s += 0.1f;
    if (!temp_ok || v > c->v_abort || *t_s > c->t_max_s)
        *s = CHG_FAULT;

    switch (*s) {
    case CHG_PRECHARGE:
        if (v >= c->v_pre) *s = CHG_CC;
        return c->i_pre;
    case CHG_CC:
        if (v >= c->v_cv) *s = CHG_CV;
        return c->i_cc;
    case CHG_CV:
        if (i <= c->i_term) *s = CHG_DONE;
        return c->i_cc;           /* voltage loop is now limiting */
    default:                      /* CHG_DONE, CHG_FAULT */
        return 0.0f;
    }
}
```

The firmware never commands the CV phase directly; it only decides when charging may start and must stop. That division (fast analogue or fixed-function loops, slow supervisory firmware) is typical of every safe power-electronic product.

Worked example — CC–CV timing of a phone cell. A 3 Ah NMC cell is charged at 1C (3 A) to 4.2 V and terminated at C/20 (0.15 A). Model its open-circuit voltage as rising linearly from 3.3 V (empty) to 4.2 V (full), with a lumped resistance of 50 mΩ.

- CC ends when $\mathrm{OCV} + IR = 4.2$ V, i.e. at an OCV of $4.2 - 0.15 = 4.05$ V, a state of charge of $0.75/0.9 = 83\,\%$. From 10 % this takes 0.733×3 Ah$/3$ A $= 0.733$ h, 44 min.
- In CV, $I = (4.2 - \mathrm{OCV})/R$ decays exponentially with time constant $\tau = RQ/(dV_{oc}/ds) = 0.05 \times 10\,800/0.9 = 600$ s. Falling from 3 A to 0.15 A takes $\tau \ln 20 = 1800$ s, 30 min, to add the last 17 %.

The CV phase takes 40 % of the time for 17 % of the charge, the familiar slow finish of every phone. Fast-charge protocols raise the CC current, not the CV voltage, and split the CC phase into steps that follow the cell's lithium-plating limit.

> **Status of the principle: CC–CV charge timing with a linear-OCV, single-resistance cell**
> *Approximate*. Captures the automatic CC-to-CV transition and the exponential current decay; real cells have a curved OCV, a diffusion time constant and temperature-dependent resistance, so the CV phase is usually longer than predicted and the transition point moves with temperature and age.

EV chargers

An EV has an **on-board charger** (OBC) for AC charging and accepts **DC fast charging** from an off-board charger that bypasses the OBC. Both follow the same two-stage pattern (Figure 32.4):

1. **PFC stage.** A boost converter (Chapter 30) after the rectifier, or a bridgeless **totem-pole** PFC with GaN or SiC switches, draws a sinusoidal input current at unity power factor (IEC 61000-3-2/-12 compliance) and regulates an intermediate bus of 400–800 V.
2. **Isolated DC–DC stage.** An **LLC resonant** converter or a **dual-active-bridge** (DAB) converter with a high-frequency transformer gives galvanic isolation (required by IEC 61851-1 and ISO 6469) and controls the battery current; the DAB is bidirectional, enabling vehicle-to-load and vehicle-to-grid operation.

Typical OBCs are 3.7–7.4 kW single-phase or 11–22 kW three-phase at 94–96 % efficiency; DC fast chargers are 50–400 kW, built from 20–60 kW modules in parallel, with CCS, CHAdeMO, GB/T or NACS connectors and charging communication per ISO 15118 or DIN 70121.

Worked example — a 7.4 kW single-phase OBC. The OBC runs from 230 V, 32 A with a boost PFC at 65 kHz to a 400 V bus, and charges a 77 kWh battery from 10 % to 80 %.

- Input current peak at 7.4 kW drawn from the line: $\sqrt{2} \times 7400/230 = 45.5$ A.
- PFC inductor for a ripple of 20 % of that peak at the line-voltage peak (325 V, duty $1 - 325/400 = 0.187$): $L = 325 \times 0.187/(0.2 \times 45.5 \times 65\,000) = 103\ \mu$H, so a 100 µH inductor is chosen. The ripple is worst where the input is half the output (200 V), $200 \times 0.5/(100 \times 10^{-6} \times 65\,000) = 15.4$ A peak-to-peak.
- Bus capacitor for 40 V peak-to-peak 100 Hz ripple: $7400/(2\pi 50 \times 400 \times 40) = 1.47$ mF; the LLC stage regulates through that ripple.
- Charging time: $0.7 \times 77 = 53.9$ kWh at 92 % wall-to-battery efficiency takes $53.9/(7.4 \times 0.92) = 7.9$ h; an 11 kW three-phase OBC needs 5.3 h, and a 150 kW DC charger roughly 25–30 min, limited by the battery's current acceptance and taper, not by the charger.

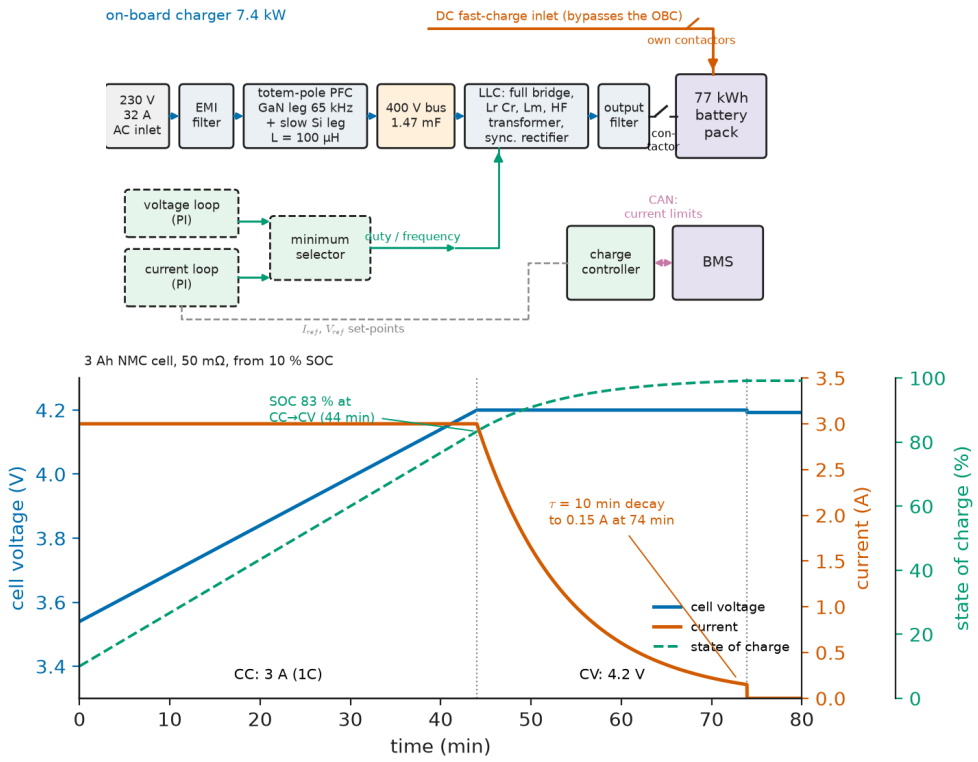


Figure 32.4 — EV on-board charger architecture and the CC-CV charge profile it delivers.

32.8 Renewable-energy converters

PV: the MPPT stage

A PV string's maximum-power voltage varies with temperature and irradiance by tens of per cent (*Electrical Energy and Machines*, Ch. 15 §15.1, which also names the MPPT algorithms: perturb-and-observe and incremental conductance). The converter must present the array with an adjustable input voltage while delivering a DC link fit for the inverter. In a single-phase string inverter, a **boost stage** does this: its input capacitor holds the array voltage, its duty ratio sets that voltage relative to the DC link, and the MPPT algorithm, running every 10–100 ms, perturbs the input-voltage reference by a fixed step and keeps the direction that raised power.

Worked example — the boost stage of a 6 kW string inverter. Twelve modules give a maximum-power voltage of 372 V at 25 °C, falling to about 330 V on a hot afternoon. The H-bridge needs a 400 V link to make 230 V.

- At 330 V the duty ratio is $1 - 330/400 = 0.175$; the input current is $6000/330 = 18.2$ A.
- Inductor for a 30 % ripple (5.45 A) at 20 kHz: $L = 330 \times 0.175/(5.45 \times 20\,000) = 0.53$ mH.
- On a cold, bright morning the array sits above 400 V and the boost stage simply conducts through its diode (or bypass relay), the H-bridge's own modulation then tracking the MPP; many inverters therefore run a variable DC link.

Three hardware details decide how well MPPT works. The input capacitor must keep the 100 Hz ripple coupled back from the single-phase output small (under about 1 % of the array voltage), or the array oscillates around its MPP and loses energy. The perturbation step must be larger than the measurement noise of ΔP but small enough that the steady oscillation costs little (1–3 V on a 400 V string is typical). And a periodic scan of the whole P–V curve is needed to escape the local maxima created by partial shading.

Transformerless PV inverters and leakage current

Removing the line-frequency transformer raises efficiency by 1–2 % but connects the PV array to the grid. A PV array has a capacitance to earth of 50–200 nF per kW (more when wet), and any switching-frequency component in the array's common-mode voltage drives leakage current through it, a safety and EMC hazard: IEC 62109-2 and VDE 0126-1-1 require disconnection when the residual current jumps by 30 mA or exceeds 300 mA. The unipolar H-bridge of Section 31.4 is the problem case; the solutions keep the array's common-mode voltage constant by disconnecting it from the grid during the freewheeling intervals. **H5** adds a fifth switch in the DC link; **HERIC** adds a bidirectional switch across the AC output; the **NPC** and **T-type** three-level legs clamp the output to the grounded neutral during freewheeling. All achieve 97.5–98.5 % European efficiency.

Wind: back-to-back converters

A full-converter (Type 4) wind turbine feeds its permanent-magnet or induction generator through a **back-to-back** converter: a machine-side inverter (run as a drive, controlling generator torque) and a grid-side inverter (an AFE controlling DC-link voltage and grid current), sharing a DC link of 1050–1200 V at 690 V (*Electrical Energy and Machines*, Ch. 27 §27.1, for the voltage sizing). A doubly fed induction generator (Type 3) uses the same back-to-back pair in its rotor circuit, rated for only the slip power, about 30 % of turbine rating, with a crowbar that short-circuits the rotor during grid faults.

During a grid fault the grid-side converter cannot export power, but the generator keeps producing it for the fraction of a second the pitch system needs to respond. The surplus goes into a **DC-link chopper** (brake resistor). Its energy rating follows directly from the grid code's ride-through duration: a 3 MW turbine required to ride through 500 ms at near-zero voltage must dump up to $3 \times 0.5 = 1.5$ MJ per event. The wind farm of the opening had choppers rated for 100 ms, a fifth of that.

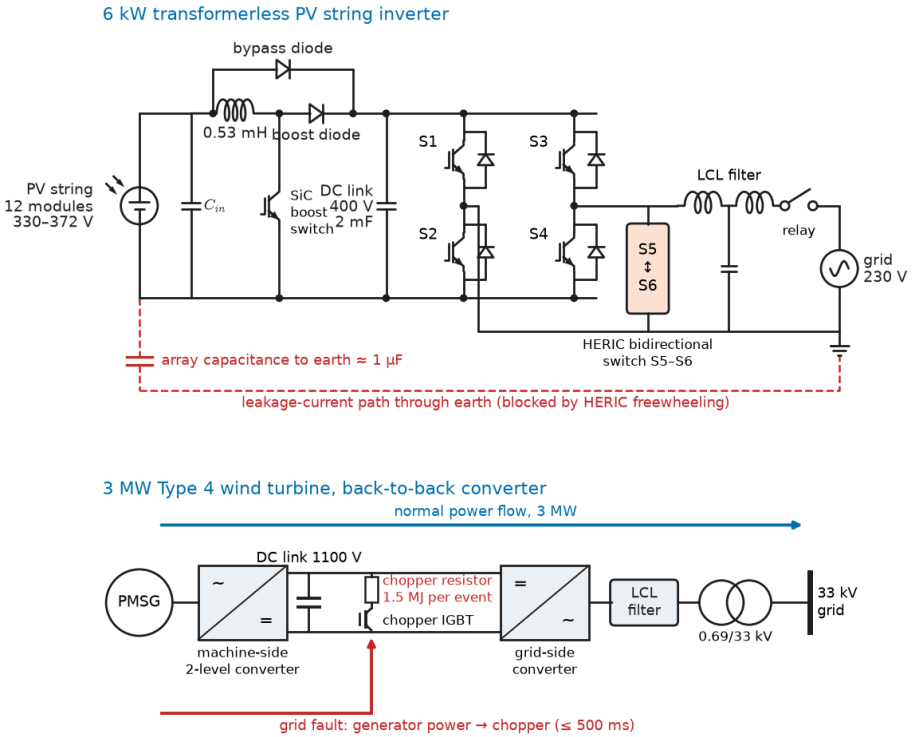


Figure 32.5 — Renewable converter power stages: a 6 kW single-phase transformerless PV inverter with boost MPPT stage and HERIC output, and a 3 MW Type 4 wind back-to-back converter.

Figure 32.5 shows both power stages, including the leakage-current path through the PV array capacitance that the HERIC switch interrupts.

32.9 Grid-connected converter fundamentals

The grid-connected voltage-source converter is the common core of the AFE, the PV and battery inverter, the wind grid-side converter, the STATCOM and the active harmonic filter. Its system behaviour (power-angle relations, P–Q control, grid-following versus grid-forming, grid codes) is owned by *Electrical Energy and Machines*, Ch. 27 §27.1–27.4; its filter by Section 31.6. What remains is the implementation: sensing, synchronisation, current control and start-up.

Synchronisation: the PLL in firmware

The synchronous-reference-frame PLL (*Electrical Energy and Machines*, Ch. 27 §27.2) transforms the measured grid voltages into a frame rotating at the estimated angle, drives the quadrature component v_q to zero with a PI controller and integrates the resulting frequency. With $v_q \approx V_m(\theta_g - \theta)$ for small errors, the loop is second order, and the PI gains for a natural frequency ω_n and damping ζ are $k_p = 2\zeta\omega_n/V_m$ and $k_i = \omega_n^2/V_m$. Listing 32.3 runs a 20 Hz PLL at 10 kHz against a grid that starts 0.5 Hz off nominal and jumps 30° in phase at 150 ms, as during a nearby fault.

```
 import numpy as np

Ts, Vm, f0 = 1e-4, 325.0, 50.0                # 10 kHz sampling, phase peak, nominal f
wn, zeta = 2 * np.pi * 20, 0.707              # PLL design: 20 Hz natural frequency
kp, ki = 2 * zeta * wn / Vm, wn**2 / Vm       # PI gains on vq (rad/s per volt)
t = np.arange(0, 0.3, Ts)
theta_g = 2 * np.pi * 50.5 * t + np.where(t >= 0.15, np.radians(30), 0.0)  # +0.5 Hz, 30 deg jump

theta, integ, err, freq = 0.0, 0.0, [], []
for tg in theta_g:
    va, vb, vc = (Vm * np.cos(tg - k * 2 * np.pi / 3) for k in range(3))
    alpha = (2 * va - vb - vc) / 3                      # Clarke (amplitude-invariant)
    beta = (vb - vc) / np.sqrt(3)
    vq = -alpha * np.sin(theta) + beta * np.cos(theta)   # Park, q-axis
    err.append(np.degrees(np.angle(np.exp(1j * (tg - theta)))))
    integ += ki * vq * Ts
    w = 2 * np.pi * f0 + kp * vq + integ                # estimated angular frequency
    theta = (theta + w * Ts) % (2 * np.pi)
    freq.append(w / (2 * np.pi))

err, freq = np.array(err), np.array(freq)
for t_end in (0.05, 0.149, 0.16, 0.20, 0.30):
    k = int(round(t_end / Ts)) - 1
    print(f"t = {t_end * 1e3:5.0f} ms  phase error {err[k]:6.2f} deg  f = {freq[k]:6.3f} Hz")
settle = t[(np.abs(err) > 2) & (t >= 0.15)].max() - 0.15
print(f"time to stay within 2 deg after the jump: {settle * 1e3:.0f} ms")
```

```
 t =    50 ms  phase error  -0.02 deg  f = 50.496 Hz
t =   149 ms  phase error   0.00 deg  f = 50.500 Hz
t =   160 ms  phase error  -1.68 deg  f = 54.403 Hz
t =   200 ms  phase error   0.24 deg  f = 50.455 Hz
t =   300 ms  phase error   0.00 deg  f = 50.500 Hz
time to stay within 2 deg after the jump: 32 ms
```

The PLL locks to the 0.5 Hz offset with zero steady phase error (the integrator holds the frequency difference) and recovers from the 30° jump within about 32 ms. The price is visible in the frequency estimate: the proportional path kicks it to 64.8 Hz at the instant of the jump and it is still 4 Hz high 10 ms later. A loss-of-mains relay fed with this raw estimate would see an enormous rate of change of frequency and disconnect, which is part of what happened in the 2019 British event described in *Electrical Energy and Machines*, Ch. 27; frequency used for protection or grid support must come from a slower, filtered estimate. Under unbalanced faults the negative-sequence voltage adds a 100 Hz ripple to v_q, and practical PLLs separate the sequences first (DSOGI or decoupled double-frame PLL).

> **Status of the principle: Linearised SRF-PLL model**
> *Approximate.* The small-angle approximation $v_q \approx V_m(\theta_g - \theta)$ makes the PLL a linear second-order loop; it holds for phase errors below about 30° and balanced voltages, and the loop gain changes with grid-voltage magnitude unless v_q is normalised, which is why voltage dips also detune an un-normalised PLL.

Current control at circuit level

In the PLL frame the converter's output currents become DC quantities i_d and i_q, regulated by two PI controllers with cross-coupling compensation $\pm\omega L i$ and grid-voltage feedforward. At frequencies well below the LCL resonance the filter looks like its total inductance $L = L_1 + L_2$ with resistance R, so the plant is $1/(sL + R)$, preceded by the digital delay of sampling, computation and PWM update: about $1.5T_s$ for single-update PWM when the currents are sampled at the start of a period and the new duty applies from the start of the next (T_s with the mid-period sampling of Listing 32.1, Section

32.4). The standard tuning cancels the plant pole with the PI zero ($T_i = L/R$) and chooses the proportional gain for a crossover frequency ω_c, $K_p = \omega_c L$. The delay then costs phase $\omega_c \times 1.5T_s$, and the phase margin is about $90° - \omega_c \times 1.5T_s$.

Worked example — tuning the 15 kW inverter's current loop. With the LCL filter of Section 31.6 ($L = 2.71$ mH, $R \approx 0.1\ \Omega$) and $f_s = 10$ kHz, sampled at the start of each period (delay 150 µs):

- $T_i = L/R = 27$ ms for any bandwidth.
- At $f_c = 1$ kHz: $K_p = 2\pi \times 1000 \times 2.71 \times 10^{-3} = 17.0\ \Omega$, but the delay costs $360° \times 1000 \times 150 \times 10^{-6} = 54°$, leaving a 36° margin: too oscillatory.
- At $f_c = 600$ Hz: $K_p = 10.2\ \Omega$, delay phase 32.4°, margin 57.6°, but only on the $L_1 + L_2$ model. On the full LCL with $R_d = 2\ \Omega$ the loop crosses over at 720 Hz with a 49° margin, and the resonance lifts the gain to +1.3 dB near 1.6 kHz, where the phase is already past −180°: the gain margin (at 1.33 kHz) is 0.3 dB, and without R_d the loop is unstable.
- At $f_c = 300$ Hz: $K_p = 5.1\ \Omega$, margin 74° on the simple model; on the full model, crossover 310 Hz, phase margin 73°, gain margin 6.3 dB. This is the setting adopted.

With an L filter the digital delay alone limits current-loop bandwidth to about $f_s/15$–$f_s/20$; with an LCL filter the resonance must also be checked on the full model, because near $f_s/6$ (1.67 kHz here) the delay has already brought the phase to −180°. Keeping 600 Hz would need active damping. If the resonance falls below about $f_s/6$, the delay makes the loop unstable unless the filter is damped passively (R_d of Section 31.6) or actively by feeding back the filter-capacitor current. Double-update PWM (sampling at both counter peak and valley) halves the delay and lets the bandwidth roughly double.

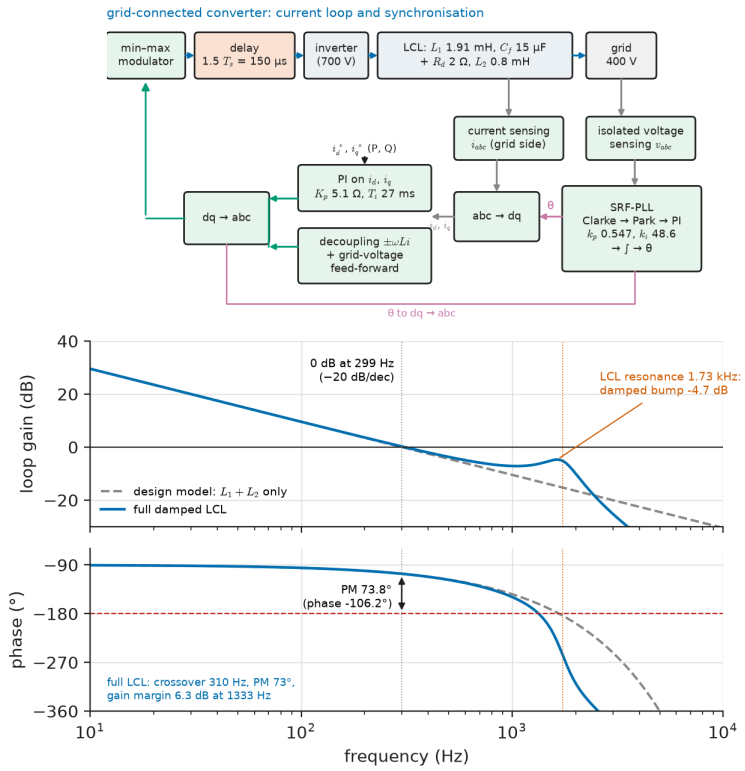


Figure 32.6 — Implementation of the grid-connected converter's current loop and synchronisation, with the open-loop Bode plot of the tuned loop.

Figure 32.6 shows the whole loop as implemented and its open-loop frequency response at the adopted 300 Hz design point.

Sensing, start-up and protection

A grid-connected converter measures the grid voltages before the relay, the converter-side or grid-side currents, and the DC-link voltage, all through isolated sensors so that the control electronics stay at safety-extra-low voltage. Its start-up sequence is fixed by physics and by the grid code: precharge the DC link (from the DC source or through resistors from the grid); run the PLL and verify voltage, frequency and phase sequence within limits for the connection delay the code requires; start PWM with the current references at zero so that the converter matches the grid voltage; close the grid relay; then ramp power at the permitted rate. Anti-islanding (*Electrical Energy and Machines*, Ch. 27 §27.5), ride-through and current limiting run continuously on top. Residual-current monitoring (Section 32.8), relay-weld detection by comparing voltages across two series relays, and an independent overvoltage trip complete the hardware protection that standards such as IEC 62109 and IEEE 1547.1 test.

What remains open

Open question	Where it is answered
How are switching losses, gate drives and snubbers designed for each device?	Chapter 29
How are the PFC boost, LLC and DAB stages of chargers designed in detail?	Chapter 30
How are rectifiers, PWM, LCL filters and semiconductor ratings derived?	Chapter 31
How does the controller decouple flux and torque (V/f, FOC, DTC, MTPA)?	*Electrical Energy and Machines*, Ch. 12
What do grid codes require during faults and frequency events, and how does grid-forming control work?	*Electrical Energy and Machines*, Ch. 27
How are state of charge, state of health and charge-power limits estimated by the BMS?	*Electrical Energy and Machines*, Ch. 16
How is the control ISR scheduled and verified, and how is drive firmware tested in the loop?	Chapter 27, Chapter 28
How are gate-driver and controller ICs themselves designed?	Chapter 33, Chapter 34

Next: Chapter 33 leaves the kilowatt scale for the femtojoule scale, building the CMOS inverter, gates and flip-flops from which every controller in this part is made.

Part VIII — Integrated Circuits and VLSI

Every circuit in this book so far has ended up, sooner or later, inside a chip: the op-amp of Part II, the gates and flip-flops of Part III, the processor and memories of Part IV, the microcontroller that runs the firmware of Part VI and the gate driver and controller of a power converter in Part VII. This part opens the chip. It starts from the complementary MOS inverter, the circuit of which billions of copies make up a modern processor, and shows how its switching threshold, noise margins, power and delay follow from the transistor models of Chapter 4. It then scales up to the design flow that turns a register-transfer description of millions of gates into a placed, routed, timed and verified layout ready for fabrication, and ends at the system level, where an engineer decides which parts of a computation belong in software on a processor and which in dedicated hardware. The thread is the same as in the rest of the book: an abstraction (a logic gate, a standard cell, a synthesised netlist, an accelerator) holds only inside limits of voltage, delay, power and area that the devices beneath it set, and good design is knowing where those limits are.

By the end of this part you should understand

- Why complementary pull-up and pull-down networks give rail-to-rail outputs with no static current, and how the inverter's transfer characteristic sets its switching threshold and noise margins.
- How dynamic power ($\alpha C V^2 f$), short-circuit power and leakage arise, and how voltage scaling, clock gating, power gating and multi-threshold libraries trade them against speed.
- How to estimate gate and wire delay with RC and Elmore models, and how to size a path of gates with logical effort.
- How static CMOS gates, transmission gates, latches and master–slave flip-flops are built and sized at the transistor level, and where setup, hold and clock-to-Q times come from.
- How a schematic becomes a layout through stick diagrams and design rules, and how layout prevents latch-up.
- How a chip is designed hierarchically, from RTL through logic synthesis, placement, routing and clock-tree synthesis to static timing analysis, power estimation, design-rule checking and verification.
- What fabrication imposes on design: process variation and corners, lithography-driven rules, yield and reliability.
- How to partition a computation between processor, FPGA and custom hardware, model its performance and verify the result at system level.

Chapters

Chapter	Central question	What it delivers
Chapter 33 — CMOS Fundamentals	Why does a pair of complementary MOSFETs make an almost ideal logic gate, and what sets its noise margins, power and delay?	Inverter VTC and switching threshold, effective-resistance switch model, dynamic and static power, propagation delay and logical effort, noise margins, static CMOS and transmission-gate logic, latches and flip-flops at transistor level, stick diagrams, lambda rules and latch-up
Chapter 34 — VLSI Design	How is a design of millions of gates taken from an RTL description to a verified layout that meets timing, power and manufacturing constraints?	Design hierarchy, RTL coding for synthesis, logic synthesis, floorplanning, placement and routing, static timing analysis, power estimation, design-rule checking, functional and physical verification, fabrication constraints
Chapter 35 — Hardware–Software Co-Design	Which parts of a computation belong in software and which in hardware, and how is the combined system modelled and verified?	Partitioning methods, processor versus FPGA trade-offs, hardware acceleration, memory and data movement, performance modelling, embedded AI deployment, real-time constraints and system-level verification

How this part connects to the rest of the book

Part VIII stands on the device models of Part I, especially the MOSFET of Chapter 4 and the parasitic thyristor of Chapter 5; on the logic, timing and sequential design of Part III (Chapters 12–15); on the architecture, memories and FPGAs of Part IV (Chapters 16–18) and the hardware description languages of Chapter 19; and, for co-design, on the programming of Part V and the embedded and real-time systems of Part VI.

Three sibling books supply what this part does not re-derive. *Engineering Physics* gives the physics of the MOS transistor and its scaling limits (Ch. 18 §18.3 and §18.9). *Materials Science and Engineering* owns the processes that make the chip, from crystal growth to lithography, etching, copper interconnects and process-induced defects (Ch. 50), and the semiconductor materials themselves (Ch. 32–35). *Circuits, Signals and Communications* provides the transmission-line and interconnect theory behind high-speed wires and clock distribution (Ch. 14), and *The Mathematics Universe* the optimisation behind placement and routing tools. Chapter 35 closes the book by bringing hardware and software back together into one design problem.

Chapter 33 — CMOS Fundamentals

Part VIII · Integrated Circuits and VLSI

A sensor-hub microcontroller is ported from a 180 nm process to 40 nm. Its active current falls by a factor of five, exactly as the datasheet of the new process promised, but its standby current rises twentyfold and the coin-cell product it lives in now lasts four months instead of two years. On another project, a register file that worked at every corner in a 1.2 V process loses data when the supply is scaled to 0.7 V for a low-power mode, because two cross-coupled inverters no longer win the fight against a write transistor. In a third, a clock tree that met timing in synthesis misses it after layout because a single long wire in metal 2 adds more delay than the eight gates it connects. Each of these follows from a handful of circuit facts about the complementary MOS inverter and its relatives: where its switching threshold lies, how much charge it moves per transition, how its delay scales with load and wire, how much noise it can reject, and what geometry the process lets it occupy. This chapter builds those facts from the transistor models of Chapter 4 up to gates, latches, flip-flops and layout.

> **Why does a pair of complementary MOSFETs make an almost ideal logic gate, and what sets its noise margins, power and delay?**
> **How are gates, latches and flip-flops built and sized at the transistor level, and how does a transistor schematic become a layout?**

Concept	Engineering destination
CMOS logic and the inverter VTC	Logic thresholds, regeneration, I/O level compatibility, standard-cell libraries
MOSFET switching	Effective resistance and capacitance models, pass transistors, transmission gates
Static and dynamic power	Battery life, thermal design power of processors, DVFS, power and clock gating
Propagation delay	Gate sizing, buffer chains, logical effort, wire delay and repeaters
Noise margins	Supply-voltage scaling limits, crosstalk and ground-bounce budgets
CMOS gates	NAND, NOR, AOI/OAI, multiplexers, tristate drivers in standard-cell libraries
Latches and flip-flops	Register design, setup and hold times, clock-to-Q delay, metastability
Layout fundamentals	Stick diagrams, lambda rules, cell area, latch-up prevention

Roadmap. The complementary principle and the inverter's transfer characteristic (33.1) → the MOSFET as a switch with resistance and capacitance (33.2) → where the power goes (33.3) → how long a transition takes, and how to size a path (33.4) → how much noise a gate tolerates (33.5) → static CMOS and transmission-gate logic (33.6) → latches and flip-flops at transistor level (33.7) → from schematic to layout (33.8).

> **Boundary.**
> MOSFET structure, threshold voltage, square-law and short-channel models, body effect and capacitances are developed in Chapter 4, from the physics sketch of *Engineering Physics*, Ch. 18 §18.3.3, which already states the CMOS inverter's static and dynamic power in one line. Boolean algebra and gates as logic are in Chapter 12, latches and flip-flops as logic elements with their timing constraints in Chapter 14, and metastability and synchronisers in Chapter 15. The fabrication steps behind each layout layer (oxidation, implantation, lithography, etching, metallisation) are owned by *Materials Science and Engineering*, Ch. 50. Chip-level timing analysis, power estimation and design-rule checking belong to Chapter 34.

33.1 CMOS logic

The complementary principle

Chapter 4 showed that an n-channel MOSFET conducts when its gate is high relative to its source and a p-channel MOSFET when its gate is low. A **CMOS** gate exploits this by building every logic function from two networks: a **pull-down network** (PDN) of nMOS transistors between the output and ground, and a **pull-up network** (PUN) of pMOS transistors between the supply V_{DD} and the output. The two networks are logical complements: for every input combination exactly one of them conducts. The output is therefore always connected, through a conducting path, to exactly one rail, and the inactive network blocks.

Three properties follow, and together they explain why CMOS displaced every other logic family by the late 1980s:

1. **Rail-to-rail output.** With no conducting path to the opposite rail, the output settles at exactly 0 or V_{DD}, independent of transistor sizes. Logic levels are set by the supply, not by device ratios (unlike nMOS or TTL logic).
2. **No static current.** In either steady state one network is off, so apart from leakage no current flows from V_{DD} to ground. Power is consumed only when nodes switch (Section 33.3).
3. **Infinite input resistance.** The gate of a MOSFET draws no DC current, so one gate can drive many others without loading their DC levels; fan-out affects speed, not levels.

> **Status of the principle: Complementary pull-up/pull-down networks**
> *Fundamental.* Follows from the definition of the circuit: if the PDN conducts exactly when $F = 0$ and the PUN exactly when $F = 1$, one and only one network conducts for every input, so the static output is a rail and no static path exists; the ideal holds up to leakage currents.

The inverter and its transfer characteristic

The simplest CMOS gate, the **inverter**, has one nMOS and one pMOS transistor with joined gates (input) and joined drains (output). Its **voltage transfer characteristic** (VTC), output voltage against input voltage, is found by equating the nMOS and pMOS drain currents at every input voltage. It has five regions:

Region	Input range	nMOS	pMOS	Output
A	$0 \le V_{in} < V_{Tn}$	Cut off	Linear	V_{DD}
B	$V_{Tn} < V_{in} < V_M$	Saturated	Linear	Slightly below V_{DD}
C	$V_{in} \approx V_M$	Saturated	Saturated	Falls steeply through V_M
D	$V_M < V_{in} < V_{DD} - \lvert V_{Tp}\rvert$	Linear	Saturated	Slightly above 0
E	$V_{DD} - \lvert V_{Tp}\rvert < V_{in} \le V_{DD}$	Linear	Cut off	0

The **switching threshold** V_M is the input voltage at which $V_{out} = V_{in}$. With both transistors saturated and the square-law model of Chapter 4 (gain factors $k_n = \mu_n C_{ox} W_n/L_n$ and $k_p = \mu_p C_{ox} W_p/L_p$), equating currents gives

$$V_M = \frac{V_{Tn} + r\,(V_{DD} - \lvert V_{Tp}\rvert)}{1 + r}, \qquad r = \sqrt{\frac{k_p}{k_n}}$$

A symmetric inverter ($V_M = V_{DD}/2$ when $V_{Tn} = \lvert V_{Tp}\rvert$) needs $k_p = k_n$, so the pMOS must be wider than the nMOS by the mobility ratio μ_n/μ_p, about 2–3 in planar silicon and nearer 1–1.5 in strained-silicon and FinFET processes. Most libraries compromise at $W_p/W_n \approx 2$, which saves area and input capacitance at the cost of a slightly low V_M; V_M depends only on the square root of the size ratio, so it is insensitive to it.

Status of the principle: Switching threshold of the CMOS inverter
Approximate. Derived from the long-channel square law with both devices saturated and channel-length modulation neglected; in short-channel devices velocity saturation makes the currents closer to linear in overdrive and V_M is better found from the α-power law or simulation, but the conclusion (set V_M by the W_p/W_n ratio, weakly) is unchanged.

Worked example — switching threshold of a 180 nm-class inverter. Take $V_{DD} = 1.8$ V, $V_{Tn} = |V_{Tp}| = 0.45$ V, $\mu_n C_{ox} = 270\ \mu$A/V² and $\mu_p C_{ox} = 110\ \mu$A/V² (illustrative long-channel values), and a pMOS twice as wide as the nMOS. Then $r = \sqrt{220/270} = 0.903$ and $V_M = (0.45 + 0.903 \times 1.35)/1.903 = 0.877$ V, 26 mV below mid-supply. Making the pMOS 2.45 times wider would centre it exactly; making it equal in width to the nMOS would move V_M only to 0.80 V. Figure 33.1 shows this VTC.

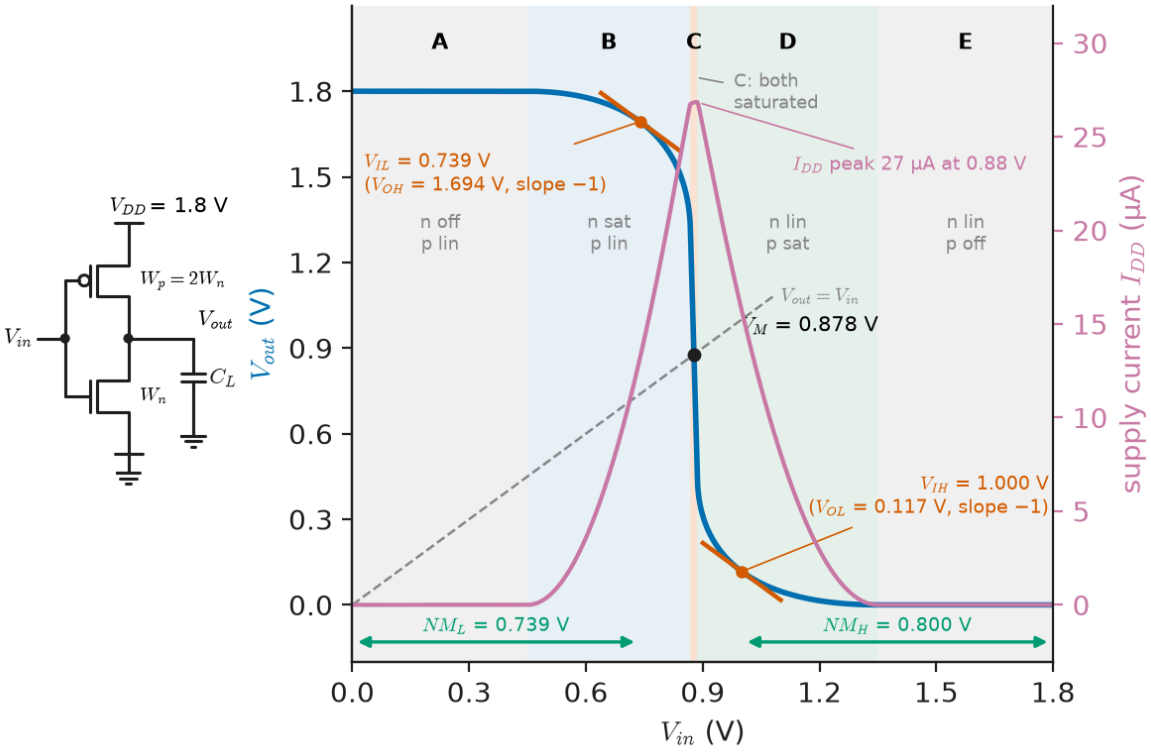


Figure 33.1 — Voltage transfer characteristic of a 1.8 V CMOS inverter with its five operating regions, switching threshold and noise margins.

33.2 MOSFET switching

The switch model

For digital timing a transistor is modelled as a switch in series with an **effective resistance** R_{eq}, driving and loaded by capacitances. A conducting nMOS discharging a load from V_{DD} to $V_{DD}/2$ starts saturated and ends in the linear region, so its resistance varies along the transition; averaging V_{DS}/I_D over the swing gives, to good accuracy for velocity-saturated devices,

$$R_{eq} \approx \frac{3}{4}\,\frac{V_{DD}}{I_{DSAT}}\left(1 - \frac{7}{9}\lambda V_{DD}\right)$$

R_{eq} is inversely proportional to width, so it is quoted per unit width. With a saturation current of 600 µA/µm for nMOS and 260 µA/µm for pMOS (typical of 180 nm at 1.8 V), $R_{eq} \approx 2.25$ kΩ·µm and 5.2 kΩ·µm respectively, ignoring λ. Table

33.1 collects representative values across process generations.

Table 33.1 — Representative CMOS process parameters (typical, high-performance logic transistors; low-power flavours have 3–10× lower leakage and 20–40 % lower drive).

Node	V_{DD} (V)	Gate dielectric (EOT)	$I_{DSAT,n}$ (µA/µm)	I_{off} per µm	Gate cap. (fF/µm)	FO4 delay (ps)
350 nm	3.3	7 nm SiO_2	500	~1 pA	1.6	150–180
180 nm	1.8	4 nm SiO_2	600	10–100 pA	1.5	65–90
90 nm	1.0–1.2	1.6 nm SiON	900–1100	10–100 nA	1.0	30–40
45 nm	1.0	1.0 nm high-k/metal gate	1200–1400	50–200 nA	0.9	15–20
16/14 nm FinFET	0.75–0.8	0.9 nm high-k	1300–1600 (per footprint)	10–100 nA	0.8	8–12
5 nm FinFET	0.7–0.75	0.8 nm high-k	1500–2000 (per footprint)	10–100 nA	0.7	4–7

The capacitances that the switches charge are of three kinds. **Gate capacitance** is about $C_{ox}WL$ plus overlap, roughly 1 fF per µm of width in modern processes, and is what each fan-out input adds. **Diffusion capacitance** of source and drain junctions, about the same per µm, loads the gate's own output (its "parasitic" or self-loading). **Wire capacitance**, about 0.15–0.2 fF per µm of length on any metal layer of any node (the geometry scales together), dominates long nets.

Pass transistors and transmission gates

A MOSFET used as a series switch between two signal nodes, rather than to a rail, is a **pass transistor**. An nMOS passes a strong 0 but a weak 1: when passing V_{DD}, its source rises until V_{GS} falls to V_{Tn} (raised further by the body effect, Chapter 4), so the output stops at about $V_{DD} - V_{Tn}$, 1.2–1.3 V of a 1.8 V supply. A gate driven by that degraded high has both its transistors partly on, drawing static current, and its noise margin shrinks. A pMOS passes a strong 1 and a weak 0.

Putting an nMOS and a pMOS in parallel with complementary gate signals gives the **transmission gate** (TG), which passes both levels fully: whichever device is losing drive near one rail, the other is gaining it. Its on-resistance is roughly constant across the signal range. Transmission gates are the switches inside multiplexers, XOR gates, latches and flip-flops (Sections 33.6–33.7) and the analogue switches of sample-and-hold circuits and ADC input multiplexers.

> **Status of the principle: Effective-resistance (RC) switch model**
> *Approximate.* Replaces the nonlinear transistor by a linear resistor fitted to the 50 % transition and the load by a lumped capacitor; it predicts delays to within 10–30 % and scales correctly with width and load, but fails for slow input edges, for strongly nonlinear loads such as long resistive wires, and in near-threshold operation, where the current is exponential in voltage.

33.3 Static and dynamic power

Dynamic power

Each time a gate output rises from 0 to V_{DD}, the supply delivers charge $C_L V_{DD}$ at voltage V_{DD}, energy $C_L V_{DD}^2$; half is stored on the capacitor and half is dissipated in the pull-up network. On the falling transition the stored half is dissipated

in the pull-down network. One full cycle therefore costs $C_L V_{DD}^2$ from the supply, independent of transistor resistance. If a node makes a 0→1 transition with probability α per clock cycle (its **activity factor**) at clock frequency f,

$$P_{dyn} = \alpha\, C_L\, V_{DD}^2\, f$$

summed over all nodes. Clock nets have $\alpha = 1$, datapaths 0.1–0.3, control logic and memories often below 0.05.

> **Status of the principle: CMOS dynamic power**
> *Derived.* Follows from charge conservation and the energy delivered by a constant-voltage source charging a capacitor through any resistance; it is exact for full-swing transitions and excludes short-circuit and leakage power, glitches (which raise the effective α) and reduced-swing nodes.

The quadratic dependence on V_{DD} is the most powerful lever in low-power design. **Dynamic voltage and frequency scaling** (DVFS) lowers V_{DD} and f together when performance is not needed; **clock gating** stops the clock to idle registers, setting their α to zero; reducing capacitance by shorter wires and smaller devices helps linearly.

Short-circuit power flows during each input transition while the input passes between V_{Tn} and $V_{DD} - |V_{Tp}|$, when both transistors conduct (the supply-current peak in Figure 33.1). It stays below about 10 % of dynamic power when input and output edges have similar rise times, which is one reason libraries limit maximum slew.

Static power

An "off" transistor still conducts. Four leakage mechanisms matter:

- **Subthreshold conduction**, the dominant one, is exponential in gate overdrive:

$$I_{sub} \approx I_0\, 10^{(V_{GS} - V_T)/S}$$

with S, the **subthreshold swing**, at least $(k_B T/q)\ln 10 = 60$ mV per decade at room temperature and typically 70–100 mV per decade in planar bulk devices, 65–75 in FinFETs. Lowering V_T by 100 mV at $S = 90$ mV/dec raises leakage by $10^{100/90} = 13$ times. Drain-induced barrier lowering (Chapter 4) makes V_T fall with V_{DS}, so leakage rises with V_{DD} too.

- **Gate tunnelling** through thin oxides; it became significant at 90–65 nm (1.2–1.6 nm SiON) and was cut by two to three orders of magnitude by high-k dielectrics at 45 nm.
- **Junction leakage** and **gate-induced drain leakage** (GIDL), important in low-power processes and at high temperature.

Leakage roughly doubles every 10–15 °C, so a chip's static power is often quoted at 85 °C or 125 °C junction temperature. The remedies are circuit and architectural: **multi-threshold libraries** (high-V_T cells off the critical path, low-V_T on it), **power gating** with header or footer switches that disconnect idle blocks, **body biasing** in FD-SOI processes, and **retention** registers that keep state while the rest of a block is powered down.

Worked example — DVFS and leakage in an SoC block. A processor block has 2 nF of total switched capacitance and an average activity factor of 0.1, at 0.9 V and 1 GHz.

- Dynamic power: $0.1 \times 2 \times 10^{-9} \times 0.9^2 \times 10^9 = 0.162$ W.
- In a low-performance mode at 0.7 V and 700 MHz: $0.1 \times 2 \times 10^{-9} \times 0.7^2 \times 0.7 \times 10^9 = 0.069$ W, a 58 % saving for a 30 % loss of speed.
- Leakage: 10 million transistors averaging 2 nA each at 0.9 V give 20 mA, 18 mW, about 11 % of the full-speed dynamic power but 26 % in the low mode; at 85 °C, with leakage four or five times higher, it is comparable to the dynamic power.

This is the sensor hub of the opening. Its 40 nm port cut C and V_{DD}, hence active power, but its standby current was set by subthreshold leakage of millions of low-V_T transistors that never switch off completely; a high-V_T library for the always-on domain and power gating of the rest restored the battery life.

Status of the principle: End of Dennard scaling
Empirical. Constant-field (Dennard) scaling, in which voltage, dimensions and capacitance all shrink by the same factor so that power density stays constant, held approximately until about the 130–90 nm generations; because subthreshold swing cannot fall below 60 mV per decade, V_T and hence V_{DD} stopped scaling near 0.7–1 V, and power density, not transistor count, has limited clock frequency since about 2005.

33.4 Propagation delay

Definitions and the RC estimate

The **propagation delay** t_{pHL} (t_{pLH}) is the time from the input crossing 50 % of the swing to the output crossing 50 % on its falling (rising) transition; $t_p = (t_{pHL} + t_{pLH})/2$. Rise and fall times are measured between 10 % and 90 % (some libraries use 20–80 %). With the switch model, an output node of capacitance C_L discharged through R_{eq} follows an exponential, and the 50 % point is reached after

$$t_p = \ln 2 \; R_{eq} C_L = 0.69 \, R_{eq} C_L$$

where C_L includes the gate's own diffusion capacitance, the input capacitance of every fan-out gate and the wire. The step-input assumption makes this optimistic by 10–20 % for realistic input edges.

Worked example — delay of a 180 nm inverter. An inverter with $W_n = 0.5$ µm and $W_p = 1.0$ µm drives a total load of 20 fF. Then $R_n = 0.75 \times 1.8/(600\ \mu\text{A}/\mu\text{m} \times 0.5\ \mu\text{m}) = 4.5$ kΩ and $R_p = 0.75 \times 1.8/(260 \times 1.0\ \mu\text{A}) = 5.2$ kΩ, giving $t_{pHL} = 0.69 \times 4.5\ \text{k}\Omega \times 20\ \text{fF} = 62$ ps, $t_{pLH} = 72$ ps and $t_p = 67$ ps. Each transition draws $C_L V_{DD}^2 = 65$ fJ from the supply per cycle. Doubling both widths halves the resistances, but only helps if the load is dominated by fan-out and wire rather than by the gate's own diffusion capacitance, which doubles too.

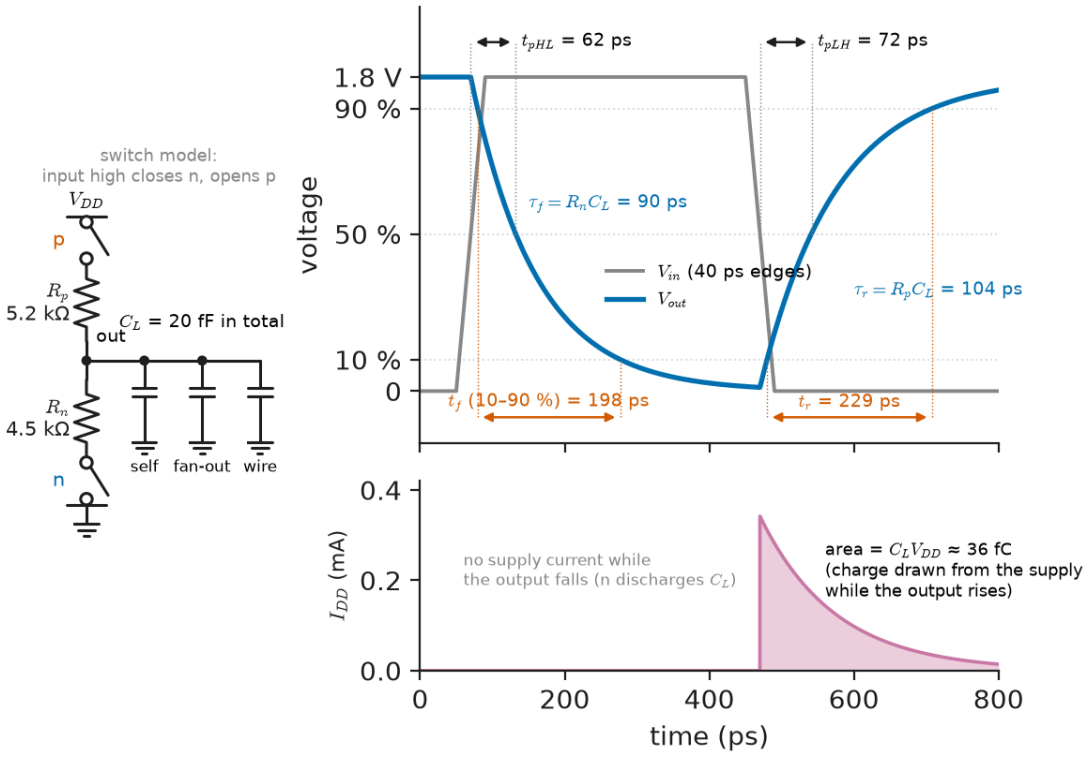


Figure 33.2 — Propagation delay of a CMOS inverter and its RC model.

Figure 33.2 shows the waveforms and the delay definitions.

Logical effort

Sizing a chain of gates by trial and error is hopeless; **logical effort** turns it into arithmetic. Express delay in units of $\tau = R_{eq} C_{in}$ of a reference inverter (about FO4/5, so 13–18 ps at 180 nm, about 1 ps at 5 nm). The delay of one gate is then

$$d = g\,h + p$$

where $h = C_{out}/C_{in}$ is the **electrical effort** (fan-out ratio), p the **parasitic delay** from the gate's own diffusion capacitance (1 for an inverter, about n for an n-input NAND or NOR), and g the **logical effort**: how much more input capacitance the gate presents than an inverter delivering the same output current. Sized for equal drive (Section 33.6), a two-input NAND has nMOS of width 2 and pMOS of width 2 per input, input capacitance 4 against the inverter's 3, so $g = 4/3$; a two-input NOR has nMOS 1 and pMOS 4, so $g = 5/3$. For a path of N stages with logical efforts g_i, branching factors b_i (the ratio of total to on-path capacitance where a signal fans out) and overall electrical effort $H = C_{load}/C_{in}$, the **path effort** is

$$F = G\,B\,H = \prod g_i \prod b_i \, \frac{C_{load}}{C_{in}}$$

and delay is minimised when every stage bears the same effort $\hat{f} = F^{1/N}$, giving

$$\hat{D} = N\,F^{1/N} + \sum p_i$$

The best number of stages makes $\hat{f}$ about 4 (between 3 and 5, where the optimum is flat), so $\hat{N} \approx \log_4 F$. Listing 33.1 applies this to two paths that drive a 1 pF load (an off-chip pad driver's input, say) from a 4 fF input.

```
 import numpy as np

G = {"inv": 1.0, "nand2": 4 / 3, "nor2": 5 / 3, "nand3": 5 / 3}   # logical effort
P = {"inv": 1.0, "nand2": 2.0, "nor2": 2.0, "nand3": 3.0}         # parasitic delay

def size_path(gates, branch, c_in, c_load):
    """Optimal stage effort, delay (in tau) and input capacitance of each stage."""
    g = np.array([G[x] for x in gates]); b = np.array(branch, float)
    F = g.prod() * b.prod() * c_load / c_in                        # path effort
    N = len(gates); f = F ** (1 / N)
    D = N * f + sum(P[x] for x in gates)
    cins, c = [], c_load                                           # walk back from the load
    for gi, bi in zip(g[::-1], b[::-1]):
        c = gi * bi * c / f
        cins.append(c)
    return F, f, D, cins[::-1]

for name, gates, branch in [("4 inverters", ["inv"] * 4, [1] * 4),
                            ("NAND2-NOR2-INV-INV", ["nand2", "nor2", "inv", "inv"], [1, 2, 1, 1])]:
    F, f, D, cins = size_path(gates, branch, c_in=4e-15, c_load=1e-12)
    print(f"{name}: F = {F:.0f}, f = {f:.2f}, D = {D:.1f} tau, "
          f"Cin = {', '.join(f'{c * 1e15:.1f}' for c in cins)} fF")
print(f"best N for F = 250: {np.log(250) / np.log(4):.2f}")
```

```
 4 inverters: F = 250, f = 3.98, D = 19.9 tau, Cin = 4.0, 15.9, 63.2, 251.5 fF
NAND2-NOR2-INV-INV: F = 1111, f = 5.77, D = 29.1 tau, Cin = 4.0, 17.3, 30.0, 173.2 fF
best N for F = 250: 3.98
```

The inverter chain is the classic buffer: each stage four times larger than the last, 19.9τ in total, about 300 ps at 180 nm (τ = 15 ps). A single inverter driving the load directly would take $250 + 1 = 251\tau$, 3.8 ns. In the second path the NOR2's output feeds two identical branches (branching factor 2), and the NAND2 and NOR2 add logical effort, so the path effort

is 1111 and the optimal stage effort 5.8, slightly above the ideal: a fifth stage would reduce the delay a little but would invert the logic. The listing's input capacitances are the sizes the designer picks from the library.

> **Status of the principle: Logical effort delay model**
> *Approximate.* Rests on the linear RC model of Section 33.2 with g and p fitted per gate type; it assumes equal rise and fall effort, ignores input slope and wire resistance, and treats branching as fixed, so it is a sizing and topology tool accurate to 10–20 %, with final numbers from static timing analysis (Chapter 34).

Wires

A wire is a distributed RC line. For a wire of resistance r and capacitance c per unit length and length l, the **Elmore delay**, the first moment of the impulse response, is $0.5\,rcl^2$, and the actual 50 % delay of the wire alone is about $0.38\,rcl^2$: both quadratic in length. A 0.28 µm wide, 0.5 µm thick copper line (2.2 µΩ·cm) has $r = 0.157\ \Omega/\mu\text{m}$ and $c \approx 0.2$ fF/µm. One millimetre has $R = 157\ \Omega$ and $C = 200$ fF, and its intrinsic delay is $0.38 \times 157 \times 200 \times 10^{-15} = 12$ ps, small beside the driver's $R_{eq}C$. Ten millimetres has $R = 1.57$ kΩ, $C = 2$ pF and an intrinsic delay of 1.2 ns, a hundred times larger. Long wires are therefore broken by **repeaters** (inverters every 1–3 mm in older nodes, every 100–300 µm in the narrow lower metals of advanced ones), which make delay linear in length again; and timing-critical long nets are routed on the thick upper metal layers. As lines narrow, surface and grain-boundary scattering and barrier layers raise copper's effective resistivity (*Materials Science and Engineering*, Ch. 50 §50.9), which is why wire delay has grown relative to gate delay at every node since 130 nm: the clock tree of the opening.

> **Status of the principle: Elmore delay of RC networks**
> *Approximate.* The first moment of the impulse response is an upper bound on the 50 % delay of an RC tree with monotone step response and is exact in the limit of one dominant pole; it ignores inductance (significant for wide, fast clock and power lines above a few GHz) and coupling to neighbouring wires.

33.5 Noise margins

Definitions

A logic gate must tolerate noise on its input without passing it on. The input voltage ranges a gate accepts as low and high are bounded by the points where the VTC's slope is -1: below V_{IL} the output stays high and the gain magnitude is below one, so input noise is attenuated; above V_{IH} the same holds for a low output; between them is the **transition region**, where the gain exceeds one and small disturbances are amplified. With driver output levels V_{OH} and V_{OL}, the **noise margins** are

$$NM_L = V_{IL} - V_{OL}, \qquad NM_H = V_{OH} - V_{IH}$$

For a static CMOS driver $V_{OL} = 0$ and $V_{OH} = V_{DD}$; a conservative variant takes the output levels at the unity-gain points instead. Listing 33.2 computes the VTC of the inverter of Section 33.1 numerically (square law with $\lambda = 0.1\ \text{V}^{-1}$) and extracts the threshold, the unity-gain points and the noise margins.

```
 import numpy as np
from scipy.optimize import brentq

VDD, VTn, VTp, lam = 1.8, 0.45, 0.45, 0.1        # V, V, |V|, 1/V
kn, kp = 270e-6 * 1.0, 110e-6 * 2.0              # A/V^2: k' x (W/L), Wp/Wn = 2

def ids(k, vgs, vds, vt):                         # square law with channel-length modulation
    if vgs <= vt: return 0.0
    vov = vgs - vt
    core = k * (vov * vds - vds**2 / 2) if vds < vov else k / 2 * vov**2
    return core * (1 + lam * vds)

def vout(vin):                                    # solve I_n = I_p for the output node
    f = lambda vo: ids(kn, vin, vo, VTn) - ids(kp, VDD - vin, VDD - vo, VTp)
    return brentq(f, 0.0, VDD)

vin = np.linspace(0.0, VDD, 3601)
vo = np.array([VDD if v <= VTn else 0.0 if v >= VDD - VTp else vout(v) for v in vin])
gain = np.gradient(vo, vin)
VM = vin[np.argmin(np.abs(vo - vin))]
idx = np.where(gain <= -1)[0]
VIL, VIH = vin[idx[0]], vin[idx[-1]]
VOH, VOL = np.interp(VIL, vin, vo), np.interp(VIH, vin, vo)
print(f"VM = {VM:.3f} V, peak gain = {gain.min():.1f}")
print(f"VIL = {VIL:.3f} V (VOH = {VOH:.3f}), VIH = {VIH:.3f} V (VOL = {VOL:.3f})")
print(f"NML = {VIL:.3f} V, NMH = {VDD - VIH:.3f} V (driver static VOL = 0, VOH = VDD)")
r = np.sqrt(kp / kn)
print(f"analytic VM = {(VTn + r * (VDD - VTp)) / (1 + r):.3f} V")
```

```
 VM = 0.878 V, peak gain = -48.6
VIL = 0.739 V (VOH = 1.694), VIH = 1.000 V (VOL = 0.117)
NML = 0.739 V, NMH = 0.800 V (driver static VOL = 0, VOH = VDD)
analytic VM = 0.877 V
```

Worked example — reading the noise margins. The transition region is only 0.26 V wide (from 0.739 to 1.000 V) because the peak gain is about 49; the margins are 0.74 V low and 0.80 V high, 41 % and 44 % of the supply, close to the ideal of 50 %. With the conservative definition they are $0.739 - 0.117 = 0.62$ V and $1.694 - 1.000 = 0.69$ V. The numerically found V_M agrees with the formula of Section 33.1 to 1 mV, since channel-length modulation shifts both currents nearly equally. Scaling the same design to $V_{DD} = 0.7$ V with $V_T = 0.35$ V leaves a supply only twice the threshold; the gain falls, the transition region widens relative to the supply and the margins shrink to well under 0.2 V, which is why near-threshold designs need careful noise budgeting and why the register file of the opening failed at 0.7 V.

Regeneration and the noise budget

CMOS logic restores levels: because the gain magnitude exceeds one in the transition region and is far below one at the rails, a degraded level passing through a chain of gates is pushed back toward the rails at each stage. This **regenerative property** is what makes the digital abstraction of Chapter 12 work: errors do not accumulate. The noise margin must, however, cover every disturbance at once. The main contributors are **crosstalk** (capacitive coupling from a switching neighbour, a fraction $C_c/(C_c + C_g)$ of its swing, 10–30 % for long parallel wires), **supply and ground bounce** ($L\,di/dt$ on package and grid inductance and IR drop across the power grid, typically budgeted at 5–10 % of V_{DD}), and **process, voltage and temperature variation** shifting V_M from gate to gate. Static noise analysis in sign-off tools checks the sum.

> **Status of the principle: Unity-gain definition of noise margins**
> *Approximate.* A static, single-stage criterion: it treats noise as a DC offset and assumes a chain of identical gates; dynamic noise (short glitches that the next gate's capacitance filters) and correlated noise on supply and input are assessed by simulation-based noise analysis.

33.6 CMOS gates

Static CMOS: NAND, NOR and complex gates

Parallel nMOS transistors in the PDN implement OR of their inputs (the output is pulled low if any input is high), series nMOS implement AND. The PUN is the **dual**: series becomes parallel and vice versa. A two-input **NAND** has two series nMOS and two parallel pMOS; a two-input **NOR** two parallel nMOS and two series pMOS. Static CMOS gates are inherently inverting, because nMOS pull down when their inputs go high; AND and OR are a NAND or NOR followed by an inverter.

A **complex gate** realises any inverted sum of products in one stage. The **AOI21** (AND–OR–INVERT), $Y = \overline{AB + C}$, has a PDN of A and B in series, in parallel with C, and a PUN of A and B in parallel, in series with C: six transistors instead of the ten of a NAND2 followed by a NOR2 and an inverter. AOI and OAI gates are the backbone of adders and multiplexers in standard-cell libraries.

Sizing for equal drive. Each gate is sized so that its worst-case pull-up and pull-down resistances equal those of a reference inverter (nMOS width 1, pMOS width 2). Transistors in series add resistance, so each of n series transistors must be n times wider. A NAND2 has nMOS of width 2 and pMOS of width 2; a NOR2 has nMOS of width 1 and pMOS of width 4; a NAND3 nMOS 3, pMOS 2; a NOR3 nMOS 1, pMOS 6. These sizes give the logical efforts of Section 33.4, and they show why designers prefer NAND to NOR: series pMOS, already the weaker device, become very wide. Stacks deeper than three or four are avoided because the series resistance and the internal-node capacitance grow together. Stacks do have one benefit: two off transistors in series leak 5–10 times less than one (the **stack effect**), because the intermediate node rises and reverse-biases the upper device's gate–source.

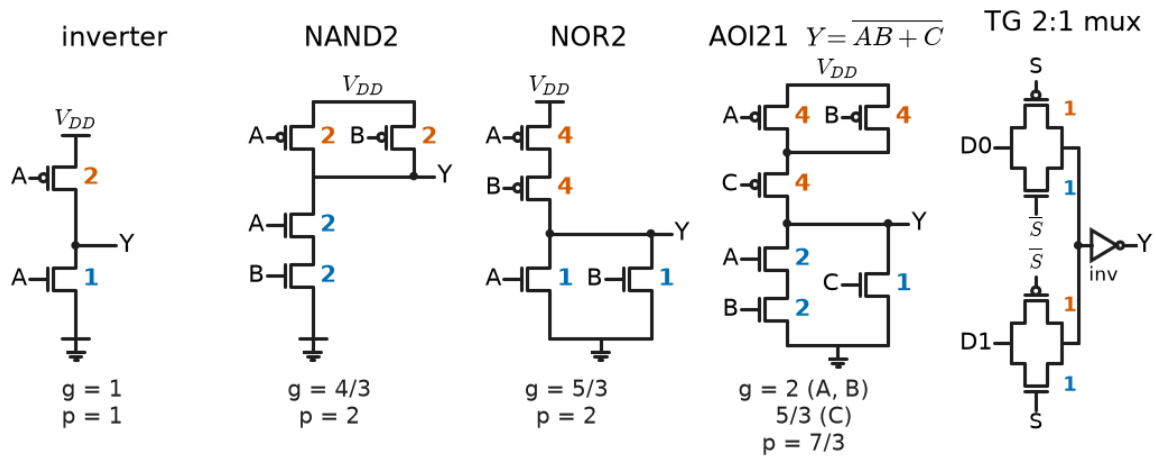


Figure 33.3 — Static CMOS gates sized for equal drive: inverter, NAND2, NOR2, AOI21 and a transmission-gate 2:1 multiplexer.

Transmission-gate and pass-transistor logic

Transmission gates make some functions much cheaper. A **2:1 multiplexer** is two TGs driven by a select signal and its complement, joined at the output: four transistors plus an inverter for the select, against 12 for a static AOI-based mux. An **XOR** in TG form uses six to eight transistors against 12 in static CMOS. The penalty is that a TG has no gain and no connection to the rails: a chain of n TGs has an RC delay growing as n^2 (Elmore again) and passes noise straight through, so TG stages are limited to two or three in series and are followed by a restoring inverter. **Tristate buffers**, a static inverter with series enable transistors in both networks (or an inverter followed by a TG), let several drivers share a bus.

Ratioed and dynamic logic

Two other styles appear in specialised circuits. **Pseudo-nMOS** replaces the PUN by one always-on pMOS: fewer transistors and fast wide NORs, at the cost of static current when the output is low and a low level set by the ratio of pull-up to pull-down strength; it survives in some ROM and PLA arrays. **Dynamic (domino) logic** precharges the output high during one clock phase and conditionally discharges it through an nMOS network in the other: smaller input capacitance and faster wide gates, but it needs a clock, keepers against leakage and careful noise design. High-performance processors used domino logic heavily until leakage and design effort pushed most datapaths back to static CMOS around the 65–45 nm generations.

> **Status of the principle: Duality of pull-up and pull-down networks**
> *Derived.* From De Morgan's laws (Chapter 12): if the PDN conducts when $F = 0$ for a series–parallel function, its dual network of complementary devices conducts exactly when $F = 1$; the construction holds for any series–parallel network and guarantees rail-to-rail static outputs.

33.7 Latches and flip-flops

The bistable element

Storage in CMOS rests on two inverters connected in a loop. The loop has three equilibrium points where the two VTCs (one mirrored) intersect: two stable ones at the rails and one unstable one at V_M, where the loop gain exceeds one. A loop disturbed away from the unstable point diverges exponentially toward a stable state with time constant $\tau_m \approx C/g_m$ of the inverters, roughly one FO1 delay; how long it can hover near the unstable point is the **metastability** problem whose statistics and synchroniser remedies are treated in Chapter 15. The SRAM cell built from the same loop is in Chapter 18.

The transmission-gate latch

A **D latch** must store a value and let a new one in when its clock is active. The standard static CMOS latch has an input transmission gate, an inverter, and a feedback path through a second inverter and a second transmission gate (or a clocked inverter). When the clock ϕ is high, the input TG is on and the feedback TG off: the latch is **transparent** and Q follows D after one TG and one inverter delay. When ϕ falls, the input TG turns off and the feedback TG on, closing the loop and holding the value. Because the feedback is broken while new data enter, the input driver never fights the stored value; a cheaper version with a weak always-on feedback inverter (a **keeper**) saves the second TG but makes writing a ratioed contest, the failure of the register file in the opening when its supply was lowered and the write driver weakened faster than the keeper.

A **clocked inverter** (C²MOS stage: an inverter with an extra series pMOS and nMOS gated by $\overline{\phi}$ and ϕ) combines the inverter and the transmission gate in one stage and is equivalent for storage purposes; most standard-cell latches use clocked inverters in the feedback path.

The master–slave flip-flop

An edge-triggered **D flip-flop** is two latches in series, transparent on opposite clock phases. With the **master** transparent while the clock is low and the **slave** while it is high, data pass through the master during the low phase, are frozen in the master at the rising edge, and appear at Q through the slave, which is then transparent; while the clock is high, the closed master isolates Q from changes at D. A static TG master–slave flip-flop uses about 16–24 transistors including local clock inverters, and its timing parameters follow from its internal paths:

- **Setup time** t_{setup}: D must arrive early enough to propagate through the master's input TG and inverter and set up the feedback loop before the clock edge closes it; typically 1–2 FO4.
- **Hold time** t_{hold}: D must not change until the master's input TG has fully turned off after the edge; with locally generated $\overline{\phi}$, slow complementary clocks can make both TGs briefly conduct, so hold time is zero or slightly negative in a well-designed cell and positive under clock skew.
- **Clock-to-Q delay** t_{cq}: the slave's TG plus its inverter and output buffer; typically 2–3 FO4.

These three numbers enter every timing constraint of Chapter 14 ($T_{clk} \geq t_{cq} + t_{logic} + t_{setup} + t_{skew}$, and $t_{cq} + t_{logic,min} \geq t_{hold} + t_{skew}$). The flip-flop overhead, $t_{cq} + t_{setup}$, is 3–5 FO4, a large fraction of a high-speed processor's 15–20 FO4 cycle, which is why such processors use **pulsed latches** (a latch made transparent for a narrow pulse after each clock edge) to shave a gate delay from every pipeline stage.

Worked example — the cycle time budget. At 180 nm with FO4 = 75 ps, a register with $t_{cq} = 2.5$ FO4 and $t_{setup} = 1.5$ FO4 costs 300 ps of every cycle. A target clock of 500 MHz (2 ns) leaves $2000 - 300 - 100 = 1600$ ps for logic after a 100 ps skew allowance, about 21 FO4 of logic per stage. At 1 GHz only 600 ps, 8 FO4, remains: pipeline stages must be cut in half, and the register overhead becomes a third of the cycle. Porting the design to 45 nm (FO4 ≈ 17 ps) reduces all three terms together, which is why process migration speeds up the clock without changing the logic depth.

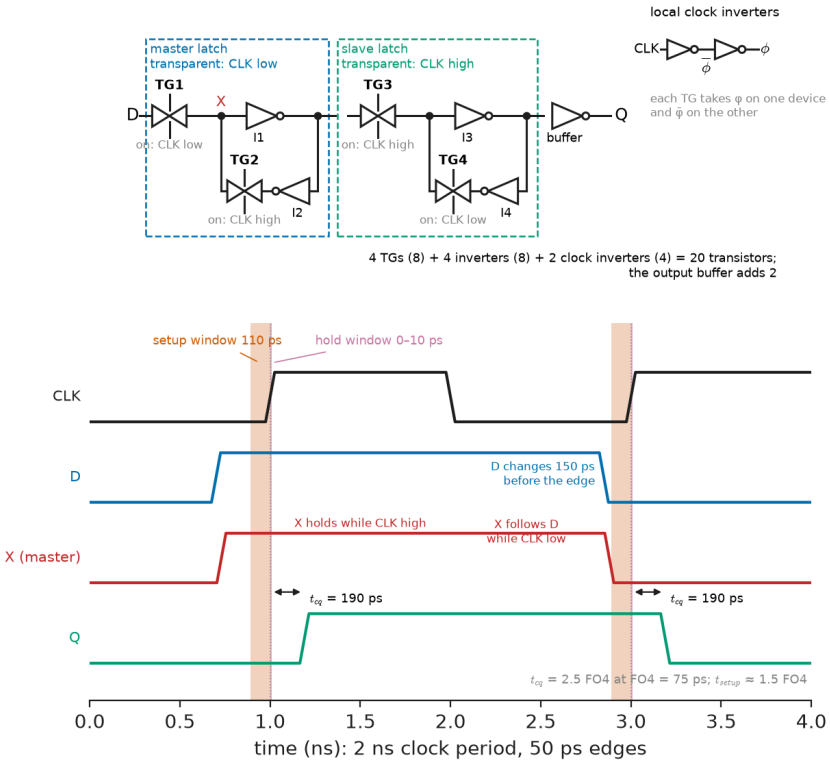


Figure 33.4 — Static transmission-gate master-slave D flip-flop and its timing parameters.

Figure 33.4 shows the circuit and its timing; the windows marked around the clock edge are the setup and hold times that static timing analysis checks on every register (Chapter 34).

> **Status of the principle: Bistability of a cross-coupled inverter pair**
> *Derived*. Follows from the fixed points of two cascaded transfer characteristics: with loop gain above one at the crossing, the middle equilibrium is unstable and the outer ones stable; it holds as long as each inverter's peak gain exceeds one, which fails when the supply approaches the threshold voltage or the cell is overpowered by a write driver.

33.8 Layout fundamentals

Layers seen from above

A layout is the set of mask polygons from which the chip is made. For a basic CMOS process the designer draws, from the substrate up:

- **n-well**: the region doped n-type to hold pMOS transistors in a p-type substrate (twin-well processes also draw p-wells);
- **active** (diffusion): areas where transistors and substrate or well contacts are formed, bounded by shallow trench isolation;
- **polysilicon** (or the metal-gate placeholder in high-k processes): a transistor exists wherever poly crosses active, with channel length set by the poly width and channel width by the active width;
- **n+ and p+ select**: the implant masks that dope source and drain regions on either side of the gate;
- **contact**: holes from metal 1 down to active or poly;
- **metal 1, via 1, metal 2, …**: the interconnect stack, 6–15 levels in modern logic processes.

The processes that turn each mask into structure (oxidation, implantation, lithography, etching, silicide, copper damascene) are described in *Materials Science and Engineering*, Ch. 50 §50.4–50.9; the layout designer sees them only through the design rules.

Lambda rules

Design rules state the minimum widths, spacings, enclosures and extensions the process can reliably build. In the scalable approach introduced by Mead and Conway, every rule is a multiple of a single parameter λ, half the minimum drawn gate length, so that a layout can be ported between processes by scaling. Table 33.2 lists representative rules in the style of the MOSIS scalable CMOS (SCMOS) set; exact values differ between rule decks.

Table 33.2 — Representative scalable (lambda) design rules.

Rule	Value
Poly width (minimum channel length)	2λ
Poly spacing	2λ
Active width (minimum channel width)	3λ
Active spacing	3λ
Poly extension beyond active (gate end-cap)	2λ
Active extension beyond poly (source/drain)	3λ
Poly to unrelated active spacing	1λ
Contact size	$2\lambda \times 2\lambda$
Active or poly enclosure of contact	1λ
Contact to gate spacing	2λ
Metal 1 width and spacing	3λ
Via size; metal 2 width and spacing	2λ; 3λ and 4λ
n-well width; well enclosure of p+ active	10λ; 5λ

Each rule has a physical reason: poly extension prevents a misaligned gate from leaving a conducting sliver of active; contact enclosure tolerates overlay error; metal spacing avoids shorts from lithographic bridging; well enclosure keeps the p+ junction away from the well edge where the doping changes.

> **Status of the principle: Scalable lambda design rules**
> *Approximate*. A single scaling parameter captures the proportions of processes down to about 0.25 μm; below that, rules no longer scale together (metal pitches, via enclosures, well spacings and gate-to-contact distances follow different physics), lithography introduces forbidden pitches and orientation rules, multiple patterning adds colouring constraints, and real rule decks are written in absolute dimensions with hundreds to thousands of rules (Chapter 34).

Stick diagrams and cell layout

A **stick diagram** is a topological sketch of a layout: lines in the colour of each layer (poly, n- and p-diffusion, metal), crosses for contacts, no dimensions. It decides the order of transistors and the routing before any rule is applied. For an inverter, two horizontal strips of diffusion (p-diffusion in the n-well at the top, n-diffusion at the bottom) are crossed by one vertical poly line, the input; metal 1 power rails run horizontally at top (V_{DD}) and bottom (ground), and a metal 1 strap joins the two drains as the output.

For multi-input gates the transistor order matters. If the nMOS and pMOS transistors can be ordered along a single unbroken strip of diffusion each, with the same input order in both, every poly line crosses both strips straight and adjacent transistors share source and drain regions, minimising area and diffusion capacitance. Such an order is an **Euler path** through the graph of each network, visiting every transistor (edge) once. For the AOI21 the order A–B–C works for both networks; a gate with no common Euler path needs a break in the diffusion and costs width.

Standard cells fix the height (typically 7–12 metal-1 tracks), the positions of the power rails and the well boundary, so that cells abut in rows with shared rails; their width varies with function. Figure 33.5 shows an inverter cell drawn to the rules of Table 33.2, about 24λ wide and 40λ high including rails.

Worked example — the area of an inverter cell. In a 180 nm process ($\lambda = 0.09$ μm) the $24\lambda \times 40\lambda = 960\lambda^2$ cell occupies $960 \times 0.09^2 = 7.8\ \mu\text{m}^2$, close to the 5–10 μm² of real 180 nm libraries. The same 960 λ^2 at 45 nm ($\lambda = 22.5$

nm) would be 0.49 µm², but a real 45 nm inverter cell is closer to 0.5–1 µm² because fixed-pitch metal and contact rules prevent pure scaling; in FinFET processes width is quantised in fins (each fin about 30–50 nm effective width) and cell area is counted in track heights and contacted-poly pitches rather than λ.

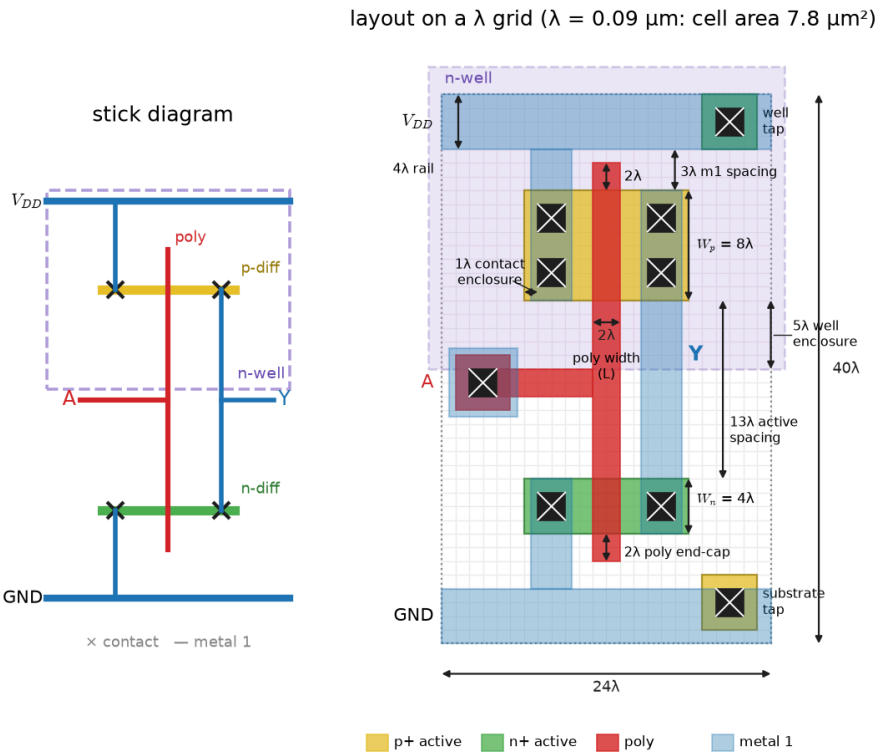


Figure 33.5 — Stick diagram and lambda-rule layout of a CMOS inverter cell.

Latch-up

The n-well, p-substrate and the p+ and n+ diffusions of every CMOS inverter form a parasitic pnpn structure: a vertical pnp transistor (p+ source in the n-well, the well, the substrate) and a lateral npn (n+ source, substrate, well), connected as the thyristor of Chapter 5. If a transient (an I/O pin driven beyond the rails, a supply surge, an ionising particle) injects enough current to forward-bias either base–emitter junction through the well or substrate resistance, the structure latches and short-circuits V_{DD} to ground until power is removed, often destroying the chip. Layout prevents it by keeping the well and substrate resistances low: **well and substrate taps** (contacts to V_{DD} and ground) placed at regular intervals, tens of micrometres apart (standard-cell rows include tap cells or taps in every cell); **guard rings** of tied diffusion around I/O transistors, which collect injected carriers; and spacing between I/O nMOS and pMOS. Latch-up immunity is qualified by JEDEC JESD78, which injects ±100 mA into each I/O pin and overvoltages the supplies without triggering latch-up.

What remains open

Open question	Where it is answered
How do short-channel effects, body effect and capacitances of a single MOSFET arise and how are they modelled?	Chapter 4
How are latches and flip-flops used in registers, counters and state machines, and how are clock-domain crossings made safe?	Chapter 14, Chapter 15
How are SRAM and DRAM cells and arrays built?	Chapter 18
How are millions of gates synthesised, placed, routed, timed, power-analysed and checked against full design-rule decks?	Chapter 34
How are the masks turned into silicon (lithography, implantation, etching, metallisation)?	*Materials Science and Engineering*, Ch. 50
How are energy and delay traded at system level between processors and accelerators?	Chapter 35

Next: Chapter 34 scales these gates and flip-flops to complete chips, following a design from RTL through synthesis, placement, routing and timing sign-off to the fabrication constraints that bound it.

Chapter 34 — VLSI Design

Part VIII · Integrated Circuits and VLSI

An automotive radar processor in a 28 nm process passes every simulation, synthesises cleanly and fits its 25 mm² die budget. Two weeks before tape-out the sign-off timing run reports that one path in the FFT engine misses its 1 GHz clock by 42 ps — but only at the slow-process, 0.81 V, 125 °C corner. At the opposite corner (fast process, 0.99 V, −40 °C) another path fails its *hold* check by 30 ps; that one cannot be fixed by slowing the clock, and in silicon the chip would work at no frequency at all. The physical-verification run lists 1 400 antenna violations, the power-grid analysis shows a 61 mV droop under the DSP array, and the yield model says a 5 % larger die would add 0.06 USD to every part — 0.3 million USD over a five-million-unit lifetime. None of this is visible in the Verilog. Each problem comes from turning a register-transfer description into millions of placed and wired transistors, manufactured at a given yield on a real process with real variation. This chapter is about that transformation.

> **How does a design described as registers and logic become a manufacturable layout, and which models decide whether it meets timing, power and area?**
> **Where does each abstraction of the VLSI flow — the synthesised netlist, the timing graph, the power estimate, the design rules — hold, and where does the silicon depart from it?**

Concept	Engineering destination
Design hierarchy and abstraction levels	Partitioning a system-on-chip into blocks, IP reuse, NRE budgeting
RTL design for synthesis	Synthesis-friendly Verilog/VHDL, pipelining, clock gating, reset strategy
Logic synthesis and timing constraints	Standard-cell netlists, SDC files, area–delay trade-offs
Floorplanning and power-grid design	Die size, macro placement, IR-drop budgets
Placement, clock-tree synthesis, routing	Congestion, clock skew, wire delay, repeater insertion
Static timing analysis	Setup and hold slack, PVT corners, on-chip variation, sign-off
Power estimation	Battery life, thermal design power, package and regulator choice
DRC, LVS, antenna and density rules	A layout the foundry will accept and that matches the schematic
Functional, formal and test verification	First-silicon success, test coverage, defect levels in DPPM
Yield, variability, design for manufacturing	Cost per good die, SRAM margins, redundant vias and fill

Roadmap. How a chip is organised and what it costs (34.1) → writing RTL that synthesises well (34.2) → synthesis to gates under constraints (34.3) → floorplan and power grid (34.4) → placement, clock tree and routing (34.5) → static timing analysis (34.6) → power estimation (34.7) → design-rule checking and LVS (34.8) → verification and test (34.9) → yield, variability and manufacturing constraints (34.10).

> **Boundary.**
> Verilog, SystemVerilog and VHDL syntax, simulation semantics and test benches are developed in Chapter 19; CMOS gates, their delay, static and dynamic power, latches and flip-flops and layout fundamentals (layers, stick diagrams, lambda rules) in Chapter 33; setup and hold constraints, clock-domain crossings and reset strategies in Chapter 14 §14.8–14.10 and Chapter 15; SDC constraints, assertions, coverage and formal checking in Chapter 19 §19.8–19.11. The manufacturing steps themselves — lithography, etching, copper damascene metallization, low-k dielectrics, CMP and process-induced defects — are treated as materials processes in *Materials Science and Engineering*, Ch. 50. This chapter uses those results as constraints on design.

34.1 Design hierarchy

Abstraction levels

A chip is described at several levels of abstraction, each hiding the level below behind a model; the design flow is the chain of tools that translates each level into the next and checks that behaviour is preserved (Figure 34.1).

Table 34.1 — Abstraction levels of a digital IC.

Level	Primitive objects	Description	Typical tools and checks
System / architecture	Processors, accelerators, memories, buses	C/C++, SystemC, spreadsheets, performance models	Architecture exploration (Chapter 35)
Register-transfer (RTL)	Registers, adders, multiplexers, FSMs	Verilog, SystemVerilog, VHDL	Simulation, lint, formal property checking
Gate (netlist)	Standard cells: NAND2, DFF, MUX2, ICG	Structural Verilog netlist plus Liberty timing	Equivalence checking, STA, gate-level simulation
Transistor	MOSFETs, parasitic R and C	SPICE netlist	Circuit simulation, cell characterisation
Layout	Polygons on mask layers	GDSII or OASIS	DRC, LVS, parasitic extraction

The vertical hierarchy is matched by a horizontal one. A system-on-chip (SoC) is divided into blocks — a CPU cluster, a DSP, a DDR controller, PHYs, SRAM macros, PLLs and ADCs — each designed or bought as intellectual property (IP) and verified separately, then assembled at the top level. A licensed Cortex-M core arrives as RTL (soft IP); a DDR4 PHY arrives as a finished layout for one process (hard IP). Analog blocks follow a custom flow (Chapter 10) and enter the digital flow as hard macros with timing and power models.

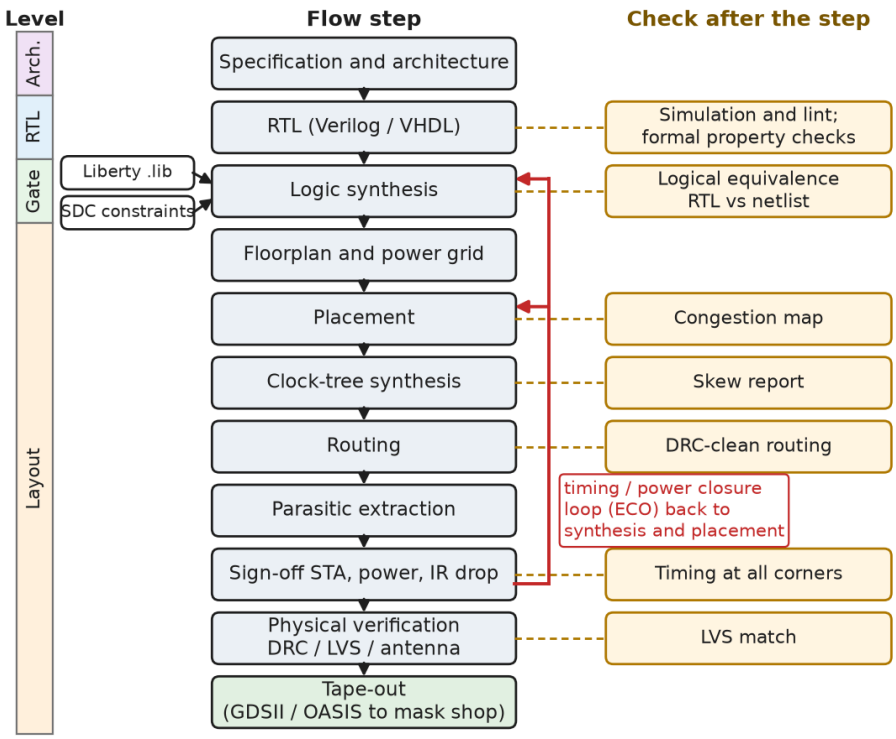


Figure 34.1 — The digital VLSI design flow from specification to tape-out, with the check performed after each step.

Why hierarchy is also a wiring problem

The number of signals that must cross a block boundary grows with the block's size, but sub-linearly. Empirically, a block of G gates has

$$T = t\,G^{p}$$

external terminals, with $t \approx 3$–4 terminals per gate and Rent exponent $p \approx 0.5$–0.75 for random logic. A 100 000-gate block with $t = 4$ and $p = 0.6$ needs about $4 \times 10^{5\times 0.6} = 4\,000$ terminals; a 10-million-gate chip would need 63 000, far beyond any package, so the rule breaks at the chip boundary and off-chip traffic is serialised into a few fast links (Chapter 26). Inside the chip, Rent's rule sets the wire-length distribution, hence the number of metal layers and the share of delay and power spent in wires.

> **Status of the principle: Rent's rule**
> *Empirical.* A power-law fit to the terminal counts of partitioned logic, valid over several decades of block size inside a chip; it fails at the package boundary and for regular arrays, and p must be measured for each design style.

What a chip costs before it is made

VLSI economics are set by non-recurring engineering (NRE) cost, which grows steeply with each node.

Table 34.2 — Representative process nodes and non-recurring costs (typical; varies by foundry, library and design).

Node	Core V_{DD} (V)	FO4 delay (ps)	NAND2 area (µm²)	6T SRAM cell (µm²)	Metal layers	Mask set (MUSD)
180 nm	1.8	60–90	≈ 10	≈ 5	5–6	≈ 0.1
65 nm	1.0–1.2	20–30	≈ 1.4	≈ 0.55	8–9	0.5–1
28 nm (planar HKMG)	0.9–1.0	15–20	≈ 0.5	0.12–0.15	9–11	1.5–3
7 nm (FinFET)	0.75	8–12	≈ 0.05	≈ 0.027	13–15	10–15
3 nm (FinFET/GAA)	0.70–0.75	6–9	≈ 0.03	≈ 0.021	15–19	25–40

A 28 nm re-spin costs a few million USD and three months; at 3 nm, tens of millions. Every step below exists to make first silicon work.

34.2 RTL design

The synchronous discipline

The flow from RTL to layout works because almost all digital chips obey one rule: **every storage element is an edge-triggered flip-flop on a clock, and every combinational path between flip-flops settles within a clock period.** Under that rule behaviour can be simulated cycle by cycle without delays, synthesis can optimise logic freely between registers, and timing can be checked separately by static analysis (Section 34.6). Latches, combinational loops and hand-gated clocks break the rule and appear only in reviewed places (clock-gating cells, synchronisers, analog interfaces).

Good ASIC RTL adds a few conventions to the language rules of Chapter 19:

- **Few clock domains.** Each crossing needs a synchroniser or asynchronous FIFO (Chapter 15) and a constraint that excludes it from timing.
- **Deliberate reset** (Chapter 14 §14.10). In an ASIC, datapath registers overwritten before use get no reset at all, saving area and the routing of a high-fanout net.
- **Pipelining.** Registers cut long arithmetic paths so each stage fits the period; latency in cycles grows, while throughput stays one result per cycle at a shorter period, so results per second rise.
- **Clock enables, not hand-gated clocks.** A register written as "update when `en`" lets synthesis insert an integrated clock-gating (ICG) cell — a latch plus an AND gate, characterised to be glitch-free — that stops the clock to a whole register bank, the most effective dynamic-power technique (Section 34.7).
- **No inferred latches.** Every combinational block assigns every output on every path.

A synthesis-ready datapath

Listing 34.1 is a two-stage pipelined multiply–accumulate unit of the kind replicated many times in the radar processor's FFT and filter engines.

```
 // Two-stage pipelined multiply-accumulate, Verilog-2001, synthesis-ready
module mac_pipe #(
    parameter W  = 16,
    parameter AW = 40
) (
    input  wire                 clk,
    input  wire                 rst_n,   // synchronous, active low
    input  wire                 en,      // input valid; becomes an ICG enable
    input  wire                 clr,     // first sample of a new accumulation
    input  wire signed [W-1:0]  a,
    input  wire signed [W-1:0]  b,
    output reg  signed [AW-1:0] acc
);
    reg signed [2*W-1:0] prod_q;
    reg                  clr_q, vld_q;

    always @(posedge clk) begin
        if (!rst_n) begin
            vld_q <= 1'b0;
            acc   <= {AW{1'b0}};
        end else begin
            vld_q <= en;
            if (vld_q)                    // stage 2: accumulate
                acc <= (clr_q ? {AW{1'b0}} : acc)
                     + {{(AW-2*W){prod_q[2*W-1]}}, prod_q};
        end
    end

    always @(posedge clk)                 // stage 1: no reset needed
        if (en) begin
            prod_q <= a * b;
            clr_q  <= clr;
        end
endmodule
```

The critical path is the slower of the 16 × 16 multiplier and the 40-bit adder rather than their sum; at 28 nm the multiplier alone is about 30 FO4 delays (0.5 ns), so an unpipelined version would not reach 1 GHz. The 8 guard bits of the accumulator allow 256 full-scale accumulations without overflow. The stage-1 registers have no reset, because they

are ignored until `vld_q` is set, and their `en` condition is exactly the pattern synthesis turns into one ICG cell driving 33 flip-flops.

34.3 Logic synthesis

What synthesis does

Logic synthesis turns RTL into a netlist of cells from a **standard-cell library**: hundreds to thousands of pre-characterised layouts (inverters and buffers in many drive strengths, NAND, NOR, AOI/OAI gates, multiplexers, adders, flip-flops with scan, reset and enable, ICG cells, level shifters), all of one height so they abut in rows. Each cell is described in a **Liberty** (.lib) file at every process–voltage–temperature (PVT) corner: delay and output transition as look-up tables indexed by input transition and output load (NLDM, or the current-source models CCS and ECSM at advanced nodes), pin capacitances, setup and hold tables, and internal and leakage power.

Synthesis proceeds in three steps:

1. **Elaboration.** The RTL becomes a generic network of operators, registers and multiplexers; FSMs may be re-encoded (binary, Gray, one-hot).
2. **Technology-independent optimisation.** Boolean minimisation, sharing, constant propagation, retiming, and choice of arithmetic architecture (ripple or parallel-prefix adders, Booth or Wallace-tree multipliers).
3. **Technology mapping and sizing.** The network is covered by library cells for minimum area under the timing constraints; critical cells are then upsized, buffered or swapped to faster threshold-voltage flavours, and cells with slack are downsized.

Optimal mapping is an NP-hard covering problem; tools decompose the network into trees or cuts and use dynamic programming and heuristics, so "the" area of a design is never unique.

> **Status of the principle: Logic-synthesis optimality**
> *Approximate.* Synthesis returns a good, not optimal, cover of the logic: tree-decomposition mapping is exact only within each tree, and sizing uses the library's table models, which are themselves interpolations of characterised SPICE data.

Timing constraints drive everything

Synthesis learns what "fast enough" means only from the constraints, written in SDC and reused unchanged by placement, routing and sign-off; the core commands — clocks, input and output delays, false and multicycle paths, clock groups — are those of Chapter 19 §19.8, Table 19.5. An ASIC block adds constraints for what does not exist yet at synthesis time: the clock tree, and the cells and wires of its neighbours. Listing 34.2 shows those additions for the radar block (ns, pF).

```
create_clock -name clk -period 1.000 [get_ports clk]
set_clock_uncertainty -setup 0.080 [get_clocks clk]  ;# pre-CTS: jitter + skew estimate
set_clock_uncertainty -hold  0.020 [get_clocks clk]
set_clock_transition 0.040 [get_clocks clk]          ;# ideal clock until CTS
set_driving_cell -lib_cell BUFX4 [remove_from_collection [all_inputs] [get_ports clk]]
set_load 0.010 [all_outputs]
set_max_transition 0.150 [current_design]
set_max_fanout 32 [current_design]
```

Before clock-tree synthesis the clock is ideal, so the setup uncertainty carries an estimate of the skew still to come (here 50 ps on top of 30 ps of jitter and margin); after CTS the tool switches to propagated clocks and the uncertainty shrinks

to 30 ps. The driving cell and load model the neighbouring blocks' gates and wires, and the transition and fanout limits keep every net inside the range over which the library's tables were characterised.

Tightening the clock constraint makes synthesis choose faster architectures and larger cells, and area rises steeply near the fastest achievable delay: a 32-bit adder in a 28 nm library occupies about 150 µm² as a ripple-carry adder at 2 ns but about three times that as a parallel-prefix adder at 250 ps. Over-constraining a block wastes area and leakage.

Worked example — Estimating the radar block's area. Synthesis reports 2.0 million NAND2-equivalent gates of logic and 4 Mbit of SRAM. At 28 nm a NAND2 occupies about 0.5 µm², and placement can use about 70 % of a row's area before routing congestion becomes unmanageable, so the logic needs

$$A_{\text{logic}} = \frac{2.0 \times 10^{6} \times 0.5\ \mu\text{m}^2}{0.70} = 1.43\ \text{mm}^2$$

The SRAM, built from 0.127 µm² high-density bitcells with an array efficiency (bitcells over total macro area, including decoders, sense amplifiers and redundancy) of about 60 %, needs

$$A_{\text{SRAM}} = \frac{4 \times 1\,048\,576 \times 0.127\ \mu\text{m}^2}{0.60} = 0.89\ \text{mm}^2$$

About 2.3 mm² in all, plus 15–25 % for power straps, macro halos and wiring channels.

34.4 Physical design

Physical design turns the netlist into polygons in stages, each fixing some decisions and passing better estimates to the next.

Floorplanning

The **floorplan** fixes the die or block outline, the I/O pads or bumps, the locations of hard macros (SRAMs, PLLs, PHYs, analog blocks) and the standard-cell regions (Figure 34.2). Macros go to block edges with pins facing the logic they serve; blocks that exchange many signals sit side by side, because a wire across a 10 mm die costs several cycles at 1 GHz (Section 34.5); noisy digital blocks are separated from analog ones by guard rings, separate supplies and deep-n-well isolation; and switchable or separately supplied power domains get their own regions (Section 34.7).

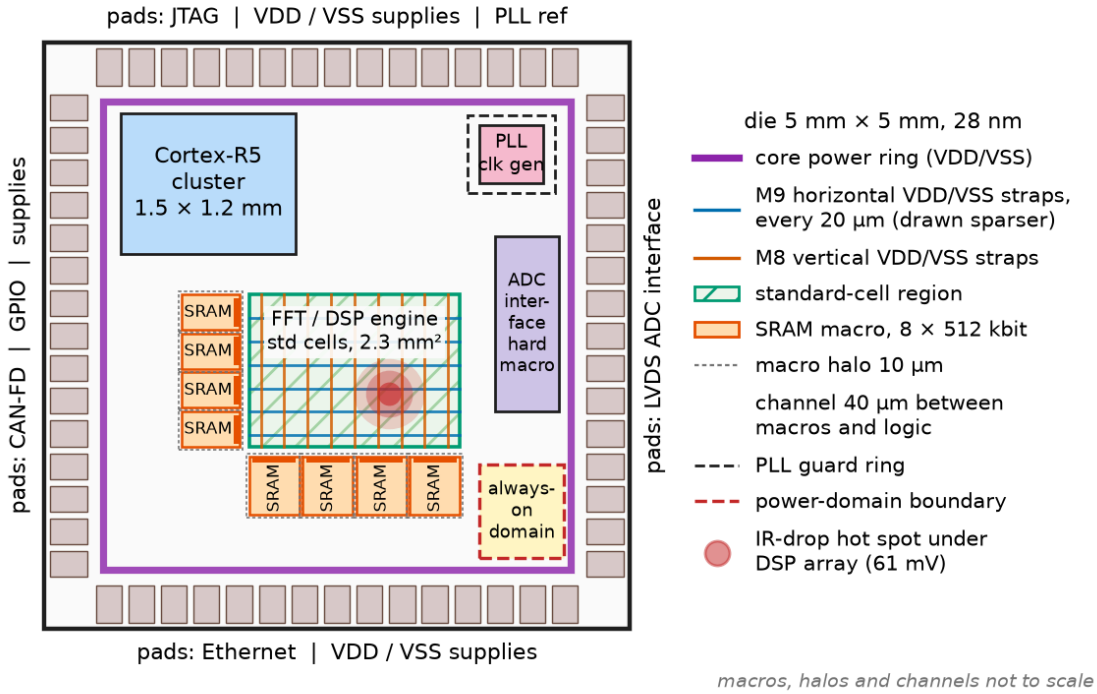


Figure 34.2 — Floorplan of a 5 mm x 5 mm 28 nm radar SoC with hard macros, standard-cell regions and the power grid.

Power-grid design and IR drop

Every cell draws current through a metal mesh: a core ring, wide straps on the thick top layers, narrower straps below and the rails of the cell rows on metal 1, joined by via stacks. The mesh resistance causes a static **IR drop**; package inductance against on-chip decoupling causes a dynamic droop when activity changes suddenly. Near 0.9 V a 28 nm gate slows by roughly 1–1.5 % per 10 mV lost, so sign-off assumes the worst supply the grid delivers; a common budget is 5 % of V_{DD}, static plus dynamic.

Worked example — Static IR drop in a strap grid. The DSP region is 2 mm wide and draws 0.5 A at 0.9 V. Vertical VDD straps on a thick metal layer (sheet resistance 0.02 Ω/□) are 2 µm wide, run 2 mm between core rings on both sides, and repeat every 20 µm, giving 100 straps each carrying $0.5/100 = 5$ mA spread uniformly along its length. Each strap has a resistance

$$R_s = 0.02\ \Omega \times \frac{2000\ \mu\text{m}}{2\ \mu\text{m}} = 20\ \Omega$$

and a line fed at both ends with uniformly distributed load current I has its largest drop at its centre, $\Delta V = I R_s / 8$:

$$\Delta V = \frac{5\ \text{mA} \times 20\ \Omega}{8} = 12.5\ \text{mV}$$

The VSS grid adds the same, so cells at the centre see a 25 mV (2.8 %) lower supply — inside a 5 % budget. The opening example's 61 mV droop came from a hot spot drawing three times the average current density; doubling the strap density there fixed it at the cost of routing tracks. Flip-chip packages, with a bump every 150–200 µm, feed the grid from above and cut the drop by an order of magnitude.

> **Status of the principle: Static IR-drop analysis**
> *Approximate*. The grid is a linear resistive network solved by nodal analysis (*Circuits, Signals and Communications*, Ch. 3) with cell currents averaged over a cycle; it misses the dynamic droop set by package inductance, decoupling capacitance and simultaneous switching, which needs a time-domain (vectored or vectorless) analysis.

The power grid also has to survive **electromigration**: the average current density in each strap and via must stay below the foundry's limit (of order 1–2 mA/μm of width for Cu at 105–125 °C), set from Black's equation for the median time to failure (*Materials Science and Engineering*, Ch. 44 §44.8).

34.5 Placement and routing

Placement

Placement assigns each cell a legal site in a row. *Global placement* minimises an estimate of total wirelength — usually each net's half-perimeter bounding box (HPWL) — while spreading cells out of overfilled regions, by analytic optimisation (the gradient methods of *The Mathematics Universe*, Ch. 23); *legalisation* snaps cells to sites and *detailed placement* swaps neighbours. Critical nets get higher weights, and cells are resized and buffered as placement proceeds. The key outputs are timing with real distances and a **congestion map** showing where more wires want to pass than there are tracks; congestion is fixed by lowering local utilisation or moving macros, not by hoping the router copes.

Clock-tree synthesis

Until now the clock has been ideal. **Clock-tree synthesis** (CTS) builds the real buffer-and-wire network from the PLL output to the 10^5–10^6 flip-flops of a block (Figure 34.3). Its targets are **skew** between communicating flip-flops below 20–50 ps at 28 nm; small **insertion delay** (typically 300–600 ps for a large block), because every tree buffer varies with PVT and only the shared part of the tree is common to launch and capture; and sink **transition times** within the library's characterised range (40–80 ps). Structures range from symmetric H-trees through clustered buffered trees to **clock meshes**, which short the tree's last level together and trade 20–40 % more clock power for skew below 10 ps. Toggling every cycle into every flip-flop, the clock network typically takes 25–45 % of a block's dynamic power before gating.

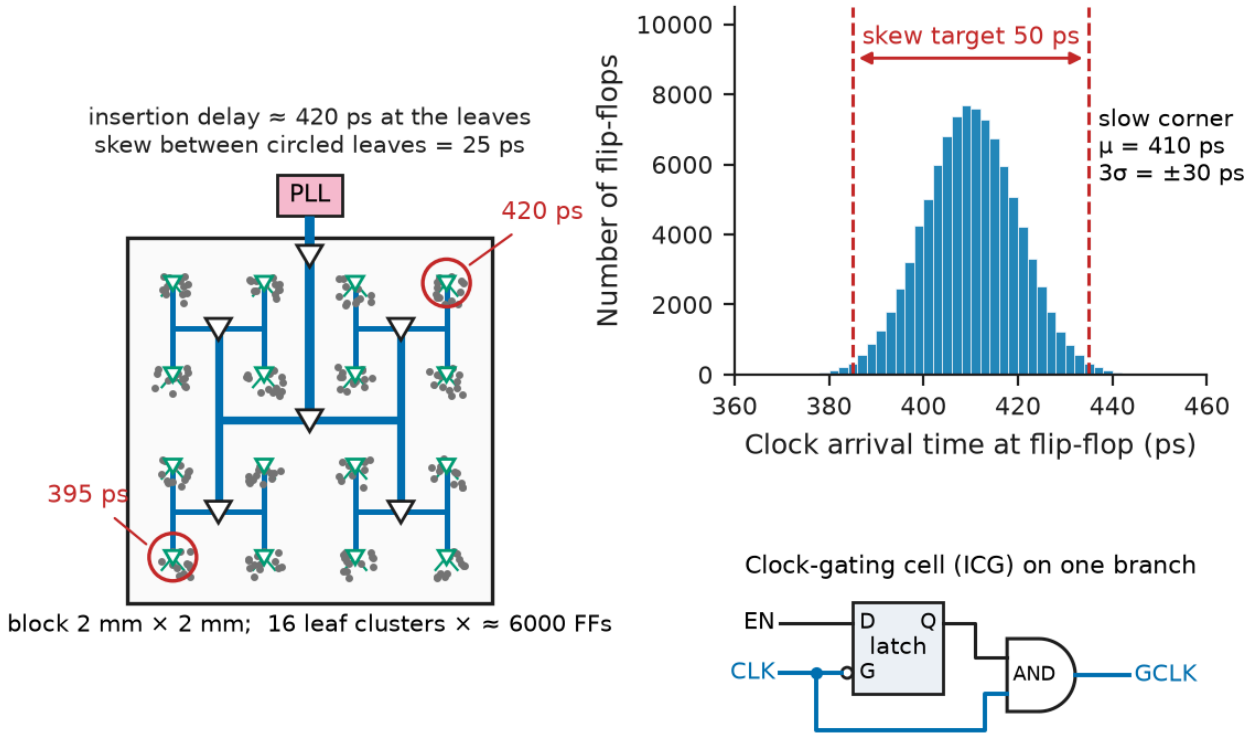


Figure 34.3 — A buffered H-tree clock distribution and the resulting arrival-time spread at the flip-flops.

Delivering the clock deliberately late to the capture flip-flop of a long path borrows time from the next stage; tools use this **useful skew** routinely to fix setup, at the cost of hold margin downstream.

Routing

Routing connects every net with metal and vias under the design rules of Section 34.8: a *global router* assigns nets to coarse tiles and layers, and the *detailed router* draws exact wires obeying every spacing, enclosure and multiple-patterning rule. A 28 nm process offers 9–11 copper layers: tightly pitched lower layers (about 90–100 nm) for local wiring, intermediate layers at twice the pitch, and thick top layers for power, clock trunks and long buses. Resistance per unit length rises steeply as wires shrink, because copper's effective resistivity climbs from 1.7 μΩ·cm in bulk to 4–8 μΩ·cm in narrow lines (*Materials Science and Engineering*, Ch. 50 §50.9), while capacitance stays near 0.15–0.25 fF/μm on every layer.

Table 34.3 — Representative on-chip wire parasitics (typical, 28 nm class).

Layer class	Width/pitch (nm)	r (Ω/μm)	c (fF/μm)	0.38 rc (ps/mm²)	Use
Local (M1–M3)	45/90	6–10	0.18–0.22	≈ 600	Inside cells and between neighbours
Intermediate (2×)	100/200	1.5–2.5	0.18–0.22	≈ 150	Block-level routes up to ~1 mm
Top thick	400/800 and wider	0.05–0.2	0.20–0.25	4–20	Clock trunks, power, long buses

Wire delay and repeaters

A wire of length L with resistance r and capacitance c per unit length is a distributed RC line with 50 % delay $0.38\,rcL^2$, against $0.69RC$ for a lumped RC (first-order transients: *Circuits, Signals and Communications*, Ch. 6; inductance and

transmission-line behaviour matter only for wide, low-resistance top-layer wires with fast edges, *Circuits, Signals and Communications*, Ch. 14 §14.10). Driven by a gate of output resistance R_d into a load C_L, the Elmore-type estimate is

$$t_{50} \approx 0.69 R_d\,(cL + C_L) + 0.69\,rL\,C_L + 0.38\,rcL^2$$

The quadratic term makes wire delay overtake gate delay beyond a few hundred micrometres. Cutting the wire into N segments driven by repeaters h times a minimum inverter (resistance R_0, input and parasitic capacitances C_0, C_p) makes growth linear; minimising delay gives

$$N_{\text{opt}} = \sqrt{\frac{0.38\,rcL^2}{0.69\,R_0(C_0 + C_p)}}, \qquad h_{\text{opt}} = \sqrt{\frac{R_0\,c}{r\,C_0}}$$

> **Status of the principle: Distributed-RC wire delay and optimal repeaters**
> *Approximate*. Derived from the Elmore delay of an RC ladder with linearised driver resistances; accurate to 10–20 % for on-chip wires with R_{wire} comparable to or larger than the driver resistance. It fails for low-resistance top-metal wires (inductance, $L\,di/dt$ ringing), ignores slope effects and coupling capacitance to switching neighbours, and assumes repeaters can be placed anywhere along the route.

Worked example — A 2 mm route across the DSP block. Take a 28 nm minimum inverter with $R_0 = 5$ kΩ and $C_0 = C_p = 1$ fF, which gives an FO4 delay of $0.69 \times 5\,\text{k}\Omega \times (1+4)\,\text{fF} = 17$ ps, consistent with Table 34.2. On an intermediate layer with $r = 2$ Ω/µm and $c = 0.2$ fF/µm, the 2 mm wire has $R_w = 4$ kΩ and $C_w = 400$ fF, and its intrinsic delay alone is $0.38 \times 4\,\text{k}\Omega \times 400\,\text{fF} = 608$ ps. Driven by a single 22× inverter it takes 738 ps. With repeaters,

$$N_{\text{opt}} = \sqrt{\frac{0.38 \times 4\,000 \times 400 \times 10^{-15}}{0.69 \times 5\,000 \times 2 \times 10^{-15}}} = 9.4, \qquad h_{\text{opt}} = \sqrt{\frac{5\,000 \times 0.2}{2 \times 1}} = 22$$

Nine 22× repeaters bring the delay to 253 ps — a quarter of the cycle, too much to share a stage with logic, so the route gets its own pipeline register. On a local layer with $r = 8$ Ω/µm the unrepeated delay would be 2.7 ns, and even 19 optimal repeaters only reach 506 ps. This is why routers promote long nets to upper layers and why physical synthesis inserts tens of thousands of buffers into a large block.

At pitches below 100 nm most of a wire's capacitance is to its neighbours, so a switching *aggressor* injects a glitch into a quiet *victim* or changes its delay when both switch (the effective coupling capacitance ranges from zero to twice its value). Sign-off therefore uses crosstalk-aware delay calculation, and routers fix violations by spacing, shielding clocks with grounded wires or changing layers.

34.6 Timing analysis

Static timing analysis

Static timing analysis (STA) checks every path against its constraints without simulating any input pattern. Netlist and parasitics become a directed acyclic graph of cell pins, cell arcs and net delays; the tool propagates the latest and earliest **arrival times** forward from every start point (launching clock pins, inputs) and **required times** backward from every end point (capturing data pins, outputs). Their difference is the **slack**, negative for a violation. The analysis is linear in graph size, so a million-cell block is checked at a dozen corners in minutes — exhaustively, as simulation never could.

The setup and hold constraints of a register-to-register path, with their skew and jitter terms and the reason hold violations survive any clock slow-down, are derived in Chapter 14 §14.8. On chip, skew is no longer a single number: it is the difference between the propagated latencies t_L (launch) and t_C (capture) of two branches of the clock tree, and the remaining jitter and margin are lumped into an uncertainty t_{unc}. STA then evaluates, for every path,

$$\text{setup slack} = \big(T + t_C - t_{\text{unc}} - t_{\text{setup}}\big) - \big(t_L + t_{cq} + t_{\text{logic}}\big)_{\max}, \qquad \text{hold slack} = \big(t_L + t_{cq} + t_{\text{logic}}\big)_{\min} - \big(t_C + t_{\text{hold}} + t_{\text{unc}}\big)$$

with maximum delays at the slow corner for setup and minimum delays at the fast corner for hold (Figure 34.4).

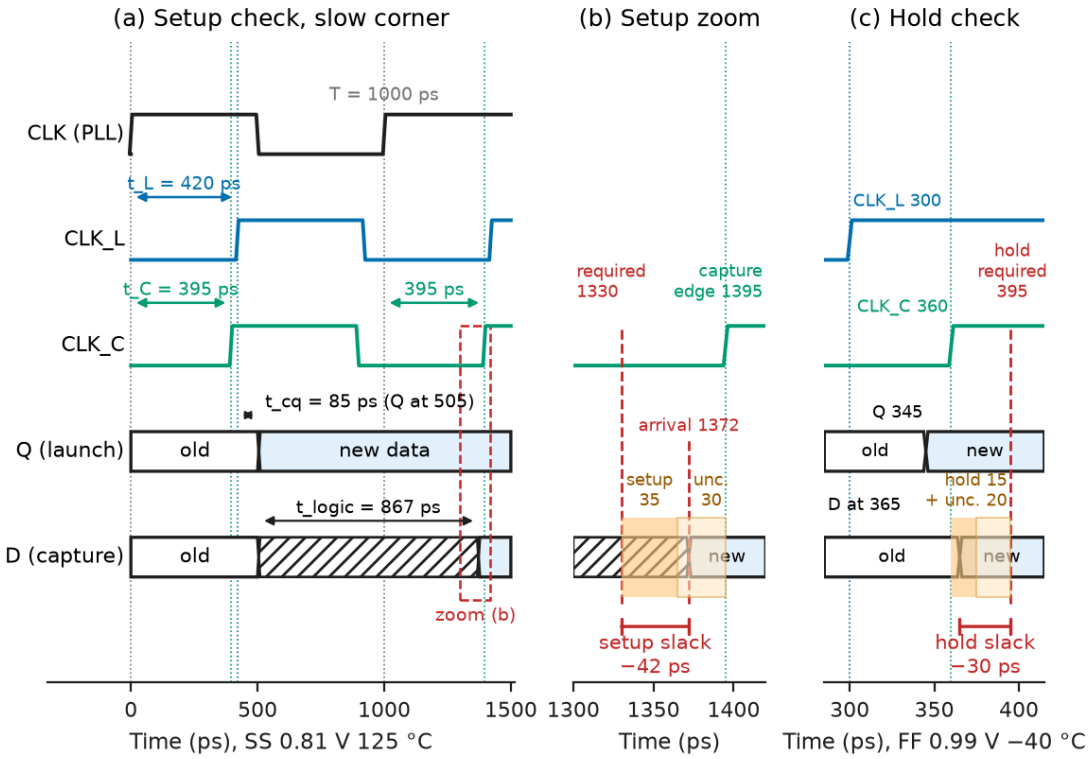


Figure 34.4 — Setup and hold checks on a register-to-register path with clock skew and uncertainty.

Worked example — The radar path at two corners. At the slow corner (SS, 0.81 V, 125 °C) the failing path has $t_L = 420$ ps, $t_{cq} = 85$ ps, $t_{\text{logic}} = 867$ ps, $t_C = 395$ ps, $t_{\text{setup}} = 35$ ps and a setup uncertainty of 30 ps, with $T = 1000$ ps:

$$\text{required} = 1000 + 395 - 30 - 35 = 1330 \text{ ps}, \qquad \text{arrival} = 420 + 85 + 867 = 1372 \text{ ps}$$

so the setup slack is -42 ps. Swapping the six cells on the path to low-V_t versions and upsizing two of them saves 60 ps, giving $+18$ ps at a cost of extra leakage on those cells only. At the fast corner (FF, 0.99 V, −40 °C) a short path between neighbouring registers has $t_L = 300$ ps, $t_{cq,\text{min}} = 45$ ps and only 20 ps of logic, while its capture clock arrives late, $t_C = 360$ ps, because the capture flip-flop sits at the far end of a different clock branch; with $t_{\text{hold}} = 15$ ps and 20 ps hold uncertainty,

$$\text{hold slack} = (300 + 45 + 20) - (360 + 15 + 20) = -30 \text{ ps}$$

The fix is a 35 ps delay cell in the data path (after checking setup at the slow corner) or rebalanced clock branches.

Listing 34.3 is the core of a block-based STA engine: a topological traversal propagating the latest arrival time to each endpoint. Real tools add rise and fall transitions, slews, early arrivals and millions of nodes.

```
 # Tiny block-based STA: late arrival times on a DAG, setup slack at each endpoint.
from graphlib import TopologicalSorter

T_CLK, UNC = 1000.0, 30.0                       # ps
launch = {"ffA": 420.0, "ffB": 410.0}           # clock latency at launch flops
capture = {"ffC": (395.0, 35.0), "ffD": (430.0, 35.0)}   # (latency, setup)
CLK_Q = 85.0
arcs = {                                        # (from, to): delay in ps (SS corner)
    ("ffA", "u1"): 210, ("ffB", "u1"): 160, ("u1", "u2"): 240,
    ("u2", "u3"): 190, ("u3", "ffC"): 227, ("u2", "ffD"): 300, ("ffB", "ffD"): 520,
}
preds = {}
for (a, b) in arcs:
    preds.setdefault(b, set()).add(a)
    preds.setdefault(a, set())

arrival = {}
for node in TopologicalSorter(preds).static_order():
    if node in launch:
        arrival[node] = launch[node] + CLK_Q
    else:
        arrival[node] = max(arrival[p] + arcs[(p, node)] for p in preds[node])

for ff, (lat, setup) in capture.items():
    required = T_CLK + lat - UNC - setup
    print(f"{ff}: arrival {arrival[ff]:6.0f} ps  required {required:6.0f} ps"
          f"  slack {required - arrival[ff]:+5.0f} ps")
```

It reproduces the −42 ps slack into `ffC` and reports +110 ps for `ffD`, whose worst path runs through `u2`, not the 520 ps arc from `ffB`.

Corners, modes and on-chip variation

The foundry supplies libraries at **PVT corners** and interconnect models at **RC corners**, and sign-off checks every relevant combination in every operating mode (functional, scan, low-power): multi-corner multi-mode (MCMM) analysis.

Table 34.4 — Representative sign-off corners for a 28 nm automotive block (nominal 0.90 V).

Corner	Process	V_{DD}	T_j	Interconnect	Main check
SS 0.81 V 125 °C	slow n, slow p	−10 %	125 °C (150 °C for AEC-Q100 Grade 1, rated 125 °C ambient; higher for Grade 0)	RC-worst	Setup
SS 0.81 V −40 °C	slow n, slow p	−10 %	−40 °C	RC-worst	Setup (temperature inversion)
TT 0.90 V 25 °C	typical	nominal	25 °C	typical	Power reporting
FF 0.99 V −40 °C	fast n, fast p	+10 %	−40 °C	C-best	Hold
FF 0.99 V 125 °C	fast n, fast p	+10 %	125 °C	C-best	Hold, leakage, EM

At low supply voltage a gate can be slower at −40 °C than at 125 °C (**temperature inversion**: the threshold rises faster than mobility improves), so both extremes are checked for setup.

Corners model die-to-die variation. **On-chip variation** (OCV) within a die is handled by derates. A flat ±3 % derate on the radar path (late on launch clock and data, early on capture clock) costs $0.03 \times 1372 + 0.03 \times 395 = 53$ ps of slack. The clock path shared by launch and capture cannot be fast and slow at once, so **clock-reconvergence pessimism removal** (CRPR) returns $0.06 \times 300 = 18$ ps for a 300 ps common path. Flat derates over-penalise long paths, whose random stage variations average out; advanced and parametric OCV (AOCV, POCV) give each cell a mean and standard deviation and combine paths statistically.

> **Status of the principle: Corner-based timing sign-off**
> *Approximate.* Corners bound global variation by placing all devices at the same extreme, which no real die does; OCV derates and statistical models add local variation. The method is deliberately conservative, and the margin it leaves is part of the price of first-silicon success.

34.7 Power estimation

Where the power goes

The switching power $P_{\text{dyn}} = \alpha C V_{DD}^2 f$, short-circuit power and the leakage mechanisms are derived in Chapter 33; the task here is to estimate them for millions of cells before silicon exists, well enough to choose a package, a regulator and a battery. Tools combine the netlist with extracted capacitances, the Liberty power tables at the chosen corner, and **switching activity** on every net — propagated statistically (vectorless) or recorded from simulation of real workloads (VCD, FSDB or SAIF files). Activity dominates the accuracy: the same block idling and running a dense FFT differs 3–10× in dynamic power. Gate-level simulation with delays also captures **glitch power**, 10–30 % of the power of deep arithmetic such as multipliers.

Worked example — Power budget of the radar DSP block. At the typical corner (0.9 V, 1 GHz) the block has 2 million gates whose nets carry on average 1.5 fF each (pin plus wire) and toggle with activity $\alpha = 0.1$, and 150 000 flip-flops each loading the clock network with about 3 fF (pin plus its share of tree wiring and buffers), with the clock's $\alpha = 1$:

$$P_{\text{data}} = 0.1 \times 2 \times 10^6 \times 1.5\ \text{fF} \times (0.9\ \text{V})^2 \times 1\ \text{GHz} = 0.243\ \text{W}$$

$$P_{\text{clock}} = 150\,000 \times 3\ \text{fF} \times (0.9\ \text{V})^2 \times 1\ \text{GHz} = 0.365\ \text{W}$$

Ungated, the clock is 60 % of dynamic power; gating 60 % of the registers on an average cycle cuts it to 0.146 W. With 10 % for short-circuit current the dynamic total is $1.1 \times (0.243 + 0.146) = 0.43$ W. Leakage at about 5 nW per gate is 10 mW at 25 °C but ten times that at a 125 °C junction, so the hot total is about 0.53 W. Package and regulator are sized for this plus margin, with the thermal-resistance model of *Engineering Physics*, Ch. 14 §14.7.

Table 34.5 — Effect of common low-power techniques (typical ranges).

Technique	What it saves	Typical saving	Cost
Clock gating (ICG insertion)	Clock and register dynamic power	20–50 % of block dynamic power	Small area; gating-enable timing
Multi-V_t cell assignment	Leakage	2–5× lower leakage, few % delay	Library flavours, sign-off effort
Operand isolation, data gating	Glitch and datapath power	5–20 % in arithmetic blocks	Extra gates
Dynamic voltage and frequency scaling (DVFS)	Dynamic power $\propto V^2 f$	2–4× at reduced performance	Regulator, characterisation at several voltages
Power gating (header/footer switches)	Leakage when idle	10–50× lower idle leakage	Switch area (5–10 %), wake-up time µs, state retention
Multiple voltage domains	Dynamic power of slow blocks	20–40 %	Level shifters, more corners

Supply-changing techniques are described in a **power-intent** file (IEEE 1801 UPF) that every tool reads, so all agree which domains are on and which signals must be isolated.

> **Status of the principle: Leakage–temperature dependence**
> *Empirical.* Subthreshold leakage rises roughly 5–20× between 25 °C and 125 °C junction temperature, depending on threshold voltage, process and corner; tools use characterised tables, not a formula, and the dependence couples power, temperature and package into one loop (thermal runaway in the worst case).

34.8 Design-rule checking

Design rules

A foundry guarantees its process only for layouts that obey its **design rules**, thousands of geometric constraints coded as a rule deck for a physical-verification tool. They exist because the steps of *Materials Science and Engineering*, Ch. 50 §50.7–50.10 — lithography near its resolution limit, etching with finite anisotropy, CMP with pattern-dependent dishing and erosion — reproduce shapes reliably only within limits.

Table 34.6 — Kinds of design rule (values illustrative of a 28 nm-class process, not from any foundry's manual).

Rule	Example	Physical reason
Minimum width	M1 width ≥ 45 nm	Line must print and not neck or open
Minimum spacing	M1 space ≥ 45 nm; larger next to wide lines	Bridging defects, lithographic resolution
Enclosure	Via must be enclosed by metal ≥ 10 nm on two sides	Overlay error between layers
Minimum area	M1 area ≥ 0.01 µm²	Small islands lift off or do not print
Density	Each layer 20–80 % filled in every 50 µm window	Uniform CMP and etch loading
Antenna ratio	Metal area connected to a gate ≤ 400× gate area	Plasma charging of thin gate oxide
Multiple patterning	Same-colour M1 shapes ≥ 2 pitches apart	Two masks split one dense layer
End-of-line, tip-to-tip	Line end to line end ≥ 60 nm	Line-end shortening in lithography

Design-rule checking (DRC) runs these rules on the full layout and reports every violation; a chip is taped out only "DRC clean", with any waived violation agreed in writing with the foundry.

Antenna and density rules

During plasma etching a wire collects charge; if it connects only to a gate (the diffusion that would bleed it off is reached through a metal layer not yet built), the charge stresses the gate oxide (plasma-induced damage, *Materials Science and Engineering*, Ch. 50 §50.12). The **antenna rule** limits the ratio of connected metal area to gate area at each layer.

Worked example — An antenna violation. A 600 µm signal wire on metal 2, 0.05 µm wide, runs from a driver to the input of a minimum inverter with total gate area $0.3\ \mu\text{m} \times 0.03\ \mu\text{m} = 0.009\ \mu\text{m}^2$. While metal 2 is etched, the connection to the driver's drain exists only through metal 3. The antenna ratio is

$$\frac{0.05 \times 600\ \mu\text{m}^2}{0.009\ \mu\text{m}^2} = 3\,333$$

eight times an illustrative limit of 400. The router either breaks the wire near the gate and hops briefly to metal 3 (a "jumper"), so the long segment is not attached to the gate while it is etched, or adds an antenna diode beside the gate. The opening example's 1 400 violations were of this kind and were cleared automatically.

Density rules exist because CMP polishes sparse regions faster than dense ones (*Materials Science and Engineering*, Ch. 50 §50.12); after routing, tools insert **dummy fill** into empty areas and re-extract timing, because fill adds capacitance.

LVS, ERC and extraction

Layout-versus-schematic (LVS) extracts devices and connectivity from the polygons and compares them with the netlist, catching shorts, opens, missing well taps and mis-connected macros: it is the definitive check that the drawn chip is the designed chip. **Electrical rule checks** (ERC) find floating gates, contending drivers, missing substrate ties (latch-up) and unprotected I/O devices (ESD, tested to JEDEC JS-001 and JS-002). **Parasitic extraction** (PEX) computes every wire's R and C from the layer stack and writes them (SPEF) for sign-off timing, power and IR drop, which are only as correct as the extraction.

34.9 Verification

Functional verification

A timing-clean, DRC-clean chip can still be wrong. Test benches, assertions, coverage, formal property checking and equivalence checking — and what each finds and misses — are developed in Chapter 19 §19.9–19.11. What changes for an ASIC is scale and irreversibility: functional verification takes more than half the engineering effort of a large SoC, and a missed bug costs a re-spin (Table 34.2). An SoC sign-off therefore stacks the methods:

- **Block-level constrained-random simulation** in a reusable methodology (UVM, IEEE 1800.2), with reference models, scoreboards and functional-coverage closure as the tape-out criterion; SoC-level tests run the real boot code and drivers on the processor models.
- **Formal apps** beyond property checking: structural **clock- and reset-domain-crossing** checks over the whole netlist, connectivity checks of the top-level wiring and pad multiplexing, and X-propagation analysis.
- **Logical equivalence checking** at every hand-off — RTL to synthesised netlist, netlist to scan-inserted netlist, placed and clock-tree netlist, and every engineering change order (ECO) — so that gate-level simulation is needed only for resets, power-up, asynchronous interfaces and test modes.
- **Low-power verification** of the UPF intent: every signal leaving a switchable domain is isolated, every level crossing has a shifter, retention registers save and restore correctly through a power-down sequence.
- **Emulation and FPGA prototyping** at 1–100 MHz, to boot operating systems and run application software before silicon (Chapter 35).

Design for test

Verification asks whether the design is right; **manufacturing test** asks whether each die is the design, since one particle can short two wires on one die in thousands. **Design for test** (DFT) makes testing a million-gate block on automatic test equipment tractable. **Scan** replaces every flip-flop by one with a multiplexed input and stitches them into shift chains: the tester shifts a pattern in, pulses the functional clock once or twice, and shifts the response out, with on-chip compression cutting test time 10–100×. **ATPG** computes patterns for fault models — stuck-at, transition (two at-speed pulses), path-delay and cell-aware faults inside cells. **Memory BIST** runs march algorithms on every SRAM and repairs failing rows or columns from spares by blowing fuses, and **boundary scan** (IEEE 1149.1 JTAG) tests board interconnect.

The quality of a test is its **fault coverage** T_c, the fraction of modelled faults detected. The defect level — the fraction of dies that pass the test but are faulty — follows approximately the Williams–Brown relation

$$DL = 1 - Y^{\,1 - T_c}$$

where Y is the process yield.

> **Status of the principle: Williams–Brown defect level**
> *Approximate.* It assumes that faults are equally likely and independent and that fault coverage equals defect coverage; real defects cluster and are not all modelled by stuck-at faults, so measured field-return rates are used to calibrate it.

Worked example — Test coverage for an automotive part. With a yield of 80 %, stuck-at coverage of 99 % gives

$$DL = 1 - 0.8^{\,0.01} = 2.23 \times 10^{-3} = 2\,230 \text{ DPPM}$$

(defective parts per million); 99.9 % gives 223 DPPM and 99.99 % gives 22 DPPM. Automotive customers expect below 1 DPPM, so stuck-at testing alone is never enough: the radar processor's test program adds at-speed transition and cell-aware patterns, IDDQ screening, memory BIST at several voltages and a voltage-stress step for latent defects, following AEC-Q100 and ISO 26262.

34.10 Fabrication constraints

Yield

The defects that process steps introduce are described in *Materials Science and Engineering*, Ch. 50 §50.12. The designer sees them through the density D_0 of **killer defects** — those that cause a short or open. If they fall independently and uniformly, a die of area A escapes them all with the Poisson yield

$$Y = e^{-AD_0}$$

Real defects cluster on wafers (a scratch, a contaminated region, an edge effect), which leaves more dies untouched than Poisson predicts. The negative-binomial model with clustering parameter α (typically 1–4) captures this:

$$Y = \left(1 + \frac{AD_0}{\alpha}\right)^{-\alpha}$$

which tends to the Poisson form as $\alpha \to \infty$ (distributions from *The Mathematics Universe*, Ch. 20 §20.4). Mature processes reach $D_0 \approx 0.03$–0.1 cm^{-2}; new nodes start higher. **Parametric** losses (dies that work but miss frequency or leakage limits) and systematic, layout-sensitive losses come on top.

> **Status of the principle: Defect-limited yield models**
> *Approximate.* Poisson and negative-binomial yields follow from assuming randomly located point defects with a fixed (or gamma-distributed) density; D_0 and α are fitted to each fab's data, and the models do not describe parametric or systematic yield loss.

The number of whole dies on a wafer of diameter d is approximately $\pi(d/2)^2/A - \pi d/\sqrt{2A}$, the second term counting partial dies lost at the edge. Listing 34.4 combines the two into a cost per good die.

```
 # Gross dies per wafer, defect-limited yield and cost per good die.
import math

def dies_per_wafer(area_mm2, d_mm=300.0):
    return int(math.pi * (d_mm / 2) ** 2 / area_mm2 - math.pi * d_mm / math.sqrt(2 * area_mm2))

def yield_nb(area_mm2, d0_per_cm2, alpha=3.0):      # negative binomial (clustered defects)
    return (1 + area_mm2 / 100 * d0_per_cm2 / alpha) ** -alpha

for node, wafer_usd, d0 in (("28 nm", 3000, 0.05), ("5 nm", 17000, 0.10)):
    for area in (25, 100, 600):
        n = dies_per_wafer(area)
        y = yield_nb(area, d0)
        print(f"{node:5s} {area:4d} mm2: {n:5d} dies, yield {y:5.1%}, "
              f"{wafer_usd / (n * y):8.2f} USD per good die")
```

Table 34.7 — Output of Listing 34.4 (300 mm wafer, $\alpha = 3$; wafer prices are representative).

Node (wafer price)	Die area (mm²)	Gross dies	Yield	Cost per good die (USD)
28 nm (3 000 USD), $D_0 = 0.05$ cm^{-2}	25	2 694	98.8 %	1.13
	100	640	95.2 %	4.93
	600	90	75.1 %	44.37
5 nm (17 000 USD), $D_0 = 0.10$ cm^{-2}	25	2 694	97.5 %	6.47
	100	640	90.6 %	29.31
	600	90	57.9 %	326.40

Worked example — Why large chips are split into chiplets. A 600 mm² data-centre accelerator at 5 nm costs 326 USD per good die before packaging and test (Table 34.7): a 24-fold area increase over a 25 mm² die costs 50 times as much, because both the gross die count and the yield fall. Splitting it into four 150 mm² chiplets, each with gross count $\pi \times 150^2/150 - \pi \times 300/\sqrt{300} = 471 - 54 \approx 416$ and yield $(1 + 1.5 \times 0.1/3)^{-3} = 86.4\ \%$, costs $17\,000/(416 \times 0.864) = 47.3$ USD per chiplet, or 189 USD for four — 42 % less silicon cost, partly repaid in die-to-die interfaces, advanced packaging and the energy of moving data between chiplets (Chapter 35). Large server processors and GPUs take this route; a 25 mm² radar chip gains nothing from it.

Variability

Random dopant fluctuation, line-edge roughness and work-function granularity make the thresholds of identical adjacent transistors differ, with a standard deviation that scales as the inverse square root of gate area (Pelgrom's rule):

$$\sigma_{\Delta V_t} = \frac{A_{V_t}}{\sqrt{WL}}$$

with $A_{V_t} \approx 1$–2.5 mV·µm for modern high-k metal-gate processes (and 3–5 mV·µm in older planar nodes).

> **Status of the principle: Pelgrom mismatch scaling**
> *Empirical.* The inverse-square-root dependence follows from averaging many independent microscopic fluctuations over the gate area, and the coefficient A_{V_t} is measured on test structures for each process; it under-predicts mismatch for very short channels and ignores systematic gradients across the die.

Worked example — SRAM margins. A 28 nm SRAM pass-gate transistor with $W = 0.1$ µm and $L = 0.03$ µm in a process with $A_{V_t} = 1.8$ mV·µm has

$$\sigma_{\Delta V_t} = \frac{1.8\ \text{mV}\,\mu\text{m}}{\sqrt{0.1 \times 0.03}\ \mu\text{m}} = 33\ \text{mV}$$

— over 10 % of the threshold itself. For a 90 % chance that no cell of the 4 Mbit memory of Section 34.3 fails, the per-cell failure probability must be below $0.1/4\,194\,304 = 2.4 \times 10^{-8}$: the cell must work out to 5.5σ of a Gaussian margin. Hence foundry-characterised bitcells, spare rows and columns, and an SRAM minimum voltage $V_{\min}$ that limits voltage scaling (Section 34.7) long before logic fails. The same rule sizes matched analog pairs (Chapter 10).

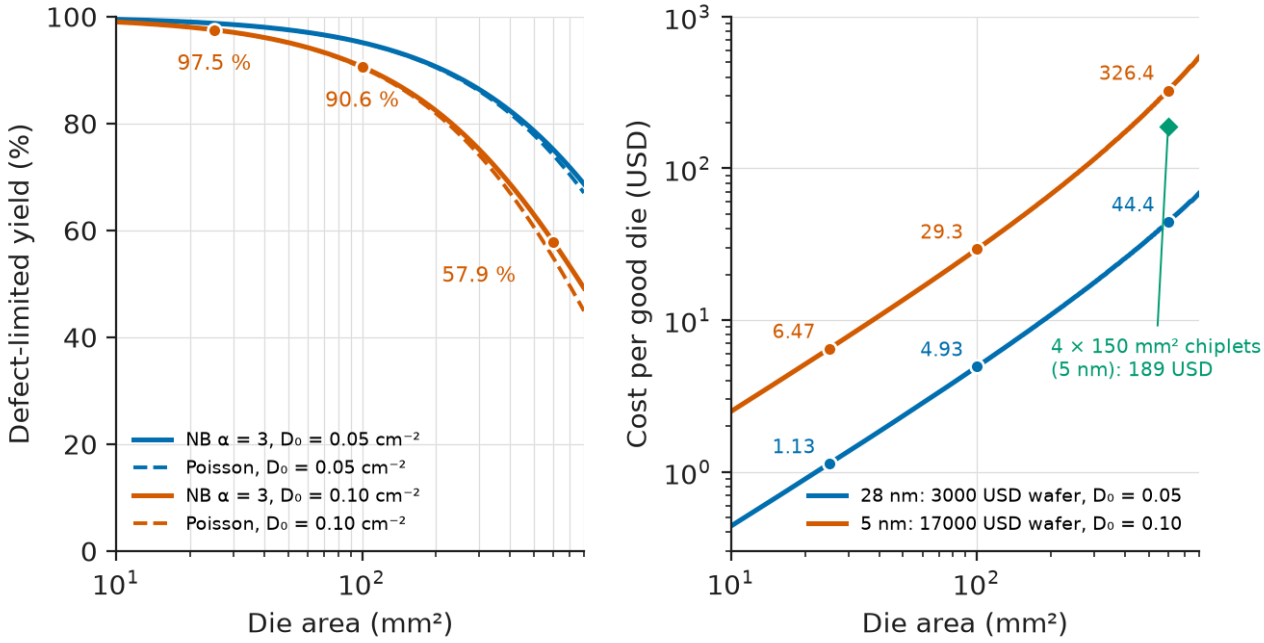


Figure 34.5 — Defect-limited yield and cost per good die versus die area for two defect densities.

Design for manufacturing and reliability

Design for manufacturing (DFM) raises yield beyond what minimum rules guarantee, by shrinking the area at risk (Figure 34.5): **redundant vias** wherever space allows, since single vias are among the commonest opens; **wire spreading** in uncongested areas to cut the critical area for particle shorts and opens; **restricted design rules** — at 28 nm and below, gates on one fixed pitch and orientation with dummy gates at cell edges, because lithography (*Materials Science and Engineering*, Ch. 50 §50.7) prints regular gratings far better than arbitrary shapes, and FinFETs quantise widths to whole fins; **lithography hotspot checks** that simulate the printed image; and density-balancing fill.

A chip must also last — ten years for consumer parts, fifteen or more for automotive. Wear-out enters as margins: electromigration limits (Section 34.4); bias-temperature instability (BTI) and hot-carrier injection, which shift thresholds over years and are covered by "aged" libraries or a few per cent of timing margin; and time-dependent dielectric breakdown, which caps the supply voltage. The physics is in *Materials Science and Engineering*, Ch. 44 §44.8; the designer keeps currents, voltages and temperatures inside the limits the foundry derives from it.

What remains open

Open question	Where it is answered
How is the CMOS gate whose delay, power and layout this chapter assumes built and sized?	Chapter 33
How are HDL descriptions, test benches and FPGA implementations written and simulated?	Chapter 19
How are FPGA fabrics organised, and when is an FPGA used instead of an ASIC?	Chapter 18; Chapter 35
How do clock-domain crossings and metastability enter the design?	Chapter 14; Chapter 15
How are the analog blocks of a mixed-signal SoC (references, PLLs, ADCs) designed?	Chapter 10
How do lithography, etching, metallization and CMP create the rules and defects used here?	*Materials Science and Engineering*, Ch. 50
What sets electromigration, BTI and dielectric-breakdown lifetimes?	*Materials Science and Engineering*, Ch. 44
How do on-chip and off-chip interconnects behave as transmission lines at high edge rates?	*Circuits, Signals and Communications*, Ch. 14
What should be built in silicon at all, and how is the split between hardware and software decided and verified?	Chapter 35

Next: Chapter 35 steps back from how a chip is built to what should be built — how computation is partitioned between processors, FPGAs, GPUs and custom accelerators, and how the whole hardware–software system is modelled and verified.

Chapter 35 — Hardware–Software Co-Design

Part VIII · Integrated Circuits and VLSI

A wireless vibration sensor bolted to a paper-mill pump classifies bearing faults with a small convolutional network. On its Cortex-M4 at 80 MHz one inference takes 56 ms and 0.68 mJ — fine at one inference a second, hopeless when the customer asks for continuous classification at 100 Hz on the same battery. The team moves to a microcontroller with a neural processing unit (NPU) whose datasheet promises 250 times the throughput, and measures a speed-up of only 38: the network now runs in 0.2 ms, but the FFT that prepares its input still runs on the CPU and takes 1.2 ms. In a data centre two buildings away, a recommendation model runs on a GPU rated at 624 TOPS and achieves less than 1 % of that, because at batch size one every weight is fetched from memory to be used exactly once. Both teams bought the right hardware for the wrong part of their problem. Deciding which computation goes where — processor, FPGA, GPU, NPU or custom silicon — and predicting, before committing, what the whole system will achieve, is hardware–software co-design. It is the last step of the book's argument: every abstraction built in the earlier chapters is now a resource to be allocated.

> **Which parts of a computation belong in software on a processor, and which in an FPGA, a GPU, an NPU or an ASIC — and what will the split really gain?**
> **How do memory traffic, numerical precision and deadlines limit an accelerated system, and how is the combination verified before it is built?**

Concept	Engineering destination
Partitioning and offload break-even	Deciding what to accelerate; sizing DMA and interfaces
Processor, FPGA, GPU, ASIC trade-offs	Platform choice by throughput, energy per operation, NRE and volume
Amdahl's law, custom instructions, HLS	Realistic speed-up estimates; accelerator design from C++
Memory hierarchy and data-movement energy	Tiling, data reuse, scratchpads, DMA double buffering
Roofline and Little's law	Diagnosing memory- versus compute-bound kernels; required concurrency
Quantisation and int8 inference	Neural networks on MCUs, NPUs and FPGAs within flash, RAM and energy budgets
Real-time constraints with accelerators	Worst-case latency budgets for control loops and perception pipelines
Co-simulation and virtual prototypes	Running software on hardware that does not yet exist

Roadmap. What to put where (35.1) → the platforms compared with numbers (35.2) → how much acceleration buys, and how accelerators are built (35.3) → why moving data costs more than computing on it (35.4) → predicting performance before building (35.5) → deploying neural networks on small hardware (35.6) → meeting deadlines with accelerators in the loop (35.7) → verifying hardware and software together (35.8).

> **Boundary.**
> Qualitative hardware–software partitioning for microcontroller products — which jobs go to on-chip peripherals — is in Chapter 25 §25.2; FPGA architecture in Chapter 18; caches and memory hierarchy in Chapter 16 §16.7–16.8; parallel computing and profiling in Chapter 24; scheduling, WCET and RTOS behaviour in Chapter 27; how a custom chip is designed in Chapter 34. Quantisation noise and fixed-point formats are from *Circuits, Signals and Communications*, Ch. 21 §21.2 and §21.5, which also lists DSP implementation platforms qualitatively (§21.3). This chapter makes those choices quantitative.

35.1 Partitioning computation

From profile to task graph

Co-design starts from a working software implementation and a measurement, not from a block diagram. The application is written (usually in C, C++ or Python with NumPy), run on representative data, and profiled (Chapter 24). The profile is redrawn as a **task graph**: nodes are computations with their operation counts, edges are the data passed between them with their sizes. For the vibration sensor: ADC samples (4 096 × 16 bit) → windowing and 1 024-point FFTs → log-magnitude spectrogram (49 × 40 values) → CNN (2.7 million multiply–accumulates, MACs) → decision → radio message. Figure 35.1 maps such a graph onto a typical SoC.

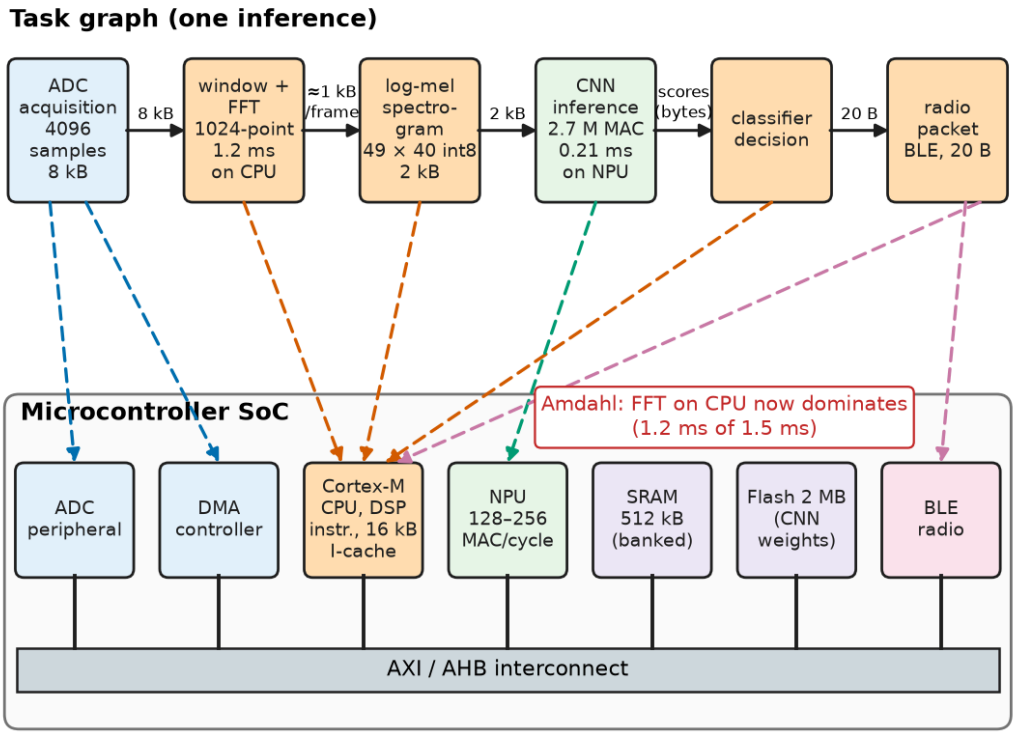


Figure 35.1 — A task graph for a sensor-inference pipeline mapped onto the resources of a microcontroller SoC with an NPU.

What favours hardware, what favours software

Table 35.1 — Properties of a computation and where they push it.

Property	Pushes towards hardware (FPGA, NPU, ASIC)	Pushes towards software (CPU)
Operation count per byte of data	High and regular (convolution, FFT, matrix multiply)	Low, irregular (parsing, protocol stacks)
Parallelism	Wide data parallelism or deep pipelines	Sequential dependence, data-dependent branches
Precision	Fixed, narrow (int8, int16, custom widths)	Mixed, floating point, wide dynamic range
Determinism	Cycle-exact latency needed (µs control loops)	Soft deadlines, average throughput
Stability of the algorithm	Frozen standard (AES, CRC, H.264, LDPC)	Changing, field-updatable, many variants
Volume and cost target	High volume (ASIC) or low-latency niche (FPGA)	Low volume, short time to market

Offload break-even

Moving a computation to an accelerator is worth it only if the time saved exceeds the cost of getting the data there and back. For a kernel that takes t_{sw} in software and t_{hw} on the accelerator,

$$t_{\text{setup}} + t_{\text{in}} + t_{hw} + t_{\text{out}} < t_{sw}$$

where t_{setup} is driver and synchronisation overhead (cache maintenance, descriptor set-up, interrupt) and t_{in}, t_{out} are the transfer times. When transfers and computation can overlap in a streaming pipeline, the steady-state cost per item becomes the largest of t_{in}, t_{hw} and t_{out} instead of their sum.

Worked example — FFT offload over AXI. A 1 024-point complex FFT (single-precision, 8 kB per transform) takes 15 µs on a Cortex-A53 core with NEON. An FPGA FFT core in the same Zynq-class device computes one in 1 024 cycles at 250 MHz, 4.1 µs, but data move over an AXI port at an effective 1.6 GB/s, so each direction takes $8\,192/1.6\times10^{9} = 5.12$ µs, and the driver costs 3 µs per call. One transform at a time costs $3 + 5.12 + 4.1 + 5.12 = 17.3$ µs — slower than software. Streamed in batches with input and output DMA running concurrently, the cost per transform falls to 5.12 µs (transfer-bound), a speed-up of $15/5.12 = 2.9$. The FFT core already outruns the link that feeds it, so a faster core would gain nothing; the partitioning decision is really a decision about the interface, and it is worth taking only if the FFT results stay on the FPGA side for further processing.

Granularity is the third design variable. A **custom instruction** (a few cycles; no data movement beyond registers), a **tightly coupled coprocessor** (tens to thousands of cycles, sharing the CPU's local memory) and a **loosely coupled accelerator** on the system bus with its own DMA (thousands of cycles and up) amortise very different overheads; the coarser the offload, the larger the amount of work that must be handed over at once to make it pay.

35.2 Processor versus FPGA

Temporal and spatial computation

A processor computes *in time*: one instruction stream, a few wide functional units, reused every cycle, at 0.1–5 GHz. Its strength is generality; its cost is that most of the energy of each instruction goes into fetching, decoding, scheduling and moving operands through register files and caches rather than into the arithmetic — for a scalar 32-bit add on a modern core, well over 90 %. SIMD and vector units (Arm Helium and NEON, x86 AVX-512, RISC-V V) amortise that overhead over 4–64 operations per instruction.

An FPGA computes *in space* (Chapter 18): the algorithm is laid out as a circuit of LUTs, DSP slices and block RAMs, each doing one job every cycle at 200–600 MHz. There is no instruction overhead, and pipelines thousands of operations deep run with cycle-exact latency. The price is the programmable interconnect: compared with a standard-cell ASIC in the same process, an FPGA implementation typically needs 20–40 times the area, runs 3–4 times slower and uses roughly 7–14 times the dynamic power, the gap narrowing when the design maps onto hard DSP and memory blocks. A GPU sits between the two: thousands of simple cores executing the same instruction on different data (SIMT), with very high memory bandwidth, excellent for large batched dense arithmetic and poor for small, latency-critical, branchy work. An ASIC or a fixed-function NPU removes both the instruction and the interconnect overhead for one class of computation.

Table 35.2 — Representative compute platforms for int8 inference and signal processing (peak values; sustained values are often 10–60 % of peak).

Platform (example)	Peak int8 throughput	Power	Energy per op at peak	NRE	Latency behaviour
MCU core (Cortex-M4, 80 MHz, SIMD MAC)	0.32 GOPS	≈ 12 mW	≈ 40 pJ	Firmware only	Predictable; interrupts add jitter
Server CPU (32 cores, AVX-512 VNNI, 2.5 GHz)	≈ 20 TOPS	≈ 250 W	≈ 12 pJ	Software only	Caches, OS: ms-scale tails
MCU-class NPU (256 MAC/cycle, 500 MHz)	256 GOPS	≈ 0.12 W	≈ 0.5 pJ	Model compilation	Deterministic per network
FPGA (2 520 DSP slices, 600 MHz)	≈ 6 TOPS	20–30 W	3–5 pJ	0.1–2 MUSD design	Cycle-exact pipelines
Data-centre GPU (A100 class)	624 TOPS	≈ 400 W	≈ 0.6 pJ	Software only	Batching, kernel launch µs–ms
Edge inference ASIC (4 TOPS at 2 W)	4 TOPS	≈ 2 W	≈ 0.5 pJ	5–50 MUSD at 28–16 nm; 100 MUSD+ at 5 nm	Deterministic

(Throughput is counted as two operations per MAC. The NPU figure is per MAC array alone; the system around it adds SRAM and DMA energy, Section 35.4.)

> **Status of the principle: Energy-efficiency ordering of compute platforms**
> *Empirical.* The ordering CPU → GPU/FPGA → NPU/ASIC, each step gaining roughly an order of magnitude in operations per joule for regular arithmetic, is observed across process generations; it reverses for irregular, branchy or rapidly changing workloads, where the specialised platforms sit idle or cannot express the computation.

When an FPGA, when an ASIC

The choice between programmable and custom hardware is mostly economics. With FPGA unit price c_F, ASIC unit price c_A and ASIC NRE N_A (masks, design, verification, test development; Chapter 34), the ASIC is cheaper above the volume

$$V^* = \frac{N_A}{c_F - c_A}$$

> **Status of the principle: FPGA–ASIC break-even volume**
> *Derived.* It follows from equating total costs linear in volume; it ignores time to market, the cost of a re-spin, FPGA design cost, and the option value of reprogramming in the field, which often dominate the decision.

Worked example — An industrial camera's image-processing pipeline. A mid-range FPGA that implements the pipeline costs 120 USD in volume; an equivalent 28 nm ASIC would cost 8 USD per unit after an NRE of 4 MUSD. The break-even volume is

$$V^* = \frac{4 \times 10^6}{120 - 8} = 35\,700 \text{ units}$$

A machine-vision camera selling 5 000 units a year stays on the FPGA, and keeps the ability to add features in the field; a consumer dash-camera selling a million units a year moves to an ASIC — or, more often, to a commercial SoC that already contains a suitable image processor and NPU, which removes the NRE altogether. FPGAs also win outright where the algorithm is not yet frozen (5G radio prototypes, new radar waveforms), where volumes are small and lives long (test equipment, aerospace, defence), and where nanosecond-scale deterministic latency is needed (high-frequency trading, protection relays, motor-drive current loops; Section 35.7).

35.3 Hardware acceleration

Amdahl's law

If a fraction f of a program's run time is accelerated by a factor s, and the offload adds an overhead equal to a fraction o of the original run time, the overall speed-up is

$$S = \frac{1}{(1 - f) + \frac{f}{s} + o}$$

and even an infinitely fast accelerator cannot beat $1/(1 - f)$.

> **Status of the principle: Amdahl's law**
> *Derived*. It follows from splitting a fixed workload's run time into a serial and an accelerable part; it assumes a fixed problem size and no overlap between the parts. When the problem grows with the available hardware (larger images, more channels), Gustafson's scaled form applies instead, and overlap of transfers with computation reduces the effective o.

Worked example — The vibration sensor's speed-up. Profiling the original firmware shows 80 % of the time in the CNN layers. An accelerator 20 times faster on those layers, with transfers and synchronisation costing 5 % of the original time, gives

$$S = \frac{1}{0.2 + 0.8/20 + 0.05} = \frac{1}{0.29} = 3.4$$

against 4.2 without the overhead and a ceiling of 5 for any accelerator. In the opening example the NPU was 267 times faster on the network itself (56.25/0.211), but the end-to-end time went from $1.2 + 56.25 = 57.45$ ms to $1.2 + 0.21 + 0.1 = 1.51$ ms, a speed-up of 38: the 1.2 ms FFT is now 80 % of the remaining time, so the next step is to move the FFT too — into the CPU's DSP instructions with a tuned library, or into a hardware FFT block — not to buy a faster NPU.

Ways to accelerate

- **Instruction-set extensions.** Many processors reserve opcode space for custom instructions (Chapter 16 for RISC-V's custom opcodes); Arm's Custom Datapath Extension does the same for Cortex-M33/M55-class cores. A CRC step, a Galois-field multiply or a saturating fixed-point MAC becomes a one-cycle instruction with no data movement at all, typically gaining 2–10× on the kernel.
- **Coprocessors** sharing the CPU's tightly coupled memory, for kernels of hundreds of cycles (crypto, FFT butterflies).
- **Streaming accelerators** with their own DMA on the system interconnect (AXI), the usual FPGA and NPU arrangement.
- **Systolic arrays** for matrix arithmetic: a grid of MAC units through which operands flow from neighbour to neighbour, so each operand fetched from memory is used by a whole row or column of units. A 128×128 array reuses each fetched weight 128 times, the property that makes NPUs and tensor units efficient (Section 35.4).

High-level synthesis

Writing an accelerator in RTL (Chapter 19) gives full control at a high design cost. **High-level synthesis** (HLS) compiles C or C++ with directives into RTL: the tool schedules operations into clock cycles, binds them to hardware units and builds the control FSM. Listing 35.1 is a 32-tap FIR filter in the style accepted by AMD Vitis HLS.

```
 // 32-tap FIR for high-level synthesis (C++17; Vitis-HLS-style pragmas, illustrative)
#include <cstdint>

constexpr int N_TAPS = 32;

void fir32(const int16_t *x_in, int32_t *y_out, int n_samples,
           const int16_t coef[N_TAPS])
{
#pragma HLS INTERFACE m_axi port=x_in  bundle=gmem0 depth=4096
#pragma HLS INTERFACE m_axi port=y_out bundle=gmem1 depth=4096
#pragma HLS ARRAY_PARTITION variable=coef complete
    static int16_t shift_reg[N_TAPS];
#pragma HLS ARRAY_PARTITION variable=shift_reg complete

sample_loop:
    for (int n = 0; n < n_samples; ++n) {
#pragma HLS PIPELINE II=1
        int32_t acc = 0;
    tap_loop:
        for (int k = N_TAPS - 1; k > 0; --k) {
            shift_reg[k] = shift_reg[k - 1];
            acc += static_cast<int32_t>(shift_reg[k]) * coef[k];
        }
        shift_reg[0] = x_in[n];
        acc += static_cast<int32_t>(shift_reg[0]) * coef[0];
        y_out[n] = acc;
    }
}
```

The `PIPELINE II=1` directive asks for one new sample every clock (initiation interval 1), which forces the inner loop to be fully unrolled into 32 parallel multipliers — 32 DSP slices — and requires the coefficient and delay-line arrays to be split into individual registers (`ARRAY_PARTITION`), since a block RAM has only two ports. At 300 MHz the filter processes 300 MS/s, 9.6 GMAC/s. With Q15 coefficients whose absolute values sum to at most one, the Q30 accumulator cannot overflow 32 bits. The same source compiles and runs as ordinary C++, which is how it is tested (Section 35.8); the pragmas are ignored by a normal compiler. HLS removes most hand-written RTL for datapaths, but not the need to think in hardware: the code above is fast only because it was written as a shift register, and an innocent-looking pointer dereference inside the loop could raise the initiation interval tenfold.

35.4 Memory and data movement

Data movement costs more than arithmetic

The energy of an arithmetic operation is set by the switched capacitance of a small circuit; the energy of fetching its operands is set by the length of the wires to the memory that holds them (*Circuits, Signals and Communications*, Ch. 14 §14.10 for off-chip links; Chapter 34 for on-chip wires). Widely quoted measurements for a 45 nm process show how lopsided the comparison is (Figure 35.2).

Table 35.3 — Energy per operation and per access (45 nm, 0.9 V; after Horowitz, ISSCC 2014). Newer nodes lower all entries, but the ratios persist.

Operation	Energy (pJ)	Relative to 8-bit add
8-bit integer add	0.03	1
8-bit integer multiply	0.2	7
32-bit integer add	0.1	3
32-bit integer multiply	3.1	100
16-bit floating-point multiply	1.1	37
32-bit floating-point multiply	3.7	120
64-bit read, 8 kB SRAM	10	330
64-bit read, 32 kB SRAM	20	670
64-bit read, 1 MB SRAM	100	3 300
64-bit DRAM access	1 300–2 600	43 000–87 000

Modern LPDDR5 and HBM interfaces have reduced DRAM energy to roughly 3–8 pJ per bit (0.2–0.5 nJ per 64 bits), still about a thousand times an 8-bit MAC.

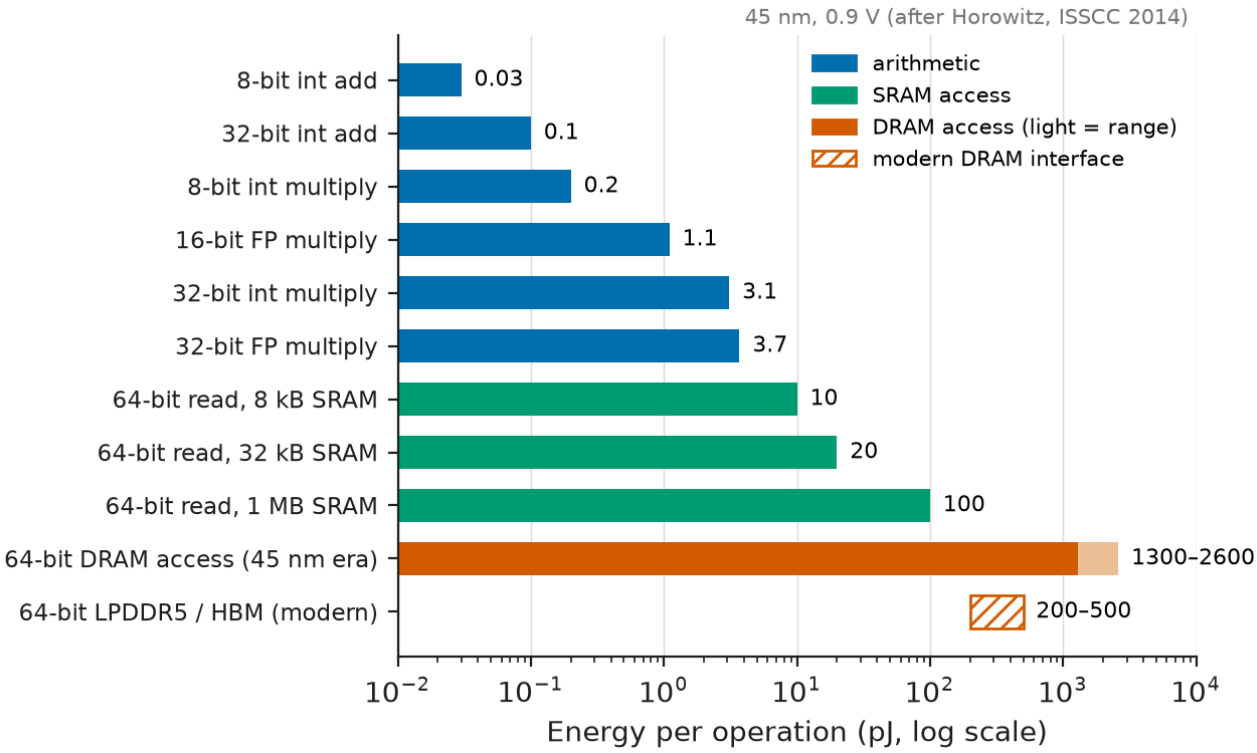


Figure 35.2 — Energy of arithmetic operations compared with memory accesses, on a logarithmic scale.

> **Status of the principle: Dominance of data-movement energy**
> *Empirical.* Measured energies across many processes show memory access costing 10–10 000 times an arithmetic operation, depending on distance; the ratio follows from wire capacitance per unit length staying nearly constant while logic shrinks, and it is the main reason specialised accelerators are efficient.

Worked example — Energy of one convolution layer. A 3×3 convolution with 64 input and 64 output channels on a 56×56 feature map performs $56 \times 56 \times 64 \times 64 \times 9 = 115.6$ million MACs. At 0.23 pJ per int8 MAC (multiply plus add) the arithmetic costs 26.6 µJ. The layer's data are 36 864 bytes of weights and 200 704 bytes each of input and

output activations, 438 kB in all; reading each byte once from DRAM at about 160 pJ per byte (1.3 nJ per 64 bits) costs 71 µJ — already 2.7 times the arithmetic, even with perfect reuse. With no reuse at all, fetching two operand bytes from DRAM for every MAC would cost $2 \times 115.6 \times 10^6 \times 162.5$ pJ $= 37.6$ mJ, 1 400 times the arithmetic; fetching them from a 32 kB SRAM instead (2.5 pJ per byte) would cost 0.58 mJ. The design of every efficient accelerator is the design of the reuse that turns the third number into the second.

Reuse, tiling and dataflow

The tools for reducing data movement are the same on a CPU, a GPU, an FPGA or an NPU:

- **Tiling.** Split a large computation into blocks that fit the fastest memory level (registers, a 32–512 kB scratchpad, a cache), and finish all the work on a block before evicting it.
- **Dataflow choice.** Keep one operand stationary in the processing elements and stream the others past it: *weight-stationary* arrays hold weights and stream activations; *output-stationary* arrays hold partial sums. Which is best depends on the layer shape, and NPU compilers choose per layer.
- **Scratchpads and DMA.** A software-managed scratchpad has no tags or misses, so its timing is deterministic; a DMA engine fills one half while the accelerator computes on the other (**double buffering**), hiding transfer time when compute time per tile exceeds transfer time.
- **Narrow data.** int8 instead of float32 quarters both storage and traffic (Section 35.6).
- **Coherence discipline.** When a DMA engine writes memory that a cached CPU then reads, the CPU's cache must be invalidated first, and cleaned before the DMA reads CPU-written data; a missing cache-maintenance call produces the classic bug of an accelerator that "works in the debugger" and returns stale results at full speed. Hardware-coherent interconnects (ACE, CHI, CXL) remove the problem at a cost in area and latency.

Table 35.4 — Representative memory and interconnect bandwidths (peak, per direction where relevant).

Interface	Configuration	Peak bandwidth
On-chip SRAM bank	128 bit at 500 MHz	8 GB/s per bank
AXI4 port	128 bit at 250 MHz	4 GB/s
LPDDR4X	32 bit at 4 266 MT/s	17 GB/s
DDR4-3200 DIMM	64 bit	25.6 GB/s
DDR5-4800 DIMM	64 bit (2 × 32)	38.4 GB/s
PCIe 4.0 × 16	16 GT/s per lane, 128b/130b	31.5 GB/s
HBM2e stack	1 024 bit at 3.2–3.6 Gb/s	410–460 GB/s

35.5 Performance modeling

A hierarchy of models

Performance can be estimated at several levels of effort and fidelity: analytical models (Amdahl, roofline, Little's law, a spreadsheet of operation counts and bandwidths) in minutes; instruction-set and cycle-approximate simulators (gem5, or the performance report of an NPU compiler such as Arm's Vela) in hours; RTL simulation or FPGA prototypes in days; and measurement on silicon with hardware performance counters — Linux `perf` on application processors, the cycle counter `DWT->CYCCNT` on Cortex-M cores — which is the ground truth against which all models are calibrated. The right first step is almost always the analytical model: it is wrong in detail, but it shows which resource limits the design and how far it is from that limit.

The roofline model

A kernel with **arithmetic intensity** I — operations per byte moved from main memory — running on a machine with peak compute π (operations per second) and memory bandwidth β (bytes per second) can attain at most

$$P = \min\left(\pi,\ \beta I\right)$$

The two lines cross at the **ridge point** $I_r = \pi/\beta$. Kernels left of the ridge are memory-bound: faster arithmetic does nothing for them, and only more bandwidth or more reuse (higher I) helps. Kernels right of it are compute-bound (Figure 35.3).

> **Status of the principle: Roofline model**
> *Approximate.* It is an upper bound assuming perfect overlap of computation and memory transfer and a single bandwidth-limited memory level; real kernels sit below it because of latency, cache conflicts, pipeline stalls and limited parallelism, and extended rooflines add ceilings for each cache level and for vector or tensor-unit use.

Listing 35.2 applies the model to four kernels on the platforms of Table 35.2.

```
 # Roofline: attainable throughput = min(peak, bandwidth x arithmetic intensity).
platforms = {                       # peak int8 ops/s, memory bandwidth bytes/s
    "Cortex-M4 @ 80 MHz": (0.32e9, 0.32e9),
    "Small NPU (256 MAC)": (256e9, 4e9),
    "FPGA, 2520 DSP": (6.05e12, 19.2e9),
    "Data-centre GPU": (624e12, 2.0e12),
}
kernels = {                         # ops per byte moved from main memory
    "GEMV 1024x1024 (batch 1)": 2.0,
    "Depthwise 3x3 conv": 9.0,
    "3x3 conv, 64 ch": 528.0,
    "GEMM 1024^3": 683.0,
}
for pname, (peak, bw) in platforms.items():
    print(f"{pname}: ridge at {peak / bw:6.1f} ops/B")
    for kname, ai in kernels.items():
        att = min(peak, bw * ai)
        bound = "compute" if att == peak else "memory"
        print(f"   {kname:26s} {att / 1e9:10.2f} GOPS ({bound}-bound, {att / peak:6.1%} of peak)")
```

The output says that the microcontroller (ridge at 1 op/B, because its SRAM keeps up with its single MAC unit) is compute-bound on everything. The NPU, with its ridge at 64 ops/B, reaches only 3.1 % of peak on a batch-1 matrix–vector product and 14 % on a depthwise convolution, but 100 % on a dense convolution whose weights and activations are reused from on-chip memory ($I = 2 \times 115.6 \times 10^6 / 438\,272 = 528$ ops/B for the layer above). The GPU, with its ridge at 312 ops/B, attains 4 TOPS of its 624 on the batch-1 GEMV — 0.6 % — which is the data-centre puzzle of the opening: the fix is to batch requests, raising I, not to buy a faster GPU.

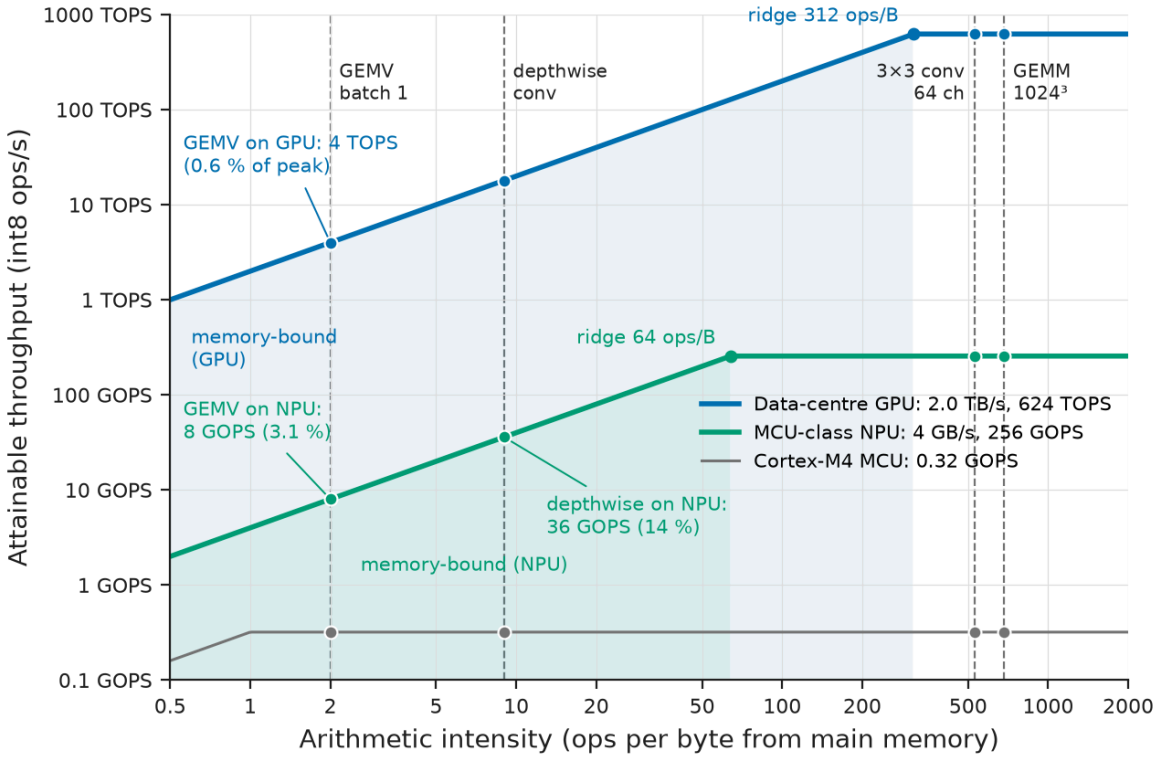


Figure 35.3 — Roofline model for an MCU-class NPU and a data-centre GPU with representative kernels.

Little's law and the concurrency a memory system needs

A memory system with latency W that is to deliver bandwidth β in requests of B bytes must have, on average,

$$N = \frac{\beta W}{B}$$

requests in flight. This is Little's law ($N = \lambda W$, with arrival rate $\lambda = \beta / B$) applied to memory.

> **Status of the principle: Little's law**
> *Derived*. It holds for any stable queueing system in steady state, independent of arrival and service distributions; it says nothing about how latency grows as the system approaches saturation, which needs a queueing model.

Worked example — Why one core cannot saturate DRAM. A DDR4-3200 channel delivers 25.6 GB/s with a loaded latency of about 90 ns and 64-byte cache lines, so keeping it busy needs $25.6 \times 10^9 \times 90 \times 10^{-9} / 64 = 36$ outstanding line requests. A core that can track 10 outstanding misses (its miss-status holding registers) is limited to $10 \times 64\ \text{B} / 90\ \text{ns} = 7.1$ GB/s however fast it computes. Accelerators and GPUs reach high bandwidth precisely by keeping hundreds of requests in flight, and DMA engines are given deep outstanding-transaction queues for the same reason.

Pipelines obey the same arithmetic in the time domain: a hardware pipeline with initiation interval II at clock f delivers f/II results per second regardless of its depth; its depth sets only the latency.

35.6 Embedded AI deployment

The deployment flow

Neural networks are trained in floating point on GPUs and deployed on whatever the product contains: a microcontroller core, an NPU, an FPGA or a mobile GPU. The deployment flow is a co-design loop in its own right: choose a model

architecture that fits the target (depthwise-separable convolutions, small input resolution); **quantise** it to 8-bit integers; compile it for the target (TensorFlow Lite for Microcontrollers with Arm's CMSIS-NN kernels on Cortex-M, the Vela compiler for Ethos-U NPUs, Vitis AI for AMD FPGAs, TensorRT for NVIDIA GPUs, vendor tools for others); measure latency, memory and accuracy on the real device; and iterate.

Quantisation

> **Boundary.**
> The uniform quantiser, its noise model (≈ 6.02 dB of SQNR per bit) and fixed-point Q formats with guard bits and rounding modes are from *Circuits, Signals and Communications*, Ch. 21 §21.2 and §21.5; two's-complement integers from Chapter 11. Here they are applied to tensors.

Integer inference represents each real tensor value r by an integer q through an **affine mapping** with a real scale S and an integer zero point Z:

$$r = S\,(q - Z)$$

Weights are usually quantised *symmetrically* ($Z = 0$, $q \in [-127, 127]$) and activations *asymmetrically* (a zero point lets a post-ReLU range such as 0 to 6 use all 256 codes). A layer $y = \sum_i w_i x_i$ then becomes integer arithmetic: products of int8 values accumulate exactly in an int32 register, and the result is rescaled to the output's scale by the real factor $M = S_w S_x / S_y$, itself implemented in integers as a 32-bit fixed-point multiplier M_0 and a right shift n, $M = M_0\,2^{-31}\,2^{-n}$, with $0.5 \le M_0 2^{-31} < 1$ (Figure 35.4). No floating-point hardware is needed anywhere.

Two choices dominate the accuracy. **Per-channel** weight scales (one S_w for each output channel) instead of one per tensor protect channels whose weights are small. **Post-training quantisation** (PTQ), which calibrates activation ranges on a few hundred representative inputs, typically costs less than 1 % accuracy for convolutional networks at int8; **quantisation-aware training** (QAT), which simulates quantisation during training, recovers most of the remainder and is needed for int4 or for sensitive models. Listing 35.3 shows the effect of per-channel scales and the integer rescaling step.

```
 # Symmetric int8 weights (per-tensor vs per-channel), int32 accumulation, requantisation.
import numpy as np

rng = np.random.default_rng(1)
W = rng.normal(0, 1, (16, 64)) * np.linspace(0.02, 0.4, 16)[:, None]  # 16 output channels
x = rng.uniform(0, 2.0, 64)                                           # post-ReLU activations

def quant_sym(w, axis=None):
    s = np.max(np.abs(w), axis=axis, keepdims=axis is not None) / 127.0
    return np.clip(np.round(w / s), -127, 127).astype(np.int8), s

def sqnr_db(ref, approx):
    return 10 * np.log10(np.sum(ref**2) / np.sum((ref - approx) ** 2))

s_x = 2.0 / 255.0                                   # asymmetric uint8 input, zero point 0
q_x = np.round(x / s_x).astype(np.int32)
y_ref = W @ x
for name, axis in (("per-tensor", None), ("per-channel", 1)):
    q_w, s_w = quant_sym(W, axis)
    acc = q_w.astype(np.int32) @ q_x                # exact int32 accumulators
    y = acc * (np.ravel(s_w) * s_x)                 # real-valued result before requantisation
    w_hat = q_w * s_w
    worst = min(sqnr_db(W[i], w_hat[i]) for i in range(16))
    print(f"{name:11s}: output SQNR {sqnr_db(y_ref, y):5.1f} dB, worst-channel weight"
          f" SQNR {worst:5.1f} dB, max |acc| {np.max(np.abs(acc))}")

# Requantise to int8 output with an integer multiplier M0 (Q31) and a right shift.
# Illustrative layer, not W above: S_x = 0.02, S_w = 0.0041, S_y = 0.05.
M = 0.02 * 0.0041 / 0.05
shift = int(np.floor(-np.log2(M)))
M0 = int(round(M * 2**shift * 2**31))
acc = 12345
y_q = (acc * M0 + (1 << (30 + shift))) >> (31 + shift)
print(f"M = {M:.6f} = {M0}/2^31 x 2^-{shift}; acc {acc} -> {y_q} (exact {acc * M:.2f})")
```

The layer's weight magnitudes vary twentyfold across its 16 output channels. With one scale for the whole tensor the output SQNR is 40.8 dB, but the channel with the smallest weights is represented with only 15.3 dB, because it uses just a handful of the 255 codes; per-channel scales raise that channel to 43.0 dB and the output SQNR to 46.5 dB. The largest accumulator value, 170 367, needs 19 bits including the sign, far inside int32. The requantisation step uses an illustrative layer rather than the matrix above (whose per-tensor scale is $1.31/127 = 0.0103$): with $S_x = 0.02$, $S_w = 0.0041$ (weights within ± 0.52) and $S_y = 0.05$, the rescaling factor $M = 0.02 \times 0.0041 / 0.05 = 0.00164$ is represented as $M_0 = 1\,803\,199\,070$ with a shift of 9, and maps an accumulator of 12 345 to 20, the correctly rounded value of 20.25.

On the device the same arithmetic is a short C loop. Listing 35.4 computes one output channel of a fully connected int8 layer the way CMSIS-NN-style kernels do, in portable C.

```
#include <stdint.h>

/* acc * M0 * 2^-31 * 2^-shift with round-half-up; shift >= 0. */
static inline int32_t requantise(int32_t acc, int32_t m0, int shift)
{
    int64_t prod = (int64_t)acc * m0;
    int64_t half = (int64_t)1 << (30 + shift);
    return (int32_t)((prod + half) >> (31 + shift));
}

/* One output channel of an int8 fully connected layer (symmetric weights). */
int8_t fc_channel(const int8_t *x, const int8_t *w, int n, int32_t x_zero,
                  int32_t bias, int32_t m0, int shift, int32_t y_zero)
{
    int32_t acc = bias;                     /* bias pre-scaled to S_w * S_x */
    for (int i = 0; i < n; ++i)
        acc += ((int32_t)x[i] - x_zero) * (int32_t)w[i];
    int32_t y = requantise(acc, m0, shift) + y_zero;
    if (y > 127)  y = 127;                  /* saturate to int8 */
    if (y < -128) y = -128;
    return (int8_t)y;
}
```

Right-shifting a negative signed integer is implementation-defined in C11; every mainstream compiler for Arm, RISC-V and x86 implements it as an arithmetic shift, and the production kernels rely on that (or on the target's saturating-rounding instructions). On a Cortex-M4 the inner loop is rewritten with the `SMLAD` instruction to perform two 16-bit MACs per cycle, which is where Table 35.2's 2 MACs per cycle peak comes from; on a Cortex-M55 the Helium vector extension raises it to 8 int8 MACs per cycle.

Memory budgets and targets

On a microcontroller the binding constraints are usually memory, not arithmetic. Weights are constant and live in flash; activations live in an SRAM **tensor arena** whose peak size the converter computes by planning which tensors are alive at once. A 38 600-parameter keyword-spotting or vibration network needs 38.6 kB of flash at int8 instead of 154 kB at float32, and an arena of a few tens of kilobytes; a 300 000-parameter image classifier at 96×96 resolution needs about 300 kB of flash and 100–250 kB of arena, which rules out parts with less than 256 kB of SRAM. On FPGAs, int8 MACs pack two per DSP slice (the source of Table 35.2's FPGA figure) and weights for small networks stay entirely in on-chip block RAM, eliminating the DRAM term of Section 35.4.

Worked example — Vibration classifier on a CPU and on an NPU. The network performs 2.7 million MACs per inference. On the Cortex-M4 at 80 MHz, CMSIS-NN kernels sustain about 0.6 MAC per cycle (memory accesses, im2col copies and requantisation take the rest of the 2-per-cycle peak), so one inference takes

$$t = \frac{2.7 \times 10^{6}}{0.6 \times 80 \times 10^{6}} = 56\ \text{ms}$$

and, at 12 mW for core and memory, 0.68 mJ. An Ethos-U55-class NPU with 128 MACs per cycle at 250 MHz, at a utilisation of 40 % on this small network, takes $2.7 \times 10^{6} / (128 \times 0.4 \times 250 \times 10^{6}) = 0.21$ ms, and at 30 mW uses 6.3 µJ — 107 times less energy per inference. At 100 inferences per second the network itself then costs 0.63 mW on average, and the battery budget is dominated by the 1.2 ms CPU FFT, the radio and the sensor, which is where the next round of co-design goes.

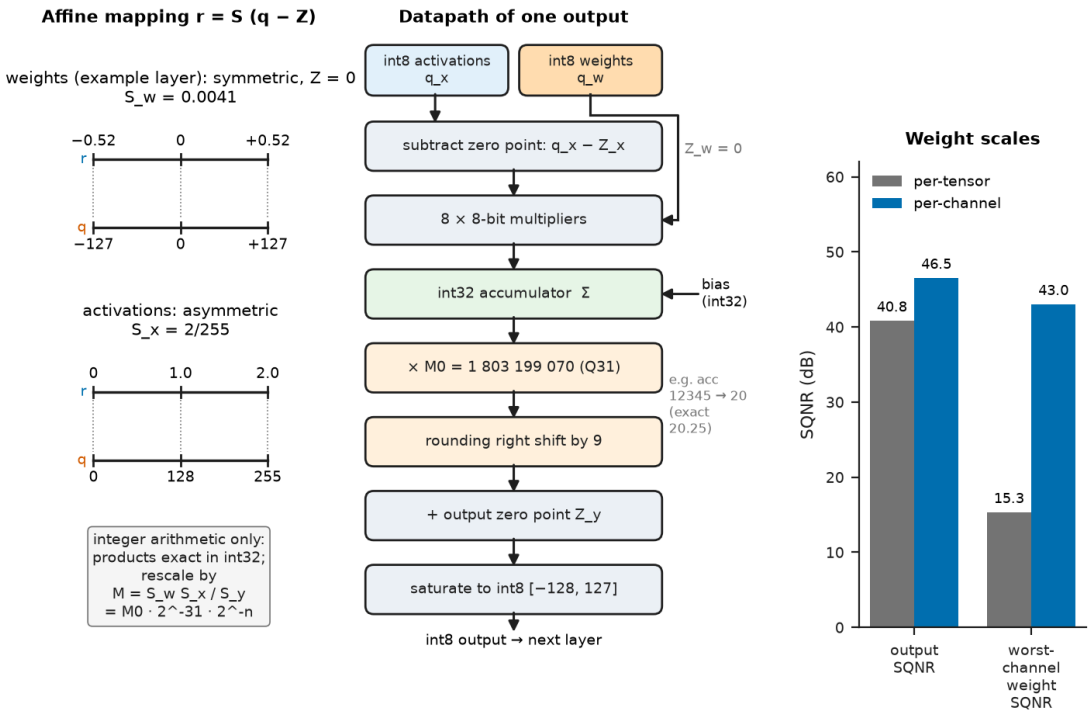


Figure 35.4 — Integer-only inference of one quantised layer: affine mapping, int32 accumulation and requantisation.

35.7 Real-time constraints

> **Boundary.**
> Deadlines, worst-case execution time (WCET), interrupt latency, jitter and schedulability analysis are developed in Chapter 27; this section adds what accelerators change.

An accelerator changes a real-time system in three ways. It **shortens** the computation, often by orders of magnitude. It **adds** fixed costs — set-up, DMA transfers, a completion interrupt — that must be included in the WCET. And it introduces **interference**: an accelerator streaming from DRAM competes with the CPU, the display controller and other masters for the same memory, so the CPU's cache-miss latency, and the accelerator's own transfer time, depend on what everything else is doing. Platforms differ sharply in how analysable their timing is: an FPGA pipeline is cycle-exact; an NPU running one compiled network from private SRAM is nearly deterministic; a GPU, which batches work, schedules kernels non-preemptively and shares memory with a host operating system, is designed for throughput and typically shows millisecond-scale latency tails.

Worked example — A 20 kHz current loop: MCU or FPGA. A motor drive's current controller runs every 50 µs: sample phase currents, Clarke and Park transforms, two PI controllers, inverse transforms, space-vector PWM update (the control law itself is from *Electrical Energy and Machines*, Ch. 12). On a 200 MHz Cortex-M4 the interrupt service routine takes 2 400 cycles (12 µs) typically, but 4 200 cycles (21 µs) worst case once flash wait states, a cache miss and preemption by a higher-priority communication interrupt are included: 42 % of the period, with 9 µs of jitter in when the PWM update lands. The same loop in FPGA fabric takes 120 cycles at 100 MHz — 1.2 µs, the same every time — which allows the loop to run at 100 kHz for a high-speed spindle or a GaN inverter. The partition used in many industrial drives is exactly this: the fast, fixed current loop and protection in FPGA logic or a dedicated control accelerator, the slower speed loop, communication and diagnostics in software.

Worked example — An automotive camera pipeline. A front camera produces 30 frames per second, a frame every 33.3 ms, and the braking function requires each frame's result within one frame period. The stages are image signal

processing (8 ms), DMA of the scaled frame into NPU memory (1.2 ms), CNN inference on the NPU (14 ms), tracking and decision on the CPU (3 ms) and a CAN-FD message (1 ms): end-to-end latency 27.2 ms. Because the stages work on successive frames concurrently (Figure 35.5), the frame *rate* could rise to $1/14$ ms $= 71$ fps, limited by the slowest stage, but the *latency* of each frame stays 27.2 ms. Measured under full load, though, inference stretches to 20 ms when the ISP and the display controller saturate DRAM, making the latency $8 + 1.2 + 20 + 3 + 1 = 33.2$ ms — 0.1 ms inside the deadline. The fix is architectural, not a faster NPU: memory-bandwidth regulation (quality-of-service limits on the other DRAM masters), keeping the network's weights in on-chip SRAM, or reducing the input resolution. Safety standards make this analysis mandatory — ISO 26262 for road vehicles, DO-178C for airborne software and DO-254 for airborne FPGA and ASIC hardware, IEC 62304 for medical software.

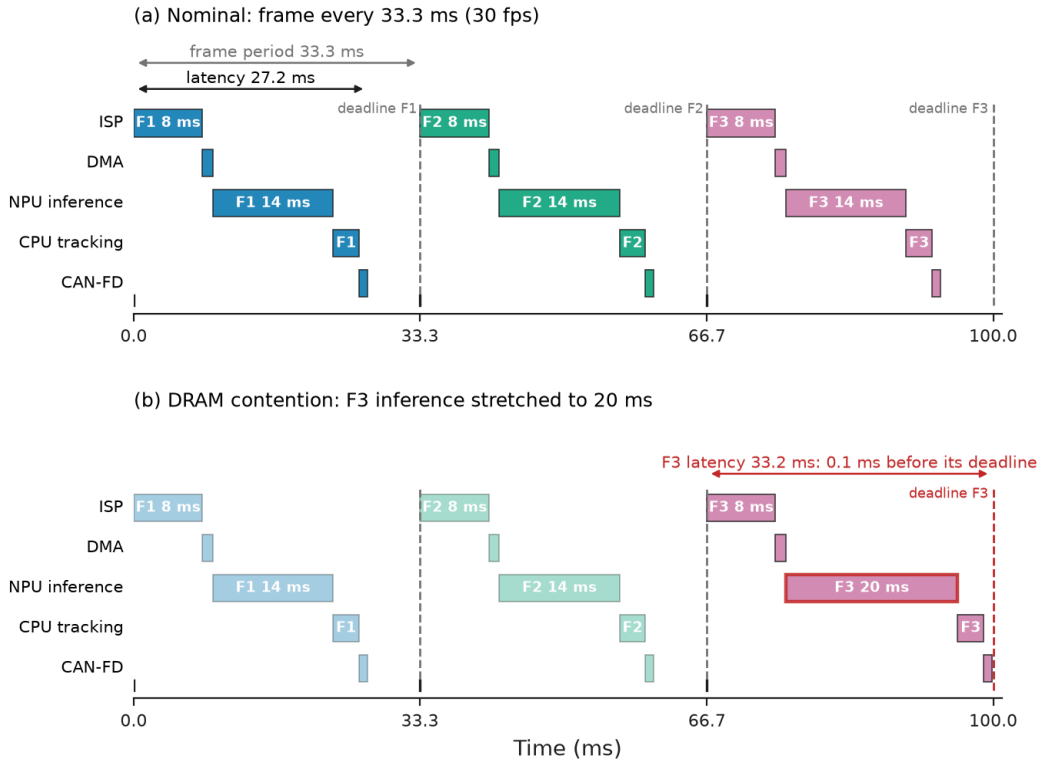


Figure 35.5 — Pipelined processing of camera frames showing per-frame latency and frame rate, nominal and under memory contention.

35.8 System-level verification

Verifying hardware and software together

The hardware of Chapter 34 and the software of Parts V and VI can each be verified separately and still fail together: a driver writes a register in the wrong order, an interrupt arrives before the accelerator's status is valid, a DMA descriptor uses a physical address the IOMMU does not map, a quantised model compiled with one tool version behaves differently on the device. **System-level verification** runs the real software on a model of the real hardware, as early as possible. The methods form a ladder of speed against fidelity.

Table 35.5 — Platforms for running software on hardware under development (typical speeds for a multi-core SoC).

Platform	What models the hardware	Speed	Fidelity	Available
Virtual prototype (loosely timed)	SystemC/TLM-2.0 models, instruction-set simulators	100–1 000 MIPS	Functional, register-accurate; no cycle timing	Before RTL
Virtual prototype (approximately timed)	TLM models with timing annotations	1–50 MIPS	Performance estimates within 10–30 %	Before RTL
Co-simulation	RTL of one block in a simulator, rest as virtual models or C via DPI	1–100 kHz	Cycle-exact for the RTL block	With block RTL
RTL simulation of the SoC	Full RTL	10–1 000 Hz	Cycle-exact	With full RTL
Hardware emulation	RTL mapped to a dedicated emulator	0.5–5 MHz	Cycle-exact, full visibility	With full RTL
FPGA prototype	RTL partitioned over FPGAs	5–100 MHz	Cycle-exact, limited visibility	With full RTL
Hardware-in-the-loop	Real device with simulated plant	Real time	Real hardware	With silicon or board

A **virtual prototype** models each block at the level the software sees — its registers, interrupts and DMA behaviour — in SystemC with the TLM-2.0 transaction-level interfaces (IEEE 1666), around fast instruction-set simulators of the processors (QEMU, Arm Fast Models). It lets firmware and drivers be written and tested months before RTL exists and later serves as the reference against which the RTL is checked. **Co-simulation** couples models at different levels: an accelerator's RTL runs in an HDL simulator while the processor runs as an instruction-set simulator, or a C/C++ reference model is called directly from a SystemVerilog test bench through the DPI-C interface to predict each output (Figure 35.6). **Hardware-in-the-loop** testing of the finished device against a simulated plant is the last rung (Chapter 28 §28.9).

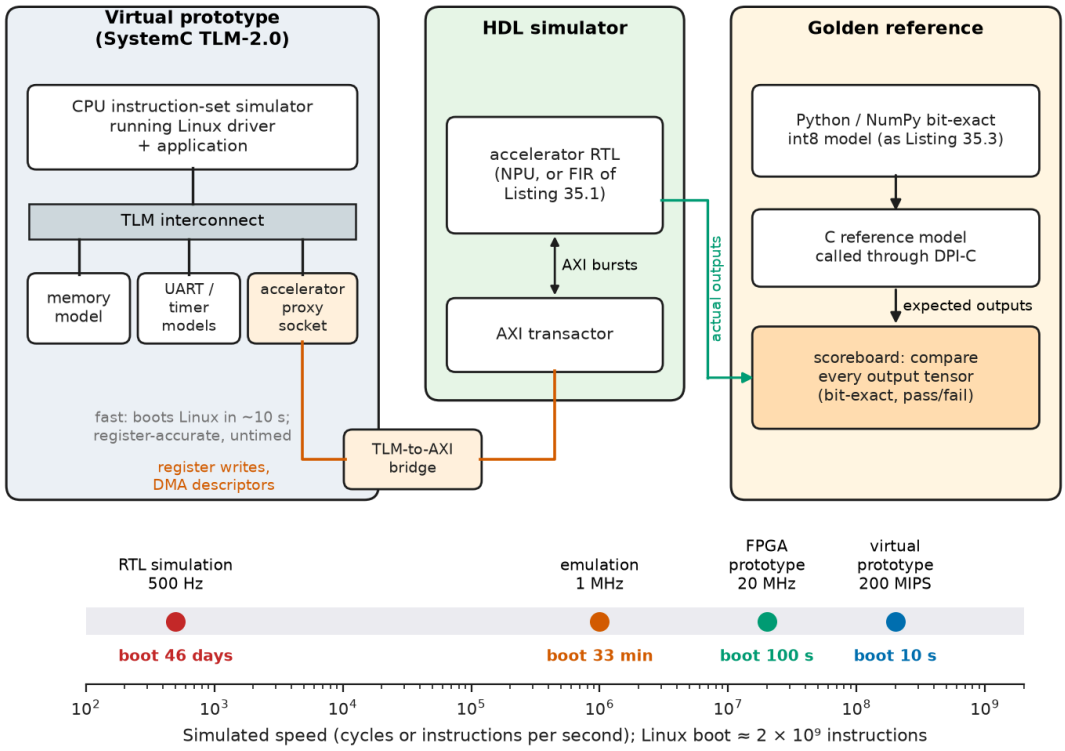


Figure 35.6 — A virtual prototype with an RTL accelerator in co-simulation, and the bit-exact reference model used as the golden check.

Worked example — How long to boot an operating system? Booting embedded Linux to a shell on the SoC takes about 2×10^9 instructions, roughly as many cycles. RTL simulation of the full SoC at 500 cycles per second needs 4×10^6 s, 46 days; an emulator at 1 MHz needs 2 000 s, 33 minutes; an FPGA prototype at 20 MHz, 100 s; a loosely timed virtual prototype at 200 MIPS, 10 s. Hence the division of labour: software bring-up and regression on the virtual prototype, hardware–software integration and performance validation on emulation and FPGA prototypes, cycle-level debugging of individual blocks in RTL simulation or co-simulation.

Golden models and bit-exactness

The most effective single practice in co-design verification is a **bit-exact golden model**: an executable reference — in Python, C or C++ — that computes exactly the integers the hardware should produce, including rounding, saturation and quantisation, as Listings 35.3 and 35.4 do. The HLS source of Listing 35.1 is already such a model; an NPU compiler's reference kernels are another. Every level of the ladder is compared against it: the RTL in co-simulation output by output, the FPGA prototype on recorded data sets, the silicon in production test. Floating-point stages are compared with a tolerance derived from the numerical analysis (*Circuits, Signals and Communications*, Ch. 21 §21.6), never with "looks close". For machine-learning systems a second level of checking is needed above bit-exactness: the deployed int8 model's accuracy on a held-out test set, measured on the target, must match the accuracy recorded when the model was quantised, because a model that is bit-exact to a badly calibrated reference is exactly as wrong as its reference.

> **Status of the principle: Bit-exact reference modelling**
> *Approximate* as a guarantee of correctness. Matching a reference proves the implementation computes what the reference computes on the tested inputs, no more; coverage of the input space (Chapter 19) and validation of the reference against the specification remain separate obligations.

What remains open

Open question	Where it is answered
How do the transistors, gates and memories that every platform here is built from actually work?	Chapter 4; Chapter 33; Chapter 18
How are processors, caches and interrupts organised, and how do instructions execute on a microcontroller?	Chapter 16; Chapter 17
How is accelerator RTL written, constrained and tested, and how is a custom chip carried to tape-out?	Chapter 19; Chapter 34
How are firmware, drivers and real-time schedules built around the accelerators?	Chapter 25; Chapter 27; Chapter 28
How are the analog front ends and converters that feed these systems designed?	Chapter 10; *Circuits, Signals and Communications*, Ch. 19
Which FFTs and digital filters should the accelerators compute, and with what numerical error?	*Circuits, Signals and Communications*, Ch. 22, Ch. 23, Ch. 21
How are the feedback loops that run on these controllers designed and proved stable?	*Circuits, Signals and Communications*, Ch. 34, Ch. 35
How do the radios and links that carry the results work?	*Circuits, Signals and Communications*, Ch. 27, Ch. 30
How do embedded controllers drive machines, and how are field-oriented control and its current loops tuned?	*Electrical Energy and Machines*, Ch. 10, Ch. 12; power stage in Chapter 32
How do power converters, storage and data centres connect to the grid, and how is their energy managed?	*Electrical Energy and Machines*, Ch. 16, Ch. 27, Ch. 28; converters in Chapters 29–31
How will computation keep scaling as transistors stop getting cheaper — chiplets, 3D stacking, in-memory and analog computing?	Open research; the yield and data-movement arithmetic of Chapter 34 and Section 35.4 is where every proposal must start

The book began with a nonlinear device and ends with a decision about where to put a computation; between them lies one idea, applied at every level — a physical device, biased and switched, made into a reliable abstraction that holds only within the limits of voltage, current, time, power and temperature set by the device beneath it, and the engineer's craft is to know both the abstraction and its limits.